

Fortgeschrittene mathematische Methoden in der Statistik

Skript zur Vorlesung

Fabian Scheipl

Institut für Statistik, LMU München

Inhaltsverzeichnis

1	Worum geht's?	1
1.1	Worum geht es in diesem Skript?	1
1.2	Landkarte des Skripts	4
2	Algorithmen & Komplexität	7
2.1	Numerische Probleme und Algorithmen	7
2.2	Algorithmen konkret: Fibonacci und Verwandte	11
2.3	Aufwand und Komplexität	15
2.4	Landau-Symbole und Rechenregeln	19
2.5	Fibonacci: Komplexitätsanalyse	24
3	Spur & Matrixnormen	31
3.1	Die Spur einer Matrix	31
3.2	Matrixnormen: Definition und Beispiele	36
3.3	Operatornormen	41
3.4	Schattennormen	47
3.5	Eigenschaften von Matrixnormen	52
3.6	Zusammenfassung	60
4	Fehler, Kondition & Stabilität	65
4.1	Fehlermaße und Fehlerzerlegung	65
4.2	Kondition	70
4.3	Stabilität von Algorithmen	77
4.4	Zusammenfassung	82
5	Lineare Gleichungssysteme	85
5.1	Numerische lineare Algebra: Grundlagen	85
5.2	Lineare Gleichungssysteme	87
5.3	Die LU-Zerlegung	92

5.4 Die Cholesky-Zerlegung	99
5.5 Zusammenfassung	105
6 Die Singulärwertzerlegung	107
6.1 Motivation	107
6.2 Singulärwerte und Singulärvektoren	111
6.3 Reduzierte SVD und Pseudoinverse	121
6.4 Anwendungen	130
6.5 Zusammenfassung	139
7 Kleinste Quadrate	143
7.1 Kleinste Quadrate: Problem und Motivation	143
7.2 Kondition des Kleinste-Quadrate-Problems	149
7.3 Normalgleichungen und Cholesky-Zerlegung	154
7.4 QR-Zerlegung und Gram-Schmidt-Verfahren	158
7.5 Konstruktion von Q: Givens-Rotationen und Householder-Spiegelungen	164
7.6 Pseudoinverse, SVD-Lösung und Methodenvergleich	172
8 Numerische Lineare Algebra: Iteration & Zufall	179
8.1 Eigenwertprobleme: Potenzmethode und QR-Iteration	179
8.2 Anwendungen: PageRank, PCA und approximative SVD	191
8.3 Iterative Löser für lineare Gleichungssysteme	194
8.4 Probabilistische Methoden: Matrix-Sketching	202
8.5 Zusammenfassung	212
9 Tensoren & Tensorprodukte	215
9.1 Multilineare Abbildungen	215
9.2 Tensoren	219
9.3 Produkte von Tensoren	226
9.4 Tensorprodukt von Vektorräumen	237
9.5 Zusammenfassung	244
10 Differentialrechnung	251
10.1 Ableitung als lineare Approximation	251
10.2 Der Gradient: Vektor zu Skalar	257

10.3 Die Jacobimatrix: Vektor zu Vektor	265
10.4 Ableitungen mit Matrizen	275
10.5 Stetigkeit und Linearität	285
10.6 Produkt- und Kettenregel	292
10.7 Ableitungen höheren Grades	302
10.8 Taylorapproximation	315
10.9 Zusammenfassung	326
11 Konvexität	335
11.1 Konvexkombinationen und konvexe Hülle	335
11.2 Konvexe Mengen	342
11.3 Projektion und konvexe Funktionen	354
11.4 Eigenschaften konvexer Funktionen	365
11.5 Konvexe Optimierung und Zusammenfassung	378
12 Nichtlineare Gleichungen & Optimierung	389
12.1 Nichtlineare Gleichungen	389
12.2 Optimalität und Sattelpunkte	402
12.3 Nelder-Mead und Gradientenabstieg	411
12.4 Newton, Quasi-Newton und SGD	424
12.5 Beschränkte Optimierung	437
12.6 Optimierung in R und Zusammenfassung	443
13 Funktionsapproximation	449
13.1 Approximation, Interpolation, Glättung	449
13.2 Interpolation durch Basisdarstellung	454
13.3 Polynominterpolation	461
13.4 Splines und B-Splines	469
13.5 Minimale Krümmung	479
13.6 Approximationsfehler	486
13.7 Glättung und Regression	492
13.8 Bias-Varianz und Modellwahl	501
13.9 Multivariat und Zusammenfassung	510

Worum geht's?

1.1 Worum geht es in diesem Skript?

Statistik war einmal Papier-und-Bleistift-Arbeit an kleinen Tabellen. Heute ist sie (genau wie das maschinelle Lernen) durch und durch *computational*: Jedes Modell, das wir fitten, jede Prognose, die wir berechnen, entsteht durch einen Algorithmus auf einem Rechner. In diesem Skript lernen wir die mathematischen Konzepte, mit denen wir solche Algorithmen entwickeln, verstehen und beurteilen können. Es geht also nicht um neue Statistik, sondern um die Frage: *Wie rechnet man das eigentlich – schnell, sparsam und zuverlässig?*

Warum Numerik?

Ein Beispiel zeigt, warum die Formeln aus den Statistik-Vorlesungen allein nicht reichen. Wir wollen ein lineares Regressionsmodell mit $p = 10\,000$ Merkmalen auf $n = 100\,000$ Beobachtungen trainieren, also das Kleinste-Quadrate-Problem

$$\min_{\beta \in \mathbb{R}^p} \|\mathbf{X}\beta - \mathbf{y}\|_2^2$$

lösen. Die Theorie liefert die Formel $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$. Wer sie wörtlich in den Rechner tippt, erlebt gleich drei Überraschungen. Erstens die *Kondition*: Die Matrix $\mathbf{X}^\top \mathbf{X}$ kann so empfindlich sein, dass winzige Rundungsfehler das Ergebnis komplett verfälschen. Zweitens die *Komplexität*: Das Invertieren kostet $O(p^3)$ Rechenoperationen, für $p = 10\,000$ rund 10^{12} Gleitkommaoperationen. Drittens der *Speicher*: $\mathbf{X}^\top \mathbf{X}$ ist eine $p \times p$ -Matrix und belegt allein schon etwa 800 MB. Bessere Wege (die QR-Zerlegung, die Singulärwertzerlegung, iterative Verfahren) sind genau der Stoff dieses Skripts.

Beispiel 1.1.1 (Komplexität der Matrixmultiplikation)

Für zwei $n \times n$ -Matrizen hängt der Rechenaufwand stark vom gewählten Algorithmus ab:

Algorithmus	Rechenoperationen für $n = 1000$	Komplexität
Naive Methode	$\approx 10^9$	$O(n^3)$
Strassen	$\approx 6 \cdot 10^8$	$O(n^{2,807})$
Bester bekannter Algorithmus	$\approx 2 \cdot 10^8$	$O(n^{2,373})$

Die Exponenten liegen scheinbar nah beieinander, wirken sich bei großen Matrizen aber stark aus. Welches Verfahren praktisch gewinnt, hängt zusätzlich von Konstanten und Speichertzugriffen ab.

Für $n = 10\,000$ stehen damit grob 10^{12} Operationen der naiven Methode etwa $2 \cdot 10^{10}$ Operationen gegenüber. Das ist ein möglicher Faktor von 50. Für die Statistik ist dieser Unterschied

entscheidend, weil Matrixmultiplikationen etwa beim Schätzen großer Modelle immer wieder anfallen.

Wie dramatisch das erste Problem werden kann, zeigt schon ein Gleichungssystem mit zwei Unbekannten:

Beispiel 1.1.2 (Ein schlecht konditioniertes Problem)

Wir lösen $Ax = b$ mit

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1,0001 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 2,0001 \end{pmatrix}.$$

Subtrahieren wir die erste Zeile von der zweiten, bleibt $0,0001 x_2 = 0,0001$, also $x_2 = 1$ und damit $x_1 = 1$: Die exakte Lösung ist $x = (1, 1)^\top$. Nun stören wir die rechte Seite minimal (etwa so, wie es ein Rundungsfehler täte) im zweiten Eintrag:

$$\tilde{b} = \begin{pmatrix} 2 \\ 2,0002 \end{pmatrix} \implies 0,0001 x_2 = 0,0002 \implies \tilde{x} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}.$$

Die winzige Störung wird bei der Division durch 0,0001 massiv verstärkt: Aus $x_2 = 1$ wird $\tilde{x}_2 = 2$, aus $x_1 = 1$ wird $\tilde{x}_1 = 0$ – ein Fehler von 100 % in der Lösung, ausgelöst durch einen Datenfehler von 0,005 %. Solche Probleme heißen *schlecht konditioniert*; die Konditionszahl misst das präzise und begleitet uns durch das ganze erste Drittel des Skripts.

Die drei Themenblöcke

Das Skript besteht aus drei Blöcken, die aufeinander aufbauen. Der erste Block, *Numerische Lineare Algebra*, klärt zunächst, was einen guten Algorithmus ausmacht (Kondition, Stabilität, Komplexität), und entwickelt dann Matrix-Zerlegungen, mit denen wir Gleichungssysteme und KQ-Probleme $Ax \approx b$ zuverlässig lösen, statt naiv zu invertieren.

Der zweite Block, *Analysis und Optimierung*, verallgemeinert die Ableitung auf Funktionen mit Vektor-, Matrix- und Tensor-Argumenten. Wozu? Beim Training neuronaler Netze müssen wir Verlustfunktionen wie $L(W) = \frac{1}{2} \|XW - Y\|_F^2$ nach einer ganzen Matrix W ableiten. Naiv wären das so viele einzelne partielle Ableitungen, wie W Einträge hat, bei 1000×10 schon 10 000; der Matrix-Kalkül liefert stattdessen eine einzige Zeile, $\partial L / \partial W = X^\top (XW - Y)$, und die ist auch noch effizient implementierbar. Darauf setzt die numerische Optimierung auf: Gradientenverfahren und ihre Verwandten finden Minima von Funktionen, die nicht konvex, hochdimensional (GPT-3: $1,75 \cdot 10^{11}$ Parameter!) und nur häppchenweise auswertbar sind.

Der dritte Block, *Funktionsapproximation*, fragt: Wie ersetzen wir eine komplizierte Funktion durch eine einfache, mit der sich gut rechnen lässt? Das klassische Werkzeug ist die Taylor-Approximation: Schon $e^x \approx 1 + x + \frac{x^2}{2}$ liefert bei $x = 0,5$ den Wert 1,625 statt $e^{0,5} = 1,6487 \dots$, ein relativer Fehler von unter 2 %. Später kommen flexiblere Bausteine wie Splines dazu, mit denen die Statistik glatte Funktionen aus Daten schätzt.

Die drei Blöcke bauen aufeinander auf; welches Kapitel im Einzelnen worauf aufbaut, zeigt die Landkarte in Abschnitt 1.2.

Wie dieses Skript funktioniert

Drei Elemente helfen beim Lesen. *Erstens*: Gestrichelt unterstrichene Begriffe wie Eigenwerte öffnen Tooltips, die das nötige Vorwissen auffrischen, oft mit weiteren Links darin, sodass wir uns bei Bedarf bis zu den Grundlagen durchhangeln können. *Zweitens*: Blaue „Interaktiv“-Kästen enthalten Widgets zum Selbst-Ausprobieren, jeweils mit einer Aufgabe und der Auswertung dazu; sie gehören zum roten Faden. Ausklappbare „Vertiefung“-Boxen sammeln dagegen Zusatzstoff – Exkurse, längere Beweise, Übersichten –, den wir beim ersten Lesen überspringen können; die erste steht am Ende von Abschnitt 1.2. *Drittens*: Beweise sind als Stepper gebaut: Wir decken sie Schritt für Schritt auf, und jeder Schritt trägt seine Begründung. Am Ende jedes Abschnitts verweist eine „Vertiefung“-Zeile auf Literatur zum Weiterlesen.

Selbsttest: Reicht das Vorwissen?

Das Skript setzt Lineare Algebra I, Analysis I und etwas R-Programmierung voraus. Die folgenden Fragen sollten wir (zumindest nach kurzem Nachdenken) beantworten können. Wer hängt, klappt die Lösung auf und frischt das Thema per Tooltip auf; sehr zu empfehlen ist außerdem, die geometrische Intuition mit den Videos von *3blue1brown* („Essence of Linear Algebra“) aufzubauen.

Lineare Algebra:

1. Was ist der Rang einer Matrix? Wie berechnen wir ihn?

Der Rang ist die Anzahl linear unabhängiger Spalten (gleichwertig: Zeilen), also die Dimension des Bildes der zugehörigen Abbildung. Von Hand berechnen wir ihn z. B. per Gauß-Elimination: Rang = Anzahl der Nichtnullzeilen in der Stufenform.

2. Was bedeutet es, dass eine Matrix A invertierbar ist?

Es gibt eine Matrix A^{-1} mit $AA^{-1} = A^{-1}A = I$ (Inverse). Das setzt voraus, dass A quadratisch ist und vollen Rang hat; gleichwertig: 0 ist kein Eigenwert, und $Ax = b$ hat für jedes b genau eine Lösung.

3. Was sind Eigenwerte und Eigenvektoren? Wie finden wir sie?

Ein Eigenvektor $v \neq 0$ mit Eigenwert λ erfüllt $Av = \lambda v$: Die Matrix skaliert v nur, statt seine Richtung zu ändern. Von Hand finden wir Eigenwerte als Nullstellen des charakteristischen Polynoms $\det(A - \lambda I) = 0$; am Rechner mit numerischen Verfahren.

4. Was bedeutet das Matrix-Vektor-Produkt Ax geometrisch?

Ax wendet die lineare Abbildung A auf den Vektor x an (Matrix-Vektor-Produkt): Je nach A wird x gestreckt, gedreht, gespiegelt, geschert oder in einen Unterraum projiziert.

5. Wann bilden Vektoren $v_1, \dots, v_n$ eine Basis des $\mathbb{R}^n$? Was ist ihr Span?

Die Vektoren bilden eine Basis des $\mathbb{R}^n$, wenn sie linear unabhängig sind und den gesamten $\mathbb{R}^n$ aufspannen. Bei n Vektoren im $\mathbb{R}^n$ genügt bereits eine dieser beiden Bedingungen. Ihr Span ist die Menge aller Linearkombinationen $\alpha_1 v_1 + \dots + \alpha_n v_n$ mit $\alpha_1, \dots, \alpha_n \in \mathbb{R}$.

Analysis:

1. Was bedeutet es, dass eine Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$ stetig ist?

Kleine Änderungen im Argument führen zu kleinen Änderungen im Funktionswert (Stetigkeit). Formal: Zu jedem $\varepsilon > 0$ gibt es ein $\delta > 0$, sodass $|x - x_0| < \delta$ stets $|f(x) - f(x_0)| < \varepsilon$ erzwingt.

2. Was ist die Ableitung $f'(x_0)$ geometrisch?

Die Steigung der Tangente an den Graphen von f im Punkt x_0 , also die momentane Änderungsrate von f an dieser Stelle (Ableitung).

3. Was unterscheidet ein lokales von einem globalen Minimum?

Ein lokales Minimum x_0 erfüllt $f(x_0) \leq f(x)$ nur für alle x in einer Umgebung von x_0 ; ein globales Minimum für *alle* x im Definitionsbereich. Bei nichtkonvexen Problemen können Abstiegsverfahren an lokalen Minima oder anderen stationären Punkten enden. Bei konvexen Zielfunktionen ist dagegen jedes lokale Minimum bereits global.

Vertiefung: Heath §1 (wissenschaftliches Rechnen: Näherungen, Fehler, Kondition im Überblick); MML, Vorwort und Teil I als Panorama der Mathematik hinter dem maschinellen Lernen.

1.2 Landkarte des Skripts

Die dreizehn Kapitel dieses Skripts sind zum Durchlesen von vorne nach hinten gebaut – aber sie hängen enger zusammen, als eine lineare Reihenfolge zeigen kann. Manche Kapitel sind Fundament für fast alles Spätere (die Normen aus Kapitel 3, die Fehleranalyse aus Kapitel 4), andere greifen weit voraus oder zurück: Die Funktionsapproximation in Teil 3 etwa braucht kein einziges Ergebnis aus Teil 2, dafür umso mehr aus Teil 1. Die folgende Karte macht diese Struktur sichtbar.

Interaktiv: Landkarte: worauf ein Kapitel aufbaut und wohin es führt

Interaktives Element — nur in der Web-Fassung

Die Kästen sind die Kapitel in Lesereihenfolge, die Bögen sind Abhängigkeiten über Nachbarkapitel hinweg. Wählen wir ein Kapitel aus, hebt die Karte hervor, worauf es aufbaut und wohin es führt – nützlich vor dem Lesen (*was sollte ich parat haben?*) genauso wie bei der Prüfungsvorbereitung (*wofür brauche ich das noch?*).

Vertiefung: Die Kapitel im Detail

Die drei Teile des Skripts

Teil 1 – Numerische lineare Algebra (Kapitel 2–9). Die Leitfrage: *Wie lassen sich Probleme der linearen Algebra auf dem Rechner effizient und genau lösen?* Kapitel 2 stellt dafür die Sprache bereit – was ein numerisches Problem, ein Algorithmus und sein Aufwand in Landau-Notation ist. Kapitel 3 liefert die Messwerkzeuge (Spur und Matrixnormen), Kapitel 4 die zentrale Unterscheidung des ganzen Skripts: *Kondition* ist eine Eigenschaft des Problems, *Stabilität* eine Eigenschaft des Algorithmus – nur an letzterer können wir drehen. Darauf bauen die Rechenkerne auf: Kapitel 5 löst Gleichungssysteme über Zerlegungen (LU und Cholesky) statt über die goldene Regel des Skripts zu verstoßen – *niemals eine Matrix invertieren!* –, Kapitel 6 entwickelt mit der Singulärwertzerlegung die wichtigste Matrixzerlegung der angewandten Mathematik, und Kapitel 7 vergleicht drei Lösungswege für das Kleinste-Quadrate-Problem – Normalengleichungen, QR, SVD – nach genau den Kriterien aus Kapitel 4. Kapitel 8 tauscht dann Exaktheit gegen Geschwindigkeit (Iteration und Zufall, von der Potenzmethode bis zum Matrix-Sketching), und Kapitel 9 baut mit Tensoren und Tensorprodukten multivariate Strukturen aus univariaten Bausteinen.

Teil 2 – Analysis & Optimierung (Kapitel 10–12). Die Leitfrage: *Wie differenziert und optimiert man Funktionen jenseits des Eindimensionalen?* Kapitel 10 verallgemeinert die Ableitung nach dem Prinzip „Ableitung = lineare Approximation“ auf Vektoren und Matrizen (Gradient, Jacobi-Matrix) und baut

darauf die Rechenregeln, die Hesse-Matrix und die Taylor-Entwicklung auf; nebenbei fallen die Normalengleichungen aus Kapitel 7 als Rechnung im Matrixkalkül ab. Kapitel 11 behandelt Konvexität als das Versprechen, das lokale Verfahren global macht: Bei konvexen Zielfunktionen ist jedes lokale Minimum bereits das globale. Kapitel 12 setzt darauf die Verfahren, sortiert nach der Ordnung der verwendeten Ableitungen: Nelder-Mead (nullte), Gradientenabstieg mit Line Search und SGD (erste), Newton und Quasi-Newton (zweite), dazu Optimierung unter Nebenbedingungen. Eine feinere Karte dieses Teils – von der Fréchet-Ableitung bis zu den KKT-Bedingungen – steht am Anfang von Kapitel 10.

Teil 3 – Funktionsapproximation (Kapitel 13). Die Leitfrage: *Wie ersetzen wir eine komplizierte Funktion durch eine einfache, mit der sich gut rechnen lässt?* Kapitel 13 überträgt zuerst die Idee der Basis aus dem $\mathbb{R}^n$ auf Funktionenräume, stellt Funktionen als Linearkombination $\hat{f} = \sum_k a_k \phi_k$ von Basisfunktionen dar – die Koeffizienten liefert wieder ein Gleichungssystem oder ein KQ-Problem – und erklärt, warum Polynominterpolation scheitert (Runge-Phänomen) und B-Splines funktionieren. Die zweite Hälfte des Kapitels macht daraus Statistik: Glättung verrauschter Daten als Regressionsproblem, die Bias-Varianz-Abwägung bei der Wahl der Basisgröße und der Fluch der Dimension im Multivariaten, dem additive Modelle (GAMs) entkommen. Dieser Teil setzt Teil 2 *nicht* voraus – seine Anker liegen in den Kapiteln 1, 4, 5, 7 und 9; die feinere Karte dazu steht am Anfang von Kapitel 13.

Vertiefung: Was sich durch das ganze Skript zieht

Vier Themen tauchen in allen drei Teilen wieder auf; sie sind das eigentliche Gerüst des Skripts.

Stabilität. Wähle Algorithmen, die Fehler nicht verstärken. Die Konditionszahl des Problems ist uns vorgegeben, die Stabilität des Algorithmus nicht – deshalb Zerlegungen statt Inversion, orthogonale Matrizen (perfekt konditioniert, $\kappa = 1$) wo immer möglich, und schlecht konditionierte Schritte so früh wie möglich.

Rechenaufwand. Ein kleines Einmaleins der Komplexitäten, das sich einzuprägen lohnt: Vektoroperationen $O(n)$, Matrix-Vektor-Produkte $O(n^2)$, Matrixzerlegungen $O(n^3)$, iterative Verfahren $O(n^2 \log(1/\epsilon))$ – und als Warnung der Fluch der Dimension mit $O(K^p)$.

Matrixzerlegungen. Das Werkzeugregal von Teil 1, auf das alles Spätere zurückgreift:

Zerlegung	Form	Kapitel	typischer Einsatz
LU	$A = LU$	5	allgemeine Gleichungssysteme
Cholesky	$A = LL^T$	5	SPD-Matrizen, Simulation aus $\mathcal{N}(\mathbf{0}, \Sigma)$
QR	$A = QR$	7	Kleinste Quadrate, stabil
SVD	$A = U\Sigma V^T$	6	universell: Rang, Pseudoinverse, Approximation

Von exakt zu approximativ. Die Problemklassen des Skripts bilden eine Treppe wachsender Allgemeinheit – und abnehmender Garantien: lineare Probleme haben exakte Lösungen per Zerlegung; Kleinste Quadrate lösen wir als Projektion; nichtlineare Gleichungen nur noch iterativ (Newton); allgemeine Optimierung mit Gradientenverfahren, die lokal suchen; erst Konvexität gibt globale Garantien zurück; und die Funktionsapproximation handelt schließlich Genauigkeit gegen Aufwand.

Vertiefung: Die Referenzen des Skripts, nach Teilen sortiert – zu Teil 1 Heath, Scientific Computing, §1–4 (und §11 für die iterativen Verfahren) sowie MML §2–4; zu Teil 2 MML §5 und §7 und Heath §5–6, für die Konvexität Boyd & Vandenberghe, Convex Optimization (frei verfügbar); zu Teil 3 Heath §7. Für die geometrische Intuition zur linearen Algebra außerdem die 3blue1brown-Videoreihe „Essence of Linear Algebra“.

Algorithmen & Komplexität

2.1 Numerische Probleme und Algorithmen

Mit diesem Kapitel beginnt die eigentliche Numerik. Statistische Methoden sind am Ende immer Rechenvorschriften, die ein Computer ausführt, und Computer rechnen anders als die Mathematik auf dem Papier. In diesem Kapitel klären wir zuerst, was ein *numerisches Problem* und was ein *Algorithmus* überhaupt ist. Dann sehen wir an konkreten Beispielen, warum es entscheidend darauf ankommt, *wie* wir etwas berechnen: dieselbe Formel kann als Rechenvorschrift brillant oder katastrophal sein. In den folgenden Abschnitten fragen wir dann, wie *teuer* ein Algorithmus ist, und entwickeln mit den Landau-Symbolen die Sprache, um Rechenaufwand zu vergleichen.

Verwendete Vorkenntnisse

Wir brauchen in diesem Kapitel nur Grundbegriffe aus dem ersten Semester: aus der Analysis Folgen und ihre Grenzwerte sowie die Grundidee der Konvergenz; aus der linearen Algebra Matrizen und ihre Grundoperationen, insbesondere Matrix-Vektor- und Matrixmultiplikation; dazu ein erstes Gespür dafür, wie viele Einzelrechnungen darin stecken.

Numerische Probleme

Was ist ein numerisches Problem? Kurz gesagt: eine Rechenaufgabe, deren Lösung aus einer oder mehreren Zahlen besteht. Drei typische Beispiele aus der Statistik: der Wert eines Integrals (etwa eine Wahrscheinlichkeit als Fläche unter einer Dichte), die Lösung eines linearen Gleichungssystems (etwa die Koeffizienten einer Regression) oder die Basis-Koeffizienten einer Funktionsapproximation. Allgemein fassen wir das so:

Definition 2.1.1 (Numerisches Problem)

Ein *numerisches Problem* ist eine Aufgabe der Form: *Gegeben ein Problem f mit Input $\mathbf{x}$, berechne die Lösung $f(\mathbf{x})$.*

Die Schreibweise ist bewusst abstrakt: f ist die mathematische Abbildung von den Eingabedaten auf die exakte Lösung; beim Gleichungssystem $\mathbf{A}\mathbf{y} = \mathbf{b}$ etwa $f(\mathbf{A}, \mathbf{b}) = \mathbf{A}^{-1}\mathbf{b}$. Ob und wie wir $f(\mathbf{x})$ tatsächlich ausrechnen können, ist damit noch völlig offen. Und da beginnen die Schwierigkeiten.

Zwei grundsätzliche Schwierigkeiten

Computer sind so gut wie nie exakt. Das liegt nicht an schlampiger Technik, sondern an zwei prinzipiellen Beschränkungen jeder endlichen Maschine:

Bemerkung 2.1.2 (Grenzen des Rechnens)

- Computer können nur *endlich viele Zahlen mit endlich vielen Stellen* darstellen. Schon

$$\pi = 3,141592653589793238462643383279 \dots$$

passt in keinen endlichen Speicher. Gespeichert wird immer nur eine gerundete Näherung.

- Computer können nur *endlich viele Rechenoperationen* ausführen. Schon die harmlose Exponentialfunktion ist über eine unendliche Reihe definiert,

$$e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!},$$

und unendlich viele Summanden kann keine Maschine aufaddieren. Jede Auswertung bricht irgendwo ab.

Deshalb ist es wichtig, *wie* wir Dinge berechnen. Wie schlimm kann es schon sein? Prüfen wir unsere Intuition an vier kurzen R-Ausdrücken. Tippen wir bei jedem zuerst, was herauskommt, und decken erst dann die Ausgabe auf:

Interaktives Element — nur in der Web-Fassung

Halten wir die Beobachtungen fest: Computer können nur endlich viele Zahlen darstellen. Das führt zu *Rundungsfehlern* (Rundungsfehler; ausführlich in Kapitel 4, nach den Normen aus Kapitel 3), und diese Fehler können sich *akkumulieren*. Der Fehler einer einzelnen Operation ist meist vernachlässigbar klein. Gefährlich wird die Summe vieler kleiner Fehler. Und weil jede Operation frisch rundet, spielt sogar die *Reihenfolge* unserer Rechenschritte eine Rolle. Eine erste praktische Empfehlung, die wir gleich zweimal in Aktion sehen: besonders große und besonders kleine Zahlen im selben Ausdruck vermeiden.

Zwei warnende Beispiele

Die beiden folgenden Beispiele zeigen die zwei wichtigsten Mechanismen, mit denen Gleitkomma-rechnung schiefgeht. Beide sehen auf dem Papier völlig harmlos aus.

Beispiel 2.1.3 (Katastrophale Auslöschung)

Wir berechnen die Varianz von $\mathbf{x}$ mit der bekannten Verschiebungsformel

$$\text{Var}(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n x_i^2 - \left(\frac{1}{n} \sum_{i=1}^n x_i \right)^2$$

(wir teilen der Einfachheit halber durch n statt $n - 1$; am Phänomen ändert das nichts). Die Daten sind vier kleine Zahlen, verschoben um eine große Konstante:

$$\mathbf{x} = (4 + 10^9, 7 + 10^9, 13 + 10^9, 16 + 10^9).$$

Rechnen wir zunächst exakt. Der Mittelwert ist $\bar{x} = 10^9 + 10$, die Abweichungen davon sind $-6, -3, 3, 6$, also

$$\text{Var}(x) = \frac{36 + 9 + 9 + 36}{4} = \frac{90}{4} = 22,5.$$

Für die beiden Terme der Verschiebungsformel gilt exakt

$$\frac{1}{n} \sum_{i=1}^n x_i^2 = 10^{18} + 2 \cdot 10^{10} + 122,5, \quad \bar{x}^2 = 10^{18} + 2 \cdot 10^{10} + 100,$$

und die Differenz $122,5 - 100 = 22,5$ stimmt. Aber: Beide Terme sind riesig ($\approx 10^{18}$), und ihre gesamte Information über die Varianz steckt in den letzten drei Ziffern. Bei der Größenordnung 10^{18} liegen benachbarte Gleitkommazahlen aber schon 128 auseinander. Feiner kann die Maschine dort nicht auflösen. In R (Doppelpräzision) werden deshalb *beide* Terme auf dieselbe Maschinenzahl gerundet:

$$1\,000\,000\,020\,000\,000\,128 - 1\,000\,000\,020\,000\,000\,128 = 0.$$

Die berechnete „Varianz“ ist 0 statt 22,5 – nicht ungefähr falsch, sondern komplett informationsfrei. Die zweistufige Rechnung $\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ liefert dagegen exakt 22,5, denn sie subtrahiert die großen Zahlen, *bevor* quadriert wird, und arbeitet danach nur noch mit den kleinen Abweichungen.

Ursache des Desasters ist die Subtraktion zweier fast gleich großer Zahlen: Die übereinstimmenden führenden Ziffern heben sich weg, übrig bleiben nur die (verrauschten) hinteren Stellen. Dieses Phänomen heißt *katastrophale Auslöschung* (engl. *catastrophic cancellation*) und ist der Klassiker unter den numerischen Fallen.

Beispiel 2.1.4 (Verletzte Assoziativität)

Gleitkomma-Addition ist nicht assoziativ! Seien $x = 10^{30}$, $y = -10^{30}$ und $z = 1$. Mathematisch ist $x + y + z = 1$, egal wie wir klammern. Die Maschine rechnet:

$$(x + y) + z = (10^{30} + (-10^{30})) + 1 = 0 + 1 = 1,$$

$$x + (y + z) = 10^{30} + ((-10^{30}) + 1) \stackrel{!!}{\approx} 10^{30} + (-10^{30}) = 0.$$

Die erste Klammerung ist exakt: $x + y = 0$ löscht die riesigen Zahlen sauber aus, danach überlebt die 1. In der zweiten Klammerung muss die Maschine $-10^{30} + 1$ als Gleitkommazahl speichern. Aber bei der Größenordnung 10^{30} liegen benachbarte Maschinenzahlen etwa 10^{14} auseinander. Die 1 ist viel kleiner als diese Auflösung, das Zwischenergebnis wird auf -10^{30} zurückgerundet, und am Ende steht 0 statt 1.

⇒ Die Reihenfolge (Klammerung) einer Summe beeinflusst das Ergebnis. Das haben wir oben im Selbsttest bei `sum(x) - sum(rev(x))` beobachtet: vorwärts und rückwärts summiert ergeben dieselben 10^5 Zahlen verschiedene Summen.

Die beiden Beispiele sehen verschieden aus, aber sie scheitern am selben Mechanismus: An der Stelle, an der die Maschine rechnet, liegen benachbarte darstellbare Zahlen weiter auseinander als die Größe, die wir eigentlich suchen. Wie weit dürfen wir die Verschiebung treiben, bis es so weit ist? Das lässt sich nicht ansehen, das müssen wir ausprobieren.

Interaktiv: Zwei Gesichter derselben Auslöschung

Schätzfrage. Bei welcher Verschiebung c verliert die Verschiebungsformel die Varianz zum ersten Mal?

Lösung: e^8

Die Tafel macht das Kriterium sichtbar: Solange die orange Auflösung links von der grünen Zielgröße steht, überlebt das Ergebnis; sobald sie rechts davon steht, ist es weg. Die Verschiebungsformel verliert schon bei $c = 10^8$ die erste Nachkommastelle (22 statt 22,5) und ab $c = 10^9$ jede Information; die Klammerung der Summe hält bis 10^{15} und bricht bei 10^{16} .

Algorithmen

Die Beispiele zeigen: Zum numerischen Problem f gehört immer noch eine zweite Zutat, nämlich die konkrete Rechenvorschrift, mit der wir $f(\mathbf{x})$ zu berechnen versuchen. Für dieselbe Varianz gab es zwei Vorschriften mit drastisch verschiedenem Ausgang. Diese zweite Zutat bekommt jetzt einen Namen:

Definition 2.1.5 (Algorithmus)

Ein *Algorithmus* ist ein Verfahren $\tilde{f} = \tilde{f}_s \circ \dots \circ \tilde{f}_1$, das für Inputs $\mathbf{x}$ eine mögliche Lösung $\tilde{f}(\mathbf{x})$ berechnet.

Lesen wir die Definition genau. Ein Algorithmus ist eine Verkettung $\tilde{f}_s \circ \dots \circ \tilde{f}_1$ endlich vieler elementarer Rechenschritte $\tilde{f}_1, \dots, \tilde{f}_s$: erst wird $\tilde{f}_1$ auf den Input angewandt, dann $\tilde{f}_2$ auf dessen Ergebnis, und so weiter. Die Tilde ist Programm: $\tilde{f}$ ist nicht dasselbe wie f , sondern nur ein Versuch, f nachzubauen; die Definition verspricht vorsichtig nur eine „mögliche Lösung“. Wie gut $\tilde{f}(\mathbf{x})$ die wahre Lösung $f(\mathbf{x})$ trifft, ist genau die Frage, die uns in Kapitel 4 beschäftigen wird. Zu einem Problem f gibt es dabei meist viele verschiedene Algorithmen, und sie unterscheiden sich in Genauigkeit *und* Rechenaufwand.

Bemerkung 2.1.6 (Arten von Algorithmen)

Die gebräuchlichen Bezeichnungen beschreiben drei *unabhängige Achsen*:

- *exakt oder approximativ*: Trifft das Verfahren die mathematische Lösung in exakter Arithmetik, oder entsteht bereits durch das Verfahren ein Approximationsfehler? Eine abgebrochene Reihe für e^x ist approximativ.
- *direkt oder numerisch iterativ*: Liefert eine vorab endliche Folge von Rechenschritten die Lösung, oder erzeugt das Verfahren eine Folge von Näherungen mit Abbruchkriterium? Das Gauß-Verfahren ist direkt; Newton- und Gradientenverfahren sind iterativ.
- *deterministisch oder randomisiert*: Ist das Ergebnis bei gleichem Input vollständig festgelegt, oder verwendet das Verfahren Zufall? Die Monte-Carlo-Integration ist randomisiert.

Eine Programmschleife allein macht ein Verfahren noch nicht *numerisch iterativ*: Auch ein direkter Algorithmus kann Schleifen enthalten. Umgekehrt können sich die Achsen beliebig kombinieren. Das stochastische Gradientenverfahren etwa ist numerisch iterativ, approximativ und randomisiert.

Im nächsten Abschnitt sehen wir an einem klassischen Beispiel (den Fibonacci-Zahlen), dass zwei exakte Algorithmen für dasselbe Problem sich im Rechenaufwand so drastisch unterscheiden kön-

nen, dass der eine praktisch unbrauchbar ist.

Selbsttest

1. Ab welchem Exponenten k liefern im Widget oben die beiden Klammerungen von $10^k + (-10^k) + 1$ verschiedene Ergebnisse?

Lösung: 16

Bei 10^{15} liegen benachbarte Maschinenzahlen 0,125 auseinander, die 1 überlebt also die Zwischensumme. Bei 10^{16} ist der Abstand bereits 2, und $-10^{16} + 1$ wird auf -10^{16} zurückgerundet (Beispiel 2.1.4).

2. Die zweistufige Varianzformel $\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$ versagt bei genügend großer Verschiebung genauso wie die Verschiebungsformel.

Antwort: Falsch

Sie liefert im Widget für jedes k exakt 22,5. Sie subtrahiert die großen Zahlen, *bevor* quadriert wird, und rechnet danach nur noch mit den kleinen Abweichungen $-6, -3, 3, 6$; zwei riesige, fast gleiche Zahlen treffen dabei nie aufeinander.

3. Ein Rundungsfehler pro Operation ist harmlos, gefährlich ist erst das Zusammenspiel vieler Operationen.

Antwort: Wahr

Darum ging es in beiden Beispielen: Jede einzelne Rundung ist relativ winzig, aber eine Subtraktion fast gleich großer Zahlen macht aus diesem winzigen relativen Fehler einen riesigen relativen Fehler des Ergebnisses.

Vertiefung: Heath §1.1–1.2 (wissenschaftliches Rechnen, Näherungen und Fehlerquellen); die Gleitkomma-Arithmetik hinter den Beispielen behandelt Heath §1.3 sowie unser Kapitel 4, mit Normen aus Kapitel 3.

2.2 Algorithmen konkret: Fibonacci und Verwandte

In Abschnitt 2.1 haben wir Algorithmen abstrakt definiert: als Verkettung $\tilde{f} = \tilde{f}_s \circ \dots \circ \tilde{f}_1$ elementarer Rechenschritte. Diese Definition bleibt blass, solange wir sie nicht einmal vollständig durchspielen. Das tun wir jetzt an einem Problem, das einfach genug ist, um jeden einzelnen Schritt hinzuschreiben, und trotzdem reich genug, um uns durch das ganze Kapitel zu begleiten: den Fibonacci-Zahlen. Danach ordnen wir das Beispiel in die Landschaft der Algorithmenarten ein, die uns in Statistik und Machine Learning begegnen, und überlegen, was einen *guten* Algorithmus eigentlich auszeichnet.

2.2.1 Beispiel: Fibonacci-Zahlen

Definition 2.2.1 (Fibonacci-Zahlen)

Die *Fibonacci-Zahlen* sind die Folge $(x_n)_{n \in \mathbb{N}}$ mit

$$x_1 = 0, \quad x_2 = 1, \quad x_{n+1} = x_n + x_{n-1} \quad (n \geq 2).$$

Jedes Folgenglied ist also die Summe seiner beiden Vorgänger; die Folge beginnt mit 0, 1, 1, 2, 3, 5, 8, 13, 21, ... (Manche Bücher lassen die Folge bei 1, 1 oder mit dem Index 0 beginnen; wir bleiben bei der obigen Konvention.) Unser numerisches Problem lautet: *Berechne die ersten n Fibonacci-Zahlen.* In der Sprache von Abschnitt 2.1: Das Problem ist die Abbildung f mit Input n und Lösung $f(n) = (x_1, \dots, x_n)$.

Wie berechnen wir das? Die Definition selbst gibt den Weg vor: Wir bauen den Ergebnisvektor von links nach rechts auf und gewinnen jeden neuen Eintrag durch *eine* Addition aus den beiden zuletzt berechneten. Als Verkettung elementarer Schritte (Komposition von Funktionen) aufgeschrieben:

Algorithmus 2.2.2 (Fibonacci, schleifenbasiert)

$$f : \mathbb{N} \rightarrow \bigcup_{n \geq 1} \mathbb{N}_0^n, \quad f(n) = (x_1, \dots, x_n).$$

Für $n \geq 1$ setzt der Algorithmus $x_1 = 0$, für $n \geq 2$ zusätzlich $x_2 = 1$, und führt dann aus:

```
für i = 2, ..., n-1:
    x[i+1] = x[i] + x[i-1]
gib (x[1], ..., x[n]) zurück
```

Jeder Schleifendurchlauf hängt an das bisher berechnete Tupel die Summe aus dem **letzten** und dem **vorletzten** Element als **neues Element** an. Nach dem letzten Durchlauf steht das vollständige Ergebnis da. Der Zielraum hängt von der Eingabe n ab; deshalb schreiben wir oben die disjunkte Vereinigung der möglichen Vektorräume statt des formal unzulässigen $\mathbb{N}_0^n$ als festen Zielraum.

Beispiel 2.2.3 (Die ersten 6 Fibonacci-Zahlen)

Berechnen wir $\tilde{f}(6)$ Schritt für Schritt. Die Farben verfolgen die Rollen aus Algorithmus 2.2.2: **letztes Element**, **vorletztes Element**, **neu berechnete Summe**.

$$\begin{aligned} \tilde{f}_1(6) &= 0 \\ \tilde{f}_2(6) &= (0, 1) \\ \tilde{f}_3(6) &= (0, 1, 1) \\ \tilde{f}_4(6) &= (0, 1, 1, 2) \\ \tilde{f}_5(6) &= (0, 1, 1, 2, 3) \\ \tilde{f}_6(6) &= (0, 1, 1, 2, 3, 5) \end{aligned}$$

Ergebnis: $\tilde{f}(6) = (0, 1, 1, 2, 3, 5)$ – vier Additionen (1, 2, 3, 5) für sechs Zahlen.

2.2.2 Vom Algorithmus zum Programm

Der Algorithmus lässt sich fast wörtlich in Code übersetzen. Lege zuerst einen Vektor der Länge n an, mit Nullen vorbelegt; damit ist $x_1 = 0$ bereits erledigt. Setze dann (falls $n > 1$) das zweite Element auf 1. Durchlaufe schließlich $i = 2, \dots, n-1$ und setze in jedem Durchlauf $x_{i+1} = x_i + x_{i-1}$. In R:

```
fibonacci <- function(n) {
  x <- numeric(n) # Vektor (0, ..., 0) der Länge n
  if (n > 1) {
    x[2] <- 1
```

```

}
if (n > 2) {
  for (i in 2:(n - 1)) {
    x[i + 1] <- x[i] + x[i - 1]
  }
}
x
}

```

Eine Schleife, eine Addition pro Durchlauf, und jedes Zwischenergebnis wird genau einmal berechnet und dann wiederverwendet.

Interaktiv: Wie oft rechnet die Rekursion doppelt?

Dass hier *nichts doppelt* gerechnet wird, ist keine Selbstverständlichkeit. Die Definition 2.2.1 legt nämlich noch einen zweiten, verführerisch eleganten Weg nahe: Um x_n zu berechnen, rufe dieselbe Rechenvorschrift rekursiv für x_{n-1} und x_{n-2} auf, ohne Zwischenergebnisse zu speichern. Das liefert dieselben Zahlen, aber zu einem absurden Preis: Beide Teilaufufe berechnen große Teile der Folge unabhängig voneinander noch einmal, und deren Teilaufufe wieder. Wie oft ist „noch einmal“ – zweimal, fünfmal, zwanzigmal? Das ist keine Geschmacksfrage, sondern eine Zählaufgabe.

Schätzfrage. Beim Ziel x_8 : wie oft rechnet die naive Rekursion die Zahl x_3 von vorn aus?

Lösung: 8 mal

Der Stepper zählt es aus. Bis x_8 kommt die Iteration mit sechs Additionen aus und hat dabei *alle* acht Zahlen berechnet; die Rekursion braucht für die *eine* Zahl x_8 schon 20 Additionen und 41 Aufrufe, weil sie x_3 achtmal und x_2 dreizehnmal von vorn ausrechnet. Bis x_{15} sind es 609 Additionen gegen 13. Die genaue Analyse, wie schnell die naive Rekursion explodiert, folgt in Abschnitt 2.5.

2.2.3 Selbsttest

Ordnen wir Algorithmus 2.2.2 in die Arten aus Abschnitt 2.1 ein. Für jede Aussage: wahr oder falsch?

1. Der Algorithmus ist *exakt*.

Antwort: Wahr

Es gilt $f(n) = \tilde{f}(n)$: Der Algorithmus liefert genau die definierten Fibonacci-Zahlen, keine Näherung. (Die Einschränkung „bis auf Rundungsfehler“ aus der Definition greift hier erst, wenn x_n für sehr großes n den exakt darstellbaren Ganzzahlbereich der Gleitkommazahlen verlässt.)

2. Der Algorithmus ist *approximativ*.

Antwort: Falsch

Approximativ hieße $f(n) \approx \tilde{f}(n)$: der Algorithmus bricht etwa bei einem Toleranzlevel ab, bevor die exakte Lösung erreicht ist. Hier gibt es nichts zu approximieren: Nach n Schritten steht das exakte Ergebnis fest.

3. Der Algorithmus ist *numerisch iterativ*.

Antwort: Falsch

Er enthält zwar eine Schleife, erzeugt aber keine Folge von Näherungen mit einem Abbruchkriterium: Nach einer vorab durch n festgelegten Zahl von Schritten steht das exakte Ergebnis

fest. In der numerischen Terminologie ist er daher *direkt*. Die häufige Bezeichnung „iterative Fibonacci-Variante“ meint hier nur die schleifenbasierte Implementierung im Gegensatz zur Rekursion.

4. Der Algorithmus ist *probabilistisch*.

Antwort: Falsch

Es kommt kein Zufall vor: Der Algorithmus ist deterministisch und liefert bei jedem Aufruf mit demselben n exakt dasselbe Ergebnis.

5. Wie viele Additionen braucht die naive Rekursion im Stepper oben für die Zahl x_{10} ?

Lösung: 54

Der Zähler steht bei Ziel $n = 10$ und Schritt 10 auf 54; die Iteration kommt für dieselben zehn Zahlen mit 8 Additionen aus.

2.2.4 Algorithmenarten in ML und Statistik

Der Fibonacci-Algorithmus ist also exakt, direkt und deterministisch. Diese drei Angaben liegen auf den unabhängigen Achsen aus Abschnitt 2.1. Wo auf diesen Achsen liegen die Verfahren, mit denen wir in Statistik und Machine Learning tatsächlich arbeiten?

Beispiel 2.2.4 (Algorithmenarten in ML und Statistik)

Matrixmultiplikation und Gauß-Elimination sind wie unser Fibonacci-Algorithmus exakt, direkt und deterministisch. Gradientenabstieg und Newton-Verfahren erzeugen dagegen Näherungsfolgen und stoppen nach einem Abbruchkriterium: numerisch iterativ und damit zugleich approximativ. Monte-Carlo-Integration, MCMC und randomisierte Quicksort-Varianten verwenden zusätzlich Zufall. Kombiniert sind die Achsen dabei fast immer (Bemerkung 2.1.6).

2.2.5 Was ist ein guter Algorithmus?

Bemerkung 2.2.5 (Eine Analogie)

Problem: *Koche Schweinsbraten wie bei Oma*. Dann entspricht der Algorithmus dem Kochrezept, und der Computer dem Koch, der sich zwar an das Rezept hält, dabei aber Fehler macht.

Die Analogie trägt weiter, als sie zunächst wirkt. Erstens: Für dasselbe Gericht gibt es viele Rezepte, und sie sind nicht gleich gut, genau wie unsere beiden Fibonacci-Varianten dasselbe Problem lösen, aber zu drastisch verschiedenen Kosten. Zweitens: Ein gutes Rezept ist *robust* gegen die kleinen Ungenauigkeiten des Kochs; ein gutes numerisches Verfahren verstärkt die unvermeidlichen Rundungsfehler des Computers nicht unnötig. Zusammengefasst wollen wir Algorithmen, die

- möglichst exakte Ergebnisse liefern,
- schnell sind,
- wenig Speicher brauchen.

Diese drei Wünsche stehen oft in Konkurrenz: Mehr Genauigkeit kostet in der Regel Rechenzeit oder Speicher, und der schnellste Weg ist nicht immer der stabilste. Um solche Abwägungen tref-

fen zu können, müssen wir „schnell“ und „wenig Speicher“ erst einmal präzise messen. Das ist das Thema des nächsten Abschnitts. Wie wir „möglichst exakt“ trotz Rundungsfehlern erreichen, beschäftigt uns dann ausführlich in Kapitel 4; Kapitel 3 stellt dafür zunächst Normen als Fehlermaße bereit.

Vertiefung: Heath §1.1 (Näherungen und Fehlerquellen im wissenschaftlichen Rechnen); Cormen, Leiserson, Rivest & Stein, Introduction to Algorithms, der Klassiker zu Entwurf und Analyse von Algorithmen.

2.3 Aufwand und Komplexität

In Abschnitt 2.2 haben wir gesehen, dass zwei Algorithmen dasselbe Problem lösen und sich dabei dramatisch unterschiedlich anfühlen können: Der Aufruf-Zähler der naiven Fibonacci-Rekursion explodiert schon für kleine n , die iterative Variante ist sofort fertig. „Fühlt sich langsam an“ ist aber kein mathematischer Begriff. In diesem Abschnitt machen wir daraus eine Größe, die wir zählen können (den Aufwand eines Algorithmus), und lernen dann, wie man Aufwände sinnvoll vergleicht: nicht über exakte Zahlen, sondern über ihr *Skalierungsverhalten*, die Komplexität.

2.3.1 Zeit- und Speicheraufwand

Was kostet ein Algorithmus? Der Computer führt ihn letztlich als Folge *elementarer Rechenoperationen* aus: Additionen, Subtraktionen, Multiplikationen und Divisionen von Gleitkommazahlen. Jede dieser Operationen kostet ungefähr gleich viel Zeit, also ist ihre *Anzahl* ein gutes Maß für die Laufzeit. Genauso können wir zählen, wie viele Zahlen der Algorithmus unterwegs im Speicher halten muss.

Definition 2.3.1 (Zeit- und Speicheraufwand)

Ein Algorithmus werde durch elementare Operationen ausgeführt: Rechenoperationen $f_i \in \{+, -, \cdot, /\}$, aber auch Vergleiche, Zuweisungen und Speicherzugriffe, jede mit (etwa) konstantem Aufwand.

- Der *Zeitaufwand* des Algorithmus ist die Anzahl dieser elementaren Operationen. Höherer Zeitaufwand bedeutet (ungefähr) längere Laufzeit.
- Der *Speicheraufwand* ist (in etwa) die Anzahl der gespeicherten und zwischengespeicherten Zahlen. Höherer Speicheraufwand bedeutet mehr benötigten Speicherplatz.

Elementare Gleitkomma-Operationen heißen in der Numerik-Literatur auch *FLOPs* (engl. *floating point operations*); „der Algorithmus braucht 2nd FLOPs“ ist also nur eine kompakte Sprechweise für unseren Zeitaufwand. Der Zusatz „ungefähr“ in der Definition ist übrigens Absicht: Reale Laufzeiten hängen auch von Speicherzugriffen, Zwischenspeichern (Caches) und Parallelisierung ab. Für den Vergleich von Algorithmen ist das Zählen der Operationen trotzdem das richtige Werkzeug, wie wir gleich sehen werden.

2.3.2 Beispiel: Matrix-Vektor-Multiplikation

Zählen wir das an einem Arbeitspferd der Statistik konkret durch: dem Matrix-Vektor-Produkt $y = Ax$. Es steckt in jeder Vorhersage eines linearen Modells und in jeder Schicht eines neuronalen Netzes. Sein Aufwand ist also alles andere als eine akademische Frage. Wir verfolgen die Matrix

A in Rot und den Vektor **x** in Blau durch die Rechnung; das Ergebnis **y** erscheint in Grün.

Beispiel 2.3.2 (Matrix-Vektor-Multiplikation)

Berechne $y = Ax$ für $A \in \mathbb{R}^{3 \times 2}$, $x \in \mathbb{R}^2$, konkret

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \quad x = \begin{pmatrix} 7 \\ 8 \end{pmatrix}.$$

Jede Komponente von **y** ist das Skalarprodukt einer Zeile von **A** mit **x**:

$$y_1 = 1 \cdot 7 + 2 \cdot 8 = 7 + 16 = 23 \quad (2 \text{ Mult.}, 1 \text{ Add.})$$

$$y_2 = 3 \cdot 7 + 4 \cdot 8 = 21 + 32 = 53 \quad (2 \text{ Mult.}, 1 \text{ Add.})$$

$$y_3 = 5 \cdot 7 + 6 \cdot 8 = 35 + 48 = 83 \quad (2 \text{ Mult.}, 1 \text{ Add.})$$

- *Zeitaufwand*: $3 \cdot 2 = 6$ Multiplikationen und $3 \cdot 1 = 3$ Additionen, zusammen 9 Operationen.
- *Speicheraufwand*: 6 Zahlen für **A**, 2 für **x**, 3 für **y**, zusammen 11 Zahlen.

Das Muster aus dem Beispiel verallgemeinert sich direkt auf beliebige Dimensionen:

Satz 2.3.3 (Aufwand der Matrix-Vektor-Multiplikation)

Sei $A \in \mathbb{R}^{n \times d}$ und $x \in \mathbb{R}^d$. Die Berechnung von $y = Ax$ hat

- Zeitaufwand nd Multiplikationen + $n(d-1)$ Additionen = $n(2d-1) \approx 2nd$ Operationen,
- Speicheraufwand $nd + d + n$ Zahlen (für **A**, **x** und **y**).

Vertiefung: Beweis durch genaues Zählen der Operationen

Beweis.

(1)

$$y_i = \sum_{j=1}^d a_{ij} x_j, \quad i = 1, \dots, n$$

Jede Komponente y_i kostet damit d Multiplikationen und $d-1$ Additionen.

Definition des Matrix-Vektor-Produkts, komponentenweise in Summenschreibweise: d Produkte $a_{ij} x_j$, und um d Summanden aufzuaddieren, braucht es $d-1$ Additionen

(2)

$$n \cdot d + n \cdot (d-1) = n(2d-1) = 2nd - n \approx 2nd$$

*n Komponenten $y_1, \dots, y_n$, jede gleich teuer; für großes d ist der Term $-n$ gegenüber $2nd$ vernachlässigbar. Den Speicheraufwand erhalten wir, indem wir die beteiligten Objekte zählen: die Eingaben **A** (nd Einträge) und **x** (d Einträge) plus das Ergebnis **y** (n Einträge)*

□

Die Faustregel lautet also: *Matrix-Vektor kostet $\approx 2nd$ Operationen*, je zwei pro Matrixeintrag, eine Multiplikation und eine Addition. Dieselbe Zählung funktioniert für die Matrix-Matrix-Multiplikation $C = AB$ mit $B \in \mathbb{R}^{d \times m}$: Sie besteht aus m Matrix-Vektor-Produkten (eines pro Spalte von **B**), kostet also $\approx 2ndm$ Operationen. Damit stellt sich sofort die Frage, die uns den Rest des Kapitels beschäftigt: Was passiert mit diesen Zahlen, wenn das Problem größer wird? Verdoppeln wir alle Dimensionen, verdoppelt sich dann auch der Aufwand?

Interaktiv: FLOP-Zähler: Was kosten Matrix-Vektor- und Matrix-Matrix-Produkt?

Interaktives Element — nur in der Web-Fassung

Der Zähler zeigt: Nein. Beim Matrix-Vektor-Produkt stecken zwei Dimensionen im Produkt $2nd$, also wird der Aufwand beim Verdoppeln aller Dimensionen viermal so groß; beim Matrix-Matrix-Produkt sind es drei Dimensionen und damit der Faktor acht. Der Speicher wächst dabei langsamer als die Rechenzeit.

Diese Faktoren, nicht die absoluten Zahlen, sind das, was einen Algorithmus praktisch brauchbar oder unbrauchbar macht.

Selbsttest: Operationen und Speicher zählen

Zeit für einen Selbsttest. Versuchen wir uns an zwei Fragen zum Aufwand. Gegeben sind $\mathbf{A} \in \mathbb{R}^{n \times d}$ und $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$, und wir wollen

$$\mathbf{z} = f(\mathbf{A}, \mathbf{x}, \mathbf{y}) = \mathbf{A}(\mathbf{x} - \mathbf{y}) = \sum_{i=1}^d (x_i - y_i) \mathbf{A}_i$$

berechnen, also eine Linearkombination der Spalten $\mathbf{A}_i$ von $\mathbf{A}$. Der Algorithmus arbeitet die Summe spaltenweise ab:

Algorithmus 2.3.4 (Spaltenweise Auswertung)

1. Initialisiere $\mathbf{z} = \mathbf{0} \in \mathbb{R}^n$.
2. Für $i = 1, \dots, d$: berechne den Skalar $(x_i - y_i)$ und aktualisiere $\mathbf{z} \leftarrow \mathbf{z} + (x_i - y_i) \cdot \mathbf{A}_i$.

2.3.3 Komplexität: Wie skaliert der Aufwand?

Beim Nachrechnen der Quizfragen ist vielleicht aufgefallen: Ob nun $2nd + d$ oder $2nd - n$ oder $2nd$ herauskommt, hängt von Buchhaltungsdetails ab: zählt man die Initialisierung mit? Das Zwischenergebnis? Solche Details sind für den Vergleich von Algorithmen unwichtig. Die *exakte* Anzahl an Operationen und gespeicherten Zahlen interessiert uns nicht. Was uns interessiert, ist, *wie der Aufwand mit der Größe des Problems wächst*: Was passiert, wenn wir statt 1000 Datenpunkten 10 000 haben? Statt 10 Kovariablen 1000?

Definition 2.3.5 (Komplexität)

Wie der Zeit- bzw. Speicheraufwand eines Algorithmus mit der Größe des Problems *skaliert*, nennen wir die *Laufzeitkomplexität* bzw. *Speicherkomplexität* des Algorithmus.

Beispiel 2.3.6

Ein Algorithmus benötige $4n^3 + 16n^2 + 239$ Operationen. Für großes n ist $16n^2 + 239$ gegenüber $4n^3$ vernachlässigbar: Bei $n = 100$ steuert der kubische Term $4 \cdot 10^6$ Operationen bei, die restlichen Terme nur 160 239, rund 4 % des Gesamtaufwands. Bei $n = 1000$ sind es nur noch

0,4 %. Der Algorithmus *skaliert* also wie $4n^3$; seine Komplexität ist von *kubischer Ordnung*, und wir schreiben kurz: „ $O(n^3)$ “.

Die Schreibweise $O(n^3)$ lässt dabei bewusst auch den Vorfaktor 4 weg; es zählt allein die *Ordnung* des Wachstums. Was genau hinter dem großen O steckt, definieren wir sauber in Abschnitt 2.4; hier genügt uns die Lesart „wächst höchstens wie“.

2.3.4 Komplexitätsklassen

Die wichtigsten Wachstumsordnungen haben Namen, und es lohnt sich, für jede ein Gefühl zu entwickeln. Der Schlüssel dazu ist die Frage: *Was passiert mit dem Aufwand, wenn sich die Problemgröße verdoppelt?*

Bemerkung 2.3.7 (Interpretation der Komplexitätsklassen)

Die folgenden Namen beschreiben das typische Wachstum eines Aufwands. Sie helfen uns, die Größenordnungen einzuordnen; eine formale obere Schranke schreiben wir später mit O .

- *Konstant*: Die Anzahl der Operationen bleibt von n unabhängig.
- *Logarithmisch*: Verdoppelt sich n , kommt im Modell $c \log_2 n$ ein konstanter Aufwand hinzu, denn $\log_2(2n) = \log_2(n) + 1$ (Logarithmus).
- *Linear*: Verdoppelt sich n , verdoppelt sich der führende Aufwand.
- *Quadratisch*: Verdoppelt sich n , vervierfacht sich der führende Aufwand.
- *Exponentiell*: Verdoppelt sich n , *quadriert* sich die Anzahl der Operationen, denn $2^{2n} = (2^n)^2$. Schon ein einziger Schritt $n \rightarrow n + 1$ *verdoppelt* den Aufwand.

Algorithmen mit exponentieller Laufzeit sind damit schon für moderate Problemgrößen häufig praktisch unbrauchbar. So steht es um die naive Fibonacci-Rekursion aus Abschnitt 2.2, wie wir in Abschnitt 2.5 genau nachrechnen werden. In der numerischen linearen Algebra bewegen wir uns dagegen meist zwischen $O(n)$ (Vektoroperationen), $O(n^2)$ (Matrix-Vektor-Produkte, siehe Satz 2.3.3 mit $d = n$) und $O(n^3)$ (Matrix-Zerlegungen, Matrix-Matrix-Produkte).

Wie weit diese Klassen auseinanderliegen, zeigt schon ein Bild mit drei Kurven. Auf der linearen Skala links verlässt 2^n den Bildausschnitt bereits bei $n = 10$ als fast senkrechte Wand; alles andere liegt darunter platt am Boden. Erst die logarithmische Skala rechts macht die Klassen vergleichbar, und dort wird 2^n zu einer Geraden:

Interaktives Element — nur in der Web-Fassung

Vertiefung: Wie groß ist exponentieller Aufwand?

An dieser Geraden lässt sich der Preis exponentieller Verfahren ablesen. Bei $n = 200$ braucht ein $O(2^n)$ -Algorithmus rund $1,6 \cdot 10^{60}$ Operationen. Selbst ein Superrechner mit 10^{18} Operationen pro Sekunde rechnete daran $5 \cdot 10^{34}$ Jahre, und das Universum ist erst etwa 10^{10} Jahre alt. Wer feiner mit den Klassen spielen will (welche Klassen sichtbar sind, welcher Vorfaktor, welche Skala), findet den zugehörigen Explorer in Abschnitt 2.4.

Bemerkung 2.3.8 (Vorsicht: Konstanten!)

Die $O(\cdot)$ -Notation ignoriert konstante Faktoren und Terme niedrigerer Ordnung. Ein Algorithmus mit $1000n + 10\,000$ Operationen ist $O(n)$, aber trotzdem *langsamer* als ein $O(n^2)$ -Algorithmus mit n^2 Operationen, solange $n^2 < 1000n + 10\,000$ gilt. Bei $n = 100$ etwa stehen 110 000 Operationen gegen nur 10 000. Die Komplexitätsklasse sagt, wer für *hinreichend große* Probleme gewinnt, nicht, wer bei *unserem konkreten* Problem gewinnt.

Selbsttest: Aufwand und Ordnung

1. Ab welchem n ist ein Algorithmus mit n^2 Operationen schneller als einer mit $1000n + 10\,000$ Operationen?

Lösung: 1010

Der Schnittpunkt liegt bei $n = 500 + \sqrt{260\,000} \approx 1009,9$. Für $n = 1009$ stehen 1 018 081 gegen 1 019 000 Operationen, für $n = 1010$ dagegen 1 020 100 gegen 1 020 000.

2. Um welchen Faktor wächst der Zeitaufwand des Matrix-Matrix-Produkts, wenn wir im FLOP-Zähler n , d und m gleichzeitig verdoppeln?

Lösung: 8

In $2ndm$ stecken drei Dimensionen, also $2^3 = 8$. Beim Matrix-Vektor-Produkt sind es nur zwei Dimensionen und damit der Faktor 4.

3. Ein $O(n)$ -Algorithmus ist für jedes n schneller als ein $O(n^2)$ -Algorithmus.

Antwort: Falsch

Die Landau-Notation ignoriert Vorfaktoren, und die können jede endliche Problemgröße dominieren (Bemerkung 2.3.8). Die Aussage gilt nur für *hinreichend große* n .

Vertiefung: Heath §1.1 (Kosten und Genauigkeit wissenschaftlichen Rechnens); Heath §2.4.5 (Operationen zählen am Beispiel des Gauß-Verfahrens).

2.4 Landau-Symbole und Rechenregeln

In Abschnitt 2.3 haben wir den Aufwand von Algorithmen gemessen, indem wir elementare Operationen gezählt haben. Das Ergebnis sind Ausdrücke wie $4n^3 + 16n^2 + 239$: exakt, aber unhandlich. Für den Vergleich von Algorithmen interessiert uns fast immer nur eine Frage: Wie schnell wächst der Aufwand, wenn die Problemgröße n groß wird? Die konkreten Vorfaktoren hängen ohnehin von Maschine, Programmiersprache und Zählweise ab. Wir brauchen also eine Notation, die genau das Wachstumsverhalten festhält und alles Unwesentliche wegwirft. Das leisten die *Landau-Symbole* (engl. *big-O notation*).

2.4.1 Klein-o und Groß-O

Die Idee: Wir vergleichen den Aufwand a_n mit einer möglichst einfachen Vergleichsfolge b_n (etwa n^2 oder 2^n), indem wir den Grenzwert des Quotienten a_n/b_n betrachten. Bleibt der Quotient beschränkt, wächst a_n höchstens so schnell wie b_n ; verschwindet er sogar, wächst a_n echt langsamer.

Definition 2.4.1 (Landau-Symbole)

Seien a_n, b_n Folgen, wobei b_n ab einem Index ungleich null ist.

- $a_n = o(b_n)$ („ a_n ist klein-o von b_n “), wenn

$$\lim_{n \rightarrow \infty} \frac{a_n}{b_n} = 0.$$

- $a_n = O(b_n)$ („ a_n ist groß-O von b_n “), wenn

$$\limsup_{n \rightarrow \infty} \left| \frac{a_n}{b_n} \right| < \infty.$$

Die Interpretation der Symbole:

- $a_n = o(b_n)$: a_n wächst *langsamer* als b_n ; gegenüber b_n wird a_n vernachlässigbar klein.
- $a_n = O(b_n)$: a_n wächst *höchstens so schnell* wie b_n ; die Vergleichsfolge b_n ist gegenüber a_n nicht vernachlässigbar.

Vertiefung: Technische Feinheiten der Landau-Definition

Bemerkung 2.4.2 (Zwei Feinheiten der Definition)

Warum Limes superior statt Grenzwert? Der Quotient a_n/b_n muss nicht konvergieren; er darf zum Beispiel oszillieren. Der Limes superior (der größte Häufungswert, gebildet als Grenzwert der Suprema der Restfolgen) existiert dagegen immer, notfalls als ∞ . Die Bedingung $\limsup_{n \rightarrow \infty} |a_n/b_n| < \infty$ bedeutet schlicht: Ab irgendeinem Index gilt $|a_n| \leq C \cdot |b_n|$ für eine Konstante C .

Das Gleichheitszeichen ist ein Notationsmissbrauch: $a_n = O(b_n)$ liest man besser als „ a_n gehört zur Klasse $O(b_n)$ “. Insbesondere ist die Beziehung nicht symmetrisch: Aus $5n = O(n^2)$ folgt nicht $n^2 = O(5n)$.

2.4.2 Rechenbeispiele

Wie weist man eine Landau-Beziehung konkret nach? Wir bilden den Quotienten und rechnen seinen Grenzwert aus. In den folgenden drei Beispielen verfolgt **Rot** die untersuchte Folge a_n und **Blau** die Vergleichsfolge b_n .

Beispiel 2.4.3

- (a) Wir zeigen $3n^2 + 5n = O(n^2)$:

$$\lim_{n \rightarrow \infty} \frac{3n^2 + 5n}{n^2} = \lim_{n \rightarrow \infty} \left(3 + \frac{5}{n} \right) = 3 < \infty \implies 3n^2 + 5n = O(n^2). \quad \boxtimes$$

Der Quotient konvergiert gegen 3, also ist auch sein Limes superior $3 < \infty$.

- (b) Wir zeigen $5n = o(n^2)$:

$$\lim_{n \rightarrow \infty} \frac{5n}{n^2} = \lim_{n \rightarrow \infty} \frac{5}{n} = 0 \implies 5n = o(n^2). \quad \boxtimes$$

- (c) Gilt $n^2 = O(n)$? Nein!

$$\lim_{n \rightarrow \infty} \frac{n^2}{n} = \lim_{n \rightarrow \infty} n = \infty \quad (\text{divergiert!}) \implies n^2 \neq O(n).$$

Der Quotient wächst über jede Schranke hinaus; n^2 wächst echt schneller als n .

Vertiefung: Warum manchmal Theta?

Groß-O ist absichtlich nur eine obere Schranke. Daher heißt „ein $O(n^2)$ -Algorithmus“ nicht, dass seine Laufzeit tatsächlich quadratisch ist. Ein Sortierverfahren mit Laufzeit $O(n \log n)$ ist nämlich automatisch auch $O(n^2)$, weil $n \log n$ für große n höchstens so groß wie n^2 ist. Die zweite Aussage ist richtig, unterscheidet das Verfahren aber nicht von einem tatsächlich quadratischen Verfahren.

Wollen wir die Ordnung von beiden Seiten festlegen, verwenden wir $a_n = \Theta(b_n)$. Das bedeutet zugleich $a_n = O(b_n)$ und $b_n = O(a_n)$: Bis auf konstante Faktoren wachsen beide Folgen gleich schnell. Für $3n^2 + 5n$ gilt also $\Theta(n^2)$; $O(n^3)$ wäre zwar ebenfalls wahr, aber als Beschreibung zu grob. Im Hauptstrang genügt uns meist die obere Schranke O , weil sie für Laufzeit- und Fehlerabschätzungen bereits die entscheidende Garantie liefert.

2.4.3 Rechenregeln

Aufwandsausdrücke entstehen durch Hintereinanderausführung (Addition der Kosten) und Verschachtelung (Multiplikation der Kosten) von Algorithmus-Bausteinen. Praktischerweise vertragen sich die Landau-Symbole genau mit diesen beiden Operationen, so dass wir die Grenzwertrechnung nicht jedes Mal neu machen müssen:

Lemma 2.4.4 (Rechenregeln für Landau-Symbole)

Seien $a_n, b_n > 0$ Vergleichsfolgen.

1. Sei $f_n = O(a_n)$ und $g_n = O(b_n)$. Dann gilt

$$f_n + g_n = O(a_n + b_n) \quad \text{und} \quad f_n \cdot g_n = O(a_n \cdot b_n).$$

2. Sei $f_n = O(a_n)$ und $g_n = o(b_n)$. Dann gilt

$$f_n \cdot g_n = o(a_n \cdot b_n).$$

3. Sei $f_n = O(a_n + b_n)$ mit $a_n = O(b_n)$. Dann gilt

$$f_n = O(b_n).$$

Die Positivität der Vergleichsfolgen setzen wir voraus, damit sich in $a_n + b_n$ nichts wegheben kann. Für Aufwandsvergleiche ist das keine Einschränkung, denn Operationenzahlen sind positiv. Die Beweise sind kurze Grenzwertargumente.

Vertiefung: Warum die Rechenregeln gelten

Beweis.

(1) *Regel 1, Addition.* Wir schätzen den Quotienten ab:

$$\frac{|f_n + g_n|}{a_n + b_n} \leq \frac{|f_n|}{a_n + b_n} + \frac{|g_n|}{a_n + b_n} \leq \frac{|f_n|}{a_n} + \frac{|g_n|}{b_n}.$$

Dreiecksungleichung; danach verkleinern wir die Nenner ($a_n + b_n \geq a_n$ bzw. $\geq b_n$, da beide Folgen positiv sind), was die Brüche höchstens vergrößert

(2)

$$\limsup_{n \rightarrow \infty} \frac{|f_n + g_n|}{a_n + b_n} < \infty \implies f_n + g_n = O(a_n + b_n).$$

beide Summanden haben nach Voraussetzung endlichen Limes superior, und der Limes superior einer Summe ist höchstens die Summe der Limes superiores

(3) *Regel 1, Multiplikation, und Regel 2.* Der Quotient zerfällt in ein Produkt:

$$\frac{|f_n \cdot g_n|}{a_n \cdot b_n} = \frac{|f_n|}{a_n} \cdot \frac{|g_n|}{b_n}.$$

Betrag und Bruch faktorisieren, beide Faktoren kennen wir schon

(4) Der erste Faktor ist beschränkt ($f_n = O(a_n)$). Ist $g_n = O(b_n)$, so ist auch der zweite Faktor beschränkt und das Produkt bleibt beschränkt: $f_n g_n = O(a_n b_n)$. Ist dagegen $g_n = o(b_n)$, so konvergiert der zweite Faktor gegen 0 und damit das ganze Produkt: $f_n g_n = o(a_n b_n)$.

das Produkt zweier beschränkter Folgen ist beschränkt; das Produkt einer beschränkten Folge mit einer Nullfolge ist eine Nullfolge

(5) *Regel 3.* Zuerst zeigen wir $a_n + b_n = O(b_n)$:

$$\frac{a_n + b_n}{b_n} = \frac{a_n}{b_n} + 1 = O(1).$$

$a_n = O(b_n)$ macht den ersten Summanden beschränkt; die Konstante 1 ist es sowieso

(6)

$$\frac{|f_n|}{b_n} = \frac{|f_n|}{a_n + b_n} \cdot \frac{a_n + b_n}{b_n} \implies f_n = O(b_n).$$

Produkt zweier beschränkter Folgen; das ist gerade Regel 1 (Multiplikation) mit $b_n \cdot 1$ als Vergleichsfolge

□

2.4.4 Anwendung: auf den dominanten Term reduzieren

Sehen wir die Regeln im Einsatz. Wieder verfolgt Rot die Folge f_n und Grün die Folge g_n .

Beispiel 2.4.5 (Addition und Multiplikation)

Gegeben seien $f_n = 3n^2 = O(n^2)$ und $g_n = 5n = O(n)$.

Addition (Regel 1, dann Regel 3):

$$f_n + g_n = 3n^2 + 5n = O(n^2 + n) = O(n^2), \quad \text{denn } n = O(n^2).$$

Multiplikation (Regel 1):

$$f_n \cdot g_n = 3n^2 \cdot 5n = 15n^3 = O(n^2 \cdot n) = O(n^3).$$

Beispiel 2.4.6 (Vereinfachung eines Aufwandsausdrucks)

Ein Algorithmus benötige $4n^3 + 16n^2 + 239$ Operationen. Rot verfolgt jetzt den am schnellsten wachsenden Term:

$$4n^3 + 16n^2 + 239 = O(n^3 + n^2 + 1) = O(n^3),$$

denn $n^2 = O(n^3)$ und $1 = O(n^3)$, zweimal Regel 3 angewandt. Dass auch der Vorfaktor 4 verschwindet, liegt an der Definition: Für jede Konstante $c > 0$ ist $\limsup_{n \rightarrow \infty} |c b_n / b_n| = c < \infty$, also $c \cdot b_n = O(b_n)$.

Wir können komplexe Aufwandsausdrücke also auf ihren dominanten Term reduzieren. Deshalb sprechen wir von einem „ $O(n^3)$ -Algorithmus“, ohne Vorfaktoren oder niedrigere Terme zu nennen: Für großes n bestimmt allein der dominante Term, wie sich die Laufzeit verhält. Und die üblichen Komplexitätsklassen bilden eine strikte Hierarchie, in der jede klein-o-mäßig langsamer wächst als die nächste:

$$\log n, \quad n, \quad n \log n, \quad n^2, \quad n^3, \quad 2^n.$$

Wie drastisch unterscheiden sich diese Klassen tatsächlich, und ab welchem n überholt ein schneller wachsender Term jeden noch so großen Vorfaktor? Der Explorer unten lässt beides ausprobieren; der Logarithmus-Trick der log-Skala macht dabei aus 2^n eine Gerade.

Interaktiv: Wachstumsraten-Explorer: wer dominiert wen, und ab wann?

Interaktives Element — nur in der Web-Fassung

Die Antwort auf das „ab wann“ fällt ernüchternd aus. Ohne Vorfaktor überholt 2^n die Kurve n^2 endgültig ab $n = 5$; mit dem tausendfachen Vorfaktor $c = 1000$ erst ab $n = 19$. Vierzehn Schritte, mehr kauft der Faktor 1000 nicht. Bei den polynomialen Klassen ist es genauso durchsichtig: $n^3 > c n^2$ gilt schlicht ab $n > c$. Das ist die praktische Bedeutung von $c n^2 = O(n^2)$ und $n^2 = o(n^3)$: Vorfaktoren verschieben die Schwelle, die Reihenfolge drehen sie nie um.

2.4.5 Selbsttest

Welche der folgenden Aussagen sind wahr? Erst selbst entscheiden (Quotient bilden!), dann die Lösung aufklappen.

Wahr. $\lim_{n \rightarrow \infty} \frac{2n}{n^2} = \lim_{n \rightarrow \infty} \frac{2}{n} = 0 < \infty$; es gilt sogar die stärkere Aussage $2n = o(n^2)$. Die informativere obere Schranke ist $2n = O(n)$.

Wahr. Mit $b_n = 1$ ist $\lim_{n \rightarrow \infty} \frac{7/n}{1} = 0$. Landau-Symbole beschreiben also nicht nur Wachstum, sondern auch das Abklingen von Nullfolgen. So werden wir später Approximations- und Rundungsfehler klassifizieren.

Falsch. $\lim_{n \rightarrow \infty} \frac{8n^3 + 7n^2 + n}{n} = \lim_{n \rightarrow \infty} (8n^2 + 7n + 1) = \infty$; der Quotient divergiert. Richtig wäre $O(n^3)$: der dominante Term entscheidet (Beispiel 2.4.6).

Wahr. $\lim_{n \rightarrow \infty} \frac{n^{-3} + n^{-2}}{n^{-2}} = \lim_{n \rightarrow \infty} \left(\frac{1}{n} + 1 \right) = 1 < \infty$. ☒ Auch $n^{-3} + n^{-2} = o(1)$ wäre richtig (die Folge ist eine Nullfolge), aber $O(n^{-2})$ ist die schärfere Aussage: Sie sagt nicht nur *dass*, sondern

wie *schnell* die Folge verschwindet.

1. Setzen wir im Explorer oben den Vorfaktor auf $c = 100$ und schalten $c \cdot n^2$ und 2^n an: Ab welchem n gilt endgültig $2^n > c \cdot n^2$?

Lösung: 15

Das Verdikt des Widgets nennt die Schwelle; nachrechnen lässt sie sich auch von Hand: $2^{14} = 16\,384$ liegt noch unter $100 \cdot 14^2 = 19\,600$, aber $2^{15} = 32\,768$ liegt über $100 \cdot 15^2 = 22\,500$, und danach zieht die Exponentialfunktion endgültig davon.

2. Aus $a_n = O(b_n)$ folgt $b_n = O(a_n)$.

Antwort: Falsch

Das Gleichheitszeichen in $a_n = O(b_n)$ bezeichnet keine symmetrische Beziehung: $5n = O(n^2)$ gilt, $n^2 = O(5n)$ nicht.

3. Gilt $a_n = o(b_n)$, so gilt auch $a_n = O(b_n)$.

Antwort: Wahr

Konvergiert der Quotient gegen 0, so ist er insbesondere beschränkt. Klein-o ist die stärkere Aussage; eine informative obere Schranke kann aber eine andere Vergleichsfolge verwenden.

Damit haben wir das Handwerkszeug beisammen. Im nächsten Abschnitt setzen wir es ein, um die Komplexität unserer Fibonacci-Algorithmen aus Abschnitt 2.2 präzise zu bestimmen – mit einem drastischen Ergebnis.

Vertiefung: Heath §1.1 (*Aufwand und Genauigkeit numerischer Verfahren*); Cormen, Leiserson, Rivest & Stein, Introduction to Algorithms, Kap. 3 (*asymptotische Notation, einschließlich scharfer Schranken*).

2.5 Fibonacci: Komplexitätsanalyse

Jetzt wird geerntet. In Abschnitt 2.2 haben wir zwei Algorithmen für dasselbe Problem kennengelernt (die ersten n Fibonacci-Zahlen zu berechnen) und im Widget dort beobachtet, dass die rekursive Variante schon für moderate n unangenehm viele Aufrufe produziert. In Abschnitt 2.3 haben wir gelernt, Aufwand zu zählen, und in Abschnitt 2.4, die Zählerei mit Landau-Symbolen auf ihre Ordnung einzudampfen. Mit diesen Werkzeugen können wir den Unterschied zwischen beiden Varianten nun präzise machen. Und er ist drastischer, als man zunächst vermuten würde: Es geht nicht um einen konstanten Faktor, sondern um *linear gegen exponentiell*.

2.5.1 Die iterative Variante: linearer Aufwand

Erinnern wir uns an den iterativen Algorithmus aus Abschnitt 2.2: Er legt einen Ergebnisvektor der Länge n an, setzt die Startwerte $x_1 = 0$ und $x_2 = 1$ und füllt dann in einer einzigen Schleife jeden weiteren Eintrag als Summe seiner beiden Vorgänger: $x_{i+1} = x_i + x_{i-1}$ für $i = 2, \dots, n-1$. Zählen wir die Schritte, so wie wir es beim Matrix-Vektor-Produkt in Abschnitt 2.3 geübt haben.

Satz 2.5.1 (Komplexität der iterativen Variante)

Der iterative Fibonacci-Algorithmus berechnet die ersten n Fibonacci-Zahlen mit Zeitkomplexität $O(n)$ und Speicherkomplexität $O(n)$.

Vertiefung: Beweis der linearen Laufzeit*Beweis.*

- (1)
- Initialisierung: n Operationen.*

das Anlegen des Ergebnisvektors schreibt n Nullen, eine Schreiboperation pro Eintrag

(2)

Schleife: $(n - 2) \cdot 3$ Operationen.*die Schleife durchläuft $i = 2, \dots, n - 1$, das sind $(n - 1) - 2 + 1 = n - 2$ Durchläufe; pro Durchlauf zählen wir 1 Addition, 1 Zuweisung und 1 Indexrechnung; ob man die Indexrechnungen mitzählt, ist Konvention, für die Ordnung ist es egal (Bemerkung 2.5.2)*

(3)

$$a_n = n + 3(n - 2) + c = 4n + (c - 6) \implies a_n = O(n).$$

Summen- und Dominanzregel aus Abschnitt 2.4: mit $n = O(n)$ und $1 = O(n)$ ist $O(n) + O(n) + O(1) = O(n)$; die Konstante c deckt Funktionsaufruf und die beiden if-Abfragen ab

- (4)
- Speicher: $n + O(1) = O(n)$ Speicherzellen.*

gespeichert werden der Vektor x mit n Einträgen und konstant viele Hilfsgrößen (n , Laufindex i)

□

Bemerkung 2.5.2 (Zählen ist Konvention, die Ordnung nicht)

Ob wir pro Schleifendurchlauf 3, 5 oder 10 Elementaroperationen ansetzen, ändert am Ergebnis nichts: Jede konstante Zahl von Operationen pro Durchlauf liefert einen Gesamtaufwand der Form $c_1 n + c_2$, und der ist immer $O(n)$. Genau dafür haben wir die Landau-Notation in Abschnitt 2.4 eingeführt: Sie macht die Analyse unabhängig von solchen Zählkonventionen. Schneller als linear geht es für dieses Problem übrigens prinzipiell nicht. Allein das Hinschreiben der n Ergebniszahlen kostet schon n Schritte. Die iterative Variante ist also ordnungsoptimal.

2.5.2 Die naive Rekursion: exponentieller Aufwand

Die zweite Variante aus Abschnitt 2.2 übersetzt die mathematische Definition wörtlich in Code: Eine Funktion `fib_rek( $n$ )` gibt für $n \leq 1$ direkt n zurück und ruft sich sonst zweimal selbst auf, mit $n - 1$ und $n - 2$, und addiert die Ergebnisse. In R:

```
fib_rek <- function(n) {
  if (n <= 1) {
    return(n)
  }
  return(fib_rek(n - 1) + fib_rek(n - 2))
}
```

Das ist verführerisch elegant: der Code *ist* praktisch die Rekursionsformel. Aber die Eleganz täuscht.

Bemerkung 2.5.3 (Was wird hier eigentlich berechnet?)

Zwei Feinheiten sind dabei wichtig: Erstens berechnet `fib_rek( $n$ )` nur eine *einzelne* Fibonacci-Zahl F_n (mit der Zählung $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$), nicht den ganzen

Vektor der ersten n Zahlen; in der Notation aus Abschnitt 2.2 gilt $F_n = x_{n+1}$. Zweitens macht das den Vergleich nicht etwa unfair, sondern erst recht vernichtend: Die Rekursion braucht schon für *eine* Zahl exponentiell viele Schritte, während die Iteration *alle* ersten n Zahlen in $O(n)$ liefert.

Was kostet ein Aufruf von `fib_rek(n)`? Außer den beiden rekursiven Aufrufen passiert pro Aufruf nur konstant viel (ein Vergleich, eine Addition). Das richtige Aufwandsmaß ist deshalb die *Gesamtzahl der Funktionsaufrufe*: Nennen wir sie $T(n)$, den Startaufruf mitgezählt. Diese Folge erbt die Fibonacci-Struktur des Algorithmus:

Lemma 2.5.4 (Rekurrenz der Aufrufzahl)

Die Aufrufzahl $T(n)$ von `fib_rek(n)` erfüllt

$$T(0) = T(1) = 1, \quad T(n) = 1 + T(n-1) + T(n-2) \quad (n \geq 2).$$

Für $n \leq 1$ greift die Abbruchbedingung: Die Funktion kehrt sofort zurück, ohne sich selbst aufzurufen. Für $n \geq 2$ besteht der Aufrufbaum aus der Wurzel (dem Aufruf selbst) und den beiden vollständigen Teilbäumen von `fib_rek(n-1)` und `fib_rek(n-2)`, zusammen also $1 + T(n-1) + T(n-2)$ Aufrufe.

Bevor wir $T(n)$ allgemein abschätzen, sehen wir uns den Aufrufbaum einmal konkret an. Dann ist auch sofort klar, *woher* die Explosion kommt.

Beispiel 2.5.5 (Der Aufrufbaum für $n = 5$)

Der Aufruf `fib_rek(5)` erzeugt den folgenden Baum (jeder Knoten ist ein Funktionsaufruf):

Interaktives Element – nur in der Web-Fassung

Zählen wir nach: F_5 und F_4 werden je einmal berechnet, F_3 aber 2-mal, F_2 schon 3-mal, F_1 sogar 5-mal und F_0 noch 3-mal, zusammen $1 + 1 + 2 + 3 + 5 + 3 = 15$ Aufrufe. Das passt zum Lemma: Aus $T(2) = 3$ und $T(3) = 1 + 3 + 1 = 5$ folgt $T(4) = 1 + 5 + 3 = 9$ und $T(5) = 1 + 9 + 5 = 15$. $\square$

Die Wurzel des Übels ist rot markiert: Der komplette Teilbaum unter `fib_rek(3)` wird zweimal durchgerechnet, denn die Rekursion „vergisst“ alles, was sie schon berechnet hat. Bei größerem n verdoppeln sich diese Dopplungen immer weiter. Die iterative Variante berechnet dagegen jede Zahl genau einmal, weil sie die Zwischenergebnisse im Vektor aufbewahrt.

Jetzt die allgemeine Analyse. Kurz und bündig argumentiert: Jeder Aufruf erzeugt bis zu 2 weitere Aufrufe, diese wieder je 2, und so fort über bis zu n Ebenen, insgesamt höchstens $1 + 2 + 4 + \dots + 2^n$ Aufrufe, also $O(2^n)$. Das folgende Resultat macht dieses Argument präzise und ergänzt die Gegenrichtung: Das Wachstum ist auch wirklich exponentiell, nicht nur durch eine Exponentialfunktion beschränkt.

Satz 2.5.6 (Exponentielle Laufzeit der naiven Rekursion)

Für alle $n \geq 0$ gilt

$$(\sqrt{2})^{n-1} \leq T(n) \leq 2^{n+1} - 1.$$

Insbesondere ist $T(n) = O(2^n)$, und für jedes feste $k \in \mathbb{N}$ gilt $n^k = o(T(n))$: Die Aufrufzahl

wächst exponentiell und überholt jedes Polynom. Die Speicherkomplexität ist dagegen nur $O(n)$.

Der Speicherbedarf bleibt bescheiden, weil der Aufrufbaum nie ganz gleichzeitig gespeichert wird: „Offen“ ist immer nur der Pfad von der Wurzel zum aktuellen Aufruf, und der ist höchstens n Aufrufe lang. Auf dem Aufruf-Stapel (call stack) liegen also höchstens n Aufrufe, das sind $O(n)$ Speicherzellen.

Vertiefung: Woher die beiden Schranken für die Aufrufzahl kommen

Beweis.

- (1) *Obere Schranke*, per vollständiger Induktion: Für $n \leq 1$ gilt $T(n) \leq 2^{n+1} - 1$.

Induktionsanfang: $T(0) = T(1) = 1$ und $2^1 - 1 = 1$, $2^2 - 1 = 3$

- (2)

$$T(n) = 1 + T(n-1) + T(n-2) \leq 1 + (2^n - 1) + (2^{n-1} - 1) = 2^n + 2^{n-1} - 1 \leq 2^{n+1} - 1.$$

Rekurrenz aus Lemma 2.5.4, dann Induktionsvoraussetzung für $n-1$ und $n-2$ einsetzen; zuletzt $2^n + 2^{n-1} = 3 \cdot 2^{n-1} \leq 4 \cdot 2^{n-1} = 2^{n+1}$

- (3)

$$T(n) = O(2^n).$$

Definition von O aus Abschnitt 2.4: $T(n)/2^n \leq 2$ für alle n , der Limes superior ist also endlich

- (4) *Untere Schranke:* Für $n \geq 2$ ist

$$T(n) \geq T(n-1) + T(n-2) \geq 2T(n-2).$$

T ist monoton wachsend (in der Rekurrenz kommt zu $T(n-1)$ nur Positives hinzu), also $T(n-1) \geq T(n-2)$

- (5)

$$T(n) \geq 2T(n-2) \geq 4T(n-4) \geq \dots \geq 2^{\lfloor n/2 \rfloor} \geq 2^{(n-1)/2} = (\sqrt{2})^{n-1}.$$

die Ungleichung $\lfloor n/2 \rfloor$ -mal anwenden; bei jedem Schritt verdoppelt sich der Faktor wie bei einer geometrischen Folge, bis das Argument 0 oder 1 erreicht (dort ist $T = 1$); schließlich $\lfloor n/2 \rfloor \geq (n-1)/2$

- (6)

$$\frac{n^k}{T(n)} \leq \frac{n^k}{(\sqrt{2})^{n-1}} \rightarrow 0 \quad \Rightarrow \quad n^k = o(T(n)).$$

exponentiell schlägt polynomiell: Der Grenzwert $n^k/q^n \rightarrow 0$ für jedes $q > 1$ ist ein Analysis-Standardresultat, hier mit $q = \sqrt{2}$

□

Damit steht die Ordnung fest, aber nicht die Basis: Zwischen $\sqrt{2}$ und 2 ist viel Platz. Zählen wir die Aufrufe für wachsendes n also einfach aus und tragen sie logarithmisch auf. Dann wird aus jedem exponentiellen Wachstum eine Gerade, und deren Steigung verrät die Basis.

Interaktiv: Gezählte Schritte gegen die Landau-Vorhersage

Interaktives Element — nur in der Web-Fassung

Dass wir dafür die logarithmische Skala brauchen, um beide Varianten überhaupt in ein gemeinsames Bild zu bekommen, ist selbst schon die halbe Pointe. Die Steigung der roten Punkte liegt dabei sichtbar unter $\log_{10} 2 \approx 0,301$. Woran liegt das?

Vertiefung: Die exakte Aufrufzahl

Bemerkung 2.5.7 (Wie schlimm ist es wirklich?)

Zwischen unserer unteren Schranke (Basis $\sqrt{2} \approx 1,41$) und der oberen (Basis 2) klappt noch eine Lücke: Die Schranke $O(2^n)$ ist zwar korrekt, aber nicht scharf. Die Aufrufzahl lässt sich sogar exakt angeben, und sie ist selbst fast eine Fibonacci-Zahl: Aus der Rekurrenz folgt per Induktion $T(n) = 2F_{n+1} - 1$. Der Anfang stimmt wegen $T(0) = 1 = 2F_1 - 1$ und $T(1) = 1 = 2F_2 - 1$, und der Schritt ist

$$T(n) = 1 + (2F_n - 1) + (2F_{n-1} - 1) = 2(F_n + F_{n-1}) - 1 = 2F_{n+1} - 1.$$

Die Fibonacci-Zahlen selbst wachsen wie φ^n mit dem goldenen Schnitt $\varphi = (1 + \sqrt{5})/2 \approx 1,618$ (Vertiefung unten), also ist $T(n) = O(\varphi^n)$: Der wahre Wachstumsfaktor liegt sauber zwischen unseren Schranken. Für das Urteil „katastrophal langsam“ ist das aber einerlei: Jede Basis $q > 1$ bedeutet, dass eine um 1 größere Eingabe den Aufwand um den Faktor q vervielfacht.

Vertiefung: Der goldene Schnitt im Aufrufbaum

Woher kommt die Basis φ ? Für die Fibonacci-Zahlen gilt die Binet-Formel $F_n = (\varphi^n - \psi^n)/\sqrt{5}$ mit $\varphi = (1 + \sqrt{5})/2$ (dem goldenen Schnitt) und $\psi = 1 - \varphi \approx -0,618$. Weil $|\psi| < 1$ ist, stirbt der zweite Term aus, und es bleibt $F_{n+1} \approx \varphi^{n+1}/\sqrt{5}$; mit $T(n) = 2F_{n+1} - 1$ also $T(n) = O(\varphi^n)$.

Im Widget oben ist genau das zu sehen: Die gezählten Aufrufe liegen auf einer Geraden mit Steigung $\log_{10} \varphi \approx 0,209$, nicht auf der steileren 2^n -Geraden mit Steigung $\log_{10} 2 \approx 0,301$.

2.5.3 Der Vergleich: Ordnung schlägt Konstante

Was bedeuten $O(n)$ gegen $O(\varphi^n)$ in echten Zahlen? Setzen wir unsere gezählten Aufwände ein – iterativ $4n - 6$ Operationen (die additive Konstante c aus dem Beweis von Satz 2.5.1 lassen wir weg), naiv rekursiv $T(n) = 2F_{n+1} - 1$ Aufrufe – und rechnen als grobes Modell mit 10^9 Elementarschritten pro Sekunde (ein realer Funktionsaufruf in R kostet deutlich mehr, das macht es nur schlimmer):

n	iterativ ($4n - 6$)	naiv ($T(n)$)	Zeit naiv (Modell)
20	74	21 891	$\approx 22 \mu\text{s}$
30	114	2 692 537	$\approx 2,7 \text{ ms}$
50	194	$\approx 4,1 \cdot 10^{10}$	$\approx 41 \text{ s}$
100	394	$\approx 1,1 \cdot 10^{21}$	$\approx 36\,000 \text{ Jahre}$

Die iterative Spalte bleibt dabei durchgehend im Nanosekundenbereich. Das ist die zentrale Botschaft dieses Kapitels: Der Unterschied zwischen den beiden Varianten ist kein Implementierungsdetail und keine Frage schnellerer Hardware: Ein tausendmal schnellerer Rechner verschiebt die 36 000 Jahre lediglich auf 36 Jahre. Nur ein *besserer Algorithmus* hilft, und die Landau-Notation ist das Instrument, mit dem wir „besser“ präzise ausdrücken: *Die iterative Lösung ist dramatisch effizienter; sie liegt in einer anderen Komplexitätsklasse.*

Selbsttest

Vier Aussagen zu diesem Abschnitt. Welche sind wahr?

1. Die Laufzeit der iterativen Variante ist $O(n^2)$.

Antwort: Wahr

O ist nur eine *obere* Schranke: $4n - 6$ wächst höchstens so schnell wie n^2 , also gilt $O(n^2)$; die Aussage ist wahr, aber wenig informativ. Die scharfe Beschreibung ist $O(n)$, und mehr noch: $4n - 6$ ist sogar $o(n^2)$.

2. Die naive Rekursion braucht exponentiell viel *Speicher*, weil ihr Aufrufbaum exponentiell viele Knoten hat.

Antwort: Falsch

Der Baum wird nie ganz gleichzeitig gespeichert: Auf dem Aufruf-Stapel liegt immer nur der aktive Pfad von der Wurzel zum aktuellen Aufruf, und der ist höchstens n Aufrufe lang; Speicherkomplexität $O(n)$ (Satz 2.5.6). Exponentiell ist nur die *Zeit*.

3. Zählt man pro Schleifendurchlauf 5 statt 3 Operationen, ändert sich die Zeitkomplexität der iterativen Variante nicht.

Antwort: Wahr

Konstante Faktoren und additive Konstanten verschwinden in der Landau-Notation: $n + 5(n - 2) + c = O(n)$ genauso wie $n + 3(n - 2) + c$ (Bemerkung 2.5.2). Diese Robustheit gegen Zählkonventionen ist der Zweck der Notation.

4. Aus $T(n) \geq 2T(n - 2)$ für alle $n \geq 2$ (mit $T(0), T(1) \geq 1$) folgt bereits, dass T mindestens exponentiell wächst.

Antwort: Wahr

Wiederholtes Einsetzen liefert $T(n) \geq 2^{\lfloor n/2 \rfloor} \geq (\sqrt{2})^{n-1}$, geometrisches Wachstum mit Basis $\sqrt{2} > 1$. So haben wir die untere Schranke in Satz 2.5.6 bewiesen.

5. Wie viele Jahre rechnet die naive Rekursion im Modell des Widgets oben bei $n = 80$?

Lösung: 2

Der Ablesekasten des Widgets nennt bei $n = 80$ rund $7,6 \cdot 10^{16}$ Aufrufe; bei 10^9 Schritten pro Sekunde sind das etwa $7,6 \cdot 10^7$ Sekunden, also gut zwei Jahre. Die Iteration braucht für dieselbe Zahl 314 Operationen.

Zusammenfassung des Kapitels

In diesem Kapitel haben wir den Grundwortschatz des Skripts aufgebaut: Ein *numerisches Problem* ist eine Abbildung f , die Eingabedaten $\mathbf{x}$ eine gesuchte Lösung $f(\mathbf{x})$ zuordnet, und ein *Algorithmus* ist eine endliche Folge elementarer Rechenschritte, die diese Lösung exakt oder näherungsweise berechnet (Abschnitte 2.1–2.2). Gute Algorithmen erkennen wir daran, dass sie mit wenig Laufzeit und Speicher auskommen; beides messen wir als Funktion der Problemgröße n (Abschnitt 2.3) und beschreiben es nur noch durch seine Ordnung, mit Landau-Symbolen und ihren Rechenregeln (Abschnitt 2.4). Die Fibonacci-Fallstudie dieses Abschnitts hat gezeigt, wie viel diese komprimierte Sprache leistet: $O(n)$ gegen $O(\varphi^n)$ markiert den Unterschied zwischen „sofort fertig“ und „Jahrtausende“. Im nächsten Kapitel kommt die Sprache für Fehlermaße hinzu: Normen. Kapitel 4 nutzt sie dann, um Rundungsfehler, Kondition und Stabilität präzise zu behandeln.

Vertiefung: Heath §1.1 (wissenschaftliches Rechnen: Probleme, Algorithmen und die Rolle des Aufwands).

Spur & Matrixnormen

3.1 Die Spur einer Matrix

Mit diesem Kapitel beginnt der Numerik-Teil des Skripts in der linearen Algebra. Bevor wir in den folgenden Kapiteln Gleichungssysteme zerlegen und lösen, brauchen wir ein Handwerkszeug, das dort auf Schritt und Tritt auftaucht: Wir müssen Matrizen *messen* können. Wie groß ist eine Matrix? Wie stark verstärkt sie Fehler? Wann konvergiert ein iteratives Verfahren? Alle diese Fragen verlangen, eine ganze Tabelle voller Zahlen zu einer einzigen aussagekräftigen Kennzahl zu verdichten. Dieses Kapitel stellt die beiden wichtigsten solchen Kennzahlen vor: die *Spur* (dieser Abschnitt) und die *Matrixnormen* (Rest des Kapitels). Beide begegnen uns in der Statistik ständig: Die Gesamtvarianz eines Zufallsvektors ist zum Beispiel die Spur seiner Kovarianzmatrix.

Bemerkung 3.1.1 (Verwendete Vorkenntnisse)

Dieses Kapitel setzt voraus:

- aus der linearen Algebra: Matrixoperationen (Addition, Multiplikation, Transposition), Eigenwerte und Eigenvektoren, Orthogonalität, Skalarprodukte sowie grundlegende Vektornormen (euklidische Norm $\|\mathbf{x}\|_2$ und p -Normen),
- aus der Analysis: Grenzwerte und Suprema, Maximum und Minimum von Funktionen.

3.1.1 Definition und erste Beispiele

Die einfachste Art, eine quadratische Matrix zu einer Zahl zu verdichten, ist zugleich die billigste: Wir addieren einfach die Diagonalelemente. Das klingt zunächst willkürlich. Warum ausgerechnet die Diagonale? Die Antwort liefert dieser Abschnitt: Diese Summe ist eine erstaunlich stabile Kenngröße der Matrix, die sich von Basiswechseln nicht beeindrucken lässt und die Summe der Eigenwerte liefert, ohne dass wir auch nur einen davon ausrechnen müssten.

Definition 3.1.2 (Spur)

Die *Spur* (*trace*) einer quadratischen Matrix $A \in \mathbb{R}^{n \times n}$ ist die Summe ihrer Diagonalelemente:

$$\operatorname{tr}(A) = \sum_{i=1}^n a_{ii}.$$

Zwei Dinge halten wir gleich fest: Erstens ist die Spur *nur für quadratische Matrizen* definiert, denn bei einer rechteckigen Matrix hat „die Diagonale“ keine sinnvolle Bedeutung. Zweitens ist die Spur eine *skalare Invariante* der Matrix: Sie ändert sich nicht, wenn wir dieselbe lineare Abbildung in einer anderen Basis darstellen (das ist die Ähnlichkeitsinvarianz, die wir gleich in Satz 3.1.4 beweisen). Nebenbei bemerkt ist die Spur auch numerisch ein Schnäppchen: $n - 1$ Additionen, keine einzige Multiplikation.

Beispiel 3.1.3

Für die Einheitsmatrix stehen auf der Diagonale n Einsen, also $\text{tr}(\mathbf{I}_n) = n$. Zwei konkrete Rechnungen (die Diagonalelemente sind grün markiert):

$$\text{tr} \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = 1 + 4 = 5, \quad \text{tr} \begin{pmatrix} 2 & -1 & 0 \\ 0 & 3 & 5 \\ 1 & 0 & -2 \end{pmatrix} = 2 + 3 + (-2) = 3.$$

Alle Einträge abseits der Diagonale sind für die Spur unsichtbar; die 5 im zweiten Beispiel könnte genauso gut 5000 sein.

3.1.2 Rechenregeln und zyklische Vertauschung

Der eigentliche Wert der Spur liegt in ihren Rechenregeln. Sie machen aus der harmlosen Diagonalsumme ein Werkzeug, mit dem sich viele Matrixausdrücke drastisch vereinfachen lassen.

Satz 3.1.4 (Eigenschaften der Spur)

Für Matrizen $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$, invertierbares $\mathbf{P} \in \mathbb{R}^{n \times n}$ und $c \in \mathbb{R}$ gilt:

1. *Additiv*: $\text{tr}(\mathbf{A} + \mathbf{B}) = \text{tr}(\mathbf{A}) + \text{tr}(\mathbf{B})$
2. *Homogen*: $\text{tr}(c\mathbf{A}) = c \cdot \text{tr}(\mathbf{A})$
3. *Zyklisch*: $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$
4. *Ähnlichkeitsinvariant*: $\text{tr}(\mathbf{PAP}^{-1}) = \text{tr}(\mathbf{A})$
5. *Transpositionsinvariant*: $\text{tr}(\mathbf{A}^\top) = \text{tr}(\mathbf{A})$

Die Eigenschaften 1, 2 und 5 folgen direkt aus der Definition: Addition und Skalierung wirken eintragsweise, also insbesondere auf jedes Diagonalelement einzeln, und das Transponieren spiegelt die Matrix an der Diagonale, wobei die Diagonalelemente selbst bleiben, wo sie sind. Interessant sind die Eigenschaften 3 und 4. Ihr ausführlicher Beweis ist für den weiteren Kurs nicht erforderlich.

Vertiefung: Beweis der Spur-Rechenregeln

Beweis.

- (1) Zu Eigenschaft 3: Wir schreiben beide Seiten als Doppelsumme aus.

$$\text{tr}(\mathbf{AB}) = \sum_{i=1}^n (\mathbf{AB})_{ii} = \sum_{i=1}^n \sum_{k=1}^n a_{ik} b_{ki}$$

Definition der Spur; der Diagonaleintrag $(\mathbf{AB})_{ii}$ ist das Skalarprodukt aus i -ter Zeile von $\mathbf{A}$ und i -ter Spalte von $\mathbf{B}$ (Matrixprodukt)

- (2)

$$= \sum_{k=1}^n \sum_{i=1}^n b_{ki} a_{ik} = \sum_{k=1}^n (\mathbf{BA})_{kk} = \text{tr}(\mathbf{BA})$$

endliche Summen dürfen wir vertauschen, und die Faktoren a_{ik}, b_{ki} sind Skalare, kommutieren also

- (3) Eigenschaft 4 ist damit ein Einzeiler:

$$\text{tr}(\mathbf{PAP}^{-1}) = \text{tr}((\mathbf{PA})\mathbf{P}^{-1}) = \text{tr}(\mathbf{P}^{-1}(\mathbf{PA})) = \text{tr}(\mathbf{A}).$$

Eigenschaft 3, angewendet auf die beiden Faktoren $\mathbf{PA}$ und $\mathbf{P}^{-1}$; dann $\mathbf{P}^{-1}\mathbf{P} = \mathbf{I}$ (Inverse)

□

Eigenschaft 3 ist die *wichtigste* Eigenschaft der Spur. Wenden wir sie auf ein Produkt aus drei Matrizen an (mit den Faktorpaaren $\mathbf{A} \cdot (\mathbf{BC})$ bzw. $(\mathbf{AB}) \cdot \mathbf{C}$), erhalten wir die *zyklische Vertauschung*:

$$\text{tr}(\mathbf{ABC}) = \text{tr}(\mathbf{BCA}) = \text{tr}(\mathbf{CAB}).$$

Wir dürfen die Faktoren unter der Spur also „im Kreis herumschieben“, so als stünden sie auf einem Karussell: Die Reihenfolge im Kreis bleibt erhalten, nur der Startpunkt wandert.

Bemerkung 3.1.5 (Vorsicht: Was die Spur nicht kann)

- Die Spur ist *nicht* multiplikativ: $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$, aber $\text{tr}(\mathbf{AB}) \neq \text{tr}(\mathbf{A}) \cdot \text{tr}(\mathbf{B})$! Kleinstes Gegenbeispiel: $\mathbf{A} = \mathbf{B} = \mathbf{I}_2$, denn $\text{tr}(\mathbf{I}_2\mathbf{I}_2) = \text{tr}(\mathbf{I}_2) = 2$, aber $\text{tr}(\mathbf{I}_2) \cdot \text{tr}(\mathbf{I}_2) = 4$.
- Erlaubt sind nur *zyklische* Vertauschungen, keine beliebigen: Im Allgemeinen ist $\text{tr}(\mathbf{ABC}) \neq \text{tr}(\mathbf{ACB})$, weil sich die Umordnung $\mathbf{ACB}$ nicht durch Weiterdrehen aus $\mathbf{ABC}$ erzeugen lässt.

Vertiefung: Anwendung: Freiheitsgrade in der linearen Regression

Beispiel 3.1.6 (Zyklische Vertauschung in der Statistik)

Ein Vorgeschmack darauf, warum die zyklische Vertauschung in der Statistik ein Dauerbrenner ist: In der linearen Regression mit Designmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (voller Spaltenrang) erzeugt die „Hutmatrix“ $\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T \in \mathbb{R}^{n \times n}$ die gefitteten Werte $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$. Ihre Spur zählt die effektiven Parameter des Modells. Die zyklische Vertauschung berechnet sie, ohne dass wir einen einzigen Eintrag der $n \times n$ -Matrix $\mathbf{H}$ kennen müssen:

$$\text{tr}(\mathbf{H}) = \text{tr}(\mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T) = \text{tr}((\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X}) = \text{tr}(\mathbf{I}_p) = p.$$

Im zweiten Schritt haben wir den Faktor $\mathbf{X}$ einmal im Kreis nach hinten geschoben; dann kürzt sich $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{X}$ zur $p \times p$ -Einheitsmatrix.

3.1.3 Spur und Eigenwerte

Warum ist ausgerechnet die Diagonalsumme so invariant? Das folgende Resultat liefert die tiefere Erklärung: Die Spur ist in Wahrheit eine Kennzahl der Eigenwerte – und die hängen nur von der linearen Abbildung ab, nicht von der Basis, in der wir sie aufschreiben.

Satz 3.1.7 (Spur als Summe der Eigenwerte)

Für eine Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ mit Eigenwerten $\lambda_1, \dots, \lambda_n$ gilt

$$\text{tr}(\mathbf{A}) = \sum_{i=1}^n \lambda_i.$$

Vertiefung: Beweisskizze für diagonalisierbare Matrizen

Beweis.

- (1) Wir führen den Beweis für *diagonalisierbare* A . Sei dazu

$$A = PDP^{-1} \quad \text{mit} \quad D = \text{diag}(\lambda_1, \dots, \lambda_n)$$

eine Diagonalisierung von A .

A diagonalisierbar heißt gerade: Es gibt eine Basis aus Eigenvektoren (Spalten von P), und in dieser Basis ist die Abbildung eine Diagonalmatrix mit den Eigenwerten auf der Diagonale

- (2)

$$\text{tr}(A) = \text{tr}(PDP^{-1}) = \text{tr}(D)$$

Ähnlichkeitsinvarianz der Spur (Satz 3.1.4, Eigenschaft 4): ähnliche Matrizen haben dieselbe Spur

- (3)

$$\text{tr}(D) = \sum_{i=1}^n \lambda_i.$$

Definition der Spur: Die Diagonalelemente von D sind genau die Eigenwerte

□

Der Satz gilt auch für nicht diagonalisierbare Matrizen; der allgemeine Beweis läuft über einen Koeffizientenvergleich im charakteristischen Polynom $\det(A - \lambda I)$ (Determinante) und ist hier nicht unser Thema. Zwei Feinheiten sind trotzdem erwähnenswert: Die Eigenwerte zählen wir mit ihrer algebraischen Vielfachheit, und sie dürfen komplex sein (eine reelle Matrix kann komplexe Eigenwerte haben). Die treten dann aber immer als konjugierte Paare $a \pm bi$ auf, deren Imaginärteile sich in der Summe wegheben: Die Spur bleibt reell, wie es sich für eine Summe reeller Diagonalelemente gehört.

Interaktiv: Was macht ein Eintrag neben der Diagonale mit Eigenwerten und Spur?

Wie belastbar diese Aussage ist, merken wir erst, wenn wir an der Matrix rütteln. Verändern wir einen Eintrag *außerhalb* der Diagonale: Was macht das mit den beiden Eigenwerten, und was mit ihrer Summe?

Interaktives Element — nur in der Web-Fassung

Die Eigenwerte reagieren empfindlich. Ein kleiner Nebendiagonaleintrag schiebt sie auseinander, lässt sie zusammenfallen oder hebt sie ganz aus der reellen Achse heraus; die Spur bemerkt davon nichts, weil sie nur die Diagonale abliest. Auch im komplexen Fall bleibt die Bilanz stehen: Die beiden Eigenwerte sind dann konjugiert, ihre Imaginärteile heben sich in der Summe weg, und übrig bleiben zwei gleiche Realteile, zusammen wieder die Spur.

3.1.4 Die Frobenius-Norm

Die Spur verdichtet eine Matrix zu einer Zahl, aber als Maß für die „Größe“ einer Matrix taugt sie nicht: Sie ignoriert alle Einträge abseits der Diagonale, kann negativ werden und ist für rechteckige Matrizen gar nicht definiert. Für ein echtes Größenmaß liegt eine andere Idee nahe: Wir behandeln die $m \cdot n$ Einträge der Matrix wie einen langen Vektor und nehmen dessen euklidische Norm.

Definition 3.1.8 (Frobenius-Norm)

Die *Frobenius-Norm* einer Matrix $A \in \mathbb{R}^{m \times n}$ ist

$$\|A\|_F := \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}.$$

Bemerkung 3.1.9

- Die Frobenius-Norm ist das exakte Analogon zur euklidischen Vektornorm: Quadrieren, aufsummieren, Wurzel ziehen, nur eben über alle $m \cdot n$ Einträge. Sie ist insbesondere auch für rechteckige Matrizen definiert.
- In der Literatur heißt sie auch *Hilbert-Schmidt-Norm*.
- Sie misst so etwas wie die „Größe“ der Matrix. Ob sie das *richtige* Maß für unsere Zwecke ist, klären wir im nächsten Abschnitt.

Das folgende Resultat verknüpft die beiden Begriffe dieses Abschnitts: Die Frobenius-Norm lässt sich vollständig durch die Spur ausdrücken. Damit stehen ihr alle Rechenregeln aus Satz 3.1.4 zur Verfügung, insbesondere die zyklische Vertauschung.

Satz 3.1.10 (Frobenius-Norm über die Spur)

Für $A \in \mathbb{R}^{m \times n}$ gilt

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \sqrt{\operatorname{tr}(A^\top A)}.$$

Beweis.

- (1) Wir berechnen $A^\top A \in \mathbb{R}^{n \times n}$ eintragsweise:

$$A^\top A = \begin{pmatrix} \sum_{i=1}^m a_{i1}^2 & \sum_{i=1}^m a_{i1}a_{i2} & \cdots \\ \sum_{i=1}^m a_{i2}a_{i1} & \sum_{i=1}^m a_{i2}^2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

Eintrag (j, k) von $A^\top A$ ist das Skalarprodukt aus j -ter Zeile von $A^\top$ (also j -ter Spalte von A) und k -ter Spalte von A ; auf der Diagonale ($j = k$, grün) steht also jede Spalte im Skalarprodukt mit sich selbst

- (2)

$$\Rightarrow \operatorname{tr}(A^\top A) = \sum_{j=1}^n \sum_{i=1}^m a_{ij}^2 = \sum_{i=1}^m \sum_{j=1}^n a_{ij}^2$$

die Spur summiert die grünen Diagonalelemente; zusammen durchlaufen die beiden Summen alle $m \cdot n$ Einträge von A genau einmal

- (3)

$$\Rightarrow \sqrt{\operatorname{tr}(A^\top A)} = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2} = \|A\|_F.$$

Wurzel ziehen auf beiden Seiten; Definition 3.1.8

Diese Spur-Darstellung ist mehr als eine Kuriosität: Sie macht die Frobenius-Norm in Rechnungen handhabbar, in denen die Doppelsumme unhandlich wäre. Ob die Frobenius-Norm allerdings das misst, was wir bei der Fehleranalyse wirklich brauchen (nämlich wie stark eine Matrix Vektoren verzerren kann), ist eine andere Frage. Ihr gehen wir im nächsten Abschnitt nach, wo wir den Begriff der Matrixnorm systematisch entwickeln.

Selbsttest

1. Die Spur einer Matrix ändert sich, wenn wir einen Eintrag außerhalb der Diagonale verändern.

Antwort: Falsch

Die Spur summiert ausschließlich die Diagonalelemente (Definition 3.1.2). Die Eigenwerte ändern sich dabei sehr wohl, ihre Summe aber nicht.

2. Eine reelle Matrix mit komplexen Eigenwerten hat trotzdem eine reelle Spur.

Antwort: Wahr

Komplexe Eigenwerte reeller Matrizen treten als konjugierte Paare $a \pm bi$ auf. In der Summe heben sich die Imaginärteile weg, und Satz 3.1.7 liefert eine reelle Zahl, wie es die Diagonalsumme verlangt.

3. Stellen wir im Spur-Widget die Voreinstellung „defekt (Jordan)“ ein. Welchen Wert zeigt die Summe der Eigenwerte?

Lösung: 4

Vier. Die Voreinstellung ist $A = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}$ mit dem doppelten Eigenwert 2. Satz 3.1.7 zählt Eigenwerte mit ihrer algebraischen Vielfachheit, die Summe ist also $2 + 2 = 4 = \text{tr}(A)$ und nicht 2. Die Matrix ist dabei nicht einmal diagonalisierbar; der Beweis aus dem Skript greift hier nicht, die Aussage gilt trotzdem.

Vertiefung: Heath §2; MML §4.1.

3.2 Matrixnormen: Definition und Beispiele

In Abschnitt 3.1 haben wir mit der Frobenius-Norm bereits eine erste Möglichkeit kennengelernt, die „Größe“ einer Matrix in einer einzigen Zahl zusammenzufassen. In diesem Abschnitt gehen wir das Thema grundsätzlicher an: Wir legen axiomatisch fest, was eine Matrixnorm überhaupt ist, konstruieren eine ganze Familie von Beispielen und entdecken dann, dass diese naheliegenden Normen einen blinden Fleck haben. Das motiviert die Operatornormen des nächsten Abschnitts.

3.2.1 Warum Matrixnormen?

Für Vektoren sind Normen unser wichtigstes Messinstrument: Sie machen aus einem Fehlervektor eine Fehlergröße, mit der wir rechnen und argumentieren können. Dasselbe brauchen wir für Matrizen, und zwar aus vier konkreten Anlässen:

- die „Größe“ einer Matrix messen, analog zur Länge eines Vektors;

- *Fehler* quantifizieren: Wie weit liegt eine mit Rundungsfehlern gespeicherte oder näherungsweise berechnete Matrix von der exakten entfernt?
- *Konvergenz* von Algorithmen analysieren: Iterative Verfahren erzeugen Folgen von Matrizen oder Vektoren, und die Aussage „ $A_k \rightarrow A$ “ braucht ein Abstandsmaß (Konvergenz);
- die *Konditionierung* von Problemen verstehen: Die Konditionszahl einer Matrix wird über Matrixnormen definiert.

Frischen wir zunächst das Vorwissen auf. Für Vektoren kennen wir neben der euklidischen Norm $\|x\|_2$ die ganze Familie der p -Normen, und ihr „Aussehen“ lässt sich am besten über ihre Einheitskugeln $\{x : \|x\|_p = 1\}$ vergleichen: Raute, Kreis, Quadrat.

Interaktiv: Auffrischung: Was misst die Norm an der Einheitskugel?

Bevor wir weiterlesen, lohnt ein Blick auf diese Formen, und zwar auf die Frage, was sie mit der Zahl $\|x\|_p$ zu tun haben.

Interaktives Element — nur in der Web-Fassung

Die Norm eines Vektors ist damit ablesbar geworden: Sie ist der Faktor, mit dem wir die Einheitskugel aufblasen müssen, damit ihr Rand durch die Spitze von x läuft. Für $p \geq 1$ ist diese Kugel konvex, und die Konvexität ist nichts anderes als die Dreiecksungleichung in geometrischer Gestalt.

Diese Denkfigur nehmen wir mit: Auch bei Matrixnormen werden wir gleich fragen, welche Menge eine Norm zur „Einheitskugel“ erklärt.

3.2.2 Die Axiome

Was macht eine Funktion auf Matrizen zu einer „Norm“? Dieselben drei Forderungen, die wir schon von Vektornormen kennen:

Definition 3.2.1 (Matrixnorm)

Eine Funktion $\|\cdot\| : \mathbb{R}^{m \times n} \rightarrow [0, \infty)$ heißt *Matrixnorm*, wenn für alle $A, B \in \mathbb{R}^{m \times n}$ und $c \in \mathbb{R}$ gilt:

1. *Definitheit*: $\|A\| = 0 \iff A = 0$
2. *absolute Homogenität*: $\|cA\| = |c| \cdot \|A\|$
3. *Dreiecksungleichung*: $\|A + B\| \leq \|A\| + \|B\|$

Bemerkung 3.2.2

Das sind exakt die Axiome einer Vektornorm, und das ist kein Zufall: Die Menge $\mathbb{R}^{m \times n}$ ist ein Vektorraum (der Dimension mn), und eine Matrixnorm ist schlicht eine Norm auf diesem Raum. Umgekehrt ist jede Vektornorm eine Matrixnorm auf dem Raum $\mathbb{R}^{n \times 1}$ der einspaltigen Matrizen. Was eine Matrixnorm über diese Grundaxiome hinaus leisten *sollte* (etwa gut mit dem Matrixprodukt zusammenzuspielen), untersuchen wir in Abschnitt 3.5.

3.2.3 Matrixnormen durch Vektorisierung

Wie kommen wir am schnellsten an Matrixnormen? Mit einem einfachen Rezept: Wir rollen die Matrix zu einem langen Vektor aus und wenden darauf eine Vektornorm an, die wir schon kennen.

Definition 3.2.3 (Vektorisierung)

Die *Vektorisierung* einer Matrix $A \in \mathbb{R}^{m \times n}$ stapelt ihre Spalten zu einem Vektor der Länge mn :

$$\text{vec}(A) = (a_{11}, \dots, a_{m1}, a_{12}, \dots, a_{m2}, \dots, a_{1n}, \dots, a_{mn})^\top \in \mathbb{R}^{mn}.$$

Dass dieses Rezept wirklich immer eine Matrixnorm liefert, ist einfach zu sehen: Die Vektorisierung sortiert die Einträge ja nur um, und damit erbt $A \mapsto \|\text{vec}(A)\|$ alle drei Axiome von der Vektornorm.

Vertiefung: Warum jede Vektornorm auf diesem Weg eine Matrixnorm liefert

Satz 3.2.4

Ist $\|\cdot\|$ eine Vektornorm auf $\mathbb{R}^{mn}$, so ist

$$A \mapsto \|\text{vec}(A)\|$$

eine Matrixnorm auf $\mathbb{R}^{m \times n}$.

Beweis.

- (1) Die Abbildung $\text{vec} : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{mn}$ ist linear und bijektiv:

$$\begin{aligned} \text{vec}(A + B) &= \text{vec}(A) + \text{vec}(B), & \text{vec}(cA) &= c \text{vec}(A), \\ \text{vec}(A) = 0 &\Leftrightarrow A = 0. \end{aligned}$$

vec sortiert die Einträge nur um: Jeder Eintrag von A landet an genau einer Position des langen Vektors, Addition und Skalarmultiplikation wirken eintragsweise

- (2)

$$\|\text{vec}(A)\| = 0 \Leftrightarrow \text{vec}(A) = 0 \Leftrightarrow A = 0.$$

Definitheit der Vektornorm auf $\mathbb{R}^{mn}$, dann die letzte Äquivalenz aus dem vorigen Schritt

- (3)

$$\|\text{vec}(cA)\| = \|c \text{vec}(A)\| = |c| \cdot \|\text{vec}(A)\|.$$

erst Linearität von vec, dann absolute Homogenität der Vektornorm

- (4)

$$\|\text{vec}(A + B)\| = \|\text{vec}(A) + \text{vec}(B)\| \leq \|\text{vec}(A)\| + \|\text{vec}(B)\|.$$

erst Additivität von vec, dann Dreiecksungleichung der Vektornorm

□

Die drei wichtigsten Vertreter dieser Bauart entstehen aus den drei vertrauten Vektornormen:

Beispiel 3.2.5 (Vektorisierungsnormen)

- *Frobenius-Norm* (aus der 2-Norm):

$$\|\mathbf{A}\|_F = \|\text{vec}(\mathbf{A})\|_2 = \sqrt{\sum_{i,j} a_{ij}^2}$$

Das stimmt mit der Definition aus Abschnitt 3.1 überein, wo wir außerdem $\|\mathbf{A}\|_F = \sqrt{\text{tr}(\mathbf{A}^\top \mathbf{A})}$ gezeigt haben.

- *Summennorm* (aus der 1-Norm):

$$\|\mathbf{A}\|_S = \|\text{vec}(\mathbf{A})\|_1 = \sum_{i,j} |a_{ij}|$$

- *Maximumsnorm* (aus der ∞ -Norm):

$$\|\mathbf{A}\|_M = \|\text{vec}(\mathbf{A})\|_\infty = \max_{i,j} |a_{ij}|$$

Interaktiv: Frobenius-, Summen- und Maximumsnorm: was sehen sie, was nicht?

Diese drei Normen sind schnell hingeschrieben. Aber leisten sie auch, was wir von einem Größenmaß erwarten? Prüfen wir das an einer Frage, die wir gleich brauchen werden: Können zwei Matrizen, die als Abbildung nichts miteinander zu tun haben, in allen drei Normen denselben Wert haben?

Interaktives Element — nur in der Web-Fassung

Der Tauschknopf beantwortet die Frage: Verschieben wir die vier Einträge auf andere Plätze, bleibt jede der drei Zahlen stehen, während aus der Identität etwa eine singuläre Matrix wird. Alle drei Normen sehen nur die Multimenge der Einträge, nicht ihre Anordnung.

3.2.4 Was elementweise Normen nicht sehen

Diese Vektorisierungsnormen sind billig zu berechnen und erfüllen alle Axiome. Trotzdem haben sie einen konzeptionellen Haken: Sie behandeln die Matrix wie einen langen Vektor; jede Umordnung der Einträge lässt die Norm unverändert. Eine Matrix ist aber mehr als eine Tabelle von Zahlen: Sie repräsentiert die lineare Abbildung $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$, und von dieser Abbildung wissen die elementweisen Normen nichts. Ihnen fehlt damit auch eine direkte geometrische Interpretation: Was genau $\|\mathbf{A}\|_F = 5$ über die Wirkung von $\mathbf{A}$ auf Vektoren aussagt, bleibt unklar. Wie drastisch das schiefgehen kann, zeigt das folgende Beispiel.

Beispiel 3.2.6 (Gleiche Frobenius-Norm, völlig verschiedene Abbildungen)

Betrachten wir die drei Matrizen

$$\mathbf{A}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{A}_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{A}_3 = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{pmatrix}.$$

Ihre Frobenius-Normen rechnen wir direkt aus:

$$\|\mathbf{A}_1\|_F = \sqrt{1^2 + 0^2 + 0^2 + 1^2} = \sqrt{2}, \quad \|\mathbf{A}_2\|_F = \sqrt{0^2 + 1^2 + 1^2 + 0^2} = \sqrt{2},$$

$$\|\mathbf{A}_3\|_F = \sqrt{(\sqrt{2})^2 + 0^2 + 0^2 + 0^2} = \sqrt{2}.$$

Alle drei Matrizen haben also *dieselbe* Frobenius-Norm $\sqrt{2}$. Als Abbildungen könnten sie kaum unterschiedlicher sein. Verfolgen wir den Vektor $\mathbf{x} = (1, 0)^\top$:

- $\mathbf{A}_1 \mathbf{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \mathbf{x}$: Die Identität lässt *jeden* Vektor unverändert.
- $\mathbf{A}_2 \mathbf{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$: Die Permutationsmatrix $\mathbf{A}_2$ vertauscht die beiden Koordinaten. Geometrisch ist das eine Spiegelung an der Winkelhalbierenden $x_2 = x_1$ (keine Drehung; es gilt $\det \mathbf{A}_2 = -1$, siehe Determinante).
- $\mathbf{A}_3 \mathbf{x} = \begin{pmatrix} \sqrt{2} \\ 0 \end{pmatrix}$: $\mathbf{A}_3$ streckt die erste Koordinate um den Faktor $\sqrt{2}$ und löscht die zweite komplett: $\mathbf{A}_3(0, 1)^\top = \mathbf{0}$. Diese Matrix ist singulär (nicht invertierbar); sie kollabiert die ganze Ebene auf die x_1 -Achse.

Zwei längentreue Abbildungen und eine, die eine ganze Dimension vernichtet – und die Frobenius-Norm kann sie nicht auseinanderhalten.

Dasselbe Problem zeigt sich aus einer zweiten Richtung, wenn wir die Dimension wachsen lassen:

Beispiel 3.2.7 (Identität in wachsender Dimension)

Für die Identitätsmatrix $\mathbf{I}_n \in \mathbb{R}^{n \times n}$ (mit n Einsen auf der Diagonale und sonst Nullen) liefern die drei Vektorisierungsnormen

$$\|\mathbf{I}_n\|_F = \sqrt{\underbrace{1 + \dots + 1}_n} = \sqrt{n}, \quad \|\mathbf{I}_n\|_S = n, \quad \|\mathbf{I}_n\|_M = 1.$$

Die Abbildung $\mathbf{x} \mapsto \mathbf{I}_n \mathbf{x} = \mathbf{x}$ ändert überhaupt nichts, trotzdem wachsen Frobenius- und Summennorm mit n über alle Grenzen.

Eine Norm, die die Matrix als *Abbildung* misst, sollte der Identität den Wert 1 geben, unabhängig von der Dimension. Die Maximumsnorm trifft diesen Wert hier zwar zufällig, bleibt aber genauso blind für das, was $\mathbf{A}$ mit Vektoren *tut* (im Matrix-Vektor-Produkt $\mathbf{A}\mathbf{x}$). Die Lösung sind *Operatornormen*, die die Transformation $\mathbf{x} \mapsto \mathbf{A}\mathbf{x}$ direkt vermessen: Wie stark kann $\mathbf{A}$ einen Vektor höchstens verlängern? Ihnen widmen wir den nächsten Abschnitt.

Selbsttest

1. Vertauschen wir zwei Einträge einer Matrix, so ändert sich keine der drei Vektorisierungsnormen.

Antwort: Wahr

Alle drei entstehen als Vektornorm von $\text{vec}(\mathbf{A})$, und die 1-, 2- und ∞ -Norm eines Vektors hängen nicht von der Reihenfolge seiner Einträge ab. Genau diese Blindheit führt in Abschnitt 3.3 zu den Operatornormen.

2. Die p -Normen sind für jedes $p > 0$ definiert.

Antwort: Falsch

Für $p < 1$ ist die Dreiecksungleichung verletzt: Im Widget beult sich die Einheitskugel nach

innen, und für $p = 0,5$ ist $\|\mathbf{e}_1 + \mathbf{e}_2\|_p = 4 > 2 = \|\mathbf{e}_1\|_p + \|\mathbf{e}_2\|_p$. Das dritte Normaxiom (für Matrizen: Definition 3.2.1) verlangt daher $p \geq 1$.

3. Setzen wir im Einheitskugel-Widget das Kästchen $p = \infty$ und die Regler auf $\mathbf{x} = (-1,2; 0,9)$. Welchen Wert zeigt das Widget für $\|\mathbf{x}\|_p$?

Lösung: 1,2

1,2. Die Maximumsnorm nimmt den größten Betrag, hier $|-1,2| = 1,2$; die zweite Koordinate spielt keine Rolle. Zum Vergleich: $\|\mathbf{x}\|_2 = 1,5$ und $\|\mathbf{x}\|_1 = 2,1$. Die skalierte Kugel ist entsprechend ein Quadrat der halben Kantenlänge 1,2, dessen Rand genau durch die Pfeilspitze läuft.

Vertiefung: Heath §2.3.

3.3 Operatornormen

Die elementweisen Matrixnormen aus Abschnitt 3.2 haben einen blinden Fleck: Sie behandeln die Matrix als bloße Zahlentabelle. Die Identitätsmatrix $\mathbf{I}_n$ etwa verändert als Abbildung überhaupt nichts; trotzdem wächst ihre Frobenius-Norm $\|\mathbf{I}_n\|_F = \sqrt{n}$ mit der Dimension. Was die Matrix als lineare Abbildung $\mathbf{x} \mapsto \mathbf{Ax}$ tut, sehen diese Normen nicht. In diesem Abschnitt bauen wir Normen, die genau das messen: Wie stark kann $\mathbf{A}$ einen Vektor strecken?

3.3.1 Definition und Interpretation

Die Idee: Wir schicken alle Vektoren $\mathbf{x} \neq \mathbf{0}$ durch die Abbildung und vergleichen die Länge des Bildes $\mathbf{Ax}$ mit der Länge des Urbildes $\mathbf{x}$, gemessen mit einer Vektornorm unserer Wahl. Der größte auftretende Streckfaktor ist die Norm der Matrix.

Definition 3.3.1 (Operatornorm)

Sei $\|\cdot\|_V$ eine Vektornorm. Die *Operatornorm* (auch: *induzierte Norm*) von $\mathbf{A} \in \mathbb{R}^{m \times n}$ ist

$$\|\mathbf{A}\|_V := \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{Ax}\|_V}{\|\mathbf{x}\|_V} = \max_{\|\mathbf{x}\|_V=1} \|\mathbf{Ax}\|_V.$$

Der *Normquotient* $s_{\mathbf{A}}(\mathbf{x}) = \|\mathbf{Ax}\|_V / \|\mathbf{x}\|_V$ gibt an, um welchen Faktor $\mathbf{A}$ den Vektor $\mathbf{x}$ streckt. Eine große Operatornorm bedeutet also: Es gibt mindestens eine Richtung, in der $\mathbf{A}$ stark streckt. Eine kleine Operatornorm bedeutet: $\mathbf{A}$ staucht jeden Vektor (oder lässt ihn höchstens schwach wachsen). Die zweite Gleichheit in der Definition gilt, weil der Normquotient skaleninvariant ist: Ersetzen wir $\mathbf{x}$ durch $c\mathbf{x}$ mit $c \neq 0$, kürzt sich $|c|$ heraus. Wir dürfen uns deshalb auf Vektoren der Länge 1 beschränken.

Vertiefung: Zwei Normen, und warum wir das Maximum schreiben dürfen

Bemerkung 3.3.2

- Genau genommen brauchen wir zwei Normen: eine auf $\mathbb{R}^n$ für $\mathbf{x}$ und eine auf $\mathbb{R}^m$ für $\mathbf{Ax}$. Wir verwenden stets dieselbe Normfamilie in der jeweils passenden Dimension und schreiben dafür dasselbe Symbol.
- Warum dürfen wir „max“ statt „sup“ schreiben? Die Funktion $\mathbf{x} \mapsto \|\mathbf{Ax}\|_V$ ist stetig, und die Einheitskugel $\{\mathbf{x} : \|\mathbf{x}\|_V = 1\}$ ist eine abgeschlossene und beschränkte Menge; das Maximum wird also tatsächlich angenommen.

Schauen wir uns das geometrisch an. In der euklidischen Norm ($p = 2$) ist die Menge aller Einheitsvektoren der Einheitskreis. Sein Bild unter einer 2×2 -Matrix ist eine Ellipse, und die Operatornorm ist die längste Halbachse dieser Ellipse.

Beispiel 3.3.3 (Visualisierung)

Betrachten wir $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$. Wie wir in Satz 3.3.7 beweisen werden, ist $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$.

Rechnen wir nach:

$$\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix}.$$

Die Eigenwerte sind die Nullstellen des charakteristischen Polynoms $\lambda^2 - 6\lambda + 4$, also $\lambda_{1,2} = 3 \pm \sqrt{5}$, und damit

$$\|\mathbf{A}\|_2 = \sqrt{3 + \sqrt{5}} \approx 2,29.$$

In R können wir das direkt nachprüfen (crossprod(A) berechnet $\mathbf{A}^\top \mathbf{A}$):

```
A <- matrix(c(2, 1,
              0, 1), nrow = 2, byrow = TRUE)
AtA <- crossprod(A) # t(A) %*% A
sqrt(eigen(AtA)$values)
#> [1] 2.288246 0.874032
```

Der größte Wert ist die Operatornorm: Kein Einheitsvektor wird um mehr als den Faktor 2,29 gestreckt. Der kleinere Wert 0,874 ist die kürzeste Halbachse der Bildellipse, also die stärkste Stauchung. Dieselbe Matrix begegnet uns in Beispiel 3.4.6 (Abschnitt 3.4) wieder. Dort berechnen wir ihre Schattennormen aus genau diesen Eigenwerten.

Interaktiv: Wie schwankt der Streckfaktor, wenn der Vektor um den Einheitskreis wandert?

Gerechnet haben wir das Maximum damit, gesehen haben wir es nicht. Die Definition läuft über *alle* Einheitsvektoren, ausprobieren können wir immer nur einen. Wie stark schwankt der Streckfaktor $\|\mathbf{Ax}\|_2$ überhaupt, wenn $\mathbf{x}$ einmal ganz um den Einheitskreis wandert, und in welche Richtung muss $\mathbf{x}$ zeigen, damit er sein Maximum erreicht?

Interaktives Element — nur in der Web-Fassung

Der Streckfaktor pendelt zwischen den beiden Halbachsen der Bildellipse hin und her, und

er tut das zweimal pro Umlauf: Die Kurve rechts hat zwei Hochpunkte, an denen die Ellipse den roten Kreis von innen berührt. Nur der obere Wert ist die Operatornorm. Die Richtung $\mathbf{x}^*$, in der das passiert, ist keine der Koordinatenachsen, sondern liegt schief im Raum; welche Richtung es genau ist, klärt Satz 3.3.7 weiter unten. Und die beiden Sonderfälle sind schnell durchgespielt: Bei der Drehung ist die Kurve eine Waagrechte, jede Richtung ist Maximalrichtung; bei der singulären Matrix fällt die Kurve irgendwo auf null, ohne dass das Maximum davon etwas merkt.

3.3.2 Die wichtigsten Operatornormen

Jede p -Norm $\|\mathbf{v}\|_p = (\sum_i |v_i|^p)^{1/p}$ induziert ihre eigene Operatornorm. Drei Fälle sind so wichtig, dass sie eigene Namen tragen, und für zwei davon gibt es erfreulich einfache Formeln:

Satz 3.3.4 (Induzierte p -Normen)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit Einträgen a_{ij} . Dann gilt:

Spektralnorm ($p = 2$):

$$\|\mathbf{A}\|_2 = \max_{\|\mathbf{x}\|_2=1} \|\mathbf{A}\mathbf{x}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$$

Spaltensummennorm ($p = 1$):

$$\|\mathbf{A}\|_1 = \max_{\|\mathbf{x}\|_1=1} \|\mathbf{A}\mathbf{x}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^m |a_{ij}|$$

Zeilensummennorm ($p = \infty$):

$$\|\mathbf{A}\|_\infty = \max_{\|\mathbf{x}\|_\infty=1} \|\mathbf{A}\mathbf{x}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^n |a_{ij}|$$

Die 1-Norm einer Matrix ist also einfach die größte betragsmäßige Spaltensumme, die ∞ -Norm die größte betragsmäßige Zeilensumme. Beide lassen sich ohne jede Eigenwertrechnung ablesen. Als Eselsbrücke: Die 1 steht senkrecht wie eine Spalte, das Symbol ∞ liegt waagrecht wie eine Zeile. Die Spektralnorm ist teurer zu berechnen, dafür passt sie zur euklidischen Geometrie: Sie ist die längste Halbachse der Bildellipse aus Beispiel 3.3.3. Ihre Formel beweisen wir in Satz 3.3.7; für die Spaltensummennorm holen wir den Beweis in der folgenden Vertiefung nach, der Fall $p = \infty$ verläuft völlig analog und ist eine gute Übung.

Vertiefung: Beweis der Spaltensummenformel

Wir zeigen $\|\mathbf{A}\|_1 = \max_j c_j$ mit den Spaltensummen $c_j := \sum_{i=1}^m |a_{ij}|$. Die Farbe Grün verfolgt dabei die Spaltensummen durch die gesamte Rechnung.

Beweis.

(1)

$$\|\mathbf{Ax}\|_1 = \sum_{i=1}^m \left| \sum_{j=1}^n a_{ij} x_j \right| \leq \sum_{i=1}^m \sum_{j=1}^n |a_{ij}| |x_j|$$

i-te Komponente von Ax ausschreiben; Dreiecksungleichung $|u+v| \leq |u| + |v|$ im Betrag

(2)

$$\sum_{i=1}^m \sum_{j=1}^n |a_{ij}| |x_j| = \sum_{j=1}^n |x_j| \sum_{i=1}^m |a_{ij}| = \sum_{j=1}^n |x_j| c_j$$

Summationsreihenfolge tauschen (endliche Summen), dann $\sum_i |a_{ij}| = c_j$ einsetzen

(3)

$$\sum_{j=1}^n |x_j| c_j \leq \left(\max_j c_j \right) \sum_{j=1}^n |x_j| = \max_j c_j$$

jede Spaltensumme durch die größte abschätzen; $\sum_j |x_j| = \|\mathbf{x}\|_1 = 1$

(4)

$$\|\mathbf{Ae}_{j^*}\|_1 = \sum_{i=1}^m |a_{ij^*}| = c_{j^*} = \max_j c_j$$

die Schranke wird angenommen: Für den Einheitsvektor $\mathbf{e}_{j^}$ zur maximalen Spalte j^* ist $\mathbf{Ae}_{j^*}$ genau die j^* -te Spalte von A*

□

3.3.3 Einschub: Orthogonalmatrizen

Für den Beweis der Spektralnorm-Formel brauchen wir eine besondere Klasse von Matrizen: solche, die Längen überhaupt nicht verändern. Sie werden uns im ganzen Skript immer wieder begegnen, gerade *weil* sie Normen unangetastet lassen.

Definition 3.3.5 (Orthogonalmatrix)

Eine Matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ heißt *Orthogonalmatrix*, wenn ihre Spalten orthonormale Vektoren sind, also

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}.$$

Bemerkung 3.3.6 (Eigenschaften von Orthogonalmatrizen)

- Geometrisch beschreibt $\mathbf{Q}$ eine Rotation und/oder Spiegelung in $\mathbb{R}^n$.
- $\mathbf{Q}$ erhält die euklidische Norm: Für alle $\mathbf{x} \in \mathbb{R}^n$ gilt

$$\|\mathbf{Qx}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{Q}^\top \mathbf{Q} \mathbf{x}} = \sqrt{\mathbf{x}^\top \mathbf{x}} = \|\mathbf{x}\|_2.$$

Insbesondere ist $\|\mathbf{Q}\|_2 = 1$: Jeder Normquotient ist exakt 1.

- $\mathbf{Q}^{-1} = \mathbf{Q}^\top$. Die Inverse ist gratis: Statt eines Gleichungssystems genügt Transponieren.
- Alle Eigenwerte haben Betrag 1: Aus $\mathbf{Qx} = \lambda \mathbf{x}$ mit $\mathbf{x} \neq \mathbf{0}$ folgt $\|\mathbf{x}\|_2 = \|\mathbf{Qx}\|_2 = |\lambda| \|\mathbf{x}\|_2$, also $|\lambda| = 1$.

- Mit Q ist auch $Q^{-1} = Q^T$ orthogonal (denn $(Q^T)^T Q^T = Q Q^T = I$), hat also ebenfalls Spektralnorm 1. Für die Konditionszahl folgt $\kappa_2(Q) = \|Q\|_2 \|Q^{-1}\|_2 = 1$: Multiplikation mit Q oder Q^{-1} verstärkt relative Störungen in der 2-Norm nicht. Stabil implementierte orthogonale Transformationen sind deshalb ein Arbeitspferd der numerischen linearen Algebra (mehr dazu bei der QR-Zerlegung in Kapitel 7).
- Produkte bleiben orthogonal: Sind $Q_1, \dots, Q_k$ orthogonal, dann auch $Q = Q_1 \cdots Q_k$. Warum? Weil $(Q_1 Q_2)^T Q_1 Q_2 = Q_2^T Q_1^T Q_1 Q_2 = Q_2^T Q_2 = I$ (Transponierregeln), und dann induktiv weiter.

3.3.4 Spektralnorm und Spektralzerlegung

Jetzt können wir die Formel $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$ beweisen. Der Schlüssel: $A^T A$ ist symmetrisch und besitzt deshalb eine Spektralzerlegung $A^T A = P \Lambda P^T$ mit einer *orthogonalen* Eigenvektormatrix P – und genau die Normerhaltung aus Bemerkung 3.3.6 macht den Beweis kurz. Die Eigenwerte sind dabei automatisch nichtnegativ, denn $x^T A^T A x = \|Ax\|_2^2 \geq 0$ ($A^T A$ ist positiv semidefinit).

Satz 3.3.7 (Spektralnorm und Spektralzerlegung)

Sei $A \in \mathbb{R}^{m \times n}$ mit Rang r und Spektralzerlegung $A^T A = P \Lambda P^T$, wobei $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_r, 0, \dots, 0)$ mit $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_r > 0$. Dann gilt

$$\|A\|_2 = \sqrt{\lambda_1} = \sqrt{\lambda_{\max}(A^T A)}.$$

Vertiefung: Beweis der Spektralnormformel

Beweis.

(1)

$$\|Ax\|_2^2 = x^T A^T A x = x^T P \Lambda P^T x$$

euklidische Norm als Skalarprodukt schreiben, dann die Spektralzerlegung einsetzen

(2)

$$x^T P \Lambda P^T x = (\Lambda^{1/2} P^T x)^T (\Lambda^{1/2} P^T x) = \|\Lambda^{1/2} P^T x\|_2^2$$

$\Lambda = \Lambda^{1/2} \Lambda^{1/2}$ mit $\Lambda^{1/2} = \text{diag}(\sqrt{\lambda_i})$ aufspalten und die Faktoren den beiden Seiten zuschlagen, wieder eine quadrierte Norm

(3)

$$\max_{\|x\|_2=1} \|\Lambda^{1/2} P^T x\|_2^2 = \max_{\|y\|_2=1} \|\Lambda^{1/2} y\|_2^2$$

Substitution $y = P^T x$: Weil P orthogonal ist, gilt $\|y\|_2 = \|x\|_2$ (Bemerkung 3.3.6), und $x \mapsto P^T x$ ist bijektiv; y durchläuft also die Einheitssphäre, wenn x dies tut

(4)

$$\|\Lambda^{1/2} y\|_2^2 = \lambda_1 y_1^2 + \lambda_2 y_2^2 + \dots + \lambda_r y_r^2 \leq \lambda_1 (y_1^2 + \dots + y_r^2) \leq \lambda_1$$

$\Lambda^{1/2}$ ist diagonal; jeden Koeffizienten durch den größten Eigenwert λ_1 abschätzen und $\sum_i y_i^2 = 1$ benutzen

(5)

$$\|A\|_2^2 = \max_{\|x\|_2=1} \|Ax\|_2^2 = \lambda_1$$

die Schranke wird angenommen: Für $y = e_1$ (also $x =$ erste Spalte von P , der Eigenvektor zu λ_1) gilt Gleichheit; Wurzel ziehen liefert die Behauptung

Die Interpretation: Der größte Eigenwert von $A^T A$ ist die quadrierte maximale Streckung, und die Richtung der stärksten Streckung ist der zugehörige Eigenvektor, im Widget oben die rot gestrichelte Richtung $\mathbf{x}^*$. Noch intuitiver wird das über die Singulärwerte von A und die Singulärwertzerlegung; dazu später mehr.

3.3.5 Beispiele

Beispiel 3.3.8

Berechnen wir alle drei Operatornormen von $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$.

Spaltensummennorm (Grün verfolgt die erste, Blau die zweite Spalte):

$$\|A\|_1 = \max\{|2| + |0|, |1| + |3|\} = \max\{2, 4\} = 4.$$

Zeilensummennorm (Rot verfolgt die erste, Violett die zweite Zeile):

$$\|A\|_\infty = \max\{|2| + |1|, |0| + |3|\} = \max\{3, 3\} = 3.$$

Spektralnrm (hier müssen wir rechnen):

$$A^T A = \begin{pmatrix} 2 & 0 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 10 \end{pmatrix}.$$

Das charakteristische Polynom ist $\lambda^2 - 14\lambda + 36$ (Spur 14, Determinante $4 \cdot 10 - 2 \cdot 2 = 36$), mit Nullstellen $\lambda_{1,2} = 7 \pm \sqrt{13}$. Also

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sqrt{7 + \sqrt{13}} \approx \sqrt{10,61} \approx 3,26.$$

Die drei Normen liefern verschiedene Werte (4, 3,26, 3), denn sie messen die maximale Streckung ja auch in verschiedenen Geometrien. Weit auseinander liegen können sie aber nie; mehr dazu in Abschnitt 3.5.

Beispiel 3.3.9 (Identitätsmatrix)

Für die Identitätsmatrix I_n gilt in jeder Operatornorm $\|I_n \mathbf{x}\| / \|\mathbf{x}\| = 1$ für alle $\mathbf{x} \neq \mathbf{0}$, also

$$\|I_n\|_1 = \|I_n\|_2 = \|I_n\|_\infty = 1. \quad \boxtimes$$

Das ist genau das Verhalten, das wir uns eingangs gewünscht haben: Eine Abbildung, die nichts verändert, bekommt Norm 1, unabhängig von der Dimension. Der Schönheitsfehler der elementweisen Normen ($\|I_n\|_F = \sqrt{n}$) ist behoben.

Selbsttest

- Berechnen wir $\|A\|_1$ und $\|A\|_\infty$ für $A = \begin{pmatrix} 1 & -2 \\ 3 & 4 \end{pmatrix}$, ganz ohne Eigenwerte.

Spaltensummen: $|1| + |3| = 4$ und $|-2| + |4| = 6$, also $\|A\|_1 = 6$. Zeilensummen: $|1| + |-2| = 3$ und $|3| + |4| = 7$, also $\|A\|_\infty = 7$. Vorsicht: Die Beträge nicht vergessen, denn das Minuszeichen

in -2 zählt positiv.

- **Wahr oder falsch:** Für jede Orthogonalmatrix Q gilt $\|Q\|_2 = 1$.

Wahr. Wegen $\|Qx\|_2 = \|x\|_2$ (Bemerkung 3.3.6) ist jeder Normquotient gleich 1, also auch das Maximum. Geometrisch: Eine Drehung oder Spiegelung streckt nichts, der Einheitskreis bleibt ein Einheitskreis.

1. Stellen wir im Operatornorm-Widget die Voreinstellung „zehnfach“ ein, also $A = \begin{pmatrix} 20 & 10 \\ 0 & 10 \end{pmatrix}$. Welchen Wert zeigt es für $\|A\|_2$?

Lösung: 22.882

22,882, also genau das Zehnfache des Werts aus Beispiel 3.3.3. Das muss so sein: Multiplizieren wir eine Matrix mit 10, so wird jedes Bild zehnmal so lang, und die absolute Homogenität aus Definition 3.2.1 liefert $\|10A\|_2 = 10\|A\|_2$. Die Ellipse behält ihre Gestalt, die Maximalrichtung x^* bleibt dieselbe, nur der Maßstab wächst.

2. Bei der Voreinstellung „singulär“ zeigt das Widget die Operatornorm 0, weil eine Richtung ganz platt gedrückt wird.

Antwort: Falsch

Die Operatornorm ist ein *Maximum* über Richtungen, kein Minimum. Die Bildellipse entartet zu einer Strecke, ihre kurze Halbachse ist $\sigma_2 = 0$, die lange aber 2,5 – und nur die ist $\|A\|_2$ (Definition 3.3.1).

Vertiefung: Heath §2.3.2.

3.4 Schattennormen

Wir kennen inzwischen zwei Rezepte für Matrixnormen. Die elementweisen Normen aus Abschnitt 3.2 behandeln die Matrix als Zahlenhaufen und verlieren dabei jede Information über die Transformation, die sie beschreibt. Die Operatornormen aus Abschnitt 3.3 messen die Transformation direkt, aber nur ihre *stärkste* Streckung. Eine Matrix streckt verschiedene Richtungen jedoch verschieden stark, und diese vollständige Streckungsinformation steckt in *allen* Eigenwerten von $A^T A$, nicht nur im größten. Die dritte Normfamilie dieses Kapitels nutzt sie alle.

3.4.1 Von Eigenwerten zu Singulärwerten

Erinnern wir uns an den Satz über die Spektralnrm aus Abschnitt 3.3: Dort war $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$. Die Matrix $A^T A$ ist symmetrisch, ihre Eigenwerte sind also reell und sogar nie negativ, denn $x^T A^T A x = \|Ax\|_2^2 \geq 0$ für alle x (die Matrix ist positiv semidefinit). Wir dürfen daher aus allen Eigenwerten die Wurzel ziehen:

$$\sigma_i := \sqrt{\lambda_i}, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0 \text{ Eigenwerte von } A^T A.$$

Diese Zahlen $\sigma_1 \geq \dots \geq \sigma_n \geq 0$ heißen die *Singulärwerte* (singular values) von A . Geometrisch sind sie die Halbachsenlängen der Ellipse, in die A die Einheitssphäre abbildet: σ_1 ist die stärkste Streckung (die Spektralnrm!), σ_n die schwächste. Systematisch entwickeln wir das später mit der Singulärwertzerlegung; hier genügt uns: Der Vektor $(\sigma_1, \dots, \sigma_n)$ ist der vollständige „Streckungs-Fingerabdruck“ der Transformation. Auf diesen Vektor wenden wir jetzt eine Vektornorm an:

Definition 3.4.1 (Schatten-p-Norm)

Sei $A \in \mathbb{R}^{m \times n}$, und sei $\lambda = (\lambda_1, \dots, \lambda_n)$ der Vektor der Eigenwerte von $A^\top A$. Die Schatten- p -Norm von A ist für $1 \leq p < \infty$

$$\|A\|_{S,p} := \|\lambda^{1/2}\|_p = \left(\sum_{i=1}^n \lambda_i^{p/2} \right)^{1/p} = \left(\sum_{i=1}^n \sigma_i^p \right)^{1/p}$$

und für $p = \infty$

$$\|A\|_{S,\infty} := \max_{1 \leq i \leq n} \sigma_i = \sigma_1.$$

Bemerkung 3.4.2

- Das Bauprinzip ist dasselbe wie bei der Vektorisierung in Abschnitt 3.2: „Wende eine Vektor- p -Norm an.“ Nur wenden wir sie nicht auf die rohen Einträge $\text{vec}(A)$ an, sondern auf den Singulärwertvektor, also auf die geometrische Wirkung der Matrix statt auf ihre Zahlen.
- Der Fall $p = \infty$ fügt sich nahtlos ein: Wie bei Vektornormen ist die Maximumsnorm der Grenzwert der p -Normen für $p \rightarrow \infty$.
- Schatten- p -Normen erfüllen tatsächlich alle drei Normaxiome aus Abschnitt 3.2. Definitheit und absolute Homogenität lassen sich direkt nachrechnen; die Dreiecksungleichung ist überraschend knifflig, wir verzichten hier auf den Beweis.

3.4.2 Die drei wichtigen Spezialfälle

Drei Werte von p liefern genau die Normen, die in der Praxis dominieren, und zwei davon kennen wir schon. Der Schlüssel zum überraschendsten Fall ist die Verbindung zwischen Frobenius-Norm und Spur:

Satz 3.4.3 (Frobenius-Norm und Spur)

Für $A \in \mathbb{R}^{m \times n}$ mit Rang r und Eigenwerten $\lambda_1 \geq \dots \geq \lambda_r > 0$ von $A^\top A$ (alle übrigen sind null) gilt

$$\|A\|_F^2 = \text{tr}(A^\top A) = \sum_{i=1}^r \lambda_i.$$

Beweis.

(1)

$$\text{tr}(\mathbf{A}^\top \mathbf{A}) = \sum_{j=1}^n (\mathbf{A}^\top \mathbf{A})_{jj} = \sum_{j=1}^n \sum_{i=1}^m a_{ij}^2 = \|\mathbf{A}\|_F^2$$

Der Diagonaleintrag $(\mathbf{A}^\top \mathbf{A})_{jj}$ ist das Skalarprodukt der j -ten Spalte von $\mathbf{A}$ mit sich selbst, also $\sum_{i=1}^m a_{ij}^2$; Aufsummieren über alle j ergibt die Summe aller quadrierten Einträge, genau die quadrierte Frobenius-Norm aus Abschnitt 3.2

(2)

$$\text{tr}(\mathbf{A}^\top \mathbf{A}) = \text{tr}(\mathbf{P} \mathbf{\Lambda} \mathbf{P}^\top) = \text{tr}(\mathbf{\Lambda} \mathbf{P}^\top \mathbf{P}) = \text{tr}(\mathbf{\Lambda}) = \sum_{i=1}^r \lambda_i$$

Spektralzerlegung $\mathbf{A}^\top \mathbf{A} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^\top$ mit orthogonalem $\mathbf{P}$ und $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_r, 0, \dots, 0)$; dann Zyklizität der Spur aus Abschnitt 3.1 und $\mathbf{P}^\top \mathbf{P} = \mathbf{I}$

□

Damit fallen die Spezialfälle wie reife Früchte:

Korollar 3.4.4 (Spezialfälle der Schatten-p-Norm)

- $p = \infty$: $\|\mathbf{A}\|_{S,\infty} = \sigma_1 = \sqrt{\lambda_{\max}} = \|\mathbf{A}\|_2$, die **Spektralnorm** aus Abschnitt 3.3.
- $p = 2$: $\|\mathbf{A}\|_{S,2} = (\sum_i \lambda_i)^{1/2} = \sqrt{\text{tr}(\mathbf{A}^\top \mathbf{A})} = \|\mathbf{A}\|_F$, die **Frobenius-Norm** (nach Satz 3.4.3).
- $p = 1$: $\|\mathbf{A}\|_{S,1} = \sum_{i=1}^r \sigma_i =: \|\mathbf{A}\|_*$, die **Nuklearnorm** (nuclear norm), für uns neu.

Der Fall $p = 2$ ist eine echte Überraschung. Die Frobenius-Norm hatten wir in Abschnitt 3.2 als elementweise Norm eingeführt, als Musterbeispiel einer Norm, die von der Transformation nichts weiß. Jetzt stellt sich heraus: Sie ist zugleich eine Schatten-Norm, also vollständig durch die Singulärwerte bestimmt. Sie weiß mehr über die Transformation, als ihre Definition vermuten lässt. Das erklärt auch im Nachhinein das Beispiel aus Abschnitt 3.2, in dem drei völlig verschiedene Transformationen dieselbe Frobenius-Norm hatten: Ihre quadrierten Singulärwerte summierten sich jeweils zu 2.

Vertiefung: Nuklearnorm und Niedrigrang-Probleme

Bemerkung 3.4.5 (Nuklearnorm und Niedrigrang-Probleme)

Warum eine dritte Norm, wenn wir schon zwei haben? Der Rang von $\mathbf{A}$ ist die Anzahl der Singulärwerte $\sigma_i > 0$. Rang-Minimierung ist ein kombinatorisch schwieriges Problem; die Nuklearnorm $\sum_i \sigma_i$ ist dagegen konvex und wird klein, wenn viele Singulärwerte (nahezu) verschwinden. Sie dient deshalb in Statistik und maschinellem Lernen als gutmütiger Ersatz für den Rang, etwa bei der Niedrigrang-Approximation und beim Vervollständigen von Matrizen mit fehlenden Einträgen (Empfehlungssysteme).

Rechnen wir die drei Normen einmal konkret aus, und zwar für die Matrix, deren Streckung wir in Abschnitt 3.3 visualisiert haben.

Beispiel 3.4.6

Sei $A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$. Zuerst die Eigenwerte von $A^T A$:

$$A^T A = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 2 \end{pmatrix},$$

$$\det(A^T A - \lambda I) = (4 - \lambda)(2 - \lambda) - 4 = \lambda^2 - 6\lambda + 4 \stackrel{!}{=} 0$$

$$\implies \lambda_1 = 3 + \sqrt{5} \approx 5,236, \quad \lambda_2 = 3 - \sqrt{5} \approx 0,764.$$

Die Singulärwerte sind die Wurzeln daraus:

$$\sigma_1 = \sqrt{3 + \sqrt{5}} \approx 2,288, \quad \sigma_2 = \sqrt{3 - \sqrt{5}} \approx 0,874.$$

Spektralnorm ($p = \infty$): der größte Singulärwert,

$$\|A\|_{S,\infty} = \sigma_1 \approx 2,288.$$

Frobenius-Norm ($p = 2$): über die Singulärwerte und, zur Probe, *elementweise*, beides muss nach Korollar 3.4.4 dasselbe ergeben:

$$\|A\|_{S,2} = \sqrt{\lambda_1 + \lambda_2} = \sqrt{6}, \quad \sqrt{2^2 + 1^2 + 0^2 + 1^2} = \sqrt{6} \approx 2,449. \quad \boxtimes$$

Nuklearnorm ($p = 1$): die Summe der Singulärwerte:

$$\|A\|_{S,1} = \sigma_1 + \sigma_2 = \sqrt{10} \approx 3,162.$$

In R genügt für die Singulärwerte ein Einzeiler:

```
A <- matrix(c(2, 0, 1, 1), 2, 2)
sqrt(eigen(crossprod(A))$values)
#> [1] 2.288246 0.874032
```

3.4.3 Unitäre Invarianz

Was macht die Schatten-Familie so besonders? Drehen wir eine Matrix (genauer: multiplizieren wir sie mit einer Orthogonalmatrix), dann ändern sich ihre Einträge komplett. Elementweise Normen wie Summen- oder Maximumsnorm ändern sich mit. Die Streckungswirkung der Transformation bleibt aber dieselbe, nur die Richtungen rotieren. Da Schattennormen ausschließlich von den Streckungsfaktoren abhängen, dürfen sie sich nicht ändern – und das lässt sich beweisen:

Satz 3.4.7 (Unitäre Invarianz)

Seien $P \in \mathbb{R}^{m \times m}$ und $Q \in \mathbb{R}^{n \times n}$ orthogonal. Dann gilt für alle $A \in \mathbb{R}^{m \times n}$ und alle $p \in [1, \infty]$:

$$\|PAQ\|_{S,p} = \|A\|_{S,p}.$$

Vertiefung: Beweis der unitären Invarianz

Beweis.

(1)

$$(\mathbf{PAQ})^T(\mathbf{PAQ}) = \mathbf{Q}^T \mathbf{A}^T \mathbf{P}^T \mathbf{PAQ} = \mathbf{Q}^T (\mathbf{A}^T \mathbf{A}) \mathbf{Q}$$

Transponierregel $(\mathbf{BC})^T = \mathbf{C}^T \mathbf{B}^T$ und Orthogonalität $\mathbf{P}^T \mathbf{P} = \mathbf{I}$

(2) $(\mathbf{PAQ})^T(\mathbf{PAQ})$ und $\mathbf{A}^T \mathbf{A}$ haben dieselben Eigenwerte $\lambda_1, \dots, \lambda_n$.

$\mathbf{Q}^T = \mathbf{Q}^{-1}$, also ist $\mathbf{Q}^T (\mathbf{A}^T \mathbf{A}) \mathbf{Q}$ ähnlich zu $\mathbf{A}^T \mathbf{A}$, und ähnliche Matrizen haben dasselbe charakteristische Polynom: $\det(\mathbf{Q}^T \mathbf{M} \mathbf{Q} - \lambda \mathbf{I}) = \det(\mathbf{Q}^T (\mathbf{M} - \lambda \mathbf{I}) \mathbf{Q}) = \det(\mathbf{M} - \lambda \mathbf{I})$

(3) Also haben $\mathbf{PAQ}$ und $\mathbf{A}$ dieselben Singulärwerte und damit dieselbe Schatten- p -Norm, für jedes p gleichzeitig.

Schatten- p -Normen hängen nach Definition 3.4.1 nur vom Eigenwertvektor λ ab

□

Eine bemerkenswerte Konsequenz für die Frobenius-Norm: Ihrer elementweisen Definition sieht man die Invarianz $\|\mathbf{PAQ}\|_F = \|\mathbf{A}\|_F$ überhaupt nicht an; erst der Umweg über Korollar 3.4.4 macht sie offensichtlich. Für Summen- und Maximumsnorm gilt nichts dergleichen.

Vertiefung: Robert Schatten und Approximationsprobleme

Bemerkung 3.4.8 (Namensgeber und Anwendung)

- Benannt sind die Normen nach dem Mathematiker *Robert Schatten* (1911–1977), der sie im Kontext unendlichdimensionaler Operatoren untersuchte.
- Die unitäre Invarianz macht Schattennormen zum Werkzeug der Wahl für *Approximationsprobleme*: Wie gut eine Matrix eine andere approximiert, sollte nicht davon abhängen, in welchem (orthogonalen) Koordinatensystem wir die Daten aufschreiben. Messen wir Approximationsfehler in einer Schatten-Norm, ist genau das garantiert. Darauf kommen wir bei der Niedrigrang-Approximation zurück.

Interaktiv: Was überlebt eine Drehung? Schattennormen gegen elementweise Normen

Satz 3.4.7 ist damit bewiesen, aber noch nicht wirklich geglaubt. Eine Drehung schreibt ja jeden einzelnen Eintrag der Matrix um. Welche der Zahlen, die wir aus einer Matrix ablesen können, überstehen das unverändert, und welche wandern mit?

Interaktives Element — nur in der Web-Fassung

Die Antwort teilt unsere Normen sauber in zwei Lager. Ellipse, Halbachsen und alles, was aus ihnen gebaut ist – Spektral-, Frobenius- und Nuklearnorm – stehen still, während die Einträge von $\mathbf{Q}_\theta \mathbf{A}$ munter durchwechseln; die gemeldete Abweichung über den durchfahrenen Drehbereich ist reines Rundungsrauschen in der Größenordnung 10^{-16} . Summen- und Maximumsnorm dagegen bewegen sich sichtbar mit. Sie sind eben elementweise definiert, und die Einträge sind genau das, was eine Drehung ändert.

Selbsttest

- Sei $Q \in \mathbb{R}^{n \times n}$ orthogonal. Was sind die Singulärwerte von Q , und damit ihre Spektral-, Frobenius- und Nuklearnorm?

Wegen $Q^T Q = I$ sind alle Eigenwerte von $Q^T Q$ gleich 1, also $\sigma_1 = \dots = \sigma_n = 1$. Damit ist $\|Q\|_{S,\infty} = 1$, $\|Q\|_{S,2} = \sqrt{n}$ und $\|Q\|_{S,1} = n$. Das passt zu Abschnitt 3.2: Auch dort war $\|I_n\|_F = \sqrt{n}$; nur die Spektralnorm honoriert, dass eine Orthogonalmatrix „nichts streckt“.

- Warum kann man die Nuklearnorm als „weiche“ Version des Rangs auffassen?

Der Rang ist die *Anzahl* der Singulärwerte $\sigma_i > 0$, eine Zählgröße, die springt, sobald ein Singulärwert exakt null wird, und die sich deshalb schlecht optimieren lässt. Die Nuklearnorm $\sum_i \sigma_i$ ersetzt das Zählen durch Summieren: Sie ist als Norm konvex und wird genau dann klein, wenn viele Singulärwerte nahe null liegen, also wenn die Matrix „fast niedrigen Rang“ hat.

1. Drehen wir im Schattennorm-Widget den Regler von 0° auf 90° (die Matrix bleibt die aus Beispiel 3.4.6). Welchen Wert zeigt es dann für die Nuklearnorm $\|Q_\theta A\|_{S,1}$?

Lösung: 3.162

$3,162 = \sqrt{10} = \sigma_1 + \sigma_2$, denselben Wert wie bei 0° . Alle vier Einträge der Matrix sind unterwegs andere geworden, die Singulärwerte nicht: Das ist Satz 3.4.7 an einer konkreten Zahl.

2. Weil die Drehung eine Isometrie ist, bleibt im Widget jede der angezeigten Normen konstant.

Antwort: Falsch

Konstant bleiben nur die Schattennormen, die aus den Singulärwerten gebaut sind. Summen- und Maximumsnorm lesen die Einträge ab und wandern mit; bei $\theta = 45^\circ$ etwa steigt die Summennorm von 4,000 auf 4,243. Satz 3.4.7 gilt für Schattennormen, nicht für alle Matrixnormen.

Vertiefung: MML §4.5 (SVD-Bezug: dort werden die Singulärwerte, auf denen die Schattennormen aufbauen, über die Singulärwertzerlegung systematisch entwickelt).

3.5 Eigenschaften von Matrixnormen

In den Abschnitten 3.2 bis 3.4 haben wir einen ganzen Zoo von Matrixnormen kennengelernt: elementweise Normen wie die Frobenius- und die Maximumsnorm, die Operatornormen und die Schatten-Normen. Drei Fragen drängen sich jetzt auf. Erstens: Wie hängen all diese Normen zusammen? Kann dieselbe Matrix in einer Norm „klein“ und in einer anderen „riesig“ sein? Zweitens: Wie verhalten sich Matrixnormen unter der wichtigsten Matrixoperation überhaupt, der Multiplikation? Und drittens: Was haben wir davon? Die Antworten auf die ersten beiden Fragen heißen *Normenäquivalenz*, *Submultiplikativität* und *Verträglichkeit*; die Antwort auf die dritte führt uns zur Konditionszahl und zur Fehleranalyse, also mitten hinein in die Numerik.

3.5.1 Normenäquivalenz

Beginnen wir mit der ersten Frage. Für Vektornormen auf $\mathbb{R}^n$ gilt bekanntlich: Alle Normen sind äquivalent, d. h. je zwei Normen schätzen sich gegenseitig bis auf konstante Faktoren ab. Weil Matrixnormen auf $\mathbb{R}^{m \times n}$ denselben Axiomen gehorchen (eine Matrix ist aus Sicht der Normaxiome nur ein langer Vektor mit mn Einträgen), überträgt sich dieses Resultat direkt:

Satz 3.5.1 (Alle Matrixnormen sind äquivalent)

Für je zwei Matrixnormen $\|\cdot\|_a$ und $\|\cdot\|_b$ auf $\mathbb{R}^{m \times n}$ (bei festen m, n) existieren Konstanten $c, C > 0$ mit

$$c \|\mathbf{A}\|_a \leq \|\mathbf{A}\|_b \leq C \|\mathbf{A}\|_a \quad \text{für alle } \mathbf{A} \in \mathbb{R}^{m \times n}.$$

Der Beweis läuft wie im Vektorfall über ein Kompaktheitsargument (jede Norm ist als Funktion stetig und nimmt auf der Einheitssphäre einer Referenznorm ihr Minimum und Maximum an); wir übernehmen das Resultat hier ohne Beweis. Interessanter für die Praxis ist, *wie groß* die Konstanten für die konkreten Normen aus den letzten Abschnitten ausfallen:

Beispiel 3.5.2 (Explizite Äquivalenzkonstanten)

Für $\mathbf{A} \in \mathbb{R}^{m \times n}$ gilt unter anderem:

$$\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{\min(m, n)} \|\mathbf{A}\|_2,$$

$$\|\mathbf{A}\|_F \leq \|\mathbf{A}\|_* \leq \sqrt{\min(m, n)} \|\mathbf{A}\|_F,$$

$$\frac{1}{\sqrt{n}} \|\mathbf{A}\|_\infty \leq \|\mathbf{A}\|_2 \leq \sqrt{m} \|\mathbf{A}\|_\infty.$$

($\|\cdot\|_\infty$ ist hier die Zeilensummennorm aus Abschnitt 3.3, nicht zu verwechseln mit der elementweisen Maximumsnorm $\|\cdot\|_M$.) Die Konstanten hängen also nur über die Dimensionen m, n von der Matrix ab, und sie wachsen mit der Dimension. Für sehr große Matrizen können zwei Normen also durchaus um Größenordnungen auseinanderliegen.

Vertiefung: Beweis der Kette zwischen Spektral- und Frobenius-Norm

Die erste Kette können wir mit dem Schatten-Blick aus Abschnitt 3.4 vollständig beweisen. Dort haben wir gesehen: Sind $\lambda_1 \geq \dots \geq \lambda_r > 0$ die positiven Eigenwerte von $\mathbf{A}^\top \mathbf{A}$ (mit $r = \text{rang}(\mathbf{A})$), so ist $\|\mathbf{A}\|_2 = \sqrt{\lambda_1}$ und $\|\mathbf{A}\|_F = \sqrt{\sum_{i=1}^r \lambda_i}$. Wir verfolgen λ_1 rot und die Eigenwertsumme grün:

Beweis.

(1)

$$\|\mathbf{A}\|_2^2 = \lambda_1 \leq \sum_{i=1}^r \lambda_i = \|\mathbf{A}\|_F^2$$

alle λ_i sind positiv, die Summe ist also mindestens so groß wie ihr größter Summand λ_1

(2)

$$\|\mathbf{A}\|_F^2 = \sum_{i=1}^r \lambda_i \leq r \cdot \lambda_1 \leq \min(m, n) \cdot \lambda_1 = \min(m, n) \cdot \|\mathbf{A}\|_2^2$$

jeder der r Summanden ist höchstens λ_1 ; und der Rang erfüllt $r \leq \min(m, n)$

(3)

$$\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{\min(m, n)} \|\mathbf{A}\|_2$$

Wurzelziehen ist monoton, erhält also beide Ungleichungen

□

Bemerkung 3.5.3 (Äquivalenz ist nicht Gleichheit)

Vorsicht: Äquivalenz heißt nur, dass keine Norm „unendlich viel größer“ sein kann als eine andere. Verschiedene Normen können Matrizen aber durchaus *unterschiedlich ordnen*. Betrachten wir

$$\mathbf{A} = \mathbf{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1,2 & 0 \\ 0 & 0 \end{pmatrix}.$$

In der Frobenius-Norm ist $\mathbf{A}$ die größere Matrix: $\|\mathbf{A}\|_F = \sqrt{2} \approx 1,41 > 1,2 = \|\mathbf{B}\|_F$. In der Spektralnorm ist es umgekehrt: $\|\mathbf{A}\|_2 = 1 < 1,2 = \|\mathbf{B}\|_2$. Beide Antworten sind richtig, denn jede Norm misst eben etwas anderes: Die Frobenius-Norm summiert die „Gesamtmasse“ aller Einträge, die Spektralnorm die maximale Streckung. Welche Norm die passende ist, entscheidet die Anwendung.

Interaktiv: Wie scharf sind die Äquivalenzkonstanten?

Eine Frage bleibt offen: Sind die Konstanten in Beispiel 3.5.2 nur bequeme Abschätzungen, oder lässt sich der Faktor $\sqrt{\min(m, n)}$ noch drücken? Anders gefragt: Gibt es Matrizen, für die eine der beiden Ungleichungen mit Gleichheit steht? Weil alle drei beteiligten Normen nur von den Singulärwerten abhängen, können wir das für 2×2 vollständig durchprobieren.

Interaktives Element — nur in der Web-Fassung

Beide Enden der Kette werden angenommen, und zwar genau an den Rändern des zulässigen Bereichs: unten, wenn der zweite Singulärwert verschwindet, oben, wenn beide gleich sind. Dazwischen ist der Quotient echt kleiner als $\sqrt{2}$. Verbessern lässt sich die Konstante also nicht, und sie wird schlecht: Im $\mathbb{R}^{n \times n}$ steht dort $\sqrt{n}$, und für $\mathbf{A} = \mathbf{I}_n$ ist diese Schranke scharf.

3.5.2 Submultiplikativität

Kommen wir zur zweiten Frage. Die Normaxiome aus Abschnitt 3.2 regeln per Dreiecksungleichung, wie sich Normen unter *Addition* verhalten: $\|\mathbf{A} + \mathbf{B}\| \leq \|\mathbf{A}\| + \|\mathbf{B}\|$. Über das *Matrixprodukt* sagen sie dagegen gar nichts. Für die Numerik ist das Produkt aber die zentrale Operation: Algorithmen sind Ketten von Matrixmultiplikationen, und wir wollen abschätzen können, wie stark so eine Kette Fehler aufbläht. Normen, die das erlauben, bekommen einen eigenen Namen:

Definition 3.5.4 (Submultiplikative Matrixnorm)

Eine Matrixnorm $\|\cdot\|$ heißt *submultiplikativ*, wenn für alle Matrizen mit passenden Formaten ($\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$) gilt:

$$\|\mathbf{AB}\| \leq \|\mathbf{A}\| \cdot \|\mathbf{B}\|.$$

Die gute Nachricht: Die wichtigsten Normen aus diesem Kapitel haben diese Eigenschaft.

Satz 3.5.5 (Operatornormen sind submultiplikativ)

Jede Operatornorm (Abschnitt 3.3) ist submultiplikativ.

Der Beweis ruht auf einer Ungleichung, die wir im Rest des Kapitels immer wieder brauchen und deshalb eigens festhalten:

Bemerkung 3.5.6 (Die Operatornorm als Streckungsschranke)

Für jede Matrix M und jeden Vektor y gilt

$$\|My\|_V \leq \|M\|_V \cdot \|y\|_V.$$

Die Operatornorm ist ja als Maximum des Quotienten $\|My\|_V / \|y\|_V$ definiert, also mindestens so groß wie dieser Quotient an jeder einzelnen Stelle $y \neq 0$; für $y = 0$ stehen links und rechts null.

Vertiefung: Beweis der Submultiplikativität von Operatornormen

Beweis.

- (1) Sei x ein Vektor mit $\|x\|_V = 1$. Dann:

$$\|(AB)x\|_V = \|A(Bx)\|_V \leq \|A\|_V \|Bx\|_V \leq \|A\|_V \|B\|_V.$$

Bemerkung 3.5.6 zweimal anwenden: erst auf A mit dem Vektor Bx , dann auf B mit x , wobei $\|x\|_V = 1$

- (2)

$$\|AB\|_V = \max_{\|x\|_V=1} \|(AB)x\|_V \leq \|A\|_V \cdot \|B\|_V$$

die Operatornorm von AB ist das Maximum der linken Seite über alle x mit $\|x\|_V = 1$; die rechte Seite hängt nicht von x ab

□

Auch alle Schatten-Normen aus Abschnitt 3.4 (insbesondere die Frobenius-Norm und die Nuklearnorm) sind submultiplikativ. Der Beweis braucht Ungleichungen für Singulärwerte von Produkten, die wir hier nicht entwickeln; wir merken uns das Resultat.

Interaktiv: Gilt $\|AB\| \leq \|A\| \cdot \|B\|$ für jede Matrixnorm?

Damit sind die Operator- und die Schattennormen versorgt. Bleibt die Frage, ob wir uns die Ungleichung überhaupt merken müssen oder ob sie ohnehin für jede Matrixnorm gilt. Tippen wir zuerst, bevor wir weiterlesen.

Interaktives Element — nur in der Web-Fassung

Es gibt sie, die Norm, die aus der Reihe tanzt, und es ist die elementweise Maximumsnorm. Schon das einfachste Paar bringt sie zu Fall: die Einsermatrix mit sich selbst multipliziert.

Rechnen wir diesen Fall von Hand nach.

Beispiel 3.5.7 (Die Maximumsnorm ist nicht submultiplikativ)

Betrachten wir die Einsermatrix $A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ und ihr Quadrat. Jeder Eintrag von A^2 ist ein Skalarprodukt aus einer Zeile und einer Spalte voller Einsen:

$$\mathbf{A}^2 = \begin{pmatrix} 1 \cdot 1 + 1 \cdot 1 & 1 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 1 \cdot 1 & 1 \cdot 1 + 1 \cdot 1 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}.$$

Damit ist

$$\|\mathbf{A}^2\|_M = 2 > 1 = 1 \cdot 1 = \|\mathbf{A}\|_M \cdot \|\mathbf{A}\|_M.$$

Die Maximumsnorm „übersieht“, dass bei der Matrixmultiplikation viele Produkte *aufsummiert* werden, denn sie schaut nur auf den größten einzelnen Eintrag.

Bemerkung 3.5.8 (Reparatur: die Gesamtnorm)

Der Defekt lässt sich durch einen dimensionsabhängigen Faktor beheben: Die *Gesamtnorm* $\|\mathbf{A}\|_G := \sqrt{mn} \|\mathbf{A}\|_M$ ist submultiplikativ. Der Nachweis ist eine gute Übung. Er benutzt genau die Summenstruktur der Matrixmultiplikation, die die Maximumsnorm ignoriert. (Probe am Beispiel 3.5.7 mit $m = n = 2$, also $\sqrt{mn} = 2$: $\|\mathbf{A}\|_G = 2 \cdot 1 = 2$ und $\|\mathbf{A}^2\|_G = 2 \cdot 2 = 4 \leq 4 = \|\mathbf{A}\|_G^2$, es passt.)

Warum ist Submultiplikativität so wertvoll? Drei Gründe. Erstens kontrolliert sie die *Fehlerfortpflanzung*: Für eine Kette $\mathbf{A}_1 \mathbf{A}_2 \cdots \mathbf{A}_k$ folgt durch wiederholtes Anwenden $\|\mathbf{A}_1 \cdots \mathbf{A}_k\| \leq \|\mathbf{A}_1\| \cdots \|\mathbf{A}_k\|$; die Normen der Faktoren deckeln die Norm des Produkts. Zweitens ermöglicht sie Konvergenzanalysen iterativer Verfahren: Aus $\|\mathbf{A}\| < 1$ folgt $\|\mathbf{A}^k\| \leq \|\mathbf{A}\|^k \rightarrow 0$, die Potenzen einer „kontrahierenden“ Matrix sterben also garantiert aus. Drittens verbindet sie Normen mit dem Spektralradius: Man kann zeigen, dass $\rho(\mathbf{A}) \leq \|\mathbf{A}\|$ für jede submultiplikative Norm gilt: Jede solche Norm ist eine obere Schranke für die betragsgrößten Eigenwerte.

3.5.3 Verträglichkeit von Normen

Submultiplikativität vergleicht Matrixnormen mit Matrixnormen. Genauso oft brauchen wir aber ein gemischtes Szenario: Eine Matrix wirkt auf einen *Vektor*, und wir wollen $\|\mathbf{A}\mathbf{x}\|$ durch die Norm von $\mathbf{A}$ und die Norm von $\mathbf{x}$ abschätzen. Das führt auf folgenden Begriff:

Definition 3.5.9 (Verträgliche Norm)

Eine Matrixnorm $\|\cdot\|$ ist *verträglich* (kompatibel) mit einer Vektornorm $\|\cdot\|_V$, wenn für alle $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{x} \in \mathbb{R}^n$ gilt:

$$\|\mathbf{A}\mathbf{x}\|_V \leq \|\mathbf{A}\| \cdot \|\mathbf{x}\|_V.$$

(Dabei bezeichnet $\|\cdot\|_V$ die Vektornorm gleichen Typs auf $\mathbb{R}^n$ und auf $\mathbb{R}^m$.)

Satz 3.5.10 (Wichtige Verträglichkeiten)

1. Jede submultiplikative Matrixnorm ist mit „sich selbst“ verträglich, d. h. mit der Vektornorm, die entsteht, wenn wir $\mathbf{x} \in \mathbb{R}^n$ als $n \times 1$ -Matrix lesen.
2. Jede Operatornorm ist mit der Vektornorm verträglich, die sie induziert.
3. Die Frobenius-Norm ist mit der euklidischen Norm verträglich: $\|\mathbf{A}\mathbf{x}\|_2 \leq \|\mathbf{A}\|_F \|\mathbf{x}\|_2$.

Vertiefung: Beweis der drei Verträglichkeiten*Beweis.*

- (1) Zu (1): Lesen wir $\mathbf{x}$ als $n \times 1$ -Matrix, so ist $\mathbf{Ax}$ ein Matrixprodukt, und die Submultiplikativität liefert direkt

$$\|\mathbf{Ax}\| \leq \|\mathbf{A}\| \cdot \|\mathbf{x}\|.$$

Submultiplikativität (Definition 3.5.4), angewendet auf das Matrixprodukt einer $m \times n$ - mit einer $n \times 1$ -Matrix

- (2) Zu (2): Das ist genau Bemerkung 3.5.6: Für $\mathbf{x} \neq \mathbf{0}$ gilt

$$\frac{\|\mathbf{Ax}\|_V}{\|\mathbf{x}\|_V} \leq \max_{\mathbf{y} \neq \mathbf{0}} \frac{\|\mathbf{Ay}\|_V}{\|\mathbf{y}\|_V} = \|\mathbf{A}\|_V.$$

die Operatornorm ist als Maximum des Quotienten definiert, also ist sie mindestens so groß wie der Quotient an jeder einzelnen Stelle $\mathbf{x} \neq \mathbf{0}$

- (3) Zu (3): Schreiben wir $\mathbf{A}$ zeilenweise mit Zeilenvektoren $\mathbf{a}_1^\top, \dots, \mathbf{a}_m^\top$. Die i -te Komponente von $\mathbf{Ax}$ ist das Skalarprodukt $\mathbf{a}_i^\top \mathbf{x}$, also

$$\|\mathbf{Ax}\|_2^2 = \sum_{i=1}^m (\mathbf{a}_i^\top \mathbf{x})^2 \leq \sum_{i=1}^m \|\mathbf{a}_i\|_2^2 \|\mathbf{x}\|_2^2.$$

Cauchy-Schwarz-Ungleichung für jedes einzelne Skalarprodukt $\mathbf{a}_i^\top \mathbf{x}$

- (4)

$$\|\mathbf{Ax}\|_2^2 \leq \left(\sum_{i=1}^m \|\mathbf{a}_i\|_2^2 \right) \|\mathbf{x}\|_2^2 = \|\mathbf{A}\|_F^2 \|\mathbf{x}\|_2^2$$

die Summe der quadrierten Zeilennormen ist genau die Summe aller quadrierten Einträge, also $\|\mathbf{A}\|_F^2$; dann Wurzel ziehen

□

Wozu das Ganze? Verträglichkeit ist das Werkzeug für *Fehlerabschätzungen*. Steckt in einem Vektor ein Fehler $\mathbf{e}$ (etwa aus Rundungsfehlern früherer Rechenschritte) und multiplizieren wir mit $\mathbf{A}$, dann garantiert $\|\mathbf{Ae}\|_V \leq \|\mathbf{A}\| \cdot \|\mathbf{e}\|_V$: Ein kleiner Eingangsfehler bleibt nach der Multiplikation kontrolliert, denn die Matrixnorm ist der Verstärkungsfaktor im schlimmsten Fall. Diese Denkfigur bauen wir jetzt zu den zwei wichtigsten Anwendungen aus.

3.5.4 Matrixnormen und Konditionierung

Die Konditionszahl misst, wie stark ein Problem relative Eingabefehler verstärkt. Für das Lösen linearer Gleichungssysteme $\mathbf{Ax} = \mathbf{b}$ lässt sich dieser Verstärkungsfaktor direkt aus Matrixnormen gewinnen. Das rechnen wir im Kapitel über lineare Gleichungssysteme nach, halten die Definition aber schon hier fest:

Definition 3.5.11 (Konditionszahl einer Matrix)

Für eine invertierbare Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ und eine submultiplikative Matrixnorm $\|\cdot\|$ ist die *Konditionszahl*

$$\kappa(\mathbf{A}) = \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\|;$$

für singuläre Matrizen setzen wir $\kappa(\mathbf{A}) = \infty$. Speziell für die Spektralnorm ergibt sich die *Spektralkondition*

$$\kappa_2(\mathbf{A}) = \|\mathbf{A}\|_2 \cdot \|\mathbf{A}^{-1}\|_2 = \frac{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})^{1/2}}{\lambda_{\min}(\mathbf{A}^\top \mathbf{A})^{1/2}}.$$

Diese Definition ist für jede submultiplikative Matrixnorm algebraisch sinnvoll. Die geometrische Interpretation als Verhältnis von Streckungsfaktoren gilt jedoch für *induzierte Operatornormen*. Bei anderen Normen kann auch die Normierung anders aussehen: Beispielsweise ist $\kappa_F(\mathbf{I}_n) = n$, nicht 1.

Die zweite Gleichheit rechnen wir hier nicht nach; mit der Singulärwertzerlegung fällt sie in Kapitel 6 von selbst ab. Inhaltlich misst die Konditionszahl in der Spektralnorm das *Verhältnis der extremen Streckungsfaktoren*: maximale Streckung durch $\mathbf{A}$ geteilt durch minimale Streckung, also den Quotienten aus größtem und kleinstem Singulärwert. Für *symmetrische* Matrizen sind die Singulärwerte gerade die Beträge der Eigenwerte; dort ist $\kappa_2(\mathbf{A}) = \max_i |\lambda_i| / \min_i |\lambda_i|$.

Bemerkung 3.5.12 (Interpretation der Konditionszahl)

Für eine induzierte Operatornorm gilt:

- $\kappa(\mathbf{A}) \approx 1$: gut konditioniert; $\mathbf{A}$ streckt alle Richtungen ungefähr gleich stark.
- $\kappa(\mathbf{A}) \gg 1$: schlecht konditioniert; manche Richtungen werden enorm gestreckt, andere fast plattgedrückt.
- $\kappa(\mathbf{A}) = \infty$: singulär.

Kleiner als 1 kann die Konditionszahl (in einer Operatornorm) nie werden: Wegen $\|\mathbf{I}\| = 1$ und der Submultiplikativität gilt

$$1 = \|\mathbf{I}\| = \|\mathbf{A}\mathbf{A}^{-1}\| \leq \|\mathbf{A}\| \cdot \|\mathbf{A}^{-1}\| = \kappa(\mathbf{A}).$$

Den Idealwert $\kappa_2 = 1$ erreichen zum Beispiel Orthogonalmatrizen: Sie lassen alle euklidischen Längen unverändert (Abschnitt 3.3), strecken also jede Richtung mit dem Faktor 1.

3.5.5 Normen in der Fehleranalyse

Zum Abschluss zeigen wir an einem konkreten Resultat, wie Matrixnormen in der Fehleranalyse arbeiten. Angenommen, wir lösen $\mathbf{A}\mathbf{x} = \mathbf{b}$ numerisch und erhalten (dank Rundungsfehlern) nicht die exakte Lösung, sondern eine Näherung $\tilde{\mathbf{x}}$. Die *Rückwärts-Fehleranalyse* (backward error analysis) stellt die Frage geschickt um: Statt zu fragen, wie falsch $\tilde{\mathbf{x}}$ ist (Vorwärtsfehler), fragen wir: *Für welches gestörte Problem ist $\tilde{\mathbf{x}}$ die exakte Lösung?* Ist die nötige Störung winzig, hat der Algorithmus sein Bestes getan; alles Weitere ist Sache der Kondition des Problems.

Satz 3.5.13 (Rückwärtsfehler beim linearen Gleichungssystem)

Sei $\tilde{\mathbf{x}} \neq \mathbf{0}$ eine Näherungslösung von $\mathbf{A}\mathbf{x} = \mathbf{b}$ mit Residuum $\mathbf{r} = \mathbf{A}\tilde{\mathbf{x}} - \mathbf{b}$. Dann löst $\tilde{\mathbf{x}}$ das gestörte System

$$(\mathbf{A} + \Delta\mathbf{A})\tilde{\mathbf{x}} = \mathbf{b} \quad \text{mit} \quad \Delta\mathbf{A} = -\frac{\mathbf{r}\tilde{\mathbf{x}}^\top}{\|\tilde{\mathbf{x}}\|_2^2}$$

exakt, und die relative Größe der Störung ist

$$\frac{\|\Delta A\|_2}{\|A\|_2} = \frac{\|r\|_2}{\|A\|_2 \cdot \|\tilde{x}\|_2}.$$

Vertiefung: Beweis der Rückwärtsfehler-Formel

Beweis.

(1)

$$\Delta A \tilde{x} = -\frac{r(\tilde{x}^\top \tilde{x})}{\|\tilde{x}\|_2^2} = -r$$

$r \tilde{x}^\top$ ist ein äußeres Produkt; multiplizieren wir es mit $\tilde{x}$, entsteht das Skalarprodukt $\tilde{x}^\top \tilde{x} = \|\tilde{x}\|_2^2$, das sich gegen den Nenner kürzt

(2)

$$(A + \Delta A) \tilde{x} = A \tilde{x} - r = b + r - r = b$$

Definition des Residuums: $A \tilde{x} = b + r$

(3)

$$\|\Delta A\|_2 = \frac{\|r \tilde{x}^\top\|_2}{\|\tilde{x}\|_2^2} = \frac{\|r\|_2 \|\tilde{x}\|_2}{\|\tilde{x}\|_2^2} = \frac{\|r\|_2}{\|\tilde{x}\|_2}$$

für Rang-1-Matrizen gilt $\|uv^\top\|_2 = \|u\|_2 \|v\|_2$: wegen $(uv^\top)x = (v^\top x)u$ wird $\|(uv^\top)x\|_2 = |v^\top x| \|u\|_2$ nach Cauchy-Schwarz maximal für $x = v / \|v\|_2$

(4)

$$\frac{\|\Delta A\|_2}{\|A\|_2} = \frac{\|r\|_2}{\|A\|_2 \cdot \|\tilde{x}\|_2}$$

beide Seiten durch $\|A\|_2$ teilen

□

Matrixnormen machen den Rückwärtsfehler damit *berechenbar*: Rechts stehen nur Größen, die nach der Rechnung vorliegen, und ein kleines relatives Residuum bedeutet, dass $\tilde{x}$ die exakte Lösung eines nur winzig gestörten Systems ist. Wie weit $\tilde{x}$ dann von der wahren Lösung entfernt sein kann, entscheidet die Konditionszahl aus Definition 3.5.11. Als Faustregel gilt: Vorwärtsfehler $\leq \kappa(A) \times$ Rückwärtsfehler.

Selbsttest

- Gilt $\|A^2\| \leq \|A\|^2$ für jede Matrixnorm?

Nein, das gilt nur für submultiplikative Normen. Die Maximumsnorm liefert ein Gegenbeispiel: Für die Einsermatrix aus Beispiel 3.5.7 ist $\|A^2\|_M = 2 > 1 = \|A\|_M^2$. Operator- und Schatten-Normen (also auch Spektral-, Frobenius- und Nuklearnorm) erfüllen die Ungleichung dagegen immer.

- Sei Q eine Orthogonalmatrix. Was ist $\kappa_2(Q)$?

$\kappa_2(Q) = 1$. Wegen $\|Qx\|_2 = \|x\|_2$ für alle x ist $\|Q\|_2 = 1$; und $Q^{-1} = Q^\top$ ist selbst orthogonal, also auch $\|Q^{-1}\|_2 = 1$. Orthogonalmatrizen sind perfekt konditioniert – ein Hauptgrund, warum numerische Verfahren so gerne mit ihnen arbeiten.

- Für die Einheitsmatrix gilt $\|I_n\|_2 = 1$, aber $\|I_n\|_F = \sqrt{n}$. Widerspricht das der Normen-äquivalenz (Satz 3.5.1)? Und was sagt uns das Beispiel über die Konstante $\sqrt{\min(m, n)}$ aus

Beispiel 3.5.2?

Kein Widerspruch: Satz 3.5.1 gilt bei *festen* Dimensionen, und die Konstanten dürfen von m, n abhängen. Das Beispiel zeigt sogar, dass die Konstante scharf ist: $\mathbf{I}_n$ erfüllt $\|\mathbf{I}_n\|_F = \sqrt{n} = \sqrt{\min(n, n)} \cdot \|\mathbf{I}_n\|_2$; die obere Schranke aus Beispiel 3.5.2 wird hier mit Gleichheit angenommen, besser geht es also nicht.

1. Ziehen wir im Äquivalenz-Widget den Punkt auf die Winkelhalbierende $\sigma_1 = \sigma_2$. Welchen Quotienten $\|\mathbf{A}\|_F / \|\mathbf{A}\|_2$ zeigt es dann?

Lösung: 1.414

$\sqrt{2} \approx 1,414$, unabhängig davon, wie weit außen auf der Winkelhalbierenden wir landen. Dort ist $\mathbf{A}$ ein Vielfaches der Einheitsmatrix, und die rechte Ungleichung aus Beispiel 3.5.2 steht mit Gleichheit. Auf der waagerechten Achse dagegen ist der Quotient 1.

2. Stellen wir im Submultiplikativitäts-Widget die Einsermatrix für $\mathbf{A}$ und $\mathbf{B}$ ein und wählen wir die Maximumsnorm. Welchen Quotienten $\|\mathbf{AB}\| / (\|\mathbf{A}\| \cdot \|\mathbf{B}\|)$ zeigt der Balken?

Lösung: 2

2, also doppelt so viel wie erlaubt. Alle Einträge von $\mathbf{AB}$ sind 2, alle Einträge von $\mathbf{A}$ und $\mathbf{B}$ sind 1: $2/(1 \cdot 1) = 2$ (Beispiel 3.5.7). Schalten wir bei denselben Matrizen auf eine Operatornorm um, springt der Balken auf genau 1 – dort ist die Schranke aus Satz 3.5.5 scharf, aber eben nicht verletzt.

3. Der Würfelknopf des Widgets zeigt bei jedem Klick andere Zufallsmatrizen, also sieht jeder Leser andere Beispiele.

Antwort: Falsch

Die Zahlen kommen aus einem geseedeten Generator. Die Folge der Beispiele ist für alle Leser dieselbe, und über diese Folge bleibt der Quotient in jeder Operator- und Schattenorm unter 1; nur die Maximumsnorm schießt darüber hinaus.

Vertiefung: Heath §2.3.

3.6 Zusammenfassung

Fassen wir das Kapitel zusammen. Der rote Faden war die Frage, wie wir die „Größe“ einer Matrix messen, und zwar so, dass die Zahl etwas über die lineare Abbildung $\mathbf{x} \mapsto \mathbf{Ax}$ aussagt, nicht nur über die Einträge. Als Werkzeuge haben wir die Spur als skalare Invariante und drei Familien von Matrixnormen kennengelernt.

Die Konzepte im Überblick

Konzept	Kernaussage	Abschnitt
Spur	$\text{tr}(\mathbf{A}) = \sum_{i=1}^n a_{ii}$; linear, zyklisch ($\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$) und damit ähnlichkeitsinvariant; gleich der Summe der Eigenwerte (Koeffizient des charakteristischen Polynoms)	3.1
Matrixnorm	dieselben Axiome wie bei Vektornormen: Definitheit, absolute Homogenität, Dreiecksungleichung	3.2
elementweise Normen	Vektornorm auf $\text{vec}(\mathbf{A})$: $\ \mathbf{A}\ _F$, $\ \mathbf{A}\ _S$, $\ \mathbf{A}\ _M$; billig, aber blind für die Transformation	3.2
Operatornormen	maximale Streckung $\max_{\mathbf{x} \neq 0} \ \mathbf{Ax}\ _V / \ \mathbf{x}\ _V$: Spaltensummennorm $\ \mathbf{A}\ _1$, Spektralnorm $\ \mathbf{A}\ _2$, Zeilensummennorm $\ \mathbf{A}\ _\infty$	3.3
Schatten-p-Normen	p -Norm des Vektors der Wurzeln der Eigenwerte von $\mathbf{A}^\top \mathbf{A}$; invariant unter orthogonalen Transformationen	3.4
Eigenschaften	Normäquivalenz, Submultiplikativität ($\ \mathbf{AB}\ \leq \ \mathbf{A}\ \ \mathbf{B}\ $), Verträglichkeit mit Vektornormen	3.5

Die drei Familien sind dabei kein loses Nebeneinander, sondern eng verwoben. Die Frobenius-Norm gehört gleich zwei Familien an: Sie ist elementweise Norm (die euklidische Norm von $\text{vec}(\mathbf{A})$) und zugleich die Schatten-Norm mit $p = 2$. Die Brücke schlägt die Spur: $\|\mathbf{A}\|_F^2 = \text{tr}(\mathbf{A}^\top \mathbf{A})$. Und die Spektralnorm verbindet Operator- und Schatten-Welt: Sie ist die von der euklidischen Norm induzierte Operatornorm *und* die Schatten-Norm mit $p = \infty$, berechenbar als $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$, der Anknüpfungspunkt zur Singulärwertzerlegung, die uns später noch ausführlich beschäftigen wird.

Vertiefung: Was kostet die Berechnung der einzelnen Normen?

Für die Praxis ist entscheidend, wie teuer die einzelnen Normen sind. Spalten- und Zeilensummen-norm sowie die Frobenius-Norm brauchen nur einen Durchlauf über alle Einträge. Für die exakte Spektral- und Nuklearnorm verwenden dichte Standardverfahren dagegen eine entsprechende Spektralzerlegung bzw. SVD. Benötigen wir nur eine Approximation des größten Singulärwerts, können iterative oder randomisierte Verfahren deutlich billiger sein.

Norm	Aufwand	Algorithmus
$\ \mathbf{A}\ _1$	$O(mn)$	Spaltensummen berechnen
$\ \mathbf{A}\ _\infty$	$O(mn)$	Zeilensummen berechnen
$\ \mathbf{A}\ _F$	$O(mn)$	Quadratsumme aller Einträge
$\ \mathbf{A}\ _2$	$O(\min(m^2n, mn^2))$	größten Singulärwert berechnen (dicht: SVD; approximativ: Iteration)
$\ \mathbf{A}\ _*$	$O(\min(m^2n, mn^2))$	SVD, dann Singulärwerte summieren

Vertiefung: Ausblick: Normen als Regularisierer im maschinellen Lernen

Bemerkung 3.6.1

Normen messen nicht nur Fehler: Im maschinellen Lernen steuern sie als Strafterme, *welche* Lösung ein Schätzproblem liefert. Drei prominente Beispiele, alle Varianten des KQ-Problems:

$$\text{Ridge: } \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_2^2, \quad \text{LASSO: } \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_1,$$

$$\text{Matrix Completion: } \min_{\mathbf{X}} \|\mathbf{P}_{\Omega}(\mathbf{X}) - \mathbf{P}_{\Omega}(\mathbf{M})\|_F^2 + \lambda \|\mathbf{X}\|_*.$$

Verschiedene Normen erzwingen verschiedene Lösungseigenschaften: Die 2-Norm schrumpft alle Koeffizienten gleichmäßig, die 1-Norm setzt viele exakt auf null, und die Nukleornorm aus Abschnitt 3.4 begünstigt Lösungen mit kleinem Rang (Niedrigrang-Approximation).

Selbsttest

Prüfen wir zum Abschluss, ob die Kernaussagen sitzen. Welche der folgenden Aussagen über die Spur sind wahr ($\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$)?

1. $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{A}) \cdot \text{tr}(\mathbf{B})$

Antwort: Falsch

Die Spur ist mit dem Matrixprodukt nicht verträglich. Gegenbeispiel $\mathbf{A} = \mathbf{B} = \mathbf{I}_2$: links steht $\text{tr}(\mathbf{I}_2) = 2$, rechts $2 \cdot 2 = 4$.

2. $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$

Antwort: Wahr

Das ist die zyklische Eigenschaft aus Abschnitt 3.1, die wichtigste Rechenregel der Spur.

3. $\text{tr}(\mathbf{A}^T) = -\text{tr}(\mathbf{A})$

Antwort: Falsch

Transponieren spiegelt an der Diagonale und lässt die Diagonaleinträge unverändert; es gilt $\text{tr}(\mathbf{A}^T) = \text{tr}(\mathbf{A})$, ohne Vorzeichenwechsel.

4. $\text{tr}(\mathbf{I}_n) = n$

Antwort: Wahr

Auf der Diagonale stehen n Einsen, also ist die Summe n .

5. Ist $\mathbf{A}$ invertierbar, so gilt $\text{tr}(\mathbf{A}^{-1}) = 1/\text{tr}(\mathbf{A})$.

Antwort: Falsch

Gegenbeispiel $\mathbf{A} = 2\mathbf{I}_2$: dann ist $\mathbf{A}^{-1} = \frac{1}{2}\mathbf{I}_2$, also $\text{tr}(\mathbf{A}^{-1}) = 1$, aber $1/\text{tr}(\mathbf{A}) = \frac{1}{4}$.

Berechnen wir für $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$ die Spaltensummennorm $\|\mathbf{A}\|_1$, die Zeilensummennorm $\|\mathbf{A}\|_{\infty}$ und die Frobenius-Norm $\|\mathbf{A}\|_F$.

Spaltensummennorm: Die Beträge spaltenweise summieren; die **erste Spalte** gewinnt:

$$\|\mathbf{A}\|_1 = \max\{|1| + |3|, |2| + |0|\} = \max\{4, 2\} = 4.$$

Zeilensummennorm: Die Beträge zeilenweise summieren; hier liefern **beide Zeilen** denselben Wert:

$$\|\mathbf{A}\|_{\infty} = \max\{|1| + |2|, |3| + |0|\} = \max\{3, 3\} = 3.$$

Frobenius-Norm: Wurzel aus der Quadratsumme aller Einträge:

$$\|\mathbf{A}\|_F = \sqrt{1^2 + 2^2 + 3^2 + 0^2} = \sqrt{14} \approx 3,74.$$

Probe über die Spur (Abschnitt 3.2): $\mathbf{A}^{\top} \mathbf{A} = \begin{pmatrix} 10 & 2 \\ 2 & 4 \end{pmatrix}$, also $\text{tr}(\mathbf{A}^{\top} \mathbf{A}) = 10 + 4 = 14 = \|\mathbf{A}\|_F^2$. ☒ Zum

Vergleich: Die Spektralnorm ist $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^{\top} \mathbf{A})} = \sqrt{7 + \sqrt{13}} \approx 3,26$, wie es sein muss kleiner als die Frobenius-Norm.

Wie geht es weiter?

Warum der ganze Aufwand? Weil Matrixnormen das zentrale Messwerkzeug der *Fehleranalyse* sind, mit der wir uns im nächsten Kapitel beschäftigen. Konditionszahl, Rückwärtsfehler, Submultiplikativität und Verträglichkeit aus Abschnitt 3.5 sind genau die Begriffe, mit denen wir dort abschätzen, wie viel Vertrauen eine numerisch berechnete Lösung verdient.

Vertiefung: Heath §2.3; MML §4.1/4.5.

Fehler, Kondition & Stabilität

4.1 Fehlermaße und Fehlerzerlegung

In Kapitel 2 haben wir gesehen, dass Computer praktisch nie exakt rechnen: Schon das Speichern einer Zahl im Gleitkommaformat erzeugt Rundungsfehler, und jede Rechenoperation kann weitere hinzufügen (Abschnitt 2.1). Für Statistiker/innen kommt noch etwas Grundsätzlicheres dazu: Unsere Inputs sind Daten, und Daten sind gemessen, also selbst schon fehlerbehaftet, bevor der Computer überhaupt anfängt zu rechnen. Dieses Kapitel entwickelt das Handwerkszeug, um mit beidem systematisch umzugehen. Wir klären zuerst, wie man Fehler überhaupt *misst*, und zerlegen dann den Gesamtfehler eines berechneten Ergebnisses in seine zwei grundverschiedenen Quellen. Diese Zerlegung ist der rote Faden des ganzen Kapitels: Sie führt direkt auf die Begriffe *Kondition* (Abschnitt 4.2) und *Stabilität* (Abschnitt 4.3).

Bemerkung 4.1.1 (Verwendete Vorkenntnisse)

Dieses Kapitel setzt voraus:

- aus Kapitel 2: den Algorithmusbegriff und die Komplexitätsanalyse (Abschnitte 2.1–2.3) sowie das Verständnis, dass Algorithmen meist nur *approximative* Lösungen berechnen,
- aus der linearen Algebra: Vektor- und Matrixnormen (Kapitel 3, insbesondere Abschnitt 3.2) mit den Normeigenschaften Definitheit, Homogenität und Dreiecksungleichung, außerdem Matrixoperationen und Matrix-Vektor-Produkte,
- aus der Analysis: Grenzwerte und Konvergenz, Stetigkeit und die Grundlagen der Taylor-Approximation.

4.1.1 Absolute und relative Fehler

Beginnen wir mit dem einfachsten Fall: Eine einzelne Zahl soll gespeichert oder berechnet werden, und statt des **tatsächlichen Werts** erhalten wir einen **fehlerhaften Wert**. Die naheliegendste Art, den Fehler zu quantifizieren, ist die Differenz der beiden:

$$\text{absoluter Fehler} = \text{fehlerhafter Wert} - \text{tatsächlicher Wert}.$$

Der absolute Fehler hat aber einen Schönheitsfehler: Er hängt von der Skala ab. Ein Fehler von einem Meter ist beim Vermessen eines Zimmers eine Katastrophe, beim Abstand zwischen München und Hamburg belanglos. Deshalb ist es üblicher, Fehler *relativ* zum tatsächlichen Wert zu betrachten:

$$\text{relativer Fehler} = \frac{\text{fehlerhafter Wert} - \text{tatsächlicher Wert}}{\text{tatsächlicher Wert}}.$$

Der Vorteil: Der relative Fehler ist einheitenfrei und lässt sich als Prozentzahl lesen. Der Nachteil: Der tatsächliche Wert darf nicht 0 sein. Lösen wir die Definition nach dem fehlerhaften Wert auf, erhalten wir eine Umformung, die uns noch oft begegnen wird:

$$\text{fehlerhafter Wert} = \text{tatsächlicher Wert} \cdot (1 + \text{relativer Fehler}).$$

Ein fehlerbehafteter Wert ist also der wahre Wert, multiplikativ verzerrt um den Faktor $1 + \text{relativer Fehler}$. So lässt sich auch das aus Kapitel 2 bekannte Runden im Gleitkommaformat beschreiben: Dort ist der relative Fehler höchstens die Maschinengenauigkeit.

4.1.2 Fehlermaße und Fehlerschranken

In der Statistik sind unsere Ergebnisse selten einzelne Zahlen: Wir rechnen mit Vektoren von Schätzern, mit Kovarianzmatrizen, mit ganzen Funktionen. Um Fehler auch dort messen zu können, brauchen wir statt des Betrags eine Norm. Genau dafür haben wir in Kapitel 3 die Normbegriffe bereitgestellt; jetzt zählt sich das aus. Die folgende Definition überträgt beide Fehlerbegriffe wörtlich auf beliebige Vektorräume:

Definition 4.1.2 (Fehlermaß)

Sei V ein Vektorraum mit (Semi-)Norm $\|\cdot\|$ und $\tilde{\mathbf{v}} \in V$ eine Approximation zu $\mathbf{v} \in V$. Dann ist

- der *absolute Fehler*

$$\Delta_{\mathbf{v}} = \tilde{\mathbf{v}} - \mathbf{v},$$

- der *relative Fehler*

$$\delta_{\mathbf{v}} = \frac{\|\tilde{\mathbf{v}} - \mathbf{v}\|}{\|\mathbf{v}\|} = \frac{\|\Delta_{\mathbf{v}}\|}{\|\mathbf{v}\|} \quad (\text{sofern } \|\mathbf{v}\| \neq 0).$$

Beachten wir den kleinen, aber wichtigen Unterschied: Der absolute Fehler $\Delta_{\mathbf{v}}$ ist selbst ein Element von V und sagt uns auch, *in welche Richtung* die Approximation danebenliegt. Der relative Fehler $\delta_{\mathbf{v}}$ ist dagegen eine einzelne nichtnegative Zahl. Für Skalare ($V = \mathbb{R}$ mit dem Betrag als Norm) liefert die Definition gerade den Betrag des relativen Fehlers aus Abschnitt 4.1.1.

Oft kennen wir den Fehler nicht exakt, sonst könnten wir ihn ja einfach abziehen und exakt rechnen. Was wir realistischerweise angeben können, ist eine Garantie nach oben: Gilt $\|\Delta_{\mathbf{v}}\| \leq \varepsilon$ bzw. $\delta_{\mathbf{v}} \leq \varepsilon$, so nennen wir ε eine *absolute* bzw. *relative Fehlerschranke*. Das folgende Lemma zeigt, was eine relative Fehlerschranke praktisch wert ist: Sie kontrolliert, wie stark die Norm der Approximation von der Norm des wahren Werts abweichen kann.

Lemma 4.1.3 (Fehlerschranken)

Es gilt

$$\|\mathbf{z}\| (1 - \delta_{\mathbf{z}}) \leq \|\tilde{\mathbf{z}}\| \leq \|\mathbf{z}\| (1 + \delta_{\mathbf{z}}).$$

Beweis.

(1)

$$\delta_z = \frac{\|\tilde{z} - z\|}{\|z\|} \implies \|\tilde{z} - z\| = \delta_z \|z\|$$

Definition des relativen Fehlers δ_z , beide Seiten mit $\|z\| > 0$ multipliziert

(2)

$$\left| \|\tilde{z}\| - \|z\| \right| \leq \|\tilde{z} - z\| = \delta_z \|z\|$$

umgekehrte Dreiecksungleichung: $\|a\| - \|b\| \leq \|a - b\|$ für alle $a, b \in V$

(3)

$$-\delta_z \|z\| \leq \|\tilde{z}\| - \|z\| \leq \delta_z \|z\|$$

Definition des Betrags: $|t| \leq c \iff -c \leq t \leq c$

(4)

$$\|z\| - \delta_z \|z\| \leq \|\tilde{z}\| \leq \|z\| + \delta_z \|z\|$$

überall $\|z\|$ addiert; Ausklammern liefert die Behauptung

□

Ein relativer Fehler von zum Beispiel $\delta_z = 0,01$ garantiert also: Die Norm der Approximation liegt zwischen 99 % und 101 % der wahren Norm. Rechnen wir das Ganze einmal konkret an einem Vektor durch.

Beispiel 4.1.4 (Fehlermaße für Vektoren)

Sei $\mathbf{v} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ und $\tilde{\mathbf{v}} = \begin{pmatrix} 3,2 \\ 4,3 \end{pmatrix}$ mit der euklidischen Norm $\|\cdot\|_2$.

Absoluter Fehler, komponentenweise Differenz:

$$\Delta_{\mathbf{v}} = \tilde{\mathbf{v}} - \mathbf{v} = \begin{pmatrix} 3,2 \\ 4,3 \end{pmatrix} - \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0,2 \\ 0,3 \end{pmatrix}.$$

Norm des absoluten Fehlers:

$$\|\Delta_{\mathbf{v}}\|_2 = \sqrt{0,2^2 + 0,3^2} = \sqrt{0,04 + 0,09} = \sqrt{0,13} \approx 0,361.$$

Relativer Fehler, wegen $\|\mathbf{v}\|_2 = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$:

$$\delta_{\mathbf{v}} = \frac{\|\Delta_{\mathbf{v}}\|_2}{\|\mathbf{v}\|_2} = \frac{\sqrt{0,13}}{5} \approx \frac{0,361}{5} \approx 0,072 = 7,2\%.$$

Fehlerschranke: Wegen $\delta_{\mathbf{v}} \approx 0,072 \leq 0,1$ ist $\varepsilon = 0,1$ eine relative Fehlerschranke.

Zur Probe können wir auch Lemma 4.1.3 nachrechnen: Es ist $\|\tilde{\mathbf{v}}\|_2 = \sqrt{3,2^2 + 4,3^2} = \sqrt{28,73} \approx 5,3600$, und tatsächlich gilt $5 \cdot (1 - \delta_{\mathbf{v}}) \approx 4,6394 \leq 5,3600 \leq 5,3606 \approx 5 \cdot (1 + \delta_{\mathbf{v}})$.

Interaktiv: Fehlermaß-Rechner: $\mathbf{v}$ und $\tilde{\mathbf{v}}$ in der Ebene

Für den Vektor aus Beispiel 4.1.4 bedeutet $\delta_{\mathbf{v}} \leq 0,1$ nichts anderes als: Die Spitze von $\tilde{\mathbf{v}}$ darf höchstens um $0,1 \cdot \|\mathbf{v}\|_2$ danebenliegen. Wie groß ist dieser Spielraum in der Ebene, und wie viel Information über $\|\tilde{\mathbf{v}}\|$ steckt am Ende in Lemma 4.1.3?

Interaktives Element — nur in der Web-Fassung

Der Rechner übersetzt beide Fehlermaße in Geometrie: Der absolute Fehler Δ_v ist der rote Pfeil zwischen den beiden Spitzen, der relative Fehler misst dessen Länge an $\|\tilde{v}\|_2$. Der gestrichelte Kreis um $\tilde{v}$ hat den Radius $0,1 \cdot 5 = 0,5$: Solange die blaue Spitze darin liegt, ist $\varepsilon = 0,1$ eine gültige Fehlerschranke. Der orange Ring um den Ursprung ist Lemma 4.1.3 als Bild – bei den 7,2 % aus Beispiel 4.1.4 noch schmal, bei 40 % lässt er für $\|\tilde{v}\|_2$ fast alles zu.

4.1.3 Fehlerzerlegung

Jetzt zur zweiten Leitfrage: *Woher* kommt der Fehler in einem berechneten Ergebnis? Formalisieren wir dazu die Situation. Sei f ein Problem (eine Funktion, die wir auswerten wollen) mit idealem Input x . Was der Computer stattdessen ausführt, ist ein Algorithmus $\tilde{f}$ (eine Näherung an f), und zwar am tatsächlichen Input $\tilde{x}$, einer Näherung an x :

$$\text{Input } \tilde{x} \longrightarrow \text{Algorithmus } \tilde{f} \longrightarrow \text{Ergebnis } \tilde{f}(\tilde{x}).$$

Weicht das Ergebnis $\tilde{f}(\tilde{x})$ vom idealen Wert $f(x)$ ab, kann das an zwei ganz verschiedenen Stellen liegen:

- am *Fehler im Input*, also $\tilde{x} \neq x$, etwa durch Messen oder Speichern von x . Diese Fehler sind *unvermeidbar*: Sie entstehen, bevor unser Programm die erste Zeile ausführt.
- am *Fehler im Algorithmus*, also $\tilde{f} \neq f$, etwa durch Rundung in Maschinenarithmetik oder durch bewusste Approximation (abgebrochene Reihen, Diskretisierung, endlich viele Iterationen). Diese Fehler *können wir beeinflussen*: durch die Wahl eines besseren Algorithmus.

Der zentrale Trick dieses Kapitels ist nun, den Gesamtfehler exakt in diese zwei Anteile zu zerlegen. Dazu subtrahieren und addieren wir den Zwischenwert $f(\tilde{x})$ (das ideale Problem, ausgewertet am tatsächlichen Input):

$$\underbrace{\tilde{f}(\tilde{x}) - f(x)}_{\text{Gesamtfehler}} = \underbrace{\tilde{f}(\tilde{x}) - f(\tilde{x})}_{\text{Fehler im Algorithmus (Maschinenarithmetik)}} + \underbrace{f(\tilde{x}) - f(x)}_{\text{Folgefehler aus Input (exakte Arithmetik)}} \quad (4.1.1)$$

Warum ist diese Zerlegung so nützlich? Weil die beiden Terme sauber getrennte Verantwortlichkeiten haben:

Bemerkung 4.1.5 (Stabilität und Kondition: Ausblick)

- Der erste Term $\tilde{f}(\tilde{x}) - f(\tilde{x})$ vergleicht Algorithmus und Problem *am selben Input* und hängt damit nicht vom Inputfehler ab. Ist er „klein“, nennen wir den Algorithmus *stabil* (Abschnitt 4.3).
- Der zweite Term $f(\tilde{x}) - f(x)$ vergleicht das ideale Problem f an zwei Inputs, in exakter Arithmetik, und hängt damit nicht vom Algorithmus $\tilde{f}$ ab. Ist er „klein“, nennen wir das Problem f an der Stelle x *gut konditioniert* (Abschnitt 4.2).

Die Zerlegung (4.1.1) trennt also, was der Algorithmus verbockt, von dem, was am gestörten Input liegt – und nur den ersten Anteil können wir durch bessere Software reparieren. Sehen wir uns das an einem vollständig durchgerechneten Beispiel an.

Beispiel 4.1.6 (Fehlerzerlegung: Berechnung von e^π)

Bekanntlich lässt sich die Exponentialfunktion als Potenzreihe schreiben: $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$. Angenommen, wir wollen $f(\pi) = e^\pi$ berechnen und verwenden dazu zwei Näherungen: als Input $\tilde{\pi} = 3$ (auf eine ganze Zahl gerundet) und als Algorithmus die nach dem quadratischen Glied abgebrochene Reihe $\tilde{f}(x) = \sum_{n=0}^2 \frac{x^n}{n!} = 1 + x + \frac{x^2}{2}$. Zerlegen wir den Gesamtfehler wie in (4.1.1). Am besten erst selbst probieren, dann weiterlesen. Der Algorithmus liefert $\tilde{f}(3) = 1 + 3 + \frac{9}{2} = 8,5$. Einsetzen des Zwischenwerts $f(3) = e^3$ ergibt die Zerlegung

$$\tilde{f}(3) - f(\pi) = [\tilde{f}(3) - f(3)] + [f(3) - f(\pi)].$$

Fehler im Algorithmus, also die abgebrochene Reihe gegen e^3 ; der Fehler ist gerade der weggelassene Reihenrest, mit Minuszeichen:

$$\tilde{f}(3) - f(3) = \left(1 + 3 + \frac{9}{2}\right) - e^3 = - \sum_{n=3}^{\infty} \frac{3^n}{n!} \approx 8,5 - 20,086 = -11,586.$$

Folgefehler aus dem Input, das exakte Problem f an den zwei Inputs 3 und π :

$$f(3) - f(\pi) = e^3 - e^\pi \approx 20,086 - 23,141 = -3,055.$$

Gesamtfehler, zur Probe direkt und über die Zerlegung:

$$\tilde{f}(3) - f(\pi) \approx 8,5 - 23,141 = -14,641 = (-11,586) + (-3,055). \quad \square$$

Beide Anteile sind hier negativ, und das aus jeweils gutem Grund: Die abgebrochene Reihe lässt nur positive Terme weg und unterschätzt e^3 deshalb systematisch; und wegen $3 < \pi$ und der Monotonie von e^x ist auch $e^3 < e^\pi$. Der Algorithmusfehler dominiert klar ($-11,6$ gegenüber $-3,1$). Hier würde sich also zuerst ein besserer Algorithmus lohnen (mehr Reihenglieder), nicht ein genauerer Input.

Bleibt das so? Den Algorithmusfehler drücken wir mit mehr Reihengliedern, den Folgefehler nur mit einem genaueren $\tilde{\pi}$.

Interaktiv: Fehlerzerlegungs-Explorer: Abbruchordnung N und Input-Genauigkeit k

Ab welcher Abbruchordnung ist der Algorithmus so gut, dass nicht mehr er, sondern der Input das Ergebnis begrenzt?

Interaktives Element – nur in der Web-Fassung

Bei $\tilde{\pi} = 3$ kippt die Dominanz zwischen $N = 4$ und $N = 6$: Bis $N = 3$ ist der Algorithmusfehler mehr als doppelt so groß wie der Folgefehler, ab $N = 6$ ist es umgekehrt. Ab da kostet jedes weitere Reihenglied Rechenzeit, ohne den Gesamtfehler zu bewegen, denn der zweite Summand in (4.1.1) enthält N überhaupt nicht. Wer den Fehler dann noch drücken will, muss am Input ansetzen.

Damit haben wir das Vokabular beisammen: Fehler messen wir mit Normen (absolut oder relativ), und den Gesamtfehler eines berechneten Ergebnisses zerlegen wir in den Algorithmus-Anteil und den Folgefehler aus dem Input. Die nächsten beiden Abschnitte quantifizieren die zwei Anteile: Die *Kondition* eines Problems misst, wie stark Inputfehler im Ergebnis verstärkt werden; die *Stabilität* beurteilt, wie viel Fehler der Algorithmus selbst hinzufügt.

Selbsttest

1. Ein absoluter Fehler von 0,36 ist ein kleiner Fehler.

Antwort: Falsch

Ohne Bezugsgröße sagt der absolute Fehler nichts. In Beispiel 4.1.4 sind 0,36 gemessen an $\|\mathbf{v}\|_2 = 5$ gerade 7,2 %, bei einem Vektor der Länge 0,5 wären dieselben 0,36 ein relativer Fehler von über 70 %. Genau deshalb steht neben dem absoluten Fehler immer auch der relative.

2. Der absolute Fehler $\Delta_{\mathbf{v}}$ trägt mehr Information als der relative Fehler $\delta_{\mathbf{v}}$.

Antwort: Wahr

$\Delta_{\mathbf{v}}$ ist ein Element von V und sagt auch, in welche *Richtung* die Näherung danebenliegt; $\delta_{\mathbf{v}}$ ist eine einzelne nichtnegative Zahl (Definition 4.1.2). Aus $\Delta_{\mathbf{v}}$ und $\mathbf{v}$ lässt sich $\delta_{\mathbf{v}}$ berechnen, umgekehrt nicht.

3. Mehr Reihenglieder in $\tilde{f}$ verkleinern beide Anteile der Zerlegung (4.1.1).

Antwort: Falsch

Nur den ersten. Der Folgefehler $f(\tilde{\mathbf{x}}) - f(\mathbf{x})$ vergleicht das *ideale* Problem an zwei Inputs und weiß von $\tilde{f}$ gar nichts (Bemerkung 4.1.5). Deshalb läuft der Gesamtfehler im Explorer gegen den Folgefehler und nicht gegen null.

4. Ziehen wir im Fehlermaß-Rechner die blaue Spitze bei $\mathbf{v} = (3, 4)^T$ genau auf den gestrichelten Kreis. Welchen Wert zeigt der Rechner dann für $\|\Delta_{\mathbf{v}}\|_2$?

Lösung: 0.5

Der gestrichelte Kreis hat den Radius $0,1 \cdot \|\mathbf{v}\|_2$, und $\|\mathbf{v}\|_2 = 5$. Auf dem Kreis ist also $\|\Delta_{\mathbf{v}}\|_2 = 0,5$ und damit $\delta_{\mathbf{v}} = 0,5/5 = 10\%$: der Grenzfall der Fehlerschranke $\varepsilon = 0,1$ aus Beispiel 4.1.4.

Vertiefung: Heath §1.2.

4.2 Kondition

In Abschnitt 4.1 haben wir den Gesamtfehler einer numerischen Rechnung in zwei Teile zerlegt: den Fehler des Algorithmus selbst und den Folgefehler, der aus einem fehlerhaften Input entsteht. Um den zweiten Teil geht es jetzt. Inputfehler betrachten wir als *unvermeidbar*: Messwerte sind ungenau, Daten wurden gerundet gespeichert, und schon das Ablegen einer reellen Zahl als Gleitkommazahl erzeugt einen relativen Rundungsfehler von der Größenordnung der Maschinengenauigkeit.

Die entscheidende Beobachtung: Wie stark sich solche Inputfehler im Ergebnis niederschlagen, ist eine Eigenschaft des Problems f , noch bevor irgendein Algorithmus ins Spiel kommt. Stellen wir uns die Menge aller Inputs vor, die wir von $\mathbf{x}$ nicht unterscheiden können, weil ihr relativer Fehler unter einer Toleranz δ liegt:

$$E = \{\tilde{x} : \|\tilde{x} - x\| / \|x\| \leq \delta\}.$$

Jeder Punkt in E ist ein plausibler Input, also ist jeder Punkt der Bildmenge $R = f(E)$ ein plausibles Ergebnis. Ist R groß im Vergleich zu E , bläst f kleine Eingabestörungen zu großen Ergebnisänderungen auf. Wir nennen das Problem dann *schlecht konditioniert*. Vorsicht: Kein noch so guter Algorithmus kann das reparieren, denn er sieht nur $\tilde{x}$ und kann nicht wissen, welcher Punkt von E der „wahre“ Input war.

4.2.1 Ein Warnbeispiel: der Kehrwert

Wie schlimm kann es werden? Ein bewusst extremes Beispiel:

Beispiel 4.2.1 (Kehrwert nahe null)

Sei $f(x) = x^{-1}$ mit Input $x = 10^{-17}$, und sei der gestörte Input $\tilde{x} = x + \varepsilon$ mit $\varepsilon \in (-10^{-17}, 10^{-17})$. Dann durchläuft $\tilde{x}$ das Intervall $(0, 2 \cdot 10^{-17})$ und damit

$$f(\tilde{x}) = \frac{1}{\tilde{x}} \in \left(\frac{1}{2} \cdot 10^{17}, \infty\right).$$

Der Output kann also um Größenordnungen vom wahren Wert $f(x) = 10^{17}$ abweichen. Rechnen wir den relativen Outputfehler explizit aus. Die Störung ε verfolgen wir in Rot:

$$f(\tilde{x}) - f(x) = \frac{1}{x + \varepsilon} - \frac{1}{x} = \frac{x - (x + \varepsilon)}{(x + \varepsilon)x} = \frac{-\varepsilon}{(x + \varepsilon)x},$$

und nach Division durch $f(x) = 1/x$:

$$\frac{|f(\tilde{x}) - f(x)|}{|f(x)|} = \frac{|\varepsilon|}{|x + \varepsilon|} = \frac{|x|}{|x + \varepsilon|} \cdot \frac{|\varepsilon|}{|x|}.$$

Der *relative Outputfehler* ist also der *relative Inputfehler*, multipliziert mit dem *Verstärkungsfaktor* $|x|/|x + \varepsilon|$. Für $\varepsilon \rightarrow -10^{-17}$ wächst dieser Faktor über alle Grenzen; insgesamt durchläuft der relative Outputfehler das gesamte Intervall $[0, \infty)$. Und das, obwohl die Störung absolut winziger ist als jede Messgenauigkeit der Welt: $|\varepsilon| < 10^{-17}$. Relativ zu x kann sie allerdings bis zu 100 % betragen. Diesen Unterschied schauen wir uns gleich noch genauer an.

Interaktiv: Kondition-Spielwiese: der Kehrwert unter Störungen

Wie schnell wächst dieser Verstärkungsfaktor eigentlich? Die Frage lässt sich schätzen, bevor wir sie ausrechnen: Steht der Input bei $x = 0,6$, wie weit müssen wir ihn dann stören, damit der relative Outputfehler zehnmal so groß wird wie der relative Inputfehler?

Schätzfrage. Wie groß muss die Störung ε bei $x = 0,6$ sein, damit der relative Outputfehler zehnmal so groß ist wie der relative Inputfehler?

Lösung: $-0,54 \varepsilon$

Die Verstärkung ist $|x|/|x + \varepsilon|$. Zehnfach heißt $|x + \varepsilon| = x/10 = 0,06$, also $\varepsilon = -0,54$: 90 % der Strecke von x zur Polstelle.

Das Widget zeigt beide Regime an einem Bild. Solange ε klein gegenüber x ist, sind roter und grüner Balken fast gleich lang, die Verstärkung liegt nahe 1. Schieben wir $\tilde{x}$ dagegen an die Polstelle, wächst der grüne Outputbalken über den Bildrand hinaus, während der rote Inputbalken kaum länger wird. Bei $x = 0,6$ und $\varepsilon = -0,54$ ist die Verstärkung genau 10, bei

$\varepsilon = -0,45$ (der Voreinstellung) ist sie 4.

4.2.2 Konditionszahlen

Um „sensitiv gegenüber Inputfehlern“ quantitativ zu fassen, setzen wir Output- und Inputfehler direkt ins Verhältnis, einmal für absolute, einmal für relative Fehler. Im Folgenden betrachten wir Abbildungen $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$; $\|\cdot\|$ bezeichnet eine beliebige Vektornorm bzw. die von ihr induzierte Operatornorm für Matrizen (Abschnitt 3.3).

Definition 4.2.2 (Konditionszahl)

Sei f ein Problem mit Input $\mathbf{x}$. Für die relative Konditionszahl setzen wir $\|\mathbf{x}\| \neq 0$ und $\|f(\mathbf{x})\| \neq 0$ voraus.

- Die *absolute Konditionszahl* ist

$$\kappa_{abs}(f, \mathbf{x}) = \limsup_{\substack{\mathbf{h} \rightarrow 0 \\ \mathbf{h} \neq 0}} \frac{\|f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})\|}{\|\mathbf{h}\|}.$$

- Die *relative Konditionszahl* ist

$$\kappa_{rel}(f, \mathbf{x}) = \limsup_{\substack{\mathbf{h} \rightarrow 0 \\ \mathbf{h} \neq 0}} \frac{\|f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})\| / \|f(\mathbf{x})\|}{\|\mathbf{h}\| / \|\mathbf{x}\|}.$$

Bemerkung 4.2.3 (Wie lesen wir das?)

- Der Limes superior betrachtet alle Richtungen, aus denen kleine Störungen gegen null gehen. κ ist also der größte asymptotische Verstärkungsfaktor *im Grenzfall* verschwindender Störungen. Konditionszahlen sind lokale Größen, und sie hängen sowohl vom Problem f als auch von der konkreten Stelle $\mathbf{x}$ ab.
- Für differenzierbares $f : \mathbb{R} \rightarrow \mathbb{R}$ können wir die Konditionszahlen direkt ausrechnen: Wegen $f(\tilde{x}) - f(x) \approx f'(x)(\tilde{x} - x)$ (Taylor-Näherung erster Ordnung) gilt

$$\kappa_{abs} = |f'(x)| \quad \text{und} \quad \kappa_{rel} = \frac{|f'(x)| |x|}{|f(x)|}.$$

- Meist ist κ_{rel} die relevantere Größe: Gleitkommarechnung erzeugt naturgemäß *relative* Fehler (Abschnitt 4.1), und relative Fehler sind unabhängig von Maßeinheiten und Skalierung.

Die Konditionszahl sortiert Probleme in gutartige und böartige, mit einem wichtigen Extremfall:

Bemerkung 4.2.4 (Interpretation)

- *gut konditioniert* ($\kappa \lesssim 1$): Ein Inputfehler führt zu einem ähnlich großen oder kleineren Outputfehler.
- *schlecht konditioniert* ($\kappa \gg 1$): Ungünstig gerichtete kleine Inputfehler können zu viel größeren Outputfehlern führen. Als Faustregel bei $\kappa_{rel} \approx 10^k$: Wir verlieren bis zu k

signifikante Dezimalstellen gegenüber der Genauigkeit des Inputs.

- *schlecht gestellt* (engl. *ill-posed*, $\kappa = \infty$): Bei Inputfehlern ist das Problem praktisch nicht lösbar: Beliebig kleine Störungen können das Ergebnis beliebig stark verfälschen.

Mit den Formeln aus Bemerkung 4.2.3 können wir nun auch das Rätsel aus dem Warnbeispiel auflösen:

Beispiel 4.2.5 (Der Kehrwert, aufgelöst)

Für $f(x) = x^{-1}$ ist $f'(x) = -x^{-2}$, also an der Stelle $x = 10^{-17}$:

$$\kappa_{abs} = |f'(x)| = \frac{1}{x^2} = 10^{34}, \quad \kappa_{rel} = \frac{|f'(x)| |x|}{|f(x)|} = \frac{x^{-2} \cdot x}{x^{-1}} = 1.$$

Absolut gemessen ist das Problem also katastrophal sensitiv ($\kappa_{abs} = 10^{34}$), relativ gemessen dagegen völlig harmlos: Asymptotisch wird ein relativer Inputfehler von 0,1 % zu einem relativen Outputfehler von ebenfalls 0,1 %. Das Drama in Beispiel 4.2.1 entsteht, weil die Störungen dort *relativ zu x* eben nicht klein waren (bis zu 100 %): Ein absoluter Fehler fester Größe (etwa ein additiver Messfehler) bedeutet bei winzigem x eine riesige relative Störung. Merken wir uns: *Welche* Konditionszahl zählt, hängt davon ab, welche Fehlerart im Input unvermeidbar ist. Im Widget oben sehen wir beide Regime: Für kleine ε/x liegt die Verstärkung nahe 1, erst wenn ε in die Größenordnung von x kommt, explodiert sie.

4.2.3 Kondition eines linearen Gleichungssystems

Nun zum wichtigsten Beispiel dieses Kapitels. Wir lösen ein lineares Gleichungssystem $\mathbf{A}\mathbf{y} = \mathbf{x}$ mit invertierbarem $\mathbf{A} \in \mathbb{R}^{n \times n}$, wobei die rechte Seite $\mathbf{x}$ der fehlerbehaftete Input ist. Als Problem geschrieben: $f(\mathbf{x}) = \mathbf{A}^{-1}\mathbf{x} = \mathbf{y}$, wobei die Inverse hier nur die Lösungsabbildung bezeichnet, nicht etwa eine Empfehlung, sie auszurechnen. Wie sensitiv reagiert die Lösung $\mathbf{y}$ auf Störungen von $\mathbf{x}$? Die Antwort führt genau auf die *Konditionszahl der Matrix* $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$, die wir in Abschnitt 3.5 kennengelernt haben.

Satz 4.2.6 (Kondition der Lösung eines LGS)

Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$ invertierbar und $f(\mathbf{x}) = \mathbf{A}^{-1}\mathbf{x}$. Bezüglich einer beliebigen Vektornorm und der von ihr induzierten Operatornorm gilt dann

$$\kappa_{abs} = \|\mathbf{A}^{-1}\|, \quad \kappa_{rel} = \|\mathbf{A}^{-1}\| \frac{\|\mathbf{x}\|}{\|\mathbf{A}^{-1}\mathbf{x}\|} \leq \|\mathbf{A}\| \|\mathbf{A}^{-1}\| = \kappa(\mathbf{A}).$$

Beweis.

(1)

$$f(\tilde{\mathbf{x}}) - f(\mathbf{x}) = \mathbf{A}^{-1}\tilde{\mathbf{x}} - \mathbf{A}^{-1}\mathbf{x} = \mathbf{A}^{-1}\mathbf{h}$$

f ist linear; wir kürzen die Störung als $\mathbf{h} = \tilde{\mathbf{x}} - \mathbf{x}$ ab

(2)

$$\|f(\tilde{\mathbf{x}}) - f(\mathbf{x})\| = \|\mathbf{A}^{-1}\mathbf{h}\| \leq \|\mathbf{A}^{-1}\| \|\mathbf{h}\| \implies \kappa_{abs} \leq \|\mathbf{A}^{-1}\|$$

definierende Eigenschaft der Operatornorm: $\|\mathbf{A}^{-1}\mathbf{v}\| \leq \|\mathbf{A}^{-1}\| \|\mathbf{v}\|$ für alle $\mathbf{v}$ (Abschnitt 3.3)

- (3) Keine kleinere Konstante genügt: Wählen wir $\mathbf{h}$ (beliebig klein) entlang einer maximierenden Richtung, gilt in der Ungleichung Gleichheit. Also ist $\kappa_{abs} = \|\mathbf{A}^{-1}\|$.

die Operatornorm ist ein Maximum: $\|\mathbf{A}^{-1}\| = \max_{\mathbf{v} \neq 0} \|\mathbf{A}^{-1}\mathbf{v}\| / \|\mathbf{v}\|$, und eine maximierende Richtung dürfen wir beliebig klein skalieren

(4)

$$\frac{\|\mathbf{A}^{-1}\mathbf{h}\| / \|\mathbf{A}^{-1}\mathbf{x}\|}{\|\mathbf{h}\| / \|\mathbf{x}\|} = \frac{\|\mathbf{A}^{-1}\mathbf{h}\|}{\underbrace{\|\mathbf{h}\|}_{\leq \|\mathbf{A}^{-1}\|}} \cdot \frac{\|\mathbf{x}\|}{\|\mathbf{A}^{-1}\mathbf{x}\|} \implies \kappa_{rel} = \|\mathbf{A}^{-1}\| \frac{\|\mathbf{x}\|}{\|\mathbf{A}^{-1}\mathbf{x}\|}$$

den Quotienten aus relativem Output- und Inputfehler umsortieren; dann liefern Schritt 2 und 3 wieder Schranke und Gleichheitsfall für den ersten Faktor

(5)

$$\kappa_{rel} = \|\mathbf{A}^{-1}\| \frac{\|\mathbf{A}\mathbf{y}\|}{\|\mathbf{y}\|} \leq \|\mathbf{A}\| \|\mathbf{A}^{-1}\| = \kappa(\mathbf{A})$$

substituiere $\mathbf{y} = \mathbf{A}^{-1}\mathbf{x}$, also $\mathbf{x} = \mathbf{A}\mathbf{y}$; Operatornorm-Schranke $\|\mathbf{A}\mathbf{y}\| \leq \|\mathbf{A}\| \|\mathbf{y}\|$

□

Bemerkung 4.2.7 (Konditionszahl einer Matrix)

- Die relative Kondition des Gleichungslösens hängt von der rechten Seite $\mathbf{x}$ ab; $\kappa(\mathbf{A}) = \|\mathbf{A}\| \|\mathbf{A}^{-1}\|$ ist die von $\mathbf{x}$ unabhängige Worst-Case-Schranke – und sie wird für ungünstige $\mathbf{x}$ auch angenommen. Das erklärt im Nachhinein, warum genau dieses Produkt in Abschnitt 3.5 den Namen *Konditionszahl von A* bekommen hat.
- Für die von $\|\cdot\|_2$ induzierte Operatornorm (*Spektralnorm*) gilt

$$\kappa_2(\mathbf{A}) = \frac{\sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}}{\sqrt{\lambda_{\min}(\mathbf{A}^\top \mathbf{A})}} = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})},$$

das Verhältnis des größten zum kleinsten Singulärwert (bzw. der Wurzeln der extremen Eigenwerte von $\mathbf{A}^\top \mathbf{A}$).

Satz 4.2.6 hat zwei Aussagen, und die zweite ist die überraschende: Die relative Kondition hängt von der rechten Seite $\mathbf{x}$ ab, aber die Schranke $\kappa(\mathbf{A})$ wird für ungünstige $\mathbf{x}$ auch wirklich angenommen.

Interaktiv: Kondition eines Gleichungssystems: aus dem Kreis wird eine Ellipse

Sehen wir uns an, was das geometrisch heißt. Ein Kreis plausibler rechter Seiten um $\mathbf{x}$ wird von $\mathbf{A}^{-1}$ zu einer Ellipse verzogen; für welche $\mathbf{x}$ ist diese Ellipse groß gemessen an $\|\mathbf{A}^{-1}\mathbf{x}\|$, und für welche nicht?

Interaktives Element – nur in der Web-Fassung

Beide Tafeln zeigen dieselbe Störung, einmal vor und einmal nach dem Lösen. In der Voreinstellung $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1,05 \end{pmatrix}$, $\mathbf{x} = (2; 2,05)^\top$, also $\mathbf{y} = (1; 1)^\top$ schöpft $\kappa_{rel}(f, \mathbf{x}) \approx 82,0$ die Schranke $\kappa(\mathbf{A}) \approx 82,0$ zu 99,99 % aus: Aus 5 % Inputfehler werden 410 % Outputfehler, aus

dem kleinen roten Kreis links wird die lange Nadel rechts. Ziehen wir $\mathbf{x}$ dagegen in Richtung $(1; -1)^\top$, fällt κ_{rel} auf 1, obwohl die Matrix dieselbe bleibt.

Für eine Diagonalmatrix ist $\mathbf{A}^\top \mathbf{A} = \text{diag}(4, 25, 0,01)$, die Singulärwerte sind also die Beträge der Diagonaleinträge: $\sigma \in \{2, 5, 0,1\}$. Damit

$$\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}}{\sigma_{\min}} = \frac{5}{0,1} = 50.$$

Vorsicht vor der Distraktor-Antwort $20 = 2/0,1$: Der größte Singulärwert ist 5, nicht der erste Diagonaleintrag 2. Die Werte 2 und 5 wären nur Normen, keine Konditionszahlen; κ_2 misst das *Verhältnis* von stärkster zu schwächster Streckung.

4.2.4 Aufgabe: die Kondition einer Summe

Zum Abschluss eine Aufgabe. Versuchen wir es erst selbst, bevor wir die Lösung aufklappen. Sie sieht harmlos aus, trägt aber den Keim eines Phänomens in sich, das wir schon kennen.

Beispiel 4.2.8 (Aufgabe: Kondition der Summe)

Sei $f(\mathbf{x}) = x_1 + x_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}^\top \mathbf{x} = \mathbf{1}^\top \mathbf{x}$ mit $\mathbf{x} \in \mathbb{R}^2$ und der euklidischen Norm $\|\cdot\|_2$.

1. Was sind die Konditionszahlen?
2. Für welche $\mathbf{x}$ ist das Problem schlecht konditioniert?

Lösung

1. Für die Störung $\mathbf{h} = \tilde{\mathbf{x}} - \mathbf{x}$ liefert die Cauchy-Schwarz-Ungleichung

$$|f(\tilde{\mathbf{x}}) - f(\mathbf{x})| = |\mathbf{1}^\top \mathbf{h}| \leq \|\mathbf{1}\|_2 \|\mathbf{h}\|_2 = \sqrt{2} \|\mathbf{h}\|_2,$$

mit Gleichheit genau dann, wenn $\mathbf{h}$ parallel zu $\mathbf{1} = (1, 1)^\top$ liegt. Solche beliebig kleinen Störungen gibt es, also ist die Schranke scharf: Der Gleichheitsfall von Cauchy-Schwarz liefert hier nicht nur $\kappa_{abs} \leq \sqrt{2}$, sondern $\kappa_{abs} = \sqrt{2}$. Da f linear ist, tritt derselbe Gleichheitsfall auch beim relativen Fehlerquotienten auf, und wie im Beweis von Satz 4.2.6 folgt

$$\kappa_{rel} = \kappa_{abs} \frac{\|\mathbf{x}\|_2}{|f(\mathbf{x})|} = \frac{\sqrt{2} \|\mathbf{x}\|_2}{|x_1 + x_2|}.$$

2. Schlecht konditioniert ist das Problem, wenn $\kappa_{rel} \gg 1$, also wenn $|x_1 + x_2| \approx 0$, während $\|\mathbf{x}\|_2$ nicht klein ist, sprich für $x_1 \approx -x_2$, nahe der Antidiagonalen. Dort heben sich die beiden Summanden fast auf: Das ist exakt die Auslöschung aus Abschnitt 2.1, jetzt als Konditionsaussage über das *Problem* „addiere zwei Zahlen“, unabhängig davon, wie wir die Summe ausrechnen. Auf der Diagonalen $x_1 = x_2$ gilt dagegen $\kappa_{rel} = 1$: besser geht es nicht.

Interaktiv: Kondition-Spielwiese: die Summe zweier Zahlen in der Ebene

Die Antwort auf Teil 2 lässt sich auch als Landkarte über der Ebene lesen: Für welche Punkte $\mathbf{x}$ ist die Summe gutmütig, für welche katastrophal, und wie schnell wechselt das eine ins andere?

Die Karte macht zwei Dinge sichtbar, die in der Formel stecken, aber leicht zu übersehen sind. Erstens ist sie entlang jedes Strahls durch den Ursprung einfarbig: Da Zähler und Nenner von κ_{rel} beide homogen vom Grad 1 in $\mathbf{x}$ sind, zählt allein die *Richtung*. Zweitens liegt der ganze Ärger in einer beliebig dünnen Umgebung der Antidiagonalen: Bei $\mathbf{x} = (1,5; -1,45)^\top$ ist $\kappa_{rel} \approx 59$, also gut zwei verlorene Dezimalstellen; ein Schritt weiter auf $(1,5; -1,5)^\top$ und das Problem ist schlecht gestellt.

Diese Rechnung ist mehr als eine Fingerübung: In Abschnitt 4.3 wird genau die Kondition der Differenz zweier fast gleicher Zahlen erklären, warum die Verschiebungsformel für die Varianz numerisch scheitert.

Selbsttest

1. Ein schlecht konditioniertes Problem lässt sich durch einen besseren Algorithmus retten.

Antwort: Falsch

Die Kondition ist eine Eigenschaft des Problems f an der Stelle $\mathbf{x}$, nicht der Rechenvorschrift. Ein Algorithmus sieht nur $\tilde{\mathbf{x}}$ und kann nicht wissen, welcher Punkt der ununterscheidbaren Menge E der wahre Input war. Ein *stabiler* Algorithmus liefert trotzdem etwas: nämlich nicht mehr Fehler, als das Problem erzwingt.

2. Die absolute Konditionszahl von $f(x) = 1/x$ ist bei $x = 10^{-17}$ riesig, also ist das Problem dort in jedem Sinn schlecht konditioniert.

Antwort: Falsch

$\kappa_{abs} = 10^{34}$, aber $\kappa_{rel} = 1$ (Beispiel 4.2.5). Welche der beiden zählt, hängt davon ab, welche Fehlerart im Input unvermeidbar ist. Bei Gleitkommarechnung sind das *relative* Fehler, und die gibt der Kehrwert unverändert weiter.

3. Für dieselbe Matrix A kann das Lösen von $A\mathbf{y} = \mathbf{x}$ je nach rechter Seite gut oder schlecht konditioniert sein.

Antwort: Wahr

Nach Satz 4.2.6 ist $\kappa_{rel} = \|A^{-1}\| \|\mathbf{x}\| / \|A^{-1}\mathbf{x}\|$, und dieser Quotient hängt von der Richtung von $\mathbf{x}$ ab. Er schöpft die Schranke $\kappa(A)$ nur für ungünstige rechte Seiten aus; im LGS-Widget sind das die Richtungen nahe 45° .

4. Stellen wir im Widget zur Kondition der Summe die Voreinstellung „Diagonale“ ein. Welchen Wert zeigt der Readout für κ_{rel} ?

Lösung: 1

Eins. Auf der Diagonalen $x_2 = x_1$ ist $\|\mathbf{x}\|_2 = \sqrt{2}|x_1|$ und $|x_1 + x_2| = 2|x_1|$, also $\kappa_{rel} = \sqrt{2} \cdot \sqrt{2}|x_1| / (2|x_1|) = 1$. Besser als 1 geht nicht: Relative Inputfehler werden dort überhaupt nicht verstärkt.

Vertiefung: Heath §1.2.5–1.2.6 (Fehlerfortpflanzung, Sensitivität und Kondition).

4.3 Stabilität von Algorithmen

Erinnern wir uns an die Fehlerzerlegung aus Abschnitt 4.1: Wenn wir ein Problem f mit einem Algorithmus $\tilde{f}$ auf einem fehlerbehafteten Input $\tilde{\mathbf{x}}$ lösen, zerfällt der Gesamtfehler in zwei Teile:

$$\text{Gesamtfehler} = \tilde{f}(\tilde{\mathbf{x}}) - f(\mathbf{x}) = \underbrace{\tilde{f}(\tilde{\mathbf{x}}) - f(\tilde{\mathbf{x}})}_{\text{Fehler im Algorithmus}} + \underbrace{f(\tilde{\mathbf{x}}) - f(\mathbf{x})}_{\text{Folgefehler aus Input}}$$

Den **zweiten Summanden** haben wir in Abschnitt 4.2 untersucht: Er hängt nur vom Problem ab, nicht vom Algorithmus, und die Konditionszahl sagt uns, wie stark er den unvermeidbaren Inputfehler verstärkt. Jetzt nehmen wir uns den **ersten Summanden** vor: den Fehler, den der Algorithmus *selbst* hinzufügt: durch Rundungsfehler in jeder einzelnen Gleitkomma-Operation, durch abgebrochene Reihen, durch geschätzte Zwischengrößen. Dieser Anteil hängt nicht vom Inputfehler ab; er entsteht, selbst wenn der Input exakt ist. Ganz analog zur Kondition geben wir ihm einen Namen: Ist er „klein“, nennen wir den Algorithmus $\tilde{f}$ *stabil*. Was „klein“ heißt, lässt sich präzise fassen.

Vertiefung: Vorwärts- und Rückwärtsstabilität, präzise

Definition 4.3.1 (Vorwärts- und Rückwärtsstabilität)

Sei f ein Problem, $\tilde{f}$ ein Gleitkomma-Algorithmus und $\hat{\mathbf{y}} = \tilde{f}(\mathbf{x})$ sein Ergebnis für einen exakt dargestellten Input $\mathbf{x}$. Für $\|\mathbf{x}\| \neq 0$ und $\|f(\mathbf{x})\| \neq 0$ heißen

$$\eta_{\text{vor}} = \frac{\|\hat{\mathbf{y}} - f(\mathbf{x})\|}{\|f(\mathbf{x})\|}$$

der *relative Vorwärtsfehler* und

$$\eta_{\text{rck}} = \inf \left\{ \frac{\|\Delta \mathbf{x}\|}{\|\mathbf{x}\|} : \hat{\mathbf{y}} = f(\mathbf{x} + \Delta \mathbf{x}) \right\}$$

der *relative Rückwärtsfehler*. Der Algorithmus heißt *rückwärtsstabil*, wenn $\eta_{\text{rck}} = O(u)$ ist, wobei u die Rundungseinheit bezeichnet. Er heißt *vorwärtsstabil*, wenn sein Vorwärtsfehler bis auf moderate Faktoren nicht größer ist als der durch Kondition und Rundung erwartbare Fehler, $\eta_{\text{vor}} = O(\kappa_{\text{rel}}(f, \mathbf{x})u)$.

Die Konstanten in den O -Aussagen dürfen von Dimension und Algorithmus abhängen, sollen aber nicht durch eine vermeidbare numerische Katastrophe riesig werden. Rückwärtsstabilität bedeutet besonders anschaulich: Das berechnete Ergebnis ist die exakte Lösung eines nur geringfügig gestörten Problems. Zusammen mit der Kondition liefert das typischerweise eine Vorwärtsfehlerschranke.

Wichtig ist die Arbeitsteilung: Die *Kondition* ist eine Eigenschaft des *Problems*, die wir nicht wählen können. Die *Stabilität* ist eine Eigenschaft des *Algorithmus*, und den wählen wir sehr wohl. Deshalb lohnt sich dieser Abschnitt: Für dasselbe Problem können zwei mathematisch äquivalente Rechenwege dramatisch unterschiedlich stabil sein.

Eine verwandte Idee aus dem maschinellen Lernen: SGD

Das nächste Beispiel illustriert die *dynamische* Stabilität einer Iteration. Es ist eine nützliche Analogie, aber nicht dasselbe wie die eben definierte numerische Vorwärts- oder Rückwärtsstabilität.

Beispiel 4.3.2 (Stochastic Gradient Descent)

Beim Training von Machine-Learning-Modellen minimieren wir eine Verlustfunktion $L(\theta)$ mit dem Gradientenabstieg in seiner stochastischen Variante (SGD):

$$\theta_{k+1} = \theta_k - \alpha \widehat{\nabla} L(\theta_k),$$

wobei $\widehat{\nabla} L(\theta_k)$ nur eine *Schätzung* des Gradienten ist (berechnet auf einem zufälligen Teil der Daten, in endlicher Gleitkomma-Genauigkeit) und $\alpha > 0$ die *Lernrate* (learning rate).

- Eine relativ zur Krümmung von L hinreichend kleine Lernrate kann Fehler in bestimmten Richtungen dämpfen und unter geeigneten Annahmen Konvergenz ermöglichen.
- Eine zu große Lernrate kann Iterierte oszillieren oder divergieren lassen. Zahlenwerte wie 0,001 oder 1 sind ohne Skalierung und Krümmung des konkreten Problems nicht allgemein als „klein“ oder „groß“ einzuordnen.

Die Lernrate beeinflusst also die Stabilität der Iterationsdynamik. Ob SGD konvergiert und wogegen, hängt zusätzlich von Annahmen an L , an die Gradientenschätzung und an den Lernratenplan ab.

Interaktiv: Die Lernrate als Stabilitätsregler: SGD zum Ausprobieren

Wie scharf ist der Umschlag von Konvergenz zu Divergenz? Machen wir das Modell so klein, dass wir es ganz überblicken: $L(\theta) = \theta^2$ mit exaktem Gradienten 2θ , Start bei $\theta_0 = 2,5$. Ab welcher Lernrate laufen die Iterierten nicht mehr auf das Minimum zu, sondern davon weg?

Interaktives Element — nur in der Web-Fassung

Die Rechnung dahinter passt in eine Zeile. Ohne Rauschen ist $\theta_{k+1} = \theta_k - \alpha \cdot 2\theta_k = (1 - 2\alpha) \theta_k$, der Abstand zum Minimum wird in jedem Schritt also mit $\rho(\alpha) = |1 - 2\alpha|$ multipliziert. Das erklärt alle vier Regime auf einmal: monotone Konvergenz für $\alpha < 0,5$, ein einziger Schritt bei $\alpha = 0,5$, Oszillation mit schrumpfender Amplitude für $0,5 < \alpha < 1$, und ab $\alpha > 1$ Divergenz. Die Schwelle ist keine Eigenschaft von SGD, sondern der Krümmung von L : Für $L(\theta) = \frac{c}{2} \theta^2$ läge sie bei $\alpha = 2/c$.

Zusammengesetzte Algorithmen: die wichtigste Erkenntnis

Praktisch jeder Algorithmus ist eine Kette von Teilschritten. Was passiert mit den Fehlern der frühen Schritte, wenn spätere Schritte darauf aufbauen? Das folgende Resultat verknüpft die Stabilität eines zusammengesetzten Algorithmus mit der *Kondition seiner Teilschritte*. Es ist die wichtigste Erkenntnis dieses Abschnitts. Wir betrachten dazu ein Problem, das als Verkettung zweier Teilprobleme geschrieben ist.

Satz 4.3.3 (Fehlerfortpflanzung in einer Komposition)

Sei $f = h \circ g$, $y = g(x) \neq 0$ und $\tilde{y} = \tilde{g}(x)$. Wir setzen $h(y) \neq 0$ voraus und definieren

$$\eta_g = \frac{\|\tilde{y} - y\|}{\|y\|}, \quad \eta_h = \frac{\|\tilde{h}(\tilde{y}) - h(\tilde{y})\|}{\|h(\tilde{y})\|}.$$

Ist h in y stetig und $\tilde{y} \rightarrow y$, dann gilt in erster Ordnung

$$\frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\mathbf{y})\|}{\|h(\mathbf{y})\|} \leq (1 + o(1))\eta_h + (\kappa_{rel}(h, \mathbf{y}) + o(1))\eta_g.$$

Der erste Term ist der relative algorithmische Fehler des zweiten Schritts am tatsächlich übergebenen Input. Der zweite Term ist der Fehler des ersten Schritts, verstärkt durch die lokale Kondition des zweiten Problems. Die $o(1)$ -Terme machen explizit, dass dies eine asymptotische Aussage für verschwindende Zwischenfehler ist, keine exakte Ungleichung für beliebig große Störungen.

Beweis.

- (1) Durch Addieren und Subtrahieren von $h(\tilde{\mathbf{y}})$ sowie die Dreiecksungleichung erhalten wir exakt

$$\frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\mathbf{y})\|}{\|h(\mathbf{y})\|} \leq \frac{\|\tilde{h}(\tilde{\mathbf{y}}) - h(\tilde{\mathbf{y}})\|}{\|h(\mathbf{y})\|} + \frac{\|h(\tilde{\mathbf{y}}) - h(\mathbf{y})\|}{\|h(\mathbf{y})\|}.$$

- (2) Stetigkeit und $h(\mathbf{y}) \neq \mathbf{0}$ liefern $\|h(\tilde{\mathbf{y}})\|/\|h(\mathbf{y})\| = 1 + o(1)$. Daher ist der erste Summand gleich $(1 + o(1))\eta_h$.
- (3) Nach Definition der relativen Konditionszahl ist der zweite Summand entlang $\tilde{\mathbf{y}} \rightarrow \mathbf{y}$ höchstens

$$(\kappa_{rel}(h, \mathbf{y}) + o(1)) \frac{\|\tilde{\mathbf{y}} - \mathbf{y}\|}{\|\mathbf{y}\|} = (\kappa_{rel}(h, \mathbf{y}) + o(1))\eta_g.$$

□

Bemerkung 4.3.4 (Was folgt daraus – und was nicht?)

Ein schlecht konditionierter später Teilschritt kann Fehler aus früheren Schritten stark verstärken. Daraus folgt aber keine allgemeine Regel, schlecht konditionierte Operationen einfach „nach vorn“ zu verschieben: Eine Umordnung verändert Zwischengrößen, Rundungsfehler und oft auch die relevanten Konditionen. Verlässlichere Strategien sind, das Teilproblem umzuformulieren, große beinahe gleiche Zwischengrößen zu vermeiden und eine rückwärts-stabile Implementierung zu verwenden.

Anwendung: Stabilität der Varianzberechnung

Wenden wir den Satz auf ein Beispiel an, das wir aus Abschnitt 2.1 schon kennen. Dort haben wir *beobachtet*, dass die Verschiebungsformel für die Varianz katastrophal versagen kann. Jetzt können wir *erklären*, warum, und zwar quantitativ.

Beispiel 4.3.5 (Zwei Algorithmen für die Varianz)

Problem: Berechne die Stichprobenvarianz $f(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2$.

Algorithmen: Wir vergleichen die zweistufige Rechnung (erst zentrieren, dann quadrieren) mit der Verschiebungsformel:

$$\tilde{f}_1(\mathbf{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2, \quad \tilde{f}_2(\mathbf{x}) = \left(\frac{1}{n} \sum_{i=1}^n x_i^2 \right) - \bar{x}^2.$$

Mathematisch sind beide identisch. Als Rechenvorschriften sind sie es nicht: $\tilde{f}_2$ ist eine Verkettung $\tilde{f}_2 = \tilde{h} \circ \tilde{g}$ im Sinne von Satz 4.3.3. Der erste Schritt $\tilde{g}$ berechnet das Paar

$$g(\mathbf{x}) = (a, b) \quad \text{mit} \quad a = \frac{1}{n} \sum_{i=1}^n x_i^2 \quad \text{und} \quad b = \bar{x}^2,$$

und der *letzte* Schritt bildet die Differenz $h(a, b) = a - b$.

Nach Satz 4.3.3 müssen wir also die Kondition des letzten Schritts an der Übergabestelle untersuchen. Rechnen wir sie aus.

Lemma 4.3.6 (Kondition der Differenz)

Für $h(a, b) = a - b$ gilt bezüglich der euklidischen Norm

$$\kappa_{\text{rel}}(h, (a, b)) = \sqrt{2} \frac{\sqrt{a^2 + b^2}}{|a - b|}.$$

Der Beweis ist Schritt für Schritt der von Beispiel 4.2.8, nur mit $(1, -1)^\top$ statt $(1, 1)^\top$: Auch dort liefert die Cauchy-Schwarz-Ungleichung $\kappa_{\text{abs}} = \sqrt{2}$, und die Division durch $|h(a, b)| = |a - b|$ ergibt die Behauptung.

Vertiefung: Die Rechnung im Detail

Beweis.

(1)

$$h(\tilde{a}, \tilde{b}) - h(a, b) = (\tilde{a} - a) - (\tilde{b} - b) = \begin{pmatrix} 1 \\ -1 \end{pmatrix}^\top \begin{pmatrix} \tilde{a} - a \\ \tilde{b} - b \end{pmatrix}$$

h ist linear: Die Differenz zweier Funktionswerte ist h , angewandt auf die Differenz der Inputs, geschrieben als Skalarprodukt mit dem Vektor $(1, -1)^\top$

(2)

$$|h(\tilde{a}, \tilde{b}) - h(a, b)| \leq \sqrt{2} \left\| \begin{pmatrix} \tilde{a} - a \\ \tilde{b} - b \end{pmatrix} \right\|_2$$

Cauchy-Schwarz-Ungleichung mit $\|(1, -1)^\top\|_2 = \sqrt{2}$; für Störungen proportional zu $(1, -1)^\top$ gilt Gleichheit, die Schranke wird also angenommen; $\sqrt{2}$ ist die kleinste solche Konstante und damit $\kappa_{\text{abs}} = \sqrt{2}$

(3)

$$\frac{|h(\tilde{a}, \tilde{b}) - h(a, b)|}{|a - b|} \leq \underbrace{\sqrt{2} \frac{\sqrt{a^2 + b^2}}{|a - b|}}_{=\kappa_{\text{rel}}} \cdot \frac{\left\| \begin{pmatrix} \tilde{a} - a \\ \tilde{b} - b \end{pmatrix} \right\|_2}{\sqrt{a^2 + b^2}}$$

Definition der relativen Konditionszahl (Abschnitt 4.2): beide Seiten durch $|h(a, b)| = |a - b|$ teilen und mit der Inputnorm $\|(a, b)^\top\|_2 = \sqrt{a^2 + b^2}$ erweitern

□

Die Formel sagt präzise, wann es gefährlich wird: Sind a und b groß, liegen aber *nah beieinander* ($a \approx b$), dann ist der Zähler riesig und der Nenner winzig: κ_{rel} explodiert, und der letzte Schritt verstärkt *alle* vorangegangenen Fehler mit diesem Faktor. Genau das passiert bei der Varianz von Daten mit großem Mittelwert: Für $x_i = c + z_i$ mit Verschiebung c und Varianz 1 ist $a \approx c^2 + 1$ und $b \approx c^2$, während die Differenz (die Varianz) bei 1 bleibt. Damit wächst

$$\kappa_{\text{rel}} \approx \sqrt{2} \frac{\sqrt{2} c^2}{1} = 2c^2$$

quadratisch in der Verschiebung. Das ist die katastrophale Auslöschung aus Abschnitt 2.1. Dort können wir mit der interaktiven Demo nachspielen, ab welcher Verschiebung die Formel kippt. Und Bemerkung 4.3.4 erklärt, warum $\tilde{f}_1$ so viel besser dasteht: Auch $\tilde{f}_1$ subtrahiert fast gleich große Zahlen ($x_i - \bar{x}$ mit $x_i \approx \bar{x} \approx c$), aber dieser schlecht konditionierte Schritt kommt *zuerst* und arbeitet auf Zahlen der Größenordnung c . Bei $\tilde{f}_2$ kommt die Subtraktion ganz am *Ende*, nach dem Quadrieren, auf Zahlen der Größenordnung c^2 . Der Verstärkungsfaktor ist $\sim c^2$ statt $\sim c$, und er trifft alle vorher akkumulierten Rundungsfehler.

Beispiel 4.3.7 (Die Instabilität in R)

Ein Zahlenexperiment dazu: $n = 1000$ standardnormalverteilte Werte, um $c = 10^{10}$ verschoben. Die wahre Varianz ist 1.

```
set.seed(5)
x <- rnorm(10^3, mean = 0, sd = 1) + 10^10
mean((x - mean(x))^2)
#> [1] 1.023151
mean(x^2) - mean(x)^2
#> [1] 16384
```

Die zweistufige Rechnung $\tilde{f}_1$ liefert 1,023, eine völlig plausible Stichprobenvarianz. Die Verschiebungsformel $\tilde{f}_2$ liefert 16384: um mehr als vier Größenordnungen daneben. Passt das zu unserer Analyse? Mit $c = 10^{10}$ ist $\kappa_{rel} \approx 2c^2 = 2 \cdot 10^{20}$; die Zwischenergebnisse $a, b \approx 10^{20}$ tragen relative Rundungsfehler der Größenordnung $\varepsilon \approx 2,2 \cdot 10^{-16}$, also erwarten wir relative Fehler im Ergebnis bis etwa $\kappa_{rel} \cdot \varepsilon \approx 4 \cdot 10^4$. Beobachtet: $16384/1,02 \approx 1,6 \cdot 10^4$. Die Größenordnung stimmt.

Ein hübsches Detail: $16384 = 2^{14}$ ist genau der Abstand zweier benachbarter Maschinenzahlen in der Größenordnung 10^{20} . Was die Verschiebungsformel als „Varianz“ ausgibt, ist hier also schlicht ein einzelner Rundungsschritt der Zwischenergebnisse – mit den Daten hat diese Zahl nichts mehr zu tun.

Interaktiv: Wie schlimm wird es? Der κ -Rechner für den letzten Schritt

Bei $c = 10^{10}$ ging alles verloren. Aber wo genau liegt die Grenze? Anders gefragt: Ab welcher Verschiebung c frisst die Kondition des letzten Schritts die 16 Dezimalstellen doppelter Genauigkeit auf?

Interaktives Element — nur in der Web-Fassung

Der Rechner macht die Faustregel aus Bemerkung 4.2.4 zur Ablesehilfe: $\log_{10} \kappa_{rel}$ zählt die verlorenen Dezimalstellen. Bei $c = 10^5$ sind das schon 10 von 16, bei $c = 10^8$ genau 16, und darüber bleibt nichts mehr übrig – im Einklang mit Beispiel 4.3.7.

Selbsttest

Prüfen wir das Verständnis. Erst selbst überlegen, dann aufklappen.

1. Sei $f = h \circ g$, und h sei an der Stelle $g(x)$ schlecht konditioniert. Ist der Algorithmus $\tilde{f} = \tilde{h} \circ \tilde{g}$ dann zwangsläufig instabil?

Nein. Satz 4.3.3 liefert eine lokale *obere Schranke*: Die schlechte Kondition von h kann ungünstig gerichtete Fehler von $\tilde{g}$ stark verstärken. Ist $\tilde{g}$ (nahezu) exakt oder liegt sein Fehler in

einer wenig empfindlichen Richtung, kann der resultierende Fehler trotzdem klein bleiben. Eine schlecht konditionierte Zwischenstelle ist deshalb ein Indiz, kein Beweis für Instabilität.

2. Auch $\tilde{f}_1$ subtrahiert fast gleich große Zahlen ($x_i - \bar{x}$ mit $x_i \approx \bar{x}$). Warum ist $\tilde{f}_1$ trotzdem viel stabiler als $\tilde{f}_2$?

Entscheidend ist nicht allein die Position der Subtraktion. $\tilde{f}_1$ bildet direkt die Abweichungen und arbeitet danach mit deren Quadraten. $\tilde{f}_2$ bildet dagegen zunächst zwei Aggregate der Größenordnung c^2 und versucht, ihre kleine Differenz als Varianz zurückzugewinnen; deren Information kann schon in den Rundungen der großen Aggregate verschwunden sein. Die Zentrierung vermeidet genau diese großen, beinahe gleichen Zwischenwerte.

3. Beim Training eines neuronalen Netzes beginnt der Verlust nach wenigen Epochen zu oszillieren und wächst dann über alle Grenzen. Welche Stellschraube prüfen wir zuerst, und warum?

Die Lernrate α . Oszillation mit wachsender Amplitude ist das Signaturverhalten eines instabilen Iterationsverfahrens: Der Verstärkungsfaktor pro Schritt liegt über 1, jeder Schritt bläht die Fehler des geschätzten Gradienten weiter auf. Eine kleinere Lernrate (oder Gradient Clipping) drückt den Faktor wieder unter 1. In der SGD-Demo oben lässt sich der Umschlag direkt beobachten.

1. Stellen wir im κ -Rechner den Modus „an Verschiebung c gekoppelt“ und den Exponenten $k = 5$ ein. Wie viele Dezimalstellen gehen laut Rechner verloren?

Lösung: 10,3

Rund 10,3. Mit $a = c^2 + 1$ und $b = c^2$ bei $c = 10^5$ ist $\kappa_{rel} = \sqrt{2} \sqrt{a^2 + b^2} \approx 2c^2 = 2 \cdot 10^{10}$, also $\log_{10} \kappa_{rel} \approx 10,3$. Von den etwa 16 Stellen doppelter Genauigkeit bleiben knapp 6 übrig, die Verschiebungsformel liefert hier noch eine Zahl, aber nur noch mit einer Handvoll gültiger Stellen.

2. In der SGD-Demo: Bei welcher Lernrate $\alpha > 0,5$ springen die Iterierten mit konstanter Amplitude um das Minimum, ohne näher zu kommen oder wegzulaufen?

Lösung: 1

Bei $\alpha = 1$. Dort ist $|1 - 2\alpha| = 1$, also $\theta_{k+1} = -\theta_k$: Die Iterierten pendeln zwischen 2,5 und -2,5. Die zweite Lösung von $|1 - 2\alpha| = 1$ ist $\alpha = 0$, und dort passiert schlicht gar nichts.

Vertiefung: Heath §1.2.7 (Stabilität und Genauigkeit).

4.4 Zusammenfassung

Fassen wir das Kapitel zusammen. Der rote Faden war die Frage, warum ein berechnetes Ergebnis $\tilde{f}(\tilde{\mathbf{x}})$ vom idealen Ergebnis $f(\mathbf{x})$ abweicht, und wie wir diese Abweichung messen, zerlegen und den beiden Schuldigen zuordnen: dem *Problem* und dem *Algorithmus*.

Die Konzepte im Überblick

Konzept	Kernaussage	Abschnitt
absoluter / relativer Fehler	$\Delta_v = \tilde{v} - v$ bzw. $\delta_v = \ \tilde{v} - v\ / \ v\ $, gemessen in einer Norm unserer Wahl	4.1
Fehlerzerlegung	Gesamtfehler = algorithmischer Fehler $\tilde{f}(\tilde{x}) - f(\tilde{x})$ + Input-Folgefehler $f(\tilde{x}) - f(x)$	4.1
Kondition	Sensitivität eines Problems bezüglich Inputfehlern, unabhängig vom verwendeten Algorithmus	4.2
Konditionszahlen	κ_{abs} und κ_{rel} : kleinste Verstärkungsfaktoren von absolutem bzw. relativem Inputfehler; gut konditioniert ($\kappa \lesssim 1$), schlecht konditioniert ($\kappa \gg 1$), schlecht gestellt ($\kappa = \infty$)	4.2
Konditionszahl einer Matrix	$\kappa(A) = \ A\ \ A^{-1}\ $ in einer Matrixnorm: obere Schranke für die relative Kondition des Problems $x \mapsto A^{-1}x$ (eingeführt in Abschnitt 3.5)	4.2
Stabilität	Eigenschaft eines Algorithmus: $\tilde{f}$ ist stabil, wenn der algorithmische Fehler $\tilde{f}(\tilde{x}) - f(\tilde{x})$ klein ist	4.3
zusammengesetzte Algorithmen	Fehlerfortpflanzung über Teilschritte: Die Kondition eines späteren Teilschritts verstärkt die Fehler aller früheren	4.3

Kondition und Stabilität: die Arbeitsteilung

Die *Kondition* kontrolliert den orangenen Term der Fehlerzerlegung: wie stark f selbst (in exakter Arithmetik) Inputfehler verstärkt. Daran kann kein noch so guter Algorithmus etwas ändern. Die *Stabilität* kontrolliert den roten Term: wie viel zusätzlichen Fehler die Umsetzung $\tilde{f}$ durch Rundungsfehler in Gleitkommaarithmetik obendrauf legt. Ein stabiler Algorithmus für ein schlecht konditioniertes Problem liefert also trotzdem ungenaue Ergebnisse – aber nicht ungenauer, als das Problem es erzwingt.

Bemerkung 4.4.1 (Faustregel, mit Einschränkung)

Schlecht konditionierte Teilschritte möglichst früh ausführen!

Warum? Nach der Fehlerfortpflanzungs-Schranke aus Abschnitt 4.3 verstärkt die Kondition eines Teilschritts die angesammelten Fehler aller *vorangegangenen* Schritte. Ein schlecht konditionierter Schritt am Ende der Kette trifft auf einen Input, der schon viele Fehler trägt, und bläst sie alle auf. Derselbe Schritt am Anfang sieht nur den unvermeidbaren Inputfehler. Genau das haben wir am Varianz-Beispiel gesehen: Die Verschiebungsformel hebt sich die schlecht konditionierte Subtraktion (Stichwort Auslöschung) bis zum Schluss auf, die Zweipass-Formel zieht sie vor und ist deshalb stabil.

Vorsicht: Das ist eine Faustregel, kein Satz – die Einschränkungen und die verlässlicheren Strategien stehen in Bemerkung 4.3.4.

Wie geht es weiter?

Im nächsten Kapitel wenden wir das neue Vokabular auf das wichtigste Rechenproblem der Statistik an: die numerische Lösung linearer Gleichungssysteme $Ax = b$. Die Konditionszahl $\kappa(A)$ sagt uns dann, wie genau die Lösung überhaupt sein *kann* – und die Stabilitätsanalyse, welche Lösungsverfahren diese Genauigkeit auch wirklich erreichen.

Selbsttest zum Kapitel

1. Ein stabiler Algorithmus für ein schlecht konditioniertes Problem liefert im Allgemeinen ungenaue Ergebnisse.

Antwort: Wahr

Die Stabilität begrenzt nur den roten Term der Zerlegung, den der Algorithmus selbst beisteuert. Den orangen Term erzwingt das Problem: Ist es schlecht konditioniert, verstärkt es den unvermeidbaren Inputfehler, und daran ändert kein Algorithmus etwas. Die richtige Erwartung lautet: nicht ungenauer, als das Problem es erzwingt.

2. Im Widget zur Kondition eines Gleichungssystems (Abschnitt 4.2) ist die Voreinstellung „schlecht konditioniert“ gewählt. Welchen Wert zeigt der Readout für die Schranke $\kappa(\mathbf{A})$?

Lösung: 82

Rund 82. Die Matrix ist $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1,05 \end{pmatrix}$ mit den Singulärwerten $\sigma_{\max} \approx 2,025$ und $\sigma_{\min} \approx 0,0247$, also $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min} \approx 82,0$ (Bemerkung 4.2.7). Für die Voreinstellung der rechten Seite wird diese Schranke fast exakt angenommen.

Vertiefung: Heath §1.2.

Lineare Gleichungssysteme

5.1 Numerische lineare Algebra: Grundlagen

Ab hier geht es um *numerische lineare Algebra*. Sie fragt, wie sich die Objekte und Operationen der linearen Algebra auf dem Rechner tatsächlich berechnen lassen, und zwar schnell und verlässlich. Auf dem Programm stehen numerische Methoden für

- die Matrizenrechnung (Addition, Multiplikation),
- lineare Gleichungssysteme (der Kern dieses Kapitels, ab Abschnitt 5.2),
- Kleinste-Quadrate-Probleme (Kapitel 7),
- Eigenwertprobleme und mehr.

Dabei achten wir bei jeder Methode auf dieselben zwei Gütekriterien, die wir in den letzten Kapiteln aufgebaut haben: auf die Kondition (wie stark verstärken sich kleine Fehler?) und auf die Komplexität (wie wächst der Aufwand mit der Problemgröße?).

Was wir mitbringen

Dieses Kapitel führt die Fäden der bisherigen Kapitel zusammen. Wir brauchen:

- aus Kapitel 2 den Algorithmus-Begriff, insbesondere *direkte Methoden* (endlich viele Schritte, exakte Lösung in exakter Arithmetik) und die Komplexitätsanalyse mit Landau-Symbolen,
- aus Kapitel 4 die Kondition von Problemen und die Stabilität von Algorithmen,
- aus Kapitel 3 die Matrixnormen und die Orthogonalmatrizen mit ihrer perfekten Kondition $\kappa_2(Q) = 1$.

Dazu kommt Grundwissen aus der Linearen Algebra I: das Lösen linearer Gleichungssysteme mit grundlegenden Methoden, Dreiecksmatrizen, symmetrische und positiv definite Matrizen sowie die Matrixinverse. Über Letztere lernen wir in diesem Kapitel vor allem eines: dass wir sie numerisch *nicht* berechnen.

Matrizenrechnung

Beginnen wir mit den Grundoperationen. Ihre algorithmische Berechnung folgt für gewöhnlich direkt den mathematischen Formeln. Für die Addition von $A, B \in \mathbb{R}^{n \times m}$ heißt das

$$A + B = \begin{pmatrix} a_{11} + b_{11} & \cdots & a_{1m} + b_{1m} \\ \vdots & & \vdots \\ a_{n1} + b_{n1} & \cdots & a_{nm} + b_{nm} \end{pmatrix},$$

und für die Matrixmultiplikation von $A \in \mathbb{R}^{n \times m}$ mit $B \in \mathbb{R}^{m \times k}$

$$AB = \begin{pmatrix} \sum_{i=1}^m a_{1i}b_{i1} & \cdots & \sum_{i=1}^m a_{1i}b_{ik} \\ \vdots & & \vdots \\ \sum_{i=1}^m a_{ni}b_{i1} & \cdots & \sum_{i=1}^m a_{ni}b_{ik} \end{pmatrix}.$$

Jeder Eintrag des Produkts ist also ein Skalarprodukt einer Zeile von A mit einer Spalte von B .

Bemerkung 5.1.1 (Kondition der Grundoperationen)

Wie gut oder schlecht Matrixaddition und -multiplikation konditioniert sind, untersuchen wir in der Übung. Das Handwerkszeug dafür steht in Kapitel 4.

Die Komplexität der beiden Formeln können wir dagegen sofort bestimmen, indem wir Operationen zählen wie in Kapitel 2. Prüfen wir uns zuerst selbst.

Was ist die algorithmische Komplexität der *Addition* von $A, B \in \mathbb{R}^{n \times m}$? Zur Auswahl stehen $O(nm)$, $O(n + m)$ und $O(n^2m^2)$.

1. Die Addition gelingt in $O(nm)$ Operationen, und schneller geht es größenordnungsmäßig auch nicht.

Antwort: Wahr

Das Ergebnis hat nm Einträge, und jeder Eintrag kostet genau eine Addition $a_{ij} + b_{ij}$; zusammen nm Additionen, also $O(nm)$. Weniger ist nicht drin, denn allein das Hinschreiben der nm Ergebniseinträge braucht schon nm Schritte.

2. Die Addition gelingt sogar in $O(n + m)$ Operationen.

Antwort: Falsch

$n + m$ ist viel kleiner als nm (für $n = m = 100$ etwa 200 gegen 10 000). Ein Algorithmus mit nur $O(n + m)$ Schritten könnte nicht einmal alle Ergebniseinträge schreiben.

3. Der Aufwand wächst wie n^2m^2 , weil jeder Eintrag von A mit jedem Eintrag von B verrechnet werden muss.

Antwort: Falsch

Die Begründung stimmt nicht: Addiert wird nur *paarweise* an gleicher Position (i, j) , kein Eintrag trifft je einen anderen. Als reine obere Schranke wäre $O(n^2m^2)$ zwar formal korrekt (Landau-Symbole schließen Überschätzung nicht aus, vgl. Kapitel 2), aber als Beschreibung des Wachstums grob falsch; die scharfe Ordnung ist nm .

Und die *Multiplikation* von $A \in \mathbb{R}^{n \times m}$ mit $B \in \mathbb{R}^{m \times k}$? Zur Auswahl stehen $O(nk)$, $O(nmk)$ und $O(nm + k)$.

1. Die Multiplikation kostet $O(nk)$ Operationen, denn das Produkt hat nk Einträge.

Antwort: Falsch

nk zählt nur die Ergebniseinträge. Jeder einzelne ist aber ein Skalarprodukt der Länge m und kostet selbst m Multiplikationen und $m - 1$ Additionen; der Faktor m fehlt.

2. Die Multiplikation nach der Produktformel kostet $O(nmk)$ Operationen.

Antwort: Wahr

Für jede der $n \cdot k$ Kombinationen aus Zeile und Spalte fallen m Multiplikationen und $m - 1$ Additionen an, insgesamt also $nk(2m - 1) = O(nmk)$ Operationen. Für quadratische Matrizen ($n = m = k$) ist das der bekannte $O(n^3)$ -Aufwand.

3. Es reichen $O(nm + k)$ Operationen.

Antwort: Falsch

Für $n = m = k$ wüchse $nm + k = n^2 + n$ nur quadratisch, die Produktformel braucht aber kubischen Aufwand n^3 . Die Summe $nm + k$ unterschätzt den Aufwand also drastisch, sobald die Dimensionen wachsen.

Vertiefung: Schnellere Matrixmultiplikation

Geht Matrixmultiplikation schneller?

Erstaunlicherweise ja, zumindest auf dem Papier. Für das Produkt zweier quadratischer Matrizen $A, B \in \mathbb{R}^{n \times n}$ gibt es Algorithmen mit besserer Komplexität als $O(n^3)$. Der erste stammt von Strassen (1969): Er zerlegt beide Matrizen in vier Blöcke und kommt mit sieben statt acht Blockmultiplikationen aus. Rekursiv angewandt kostet das $O(n^{\log_2 7})$ Operationen, also etwa $O(n^{2.8})$, denn $\log_2 7 \approx 2.807$. Seither wurde der Exponent in kleinen Schritten weiter gedrückt; der aktuelle Rekord liegt bei etwa $O(n^{2.37})$ (Alman et al., 2024), und wesentlich Besseres ist bislang nicht bekannt. Die Suche nach neuen Multiplikationsschemata geht weiter. 2022 hat etwa DeepMinds AlphaTensor per Reinforcement Learning bisher unbekannte Varianten für kleine Matrixformate gefunden.

Bemerkung 5.1.2 (Warum wir trotzdem mit $O(nmk)$ rechnen)

Die Landau-Notation versteckt konstante Faktoren, und bei den schnellen Multiplikationsalgorithmen sind diese Konstanten meist so groß, dass sich die bessere Ordnung erst bei riesigen Matrizen auszahlen würde. In der Praxis werden die Verfahren deshalb kaum verwendet. Wir rechnen in diesem Skript der Einfachheit halber immer mit $O(nmk)$ für die Multiplikation und $O(nm)$ für die Addition.

Damit ist der Rahmen gesteckt. Ab dem nächsten Abschnitt wenden wir uns dem wichtigsten Problem der numerischen linearen Algebra zu: dem Lösen von $A\mathbf{x} = \mathbf{b}$.

Vertiefung: Heath §2.1 (der Einstieg in Heath Kap. 2, das lineare Gleichungssysteme behandelt); zur Komplexitätsanalyse Heath §1.1.

5.2 Lineare Gleichungssysteme

Problemstellung

In Abschnitt 5.1 haben wir gezählt, was Matrixaddition und Matrixmultiplikation kosten. Jetzt setzen wir diese Bausteine zu der Aufgabe zusammen, die in der Statistik ständig auftritt: Wir beschäftigen uns mit der Lösung eines linearen Gleichungssystems (LGS). Gesucht ist also der Vektor $\mathbf{x}$ mit

$$A\mathbf{x} = \mathbf{b}.$$

Ob Kleinste-Quadrate-Schätzer (Abschnitt 7.1) oder gemischtes Modell: am Ende steht fast immer ein solches System.

Die Aufgabe ist nur dann sinnvoll gestellt, wenn es genau eine Lösung gibt. Wir nehmen deshalb in diesem Kapitel an, dass $A \in \mathbb{R}^{n \times n}$ quadratisch und invertierbar ist. Auf dem Papier ist damit alles erledigt: Die Lösung lässt sich als $x = A^{-1}b$ schreiben. Vorsicht: Diese Formel ist eine Schreibweise, kein Rechenweg.

Bemerkung 5.2.1 (Für ein LGS keine explizite Inverse bilden)

Wer nur $Ax = b$ lösen will, sollte die Inverse nicht explizit bilden:

Löse das System direkt.

Warum ist die Formel $x = A^{-1}b$ als Algorithmus so schlecht?

- *Aufwand:* A^{-1} explizit auszurechnen heißt, die n Systeme $Az_j = e_j$ für alle Einheitsvektoren e_j zu lösen und anschließend noch das Produkt $A^{-1}b$ zu bilden. Wer nur x braucht, erledigt also n Lösungen statt einer. In der Größenordnung bleibt das zwar beim Aufwand einer einzigen Elimination, weil sich deren teure Vorarbeit für alle n rechten Seiten wiederverwenden lässt; in der versteckten Konstanten kostet der Umweg aber ein Mehrfaches.
- *Genauigkeit:* Das explizite Invertieren muss nicht in jedem Fall scheitern. Bei einer schlecht konditionierten Matrix können die zusätzlichen Rundungen der Inversenbildung und des anschließenden Matrix-Vektor-Produkts den Fehler aber deutlich vergrößern. Für eine einzelne rechte Seite gibt es keinen Genauigkeitsvorteil. Gauß-Elimination *mit partieller Pivotierung* ist in der Praxis meist rückwärtsstabil und vermeidet diesen Umweg. Wie genau die Lösung sein kann, begrenzt in beiden Fällen die Konditionszahl $\kappa(A)$ (Abschnitt 3.5 und Abschnitt 4.2).

Explizite Inversen haben legitime Anwendungen, wenn die Inverse selbst das gesuchte Objekt ist. Die Warnung bezieht sich auf das unnötige Bilden von A^{-1} zum Lösen eines Systems. Wo immer in einer Formel $A^{-1}b$ steht, lösen wir in der Implementierung stattdessen das LGS $Ax = b$.

Vertiefung: Ein empfindliches Zahlenbeispiel zur expliziten Inversen

Beispiel 5.2.2 (Hilbert-Matrix: Inverse gegen direktes Lösen)

Betrachten wir die Hilbert-Matrix H_n mit Einträgen $h_{ij} = 1/(i + j - 1)$ und wählen die exakte Lösung $\mathbf{x} = (1, \dots, 1)^\top$. Dann ist $\mathbf{b} = H_n \mathbf{x}$ bekannt.

Für $n = 11$ haben wir beide Wege in JavaScript mit derselben einfachen Gauß-Elimination mit partieller Pivotierung nachgerechnet. Beim direkten Lösen erhalten wir den relativen Fehler $\|\hat{\mathbf{x}} - \mathbf{x}\|_\infty / \|\mathbf{x}\|_\infty \approx 9,66 \cdot 10^{-3}$. Bilden wir zuerst die explizite Inverse, steigt er auf etwa $3,73 \cdot 10^{-1}$. Der Umweg ist in dieser Rechnung also rund 38,6-mal ungenauer. Die konkreten Werte hängen von Algorithmus, Reihenfolge der Operationen und Gleitkommaarithmetik ab. Sie sind keine allgemeinen Fehlerschranken. In dieser Rechnung fügt das explizite Invertieren einen empfindlichen Rechenweg hinzu.

Der R-Code des Beispiels verwendet `solve(A, b)` zum direkten Lösen und `solve(A) %*% b` für den Umweg über die Inverse:

```
n <- 11 # Hilbert-Matrix
A <- outer(1:n, 1:n,
           function(i, j) 1 / (i + j - 1))
x <- rep(1, n)
b <- A %*% x

range((solve(A) %*% b) - x) # Fehler über Inverse
range(solve(A, b) - x)      # Fehler direkte Lösung
```

Interaktiv: Kondition und Rechenweg

Interaktives Element — nur in der Web-Fassung

Der Regler zeigt dieselbe Rechnung für mehrere Ordnungen. Die Konditionszahl erklärt die grundsätzliche Empfindlichkeit; der Vergleich der beiden Fehler isoliert den zusätzlichen Preis des Inversenwegs in der hier verwendeten Arithmetik.

Gauß-Elimination und Rückwärtssubstitution

Wie lösen wir stattdessen? Wir erinnern uns an das Gauß'sche Eliminationsverfahren aus der linearen Algebra. Es besteht aus zwei Phasen:

Algorithmus 5.2.3 (Gauß-Elimination mit partieller Pivotierung)

1. Wähle in Eliminationsschritt k unter den Zeilen $k, \dots, n$ einen Eintrag mit maximalem Betrag in Spalte k und tausche seine Zeile nach oben. Eliminiere anschließend die Einträge darunter. So entsteht aus $(A \mid \mathbf{b})$ eine Zeilenstufenform $(U \mid \mathbf{c})$ mit oberer Dreiecksmatrix U .
2. Löse das gestaffelte System

$$\begin{array}{ccccccc} u_{11}x_1 & + & u_{12}x_2 & + & \dots & + & u_{1n}x_n & = & c_1 \\ & & u_{22}x_2 & + & \dots & + & u_{2n}x_n & = & c_2 \\ & & & & \ddots & & \vdots & & \vdots \\ & & & & & & u_{nn}x_n & = & c_n \end{array}$$

durch *Rückwärtssubstitution* (backward substitution):

$$x_n = c_n/u_{nn}, \quad x_i = \left(c_i - \sum_{j=i+1}^n u_{ij}x_j \right) / u_{ii} \quad (i \leq n-1). \quad (5.2.1)$$

Die letzte Gleichung des gestaffelten Systems enthält nur noch x_n und verrät diesen Wert sofort. Danach arbeiten wir uns Zeile für Zeile nach oben. Im Farbcode dieses Kapitels: Ist Zeile i an der Reihe (blau), sind die Komponenten aus den Zeilen darunter schon bekannt (grün). Die Zeile setzt sie ein und teilt zum Schluss durch ihr Diagonalelement, das Pivot (rot):

$$x_i = \left(c_i - \sum_{j=i+1}^n u_{ij} x_j \right) / u_{ii}.$$

Mit partieller Pivotierung garantiert die Invertierbarkeit in jedem Schritt ein von null verschiedenes Pivot; *ohne* Zeilentausch reicht sie nicht, denn ein Nullpivot kann trotzdem auftreten (Beispiel 5.3.6). Auch ein sehr kleines Pivot ist in Gleitkommaarithmetik gefährlich – deshalb wählen wir den größten verfügbaren Betrag.

Interaktiv: Rückwärtseinsetzen Schritt für Schritt

Die Formel (5.2.1) bestimmt die Komponenten von unten nach oben. Wie viele Divisionen braucht das voreingestellte 3×3-System?

Interaktives Element — nur in der Web-Fassung

Wie teuer ist die zweite Phase? Prüfen wir die vier Antwortmöglichkeiten einzeln (Landau-Notation zur Erinnerung in Abschnitt 2.4).

1. Wie viele Divisionen hat das voreingestellte Rückwärtseinsetzen im Widget?

Lösung: 3

Eine pro Diagonalzeile, also drei.

2. Die Rückwärtssubstitution für ein $n \times n$ -System hat die Komplexität $O(1)$.

Antwort: Falsch

Konstanter Aufwand ist unmöglich: Schon um jeden Eintrag von U einmal anzufassen, braucht es mit n wachsende Arbeit.

3. Die Rückwärtssubstitution für ein $n \times n$ -System hat die Komplexität $O(n)$, denn sie besteht aus n Schritten.

Antwort: Falsch

Die Schrittzahl stimmt, aber die Schritte sind nicht konstant teuer: Schritt i setzt die $n-i$ schon bekannten Komponenten ein, im ungünstigsten Fall also $n-1$ Stück.

4. Die Rückwärtssubstitution für ein $n \times n$ -System hat die Komplexität $O(n^2)$.

Antwort: Wahr

Es sind n Schritte, und Schritt i kostet nach Formel (5.2.1) je $n-i$ Multiplikationen und ebenso viele Subtraktionen, dazu eine Division; im ungünstigsten Fall sind das $n-1$ Multiplikationen pro Schritt. Aufsummiert ergibt das $n(n-1)/2 \approx n^2/2$ Multiplikationen und

insgesamt genau n^2 Rechenoperationen: die Größenordnung ist $O(n^2)$.

5. Die Rückwärtssubstitution für ein $n \times n$ -System hat die Komplexität $O(n^3)$.

Antwort: Falsch

Das überschätzt den Aufwand um einen Faktor n . (Formal enthält $O(n^3)$ zwar auch alle $O(n^2)$ -Algorithmen, gemeint ist hier aber die Größenordnung des tatsächlichen Aufwands, und die ist n^2 .) Bei $O(n^3)$ liegt erst die Eliminationsphase selbst, wie wir in Abschnitt 5.3 sehen werden.

Warum Matrixzerlegungen?

Die Gauß-Elimination löst unser LGS. In den nächsten Abschnitten organisieren wir sie neu, nämlich als *Matrixzerlegung* (matrix decomposition). Die Grundidee ist „Teile und herrsche“ für Matrizen: Statt direkt mit A zu arbeiten, schreiben wir $A = BC$ mit „einfachen“ Faktoren B und C .

Eine Analogie aus der Zahlenwelt ist die Primfaktorzerlegung. Es gilt $84 = 2^2 \cdot 3 \cdot 7$, und wer diese Zerlegung einmal kennt, rechnet bequemer weiter:

$$84 \cdot 126 = (2^2 \cdot 3 \cdot 7) \cdot (2 \cdot 3^2 \cdot 7) = 2^3 \cdot 3^3 \cdot 7^2.$$

Die Faktoren machen die Struktur der Zahl sichtbar, und Multiplizieren schrumpft zum Addieren von Exponenten. Dasselbe wollen wir für Matrizen: Faktoren finden, an denen sich billig rechnen lässt. Drei Familien haben sich als „einfach“ bewährt:

- **Dreiecksmatrizen:** Ein Dreieckssystem lösen wir per Vorwärts- bzw. Rückwärtssubstitution in $O(n^2)$ statt $O(n^3)$, wie eben gesehen.
- **Orthogonale Matrizen:** Sie verstärken Störungen in der 2-Norm nicht und sind trivial zu invertieren wegen $Q^{-1} = Q^T$.
- **Diagonalmatrizen:** Invertieren, Multiplizieren und Potenzieren laufen elementweise auf der Diagonalen, alle Operationen sind trivial.

Was bringt uns eine Zerlegung konkret? Vier Dinge:

1. **Mehrere LGS mit derselben Matrix effizient lösen.** Die Zerlegung bezahlen wir einmal; danach kostet jedes weitere System $Ax = b_i$ mit neuer rechter Seite nur noch die billigen Dreiecks- oder Diagonallösungen. In der Statistik ist das der Normalfall, etwa wenn dieselbe Modellmatrix auf viele Zielvariablen trifft.
2. **Numerische Stabilität verbessern.** Wer die Arbeit in gut konditionierte Teilschritte zerlegt, etwa über orthogonale Faktoren, vermeidet unnötige Fehlerverstärkung (Abschnitt 4.3).
3. **Struktur und Eigenschaften von A erkennen.** An den Faktoren lässt sich ablesen, ob A (fast) singulär oder positiv definit ist; solche Diagnosen fallen bei der Zerlegung gratis mit ab.
4. **Grundlage für weitere Algorithmen.** Auf Zerlegungen bauen viele Verfahren auf, von der QR-Zerlegung für das KQ-Problem (Abschnitt 7.4) bis zur Spektralzerlegung und der Singulärwertzerlegung (SVD), die uns in einem späteren Kapitel begegnet.

Den Anfang macht im nächsten Abschnitt die LU-Zerlegung: Sie ist nichts anderes als die Gauß-Elimination, klug verbucht.

Vertiefung: Heath §2.1–2.2 (Problemstellung und Lösbarkeit) sowie §2.4.2 (Dreieckssysteme und Rückwärtssubstitution).

5.3 Die LU-Zerlegung

Elimination als Zerlegung

Die Gauß-Elimination aus Abschnitt 5.2 verwandelt $(A \mid b)$ durch Zeilenoperationen in ein gestaffeltes System. In diesem Abschnitt wechseln wir den Blickwinkel: Die Elimination rechnet nicht nur ein konkretes System um, sie zerlegt die Matrix selbst. Alles, was wir dabei tun, lässt sich in zwei Faktoren verbuchen:

- L (*lower*) speichert sämtliche Eliminationsschritte,
- U (*upper*) ist die entstehende obere Dreiecksmatrix.

Definition 5.3.1 (LU-Zerlegung)

Eine Darstellung

$$A = LU$$

von $A \in \mathbb{R}^{n \times n}$ mit einer unteren Dreiecksmatrix L mit Einsen auf der Diagonalen und einer oberen Dreiecksmatrix U heißt *LU-Zerlegung* (LU decomposition) von A .

Warum lohnt sich diese Buchführung? Sobald wir $A = LU$ einmal berechnet haben, können wir $Ax = b_i$ für jedes b_i billig lösen, denn beide Faktoren sind Dreiecksmatrizen:

Bemerkung 5.3.2 (Merkregel: ein LGS, zwei Dreieckssysteme)

Aus der Zerlegung wird ein zweistufiger Lösungsweg:

$$Ax = b \implies L \underbrace{Ux}_{=y} = b,$$

wir lösen also nacheinander zwei gestaffelte Systeme:

1. $Ly = b$ durch Vorwärtssubstitution (forward substitution), von oben nach unten;
2. $Ux = y$ durch Rückwärtssubstitution (Abschnitt 5.2), von unten nach oben.

Beide Teilschritte kosten nur $O(n^2)$ Operationen.

Eliminationsmatrizen

Wie sehen die Faktoren aus? Wir gehen die Elimination noch einmal durch, diesmal in Matrixsprache. Von links nach rechts arbeiten wir uns durch die Spalten und entfernen alle Einträge unter der Diagonalen. Nennen wir das aktuelle System $(M \mid r)$. Die k -te Spalte räumen wir, indem wir von links mit der *Eliminationsmatrix* L_k multiplizieren:

$$\mathbf{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & & 1 \end{pmatrix} \quad \text{mit} \quad l_{ik} = \frac{m_{ik}}{m_{kk}}. \quad (5.3.1)$$

$\mathbf{L}_k$ ist eine Einheitsmatrix mit Zusatzeinträgen in Spalte k . Das Produkt $\mathbf{L}_k \mathbf{M}$ zieht für jedes $i > k$ von Zeile i das l_{ik} -fache der Pivotzeile k ab und erzeugt so die Nullen unter dem Pivot m_{kk} . Nach $n-1$ solchen Schritten sind alle Spalten abgeräumt; ein n -ter Schritt wäre arbeitslos, denn unterhalb von m_{nn} steht kein Eintrag mehr ($\mathbf{L}_n$ wäre schlicht die Einheitsmatrix).

Satz 5.3.3 (Die Gauß-Elimination liefert eine LU-Zerlegung)

Läuft die Gauß-Elimination für $\mathbf{A} \in \mathbb{R}^{n \times n}$ ohne Nullpivot durch, so ist

$$\mathbf{U} = \mathbf{L}_{n-1} \cdots \mathbf{L}_1 \mathbf{A}$$

eine obere Dreiecksmatrix, und es gilt $\mathbf{A} = \mathbf{LU}$ mit

$$\mathbf{L} = \mathbf{L}_1^{-1} \cdots \mathbf{L}_{n-1}^{-1} = \begin{pmatrix} 1 & & & & \\ l_{21} & 1 & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & & & \ddots & \\ l_{n1} & \cdots & \cdots & l_{n(n-1)} & 1 \end{pmatrix}. \quad (5.3.2)$$

$\mathbf{L}$ ist also eine untere Dreiecksmatrix mit Einsen auf der Diagonalen, und unter der Diagonalen stehen unverändert die Multiplikatoren l_{ik} aus der Elimination.

Vertiefung: Die Eliminationsmatrizen sauber zusammenrechnen

Der Beweis rechnet mit einer kompakten Schreibweise für $\mathbf{L}_k$; die eigentliche Arbeit steckt darin, dass sich die Inversen ohne jede Nebenwirkung aufeinanderstapeln.

Beweis.

- (1) Wir schreiben L_k kompakt als $L_k = I - l_k e_k^T$, wobei e_k der k -te Einheitsvektor ist und $l_k = (0, \dots, 0, l_{(k+1)k}, \dots, l_{nk})^T$ die Multiplikatoren der k -ten Spalte sammelt; die ersten k Komponenten von l_k sind null.

$l_k e_k^T$ ist die Matrix, die in Spalte k unterhalb der Diagonalen die Einträge l_{ik} trägt und sonst nur Nullen; abgezogen von I ergibt das die Gestalt aus (5.3.1)

- (2) Es gilt $L_k^{-1} = I + l_k e_k^T$: Die Inverse macht die Zeilenoperationen rückgängig, indem sie die abgezogenen Vielfachen wieder addiert. Nur die Vorzeichen der Multiplikatoren kippen.

Ausmultiplizieren: $(I - l_k e_k^T)(I + l_k e_k^T) = I - l_k (e_k^T l_k) e_k^T = I$, denn $e_k^T l_k$ ist die k -te Komponente von l_k , und die ist null

- (3) Für das Produkt der Inversen gilt

$$L = (I + l_1 e_1^T) \cdots (I + l_{n-1} e_{n-1}^T) = I + \sum_{k=1}^{n-1} l_k e_k^T,$$

und die rechte Seite ist die Matrix aus (5.3.2): Einsen auf der Diagonalen, darunter die Multiplikatoren, jeder an seinem Platz.

Beim Ausmultiplizieren verschwinden alle gemischten Terme: Sie enthalten Faktoren der Form $e_j^T l_k$ mit $j < k$, und die j -te Komponente von l_k ist für $j \leq k$ null

- (4) Insbesondere ist L als Summe der Einheitsmatrix und von Beiträgen unterhalb der Diagonalen eine untere Dreiecksmatrix mit Einsen auf der Diagonalen. Auflösen von $U = L_{n-1} \cdots L_1 A$ nach A ergibt schließlich $A = L_1^{-1} \cdots L_{n-1}^{-1} U = LU$.

wir multiplizieren von links nacheinander mit $L_{n-1}^{-1}, \dots, L_1^{-1}$; diese Inversen existieren nach Schritt 2

□

Die Eliminationsmatrizen treffen bei der klassischen Elimination auch die rechte Seite: Aus b wird $c = L_{n-1} \cdots L_1 b$, das gestaffelte Endsystem ist $(U \mid c)$. Mit gespeichertem L dürfen wir uns dieses Mitschleppen sparen, denn die Vorwärtssubstitution $Ly = b$ liefert später denselben Vektor: $y = L^{-1}b = c$.

Ein vollständiges Beispiel

Rechnen wir das einmal komplett durch, im Farbcode dieses Kapitels: Pivots rot, Multiplikatoren blau, fertige L -Einträge grün.

Beispiel 5.3.4 (LU-Zerlegung einer 3×3-Matrix)

Sei

$$A = \begin{pmatrix} 2 & 1 & -1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}.$$

Schritt 1: Spalte 1 räumen. Das Pivot ist $m_{11} = 2$, die Multiplikatoren nach Formel (5.3.1) sind

$$l_{21} = \frac{4}{2} = 2, \quad l_{31} = \frac{-2}{2} = -1.$$

Damit ist

$$L_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}, \quad L_1 A = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 8 & 1 \end{pmatrix}.$$

Schritt 2: Spalte 2 räumen. Das Pivot ist jetzt -8 , und mit

$$l_{32} = \frac{8}{-8} = -1, \quad L_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

erreichen wir die Dreiecksform:

$$L_2 L_1 A = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} = U.$$

Ergebnis: Die Multiplikatoren wandern mit ihrem ursprünglichen Vorzeichen nach L :

$$L = L_1^{-1} L_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix}.$$

Die Probe $LU = A$ geht auf (nachrechnen!).

Wann existiert die Zerlegung?

Die Konstruktion hat eine Schwachstelle: In jedem Schritt teilen wir durch das Pivot m_{kk} . Steht dort eine Null, bricht die Elimination ab, und zwar auch dann, wenn A invertierbar ist. Invertierbarkeit allein garantiert also noch keine LU-Zerlegung; sie garantiert nur, dass sich das Problem durch Zeilentausch beheben lässt.

Satz 5.3.5 (Existenz der LU-Zerlegung)

Sei $A \in \mathbb{R}^{n \times n}$ invertierbar. Dann gilt:

1. Die LU-Zerlegung $A = LU$ aus Definition 5.3.1 existiert genau dann, wenn die Gauß-Elimination an A ohne Nullpivot durchläuft. In diesem Fall ist sie eindeutig.
2. Es gibt stets eine Permutationsmatrix P (sie vertauscht Zeilen), sodass die Elimination an PA ohne Nullpivot durchläuft; es gilt dann $PA = LU$.

Die Richtung „kein Nullpivot $\Rightarrow$ Zerlegung existiert“ ist Satz 5.3.3; auf die übrigen Beweisteile verzichten wir. Wichtiger ist ein Warnbeispiel, das zeigt, dass die Bedingung in Teil 1 keine leere Vorsichtsmaßnahme ist:

Beispiel 5.3.6 (Invertierbar, aber keine LU-Zerlegung)

Die Vertauschungsmatrix

$$A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

ist invertierbar ($\det A = -1$). Trotzdem scheitert die Elimination sofort, denn das erste Pivot ist $m_{11} = 0$. Auch keine andere Konstruktion hilft: Der Ansatz

$$LU = \begin{pmatrix} 1 & 0 \\ l & 1 \end{pmatrix} \begin{pmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{pmatrix} = \begin{pmatrix} u_{11} & u_{12} \\ l u_{11} & l u_{12} + u_{22} \end{pmatrix}$$

erzwingt $u_{11} = 0$ aus dem Eintrag links oben und zugleich $l u_{11} = 1$ aus dem Eintrag links unten: ein Widerspruch.

Ein Zeilentausch löst das Problem dagegen sofort auf, denn mit $P = A$ ist $PA = I = I \cdot I$ eine (zugegeben langweilige) LU-Zerlegung.

Lösen mit der LU-Zerlegung

Angenommen, wir haben eine Faktorisierung $A = LU$ gefunden. Die Merkregel aus Bemerkung 5.3.2 wird jetzt zum Algorithmus:

Algorithmus 5.3.7 (Lösen von $Ax = b$ mit der LU-Zerlegung)

1. *Vorwärtssubstitution*: Löse $Ly = b$ von oben nach unten,

$$y_i = b_i - \sum_{j=1}^{i-1} l_{ij} y_j \quad (i = 1, \dots, n);$$

eine Division ist nicht nötig, denn auf der Diagonalen von L stehen Einsen.

2. *Rückwärtssubstitution*: Löse $Ux = y$ von unten nach oben wie in Formel (5.2.1),

$$x_i = \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right) / u_{ii} \quad (i = n, \dots, 1).$$

Beispiel 5.3.8 (Fortsetzung: Lösen mit der Zerlegung)

Wir nehmen L und U aus Beispiel 5.3.4 und lösen $Ax = b$ für $b = (5, -2, 9)^T$.

Schritt 1: Vorwärtssubstitution für $Ly = b$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix} \implies \begin{aligned} y_1 &= 5, \\ y_2 &= -2 - 2 \cdot 5 = -12, \\ y_3 &= 9 - (-1) \cdot 5 - (-1) \cdot (-12) = 9 + 5 - 12 = 2. \end{aligned}$$

Schritt 2: Rückwärtssubstitution für $Ux = y$ mit $y = (5, -12, 2)^T$; die Divisoren sind die Diagonalelemente u_{ii} :

$$\begin{aligned} x_3 &= 2/3 = \frac{2}{3}, \\ x_2 &= (-12 - 2 \cdot \frac{2}{3}) / (-8) = \frac{5}{3}, \\ x_1 &= (5 - 1 \cdot \frac{5}{3} - (-1) \cdot \frac{2}{3}) / 2 = 2. \end{aligned}$$

Die Lösung ist $x = (2, \frac{5}{3}, \frac{2}{3})^T$; die Probe $Ax = b$ geht auf.

Interaktiv: LU-Zerlegung Schritt für Schritt

Welche Einträge speichert die LU-Zerlegung zusätzlich zur Arbeitsmatrix?

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, werden die Multiplikatoren der Elimination in **L** gespeichert, während die Arbeitsmatrix zu **U** wird.

Pivotierung

Was tun, wenn ein Pivot $m_{kk} = 0$ auftritt, wie in Beispiel 5.3.6? Wir tauschen die Pivotzeile gegen eine Zeile weiter unten, in deren k -ter Spalte keine Null steht; bei invertierbarem **A** gibt es so eine Zeile immer. Dieses Umsortieren heißt *Pivotierung* (pivoting). Buchhalterisch sammeln wir alle Vertauschungen in einer Permutationsmatrix **P** und erhalten die pivotierte Zerlegung $\mathbf{PA} = \mathbf{LU}$ aus Satz 5.3.5. Zum Lösen von $\mathbf{Ax} = \mathbf{b}$ substituieren wir dann entlang $\mathbf{LUx} = \mathbf{Pb}$: Die rechte Seite wird mitvertauscht, sonst ändert sich nichts.

Pivotierung ist aber mehr als eine Notfallmaßnahme für exakte Nullen. Auch ein sehr kleines Pivot $m_{kk} \approx 0$ ist gefährlich: Die Multiplikatoren $l_{ik} = m_{ik}/m_{kk}$ werden dann riesig, beim Abziehen der aufgeblähten Pivotzeile gehen die ursprünglichen Einträge der Matrix in Rundungsfehlern unter, und der Eliminationsschritt wird instabil. Der Tausch zum betragsgrößten Pivot der Spalte (*partielle Pivotierung*) hält alle Multiplikatoren im Betrag bei höchstens 1 und macht die Elimination in der Praxis stabil. Numerikbibliotheken pivotieren deshalb grundsätzlich, auch wenn kein Pivot exakt null ist.

Interaktiv: Kleine Pivots, große Fehler

Was verändert der Zeilentausch am Rundungsfehler eines kleinen Pivots?

Interaktives Element — nur in der Web-Fassung

Ohne Zeilentausch wächst der Fehler mit jedem Zehnerschritt von ϵ mit: Bei $\epsilon = 10^{-12}$ stimmen nur noch vier Nachkommastellen, und ab $\epsilon \approx 10^{-16}$ schluckt die Subtraktion $1 - 1/\epsilon$ den Eintrag a_{22} vollständig – die berechnete Zerlegung gehört dann zu einer anderen Matrix. Die Zeile mit Zeilentausch bleibt über den ganzen Reglerbereich bei Maschinengenauigkeit. Am Problem ändert der Tausch nichts, nur am Rechenweg.

Komplexität

Was kostet die Zerlegung? Zählen wir zuerst einen einzelnen Schritt nach. Im k -ten Schritt multiplizieren wir das aktuelle System mit $\mathbf{L}_k$; die ersten $k - 1$ Spalten der Arbeitsmatrix haben ihre Nullen unter der Diagonalen bereits. Vor dem Nachrechnen ein Selbsttest (Landau-Notation zur Erinnerung in Abschnitt 2.4):

1. Für den k -ten Schritt müssen wir das volle Matrixprodukt $\mathbf{L}_k \mathbf{M}$ ausrechnen; er kostet also $O(n^3)$ Operationen.

Antwort: Falsch

Niemand multipliziert L_k wirklich aus. L_k unterscheidet sich von der Einheitsmatrix nur in Spalte k : Das Produkt zieht lediglich von den Zeilen $k + 1, \dots, n$ je ein Vielfaches der Pivotzeile ab; alle anderen Zeilen bleiben unverändert stehen.

2. Im k -ten Schritt ändern sich nur die Einträge im rechten unteren $(n - k) \times (n - k)$ -Block (und die rechte Seite); der Aufwand ist $O((n - k)^2)$.

Antwort: Wahr

Betroffen sind die Zeilen $k + 1, \dots, n$; links von Spalte k stehen dort schon Nullen, und die bleiben null. In Spalte k entsteht die neue Null, neu ausgerechnet werden die Spalten $k + 1, \dots, n$ und der Eintrag der rechten Seite. Das sind $n - k$ Zeilen mit je $n - k + 1$ Aktualisierungen, in der Summe $(n - k)(n - k + 1)$ Rechenschritte, also $O((n - k)^2)$.

3. $O(n^2)$ ist eine korrekte obere Schranke für den Aufwand des k -ten Schritts.

Antwort: Wahr

Wegen $(n - k)^2 \leq n^2$ stimmt das; die Schranke ist nur nicht scharf, denn späte Schritte sind viel billiger. Für die Gesamtkosten macht das keinen Unterschied: Beide Abschätzungen liefern aufsummiert $O(n^3)$.

4. Die Schritte werden mit wachsendem k teurer; der k -te Schritt kostet $O(k^3)$ Operationen.

Antwort: Falsch

Umgekehrt: Der aktive Block schrumpft von Schritt zu Schritt. Der erste Schritt ist mit etwa n^2 Aktualisierungen der teuerste; der letzte bearbeitet nur noch die unterste Zeile, also eine neue Null, einen aktualisierten Eintrag und die rechte Seite.

Satz 5.3.9 (Komplexität der LU-Zerlegung)

Für invertierbares $A \in \mathbb{R}^{n \times n}$ gilt:

1. Die Berechnung der LU-Zerlegung kostet $O(n^3)$ Operationen.
2. Vorwärts- und Rückwärtssubstitution kosten je $O(n^2)$.
3. Das Lösen von J Systemen $Ax = b_j$, $j = 1, \dots, J$, mit derselben Matrix kostet insgesamt $O(n^3 + Jn^2)$.

Teil 3 lebt davon, dass L und U nur von A abhängen und nicht von b : Die teure Elimination fällt kein zweites Mal an, für jede weitere rechte Seite bleiben nur die beiden billigen Substitutionen. Dieses Muster trägt weit über die LU-Zerlegung hinaus – auch die QR-Zerlegung für Kleinste-Quadrate-Probleme (Abschnitt 7.4) folgt ihm.

Vertiefung: Herleitung der Aufwandsordnung und exakte Kosten

Beweis.

- (1) Der k -te Eliminationsschritt kostet $O((n - k)^2)$ Operationen, aufsummiert über die $n - 1$ Schritte also

$$\sum_{k=1}^{n-1} O((n - k)^2) = O\left(\sum_{j=1}^{n-1} j^2\right) = O(n^3).$$

Die $O(n^2)$ eines Substitutionspaars haben wir in Abschnitt 5.2 gezählt; für J rechte Seiten kommen sie J -mal zur einmaligen Zerlegung hinzu.

Substitution $j = n - k$; die Quadratsumme ist $\sum_{j=1}^{n-1} j^2 = \frac{(n-1)n(2n-1)}{6} \approx \frac{n^3}{3}$

□

Wer die Multiplikationen exakt zählt, findet rund $n^3/3$ für die Zerlegung und rund n^2 für ein Substitutionspaar. Mit diesen Konstanten rechnet das folgende Widget:

Interaktiv: Einmal zerlegen oder jedes Mal neu?

Ab wann überwiegt der einmalige Aufwand der Zerlegung nicht mehr?

Interaktives Element — nur in der Web-Fassung

Bei $n = 100$ und $J = 50$ rechten Seiten kostet der Neuansatz rund das Zwanzigfache: Die Zerlegung wird fünfzigmal statt einmal bezahlt, während die Substitutionen mit ihren n^2 kaum ins Gewicht fallen. Schon ab der zweiten rechten Seite liegt die gespeicherte Zerlegung vorn.

1. Ab wie vielen rechten Seiten ist die gespeicherte LU-Zerlegung im Kosten-Widget günstiger?

Lösung: 2

Ab der zweiten rechten Seite fällt die Zerlegung nur einmal an.

Bemerkung 5.3.10 (Struktur ausnutzen)

Die Laufzeit lässt sich weiter drücken, wenn A besondere Eigenschaften erfüllt, zum Beispiel:

- symmetrisch: $A = A^T$;
- positiv definit: $\mathbf{x}^T A \mathbf{x} > 0$ für alle $\mathbf{x} \neq \mathbf{0}$;
- *Bandeigenschaft*: $a_{ij} = 0$ für alle $|i - j| > \beta$, das heißt, außerhalb eines Bandes der Breite β um die Diagonale stehen nur Nullen;
- dünn besetzt (*sparse*): $a_{ij} = 0$ für die allermeisten (i, j) .

Für symmetrische, positiv definite Matrizen halbiert etwa die Cholesky-Zerlegung des nächsten Abschnitts (Abschnitt 5.4) den Aufwand. Für all diese Fälle existieren hervorragende, über Jahrzehnte gereifte Implementierungen; man schreibt solche Algorithmen selten selbst.

Vertiefung: Heath §2.4 (Gauß-Elimination, LU-Zerlegung, Pivotierung) und §2.5 (Spezialstrukturen).

5.4 Die Cholesky-Zerlegung

Die LU-Zerlegung funktioniert, notfalls mit Pivotierung, für jede invertierbare Matrix. Viele Matrizen, die uns in der Statistik begegnen, haben aber deutlich mehr Struktur: Sie sind symmetrisch, und ihre quadratische Form ist strikt positiv. Es lohnt sich, diese Struktur auszunutzen. Für solche Matrizen gibt es eine maßgeschneiderte Zerlegung, die mit halbem Aufwand auskommt und ganz ohne Pivotierung stabil bleibt: die Cholesky-Zerlegung.

SPD-Matrizen

Der entscheidende Spezialfall trägt eine Abkürzung im Namen: SPD-Matrizen sind symmetrisch und positiv definit.

Definition 5.4.1 (SPD-Matrix)

Eine Matrix $A \in \mathbb{R}^{n \times n}$ heißt *SPD*, wenn

- $A = A^\top$ gilt (A also symmetrisch ist) und
- $\mathbf{x}^\top A \mathbf{x} > 0$ für alle $\mathbf{x} \in \mathbb{R}^n$ mit $\mathbf{x} \neq \mathbf{0}$ gilt (A also *positiv definit* ist).

Verlangen wir statt der strikten Ungleichung nur $\mathbf{x}^\top A \mathbf{x} \geq 0$, und zwar für alle $\mathbf{x} \in \mathbb{R}^n$, heißt A *positiv semidefinit*. Den Nullvektor müssen wir dabei nicht mehr ausnehmen, denn $\mathbf{0}^\top A \mathbf{0} = 0$ erfüllt die Bedingung ohnehin. Der Unterschied zur Definitheit wirkt klein, entscheidet aber später darüber, ob die Cholesky-Zerlegung ohne Zusatztricks durchläuft.

Interaktiv: Welche Richtung widerlegt positive Definitheit?

Eine Richtung kann positive Definitheit widerlegen, aber niemals allein beweisen. Finden wir im folgenden Test die Richtung, die den Gegenbeweis liefert.

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, genügt bei einer nicht-SPD-Matrix ein Vektor mit $\mathbf{x}^\top A \mathbf{x} \leq 0$. Für SPD muss die strikte Ungleichung dagegen für alle Richtungen gelten.

Das Paradebeispiel liefert die Statistik selbst: Jede Kovarianzmatrix $\Sigma = \text{Var}(\mathbf{x})$ ist symmetrisch und positiv semidefinit, denn für jeden festen Vektor $\mathbf{a} \neq \mathbf{0}$ ist $\mathbf{a}^\top \Sigma \mathbf{a} = \text{Var}(\mathbf{a}^\top \mathbf{x}) \geq 0$, eine Varianz. SPD ist Σ genau dann, wenn keine nichttriviale Linearkombination $\mathbf{a}^\top \mathbf{x}$ mit $\mathbf{a} \neq \mathbf{0}$ fast sicher konstant ist, die Verteilung also nicht auf einer Hyperebene zusammenfällt.

SPD-Matrizen in Statistik und ML

Wo immer in Statistik und maschinellem Lernen gerechnet wird, tauchen SPD-Matrizen auf. Drei große Familien:

1. **Kovarianzmatrizen** $\Sigma = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top]$: Sie parametrisieren multivariate Normalverteilungen $\mathcal{N}_p(\boldsymbol{\mu}, \Sigma)$ und stehen im Zentrum der Hauptkomponentenanalyse (PCA).
2. **Gram-Matrizen** $\mathbf{K} = \mathbf{X}^\top \mathbf{X}$, etwa für eine Designmatrix $\mathbf{X}$ in der Regression: Sie bilden die linke Seite der Normalgleichungen $\mathbf{X}^\top \mathbf{X} \boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$ für *Kleinste-Quadrate*-Probleme (Kapitel 7). SPD ist $\mathbf{K}$ genau dann, wenn $\mathbf{X}$ vollen Spaltenrang hat; sonst bleibt nur positive Semidefinitheit. Auch Kernel-Matrizen in Support Vector Machines, Gauß-Prozessen und der nichtparametrischen Glättung gehören in diese Familie.
3. **Hesse-Matrizen** $\mathbf{H} = \left[\frac{\partial^2}{\partial x_i \partial x_j} f(\mathbf{x}) \right]_{i,j}$ (Hesse-Matrix): In der Optimierung charakterisieren sie Konvexität und treiben das Newton-Verfahren und andere Optimierer zweiter Ordnung an. Ihr statistisches Gegenstück ist die Fisher-Information $\mathbb{E} \left[-\frac{\partial^2}{\partial \theta_i \partial \theta_j} \log L(\boldsymbol{\theta}) \right]_{i,j}$ der Log-Likelihood.

Warum lohnt sich diese Klasse für die Numerik? SPD-Matrizen garantieren:

- Existenz der Cholesky-Zerlegung (Satz 5.4.2 unten); legen wir die Diagonale von $\mathbf{L}$ auf positive Werte fest, ist die Zerlegung sogar eindeutig.
- Numerische Stabilität ganz ohne Pivotierung.

- Alle Eigenwerte sind positiv, insbesondere hat A vollen Rang und ist invertierbar.

Der Zerlegungssatz

Bei einer symmetrischen Matrix steckt die gesamte Information schon in einer Dreieckshälfte. Es wäre verschwenderisch, wenn die Zerlegung das ignorieren würde. Tatsächlich können wir für SPD-Matrizen die Faktoren L und U der LU-Zerlegung so wählen, dass $U = L^T$ gilt: Die Symmetrie von A vererbt sich an die Zerlegung.

Satz 5.4.2 (Cholesky-Zerlegung)

Jede SPD-Matrix $A \in \mathbb{R}^{n \times n}$ lässt sich zerlegen als

$$A = LL^T, \quad (5.4.1)$$

wobei L eine untere Dreiecksmatrix ist.

Statt zweier verschiedener Faktoren speichern wir nur noch L , und auch die Rechnung halbiert sich: Die Cholesky-Zerlegung kommt mit rund $n^3/6$ Multiplikationen aus, die volle LU-Zerlegung derselben Matrix braucht rund $n^3/3$ (Abschnitt 5.3). An der Größenordnung $O(n^3)$ ändert das nichts (Abschnitt 2.4), an der Rechenzeit sehr wohl.

Vertiefung: Warum die Zerlegung immer existiert: der Induktionsbeweis

Der Beweis ist eine Induktion über die Dimension n und liefert nebenbei gleich ein Rezept, wie sich L Spalte für Spalte berechnen lässt.

Beweis.

- (1) **Induktionsanfang** $n = 1$: Hier ist $A = (a)$ mit $a \in \mathbb{R}$, und aus der Definitheit folgt $a > 0$. Also leistet $L = (\sqrt{a})$ das Verlangte, denn $LL^T = (\sqrt{a})^2 = a$.

mit dem Testvektor $x = 1 \neq 0$ liefert die Definitheit $xax = a > 0$; die Wurzel existiert also

- (2) **Induktionsschritt** $n > 1$: Die Behauptung gelte für $(n-1) \times (n-1)$ -Matrizen. Wir zerlegen A in Blöcke,

$$A = \begin{pmatrix} a & \mathbf{c}^T \\ \mathbf{c} & \mathbf{B} \end{pmatrix}, \quad a \in \mathbb{R}, \quad \mathbf{c} \in \mathbb{R}^{n-1}, \quad \mathbf{B} \in \mathbb{R}^{(n-1) \times (n-1)},$$

und halten fest, dass $a > 0$ gilt.

$a = \mathbf{e}_1^T A \mathbf{e}_1 > 0$, weil A positiv definit und $\mathbf{e}_1 \neq 0$ ist

- (3) Wir setzen $\mathbf{l} = \mathbf{c}/\sqrt{a}$ und betrachten die $(n-1) \times (n-1)$ -Matrix $\mathbf{B} - \mathbf{l}\mathbf{l}^T = \mathbf{B} - \mathbf{c}\mathbf{c}^T/a$. Sie ist symmetrisch.

$\mathbf{B}$ ist als Diagonalblock der symmetrischen Matrix A selbst symmetrisch, und $(\mathbf{l}\mathbf{l}^T)^T = \mathbf{l}\mathbf{l}^T$

- (4) $\mathbf{B} - \mathbf{l}\mathbf{l}^T$ ist auch positiv definit. Zu beliebigem $\mathbf{x} \in \mathbb{R}^{n-1}$ mit $\mathbf{x} \neq 0$ wählen wir den Testvektor

$$\mathbf{y} = \begin{pmatrix} -\frac{\mathbf{c}^T \mathbf{x}}{a} \\ \mathbf{x} \end{pmatrix} \in \mathbb{R}^n$$

und rechnen zuerst

$$A\mathbf{y} = \begin{pmatrix} a \cdot \left(-\frac{\mathbf{c}^T \mathbf{x}}{a}\right) + \mathbf{c}^T \mathbf{x} \\ \mathbf{c} \cdot \left(-\frac{\mathbf{c}^T \mathbf{x}}{a}\right) + \mathbf{B}\mathbf{x} \end{pmatrix} = \begin{pmatrix} 0 \\ (\mathbf{B} - \frac{\mathbf{c}\mathbf{c}^T}{a})\mathbf{x} \end{pmatrix}.$$

blockweises Ausmultiplizieren; in der ersten Komponente kürzt sich a heraus, es bleibt $-\mathbf{c}^T \mathbf{x} + \mathbf{c}^T \mathbf{x} = 0$

(5) Damit folgt

$$\mathbf{x}^\top \left(\mathbf{B} - \frac{\mathbf{c}\mathbf{c}^\top}{a} \right) \mathbf{x} = \mathbf{y}^\top \mathbf{A} \mathbf{y} > 0.$$

im Produkt $\mathbf{y}^\top (\mathbf{A} \mathbf{y})$ trifft die erste Komponente von $\mathbf{y}$ auf die 0, übrig bleibt genau $\mathbf{x}^\top (\mathbf{B} - \mathbf{c}\mathbf{c}^\top/a) \mathbf{x}$; und $\mathbf{y}^\top \mathbf{A} \mathbf{y} > 0$, weil $\mathbf{A}$ SPD ist und $\mathbf{y} \neq \mathbf{0}$ (schon der untere Block $\mathbf{x}$ ist es)

(6) $\mathbf{B} - \mathbf{I}\mathbf{I}^\top$ ist also eine SPD-Matrix der Dimension $n - 1$, und nach Induktionsvoraussetzung existiert eine untere Dreiecksmatrix $\tilde{\mathbf{L}}$ mit $\mathbf{B} - \mathbf{I}\mathbf{I}^\top = \tilde{\mathbf{L}}\tilde{\mathbf{L}}^\top$. Wir setzen

$$\mathbf{L} = \begin{pmatrix} \sqrt{a} & \mathbf{0}^\top \\ \mathbf{1} & \tilde{\mathbf{L}} \end{pmatrix}$$

und prüfen blockweise nach:

$$\mathbf{L}\mathbf{L}^\top = \begin{pmatrix} a & \sqrt{a}\mathbf{1}^\top \\ \sqrt{a}\mathbf{1} & \mathbf{I}\mathbf{I}^\top + \tilde{\mathbf{L}}\tilde{\mathbf{L}}^\top \end{pmatrix} = \begin{pmatrix} a & \mathbf{c}^\top \\ \mathbf{c} & \mathbf{B} \end{pmatrix} = \mathbf{A}.$$

$\sqrt{a}\mathbf{1} = \mathbf{c}$ nach Definition von $\mathbf{1}$, und $\mathbf{I}\mathbf{I}^\top + \tilde{\mathbf{L}}\tilde{\mathbf{L}}^\top = \mathbf{I}\mathbf{I}^\top + (\mathbf{B} - \mathbf{I}\mathbf{I}^\top) = \mathbf{B}$

□

Achten wir zum Schluss auf den Quantor: Die strikte Ungleichung $\mathbf{x}^\top (\mathbf{B} - \mathbf{I}\mathbf{I}^\top) \mathbf{x} > 0$ gilt natürlich nur für $\mathbf{x} \neq \mathbf{0}$; deshalb haben wir ihn überall mitgeführt.

Die Konstruktion liefert ein $\mathbf{L}$ mit lauter positiven Diagonaleinträgen, denn auf der Diagonalen steht in jedem Schritt eine Wurzel. Mit dieser Normierung ist die Zerlegung dann auch eindeutig: Ohne sie dürften wir in einer beliebigen Spalte von $\mathbf{L}$ alle Vorzeichen umdrehen, ohne $\mathbf{L}\mathbf{L}^\top$ zu verändern.

Cholesky von Hand: Koeffizientenvergleich

Wie kommen wir konkret an $\mathbf{L}$? Für kleine Matrizen genügt ein Koeffizientenvergleich: Wir multiplizieren den Ansatz $\mathbf{L}\mathbf{L}^\top$ aus und vergleichen Eintrag für Eintrag mit $\mathbf{A}$.

Beispiel 5.4.3 (Cholesky-Zerlegung einer 2×2-Matrix)

Wir berechnen die Cholesky-Zerlegung von $\mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$.

Ansatz: $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ mit $\mathbf{L} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$.

Ausmultiplizieren:

$$\mathbf{L}\mathbf{L}^\top = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix} \stackrel{!}{=} \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}.$$

Koeffizientenvergleich, von links oben nach rechts unten (der jeweils verglichene Eintrag von $\mathbf{A}$ rot, fertig berechnete Einträge von $\mathbf{L}$ grün):

$$\begin{aligned} l_{11}^2 &= 4 &\implies l_{11} &= 2, \\ l_{11}l_{21} &= 2 &\implies l_{21} &= 2/2 = 1, \\ l_{21}^2 + l_{22}^2 &= 3 &\implies l_{22} &= \sqrt{3-1} = \sqrt{2}. \end{aligned}$$

Ergebnis: $L = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix}$, und die Probe bestätigt $LL^T = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix} = A$.

Jede Gleichung enthielt genau eine neue Unbekannte; in dieser Reihenfolge löst sich das System also von selbst auf. Und dass unter der Wurzel mit $3 - 1 = 2 > 0$ etwas Positives stand, war kein Glück: Der Beweis von Satz 5.4.2 zeigt, dass das für SPD-Matrizen immer so ausgeht.

Für allgemeines n liefert derselbe Vergleich das Rezept, das der Beweis schon angedeutet hat. Der Eintrag (i, j) von LL^T ist $\sum_{k=1}^n l_{ik}l_{jk}$, und weil L untere Dreiecksmatrix ist, brechen die Summen früh ab. Wir lösen Spalte für Spalte auf, jeweils nach dem einzigen noch unbekannten Eintrag:

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}, \quad l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik}l_{jk} \right) \quad (i > j). \quad (5.4.2)$$

Auf der Diagonalen steht also eine Wurzel, darunter eine Division durch den zuletzt berechneten Diagonaleintrag.

Interaktiv: Cholesky-Zerlegung Schritt für Schritt

Ab 3×3 entsteht pro Eintrag ein Rechenschritt. In welcher Reihenfolge füllt sich L ?

Interaktives Element — nur in der Web-Fassung

Der Stepper macht die spaltenweise Reihenfolge sichtbar: Auf einen Diagonaleintrag folgen die Einträge darunter.

Anwendung: korrelierte Zufallsvektoren simulieren

Zufallszahlengeneratoren liefern uns unabhängige Standardnormalvariablen, also Vektoren $\mathbf{x}$ mit $\mathbb{E}[\mathbf{x}] = \mathbf{0}$ und $\text{Var}(\mathbf{x}) = \mathbf{I}_d$. Gebraucht werden in der Statistik aber meist *korrelierte* Ziehungen mit vorgegebener Kovarianzmatrix Σ . Der Cholesky-Faktor schlägt die Brücke: Als lineare Abbildung verformt er unkorreliertes Rauschen in genau die gewünschte Abhängigkeitsstruktur. Nachrechnen lässt sich das mit zwei Eigenschaften des Erwartungswerts: Er ist linear, und konstante Matrizen dürfen wir aus ihm herausziehen.

Satz 5.4.4 (Kovarianz unter dem Cholesky-Faktor)

Sei $\mathbf{x} \in \mathbb{R}^d$ ein Zufallsvektor mit $\mathbb{E}[\mathbf{x}] = \mathbf{0}$ und $\text{Var}(\mathbf{x}) = \mathbf{I}_d$, und sei $\Sigma \in \mathbb{R}^{d \times d}$ SPD mit Cholesky-Zerlegung $\Sigma = LL^T$. Dann hat $\mathbf{y} = L\mathbf{x}$ den Erwartungswert $\mathbf{0}$ und die Kovarianzmatrix

$$\text{Var}(\mathbf{y}) = \Sigma.$$

Beweis.

(1) Wegen $\mathbb{E}[\mathbf{y}] = L\mathbb{E}[\mathbf{x}] = \mathbf{0}$ ist $\text{Var}(\mathbf{y}) = \mathbb{E}[\mathbf{y}\mathbf{y}^T]$, und damit

$$\text{Var}(\mathbf{y}) = \mathbb{E}[L\mathbf{x}\mathbf{x}^T L^T] = L\mathbb{E}[\mathbf{x}\mathbf{x}^T] L^T = L\mathbf{I}_d L^T = \Sigma.$$

der Erwartungswert ist linear und $\mathbf{L}$ eine feste, nicht zufällige Matrix, darf also aus ihm herausgezogen werden; für zentrierte Vektoren ist $\text{Var}(\mathbf{y}) = \mathbb{E}[\mathbf{y}\mathbf{y}^\top]$, insbesondere $\mathbb{E}[\mathbf{x}\mathbf{x}^\top] = \text{Var}(\mathbf{x}) = \mathbf{I}_d$; zuletzt die Zerlegung (5.4.1)

□

Bemerkung 5.4.5 (Ziehen aus der multivariaten Normalverteilung)

So simulieren wir aus $\mathcal{N}(\boldsymbol{\mu}, \Sigma)$: Wir erzeugen $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d)$ aus d unabhängigen Standardnormalvariablen, berechnen einmal den Cholesky-Faktor $\mathbf{L}$ von Σ und geben $\boldsymbol{\mu} + \mathbf{L}\mathbf{z}$ zurück. Als lineare Transformation eines normalverteilten Vektors ist $\boldsymbol{\mu} + \mathbf{L}\mathbf{z}$ wieder normalverteilt, nach Satz 5.4.4 mit Erwartungswert $\boldsymbol{\mu}$ und Kovarianz Σ . Die Zerlegung bezahlen wir dabei nur einmal; jede weitere Ziehung kostet nur noch ein Matrix-Vektor-Produkt.

Interaktiv: Wie $\mathbf{L}$ eine runde Punktwolke verformt

Welche Veränderung der Wolke erwarten wir, wenn $|\rho|$ fast eins wird?

Im zweidimensionalen Fall lässt sich Satz 5.4.4 direkt ansehen. Aus σ_1, σ_2 und der Korrelation ρ bauen wir

$$\Sigma = \begin{pmatrix} \sigma_1^2 & \rho \sigma_1 \sigma_2 \\ \rho \sigma_1 \sigma_2 & \sigma_2^2 \end{pmatrix},$$

und (5.4.2) liefert den Cholesky-Faktor in geschlossener Form:

$$\mathbf{L} = \begin{pmatrix} \sigma_1 & 0 \\ \rho \sigma_2 & \sigma_2 \sqrt{1 - \rho^2} \end{pmatrix}.$$

Interaktives Element — nur in der Web-Fassung

$\mathbf{L}$ streckt die erste Koordinate mit σ_1 und mischt der zweiten über $\rho \sigma_2$ einen Anteil davon bei: Aus der runden Wolke wird eine gestreckte, für $\rho \neq 0$ geneigte Ellipse. Läuft $|\rho|$ gegen eins, geht der Eintrag $\sigma_2 \sqrt{1 - \rho^2}$ gegen null, die Wolke fällt auf eine Gerade zusammen und Σ wird singulär.

Vertiefung: Cholesky mit Pivotierung: semidefinite und rangdefiziente Matrizen

Was passiert, wenn $\mathbf{A}$ nur positiv *semidefinit* ist (also $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ statt > 0), etwa weil $\mathbf{A}$ singulär ist? Oder wenn $\mathbf{A}$ zwar SPD, aber so schlecht konditioniert ist, dass Rundungsfehler den Unterschied verwischen? Dann läuft der Koeffizientenvergleich irgendwann auf eine Wurzel aus null oder sogar aus einer negativen Zahl, und die gewohnte Rechnung bricht zusammen. Die Rettung kennen wir sinngemäß schon von der LU-Zerlegung: Pivotierung. Diesmal vertauschen wir Zeilen *und* Spalten symmetrisch (*complete pivoting*), damit die Symmetrie erhalten bleibt:

$$\mathbf{P}^\top \mathbf{A} \mathbf{P} = \mathbf{L} \mathbf{L}^\top$$

mit einer Permutationsmatrix $\mathbf{P}$. Diese Variante hat bemerkenswerte Eigenschaften:

- **Rang-aufdeckend:** Nullspalten in $\mathbf{L}$ zeigen direkt den Rangabfall von $\mathbf{A}$ an; die Zerlegung bestimmt also nebenbei den Rang der Matrix.
- **Stabil:** Auch für semidefinite Matrizen bleibt die pivotierte Cholesky-Zerlegung numerisch stabil (Higham, 1990).
- **Praktisch relevant** in Statistik und ML: für Kernel-Matrizen (Gauß-Prozesse, SVMs), für niedrig-rangige approximative Zerlegungen mit Aufwand $O(k^2 n)$ statt $O(n^3)$ bei Abbruch nach

k Spalten, und für Gram-Matrizen $\mathbf{X}^\top \mathbf{X}$ im Fall $p > n$ (mehr Merkmale als Beobachtungen), die zwangsläufig singulär sind.

Vertiefung: Heath §2.5 (spezielle lineare Systeme, insbesondere symmetrische und positiv definite Matrizen).

5.5 Zusammenfassung

Zum Abschluss stellen wir die beiden Zerlegungen dieses Kapitels nebeneinander. Beide lösen $\mathbf{Ax} = \mathbf{b}$ nach demselben Plan: einmal zerlegen, dann pro rechter Seite zwei Dreieckssysteme substituieren. Die Unterschiede liegen in den Voraussetzungen, im Aufwand und in der Stabilität.

LU und Cholesky im Vergleich

	LU-Zerlegung	Cholesky-Zerlegung
Zerlegung	$\mathbf{PA} = \mathbf{LU}$	$\mathbf{A} = \mathbf{LL}^\top$
geeignet für	jede invertierbare Matrix	SPD-Matrizen
Pivotierung	für die Stabilität nötig	nicht nötig
Aufwand	rund $n^3/3$ Multiplikationen	rund $n^3/6$, also halb so viel
scheitert	ohne Pivotierung an Nullpivots; kleine Pivots machen sie instabil	in exakter Arithmetik bei nicht-SPD-Matrizen; numerisch eventuell nahe der Semidefinitheitsgrenze

Die Stabilität der LU-Zerlegung entscheidet sich am Pivot (Abschnitt 5.3): Partielle Pivotierung hält alle Multiplikatoren im Betrag bei höchstens 1. Ein Stabilitätsbeweis ist das nicht, aber in der Praxis gilt die pivotierte LU-Zerlegung als stabil (Higham, 2020). Perfekte Genauigkeit bekommen wir auch damit nicht: Selbst ein stabiler Algorithmus liefert nur eine Näherung, deren relativer Fehler in der Größenordnung $\kappa(\mathbf{A}) \cdot \varepsilon$ liegen kann, mit der Konditionszahl $\kappa(\mathbf{A})$ aus Abschnitt 3.5 und der Maschinengenauigkeit ε . Ist $\mathbf{A}$ schlecht konditioniert, liegt das am Problem, nicht am Algorithmus.

Die Cholesky-Zerlegung braucht für eine SPD-Matrix keine Pivotierung und ist rückwärtsstabil. In exakter Arithmetik läuft sie für jede SPD-Matrix durch und scheitert für Matrizen, die nicht positiv definit sind. In Gleitkommaarithmetik kann eine Matrix nahe der Grenze zur Semidefinitheit wegen Daten- oder Rundungsfehlern als nicht positiv definit erscheinen. Ein Abbruch ist daher ein Diagnosehinweis, aber kein Beweis dafür, dass allein eine große Konditionszahl die Ursache war. Für nur *semidefinite* Matrizen springt die pivotierte Variante $\mathbf{P}^\top \mathbf{AP} = \mathbf{LL}^\top$ ein (Abschnitt 5.4).

Was wir mitnehmen

Drei Botschaften bleiben:

- **Für ein LGS keine explizite Inverse bilden.** Wo in einer Formel $\mathbf{A}^{-1}\mathbf{b}$ steht, lösen wir in der Implementierung direkt das LGS $\mathbf{Ax} = \mathbf{b}$ (Bemerkung 5.2.1 in Abschnitt 5.2).
- Die **LU-Zerlegung** $\mathbf{PA} = \mathbf{LU}$ ist die Gauß-Elimination in Matrixform: einmal zerlegen für $O(n^3)$, danach kostet jede weitere rechte Seite nur noch $O(n^2)$.

- Die **Cholesky-Zerlegung** $A = LL^T$ nutzt Symmetrie und Definitheit aus, halbiert den Aufwand und liefert nebenbei das Werkzeug, mit dem wir korrelierte Zufallsvektoren simulieren (Abschnitt 5.4). Kovarianz- und Gram-Matrizen tragen diese Struktur von Haus aus, Hesse-Matrizen nur unter zusätzlichen Krümmungsannahmen.

In Kapitel 6 lernen wir mit der Singulärwertzerlegung (SVD) die vielseitigste aller Matrixzerlegungen kennen; sie funktioniert für beliebige, auch nicht quadratische Matrizen.

Selbsttest

Prüfen wir die Kernaussagen des Kapitels.

1. Um $Ax = b$ numerisch zu lösen, berechnen wir am besten zuerst A^{-1} und bilden dann das Produkt $A^{-1}b$.

Antwort: Falsch

Das explizite Invertieren löst versteckt n Gleichungssysteme statt einem. Die Größenordnung $O(n^3)$ bleibt dabei zwar erhalten, weil eine Zerlegung für alle n rechten Seiten reicht, doch der Umweg kostet ein Mehrfaches an Rechenzeit und stapelt zusätzliche Rundungsfehler auf. Wir lösen stattdessen direkt das LGS (Bemerkung 5.2.1).

2. Liegt $PA = LU$ einmal vor, kostet jede weitere rechte Seite b nur noch $O(n^2)$ Operationen.

Antwort: Wahr

Pro rechter Seite fallen eine Vorwärts- und eine Rückwärtssubstitution an, zusammen rund n^2 Multiplikationen (Satz 5.3.9); die teure Zerlegung hängt nicht von b ab.

3. Jede invertierbare Matrix A besitzt eine Zerlegung $A = LU$.

Antwort: Falsch

Schon die Vertauschungsmatrix $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ startet mit dem Pivot 0 und hat keine LU-Zerlegung (Beispiel 5.3.6). Mit Zeilenvertauschungen existiert aber immer $PA = LU$ (Satz 5.3.5).

4. Eine einzige Richtung $x \neq 0$ mit $x^T Ax \leq 0$ widerlegt, dass A positiv definit ist.

Antwort: Wahr

Definition 5.4.1 verlangt strikte Positivität für *alle* von null verschiedenen Richtungen; ein Gegenbeispiel genügt daher zum Widerlegen. Umgekehrt beweist keine noch so lange Liste bestandener Richtungen die Definitheit.

5. Für eine SPD-Matrix ist die Cholesky-Zerlegung etwa doppelt so schnell wie die LU-Zerlegung und kommt ohne Pivotierung aus.

Antwort: Wahr

Wegen $U = L^T$ muss nur ein Faktor berechnet und gespeichert werden, rund $n^3/6$ statt $n^3/3$ Multiplikationen; und die positive Definitheit garantiert positive Pivots, sodass kein Tauschen nötig ist (Abschnitt 5.4).

Vertiefung: Heath §2.4 (LU-Zerlegung, Pivotierung, Stabilität) und §2.5 (SPD-Systeme und Cholesky).

Die Singulärwertzerlegung

6.1 Motivation

In Kapitel 5 haben wir Matrizen zerlegt, um Gleichungssysteme zu lösen. Die LU-Zerlegung braucht dafür eine quadratische Matrix, die Cholesky-Zerlegung sogar eine symmetrische und positiv definite. Die Matrizen, mit denen wir in der Statistik tatsächlich rechnen, sehen selten so aus: Eine Designmatrix hat n Zeilen für die Beobachtungen und p Spalten für die Merkmale, und $n = p$ ist der Ausnahmefall. Dieses Kapitel baut eine Zerlegung, die für jede Matrix funktioniert, ganz gleich welches Format sie hat.

Was wir brauchen

Sammeln wir zuerst das Vorwissen ein, auf dem das Kapitel steht.

- Die *Spektralnorm* $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$, also der größte Streckfaktor der Abbildung $x \mapsto Ax$ (Abschnitt 3.3).
- Orthogonalmatrizen Q mit $Q^T Q = I$. Sie erhalten Längen und Winkel, ihre Inverse ist die Transponierte (Abschnitt 3.3).
- Matrixzerlegungen als Werkzeug: eine Matrix in leicht handhabbare Faktoren aufspalten und die eigentliche Rechnung dann auf den Faktoren machen (Abschnitt 5.2).
- Aus der linearen Algebra:
 - Eigenwerte und Eigenvektoren, $Av = \lambda v$;
 - der Rang einer Matrix;
 - die fundamentalen Unterräume $\text{col}(A)$, $\text{Kern}(A)$ und $\text{col}(A^T)$ (Spaltenraum und Kern);
 - Projektionen auf Unterräume.

Diagonalisierung ist limitiert

Warum brauchen wir überhaupt etwas Neues? Die Eigenwertzerlegung $A = PDP^{-1}$ ist das Standardwerkzeug, um eine Matrix in ihre Wirkungsrichtungen zu zerlegen. Nur ist sie doppelt eingeschränkt.

Erstens verlangt schon die Gleichung $Av = \lambda v$, dass Av und v im selben Raum liegen. Die Matrix muss also quadratisch sein; für $A \in \mathbb{R}^{m \times n}$ mit $m \neq n$ stellt sich die Frage nach Eigenwerten gar nicht erst. Und selbst unter den quadratischen Matrizen hat nicht jede eine Eigenwertzerlegung: $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ besitzt nur den Eigenwert 0, dessen Eigenraum eindimensional ist. Eine Basis aus Eigenvektoren kommt so nicht zustande.

Zweitens nützt uns die Zerlegung vor allem dann etwas, wenn P orthogonal gewählt werden kann,

denn nur dann verzerrt der Basiswechsel nichts. Das gelingt genau für symmetrische Matrizen (Spektralsatz, Abschnitt 3.3). Ein Blick auf die Rechenregel $(\mathbf{PDP}^\top)^\top = \mathbf{PD}^\top \mathbf{P}^\top = \mathbf{PDP}^\top$ zeigt auch, warum: Was so zerlegbar ist, ist automatisch symmetrisch.

Bemerkung 6.1.1 (Wo die Diagonalisierung aufhört)

- Die Eigenwertzerlegung setzt eine **quadratische** Matrix voraus, und auch dann gibt es sie nicht immer.
- Orthogonal diagonalisierbar, also $\mathbf{A} = \mathbf{PDP}^\top$ mit orthogonalem $\mathbf{P}$, sind nur die **symmetrischen** Matrizen.
- Die meisten Matrizen, die uns begegnen, sind weder das eine noch das andere.

Was wir wollen

Gesucht ist eine Zerlegung, die dieselbe Arbeit leistet, aber ohne diese Bedingungen:

$$\mathbf{A} = \mathbf{PDQ}^{-1}. \quad (6.1.1)$$

Drei Dinge wünschen wir uns dabei:

1. Sie existiert für *beliebige* Matrizen $\mathbf{A} \in \mathbb{R}^{m \times n}$.
2. $\mathbf{D}$ ist „diagonal“, trägt also außerhalb der Diagonalen nur Nullen. Bei einer rechteckigen Matrix müssen wir dafür genauer sagen, was Diagonale heißen soll; das holen wir nach, sobald wir die Bausteine haben.
3. $\mathbf{P}$ und $\mathbf{Q}$ sind Orthogonalmatrizen.

Punkt 3 ist der eigentliche Anspruch. Orthogonalmatrizen drehen und spiegeln, mehr nicht. Steckt die ganze Verzerrung in $\mathbf{D}$, dann lesen wir an den Diagonaleinträgen direkt ab, was die Abbildung mit dem Raum anstellt. Nebenbei wird die Zerlegung dadurch bequem: Für orthogonales $\mathbf{Q}$ ist $\mathbf{Q}^{-1} = \mathbf{Q}^\top$, aus (6.1.1) wird also $\mathbf{A} = \mathbf{PDQ}^\top$, und niemand muss eine Inverse ausrechnen.

Die Namen der Faktoren wechseln später noch. Sobald wir wissen, woraus sie bestehen, heißt die Zerlegung $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ und trägt den Namen Singulärwertzerlegung. Die Farben bleiben uns im ganzen Kapitel erhalten: orange die Streckfaktoren, blau die rechte Seite $\mathbf{V}$, grün die linke Seite $\mathbf{U}$.

Geometrische Intuition

Woher soll so eine Zerlegung kommen? Die Antwort steckt in der Geometrie. Statt einen einzelnen Vektor durch die Abbildung zu schicken, schicken wir alle Einheitsvektoren auf einmal.

Betrachten wir also die Einheitssphäre, alle $\mathbf{x}$ mit $\|\mathbf{x}\| = 1$. Ihre Bilder $\mathbf{Ax}$ haben ganz verschiedene Längen: Manche Richtungen streckt $\mathbf{A}$, andere staucht sie. Die Frage, die uns durch dieses Kapitel trägt, lautet: Welche Richtung wird am stärksten gestreckt, welche am zweitstärksten, und so weiter?

Für eine 2×2 -Matrix wird aus dem Einheitskreis eine Ellipse; ist $\mathbf{A}$ singulär, so entartet sie zu einer Strecke. Der längste Bildvektor liegt auf ihrer Hauptachse, der kürzeste auf der Nebenachse, und diese Halbachsenlängen sind uns als Singulärwerte in Abschnitt 3.4 schon begegnet. Bildet

A dagegen von $\mathbb{R}^3$ in die Ebene ab, so wird eine ganze Richtung platt gedrückt, und aus der Einheitssphäre wird eine ausgefüllte Ellipse. An den Hauptachsen ändert das nichts.

Bleiben wir konkret und nehmen wir die Matrix aus Beispiel 3.3.3,

$$A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}.$$

Interaktiv: Welche Richtung streckt die Matrix am stärksten?

Welche Richtung streckt sie am stärksten, und um welchen Faktor? Das Widget schickt den ganzen Einheitskreis durch A und trägt rechts die Länge des Bildvektors über dem Winkel auf. Wir bleiben dabei beim 2×2 -Fall, in dem Urbild und Bild in derselben Ebene liegen; bildet A zwischen Räumen verschiedener Dimension ab, ändert das an der Aussage nichts.

Interaktives Element — nur in der Web-Fassung

Beispiel 6.1.2 (Der Einheitskreis wird zur Ellipse)

Aus Kapitel 3 kennen wir die beiden Streckfaktoren von A bereits: $2,288 = \sqrt{3 + \sqrt{5}}$ ist die längste Halbachse der Bildellipse, $0,874$ die kürzeste. Am stärksten gestreckt wird die Richtung

$$\mathbf{x}^* \approx \begin{pmatrix} 0,851 \\ 0,526 \end{pmatrix}, \quad A\mathbf{x}^* \approx \begin{pmatrix} 2,227 \\ 0,526 \end{pmatrix}, \quad \|A\mathbf{x}^*\| \approx 2,288,$$

ein Einheitsvektor, der rund $31,7^\circ$ über der x_1 -Achse liegt. Die dazu senkrechte Richtung $\mathbf{x}^\perp \approx (-0,526, 0,851)^\top$ hat das Bild $A\mathbf{x}^\perp \approx (-0,201, 0,851)^\top$ der Länge $0,874$, wird also am schwächsten gestreckt.

Nun die Beobachtung, auf der das ganze Kapitel ruht: Die beiden Bilder stehen wieder senkrecht aufeinander,

$$A\mathbf{x}^{*\top} A\mathbf{x}^\perp \approx 2,227 \cdot (-0,201) + 0,526 \cdot 0,851 = -0,4476 + 0,4476 = 0$$

(bis auf Rundung). Zwei senkrechte Richtungen im Urbild, zwei senkrechte Richtungen im Bild, dazwischen nur zwei Streckfaktoren: Dieses Muster schreiben wir in Abschnitt 6.2 als Zerlegung auf. Woher $\mathbf{x}^*$ kommt und warum die Bilder senkrecht bleiben, klärt sich dort ebenfalls.

Die Grundidee

Wie finden wir die Richtung der stärksten Streckung? Wir schreiben die Frage als Optimierungsproblem:

$$\max_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|. \quad (6.1.2)$$

Ein Maximum gibt es tatsächlich, denn die Einheitssphäre ist abgeschlossen und beschränkt und $\mathbf{x} \mapsto \|A\mathbf{x}\|$ ist stetig. Der Maximalwert steht sogar schon in unserer Vorwissensliste: Er ist die Spektralnorm $\|A\|_2$, die Abschnitt 3.3 als größten Streckfaktor einführt. Dort haben wir nach dem

Wert gefragt, hier fragen wir nach der Maximalstelle, und gleich danach nach dem, was in den übrigen Richtungen passiert.

Unhandlich an (6.1.2) ist die Zielfunktion: In der Norm steckt eine Wurzel über einer Quadratsumme. Der Ausweg ist ein alter Trick. Wir maximieren stattdessen das Quadrat.

Satz 6.1.3 (Streckung als quadratische Form)

Für $A \in \mathbb{R}^{m \times n}$ und alle $x \in \mathbb{R}^n$ gilt

$$\|Ax\|^2 = (Ax)^\top (Ax) = x^\top (A^\top A) x. \quad (6.1.3)$$

Auf der Einheitssphäre haben $\|Ax\|$ und $\|Ax\|^2$ dieselben Maximalstellen.

Vertiefung: Die Identität im Detail

Beweis.

- (1) Für jeden Vektor $y \in \mathbb{R}^m$ ist $\|y\|^2 = y^\top y$. Mit $y = Ax$ folgt die erste Gleichheit in (6.1.3).

Definition der euklidischen Norm: $\|y\| = \sqrt{\sum_i y_i^2}$, und das Skalarprodukt $y^\top y$ ist genau $\sum_i y_i^2$

- (2) Ausmultiplizieren liefert die zweite Gleichheit:

$$(Ax)^\top (Ax) = x^\top A^\top Ax = x^\top (A^\top A) x.$$

$(Ax)^\top = x^\top A^\top$, die Transposition dreht die Reihenfolge des Produkts um; danach klammern wir nur um

- (3) Die Abbildung $t \mapsto t^2$ wächst auf $[0, \infty)$ streng monoton, und Normen sind nie negativ. Ein x maximiert daher $\|Ax\|$ genau dann, wenn es $\|Ax\|^2$ maximiert.

strenge Monotonie erhält die Ordnung: $\|Ax_1\| \leq \|Ax_2\|$ gilt genau dann, wenn $\|Ax_1\|^2 \leq \|Ax_2\|^2$ gilt

□

Die rechte Seite von (6.1.3) ist eine quadratische Form, und zwar die zur Matrix $A^\top A$. Damit hat sich die Lage grundlegend geändert. Ist A vom Format $m \times n$, so ist $A^\top A$ vom Format $n \times n$, also quadratisch, unabhängig davon, ob A mehr Zeilen oder mehr Spalten hat. Symmetrisch ist $A^\top A$ außerdem, das rechnet Abschnitt 6.2 nach. Der Umweg über $A^\top A$ führt uns damit zurück in die Klasse, für die die orthogonale Diagonalisierung funktioniert. Aus ihren Eigenwerten und Eigenvektoren werden dort die Singulärwerte und die Singulärvektoren.

Bemerkung 6.1.4 (Notation)

Ab hier ist $\|\cdot\|$ ohne Index immer die euklidische Norm $\|\cdot\|_2$, bei Matrizen entsprechend die davon induzierte Operatornorm, also die Spektralnorm (Abschnitt 3.3). Ist eine andere Norm gemeint, schreiben wir den Index dazu.

Selbsttest

Prüfen wir, ob die Motivation sitzt.

1. Stellen wir im Widget oben das Preset „Strecken und Stauchen“ ein, also $A = \begin{pmatrix} 2 & 0 \\ 0 & 0,5 \end{pmatrix}$, und schieben wir θ auf 45° . Wie lang ist dann der Bildvektor $\|Ax\|$?

Lösung: 1,458

Rund 1,458. Der Einheitsvektor ist $x = (0,7071, 0,7071)^\top$, sein Bild $(1,4142, 0,3536)^\top$, und

dessen Länge ist $\sqrt{2+0,125} = \sqrt{2,125} \approx 1,458$. Der Wert liegt zwischen 0,5 und 2, den beiden Streckfaktoren, und das ist kein Zufall: Auf dem Einheitskreis liegt $\|Ax\|$ immer zwischen kleinstem und größtem Singulärwert.

2. Für $A \in \mathbb{R}^{3 \times 2}$ ist die Frage nach Eigenwerten sinnlos, die Matrix $A^T A$ dagegen ist ein 2×2 -Objekt.

Antwort: Wahr

A bildet von $\mathbb{R}^2$ nach $\mathbb{R}^3$ ab; in $Av = \lambda v$ stünde links ein Vektor aus $\mathbb{R}^3$ und rechts einer aus $\mathbb{R}^2$. Das Produkt $A^T A$ ist dagegen $(2 \times 3) \cdot (3 \times 2)$, also eine quadratische 2×2 -Matrix. Diese Beobachtung ist der Grund für den Umweg aus Satz 6.1.3.

3. Jede quadratische Matrix lässt sich orthogonal diagonalisieren, also als $A = PDP^T$ mit orthogonalem P schreiben.

Antwort: Falsch

Aus $A = PDP^T$ folgt $A^T = PD^T P^T = PDP^T = A$. So zerlegbar sind also nur symmetrische Matrizen. Die quadratische Matrix $A = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$ aus Beispiel 6.1.2 ist nicht symmetrisch und fällt damit heraus.

4. Wenn $\det A = 1$ gilt, wird jede Richtung des Einheitskreises gleich stark gestreckt.

Antwort: Falsch

Die Determinante misst nur die Flächenänderung insgesamt, nicht ihre Verteilung auf die Richtungen. Für $A = \begin{pmatrix} 2 & 0 \\ 0 & 0,5 \end{pmatrix}$ ist $\det A = 1$, trotzdem wird die x_1 -Richtung um den Faktor 2 gestreckt und die x_2 -Richtung auf die Hälfte gestaucht. Gleich behandelt werden alle Richtungen nur von einem Vielfachen einer Orthogonalmatrix, also von $A = cQ$ mit $Q^T Q = I$; dann ist $\|Ax\| = |c|$ für jeden Einheitsvektor x .

5. Der Übergang von $\|Ax\|$ zu $\|Ax\|^2$ ändert weder die Maximalstelle noch den Maximalwert.

Antwort: Falsch

Die Maximalstelle bleibt, der Wert nicht: Er wird quadriert. Für die Matrix aus Beispiel 6.1.2 ist $\max_{\|x\|=1} \|Ax\| \approx 2,288$, während $\max_{\|x\|=1} \|Ax\|^2 = 3 + \sqrt{5} \approx 5,236$ herauskommt. Für die Suche nach der Richtung spielt das keine Rolle, und deshalb dürfen wir quadrieren.

Vertiefung: vgl. Heath §3.6 (Singulärwertzerlegung) sowie MML §4.5 und §4.6.

6.2 Singulärwerte und Singulärvektoren

In Abschnitt 6.1 haben wir die Frage nach der Richtung stärkster Streckung auf ein Maximierungsproblem über der Einheitssphäre zurückgeführt. Der entscheidende Kniff war, statt $\|Ax\|$ das Quadrat zu maximieren:

$$\|Ax\|^2 = (Ax)^T (Ax) = x^T (A^T A) x. \quad (6.2.1)$$

Rechts steht eine quadratische Form in $A^T A$. Diese eine Matrix trägt also die komplette Information darüber, wie stark A in welche Richtung streckt. Bevor wir daraus Singulärwerte gewinnen, sollten wir sie uns genauer ansehen. (Wie im ganzen Kapitel ist $\|\cdot\|$ dabei die euklidische Norm $\|\cdot\|_2$.)

Die Matrix $A^T A$

Zwei Eigenschaften machen $A^T A$ so angenehm, und beide fallen fast von selbst ab.

Satz 6.2.1 (Eigenschaften von $A^T A$)

Für $A \in \mathbb{R}^{m \times n}$ gilt:

1. $A^T A \in \mathbb{R}^{n \times n}$ ist symmetrisch, das heißt $(A^T A)^T = A^T A$.
2. $A^T A$ ist *positiv semidefinit*, das heißt $x^T (A^T A) x \geq 0$ für alle $x \in \mathbb{R}^n$. Insbesondere sind alle Eigenwerte λ_i von $A^T A$ nichtnegativ.

Vertiefung: Beweis der Eigenschaften von $A^T A$

Beweis.

- (1) Zu (1): Transponieren dreht die Reihenfolge eines Produkts um und hebt sich selbst auf, also

$$(A^T A)^T = A^T (A^T)^T = A^T A.$$

$$(XY)^T = Y^T X^T \text{ und } (X^T)^T = X$$

- (2) Zu (2): Die quadratische Form ist nach (6.2.1) ein Längenquadrat, und Längen werden nie negativ:

$$x^T (A^T A) x = \|Ax\|^2 \geq 0 \quad \text{für alle } x \in \mathbb{R}^n.$$

(6.2.1), von rechts nach links gelesen

- (3) Für die Eigenwerte setzen wir speziell $x = v_i$ ein, wobei v_i ein auf Länge 1 normierter Eigenvektor von $A^T A$ zum Eigenwert λ_i sei:

$$0 \leq \|Av_i\|^2 = v_i^T (A^T A) v_i = v_i^T (\lambda_i v_i) = \lambda_i \|v_i\|^2 = \lambda_i.$$

Eigenwertgleichung $A^T A v_i = \lambda_i v_i$, dann $\|v_i\| = 1$; reelle Eigenwerte mit reellen Eigenvektoren gibt es dank des Spektralsatzes, siehe direkt im Anschluss

□

Dass die Eigenwerte überhaupt reell sind und dass es eine Orthonormalbasis aus Eigenvektoren gibt, garantiert der Spektralsatz für symmetrische Matrizen. Zusammen mit Satz 6.2.1 ergibt das:

Korollar 6.2.2 ($A^T A$ ist orthogonal diagonalisierbar)

Zu $A \in \mathbb{R}^{m \times n}$ gibt es eine Orthogonalmatrix $V \in \mathbb{R}^{n \times n}$ und eine Diagonalmatrix $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ mit $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$, sodass

$$A^T A = V \Lambda V^T.$$

Die Spalten von V sind Eigenvektoren von $A^T A$ und bilden eine Orthonormalbasis des $\mathbb{R}^n$.

Damit liegt das halbe Rüstzeug für die Singulärwertzerlegung schon bereit: eine orthonormale Basis, in der $A^T A$ nichts weiter tut als achsenweises Skalieren. Rechnen wir das an einer kleinen Matrix durch, die uns durch den ganzen Abschnitt begleitet.

Beispiel 6.2.3 (Die Matrix $A^T A$ einer 3×2 -Matrix)

Sei

$$A = (\mathbf{a}_1 \quad \mathbf{a}_2) = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \in \mathbb{R}^{3 \times 2}.$$

Der Eintrag (i, j) von $A^T A$ ist das Skalarprodukt der i -ten mit der j -ten Spalte von A :

$$A^T A = \begin{pmatrix} \mathbf{a}_1^T \mathbf{a}_1 & \mathbf{a}_1^T \mathbf{a}_2 \\ \mathbf{a}_2^T \mathbf{a}_1 & \mathbf{a}_2^T \mathbf{a}_2 \end{pmatrix} = \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix}.$$

Die Symmetrie sieht man der Matrix an. Die Eigenwerte liefert das charakteristische Polynom:

$$\det(A^T A - \lambda I) = (6 - \lambda)(5 - \lambda) - 16 = \lambda^2 - 11\lambda + 14 \stackrel{!}{=} 0,$$

also

$$\lambda_{1,2} = \frac{11 \pm \sqrt{11^2 - 4 \cdot 14}}{2} = \frac{11 \pm \sqrt{65}}{2}, \quad \lambda_1 \approx 9,531, \quad \lambda_2 \approx 1,469.$$

Satz 6.2.1 verspricht nur $\lambda_i \geq 0$. Hier sind beide Eigenwerte sogar echt positiv, $A^T A$ ist also positiv definit. Der Grund: Die beiden Spalten von A sind linear unabhängig, $A\mathbf{x} = \mathbf{0}$ gilt somit nur für $\mathbf{x} = \mathbf{0}$, und ein Eigenwert 0 hätte nach der Rechnung oben einen Eigenvektor mit $A\mathbf{v} = \mathbf{0}$ verlangt. Zwei schnelle Proben: Die Summe der Eigenwerte ist 11, die Spur von $A^T A$; ihr Produkt ist 14, die Determinante $6 \cdot 5 - 4^2$.

Singulärwerte

Die Eigenwerte von $A^T A$ sind nach (6.2.1) quadrierte Längen. Wer an Streckungsfaktoren interessiert ist, zieht deshalb die Wurzel.

Definition 6.2.4 (Singulärwerte)

Für $A \in \mathbb{R}^{m \times n}$ heißen die $q = \min(m, n)$ Zahlen

$$\sigma_1 \geq \dots \geq \sigma_q \geq 0$$

die *Singulärwerte* (singular values) von A . Die positiven unter ihnen sind die Quadratwurzeln der positiven Eigenwerte von $A^T A$; bis zur Länge q wird die Liste mit Nullen ergänzt.

Bemerkung 6.2.5 (Singulärwerte sind Streckungsfaktoren)

- $\sigma_1 = \max_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|$ ist die maximale Streckung. Das ist genau das Maximierungsproblem aus Abschnitt 6.1: Über (6.2.1) maximieren wir die quadratische Form von $A^T A$, und deren größter Wert auf der Einheitssphäre ist λ_1 .
- Falls $m \geq n$, ist $\sigma_n = \min_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|$ die minimale Streckung. Für $m < n$ ist dieses Minimum wegen des nichttrivialen Kerns null; in der erweiterten Konvention entspricht es $\sigma_n = 0$.

- $\sigma_i = 0$ gilt genau dann, wenn $\lambda_i = 0$ ist.
- *Konvention bei breiten Matrizen.* Für $m < n$ hat $A^T A$ insgesamt n Eigenwerte, darunter mindestens $n - m$ strukturell erzwungene Nullen. Als Singulärwerte zählen nur die $q = \min(m, n)$ Diagonalplätze von Σ ; wer alle Eigenwerte indizieren will, ergänzt $\sigma_{q+1} = \dots = \sigma_n = 0$ zu dieser *erweiterten Liste*, ohne dass die Nullen weitere Diagonaleinträge wären.

Der größte Singulärwert ist damit nichts anderes als die *Spektralnorm* $\|A\|_2$ aus Abschnitt 3.3. Definiert war sie dort als die von der euklidischen Norm induzierte Operatornorm; Satz 3.3.7 hat sie zu $\lambda_{\max}(A^T A)^{1/2}$ ausgerechnet, und das ist σ_1 .

Beispiel 6.2.6 (Singulärwerte der Beispielmatrix)

Für A aus Beispiel 6.2.3 mit $\lambda_1 \approx 9,531$ und $\lambda_2 \approx 1,469$ ist

$$\sigma_1 = \sqrt{\lambda_1} \approx \sqrt{9,531} \approx 3,087, \quad \sigma_2 = \sqrt{\lambda_2} \approx \sqrt{1,469} \approx 1,212.$$

In der einen Richtung streckt A also um gut das Dreifache, in der anderen wächst die Länge nur um gut ein Fünftel. Ihr Verhältnis

$$\frac{\sigma_1}{\sigma_2} \approx \frac{3,087}{1,212} \approx 2,55$$

sagt, wie richtungsabhängig (*anisotrop*) diese Matrix wirkt. Für quadratische, invertierbare Matrizen ist σ_1/σ_n genau die Spektralkondition $\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2$ aus Abschnitt 3.5; dort hatten wir sie als Verhältnis der extremen Streckungsfaktoren gedeutet und angekündigt, dass die Singulärwerte diese Deutung noch durchsichtiger machen. Damit lösen wir das ein: $\|A\|_2 = \sigma_1$ ist die stärkste Streckung durch A , und $\|A^{-1}\|_2 = 1/\sigma_n$, denn die schwächste Streckung durch A ist die stärkste durch A^{-1} . Für rechteckige Matrizen mit vollem Spaltenrang, wie hier, ist σ_1/σ_n die übliche Verallgemeinerung.

Singulärvektoren

Streckungsfaktoren allein nützen wenig, solange wir nicht wissen, *welche* Richtung um σ_i gestreckt wird. Diese Richtungen stecken bereits in Korollar 6.2.2.

Definition 6.2.7 (Rechte und linke Singulärvektoren)

Sei $A \in \mathbb{R}^{m \times n}$. Für die rechten Singulärvektoren verwenden wir die in Bemerkung 6.2.5 erklärte erweiterte Liste $\sigma_1 \geq \dots \geq \sigma_n \geq 0$; bei $m < n$ enthält sie die zusätzlichen erzwungenen Nullen.

- Die *rechten Singulärvektoren* $\mathbf{v}_1, \dots, \mathbf{v}_n \in \mathbb{R}^n$ sind normierte, paarweise orthogonale Eigenvektoren von $A^T A$:

$$A^T A \mathbf{v}_i = \lambda_i \mathbf{v}_i = \sigma_i^2 \mathbf{v}_i, \quad \|\mathbf{v}_i\| = 1.$$

- Die *linken Singulärvektoren* $\mathbf{u}_i \in \mathbb{R}^m$ sind für $\sigma_i > 0$ erklärt durch

$$\mathbf{u}_i = \frac{A \mathbf{v}_i}{\sigma_i}, \quad \text{gleichwertig} \quad A \mathbf{v}_i = \sigma_i \mathbf{u}_i. \quad (6.2.2)$$

Die zweite Schreibweise in (6.2.2) ist die aussagekräftigere: A bildet die Richtung $\mathbf{v}_i$ auf die Richtung $\mathbf{u}_i$ ab und streckt dabei um σ_i . Dass die Bildrichtungen dabei aufeinander senkrecht stehen, ist der eigentliche Grund, warum die ganze Konstruktion trägt.

Satz 6.2.8 (Orthogonalität der Singulärvektoren)

Seien $\mathbf{v}_1, \dots, \mathbf{v}_n$ orthonormale rechte Singulärvektoren von A . Dann gilt

$$(\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j) = 0 \quad \text{für } i \neq j, \quad \|\mathbf{A} \mathbf{v}_i\| = \sigma_i.$$

Insbesondere sind die linken Singulärvektoren $\mathbf{u}_1, \dots, \mathbf{u}_r$ zu den r von null verschiedenen Singulärwerten orthonormal.

Vertiefung: Beweis der Orthogonalität

Beweis.

- (1) Für $i \neq j$ rechnen wir

$$(\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j) = \mathbf{v}_i^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v}_j = \mathbf{v}_i^\top (\lambda_j \mathbf{v}_j) = \lambda_j (\mathbf{v}_i^\top \mathbf{v}_j) = \lambda_j \cdot 0 = 0.$$

$(\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j) = \mathbf{v}_i^\top \mathbf{A}^\top \mathbf{A} \mathbf{v}_j$; dann die Eigenwertgleichung für $\mathbf{v}_j$ und zuletzt die Orthogonalität $\mathbf{v}_i^\top \mathbf{v}_j = 0$

- (2) Für $i = j$ liefert dieselbe Rechnung die Länge:

$$\|\mathbf{A} \mathbf{v}_i\|^2 = \mathbf{v}_i^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{v}_i = \lambda_i \|\mathbf{v}_i\|^2 = \lambda_i = \sigma_i^2.$$

(6.2.1) mit $\mathbf{x} = \mathbf{v}_i$, danach Eigenwertgleichung und $\|\mathbf{v}_i\| = 1$

- (3) Halten wir den Unterschied fest: Die $\mathbf{v}_i$ sind orthoNORMAL, ihre Bilder $\mathbf{A} \mathbf{v}_i$ dagegen nur orthogonal, mit Längen σ_i . Erst die Normierung (6.2.2) macht daraus orthonormale Vektoren,

$$\mathbf{u}_i^\top \mathbf{u}_j = \frac{(\mathbf{A} \mathbf{v}_i)^\top (\mathbf{A} \mathbf{v}_j)}{\sigma_i \sigma_j} = 0 \quad (i \neq j), \quad \|\mathbf{u}_i\| = \frac{\|\mathbf{A} \mathbf{v}_i\|}{\sigma_i} = 1.$$

dividieren dürfen wir nur bei $\sigma_i, \sigma_j > 0$, also für Indizes $i, j \leq r$; die übrigen $\mathbf{u}_i$ liefert Definition 6.2.7 gar nicht erst

□

Beispiel 6.2.9 (Singulärvektoren der Beispielmatrix)

Wir bleiben bei A aus Beispiel 6.2.3 mit $A^\top A = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix}$.

Rechte Singulärvektoren. Für $\lambda_1 \approx 9,531$ suchen wir $\mathbf{v}_1$ mit

$$\begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} v_{11} \\ v_{21} \end{pmatrix} = 9,531 \begin{pmatrix} v_{11} \\ v_{21} \end{pmatrix}.$$

Die erste Zeile heißt $(6 - 9,531) v_{11} + 4 v_{21} = 0$, also $v_{21} \approx 0,883 v_{11}$. Normieren wir (das Vorzeichen dürfen wir dabei frei wählen, siehe Bemerkung 6.2.10), so ergibt sich zusammen mit dem dazu orthogonalen zweiten Eigenvektor

$$\mathbf{v}_1 \approx \begin{pmatrix} -0,750 \\ -0,662 \end{pmatrix}, \quad \mathbf{v}_2 \approx \begin{pmatrix} 0,662 \\ -0,750 \end{pmatrix}.$$

Linke Singulärvektoren. Jetzt liefert (6.2.2) die Bildrichtungen:

$$\mathbf{u}_1 = \frac{\mathbf{A} \mathbf{v}_1}{\sigma_1} \approx \frac{1}{3,087} \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -0,750 \\ -0,662 \end{pmatrix} = \frac{1}{3,087} \begin{pmatrix} -2,074 \\ -2,162 \\ -0,750 \end{pmatrix} \approx \begin{pmatrix} -0,672 \\ -0,700 \\ -0,243 \end{pmatrix},$$

$$\mathbf{u}_2 = \frac{\mathbf{A} \mathbf{v}_2}{\sigma_2} \approx \frac{1}{1,212} \begin{pmatrix} -0,838 \\ 0,574 \\ 0,662 \end{pmatrix} \approx \begin{pmatrix} -0,691 \\ 0,474 \\ 0,546 \end{pmatrix}.$$

Probe auf Orthonormalität. Satz 6.2.8 behauptet, dass diese beiden Vektoren orthonormal sind, und die Zahlen bestätigen es:

$$\begin{aligned} \mathbf{u}_1^\top \mathbf{u}_2 &\approx (-0,672)(-0,691) + (-0,700)(0,474) + (-0,243)(0,546) \\ &= 0,4644 - 0,3318 - 0,1327 \approx 0, \end{aligned}$$

$$\|\mathbf{u}_1\| \approx \sqrt{0,672^2 + 0,700^2 + 0,243^2} = \sqrt{0,4516 + 0,4900 + 0,0590} \approx 1.$$

Bemerkung 6.2.10 (Singulärvektoren sind nicht eindeutig)

Mit $\mathbf{v}_i$ ist auch $-\mathbf{v}_i$ ein normierter Eigenvektor zum selben Eigenwert. Die Singulärwerte ändern sich dadurch nicht, wohl aber die Vektoren, und zwar paarweise: Nach (6.2.2) dreht sich mit $\mathbf{v}_i$ zugleich $\mathbf{u}_i$ um. Wer in einem Beispiel das Vorzeichen von $\mathbf{v}_2$ umdreht, muss deshalb auch $\mathbf{u}_2$ umdrehen, sonst passen linke und rechte Seite von $\mathbf{A} \mathbf{v}_2 = \sigma_2 \mathbf{u}_2$ nicht mehr zusammen. Numerische Bibliotheken treffen die Wahl nach eigenen Konventionen; zwei Programme dürfen also verschiedene Vorzeichen ausgeben, ohne dass eines von beiden falsch rechnet. Bei mehrfachen Singulärwerten ist die Freiheit sogar größer: Im zugehörigen Eigenraum lässt sich die Orthonormalbasis beliebig drehen.

Interaktiv: Die Rechnung Schritt für Schritt

Wie läuft dieser Weg an einer anderen Matrix, und was geht schief, wenn die beiden Spalten linear abhängig sind? Das Widget rechnet die Schritte von Beispiel 6.2.3 bis 6.2.9 für jede eingetragene 3×2 -Matrix nach; die beiden Vorzeichen-Knöpfe in Schritt 5 führen Bemerkung 6.2.10 vor. Unter den Schritten steht dieselbe Abbildung geometrisch: links der Einheitskreis des $\mathbb{R}^2$, rechts sein Bild im $\mathbb{R}^3$.

Interaktives Element — nur in der Web-Fassung

Fundamentale Unterräume

Bisher waren die Singulärvektoren Rechengrößen. Sie leisten aber mehr: Sie beschreiben auf einen Schlag alle Unterräume, die zu einer Matrix gehören.

Satz 6.2.11 (Charakterisierung der fundamentalen Unterräume)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit genau r von null verschiedenen Singulärwerten. In der erweiterten Liste aus Bemerkung 6.2.5 gilt also $\sigma_1 \geq \dots \geq \sigma_r > 0$ und $\sigma_{r+1} = \dots = \sigma_n = 0$. Dann gilt:

1. $\{\mathbf{v}_1, \dots, \mathbf{v}_r\}$ ist eine Orthonormalbasis des Zeilenraums $\text{col}(\mathbf{A}^\top)$.

2. $\{\mathbf{A}\mathbf{v}_1, \dots, \mathbf{A}\mathbf{v}_r\}$ ist eine Orthogonalbasis des Spaltenraums $\text{col}(\mathbf{A})$, also des Bildes von $\mathbf{A}$; normiert man sie, erhält man die Orthonormalbasis $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$.
3. $\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$ ist eine Orthonormalbasis des Kerns $\text{Kern}(\mathbf{A})$.
4. $\text{rang}(\mathbf{A}) = r$: Der Rang von $\mathbf{A}$ ist die Anzahl der von null verschiedenen Singulärwerte.

Achten wir auf den Wechsel zwischen (1) und (2): In (1) steht eine Orthonormalbasis, in (2) nur eine Orthogonalbasis. Der Grund steht in Satz 6.2.8, die Längen $\|\mathbf{A}\mathbf{v}_i\| = \sigma_i$ sind im Allgemeinen eben nicht 1. Erst das Teilen durch σ_i macht die $\mathbf{u}_i$ daraus.

Vertiefung: Warum die Singulärvektoren genau die vier Unterräume aufspannen

Der Beweis geht in vier Etappen: Kern, Umkehrung, Spaltenraum, Zeilenraum.

Beweis.

- (1) Für $i > r$ ist $\sigma_i = 0$, nach Satz 6.2.8 also $\|\mathbf{A}\mathbf{v}_i\| = 0$ und damit $\mathbf{A}\mathbf{v}_i = \mathbf{0}$. Die letzten $n - r$ rechten Singulärvektoren liegen im Kern von $\mathbf{A}$.

nur der Nullvektor hat die Länge null

- (2) Umgekehrt liegt jedes $\mathbf{x}$ mit $\mathbf{A}\mathbf{x} = \mathbf{0}$ in deren linearer Hülle. Wir entwickeln $\mathbf{x} = \sum_{i=1}^n c_i \mathbf{v}_i$ und erhalten

$$\mathbf{0} = \mathbf{A}\mathbf{x} = \sum_{i=1}^r c_i \sigma_i \mathbf{u}_i,$$

woraus $c_1 = \dots = c_r = 0$ folgt. Zusammen mit dem ersten Schritt ist $\text{Kern}(\mathbf{A}) = \text{span}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$, und diese Vektoren sind als Teil einer Orthonormalbasis orthonormal.

die Summanden mit $i > r$ fallen weg; die $\mathbf{u}_i$ sind orthonormal, also linear unabhängig, und wegen $\sigma_i > 0$ müssen alle Koeffizienten verschwinden

- (3) Die Spalten von $\mathbf{A}$ sind Linearkombinationen der $\mathbf{A}\mathbf{v}_i$ mit $i \leq r$: Für jeden Einheitsvektor $\mathbf{e}_j$ ist $\mathbf{e}_j = \sum_{i=1}^n (\mathbf{v}_i^\top \mathbf{e}_j) \mathbf{v}_i$, und Anwenden von $\mathbf{A}$ liefert $\mathbf{A}\mathbf{e}_j = \sum_{i \leq r} (\mathbf{v}_i^\top \mathbf{e}_j) \mathbf{A}\mathbf{v}_i$. Die r Vektoren $\mathbf{A}\mathbf{v}_i$ sind nach Satz 6.2.8 orthogonal und von null verschieden, also linear unabhängig. Sie bilden damit eine Orthogonalbasis von $\text{col}(\mathbf{A})$, und es folgt $\text{rang}(\mathbf{A}) = \dim \text{col}(\mathbf{A}) = r$.

$\mathbf{A}\mathbf{e}_j$ ist die j -te Spalte von $\mathbf{A}$; die Summanden mit $i > r$ fallen nach Schritt 1 weg

- (4) Für den Zeilenraum multiplizieren wir (6.2.2) von links mit $\mathbf{A}^\top$: Für $i \leq r$ ist

$$\mathbf{A}^\top \mathbf{u}_i = \frac{\mathbf{A}^\top \mathbf{A} \mathbf{v}_i}{\sigma_i} = \frac{\sigma_i^2 \mathbf{v}_i}{\sigma_i} = \sigma_i \mathbf{v}_i.$$

Jedes $\mathbf{v}_i$ mit $i \leq r$ ist also eine Linearkombination von Spalten von $\mathbf{A}^\top$, liegt somit in $\text{col}(\mathbf{A}^\top)$. Da $\dim \text{col}(\mathbf{A}^\top) = \text{rang}(\mathbf{A}^\top) = \text{rang}(\mathbf{A}) = r$ ist und die $\mathbf{v}_1, \dots, \mathbf{v}_r$ orthonormal sind, spannen sie $\text{col}(\mathbf{A}^\top)$ vollständig auf.

r linear unabhängige Vektoren in einem r -dimensionalen Raum bilden bereits eine Basis, und orthonormale Vektoren sind linear unabhängig

□

Damit liefert die SVD explizite Orthonormalbasen für alle vier fundamentalen Unterräume. Der vierte, der linke Kern $\text{Kern}(\mathbf{A}^\top)$, fällt im nächsten Beispiel nebenbei mit ab.

Beispiel 6.2.12 (Alle Unterräume einer Rang-1-Matrix)

Wir nehmen

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}, \quad A^T A = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}.$$

Singulärwerte. Das charakteristische Polynom ist $(2 - \lambda)^2 - 4 = \lambda^2 - 4\lambda = \lambda(\lambda - 4)$, also $\lambda_1 = 4$ und $\lambda_2 = 0$ und damit

$$\sigma_1 = 2, \quad \sigma_2 = 0, \quad r = 1.$$

In Richtung $\mathbf{v}_1$ streckt A um den Faktor 2, in Richtung $\mathbf{v}_2$ um den Faktor 0.

Rechte Singulärvektoren und Zeilenraum. Die Eigenvektoren von $A^T A$ sind

$$\mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix},$$

also $\text{col}(A^T) = \text{span}\{\mathbf{v}_1\} = \text{span}\{(1, 1)^T\}$. Das passt: Die beiden Zeilen von A , die nicht null sind, lauten beide $(1, 1)$.

Linker Singulärvektor und Spaltenraum. Aus (6.2.2) folgt

$$\mathbf{u}_1 = \frac{A\mathbf{v}_1}{\sigma_1} = \frac{1}{2} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix},$$

also $\text{col}(A) = \text{span}\{\mathbf{u}_1\} = \text{span}\{(1, 1, 0)^T\}$.

Kern. Nach Satz 6.2.11 ist $\text{Kern}(A) = \text{span}\{\mathbf{v}_2\} = \text{span}\{(1, -1)^T\}$, und die Probe bestätigt das: $A(1, -1)^T = (0, 0, 0)^T$.

Linker Kern. Für $\mathbf{u}_2$ liefert (6.2.2) nichts, denn $\sigma_2 = 0$; ein $\mathbf{u}_3$ sieht die Formel gar nicht erst vor, weil A nur zwei Spalten und damit nur zwei rechte Singulärvektoren hat. Wir ergänzen $\mathbf{u}_1$ stattdessen zu einer Orthonormalbasis des $\mathbb{R}^3$, etwa durch

$$\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad \mathbf{u}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

Diese beiden spannen den linken Kern auf, $\text{Kern}(A^T) = \text{span}\{\mathbf{u}_2, \mathbf{u}_3\}$, denn

$$A^T \begin{pmatrix} -1 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}.$$

Der SVD-Hauptsatz

Alle Bausteine liegen bereit. Sammeln wir die rechten Singulärvektoren als Spalten in einer Matrix, ebenso die linken, und schreiben wir die Singulärwerte auf eine Diagonale, dann steht die Zerlegung da.

Satz 6.2.13 (Singulärwertzerlegung)

Sei $A \in \mathbb{R}^{m \times n}$ mit $\text{rang}(A) = r$. Dann existiert eine Zerlegung

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T \quad (6.2.3)$$

mit

- $\mathbf{V} \in \mathbb{R}^{n \times n}$ orthogonal, in den Spalten die rechten Singulärvektoren $\mathbf{v}_1, \dots, \mathbf{v}_n$;
- $\mathbf{U} \in \mathbb{R}^{m \times m}$ orthogonal, in den ersten r Spalten die linken Singulärvektoren $\mathbf{u}_1, \dots, \mathbf{u}_r$, ergänzt zu einer Orthonormalbasis des $\mathbb{R}^m$;
- $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$ „diagonal“ mit den Singulärwerten auf der Hauptdiagonalen,

$$\mathbf{\Sigma} = \begin{pmatrix} \text{diag}(\sigma_1, \dots, \sigma_r) & \mathbf{0}_{r \times (n-r)} \\ \mathbf{0}_{(m-r) \times r} & \mathbf{0}_{(m-r) \times (n-r)} \end{pmatrix}. \quad (6.2.4)$$

Die Anführungszeichen um „diagonal“ sind nötig, weil $\mathbf{\Sigma}$ dieselbe Form wie $\mathbf{A}$ hat und für $m \neq n$ gar nicht quadratisch ist. Der folgende Beweis ist eine Skizze: Er sammelt die Konstruktion, die wir in diesem Abschnitt ohnehin schon durchgeführt haben, und stützt sich dabei auf den Spektralsatz, den wir nicht bewiesen haben.

Vertiefung: Beweisskizze des SVD-Hauptsatzes

Beweis.

- (1) **Rechte Singulärvektoren.** Nach Korollar 6.2.2 besitzt $\mathbf{A}^T \mathbf{A}$ eine Orthonormalbasis aus Eigenvektoren. Wir sortieren sie nach absteigenden Eigenwerten und stellen sie spaltenweise zusammen: $\mathbf{V} = (\mathbf{v}_1 \mid \mathbf{v}_2 \mid \dots \mid \mathbf{v}_n)$.

$\mathbf{V}$ ist orthogonal, weil seine Spalten orthonormal sind

- (2) **Linke Singulärvektoren.** Für $i \leq r$ setzen wir $\mathbf{u}_i = \mathbf{A} \mathbf{v}_i / \sigma_i$ nach (6.2.2) und sammeln sie in $\mathbf{U}_r = (\mathbf{u}_1 \mid \dots \mid \mathbf{u}_r)$.

nach Satz 6.2.8 sind diese r Vektoren orthonormal

- (3) **Vervollständigung.** Wir ergänzen $\mathbf{U}_r$ zu einer Orthonormalbasis des $\mathbb{R}^m$, etwa mit dem Gram-Schmidt-Verfahren. Das ergibt die Orthogonalmatrix $\mathbf{U} \in \mathbb{R}^{m \times m}$.

die ergänzten Spalten spannen gerade $\text{Kern}(\mathbf{A}^T)$ auf, wie in Beispiel 6.2.12

- (4) **Verifikation.** Für $i \leq r$ ist $\mathbf{U} \mathbf{\Sigma} \mathbf{V}^T \mathbf{v}_i = \mathbf{U} \mathbf{\Sigma} \mathbf{e}_i = \sigma_i \mathbf{U} \mathbf{e}_i = \sigma_i \mathbf{u}_i = \mathbf{A} \mathbf{v}_i$, und für $i > r$ steht auf beiden Seiten der Nullvektor. Beide Abbildungen stimmen also auf einer Basis des $\mathbb{R}^n$ überein und sind damit gleich.

$\mathbf{V}^T \mathbf{v}_i = \mathbf{e}_i$, weil $\mathbf{V}$ orthogonal ist; $\mathbf{\Sigma} \mathbf{e}_i = \sigma_i \mathbf{e}_i$ nach (6.2.4), und für $i > r$ ist $\mathbf{\Sigma} \mathbf{e}_i = \mathbf{0}$

Beispiel 6.2.14 (Die Gestalt von $\mathbf{\Sigma}$)

Die Blockform (6.2.4) wirkt sperriger, als sie ist. Drei Fälle:

Für ein quadratisches $\mathbf{A}$ mit $m = n = 3$ und $r = 2$ ist

$$\mathbf{\Sigma} = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Für ein rechteckiges $\mathbf{A}$ mit $m = 2$, $n = 3$ und $r = 2$ ist

$$\mathbf{\Sigma} = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{pmatrix}.$$

Und für unsere Beispielmatrix aus Beispiel 6.2.3 mit $m = 3$, $n = 2$, $r = 2$ ist

$$\Sigma \approx \begin{pmatrix} 3,087 & 0 \\ 0 & 1,212 \\ 0 & 0 \end{pmatrix}.$$

Hier hat $\mathbf{U}$ drei Spalten, wir kennen aber erst zwei. Die dritte ergänzt Schritt 3 des Beweises; sie spannt den linken Kern auf und ist bis aufs Vorzeichen

$$\mathbf{u}_3 = \frac{1}{\sqrt{14}} \begin{pmatrix} 1 \\ -2 \\ 3 \end{pmatrix} \approx \begin{pmatrix} 0,267 \\ -0,535 \\ 0,802 \end{pmatrix}, \quad \text{denn} \quad \mathbf{A}^\top \begin{pmatrix} 1 \\ -2 \\ 3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

Geometrische Interpretation

Bemerkung 6.2.15 (Merkregel: Drehen, Strecken, Drehen)

Lesen wir (6.2.3) als Abbildungsvorschrift $\mathbf{x} \mapsto \mathbf{Ax} = \mathbf{U}\Sigma\mathbf{V}^\top \mathbf{x}$, dann geschieht die Abbildung in drei Etappen:

1. $\mathbf{V}^\top$: Koordinatenwechsel im $\mathbb{R}^n$, eine Drehung oder Spiegelung auf die „optimalen“ Richtungen $\mathbf{v}_1, \dots, \mathbf{v}_n$.
2. Σ : Strecken entlang dieser Achsen um die Faktoren σ_i , Nullsetzen der Richtungen mit $i > r$ und Übergang von $\mathbb{R}^n$ nach $\mathbb{R}^m$.
3. $\mathbf{U}$: Koordinatenwechsel im $\mathbb{R}^m$, wieder eine Drehung oder Spiegelung, diesmal in die Endlage.

Die SVD zerlegt also *jede* lineare Abbildung von $\mathbb{R}^n$ nach $\mathbb{R}^m$ in Drehen, Strecken, Drehen. Aus der Einheitssphäre des $\mathbb{R}^n$ wird dabei ein Ellipsoid mit den Hauptachsenrichtungen $\mathbf{u}_i$ und den Halbachsenlängen σ_i . Drückt $\mathbf{A}$ ganze Richtungen auf null, gilt also $r < n$, dann liegt der Nullpunkt mit im Bild, und aus dem Ellipsoid wird ein ausgefülltes Ellipsoid im r -dimensionalen Spaltenraum (Abschnitt 6.1).

Interaktiv: Drehen, Strecken, Drehen an einer editierbaren Matrix

Welche der drei Etappen ist für die Form der Bildellipse zuständig, und welche nur für ihre Lage? Vier Tafeln zeigen denselben Kreis vor $\mathbf{V}^\top$, danach, nach Σ und nach $\mathbf{U}$. Voreingestellt ist $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$, die Matrix, an der Kapitel 3 die Operatornorm sichtbar gemacht hat (Abschnitt 3.3); drei weitere Knöpfe stellen eine Drehmatrix, eine Scherung und eine Matrix mit zwei gleichen Spalten ein.

Interaktives Element — nur in der Web-Fassung

Selbsttest

1. Der Rechner oben zeigt in Schritt 4 das Verhältnis σ_1/σ_2 der voreingestellten Matrix. Wie groß ist es?

Lösung: 2.547

$3,087/1,212 \approx 2,547$. Nachrechnen lässt sich das auch ohne Widget, denn Wurzeln ziehen und Teilen vertauschen: $\sqrt{9,531/1,469} = \sqrt{6,488} \approx 2,547$. Diese Zahl ist die Konditionszahl $\kappa_2(\mathbf{A})$ aus Abschnitt 3.5; die Bildellipse ist gut zweieinhalbmal so lang wie breit.

2. Für jede Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ sind alle Eigenwerte von $\mathbf{A}^\top \mathbf{A}$ nichtnegativ.

Antwort: Wahr

Das ist Satz 6.2.1(2). Der Grund steckt in (6.2.1): Für einen normierten Eigenvektor $\mathbf{v}_i$ ist $\lambda_i = \|\mathbf{A}\mathbf{v}_i\|^2$, also das Quadrat einer Länge.

3. Wenn ein Singulärwert von $\mathbf{A}$ null ist, muss $\mathbf{A}$ die Nullmatrix sein.

Antwort: Falsch

Ein einziger verschwindender Singulärwert bedeutet nur, dass $\mathbf{A}$ eine Richtung plattdrückt. Beispiel 6.2.12 zeigt eine Matrix mit $\sigma_1 = 2$ und $\sigma_2 = 0$, die alles andere als null ist; nach Satz 6.2.11 hat sie den Rang 1 und einen eindimensionalen Kern.

4. Die rechten Singulärvektoren $\mathbf{v}_i$ sind orthonormal, die Bildvektoren $\mathbf{A}\mathbf{v}_i$ dagegen im Allgemeinen nur orthogonal.

Antwort: Wahr

Genau das sagt Satz 6.2.8: Die Bilder stehen paarweise senkrecht aufeinander, haben aber die Längen σ_i . Erst die Division durch σ_i macht daraus die orthonormalen linken Singulärvektoren.

5. In der Zerlegung $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ ist Σ immer eine quadratische Diagonalmatrix.

Antwort: Falsch

Σ hat dasselbe Format wie $\mathbf{A}$, ist also für $m \neq n$ rechteckig. Quadratisch sind nur die beiden Orthogonalmatrizen: $\mathbf{U}$ ist $m \times m$, $\mathbf{V}$ ist $n \times n$.

6. Die Singulärwerte und die Singulärvektoren einer Matrix sind eindeutig bestimmt.

Antwort: Falsch

Die Singulärwerte schon, die Vektoren nicht: Mit $\mathbf{v}_i$ leistet auch $-\mathbf{v}_i$ dasselbe, und dann dreht sich nach (6.2.2) auch $\mathbf{u}_i$ um (Bemerkung 6.2.10). Bei mehrfachen Singulärwerten kommt sogar noch die freie Wahl einer Orthonormalbasis im zugehörigen Eigenraum hinzu.

Vertiefung: Heath §3.6 (Singulärwertzerlegung); vgl. MML §4.5 für die geometrische Deutung der SVD und §4.6 für die daran anschließende Rang-k-Approximation.

6.3 Reduzierte SVD und Pseudoinverse

Die Zerlegung $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$ aus Abschnitt 6.2 ist vollständig, aber verschwenderisch. Hat $\mathbf{A}$ den Rang $r < \min(m, n)$, so besteht ein guter Teil von Σ aus Nullen, und diese Nullen löschen ganze Spalten von $\mathbf{U}$ und $\mathbf{V}$ aus. In diesem Abschnitt werfen wir den Ballast ab. Übrig bleibt die reduzierte SVD, und aus ihr lesen wir eine Matrix ab, die auch eine rechteckige oder singuläre Matrix so gut umkehrt, wie es überhaupt geht.

Nullblöcke in Sigma

Für $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r$ hat Σ die Blockgestalt

$$\Sigma = \begin{pmatrix} \Sigma_r & \mathbf{0}_{r \times (n-r)} \\ \mathbf{0}_{(m-r) \times r} & \mathbf{0}_{(m-r) \times (n-r)} \end{pmatrix} \in \mathbb{R}^{m \times n}, \quad \Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r). \quad (6.3.1)$$

Nur der Block links oben ist besetzt. Die letzten $m-r$ Zeilen von Σ sind Nullzeilen, die letzten $n-r$ Spalten sind Nullspalten, und beides gilt gleichzeitig. Bei einer Matrix mit $m = 1000$, $n = 50$ und $r = 5$ heißt das: Von den 50 000 Einträgen in Σ sind ganze fünf von null verschieden.

Was macht das mit den beiden Orthogonalmatrizen? Zerlegen wir sie entsprechend. Die ersten r Spalten spielen offenbar eine andere Rolle als der Rest, also trennen wir sie ab:

$$\mathbf{U} = (\mathbf{U}_r \mid \mathbf{U}_{m-r}), \quad \mathbf{V} = (\mathbf{V}_r \mid \mathbf{V}_{n-r}), \quad (6.3.2)$$

mit $\mathbf{U}_r \in \mathbb{R}^{m \times r}$ und $\mathbf{V}_r \in \mathbb{R}^{n \times r}$. Nach Satz 6.2.11 stehen in $\mathbf{U}_r$ die linken Singulärvektoren $\mathbf{u}_1, \dots, \mathbf{u}_r$, also eine Orthonormalbasis des Spaltenraums $\text{col}(\mathbf{A})$, und in $\mathbf{V}_r$ die rechten Singulärvektoren $\mathbf{v}_1, \dots, \mathbf{v}_r$, eine Orthonormalbasis des Zeilenraums $\text{col}(\mathbf{A}^\top)$. Die abgetrennten Blöcke gehören zu den beiden Kernen: $\mathbf{V}_{n-r}$ spannt $\text{Kern}(\mathbf{A})$ auf, $\mathbf{U}_{m-r}$ den linken Kern $\text{Kern}(\mathbf{A}^\top)$.

Jetzt rechnen wir nach, was die Nullblöcke anrichten.

Satz 6.3.1 (Reduzierte Darstellung)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r \geq 1$ und einer Singulärwertzerlegung $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$. Mit den Partitionen aus (6.3.2) gilt

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top. \quad (6.3.3)$$

Die Blöcke $\mathbf{U}_{m-r}$ und $\mathbf{V}_{n-r}$ tragen zu $\mathbf{A}$ nichts bei.

Beweis.

- (1) Transponieren wir die Partition von $\mathbf{V}$, so werden aus den Spaltenblöcken übereinanderliegende Zeilenblöcke:

$$\mathbf{V}^\top = \begin{pmatrix} \mathbf{V}_r^\top \\ \mathbf{V}_{n-r}^\top \end{pmatrix}.$$

Transponieren macht aus der j-ten Spalte die j-te Zeile; der obere Block hat r Zeilen, der untere die restlichen $n-r$

- (2) Multiplizieren wir blockweise mit (6.3.1):

$$\Sigma \mathbf{V}^\top = \begin{pmatrix} \Sigma_r & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{pmatrix} \begin{pmatrix} \mathbf{V}_r^\top \\ \mathbf{V}_{n-r}^\top \end{pmatrix} = \begin{pmatrix} \Sigma_r \mathbf{V}_r^\top \\ \mathbf{0}_{(m-r) \times n} \end{pmatrix}.$$

Blockmultiplikation: oben steht $\Sigma_r \mathbf{V}_r^\top + \mathbf{0} \mathbf{V}_{n-r}^\top$, unten nur Nullen; $\mathbf{V}_{n-r}$ kommt rechts gar nicht mehr vor

- (3) Multiplizieren wir von links mit $\mathbf{U}$, ebenfalls blockweise:

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top = (\mathbf{U}_r \mid \mathbf{U}_{m-r}) \begin{pmatrix} \Sigma_r \mathbf{V}_r^\top \\ \mathbf{0} \end{pmatrix} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top + \mathbf{U}_{m-r} \mathbf{0} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top.$$

Spaltenblöcke mal Zeilenblöcke: der zweite Summand ist die Nullmatrix, ganz gleich, was in $\mathbf{U}_{m-r}$ steht

□

Die reduzierte SVD

Damit hat sich die Zerlegung von selbst verkleinert, und wir geben dem Ergebnis einen Namen.

Definition 6.3.2 (Reduzierte SVD)

Sei $A \in \mathbb{R}^{m \times n}$ mit $\text{rang}(A) = r \geq 1$. Die Darstellung (6.3.3),

$$A = U_r \Sigma_r V_r^T,$$

heißt *reduzierte SVD* (reduced SVD) von A . Dabei ist

- $U_r \in \mathbb{R}^{m \times r}$ die Matrix der ersten r linken Singulärvektoren,
- $V_r \in \mathbb{R}^{n \times r}$ die Matrix der ersten r rechten Singulärvektoren,
- $\Sigma_r = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$ die Diagonalmatrix der von null verschiedenen Singulärwerte.

Bemerkung 6.3.3 (Was die reduzierte Form spart, und was sie kostet)

Drei Beobachtungen zur reduzierten Fassung:

1. *Kompakter*. Die volle Zerlegung speichert $m^2 + n^2 + \min(m, n)$ Zahlen (beide Orthogonalmatrizen plus die Diagonale von Σ), die reduzierte nur $r(m+n+1)$. Im Beispiel oben mit $m = 1000$, $n = 50$ und $r = 5$ sind das 5255 statt 1 002 550 Zahlen, also nicht einmal ein Prozent. Sogar A selbst braucht mit 50 000 Einträgen noch fast das Zehnfache.
2. *Rechenzeit*. Wer Ax über die drei kleinen Faktoren auswertet, braucht wieder $r(m+n+1)$ Multiplikationen statt mn , also dieselbe Schwelle wie in Punkt 1.
3. Σ_r ist invertierbar. Sie ist quadratisch, und auf ihrer Diagonalen stehen ausschließlich positive Zahlen. Das ist der Punkt, an dem der Rest dieses Abschnitts ansetzt.

Eines geht dabei verloren: U_r und V_r sind für $r < m$ bzw. $r < n$ keine Orthogonalmatrizen mehr, denn sie sind nicht quadratisch. Ihre Spalten bleiben orthonormal, es gilt also

$$U_r^T U_r = I_r \quad \text{und} \quad V_r^T V_r = I_r, \quad (6.3.4)$$

aber die Produkte in der anderen Reihenfolge, $U_r U_r^T$ und $V_r V_r^T$, sind im Allgemeinen keine Einheitsmatrizen. Was sie stattdessen sind, klären wir gleich; die Unterscheidung ist der wunde Punkt jeder Rechnung mit reduzierten Faktoren.

Beispiel 6.3.4 (Reduzierte SVD einer Rang-1-Matrix)

Nehmen wir die Matrix aus Beispiel 6.2.12,

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}, \quad \sigma_1 = 2, \quad \sigma_2 = 0, \quad r = 1.$$

Die volle Zerlegung braucht ein 3×3 -großes U , ein 3×2 -großes Σ und ein 2×2 -großes V . Bei $r = 1$ bleiben davon die erste Spalte von U , die erste Spalte von V und der einzige positive Singulärwert übrig:

$$\mathbf{U}_1 = \mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \Sigma_1 = (2), \quad \mathbf{V}_1 = \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Die Probe bestätigt (6.3.3):

$$\mathbf{U}_1 \Sigma_1 \mathbf{V}_1^T = 2 \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix} = \mathbf{A}.$$

Aus drei Matrizen mit 9, 6 und 4 Einträgen sind 3 + 1 + 2 Zahlen geworden. Ganz rechts steht ein äußeres Produkt von zwei Vektoren; dass sich jede Matrix so als Summe von r solchen Produkten schreiben lässt, ist der Ausgangspunkt von Abschnitt 6.4.

Interaktiv: Welche Blöcke fallen weg?

Welche Teile der Zerlegung verschwinden beim Schrumpfen, und wie viel Speicher bringt das? Das Widget zeichnet die volle Zerlegung als Blockschema mit frei einstellbaren Formaten. Ein Klick auf einen Block sagt, wozu er da ist; der Umschalter führt Satz 6.3.1 vor und wirft alles heraus, was auf einen Nullblock von Σ trifft. Die Zeile ganz unten zählt die gespeicherten Zahlen aus Bemerkung 6.3.3 mit.

Interaktives Element — nur in der Web-Fassung

Zwei Fälle lohnen den Vergleich: Bei $r = 1$ fällt der Speicherbedarf unter den der Matrix selbst, bei $r = m = n$ schrumpft dagegen gar nichts, und die reduzierte Zerlegung ist die volle.

Die Moore-Penrose-Pseudoinverse

Ein Gleichungssystem $\mathbf{Ax} = \mathbf{b}$ lösen wir mit $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$, sofern $\mathbf{A}$ quadratisch und invertierbar ist. In der Statistik ist das der Ausnahmefall: Designmatrizen sind rechteckig, und auch quadratische Matrizen sind manchmal singulär. Was setzen wir an die Stelle von $\mathbf{A}^{-1}$, wenn es die Inverse gar nicht gibt?

Die reduzierte SVD liefert die Antwort fast von selbst, denn sie zerlegt die Abbildung $\mathbf{x} \mapsto \mathbf{Ax}$ in drei Etappen: $\mathbf{V}_r^T$ liest die Koordinaten von $\mathbf{x}$ bezüglich $\mathbf{v}_1, \dots, \mathbf{v}_r$ ab, Σ_r streckt jede dieser Koordinaten um ihren Singulärwert, und $\mathbf{U}_r$ setzt das Ergebnis in die Basis $\mathbf{u}_1, \dots, \mathbf{u}_r$ des Spaltenraums zusammen. Auf diesen r Koordinaten lässt sich jede Etappe rückwärts durchlaufen: Die beiden äußeren sind Übergänge zwischen orthonormalen Systemen, Transponieren macht sie rückgängig; die mittlere ist eine Streckung um lauter positive Faktoren, Division macht sie rückgängig. Was $\mathbf{x}$ außerhalb des Zeilenraums mitbrachte, ist dagegen verloren, denn $\mathbf{A}$ hat es auf null gedrückt. Gehen wir den Weg trotzdem rückwärts und sehen wir nach, wie weit wir damit kommen.

Definition 6.3.5 (Moore-Penrose-Pseudoinverse)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r \geq 1$ und reduzierter SVD $\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T$. Die *Pseudoinverse* (Moore-Penrose-Inverse) von $\mathbf{A}$ ist

$$\mathbf{A}^+ := \mathbf{V}_r \boldsymbol{\Sigma}_r^{-1} \mathbf{U}_r^\top \in \mathbb{R}^{n \times m}. \quad (6.3.5)$$

Bemerkung 6.3.6 (Zum Bau der Pseudoinversen)

Vier Anmerkungen zu dieser Definition:

- Die Inverse in der Mitte ist harmlos, denn $\boldsymbol{\Sigma}_r$ ist eine Diagonalmatrix mit positiven Einträgen:

$$\boldsymbol{\Sigma}_r^{-1} = \text{diag}\left(\frac{1}{\sigma_1}, \dots, \frac{1}{\sigma_r}\right).$$

Genau hier zählt sich die Reduktion aus. In $\boldsymbol{\Sigma}$ selbst stehen Nullen auf der Diagonalen, und durch null teilt niemand.

- Die Reihenfolge der drei Faktoren dreht sich um, wie wir es von $(\mathbf{XY})^{-1} = \mathbf{Y}^{-1}\mathbf{X}^{-1}$ gewohnt sind: Was zuletzt angewandt wurde, wird zuerst rückgängig gemacht.
- Die Formate passen zusammen. $\mathbf{A}$ bildet von $\mathbb{R}^n$ nach $\mathbb{R}^m$ ab, also muss $\mathbf{A}^+$ von $\mathbb{R}^m$ nach $\mathbb{R}^n$ abbilden, und tatsächlich ist $\mathbf{A}^+$ vom Format $(n \times r) \cdot (r \times r) \cdot (r \times m) = n \times m$.
- Die Singulärvektoren liegen nach Bemerkung 6.2.10 nicht eindeutig fest, auf $\mathbf{A}^+$ schlägt diese Freiheit aber nicht durch: Die Pseudoinverse hängt allein an $\mathbf{A}$, nicht an der gewählten Zerlegung.

Vertiefung: Warum die Pseudoinverse nicht an der gewählten Zerlegung hängt

Beim Vorzeichen sieht man es sofort: Dreht sich das von $\mathbf{v}_i$, so dreht sich nach (6.2.2) auch das von $\mathbf{u}_i$, und in der Summendarstellung

$$\mathbf{A}^+ = \sum_{i=1}^r \frac{1}{\sigma_i} \mathbf{v}_i \mathbf{u}_i^\top$$

heben die beiden Vorzeichen einander auf. Bei einem mehrfachen Singulärwert ist die Freiheit größer, das Ergebnis aber dasselbe: Dort darf die Orthonormalbasis des Eigenraums gedreht werden, $\mathbf{V}_r \mapsto \mathbf{V}_r \mathbf{Q}$ und mit (6.2.2) zugleich $\mathbf{U}_r \mapsto \mathbf{U}_r \mathbf{Q}$, und weil $\boldsymbol{\Sigma}_r^{-1}$ in diesem Block ein Vielfaches der Einheitsmatrix ist, kürzt sich $\mathbf{Q}\mathbf{Q}^\top = \mathbf{I}$ ebenso weg.

Beispiel 6.3.7 (Fortsetzung: die Pseudoinverse der Rang-1-Matrix)

Für $\mathbf{A}$ aus Beispiel 6.3.4 setzen wir die drei Faktoren in umgekehrter Reihenfolge zusammen:

$$\mathbf{A}^+ = \mathbf{V}_1 \boldsymbol{\Sigma}_1^{-1} \mathbf{U}_1^\top = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} \cdot \frac{1}{2} \cdot \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 0 \end{pmatrix} = \frac{1}{4} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}.$$

Rechnen wir die beiden Produkte aus, die uns gleich beschäftigen werden:

$$\mathbf{A}\mathbf{A}^+ = \frac{1}{2} \begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} = \mathbf{u}_1 \mathbf{u}_1^\top, \quad \mathbf{A}^+\mathbf{A} = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = \mathbf{v}_1 \mathbf{v}_1^\top.$$

Beides sind keine Einheitsmatrizen, und beides kann auch nicht sein, wenn auch aus je eigenem Grund. $\mathbf{A}^+\mathbf{A} \neq \mathbf{I}_2$: $\mathbf{A}$ drückt die Richtung $\mathbf{v}_2$ auf null, und was einmal verschwunden ist, holt keine Rechnung zurück. $\mathbf{A}\mathbf{A}^+ \neq \mathbf{I}_3$: Der Spaltenraum ist nur eine Gerade im $\mathbb{R}^3$,

und AA^+b landet immer auf ihr, welches b wir auch einsetzen. Was die beiden Matrizen stattdessen tun, zeigt ein Testvektor. Für $b = (1, 3, 5)^T$ ist

$$AA^+b = \begin{pmatrix} 2 \\ 2 \\ 0 \end{pmatrix},$$

und das ist genau der Punkt von $\text{col}(A) = \text{span}\{(1, 1, 0)^T\}$, der b am nächsten liegt: Die ersten beiden Einträge werden gemittelt, der dritte fällt weg. Wir sehen eine Projektion.

Eigenschaften der Pseudoinversen

Der Testvektor hat den allgemeinen Sachverhalt schon verraten. Halten wir ihn fest.

Satz 6.3.8 (Eigenschaften der Pseudoinversen)

Sei $A \in \mathbb{R}^{m \times n}$ mit $\text{rang}(A) = r \geq 1$ und Pseudoinverser A^+ . Dann gilt:

1. $AA^+x = \text{proj}_{\text{col}(A)}x$ für alle $x \in \mathbb{R}^m$;
2. $A^+Ax = \text{proj}_{\text{col}(A^T)}x$ für alle $x \in \mathbb{R}^n$;
3. $AA^+A = A$;
4. $A^+AA^+ = A^+$.

Dabei ist proj_U die orthogonale Projektion auf den Untervektorraum U .

Die Punkte 3 und 4 heißen *Konsistenzbedingungen*: In den Dreierprodukten AA^+A und A^+AA^+ hebt ein benachbartes Paar einander weg, genau wie es eine echte Inverse täte. Für sich allein sind AA^+ und A^+A dagegen keine Einheitsmatrizen.

Wer die reduzierten Faktoren einsetzt, sieht schnell, was passiert:

$$AA^+ = U_r \Sigma_r V_r^T V_r \Sigma_r^{-1} U_r^T = U_r U_r^T,$$

denn die inneren Faktoren heben sich nach (6.3.4) weg; genauso wird $A^+A = V_r V_r^T$. Übrig bleiben genau die beiden Projektionsmatrizen: Die Aussagen 1 und 2 sagen, worauf sie projizieren, und die Aussagen 3 und 4 fallen danach ab.

Vertiefung: Warum die beiden Produkte orthogonale Projektionen sind

Die Rechnung ist lang, aber sie macht die Projektionsmatrix explizit: Sie zeigt, dass $U_r U_r^T$ auf $\text{col}(A)$ nichts ändert und auf dem orthogonalen Komplement alles auf null drückt.

Beweis.

- (1) Geben wir dem Produkt einen Namen:

$$P := AA^+ = U_r U_r^T.$$

in $AA^+ = U_r \Sigma_r V_r^T V_r \Sigma_r^{-1} U_r^T$ ist $V_r^T V_r = I_r$ nach (6.3.4), danach $\Sigma_r \Sigma_r^{-1} = I_r$

- (2) P ist symmetrisch und idempotent:

$$P^T = (U_r U_r^T)^T = U_r U_r^T = P, \quad P^2 = U_r (U_r^T U_r) U_r^T = U_r U_r^T = P.$$

Transponieren dreht das Produkt um und trifft es dadurch selbst; für $\mathbf{P}^2$ klammern wir die inneren Faktoren zusammen und nutzen wieder (6.3.4)

- (3) Auf $\text{col}(\mathbf{A})$ ändert $\mathbf{P}$ nichts. Jedes $\mathbf{y} \in \text{col}(\mathbf{A})$ lässt sich als $\mathbf{y} = \mathbf{U}_r \mathbf{c}$ mit einem Koeffizientenvektor $\mathbf{c} \in \mathbb{R}^r$ schreiben, und damit ist

$$\mathbf{P}\mathbf{y} = \mathbf{U}_r(\mathbf{U}_r^\top \mathbf{U}_r)\mathbf{c} = \mathbf{U}_r \mathbf{c} = \mathbf{y}.$$

die Spalten von $\mathbf{U}_r$ sind nach Satz 6.2.11 eine Orthonormalbasis von $\text{col}(\mathbf{A})$, jedes $\mathbf{y}$ dort ist also eine Linearkombination von ihnen

- (4) Auf dem orthogonalen Komplement macht $\mathbf{P}$ alles platt. Steht $\mathbf{z}$ senkrecht auf $\text{col}(\mathbf{A})$, so ist $\mathbf{u}_i^\top \mathbf{z} = 0$ für $i = 1, \dots, r$, also $\mathbf{U}_r^\top \mathbf{z} = \mathbf{0}$ und damit

$$\mathbf{P}\mathbf{z} = \mathbf{U}_r(\mathbf{U}_r^\top \mathbf{z}) = \mathbf{0}.$$

der i -te Eintrag von $\mathbf{U}_r^\top \mathbf{z}$ ist das Skalarprodukt $\mathbf{u}_i^\top \mathbf{z}$, und die $\mathbf{u}_i$ spannen $\text{col}(\mathbf{A})$ auf

- (5) Beides zusammen ergibt Aussage 1. Jedes $\mathbf{x} \in \mathbb{R}^m$ zerlegt sich eindeutig in $\mathbf{x} = \mathbf{y} + \mathbf{z}$ mit $\mathbf{y} \in \text{col}(\mathbf{A})$ und $\mathbf{z} \perp \text{col}(\mathbf{A})$, und nach den beiden Vorschriften ist

$$\mathbf{P}\mathbf{x} = \mathbf{P}\mathbf{y} + \mathbf{P}\mathbf{z} = \mathbf{y} = \text{proj}_{\text{col}(\mathbf{A})}\mathbf{x}.$$

$\mathbf{P} = \mathbf{U}_r \mathbf{U}_r^\top$ ist also die orthogonale Projektionsmatrix auf $\text{col}(\mathbf{A})$. Bild und Kern lesen wir gleich mit ab: $\text{col}(\mathbf{P}) = \text{col}(\mathbf{A})$ und $\text{Kern}(\mathbf{P}) = \text{col}(\mathbf{A})^\perp$. Schritt 2 bestätigt das von der anderen Seite, denn $\mathbf{P}^2 = \mathbf{P}$ macht $\mathbf{P}$ zu einer Projektion und $\mathbf{P}^\top = \mathbf{P}$ zu einer orthogonalen.

die Spalten von $\mathbf{U} = (\mathbf{U}_r \mid \mathbf{U}_{m-r})$ sind eine Orthonormalbasis des $\mathbb{R}^m$; die ersten r spannen $\text{col}(\mathbf{A})$ auf, die übrigen den linken Kern $\text{Kern}(\mathbf{A}^\top) = \text{col}(\mathbf{A})^\perp$, und die Entwicklung von $\mathbf{x}$ in dieser Basis liefert die Zerlegung

- (6) Aussage 2 geht wörtlich genauso, nur mit den blauen Faktoren:

$$\mathbf{A}^+ \mathbf{A} = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^\top \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top = \mathbf{V}_r \mathbf{V}_r^\top.$$

jetzt treffen sich $\mathbf{U}_r^\top \mathbf{U}_r = \mathbf{I}_r$ in der Mitte; die Spalten von $\mathbf{V}_r$ sind eine Orthonormalbasis von $\text{col}(\mathbf{A}^\top)$, also ist $\mathbf{V}_r \mathbf{V}_r^\top$ nach demselben Argument die Projektion auf den Zeilenraum

- (7) Die Konsistenzbedingungen fallen jetzt ab:

$$\mathbf{A}\mathbf{A}^+ \mathbf{A} = \mathbf{U}_r \mathbf{U}_r^\top \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top = \mathbf{A},$$

$$\mathbf{A}^+ \mathbf{A} \mathbf{A}^+ = \mathbf{V}_r \mathbf{V}_r^\top \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^\top = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^\top = \mathbf{A}^+.$$

beide Male steht (6.3.4) in der Mitte; anschaulich: jede Spalte von $\mathbf{A}$ liegt in $\text{col}(\mathbf{A})$, und dort lässt die Projektion nach Schritt 3 alles unverändert

□

Für Matrizen mit vollem Zeilen- oder Spaltenrang wird eine der beiden Projektionen zur Identität, und dann sieht die Pseudoinverse einer echten Inversen sehr ähnlich.

Korollar 6.3.9 (Spezialfälle)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r$.

1. Ist $r = m$ (voller Zeilenrang), so gilt $\mathbf{A}\mathbf{A}^+ = \mathbf{I}_m$.
2. Ist $r = n$ (voller Spaltenrang), so gilt $\mathbf{A}^+ \mathbf{A} = \mathbf{I}_n$.
3. Ist $\mathbf{A}$ quadratisch und invertierbar, so gilt $\mathbf{A}^+ = \mathbf{A}^{-1}$.

Vertiefung: Beweis der Spezialfälle

Beweis.

- (1) Zu 1 und 2: Für $r = m$ ist $\mathbf{U}_r \in \mathbb{R}^{m \times m}$ quadratisch mit orthonormalen Spalten, also eine Orthogonalmatrix, und aus $\mathbf{U}_r^T \mathbf{U}_r = \mathbf{I}_m$ folgt dann auch $\mathbf{A} \mathbf{A}^+ = \mathbf{U}_r \mathbf{U}_r^T = \mathbf{I}_m$. Für $r = n$ liefert dasselbe Argument mit $\mathbf{V}_r$ die Gleichung $\mathbf{A}^+ \mathbf{A} = \mathbf{V}_r \mathbf{V}_r^T = \mathbf{I}_n$.

bei einer quadratischen Matrix ist eine einseitige Inverse automatisch die beidseitige; geometrisch: $\text{col}(\mathbf{A})$ ist ganz $\mathbb{R}^m$ bzw. $\text{col}(\mathbf{A}^T)$ ganz $\mathbb{R}^n$, und die Projektion auf den ganzen Raum ändert nichts

- (2) Zu 3: Ist $\mathbf{A}$ quadratisch und invertierbar, so ist $r = m = n$, beide Fälle greifen gleichzeitig, und $\mathbf{A}^+$ ist eine beidseitige Inverse von $\mathbf{A}$.

die Inverse einer Matrix ist eindeutig: aus $\mathbf{B} \mathbf{A} = \mathbf{I}$ und $\mathbf{A} \mathbf{C} = \mathbf{I}$ folgt $\mathbf{B} = \mathbf{B} \mathbf{A} \mathbf{C} = \mathbf{C}$

□

Der zweite Spezialfall ist der für die Statistik wichtigste, denn eine Designmatrix hat in aller Regel mehr Zeilen als Spalten. Er lässt sich sogar ohne SVD ausrechnen.

Bemerkung 6.3.10 (Voller Spaltenrang: eine geschlossene Formel)

Hat $\mathbf{A} \in \mathbb{R}^{m \times n}$ vollen Spaltenrang $r = n$, so ist $\mathbf{V}_r$ orthogonal, und $\mathbf{A}^T \mathbf{A}$ lässt sich mit (6.3.3) und (6.3.4) ausrechnen:

$$\mathbf{A}^T \mathbf{A} = \mathbf{V}_r \Sigma_r \mathbf{U}_r^T \mathbf{U}_r \Sigma_r \mathbf{V}_r^T = \mathbf{V}_r \Sigma_r^2 \mathbf{V}_r^T, \quad \text{also} \quad (\mathbf{A}^T \mathbf{A})^{-1} = \mathbf{V}_r \Sigma_r^{-2} \mathbf{V}_r^T.$$

Multiplizieren wir das mit $\mathbf{A}^T = \mathbf{V}_r \Sigma_r \mathbf{U}_r^T$, so bleibt

$$(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T = \mathbf{V}_r \Sigma_r^{-2} \underbrace{\mathbf{V}_r^T \mathbf{V}_r}_{=\mathbf{I}_n} \Sigma_r \mathbf{U}_r^T = \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T = \mathbf{A}^+. \quad (6.3.6)$$

Bei vollem Spaltenrang ist die Pseudoinverse also nichts anderes als $(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$. Wer diese Formel schon einmal gesehen hat, erkennt die aufgelöste Form der Normalengleichungen wieder.

Beispiel 6.3.11 (Pseudoinverse der Beispielmatrix)

Kehren wir zu der Matrix zurück, die uns durch Abschnitt 6.2 begleitet hat:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix}, \quad \mathbf{A}^T \mathbf{A} = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix}, \quad \sigma_1 \approx 3,087, \quad \sigma_2 \approx 1,212.$$

Beide Singulärwerte sind positiv, also ist $r = 2 = n$: voller Spaltenrang. Die reduzierte SVD wirft hier nur die dritte Spalte von $\mathbf{U}$ weg, jene $\mathbf{u}_3$ aus Beispiel 6.2.14, die den linken Kern aufspannt. Weil $r = n$ ist, dürfen wir (6.3.6) benutzen und die Singulärwerte ganz umgehen. Mit $\det(\mathbf{A}^T \mathbf{A}) = 30 - 16 = 14$ ist

$$(\mathbf{A}^T \mathbf{A})^{-1} = \frac{1}{14} \begin{pmatrix} 5 & -4 \\ -4 & 6 \end{pmatrix}, \quad \mathbf{A}^+ = \frac{1}{14} \begin{pmatrix} 5 & -4 \\ -4 & 6 \end{pmatrix} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 0 \end{pmatrix} = \frac{1}{14} \begin{pmatrix} -3 & 6 & 5 \\ 8 & -2 & -4 \end{pmatrix}.$$

Korollar 6.3.9 verspricht $\mathbf{A}^+ \mathbf{A} = \mathbf{I}_2$, und die Rechnung hält Wort:

$$\frac{1}{14} \begin{pmatrix} -3 & 6 & 5 \\ 8 & -2 & -4 \\ 1 & 0 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} = \frac{1}{14} \begin{pmatrix} -3+12+5 & -6+6+0 \\ 8-4-4 & 16-2+0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

In der anderen Reihenfolge kommt dagegen keine Einheitsmatrix heraus, denn $\text{col}(\mathbf{A})$ ist nur eine Ebene im $\mathbb{R}^3$: Es ist $\mathbf{A}\mathbf{A}^+ = \mathbf{I}_3 - \mathbf{u}_3\mathbf{u}_3^\top$, die orthogonale Projektion auf genau diese Ebene.

Bemerkung 6.3.12 (Ausblick: Kleinste Quadrate)

Aussage 1 aus Satz 6.3.8 ist mehr als eine hübsche Formel. Ein überbestimmtes System $\mathbf{A}\mathbf{x} = \mathbf{b}$ hat in aller Regel keine Lösung, weil $\mathbf{b}$ nicht in $\text{col}(\mathbf{A})$ liegt. Das Beste, was wir erreichen können, ist der Punkt von $\text{col}(\mathbf{A})$, der $\mathbf{b}$ am nächsten liegt, und den kennen wir jetzt: Es ist $\mathbf{A}\mathbf{A}^+\mathbf{b} = \text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}$. Der Vektor

$$\hat{\mathbf{x}} = \mathbf{A}^+\mathbf{b}$$

löst deshalb das Kleinste-Quadrate-Problem $\min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|$; der rote Term ist das Residuum, und kleiner als $\|\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b} - \mathbf{b}\|$ wird es nicht. Bei vollem Spaltenrang ist $\hat{\mathbf{x}}$ nach (6.3.6) die vertraute Lösung der Normalgleichungen; bei kleinerem Rang gibt es unendlich viele Lösungen, und $\mathbf{A}^+\mathbf{b}$ wählt unter ihnen die mit der kleinsten Norm. Ausgeführt wird das alles in Kapitel 7, wo die Pseudoinverse als robustester der drei Lösungswege auftritt.

Interaktiv: Die kürzeste unter unendlich vielen Lösungen

Was heißt „die mit der kleinsten Norm“ geometrisch? Das Widget stellt beide Räume nebeneinander: rechts den Bildraum mit $\mathbf{b}$, dem Spaltenraum $\text{col}(\mathbf{A})$ und dem Residuum, links den Urbildraum mit der Menge aller Kleinste-Quadrate-Lösungen. Ist $\mathbf{A}$ singulär, so ist diese Menge eine ganze Gerade.

Interaktives Element — nur in der Web-Fassung

Selbsttest

1. Stellen wir im Blockschema oben das Preset $r < \min(m, n)$ ein, also $m = 5$, $n = 4$, $r = 2$, und schalten wir auf die reduzierte Zerlegung um. Wie viele Zahlen speichert sie?

Lösung: 20

$r \cdot (m + n + 1) = 2 \cdot 10 = 20$. Die volle Zerlegung braucht $m^2 + n^2 + \min(m, n) = 25 + 16 + 4 = 45$ Zahlen, die Matrix selbst $m \cdot n = 20$. Bei diesen Formaten lohnt die Reduktion gegenüber der vollen Zerlegung deutlich, gegenüber $\mathbf{A}$ selbst aber noch gar nicht: Erst kleines r bei großem m und n spart wirklich (Bemerkung 6.3.3).

2. Im Pseudoinversen-Widget steht voreingestellt $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ mit $\mathbf{b} = (1, 5)^\top$. Wie groß ist die Norm $\|\mathbf{A}^+\mathbf{b}\|$ der Minimalnorm-Lösung?

Lösung: 2,121

$\|\mathbf{A}^+\mathbf{b}\| = 2,121$. Es ist $\mathbf{A}^+ = \frac{1}{4} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, also $\mathbf{A}^+\mathbf{b} = (1,5, 1,5)^\top$ mit der Länge $1,5\sqrt{2}$. Jede andere Lösung liegt weiter außen: Bei $t = 1$ auf der Lösungsgeraden sind es schon 2,345, während das Residuum unverändert 2,828 bleibt.

3. In der reduzierten SVD $A = U_r \Sigma_r V_r^T$ sind U_r und V_r Orthogonalmatrizen.

Antwort: Falsch

Ihre Spalten sind orthonormal, es gilt also $U_r^T U_r = I_r$ und $V_r^T V_r = I_r$. Orthogonal heißt eine Matrix aber erst, wenn sie quadratisch ist, und das sind die beiden nur für $r = m$ bzw. $r = n$. In der anderen Reihenfolge sind die Produkte im Allgemeinen keine Einheitsmatrizen, sondern Projektionen (Satz 6.3.8).

4. Für eine Matrix $A \in \mathbb{R}^{1000 \times 50}$ vom Rang 5 haben die drei Faktoren der reduzierten SVD die Formate 1000×5 für U_5 , 5×5 für Σ_5 und 50×5 für V_5 .

Antwort: Wahr

So ist die reduzierte SVD gebaut (Definition 6.3.2). Der Speicherbedarf sinkt dabei von $1000^2 + 50^2 + 50$ auf $5 \cdot (1000 + 50 + 1)$ Zahlen, also von gut einer Million auf 5255.

5. Die Pseudoinverse erfüllt $AA^+ = I_m$ für jede Matrix A .

Antwort: Falsch

Das gilt nur bei vollem Zeilenrang $r = m$ (Korollar 6.3.9). Im Allgemeinen ist $AA^+ = U_r U_r^T$ die Projektion auf $\text{col}(A)$, und die ist von der Einheitsmatrix weit entfernt, sobald $\text{col}(A)$ ein echter Untervektorraum des $\mathbb{R}^m$ ist. In Beispiel 6.3.7 etwa hat AA^+ eine Nullzeile.

6. Ist A quadratisch und invertierbar, so ist $A^+ = A^{-1}$.

Antwort: Wahr

Dann ist $r = m = n$, und beide Spezialfälle aus Korollar 6.3.9 greifen gleichzeitig: A^+ ist eine beidseitige Inverse, und die ist eindeutig. Die Pseudoinverse verallgemeinert die Inverse also wirklich, sie ersetzt sie nicht.

7. Weil $AA^+A = A$ gilt, ist A^+A die Einheitsmatrix.

Antwort: Falsch

Das eine folgt nicht aus dem anderen. In Beispiel 6.3.7 ist $A^+A = \frac{1}{2} \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$, und trotzdem stimmt $AA^+A = A$. Der Grund: Die Projektion A^+A lässt den Zeilenraum $\text{col}(A^T)$ punktweise fest, und mehr braucht es nicht, weil A ohnehin nur auf diesem Teilraum etwas anderes als den Nullvektor produziert.

Vertiefung: Heath §3.6 (SVD und Pseudoinverse); vgl. MML §4.5 für die reduzierten Varianten der Zerlegung und §4.6 für den Anschluss an die Rang-k-Approximation.

6.4 Anwendungen

Die Zerlegung steht. Was fangen wir damit an? Bisher war die SVD vor allem eine Aussage über die Geometrie einer Abbildung. In diesem Abschnitt wird sie zum Werkzeug. Wir lesen die Spektralnrm direkt aus ihr ab, zerlegen A in eine Summe denkbar einfacher Bausteine und gewinnen daraus die beste Approximation von kleinem Rang. Zwei Anwendungen führen vor, wozu das gut ist: Bilder komprimieren und Filmbewertungen vorhersagen.

Die Spektralnrm ist der größte Singulärwert

Die Spektralnrm ist in Abschnitt 3.3 als die von der euklidischen Norm induzierte Operatornorm definiert, also als maximale Streckung $\max_{\|x\|=1} \|Ax\|$. Satz 3.3.7 hat sie dort zu $\sqrt{\lambda_{\max}(A^T A)}$ ausge-

rechnet. So notiert verlangt sie einen Umweg: erst $A^T A$ aufstellen, dann dessen größten Eigenwert bestimmen, dann die Wurzel ziehen. Wer die SVD kennt, liest den Wert dagegen ab.

Satz 6.4.1 (Spektralnorm und größter Singulärwert)

Sei $A \in \mathbb{R}^{m \times n}$ mit Singulärwerten $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n \geq 0$. Dann gilt

$$\|A\|_2 = \sigma_1 = \sigma_{\max}(A). \quad (6.4.1)$$

Vertiefung: Warum die Spektralnorm gleich dem größten Singulärwert ist

Beweis.

- (1) Nach Satz 3.3.7 ist $\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)}$, wobei $\lambda_{\max}(A^T A)$ der größte Eigenwert von $A^T A$ ist.
definiert ist $\|A\|_2$ in Abschnitt 3.3 als Operatornorm, nicht über $\lambda_{\max}$; die Wurzel ist reell, weil $A^T A$ nach Satz 6.2.1 nur nichtnegative Eigenwerte hat
- (2) Definition 6.2.4 setzt $\sigma_i = \sqrt{\lambda_i}$ für die absteigend sortierten Eigenwerte $\lambda_1 \geq \dots \geq \lambda_n$ von $A^T A$. Insbesondere ist $\lambda_{\max}(A^T A) = \lambda_1$ und damit $\sigma_1 = \sqrt{\lambda_1}$.
::why[die Singulärwerte erben die Sortierung der Eigenwerte, weil die Wurzel auf $[0, \infty)$ monoton wächst]
- (3) Beide Ausdrücke stimmen also überein:

$$\|A\|_2 = \sqrt{\lambda_{\max}(A^T A)} = \sqrt{\lambda_1} = \sigma_1.$$

erste Gleichheit nach Schritt 1, zweite wegen $\lambda_{\max} = \lambda_1$, dritte nach Schritt 2

□

Die Gleichung (6.4.1) sieht man gelegentlich als Definition geschrieben. Das wäre eine Doppelbelegung: Die Matrixnorm $\|A\|_2$ trägt aus Kapitel 3 bereits eine Definition, und dass sie mit σ_1 übereinstimmt, ist eine Aussage, die bewiesen werden will. Das Bild dazu kennen wir aus Abschnitt 6.1: σ_1 ist die längste Halbachse der Bildellipse.

Die SVD als Summe von Rang-1-Matrizen

Die Zerlegung $A = U \Sigma V^T$ verpackt alles in drei Matrizen. Multiplizieren wir sie aus, kommt eine Darstellung zum Vorschein, die für Anwendungen viel handlicher ist. Dass Σ außerhalb der Diagonalen nur Nullen trägt, schaltet nämlich fast alle Summanden ab: Übrig bleibt für jeden von null verschiedenen Singulärwert genau ein Term.

Satz 6.4.2 (Summenform der SVD)

Sei $A \in \mathbb{R}^{m \times n}$ mit $\text{rang}(A) = r$ und der Zerlegung $A = U \Sigma V^T$ aus Satz 6.2.13 (Abschnitt 6.2). Dann gilt

$$A = \sum_{i=1}^r \sigma_i u_i v_i^T. \quad (6.4.2)$$

Jeder Summand $u_i v_i^T \in \mathbb{R}^{m \times n}$ hat Rang 1, und in der Frobeniusnorm aus Abschnitt 3.1 gilt

$$\|u_i v_i^T\|_F = 1.$$

Vertiefung: Herleitung der Rang-1-Summenform

Beweis.

- (1) Wir schreiben den Eintrag (a, b) des Produkts aus:

$$(\mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top)_{ab} = \sum_{i=1}^m \sum_{j=1}^n U_{ai} \Sigma_{ij} V_{bj} = \sum_{i=1}^r \sigma_i U_{ai} V_{bi}.$$

Σ_{ij} ist null, außer für $i = j \leq r$; dort steht σ_i . Von der Doppelsumme bleibt also nur die Diagonale übrig

- (2) Die rechte Seite ist genau der Eintrag (a, b) von $\sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$, denn $(\mathbf{u}_i \mathbf{v}_i^\top)_{ab} = U_{ai} V_{bi}$. Da die Einträge übereinstimmen, stimmen die Matrizen überein, und (6.4.2) ist gezeigt.

das äußere Produkt $\mathbf{x}\mathbf{y}^\top$ hat an der Stelle (a, b) den Eintrag $x_a y_b$; hier ist $\mathbf{u}_i$ die i -te Spalte von $\mathbf{U}$ und $\mathbf{v}_i$ die i -te Spalte von $\mathbf{V}$

- (3) Die beiden Zusatzaussagen sind schnell erledigt. Jede Spalte von $\mathbf{u}_i \mathbf{v}_i^\top$ ist ein Vielfaches von $\mathbf{u}_i$, der Spaltenraum also eindimensional und der Rang damit 1; und für die Frobeniusnorm faktorisiert die Doppelsumme:

$$\|\mathbf{u}_i \mathbf{v}_i^\top\|_F^2 = \left(\sum_{a=1}^m U_{ai}^2 \right) \left(\sum_{b=1}^n V_{bi}^2 \right) = 1 \cdot 1 = 1.$$

$\mathbf{u}_i$ und $\mathbf{v}_i$ sind Spalten von Orthogonalmatrizen, haben also Länge 1 und sind insbesondere nicht der Nullvektor; in der Doppelsumme hängt U_{ai}^2 nicht von b und V_{bi}^2 nicht von a ab, beide Klammern sind quadrierte Längen von Einheitsvektoren

□

Die Summenform trägt dieselbe Information wie die reduzierte SVD $\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$ aus Abschnitt 6.3, nur als Summe statt als Produkt geschrieben. Die Schreibweise macht aber etwas sichtbar, das im Produkt versteckt bleibt: Die Matrix $\mathbf{A}$ ist eine gewichtete Summe von r Bausteinen, die in der Frobeniusnorm alle gleich groß sind. Was die Bausteine unterscheidet, sind allein die Gewichte σ_i . Ein großes σ_i markiert eine Richtung, in der viel von $\mathbf{A}$ steckt, ein kleines eine, in der wenig passiert. Und weil die Singulärwerte absteigend sortiert sind, stehen die wichtigen Terme vorn.

Daraus wird sofort eine Idee: Nehmen wir nur die ersten Summanden und lassen den Rest weg.

Die beste Approximation von kleinem Rang

Definition 6.4.3 (Rang-k-Approximation)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r$ und der Summenform (6.4.2). Für $k \leq r$ heißt

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \quad (6.4.3)$$

die *Rang-k-Approximation* (low-rank approximation) von $\mathbf{A}$. Für $k \geq 1$ hat sie tatsächlich Rang k , denn wegen $k \leq r$ sind alle beteiligten σ_i positiv und die $\mathbf{u}_i$ orthonormal.

Wie in Abschnitt 6.3 lässt sich die Summe wieder zu einem Produkt zusammenfassen: Mit den ersten k Spalten $\mathbf{U}_k$ von $\mathbf{U}$, den ersten k Spalten $\mathbf{V}_k$ von $\mathbf{V}$ und $\mathbf{\Sigma}_k = \text{diag}(\sigma_1, \dots, \sigma_k)$ ist gleichbedeutend

$$\mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top.$$

Das Abschneiden ist schnell erledigt. Interessant ist, was wir dabei verlieren und ob eine andere Matrix vom Rang k es besser könnte. Auf beide Fragen gibt derselbe Satz Antwort, und die Antwort ist bemerkenswert glatt.

Satz 6.4.4 (Eckart und Young: beste Approximation von Rang k)

Sei $A \in \mathbb{R}^{m \times n}$ mit Singulärwerten $\sigma_1 \geq \dots \geq \sigma_r > 0$ und sei $k < r$. Dann löst A_k aus (6.4.3) beide Approximationsprobleme

$$\min_{\substack{B \in \mathbb{R}^{m \times n} \\ \text{rang}(B) \leq k}} \|A - B\|_F \quad \text{und} \quad \min_{\substack{B \in \mathbb{R}^{m \times n} \\ \text{rang}(B) \leq k}} \|A - B\|_2,$$

und die zugehörigen Fehler sind

$$\|A - A_k\|_F = \sqrt{\sum_{i=k+1}^r \sigma_i^2}, \quad \|A - A_k\|_2 = \sigma_{k+1}. \quad (6.4.4)$$

Wir verzichten auf den Beweis; nachlesen lässt er sich in der Literatur (vgl. Heath §3.6; MML §4.6). Die Namensgebung ist übrigens verkürzt: Eckart und Young (1936) haben den Fall der Frobeniusnorm behandelt, die Aussage für die Spektralnorm geht auf Mirsky (1960) zurück. Die Fehlerformeln selbst sind leicht zu merken und sagen viel:

Bemerkung 6.4.5 (Was der Satz liefert und was nicht)

- **Der Fehler steht in den weggeworfenen Singulärwerten.** In der Spektralnorm ist es der größte weggelassene Wert σ_{k+1} , in der Frobeniusnorm die Wurzel aus der Summe aller weggelassenen Quadrate. Wir kennen den Approximationsfehler also, bevor wir approximieren.
- **Beide Normen führen zum selben Sieger.** Das ist keine Selbstverständlichkeit: Zwei verschiedene Gütemaße haben im Allgemeinen zwei verschiedene Optima. Hier fallen sie zusammen.
- **Der Satz sagt, dass A_k das Minimum annimmt, nicht dass es der einzige Minimierer ist.** In der Frobeniusnorm ist A_k eindeutig, solange $\sigma_k > \sigma_{k+1}$ gilt; bei Gleichheit gibt es mehrere. In der Spektralnorm sieht es schlechter aus. Für $A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}$ und $k = 1$ etwa hat jede Matrix $B = \begin{pmatrix} b & 0 \\ 0 & 0 \end{pmatrix}$ mit $b \in [1, 5]$ den Fehler $\|A - B\|_2 = \max(|3 - b|, 2) = 2 = \sigma_2$. A_1 ist also *eine* beste Approximation, nicht *die*.

Wie wählen wir k ?

Der Satz von Eckart und Young sagt, wie gut jedes k ist. Er sagt nicht, welches wir nehmen sollen. Diese Entscheidung ist ein Kompromiss zwischen Genauigkeit und Aufwand, und dafür gibt es drei gebräuchliche Kriterien.

Bemerkung 6.4.6 (Drei Kriterien für die Wahl von k)

1. *Relativer Fehler in der Spektralnorm.* Aus (6.4.4) folgt

$$\frac{\|A - A_k\|_2}{\|A\|_2} = \frac{\sigma_{k+1}}{\sigma_1},$$

denn der Nenner ist nach Satz 6.4.1 gerade σ_1 . Wir wählen das kleinste k , für das dieser Quotient unter einer Schranke bleibt.

2. *Relativer Fehler in der Frobeniusnorm.* Mit derselben Rechnung und der Identität $\|\mathbf{A}\|_F = \sqrt{\sum_i \sigma_i^2}$ aus Korollar 3.4.4 ist

$$\frac{\|\mathbf{A} - \mathbf{A}_k\|_F}{\|\mathbf{A}\|_F} = \frac{\sqrt{\sum_{i=k+1}^r \sigma_i^2}}{\sqrt{\sum_{i=1}^r \sigma_i^2}}.$$

Dieses Maß nimmt alle Richtungen zusammen, das erste nur die schlechteste.

3. *Energiekriterium.* Statt den Fehler zu beschränken, verlangen wir einen Mindestanteil an erklärter „Energie“:

$$\frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^r \sigma_i^2} \geq p,$$

üblich sind $p = 0,9$ oder $p = 0,99$. Das ist nur eine Umformulierung von Kriterium 2, denn die linke Seite ist 1 minus dem Quadrat des relativen Frobeniusfehlers.

Vorsicht bei Kriterium 3: Wenn σ_1 die übrigen Singulärwerte weit überragt, ist der Energieanteil schon bei $k = 1$ nahe bei eins, obwohl die Approximation noch sehr grob ist. Ein Kriterium, das so leicht zufrieden ist, taugt wenig als alleiniger Ratgeber.

Vertiefung: Typische Singulärwert-Verläufe und die Ellenbogen-Heuristik

Bemerkung 6.4.7 (Typische Singulärwert-Verläufe und der Ellenbogen)

Wie gut sich eine Matrix überhaupt komprimieren lässt, steht im Verlauf ihrer Singulärwerte. Drei Muster begegnen uns immer wieder:

1. *Schneller Abfall*, etwa $\sigma_i \approx c \cdot e^{-\alpha i}$. Wenige Werte dominieren, alles Weitere ist Kleinkram. Solche Matrizen sind der ideale Fall für eine Rang- k -Approximation.
2. *Linearer Abfall*, $\sigma_i \approx c - \alpha i$. Die Approximation gelingt noch ordentlich, der Kompressionsfaktor bleibt aber mittelmäßig.
3. *Langsamer Abfall*, $\sigma_i \approx c \cdot i^{-\alpha}$. Viele Komponenten tragen ähnlich viel bei; ein kleines k verliert dann zwangsläufig Substanz.

Heuristik: Tragen wir $\log \sigma_i$ gegen i auf, so wird ein exponentieller Abfall zur Geraden. Interessant ist der Knick, an dem die Kurve flach wird, der sogenannte *Ellenbogen* (elbow). Er trennt die Richtungen, in denen wirklich Struktur steckt, von denen, in denen nur noch Rauschen sitzt, und liefert einen brauchbaren Startwert für k . Eine Garantie ist das nicht; manche Spektren haben schlicht keinen Ellenbogen.

1. Ein Spektrum mit langsam abfallenden Singulärwerten, $\sigma_i \approx c \cdot i^{-\alpha}$, eignet sich besonders gut für eine Rang- k -Approximation.

Antwort: Falsch

Es ist der ungünstigste Fall: Viele Komponenten tragen ähnlich viel bei, der Fehler $\sqrt{\sum_{i>k} \sigma_i^2}$ sinkt entsprechend zäh, und ein kleines k verliert echte Struktur. Gut komprimieren lässt sich der exponentielle Abfall.

Anwendung: Datenkompression

Ein Graustufenbild ist eine Matrix. Jeder Pixel ist ein Eintrag, seine Helligkeit der Wert. Farbbilder bestehen aus drei solchen Matrizen, einer pro Kanal. Damit ist die Bildkompression eine reine Übung in Rang- k -Approximation.

Beispiel 6.4.8 (Ein Bild mit 659×512 Pixeln)

Das Bild sei $A \in \mathbb{R}^{659 \times 512}$. Vollständig gespeichert sind das

$$659 \cdot 512 = 337\,408 \text{ Zahlen (pro Kanal).}$$

Für die Rang-50-Approximation brauchen wir dagegen nur die ersten 50 Spalten von U , die ersten 50 Singulärwerte und die ersten 50 Spalten von V :

$$\underbrace{659 \cdot 50}_{U_{50}} + \underbrace{50}_{\Sigma_{50}} + \underbrace{50 \cdot 512}_{V_{50}} = 32\,950 + 50 + 25\,600 = 58\,600.$$

Das sind $58\,600/337\,408 \approx 17,4\%$ des Originals, wir sparen also rund 83% des Speichers. Beliebig weit trägt das nicht. Der Speicherbedarf wächst linear in k , und sobald $k \cdot (659 + 512 + 1) > 337\,408$ ist, also ab $k = 288$, legen wir mehr Zahlen ab als das Bild selbst hat. Kompression gibt es hier nur für $k \leq 287$.

Algorithmus 6.4.9 (Kompression mit der SVD)

1. Berechne die SVD $A = U\Sigma V^T$.
2. Wähle k anhand der Kriterien aus Bemerkung 6.4.6.
3. Bilde die Approximation $A_k = U_k \Sigma_k V_k^T$.
4. Speichere nur U_k , Σ_k und V_k ; das Bild wird bei Bedarf daraus zurückgerechnet.

Interaktiv: Bildkompression zum Schieben

Wie viele Rang-1-Terme braucht ein Bild eigentlich? Probieren wir es aus, an einem synthetisch erzeugten Graustufenbild; als Matrix verhält es sich genauso wie ein Foto. Der Schieber gibt k vor, die dritte Tafel zeigt die Differenz $A - A_k$ als Bild, und die beiden Kurven sind die Fehlerformeln aus (6.4.4), an diesem Bild ausgewertet.

Interaktives Element — nur in der Web-Fassung

Mit wachsendem Rang sinken beide Fehler; die Kurven im Widget zeigen, wie schnell die zusätzlichen Terme das Bild verbessern.

Wer lieber mit echten Fotos spielt, findet unter shinyapps.io eine Shiny-App zur abgeschnittenen SVD.

Anwendung: Empfehlungssysteme

Der zweite Klassiker heißt *kollaboratives Filtern* (collaborative filtering). Ein Streamingdienst kennt m Nutzer und n Filme und will für jedes Paar vorhersagen, wie gut der Film ankommt. Die Bewertungen bilden eine Matrix, aber eine mit einem Haken: Fast niemand hat fast alles gesehen.

Beispiel 6.4.10 (Eine dünn besetzte Bewertungsmatrix)

Zeilen sind Nutzer, Spalten sind Filme, ein Fragezeichen steht für eine fehlende Bewertung:

$$\mathbf{R} = \begin{pmatrix} 5 & ? & 1 & ? & 4 \\ ? & 3 & ? & 4 & ? \\ 2 & 1 & ? & ? & 5 \\ ? & 5 & 4 & 3 & ? \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

Hier sind $m = 4$ und $n = 5$, und von 20 möglichen Einträgen sind 11 bekannt. In der Praxis ist die Matrix noch weit dünner besetzt: Millionen von Nutzern, Hunderttausende von Titeln, und pro Zeile eine Handvoll Bewertungen.

Füllen wir die Lücken mit den Spaltenmitteln der bekannten Einträge, so sind das der Reihe nach 3,5, 3, 2,5, 3,5 und 4,5.

Warum sollte eine Zerlegung dabei helfen? Weil Geschmack Struktur hat. Die Bewertungen eines Nutzers hängen untereinander zusammen: Wer zwei Science-Fiction-Filme mag, mag wahrscheinlich auch den dritten. Die Vorlieben von Millionen Nutzern lassen sich deshalb erstaunlich gut durch wenige Zahlen beschreiben. In der Sprache dieses Kapitels heißt das: Die Bewertungsmatrix ist näherungsweise von kleinem Rang.

Algorithmus 6.4.11 (Empfehlungen über eine Rang-k-Approximation)

1. Ergänze die fehlenden Einträge grob, etwa durch die Spaltenmittel der bekannten Bewertungen.
2. Berechne von der aufgefüllten Matrix die Rang-k-Approximation $\mathbf{R}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$.
3. Lies die Vorhersagen an den zuvor fehlenden Stellen von $\mathbf{R}_k$ ab.

Bemerkung 6.4.12 (Was die Faktoren bedeuten)

Die drei Bestandteile der Zerlegung lassen sich hier ungewöhnlich gut deuten:

- Die *rechten* Singulärvektoren $\mathbf{v}_j \in \mathbb{R}^n$ leben im Filmraum. Sie sind gelernte Genres: $\mathbf{v}_1$ könnte eine Achse von Action nach Drama beschreiben, $\mathbf{v}_2$ eine von Komödie nach Thriller. Diese Achsen hat niemand vorgegeben, sie fallen aus den Daten.
- Die *linken* Singulärvektoren $\mathbf{u}_i \in \mathbb{R}^m$ leben im Nutzerraum und geben an, wie stark die einzelnen Nutzer auf diese Genre-Achsen ansprechen.
- Die Singulärwerte σ_i gewichten, wie viel eine Achse überhaupt erklärt.

Eine Vorhersage ist damit nichts anderes als ein Skalarprodukt: Nutzerprofil mal Genreprofil, gewichtet mit σ_i und über die ersten k Achsen summiert.

Interaktiv: Rang-k-Glättung einer Bewertungsmatrix

Wie gut trifft dieses Verfahren eine Bewertung, die es nicht kennt? Das Widget rechnet Beispiel 6.4.10 durch und lässt uns eine bekannte Bewertung zurückhalten; sie wird damit zur Testgröße. Der Vergleich der drei Auffüllregeln zeigt zusätzlich, wie stark die Vorhersage an Schritt 1 von Algorithmus 6.4.11 hängt.

Interaktives Element — nur in der Web-Fassung

Ein Einwand bleibt, und er ist gewichtig: Schritt 1 erfindet Daten, und Schritt 2 behandelt diese Erfindungen wie Messwerte. Denn $\mathbf{R}_k$ minimiert nach Satz 6.4.4 den Fehler über *alle* Einträge, also auch über die selbstgesetzten Mittelwerte.

Vertiefung: Stärken und Schwächen des Verfahrens, und was danach kommt

Bemerkung 6.4.13 (Stärken, Schwächen, Ausblick)

Für den Ansatz spricht einiges:

- **Speicher.** Statt $m \cdot n$ Zahlen genügen $(m+n) \cdot k + k$, und für $k \ll \min(m, n)$ ist das drastisch weniger.
- **Glättung.** Das Abschneiden wirkt als *Regularisierung*: Die Rang- k -Approximation kann den Zufall in einzelnen Bewertungen gar nicht nachbilden, weil ihr die Freiheitsgrade fehlen.
- **Robustheit gegen Lücken.** Auch bei sehr spärlichen Daten kann das Verfahren brauchbare Schätzungen liefern, solange die zugrunde liegende Struktur wirklich niedrigdimensional ist.

Gegen ihn spricht der Einwand oben, und modernere Verfahren umgehen ihn: Unter dem Stichwort *Matrix Completion* sucht man direkt eine Matrix von kleinem Rang, die nur an den beobachteten Stellen gut passt; die (nichtnegative) Matrixfaktorisierung ist die bekannteste Variante.

Wann lohnt sich die SVD?

Bemerkung 6.4.14 (Ideale Situationen für die SVD)

1. *Niedrig-Rang-Strukturen:* $\text{rang}(\mathbf{A}) \ll \min(m, n)$, oder wenigstens näherungsweise.
2. *Schnell abfallende Singulärwerte:* wenige Richtungen dominieren.
3. *Überbestimmte Systeme:* $m > n$ und womöglich inkonsistent, wie bei Kleinste-Quadrate-Problemen (Kapitel 7).
4. *Datenanalyse:* Die Hauptkomponentenanalyse (PCA) lässt sich direkt aus der SVD der zentrierten Datenmatrix $\mathbf{X}$ ablesen, ohne dass wir das Kreuzprodukt $\mathbf{X}^\top \mathbf{X}$ je aufstellen müssen. Dieses Kreuzprodukt ist bis auf einen Faktor die empirische Kovarianzmatrix; ausgeführt wird die PCA in Abschnitt 8.2.
5. *Numerische Robustheit:* als Alternative zu den Normalgleichungen (Abschnitt 7.3).

Punkt 4 und Punkt 5 haben denselben Kern. Das Produkt $\mathbf{X}^\top \mathbf{X}$ zu bilden kostet Genauigkeit, denn es quadriert die Konditionszahl (Abschnitt 3.5). Die SVD arbeitet direkt an $\mathbf{X}$ und vermeidet diesen Schritt.

Umsonst ist die Robustheit nicht.

Vertiefung: Genauer Aufwandsvergleich

Bemerkung 6.4.15 (Rechenaufwand)

Für $A \in \mathbb{R}^{m \times n}$ gilt in der Landau-Notation (Abschnitt 2.4):

- **SVD:** $O(\min(m^2n, mn^2))$; für $m \gg n$ sind das rund $4mn^2$ Operationen.
- **Normalgleichungen:** $O(mn^2 + n^3)$; für $m \gg n$ dominiert der Aufbau von $A^T A$ mit rund mn^2 Operationen.

Für $m \gg n$ ist die SVD also grob viermal so teuer. Das ist der übliche Kompromiss: Geschwindigkeit gegen Stabilität. Wer schlecht konditionierte Daten hat, zahlt den Faktor gern.

SVD und Eigenwertzerlegung im Vergleich

Zum Abschluss stellen wir die beiden Zerlegungen nebeneinander, die dieses und das dritte Kapitel getragen haben. Gemeint ist in der mittleren Spalte die Eigenwertzerlegung $A = PDP^{-1}$ mit beliebigem invertierbarem P ; erst im symmetrischen Fall wird daraus die Spektralzerlegung mit orthogonalem P aus Abschnitt 3.3.

Eigenschaft	Eigenwertzerlegung	SVD
Matrixtyp	quadratisch	beliebig, $m \times n$
orthogonale Faktoren	nur für symmetrische Matrizen	immer
numerische Stabilität	kann heikel werden	stabil
singuläre Matrizen	Eigenwert 0; ohne Basis aus Eigenvektoren gibt es die Zerlegung gar nicht	unkritisch, $\sigma_i = 0$ zeigt den Rangabfall an
geometrische Deutung	Hauptachsen (im symmetrischen Fall)	optimale Koordinaten in Urbild und Bild

Selbsttest

1. Stellen wir im Rang-k-Explorer oben $k = 2$ ein. Wie viel Prozent der Energie $\sum_i \sigma_i^2$ stecken dann schon in A_2 ?

Lösung: 98.59

98,59 %. Die ersten beiden Singulärwerte des Testbilds sind 26,475 und 6,060, alle übrigen zusammen tragen nur noch den Rest bei. Trotzdem sieht die Rekonstruktion bei $k = 2$ noch sichtbar falsch aus: Der Energie-Anteil misst Quadrate und wird deshalb schon von den großflächigen Helligkeiten fast ausgeschöpft, während die feinen Kanten in den kleinen σ_i stecken. Ein Kriterium allein reicht für die Wahl von k nicht (Bemerkung 6.4.6).

2. Halten wir im Empfehlungs-Widget Adas Bewertung für „Sternenstaub“ zurück, also die 5 links oben, und setzen wir $k = 2$ bei Auffüllung mit dem Filmmittel. Welche Vorhersage steht danach in diesem Feld?

Lösung: 1.8

Rund 1,8, bei einem wahren Wert von 5. Der Grund ist Schritt 1 von Algorithmus 6.4.11: Ohne Adas Bewertung sinkt das Mittel der Spalte „Sternenstaub“ von 3,5 auf 2,0, und dieser selbst gesetzte Wert bestimmt die Vorhersage fast allein. Bei elf bekannten Bewertungen sagt das Ergebnis mehr über die Füllregel als über Ada; genau das ist der Einwand gegen das Auffüllen in Schritt 1.

3. Die Spektralnorm $\|A\|_2$ ist der betragsgrößte Eigenwert von A .

Antwort: Falsch

Sie ist der größte *Singulärwert* (Satz 6.4.1). Für symmetrische Matrizen fallen beide Größen bis aufs Vorzeichen zusammen, sonst nicht. Gegenbeispiel: $A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ hat den doppelten Eigenwert 0, aber $A^T A = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$ und damit $\|A\|_2 = 1$. Für nicht quadratische Matrizen existieren Eigenwerte ohnehin nicht.

4. Unter allen Matrizen vom Rang höchstens k minimiert A_k den Fehler gleichzeitig in der Frobenius- und in der Spektralnorm.

Antwort: Wahr

So steht es in Satz 6.4.4. Bemerkenswert daran ist, dass zwei verschiedene Gütemaße dieselbe Lösung zulassen; erwarten müsste man das nicht. Der einzige Minimierer ist A_k deshalb noch lange nicht, siehe Bemerkung 6.4.5.

5. Für $A = \begin{pmatrix} 3 & 0 \\ 0 & 2 \end{pmatrix}$ ist $\begin{pmatrix} 2,5 & 0 \\ 0 & 2,5 \end{pmatrix}$ die beste Rang-1-Approximation.

Antwort: Falsch

Diese Matrix hat Rang 2, ist also gar nicht zugelassen. Die Singulärwerte von A sind $\sigma_1 = 3$ und $\sigma_2 = 2$, die beste Rang-1-Approximation ist $A_1 = \begin{pmatrix} 3 & 0 \\ 0 & 0 \end{pmatrix}$, und der Fehler beträgt $\|A - A_1\|_2 = \sigma_2 = 2$.

6. Solange $k < \min(m, n)$ gilt, spart die Rang- k -Approximation immer Speicher.

Antwort: Falsch

Sie kostet $k(m + n + 1)$ Zahlen gegen mn für die volle Matrix, und das ist für großes k mehr. Bei $m = n = 100$ und $k = 50$ etwa speichern wir $50 \cdot 201 = 10\,050$ statt $10\,000$ Zahlen. Ein Gewinn entsteht erst für $k < mn/(m + n + 1) \approx 49,8$, hier also für $k \leq 49$.

7. Ist $\sigma_{k+1} = 0$, so ist die Rang- k -Approximation exakt.

Antwort: Wahr

Weil die Singulärwerte absteigend sortiert sind, folgt aus $\sigma_{k+1} = 0$ sofort $\text{rang}(A) \leq k$. Die Summenform (6.4.2) hat dann gar nicht mehr als k Summanden, also ist $A_k = A$. Satz 6.4.4 selbst setzt $k < r$ voraus und greift hier nicht mehr. Setzen wir $\sigma_i = 0$ für $i > r$, sagt (6.4.4) aber dasselbe: Der Fehler ist null.

Vertiefung: vgl. Heath §3.6 (Singulärwertzerlegung und ihre Anwendungen) sowie MML §4.5 und §4.6 (SVD und Matrixapproximation).

6.5 Zusammenfassung

Dieses Kapitel hatte ein einziges Ziel: eine Zerlegung, die für jede Matrix funktioniert, gleich welches Format sie hat und ob sie symmetrisch ist. Seit Abschnitt 6.2 steht sie da, und die beiden folgenden Abschnitte haben gezeigt, was sich mit ihr anfangen lässt. Sammeln wir ein, was bleiben soll.

Was wir gelernt haben

Bemerkung 6.5.1 (Die fünf Kernkonzepte)

1. **Die SVD erweitert die Eigenwertzerlegung.** Sie verlangt weder eine quadratische noch eine symmetrische Matrix und existiert für jedes $A \in \mathbb{R}^{m \times n}$ (Abschnitt 6.1).
2. **Singulärwerte sind Wurzeln von Eigenwerten:** $\sigma_i(A) = \sqrt{\lambda_i(A^T A)}$ (Definition 6.2.4). Der Umweg über $A^T A$ verwandelt eine beliebige Matrix in eine symmetrische, auf die der Spektralsatz passt.
3. **Der Hauptsatz:** $A = U \Sigma V^T$ mit orthogonalem $U \in \mathbb{R}^{m \times m}$, orthogonalem $V \in \mathbb{R}^{n \times n}$ und „diagonalem“ $\Sigma \in \mathbb{R}^{m \times n}$ (Satz 6.2.13).
4. **Reduzierte SVD:** $A = U_r \Sigma_r V_r^T$ mit $r = \text{rang}(A)$. Die Nullblöcke von Σ tragen nichts bei und dürfen weg (Definition 6.3.2).
5. **Pseudoinverse:** $A^+ = V_r \Sigma_r^{-1} U_r^T$, die Verallgemeinerung der Inversen auf beliebige Matrizen (Definition 6.3.5).

Angewendet haben wir das an drei Stellen. Die Pseudoinverse löst Kleinste-Quadrate-Probleme über $\hat{x} = A^+ b$, und zwar auch dann, wenn die Normalengleichungen mangels vollen Spaltenrangs versagen (Bemerkung 6.3.12). Die abgeschnittene Zerlegung liefert die beste Approximation niedrigen Rangs, die es überhaupt gibt, mit einem Fehler, den die weggelassenen Singulärwerte exakt beziffern (Abschnitt 6.4). Und weil diese Approximation gleichzeitig Speicher spart und Rauschen glättet, ist sie das Arbeitspferd für Datenkompression und Datenanalyse, von der Hauptkomponentenanalyse (Abschnitt 8.2) bis zum Empfehlungssystem (Beispiel 6.4.10).

Querverbindungen

Die SVD ist kein isoliertes Werkzeug. Über die Eigenwerte von $A^T A$ hängt sie an der Eigenwerttheorie (Satz 6.2.1, Korollar 6.2.2), über U und V an den fundamentalen Unterräumen samt fertigen Orthonormalbasen (Satz 6.2.11), über AA^+ und A^+A an den Projektionen (Satz 6.3.8) und über die Minimierungsprobleme in Abschnitt 6.4 und Kapitel 7 an der Optimierung. Zwei Verbindungen lohnen den eigenen Blick:

- **Normen:** Der größte Singulärwert ist die Spektralnorm, $\|A\|_2 = \sigma_1$ (Abschnitt 3.3); die Frobeniusnorm nimmt alle Singulärwerte zusammen, $\|A\|_F = \sqrt{\sum_i \sigma_i^2}$ (Abschnitt 3.4). Für quadratische, invertierbare Matrizen ist der Quotient σ_1/σ_n die Spektralkondition $\kappa_2(A)$ (Abschnitt 3.5); sie beschränkt, wie stark ein relativer Datenfehler in der Lösung anwachsen kann (Abschnitt 4.2).
- **Matrixzerlegungen:** LU und Cholesky zerlegen, um Gleichungssysteme schnell zu lösen. Die SVD zerlegt, um die Geometrie einer Abbildung sichtbar zu machen. Sie ist die teuerste der drei und die einzige, die für jede Matrix existiert.

So viele Fäden laufen selten in einer einzigen Zerlegung zusammen. Wer die SVD einer Matrix kennt, kennt ihren Rang, ihre Norm, ihre Kondition, ihre fundamentalen Unterräume und die beste Näherung jeder Stufe.

Praktische Hinweise

Drei Zuschnitte der Zerlegung sind gebräuchlich, und ihre Namen werden gern verwechselt.

Variante	Zerlegung	Formate	Wozu
volle SVD	$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$	$\mathbf{U}$ ist $m \times m$, $\mathbf{V}$ ist $n \times n$	exakt, alle fundamentalen Unterräume auf einmal
reduzierte SVD	$\mathbf{A} = \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^\top$	$\mathbf{U}_r$ ist $m \times r$, $\mathbf{V}_r$ ist $n \times r$	exakt, ohne die Nullblöcke
truncated SVD	$\mathbf{A} \approx \mathbf{A}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top, k < r$	$\mathbf{U}_k$ ist $m \times k$, $\mathbf{V}_k$ ist $n \times k$	Approximation, Kompression

Nur die dritte Variante verliert Information, und wie viel, sagen die abgeschnittenen Singulärwerte. Die Wahl von k ist deshalb keine technische Kleinigkeit, sondern eine Modellierungsentscheidung; die Kriterien und die Kriterien stehen in Bemerkung 6.4.6; typische Singulärwert-Verläufe und die Ellenbogen-Heuristik sind als Vertiefung markiert.

Ein Hinweis wiegt schwerer, als er klingt. Die Definition der Singulärwerte führt über die Eigenwerte von $\mathbf{A}^\top \mathbf{A}$; als *Rechenweg* ist dieser Umweg trotzdem der schlechteste von allen. Das Produkt quadriert die Kondition, $\kappa_2(\mathbf{A}^\top \mathbf{A}) = \kappa_2(\mathbf{A})^2$, und beim Aufstellen verschwinden kleine Beiträge neben großen im Rundungsfehler. Was dann noch fehlt, müsste eine Differenz fast gleicher Zahlen hergeben, also ausgerechnet Auslöschung. Die legt einen Verlust aber nur offen, statt ihn zu heilen (Abschnitt 4.3). Als Faustregel: Über $\mathbf{A}^\top \mathbf{A}$ sind Singulärwerte unterhalb von $\sqrt{\varepsilon} \cdot \sigma_1 \approx 1,5 \cdot 10^{-8} \cdot \sigma_1$ nicht mehr zu retten.

Vertiefung: Wie der Umweg einen Singulärwert verschluckt

Beispiel 6.5.2 (Der Umweg über $\mathbf{A}^\top \mathbf{A}$)

Betrachten wir

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ \delta & 0 \\ 0 & \delta \end{pmatrix}, \quad \delta = 10^{-8}, \quad \mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 1 + \delta^2 & 1 \\ 1 & 1 + \delta^2 \end{pmatrix}.$$

Eine Matrix der Bauart $\begin{pmatrix} a & b \\ b & a \end{pmatrix}$ hat die Eigenwerte $a+b$ und $a-b$, hier also $(1+\delta^2)+1 = 2+\delta^2$ und $(1+\delta^2)-1 = \delta^2$. Damit besitzt $\mathbf{A}$ die Singulärwerte $\sigma_1 = \sqrt{2+\delta^2} \approx 1,4142$ und $\sigma_2 = \delta = 10^{-8}$; die Matrix hat Rang 2. Der kleinere Eigenwert ist buchstäblich eine Differenz fast gleicher Zahlen. Er hängt allein an dem Summanden δ^2 im Diagonaleintrag, und genau der ist gefährdet. In doppelter Genauigkeit geht das schief. Der Abstand von 1 zur nächstgrößeren darstellbaren Zahl ist die Maschinengenauigkeit $\varepsilon \approx 2,22 \cdot 10^{-16}$, und gerundet wird zur nächstgelegenen. Weil $\delta^2 = 10^{-16}$ kleiner als die halbe Schrittweite $1,11 \cdot 10^{-16}$ ausfällt, ergibt sich $\text{fl}(1 + \delta^2) = 1$. Die berechnete Matrix ist

$$\widehat{\mathbf{A}^\top \mathbf{A}} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} \quad \text{mit den Eigenwerten 2 und 0.}$$

Die Rechnung meldet $\sigma_2 = 0$ und damit Rang 1. Ein Verfahren, das direkt mit $\mathbf{A}$ arbeitet, bestimmt σ_2 dagegen bis auf einen absoluten Fehler der Größenordnung $\varepsilon \cdot \sigma_1$, also auf etwa $3 \cdot 10^{-16}$ genau, und sieht den Rang 2.

Die gebräuchlichen Verfahren arbeiten deshalb direkt mit $\mathbf{A}$ und benutzen ausschließlich orthogonale Transformationen, die alle Singulärwerte unverändert lassen (Satz 3.4.7 in Abschnitt 3.4); der Klassiker ist der Algorithmus von Golub und Reinsch. Bei sehr großen Matrizen lohnt die volle Zerlegung ohnehin nicht: Dort berechnen wir gleich nur die k größten Singulärwerte samt ihren Singulärvektoren, ohne $\mathbf{A}$ je vollständig zu zerlegen. In R stehen beide Wege bereit:

```

s <- svd(A)                # s$d: Singulärwerte, s$u und s$v: Singulärvektoren
k <- 5
A_k <- s$u[, 1:k] %*% diag(s$d[1:k], nrow = k) %*% t(s$v[, 1:k])

irlba::irlba(A, nv = k)    # nur die k größten, für sehr große Matrizen

```

Selbsttest

Zum Abschluss fünf Aussagen über $A^T A$ und die Diagonalisierbarkeit. Welche davon sind korrekt?

1. Für $A \in \mathbb{R}^{m \times n}$ ist $A^T A \in \mathbb{R}^{m \times m}$.

Antwort: Falsch

Die Formate ergeben $(n \times m) \cdot (m \times n)$, also ist $A^T A$ eine $n \times n$ -Matrix (Satz 6.2.1). Die $m \times m$ -Matrix ist AA^T . In Beispiel 6.2.3 mit $m = 3$ und $n = 2$ war $A^T A$ entsprechend 2×2 groß.

2. Für $A \in \mathbb{R}^{m \times n}$ ist $A^T A$ symmetrisch.

Antwort: Wahr

Transponieren dreht die Reihenfolge um: $(A^T A)^T = A^T (A^T)^T = A^T A$ (Satz 6.2.1). Das gilt für jede Matrix A , ohne jede Voraussetzung.

3. Die Eigenwerte von $A^T A$ sind die Singulärwerte von A .

Antwort: Falsch

Sie sind deren *Quadrate*: $\lambda_i = \sigma_i^2$, oder umgekehrt $\sigma_i = \sqrt{\lambda_i}$ (Definition 6.2.4). In Beispiel 6.2.6 gehören zu $\lambda_1 \approx 9,531$ und $\lambda_2 \approx 1,469$ die Singulärwerte $\sigma_1 \approx 3,087$ und $\sigma_2 \approx 1,212$.

4. Jede quadratische Matrix $A \in \mathbb{R}^{n \times n}$ ist diagonalisierbar.

Antwort: Falsch

Der Jordan-Block $A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ hat nur den Eigenwert 0, und dessen Eigenraum ist die Gerade $\text{span}\{e_1\}$. Es gibt also keine Basis des $\mathbb{R}^2$ aus Eigenvektoren. Vorsicht bei der Begründung: Symmetrie ist *hinreichend* für Diagonalisierbarkeit, nicht notwendig. Die unsymmetrische Matrix $\begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$ aus Beispiel 6.1.2 hat die zwei verschiedenen Eigenwerte 2 und 1 und ist damit diagonalisierbar, nur nicht *orthogonal* diagonalisierbar (Bemerkung 6.1.1).

5. $A^T A$ ist immer orthogonal diagonalisierbar.

Antwort: Wahr

$A^T A$ ist symmetrisch, und der Spektralsatz liefert für symmetrische Matrizen eine Orthonormalbasis aus Eigenvektoren, also $A^T A = V \Lambda V^T$ mit orthogonalem V (Korollar 6.2.2). Darauf ruht die ganze Konstruktion der SVD: Die Spalten von V sind die rechten Singulärvektoren.

Vertiefung: Heath §3.6 (Singulärwertzerlegung, Pseudoinverse und ihre Anwendungen); vgl. MML §4.5 und §4.6 für die Zerlegung selbst und die Approximation niedrigen Rangs.

Kleinste Quadrate

7.1 Kleinste Quadrate: Problem und Motivation

In den bisherigen Kapiteln haben wir lineare Gleichungssysteme $\mathbf{Ax} = \mathbf{b}$ mit quadratischer Matrix gelöst, also genauso viele Gleichungen wie Unbekannte. In der Statistik ist die Lage fast immer anders: Wir haben *viel mehr* Beobachtungen als Modellparameter, und die Beobachtungen sind fehlerbehaftet. Ein Gleichungssystem mit mehr Gleichungen als Unbekannten (*überbestimmt*, engl. *overdetermined*) hat in aller Regel gar keine exakte Lösung. Was tun? Wir geben den Anspruch auf, alle Gleichungen exakt zu erfüllen, und suchen stattdessen den Vektor, der sie *möglichst gut* erfüllt, gemessen an der euklidischen Norm des Fehlers. Das ist die *Methode der Kleinsten Quadrate* (engl. *least squares*), das Arbeitspferd der gesamten Regressionsrechnung. Dieses Kapitel behandelt, wie man solche Probleme *numerisch* löst: stabil, effizient und mit kontrollierbarem Fehler.

Wir bauen dabei auf früheren Kapiteln auf: Kondition und Konditionszahl $\kappa(\mathbf{A})$, Matrixnormen (besonders $\|\cdot\|_2$), Orthogonalmatrizen, die LU- und Cholesky-Zerlegung sowie die Singulärwertzerlegung $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$ (auch in reduzierter Form) mit der Pseudoinversen $\mathbf{A}^+$. Aus der linearen Algebra brauchen wir orthogonale Projektionen auf Unterräume und die Normalengleichungen, aus der Statistik das lineare Regressionsmodell und die Methode der Kleinsten Quadrate (OLS), die wir hier aus numerischer Sicht neu betrachten.

Vom Gleichungssystem zum Ausgleichsproblem

Definition 7.1.1 (Kleinste-Quadrate-Problem, KQ-Problem)

Gegeben seien eine Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ und ein Vektor $\mathbf{b} \in \mathbb{R}^m$. Gesucht ist ein Vektor $\hat{\mathbf{x}} \in \mathbb{R}^n$, der die Norm des *Residuums* minimiert:

$$\hat{\mathbf{x}} \in \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{Ax} - \mathbf{b}\|_2.$$

Da $t \mapsto t^2$ auf $[0, \infty)$ streng monoton wächst, können wir genauso gut $\|\mathbf{Ax} - \mathbf{b}\|_2^2$ minimieren – dieselben Minimalstellen, aber ohne Wurzel. Ausgeschrieben ist das eine Summe von Quadraten,

$$\|\mathbf{Ax} - \mathbf{b}\|_2^2 = \sum_{i=1}^m ((\mathbf{Ax})_i - b_i)^2,$$

daher der Name der Methode.

Bemerkung 7.1.2

Hat das Gleichungssystem $\mathbf{Ax} = \mathbf{b}$ eine exakte Lösung $\mathbf{x}^*$, dann gilt $\|\mathbf{Ax}^* - \mathbf{b}\|_2 = 0$, und

$\mathbf{x}^*$ löst automatisch auch das KQ-Problem. Das KQ-Problem verallgemeinert also das Lösen von Gleichungssystemen. Interessant ist es vor allem für „Gleichungssysteme“ $\mathbf{Ax} \approx \mathbf{b}$, die *keine* exakte Lösung besitzen: typischerweise überbestimmte Systeme mit $m > n$, bei denen $\mathbf{b}$ wegen Messfehlern nicht im Spaltenraum von $\mathbf{A}$ liegt.

Motivation: Regression und Ausgleichsrechnung

Warum ist gerade dieses Problem so wichtig? Weil die KQ-Methode das Fundament von Statistik und Machine Learning ist: Jedes Mal, wenn wir ein lineares Modell an Daten anpassen, lösen wir ein KQ-Problem.

Beispiel 7.1.3 (Lineare Regression)

Wir wollen Immobilienpreise anhand der Wohnfläche vorhersagen. Unsere Daten sind n Beobachtungen (x_i, y_i) mit x_i = Fläche in m^2 und y_i = Preis in €. Als Modell wählen wir eine Gerade mit Fehlerterm ϵ_i :

$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \quad i = 1, \dots, n.$$

In Matrixform lauten die n Modellgleichungen $\mathbf{b} \approx \mathbf{A}\boldsymbol{\beta}$ mit

$$\mathbf{A} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}.$$

Sobald $n > 2$ Datenpunkte nicht alle exakt auf einer gemeinsamen Geraden liegen (bei verauschten Messungen praktisch immer), liegt $\mathbf{b}$ nicht im Spaltenraum von $\mathbf{A}$: Kein Parameterpaar (β_0, β_1) erfüllt alle n Gleichungen zugleich. Die KQ-Lösung

$$\hat{\boldsymbol{\beta}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$$

minimiert stattdessen die Summe der quadrierten Abweichungen $\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 = \|\mathbf{A}\hat{\boldsymbol{\beta}} - \mathbf{b}\|_2^2$ und ist damit der Kleinste-Quadrate-Schätzer (OLS) aus der Einführungsvorlesung. Woher die Lösungsformel kommt, klären die Sätze 7.1.5 und 7.1.7 unten.

Die lineare Regression ist nur der Anfang. KQ-Probleme stecken auch in *generalisierten linearen Modellen* und *additiven Modellen* (dort löst man iterativ gewichtete KQ-Probleme), im Training neuronaler Netze (der quadratische Verlust ist ein KQ-Kriterium), in der Computer Vision (Kamerakalibrierung, 3D-Rekonstruktion aus 2D-Bildern), im Signal Processing (De-Noising, Signalschätzung) und in der Geodäsie (GPS-Positionsbestimmung gleicht widersprüchliche Distanzmessungen aus). Wer KQ-Probleme schnell und stabil lösen kann, hat also einen Universalschlüssel in der Hand.

Interaktiv: Wie gut passt die Gerade?

Definition 7.1.1 verlangt das Minimum über *alle* Paare (β_0, β_1) . Wie nah kommt man denn mit bloßem Augenmaß heran? Jeder Punkt im folgenden Bild ist ein (fiktiver) Wohnungsverkauf, Fläche gegen Preis in Tsd. €; die blaue Gerade gehört zum eingestellten Parameterpaar, jede violette Strecke ist ein Residuum $y_i - \beta_0 - \beta_1 x_i$.

Interaktives Element — nur in der Web-Fassung

Das Widget verbindet die verstellte Regressionsgerade mit den quadratischen Residuen.

Die Geometrie: Projektion auf den Spaltenraum

Interaktiv: Die Projektion aus verschiedenen Blickrichtungen

Der Schlüssel zum KQ-Problem ist ein Perspektivwechsel. Während $\mathbf{x}$ alle Vektoren in $\mathbb{R}^n$ durchläuft, durchläuft $\mathbf{Ax}$ genau den Spaltenraum $\text{col}(\mathbf{A}) \subseteq \mathbb{R}^m$, also alle Linearkombinationen der Spalten von $\mathbf{A}$. Das KQ-Problem fragt daher: Welcher Punkt des Unterraums $\text{col}(\mathbf{A})$ liegt am nächsten an $\mathbf{b}$? Aus der linearen Algebra wissen wir: der Fußpunkt des Lots, also die orthogonale Projektion von $\mathbf{b}$ auf $\text{col}(\mathbf{A})$. Halten wir das Bild dazu fest: Links steht die Zeichnung, wie sie in jedem Buch über Kleinste Quadrate zu finden ist, rechts dieselbe Konfiguration im Raum, drehbar. Denn dass die linke Zeichnung so plausibel aussieht, liegt auch an der geschickt gewählten Blickrichtung.

Interaktives Element — nur in der Web-Fassung

Drehen wir die rechte Tafel, bleibt dieselbe Projektion sichtbar: Der Lotfuß liegt im Spaltenraum, das Residuum steht senkrecht darauf.

Das folgende Resultat macht die Anschauung präzise.

Satz 7.1.4 (KQ-Lösung als Projektion)

Die Lösungsmenge des KQ-Problems zu $\mathbf{A}$ und $\mathbf{b}$ gleicht der Lösungsmenge des linearen Gleichungssystems

$$\mathbf{A}\hat{\mathbf{x}} = \hat{\mathbf{b}} \quad \text{mit } \hat{\mathbf{b}} = \text{proj}_{\text{col}(\mathbf{A})}\mathbf{b}.$$

Beweis.

- (1) Wir zerlegen $\mathbf{b}$ orthogonal bezüglich $S := \text{col}(\mathbf{A})$:

$$\mathbf{b} = \hat{\mathbf{b}} + \mathbf{r}, \quad \hat{\mathbf{b}} = \text{proj}_S \mathbf{b} \in S, \quad \mathbf{r} \perp S.$$

$\mathbb{R}^m = \text{col}(\mathbf{A}) \oplus \text{col}(\mathbf{A})^\perp$: Zerlegung in Unterraum und orthogonales Komplement existiert und ist eindeutig

- (2) Für ein beliebiges $\mathbf{x} \in \mathbb{R}^n$ schreiben wir das Residuum als

$$\mathbf{b} - \mathbf{Ax} = \underbrace{(\hat{\mathbf{b}} - \mathbf{Ax})}_{\in S} + \underbrace{\mathbf{r}}_{\perp S}.$$

$\mathbf{Ax} \in S$ und $\hat{\mathbf{b}} \in S$; S ist ein Untervektorraum, also liegt auch die Differenz in S

- (3)

$$\|\mathbf{b} - \mathbf{Ax}\|_2^2 = \|\hat{\mathbf{b}} - \mathbf{Ax}\|_2^2 + \|\mathbf{r}\|_2^2 \geq \|\mathbf{r}\|_2^2.$$

Satz des Pythagoras: für $\mathbf{u} \perp \mathbf{v}$ gilt $\|\mathbf{u} + \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2$

- (4) Die untere Schranke $\|\mathbf{r}\|_2^2$ hängt nicht von $\mathbf{x}$ ab, und Gleichheit gilt genau für $\mathbf{Ax} = \hat{\mathbf{b}}$. Also minimiert $\mathbf{x}$ das KQ-Kriterium genau dann, wenn $\mathbf{Ax} = \hat{\mathbf{b}}$ gilt.

$$\|\mathbf{v}\|_2 = 0 \Leftrightarrow \mathbf{v} = \mathbf{0} \text{ (Definitheit der Norm)}$$

□

Der Beweis zeigt mehr als die Behauptung: Das optimale Residuum ist genau der Anteil $\mathbf{r}$ von $\mathbf{b}$, der *senkrecht* auf $\text{col}(\mathbf{A})$ steht: der Teil der Daten, den das Modell prinzipiell nicht erklären kann. Vorsicht: Das Gleichungssystem $\mathbf{Ax} = \hat{\mathbf{b}}$ ist stets lösbar (denn $\hat{\mathbf{b}} \in \text{col}(\mathbf{A})$), aber seine Lösung ist nur dann eindeutig, wenn die Spalten von $\mathbf{A}$ linear unabhängig sind. Dazu gleich mehr.

Interaktiv: Wo genau steht das Residuum senkrecht?

Kleiner geht ein KQ-Problem nicht: $m = 2, n = 1$, die Matrix hat als einzige Spalte $\mathbf{a} = (3, 1)^\top$, gesucht ist ein einzelner Skalar x , und $\text{col}(\mathbf{A})$ ist die graue Ursprungsgerade. Für welches x wird $\|\mathbf{r}\|_2^2$ am kleinsten – und was ist dort mit dem Winkel los?

Interaktives Element – nur in der Web-Fassung

Das Widget verbindet die Projektion auf den Spaltenraum mit dem senkrechten Residuum aus Satz 7.1.4.

Normalengleichungen und eindeutige Lösung

Satz 7.1.4 ist geometrisch erhellend, aber unpraktisch: Um $\hat{\mathbf{b}}$ auszurechnen, müssten wir die Projektion schon kennen. Die Orthogonalitätsbedingung lässt sich aber in ein Gleichungssystem übersetzen, das nur $\mathbf{A}$ und $\mathbf{b}$ enthält: die aus der linearen Algebra bekannten Normalengleichungen.

Satz 7.1.5 (Normalengleichungen)

Die Lösungsmenge des KQ-Problems gleicht der Lösungsmenge der *Normalengleichungen*

$$\mathbf{A}^\top \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}. \quad (7.1.1)$$

Beweis.

(1)

$$\mathbf{Ax} = \hat{\mathbf{b}} \Leftrightarrow \mathbf{b} - \mathbf{Ax} = \mathbf{r} \perp \text{col}(\mathbf{A}).$$

Satz 7.1.4 mit der Zerlegung aus seinem Beweis: Genau dann besteht das Residuum nur noch aus dem zu $\text{col}(\mathbf{A})$ senkrechten Anteil

- (2) Senkrecht auf dem ganzen Spaltenraum zu stehen heißt, senkrecht auf jeder einzelnen Spalte $\mathbf{a}_j$ zu stehen:

$$\mathbf{b} - \mathbf{Ax} \perp \text{col}(\mathbf{A}) \Leftrightarrow \mathbf{a}_j^\top (\mathbf{b} - \mathbf{Ax}) = 0 \quad \forall j \Leftrightarrow \mathbf{A}^\top (\mathbf{b} - \mathbf{Ax}) = \mathbf{0}.$$

$\text{col}(\mathbf{A}) = \text{span}\{\mathbf{a}_1, \dots, \mathbf{a}_n\}$: senkrecht auf dem Spann heißt senkrecht auf jedem Erzeuger; $\mathbf{A}^\top \mathbf{v}$ stapelt gerade die Skalarprodukte $\mathbf{a}_j^\top \mathbf{v}$

(3)

$$\mathbf{A}^\top \mathbf{b} - \mathbf{A}^\top \mathbf{A} \hat{\mathbf{x}} = \mathbf{0} \quad \Leftrightarrow \quad \mathbf{A}^\top \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^\top \mathbf{b}.$$

Ausmultiplizieren und Umstellen

□

Bemerkung 7.1.6 (Analysis-Perspektive)

Dieselben Gleichungen liefert auch die Analysis, ganz ohne Geometrie: Die Zielfunktion $g(\mathbf{x}) = \|\mathbf{Ax} - \mathbf{b}\|_2^2 = \mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} - 2 \mathbf{b}^\top \mathbf{A} \mathbf{x} + \mathbf{b}^\top \mathbf{b}$ ist eine konvexe quadratische Funktion mit Gradient

$$\nabla g(\mathbf{x}) = 2 \mathbf{A}^\top \mathbf{A} \mathbf{x} - 2 \mathbf{A}^\top \mathbf{b}.$$

Nullsetzen ergibt exakt (7.1.1); wegen der Konvexität ist die notwendige Bedingung auch hinreichend. Diese Sichtweise begegnet uns in Kapitel 12 wieder (vgl. MML §7.1).

Bleibt die Frage nach der Eindeutigkeit. Sie hängt am Rang von $\mathbf{A}$: Genau bei vollem Spaltenrang gibt es nur eine Lösung.

Satz 7.1.7 (Eindeutige Lösung bei vollem Spaltenrang)

Falls $\text{rang}(\mathbf{A}) = n$, ist $\mathbf{A}^\top \mathbf{A}$ invertierbar und das KQ-Problem hat die eindeutige Lösung

$$\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b} =: \mathbf{A}^+ \mathbf{b}.$$

Beweis.

- (1) Sei $\mathbf{x} \neq \mathbf{0}$ beliebig. Wegen $\text{rang}(\mathbf{A}) = n$ sind die Spalten von $\mathbf{A}$ linear unabhängig, also $\mathbf{Ax} \neq \mathbf{0}$.
volle Spaltenzahl im Rang $\implies$ Spalten linear unabhängig $\implies$ Kern = $\{\mathbf{0}\}$
- (2) Damit ist $\mathbf{A}^\top \mathbf{A}$ symmetrisch und positiv definit:

$$\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} = \|\mathbf{Ax}\|_2^2 > 0 \quad \text{für alle } \mathbf{x} \neq \mathbf{0}.$$

$\mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x} = (\mathbf{Ax})^\top (\mathbf{Ax})$; Definitheit der Norm

- (3) Eine positiv definite Matrix ist invertierbar. Die Normalengleichungen (7.1.1) haben daher genau eine Lösung, und Auflösen liefert $\hat{\mathbf{x}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$.

positiv definit $\implies$ alle Eigenwerte $> 0 \implies \det(\mathbf{A}^\top \mathbf{A}) \neq 0$

□

Die Matrix $\mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ ist genau die aus dem SVD-Kapitel bekannte Pseudoinverse von $\mathbf{A}$ (eingeschränkt auf den Fall $\text{rang}(\mathbf{A}) = n$): Für quadratisches, invertierbares $\mathbf{A}$ gilt $\mathbf{A}^+ = \mathbf{A}^{-1}$, und allgemein löst $\mathbf{A}^+ \mathbf{b}$ das KQ-Problem. Wie man $\hat{\mathbf{x}}$ *numerisch* am besten berechnet (über die Normalengleichungen sicher *nicht* immer!), ist das Thema der Abschnitte 7.3 bis 7.6; zuvor klären wir in Abschnitt 7.2, wie gut das Problem überhaupt *konditioniert* ist.

Beispiel 7.1.8 (Rechenbeispiel: Regressionsgerade durch drei Punkte)

Rechnen wir die Lösungsformel einmal komplett durch. Drei Wohnungen wurden verkauft:

50 m² für 250, 75 m² für 350 und 100 m² für 500 (Preise in Tsd. €). Mit dem Modell aus Beispiel 7.1.3:

$$\mathbf{A} = \begin{pmatrix} 1 & 50 \\ 1 & 75 \\ 1 & 100 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 250 \\ 350 \\ 500 \end{pmatrix}.$$

Wir bilden die Bausteine der Normalgleichungen:

$$\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 3 & 225 \\ 225 & 18\,125 \end{pmatrix}, \quad \mathbf{A}^\top \mathbf{b} = \begin{pmatrix} 1\,100 \\ 88\,750 \end{pmatrix}.$$

Die beiden Spalten von $\mathbf{A}$ sind linear unabhängig, nach Satz 7.1.7 ist die Lösung also eindeutig. Auflösen des 2×2 -Systems $\mathbf{A}^\top \mathbf{A} \hat{\boldsymbol{\beta}} = \mathbf{A}^\top \mathbf{b}$ liefert

$$\hat{\boldsymbol{\beta}} = \begin{pmatrix} -25/3 \\ 5 \end{pmatrix} \approx \begin{pmatrix} -8,33 \\ 5 \end{pmatrix}.$$

Jeder Quadratmeter kostet also geschätzt 5 000 € mehr. Zur Probe berechnen wir die angepassten Werte und das Residuum:

$$\hat{\mathbf{b}} = \mathbf{A} \hat{\boldsymbol{\beta}} = \frac{1}{3} \begin{pmatrix} 725 \\ 1\,100 \\ 1\,475 \end{pmatrix} \approx \begin{pmatrix} 241,7 \\ 366,7 \\ 491,7 \end{pmatrix}, \quad \mathbf{r} = \mathbf{b} - \hat{\mathbf{b}} = \frac{1}{3} \begin{pmatrix} 25 \\ -50 \\ 25 \end{pmatrix}.$$

Tatsächlich gilt $\mathbf{A}^\top \mathbf{r} = \mathbf{0}$: Die Residuen summieren sich zu $(25 - 50 + 25)/3 = 0$, und ebenso ist $\sum_{i=1}^n x_i r_i = (50 \cdot 25 - 75 \cdot 50 + 100 \cdot 25)/3 = 0$. Das Residuum steht also senkrecht auf beiden Spalten, genau wie Satz 7.1.5 verspricht.

Selbsttest

Zum Abschluss vier Aussagen über die Pseudoinverse und Projektionen. Welche sind wahr? Es sei dabei $\text{rang}(\mathbf{A}) = n$ mit $m > n$ und $\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b}$.

1. $\mathbf{A} \mathbf{A}^+ = \mathbf{I}_m$

Antwort: Falsch

$\mathbf{A} \mathbf{A}^+ = \mathbf{A} (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ ist die Projektionsmatrix auf $\text{col}(\mathbf{A})$; die Identität auf $\mathbb{R}^m$ ist das nur, wenn $\text{col}(\mathbf{A}) = \mathbb{R}^m$ gilt, also für invertierbares quadratisches $\mathbf{A}$. (Richtig ist dagegen stets $\mathbf{A}^+ \mathbf{A} = \mathbf{I}_n$ bei vollem Spaltenrang.)

2. $\mathbf{A} \mathbf{A}^+ \mathbf{b} = \text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}$

Antwort: Wahr

$\mathbf{A} \mathbf{A}^+ \mathbf{b} = \mathbf{A} \hat{\mathbf{x}} = \hat{\mathbf{b}}$: die KQ-Lösung bildet $\mathbf{b}$ gerade auf seine Projektion in den Spaltenraum ab (Satz 7.1.4).

3. $\mathbf{r} = \mathbf{b} - \mathbf{A} \hat{\mathbf{x}} \perp \text{col}(\mathbf{A})$

Antwort: Wahr

Das ist genau die Aussage der Normalgleichungen: $\mathbf{A}^\top (\mathbf{b} - \mathbf{A} \hat{\mathbf{x}}) = \mathbf{0}$ heißt, das Residuum steht senkrecht auf allen Spalten von $\mathbf{A}$.

4. $\mathbf{b} = \text{proj}_V \mathbf{b} + \text{proj}_{V^\perp} \mathbf{b}$ für jeden Untervektorraum $V \subseteq \mathbb{R}^m$

Antwort: Wahr

Wegen $\mathbb{R}^m = V \oplus V^\perp$ zerfällt jeder Vektor eindeutig in seine Anteile in V und im orthogonalen Komplement $V^\perp$.

5. Im Miniaturbeispiel oben ($\mathbf{a} = (3, 1)^\top$, $\mathbf{b} = (1, 2)^\top$) schieben wir den Kandidaten so weit, bis das Residuum senkrecht steht. Welchen Wert hat dort $\|\mathbf{r}\|_2^2$?

Lösung: 2.5

Der senkrechte Stand wird bei $x = 1/2$ erreicht, also $\mathbf{r} = (1, 2)^\top - \frac{1}{2}(3, 1)^\top = (-0,5; 1,5)^\top$ und $\|\mathbf{r}\|_2^2 = 0,25 + 2,25 = 2,5$. Das ist das Minimum aus Satz 7.1.4; jeder andere Kandidat liefert einen größeren Wert.

Vertiefung: Heath §3.1–3.2; MML §7.1 (Gradienten-Perspektive) und §9.2 (lineare Regression aus statistischer Sicht).

7.2 Kondition des Kleinste-Quadrate-Problems

Bevor wir in den folgenden Abschnitten Algorithmen zur Lösung des KQ-Problems entwickeln, stellen wir dieselbe Frage wie seinerzeit bei den linearen Gleichungssystemen: Wie empfindlich reagiert die Lösung $\hat{\mathbf{x}}$ auf kleine Störungen der Eingaben? Die Antwort gibt die Kondition des Problems, und sie legt fest, welche Genauigkeit wir von *jedem* Algorithmus bestenfalls erwarten dürfen. Denn die Kondition ist eine Eigenschaft des Problems selbst, nicht des Lösungswegs. Beim KQ-Problem gibt es zwei Eingaben, die gestört sein können: den Datenvektor $\mathbf{b}$ und die Matrix $\mathbf{A}$. Wir behandeln beide Fälle getrennt.

Vorbemerkungen

Wir untersuchen nun die Kondition von KQ-Problemen $\mathbf{A}\mathbf{x} \approx \mathbf{b}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{b} \in \mathbb{R}^m$. Dabei beschränken wir uns auf den Fall $m \geq n$ und $\text{rang}(\mathbf{A}) = n$. Dann existiert, wie in Abschnitt 7.1 gesehen, eine eindeutige Lösung $\hat{\mathbf{x}}$. Als Norm verwenden wir im Folgenden stets $\|\cdot\| = \|\cdot\|_2$: Das KQ-Problem minimiert gerade einen Abstand in der euklidischen Norm, sie ist also die natürliche Wahl.

Die Abbildung von den Daten zur Lösung schreiben wir als $\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b}$. Hier bezeichnet $\mathbf{A}^+$ die *Pseudoinverse* von $\mathbf{A}$, die wir in Abschnitt 7.6 genauer kennenlernen. Für den Moment genügt: $\mathbf{A}^+$ ist die Matrix, die jedem $\mathbf{b}$ seine eindeutige KQ-Lösung zuordnet, und für quadratisches, reguläres $\mathbf{A}$ gilt schlicht $\mathbf{A}^+ = \mathbf{A}^{-1}$. Damit können wir die vertraute Definition der Konditionszahl auf rechteckige Matrizen übertragen:

Definition 7.2.1

Für eine nicht notwendigerweise quadratische Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $m \geq n$ definieren wir die *Konditionszahl*

$$\kappa(\mathbf{A}) = \begin{cases} \infty, & \text{rang}(\mathbf{A}) < n, \\ \|\mathbf{A}\| \|\mathbf{A}^+\|, & \text{sonst.} \end{cases}$$

Bemerkung 7.2.2

Für quadratisches, reguläres A ist $A^+ = A^{-1}$, und wir erhalten die bekannte Konditionszahl $\kappa(A) = \|A\| \|A^{-1}\|$ zurück. In der 2-Norm gilt außerdem (mit den Singulärwerten aus der SVD)

$$\kappa_2(A) = \frac{\sigma_{\max}}{\sigma_{\min}},$$

denn $\|A\|_2 = \sigma_{\max}$ und $\|A^+\|_2 = 1/\sigma_{\min}$. Die Fallunterscheidung in Definition 7.2.1 ist dabei konsistent: Nähert sich A dem Rangdefekt, geht $\sigma_{\min} \rightarrow 0$ und damit $\kappa_2(A) \rightarrow \infty$.

Interaktiv: Wie schnell wächst $\kappa_2(A)$, wenn die Spalten zusammenrücken?

Nehmen wir zwei Einheitsspalten im Winkel α zueinander. Für $\alpha \rightarrow 0$ werden sie linear abhängig, für $\alpha = 90^\circ$ stehen sie senkrecht. Dazwischen: Wie schnell wächst $\kappa_2(A)$, wenn wir α halbieren?

Interaktives Element — nur in der Web-Fassung

Das Widget zeigt die stetige Verschlechterung der Kondition auf dem Weg zum Rangdefekt.

Kondition bezüglich b

Beginnen wir mit dem statistisch wichtigsten Fall: A ist fix, aber der Datenvektor b ist gestört, etwa durch Messfehler oder schlicht durch Rundungsfehler beim Einlesen. Das Problem ist dann die Abbildung $f(b) = A^+b$, und wir fragen: Wie stark ändert sich $f(b)$ relativ, wenn wir b durch ein gestörtes $\tilde{b}$ ersetzen?

Satz 7.2.3

Sei $A \in \mathbb{R}^{m \times n}$ fix mit $\text{rang}(A) = n$, und sei f das Problem mit Lösung $f(b) = A^+b$. Für $f(b) \neq 0$ ist die exakte relative Konditionszahl bezüglich b

$$\kappa_{\text{rel}}(f, b) = \|A^+\|_2 \frac{\|b\|_2}{\|A^+b\|_2}.$$

Sie erfüllt die geometrisch anschauliche obere Schranke

$$\kappa_{\text{rel}}(f, b) \leq \kappa_2(A) \frac{\|b\|_2}{\|\text{proj}_{\text{col}(A)} b\|_2}.$$

Die exakte Formel folgt unmittelbar daraus, dass die Ableitung der linearen Abbildung f die Matrix A^+ ist. Die obere Schranke zeigt zusätzlich, wie viel von b im Spaltenraum (Bild) $\text{col}(A)$ liegt. Sie muss für ein festes b nicht scharf sein. Wir verfolgen die Störung $\tilde{b} - b$ in Blau und die Projektion $\text{proj}_{\text{col}(A)} b$ in Grün.

Vertiefung: Beweis der Konditionsschranke bezüglich b

Beweis.

(1)

$$\frac{\|A^+ \tilde{\mathbf{b}} - A^+ \mathbf{b}\|}{\|A^+ \mathbf{b}\|} = \frac{\|A^+ (\tilde{\mathbf{b}} - \mathbf{b})\|}{\|A^+ \mathbf{b}\|}$$

A^+ ist eine Matrix, die Abbildung $\mathbf{b} \mapsto A^+ \mathbf{b}$ also linear

(2)

$$\leq \frac{\|A^+\| \|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|A^+ \mathbf{b}\|}$$

Submultiplikativität der natürlichen Matrixnorm: $\|A^+ \mathbf{v}\| \leq \|A^+\| \|\mathbf{v}\|$

(3)

$$= \frac{\kappa_2(A)}{\|A\| \|A^+ \mathbf{b}\|} \|\tilde{\mathbf{b}} - \mathbf{b}\|$$

Definition 7.2.1: $\kappa_2(A) = \|A\| \|A^+\|$, also $\|A^+\| = \kappa_2(A) / \|A\|$

(4)

$$\leq \frac{\kappa_2(A)}{\|AA^+ \mathbf{b}\|} \|\tilde{\mathbf{b}} - \mathbf{b}\|$$

nochmals Submultiplikativität: $\|AA^+ \mathbf{b}\| \leq \|A\| \|A^+ \mathbf{b}\|$; wir verkleinern den Nenner, der Bruch kann nur wachsen

(5)

$$= \kappa_2(A) \frac{\|\mathbf{b}\|}{\|\text{proj}_{\text{col}(A)} \mathbf{b}\|} \cdot \frac{\|\tilde{\mathbf{b}} - \mathbf{b}\|}{\|\mathbf{b}\|}$$

$AA^+ \mathbf{b} = A\hat{\mathbf{x}} = \text{proj}_{\text{col}(A)} \mathbf{b}$: Die KQ-Lösung ist die orthogonale Projektion von $\mathbf{b}$ auf $\text{col}(A)$ (Abschnitt 7.1); anschließend erweitern wir mit $\|\mathbf{b}\|$

□

Vertiefung: Geometrische Extremfälle der Konditionsschranke

Geometrische Interpretation der oberen Schranke

Der Faktor $\|\mathbf{b}\| / \|\text{proj}_{\text{col}(A)} \mathbf{b}\|$ hat eine anschauliche Bedeutung. Erinnern wir uns an den Satz des Pythagoras: $\mathbf{b}$ zerlegt sich orthogonal in seinen Anteil im Spaltenraum und seinen Anteil im orthogonalen Komplement,

$$\|\mathbf{b}\|^2 = \|\text{proj}_{\text{col}(A)} \mathbf{b}\|^2 + \|\text{proj}_{\text{col}(A)^\perp} \mathbf{b}\|^2 \geq \|\text{proj}_{\text{col}(A)} \mathbf{b}\|^2,$$

also ist der Faktor stets ≥ 1 . Das macht die *obere Schranke* mindestens so groß wie $\kappa_2(A)$, nicht notwendig die exakte Kondition. Bezeichnet θ den Winkel zwischen $\mathbf{b}$ und $\text{col}(A)$ (siehe die Abbildung in Abschnitt 7.1), dann ist $\|\text{proj}_{\text{col}(A)} \mathbf{b}\| = \cos(\theta) \|\mathbf{b}\|$ und damit

$$\kappa_{\text{rel}}(f, \mathbf{b}) \leq \frac{\kappa_2(A)}{\cos(\theta)}.$$

Was sind nun die geometrischen Extremfälle dieser Schranke?

- **Best case:** $\mathbf{b} = \text{proj}_{\text{col}(A)} \mathbf{b}$, d.h. $\mathbf{b} \in \text{col}(A)$ und $\theta = 0$. Dann ist $\cos(\theta) = 1$, und die Schranke reduziert sich auf $\kappa_2(A)$; die exakte Kondition am konkreten $\mathbf{b}$ kann kleiner sein.
- **Worst case:** $\text{proj}_{\text{col}(A)} \mathbf{b} = \mathbf{0}$, d.h. $\mathbf{b} \perp \text{col}(A)$ und $\theta = 90^\circ$. Dann ist $\hat{\mathbf{x}} = A^+ \mathbf{b} = \mathbf{0}$, der relative Outputfehler also gar nicht definiert. Setzt man die relative Kondition in diesem Fall konventionsgemäß auf ∞ , passt das zum Grenzwert: Schon eine winzige Störung mit Anteil in $\text{col}(A)$ erzeugt einen beliebig großen *relativen* Fehler in der Lösung.

Für die Regression heißt das vorsichtiger: Ein kleiner Projektionsanteil kann die Bestimmung der Koeffizienten relativ empfindlich machen. Die exakte Empfindlichkeit hängt aber zusätzlich von der Richtung von $\mathbf{b}$ innerhalb des Spaltenraums und von den Singulärrichtungen von $\mathbf{A}$ ab.

Interaktiv: Wann ist die obere Schranke scharf?

Nehmen wir das einfachste nichttriviale KQ-Problem: $m = 2$, $n = 1$, $\mathbf{A} = (1, 0)^\top$, also $\kappa_2(\mathbf{A}) = 1$ und $\text{col}(\mathbf{A})$ gleich der horizontalen Achse. Die ganze Empfindlichkeit steckt dann im Winkel θ . Bei welcher Richtung der Störung $\Delta \mathbf{b}$ wird die Schranke tatsächlich angenommen – und bei welcher passiert gar nichts?

Interaktives Element – nur in der Web-Fassung

Das Widget trennt den störenden Projektionsanteil vom Anteil, der im Residuum verbleibt.

Kondition bezüglich $\mathbf{A}$

Nun der umgekehrte Fall: $\mathbf{b}$ ist fix, aber die Matrix $\mathbf{A}$ selbst ist gestört. Auch das ist praktisch relevant: etwa durch Messfehler in den Kovariablen oder durch Rundungsfehler, die beim Speichern von $\mathbf{A}$ und in den Zwischenschritten der Algorithmen zwangsläufig entstehen. Das Problem ist jetzt die Abbildung $f(\mathbf{A}) = \mathbf{A}^+ \mathbf{b}$. Das Ergebnis sieht auf den ersten Blick ähnlich aus wie Satz 7.2.3, hat aber eine entscheidend andere Struktur:

Satz 7.2.4 (Störung der Designmatrix, erste Ordnung)

Sei $\mathbf{b} \in \mathbb{R}^m$ fix, $\mathbf{A}$ habe vollen Spaltenrang und $f(\mathbf{A}) = \mathbf{A}^+ \mathbf{b} \neq \mathbf{0}$. Für $\Delta \mathbf{A} \rightarrow \mathbf{0}$ und $\mathbf{A} + \Delta \mathbf{A}$ weiterhin von vollem Spaltenrang gilt

$$\frac{\|f(\mathbf{A} + \Delta \mathbf{A}) - f(\mathbf{A})\|}{\|f(\mathbf{A})\|} \leq \left(\kappa_2(\mathbf{A}) + \kappa_2(\mathbf{A})^2 \frac{\|\mathbf{r}\|}{\|\text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}\|} + o(1) \right) \frac{\|\Delta \mathbf{A}\|}{\|\mathbf{A}\|},$$

wobei $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ das Residuum des ungestörten Problems bezeichnet.

Bemerkung 7.2.5

Der Beweis ist (anders als der von Satz 7.2.3) eine sehr aufwändige und, offen gesagt, langweilige Rechnung. Wir verzichten darauf und verweisen auf die Literatur (vgl. Heath §3.3).

Interpretation

Der Koeffizient der lokalen oberen Schranke enthält zunächst $\kappa_2(\mathbf{A})$. Der zweite Term koppelt das *Quadrat* der Konditionszahl an das Verhältnis von Residuum zu Projektion. Mit dem Winkel θ von oben ist $\|\mathbf{r}\| = \sin(\theta) \|\mathbf{b}\|$ (das Residuum ist gerade die zu $\text{col}(\mathbf{A})$ orthogonale Komponente von $\mathbf{b}$), also

$$\kappa_{lok}^{oben} = \kappa_2(\mathbf{A}) + \kappa_2(\mathbf{A})^2 \tan(\theta).$$

- **Best case:** $\mathbf{r} = \mathbf{0}$, d.h. $\mathbf{A}\mathbf{x} = \mathbf{b}$ ist exakt lösbar. Dann ist $\tan(\theta) = 0$, und die lokale obere Schranke reduziert sich auf $\kappa_2(\mathbf{A})$, wie bei einem gewöhnlichen linearen Gleichungssystem.

- **Worst case:** $\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b} = \mathbf{0}$, aber $\mathbf{r} \neq \mathbf{0}$: Dann ist $\mathbf{Ax} = \mathbf{b}$ nicht lösbar, $\mathbf{b} \perp \text{col}(\mathbf{A})$ und $\theta = 90^\circ$. Wie oben ist $\hat{\mathbf{x}} = \mathbf{0}$, und die relative Kondition wird konventionsgemäß als ∞ behandelt.
- **Normalfall:** $\mathbf{r} \neq \mathbf{0}$ und $\text{proj}_{\text{col}(\mathbf{A})}\mathbf{b} \neq \mathbf{0}$. Dann kann die obere Schranke einen Term der Ordnung $\kappa_2(\mathbf{A})^2$ enthalten.

Vorsicht also: Sobald das Residuum nicht verschwindet (und in der Statistik verschwindet es praktisch nie), kann eine Störung von $\mathbf{A}$ mit dem Quadrat der Konditionszahl verstärkt werden. Dieses Quadrat wird uns in Abschnitt 7.3 gleich wieder begegnen: Der Lösungsweg über die Normalengleichungen handelt es sich *immer* ein, also auch im gutartigen Fall kleiner Residuen, in dem das Problem selbst nur mit $\kappa_2(\mathbf{A})$ reagiert.

Interaktiv: Die quadrierte Kondition im Experiment

Wir stören eine konkrete Problemfamilie mit einstellbarem ε :

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ \varepsilon & -\varepsilon \\ 0 & 0 \end{pmatrix}, \quad \tilde{\mathbf{A}} = \mathbf{A} + \mathbf{E} \text{ mit } \mathbf{E} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \\ -\varepsilon & \varepsilon \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 0 \\ b_3 \end{pmatrix}.$$

Hier ist $\kappa_2(\mathbf{A}) = 1/\varepsilon$, und die relative Störung ist $\|\mathbf{E}\|_2 / \|\mathbf{A}\|_2 = \varepsilon$. Die ungestörte Lösung ist stets $\hat{\mathbf{x}} = (1/2, 1/2)^\top$, unabhängig von ε und b_3 . Die Frage ist: Wie stark muss das Residuum sein, damit der quadrierte Term die Schranke übernimmt?

Interaktives Element — nur in der Web-Fassung

Das Widget trennt das lineare vom quadrierten Konditionsregime der Problembeschreibung.

Selbsttest

1. Stellen wir im Rangdefekt-Widget den Spaltenwinkel auf $\alpha = 12^\circ$. Welchen Wert zeigt es dann für $\kappa_2(\mathbf{A})$?

Lösung: 9.51

Für diese Matrizenfamilie ist $\kappa_2(\mathbf{A}) = \cot(\alpha/2)$, bei $\alpha = 12^\circ$ also $\cot(6^\circ) \approx 9.51$. Das entspricht knapp einer verlorenen Dezimalstelle: $\log_{10}(9.51) \approx 0.98$.

2. Eine Störung von $\mathbf{b}$ senkrecht zu $\text{col}(\mathbf{A})$ verändert $\hat{\mathbf{x}}$ genauso stark wie eine gleich große Störung längs $\text{col}(\mathbf{A})$.

Antwort: Falsch

Sie verändert $\hat{\mathbf{x}}$ gar nicht. Das Winkel-Widget zeigt es unmittelbar: bei $\varphi = 90^\circ$ bleibt $\Delta\hat{\mathbf{x}} = \mathbf{0}$, die ganze Störung landet im Residuum. Ausgeschöpft wird von der Schranke aus Satz 7.2.3 der Anteil $|\cos(\varphi)|$.

3. Bei exakt lösbarem $\mathbf{Ax} = \mathbf{b}$ reduziert sich die lokale Schranke aus Satz 7.2.4 auf $\kappa_2(\mathbf{A})$.

Antwort: Wahr

Exakt lösbar heißt $\mathbf{r} = \mathbf{0}$, also $\tan(\theta) = 0$; der Term $\kappa_2(\mathbf{A})^2 \tan(\theta)$ verschwindet, und übrig bleibt genau $\kappa_2(\mathbf{A})$ wie bei einem gewöhnlichen linearen Gleichungssystem.

4. Die Konditionszahl $\kappa_2(\mathbf{A})$ hängt davon ab, mit welchem Verfahren wir das KQ-Problem lösen.

Antwort: Falsch

Die Kondition ist eine Eigenschaft des Problems, nicht des Lösungswegs – genau das ist der Grund, warum wir sie vor den Algorithmen behandeln. Der Lösungsweg entscheidet nur, wie nah ein Verfahren an diese unvermeidbare Empfindlichkeit herankommt (Stabilität).

Vertiefung: Heath §3.1–3.3, insbesondere §3.3 („Sensitivity and Conditioning“) mit der vollständigen Störungsanalyse inklusive des Beweises von Satz 7.2.4; zum KQ-Problem als Optimierungsproblem und seiner Rolle in der Regression außerdem MML §7.1 und §9.2.

7.3 Normalengleichungen und Cholesky-Zerlegung

Aus Abschnitt 7.1 wissen wir, dass das Kleinste-Quadrate-Problem für $A \in \mathbb{R}^{m \times n}$ mit $\text{rang}(A) = n$ die eindeutige Lösung $\hat{x} = (A^T A)^{-1} A^T b$ besitzt, charakterisiert durch die Normalengleichungen $A^T A \hat{x} = A^T b$ (Satz 7.1.5). Das ist eine Formel – aber noch kein Rechenverfahren. In diesem Abschnitt entwickeln wir das erste und einfachste Lösungsverfahren: Wir lösen die Normalengleichungen mit der Cholesky-Zerlegung und sehen dann, warum dieser bequeme Weg numerisch seinen Preis hat. Das motiviert am Ende des Abschnitts die orthogonalen Transformationen, die uns im Rest des Kapitels beschäftigen werden.

7.3.1 Warum wir niemals invertieren

Wie berechnen wir $\hat{x}$ nun konkret? Betrachten wir dazu eine kleine Programmieraufgabe. Gegeben ist eine R-Funktion, die die Lösungsformel $\hat{x} = (A^T A)^{-1} A^T b$ wörtlich umsetzt: Sie bildet also erst die Inverse von $A^T A$ und multipliziert dann:

```
kq_solve <- function(A, b) {
  solve(t(A) %*% A) %*% t(A) %*% b
}
```

Aufgabe: Finde den Fehler in der Funktion. Der Fehler ist kein Tippfehler, sondern der Ansatz selbst:

Bemerkung 7.3.1 (Merkregel)

Für ein LGS keine explizite Inverse bilden!

Die Formel $\hat{x} = (A^T A)^{-1} A^T b$ ist ein *mathematisches* Objekt, keine Rechenvorschrift. Numerisch lösen wir stattdessen das lineare Gleichungssystem $(A^T A) \hat{x} = A^T b$, mit einer geeigneten Zerlegung der Matrix.

Warum ist das explizite Invertieren so schlecht? Erstens ist es teurer: Die Inverse zu berechnen heißt, n lineare Gleichungssysteme zu lösen (eines pro Spalte von I), obwohl wir nur die Lösung zu *einer* rechten Seite brauchen. Zweitens bringt die anschließende Multiplikation zusätzliche Rundungsfehler, ohne für eine einzelne rechte Seite einen Genauigkeitsvorteil zu bieten; eine stabile Zerlegung mit Substitution löst dasselbe System direkter. Das heißt nicht, dass das Berechnen einer Inversen grundsätzlich instabil wäre – nur, dass sie hier das falsche Zwischenprodukt ist. Drittens kann die Inverse Struktur zerstören: Die Inverse einer dünnbesetzten Matrix ist typischerweise voll besetzt. Genau wie beim Gauß-Verfahren mit seiner LR-Zerlegung gilt also: Zerlegen und substituieren statt invertieren.

7.3.2 Das Cholesky-Verfahren

Welche Zerlegung passt zu $A^T A$? Die entscheidende Struktur haben wir im Beweis von Satz 7.1.7 schon gesehen: Für $\text{rang}(A) = n$ ist $A^T A$ symmetrisch, denn $(A^T A)^T = A^T A$, und positiv definit, denn $x^T (A^T A) x = \|Ax\|_2^2 > 0$ für jedes $x \neq 0$.

Symmetrisch positiv definit, kurz *SPD* (engl. *symmetric positive definite*): das ist genau die Klasse von Matrizen, für die die Cholesky-Zerlegung $M = LL^T$ mit einer unteren Dreiecksmatrix L existiert. Damit zerfällt das KQ-Problem in eine Kette einfacher Teilschritte:

Algorithmus 7.3.2 (Cholesky-Verfahren für das KQ-Problem)

1. Berechne $M = A^T A$ und $c = A^T b$.
2. Berechne die Cholesky-Zerlegung $M = LL^T$.
3. Löse $Ly = c$ durch Vorwärtssubstitution.
4. Löse $L^T \hat{x} = y$ durch Rückwärtssubstitution.

Schritt 1 ersetzt das $m \times n$ -Ausgangsproblem durch ein quadratisches $n \times n$ -Gleichungssystem. Die Schritte 2–4 lösen dieses System dann so, wie wir es von der LR-Zerlegung kennen: zerlegen, dann zweimal substituieren, denn Dreieckssysteme können wir direkt auflösen. Rechnen wir das einmal vollständig durch.

Beispiel 7.3.3

Wie im Wohnungspreis-Beispiel 7.1.8 aus Abschnitt 7.1 fitten wir eine Regressionsgerade $y = \beta_0 + \beta_1 x$, diesmal mit bewusst kleinen Zahlen, durch die drei Punkte (1, 2), (2, 3), (3, 5):

$$A = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}.$$

Dabei landen die x -Werte 1, 2, 3 der Datenpunkte in der zweiten Spalte von A (die Einerspalte davor gehört zum Achsenabschnitt β_0), und die y -Werte 2, 3, 5 bilden den Vektor b .

Schritt 1: M und c aufstellen.

$$M = A^T A = \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}, \quad c = A^T b = \begin{pmatrix} 10 \\ 23 \end{pmatrix}.$$

Schritt 2: Cholesky-Zerlegung $M = LL^T$. Für 2×2 lauten die Formeln $\ell_{11} = \sqrt{m_{11}}$, $\ell_{21} = m_{21}/\ell_{11}$, $\ell_{22} = \sqrt{m_{22} - \ell_{21}^2}$:

$$\ell_{11} = \sqrt{3}, \quad \ell_{21} = \frac{6}{\sqrt{3}} = 2\sqrt{3}, \quad \ell_{22} = \sqrt{14 - (2\sqrt{3})^2} = \sqrt{2},$$

$$\text{also } L = \begin{pmatrix} \sqrt{3} & 0 \\ 2\sqrt{3} & \sqrt{2} \end{pmatrix}.$$

Schritt 3: Vorwärtssubstitution $Ly = c$, Zeile für Zeile von oben.

$$y_1 = \frac{10}{\sqrt{3}} = \frac{10}{\sqrt{3}}, \quad y_2 = \frac{23 - 2\sqrt{3} \cdot \frac{10}{\sqrt{3}}}{\sqrt{2}} = \frac{23 - 20}{\sqrt{2}} = \frac{3}{\sqrt{2}}.$$

Schritt 4: Rückwärtssubstitution $\mathbf{L}^\top \hat{\mathbf{x}} = \mathbf{y}$, Zeile für Zeile von unten.

$$x_2 = \frac{3/\sqrt{2}}{\sqrt{2}} = \frac{3}{2}, \quad x_1 = \frac{\frac{10}{\sqrt{3}} - 2\sqrt{3} \cdot \frac{3}{2}}{\sqrt{3}} = \frac{10}{3} - 3 = \frac{1}{3}.$$

Die KQ-Lösung ist also $\hat{\mathbf{x}} = \left(\frac{1}{3}, \frac{3}{2}\right)^\top$, die Regressionsgerade $\hat{y} = \frac{1}{3} + \frac{3}{2}x$. Probe mit den Normalengleichungen:

$$\mathbf{M} \hat{\mathbf{x}} = \begin{pmatrix} 3 \cdot \frac{1}{3} + 6 \cdot \frac{3}{2} \\ 6 \cdot \frac{1}{3} + 14 \cdot \frac{3}{2} \end{pmatrix} = \begin{pmatrix} 10 \\ 23 \end{pmatrix} = \mathbf{c}. \quad \square$$

Auch die geometrische Deutung aus Abschnitt 7.1 können wir nachprüfen: Das Residuum $\hat{\mathbf{r}} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \left(\frac{1}{6}, -\frac{1}{3}, \frac{1}{6}\right)^\top$ erfüllt $\mathbf{A}^\top \hat{\mathbf{r}} = \mathbf{0}$: Es steht senkrecht auf beiden Spalten von $\mathbf{A}$.

7.3.3 Effizient, aber möglicherweise instabil

Das Cholesky-Verfahren ist effizient: Ab Schritt 2 rechnen wir nur noch mit der kleinen $n \times n$ -Matrix $\mathbf{A}^\top \mathbf{A}$ statt mit der $m \times n$ -Matrix $\mathbf{A}$, und die Cholesky-Zerlegung nutzt deren Symmetrie aus und kostet nur etwa halb so viel wie eine LR-Zerlegung. Für gut konditionierte Probleme ist das Verfahren in der Praxis brauchbar. Es hat aber einen eingebauten Schwachpunkt, und der liegt ausgerechnet im allerersten Schritt.

Bemerkung 7.3.4 (Stabilität des Cholesky-Verfahrens)

- Fehler, die beim Aufstellen von $\mathbf{A}^\top \mathbf{A}$ passieren, beeinflussen alle folgenden Schritte; kein späterer Schritt kann sie reparieren.
- Man kann zeigen: Der relative algorithmische Fehler ist bestenfalls $O(\kappa_2(\mathbf{A})^2)$.
- Zum Vergleich: Die Best-Case-Kondition des KQ-Problems selbst ist $O(\kappa_2(\mathbf{A}))$ (Abschnitt 7.2).

Woher kommt das Quadrat? Ab Schritt 2 rechnen wir mit $\mathbf{A}^\top \mathbf{A}$, und deren Konditionszahl ist $\kappa_2(\mathbf{A}^\top \mathbf{A}) = \kappa_2(\mathbf{A})^2$, denn die Singulärwerte von $\mathbf{A}^\top \mathbf{A}$ sind die Quadrate der Singulärwerte von $\mathbf{A}$. Diese quadrierte Kondition erben alle weiteren Schritte, ganz gleich, wie empfindlich das eigentliche KQ-Problem ist. Besonders ärgerlich ist das im günstigsten Fall: Bei kleinem Residuum gilt $\kappa \approx \kappa_2(\mathbf{A})$, der Algorithmus verstärkt Fehler aber trotzdem mit $\kappa_2(\mathbf{A})^2$. Er ist also deutlich fehleranfälliger, als die Kondition des Problems erzwingt. Genau solche Verfahren nennen wir *instabil*. Es gibt noch einen zweiten Effekt, den die Konditionszahl allein gar nicht erfasst: Schon beim Ausrechnen von $\mathbf{A}^\top \mathbf{A}$ in Gleitkommaarithmetik kann Information über die Spalten von $\mathbf{A}$ verloren gehen. Im Extremfall macht ein einziger Rundungsschritt aus einer Matrix mit vollem Spaltenrang eine numerisch singuläre.

Interaktiv: Wann verliert die Kreuzproduktmatrix ihren Rang?

Sehen wir uns eine Matrix an, bei der der ganze Unterschied zwischen den Spalten in einem einzigen kleinen Summanden steckt. Das Widget führt die kritische Addition wirklich in der Gleitkommaarithmetik des Browsers aus; es rechnet also nicht nach, sondern rechnet.

Interaktives Element — nur in der Web-Fassung

Sobald ε^2 unter die Maschinengenauigkeit rutscht, also ab $\varepsilon \approx \sqrt{\varepsilon_{\text{mach}}} \approx 1,05 \cdot 10^{-8}$, hat $\text{fl}(\mathbf{A}^\top \mathbf{A})$ zwei identische Spalten und damit Rang 1. An $\mathbf{A}$ selbst ist bei diesem ε nichts passiert: Ein Verfahren, das direkt mit $\mathbf{A}$ arbeitet, merkt von dem Problem nichts. Das ist mehr, als die Konditionszahl allein hergibt: Es geht Information verloren, bevor überhaupt gerechnet wird.

7.3.4 Alternative Lösungswege: orthogonale Transformationen

Wir würden $\mathbf{A}^\top \mathbf{A}$ also gerne ganz vermeiden und direkt auf $\mathbf{A}$ operieren. Erinnern wir uns an die Ausgangsform des Problems:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2.$$

Die Idee: Wir transformieren das Problem mit einer Matrix $\mathbf{Q}$ zu einem „einfacheren“ Problem $\mathbf{QAx} \approx \mathbf{Qb}$, ohne die Lösung zu ändern. Wann ändert sich die Lösung nicht? Wenn die Zielfunktion für *jedes* $\mathbf{x}$ denselben Wert behält, das Transformieren also alle Normen unangetastet lässt. Genau das leisten Orthogonalmatrizen ($\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$, ausführlich in Abschnitt 7.4), denn für diese gilt

$$\|\mathbf{QAx} - \mathbf{Qb}\|_2 = \|\mathbf{Q}(\mathbf{Ax} - \mathbf{b})\|_2 = \|\mathbf{Ax} - \mathbf{b}\|_2.$$

Die erste Gleichheit ist nur Ausklammern, die zweite die Norminvarianz orthogonaler Matrizen, die wir dort nachrechnen. Jedes $\mathbf{x}$ liefert im transformierten Problem also exakt denselben Zielfunktionswert wie im ursprünglichen; insbesondere stimmen die Minimierer überein.

Vorsicht: Nicht jede Transformation, die ein System „einfacher“ macht, ist hier erlaubt. Das Gauß-Verfahren etwa bringt $\mathbf{A}$ zwar auf Dreiecksform, aber seine Eliminationsschritte sind Scherungen: Sie verzerren Längen und würden den Minimierer verschieben. Für KQ-Probleme brauchen wir die norm-erhaltenden Transformationen: Drehungen und Spiegelungen. Wie wir mit ihnen $\mathbf{QA}$ gezielt auf Dreiecksform bringen (die *QR-Zerlegung*), ist das Thema von Abschnitt 7.4 und Abschnitt 7.5.

Interaktiv: Welche Abbildungen erhalten Längen – und welche nicht?

Probieren wir die drei Kandidaten der Reihe nach durch: eine Drehung, eine Spiegelung und einen Eliminationsschritt des Gauß-Verfahrens. Zwei davon dürfen wir auf ein KQ-Problem loslassen, einer nicht.

Interaktives Element – nur in der Web-Fassung

Wie das Widget zeigt, bilden Drehung und Spiegelung den Einheitskreis auf sich selbst ab, die Scherung dagegen auf eine Ellipse: Bei $m = 1,2$ wird aus $\|\mathbf{v}\|_2 = 1,803$ ein $\|\mathbf{Mv}\|_2 = 3,176$, ein Faktor 1,76. Dass $\det \mathbf{M} = 1$ bleibt, hilft nichts – Flächentreue ist eben nicht Längentreue. Genau deshalb sind für KQ-Probleme nur die orthogonalen Transformationen zugelassen.

Selbsttest

1. Im Widget oben lässt sich $\varepsilon = 10^{-k}$ einstellen. Ab welchem k speichert der Rechner $\text{fl}(1 + \varepsilon^2)$ als glatte 1?

Lösung: 8

Sobald ε^2 unter $\varepsilon_{\text{mach}} = 2^{-52}$ rutscht, verschwindet es in der Addition. Das passiert ab $\varepsilon \leq 2^{-26,5} \approx 1,05 \cdot 10^{-8}$, also ab $k \approx 8$. Ab dort sind die beiden Spalten der gespeicherten Matrix identisch, ihr Rang ist 1, und die Cholesky-Zerlegung aus Algorithmus 7.3.2 bricht ab.

2. Das Cholesky-Verfahren ist instabil, weil die Cholesky-Zerlegung selbst ein schlecht konditionierter Rechenschritt ist.

Antwort: Falsch

Die Cholesky-Zerlegung ist für SPD-Matrizen sogar ausgesprochen gutartig. Der Schaden entsteht bereits in Schritt 1: Mit $A^T A$ arbeiten wir ab dann mit einer Matrix der Konditionszahl $\kappa_2(A)^2$, und kein späterer Schritt kann das reparieren (Bemerkung 7.3.4).

3. Der Eliminationsschritt des Gauß-Verfahrens darf auf ein KQ-Problem nicht angewandt werden, weil er die euklidische Norm ändert.

Antwort: Wahr

Er ist eine Scherung; das Widget oben zeigt, wie aus dem Einheitskreis eine Ellipse wird. Damit ändert sich der Wert der Zielfunktion $\|Ax - b\|_2$ und mit ihm der Minimierer. Orthogonalmatrizen lassen beides unangetastet.

4. Weil $A^T A$ nur $n \times n$ groß ist, ist der Weg über die Normalengleichungen bei großem m auch numerisch der beste.

Antwort: Falsch

Klein ist er, schnell auch – aber die Verkleinerung ist genau der Schritt, der die Konditionszahl quadriert. Effizienz und Stabilität zeigen hier in verschiedene Richtungen; den Vergleich rechnen wir in Abschnitt 7.6 aus.

Vertiefung: Heath §3.4 (Normalengleichungen, Cholesky-Verfahren, orthogonale Transformationen); Heath §3.2 zur Herleitung; MML §7.1 für den Gradientenblick auf Optimierungsprobleme.

7.4 QR-Zerlegung und Gram-Schmidt-Verfahren

Am Ende von Abschnitt 7.3 stand der Plan: Wir transformieren das Kleinste-Quadrate-Problem mit Matrizen, die die euklidische Norm unverändert lassen, bis eine Dreiecksgestalt erreicht ist, in der es trivial lösbar wird. Das vermeidet $A^T A$ und damit die quadrierte Konditionszahl. Jetzt lösen wir das Versprechen ein.

Recap: Orthogonalmatrizen

Die normerhaltenden Matrizen kennen wir schon aus der linearen Algebra. Frischen wir das kurz auf.

Definition 7.4.1 (Orthogonalmatrix)

Eine Matrix $Q \in \mathbb{R}^{n \times n}$ heißt *Orthogonalmatrix*, wenn ihre Spalten orthonormale Vektoren sind, also

$$Q^T Q = I.$$

Geometrisch beschreibt Q eine Rotation und/oder Spiegelung des $\mathbb{R}^n$, also genau die Sorte Abbildung, die Längen und Winkel unangetastet lässt. Die für uns wichtigen Eigenschaften fassen wir zusammen:

Lemma 7.4.2 (Eigenschaften von Orthogonalmatrizen)

Sei $Q \in \mathbb{R}^{n \times n}$ orthogonal. Dann gilt:

1. $Q^{-1} = Q^T$, die Inverse ist also gratis.
2. $\|Qx\|_2 = \|x\|_2$ für alle $x \in \mathbb{R}^n$.
3. Alle Eigenwerte haben Betrag 1: $|\lambda_1(Q)| = \dots = |\lambda_n(Q)| = 1$, und es gilt $\kappa_2(Q) = 1$.
4. Für Orthogonalmatrizen $Q_1, \dots, Q_n$ ist auch das Produkt $Q = Q_1 \dots Q_n$ orthogonal.

Vertiefung: Beweis der Eigenschaften von Orthogonalmatrizen

Beweis.

(1)

$$(i): \quad Q^T Q = I \implies Q^{-1} = Q^T$$

$Q^T Q = I$ per Definition; eine quadratische Matrix mit Linksinverser ist invertierbar, und die Linksinverse ist dann die Inverse

(2)

$$(ii): \quad \|Qx\|_2 = \sqrt{x^T Q^T Q x} = \sqrt{x^T x} = \|x\|_2$$

Norm als Skalarprodukt schreiben und $Q^T Q = I$ einsetzen

(3)

$$(iii): \quad \|v\|_2 = \|Qv\|_2 = \|\lambda v\|_2 = |\lambda| \|v\|_2 \implies |\lambda| = 1$$

(ii) auf einen Eigenvektor $v \neq 0$ anwenden; κ_2 ist das Produkt $\|Q\|_2 \|Q^{-1}\|_2$, und beide Faktoren sind wegen (i) und (ii) gleich 1

(4)

$$(iv): \quad (Q_1 Q_2)^T (Q_1 Q_2) = Q_2^T \underbrace{Q_1^T Q_1}_{=I} Q_2 = Q_2^T Q_2 = I$$

Definition nachrechnen für zwei Faktoren; der allgemeine Fall folgt induktiv

□

Bemerkung 7.4.3

Eigenschaft (iii) ist der numerische Kern des ganzen Abschnitts: wegen $\kappa_2(Q) = 1$ ist die Multiplikation mit Q oder Q^{-1} immer gut konditioniert und verschlechtert die Stabilität eines Algorithmus nicht. Anders als die Normalengleichungen (Abschnitt 7.3) können orthogonale Transformationen die Kondition unseres Problems also nicht verschlimmern (vgl. die Konditionsanalyse in Abschnitt 7.2). Eigenschaft (iv) erlaubt uns außerdem, Q aus vielen einfachen orthogonalen Bausteinen zusammenzusetzen; das nutzen wir in Abschnitt 7.5 aus.

Interaktiv: Woran erkennt man Drehung und Spiegelung?

Jede Orthogonalmatrix in $\mathbb{R}^{2 \times 2}$ ist eine Drehung oder eine Spiegelung – zwei Typen, mehr gibt es nicht. Welche Größe unterscheidet sie, und welche bleibt in beiden Fällen gleich?

Interaktives Element — nur in der Web-Fassung

Das Widget verbindet die Determinante mit dem Typ der orthogonalen Abbildung und ihrer Normerhaltung.

Die QR-Zerlegung

Welche „einfache Gestalt“ streben wir mit unseren orthogonalen Transformationen an? Wie beim Gauß-Verfahren: eine obere Dreiecksmatrix, denn Dreieckssysteme lösen wir mühelos durch Rückwärtseinsetzen. Das führt auf die zentrale Zerlegung dieses Kapitels:

Definition 7.4.4 (QR-Zerlegung)

Sei $A \in \mathbb{R}^{m \times n}$ mit $m \geq n$ und $Q \in \mathbb{R}^{m \times m}$ eine Orthogonalmatrix, sodass

$$Q^T A = \begin{pmatrix} R \\ 0_{(m-n) \times n} \end{pmatrix},$$

wobei $R \in \mathbb{R}^{n \times n}$ eine obere Dreiecksmatrix ist. Dann nennen wir

$$A = Q \begin{pmatrix} R \\ 0 \end{pmatrix}$$

eine *QR-Zerlegung* von A .

Bemerkung 7.4.5

Eine QR-Zerlegung in der hier dargestellten Blockform existiert für jede Matrix $A \in \mathbb{R}^{m \times n}$ mit $m \geq n$. Für breite Matrizen $m < n$ verwendet man eine entsprechend transponierte oder trapezförmige Form. Eindeutig ist sie aber nicht: schon ein Vorzeichenwechsel in einer Spalte von Q (mit passendem Vorzeichenwechsel in der zugehörigen Zeile von R) liefert eine andere gültige Zerlegung.

Ein Beispiel: QR-Zerlegung über Gram-Schmidt

Woher bekommen wir Q und R ? Ein Verfahren kennen wir bereits aus der linearen Algebra: die Gram-Schmidt-Orthonormalisierung der Spalten von A . Sie liefert orthonormale Vektoren $q_1, q_2, \dots$, die denselben Raum aufspannen wie die Spalten $a_1, a_2, \dots$, und die Koeffizienten, die dabei anfallen (Normen und Projektions-Koeffizienten), sind genau die Einträge von R . Rechnen wir ein Beispiel vollständig durch. Zur Orientierung: q_1 verfolgt den ersten Orthonormalvektor, a_2 die zweite Spalte, R_{12} den Projektionskoeffizienten, $\tilde{q}_2$ den orthogonalisierten (noch unnormierten) und q_2 den fertigen zweiten Orthonormalvektor.

Beispiel 7.4.6 (QR-Zerlegung via Gram-Schmidt)

Gegeben: $A = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix}$. Gesucht: $Q^T A = \begin{pmatrix} R \\ \mathbf{0}_{1 \times 2} \end{pmatrix}$ mit $Q \in \mathbb{R}^{3 \times 3}$ orthogonal und $R \in \mathbb{R}^{2 \times 2}$ obere Dreiecksmatrix.

Schritt 1: erste Spalte $\mathbf{a}_1 = (1, 0, 0)^T$ normieren.

$$\mathbf{q}_1 = \frac{\mathbf{a}_1}{\|\mathbf{a}_1\|_2} = \frac{1}{1} \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad R_{11} = \|\mathbf{a}_1\|_2 = 1.$$

Schritt 2: zweite Spalte $\mathbf{a}_2 = (2, 0, 2)^T$ orthogonalisieren. Zuerst die Projektion auf $\mathbf{q}_1$:

$$R_{12} = \mathbf{q}_1^T \mathbf{a}_2 = 1 \cdot 2 + 0 \cdot 0 + 0 \cdot 2 = 2.$$

Dann den Projektionsanteil abziehen:

$$\tilde{\mathbf{q}}_2 = \mathbf{a}_2 - R_{12} \mathbf{q}_1 = \begin{pmatrix} 2 \\ 0 \\ 2 \end{pmatrix} - \begin{pmatrix} 2 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix}$$

und normieren:

$$\mathbf{q}_2 = \frac{\tilde{\mathbf{q}}_2}{\|\tilde{\mathbf{q}}_2\|_2} = \frac{1}{2} \begin{pmatrix} 0 \\ 0 \\ 2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, \quad R_{22} = \|\tilde{\mathbf{q}}_2\|_2 = 2.$$

Schritt 3: zu einer Orthonormalbasis des $\mathbb{R}^3$ vervollständigen. Wir brauchen ein $\mathbf{q}_3$ orthogonal zu $\mathbf{q}_1$ und $\mathbf{q}_2$; hier geht das durch Hinsehen:

$$\mathbf{q}_3 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}.$$

Ergebnis:

$$Q = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad R = \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix}.$$

Verifikation: Zerlegung ausmultiplizieren.

$$Q \begin{pmatrix} R \\ \mathbf{0} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix} = A \quad \square$$

Beachten wir die Arbeitsteilung: Q sammelt die orthonormalen Richtungen, R protokolliert, wie sich die Originalspalten aus diesen Richtungen linear kombinieren: $\mathbf{a}_1 = R_{11} \mathbf{q}_1$, $\mathbf{a}_2 = R_{12} \mathbf{q}_1 + R_{22} \mathbf{q}_2$. Dass R eine obere Dreiecksmatrix ist, ist kein Zufall, sondern die Struktur des Verfahrens: die k -te Spalte von A braucht nur die ersten k Orthonormalvektoren.

Interaktiv: Was bleibt von $\mathbf{a}_2$ übrig, wenn es fast auf $\mathbf{a}_1$ liegt?

Sehen wir uns den zweiten Schritt genauer an. Nach Abzug der Projektion bleibt von $\mathbf{a}_2$ nur der zu $\mathbf{q}_1$ senkrechte Anteil übrig, und genau den normieren wir anschließend. Wie viel ist das noch, wenn die beiden Spalten fast dieselbe Richtung haben?

Interaktives Element — nur in der Web-Fassung

Das Widget verbindet den Winkel der Spalten mit der Größe des zu normierenden Restvektors.

Vorsicht: So elegant Gram-Schmidt auf dem Papier ist, in Gleitkommaarithmetik ist das klassische Verfahren heikel. Sind zwei Spalten fast linear abhängig, ist das Residuum nach der Projektion winzig, und beim Normieren blasen wir Rundungsfehler massiv auf: die berechneten $\mathbf{q}_k$ verlieren ihre Orthogonalität. Ein einfacher Umbau hilft: Das *modifizierte Gram-Schmidt-Verfahren* (MGS) zieht die Projektionen nacheinander von der jeweils schon aktualisierten Spalte ab statt alle auf einmal von der Originalspalte. In exakter Arithmetik ändert das nichts, in Gleitkommaarithmetik alles. Wie dramatisch der Unterschied wird, messen wir am besten selbst.

Vertiefung: Numerische Stabilität der Gram-Schmidt-Varianten

Interaktiv: Wie schnell zerbricht Gram-Schmidt – und wie repariert man es?

Interaktives Element — nur in der Web-Fassung

Das Widget vergleicht den Orthogonalitätsverlust der drei Gram-Schmidt-Varianten. Ein Detail lohnt die Aufmerksamkeit, weil es leicht in die Irre führt: Für $p \gtrsim 8$ sinkt der gemessene MGS-Fehler wieder. Das ist kein Fortschritt, sondern derselbe Effekt wie in Abschnitt 7.3 – der Rechner rundet $1 + \varepsilon^2$ bereits zu 1, und das Experiment bildet die Theorie dort nicht mehr ab.

Warum die QR-Zerlegung das KQ-Problem löst

Jetzt zahlt sich die Vorarbeit aus. Haben wir eine QR-Zerlegung, zerfällt das Kleinste-Quadrate-Problem in einen Teil, den wir exakt auf null bringen können, und einen Teil, den keine Wahl von $\mathbf{x}$ beeinflusst. Zur Orientierung im Beweis: $\mathbf{R}\mathbf{x} - \mathbf{c}_1$ verfolgt den steuerbaren, $\mathbf{c}_2$ den unvermeidbaren Anteil.

Satz 7.4.7 (KQ-Lösung über die QR-Zerlegung)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$ und $\mathbf{Q} \in \mathbb{R}^{m \times m}$ eine Orthogonalmatrix, sodass

$$\mathbf{Q}^\top \mathbf{A} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0}_{(m-n) \times n} \end{pmatrix} \quad \text{und} \quad \mathbf{Q}^\top \mathbf{b} = \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix}$$

mit $\mathbf{R} \in \mathbb{R}^{n \times n}$ invertierbar. Dann gilt

$$\hat{\mathbf{x}} = \mathbf{R}^{-1} \mathbf{c}_1 = \arg \min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2.$$

Beweis.

(1)

$$\|\mathbf{Ax} - \mathbf{b}\|_2^2 = \|\mathbf{Q}^\top(\mathbf{Ax} - \mathbf{b})\|_2^2 = \|\mathbf{Q}^\top \mathbf{Ax} - \mathbf{Q}^\top \mathbf{b}\|_2^2$$

Multiplikation mit $\mathbf{Q}^\top$ ändert die Norm nicht (Lemma 7.4.2 (ii), angewandt auf die Orthogonalmatrix $\mathbf{Q}^\top$)

(2)

$$= \left\| \begin{pmatrix} \mathbf{Rx} - \mathbf{c}_1 \\ -\mathbf{c}_2 \end{pmatrix} \right\|_2^2$$

Blockstruktur einsetzen: $\mathbf{Q}^\top \mathbf{Ax} = \begin{pmatrix} \mathbf{Rx} \\ 0 \end{pmatrix}$ und $\mathbf{Q}^\top \mathbf{b} = \begin{pmatrix} \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix}$

(3)

$$= \|\mathbf{Rx} - \mathbf{c}_1\|_2^2 + \|\mathbf{c}_2\|_2^2$$

die Quadratsumme eines gestapelten Vektors zerfällt über die Blöcke: $\left\| \begin{pmatrix} \mathbf{u} \\ \mathbf{v} \end{pmatrix} \right\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2$

(4)

$$\arg \min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2 = \arg \min_{\mathbf{x}} \|\mathbf{Rx} - \mathbf{c}_1\|_2$$

der zweite Term $\|\mathbf{c}_2\|_2^2$ hängt nicht von $\mathbf{x}$ ab, minimieren müssen wir nur den ersten

(5)

$$\|\mathbf{R}\hat{\mathbf{x}} - \mathbf{c}_1\|_2 = \|\mathbf{R}\mathbf{R}^{-1}\mathbf{c}_1 - \mathbf{c}_1\|_2 = 0 \implies \hat{\mathbf{x}} = \mathbf{R}^{-1}\mathbf{c}_1 \text{ ist Minimierer.}$$

$\mathbf{R}$ ist invertierbar, also bringt $\hat{\mathbf{x}} = \mathbf{R}^{-1}\mathbf{c}_1$ den ersten Term exakt auf null, und kleiner als null kann eine Norm nicht werden

□

In der Praxis invertieren wir $\mathbf{R}$ natürlich nicht explizit, sondern lösen das Dreieckssystem $\mathbf{R}\hat{\mathbf{x}} = \mathbf{c}_1$ durch Rückwärtseinsetzen in $\mathcal{O}(n^2)$ Operationen. Als Bonus fällt die Residuumsnorm gratis ab: aus dem Beweis lesen wir $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2 = \|\mathbf{c}_2\|_2$ ab, ohne $\hat{\mathbf{x}}$ überhaupt einsetzen zu müssen.

Was noch fehlt, ist ein stabiler Weg zu $\mathbf{Q}$: Gram-Schmidt hat uns die Zerlegung im Beispiel geliefert, ist aber, wie oben gesehen, numerisch fragil. In Abschnitt 7.5 konstruieren wir $\mathbf{Q}$ deshalb wie beim Gauß-Verfahren spaltenweise aus einfachen orthogonalen Bausteinen: Givens-Rotationen und Householder-Spiegelungen.

Selbsttest

1. Stellen wir im Gram-Schmidt-Widget die beiden Spalten auf einen Winkel von 8° ein (etwa $\varphi_1 = 120^\circ$, $\varphi_2 = 112^\circ$) bei $\|\mathbf{a}_2\|_2 = 2,2$. Wie lang ist der senkrechte Anteil $\|\tilde{\mathbf{q}}_2\|_2$?

Lösung: 0.306

Der senkrechte Anteil ist $\|\mathbf{a}_2\|_2 \sin(8^\circ) = 2,2 \cdot 0,1392 = 0,306$, also knapp 14 % der ursprünglichen Länge. Diesen Rest teilen wir anschließend durch seine eigene Länge – mitsamt aller Rundungsfehler, die in ihm stecken.

2. $\mathbf{R}$ ist genau deshalb eine obere Dreiecksmatrix, weil die k -te Spalte von $\mathbf{A}$ nur die ersten k Orthonormalvektoren braucht.

Antwort: Wahr

Das ist die Struktur des Verfahrens: $\mathbf{q}_k$ entsteht erst aus $\mathbf{a}_k$, kann also in $\mathbf{a}_1, \dots, \mathbf{a}_{k-1}$ gar nicht vorkommen. Alle Einträge unterhalb der Diagonalen sind deshalb null.

3. Klassisches und modifiziertes Gram-Schmidt rechnen mathematisch Verschiedenes aus.

Antwort: Falsch

In exakter Arithmetik liefern beide dasselbe $\mathbf{Q}$ und dasselbe $\mathbf{R}$; sie unterscheiden sich nur in der Reihenfolge der Operationen. Erst in Gleitkommaarithmetik trennen sich die Wege, und zwar um den Faktor $\kappa_2(\mathbf{A})$ im Fehler.

4. Aus dem Beweis von Satz 7.4.7 lässt sich die minimale Residuumsnorm ablesen, ohne $\hat{\mathbf{x}}$ überhaupt einzusetzen.

Antwort: Wahr

Der Beweis zerlegt die Zielfunktion in $\|\mathbf{R}\mathbf{x} - \mathbf{c}_1\|_2^2 + \|\mathbf{c}_2\|_2^2$; der erste Term wird exakt null, also ist das Minimum gerade $\|\mathbf{c}_2\|_2$. Das fällt bei der QR-Zerlegung gratis ab.

Vertiefung: Heath §3.5 (Orthogonalisierungsverfahren, QR-Zerlegung, Gram-Schmidt); MML §3.8 (Orthonormalbasen) und MML §9.2 für die KQ-Perspektive der linearen Regression.

7.5 Konstruktion von $\mathbf{Q}$: Givens-Rotationen und Householder-Spiegelungen

In Abschnitt 7.4 haben wir gesehen: Sobald eine QR-Zerlegung von $\mathbf{A}$ vorliegt, schrumpft das Kleinste-Quadrate-Problem auf ein Dreieckssystem $\mathbf{R}\hat{\mathbf{x}} = \mathbf{c}_1$ zusammen. Offen blieb die entscheidende Frage: Wie finden wir die Orthogonalmatrix $\mathbf{Q}$ überhaupt? Das holen wir jetzt nach.

Die Strategie kennen wir im Kern schon vom Gauß-Verfahren: Wir gehen von links nach rechts durch die Spalten von $\mathbf{A}$ und entfernen die Einträge unter der Diagonalen, diesmal aber durch Multiplikation mit Orthogonalmatrizen $\mathbf{Q}_k$. Warum ausgerechnet orthogonal? Weil solche Matrizen die euklidische Norm jedes Vektors erhalten; das Kleinste-Quadrate-Kriterium bleibt unter ihnen also unverändert, wie wir in Abschnitt 7.4 ausgenutzt haben. Die beiden gängigsten Bausteine für die $\mathbf{Q}_k$ sind die *Givens-Rotation* und die *Householder-Spiegelung* (engl. *Householder reflection*). Beide schauen wir uns nun im Detail an.

Die Eliminationsaufgabe

Formulieren wir zuerst präzise, was ein einzelner Eliminationsschritt leisten muss. Sei $\mathbf{a} \in \mathbb{R}^n$ beliebig. Wir suchen eine Orthogonalmatrix $\mathbf{Q}_k$, sodass in $\mathbf{Q}_k\mathbf{a}$

- die ersten $k - 1$ Komponenten unberührt bleiben,
- die letzten $n - k$ Komponenten 0 sind:

$$Q_k \mathbf{a} = \begin{pmatrix} a_1 \\ \vdots \\ a_{k-1} \\ \alpha \\ 0 \\ \vdots \\ 0 \end{pmatrix}.$$

Der Wert α ist dabei nicht frei wählbar: Orthogonale Abbildungen erhalten Normen, also muss die gesamte „Länge“ des hinteren Teilvektors $(a_k, \dots, a_n)^\top$ in der k -ten Komponente landen. Es genügt deshalb, das Problem für diesen Teilvektor zu lösen. Damit sind wir beim Kernproblem dieses Abschnitts angekommen: Gegeben $\mathbf{a}$, finde ein orthogonales Q mit

$$Q\mathbf{a} = \alpha \mathbf{e}_1 = \alpha \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}, \quad \alpha = \pm \|\mathbf{a}\|_2.$$

Für $n = 2$ und $k = 1$ ist die Intuition ein Bild wert: Wir müssen $\mathbf{a}$ längentreu auf die erste Koordinatenachse bringen. Dafür gibt es genau zwei natürliche Bewegungen:

- **Givens:** wir *drehen* $\mathbf{a}$ auf $\alpha \mathbf{e}_1$ (eine Rotation),
- **Householder:** wir *spiegeln* $\mathbf{a}$ auf $\alpha \mathbf{e}_1$ (eine Spiegelung; mehr dazu auch in der Übung).

Givens-Rotationen

Beginnen wir mit der Drehung. Aus der linearen Algebra kennen wir die Drehmatrix der Ebene. Drehungen sind orthogonal und damit genau die Art von Transformation, die wir suchen.

Definition 7.5.1 (Givens-Rotation)

Seien $c, s \in \mathbb{R}$ mit $c^2 + s^2 = 1$. Dann heißt

$$G = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}$$

Givens-Rotation. Wegen $G^\top G = (c^2 + s^2)I = I$ ist jede solche Matrix orthogonal; c und s spielen die Rolle von Kosinus und Sinus des Drehwinkels.

Wie müssen wir c und s wählen, damit die Drehung $\mathbf{a}$ auf die erste Koordinatenachse bringt? Das folgende Resultat gibt die Antwort, und die Herleitung ist erfreulich kurz.

Satz 7.5.2 (Wahl von c und s)

Sei $\mathbf{a} = (a_1, a_2)^\top \neq \mathbf{0}$. Mit

$$c = \frac{a_1}{\sqrt{a_1^2 + a_2^2}}, \quad s = \frac{a_2}{\sqrt{a_1^2 + a_2^2}}$$

gilt $\mathbf{Ga} = \begin{pmatrix} \alpha \\ 0 \end{pmatrix}$ mit $\alpha = \|\mathbf{a}\|_2$.

Vertiefung: Herleitung der Givens-Parameter

Beweis.

(1)

$$\mathbf{Ga} = \begin{pmatrix} c a_1 + s a_2 \\ -s a_1 + c a_2 \end{pmatrix} \stackrel{!}{=} \begin{pmatrix} \alpha \\ 0 \end{pmatrix}$$

Matrixprodukt ausschreiben; die zweite Komponente soll verschwinden

(2)

$$r := \sqrt{a_1^2 + a_2^2} = \|\mathbf{a}\|_2 > 0, \quad c := \frac{a_1}{r}, \quad s := \frac{a_2}{r}$$

Ansatz aus dem Satz; wegen $c^2 + s^2 = (a_1^2 + a_2^2)/r^2 = 1$ ist G tatsächlich eine Givens-Rotation

(3)

$$-s a_1 + c a_2 = \frac{-a_2 a_1 + a_1 a_2}{r} = 0$$

einsetzen; die zweite Komponente verschwindet wie gefordert

(4)

$$c a_1 + s a_2 = \frac{a_1^2 + a_2^2}{r} = r = \|\mathbf{a}\|_2 = \alpha$$

einsetzen; Definition von r

□

Dass $\alpha = \|\mathbf{a}\|_2$ herauskommt, ist kein Zufall: Eine Orthogonalmatrix kann die Norm gar nicht ändern. Und noch eine Beobachtung: Um G aufzustellen, mussten wir den Drehwinkel nie ausrechnen; c und s ergeben sich unmittelbar aus a_1 und a_2 , ganz ohne Trigonometrie.

Beispiel 7.5.3 (Givens-Rotation)

Wir eliminieren die zweite Komponente von $\mathbf{a} = (4, 3)^\top$. Zuerst die Norm:

$$r = \sqrt{4^2 + 3^2} = \sqrt{16 + 9} = \sqrt{25} = 5.$$

Damit sind Kosinus und Sinus festgelegt:

$$c = \frac{4}{5} = 0,8, \quad s = \frac{3}{5} = 0,6.$$

Anwenden und nachrechnen:

$$\mathbf{Ga} = \begin{pmatrix} 0,8 & 0,6 \\ -0,6 & 0,8 \end{pmatrix} \begin{pmatrix} 4 \\ 3 \end{pmatrix} = \begin{pmatrix} 0,8 \cdot 4 + 0,6 \cdot 3 \\ -0,6 \cdot 4 + 0,8 \cdot 3 \end{pmatrix} = \begin{pmatrix} 3,2 + 1,8 \\ -2,4 + 2,4 \end{pmatrix} = \begin{pmatrix} 5 \\ 0 \end{pmatrix}.$$

In der zweiten Komponente steht die gewünschte Null, und wegen $\|\mathbf{Ga}\|_2 = 5 = \|\mathbf{a}\|_2$ hat die Rotation die Länge nicht angetastet, für eine Orthogonalmatrix keine Überraschung.

Interaktiv: Wo landet $G\mathbf{a}$, wenn wir $\mathbf{a}$ im Bild herumziehen?

Satz 7.5.2 bestimmt c und s allein aus den beiden Komponenten von $\mathbf{a}$. Sehen wir nach, was das geometrisch bedeutet: Wohin schickt G den Vektor, und wohin das Koordinatengitter?

Interaktives Element — nur in der Web-Fassung

Das Widget verknüpft die Givens-Koeffizienten mit einer längenerhaltenden Drehung auf die Achse.

Das Rezept trägt weiter als $\mathbb{R}^2$: Auch in $\mathbb{R}^n$ können wir damit jede einzelne Komponente eines Vektors zu Null machen. Dazu suchen wir uns neben der zu eliminierenden Komponente j eine Partnerkomponente i aus und bestimmen c und s nach Satz 7.5.2 aus den beiden Einträgen a_i und a_j . Anschließend setzen wir die 2×2 -Rotation in die Zeilen und Spalten i und j einer Einheitsmatrix ein. Für $n = 5$, $i = 3$ und $j = 5$ entsteht so

$$\begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & c & 0 & s \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & -s & 0 & c \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{pmatrix} = \begin{pmatrix} a_1 \\ a_2 \\ \alpha \\ a_4 \\ 0 \end{pmatrix}.$$

Alle übrigen Komponenten bleiben unberührt, genau wie es unsere Eliminationsaufgabe verlangt. Pro Rotation gewinnen wir allerdings nur eine einzige Null. Eine ganze Spalte räumen wir deshalb mit einer Folge solcher Rotationen leer. Dabei ist nur auf die Reihenfolge zu achten: Schon erzeugte Nullen dürfen von den nachfolgenden Rotationen nicht wieder mit Werten gefüllt werden.

Bemerkung 7.5.4 (Wann Givens?)

Eine Givens-Rotation fasst genau zwei Zeilen an; alle anderen Einträge der Matrix bleiben exakt so, wie sie sind. Deshalb greifen wir zu Givens, wenn wir gezielt einzelne Nullen erzeugen wollen: bei dünnbesetzten Matrizen, deren viele Nullen wir nicht zerstören möchten, oder wenn eine bestehende QR-Zerlegung nur um eine neue Datenzeile ergänzt werden soll. Der Preis: Eliminieren wir so eine voll besetzte Matrix komplett, fällt etwa das Anderthalbfache der Rechenoperationen des Householder-Zugangs an, den wir uns als Nächstes ansehen.

Householder-Spiegelungen

Eine Rotation erledigt eine Null pro Schritt. Wollen wir *alle* Einträge unter der Diagonalen einer Spalte in einem einzigen Schlag eliminieren, brauchen wir ein anderes Werkzeug: Statt $\mathbf{a}$ auf $\alpha \mathbf{e}_1$ zu drehen, spiegeln wir es an einer geschickt gewählten Hyperebene.

Definition 7.5.5 (Householder-Spiegelung)

Sei $\mathbf{v} \in \mathbb{R}^n$, $\mathbf{v} \neq \mathbf{0}$. Die Matrix

$$\mathbf{H} = \mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^\top}{\mathbf{v}^\top \mathbf{v}}$$

heißt *Householder-Spiegelung* (engl. *Householder reflection*, *elementary reflector*) mit Spiegelvektor $\mathbf{v}$.

Warum „Spiegelung“? Der Baustein $\mathbf{P} = \mathbf{v}\mathbf{v}^\top / (\mathbf{v}^\top \mathbf{v})$, ein skaliertes dyadisches Produkt, ist die orthogonale Projektion auf die Gerade $\text{span}(\mathbf{v})$ (lineare Hülle von $\mathbf{v}$). Entsprechend projiziert $\mathbf{I} - \mathbf{P}$ auf die Hyperebene $\text{span}(\mathbf{v})^\perp$, das orthogonale Komplement. $\mathbf{H} = \mathbf{I} - 2\mathbf{P}$ geht doppelt so weit: Es zieht von $\mathbf{a}$ den $\mathbf{v}$ -Anteil gleich *zweimal* ab und landet damit auf der anderen Seite der Hyperebene; $\text{span}(\mathbf{v})^\perp$ ist der Spiegel.

Vertiefung: Warum eine Householder-Spiegelung symmetrisch und orthogonal ist

Satz 7.5.6 (Symmetrie und Orthogonalität)

Jede Householder-Spiegelung $\mathbf{H}$ ist symmetrisch und orthogonal:

$$\mathbf{H}^\top = \mathbf{H}, \quad \mathbf{H}^\top \mathbf{H} = \mathbf{H}^2 = \mathbf{I}.$$

Beweis.

(1)

$$\mathbf{P} := \frac{\mathbf{v}\mathbf{v}^\top}{\mathbf{v}^\top \mathbf{v}} \implies \mathbf{P}^\top = \mathbf{P}$$

$$(\mathbf{v}\mathbf{v}^\top)^\top = (\mathbf{v}^\top)^\top \mathbf{v}^\top = \mathbf{v}\mathbf{v}^\top \text{ (Transponieren eines Produkts)}$$

(2)

$$\mathbf{P}^2 = \frac{\mathbf{v}(\mathbf{v}^\top \mathbf{v})\mathbf{v}^\top}{(\mathbf{v}^\top \mathbf{v})^2} = \frac{\mathbf{v}\mathbf{v}^\top}{\mathbf{v}^\top \mathbf{v}} = \mathbf{P}$$

$\mathbf{v}^\top \mathbf{v}$ ist ein Skalar (Skalarprodukt) und lässt sich kürzen: $\mathbf{P}$ ist eine Projektion

(3)

$$\mathbf{H}^\top = (\mathbf{I} - 2\mathbf{P})^\top = \mathbf{I} - 2\mathbf{P}^\top = \mathbf{I} - 2\mathbf{P} = \mathbf{H}$$

Linearität des Transponierens und Schritt 1

(4)

$$\mathbf{H}^\top \mathbf{H} = \mathbf{H}^2 = (\mathbf{I} - 2\mathbf{P})^2 = \mathbf{I} - 4\mathbf{P} + 4\mathbf{P}^2 = \mathbf{I}$$

ausmultiplizieren; Schritt 2 ($\mathbf{P}^2 = \mathbf{P}$)

□

$\mathbf{H}$ ist damit symmetrisch, orthogonal und selbstinvers: Eine Spiegelung, zweimal ausgeführt, ist die Identität. Bleibt die Frage, *welchen* Spiegelvektor $\mathbf{v}$ wir wählen müssen, damit $\mathbf{a}$ auf $\alpha \mathbf{e}_1$ landet. Die Antwort kann man aus dem geometrischen Bild ablesen: Der Spiegel muss den Winkel zwischen $\mathbf{a}$ und dem Ziel halbieren, also muss $\mathbf{v}$ in Richtung der Differenz $\mathbf{a} - \alpha \mathbf{e}_1$ zeigen. Rechnen wir nach, dass diese Wahl tatsächlich funktioniert.

Satz 7.5.7 (Wahl des Spiegelvektors)

Sei $\mathbf{a} \in \mathbb{R}^n$ und $\alpha = \pm \|\mathbf{a}\|_2$ so, dass $\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1 \neq \mathbf{0}$. Dann gilt für die Householder-Spiegelung $\mathbf{H}$ mit Spiegelvektor $\mathbf{v}$:

$$\mathbf{H}\mathbf{a} = \alpha \mathbf{e}_1.$$

Vertiefung: Warum der gewählte Spiegelvektor funktioniert*Beweis.*

(1)

$$\mathbf{v}^\top \mathbf{a} = (\mathbf{a} - \alpha \mathbf{e}_1)^\top \mathbf{a} = \mathbf{a}^\top \mathbf{a} - \alpha \mathbf{e}_1^\top \mathbf{a} = \|\mathbf{a}\|_2^2 - \alpha a_1$$

ausmultiplizieren; $\mathbf{e}_1^\top \mathbf{a} = a_1$ und $\mathbf{a}^\top \mathbf{a} = \|\mathbf{a}\|_2^2$

(2)

$$\mathbf{v}^\top \mathbf{v} = \|\mathbf{a}\|_2^2 - 2\alpha a_1 + \alpha^2 = 2(\|\mathbf{a}\|_2^2 - \alpha a_1)$$

binomisch ausmultiplizieren; $\alpha^2 = \|\mathbf{a}\|_2^2$, weil $\alpha = \pm \|\mathbf{a}\|_2$

(3)

$$2 \frac{\mathbf{v}^\top \mathbf{a}}{\mathbf{v}^\top \mathbf{v}} = \frac{2(\|\mathbf{a}\|_2^2 - \alpha a_1)}{2(\|\mathbf{a}\|_2^2 - \alpha a_1)} = 1$$

Schritte 1 und 2 einsetzen; wegen $\mathbf{v} \neq \mathbf{0}$ ist der Nenner nicht 0

(4)

$$\mathbf{H}\mathbf{a} = \mathbf{a} - 2\mathbf{v} \frac{\mathbf{v}^\top \mathbf{a}}{\mathbf{v}^\top \mathbf{v}} = \mathbf{a} - \mathbf{v} = \mathbf{a} - (\mathbf{a} - \alpha \mathbf{e}_1) = \alpha \mathbf{e}_1$$

Definition 7.5.5 anwenden; Schritt 3; $\mathbf{v}$ einsetzen

□

Bemerkenswert: Der komplette Bruch kollabiert zu 1, und übrig bleibt schlicht $\mathbf{a} - \mathbf{v}$. Ein Zahlenbeispiel macht das greifbar.

Beispiel 7.5.8 (Householder-Spiegelung)

Wir suchen die Householder-Spiegelung, die in $\mathbf{a} = (1, 2, 2)^\top$ alle Komponenten außer der ersten eliminiert. Zunächst die Norm:

$$\|\mathbf{a}\|_2 = \sqrt{1^2 + 2^2 + 2^2} = \sqrt{9} = 3.$$

Da $a_1 = 1 > 0$, wählen wir $\alpha = -3$ (warum das negative Vorzeichen die richtige Wahl ist, klärt Bemerkung 7.5.9). Der Spiegelvektor ist dann

$$\mathbf{v} = \mathbf{a} - \alpha \mathbf{e}_1 = \begin{pmatrix} 1 \\ 2 \\ 2 \end{pmatrix} + 3 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 4 \\ 2 \\ 2 \end{pmatrix}.$$

Die beiden Skalarprodukte aus dem Beweis von Satz 7.5.7:

$$\mathbf{v}^\top \mathbf{a} = 4 \cdot 1 + 2 \cdot 2 + 2 \cdot 2 = 12, \quad \mathbf{v}^\top \mathbf{v} = 4^2 + 2^2 + 2^2 = 24.$$

Wie vom Satz versprochen ist $2 \cdot \frac{12}{24} = 1$, also

$$\mathbf{H}\mathbf{a} = \mathbf{a} - \mathbf{v} = \begin{pmatrix} 1 - 4 \\ 2 - 2 \\ 2 - 2 \end{pmatrix} = \begin{pmatrix} -3 \\ 0 \\ 0 \end{pmatrix} = \alpha \mathbf{e}_1.$$

Die Länge stimmt, $\|\mathbf{H}\mathbf{a}\|_2 = 3 = \|\mathbf{a}\|_2$, wie es für eine Spiegelung sein muss. Und beide Komponenten unterhalb der ersten sind mit *einer einzigen* Spiegelung verschwunden.

Interaktiv: Warum wird der $\mathbf{v}$ -Anteil zweimal abgezogen?

In $\mathbf{H} = \mathbf{I} - 2\mathbf{P}$ steckt der Faktor 2, und der ist keine Kosmetik. Zerlegen wir die Spiegelung in ihre beiden Hälften: Der Wanderpunkt $\mathbf{w}(t) = \mathbf{a} - t \mathbf{v} \frac{\mathbf{v}^\top \mathbf{a}}{\mathbf{v}^\top \mathbf{v}}$ startet bei $t = 0$ in $\mathbf{a}$. Wo liegt er bei $t = 1$, und was ist dort mit seiner Länge?

Interaktives Element — nur in der Web-Fassung

Das Widget trennt die Projektion auf die Spiegelgerade von der vollständigen Householder-Spiegelung.

Bemerkung 7.5.9 (Vorzeichenwahl)

In exakter Arithmetik funktionieren beide Vorzeichen von $\alpha = \pm \|\mathbf{a}\|_2$, es gibt schließlich zwei Spiegelebenen, die $\mathbf{a}$ auf die Achse bringen. Numerisch sind sie *nicht* gleichwertig: Die erste Komponente des Spiegelvektors ist $v_1 = a_1 - \alpha$. Hat α dasselbe Vorzeichen wie a_1 und dominiert a_1 die Norm, subtrahieren wir zwei fast gleiche Zahlen; es droht Auslöschung der führenden Ziffern, und der winzige Rest besteht fast nur noch aus Rundungsfehlern. Deshalb wählt man

$$\alpha = -\text{sign}(a_1) \|\mathbf{a}\|_2,$$

denn dann ist $v_1 = a_1 + \text{sign}(a_1) \|\mathbf{a}\|_2$ eine Addition von Zahlen gleichen Vorzeichens und damit immer gutartig. Geometrisch heißt das: Wir spiegeln stets auf das Ziel, das *weiter* von $\mathbf{a}$ entfernt liegt.

Interaktiv: Zwei Vorzeichen für α – probieren wir beide

In exakter Arithmetik sind die beiden Vorzeichen gleichwertig. Testen wir das an $\mathbf{a} = (1, \delta)^\top$ mit kleinem δ , gerechnet in einer nachgestellten Gleitkommaarithmetik, die nach jeder Operation auf t signifikante Stellen rundet. Wie viele Ziffern von $v_1 = a_1 - \alpha$ überleben in den beiden Fällen?

Interaktives Element — nur in der Web-Fassung

Das Widget zeigt, wie die Wahl des Vorzeichens die Auslöschung beim Aufbau der Spiegelung beeinflusst.

Bemerkung 7.5.10 (Aufwand)

In der Praxis stellt niemand $\mathbf{H}$ als Matrix auf. Für einen beliebigen Vektor $\mathbf{u}$ gilt

$$\mathbf{H}\mathbf{u} = \mathbf{u} - \left(2 \frac{\mathbf{v}^\top \mathbf{u}}{\mathbf{v}^\top \mathbf{v}} \right) \mathbf{v},$$

also genügen zwei Skalarprodukte und eine Vektor-Aktualisierung: $\mathcal{O}(n)$ Operationen statt $\mathcal{O}(n^2)$ für das explizite Matrix-Vektor-Produkt (Landau-Notation). Genauso wird $\mathbf{Q}$ selbst nie explizit gebildet: Man speichert nur die Spiegelvektoren $\mathbf{v}_k$ (bzw. bei Givens die Paare (c, s)) und wendet sie bei Bedarf nacheinander an.

Von den Bausteinen zur QR-Zerlegung

Setzen wir die Teile zusammen. Im k -ten Eliminationsschritt betrachten wir vom aktuellen Zwischenprodukt die k -te Spalte ab der Diagonalen, also den Teilvektor $\mathbf{a} = (a_{kk}, \dots, a_{mk})^\top$. Dazu konstruieren wir nach Satz 7.5.7 eine Spiegelung $\tilde{\mathbf{H}}_k$ mit $\tilde{\mathbf{H}}_k \mathbf{a} = \alpha \mathbf{e}_1$ (oder eine Kette von Givens-Rotationen mit derselben Wirkung) und betten sie ein:

$$\mathbf{Q}_k = \begin{pmatrix} \mathbf{I}_{k-1} & \mathbf{0} \\ \mathbf{0} & \tilde{\mathbf{H}}_k \end{pmatrix}.$$

$\mathbf{Q}_k$ ist orthogonal, lässt die ersten $k - 1$ Zeilen unberührt und macht in Spalte k alle Einträge unter der Diagonalen zu Null. Entscheidend: Die schon fertigen Spalten $1, \dots, k - 1$ bleiben fertig. Warum? Unterhalb der Diagonalen stehen dort bereits Nullen, und $\mathbf{Q}_k$ verändert nur die Zeilen $k, \dots, m$; auf diesem Nullblock wirkt $\tilde{\mathbf{H}}_k$ als $\tilde{\mathbf{H}}_k \mathbf{0} = \mathbf{0}$. Nach $p = \min(n, m - 1)$ Schritten ist obere Dreiecksgestalt erreicht:

$$\mathbf{Q}_p \cdots \mathbf{Q}_1 \mathbf{A} = \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix} \implies \mathbf{Q}^\top = \mathbf{Q}_p \cdots \mathbf{Q}_1, \quad \mathbf{A} = \mathbf{Q} \begin{pmatrix} \mathbf{R} \\ \mathbf{0} \end{pmatrix}.$$

Das Produkt von Orthogonalmatrizen ist wieder orthogonal, also ist $\mathbf{Q} = \mathbf{Q}_1^\top \cdots \mathbf{Q}_p^\top$ tatsächlich die gesuchte Orthogonalmatrix, und wir haben eine QR-Zerlegung im Sinne von Abschnitt 7.4 konstruiert. Für das Kleinste-Quadrate-Problem wenden wir dieselben $\mathbf{Q}_k$ parallel auf $\mathbf{b}$ an und lösen anschließend $\mathbf{R} \hat{\mathbf{x}} = \mathbf{c}_1$ per Rücksubstitution.

Als Faustregel zum Mitnehmen: Für voll besetzte Matrizen ist **Householder** die Standardwahl: weniger Operationen pro Spalte, hervorragende numerische Stabilität; genau so arbeiten die QR-Löser der gängigen Software (etwa hinter $\text{lm}(\cdot)$ in R). **Givens** spielt seine Stärke aus, wenn Dünnbesetztheit (sparsity) erhalten bleiben soll oder nur einzelne Einträge zu eliminieren sind. Im Vergleich zum Weg über die Normalengleichungen (Abschnitt 7.3) vermeiden beide Zugänge das Quadrieren der Konditionszahl. Dazu mehr im Methodenvergleich in Abschnitt 7.6.

Selbsttest

1. Stellen wir im Givens-Widget $\mathbf{a} = (4, 3)^\top$ ein. Welchen Wert zeigt es für s ?

Lösung: 0.6

Nach Satz 7.5.2 ist $r = \sqrt{16 + 9} = 5$, also $c = 4/5 = 0.8$ und $s = 3/5 = 0.6$. Das Ergebnis ist $\mathbf{Ga} = (5, 0)^\top$: die Länge bleibt, die zweite Komponente wird null.

2. Bei $t = 1$ ist im Householder-Widget schon die halbe Arbeit getan, denn $\mathbf{w}(1)$ hat dieselbe Länge wie $\mathbf{a}$.

Antwort: Falsch

$\mathbf{w}(1)$ liegt zwar auf der Spiegelgeraden, ist aber echt kürzer als $\mathbf{a}$ – die Ablesezeile $\|\mathbf{w}(t)\|_2$ zeigt es. Die Projektion allein ist keine Orthogonalmatrix; erst der doppelte Schritt bringt die Länge zurück.

3. Eine Givens-Rotation erzeugt pro Anwendung genau eine Null, eine Householder-Spiegelung räumt eine ganze Spalte auf einmal leer.

Antwort: Wahr

Genau darin liegt die Arbeitsteilung: Givens fasst nur zwei Zeilen an und erhält deshalb Dünnbesetztheit, Householder ist für voll besetzte Matrizen billiger (Bemerkung 7.5.4).

4. $\alpha = -\text{sign}(a_1) \|a\|_2$ ist die numerisch sichere Wahl, weil dann v_1 aus einer Addition gleicher Vorzeichen entsteht.

Antwort: Wahr

Dann ist $v_1 = a_1 + \text{sign}(a_1) \|a\|_2$, und bei einer Addition zweier Zahlen gleichen Vorzeichens kann keine Auslöschung auftreten (Bemerkung 7.5.9).

Vertiefung: Heath §3.5.1 (Householder-Transformationen) und §3.5.2 (Givens-Rotationen); Methodenvergleich in Heath §3.7; zur Rolle orthogonaler Projektionen im Kleinste-Quadrate-Kontext auch MML §3.8.

7.6 Pseudoinverse, SVD-Lösung und Methodenvergleich

Zwei Lösungswege für das Kleinste-Quadrate-Problem kennen wir schon: die Normalengleichungen mit Cholesky-Zerlegung (Abschnitt 7.3) und die QR-Zerlegung (Abschnitt 7.4 und Abschnitt 7.5). Beide setzen stillschweigend $\text{rang}(A) = n$ voraus. Was aber, wenn A rangdefizient oder fast singulär ist? Dann ist $A^T A$ singulär, die Cholesky-Zerlegung bricht ab, und auch R in der QR-Zerlegung wird singulär. Genau hier hilft die Singulärwertzerlegung (SVD) aus Kapitel 6; sie liefert das dritte und robusteste Lösungsverfahren. Am Ende des Abschnitts vergleichen wir alle drei Methoden und fassen das Kapitel zusammen.

Die SVD-Lösung über die Pseudoinverse

Erinnern wir uns an Kapitel 6: Für $A \in \mathbb{R}^{m \times n}$ mit Rang r und reduzierter SVD $A = U_r \Sigma_r V_r^T$ ist die Moore-Penrose-Pseudoinverse definiert als $A^+ = V_r \Sigma_r^{-1} U_r^T \in \mathbb{R}^{n \times m}$, wobei $\Sigma_r^{-1} = \text{diag}(1/\sigma_1, \dots, 1/\sigma_r)$ eine Diagonalmatrix ist. Die zentrale Eigenschaft aus Kapitel 6: AA^+ ist die Projektion auf den Spaltenraum $\text{col}(A)$. Das ist exakt das, was wir für Kleinste Quadrate brauchen, denn in Abschnitt 7.1 haben wir gesehen: $\hat{x}$ löst das KQ-Problem genau dann, wenn $A\hat{x} = \text{proj}_{\text{col}(A)} b$.

Satz 7.6.1 (SVD-Lösung des KQ-Problems)

Sei $A \in \mathbb{R}^{m \times n}$ mit $\text{rang}(A) = r$ und reduzierter SVD $A = U_r \Sigma_r V_r^T$, und sei $b \in \mathbb{R}^m$. Dann löst

$$\hat{x} = A^+ b = V_r \Sigma_r^{-1} U_r^T b \quad (7.6.1)$$

das Kleinste-Quadrate-Problem $\min_x \|Ax - b\|_2$. Ist $r = n$, so ist $\hat{x}$ die eindeutige Lösung. Ist $r < n$, so ist $\hat{x}$ unter allen KQ-Lösungen diejenige mit minimaler Norm:

$$A^+ b = \arg \min \{\|x\|_2 : x \text{ minimiert } \|Ax - b\|_2\}.$$

Zuerst zeigen wir, dass $A\hat{x}$ tatsächlich die Projektion von b auf $\text{col}(A)$ ist; die Farben verfolgen U_r , Σ_r und V_r durch die Rechnung.

Vertiefung: Projektionsnachweis für die SVD-Lösung

Beweis.

(1)

$$\mathbf{A}\mathbf{A}^+ = \mathbf{U}_r \Sigma_r \mathbf{V}_r^T \mathbf{V}_r \Sigma_r^{-1} \mathbf{U}_r^T$$

reduzierte SVD und Definition der Pseudoinversen einsetzen; die inneren Faktoren treffen aufeinander

(2)

$$\mathbf{A}\mathbf{A}^+ = \mathbf{U}_r \Sigma_r \Sigma_r^{-1} \mathbf{U}_r^T = \mathbf{U}_r \mathbf{U}_r^T$$

$\mathbf{V}_r^T \mathbf{V}_r = \mathbf{I}_r$, denn die Spalten von $\mathbf{V}_r$ sind orthonormal; danach $\Sigma_r \Sigma_r^{-1} = \mathbf{I}_r$

(3)

$$\mathbf{A}\hat{\mathbf{x}} = \mathbf{A}\mathbf{A}^+ \mathbf{b} = \mathbf{U}_r \mathbf{U}_r^T \mathbf{b} = \text{proj}_{\text{col}(\mathbf{A})} \mathbf{b}$$

die Spalten von $\mathbf{U}_r$ bilden eine Orthonormalbasis von $\text{col}(\mathbf{A})$, also ist $\mathbf{U}_r \mathbf{U}_r^T$ die Projektionsmatrix auf $\text{col}(\mathbf{A})$

(4) Also minimiert $\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b}$ die Norm $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$.

das ist genau die geometrische Charakterisierung der KQ-Lösung aus Abschnitt 7.1

□

Vertiefung: Warum die Pseudoinverse die Lösung mit der kleinsten Norm liefert

Für $\text{rang}(\mathbf{A}) < n$ hat das KQ-Problem unendlich viele Lösungen. Dass $\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b}$ darunter die kürzeste ist, sagt der zweite Teil von Satz 7.6.1. Im Nachweis verfolgt $\mathbf{z}$ den Anteil einer beliebigen KQ-Lösung, der im Kern von $\mathbf{A}$ liegt.

Beweis.

(1)

$$\mathbf{x} = \hat{\mathbf{x}} + \mathbf{z} \quad \text{mit} \quad \mathbf{A}\mathbf{z} = \mathbf{0}$$

jede KQ-Lösung $\mathbf{x}$ erfüllt dieselbe Projektionsgleichung $\mathbf{A}\mathbf{x} = \text{proj}_{\text{col}(\mathbf{A})} \mathbf{b} = \mathbf{A}\hat{\mathbf{x}}$; die Differenz $\mathbf{z} = \mathbf{x} - \hat{\mathbf{x}}$ liegt also im Kern

(2)

$$\hat{\mathbf{x}} = \mathbf{V}_r (\Sigma_r^{-1} \mathbf{U}_r^T \mathbf{b}) \in \text{span}(\mathbf{V}_r) = \text{col}(\mathbf{A}^T) \perp \text{Kern}(\mathbf{A}) \ni \mathbf{z}$$

$\hat{\mathbf{x}}$ ist eine Linearkombination der Spalten von $\mathbf{V}_r$; deren Spann ist der Zeilenraum $\text{col}(\mathbf{A}^T)$, und der steht senkrecht auf dem Kern (orthogonales Komplement, Kapitel 6)

(3)

$$\|\mathbf{x}\|_2^2 = \|\hat{\mathbf{x}}\|_2^2 + \|\mathbf{z}\|_2^2 \geq \|\hat{\mathbf{x}}\|_2^2$$

Satz des Pythagoras für $\hat{\mathbf{x}} \perp \mathbf{z}$; Gleichheit nur für $\mathbf{z} = \mathbf{0}$

(4) $\hat{\mathbf{x}}$ hat unter allen KQ-Lösungen die kleinste Norm und ist bei vollem Rang die einzige Lösung.

für $\text{rang}(\mathbf{A}) = n$ ist $\text{Kern}(\mathbf{A}) = \{\mathbf{0}\}$, also $\mathbf{z} = \mathbf{0}$ erzwungen

□

Rechnen wir das an einem bewusst „kaputten“ Beispiel durch, an einer Matrix, an der die Normalgleichungen scheitern.

Beispiel 7.6.2 (rangdefizientes KQ-Problem)

Betrachten wir

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \text{rang}(\mathbf{A}) = 1 < n = 2.$$

Die Normalengleichungen sind hier chancenlos: $\mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$ ist singulär, die Cholesky-Zerlegung bricht ab. Die reduzierte SVD existiert trotzdem ($r = 1$): Wegen $\mathbf{A}^\top \mathbf{A}$ mit Eigenwerten 6 und 0 ist $\sigma_1 = \sqrt{6}$, und

$$\mathbf{A} = \mathbf{u}_1 \sigma_1 \mathbf{v}_1^\top \quad \text{mit} \quad \mathbf{u}_1 = \frac{1}{\sqrt{3}} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}, \quad \sigma_1 = \sqrt{6}, \quad \mathbf{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

(Probe: $\sqrt{6} \cdot \frac{1}{\sqrt{3}} \cdot \frac{1}{\sqrt{2}} = 1$; jeder Eintrag von $\mathbf{u}_1 \sigma_1 \mathbf{v}_1^\top$ ist 1.) Die Pseudoinverse wird damit

$$\mathbf{A}^+ = \mathbf{v}_1 \sigma_1^{-1} \mathbf{u}_1^\top = \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{6}} \cdot \frac{1}{\sqrt{3}} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix},$$

und die SVD-Lösung nach (7.6.1):

$$\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b} = \frac{1}{6} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix} = \frac{1}{6} \begin{pmatrix} 6 \\ 6 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Probe: $\mathbf{A} \hat{\mathbf{x}} = (2, 2, 2)^\top$ ist genau die Projektion von $\mathbf{b}$ auf $\text{col}(\mathbf{A}) = \text{span}\{(1, 1, 1)^\top\}$ (der Mittelwert von $\mathbf{b}$ ist 2). Alle KQ-Lösungen erfüllen $x_1 + x_2 = 2$, etwa auch $(2, 0)^\top$ mit Norm 2. Unsere Lösung $\hat{\mathbf{x}} = (1, 1)^\top$ hat Norm $\sqrt{2} \approx 1,41$ – die kleinste, wie Satz 7.6.1 verspricht.

SVD vs. Normalengleichungen

Der SVD-Zugang $\hat{\mathbf{x}} = \mathbf{A}^+ \mathbf{b}$ vermeidet die Schwächen des Wegs über die Normalengleichungen (Abschnitt 7.3): Er arbeitet direkt mit $\mathbf{A}$ statt mit $\mathbf{A}^\top \mathbf{A}$, quadriert die Konditionszahl also nicht, und er behandelt singuläre und rangdefiziente Probleme automatisch, wie Beispiel 7.6.2 vorgeführt hat. (7.6.1) ist zudem eine *explizite Konstruktion* der Lösung: Sobald die SVD vorliegt, ist kein Gleichungssystem mehr zu lösen, nur noch Matrix-Vektor-Produkte auszuwerten. Ein Vorbehalt bleibt: Die Pseudoinverse regularisiert exakt unidentifizierte Nullraumanteile, aber *keine* kleinen positiven Singulärwerte, die sie mit $1/\sigma_i$ gerade verstärkt. Gegen einen Fast-Rangdefekt helfen erst Verfahren wie die abgeschnittene SVD oder Tikhonov-Regularisierung.

Praktischer Methodenvergleich

Warum dann nicht immer SVD? Weil Robustheit ihren Preis hat. Fassen wir die Praxis-Regeln zusammen:

Bemerkung 7.6.3 (Anmerkungen zur Methodenwahl)

- Die QR-Zerlegung ist stabiler, aber langsamer als der Cholesky-Weg über die Normalengleichungen.

- Die SVD ist am langsamsten, aber sinnvoll, wenn A fast singulär ist, d. h. $\sigma_{\min}(A) \approx 0 \implies \kappa(A) \gg 1$.
- Wir können dann Σ_r verkleinern, indem wir die $r - k$ kleinsten Singulärwerte auf 0 setzen; das ist die Rang- k -Approximation aus Kapitel 6, hier als gezielte Glättung eines fast singulären Problems.
- Alle genannten Methoden haben für $m \gg n$ eine Komplexität von $O(mn^2 + n^3)$, aber die Konstanten unterscheiden sich deutlich.

Interaktiv: Wie teuer ist Stabilität?

Die O -Klasse ist bei allen drei Methoden dieselbe; entscheidend sind die Konstanten. Um welchen Faktor ist QR teurer als die Normalgleichungen – und hängt dieser Faktor von der Problemgröße ab? Die SVD-Konstante hängt vom Algorithmus ab (typisch 4–12, iterative Verfahren, vgl. Kapitel 6).

Interaktives Element – nur in der Web-Fassung

Das Widget trennt die gemeinsamen Größenordnungen von den konstanten Kostenfaktoren der Verfahren.

Interaktiv: Wie viele der 16 Stellen überleben?

In doppelter Genauigkeit stehen uns rund 16 Dezimalstellen zur Verfügung. Wie viele davon eine berechnete Lösung behält, hängt von der Maschinengenauigkeit ε , von $\kappa_2(A)$ und von der Größe des Residuums ab. Bevor wir das Widget befragen, lohnt sich eine Schätzung.

Interaktives Element – nur in der Web-Fassung

Das Widget verbindet Kondition und Residuum mit der erreichbaren Genauigkeit der drei Verfahren.

Zusammenfassung: das Kapitel im Überblick

Damit haben wir alle Bausteine des Kapitels beisammen:

- **Kleinste-Quadrate-Problem** (Abschnitt 7.1): minimiere $\|Ax - b\|_2$ für ein überbestimmtes LGS. Geometrisch: $A\hat{x}$ ist die Projektion von b auf $\text{col}(A)$.
- **Kondition** (Abschnitt 7.2): bzgl. b hängt sie vom Winkel zwischen b und $\text{col}(A)$ ab, bzgl. A ist sie typischerweise $O(\kappa_2(A)^2)$, also die quadrierte Konditionszahl.
- **Normalgleichungen + Cholesky** (Abschnitt 7.3): löse $A^T A x = A^T b$, also $\hat{x} = (A^T A)^{-1} A^T b$ falls $\text{rang}(A) = n$; $O(mn^2 + n^3)$. Schnell, aber wegen κ^2 potenziell instabil. Möglich, weil $A^T A$ SPD ist (falls $\text{rang}(A) = n$).
- **QR-Zerlegung** (Abschnitte 7.4–7.5): $A = Q \begin{pmatrix} R \\ 0 \end{pmatrix}$, löse $Rx = c_1$ mit $Q^T b = \begin{pmatrix} c_1 \\ c_2 \end{pmatrix}$; $O(mn^2)$. Stabil, quadriert die Kondition nicht und ist die Standardwahl.
- **SVD und Pseudoinverse** (dieser Abschnitt): $\hat{x} = A^+ b$; $O(\min(m^2 n, mn^2))$ mit großen Konstanten. Am stabilsten, funktioniert auch für rangdefiziente A und liefert dann die Minimal-

Norm-Lösung.

Warum ist das für die Statistik zentral? Wir haben fast immer mehr Beobachtungen als Parameter ($m > n$): Das lineare Modell $\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$ ist genau ein Kleinste-Quadrate-Problem, und auch nichtlineare Probleme werden oft iterativ gelöst: linear approximieren, Approximation minimieren, wiederholen. Die Projektionsgeometrie liefert dabei die Intuition, die Analysis die Normalengleichungen ($\nabla \|\mathbf{Ax} - \mathbf{b}\|_2^2 = \mathbf{0}$), und die numerische lineare Algebra sagt uns, welchen Lösungsweg wir wählen sollten: einen Kompromiss aus Geschwindigkeit und Stabilität.

Interaktiv: Welche Methode wählen wir wann?

Zum Schluss ein Sandkasten für alles zusammen: Form von $\mathbf{A}$, Kondition und Anspruch lassen sich frei kombinieren. Welche der drei Einstellungen ändert die Empfehlung am häufigsten?

Interaktives Element — nur in der Web-Fassung

Das Widget ordnet die Methodenwahl nach Stabilität, Rang und Aufwand.

Selbsttest

1. Stellen wir im Aufwands-Widget das Seitenverhältnis auf $m/n = 10$. Welchen Wert zeigt der Marker für $W(\text{QR})/W(\text{NE})$?

Lösung: 1.87

Mit $W_{\text{NE}} = mn^2/2 + n^3/6$ und $W_{\text{QR}} = mn^2 - n^3/3$ ist das Verhältnis $(r - 1/3)/(r/2 + 1/6)$ mit $r = m/n$, für $r = 10$ also $9,667/5,167 = 1,87$. Es hängt nur von r ab, nicht von n – beide Regler für n ändern daran nichts.

2. Weil die SVD am stabilsten ist, sollte man KQ-Probleme grundsätzlich über die Pseudoinverse lösen.

Antwort: Falsch

Robustheit hat ihren Preis: Die SVD kostet ein Vielfaches von QR (das Aufwands-Widget zeigt Faktoren um 7 bis 18). Für gut konditionierte Probleme ist dieser Aufwand nicht zu rechtfertigen; sinnvoll wird die SVD bei (fast) rangdefizientem $\mathbf{A}$ (Bemerkung 7.6.3).

3. Die Pseudoinverse regularisiert exakt unidentifizierte Nullraumanteile, aber keine kleinen positiven Singulärwerte.

Antwort: Wahr

Für $\sigma_i > 0$ steht in Σ_r^{-1} der Faktor $1/\sigma_i$, und der wird bei kleinem σ_i riesig. Gegen einen Fast-Rangdefekt helfen erst die abgeschnittene SVD oder Tikhonov-Regularisierung.

4. Bei großem Residuum verlieren QR und Normalengleichungen ähnlich viele Stellen.

Antwort: Wahr

Sobald $\|\mathbf{r}\|_2 \kappa_2(\mathbf{A}) > 1$ ist, dominiert der Term $\|\mathbf{r}\|_2 \kappa_2(\mathbf{A})^2 \varepsilon$ auch den QR-Fehler. Das Genauigkeits-Widget zeigt es: Beide Balken schrumpfen dann gleichermaßen. Der Unterschied zwischen den Verfahren zeigt sich bei kleinem Residuum.

Damit endet unser Rundgang durch die Kleinste-Quadrate-Verfahren. Im nächsten Kapitel wechseln wir die Perspektive: probabilistische Sichtweisen und iterative Methoden.

Vertiefung: Heath §3.6 (SVD und Pseudoinverse) und §3.7 (Methodenvergleich); MML §7.1 verbindet Kleinste Quadrate mit dem Gradientenblick auf Optimierung.

Numerische Lineare Algebra: Iteration & Zufall

8.1 Eigenwertprobleme: Potenzmethode und QR-Iteration

Die numerische lineare Algebra ist ein großes Feld, und bislang haben wir vor allem eine ihrer Kernideen ausgereizt: die *Matrixzerlegung*. LU, Cholesky, QR und die SVD lösen ihr Problem, indem sie die Matrix in Faktoren mit handlicher Struktur schreiben und die eigentliche Rechnung in diesen Faktoren erledigen. In diesem Kapitel kommen zwei weitere Kernideen dazu:

- *Iterative Methoden* erzeugen eine Folge von Näherungen und brechen ab, sobald eine davon genau genug ist. Sie tauschen Genauigkeit gegen Laufzeit.
- *Probabilistische Methoden* ersetzen ein großes Problem durch ein kleineres, dessen Lösung der gesuchten mit hoher Wahrscheinlichkeit sehr nahe kommt.

An beiden wird aktiv geforscht. Dieser Abschnitt und Abschnitt 8.2 behandeln die iterative Idee am Eigenwertproblem, Abschnitt 8.3 an linearen Gleichungssystemen; die probabilistische Idee kommt in Abschnitt 8.4 an die Reihe.

Was wir mitbringen

Dieses Kapitel greift auf fast alles zurück, was wir bisher aufgebaut haben:

- die Komplexitätsanalyse mit der O -Notation,
- Kondition und Stabilität,
- Orthogonalmatrizen (Abschnitt 3.4) und die Spektralnorm $\|\cdot\|_2$,
- lineare Gleichungssysteme samt ihrer direkten Lösungsverfahren und die QR-Zerlegung.

Dazu kommt Grundwissen aus der linearen Algebra: Eigenwerte und Eigenvektoren und Ähnlichkeitstransformationen $\mathbf{B} = \mathbf{Q}\mathbf{A}\mathbf{Q}^{-1}$. Aus der Analysis brauchen wir die Konvergenz von Folgen und die geometrische Reihe $\sum_k p^k$ für $|p| < 1$, aus der Wahrscheinlichkeitsrechnung Zufallsvektoren, Letztere aber erst in Abschnitt 8.4.

Eigenwerte ohne charakteristisches Polynom

Gesucht sind Paare aus Eigenwert und Eigenvektor, also λ und $\mathbf{v} \neq \mathbf{0}$ mit $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$. Ein direktes Verfahren dafür kennen wir aus der linearen Algebra: die Nullstellen des charakteristischen Polynoms $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$. Als Algorithmus taugt dieser Weg allerdings wenig.

Bemerkung 8.1.1 (Warum das charakteristische Polynom kein Algorithmus ist)

Für eine $n \times n$ -Matrix hat das charakteristische Polynom den Grad n . Daran hängen gleich drei Probleme:

1. Ab $n \geq 5$ gibt es keine allgemeine Lösungsformel für Polynomnullstellen mehr. Wir müssten die Nullstellen also ohnehin iterativ suchen und hätten nichts gewonnen.
2. Der Umweg über die Koeffizienten ist numerisch instabil: Winzige Störungen der Koeffizienten können die Nullstellen weit verschieben, das Problem ist also schlecht konditioniert.
3. Er ist teuer. Schon das Aufstellen der Koeffizienten kostet $O(n^4)$ oder mehr, während wir für eine ganze Matrixzerlegung mit $O(n^3)$ auskommen.

Bleiben wir also bei den Verfahren der numerischen linearen Algebra. Auch dort gibt es direkte Methoden, doch spätestens in drei Situationen wollen wir lieber iterieren:

- bei großen Matrizen ($n > 1000$), wo schon $O(n^3)$ wehtut,
- bei dünnbesetzten Matrizen, deren Struktur direkte Verfahren zerstören, weil sie die Nullen mit Zwischenergebnissen auffüllen,
- wenn wir gar nicht alle Eigenwerte brauchen, sondern nur wenige, etwa die $k \ll n$ größten.

Iterative Verfahren spielen hier ihre Stärken aus. Sie kommen oft mit Matrix-Vektor-Produkten als einziger Operation an A aus, nutzen also die Dünnbesetztheit unverändert weiter, und der Kompromiss zwischen Genauigkeit und Laufzeit ist über die Abbruchbedingung steuerbar.

Die Potenzmethode

Das einfachste iterative Verfahren ist zugleich das anschaulichste. Die *Potenzmethode* (power iteration, nach von Mises) berechnet den betragsgrößten Eigenwert λ_1 und einen zugehörigen Eigenvektor $\mathbf{v}_1$, vorausgesetzt, dieser Eigenwert ist von den übrigen getrennt. Die Idee besteht aus einer einzigen Zeile: Wir wenden A immer wieder an und normieren nach jedem Schritt.

Algorithmus 8.1.2 (Potenzmethode)

Gegeben seien $A \in \mathbb{R}^{n \times n}$ und ein Startvektor $\mathbf{x}^{(0)} \neq \mathbf{0}$.

1. Normiere den Start: $\mathbf{x}^{(0)} \leftarrow \mathbf{x}^{(0)} / \|\mathbf{x}^{(0)}\|$.
2. Für $k = 1, 2, \dots$:
 - (a) $\mathbf{y} = A \mathbf{x}^{(k-1)}$,
 - (b) $\lambda^{(k)} = \|\mathbf{y}\|$,
 - (c) $\mathbf{x}^{(k)} = \mathbf{y} / \lambda^{(k)}$.
3. Brich ab, sobald das Residuum $\|A \mathbf{x}^{(k)} - \lambda^{(k)} \mathbf{x}^{(k)}\|$ klein genug ist.

Pro Schritt fällt genau ein Matrix-Vektor-Produkt an. Für eine volle Matrix kostet das $O(n^2)$, für eine dünnbesetzte nur so viel, wie sie Einträge ungleich null hat. Zerlegt wird nichts, umgeformt wird nichts, A selbst bleibt unangetastet.

Bemerkung 8.1.3 (Zwei Schätzungen für den Eigenwert)

Wie man den Eigenwert abliest, hängt daran, ob unterwegs normiert wird. Ohne Normierung, also für die rohe Iteration $\mathbf{z}^{(k)} = \mathbf{A}\mathbf{z}^{(k-1)}$, wächst die Länge in jedem Schritt um ungefähr den Faktor $|\lambda_1|$, und der Quotient $\|\mathbf{z}^{(k)}\| / \|\mathbf{z}^{(k-1)}\|$ strebt gegen $|\lambda_1|$. Normieren wir dagegen wie in Algorithmus 8.1.2 nach jedem Schritt, so taugt derselbe Quotient nichts mehr: Zähler und Nenner haben denselben Grenzwert, er läuft also gegen 1. Mit Normierung lautet die Schätzung schlicht $\lambda^{(k)} = \|\mathbf{A}\mathbf{x}^{(k-1)}\|$, und weil eine Länge kein Vorzeichen liefern kann, konvergiert sie gegen $|\lambda_1|$.

Vertiefung: Der Rayleigh-Quotient als zweite Schätzung

Wer das Vorzeichen mitnehmen will, verwendet den *Rayleigh-Quotienten*

$$\rho^{(k)} = \frac{\mathbf{x}^{(k)\top} \mathbf{A} \mathbf{x}^{(k)}}{\mathbf{x}^{(k)\top} \mathbf{x}^{(k)}} = \mathbf{x}^{(k)\top} \mathbf{A} \mathbf{x}^{(k)},$$

wobei der Nenner wegen der Normierung gleich 1 ist. Für symmetrische $\mathbf{A}$ ist er die bessere Wahl. Beide Schätzungen sind dort quadratische Ausdrücke im Winkel zwischen $\mathbf{x}^{(k)}$ und $\mathbf{v}_1$, ihr Fehler fällt also mit $(\lambda_2/\lambda_1)^{2k}$ und damit doppelt so schnell wie der Winkel selbst; der Rayleigh-Quotient liegt dabei um einen konstanten Faktor näher dran und bringt das Vorzeichen mit. Ist $\mathbf{A}$ nicht symmetrisch, stehen die Eigenvektoren nicht mehr senkrecht aufeinander und beide Schätzungen sind nur noch linear genau.

Ein Wort noch zum Abbruchkriterium aus Algorithmus 8.1.2: Auch dort gehört $\rho^{(k)}$ hin, sobald λ_1 negativ sein kann. Für $\lambda_1 < 0$ wechselt $\mathbf{x}^{(k)}$ nämlich in jedem Schritt das Vorzeichen, und das Residuum mit $\lambda^{(k)} = |\lambda_1|$ bliebe bei ungefähr $2|\lambda_1|$ hängen, statt gegen null zu gehen.

Warum funktioniert das? Weil jede Anwendung von $\mathbf{A}$ die Komponente in Richtung $\mathbf{v}_1$ stärker verstärkt als alle anderen.

Satz 8.1.4 (Konvergenz der Potenzmethode)

Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$ diagonalisierbar mit einer Basis aus normierten Eigenvektoren $\mathbf{v}_1, \dots, \mathbf{v}_n$ und Eigenwerten $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$. Der Startvektor habe die Darstellung $\mathbf{x}^{(0)} = \sum_{i=1}^n c_i \mathbf{v}_i$ mit $c_1 \neq 0$. Dann gilt für die Iterierten aus Algorithmus 8.1.2

$$\mathbf{x}^{(k)} = \pm \mathbf{v}_1 + \mathbf{r}_k \quad \text{mit} \quad \|\mathbf{r}_k\| \leq C \left(\left| \frac{\lambda_2}{\lambda_1} \right| \right)^k \quad \text{für alle hinreichend großen } k$$

und einer von k unabhängigen Konstanten C , sowie $\lambda^{(k)} \rightarrow |\lambda_1|$. Für $\lambda_1 > 0$ ist das Vorzeichen dabei für alle k dasselbe, für $\lambda_1 < 0$ wechselt es in jedem Schritt.

Vertiefung: Beweis der Konvergenz der Potenzmethode

Beweis.

- (1) Wegen $\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i$ ist $\mathbf{A}^k \mathbf{v}_i = \lambda_i^k \mathbf{v}_i$. Die Basisdarstellung des Startvektors überträgt sich damit auf jede Potenz von $\mathbf{A}$:

$$\mathbf{A}^k \mathbf{x}^{(0)} = \sum_{i=1}^n c_i \lambda_i^k \mathbf{v}_i.$$

Induktion über k : aus $\mathbf{A}^k \mathbf{v}_i = \lambda_i^k \mathbf{v}_i$ folgt $\mathbf{A}^{k+1} \mathbf{v}_i = \mathbf{A}(\lambda_i^k \mathbf{v}_i) = \lambda_i^k \mathbf{A} \mathbf{v}_i = \lambda_i^{k+1} \mathbf{v}_i$; anschließend die Linearität von $\mathbf{A}^k$

- (2) Jetzt klammern wir den größten Faktor aus:

$$\mathbf{A}^k \mathbf{x}^{(0)} = \lambda_1^k \left(c_1 \mathbf{v}_1 + \underbrace{\sum_{i=2}^n c_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \mathbf{v}_i}_{=\mathbf{s}_k} \right). \quad (8.1.1)$$

$\lambda_i^k = \lambda_1^k (\lambda_i / \lambda_1)^k$; das geht, weil $|\lambda_1| > |\lambda_2| \geq 0$ und damit $\lambda_1 \neq 0$ ist

- (3) Der rote Restterm verschwindet, und zwar mit einer geometrischen Rate:

$$\|\mathbf{s}_k\| \leq \sum_{i=2}^n |c_i| \left(\left| \frac{\lambda_i}{\lambda_1} \right| \right)^k \leq \left(\sum_{i=2}^n |c_i| \right) \left(\left| \frac{\lambda_2}{\lambda_1} \right| \right)^k \rightarrow 0.$$

Übrig bleibt in der Klammer von (8.1.1) nur $c_1 \mathbf{v}_1$. Das ist die Intuition hinter der Methode: Jede Anwendung von $\mathbf{A}$ dämpft die i -te Komponente gegenüber der ersten um den Faktor $|\lambda_i / \lambda_1| < 1$.

Dreiecksungleichung und $\|\mathbf{v}_i\| = 1$; danach $|\lambda_i| \leq |\lambda_2|$ für alle $i \geq 2$. Die Potenzen einer Zahl mit Betrag kleiner als 1 bilden eine Nullfolge

- (4) Die Normierung in Algorithmus 8.1.2 ändert an der *Richtung* nichts. Per Induktion gilt

$$\mathbf{x}^{(k)} = \frac{\mathbf{A}^k \mathbf{x}^{(0)}}{\|\mathbf{A}^k \mathbf{x}^{(0)}\|}.$$

für $k = 1$ ist das die Vorschrift selbst; und ist $\mathbf{x}^{(k-1)}$ das normierte $\mathbf{A}^{k-1} \mathbf{x}^{(0)}$, so ist $\mathbf{A} \mathbf{x}^{(k-1)}$ ein positives Vielfaches von $\mathbf{A}^k \mathbf{x}^{(0)}$, und Normieren löscht diesen Faktor wieder aus

- (5) Einsetzen von (8.1.1) liefert die Behauptung: Mit $\mathbf{w}_k = c_1 \mathbf{v}_1 + \mathbf{s}_k$ ist

$$\mathbf{x}^{(k)} = \frac{\lambda_1^k \mathbf{w}_k}{|\lambda_1|^k \|\mathbf{w}_k\|} = \text{sign}(\lambda_1)^k \cdot \frac{\mathbf{w}_k}{\|\mathbf{w}_k\|} \rightarrow \pm \mathbf{v}_1,$$

denn $\mathbf{w}_k \rightarrow c_1 \mathbf{v}_1$ und $c_1 \neq 0$. Das verbleibende Vorzeichen ist das von c_1 , multipliziert mit $\text{sign}(\lambda_1)^k$. Weil der Fehler in $\mathbf{w}_k$ nach Schritt 3 wie $|\lambda_2 / \lambda_1|^k$ fällt, tut es der Fehler des normierten Vektors ebenso. Für die Eigenwertschätzung folgt schließlich

$$\lambda^{(k)} = \|\mathbf{A} \mathbf{x}^{(k-1)}\| \rightarrow \|\mathbf{A}(\pm \mathbf{v}_1)\| = |\lambda_1|.$$

Normieren ist stetig, solange der Nenner nicht gegen null geht, und $\|\mathbf{w}_k\| \rightarrow |c_1| > 0$

□

Rechnen wir das an der Beispielmatrix durch.

Beispiel 8.1.5 (Potenzmethode an einer 2×2-Matrix)

Sei

$$\mathbf{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}.$$

Das charakteristische Polynom ist

$$\det(\mathbf{A} - \lambda \mathbf{I}) = (5 - \lambda)(8 - \lambda) - 4 = \lambda^2 - 13\lambda + 36 = (\lambda - 9)(\lambda - 4),$$

also $\lambda_1 = 9$ und $\lambda_2 = 4$. Die Eigenvektoren lesen wir aus $(\mathbf{A} - \lambda \mathbf{I})\mathbf{v} = \mathbf{0}$ mit $\mathbf{v} = (a, b)^\top$ ab: Für $\lambda = 9$ heißt die erste Zeile $-4a - 2b = 0$, also $b = -2a$; für $\lambda = 4$ heißt sie $a - 2b = 0$, also $a = 2b$. Normiert ergibt das

$$\mathbf{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix} \approx \begin{pmatrix} -0,447 \\ 0,894 \end{pmatrix}, \quad \mathbf{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}.$$

Die beiden stehen senkrecht aufeinander, wie es der Spektralsatz für symmetrische Matrizen verlangt. Die zu erwartende Konvergenzrate ist $\lambda_2/\lambda_1 = 4/9 \approx 0,444$.

Wir starten mit $\mathbf{x}^{(0)} = (2; 1,3)^\top$. Weil die Eigenvektoren eine Orthonormalbasis bilden, sind die Koeffizienten der Basisdarstellung einfach die Skalarprodukte:

$$c_1 = \mathbf{v}_1^\top \mathbf{x}^{(0)} = \frac{-2 + 2,6}{\sqrt{5}} \approx 0,268, \quad c_2 = \mathbf{v}_2^\top \mathbf{x}^{(0)} = \frac{4 + 1,3}{\sqrt{5}} \approx 2,370.$$

Der Start liegt also fast in Richtung $\mathbf{v}_2$, und c_1 ist knapp neunmal kleiner als c_2 . Nach (8.1.1) muss die Iteration diesen Rückstand erst aufholen, und genau das zeigen die ersten Schritte:

k	$\mathbf{x}^{(k)}$	$\lambda^{(k)}$	$\rho^{(k)}$	$\sin \angle(\mathbf{x}^{(k)}, \mathbf{v}_1)$
0	(0,838; 0,545)	–	4,063	0,994
1	(0,756; 0,654)	4,102	4,305	0,969
2	(0,554; 0,833)	4,468	5,236	0,868
3	(0,195; 0,981)	5,663	7,122	0,613
4	(-0,131; 0,991)	7,523	8,469	0,326
5	(-0,307; 0,952)	8,608	8,885	0,151
6	(-0,385; 0,923)	8,917	8,977	0,068
8	(-0,435; 0,900)	8,997	8,999	0,013
10	(-0,445; 0,896)	9,000	9,000	0,003

Drei Beobachtungen. Erstens dauert es vier Schritte, bis die Iterierte überhaupt in den richtigen Quadranten wandert; solange dominiert der $\mathbf{v}_2$ -Anteil. Zweitens schrumpft der Winkel zur grünen Richtung ab dann in jedem Schritt um ungefähr 0,444, die Rate aus Satz 8.1.4: $0,151/0,326 \approx 0,46$ und $0,068/0,151 \approx 0,45$. Drittens sind beide Eigenwertschätzungen schneller am Ziel als die Richtung: Ihr Fehler fällt pro Schritt um ungefähr $(\lambda_2/\lambda_1)^2 \approx 0,198$, also quadratisch in der Rate.

Vertiefung: Der Genauigkeitsvorsprung des Rayleigh-Quotienten

Von $k = 8$ auf $k = 10$ geht der Fehler des Rayleigh-Quotienten von $-9,0 \cdot 10^{-4}$ auf $-3,5 \cdot 10^{-5}$ zurück, der Fehler der Normschätzung von $-3,3 \cdot 10^{-3}$ auf $-1,3 \cdot 10^{-4}$. Der Rayleigh-Quotient ist also nicht schneller, aber asymptotisch um den Faktor 3,7 genauer.

Bemerkung 8.1.6 (Wann die Potenzmethode versagt)

Zwei Voraussetzungen von Satz 8.1.4 sind verletzbar, und beide Male bricht die Konvergenz zusammen.

Kein Anteil in Richtung $\mathbf{v}_1$. Ist $c_1 = 0$, so fehlt in (8.1.1) der grüne Term. Die Iteration läuft dann gegen die Eigenrichtung mit dem größten Eigenwert, der im Startvektor überhaupt vertreten ist. Für symmetrisches A sind die Eigenvektoren orthogonal, dort heißt $c_1 = \mathbf{v}_1^\top \mathbf{x}^{(0)} = 0$ also schlicht: Der Startvektor steht senkrecht auf dem gesuchten Eigenvektor. In Beispiel 8.1.5 trifft das auf $\mathbf{x}^{(0)} = c \mathbf{v}_2$ zu. Dann ist $A\mathbf{x}^{(0)} = 4\mathbf{x}^{(0)}$, die normierte Iterierte bleibt für immer stehen, und beide Schätzungen liefern hartnäckig 4 statt 9. In der Praxis ist das weniger dramatisch, als es klingt: Ein zufällig gewählter Startvektor erfüllt $c_1 \neq 0$ mit Wahrscheinlichkeit 1, und Rundungsfehler schmuggeln beim Rechnen ohnehin

einen kleinen $\mathbf{v}_1$ -Anteil hinein, den die Iteration dann geduldig aufbläst.

Kein separierter größter Eigenwert. Gilt $|\lambda_1| = |\lambda_2|$, etwa bei $\lambda_2 = -\lambda_1$ oder bei einem komplexen Eigenwertpaar, so ist die Rate $|\lambda_2/\lambda_1| = 1$ und nichts konvergiert mehr. Und selbst wenn die beiden nur *nahe* beieinanderliegen, wird die Methode quälend langsam. Beide Male brauchen wir andere Verfahren.

Interaktiv: Die Potenzmethode Schritt für Schritt

Was geschieht, wenn der Start genau auf der zweiten Eigenrichtung liegt? Ziehen wir den Start auf dem Einheitskreis und testen wir diese Vermutung.

Interaktives Element — nur in der Web-Fassung

1. Welche Rate zeigt der Potenzmethoden-Stepper für die Beispielmatrix langfristig an?

Lösung: 0.4444444444

Der orange Wert nähert sich dem Verhältnis der beiden Eigenwertbeträge.

Ähnliche Matrizen als Werkzeug

Die Potenzmethode liefert einen Eigenwert. Wollen wir alle, brauchen wir eine andere Idee: Wir überführen A in eine Matrix B , der wir die Eigenwerte ansehen können, ohne sie dabei zu verändern. Genau das leisten Ähnlichkeitstransformationen.

Satz 8.1.7 (Ähnliche Matrizen haben dieselben Eigenwerte)

Sei $B = QAQ^{-1}$ mit invertierbarem $Q \in \mathbb{R}^{n \times n}$, also B ähnlich zu A . Dann gilt:

1. A und B haben dasselbe charakteristische Polynom, insbesondere dieselben Eigenwerte mit denselben Vielfachheiten, dieselbe Determinante und dieselbe Spur.
2. Ist $\mathbf{y}$ Eigenvektor von B zum Eigenwert λ , so ist $\mathbf{x} = Q^{-1}\mathbf{y}$ Eigenvektor von A zum selben Eigenwert.

Vertiefung: Rechnung zur Ähnlichkeit

Beweis.

- (1) Zu (2): Aus $B\mathbf{y} = \lambda\mathbf{y}$ wird durch Einsetzen der Definition von B

$$QAQ^{-1}\mathbf{y} = \lambda\mathbf{y} \implies A\mathbf{Q}^{-1}\mathbf{y} = \lambda\mathbf{Q}^{-1}\mathbf{y} \implies A\mathbf{x} = \lambda\mathbf{x}.$$

Multiplikation von links mit Q^{-1} ; und $\mathbf{x} = Q^{-1}\mathbf{y} \neq \mathbf{0}$, weil Q^{-1} invertierbar ist und $\mathbf{y} \neq \mathbf{0}$

- (2) Zu (1): Wir schieben $I = QQ^{-1}$ in den zweiten Summanden und verwenden die Multiplikativität der Determinante:

$$\det(B - \lambda I) = \det(Q(A - \lambda I)Q^{-1}) = \det(Q) \det(A - \lambda I) \det(Q)^{-1} = \det(A - \lambda I).$$

Beide Matrizen haben also dasselbe charakteristische Polynom und damit dieselben Eigenwerte samt Vielfachheiten.

$QAQ^{-1} - \lambda QIQ^{-1} = Q(A - \lambda I)Q^{-1}$, und $\det(Q)^{-1} = \det(Q^{-1})$ ist wegen der Invertierbarkeit definiert

- (3) Für $\lambda = 0$ liefert das $\det(\mathbf{B}) = \det(\mathbf{A})$. Die Gleichheit der Spuren steht schon in Abschnitt 3.1: Die Spur ist zyklisch, also $\text{tr}(\mathbf{QAQ}^{-1}) = \text{tr}(\mathbf{AQ}^{-1}\mathbf{Q}) = \text{tr}(\mathbf{A})$.

alternativ über das charakteristische Polynom: Spur und Determinante sind bis aufs Vorzeichen zwei seiner Koeffizienten, und die stimmen nach Schritt 2 überein

□

Besonders bequem wird es, wenn $\mathbf{B}$ diagonal ist: Dann stehen die Eigenwerte einfach da.

Beispiel 8.1.8 (Diagonalisierung durch eine Orthogonalmatrix)

Wir bleiben bei $\mathbf{A}$ aus Beispiel 8.1.5 und wählen

$$\mathbf{Q} = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \quad \text{mit} \quad \mathbf{Q}^\top \mathbf{Q} = \mathbf{I}.$$

$\mathbf{Q}$ ist orthogonal, also ist $\mathbf{Q}^{-1} = \mathbf{Q}^\top$ und die Ähnlichkeitstransformation $\mathbf{B} = \mathbf{QAQ}^\top$ kommt ohne jede Inversion aus. Rechnen wir sie in zwei Etappen aus. Zuerst

$$\mathbf{AQ}^\top = \frac{1}{\sqrt{5}} \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix},$$

denn etwa der Eintrag links oben ist $5 \cdot 2 + (-2) \cdot 1 = 8$. Dann

$$\mathbf{B} = \mathbf{Q}(\mathbf{AQ}^\top) = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 20 & 0 \\ 0 & 45 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}.$$

Die beiden Nullen sind kein Zufall: Die Zeilen von $\mathbf{Q}$ sind gerade die Eigenvektoren von $\mathbf{A}$. Auf der Diagonalen stehen deshalb die Eigenwerte, und sie stimmen mit den Nullstellen des charakteristischen Polynoms aus Beispiel 8.1.5 überein.

Zur Reihenfolge. Auf der Diagonalen steht $(4, 9)$ und nicht $(9, 4)$. Das ist keine Eigenschaft von $\mathbf{A}$, sondern eine der Spaltenwahl von $\mathbf{Q}^\top$: Die erste Spalte von $\mathbf{Q}^\top$ ist $\mathbf{v}_2$, die zweite ist $-\mathbf{v}_1$, und in dieser Reihenfolge landen die zugehörigen Eigenwerte auf der Diagonalen. Hätten wir die Spalten getauscht, stünde dort $(9, 4)$. Wir bleiben im ganzen Kapitel bei der absteigenden Sortierung $\lambda_1 = 9 \geq \lambda_2 = 4$, so wie bei der Potenzmethode.

Auch Satz 8.1.7(2) lässt sich hier ablesen: $\mathbf{y} = \mathbf{e}_2$ ist Eigenvektor von $\mathbf{B}$ zum Eigenwert 9, und $\mathbf{x} = \mathbf{Q}^{-1}\mathbf{e}_2 = \mathbf{Q}^\top \mathbf{e}_2 = \frac{1}{\sqrt{5}}(1, -2)^\top = -\mathbf{v}_1$ ist tatsächlich Eigenvektor von $\mathbf{A}$ zu demselben Eigenwert.

Bemerkung 8.1.9 (Welche Zielgestalt, welches Verfahren)

Beispiel 8.1.8 hat geschummelt: Wir kannten die Eigenvektoren schon und haben $\mathbf{Q}$ daraus gebaut. Ein Algorithmus muss ein passendes $\mathbf{Q}$ erst finden, und zwar iterativ. Er erzeugt eine Folge $\mathbf{B}^{(k)}$ von zu $\mathbf{A}$ ähnlichen Matrizen, die einer Zielgestalt immer näher kommt. Welche Zielgestalt, das unterscheidet die Verfahren:

- Die *QR-Iteration* strebt eine obere Dreiecksmatrix an, die *Schur-Zerlegung*; auf ihrer Diagonalen stehen die Eigenwerte. Analog gibt es LU- und Cholesky-Iterationen.
- Die *Lanczos-Iteration* strebt eine Tridiagonalmatrix an, also $b_{ij} = 0$ für $|i - j| > 1$, und lässt sich auf die wichtigsten Eigenwerte einschränken. Für große dünnbesetzte Matrizen ist sie deutlich effizienter.

In der Praxis kommen zu beiden noch numerische Kniffe hinzu, die sie stabil und schnell genug machen: *Shifted QR* und *Implicitly Restarted Lanczos*.

Die QR-Iteration

Die QR-Iteration ist von verblüffender Schlichtheit. Wir zerlegen die aktuelle Matrix in QR und multiplizieren die beiden Faktoren in der falschen Reihenfolge wieder zusammen.

Algorithmus 8.1.10 (QR-Iteration)

Gegeben sei $A \in \mathbb{R}^{n \times n}$.

1. Starte mit $A^{(0)} = A$.
2. Für $k = 1, 2, \dots$:
 - (a) Berechne die QR-Zerlegung $A^{(k-1)} = Q^{(k)} R^{(k)}$.
 - (b) Setze $A^{(k)} = R^{(k)} Q^{(k)}$.

Der zweite Schritt sieht willkürlich aus, ist aber genau die Ähnlichkeitstransformation aus dem letzten Abschnitt. Wegen $R^{(k)} = (Q^{(k)})^\top A^{(k-1)}$ ist nämlich

$$A^{(k)} = R^{(k)} Q^{(k)} = (Q^{(k)})^\top A^{(k-1)} Q^{(k)}.$$

Satz 8.1.11 (Die Iterierten sind ähnlich zu A)

Mit $Q_k := Q^{(1)} Q^{(2)} \dots Q^{(k)}$ gilt für die Iterierten aus Algorithmus 8.1.10

$$A^{(k)} = Q_k^\top A Q_k. \quad (8.1.2)$$

Insbesondere ist jedes $A^{(k)}$ ähnlich zu A und hat nach Satz 8.1.7 dieselben Eigenwerte.

Vertiefung: Beweis der Ähnlichkeit aller QR-Iterierten

Beweis.

- (1) Jedes $Q^{(j)}$ ist orthogonal, also ist auch das Produkt Q_k orthogonal und es gilt $Q_k^{-1} = Q_k^\top$.

Produkte von Orthogonalmatrizen sind orthogonal: $(QP)^\top (QP) = P^\top Q^\top QP = P^\top P = I$

- (2) Die Behauptung folgt per Induktion. Für $k = 1$ ist $A^{(1)} = (Q^{(1)})^\top A Q^{(1)} = Q_1^\top A Q_1$. Gilt sie für $k - 1$, so ist

$$A^{(k)} = (Q^{(k)})^\top A^{(k-1)} Q^{(k)} = (Q^{(k)})^\top Q_{k-1}^\top A Q_{k-1} Q^{(k)} = Q_k^\top A Q_k.$$

$$Q_{k-1} Q^{(k)} = Q_k \text{ nach Definition, und } (Q_{k-1} Q^{(k)})^\top = (Q^{(k)})^\top Q_{k-1}^\top$$

□

Damit ist klar, dass die Iteration nichts kaputt macht. Warum sie aber überhaupt irgendwohin läuft, verrät erst der folgende Satz. Er ist der eigentliche Grund, warum das Verfahren funktioniert.

Satz 8.1.12 (Die QR-Iteration zerlegt die Potenzen von A)

Mit $Q_k := Q^{(1)}Q^{(2)} \dots Q^{(k)}$ und $R_k := R^{(k)}R^{(k-1)} \dots R^{(1)}$ gilt für alle $k \geq 1$

$$A^k = Q_k R_k. \quad (8.1.3)$$

Die k -te Iterierte trägt also, versteckt in ihren Faktoren, die QR-Zerlegung der k -ten Potenz von A.

Vertiefung: Wie sich die Potenzzerlegung induktiv ergibt

Beweis.

- (1) *Induktionsanfang $k = 1$:* Der erste Schritt zerlegt $A = A^{(0)} = Q^{(1)}R^{(1)} = Q_1R_1$.

für $k = 1$ sind $Q_1 = Q^{(1)}$ und $R_1 = R^{(1)}$ genau die Faktoren des ersten Iterationsschritts

- (2) Wir notieren die Rechenregel, die den Schritt trägt. Für jedes i ist $A^{(i)} = R^{(i)}Q^{(i)}$ nach Algorithmus 8.1.10 und $A^{(i)} = Q^{(i+1)}R^{(i+1)}$ nach der Zerlegung im nächsten Schritt, also

$$R^{(i)}Q^{(i)} = Q^{(i+1)}R^{(i+1)}. \quad (*)$$

dieselbe Matrix $A^{(i)}$, einmal als Ergebnis von Schritt i und einmal als Eingang von Schritt $i + 1$ gelesen

- (3) *Induktionsschritt $k \rightarrow k + 1$:* Wir setzen die Induktionsvoraussetzung ein und schieben dann k -mal $(*)$ von links nach rechts durch das Produkt.

$$\begin{aligned} A^{k+1} &= A A^k = (Q^{(1)}R^{(1)})(Q_k R_k) \\ &= Q^{(1)} \underbrace{(R^{(1)}Q^{(1)})}_{= Q^{(2)}R^{(2)}} Q^{(2)} \dots Q^{(k)} R^{(k)} \dots R^{(1)} \\ &= Q^{(1)}Q^{(2)} \underbrace{(R^{(2)}Q^{(2)})}_{= Q^{(3)}R^{(3)}} Q^{(3)} \dots Q^{(k)} R^{(k)} \dots R^{(1)} \\ &= \dots = Q^{(1)}Q^{(2)} \dots Q^{(k+1)} R^{(k+1)}R^{(k)} \dots R^{(1)} \\ &= Q_{k+1}R_{k+1}. \end{aligned}$$

in der zweiten Zeile ist $Q_k R_k$ ausgeschrieben; jede Anwendung von $()$ tauscht ein $R^{(i)}Q^{(i)}$ gegen $Q^{(i+1)}R^{(i+1)}$ und schiebt so ein weiteres Q nach vorn*

□

Bemerkung 8.1.13 (Die QR-Iteration ist eine simultane Potenzmethode)

Gleichung (8.1.3) macht das Verfahren durchschaubar. Bei der üblichen Normierung mit positiven Diagonaleinträgen in R ist die QR-Zerlegung gerade das Gram-Schmidt-Verfahren auf den Spalten. Die Spalten von Q_k sind demnach die orthonormalisierten Versionen von

$$A^k e_1, \quad A^k e_2, \quad \dots, \quad A^k e_n.$$

Jede einzelne dieser Spalten ist eine unnormierte Iterierte der Potenzmethode, gestartet in einem Einheitsvektor. Die QR-Iteration ist also eine *simultane* Potenzmethode auf allen n Einheitsvektoren zugleich, mit einer Orthonormalisierung nach jedem Schritt. Diese Orthonormalisierung ist der ganze Trick: Ohne sie würden alle n Spalten in dieselbe Richtung v_1 kippen und die Information über die übrigen Eigenrichtungen ginge verloren.

Bemerkung 8.1.14 (Wegen die QR-Iteration konvergiert)

Bei der Konvergenz ist Vorsicht geboten, denn sie gilt nicht ohne Voraussetzungen. Ein Gegenbeispiel liefert die Drehung um 90° , $A = \begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$: Hier ist $Q^{(1)} = A$ und $R^{(1)} = I$, also $A^{(1)} = A$, und die Iteration steht für immer still. Ihre Eigenwerte $\pm i$ sind komplex und betragsgleich.

Sind dagegen alle Eigenwerte reell und betragsmäßig getrennt,

$$|\lambda_1| > |\lambda_2| > \dots > |\lambda_n| > 0,$$

so konvergiert $A^{(k)}$ gegen eine obere Dreiecksmatrix mit den Eigenwerten auf der Diagonalen. Das ist die *Schur-Zerlegung* von A . Die Einträge unterhalb der Diagonalen fallen dabei linear, der Eintrag an der Stelle (i, j) mit $i > j$ wie $|\lambda_i/\lambda_j|^k$; die langsamste dieser Raten gehört zum engsten Paar benachbarter Eigenwerte.

Ist A zusätzlich **symmetrisch**, so ist wegen (8.1.2) auch jedes $A^{(k)} = Q_k^T A Q_k$ symmetrisch, und eine symmetrische obere Dreiecksmatrix ist diagonal. In diesem Fall gilt außerdem $A = V \Lambda V^T$ mit orthogonalem V aus Eigenvektoren, und mit $Q_k \rightarrow V$ folgt

$$Q_k^T A Q_k \longrightarrow V^T A V = V^T V \Lambda V^T V = \Lambda.$$

Diese Rechnung hängt ganz am Spektralsatz: Nur für symmetrisches A gibt es die Zerlegung $A = V \Lambda V^T$ mit *orthogonalem* V , und nur deshalb kürzt sich $V^T V$ in der Kette zweimal zur Einheitsmatrix weg. Im allgemeinen Fall ist der Grenzwert von Q_k der Orthogonalfaktor der Schur-Zerlegung, und von dessen Spalten ist nur die erste ein Eigenvektor. Die übrigen Eigenvektoren rechnen wir uns dort aus der Dreiecksmatrix zurück.

Beispiel 8.1.15 (QR-Iteration an der Beispielmatrix)

Wir nehmen wieder $A = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}$ und rechnen den ersten Schritt von Hand. Gram-Schmidt auf den Spalten $a_1 = (5, -2)^T$ und $a_2 = (-2, 8)^T$ liefert

$$r_{11} = \|a_1\| = \sqrt{29}, \quad q_1 = \frac{1}{\sqrt{29}} \begin{pmatrix} 5 \\ -2 \end{pmatrix}, \quad r_{12} = q_1^T a_2 = \frac{-10 - 16}{\sqrt{29}} = \frac{-26}{\sqrt{29}},$$

und nach Abzug der Projektion bleibt $w = a_2 - r_{12}q_1 = \frac{36}{29}(2, 5)^T$ mit $r_{22} = \|w\| = 36/\sqrt{29}$. Zusammen also

$$Q^{(1)} = \frac{1}{\sqrt{29}} \begin{pmatrix} 5 & 2 \\ -2 & 5 \end{pmatrix}, \quad R^{(1)} = \frac{1}{\sqrt{29}} \begin{pmatrix} 29 & -26 \\ 0 & 36 \end{pmatrix}.$$

Eine schnelle Probe: $r_{11}r_{22} = \sqrt{29} \cdot 36/\sqrt{29} = 36 = \det(A)$. Nun die Vertauschung:

$$A^{(1)} = R^{(1)}Q^{(1)} = \frac{1}{29} \begin{pmatrix} 29 & -26 \\ 0 & 36 \end{pmatrix} \begin{pmatrix} 5 & 2 \\ -2 & 5 \end{pmatrix} = \frac{1}{29} \begin{pmatrix} 197 & -72 \\ -72 & 180 \end{pmatrix} \approx \begin{pmatrix} 6,793 & -2,483 \\ -2,483 & 6,207 \end{pmatrix}.$$

Spur und Determinante sind unverändert 13 und 36: Satz 8.1.11 macht $A^{(1)}$ ähnlich zu A , und Satz 8.1.7(1) hält beide Größen unter Ähnlichkeit fest. Weiter iteriert ergibt sich (die Matrizen bleiben symmetrisch, also genügen drei Zahlen pro Zeile):

k	$a_{11}^{(k)}$	$a_{21}^{(k)}$	$a_{22}^{(k)}$	Schrumpffaktor
0	5,000	-2,000	8,000	–
1	6,793	-2,483	6,207	1,241
2	8,325	-1,709	4,675	0,688
3	8,850	-0,852	4,150	0,498
4	8,970	-0,388	4,030	0,455
5	8,994	-0,173	4,006	0,447
6	8,999	-0,077	4,001	0,445
9	9,000	-0,007	4,000	0,444

Die Diagonale läuft auf (9, 4) zu, absteigend nach Betrag, und der rote Eintrag unter der Diagonalen verschwindet. Sein Schrumpffaktor nähert sich $|\lambda_2/\lambda_1| = 0,444$ an, derselben Rate wie bei der Potenzmethode; im ersten Schritt wächst er sogar noch, die Rate ist eben eine asymptotische Aussage. Das aufgesammelte Produkt Q_9 hat die Spalten $(0,448; -0,894)^T$ und $(0,894; 0,448)^T$, also $-v_1$ und v_2 bis auf Rundung. Ein Vergleich mit Beispiel 8.1.8 lohnt sich: Dort haben wir dieselbe Diagonalisierung in einem Schritt hingeschrieben, weil wir die Eigenvektoren schon kannten. Die QR-Iteration findet sie selbst.

Interaktiv: Die QR-Iteration zum Durchklicken

Links stehen die Einträge von $A^{(k)}$, dazu die Faktoren des letzten Schritts und das aufgesammelte Produkt Q_k ; rechts fällt der Betrag der Nebendiagonalen auf logarithmischer Skala, wo lineare Konvergenz als Gerade erscheint. Neben der Beispielmatrix lohnen sich zwei weitere Eingaben: die unsymmetrische Matrix $\begin{pmatrix} 2 & 3 \\ 1 & 4 \end{pmatrix}$, bei der der Grenzwert dreieckig und nicht diagonal ist, und die Drehung aus Bemerkung 8.1.14, bei der gar nichts passiert.

Interaktives Element – nur in der Web-Fassung

Kondition und Aufwand

Bemerkung 8.1.16 (Kondition von Eigenwertproblemen)

Wie gut konditioniert ein Eigenwertproblem ist, hängt stark von der Matrix ab.

- *Eigenwerte* sind gut konditioniert, wenn A nahezu symmetrisch ist. Für exakt symmetrische Matrizen verschiebt eine ebenfalls symmetrische Störung Δ jeden Eigenwert um höchstens $\|\Delta\|_2$. Bei stark unsymmetrischen Matrizen gibt es keine solche Schranke.
- *Eigenvektoren* sind schlecht konditioniert, wenn Eigenwerte nahe beieinanderliegen. Der Grenzfall macht klar, warum: Bei $\lambda_i = \lambda_j$ spannen die zugehörigen Eigenvektoren eine ganze Ebene auf, in der jede Richtung gleich gut ist. Liegen die Eigenwerte nur nahe beieinander, ist die Richtung fast beliebig, und eine kleine Störung dreht sie weit.

Das passt zu dem, was wir an beiden Verfahren gesehen haben: Die Rate $|\lambda_2/\lambda_1|$ geht gegen 1, sobald sich die Eigenwerte nähern. Schlechte Kondition und langsame Konvergenz haben hier dieselbe Ursache.

Bemerkung 8.1.17 (Aufwand)

Der praktische QR-Eigenwertalgorithmus reduziert eine volle Matrix einmalig in $O(n^3)$ auf Hessenbergform. Danach kostet ein QR-Sweep $O(n^2)$; bei einer symmetrischen Matrix reduziert man zunächst auf Tridiagonalform, sodass ein impliziter Sweep sogar $O(n)$ kostet. Wie viele Sweeps nötig sind, hängt von Shifts, Deflation und Eigenwertverteilung ab. Ein Schritt der Potenzmethode ist mit einem Matrix-Vektor-Produkt dagegen konkurrenzlos billig, liefert aber eben auch nur einen Eigenwert.

Was sich mit der Potenzmethode allein alles anstellen lässt, sehen wir in Abschnitt 8.2 am PageRank-Algorithmus; für große dünnbesetzte Matrizen kommt dort auch die Lanczos-Iteration aus Bemerkung 8.1.9 wieder vor.

Selbsttest

1. Für $\mathbf{x}^{(0)} = c \mathbf{v}_1$ mit $c \neq 0$ versagt die Potenzmethode.

Antwort: Falsch

Im Gegenteil, das ist der bequemste aller Startvektoren: Er hat $c_1 = c \neq 0$, und die Iteration ist bereits im ersten Schritt am Ziel, denn $\mathbf{A}\mathbf{v}_1 = \lambda_1 \mathbf{v}_1$ ändert die Richtung nicht.

2. Für $\mathbf{x}^{(0)} = c \mathbf{v}_2$ mit $c \neq 0$ versagt die Potenzmethode.

Antwort: Wahr

Hier ist $c_1 = 0$, in (8.1.1) fehlt also der Term mit $\mathbf{v}_1$. Bei symmetrischem $\mathbf{A}$ heißt das anschaulich: Der Startvektor steht senkrecht auf dem gesuchten Eigenvektor. Die Iterierte bleibt in Richtung $\mathbf{v}_2$ stehen und die Schätzung liefert λ_2 statt λ_1 (Bemerkung 8.1.6).

3. Für $\mathbf{x}^{(0)} = c(\mathbf{v}_1 + \mathbf{v}_2)$ mit $c \neq 0$ versagt die Potenzmethode.

Antwort: Falsch

Dieser Start hat $c_1 = c \neq 0$, damit ist die Voraussetzung von Satz 8.1.4 erfüllt. Die Methode konvergiert mit der üblichen Rate $|\lambda_2/\lambda_1|$.

4. Ähnliche Matrizen haben dasselbe charakteristische Polynom.

Antwort: Wahr

Das ist Satz 8.1.7(1), bewiesen über $\det(\mathbf{Q}(\mathbf{A} - \lambda \mathbf{I})\mathbf{Q}^{-1}) = \det(\mathbf{A} - \lambda \mathbf{I})$. Daraus folgen gleiche Eigenwerte samt Vielfachheiten und damit auch gleiche Spur und Determinante. Die Eigenvektoren sind dagegen verschieden, sie gehen über $\mathbf{x} = \mathbf{Q}^{-1}\mathbf{y}$ auseinander hervor.

5. Die QR-Iteration konvergiert für jede reelle Matrix gegen eine obere Dreiecksmatrix.

Antwort: Falsch

Die Drehung um 90° ist ein Gegenbeispiel: Dort ist $\mathbf{Q}^{(1)} = \mathbf{A}$ und $\mathbf{R}^{(1)} = \mathbf{I}$, die Iteration bleibt also für immer bei $\mathbf{A}$ stehen (Bemerkung 8.1.14). Ihre Eigenwerte $\pm i$ sind komplex und betragsgleich. Für die Konvergenz gegen die Schur-Form brauchen wir reelle, betragsmäßig getrennte Eigenwerte.

6. Ein Schritt der Potenzmethode kostet $O(n^3)$ Operationen.

Antwort: Falsch

Ein Schritt besteht aus einem Matrix-Vektor-Produkt und einer Normierung, das sind $O(n^2)$ Operationen für eine volle Matrix und noch weniger für eine dünnbesetzte. $O(n^3)$ kostet dagegen jede einzelne QR-Iteration, weil dort in jedem Schritt eine ganze Matrix zerlegt wird.

(Bemerkung 8.1.17).

Vertiefung: Heath §4.5 zu Potenzmethode, Ähnlichkeitstransformationen und QR-Iteration.

8.2 Anwendungen: PageRank, PCA und approximative SVD

Warum lohnt sich der Aufwand aus Abschnitt 8.1? Weil einige der größten Rechenprobleme der Praxis Eigenwertprobleme sind, und zwar solche, bei denen direkte Verfahren keine Chance haben. Drei Beispiele aus Suche, Statistik und Bildverarbeitung.

Google PageRank

Das Problem: eine Rangordnung für $n \approx 10^{10}$ Webseiten. Das Modell dahinter passt in eine Zeile. Jede Seite i bekommt einen Wichtigkeits-Score x_i , und wichtig ist, wer von wichtigen Seiten verlinkt wird:

$$x_i = \sum_{j: j \rightarrow i} \frac{x_j}{d_j},$$

wobei $j \rightarrow i$ bedeutet, dass Seite j auf Seite i verlinkt, und d_j die Anzahl der ausgehenden Links von j ist: Jede Seite vererbt ihren Score gleichmäßig an alle Seiten, auf die sie zeigt. In Matrixform, mit $A = \left[\frac{I(j \rightarrow i)}{d_j} \right]_{i,j}$, heißt das

$$\mathbf{x} = A\mathbf{x},$$

ein Eigenwertproblem zum Eigenwert 1.

Bemerkung 8.2.1 (Warum ausgerechnet der Eigenwert 1?)

Hat jede Seite mindestens einen ausgehenden Link, so summiert sich jede Spalte von A zu 1: Eine Seite verteilt ihren ganzen Score und behält nichts zurück. Solche Matrizen heißen *spaltenstochastisch*, und für sie ist 1 der betragsgrößte Eigenwert; der zugehörige Eigenvektor lässt sich nichtnegativ wählen und taugt damit überhaupt erst als Score-Verteilung.

Für die Potenzmethode reicht das noch nicht ganz: Satz 8.1.4 verlangt einen *einsamen* betragsgrößten Eigenwert, und daran hakt es beim rohen Web-Graphen zweimal. Seiten ohne ausgehende Links liefern gar keine Spaltensumme 1, und zerfällt das Netz in mehrere Teile, so gehören zu jedem Teil eigene Eigenvektoren zum Eigenwert 1. Beides lässt sich reparieren. Leere Spalten füllen wir gleichverteilt auf: Wer nirgendwohin verlinkt, gibt seinen Score eben an alle weiter. Und gegen die restlichen Eigenwerte vom Betrag 1 hilft ein *Dämpfungsfaktor* α mit $0 < \alpha < 1$, mit dem wir zu $\alpha A + \frac{1-\alpha}{n} \mathbf{E}$ übergehen, wobei $\mathbf{E}$ nur Einsen enthält. Der Surfer springt damit mit Wahrscheinlichkeit $1 - \alpha$ auf eine zufällige Seite statt einem Link zu folgen. Die neue Matrix ist strikt positiv, ihr Eigenwert 1 ist einfach, und alle übrigen Eigenwerte liegen betragsmäßig bei höchstens α . Erst damit ist der PageRank-Vektor der dominante Eigenvektor, nach dem die Potenzmethode sucht, und die Rate aus Satz 8.1.4 ist durch α gedeckelt.

Vertiefung: Woher der Eigenwert 1 kommt

Beide Aussagen folgen aus zwei Beobachtungen, die wir schon kennen. Erstens hat A^T lauter Zeilen-summen 1 und hält damit den Vektor aus lauter Einsen fest; Transponieren ändert das charakteristische Polynom nicht, also ist 1 auch Eigenwert von A . Zweitens liegt kein Eigenwert betragsmäßig darüber, denn die Spaltensummennorm ist $\|A\|_1 = 1$, und der Spektralradius bleibt unter jeder submultiplikativen Norm (Abschnitt 3.5). Dass sich der zugehörige Eigenvektor nichtnegativ wählen lässt, liefert die Perron-Frobenius-Theorie für nichtnegative Matrizen.

Und warum iterativ? Direkte Verfahren an einer Matrix mit 10^{10} Zeilen sind aussichtslos. Aber A ist extrem dünn besetzt: Eine typische Seite hat nur etwa 10 ausgehende Links, der Speicherbedarf ist also $O(n)$ statt $O(n^2)$. Die Potenz-Iteration $\mathbf{x}^{(k+1)} = A\mathbf{x}^{(k)} / \|A\mathbf{x}^{(k)}\|$ braucht pro Schritt nur ein Matrix-Vektor-Produkt mit dieser dünnen Matrix, also $O(n)$ Operationen, und der Web-Graph ist so strukturiert, dass rund 50 Iterationen reichen. Zum klassischen Dämpfungsfaktor $\alpha = 0,85$ aus Bemerkung 8.2.1 passt diese Zahl gut, denn $0,85^{50} \approx 3 \cdot 10^{-4}$. Insgesamt $O(n)$ statt $O(n^3)$: Dieser Unterschied macht eine Websuche überhaupt erst möglich.

Interaktiv: PageRank auf einem Vier-Seiten-Netz

Welche Seite erhält langfristig den größten Score? Verfolgen wir dafür die Potenziteration auf dem kleinen Netz.

Interaktives Element — nur in der Web-Fassung

1. Im PageRank-Widget bleibt die Summe der vier Scores bei jedem Schritt gleich.

Antwort: Wahr

Die Linkmatrix ist spaltenstochastisch. Daher erhält die Iteration die Summe.

Hauptkomponentenanalyse (PCA)

Auch die Datenanalyse rechnet ständig Eigenwerte. Gegeben ist eine mittelwert-zentrierte Datenmatrix $X \in \mathbb{R}^{n \times p}$ (n Beobachtungen, p Variablen); gesucht sind die Richtungen maximaler Varianz, um die Daten auf wenige Dimensionen zu verdichten. Die Antwort steckt in der Kovarianzmatrix:

$$\Sigma = \frac{1}{n-1} X^T X \in \mathbb{R}^{p \times p}.$$

Ihre Eigenvektoren sind die *Hauptrichtungen* oder *Loading-Vektoren* in $\mathbb{R}^p$; die zugehörigen Eigenwerte geben an, wie viel Varianz jede Richtung erklärt. Die eigentlichen Hauptkomponenten-Scores der Beobachtungen sind Xv_j . In der Praxis rechnen wir noch schneller direkt mit der Singularwertzerlegung von X , ohne $X^T X$ je zu bilden (warum das auch numerisch klüger ist, haben wir in Kapitel 6 gesehen).

Interaktiv: PCA: Welche Richtung erklärt am meisten?

Die Kovarianzmatrix gibt eine Formel vor. Aber wie sieht die dazugehörige Richtung in einer Punktwolke aus? Drehen wir eine Projektion durch die Daten.

Interaktives Element — nur in der Web-Fassung

1. Stellen wir im PCA-Widget die blaue Richtung auf die grün gestrichelte Achse: Wie groß ist die dort angezeigte maximale Projektionsvarianz ungefähr?

Lösung: 4.43

Die Anzeige erreicht dort den ersten Eigenwert von Σ , also ungefähr 4,43. In jeder anderen Richtung ist $\mathbf{v}^\top \Sigma \mathbf{v}$ kleiner.

Vertiefung: Lanczos für sehr große PCA-Probleme

Bemerkung 8.2.2 (Warum iterativ? Die Lanczos-Abkürzung)

Moderne Anwendungen haben $p = 10^4$ bis 10^6 Variablen (Gen-Expression, Bildanalyse), gebraucht werden aber fast immer nur die k größten Eigenwerte mit $k \ll p$. Die Lanczos-Iteration kann genau diese approximieren. Arbeitet sie direkt mit X , kostet ein Matrix-Vektor-Paar $O(\text{nnz}(X))$; hinzu kommen Orthogonalisierung und eine von Spektrallücken und Toleranz abhängige Iterationszahl. Ein pauschaler Faktor p/k folgt daraus nicht. Warum wir Σ besser gar nicht erst aufstellen, zeigt dennoch ihre Größe: $100\,000^2 = 10^{10}$ Einträge sind in doppelter Genauigkeit rund 80 Gigabyte. In R übernimmt das zum Beispiel `irlba::irlba()` (iterativ) oder `irlba::svdr()` (iterativ und probabilistisch, ein Vorgeschmack auf Abschnitt 8.4).

Approximative SVD in der Praxis

Wie groß der Unterschied ist, zeigt ein Experiment: eine Rang- k -Approximation an einem Graustufen-Foto (2500×3300 Pixel), einmal mit der vollen SVD und zweimal iterativ mit `irlba`:

```
library(irlba); suppressMessages(library(magick))
# "img" ist 2500 x 3300 px (grayscale)
img <- paste0(
  "https://upload.wikimedia.org/wikipedia/commons/7/7e/",
  "Bolzano_City_Image_-_Photo_by_Giovanni_Ussi_-_In_Black_and_White_19.jpg")
X <- (image_read(img) |>
  image_data(channels = "gray") |>
  as.numeric())[,1]
r <- 50
microbenchmark::microbenchmark(
  svd_full = svd(X),
  svd_irlba = irlba::irlba(X, nv = r, nu = r),
  svd_random = irlba::svdr(X, k = r),
  times = 5)
```

Die vollständige SVD berechnet alle 2500 Singulärwerte, obwohl nur 50 gebraucht werden. `irlba` und `svdr` tasten die Matrix stattdessen nur über Matrix-Vektor-Produkte ab und arbeiten in einem Unterraum der Dimension 50; wie viel Rechenzeit das spart, misst der Vergleich oben. Und die Qualität? Dafür stellen wir die drei Rang-50-Rekonstruktionen gegenüber:

```
r <- 50
X_r <- with(svd(X, nu = r, nv = r), u %%% diag(d[1:r]) %%% t(v))
X_irlba <- with(irlba::irlba(X, nu = r, nv = r), u %%% diag(d) %%% t(v))
X_svdr <- with(irlba::svdr(X, k = r), u %%% diag(d) %%% t(v))
norm(X - X_r, "F") / norm(X, "F")
c(norm(X_r - X_irlba, "F"), norm(X_r - X_svdr, "F")) / norm(X_r, "F")
```

Die beiden Zeilen messen Verschiedenes. Die erste ist der relative Frobenius-Fehler der exakten Rang-50-Approximation, also genau die Größe, die der Satz von Eckart und Young minimiert. Die zweite hält die beiden iterativen Rekonstruktionen gegen eben diese exakte Lösung. Fällt sie deut-

lich kleiner aus als die erste, kostet die Abkürzung nichts, was neben dem Approximationsfehler ins Gewicht fiele. Genau das ist zu erwarten: `irlba` rechnet dieselbe Rang-50-Zerlegung bis auf eine einstellbare Toleranz aus, während `svdr` bewusst etwas Genauigkeit gegen Tempo tauscht. Zum Selbst-Ausprobieren: das Rang-k-Widget in Kapitel 6 und die Shiny-App zur Bildkompression.

Die Verfahren im Vergleich

Methode	Komplexität / Iteration	Anwendung	Output
Potenzmethode	$O(\text{nnz}(\mathbf{A}))$	größter Eigenwert/-vektor	1 Eigenpaar
(shifted) QR	einmalig $O(n^3)$; danach $O(n^2)$ je Hessenberg-Sweep	alle Eigenwerte	n Eigenwerte
Lanczos	$O(m \text{nnz}(\mathbf{A}) + nm^2)$ für m Schritte mit voller Reorthogonalisierung	wenige Extremal-Eigenwerte, <i>sparse</i> $\mathbf{A}$	bis zu $m \ll n$ Eigenpaare

Dabei zählt $\text{nnz}(\cdot)$ die Nicht-Null-Einträge. Zur Einordnung: Die Potenzmethode konvergiert mit der Rate $|\lambda_2/\lambda_1|$, die QR-Iteration (mit Shifts) oft quadratisch und am stabilsten. Als Faustregeln: Für *sparse* Matrizen ist Lanczos klar überlegen; wer alle Eigenwerte braucht und Speicher übrig hat, nimmt die (shifted) QR-Iteration; für wenige Eigenpaare reichen Potenzmethode (eines) oder Lanczos ($m \ll n$).

Vertiefung: Heath §4.5 (iterative Eigenwertverfahren: Potenzmethode, QR-Iteration, Lanczos); zur PCA-über-SVD-Verbindung noch einmal Abschnitt 6.4.

8.3 Iterative Löser für lineare Gleichungssysteme

Für lineare Gleichungssysteme $\mathbf{Ax} = \mathbf{b}$ und für Kleinste-Quadrate-Probleme kennen wir bisher nur *direkte* Verfahren. Die LU-Zerlegung aus Abschnitt 5.3 und die QR-Zerlegung aus Abschnitt 7.4 zerlegen die Matrix einmal vollständig und liefern danach die Lösung bis auf Rundungsfehler exakt. Der Preis steht dabei von vornherein fest, nämlich eine Operationszahl der Größenordnung n^3 , ganz unabhängig davon, wie genau wir die Lösung überhaupt brauchen.

Iterative Verfahren drehen dieses Verhältnis um. Sie erzeugen eine Folge von Näherungen $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$, die gegen die Lösung $\mathbf{x}$ strebt, und wir hören auf, sobald uns die erreichte Genauigkeit genügt. Fast alle folgen dabei demselben Bauplan, einer *Fixpunktiteration* (fixed point iteration): Eine Vorschrift wird immer wieder auf ihr eigenes Ergebnis angewendet, und die gesuchte Lösung ist gerade der Punkt, den diese Vorschrift nicht mehr bewegt. Bei großen oder dünnbesetzten Matrizen lohnt sich dieser Handel fast immer: etwas Genauigkeit gegen viel Rechenzeit.

Residuum und Korrekturschritt

Wie nähern wir uns der Lösung überhaupt schrittweise an? Dazu brauchen wir zuerst ein Maß dafür, wie gut eine Näherung ist. Die Lösung $\mathbf{x}$ kennen wir ja nicht, sonst wären wir fertig; einsetzen können wir aber immer.

Definition 8.3.1 (Residuum)

Sei $A \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ und $\mathbf{x}^{(k)} \in \mathbb{R}^n$ eine Näherung der Lösung von $A\mathbf{x} = \mathbf{b}$. Der Vektor

$$\mathbf{r}^{(k)} = \mathbf{b} - A\mathbf{x}^{(k)}$$

heißt *Residuum* (residual) von $\mathbf{x}^{(k)}$.

Das Residuum misst, wie weit die Näherung davon entfernt ist, die Gleichung zu erfüllen. Ist $\mathbf{r}^{(k)} = \mathbf{0}$, so gilt $A\mathbf{x}^{(k)} = \mathbf{b}$, und wir haben die Lösung gefunden. Solange das Residuum nicht verschwindet, korrigieren wir unsere Schätzung:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \underbrace{C\mathbf{r}^{(k)}}_{\text{Korrekturschritt}}.$$

Die Matrix $C \in \mathbb{R}^{n \times n}$ entscheidet, wie stark und in welche Richtung wir korrigieren. Sie ist der einzige Entwurfsspielraum des Verfahrens, und wir werden ihn weiter unten ausnutzen.

Bemerkung 8.3.2 (Verwandtschaft mit dem Gradientenabstieg)

Für den einfachsten Fall $C = \gamma I_n$ lautet der Schritt $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \gamma \mathbf{r}^{(k)}$, wir gehen also ein Stück in Richtung des Residuums. Das ist wörtlich ein Gradientenabstieg mit Schrittweite γ : Ist A symmetrisch und positiv definit, so hat die quadratische Form

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top A \mathbf{x} - \mathbf{b}^\top \mathbf{x}$$

den Gradienten $\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b} = -\mathbf{r}$, und ihr Minimum liegt genau bei $A\mathbf{x} = \mathbf{b}$. Das Residuum zeigt also in Richtung des steilsten Abstiegs, und γ ist die Schrittweite. Wie beim Gradientenabstieg gilt: zu kleine Schritte kriechen, zu große schießen über das Ziel hinaus.

Warum sollten wir uns überhaupt mit Näherungen zufriedengeben, wo doch Kapitel 5 ein exaktes Verfahren bereithält? Weil ein Iterationsschritt viel billiger ist als eine ganze Zerlegung. Für sehr große und für dünnbesetzte Systeme, wie sie in der Statistik etwa bei Netzwerkdaten oder Glättungsproblemen auftreten, sind iterative Verfahren deshalb oft um Größenordnungen schneller.

Die Iteration und ihr Konvergenzsatz

Halten wir die Vorschrift fest, diesmal mit Laufindex.

Definition 8.3.3 (Korrekturiteration)

Sei $A \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ und $\mathbf{x}$ die Lösung von $A\mathbf{x} = \mathbf{b}$. Für einen beliebigen Startwert $\mathbf{x}^{(0)} \in \mathbb{R}^n$ und eine Matrix $C \in \mathbb{R}^{n \times n}$ heißt

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + C(\mathbf{b} - A\mathbf{x}^{(k-1)}), \quad k = 1, 2, \dots \quad (8.3.1)$$

die von C erzeugte *Korrekturiteration*.

Multiplizieren wir die Klammer aus, so zeigt sich die Fixpunktstruktur.

Bemerkung 8.3.4 (Fixpunktform)

Mit $B = I_n - CA$ ist (8.3.1) gleichwertig zu

$$\mathbf{x}^{(k)} = \mathbf{B} \mathbf{x}^{(k-1)} + \mathbf{C} \mathbf{b}, \quad k = 1, 2, \dots \quad (8.3.2)$$

Jeder Schritt besteht also aus einer festen affinen Abbildung, angewandt auf das Ergebnis des vorigen Schritts. Die Matrix $\mathbf{B}$ heißt *Iterationsmatrix*; sie allein entscheidet, ob die Folge zusammenläuft.

Und tatsächlich braucht es nur eine Bedingung an $\mathbf{B}$, damit alles funktioniert.

Satz 8.3.5 (Konvergenz der Korrekturiteration)

Sei $\mathbf{x}$ die eindeutige Lösung von $\mathbf{A} \mathbf{x} = \mathbf{b}$. Gilt für die Spektralnorm

$$\rho := \|\mathbf{I}_n - \mathbf{C} \mathbf{A}\|_2 < 1,$$

so folgt für alle $k \geq 1$

$$\|\mathbf{x}^{(k)} - \mathbf{x}\|_2 \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}\|_2. \quad (8.3.3)$$

Insbesondere ist $\lim_{k \rightarrow \infty} \mathbf{x}^{(k)} = \mathbf{x}$, und zwar für jeden Startwert.

Zur Notation: Das ρ hier ist die *Konvergenzrate* der Iteration; mit dem Rayleigh-Quotienten $\rho^{(k)}$ aus Abschnitt 8.1 hat es nur den Buchstaben gemeinsam.

Vertiefung: Beweis des Konvergenzsatzes

Beweis.

- (1) Wir schreiben $\mathbf{B} = \mathbf{I}_n - \mathbf{C} \mathbf{A}$, also $\|\mathbf{B}\|_2 = \rho$, und bringen die Iteration nach (8.3.2) in die Form

$$\mathbf{x}^{(k)} = \mathbf{B} \mathbf{x}^{(k-1)} + \mathbf{C} \mathbf{b}, \quad k = 1, 2, \dots$$

Ausmultiplizieren von (8.3.1): $\mathbf{x} + \mathbf{C}(\mathbf{b} - \mathbf{A} \mathbf{x}) = (\mathbf{I}_n - \mathbf{C} \mathbf{A}) \mathbf{x} + \mathbf{C} \mathbf{b}$

- (2) Die Lösung $\mathbf{x}$ ist ein *Fixpunkt* dieser Abbildung:

$$\mathbf{B} \mathbf{x} + \mathbf{C} \mathbf{b} = (\mathbf{I}_n - \mathbf{C} \mathbf{A}) \mathbf{x} + \mathbf{C} \mathbf{b} = \mathbf{x} - \underbrace{\mathbf{C}(\mathbf{A} \mathbf{x} - \mathbf{b})}_{=0} = \mathbf{x}.$$

$\mathbf{x}$ löst $\mathbf{A} \mathbf{x} = \mathbf{b}$, die Klammer ist also der Nullvektor

- (3) Damit lässt sich der Fehler nach einem Schritt abschätzen. Wir ziehen die Fixpunktgleichung von der Iterationsvorschrift ab:

$$\begin{aligned} \|\mathbf{x}^{(k)} - \mathbf{x}\|_2 &= \|\mathbf{B} \mathbf{x}^{(k-1)} + \mathbf{C} \mathbf{b} - \mathbf{x}\|_2 \\ &= \|\mathbf{B} \mathbf{x}^{(k-1)} + \mathbf{C} \mathbf{b} - (\mathbf{B} \mathbf{x} + \mathbf{C} \mathbf{b})\|_2 \\ &= \|\mathbf{B}(\mathbf{x}^{(k-1)} - \mathbf{x})\|_2 \\ &\leq \rho \|\mathbf{x}^{(k-1)} - \mathbf{x}\|_2. \end{aligned}$$

in Zeile 2 ersetzen wir $\mathbf{x}$ durch $\mathbf{B} \mathbf{x} + \mathbf{C} \mathbf{b}$ (Schritt 2), die Terme $\mathbf{C} \mathbf{b}$ heben sich weg; in der letzten Zeile nutzen wir die definierende Eigenschaft der Operatornorm, $\|\mathbf{B} \mathbf{v}\|_2 \leq \|\mathbf{B}\|_2 \|\mathbf{v}\|_2$

- (4) Jeder Schritt drückt den Fehler also auf höchstens das ρ -fache. Iterieren wir die Ungleichung, so erhalten wir

$$\|\mathbf{x}^{(k)} - \mathbf{x}\|_2 \leq \rho \|\mathbf{x}^{(k-1)} - \mathbf{x}\|_2 \leq \rho^2 \|\mathbf{x}^{(k-2)} - \mathbf{x}\|_2 \leq \dots \leq \rho^k \|\mathbf{x}^{(0)} - \mathbf{x}\|_2,$$

und wegen $\rho < 1$ geht die rechte Seite gegen null.

::why[formal eine Induktion über k : Schritt 3 liefert den Induktionsschritt, der Fall $k = 1$ ist genau Schritt 3 selbst; $\rho^k \rightarrow 0$ gilt für jedes $\rho \in [0, 1]$]

□

Die Schranke (8.3.3) beschreibt lineare Konvergenz: Der Fehler schrumpft pro Schritt auf einen festen Bruchteil, nicht um eine feste Differenz. Auf logarithmischer Skala aufgetragen bleibt er damit unterhalb einer Geraden mit Steigung $\log \rho$.

Die Voraussetzung des Satzes ist übrigens stärker, als sie aussieht. Wäre A singulär, so gäbe es ein $\mathbf{v} \neq \mathbf{0}$ mit $A\mathbf{v} = \mathbf{0}$, also $(I_n - CA)\mathbf{v} = \mathbf{v}$ und damit $\rho \geq 1$. Aus $\rho < 1$ folgt somit von selbst, dass A invertierbar ist und die Lösung eindeutig.

Selbsttest: Was kostet ein Schritt?

Bevor wir über die Wahl von C nachdenken, zwei Fragen zum Aufwand. Zur Erinnerung an die Landau-Notation siehe Abschnitt 2.4.

1. Mit $C = \gamma I_n$ kostet ein Iterationsschritt $O(1)$ Operationen.

Antwort: Falsch

Schon das Ergebnis hat n Komponenten, und jede muss geschrieben werden. Unter $O(n)$ geht es also gar nicht.

2. Mit $C = \gamma I_n$ kostet ein Iterationsschritt $O(n)$ Operationen.

Antwort: Falsch

Das gilt nur für die billigen Teile. Der Schritt lautet ausgeschrieben $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \gamma(A\mathbf{x}^{(k-1)} + \gamma\mathbf{b})$: Das Skalieren mit γ und die beiden Vektoradditionen kosten $O(n)$, das Matrix-Vektor-Produkt $A\mathbf{x}^{(k-1)}$ aber mehr.

3. Mit $C = \gamma I_n$ kostet ein Iterationsschritt $O(n^2)$ Operationen.

Antwort: Wahr

Der teuerste Posten ist das Matrix-Vektor-Produkt $A\mathbf{x}^{(k-1)}$: n Komponenten mit je n Multiplikationen und $n - 1$ Additionen, zusammen $O(n^2)$. Alles Übrige ist $O(n)$. Ist A dünnbesetzt, sinkt der Aufwand sogar auf $O(\text{nnz}(A))$, wobei nnz die Zahl der Einträge ungleich null zählt.

4. Mit $C = \gamma I_n$ kostet ein Iterationsschritt $O(n^3)$ Operationen.

Antwort: Falsch

So teuer würde es nur, wenn wir die Iterationsmatrix $B = I_n - \gamma A$ in jedem Schritt neu als Matrixprodukt aufbauten. Das tut niemand: B wird gar nicht gebildet, wir wenden A direkt auf den aktuellen Vektor an. Wäre ein Schritt so teuer wie eine LU-Zerlegung, hätte das ganze Verfahren keinen Zweck.

Wie viele Schritte bis zur Genauigkeit ε ?

Satz 8.3.5 verspricht einen Fehler, der geometrisch fällt. Übersetzen wir das in die Frage, die uns beim Rechnen wirklich beschäftigt: Wie oft müssen wir iterieren, bis eine vorgegebene Toleranz ε unterschritten ist?

1. Für $\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \varepsilon$ genügen $k = O(1)$ Schritte.

Antwort: Falsch

Eine feste Schrittzahl liefert eine feste Genauigkeit, mehr nicht. Wer ε verkleinert, muss weiter iterieren.

2. Für $\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \varepsilon$ genügen $k = O(\log(1/\varepsilon))$ Schritte.

Antwort: Wahr

Der Fehler fällt geometrisch, also braucht jede weitere Dezimalstelle Genauigkeit gleich viele zusätzliche Schritte. Die Herleitung steht direkt im Anschluss als Korollar 8.3.6.

3. Für $\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \varepsilon$ brauchen wir $k = O(1/\varepsilon)$ Schritte.

Antwort: Falsch

Das wäre die Schrittzahl bei einem Fehler, der nur wie $1/k$ fällt. Hier fällt er wie ρ^k , und das ist ungleich schneller: Für $\varepsilon = 10^{-6}$ stünden hier eine Million Schritte statt der 15, die für das System aus Beispiel 8.3.11 genügen.

4. Für $\|\mathbf{x}^{(k)} - \mathbf{x}\| \leq \varepsilon$ brauchen wir $k = O(1/\varepsilon^k)$ Schritte.

Antwort: Falsch

Diese Antwort kann schon formal nicht stimmen, denn das gesuchte k steht auf beiden Seiten. Eine Schranke für k darf nur von ε , von ρ und vom Startfehler abhängen.

Rechnen wir die Antwort nach. Sie folgt allein aus der Schranke (8.3.3).

Korollar 8.3.6 (Zahl der Iterationen)

Es gelte (8.3.3) mit $\rho \in (0, 1)$, und sei $e_0 = \|\mathbf{x}^{(0)} - \mathbf{x}\|_2 > 0$ der Startfehler. Für $\varepsilon > 0$ ist $\|\mathbf{x}^{(k)} - \mathbf{x}\|_2 \leq \varepsilon$ garantiert, sobald

$$k \geq \max \left\{ 0, \left\lceil \frac{\log(e_0/\varepsilon)}{-\log \rho} \right\rceil \right\}. \quad (8.3.4)$$

Insbesondere ist $k = O(\log(1/\varepsilon))$.

Vertiefung: Herleitung der exakten Iterationszahl

Beweis.

- (1) Nach (8.3.3) genügt es, $\rho^k e_0 \leq \varepsilon$ zu erzwingen, also

$$\rho^k \leq \frac{\varepsilon}{e_0}.$$

(8.3.3) schätzt den Fehler nach oben ab; wer die obere Schranke unter ε drückt, drückt den Fehler mit

- (2) Beide Seiten sind positiv, wir dürfen also logarithmieren. Wegen $0 < \rho < 1$ ist $\log \rho < 0$, und beim Teilen durch diese negative Zahl dreht sich das Ungleichheitszeichen um:

$$k \log \rho \leq \log \frac{\varepsilon}{e_0} \quad \Longleftrightarrow \quad k \geq \frac{\log(\varepsilon/e_0)}{\log \rho} = \frac{\log(e_0/\varepsilon)}{-\log \rho}.$$

$\log$ ist streng monoton wachsend, und $\log(\varepsilon/e_0) = -\log(e_0/\varepsilon)$

- (3) Falls $\varepsilon \geq e_0$, genügt bereits $k = 0$; andernfalls ist das kleinste ganzzahlige k die Aufrundung. Beides zusammen ergibt (8.3.4). Wegen $\log(e_0/\varepsilon) = \log e_0 + \log(1/\varepsilon)$ hängt die Schrittzahl von

der geforderten Genauigkeit nur über $\log(1/\varepsilon)$ ab; der Startfehler und der Faktor $1/(-\log \rho)$ sind Konstanten. Damit ist $k = O(\log(1/\varepsilon))$.

in der Landau-Notation zählt allein das Wachstum in ε ; e_0 und ρ hängen nicht von ε ab

□

Wie freundlich diese Schranke ist, zeigen ein paar Zahlen. Für das Beispiel weiter unten ist $\rho \approx 0,405$ und $e_0 \approx 0,643$; (8.3.4) verlangt dann 8 Schritte für $\varepsilon = 10^{-3}$, 15 Schritte für $\varepsilon = 10^{-6}$ und 23 Schritte für $\varepsilon = 10^{-9}$. Drei zusätzliche Dezimalstellen kosten also jedes Mal rund sieben bis acht weitere Schritte, und jeder davon kostet $O(n^2)$.

Die Wahl von C

Bemerkung 8.3.7 (Zwei Anforderungen an C)

Die Matrix C soll gleichzeitig zwei Dinge leisten:

1. $\rho = \|\mathbf{I}_n - \mathbf{C}\mathbf{A}\|_2 < 1$ soll möglichst klein sein, damit wenige Schritte reichen. Optimal wäre $\mathbf{C} \approx \mathbf{A}^{-1}$.
2. Der Schritt (8.3.2) soll billig auszuwerten sein, also möglichst in $O(n^2)$.

Die beiden Wünsche ziehen in verschiedene Richtungen. Mit $\mathbf{C} = \mathbf{A}^{-1}$ wäre $\rho = 0$, und ein einziger Schritt träfe die Lösung exakt. Nur müssten wir dafür $\mathbf{A}^{-1}$ kennen, und das ist teurer als das Gleichungssystem selbst. Brauchbare Wahlen sind deshalb grobe, billige Näherungen der Inversen.

Algorithmus 8.3.8 (Drei klassische Wahlen von C)

Für eine dichte Matrix kosten alle drei pro Schritt $O(n^2)$. Die folgenden Angaben definieren zunächst nur die Iterationen; Konvergenz braucht jeweils zusätzliche Voraussetzungen:

1. *Richardson-Iteration*: $\mathbf{C} = \gamma \mathbf{I}_n$. Ist $\mathbf{A}$ SPD, so garantiert $0 < \gamma < 2/\lambda_{\max}(\mathbf{A})$ Konvergenz. Für eine beliebige Matrix existiert nicht notwendig ein geeignetes positives γ .
2. *Jacobi-Iteration*: $\mathbf{C} = \text{diag}(\mathbf{A}_{11}, \dots, \mathbf{A}_{nn})^{-1}$, die Inverse der Diagonalen von $\mathbf{A}$. Dafür müssen alle Diagonaleinträge von null verschieden sein. Die Korrektur teilt jede Residuumskomponente durch das zugehörige Diagonalelement, zusätzlicher Aufwand n Divisionen. Konvergenz gilt, wenn der Spektralradius der Iterationsmatrix kleiner als 1 ist; strikte Diagonaldominanz ist eine verbreitete hinreichende Bedingung.
3. *Gauss-Seidel-Iteration*: $\mathbf{C}^{-1}$ ist das untere Dreieck von $\mathbf{A}$, also die Diagonale samt allem darunter. Invertiert wird nichts: Die Korrektur $\mathbf{d} = \mathbf{C} \mathbf{r}^{(k)}$ ist die Lösung des Dreieckssystems $\mathbf{C}^{-1} \mathbf{d} = \mathbf{r}^{(k)}$, und die liefert die Vorwärtssubstitution in $O(n^2)$ Operationen. Auch hier müssen die Diagonaleinträge ungleich null sein. Konvergenz ist eine zusätzliche Eigenschaft; für SPD-Matrizen ist sie garantiert.

Alle drei nehmen einen leicht zugänglichen Teil von $\mathbf{A}$ und tun so, als wäre er die ganze Matrix. Wie gut das gelingt, hängt von $\mathbf{A}$ ab.

Beispiel 8.3.9 (Die drei Wahlen an einem 2×2 -System)

Für

$$A = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}$$

liefern die drei Rezepte die Iterationsmatrizen

$$\text{Richardson } (\gamma = 0,25) : \quad I - \gamma A = \begin{pmatrix} 0 & -0,25 \\ -0,25 & 0,25 \end{pmatrix}, \quad \rho \approx 0,405,$$

$$\text{Jacobi: } \quad I - CA = \begin{pmatrix} 0 & -\frac{1}{4} \\ -\frac{1}{3} & 0 \end{pmatrix}, \quad \rho = \frac{1}{3} \approx 0,333,$$

$$\text{Gauss-Seidel: } \quad I - CA = \begin{pmatrix} 0 & -\frac{1}{4} \\ 0 & \frac{1}{12} \end{pmatrix}, \quad \rho \approx 0,264.$$

Alle drei konvergieren, und in dieser Reihenfolge immer schneller. Hier zählt sich aus, dass Jacobi mehr über A weiß als Richardson und Gauss-Seidel mehr als Jacobi. Als Regel taugt die Reihenfolge trotzdem nicht: Es gibt Matrizen, für die Jacobi konvergiert und Gauss-Seidel nicht, und ebenso umgekehrt.

Ein Detail zum Mitnehmen: ρ ist die *Norm* der Iterationsmatrix, und die ist nur ein hinreichendes Kriterium. Die Gauss-Seidel-Matrix hier hat die Eigenwerte 0 und $\frac{1}{12}$, ihr Spektralradius ist also 0,083, und mit diesem Faktor schrumpft der Fehler auf lange Sicht. Die Norm 0,264 ist die Garantie, die schon ab dem ersten Schritt gilt.

Bemerkung 8.3.10 (Gesamtaufwand)

Sei c_{Schritt} der Aufwand einer Iteration und $\rho = \|I - CA\|_2 < 1$. Dann liefert Korollar 8.3.6 den Aufwand

$$O\left(c_{\text{Schritt}} \max\left\{1, \frac{\log(e_0/\varepsilon)}{-\log \rho}\right\}\right).$$

Nur wenn $\rho \leq \rho_0 < 1$ unabhängig von der Problemgröße bleibt, vereinfacht sich das bei dichter A zu $O(n^2 \log(e_0/\varepsilon))$. In Problemfolgen nähert sich ρ häufig der 1; dann wächst die Iterationszahl mit n und der Kondition. Deshalb spart ein iteratives Verfahren nicht automatisch einen Faktor n . Präconditionierung soll gerade ρ von 1 fernhalten. Bei dünnbesetztem A kann ein Schritt $O(\text{nnz}(A))$ kosten, während direkte Löser bei Band- oder anderer Struktur ebenfalls deutlich billiger als $O(n^3)$ sein können.

Ein Beispiel Schritt für Schritt

Beispiel 8.3.11 (Richardson-Iteration)

Gegeben seien

$$A = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \text{also} \quad x = A^{-1}b = \frac{1}{11} \begin{pmatrix} 1 \\ 7 \end{pmatrix} \approx \begin{pmatrix} 0,091 \\ 0,636 \end{pmatrix}.$$

Wir iterieren mit $\gamma = 0,25$ und starten in $x^{(0)} = (0, 0)^\top$.

Schritt 1: $r^{(0)} = b - Ax^{(0)} = (1, 2)^\top$, also

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + 0,25 \mathbf{r}^{(0)} = \begin{pmatrix} 0,25 \\ 0,5 \end{pmatrix}.$$

Schritt 2: $\mathbf{Ax}^{(1)} = (1,5, 1,75)^\top$, $\mathbf{r}^{(1)} = (-0,5, 0,25)^\top$, also

$$\mathbf{x}^{(2)} = \begin{pmatrix} 0,125 \\ 0,563 \end{pmatrix}.$$

Schritt 3: $\mathbf{Ax}^{(2)} = (1,063, 1,813)^\top$, $\mathbf{r}^{(2)} = (-0,063, 0,188)^\top$, also

$$\mathbf{x}^{(3)} = \begin{pmatrix} 0,109 \\ 0,609 \end{pmatrix} \quad \left(\text{exakt } \frac{1}{64} (7, 39)^\top \right).$$

Schritt 4: $\mathbf{Ax}^{(3)} = (1,047, 1,938)^\top$, $\mathbf{r}^{(3)} = (-0,047, 0,063)^\top$, also

$$\mathbf{x}^{(4)} = \begin{pmatrix} 0,098 \\ 0,625 \end{pmatrix} \quad \left(\text{exakt } \frac{1}{256} (25, 160)^\top \right).$$

Die Fehler $\|\mathbf{x}^{(k)} - \mathbf{x}\|_2$ lauten

$$0,643 \rightarrow 0,210 \rightarrow 0,081 \rightarrow 0,033 \rightarrow 0,013 \rightarrow \dots$$

Passt das zur Theorie? Die Iterationsmatrix $\mathbf{I} - \gamma \mathbf{A}$ ist symmetrisch, ihre Norm also der größte Eigenwertbetrag. Aus den Eigenwerten $\frac{1}{2}(7 \pm \sqrt{5}) \approx 4,618$ und $2,382$ von $\mathbf{A}$ werden die Eigenwerte $1 - 0,25 \cdot 4,618 = -0,155$ und $1 - 0,25 \cdot 2,382 = 0,405$, also

$$\rho = \|\mathbf{I} - \gamma \mathbf{A}\|_2 \approx 0,405.$$

Der Fehler sollte demnach je Schritt etwa auf das 0,405-fache fallen, das heißt um den Faktor $1/0,405 \approx 2,5$ sinken. Die beobachteten Quotienten 0,33, 0,39, 0,40, 0,40 bestätigen das und nähern sich von unten der Vorhersage an.

Interaktiv: Richardson Schritt für Schritt, mit einstellbarem γ

Bei welcher Schrittweite kippt diese Iteration? Schätzen wir die Grenze, bevor wir den Verlauf untersuchen.

Schätzfrage. Ab welchem γ beginnt diese Richardson-Iteration zu divergieren?

Lösung: 0,4330847293182009 γ

Die Auflösung markiert die Grenze der Konvergenz. Satz 8.3.5 erklärt den Wechsel: Auf der einen Seite fällt der Fehler geometrisch, auf der anderen verfehlt die Iterationsmatrix die Kontraktionsbedingung.

1. Zeigt das Richardson-Widget für eine Schrittweite eine Rate $\rho > 1$, dann erfüllt diese Wahl die Konvergenzvoraussetzung von Satz 8.3.5 nicht.

Antwort: Wahr

Die Kontraktionsschranke verlangt $\rho < 1$.

Einordnung

Bemerkung 8.3.12 (Was iterative Verfahren leisten)

- Iterative Verfahren gibt es nicht nur für lineare Gleichungssysteme und für Eigenwertprobleme (Abschnitt 8.1), sondern ebenso für Kleinste-Quadrate-Probleme (Kapitel 7).
- In Sonderfällen sind sie *exakt*. Mit $C = A^{-1}$ etwa ist $\rho = 0$, und ein Schritt genügt; auch Verfahren vom Krylov-Typ wie das der konjugierten Gradienten erreichen die Lösung in exakter Arithmetik nach höchstens n Schritten.
- Meistens ergibt sich ein Handel zwischen Laufzeit und Genauigkeit, und die Iterationszahl ist der Regler dafür. Wo die Laufzeit wichtiger ist als die letzte Stelle, sind Iterationen die Methode der Wahl.

Ein praktischer Punkt bleibt offen: Den Fehler $\|\mathbf{x}^{(k)} - \mathbf{x}\|$ können wir nicht messen, denn dazu müssten wir $\mathbf{x}$ kennen. Abgebrochen wird deshalb nach dem Residuum, das jederzeit verfügbar ist. Wegen $\mathbf{x}^{(k)} - \mathbf{x} = -A^{-1}\mathbf{r}^{(k)}$ gilt

$$\|\mathbf{x}^{(k)} - \mathbf{x}\|_2 \leq \|A^{-1}\|_2 \|\mathbf{r}^{(k)}\|_2,$$

ein kleines Residuum bürgt also nur bei gutartiger Matrix für einen kleinen Fehler. Das ist dieselbe Warnung wie bei der Konditionszahl in Abschnitt 4.2, diesmal als Abbruchkriterium.

Vertiefung: Heath §10.9 (iterative Verfahren für lineare Gleichungssysteme, inklusive Jacobi, Gauss-Seidel und konjugierten Gradienten); für die Eigenwert-Iterationen aus Abschnitt 8.1 siehe Heath §4.5.

8.4 Probabilistische Methoden: Matrix-Sketching

Die iterativen Verfahren der Abschnitte 8.1 und 8.3 handeln Genauigkeit gegen Laufzeit: Wir brechen ab, sobald uns das Erreichte genügt. Dieser Abschnitt zieht einen anderen Hebel. Wir verkleinern nicht die Zahl der Schritte, sondern das Problem selbst, und das Werkzeug dafür ist Zufall.

Zufall mit Absicht

Oft sind Probleme so groß, dass wir Genauigkeit gern gegen bessere Komplexität tauschen. Zunehmend geschieht das durch *Randomisierung* (randomization): Statt sorgfältig auszuwählen, welchen Teil der Daten wir wegwerfen, würfeln wir ihn aus.

Bemerkung 8.4.1 (Hauptidee der Randomisierung)

Wir ersetzen ein großes Problem durch ein zufällig erzeugtes, kleines Problem, das höchstwahrscheinlich eine ähnliche Lösung hat.

Das Wort „höchstwahrscheinlich“ ist dabei wörtlich gemeint. Ein randomisiertes Verfahren darf danebenliegen, und die Fehlerwahrscheinlichkeit δ ist ein Parameter, den wir einstellen. Wir kaufen uns Rechenzeit, indem wir ein kontrolliertes Risiko eingehen.

Warum sollte das gutgehen? Wer eine Punktwolke im $\mathbb{R}^{10\,000}$ auf ein paar hundert Koordinaten eindampft, wirft weit über 90 % der Koordinaten weg. Dass die Wolke danach noch dieselbe Form

hat, klingt zunächst absurd. Der Grund liegt in einer Eigenart hochdimensionaler Räume, die wir uns zuerst ansehen.

Fast orthogonale Zufallsvektoren

In der Ebene sind zwei zufällige Richtungen selten senkrecht: Der Winkel zwischen ihnen ist gleichverteilt, jeder Wert ist so gut wie jeder andere. Im $\mathbb{R}^n$ mit großem n ändert sich das Bild vollständig.

Satz 8.4.2 (Zufallsrichtungen stehen fast senkrecht aufeinander)

Seien $\mathbf{s}_1, \mathbf{s}_2 \in \mathbb{R}^n$ unabhängig und je $\mathcal{N}(\mathbf{0}, \mathbf{I}_n)$ -verteilt, und sei θ der Winkel zwischen ihnen. Dann gilt

$$\mathbb{E}[\cos \theta] = 0, \quad \text{Var}[\cos \theta] = \frac{1}{n},$$

und für jedes $t > 0$ folgt

$$\mathbb{P}\left(|\cos \theta| \geq \frac{t}{\sqrt{n}}\right) \leq \frac{1}{t^2}.$$

Der Satz sagt: Die Streuung des Kosinus schrumpft wie $1/\sqrt{n}$. In hohen Dimensionen drängt sich die Verteilung des Winkels also um 90° zusammen, zufällige Richtungen sind praktisch orthogonal.

Vertiefung: Woher die Streuung des Kosinus kommt

Beweis.

- (1) Wir normieren beide Vektoren, $\mathbf{u} = \mathbf{s}_1 / \|\mathbf{s}_1\|$ und $\mathbf{v} = \mathbf{s}_2 / \|\mathbf{s}_2\|$, und schreiben den Kosinus als Skalarprodukt

$$\cos \theta = \mathbf{u}^\top \mathbf{v}.$$

der Winkel zwischen zwei Vektoren hängt nur von ihren Richtungen ab; $s_i = 0$ tritt mit Wahrscheinlichkeit 0 ein und stört deshalb nicht

- (2) Die Standardnormalverteilung ist drehinvariant, also ist $\mathbf{u}$ gleichverteilt auf der Einheitskugel. Daraus folgt

$$\mathbb{E}[\mathbf{u}] = \mathbf{0}, \quad \mathbb{E}[\mathbf{u}\mathbf{u}^\top] = \frac{1}{n} \mathbf{I}_n. \quad (8.4.1)$$

$\mathbf{u}$ und $-\mathbf{u}$ sind gleich verteilt, also ist der Erwartungswert null; die Matrix $\mathbb{E}[\mathbf{u}\mathbf{u}^\top]$ ändert sich unter Drehungen nicht und ist deshalb ein Vielfaches der Einheitsmatrix, und der Faktor folgt aus $\text{tr} \mathbb{E}[\mathbf{u}\mathbf{u}^\top] = \mathbb{E}[\|\mathbf{u}\|^2] = 1$

- (3) Jetzt bedingen wir auf $\mathbf{v}$ und nutzen die Unabhängigkeit:

$$\mathbb{E}[\mathbf{u}^\top \mathbf{v} \mid \mathbf{v}] = \mathbb{E}[\mathbf{u}]^\top \mathbf{v} = \mathbf{0}, \quad \mathbb{E}[(\mathbf{u}^\top \mathbf{v})^2 \mid \mathbf{v}] = \mathbf{v}^\top \mathbb{E}[\mathbf{u}\mathbf{u}^\top] \mathbf{v} = \frac{\|\mathbf{v}\|^2}{n} = \frac{1}{n}.$$

Beide Werte hängen nicht mehr von $\mathbf{v}$ ab, gelten also auch unbedingt.

(8.4.1) mit $(\mathbf{u}^\top \mathbf{v})^2 = \mathbf{v}^\top \mathbf{u}\mathbf{u}^\top \mathbf{v}$; $\|\mathbf{v}\| = 1$ nach Konstruktion

- (4) Wegen $\mathbb{E}[\cos \theta] = 0$ ist $\text{Var}[\cos \theta] = \mathbb{E}[\cos^2 \theta] = 1/n$, und die Tschebyscheff-Ungleichung liefert

$$\mathbb{P}\left(|\cos \theta| \geq \frac{t}{\sqrt{n}}\right) \leq \frac{\text{Var}[\cos \theta]}{t^2/n} = \frac{1}{t^2}.$$

Tschebyscheff ist die Markov-Ungleichung, angewandt auf die quadrierte Abweichung vom Erwartungswert □

Beispiel 8.4.3 (Wie senkrecht ist fast senkrecht?)

Für $n = 10\,000$ ist $\text{Var}[\cos \theta] = 10^{-4}$, die Standardabweichung also 0,01. Mit $t = 5$ liefert Satz 8.4.2

$$\mathbb{P}(|\cos \theta| \geq 0,05) \leq \frac{1}{25} = 0,04,$$

mit Wahrscheinlichkeit mindestens 96 % liegt der Winkel also zwischen $87,1^\circ$ und $92,9^\circ$. Zum Vergleich: Für $n = 100$ erlaubt dieselbe Schranke noch $|\cos \theta| \leq 0,5$, also jeden Winkel zwischen 60° und 120° . Die Aussage lebt von großem n .

Damit wird plausibel, was wir eigentlich vorhaben. Ziehen wir m solcher Richtungen und schreiben sie als Zeilen in eine Matrix, so stehen je zwei dieser Zeilen fast senkrecht aufeinander. Ein Orthonormalsystem sind sie deshalb noch lange nicht: Für eine Gauss-Zeile $\mathbf{s}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n/m)$ ist $\mathbb{E}[\|\mathbf{s}_i\|^2] = n/m$, bei $n = 10\,000$ und $m = 50$ also 200. Um die Richtungen kümmert sich die hohe Dimension, um die Längen der Vorfaktor in der Verteilung. Die Abbildung $\mathbf{x} \mapsto \mathbf{S}\mathbf{x}$ misst die Länge von $\mathbf{x}$ dann entlang m zufälliger, fast senkrechter Achsen, und das reicht überraschend weit.

Definition 8.4.4 (Sketching-Matrix und Skizze)

Sei $m < n$. Eine Zufallsmatrix $\mathbf{S} \in \mathbb{R}^{m \times n}$ heißt *Sketching-Matrix* (sketching matrix), und für $\mathbf{x} \in \mathbb{R}^n$ heißt $\mathbf{S}\mathbf{x} \in \mathbb{R}^m$ die *Skizze* (sketch) von $\mathbf{x}$. Gesucht sind Verteilungen für $\mathbf{S}$, unter denen mit hoher Wahrscheinlichkeit

$$\|\mathbf{S}\mathbf{x} - \mathbf{S}\mathbf{y}\| \approx \|\mathbf{x} - \mathbf{y}\| \quad \text{und} \quad \angle(\mathbf{S}\mathbf{x}, \mathbf{S}\mathbf{y}) \approx \angle(\mathbf{x}, \mathbf{y})$$

gilt, die Skizze also Abstände und Winkel der Originaldaten erbt. Wie im ganzen Kapitel ist $\|\cdot\|$ die euklidische Norm.

Häufig heißt $\mathbf{S}$ eine zufällige „Projektion“. Das Wort ist mit Vorsicht zu lesen: Eine Projektion bildet einen Raum in sich ab und ist idempotent, $\mathbf{S}$ dagegen führt vom $\mathbb{R}^n$ in den kleineren $\mathbb{R}^m$. Gemeint ist nur, dass das Bild in m statt in n Dimensionen lebt.

Ein Zahlenbeispiel

Beispiel 8.4.5 (Sketching zweier Vektoren mit 10 000 Komponenten)

Ziehen wir zwei beliebige Vektoren $\mathbf{x}, \mathbf{y} \in \mathbb{R}^{10\,000}$ und eine Gauss-Sketchmatrix mit $m = 50$, also einen Faktor 200 kleiner:

```
set.seed(1312)
## Zwei beliebige n-dimensionale Vektoren
n <- 10000
x <- runif(n)
y <- runif(n)
# (m x n)-Matrix mit m << n und zufälligen Einträgen
m <- 50 # m << n !!
S <- rnorm(m * n, sd = sqrt(1 / m)) |> matrix(nrow = m, ncol = n)
Sx <- S %*% x
Sy <- S %*% y
```

```
## Distanzen ||x - y|| und ||Sx - Sy||
c(norm(x - y, "2"), norm(Sx - Sy, "2"))
## Winkel: cos(angle) = (x'y) / (||x|| ||y||)
angle <- function(x, y) (sum(x * y) / (norm(x, "2") * norm(y, "2"))) |> acos()
c(angle(x, y), angle(Sx, Sy))
```

Die Einträge sind unabhängig $\mathcal{N}(0, 1/m)$ -verteilt, eine Zeile $\mathbf{s}_i$ hat also die Kovarianzstruktur $\mathbb{E}[\mathbf{s}_i \mathbf{s}_i^\top] = \mathbf{I}_n/m$; diese Bedingung brauchen wir gleich. Für diesen Seed weicht der Abstand um rund 3 % ab, der Winkel um rund 1 % – und das, obwohl 99,5 % der Dimensionen wegfallen.

Diese Zahlen sind eine gute Ziehung, und wie gut, lässt sich für eine Gauss-Skizze exakt beziffern. Ist $\mathbf{z} \in \mathbb{R}^n$ fest, so sind die m Komponenten $\mathbf{s}_i^\top \mathbf{z}$ von $\mathbf{S}\mathbf{z}$ unabhängig $\mathcal{N}(0, \|\mathbf{z}\|^2/m)$ -verteilt; damit ist $\|\mathbf{S}\mathbf{z}\|^2 / \|\mathbf{z}\|^2$ genau χ_m^2/m -verteilt. Bei $m = 50$ hat die relative Abweichung des Abstands damit die Standardabweichung 9,97 %, praktisch die Faustregel $1/\sqrt{2m} = 10\%$; nur knapp ein Viertel aller Ziehungen bleibt unter der 3 %-Marke, und nur knapp 8 % landen unter 1 %. Das Verfahren funktioniert, aber einzelne Ziehungen streuen erheblich. Das Widget unten führt beides vor.

Interaktiv: Sketching mit verstellbarem m

Wie weit eine einzelne Ziehung danebenliegen kann, führt das Widget vor. Verändern wir die Zeilenzahl und vergleichen wir mehrere Ziehungen.

Interaktives Element — nur in der Web-Fassung

1. Wie groß ist im Sketching-Widget die eingeblendete Faustregel in Prozent bei $m = 50$?

Lösung: 10

Nach dem Einblenden zeigt das orange Band $1/\sqrt{2m} = 10\%$.

Der Einbettungssatz

Bisher haben wir plausibel gemacht, dass Sketching funktionieren kann. Jetzt beweisen wir es, und zwar mit erstaunlich wenig Aufwand: Erwartungswert, Varianz und die Tschebyscheff-Ungleichung genügen.

Satz 8.4.6 (Zufällige Einbettung eines festen Vektors)

Sei $\mathbf{S} \in \mathbb{R}^{m \times n}$ eine Zufallsmatrix, deren Zeilen $\mathbf{s}_1^\top, \dots, \mathbf{s}_m^\top$ unabhängig und identisch verteilt sind und

$$\mathbb{E}[\mathbf{s}_i \mathbf{s}_i^\top] = \frac{1}{m} \mathbf{I}_n \quad (8.4.2)$$

erfüllen, sowie für ein $K > 0$

$$\mathbb{E}[(\mathbf{s}_i^\top \mathbf{v})^4] \leq \frac{K}{m^2} \quad \text{für alle } \mathbf{v} \in \mathbb{R}^n \text{ mit } \|\mathbf{v}\| = 1. \quad (8.4.3)$$

Seien weiter $\mathbf{x} \in \mathbb{R}^n$ fest und $\delta > 0$. Dann gilt mit Wahrscheinlichkeit mindestens $1 - \delta$

$$(1 - \varepsilon) \|\mathbf{x}\|^2 \leq \|\mathbf{S}\mathbf{x}\|^2 \leq (1 + \varepsilon) \|\mathbf{x}\|^2, \quad \varepsilon = \sqrt{\frac{K}{\delta m}}. \quad (8.4.4)$$

Vertiefung: Beweis des Einbettungssatzes

Beweis.

- (1) Wir dürfen $\|\mathbf{x}\| = 1$ annehmen. Beide Seiten von (8.4.4) sind homogen vom Grad 2 in $\mathbf{x}$; für $\mathbf{x} \neq \mathbf{0}$ teilen wir die Ungleichung durch $\|\mathbf{x}\|^2$, und für $\mathbf{x} = \mathbf{0}$ ist nichts zu zeigen. Zu beweisen bleibt

$$\mathbb{P} \left(\left| \|\mathbf{S}\mathbf{x}\|^2 - 1 \right| \leq \varepsilon \right) \geq 1 - \delta.$$

mit $\mathbf{x}$ erfüllt auch $\mathbf{x}/\|\mathbf{x}\|$ die Voraussetzungen, und $\|\mathbf{S}(\mathbf{x}/\|\mathbf{x}\|)\|^2 = \|\mathbf{S}\mathbf{x}\|^2 / \|\mathbf{x}\|^2$

- (2) Die i -te Komponente von $\mathbf{S}\mathbf{x}$ ist $\mathbf{s}_i^\top \mathbf{x}$, also

$$\|\mathbf{S}\mathbf{x}\|^2 = \sum_{i=1}^m (\mathbf{s}_i^\top \mathbf{x})^2 \quad \text{und} \quad \mathbb{E} [\|\mathbf{S}\mathbf{x}\|^2] = \sum_{i=1}^m \mathbf{x}^\top \mathbb{E} [\mathbf{s}_i \mathbf{s}_i^\top] \mathbf{x} = m \mathbf{x}^\top \frac{\mathbf{I}_n}{m} \mathbf{x} = \|\mathbf{x}\|^2 = 1.$$

Im Erwartungswert trifft die Skizze die Länge also exakt.

$(\mathbf{s}_i^\top \mathbf{x})^2 = \mathbf{x}^\top \mathbf{s}_i \mathbf{s}_i^\top \mathbf{x}$, dann die Linearität des Erwartungswerts und Bedingung (8.4.2)

- (3) Die Summanden sind unabhängig, also addieren sich die Varianzen:

$$\text{Var} [\|\mathbf{S}\mathbf{x}\|^2] = \sum_{i=1}^m \text{Var} [(\mathbf{s}_i^\top \mathbf{x})^2] \leq \sum_{i=1}^m \mathbb{E} [(\mathbf{s}_i^\top \mathbf{x})^4] \leq m \cdot \frac{K}{m^2} = \frac{K}{m}.$$

Hier steckt die ganze Arbeit der vierten Momente: Sie beschränken die Streuung der Skizze.

$\text{Var}[Z] = \mathbb{E}[Z^2] - \mathbb{E}[Z]^2 \leq \mathbb{E}[Z^2]$ mit $Z = (\mathbf{s}_i^\top \mathbf{x})^2$, danach Bedingung (8.4.3) mit $\mathbf{v} = \mathbf{x}$

- (4) Die Tschebyscheff-Ungleichung schlägt die Brücke von der Varianz zur Wahrscheinlichkeit:

$$\mathbb{P} \left(\left| \|\mathbf{S}\mathbf{x}\|^2 - 1 \right| > \varepsilon \right) \leq \frac{\text{Var} [\|\mathbf{S}\mathbf{x}\|^2]}{\varepsilon^2} \leq \frac{K}{\varepsilon^2 m}.$$

Die Wahl $\varepsilon = \sqrt{K/(\delta m)}$ macht die rechte Seite zu δ , und das Gegenereignis hat damit Wahrscheinlichkeit mindestens $1 - \delta$.

Tschebyscheff mit dem in Schritt 2 berechneten Erwartungswert 1; einsetzen: $K/(\varepsilon^2 m) = K\delta m/(Km) = \delta$

□

Weil $\mathbf{S}$ linear ist, gilt Satz 8.4.6 mit $\mathbf{x} - \mathbf{y}$ sofort auch für Abstände, und Wurzelziehen macht aus der Aussage über $\|\mathbf{S}\mathbf{x}\|^2$ eine über die Länge selbst: Aus $\varepsilon = 0,1$ im Quadrat werden rund 5 % in der Länge. Damit gilt die Abstandsangabe aus Definition 8.4.4.

Vertiefung: Warum Cauchy-Schwarz zu grob ist und wie aus Quadraten Längen werden

Bemerkung 8.4.7 (Warum die Momentenbedingung an der Projektion ansetzt)

Die Varianz in Schritt 3 ließe sich auch bequemer abschätzen, nämlich über die Cauchy-Schwarz-Ungleichung $|\mathbf{s}_i^\top \mathbf{v}| \leq \|\mathbf{s}_i\| \|\mathbf{v}\|$. Dann stünde in (8.4.3) die Länge der ganzen Zeile, $\mathbb{E}[\|\mathbf{s}_i\|^4] \leq K/m^2$. Das wäre richtig, aber unbrauchbar grob: Für $\mathbf{s}_i \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n/m)$ ist

$$\mathbb{E}[\|\mathbf{s}_i\|^4] = \frac{n^2 + 2n}{m^2},$$

das zugehörige K wüchse also wie n^2 – bei $n = 10\,000$ wäre $K \approx 10^8$ statt 3, und die Schranke aus Satz 8.4.6 wäre wertlos. Der Unterschied ist gerade der verschenkte Faktor $(n^2 + 2n)/3$: Die Zeile $\mathbf{s}_i$ ist lang, ihre Projektion auf eine feste Richtung $\mathbf{v}$ aber kurz, und nur diese Projektion geht in $\|\mathbf{S}\mathbf{x}\|^2$ ein. Bedingung (8.4.3) misst deshalb direkt $\mathbb{E}[(\mathbf{s}_i^\top \mathbf{v})^4]$ und ist für Gauss-Zeilen mit $K = 3$ sogar mit Gleichheit erfüllt (Beispiel 8.4.11).

Bemerkung 8.4.8 (Von Quadraten zu Längen und Abständen)

Satz 8.4.6 spricht über $\|\mathbf{S}\mathbf{x}\|^2$, interessant sind aber Längen und Abstände.

- **Wurzelziehen.** Für $\varepsilon \leq 1$ folgt aus (8.4.4) $\sqrt{1 - \varepsilon} \leq \|\mathbf{S}\mathbf{x}\| / \|\mathbf{x}\| \leq \sqrt{1 + \varepsilon}$. Für $\varepsilon = 0,1$ heißt das $0,949 \leq \|\mathbf{S}\mathbf{x}\| / \|\mathbf{x}\| \leq 1,049$: Eine Verzerrung von 10 % im Quadrat sind rund 5 % in der Länge.
- **Von der Länge zum Abstand.** $\mathbf{S}$ ist linear, also ist $\mathbf{S}\mathbf{x} - \mathbf{S}\mathbf{y} = \mathbf{S}(\mathbf{x} - \mathbf{y})$. Wenden wir den Satz auf den festen Vektor $\mathbf{x} - \mathbf{y}$ an, so erhalten wir die Abstandsangabe aus Definition 8.4.4.
- **Mehrere Paare.** Für N Punkte und *alle* Paare brauchen wir die Aussage für $N(N - 1)/2$ Differenzvektoren gleichzeitig. Über eine Vereinigungsschranke geht das, kostet aber $\delta \rightarrow 2\delta/(N(N - 1))$ und damit ein deutlich größeres m .

Winkel behandelt der Satz zunächst gar nicht. Sie folgen aber, denn Skalarprodukte lassen sich über die Polarisationsformel aus Längen zurückgewinnen; damit ist auch die Winkelaussage aus Definition 8.4.4 gerechtfertigt.

Vertiefung: Wie die Winkeltreue aus der Längentreue folgt**Korollar 8.4.9 (Skalarprodukte bleiben erhalten)**

Seien $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ fest und $\mathbf{S}$ wie in Satz 8.4.6. Dann gilt mit Wahrscheinlichkeit mindestens $1 - 2\delta$

$$|(\mathbf{S}\mathbf{x})^\top (\mathbf{S}\mathbf{y}) - \mathbf{x}^\top \mathbf{y}| \leq \frac{\varepsilon}{2} (\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2).$$

Beweis.

- (1) Für beliebige Vektoren $\mathbf{a}, \mathbf{c}$ gilt die Polarisationsformel

$$\mathbf{a}^\top \mathbf{c} = \frac{\|\mathbf{a} + \mathbf{c}\|^2 - \|\mathbf{a} - \mathbf{c}\|^2}{4}.$$

Wir wenden sie einmal auf $\mathbf{S}\mathbf{x}, \mathbf{S}\mathbf{y}$ und einmal auf $\mathbf{x}, \mathbf{y}$ an und nutzen $\mathbf{S}\mathbf{x} \pm \mathbf{S}\mathbf{y} = \mathbf{S}(\mathbf{x} \pm \mathbf{y})$.

ausmultiplizieren: $\|\mathbf{a} + \mathbf{c}\|^2 - \|\mathbf{a} - \mathbf{c}\|^2 = 4 \mathbf{a}^\top \mathbf{c}$

- (2) Satz 8.4.6 gilt für jeden festen Vektor, insbesondere für $\mathbf{x} + \mathbf{y}$ und für $\mathbf{x} - \mathbf{y}$. Jedes der beiden Ereignisse verfehlt die Schranke mit Wahrscheinlichkeit höchstens δ , zusammen also höchstens 2δ . Treten beide ein, so ist

$$|(\mathbf{S}\mathbf{x})^\top(\mathbf{S}\mathbf{y}) - \mathbf{x}^\top\mathbf{y}| \leq \frac{\varepsilon \|\mathbf{x} + \mathbf{y}\|^2 + \varepsilon \|\mathbf{x} - \mathbf{y}\|^2}{4} = \frac{\varepsilon}{2} (\|\mathbf{x}\|^2 + \|\mathbf{y}\|^2).$$

Vereinigungsschranke für die beiden Ausnahmeeignisse; zuletzt die Parallelogrammgleichung $\|\mathbf{x} + \mathbf{y}\|^2 + \|\mathbf{x} - \mathbf{y}\|^2 = 2\|\mathbf{x}\|^2 + 2\|\mathbf{y}\|^2$

□

Für Einheitsvektoren steht rechts schlicht ε . Der Kosinus des Winkels ist das Skalarprodukt der normierten Vektoren, und Zähler wie Nenner werden bis auf Terme der Ordnung ε getroffen.

Bemerkung 8.4.10 (Ein fester Vektor ist nicht jeder Vektor)

Das Wort „fest“ in Satz 8.4.6 trägt Gewicht. Verschiebe man den Quantor „für alle $\mathbf{x} \in \mathbb{R}^n$ “ *innerhalb* der Wahrscheinlichkeit, so wäre die Aussage schlicht falsch: Unser Beweis fixiert $\mathbf{x}$, bevor $\mathbf{S}$ gezogen wird, und die Ausnahmemenge darf für jedes $\mathbf{x}$ eine andere sein. Ein Gegenargument in einer Zeile: $\mathbf{S}$ hat höchstens Rang $m < n$, besitzt also einen nichttrivialen Kern. Für ein $\mathbf{x} \neq \mathbf{0}$ mit $\mathbf{S}\mathbf{x} = \mathbf{0}$ ist (8.4.4) verletzt, ganz gleich wie $\mathbf{S}$ ausgefallen ist. Gleichmäßige Aussagen sind trotzdem möglich, aber nur auf einem Unterraum kleiner Dimension und mit Zusatzarbeit: Man kontrolliert endlich viele Richtungen eines feinen Netzes und überträgt das Ergebnis per Vereinigungsschranke auf den ganzen Unterraum. Daher rührt der Name „Einbetten von Unterräumen“, und daher stammt die Rechtfertigung für das gesketchte Kleinste-Quadrate-Problem weiter unten: Dort muss die Skizze alle Vektoren des von den Spalten von $\mathbf{A}$ und $\mathbf{b}$ aufgespannten Unterraums gleichzeitig gut treffen.

Drei Bauarten von Sketching-Matrizen

Welche Verteilungen erfüllen (8.4.2) und (8.4.3)? Drei Bauarten sind in der Praxis üblich, und alle drei lassen sich direkt nachrechnen.

Beispiel 8.4.11 (Gauss, Rademacher, Subsampling)

Gauss. $s_i \text{ iid } \mathcal{N}(0, \mathbf{I}_n/m)$. Bedingung (8.4.2) ist die Definition der Kovarianzmatrix. Für einen Einheitsvektor $\mathbf{v}$ ist $s_i^\top \mathbf{v} \sim \mathcal{N}(0, 1/m)$, und das vierte Moment einer zentrierten Normalverteilung ist das Dreifache des quadrierten zweiten:

$$\mathbb{E}[(s_i^\top \mathbf{v})^4] = 3 \left(\frac{1}{m}\right)^2 = \frac{3}{m^2}, \quad \text{also} \quad K = 3.$$

Rademacher. s_{ij} unabhängig mit $\mathbb{P}(s_{ij} = 1/\sqrt{m}) = \mathbb{P}(s_{ij} = -1/\sqrt{m}) = 1/2$. Wegen $\mathbb{E}[s_{ij}^2] = 1/m$ und $\mathbb{E}[s_{ij}s_{ik}] = 0$ für $j \neq k$ gilt wieder (8.4.2). Ausmultiplizieren der vierten Potenz liefert für $\|\mathbf{v}\| = 1$

$$\mathbb{E}[(s_i^\top \mathbf{v})^4] = \frac{3 - 2 \sum_{j=1}^n v_j^4}{m^2} \leq \frac{3}{m^2},$$

also ebenfalls $K = 3$. Besser wird die Schranke, je stärker $\mathbf{v}$ auf wenige Koordinaten konzentriert ist: Für $\mathbf{v} = \mathbf{e}_1$ ist $\sum_j v_j^4 = 1$ und damit $K = 1$, für einen gestreuten Vektor dagegen $\sum_j v_j^4 = 1/n$, also so gut wie die 3 des Gauss-Falls. Der praktische Vorteil liegt ohnehin woanders, im Erzeugen und Speichern: ein Bit pro Eintrag.

Subsampling. $\mathbf{s}_i = \sqrt{n/m} \mathbf{e}_{K_i}$ mit $K_i \sim \text{Unif}\{1, \dots, n\}$, die Skizze greift also m zufällige Koordinaten heraus und skaliert sie. Auch hier ist $\mathbb{E}[\mathbf{s}_i \mathbf{s}_i^\top] = \frac{n}{m} \cdot \frac{1}{n} \mathbf{I}_n = \mathbf{I}_n/m$. Das vierte Moment sieht aber anders aus:

$$\mathbb{E}[(\mathbf{s}_i^\top \mathbf{v})^4] = \left(\frac{n}{m}\right)^2 \frac{1}{n} \sum_{j=1}^n v_j^4 = \frac{n \sum_{j=1}^n v_j^4}{m^2}, \quad \text{also} \quad K = n \sum_{j=1}^n v_j^4.$$

Dieses K hängt von den Daten ab. Für einen gestreuten Vektor mit $v_j = \pm 1/\sqrt{n}$ ist $\sum_j v_j^4 = 1/n$ und damit $K = 1$, besser als Gauss. Für $\mathbf{v} = \mathbf{e}_1$ dagegen ist $K = n$: Die gesamte Information steckt in einer Koordinate, und die Stichprobe mit Zurücklegen erwischt sie mindestens einmal nur mit Wahrscheinlichkeit

$$1 - \left(1 - \frac{1}{n}\right)^m,$$

nicht mit m/n . Subsampling taugt also, wenn die Information über viele Koordinaten verteilt ist, und versagt, wenn sie in wenigen sitzt. Der Merksatz, Subsampling sei „nur gut für spezielle Daten“, ist genau in dieser Richtung zu lesen: Verteilte Information ist der gute Fall, wenige dominante Koordinaten der schlechte.

Anwendungen

Bemerkung 8.4.12 (Wo Skizzen helfen)

- **Matrixprodukt.** Für $\mathbf{A} \in \mathbb{R}^{p \times n}$ und $\mathbf{B} \in \mathbb{R}^{n \times q}$ ist $\mathbf{AB} \approx (\mathbf{AS}^\top)(\mathbf{SB})$. Der Grund ist (8.4.2): Wegen $\mathbb{E}[\mathbf{S}^\top \mathbf{S}] = \sum_{i=1}^m \mathbb{E}[\mathbf{s}_i \mathbf{s}_i^\top] = \mathbf{I}_n$ ist die rechte Seite erwartungstreu für $\mathbf{AB}$. Statt pnq kostet das Matrixprodukt dann $pnm + mnq + pmq$ Multiplikationen, für $p = q = n$ also rund $3n^2m$ statt n^3 .
- **Kleinste Quadrate.** $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2 \approx \min_{\mathbf{x}} \|\mathbf{SAx} - \mathbf{Sb}\|_2$, siehe Algorithmus 8.4.13.
- **Approximative Zerlegungen.** Auch die randomisierte Näherung einer Singulärwertzerlegung aus Abschnitt 8.2 arbeitet mit einer Skizze des Spaltenraums.

Ein Vorbehalt gilt für alle drei Punkte: Schneller wird ein Algorithmus nur, wenn sich $\mathbf{Sx}$ schnell berechnen lässt. Zieht $\mathbf{S}$ nur m Komponenten von $\mathbf{x}$ heraus, kostet das $O(m)$ Operationen. Eine klassische schnelle Wahl ist die *Subsampled Randomized Hadamard Transform* (SRHT). Ihre vollständige Hadamard-Transformation kostet $O(n \log n)$ Rechenschritte; speziell geprunte Varianten können unter zusätzlichen Implementationsannahmen weniger benötigen. Bei einer dichten Gauss-Matrix dagegen kostet $\mathbf{Sx}$ schon $O(mn)$ Operationen, und dann kann die Skizze teurer sein als das Problem, das sie vereinfachen soll.

Algorithmus 8.4.13 (Sketching für ein KQ-Problem)

Gegeben $\mathbf{A} \in \mathbb{R}^{n \times p}$ mit $n \gg p$, $\mathbf{b} \in \mathbb{R}^n$ und eine Skizzengröße m mit $p < m \ll n$.

1. Ziehe eine Sketching-Matrix $\mathbf{S} \in \mathbb{R}^{m \times n}$.
2. Berechne $\mathbf{SA} \in \mathbb{R}^{m \times p}$ und $\mathbf{Sb} \in \mathbb{R}^m$.
3. Löse das kleine Problem $\min_{\mathbf{x}} \|(\mathbf{SA})\mathbf{x} - \mathbf{Sb}\|_2$, etwa per QR-Zerlegung (Abschnitt 7.4).

Schritt 3 kostet $O(mp^2)$ statt $O(np^2)$ für das Ausgangsproblem. Ob sich das lohnt, entscheidet

Schritt 2: Eine dichte Gauss-Matrix braucht dort $O(mnp)$ Operationen, und wegen $m > p$ ist das mehr als die $O(np^2)$, die wir sparen wollten. Erst mit schnell anwendbaren Skizzen wie der SRHT geht die Rechnung auf.

Wie klein darf m sein?

Beispiel 8.4.14 (Dimensionsreduktion mit Matrix-Sketching)

Gegeben seien $n = 10\,000$ Dimensionen, eine Fehlerwahrscheinlichkeit $\delta = 0,05$ und eine zulässige Verzerrung $\varepsilon = 0,1$. Wie klein darf m sein? Wir lösen (8.4.4) nach m auf und setzen für Gauss-Zeilen $K = 3$ ein:

$$\varepsilon = \sqrt{\frac{K}{\delta m}} \iff m = \frac{K}{\delta \varepsilon^2} = \frac{3}{0,05 \cdot 0,01} = 6000. \quad (8.4.5)$$

Mit $m = 6000$ Dimensionen bleibt der Abstand eines vorher festgelegten Punktpaares mit Wahrscheinlichkeit 95 % auf 10 % genau erhalten, gemessen im Quadrat der Länge, in der Länge selbst auf rund 5 %, denn $\sqrt{1 \pm 0,1}$ weicht von 1 nur um rund 5 % ab. Der Speicher sinkt von 10 000 auf 6000 Zahlen je Punkt, ein Faktor $n/m \approx 1,7$, und eine einzelne Distanzberechnung kostet statt $O(n)$ nur noch $O(m)$ Operationen, ebenfalls ein Faktor n/m . Davon leben die typischen Anwendungen: die Suche nach nächsten Nachbarn und die Clusteranalyse in hohen Dimensionen. Beide berühren die Daten fast nur über Abstände, und die bleiben unter der Skizze erhalten.

Umgekehrt gelesen sagt (8.4.5) auch, was ein zehnmal kleineres $m = 600$ hergibt: bei $\delta = 0,05$ nur $\varepsilon = \sqrt{3/(0,05 \cdot 600)} = \sqrt{0,1} \approx 0,32$, und bei $\varepsilon = 0,1$ nur $\delta = 3/(600 \cdot 0,01) = 0,5$, also die Verlässlichkeit eines Münzwurfs.

Bemerkung 8.4.15 (Was der Gewinn wirklich ist)

Ein Nachtrag zu dieser Rechnung, und dabei lohnt der Blick darauf, was genau gerechnet wird: Eine einzelne euklidische Distanz kostet $O(n)$ Operationen, der Gewinn ist deshalb der Faktor $n/m \approx 1,7$ und nicht sein Quadrat. Quadratisch in der Dimension sind nur bestimmte Verfahren, etwa das Aufstellen einer Kovarianzmatrix, und nur dort schlägt $(n/m)^2$ zu Buche.

Vertiefung: Warum in der Praxis viel kleinere Skizzen genügen

Warum wird in der Praxis mit Skizzengrößen in der Größenordnung einiger hundert gearbeitet, wo die Rechnung oben $m = 6000$ verlangt? Der Grund ist unser Werkzeug: Tschebyscheff benutzt nur zwei Momente und ist entsprechend grob. Wie grob, lässt sich für Gauss-Skizzen ausnahmsweise exakt sagen. Nach Beispiel 8.4.5 ist $\|\mathbf{S}\mathbf{x}\|^2 / \|\mathbf{x}\|^2$ dort χ_m^2/m -verteilt, und deren Verteilungsfunktion verlangt für $\varepsilon = 0,1$ und $\delta = 0,05$ nur $m = 768$ Zeilen. Tschebyscheff fordert das Achtfache. Für andere Verteilungen steht diese exakte Rechnung nicht zur Verfügung, schärfere Konzentrationsungleichungen dagegen schon: Sie liefern ein m der Größenordnung $\varepsilon^{-2} \log(1/\delta)$, also eine logarithmische statt einer linearen Abhängigkeit von $1/\delta$. Der Preis dafür sind deutlich mehr Voraussetzungen und ein längerer Beweis (Literatur am Ende des Abschnitts).

Vertiefung: Vergleich weiterer Sketching-Matrizen

Vergleich der Sketching-Matrizen

Typ	Speicher	Zeit für Sx	Qualität
Gauss	$O(mn)$	$O(mn)$	optimal
Rademacher ($\pm 1/\sqrt{m}$)	$O(mn)$	$O(mn)$	optimal
Subsampling	$O(m)$	$O(m)$	gut bei gestreuten Daten
SRHT (Hadamard)	$O(n + m)$ oder reproduzierbar erzeugte Vorzeichen	$O(n \log n)$	sehr gut

Speicher und Rechenzeit stehen in Landau-Notation: einmal für das Vorhalten von S , einmal für ein einzelnes Produkt Sx .

- **Gauss** ist theoretisch optimal, aber teuer: mn Zahlen speichern und mn Multiplikationen je Vektor.
- **Rademacher** ist genauso gut, wie Beispiel 8.4.11 mit $K = 3$ zeigt, und einfacher zu erzeugen, weil nur Vorzeichen gewürfelt werden.
- **Subsampling** wählt m zufällige Koordinaten und ist damit extrem schnell. Die Qualität hängt allerdings an den Daten: Nach Beispiel 8.4.11 ist $K = n \sum_j v_j^4$, für gestreute Vektoren klein und für konzentrierte Vektoren so groß wie n .
- **SRHT** verbindet beides. Die Matrix wird nicht explizit gespeichert, sondern durch eine schnelle Transformation angewendet, daher die $O(n \log n)$ in der Tabelle. Neben den m gezogenen Indizes gehören n Vorzeichen zur Transformation; sie werden gespeichert oder reproduzierbar aus einem Seed erzeugt. Die Arbeitskopie des transformierten Vektors benötigt unabhängig davon $O(n)$ Speicher.

Selbsttest

1. Es gibt eine Matrix $S \in \mathbb{R}^{m \times n}$ mit $m < n$, die alle Abstände exakt erhält.

Antwort: Falsch

Eine solche Matrix hat höchstens Rang $m < n$, also einen nichttrivialen Kern. Für ein $x \neq 0$ mit $Sx = 0$ schrumpft der Abstand zum Nullpunkt auf 0. Sketching kann Abstände nur näherungsweise und nur mit hoher Wahrscheinlichkeit erhalten (Bemerkung 8.4.10).

2. Satz 8.4.6 macht eine Aussage über einen vorher festgelegten Vektor x , nicht gleichzeitig über alle $x \in \mathbb{R}^n$.

Antwort: Wahr

Im Beweis wird x fixiert, bevor S gezogen wird; die Ausnahmemenge darf für jedes x eine andere sein. Gleichmäßige Aussagen über einen ganzen Unterraum brauchen ein Netzargument und eine Vereinigungsschranke (Bemerkung 8.4.10).

3. Verdoppeln wir m , so halbiert sich die garantierte Verzerrung ε .

Antwort: Falsch

Nach (8.4.4) ist $\varepsilon = \sqrt{K/(\delta m)}$, also proportional zu $1/\sqrt{m}$. Verdoppeltes m drückt ε nur um den Faktor $1/\sqrt{2} \approx 0,71$; für die Halbierung brauchen wir viermal so viele Zeilen.

4. Für Gauss-Zeilen $s_i \sim \mathcal{N}(0, I_n/m)$ gilt $\mathbb{E}[(s_i^\top v)^4] = 3/m^2$ für jeden Vektor v mit $\|v\| = 1$.

Antwort: Wahr

Es ist $\mathbf{s}_i^\top \mathbf{v} \sim \mathcal{N}(0, 1/m)$, und das vierte Moment einer zentrierten Normalverteilung ist $3\sigma^4$ (Beispiel 8.4.11). Damit ist (8.4.3) mit $K = 3$ erfüllt, und zwar mit Gleichheit.

5. Subsampling ist für beliebige Daten genauso gut wie eine Gauss-Skizze.

Antwort: Falsch

Für Subsampling ist $K = n \sum_j v_j^4$ und hängt damit von den Daten ab. Bei gestreuten Vektoren ist $K = 1$ und damit besser als die 3 der Gauss-Skizze, beim Standard-Basisvektor $\mathbf{e}_1$ dagegen $K = n$. Die Tabelle oben schreibt deshalb „gut bei gestreuten Daten“ und nicht „optimal“.

Vertiefung: Die wahrscheinlichkeitstheoretischen Werkzeuge, also Zufallsmatrizen und Konzentrationsungleichungen, behandelt R. Vershynin, High-Dimensional Probability (Cambridge University Press, 2018); die Sketching-Algorithmen selbst, samt SRHT und gesketchten KQ-Problemen, P.-G. Martinsson und J. A. Tropp, Randomized numerical linear algebra: Foundations and algorithms, Acta Numerica 29 (2020).

8.5 Zusammenfassung

Bis Kapitel 7 lief fast alles über *Zerlegungen*. LU, Cholesky, QR und die SVD schreiben eine Matrix als Produkt handlicher Faktoren, und die eigentliche Rechnung findet danach in diesen Faktoren statt. Das ist exakt bis auf Rundungsfehler, und es kostet, was es kostet: rund n^3 Operationen, gleich ob wir drei Stellen brauchen oder fünfzehn. In diesem Kapitel sind zwei Bauprinzipien dazugekommen, die diesen festen Preis drücken, indem sie etwas anderes dafür hergeben.

Die drei Kernideen

Bemerkung 8.5.1 (Die drei Kernideen)

1. **Eigenwerte berechnen wir iterativ.** Das charakteristische Polynom beschreibt die Eigenwerte zwar vollständig, taugt aber nicht als Algorithmus (Bemerkung 8.1.1). Stattdessen wenden wir die Matrix immer wieder an. Die Potenzmethode (Algorithmus 8.1.2) holt mit einem Matrix-Vektor-Produkt pro Schritt das betragsgrößte Eigenpaar, sofern der größte Eigenwert einsam an der Spitze steht (Satz 8.1.4); die QR-Iteration (Algorithmus 8.1.10) liefert unter den Voraussetzungen von Bemerkung 8.1.14 alle Eigenwerte auf einmal, im Grunde als simultane Potenzmethode auf allen Koordinatenachsen (Bemerkung 8.1.13). Getragen wird die QR-Iteration von der Ähnlichkeit: Jeder Schritt ersetzt $\mathbf{A}^{(k-1)}$ durch eine ähnliche Matrix (Satz 8.1.11), und ähnliche Matrizen haben dasselbe Spektrum (Satz 8.1.7). Wie weit das trägt, zeigt Abschnitt 8.2: Page-Rank ist eine Potenzmethode auf einer dünnbesetzten Matrix mit rund 10^{10} Zeilen, die Hauptkomponentenanalyse eine Eigenwertaufgabe für die Kovarianzmatrix, und die approximative SVD liefert die k größten Singulärwerte, ohne die Matrix je vollständig zu zerlegen.
2. **Iterative Verfahren sind Fixpunktiterationen.** Aus dem Residuum $\mathbf{r}^{(k-1)} = \mathbf{b} - \mathbf{A} \mathbf{x}^{(k-1)}$ wird eine Korrektur, und ausmultipliziert steht dort die affine Vorschrift $\mathbf{x}^{(k)} = \mathbf{B} \mathbf{x}^{(k-1)} + \mathbf{C} \mathbf{b}$ mit $\mathbf{B} = \mathbf{I}_n - \mathbf{C} \mathbf{A}$ (Bemerkung 8.3.4). Ob die Folge zusammenläuft, entscheidet allein die Iterationsmatrix $\mathbf{B}$. Hinreichend dafür ist $\rho = \|\mathbf{B}\|_2 < 1$: Dann schrumpft der Fehler in jedem Schritt auf höchstens das ρ -Fache (Satz 8.3.5). Notwendig ist die kleine Norm nicht, auf lange Sicht zählt der Spektralradius (Beispiel 8.3.9). Diese Fix-

punktstruktur verbindet beide Hälften des Kapitels. Auch die Potenzmethode ist eine: Die Abbildung $\mathbf{x} \mapsto \mathbf{Ax} / \|\mathbf{Ax}\|$ lässt genau die normierten Eigenvektoren zu Eigenwerten $\lambda \neq 0$ fest, bis auf ihr Vorzeichen, und die Rate heißt dort $|\lambda_2/\lambda_1|$ statt ρ (Satz 8.1.4).

3. **Probabilistische Verfahren ersetzen ein schweres Problem durch ein zufälliges, kleineres.** Eine geeignet gezogene Matrix $\mathbf{S} \in \mathbb{R}^{m \times n}$ mit $m \ll n$ erhält die Abstände und Winkel *vorher festgelegter* Punkte mit hoher Wahrscheinlichkeit näherungsweise (Abschnitt 8.4). Für alle Vektoren zugleich gilt das nicht, schon weil $\mathbf{S}$ einen nichttrivialen Kern hat (Bemerkung 8.4.10). Statt mit $\mathbf{A}$ rechnen wir mit der Skizze $\mathbf{SA}$ weiter, und aus einem n -dimensionalen Problem wird ein m -dimensionales. Die Garantie ist dafür von anderer Art als bisher: Sie gilt nur mit Wahrscheinlichkeit $1 - \delta$.

Die zweite und die dritte Idee laufen auf dasselbe Geschäft hinaus: Wir geben Genauigkeit her und bekommen Laufzeit zurück. Nur sitzt der Regler an verschiedenen Stellen.

Laufzeit gegen Genauigkeit

Bemerkung 8.5.2 (Zwei Regler und ihre Preise)

Zugang	Regler	Kosten	Was wir bekommen
Zerlegung (Kapitel 5, Kapitel 7)	keiner	fest, Größenordnung n^3	Lösung bis auf Rundungsfehler
Iteration (8.1, 8.3)	Iterationszahl k	k mal die Kosten eines Schritts	Fehler höchstens ρ^k mal Startfehler
Sketching (8.4)	Skizzengröße m	Rechnen in $\mathbb{R}^m$ statt in $\mathbb{R}^n$	Verzerrung ε je festem Paar, mit Wahrscheinlichkeit $1 - \delta$

Die beiden Regler haben sehr verschiedene Preisschilder. Bei der Iteration fällt der Fehler geometrisch, die Schrittzahl wächst deshalb nur wie $\log(1/\varepsilon)$ (Korollar 8.3.6); die Zahlen dazu stehen bei Beispiel 8.3.11. Beim Sketching fällt die Verzerrung dagegen nur wie $1/\sqrt{m}$: Wer doppelt so viele Zeilen zieht, drückt sie auf das $1/\sqrt{2} \approx 0,71$ -fache, und eine zusätzliche Dezimalstelle verlangt die hundertfache Skizzengröße.

Ein fairer Vergleich ist das trotzdem nicht, denn die beiden ε messen Verschiedenes: einmal den Abstand zur Lösung, einmal die Verzerrung der Geometrie. Die Faustregel, die bleibt: Wo hohe Genauigkeit gefragt ist, iterieren wir; wo eine grobe Antwort genügt und n riesig ist, würfeln wir.

Ausblick

Mit diesem Kapitel endet der erste Block des Skripts, die numerische lineare Algebra (Abschnitt 1.1). Das nächste Kapitel beginnt mit Tensoren und Tensorprodukten. Ein Tensor ist ein mehrdimensionales Zahlenfeld, die Matrix also der zweidimensionale Sonderfall. Gebraucht werden sie, sobald wir nach Matrizen ableiten wollen, denn jedes Differenzieren hängt der Buchführung eine weitere Dimension an. Die Fragen bleiben dabei dieselben wie hier: zerlegen, approximieren, iterieren.

Selbsttest

Fünf Aussagen zu den Kernideen dieses Kapitels. Welche stimmen?

1. Die Eigenwerte einer großen Matrix bestimmen wir besser nicht über die Nullstellen des charakteristischen Polynoms.

Antwort: Wahr

Drei Gründe stehen in Bemerkung 8.1.1. Ab Grad 5 gibt es keine allgemeine Lösungsformel mehr, wir müssten die Nullstellen also ohnehin iterativ suchen. Der Umweg über die Koeffizienten ist schlecht konditioniert. Und schon das Aufstellen kostet mehr als eine ganze Matrixzerlegung.

2. Iterative Verfahren liefern grundsätzlich nur Näherungen; die exakte Lösung erreichen sie nie.

Antwort: Falsch

In Sonderfällen erreichen sie sie sehr wohl (Bemerkung 8.3.12). Mit $C = A^{-1}$ ist $\rho = 0$, und ein einziger Schritt trifft die Lösung; Verfahren vom Krylov-Typ wie das der konjugierten Gradienten kommen in exakter Arithmetik nach höchstens n Schritten an. Nützlich ist das selten, denn A^{-1} zu kennen wäre teurer als das Gleichungssystem selbst.

3. Die Potenzmethode konvergiert umso schneller, je näher $|\lambda_2|$ bei $|\lambda_1|$ liegt.

Antwort: Falsch

Es ist genau umgekehrt. Die Rate ist $|\lambda_2/\lambda_1|$ (Satz 8.1.4), und die geht gegen 1, sobald sich die beiden Eigenwerte nähern; im Grenzfall $|\lambda_2| = |\lambda_1|$ trägt Satz 8.1.4 gar nicht mehr (Bemerkung 8.1.6). Dieselbe Nähe macht auch die Eigenvektoren schlecht konditioniert (Bemerkung 8.1.16).

4. Verschärfen wir bei der Korrekturiteration die geforderte Genauigkeit, so wächst die nötige Schrittzahl nur logarithmisch.

Antwort: Wahr

Das ist Korollar 8.3.6: $k = O(\log(1/\varepsilon))$, weil der Fehler in jedem Schritt auf höchstens das ρ -Fache schrumpft. Bei $\rho \approx 0,405$ sind das rund zweieinhalb Schritte je Dezimalstelle, unabhängig davon, bei welcher Stelle wir gerade sind.

5. Verdoppeln wir beim Sketching die Zeilenzahl m , so halbiert sich die Verzerrung.

Antwort: Falsch

Die Schranke aus Abschnitt 8.4 fällt wie $1/\sqrt{m}$, nicht wie $1/m$. Doppelt so viele Zeilen drücken die Verzerrung deshalb nur auf das $1/\sqrt{2} \approx 0,71$ -fache; halbieren lässt sie sich erst mit der vierfachen Skizzengröße. Genauigkeit ist bei zufälligen Verfahren der teure Teil, Dimensionsreduktion der billige.

Vertiefung: Heath §4.5 zu den Eigenwertverfahren, Heath §10.9 zu den iterativen Lösern für lineare Gleichungssysteme. Die wahrscheinlichkeitstheoretischen Werkzeuge hinter den Sketching-Schranken behandelt R. Vershynin, High-Dimensional Probability, Cambridge University Press 2018.

Tensoren & Tensorprodukte

9.1 Multilineare Abbildungen

Eine lineare Abbildung nimmt einen Vektor und liefert einen Vektor. Für viele Rechenvorschriften ist das zu eng. Ein Skalarprodukt braucht zwei Vektoren, die Matrizenmultiplikation verarbeitet zwei Matrizen zu einer dritten, und hinter einer quadratischen Form $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ steckt die Abbildung $(\mathbf{x}, \mathbf{y}) \mapsto \mathbf{x}^\top \mathbf{A} \mathbf{y}$, die ebenfalls zwei Vektoren aufnimmt. Betrachten wir das ganze Tupel auf einmal, ist keine dieser Abbildungen linear. In jedem einzelnen Argument sind sie es aber sehr wohl, und diese Beobachtung trägt das gesamte Kapitel.

Was wir brauchen

Sammeln wir zuerst das Vorwissen ein, auf dem dieser Abschnitt steht.

- Vektorräume und ihre Unterräume, dazu Basis und Dimension. In einem Raum der Dimension n ist jeder Vektor durch seine n Koordinaten bezüglich einer festen Basis bestimmt.
- Lineare Abbildungen, also $f(c\mathbf{v} + c'\mathbf{v}') = cf(\mathbf{v}) + c'f(\mathbf{v}')$. Das ist die Eigenschaft, die wir gleich verallgemeinern.
- Die lineare Hülle $\text{span}(\{\mathbf{v}_1, \dots, \mathbf{v}_n\}) = \{\mathbf{w} : \mathbf{w} = \sum_i c_i \mathbf{v}_i, c_i \in \mathbb{R}\}$.
- Als Vorschau: Das *äußere Produkt* $\mathbf{v} \otimes \mathbf{w} = \mathbf{v}\mathbf{w}^\top$, das wir in Abschnitt 9.3 einführen. Das Zeichen $\otimes$ begegnet uns in diesem Kapitel noch in allgemeinerer Rolle; für das Kroneckerprodukt, die dritte Konstruktion aus Abschnitt 9.3, verwenden wir das eigene Zeichen $\otimes_K$.
- Aus der Analysis: Funktionen mehrerer Veränderlicher, $f(\mathbf{x}) = f(x_1, x_2, \dots, x_n)$.

Von einem Vektor zu einem Tupel

Bisher haben wir einen Vektor $\mathbf{v} \in V$ linear auf einen Vektor $\mathbf{w} \in W$ abgebildet. Jetzt bilden wir Paare oder allgemeiner Tupel $(\mathbf{v}_1, \dots, \mathbf{v}_n) \in V_1 \times \dots \times V_n$ ab, und zwar wieder „linear“. Die Anführungszeichen sind nötig, denn Linearität bezieht sich hier auf die *einzelnen* Argumente: Halten wir alle Argumente bis auf eines fest, so soll die verbleibende Abbildung linear sein. Solche Abbildungen heißen *multilinear*.

Definition 9.1.1 (Multilineare Abbildung)

Seien $V_1, \dots, V_n, W$ Vektorräume. Eine Abbildung $f : V_1 \times \dots \times V_n \rightarrow W$ heißt *multilinear*, wenn

$$\begin{aligned} f(\mathbf{v}_1, \dots, c\mathbf{v}_i + c'\mathbf{v}'_i, \dots, \mathbf{v}_n) \\ = c f(\mathbf{v}_1, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n) + c' f(\mathbf{v}_1, \dots, \mathbf{v}'_i, \dots, \mathbf{v}_n) \end{aligned}$$

für alle $i = 1, \dots, n$, alle $c, c' \in \mathbb{R}$ und alle $\mathbf{v}_i, \mathbf{v}'_i \in V_i$ gilt.

Die Bedingung ist n Bedingungen auf einmal, eine je Argument. Für festes i sagt sie: Frieren wir die übrigen $n - 1$ Argumente ein, so ist die verbleibende Abbildung $V_i \rightarrow W$ linear im gewohnten Sinn.

Bemerkung 9.1.2 (Bilinear, und eine Warnung)

Im Spezialfall $n = 2$ nennen wir f *bilinear*. Dieser Fall begegnet uns in den Beispielen unten immer wieder.

Achtung: Multilineare Abbildungen sind in der Regel *nicht* linear. Der Grund steckt schon in der Definition. Skalieren wir alle n Argumente gleichzeitig mit demselben c , so zieht die Homogenität den Faktor n -mal heraus, einmal je Argument:

$$f(c\mathbf{v}_1, c\mathbf{v}_2, \dots, c\mathbf{v}_n) = c^n f(\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n).$$

Für eine lineare Abbildung müsste dort c stehen. Ab $n \geq 2$ ist das ein Unterschied, sobald f überhaupt einen Wert ungleich 0 annimmt.

Die Fläche eines Rechtecks

Das kleinste interessante Beispiel steckt in der Schulgeometrie.

Beispiel 9.1.3 (Fläche eines Rechtecks)

Wir betrachten die Fläche eines Rechtecks als Funktion seiner beiden Seitenlängen:

$$f : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}, \quad (x, y) \mapsto xy.$$

Warum ist f nicht linear? Verdoppeln wir beide Seiten, setzen also $c = 2$, so wächst die Fläche auf das Vierfache:

$$f(2x, 2y) = 2x \cdot 2y = 4xy = 4f(x, y).$$

Linear wäre $f(cx, cy) = cf(x, y)$, hier also das Doppelte. Solange $xy \neq 0$ ist, klaffen die beiden Werte auseinander.

Aber f ist bilinear. Halten wir y fest, so gelten Additivität und Homogenität im ersten Argument:

$$\begin{aligned} f(x_1 + x_2, y) &= (x_1 + x_2)y = x_1y + x_2y = f(x_1, y) + f(x_2, y), \\ f(cx, y) &= (cx)y = c(xy) = cf(x, y). \end{aligned}$$

Halten wir umgekehrt x fest, läuft dieselbe Rechnung im zweiten Argument:

$$\begin{aligned} f(x, y_1 + y_2) &= x(y_1 + y_2) = xy_1 + xy_2 = f(x, y_1) + f(x, y_2), \\ f(x, cy) &= x(cy) = c(xy) = cf(x, y). \end{aligned}$$

Die Funktion ist also in jedem Argument einzeln linear. Dieselbe Rechnung ohne die geometrische Deutung zeigt zugleich, dass die Multiplikation reeller Zahlen $(x_1, x_2) \mapsto x_1x_2$ bilinear ist; gebraucht haben wir nur Distributiv-, Assoziativ- und Kommutativgesetz in $\mathbb{R}$.

Bemerkung 9.1.4 (Das Vierfache, geometrisch gelesen)

Der Faktor 4 braucht keine Rechnung. Verdoppeln wir beide Seiten, passt das alte Rechteck viermal in das neue; verdoppeln wir nur eine Seite, genau zweimal, und das ist die Linearität

im einzelnen Argument. Der Faktor $4 = 2^2$ ist der Fall $n = 2$ der allgemeinen Regel c^n aus Bemerkung 9.1.2.

Interaktiv: Verdoppeln, einmal beide Seiten und einmal nur eine

Welche der beiden Skalierungen besteht den Linearitätstest, wenn wir die Fläche zuerst nur aus dem Rechteck vorhersagen?

Interaktives Element — nur in der Web-Fassung

Das Rechteckbild zeigt damit, warum die getrennte Linearität genau die Regel ist, die das Tensorprodukt später in eine lineare Abbildung übersetzt.

Weitere Beispiele

Beispiel 9.1.5 (Skalarprodukte sind bilinear)

Jedes Skalarprodukt $f : V \times V \rightarrow \mathbb{R}, (v_1, v_2) \mapsto \langle v_1, v_2 \rangle$, ist bilinear. Im ersten Argument rechnen wir

$$\langle cv + c'v', w \rangle = \langle cv, w \rangle + \langle c'v', w \rangle = c \langle v, w \rangle + c' \langle v', w \rangle.$$

Der erste Schritt ist die Additivität, der zweite die Homogenität. Bei einer axiomatischen Einführung stehen beide schon in der Definition des Skalarprodukts. Für das Standardskalarprodukt $x^T y = \sum_i x_i y_i$ rechnen wir sie in einer Zeile nach: Das Distributivgesetz der reellen Zahlen gibt die Additivität, das Herausziehen des Skalars die Homogenität. Für das zweite Argument brauchen wir dann keine neue Rechnung: Die Symmetrie $\langle v, w \rangle = \langle w, v \rangle$ dreht die Rollen um.

Beispiel 9.1.6 (Matrizenmultiplikation als bilineare Abbildung)

Die Matrizenmultiplikation

$$\mathbb{R}^{m \times k} \times \mathbb{R}^{k \times n} \rightarrow \mathbb{R}^{m \times n}, \quad (A, B) \mapsto AB,$$

ist bilinear. In beiden Argumenten ist das nichts anderes als das Distributivgesetz für Matrizen, zusammen mit der Verträglichkeit mit Skalaren:

$$(cA + c'A')B = cAB + c'A'B, \quad A(cB + c'B') = cAB + c'AB'.$$

Linear ist auch sie nicht. Skalieren wir beide Faktoren mit c , so erhalten wir $(cA)(cB) = c^2 AB$, wie es Bemerkung 9.1.2 für $n = 2$ vorhersagt. Die Matrizenmultiplikation ist damit unser erstes Beispiel, in dem die drei beteiligten Räume im Allgemeinen verschieden sind.

Darstellung durch Koeffizienten

Multilineare Abbildungen wirken zunächst wie eine unübersichtliche Klasse. Sind die beteiligten Räume Koordinatenräume, lässt sich jede von ihnen aber durch eine endliche Liste von Zahlen vollständig beschreiben.

Satz 9.1.7 (Darstellung multilinearer Abbildungen)

Eine Abbildung $T : \mathbb{R}^{n_1} \times \dots \times \mathbb{R}^{n_k} \rightarrow \mathbb{R}^m$ ist genau dann multilinear, wenn es reelle Koeffizienten

$$A = (a_{i_1, \dots, i_k, j} : 1 \leq i_1 \leq n_1, \dots, 1 \leq i_k \leq n_k, 1 \leq j \leq m)$$

gibt, sodass

$$T(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}) = \sum_{i_1=1}^{n_1} \dots \sum_{i_k=1}^{n_k} \sum_{j=1}^m a_{i_1, \dots, i_k, j} x_{i_1}^{(1)} \dots x_{i_k}^{(k)} \mathbf{e}_j \quad (9.1.1)$$

gilt, wobei $\mathbf{e}_j$ der j -te Einheitsvektor des $\mathbb{R}^m$ ist. Die Koeffizienten $a_{i_1, \dots, i_k, j}$ sind eindeutig bestimmt.

Einen vollständigen Beweis führen wir nicht. Woher die Koeffizienten kommen, lässt sich aber in einem Satz sagen: $a_{i_1, \dots, i_k, j}$ ist die j -te Komponente des Vektors $T(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_k}) \in \mathbb{R}^m$. Damit ist auch die Eindeutigkeit klar, denn diese Werte legt T selbst fest. Eine multilineare Abbildung ist also bereits dadurch bestimmt, was sie mit Tupeln von Einheitsvektoren macht.

Vertiefung: Warum die Koeffizientendarstellung stimmt

Dass eine Abbildung der Gestalt (9.1.1) multilinear ist, ist Nachrechnen. In jedem Summanden steht aus jedem Argument genau eine Koordinate als Faktor. Ersetzen wir $\mathbf{x}^{(l)}$ durch $c\mathbf{x}^{(l)} + c'\mathbf{y}^{(l)}$, so spaltet sich jeder Summand entsprechend auf, und die rechte Seite ist in jedem Argument linear.

Für die andere Richtung entwickeln wir jedes Argument in der Standardbasis seines eigenen Raums, $\mathbf{x}^{(l)} = \sum_{i_l} x_{i_l}^{(l)} \mathbf{e}_{i_l}$ mit $\mathbf{e}_{i_l} \in \mathbb{R}^{n_l}$, und ziehen die Summen nacheinander aus T heraus. Das erlaubt die Multilinearität, und übrig bleibt

$$T(\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}) = \sum_{i_1=1}^{n_1} \dots \sum_{i_k=1}^{n_k} x_{i_1}^{(1)} \dots x_{i_k}^{(k)} T(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_k}).$$

Der Vergleich mit (9.1.1) liefert die Koeffizienten: Dort steht $a_{i_1, \dots, i_k, j}$ genau an der Stelle, an der hier die j -te Komponente von $T(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_k})$ auftaucht.

Zählen wir nach: Es sind $n_1 n_2 \dots n_k \cdot m$ Koeffizienten. Unsere Flächenfunktion aus Beispiel 9.1.3 hat $k = 2$ Argumente mit $n_1 = n_2 = 1$ und $m = 1$, kommt also mit einer einzigen Zahl aus, nämlich $a_{1,1,1} = 1$. Ein bilineares $T : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ braucht dagegen schon $2 \cdot 2 \cdot 1 = 4$ Koeffizienten. Wie wir diese Zahlen anordnen und warum daraus ein Objekt mit $k + 1$ Indexpositionen wird, bei $m = 1$ also mit k , ist das Thema von Abschnitt 9.2.

Selbsttest

1. Wenn im Rechteck-Widget eine Seite null ist, ist das Verhältnis der Flächen vor und nach dem Verdoppeln nicht definiert.

Antwort: Wahr

Das Widget zeigt dann zwei Flächen mit Wert null. Der Quotient wäre $0/0$; darum entscheidet dieser Randfall nicht zwischen Linearität und Bilinearität.

2. Für jede multilineare Abbildung f gilt $f(\mathbf{v}_1, \dots, \mathbf{v}_n) = \mathbf{0}$, sobald $\mathbf{v}_i = \mathbf{0}$ für ein i ist.

Antwort: Wahr

Setzen wir in Definition 9.1.1 den zweiten Skalar auf $c' = 0$, so bleibt im i -ten Argument die

reine Homogenität $f(\mathbf{v}_1, \dots, c\mathbf{v}_i, \dots, \mathbf{v}_n) = c f(\mathbf{v}_1, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n)$ übrig. Darin wählen wir $c = 0$ und nutzen $\mathbf{0} = 0 \cdot \mathbf{v}_i$:

$$f(\mathbf{v}_1, \dots, \mathbf{0}, \dots, \mathbf{v}_n) = f(\mathbf{v}_1, \dots, 0 \cdot \mathbf{v}_i, \dots, \mathbf{v}_n) = 0 \cdot f(\mathbf{v}_1, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n) = \mathbf{0}.$$

Welchen Wert die übrigen Argumente haben, spielt dabei keine Rolle. Ein einziges Nullargument genügt.

3. Jede bilineare Abbildung ist insbesondere linear.

Antwort: Falsch

Bemerkung 9.1.2 sagt das Gegenteil. Skalieren wir bei $n = 2$ beide Argumente mit c , so wird der Wert mit c^2 multipliziert und nicht mit c . Ein konkreter Fall aus Beispiel 9.1.3: Für $x = y = 1$ ist $f(2, 2) = 4$, während eine lineare Abbildung $2f(1, 1) = 2$ liefern müsste. Beides zugleich, bilinear und linear, ist einzig die Nullabbildung: Aus $c^2 f(\mathbf{v}, \mathbf{w}) = c f(\mathbf{v}, \mathbf{w})$ für alle c folgt $f(\mathbf{v}, \mathbf{w}) = \mathbf{0}$ für jedes Paar.

4. Halten wir in einer multilinearen Abbildung alle Argumente bis auf das i -te fest, so ist die verbleibende Abbildung $V_i \rightarrow W$ linear.

Antwort: Wahr

Das ist Definition 9.1.1, in Worte gefasst. Sie fordert genau diese eine Bedingung, und zwar für jedes i einzeln.

5. Zwei verschiedene Koeffizienten-Arrays können dieselbe multilineare Abbildung $T : \mathbb{R}^{n_1} \times \dots \times \mathbb{R}^{n_k} \rightarrow \mathbb{R}^m$ darstellen.

Antwort: Falsch

Satz 9.1.7 sichert die Eindeutigkeit zu, und der Grund steht im Anschluss an ihn: $a_{i_1, \dots, i_k, j}$ ist die j -te Komponente von $T(\mathbf{e}_{i_1}, \dots, \mathbf{e}_{i_k})$, also durch T selbst festgelegt. Wer die Koeffizienten ändert, ändert die Abbildung.

Vertiefung: vgl. MML §2.7 zu linearen Abbildungen und MML §3.2, wo bilineare Abbildungen als Vorstufe des Skalarprodukts eingeführt werden.

9.2 Tensoren

Der Darstellungssatz aus Abschnitt 9.1 (Satz 9.1.7) hat multilineare Abbildungen vollständig auf eine Liste von Zahlen zurückgeführt. Diese Liste ist mehr als ein Nebenprodukt der Rechnung. Sie ist das Objekt, um das sich der Rest des Kapitels dreht, und sie hat einen Namen. Bevor wir ihn vergeben, sehen wir nach, was aus der Liste im einfachsten Fall wird.

Der Fall $k = 1$ liefert die Matrix

Setzen wir im Darstellungssatz $k = 1$. Dann hat T nur ein Argument, ist also eine gewöhnliche lineare Abbildung $T : \mathbb{R}^n \rightarrow \mathbb{R}^m$, und von der langen Indexliste bleiben zwei Indizes übrig, $1 \leq i \leq n$ und $1 \leq j \leq m$:

$$T(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^m a_{i,j} x_i \mathbf{e}_j. \quad (9.2.1)$$

Die Doppelsumme lässt sich umsortieren. Der Faktor x_i hängt nicht von j ab, wir dürfen ihn vor die innere Summe ziehen. Was dann in der Klammer steht, ist ein fester Vektor des $\mathbb{R}^m$, der nur noch von i abhängt:

$$\mathbf{a}_i := \sum_{j=1}^m a_{i,j} \mathbf{e}_j = \begin{pmatrix} a_{i,1} \\ \vdots \\ a_{i,m} \end{pmatrix} \in \mathbb{R}^m, \quad \text{also} \quad T(\mathbf{x}) = \sum_{i=1}^n x_i \mathbf{a}_i.$$

Rechts steht eine Summe von Vektoren, gewichtet mit den Komponenten von $\mathbf{x}$. Stellen wir die $\mathbf{a}_i$ als Spalten nebeneinander, so ist das genau das Matrix-Vektor-Produkt: Mit $\mathbf{A} = (\mathbf{a}_1 \mid \cdots \mid \mathbf{a}_n) \in \mathbb{R}^{m \times n}$ gilt

$$T(\mathbf{x}) = \mathbf{A} \mathbf{x}.$$

Der Darstellungssatz reproduziert für $k = 1$ also die vertraute Tatsache, dass sich jede lineare Abbildung $\mathbb{R}^n \rightarrow \mathbb{R}^m$ als Matrix schreiben lässt. Eine Kleinigkeit lohnt dabei einen zweiten Blick.

Bemerkung 9.2.1 (Von der Koeffizientenfamilie zur Matrix)

Der Darstellungssatz nummeriert die Koeffizienten mit dem Eingangsindex vorn und dem Ausgangsindex hinten, also $a_{i,j}$ mit i für die Komponente von $\mathbf{x}$ und j für die Komponente des Bildvektors. In der Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ stehen sie andersherum: Der Ausgangsindex ist der Zeilenindex, der Eingangsindex der Spaltenindex,

$$(\mathbf{A})_{j,i} = a_{i,j}.$$

Anders herum passt das Produkt $\mathbf{A} \mathbf{x}$ gar nicht zusammen, denn $\mathbf{x}$ hat n Komponenten und $\mathbf{A}$ braucht dafür n Spalten. Wer die Familie unbesehen als Matrix liest, erhält $\mathbf{A}^\top$. Die Reihenfolge der Indexpositionen ist Konvention, ihre Bedeutung nicht.

Die Ansammlung der nm Koeffizienten nennen wir eine Matrix. Sie hat zwei Indexpositionen, und deshalb sprechen wir bei ihr von zwei „Dimensionen“; warum die Anführungszeichen nötig sind, klärt Bemerkung 9.2.6. Der Darstellungssatz sagt aber mehr: Zu jedem k gehört eine Ansammlung von Koeffizienten mit $k + 1$ Indizes, bei $m = 1$ also mit k , und jede solche Ansammlung beschreibt umgekehrt eine multilineare Abbildung. Sehen wir uns den Fall $k = 2$ an, bevor wir dem Objekt einen Namen geben.

Ein bilineares Beispiel

Beispiel 9.2.2 (Eine bilineare Abbildung auf $\mathbb{R}^2$ mal $\mathbb{R}^2$)

Wir betrachten $T : \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ mit

$$T(\mathbf{x}, \mathbf{y}) = 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2.$$

Hier ist $k = 2$, $n_1 = n_2 = 2$ und $m = 1$. Wegen $m = 1$ nimmt der Ausgangsindex nur den Wert 1 an, und $\mathbf{e}_1 = 1$ ist die Zahl Eins; wir lassen ihn weg. Übrig bleiben vier Koeffizienten a_{i_1, i_2} , die wir direkt ablesen:

$$a_{1,1} = 2, \quad a_{1,2} = 3, \quad a_{2,1} = -1, \quad a_{2,2} = 4 \quad \Rightarrow \quad \mathbf{A} = \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix}.$$

Probe. Die Darstellungsformel für $k = 2$ und $m = 1$ lautet

$$\begin{aligned} T(\mathbf{x}, \mathbf{y}) &= \sum_{i_1=1}^2 \sum_{i_2=1}^2 a_{i_1, i_2} x_{i_1} y_{i_2} \\ &= a_{1,1} x_1 y_1 + a_{1,2} x_1 y_2 + a_{2,1} x_2 y_1 + a_{2,2} x_2 y_2 \\ &= 2x_1 y_1 + 3x_1 y_2 - x_2 y_1 + 4x_2 y_2, \end{aligned}$$

also wieder die Abbildung, von der wir ausgegangen sind. Anders als in Bemerkung 9.2.1 gibt es hier keine Frage nach der Indexreihenfolge, denn beide Indizes sind Eingangsindizes: i_1 gehört zum ersten, i_2 zum zweiten Argument.

Numerisch. Für $\mathbf{x} = (1, 2)^\top$ und $\mathbf{y} = (3, 4)^\top$ ist

$$T(\mathbf{x}, \mathbf{y}) = 2 \cdot 1 \cdot 3 + 3 \cdot 1 \cdot 4 - 1 \cdot 2 \cdot 3 + 4 \cdot 2 \cdot 4 = 6 + 12 - 6 + 32 = 44.$$

In Matrixschreibweise. Die Doppelsumme ist der Ausdruck $\mathbf{x}^\top \mathbf{A} \mathbf{y}$, ausgeschrieben. Eine bilineare Abbildung mit Werten in $\mathbb{R}$ heißt eine *Bilinearform*, und genau das ist T . Setzen wir beide Argumente gleich, so entsteht daraus die quadratische Form $\mathbf{x} \mapsto \mathbf{x}^\top \mathbf{A} \mathbf{x}$, eine Abbildung mit nur noch einem Argument.

$$T(\mathbf{x}, \mathbf{y}) = \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 & 11 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 0 \cdot 3 + 11 \cdot 4 = 44.$$

Tensoren

Für $k = 1$ heißt die Koeffizientenansammlung Matrix, für $k = 2$ ebenfalls, denn bei $m = 1$ blieben auch dort zwei Indizes übrig. Bei drei Eingangsvektoren brauchen wir drei Indizes, bei k Vektoren und mehrdimensionalem Bildbereich noch einen mehr. Für all das gibt es einen gemeinsamen Namen.

Definition 9.2.3 (Tensor)

Seien $n_1, \dots, n_k \in \mathbb{N}$. Ein *Tensor* (tensor) der *Stufe* k und des Formats $n_1 \times \dots \times n_k$ ist eine durch k Indizes indizierte Familie reeller Zahlen

$$\mathbf{A} = (a_{i_1, \dots, i_k})_{1 \leq i_1 \leq n_1, \dots, 1 \leq i_k \leq n_k} \in \mathbb{R}^{n_1 \times \dots \times n_k},$$

also eine Abbildung, die jedem Indextupel $(i_1, \dots, i_k)$ aus $\{1, \dots, n_1\} \times \dots \times \{1, \dots, n_k\}$ genau eine reelle Zahl $a_{i_1, \dots, i_k}$ zuordnet.

Bemerkung 9.2.4 (Stufe, und warum eine Menge es nicht tut)

Die Stufe zählt Indexpositionen, nicht Einträge. Die ersten Fälle kennen wir alle:

Stufe k	Objekt	Format	Einträge
0	Skalar		1
1	Vektor	n	n
2	Matrix	$n_1 \times n_2$	$n_1 n_2$
3	Stapel von Matrizen	$n_1 \times n_2 \times n_3$	$n_1 n_2 n_3$

Gelegentlich liest man, ein Tensor sei eine „geordnete Menge“ reeller Zahlen. Das trifft es nicht ganz. Eine Menge kennt weder Reihenfolge noch Vielfachheit, es ist ja $\{1, 1, 2\} = \{2, 1\}$. Worauf es ankommt, ist die Zuordnung: zu jedem Indextupel gehört genau eine Zahl. Derselbe Zahlenvorrat, anders angeordnet, ergibt einen anderen Tensor. Wir sagen deshalb *indizierte Familie* oder schlicht *Anordnung*.

Tensoren derselben Größe lassen sich addieren und mit Zahlen strecken, und zwar Eintrag für Eintrag, so wie wir es von Vektoren und Matrizen gewohnt sind. Damit liegt schon eine bekannte Struktur vor.

Satz 9.2.5 (Der Raum aller Tensoren eines Formats)

Seien $n_1, \dots, n_k \in \mathbb{N}$ fest. Mit der eintragsweisen Addition

$$(A + B)_{i_1, \dots, i_k} = a_{i_1, \dots, i_k} + b_{i_1, \dots, i_k}$$

und der eintragsweisen Multiplikation mit Skalaren $(cA)_{i_1, \dots, i_k} = c a_{i_1, \dots, i_k}$ ist $\mathbb{R}^{n_1 \times \dots \times n_k}$ ein Vektorraum über $\mathbb{R}$. Seine Dimension ist $n_1 n_2 \cdots n_k$.

Vertiefung: Warum Tensoren eines festen Formats einen Vektorraum bilden

Beweis.

- (1) Jedes Vektorraumaxiom ist eine Gleichung zwischen Tensoren, und zwei Tensoren sind genau dann gleich, wenn sie an jeder Indexstelle übereinstimmen. Beide Verknüpfungen wirken eintragsweise, also bleibt an jeder Indexstelle eine Gleichung zwischen reellen Zahlen stehen. Neutrales Element ist der Nulltensor, das Negative von A der Tensor mit den Einträgen $-a_{i_1, \dots, i_k}$.

Assoziativität, Kommutativität und die Distributivgesetze gelten in $\mathbb{R}$; sie übertragen sich Stelle für Stelle, etwa $(c(A + B))_{i_1, \dots, i_k} = c a_{i_1, \dots, i_k} + c b_{i_1, \dots, i_k} = (cA + cB)_{i_1, \dots, i_k}$

- (2) Für die Dimension geben wir eine Basis an. Zu jedem Indextupel $(i_1, \dots, i_k)$ sei $\mathbf{E}^{(i_1, \dots, i_k)}$ der Tensor, der an dieser Stelle eine 1 und sonst überall 0 stehen hat. Jeder Tensor schreibt sich damit als

$$A = \sum_{i_1=1}^{n_1} \cdots \sum_{i_k=1}^{n_k} a_{i_1, \dots, i_k} \mathbf{E}^{(i_1, \dots, i_k)},$$

und die Koeffizienten dieser Darstellung sind die Einträge von A , also eindeutig. Die $\mathbf{E}^{(i_1, \dots, i_k)}$ erzeugen den Raum somit und sind linear unabhängig. Ihre Anzahl ist die Zahl der Indextupel, und die ist $n_1 n_2 \cdots n_k$.

eindeutige Darstellbarkeit ist genau die Basiseigenschaft: Existenz gibt das Erzeugendensystem, Eindeutigkeit die lineare Unabhängigkeit

□

Bemerkung 9.2.6 (Stufe und Dimension sind zwei verschiedene Zahlen)

Eine 4×4 -Matrix hat die Stufe 2, aber $\mathbb{R}^{4 \times 4}$ hat nach Satz 9.2.5 die Dimension $4^2 = 16$. Ein Tensor des Formats $4 \times 4 \times 4$ hat die Stufe 3, und der zugehörige Raum die Dimension $4^3 = 64$. Wer die Indexpositionen „Dimensionen“ nennt, meint also etwas anderes als die Dimension des Vektorraums. Im maschinellen Lernen heißen die Indexpositionen meist *Achsen* (axes), was die Verwechslung vermeidet.

Die Basisdarstellung aus dem Beweis erklärt nebenbei einen Handgriff, der in jeder Programmbibliothek vorkommt: Zählen wir die Indextupel in einer fest gewählten Reihenfolge durch, so wird aus jedem Tensor ein Vektor mit $n_1 n_2 \cdots n_k$ Komponenten und aus jeder Rechnung im Tensorraum eine Rechnung im $\mathbb{R}^{n_1 n_2 \cdots n_k}$. Die Operation heißt dort *reshape* oder *flatten*; verloren geht dabei nur die Information, welcher Index welche Bedeutung hatte. Für Matrizen bekommt sie in Abschnitt 9.5 als spaltenweise Vektorisierung einen Namen und eine Rechenregel.

Interaktiv: Ein Stufe-3-Tensor, Scheibe für Scheibe

Welche vollständige Matrix erhalten wir, wenn wir die dritte Indexposition k festhalten?

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, entspricht jede Wahl von k einer vollständigen Matrixscheibe desselben Zahlenfelds.

Anwendungen: Bilder, Stapel, Feature-Maps

Bis hierher ist ein Tensor ein Buchhaltungsobjekt. Warum er einen eigenen Namen und inzwischen sogar eigene Rechenwerke bekommt, zeigt der Blick auf Bilddaten. Ein Graustufenbild hält je Bildpunkt eine einzige Zahl fest und ist damit eine Matrix. Farbe braucht drei Zahlen je Pixel; ein Farbbild der Größe 224×224 wird deshalb zu einem Tensor der Stufe 3, $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 3}$, mit $224 \cdot 224 \cdot 3 = 150\,528$ Zahlen. Neuronale Netze verarbeiten solche Bilder bündelweise, etwa 32 Stück auf einmal, und aus dem Stapel wird ein Tensor der Stufe 4; dieselbe vierte Indexposition zählt bei einem Video die Einzelbilder in der Zeit. Eine Faltungsschicht (convolutional layer) eines faltenden Netzes bildet dann Tensoren wieder auf Tensoren ab.

Rechnungen dieser Art bestehen aus sehr vielen gleichartigen Multiplikationen und Additionen auf regelmäßig angeordneten Zahlen. Dafür lassen sich spezialisierte Rechenwerke bauen, die solche Blockoperationen am Stück ausführen: Googles TPUs und NVIDIAs Tensor Cores sind genau darauf zugeschnitten. Für das maschinelle Lernen ist diese Hardware keine Randnotiz, sondern eine Voraussetzung.

Interaktiv: Farbbild als Kanäle

Wie setzen sich die drei Zahlen eines Pixels zu seiner sichtbaren Farbe zusammen?

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, sind die drei Kanalscheiben keine drei Bilder, sondern drei Komponenten jedes Pixels desselben Bildes.

Vertiefung: Bilder, Stapel und Feature-Maps im Detail

Beispiel 9.2.7 (Ein Farbbild als Tensor der Stufe 3)

Beim Graustufenbild läuft eine Indexrichtung über die Pixel in der Waagerechten, die andere über die Pixel in der Senkrechten. Beim Farbbild $\mathbf{I} \in \mathbb{R}^{224 \times 224 \times 3}$ nummeriert der dritte Index die Kanäle:

- $\mathbf{I}_{i,j,1}$: Rot-Intensität an der Position (i, j) , ein Wert zwischen 0 und 255,
- $\mathbf{I}_{i,j,2}$: Grün-Intensität an derselben Position,
- $\mathbf{I}_{i,j,3}$: Blau-Intensität an derselben Position.

Das sind

$$224 \cdot 224 \cdot 3 = 150\,528 \approx 150 \cdot 10^3$$

Zahlen für ein einziges Bild. Welche Indexposition wofür steht, ist dabei reine Verabredung, und die Verabredungen gehen auseinander: Manche Bibliothek stellt den Kanalindex nach vorn, manche lässt ihn hinten, und ob zuerst die Zeile oder zuerst die Spalte gezählt wird, ist ebenfalls nicht überall gleich. Für die Mathematik ist das gleichgültig, für den Code nicht.

Beispiel 9.2.8 (Ein Stapel Bilder als Tensor der Stufe 4)

Ein faltendes Netz (convolutional neural network, CNN) bekommt typischerweise 32 Bilder auf einmal vorgesetzt, also

$$\mathbf{B} \in \mathbb{R}^{32 \times 224 \times 224 \times 3},$$

wobei die vier Indexpositionen der Reihe nach die Nummer des Bildes im Stapel (*Batch-Größe*), die Höhe, die Breite und den Farbkanal angeben. Nachgezählt sind das

$$32 \cdot 224 \cdot 224 \cdot 3 = 4\,816\,896 \approx 4,8 \cdot 10^6$$

Zahlen. Bei einfacher Genauigkeit mit 4 Byte je Zahl belegt allein die Eingabe eines einzigen Rechenschritts damit rund 19 Millionen Byte. Ob die vierte Indexposition die Bilder eines Stapels oder die eines Videos in der Zeit zählt, steht ihr nicht an; wir müssen es dazusagen.

Beispiel 9.2.9 (Feature-Maps: eine Abbildung von Tensoren auf Tensoren)

Eine Faltungsschicht (convolutional layer) eines CNN bildet Tensoren auf Tensoren ab, etwa

$$c : \mathbb{R}^{32 \times 224 \times 224 \times 3} \rightarrow \mathbb{R}^{32 \times 56 \times 56 \times 64}.$$

Sie senkt die räumliche Auflösung von 224×224 auf 56×56 und erhöht dabei die Zahl der Kanäle von 3 auf 64. Diese Kanäle stehen jetzt nicht mehr für Farben, sondern für Merkmale (features): Jede der 64 Ebenen hält für jede Position fest, wie stark ein bestimmtes lokales Muster dort anspricht, und heißt deshalb *Feature-Map*.

Naheliegender wäre die Vermutung, die Schicht mache den Tensor kleiner. Zählen wir nach:

$$32 \cdot 56 \cdot 56 \cdot 64 = 6\,422\,528 \approx 6,4 \cdot 10^6,$$

also mehr als die 4 816 896 Einträge des Eingangs. Die Auflösung fällt zwar um den Faktor 16, die Kanalzahl wächst aber um den Faktor $\frac{64}{3} = \frac{4}{3}$. Anschaulich wird das Prinzip an den Feature-Maps aus Block 3 des VGG16-Modells: Jede einzelne sieht aus wie ein grobes Graustufenbild, hell an den Stellen, an denen das jeweilige Muster kräftig anspricht.

64 ausgewählte Feature-Maps aus Block 3 von VGG16 für dasselbe Eingabebild. Der Block erzeugt insgesamt 256 Kanäle. Quelle: Machine Learning Mastery.

Was an einer Faltungsschicht linear ist. Die Abbildung c hat nur ein Argument, ist also keine multilineare Abbildung im Sinn von Abschnitt 9.1. Bei fest gewählten Gewichten ist sie schlicht linear im Eingangstensor. Interessant wird es, wenn wir die Gewichte als zweites Argument hinzunehmen: Jeder Ausgangseintrag ist eine Summe von Produkten aus je einem Gewicht und je einem Eingangseintrag, die Zuordnung (Gewichte, Eingang) $\mapsto$ Ausgang ist damit bilinear, also der Fall $k = 2$ des Darstellungssatzes. Zu einer vollständigen Schicht gehören meist noch ein additiver Term und eine nichtlineare Funktion, die auf jeden Eintrag einzeln wirkt. Erst diese Funktion macht aus einem neuronalen Netz mehr als eine lange Kette linearer Abbildungen.

1. Ein Farbbild mit 224×224 Pixeln und drei Farbkanälen ist ein Tensor der Stufe 3 mit 150 528 Einträgen.

Antwort: Wahr

Es ist $224 \cdot 224 \cdot 3 = 150\,528$, also rund $150 \cdot 10^3$ Zahlen (Beispiel 9.2.7). Die drei Indexpositionen sind die beiden Pixelrichtungen und der Farbkanal; in welcher Reihenfolge sie stehen, ist Verabredung und von Bibliothek zu Bibliothek verschieden.

2. Weil die Faltungsschicht $c : \mathbb{R}^{32 \times 224 \times 224 \times 3} \rightarrow \mathbb{R}^{32 \times 56 \times 56 \times 64}$ die Auflösung von 224×224 auf 56×56 senkt, hat ihr Ausgang weniger Einträge als ihr Eingang.

Antwort: Falsch

Nachgezählt: $32 \cdot 224 \cdot 224 \cdot 3 = 4\,816\,896$ gegen $32 \cdot 56 \cdot 56 \cdot 64 = 6\,422\,528$. Die Auflösung fällt um den Faktor 16, die Kanalzahl wächst um den Faktor $64/3$, zusammen wächst die Zahl der Einträge um den Faktor $4/3$ (Beispiel 9.2.9).

Selbsttest

1. Wie viele Einträge zeigt der Zahlen-Stapel des Widgets bei vier Scheiben einer 4×4 -Matrix?

Lösung: 64

Die zusätzliche Indexposition multipliziert die Zahl der Einträge mit der Zahl der Scheiben.

2. Setzen wir im Darstellungssatz $k = 1$, so ist die Koeffizientenansammlung eine Matrix $\mathbf{A}$, und es gilt $T(\mathbf{x}) = \mathbf{A}\mathbf{x}$.

Antwort: Wahr

Das ist die Rechnung zu (9.2.1): Die innere Summe $\sum_j a_{ij} \mathbf{e}_j$ ist ein fester Vektor $\mathbf{a}_i \in \mathbb{R}^m$, und $\sum_i x_i \mathbf{a}_i$ ist das Matrix-Vektor-Produkt mit $\mathbf{A} = (\mathbf{a}_1 \mid \dots \mid \mathbf{a}_n)$. Auf die Reihenfolge der Indizes müssen wir dabei achten, siehe Bemerkung 9.2.1.

3. Die Stufe eines Tensors ist die Anzahl seiner Einträge.

Antwort: Falsch

Die Stufe zählt die Indexpositionen. Ein Tensor des Formats $4 \times 4 \times 4$ hat die Stufe 3, aber $4^3 = 64$ Einträge (Definition 9.2.3).

4. Der Vektorraum $\mathbb{R}^{4 \times 4 \times 4}$ hat die Dimension 3.

Antwort: Falsch

Seine Dimension ist $4 \cdot 4 \cdot 4 = 64$, denn so viele Einheitstensoren bilden nach Satz 9.2.5 eine Basis. Die 3 ist die Stufe, also die Zahl der Indexpositionen; Bemerkung 9.2.6 hält die beiden Zahlen auseinander.

5. Ein Tensor ist durch die Menge seiner Einträge festgelegt.

Antwort: Falsch

Eine Menge kennt weder Reihenfolge noch Vielfachheit. Vertauschen wir zwei verschiedene

Einträge einer Matrix, so bleibt die Menge der vorkommenden Zahlen dieselbe, die Matrix ist aber eine andere. Festgelegt ist ein Tensor durch die Zuordnung von Indextupeln zu Zahlen (Definition 9.2.3 und Bemerkung 9.2.4).

Vertiefung: vgl. MML §5.4, wo mehrdimensionale Zahlenfelder beim Ableiten matrixwertiger Ausdrücke ganz von selbst auftreten.

9.3 Produkte von Tensoren

Aus zwei Vektoren eine Zahl zu machen, ist uns vertraut: Das leistet das Skalarprodukt. In diesem Abschnitt gehen wir in die Gegenrichtung und bauen aus zwei Objekten ein größeres. Aus zwei Vektoren wird eine Matrix, aus zwei Matrizen ein Tensor der Stufe 4 oder, anders angeordnet, eine sehr große Matrix. In der Literatur tragen alle drei Konstruktionen dasselbe Zeichen $\otimes$; der Umgang mit dieser Doppelbelegung gehört zum Thema. Wir behalten $\otimes$ dem äußeren Produkt und dem Tensorprodukt vor und schreiben die dritte Konstruktion, das Kroneckerprodukt, als $\otimes_K$.

Das äußere Produkt

Definition 9.3.1 (Äußeres Produkt)

Das *äußere Produkt* (outer product) zweier Vektoren $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ ist die Matrix

$$\mathbf{v} \otimes \mathbf{w} := \mathbf{v} \mathbf{w}^\top = \begin{pmatrix} v_1 w_1 & \cdots & v_1 w_n \\ \vdots & & \vdots \\ v_m w_1 & \cdots & v_m w_n \end{pmatrix} \in \mathbb{R}^{m \times n}, \quad (9.3.1)$$

also die Matrix mit den Einträgen $(\mathbf{v} \otimes \mathbf{w})_{ij} = v_i w_j$.

Bemerkung 9.3.2 (Inneres und äußeres Produkt)

Das Skalarprodukt heißt auch *inneres Produkt* (inner product), und der Namensvetter macht den Unterschied deutlich. Nebeneinandergestellt sind die beiden Abbildungen

$$\langle \cdot, \cdot \rangle : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}, \quad \langle \mathbf{v}, \mathbf{w} \rangle = \mathbf{v}^\top \mathbf{w},$$

$$\otimes : \mathbb{R}^m \times \mathbb{R}^n \rightarrow \mathbb{R}^{m \times n}, \quad \mathbf{v} \otimes \mathbf{w} = \mathbf{v} \mathbf{w}^\top.$$

Drei Unterschiede lohnen einen Blick. Das innere Produkt verlangt zwei Vektoren gleicher Länge, das äußere verträgt beliebige Längen. Beim inneren Produkt steht das Transponat links, beim äußeren rechts. Und heraus kommt einmal eine einzige Zahl, einmal eine ganze Matrix aus mn Zahlen.

Wer sich die Formeln nicht merken mag, zählt die Formate ab: In $\mathbf{v}^\top \mathbf{w}$ multiplizieren wir $(1 \times n)$ mit $(n \times 1)$, die langen Dimensionen treffen sich innen und verschwinden. In $\mathbf{v} \mathbf{w}^\top$ multiplizieren wir $(m \times 1)$ mit $(1 \times n)$, die langen Dimensionen stehen außen und bleiben stehen.

Beispiel 9.3.3 (Äußeres Produkt zweier Vektoren)

Für

$$\mathbf{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^2 \quad \text{und} \quad \mathbf{w} = \begin{pmatrix} -2 \\ 3 \\ -11 \end{pmatrix} \in \mathbb{R}^3$$

ist

$$\mathbf{v} \otimes \mathbf{w} = \mathbf{v} \mathbf{w}^\top = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} -2 & 3 & -11 \end{pmatrix} = \begin{pmatrix} -2 & 3 & -11 \\ -4 & 6 & -22 \end{pmatrix} \in \mathbb{R}^{2 \times 3}.$$

Die zweite Zeile ist das Doppelte der ersten, denn $v_2 = 2v_1$. Genauso ist jede Spalte ein Vielfaches von $\mathbf{v} = (1, 2)^\top$. Das ist kein Zufall dieses Beispiels, sondern die Regel.

Eine Rang-1-Matrix und was sie tut

Wie hängen die beiden Faktoren einer Rang-1-Matrix mit ihrem Kern und ihrem Bild zusammen?

Interaktiv: Ein Bildraum als Gerade

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, bestimmt $\mathbf{v}$ die Bildgerade und $\mathbf{w}$ die dazu orthogonale Kernrichtung einer Rang-1-Matrix.

Satz 9.3.4 (Eigenschaften des äußeren Produkts)

Seien $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ beide vom Nullvektor verschieden, und sei $\mathbf{M} = \mathbf{v} \otimes \mathbf{w}$. Dann gilt:

1. Die j -te Spalte von $\mathbf{M}$ ist $w_j \mathbf{v}$, die i -te Zeile ist $v_i \mathbf{w}^\top$.
2. Für alle $\mathbf{x} \in \mathbb{R}^n$ ist $\mathbf{M}\mathbf{x} = \mathbf{v} \langle \mathbf{w}, \mathbf{x} \rangle$.
3. $\text{col}(\mathbf{M}) = \text{span}\{\mathbf{v}\}$, das Bild ist also eindimensional, und $\text{rang}(\mathbf{M}) = 1$.
4. Der Kern ist $\text{Kern}(\mathbf{M}) = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{w}^\top \mathbf{x} = 0\}$, also die Hyperebene durch den Ursprung senkrecht zu $\mathbf{w}$; sie hat die Dimension $n - 1$.

Vertiefung: Beweis der Eigenschaften des äußeren Produkts

Beweis.

- (1) Zu (1): Nach (9.3.1) steht an der Stelle (i, j) der Eintrag $v_i w_j$. In der j -ten Spalte steckt damit in jedem Eintrag derselbe Faktor w_j , sie ist also $w_j \mathbf{v}$; für die Zeilen läuft es genauso mit v_i .

Ausklammern eines gemeinsamen Faktors, Eintrag für Eintrag

- (2) Zu (2): Wir setzen die Definition ein und klammern anders:

$$\mathbf{M}\mathbf{x} = (\mathbf{v} \mathbf{w}^\top) \mathbf{x} = \mathbf{v} (\mathbf{w}^\top \mathbf{x}) = \mathbf{v} \langle \mathbf{w}, \mathbf{x} \rangle.$$

Das Matrixprodukt sagt uns damit, was diese lineare Abbildung tut: Sie misst $\mathbf{x}$ gegen $\mathbf{w}$ und trägt das Ergebnis als Vielfaches von $\mathbf{v}$ ab.

Assoziativität des Matrixprodukts; $\mathbf{w}^\top \mathbf{x}$ ist eine 1×1 -Matrix, also eine Zahl, und Zahlen dürfen wir vor den Vektor ziehen

- (3) Zu (3): Nach Schritt 2 ist jedes Bild ein Vielfaches von $\mathbf{v}$, also $\text{col}(\mathbf{M}) \subseteq \text{span}\{\mathbf{v}\}$. Umgekehrt erreichen wir jedes Vielfache: Für $\mathbf{x} = t \mathbf{w} / \|\mathbf{w}\|^2$ ist $\langle \mathbf{w}, \mathbf{x} \rangle = t$ und damit $\mathbf{M}\mathbf{x} = t \mathbf{v}$. Wegen $\mathbf{v} \neq \mathbf{0}$ hat dieser Bildraum die Dimension 1.

hier brauchen wir $\mathbf{w} \neq \mathbf{0}$, sonst ist die Division nicht erlaubt und $\mathbf{M}$ die Nullmatrix; der Rang ist die Dimension des Bildes

- (4) Zu (4): Nach Schritt 2 gilt $\mathbf{M}\mathbf{x} = \mathbf{0}$ genau dann, wenn $\langle \mathbf{w}, \mathbf{x} \rangle \mathbf{v} = \mathbf{0}$ ist, und wegen $\mathbf{v} \neq \mathbf{0}$ genau dann, wenn $\mathbf{w}^\top \mathbf{x} = 0$ ist. Das ist die Menge aller zu $\mathbf{w}$ orthogonalen Vektoren. Die Probe liefert der Rangsatz: $1 + (n - 1) = n$.

ein Vielfaches $c \mathbf{v}$ ist genau dann der Nullvektor, wenn $c = 0$ ist; der Rangsatz verlangt $\text{rang}(\mathbf{M}) + \dim \text{Kern}(\mathbf{M}) = n$

□

Damit ist das äußere Produkt der Prototyp einer Rang-1-Matrix. Es gilt sogar die Umkehrung: Jede Matrix vom Rang 1 lässt sich als $\mathbf{v} \otimes \mathbf{w}$ schreiben, denn hat ihr Spaltenraum die Dimension 1, so ist jede Spalte ein Vielfaches eines festen $\mathbf{v}$, und die Vielfachen sammeln wir in $\mathbf{w}$.

Wo äußere Produkte auftauchen

Bemerkung 9.3.5 (Die SVD als Summe äußerer Produkte)

In Abschnitt 6.4 haben wir die Singulärwertzerlegung ausmultipliziert und für $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r$ die Summenform

$$\mathbf{A} = \mathbf{U}_r \Sigma_r \mathbf{V}_r^\top = \sum_{i=1}^r \sigma_i \mathbf{u}_i \otimes \mathbf{v}_i$$

erhalten (Satz 6.4.2; dort noch als $\mathbf{u}_i \mathbf{v}_i^\top$ geschrieben). Jetzt lesen wir sie mit Satz 9.3.4: Jeder Summand ist eine Rang-1-Matrix, ihr Bild ist die Gerade $\text{span}\{\mathbf{u}_i\}$, und σ_i sagt, wie stark dieser Anteil zählt. Die SVD zerlegt eine Matrix also in r denkbar einfache Bausteine. Brechen wir die Summe nach k Termen ab, entsteht die beste Rang- k -Approximation, wie der Satz von Eckart, Young und Mirsky in Abschnitt 6.4 zeigt.

Bemerkung 9.3.6 (Kovarianzmatrizen sind Mittel äußerer Produkte)

Sei $\mathbf{x}$ ein Zufallsvektor im $\mathbb{R}^n$ mit Erwartungswert $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$. Die Kovarianzmatrix ist definiert als

$$\text{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu}) \otimes (\mathbf{x} - \boldsymbol{\mu})],$$

also als Erwartungswert eines äußeren Produkts, gebildet Eintrag für Eintrag. Ausmultiplizieren und die Linearität des Erwartungswerts liefern die Verschiebungsformel

$$\text{Cov}(\mathbf{x}) = \mathbb{E}[\mathbf{x} \otimes \mathbf{x}] - \boldsymbol{\mu} \otimes \boldsymbol{\mu} = \mathbb{E}[\mathbf{x} \otimes \mathbf{x}] - \mathbb{E}[\mathbf{x}] \otimes \mathbb{E}[\mathbf{x}],$$

denn $\mathbb{E}[\mathbf{x} \otimes \boldsymbol{\mu}] = \boldsymbol{\mu} \otimes \boldsymbol{\mu} = \mathbb{E}[\boldsymbol{\mu} \otimes \mathbf{x}]$, und von den drei gleichen Termen bleibt einer mit negativem Vorzeichen übrig. Die Formel ist die mehrdimensionale Fassung von $\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$; numerisch ist sie aus demselben Grund heikel wie dort, siehe die Auslöschung in Abschnitt 2.1.

Für jede einzelne Realisierung ist $(\mathbf{x} - \boldsymbol{\mu}) \otimes (\mathbf{x} - \boldsymbol{\mu})$ nach Satz 9.3.4 eine Rang-1-Matrix; erst das Mitteln über viele Richtungen macht die Kovarianzmatrix im Regelfall zu einer Matrix von vollem Rang. Genauso gebaut ist die empirische Kovarianzmatrix aus N Beobachtungen, $\hat{\Sigma} = \frac{1}{N-1} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}}) \otimes (\mathbf{x}_i - \bar{\mathbf{x}})$, und weil die zentrierten Vektoren sich zu null summieren, ist $\text{rang}(\hat{\Sigma}) \leq N - 1$: Bei weniger Beobachtungen als Dimensionen ist die Schätzung stets singulär.

Vertiefung: Attention: Skalarprodukte in Massen

Der Aufmerksamkeitsmechanismus (attention) moderner Sprachmodelle rechnet mit drei Matrizen: $\mathbf{Q} \in \mathbb{R}^{n_q \times d_k}$ mit den Anfragen (queries) $\mathbf{q}_i^\top$ in den Zeilen, $\mathbf{K} \in \mathbb{R}^{n_k \times d_k}$ mit den Schlüsseln (keys) $\mathbf{k}_j^\top$ und $\mathbf{V} \in \mathbb{R}^{n_v \times d_v}$ mit den Werten (values). Die übliche Form lautet

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{SoftMax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V},$$

wobei SoftMax zeilenweise wirkt und das Ergebnis in $\mathbb{R}^{n_q \times d_v}$ liegt. Der interessante Faktor ist $\mathbf{Q}\mathbf{K}^\top$: Sein Eintrag (i, j) ist das innere Produkt $\langle \mathbf{q}_i, \mathbf{k}_j \rangle$, das Modell vergleicht also jede Anfrage mit jedem Schlüssel. Als ganze Matrix ist derselbe Ausdruck eine Summe äußerer Produkte, diesmal über die d_k Spalten,

$$\mathbf{Q}\mathbf{K}^\top = \sum_{l=1}^{d_k} \mathbf{q}^{(l)} \otimes \mathbf{k}^{(l)},$$

mit $\mathbf{q}^{(l)}$ und $\mathbf{k}^{(l)}$ als l -ten Spalten von $\mathbf{Q}$ und $\mathbf{K}$. Der Rang der Scorematrix $\mathbf{Q}\mathbf{K}^\top$ ist damit höchstens d_k , und weil d_k in der Praxis viel kleiner ist als die Zahl der Zeilen, steckt in den Scores eine Zerlegung von niedrigem Rang, genau wie bei der SVD. Diese Aussage gilt vor der Softmax-Transformation: Softmax ist nicht linear und erhält die Rangschranke im Allgemeinen nicht, die Matrix der Attention-Gewichte kann also vollen Rang haben.

Das Tensorprodukt

Das äußere Produkt macht aus zwei Vektoren eine Matrix, aus zwei Objekten der Stufe 1 also eines der Stufe 2. Diese Idee lässt sich auf beliebige Stufen übertragen, und die Vorschrift bleibt dieselbe: Jeder Eintrag des einen Tensors wird mit jedem Eintrag des anderen multipliziert.

Definition 9.3.7 (Tensorprodukt)

Seien $\mathbf{A} \in \mathbb{R}^{m_1 \times \dots \times m_p}$ ein Tensor der Stufe p und $\mathbf{B} \in \mathbb{R}^{n_1 \times \dots \times n_q}$ einer der Stufe q . Ihr Tensorprodukt ist der Tensor $\mathbf{A} \otimes \mathbf{B} = \mathbf{C} \in \mathbb{R}^{m_1 \times \dots \times m_p \times n_1 \times \dots \times n_q}$ der Stufe $p + q$ mit den Einträgen

$$c_{i_1, \dots, i_p, j_1, \dots, j_q} = a_{i_1, \dots, i_p} \cdot b_{j_1, \dots, j_q}, \quad (9.3.2)$$

für alle $1 \leq i_r \leq m_r$ und $1 \leq j_s \leq n_s$. Das Tensorprodukt ist also eine Abbildung

$$\otimes : \mathbb{R}^{m_1 \times \dots \times m_p} \times \mathbb{R}^{n_1 \times \dots \times n_q} \rightarrow \mathbb{R}^{m_1 \times \dots \times m_p \times n_1 \times \dots \times n_q}.$$

Bemerkung 9.3.8 (Stufen addieren sich, Einträge multiplizieren sich)

Der Ergebnistensor hat $p + q$ Indexpositionen, seine ersten p Indizes kommen von $\mathbf{A}$, die letzten q von $\mathbf{B}$. Gezählt werden dabei

$$(m_1 \cdots m_p) \cdot (n_1 \cdots n_q)$$

Einträge, denn jeder Eintrag von A trifft genau einmal auf jeden Eintrag von B .

Für $p = q = 1$ sind beide Faktoren Vektoren, und (9.3.2) wird zu $c_{i,j} = a_i b_j$. Das ist Wort für Wort Definition 9.3.1: Das äußere Produkt ist das Tensorprodukt zweier Tensoren der Stufe 1. Im Bild aus Abschnitt 9.2 heißt das: Zwei Vektoren spannen ein Gitter von Produkten auf, drei Vektoren einen Quader, und jede weitere Stufe legt eine Achse dazu.

Satz 9.3.9 (Das Tensorprodukt ist bilinear)

Für Tensoren A, A' desselben Formats, B, B' desselben Formats und $c \in \mathbb{R}$ gilt

$$(A + A') \otimes B = A \otimes B + A' \otimes B, \quad (cA) \otimes B = c(A \otimes B),$$

und ebenso im zweiten Argument. Außerdem ist das Tensorprodukt assoziativ: Für einen dritten Tensor D beliebigen Formats gilt $(A \otimes B) \otimes D = A \otimes (B \otimes D)$.

Vertiefung: Beweis der Bilinearität des Tensorprodukts

Beweis.

- (1) Alle Aussagen sind Gleichungen zwischen Tensoren, und zwei Tensoren sind genau dann gleich, wenn sie an jeder Indexstelle übereinstimmen. Dort steht jeweils eine Gleichung zwischen reellen Zahlen, die aus den Rechengesetzen in $\mathbb{R}$ folgt.

an der Stelle $(i_1, \dots, i_p, j_1, \dots, j_q)$ etwa $(a_{i_1, \dots, i_p} + a'_{i_1, \dots, i_p}) b_{j_1, \dots, j_q} = a_{i_1, \dots, i_p} b_{j_1, \dots, j_q} + a'_{i_1, \dots, i_p} b_{j_1, \dots, j_q}$; Homogenität, zweites Argument und Assoziativität gehen genauso

- (2) Das Tensorprodukt ist damit in jedem seiner beiden Argumente linear, also eine bilineare Abbildung im Sinn von Definition 9.1.1 (Abschnitt 9.1). Es ist selbst ein Beispiel für den Gegenstand dieses Kapitels.

Bilinearität ist der Fall $n = 2$ der Multilinearität

□

Beispiel 9.3.10 (Tensorprodukt dreier Vektoren)

Wir nehmen

$$\mathbf{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} 3 \\ 5 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 7 \\ 11 \end{pmatrix},$$

alle drei im $\mathbb{R}^2$. Das Tensorprodukt $T = \mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w} \in \mathbb{R}^{2 \times 2 \times 2}$ hat nach (9.3.2) die Einträge $T_{i,j,k} = u_i v_j w_k$. Die Klammerung brauchen wir dank Satz 9.3.9 nicht zu notieren. Rechnen wir alle acht Einträge aus, geordnet nach der dritten Indexposition:

$$\begin{aligned} \text{Scheibe } k = 1 \ (\mathbf{w}_1 = 7): \quad & T_{1,1,1} = 1 \cdot 3 \cdot 7 = 21, & T_{1,2,1} &= 1 \cdot 5 \cdot 7 = 35, \\ & T_{2,1,1} = 2 \cdot 3 \cdot 7 = 42, & T_{2,2,1} &= 2 \cdot 5 \cdot 7 = 70, \\ \text{Scheibe } k = 2 \ (\mathbf{w}_2 = 11): \quad & T_{1,1,2} = 1 \cdot 3 \cdot 11 = 33, & T_{1,2,2} &= 1 \cdot 5 \cdot 11 = 55, \\ & T_{2,1,2} = 2 \cdot 3 \cdot 11 = 66, & T_{2,2,2} &= 2 \cdot 5 \cdot 11 = 110. \end{aligned}$$

Als Scheiben geschrieben, wie im Viewer aus Abschnitt 9.2:

$$T_{:,1} = \begin{pmatrix} 21 & 35 \\ 42 & 70 \end{pmatrix}, \quad T_{:,2} = \begin{pmatrix} 33 & 55 \\ 66 & 110 \end{pmatrix}.$$

Beide Scheiben sind Vielfache derselben Matrix, nämlich

$$T_{:,k} = w_k \cdot (\mathbf{u} \otimes \mathbf{v}), \quad \mathbf{u} \otimes \mathbf{v} = \begin{pmatrix} 3 & 5 \\ 6 & 10 \end{pmatrix}.$$

Nach Satz 9.3.4 hat $\mathbf{u} \otimes \mathbf{v}$ den Rang 1, und das Vielfache mit $w_k \neq 0$ ändert daran nichts: Jede Scheibe dieses Stufe-3-Tensors ist eine Rang-1-Matrix. Tensoren der Form $\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}$ mit Faktoren $\neq \mathbf{0}$ heißen deshalb *Rang-1-Tensoren*. Jeder Stufe-3-Tensor ist eine Summe solcher Bausteine, denn schon die Basistensoren $\mathbf{E}^{(i,j,k)}$ aus dem Beweis von Satz 9.2.5 (Abschnitt 9.2) sind welche: Es ist $\mathbf{E}^{(i,j,k)} = \mathbf{e}_i \otimes \mathbf{e}_j \otimes \mathbf{e}_k$. Die Analogie zur SVD hat aber eine Grenze. Für Matrizen liefert Bemerkung 9.3.5 die Bausteine mit orthogonalen Faktoren, der Größe nach geordnet und mit einem Verfahren berechenbar. Ab Stufe 3 ist das im Allgemeinen nicht mehr zu haben, und schon die kleinste Anzahl nötiger Summanden ist schwer zu bestimmen.

Das Kroneckerprodukt

Für Matrizen gibt es eine zweite Lesart des Produkts, die statt eines Tensors der Stufe 4 wieder eine Matrix liefert. Sie ist überall dort im Einsatz, wo mit Matrizen weitergerechnet werden soll.

Definition 9.3.11 (Kroneckerprodukt)

Das *Kroneckerprodukt* zweier Matrizen $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{B} \in \mathbb{R}^{p \times q}$ ist die Blockmatrix

$$\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B} = \begin{pmatrix} a_{11}\mathbf{B} & \cdots & a_{1n}\mathbf{B} \\ \vdots & & \vdots \\ a_{m1}\mathbf{B} & \cdots & a_{mn}\mathbf{B} \end{pmatrix} \in \mathbb{R}^{mp \times nq}.$$

Sie besteht aus $m \cdot n$ Blöcken der Größe $p \times q$; der Block an der Stelle (i_1, i_2) ist die mit $a_{i_1 i_2}$ skalierte Kopie von $\mathbf{B}$.

Das tiefgestellte $\mathbb{K}$ in $\otimes_{\mathbb{K}}$ ist unsere Schreibweise; in der Literatur steht auch hier meist $\otimes$ (siehe Bemerkung 9.3.12).

Bemerkung 9.3.12 (Zwei Bedeutungen, zwei Zeichen)

Für zwei Matrizen $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{B} \in \mathbb{R}^{p \times q}$ sind jetzt zwei verschiedene Produkte erklärt, und beide sind gebräuchlich:

- Das *Tensorprodukt* nach Definition 9.3.7 liefert einen Tensor der Stufe 4, $C \in \mathbb{R}^{m \times n \times p \times q}$ mit $c_{i_1, i_2, j_1, j_2} = a_{i_1, i_2} b_{j_1, j_2}$.
- Das *Kroneckerprodukt* nach Definition 9.3.11 liefert eine Matrix $\mathbf{D} \in \mathbb{R}^{mp \times nq}$ mit

$$d_{(i_1-1)p+j_1, (i_2-1)q+j_2} = a_{i_1, i_2} b_{j_1, j_2}. \quad (9.3.3)$$

Dieselben $mnpq$ Zahlen, verschieden angeordnet: $\mathbf{D}$ ist die *abgeflachte* Fassung von C . Die Indexformel (9.3.3) sagt genau, wie abgeflacht wird. Der Zeilenindex läuft in m Gruppen zu je p Zeilen; die Gruppe wählt i_1 , die Position innerhalb der Gruppe wählt j_1 . Für die Spalten gilt dasselbe mit i_2 und j_2 . Der Index des ersten Faktors läuft also langsam, der des zweiten schnell.

Beide Produkte mit demselben Zeichen $\otimes$ zu schreiben, ist verbreitet; welche Bedeutung gemeint ist, verrät dann das Format des Ergebnisses. Wir sparen uns diese Rückfrage und schreiben $\otimes_K$, sobald das Kroneckerprodukt gemeint ist. Das Zeichen $\otimes$ bleibt dem äußeren Produkt und dem Tensorprodukt vorbehalten. Beim Lesen fremder Texte ist die Unterscheidung wieder Ihre Aufgabe.

An Vektoren lässt sich das Abflachen gut ablesen. Fassen wir $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ als $(m \times 1)$ - und $(n \times 1)$ -Matrizen auf, so ist ihr Kroneckerprodukt der Vektor der Länge mn , der die Zeilen des äußeren Produkts hintereinanderhängt. Steht der zweite Faktor dagegen als Zeilenvektor da, kommt das äußere Produkt selbst heraus:

$$\begin{pmatrix} 1 \\ 2 \end{pmatrix} \otimes_K \begin{pmatrix} 3 \\ 5 \\ 7 \end{pmatrix} = \begin{pmatrix} 3 & 5 & 7 & 6 & 10 & 14 \end{pmatrix}^\top, \quad \begin{pmatrix} 1 \\ 2 \end{pmatrix} \otimes_K (3 \ 5 \ 7) = \begin{pmatrix} 3 & 5 & 7 \\ 6 & 10 & 14 \end{pmatrix}.$$

Beispiel 9.3.13 (Kroneckerprodukt zweier kleiner Matrizen)

Für

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 2 & 5 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 0 & -1 \end{pmatrix}$$

ist $\mathbf{A} \otimes_K \mathbf{B} \in \mathbb{R}^{6 \times 6}$, denn $2 \cdot 3 = 6$ Zeilen und ebenso viele Spalten. Blockweise notiert und dann ausgeschrieben:

$$\mathbf{A} \otimes_K \mathbf{B} = \begin{pmatrix} 1 \cdot \mathbf{B} & 0 \cdot \mathbf{B} \\ 2 \cdot \mathbf{B} & 5 \cdot \mathbf{B} \end{pmatrix} = \begin{pmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ -1 & 0 & -1 & 0 & 0 & 0 \\ 6 & 0 & 0 & 15 & 0 & 0 \\ 0 & 4 & 0 & 0 & 10 & 0 \\ -2 & 0 & -2 & -5 & 0 & -5 \end{pmatrix}.$$

Der Block oben rechts ist die Nullmatrix, weil $a_{12} = 0$ ist. Die Struktur von $\mathbf{B}$ wiederholt sich in jedem der drei übrigen Blöcke, nur mit unterschiedlichem Vorfaktor.

Bemerkung 9.3.14 (Transponieren und Reihenfolge)

Zwei Regeln sind im Umgang mit dem Kroneckerprodukt nützlich, und die erste davon ist bequem:

$$(\mathbf{A} \otimes_K \mathbf{B})^\top = \mathbf{A}^\top \otimes_K \mathbf{B}^\top.$$

Transponieren zieht also in beide Faktoren hinein, ohne ihre Reihenfolge zu vertauschen. Der Nachweis ist ein Blick auf (9.3.3): Auf beiden Seiten steht an der Stelle mit Zeilenindex $(i_2 - 1)q + j_2$ und Spaltenindex $(i_1 - 1)p + j_1$ der Eintrag $a_{i_1, j_2} b_{j_1, i_1}$.

Die zweite Regel ist eine Warnung: Das Kroneckerprodukt ist **nicht kommutativ**, im Allgemeinen also

$$\mathbf{A} \otimes_K \mathbf{B} \neq \mathbf{B} \otimes_K \mathbf{A}.$$

Beide Produkte haben zwar dasselbe Format $mp \times nq$ und enthalten dieselben $mnpq$ Zahlen, aber an anderen Stellen. Für die Matrizen aus Beispiel 9.3.13 steht an der Stelle $(2, 1)$ in $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}$ eine 0, in $\mathbf{B} \otimes_{\mathbb{K}} \mathbf{A}$ dagegen eine 6: Dort ist der erste Block $3 \mathbf{A}$, und dessen zweite Zeile beginnt mit $3 \cdot 2$.

Interaktiv: Kroneckerprodukte selbst ausrechnen

Der Rechner zeigt beide Faktoren und das Produkt in Blockdarstellung. Warum steht in Satz 9.5.3 gerade $\mathbf{B}^T \otimes_{\mathbb{K}} \mathbf{A}$ vor $\text{vec}(\mathbf{X})$ und nicht dieselben Faktoren in der anderen Reihenfolge?

Interaktives Element — nur in der Web-Fassung

Die Blockstruktur legt fest, welche gestapelten Spalten miteinander verrechnet werden; deshalb ist die Reihenfolge der Faktoren Teil der Rechenvorschrift.

Blockstrukturen aus Kroneckerprodukten

Setzen wir eine Einheitsmatrix als einen der beiden Faktoren ein, entstehen Muster, die in der Statistik ständig vorkommen. Auf welcher Seite sie steht, macht dabei den ganzen Unterschied.

Beispiel 9.3.15 ($\mathbf{S} \otimes_{\mathbb{K}} \mathbf{I}_n$ verteilt die Einträge)

Sei $\mathbf{S} \in \mathbb{R}^{p \times p}$ mit den Einträgen s_{ij} . Dann ist jeder Block von $\mathbf{S} \otimes_{\mathbb{K}} \mathbf{I}_n$ eine Diagonalmatrix:

$$\mathbf{S} \otimes_{\mathbb{K}} \mathbf{I}_n = \begin{pmatrix} s_{11} \mathbf{I}_n & \cdots & s_{1p} \mathbf{I}_n \\ \vdots & & \vdots \\ s_{p1} \mathbf{I}_n & \cdots & s_{pp} \mathbf{I}_n \end{pmatrix} \in \mathbb{R}^{np \times np}.$$

Jeder Eintrag s_{ij} taucht n -mal auf, nämlich an den Stellen $((i-1)n + k, (j-1)n + k)$ für $k = 1, \dots, n$; alle übrigen Einträge sind null. Für $p = n = 2$ und $\mathbf{S} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ sieht das so aus:

$$\mathbf{S} \otimes_{\mathbb{K}} \mathbf{I}_2 = \begin{pmatrix} 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \\ 3 & 0 & 4 & 0 \\ 0 & 3 & 0 & 4 \end{pmatrix}.$$

Gelesen als Kovarianzstruktur heißt das: Die np Koordinaten zerfallen in p Gruppen zu je n Stück, und gekoppelt sind immer nur gleiche Positionen verschiedener Gruppen, mit der Stärke s_{ij} . Es liegen also n unabhängige Kopien der $\mathbf{S}$ -Struktur ineinandergeschoben.

Beispiel 9.3.16 ($\mathbf{I}_n \otimes_{\mathbb{K}} \mathbf{S}$ ist blockdiagonal)

Vertauschen wir die Reihenfolge, wird aus demselben Paar eine Blockdiagonalmatrix. Die Farben bleiben an ihren Matrizen, $\mathbf{S}$ steht jetzt also innen:

$$\mathbf{I}_n \otimes_{\mathbb{K}} \mathbf{S} = \begin{pmatrix} \mathbf{S} & & \\ & \mathbf{S} & \\ & & \ddots \\ & & & \mathbf{S} \end{pmatrix} \in \mathbb{R}^{n \times np},$$

mit n Kopien von $\mathbf{S} \in \mathbb{R}^{p \times p}$ auf der Diagonalen und Nullen sonst. In der Statistik ist das die Kovarianzmatrix eines Vektors, der aus n voneinander unabhängigen Blöcken besteht, die alle dieselbe Kovarianzmatrix $\mathbf{S}$ haben; ein typischer Fall sind n Personen mit je p Messungen. Der Vergleich mit Beispiel 9.3.15 zeigt die Nicht-Kommutativität von der anschaulichen Seite: Dieselben zwei Faktoren ergeben einmal eine blockdiagonale und einmal eine über die ganze Matrix verteilte Struktur.

Anwendung: separierbare Kovarianz

Beobachten wir dieselbe Größe an m Orten zu n Zeitpunkten, so haben wir mn Messwerte, und ihre Kovarianzmatrix hat das Format $mn \times mn$. Schon bei $m = 10$ Orten und $n = 50$ Zeitpunkten sind das 500×500 Einträge, weit mehr, als sich aus überschaubaren Datenmengen schätzen lässt. Das Kroneckerprodukt liefert eine sparsame Modellannahme.

Definition 9.3.17 (Separierbare Kovarianz)

Die Kovarianzmatrix $\Sigma \in \mathbb{R}^{mn \times mn}$ der Beobachtungen heißt *separierbar*, wenn sie sich als

$$\Sigma = \Sigma_T \otimes_{\mathbb{K}} \Sigma_S \quad (9.3.4)$$

schreiben lässt, mit der Kovarianzmatrix $\Sigma_T \in \mathbb{R}^{n \times n}$ zwischen den Zeitpunkten und der Kovarianzmatrix $\Sigma_S \in \mathbb{R}^{m \times m}$ zwischen den Orten.

Dabei müssen wir sagen, wie die mn Beobachtungen angeordnet sind. Nach Bemerkung 9.3.12 läuft der Index des ersten Faktors langsam und der des zweiten schnell. Zu (9.3.4) gehört also die Anordnung, in der die Zeit langsam und der Ort schnell läuft: erst alle Orte zum ersten Zeitpunkt, dann alle Orte zum zweiten Zeitpunkt und so weiter.

Beispiel 9.3.18 (Zwei Orte, zwei Zeitpunkte)

Für $m = 2$ Orte und $n = 2$ Zeitpunkte seien

$$\Sigma_T = \begin{pmatrix} 1,0 & 0,8 \\ 0,8 & 1,0 \end{pmatrix}, \quad \Sigma_S = \begin{pmatrix} 2,0 & 0,5 \\ 0,5 & 2,0 \end{pmatrix}.$$

Dann ist

$$\Sigma = \Sigma_T \otimes_{\mathbb{K}} \Sigma_S = \begin{pmatrix} 1,0 \cdot \Sigma_S & 0,8 \cdot \Sigma_S \\ 0,8 \cdot \Sigma_S & 1,0 \cdot \Sigma_S \end{pmatrix} = \begin{pmatrix} 2,0 & 0,5 & 1,6 & 0,4 \\ 0,5 & 2,0 & 0,4 & 1,6 \\ 1,6 & 0,4 & 2,0 & 0,5 \\ 0,4 & 1,6 & 0,5 & 2,0 \end{pmatrix} \in \mathbb{R}^{4 \times 4}.$$

Die vier Beobachtungen stehen in der Reihenfolge (Ort 1, Zeit 1), (Ort 2, Zeit 1), (Ort 1, Zeit 2), (Ort 2, Zeit 2). Damit lesen wir die erste Zeile ab:

Eintrag	verbindet	Wert	Herkunft
(1, 1)	Ort 1, Zeit 1 mit sich selbst	2,0	$1,0 \cdot 2,0$, eine Varianz
(1, 2)	zwei Orte, gleiche Zeit	0,5	$1,0 \cdot 0,5$
(1, 3)	gleicher Ort, zwei Zeiten	1,6	$0,8 \cdot 2,0$
(1, 4)	anderer Ort, andere Zeit	0,4	$0,8 \cdot 0,5$

Noch klarer wird die Struktur in Korrelationen. Zwei Beobachtungen am selben Ort zu verschiedenen Zeiten sind mit $1,6/2,0 = 0,8$ korreliert, zwei Beobachtungen zur selben Zeit an verschiedenen Orten mit $0,5/2,0 = 0,25$, und unterscheiden sich beide Merkmale, so ist die Korrelation das Produkt $0,8 \cdot 0,25 = 0,2$. Separierbarkeit heißt genau das: Die Korrelation zerfällt in einen zeitlichen und einen räumlichen Anteil, die sich nicht gegenseitig beeinflussen.

Bemerkung 9.3.19 (Was die Annahme spart und was sie kostet)

Eine beliebige Kovarianzmatrix ist symmetrisch, hat also $\frac{mn(mn+1)}{2}$ freie Einträge. Die separierbare Form braucht nur die Einträge der beiden Faktoren,

$$\frac{m(m+1)}{2} + \frac{n(n+1)}{2}.$$

Für $m = n = 2$ stehen 10 Parametern also 6 gegenüber, für $m = 10$ Orte und $n = 50$ Zeitpunkte sind es 125 250 gegen 1 330. Genau genommen ist die Zerlegung (9.3.4) nicht eindeutig: Wegen $\Sigma_T \otimes_K \Sigma_S = (c \Sigma_T) \otimes_K (c^{-1} \Sigma_S)$ für jedes $c > 0$ lässt sich ein Skalenfaktor zwischen den Kovarianzfaktoren verschieben. Identifizierbar ist deshalb ein Parameter weniger, im Beispiel also 5 statt 6.

Bezahlt wird die Sparsamkeit mit einer echten Einschränkung. Separierbarkeit verbietet, dass sich das räumliche Muster über die Zeit ändert. Ein Wetterfeld, dessen Zusammenhänge morgens anders aussehen als abends, lässt sich so nicht beschreiben.

Vertiefung: Eigenwerte eines Kroneckerprodukts

Dass Σ überhaupt eine Kovarianzmatrix sein kann, sichert eine weitere Rechenregel: Die Eigenwerte von $\Sigma_T \otimes_K \Sigma_S$ sind die Produkte $\lambda_i \mu_j$ der Eigenwerte beider Faktoren, denn mit Eigenvektoren $\mathbf{a}$ und $\mathbf{b}$ ist

$$(\Sigma_T \otimes_K \Sigma_S)(\mathbf{a} \otimes_K \mathbf{b}) = (\Sigma_T \mathbf{a}) \otimes_K (\Sigma_S \mathbf{b}) = \lambda_i \mu_j (\mathbf{a} \otimes_K \mathbf{b}),$$

wobei $\mathbf{a} \otimes_K \mathbf{b}$ das Kroneckerprodukt der beiden Spaltenvektoren ist. Die mittlere Gleichung ist die Regel $(\mathbf{A} \otimes_K \mathbf{B})(\mathbf{C} \otimes_K \mathbf{D}) = (\mathbf{AC}) \otimes_K (\mathbf{BD})$, die sich ebenfalls an (9.3.3) ablesen lässt. Beide Faktoren sind als Kovarianzmatrizen symmetrisch, haben nach dem Spektralsatz also je eine Orthonormalbasis aus Eigenvektoren. Laufen $\mathbf{a}$ und $\mathbf{b}$ durch diese beiden Basen, so bilden die mn Vektoren $\mathbf{a} \otimes_K \mathbf{b}$ eine Basis des $\mathbb{R}^{mn}$; damit sind das bereits alle Eigenwerte. Ist also jeder Faktor positiv definit, so ist es auch Σ . In Beispiel 9.3.18 hat Σ_T die Eigenwerte 1,8 und 0,2, Σ_S die Eigenwerte 2,5 und 1,5; die vier Produkte 4,5, 2,7, 0,5 und 0,3 sind die Eigenwerte von Σ . Ihre Summe ist 8, die Spur von Σ , und ihr Produkt ist 1,8225, die Determinante von Σ .

Interaktiv: Separierbare Kovarianz zum Schieben

Wie groß ist der Unterschied zwischen dem Speichern einer allgemeinen Kovarianzmatrix und ihrer separierbaren Beschreibung für unser Messgitter?

Interaktives Element — nur in der Web-Fassung

Die separierbare Kovarianz gewinnt ihre Sparsamkeit dadurch, dass Zeit- und Ortsstruktur in zwei Faktoren getrennt modelliert werden.

Selbsttest

1. Im Rang-1-Widget wird jeder Bildvektor ein Vielfaches von $\mathbf{v}$.

Antwort: Wahr

Der orange Pfeil liegt stets auf derselben Geraden. Das ist die Bildaussage aus Satz 9.3.4.

2. Das äußere Produkt $\mathbf{v} \otimes \mathbf{w}$ ist nur für Vektoren gleicher Länge definiert.

Antwort: Falsch

Das gilt für das innere Produkt. Das äußere Produkt $\mathbf{v} \otimes \mathbf{w} = \mathbf{vw}^\top$ verlangt keine Übereinstimmung: Für $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ entsteht eine $m \times n$ -Matrix (Definition 9.3.1, Bemerkung 9.3.2). In Beispiel 9.3.3 sind die Längen 2 und 3.

3. Für $\mathbf{v}, \mathbf{w} \neq \mathbf{0}$ bildet $\mathbf{x} \mapsto (\mathbf{v} \otimes \mathbf{w})\mathbf{x}$ den ganzen $\mathbb{R}^n$ auf die Gerade $\text{span}\{\mathbf{v}\}$ ab, und der Kern ist die Hyperebene senkrecht zu $\mathbf{w}$.

Antwort: Wahr

Das ist Satz 9.3.4: Es gilt $(\mathbf{v} \otimes \mathbf{w})\mathbf{x} = \mathbf{v}\langle \mathbf{w}, \mathbf{x} \rangle$, das Bild besteht also aus Vielfachen von $\mathbf{v}$, und der Kern besteht aus allen $\mathbf{x}$ mit $\mathbf{w}^\top \mathbf{x} = 0$. Rang 1 plus Kerndimension $n - 1$ ergibt n , wie der Rangsatz es verlangt.

4. Die beiden Scheiben $T_{:,1}$ und $T_{:,2}$ des Tensors $T = \mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w} \in \mathbb{R}^{2 \times 2 \times 2}$ sind linear unabhängig voneinander.

Antwort: Falsch

Beide sind Vielfache derselben Rang-1-Matrix, $T_{:,k} = w_k (\mathbf{u} \otimes \mathbf{v})$, also linear abhängig. In Beispiel 9.3.10 ist die zweite Scheibe das 11/7-fache der ersten. Tensoren dieser Bauart heißen gerade deshalb Rang-1-Tensoren.

5. Das Kroneckerprodukt zweier Matrizen $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{B} \in \mathbb{R}^{p \times q}$ ist ein Tensor der Stufe 4.

Antwort: Falsch

Der Stufe-4-Tensor ist das *Tensorprodukt* nach Definition 9.3.7. Das Kroneckerprodukt ordnet dieselben $mnpq$ Zahlen nach der Indexformel (9.3.3) in einer Matrix aus $\mathbb{R}^{m \times p \times n \times q}$ an, ist also die abgeflachte Fassung davon (Bemerkung 9.3.12).

6. Es gilt $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B} = \mathbf{B} \otimes_{\mathbb{K}} \mathbf{A}$, denn beide Produkte enthalten dieselben Zahlen.

Antwort: Falsch

Dieselben Zahlen stehen an verschiedenen Stellen. In Beispiel 9.3.13 ist der Eintrag $(2, 1)$ von $\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}$ gleich 0, der von $\mathbf{B} \otimes_{\mathbb{K}} \mathbf{A}$ dagegen 6 (Bemerkung 9.3.14). Die Formate stimmen zwar überein, die Matrizen nicht.

7. Es gilt $(\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B})^\top = \mathbf{A}^\top \otimes_{\mathbb{K}} \mathbf{B}^\top$.

Antwort: Wahr

Transponieren zieht in beide Faktoren hinein, ohne ihre Reihenfolge zu vertauschen; die Indexformel (9.3.3) zeigt es unmittelbar (Bemerkung 9.3.14). Vorsicht bei der Analogie zum Matrixprodukt: Dort dreht sich die Reihenfolge um, hier nicht.

8. Die Matrix $I_n \otimes_K S$ mit $S \in \mathbb{R}^{p \times p}$ ist blockdiagonal mit n Kopien von S .

Antwort: Wahr

Der Block an der Stelle (i, j) ist $\delta_{ij}S$, außerhalb der Diagonalen steht also die Nullmatrix (Beispiel 9.3.16). In der umgekehrten Reihenfolge entsteht dagegen $S \otimes_K I_n$, dessen Einträge über die ganze Matrix verteilt sind (Beispiel 9.3.15).

9. Für $m = 2$ Orte und $n = 2$ Zeitpunkte hat eine separierbare Kovarianzmatrix 6 Parameter statt der 10 einer beliebigen symmetrischen 4×4 -Matrix.

Antwort: Wahr

Es ist $\frac{mn(mn+1)}{2} = \frac{4 \cdot 5}{2} = 10$ gegen $\frac{m(m+1)}{2} + \frac{n(n+1)}{2} = 3 + 3 = 6$ (Bemerkung 9.3.19). Identifizierbar sind davon 5, weil sich ein Skalenfaktor zwischen den beiden Faktoren verschieben lässt.

Vertiefung: vgl. MML §4.5 für die Summendarstellung einer Matrix aus Rang-1-Bausteinen, auf der Bemerkung 9.3.5 aufsetzt; die Rechenregeln des Kroneckerprodukts sammelt Kapitel 2 von Magnus und Neudecker, Matrix Differential Calculus with Applications in Statistics and Econometrics.

9.4 Tensorprodukt von Vektorräumen

In Abschnitt 9.3 war $\otimes$ eine Rechenvorschrift: Aus zwei Vektoren wurde eine Matrix, aus zwei Tensoren ein Tensor höherer Stufe. Jetzt drehen wir die Blickrichtung um und fragen nach dem Raum, in dem diese Produkte leben. Aus der Operation wird damit eine Konstruktion, die aus zwei Vektorräumen einen dritten macht.

Der Raum $V \otimes W$

Bleiben wir kurz bei $V = \mathbb{R}^m$ und $W = \mathbb{R}^n$ mit dem äußeren Produkt $\mathbf{v} \otimes \mathbf{w} = \mathbf{vw}^\top$. Die Menge aller dieser Produkte ist als Vektorraum unbrauchbar, denn sie ist nicht unter Addition abgeschlossen: Die Summe zweier Matrizen vom Rang 1 hat im Allgemeinen den Rang 2. Der kleinste Vektorraum, der alle Produkte enthält, ist ihre lineare Hülle, und genau so setzen wir den Raum an.

Definition 9.4.1 (Tensorprodukt von Vektorräumen)

Seien V und W Vektorräume und

$$\otimes : V \times W \rightarrow V \otimes W, \quad (\mathbf{v}, \mathbf{w}) \mapsto \mathbf{v} \otimes \mathbf{w},$$

eine bilineare Abbildung. Das *Tensorprodukt* (tensor product) von V und W ist der von ihren Bildern erzeugte Vektorraum

$$V \otimes W = \text{span}\{\mathbf{v} \otimes \mathbf{w} : \mathbf{v} \in V, \mathbf{w} \in W\}. \quad (9.4.1)$$

Dabei hat $\otimes$ die *universelle Eigenschaft*: Jede bilineare Abbildung $f : V \times W \rightarrow Z$ in einen Vektorraum Z faktorisiert eindeutig über $\otimes$, es gibt also genau eine lineare Abbildung $\tilde{f} : V \otimes W \rightarrow Z$ mit $f(\mathbf{v}, \mathbf{w}) = \tilde{f}(\mathbf{v} \otimes \mathbf{w})$ für alle $\mathbf{v} \in V$ und $\mathbf{w} \in W$.

Die Produkte $\mathbf{v} \otimes \mathbf{w}$ selbst nennen wir *elementare Tensoren*.

Vertiefung: Die universelle Eigenschaft

Die universelle Eigenschaft legt den Raum bis auf Isomorphie fest und macht ihn unabhängig von einer zufällig gewählten konkreten Realisierung. Das Tensorprodukt *linearisiert* damit bilineare Abbildungen: Statt f auf Paaren auszuwerten, wenden wir die lineare Abbildung $\tilde{f}$ auf elementare Tensoren an. Der Spann allein würde dazu nicht genügen. Nehmen wir etwa die Nullabbildung als bilineare Abbildung, so wäre ihr Bildspann zwar der Nullraum, erfüllte aber die universelle Eigenschaft nicht und könnte die übrigen bilinearen Abbildungen nicht darstellen.

Für $V = \mathbb{R}^m$ und $W = \mathbb{R}^n$ mit dem äußeren Produkt, also $\mathbb{R}^m \otimes \mathbb{R}^n = \mathbb{R}^{m \times n}$ nach Beispiel 9.4.3, ist die Eigenschaft schnell nachgerechnet. Zu bilinearem $f: \mathbb{R}^m \times \mathbb{R}^n \rightarrow Z$ setzen wir $\tilde{f}(A) := \sum_{i=1}^m \sum_{j=1}^n a_{ij} f(e_i, e_j)$. Diese Abbildung ist linear, und wegen $\mathbf{v}\mathbf{w}^\top = \sum_{ij} v_i w_j \mathbf{E}_{ij}$ mit der Matrix $\mathbf{E}_{ij}$, die an der Stelle (i, j) eine Eins und sonst lauter Nullen trägt, liefert die Bilinearität $\tilde{f}(\mathbf{v}\mathbf{w}^\top) = f(\mathbf{v}, \mathbf{w})$; eindeutig ist sie, weil die $\mathbf{E}_{ij}$ den Raum aufspannen.

Tensorprodukte existieren und sind durch die universelle Eigenschaft bis auf einen eindeutigen Isomorphismus festgelegt, der die elementaren Tensoren respektiert. Deshalb dürfen wir mit einer bequemen konkreten Realisierung arbeiten, in den folgenden Beispielen mit Matrizen beziehungsweise Funktionen. Gebraucht wird die Eigenschaft im Skript nur noch an einer Stelle, nämlich für die lineare Unabhängigkeit der Tensorproduktbasis in Satz 9.4.7.

Für $V = \mathbb{R}^m$ und $W = \mathbb{R}^n$ sind die elementaren Tensoren genau die Matrizen vom Rang höchstens 1. Satz 9.3.4 liefert davon eine Hälfte, nämlich den Rang 1 für $\mathbf{v}, \mathbf{w} \neq \mathbf{0}$; die Umkehrung steht im Anschluss an seinen Beweis, und der Rang 0 gehört zum Produkt mit dem Nullvektor. Produkte $\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}$ dreier Vektoren heißen in Abschnitt 9.3 entsprechend Rang-1-Tensoren.

Bemerkung 9.4.2 (Rechenregeln aus der Bilinearität)

Bilinearität im Sinn von Definition 9.1.1 (Abschnitt 9.1) heißt hier ausgeschrieben: Skalare Faktoren dürfen zwischen den Argumenten wandern, und in jedem Argument darf man Summen auseinanderziehen,

$$(c \mathbf{v}) \otimes \mathbf{w} = c(\mathbf{v} \otimes \mathbf{w}) = \mathbf{v} \otimes (c \mathbf{w}),$$

$$(\mathbf{v}_1 + \mathbf{v}_2) \otimes \mathbf{w} = \mathbf{v}_1 \otimes \mathbf{w} + \mathbf{v}_2 \otimes \mathbf{w}, \quad \mathbf{v} \otimes (\mathbf{w}_1 + \mathbf{w}_2) = \mathbf{v} \otimes \mathbf{w}_1 + \mathbf{v} \otimes \mathbf{w}_2.$$

Mit $c = 0$ folgt daraus sofort $\mathbf{0} \otimes \mathbf{w} = \mathbf{v} \otimes \mathbf{0} = \mathbf{0}$. Was die Regeln dagegen nicht liefern, ist eine Umformung von $\mathbf{v}_1 \otimes \mathbf{w}_1 + \mathbf{v}_2 \otimes \mathbf{w}_2$ in ein einzelnes Produkt. Deshalb steht in (9.4.1) der Spann und nicht die Menge der Produkte.

Zwei Beispiele mit Zahlentupeln

Beispiel 9.4.3 (Das Tensorprodukt von $\mathbb{R}^m$ und $\mathbb{R}^n$)

Sei $\otimes$ das äußere Produkt aus Abschnitt 9.3, also $\mathbf{v} \otimes \mathbf{w} = \mathbf{v}\mathbf{w}^\top \in \mathbb{R}^{m \times n}$. Wir zeigen beide Inklusionen.

„ $\subseteq$ “: Jedes $\mathbf{v} \otimes \mathbf{w}$ ist eine $m \times n$ -Matrix, und $\mathbb{R}^{m \times n}$ ist ein Vektorraum (Satz 9.2.5). Mit den Produkten liegen deshalb auch alle ihre Linearkombinationen in $\mathbb{R}^{m \times n}$.

„ $\supseteq$ “: Für die Standardbasisvektoren $\mathbf{e}_i \in \mathbb{R}^m$ und $\mathbf{e}_j \in \mathbb{R}^n$ ist

$$\mathbf{e}_i \otimes \mathbf{e}_j = \mathbf{e}_i \mathbf{e}_j^\top = \mathbf{E}_{ij},$$

die Matrix mit einer 1 an der Stelle (i, j) und sonst lauter Nullen. Jede Matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ schreibt sich als $\mathbf{A} = \sum_{i=1}^m \sum_{j=1}^n a_{ij} \mathbf{E}_{ij}$, liegt also im Spann der Produkte.

Damit ist $\mathbb{R}^m \otimes \mathbb{R}^n = \mathbb{R}^{m \times n}$ eine konkrete Realisierung des Tensorprodukts, und nach Satz 9.2.5 hat dieser Raum die Dimension mn .

Beispiel 9.4.4 (Drei Faktoren)

Seien $V_1 = \mathbb{R}^m$, $V_2 = \mathbb{R}^n$, $V_3 = \mathbb{R}^q$ und $\otimes$ das Tensorprodukt von Tensoren aus Abschnitt 9.3. Nach Beispiel 9.4.3 ist $V_1 \otimes V_2 = \mathbb{R}^{m \times n}$, und ein elementarer Tensor $\mathbf{A} \otimes \mathbf{w}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{w} \in \mathbb{R}^q$ hat die Einträge $a_{ij}w_k$. Das ist ein Tensor der Stufe 3, und dasselbe Argument wie oben zeigt

$$V_1 \otimes V_2 \otimes V_3 = (\mathbb{R}^m \otimes \mathbb{R}^n) \otimes \mathbb{R}^q = \mathbb{R}^{m \times n} \otimes \mathbb{R}^q = \mathbb{R}^{m \times n \times q}.$$

Auf die Klammerung kommt es dabei nicht an: Der elementare Tensor $\mathbf{u} \otimes \mathbf{v} \otimes \mathbf{w}$ hat die Einträge $u_i v_j w_k$, gleichgültig, in welcher Reihenfolge wir die drei Produkte bilden. Die Dimension ist $m n q$, wieder nach Satz 9.2.5.

Ein einzelnes Produkt und der ganze Raum

Die beiden Ausdrücke $\mathbf{v} \otimes \mathbf{w}$ und $V \otimes W$ sehen ähnlich aus und meinen Verschiedenes. Links steht ein einzelnes Element, rechts der ganze Raum, den alle diese Elemente aufspannen. Der Unterschied ist keine Feinheit, wie das kleinste denkbare Beispiel zeigt.

Beispiel 9.4.5 (Die Einheitsmatrix ist kein elementarer Tensor)

Sei $V = W = \mathbb{R}^2$ und

$$\mathbf{A} = \mathbf{I}_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Kein einzelnes Produkt. Gäbe es $\mathbf{v}, \mathbf{w} \in \mathbb{R}^2$ mit $\mathbf{A} = \mathbf{v} \otimes \mathbf{w}$, so wären alle Spalten von $\mathbf{v} \mathbf{w}^\top$ Vielfache von $\mathbf{v}$, also $\text{rang}(\mathbf{v} \otimes \mathbf{w}) \leq 1$ (Abschnitt 9.3). Die Einheitsmatrix hat aber den Rang 2.

Aber ein Element des Raums. In $V \otimes W = \mathbb{R}^{2 \times 2}$ liegt $\mathbf{I}_2$ selbstverständlich, und zwar als Summe von zwei elementaren Tensoren:

$$\begin{aligned} \mathbf{e}_1 \otimes \mathbf{e}_1 + \mathbf{e}_2 \otimes \mathbf{e}_2 &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \end{aligned}$$

Zwei Summanden genügen also. Dass das kein Zufall ist, sagt der nächste Satz.

Satz 9.4.6 (Jede Matrix ist eine kurze Summe elementarer Tensoren)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ mit $\text{rang}(\mathbf{A}) = r$. Dann gibt es $\mathbf{u}_i \in \mathbb{R}^m$, $\mathbf{v}_i \in \mathbb{R}^n$ und Zahlen $\sigma_i > 0$ mit

$$\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \otimes \mathbf{v}_i.$$

Insbesondere kommt jede Matrix mit höchstens $\min(m, n)$ elementaren Tensoren aus.

Vertiefung: Beweis über die Spalten einer Matrix

Beweis.

- (1) Die Singulärwertzerlegung liefert $A = U\Sigma V^T$ (Satz 6.2.13 in Abschnitt 6.2), und ausmultipliziert wird daraus die Summenform

$$A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

mit $r = \text{rang}(A)$ und $\sigma_1 \geq \dots \geq \sigma_r > 0$ (Satz 6.4.2 in Abschnitt 6.4, im Kapitel schon in Bemerkung 9.3.5 aufgegriffen).

jeder Summand ist ein äußeres Produkt $\mathbf{u}_i \mathbf{v}_i^T = \mathbf{u}_i \otimes \mathbf{v}_i$, also ein elementarer Tensor

- (2) Für die Schranke bleibt $r \leq \min(m, n)$ zu begründen. Der Rang ist die Dimension des Spaltenraums $\text{col}(A) \subseteq \mathbb{R}^m$, also ist $r \leq m$. Zugleich wird $\text{col}(A)$ von den n Spalten aufgespannt, also ist $r \leq n$.

ein Unterraum des $\mathbb{R}^m$ hat höchstens die Dimension m ; ein Erzeugendensystem aus n Vektoren spannt höchstens einen n -dimensionalen Raum auf

□

Weniger als r Summanden reichen nicht. Wäre $A = \sum_{i=1}^k \mathbf{u}_i \otimes \mathbf{v}_i$ mit $k < r$, so lägen alle Spalten von A im Spann von $\mathbf{u}_1, \dots, \mathbf{u}_k$, und der Rang wäre höchstens k . Die kleinste Anzahl elementarer Tensoren, die eine Matrix darstellen, ist also genau ihr Rang, und die SVD rechnet sie aus.

Die Tensorproduktbasis

Ein Vektorraum wird durch eine Basis handhabbar. Für $V \otimes W$ müssen wir sie nicht suchen: Sie entsteht aus Basen der beiden Faktoren, indem wir alle Paare miteinander multiplizieren.

Satz 9.4.7 (Tensorproduktbasis)

Seien $\mathcal{B}_V = \{\mathbf{v}_1, \dots, \mathbf{v}_m\}$ eine Basis von V und $\mathcal{B}_W = \{\mathbf{w}_1, \dots, \mathbf{w}_n\}$ eine Basis von W . Dann ist

$$\mathcal{B}_{V \otimes W} = \{\mathbf{v}_i \otimes \mathbf{w}_j : i = 1, \dots, m, j = 1, \dots, n\}$$

eine Basis von $V \otimes W$ und heißt *Tensorproduktbasis* (tensor product basis). Insbesondere gilt

$$V \otimes W = \text{span}(\mathcal{B}_{V \otimes W}) = \left\{ \sum_{i=1}^m \sum_{j=1}^n c_{ij} \mathbf{v}_i \otimes \mathbf{w}_j : c_{11}, \dots, c_{mn} \in \mathbb{R} \right\} \quad (9.4.2)$$

und

$$\dim(V \otimes W) = \dim(V) \dim(W) = mn.$$

Dass die Produkte den Raum erzeugen, ist schnell gesehen. Wir entwickeln $\mathbf{v} \in V$ und $\mathbf{w} \in W$ in den beiden Basen, $\mathbf{v} = \sum_i a_i \mathbf{v}_i$ und $\mathbf{w} = \sum_j b_j \mathbf{w}_j$, und ziehen die Summen mit Bemerkung 9.4.2 nach außen, zuerst im ersten, dann im zweiten Argument:

$$\mathbf{v} \otimes \mathbf{w} = \left(\sum_{i=1}^m a_i \mathbf{v}_i \right) \otimes \mathbf{w} = \sum_{i=1}^m \sum_{j=1}^n a_i b_j (\mathbf{v}_i \otimes \mathbf{w}_j). \quad (9.4.3)$$

Jeder elementare Tensor liegt damit im Spann von $\mathcal{B}_{V \otimes W}$, und da die elementaren Tensoren nach Definition 9.4.1 den ganzen Raum erzeugen, gilt die Darstellung (9.4.2). Die lineare Unabhängigkeit

holt die folgende Vertiefung nach.

Vertiefung: Warum die Produkte linear unabhängig sind

Beweis.

- (1) Sei $\sum_i \sum_j c_{ij} \mathbf{v}_i \otimes \mathbf{w}_j = \mathbf{0}$. Zu festen Indizes (k, l) ist

$$(\mathbf{v}, \mathbf{w}) \mapsto a_k(\mathbf{v}) b_l(\mathbf{w})$$

bilinear, wobei a_k und b_l die k -te beziehungsweise l -te Koordinatenfunktion in den beiden Basen sind. Die universelle Eigenschaft liefert dazu eine lineare Abbildung auf $V \otimes W$. Sie schickt $\mathbf{v}_i \otimes \mathbf{w}_j$ auf 1, falls $(i, j) = (k, l)$, und sonst auf 0. Wenden wir sie auf die verschwindende Summe an, erhalten wir $c_{kl} = 0$. Da (k, l) beliebig war, verschwinden alle Koeffizienten.

die universelle Eigenschaft macht aus jeder bilinearen Koordinatenabbildung genau eine lineare Abbildung auf dem Tensorproduktraum

- (2) Die Basis $\mathcal{B}_{V \otimes W}$ enthält damit für jedes Paar (i, j) genau ein Element, also mn Stück.

die mn Produkte sind paarweise verschieden, denn sie sind nach dem ersten Schritt linear unabhängig

□

Bemerkung 9.4.8 (Eine Feinheit)

Die Koeffizienten in (9.4.2) sind beliebig. Bei einem elementaren Tensor haben sie nach (9.4.3) die besondere Form $c_{ij} = a_i b_j$, bei einem allgemeinen Element von $V \otimes W$ gerade nicht. In Beispiel 9.4.5 ist die Koeffizientenmatrix bezüglich der Standardbasen $\mathbf{C} = \mathbf{I}_2$, und $\mathbf{I}_2$ lässt sich nicht als $\mathbf{a}\mathbf{b}^\top$ schreiben.

Tensorproduktbasen für Funktionen

Der Basissatz braucht nirgends, dass die Elemente von V und W Zahlentupel sind. Nehmen wir Funktionenräume, so wird aus ihm ein Werkzeug für die Approximation multivariater Funktionen.

Wir starten univariat. Der Raum der Polynome vom Grad höchstens 1 auf $[0, 1]$ ist

$$\mathcal{P}_1 = \{p(x) = a_0 + a_1 x : a_0, a_1 \in \mathbb{R}\}$$

mit der Basis $\mathcal{B} = \{1, x\}$, also $\dim(\mathcal{P}_1) = 2$. Für zwei Funktionen $f, g : [0, 1] \rightarrow \mathbb{R}$ setzen wir

$$(f \otimes g)(x, y) := f(x) g(y). \quad (9.4.4)$$

Das Produkt zweier univariater Funktionen ist also eine Funktion auf dem Quadrat $[0, 1]^2$, und die Zuordnung $(f, g) \mapsto f \otimes g$ ist bilinear: Ein Skalar vor f landet vor dem Produkt, ebenso einer vor g , und Summen im ersten oder zweiten Argument ziehen sich punktweise auseinander. Wir benennen Funktionen dabei nach dem Ausdruck, den sie liefern: Die konstante Funktion heißt in beiden Faktoren 1, die identische im ersten x und im zweiten y . Die zweite Basis ist also dieselbe wie die erste, nur anders aufgeschrieben.

Beispiel 9.4.9 (Die Tensorproduktbasis von $\mathcal{P}_1 \otimes \mathcal{P}_1$)

Satz 9.4.7 liefert aus $\mathcal{B}_V = \{1, x\}$ und $\mathcal{B}_W = \{1, y\}$ die vier Produkte, ausgewertet nach (9.4.4):

$$\begin{aligned}\phi_1(x, y) &= (1 \otimes 1)(x, y) = 1 \cdot 1 = 1, & \phi_2(x, y) &= (x \otimes 1)(x, y) = x \cdot 1 = x, \\ \phi_3(x, y) &= (1 \otimes y)(x, y) = 1 \cdot y = y, & \phi_4(x, y) &= (x \otimes y)(x, y) = x \cdot y = xy.\end{aligned}$$

Sie bilden eine Basis von $\mathcal{P}_1 \otimes \mathcal{P}_1$, dem Raum aller bivariaten Polynome, die in jeder der beiden Variablen höchstens den Grad 1 haben. Die Dimension ist

$$\dim(\mathcal{P}_1 \otimes \mathcal{P}_1) = \dim(\mathcal{P}_1) \dim(\mathcal{P}_1) = 2 \cdot 2 = 4.$$

Nummerieren wir die Koeffizienten wie in (9.4.2), also c_{ij} mit i für den ersten und j für den zweiten Faktor, so ist c_{11} der Koeffizient von $1 \otimes 1$, c_{21} der von $x \otimes 1$, c_{12} der von $1 \otimes y$ und c_{22} der von $x \otimes y$.

Beispiel 9.4.10 (Eine bivariate Funktion in dieser Basis)

Nehmen wir

$$f(x, y) = 2 + 3x - y + 5xy.$$

Die Entwicklung ist abzulesen:

$$f = 2 \phi_1 + 3 \phi_2 + (-1) \phi_3 + 5 \phi_4, \quad c_{11} = 2, \quad c_{21} = 3, \quad c_{12} = -1, \quad c_{22} = 5.$$

Bemerkung 9.4.11 (Was der Produktbau bedeutet)

Drei Beobachtungen an dieser Basis, die sich auf jeden Tensorproduktraum von Funktionen übertragen.

1. Halten wir y fest, so ist $x \mapsto f(x, y)$ ein Polynom aus $\mathcal{P}_1$, und umgekehrt. Jeder achsenparallele Schnitt durch die Fläche ist also eine Gerade.
2. Ohne den vierten Basisvektor $x \otimes y$ ist f eine Ebene, die beiden Variablen wirken dann rein additiv. Erst c_{22} koppelt sie: Die Steigung in x -Richtung ist $c_{21} + c_{22}y$ und hängt damit vom festgehaltenen y ab.
3. Der Grad in x und der Grad in y sind je höchstens 1, der Gesamtgrad von xy ist aber 2. Tensorprodukträume begrenzen den Grad in jeder Variablen einzeln, nicht den Gesamtgrad.

Interaktiv: Vier Koeffizienten, eine Fläche

Wie erkennen wir an derselben Stelle im Quadrat und auf der Fläche, ob der gemischte Basisanteil die beiden Variablen koppelt?

Interaktives Element — nur in der Web-Fassung

Der gemischte Basisanteil ist genau der Teil, der die beiden Variablen in der Fläche koppelt.

Bemerkung 9.4.12 (Allgemeine Dimension, und was sie kostet)

Für k -variate Polynome mit Grad höchstens d in jeder Variablen ist der zugehörige Raum das k -fache Tensorprodukt $\mathcal{P}_d^{\otimes k} = \mathcal{P}_d \otimes \cdots \otimes \mathcal{P}_d$, und Satz 9.4.7 liefert seine Dimension als k -faches Produkt:

$$\dim(\mathcal{P}_d^{\otimes k}) = (d+1)^k.$$

Das wächst exponentiell in der Zahl der Variablen. Für $d = 3$ und $k = 10$ sind das $4^{10} = 1\,048\,576$ Basisfunktionen. Zum Vergleich: Der Raum der Polynome mit *Gesamtgrad* höchstens d in k Variablen hat nur $\binom{d+k}{k}$ Dimensionen, für $d = 3$ und $k = 10$ also $\binom{13}{10} = 286$. Der Produktbau ist bequem, aber teuer, und dieser Preis hat einen Namen: *Fluch der Dimensionalität* (curse of dimensionality).

In der Praxis begegnet uns die Konstruktion trotzdem überall:

- **Finite-Elemente-Methoden:** Auf Rechteckgittern werden Lösungen partieller Differentialgleichungen durch tensorbasierte Polynome approximiert.
- **Statistik:** Tensorprodukt-Splines dienen der multivariaten Regression, etwa in generalisierten additiven Modellen (generalized additive models, GAM).
- **Numerik:** Dünne Gitter (sparse grids) wählen aus den $(d+1)^k$ Basisfunktionen gezielt aus und drücken den Aufwand damit weit unter den des vollen Produktgitters.

Bemerkung 9.4.13 (Kronecker-Designmatrix für Tensorprodukt-Splines)

Seien $\mathbf{B}_x \in \mathbb{R}^{n_x \times K_x}$ und $\mathbf{B}_y \in \mathbb{R}^{n_y \times K_y}$ die Auswertungsmatrizen zweier univariater B-Spline-Basen. Für eine Koeffizientenmatrix $\mathbf{C} \in \mathbb{R}^{K_x \times K_y}$ enthält $\mathbf{F} = \mathbf{B}_x \mathbf{C} \mathbf{B}_y^\top$ die Werte der Tensorprodukt-Spline-Fläche auf dem $n_x \times n_y$ -Gitter. Der vec-Trick aus Satz 9.5.3 liefert

$$\text{vec}(\mathbf{F}) = (\mathbf{B}_y \otimes \mathbf{B}_x) \text{vec}(\mathbf{C}).$$

Die dichte Designmatrix $\mathbf{B}_y \otimes \mathbf{B}_x$ hat das Format $(n_x n_y) \times (K_x K_y)$ und damit $n_x n_y K_x K_y$ Einträge. Die beiden Faktoren speichern nur $n_x K_x + n_y K_y$ Einträge. Wir werten daher $\mathbf{B}_x \mathbf{C} \mathbf{B}_y^\top$ durch zwei kleine Matrixprodukte aus, statt die große Designmatrix aufzubauen. Das spart Speicher und meist auch Rechenzeit. In Kapitel 13 erscheint dieselbe Idee mit einer allgemeinen Designmatrix wieder.

Tensorproduktbasen sind damit das Standardwerkzeug, sobald multivariate Funktionen approximiert werden sollen. Nötig ist dafür zunächst eine gute Basis für univariate Funktionen, etwa Legendre-Polynome, eine Fourierbasis oder Wavelets. Das Tensorprodukt setzt daraus eine Basis multivariater Funktionen zusammen. Im Kapitel zur Funktionsapproximation kommen wir darauf zurück.

Selbsttest

1. Wenn wir im Widget c_{22} auf null setzen, wird die dargestellte Fläche eine Ebene.

Antwort: Wahr

Dann hängt die Steigung in x-Richtung nicht mehr vom gewählten y-Wert ab.

2. Mit dem äußeren Produkt als $\otimes$ ist $\mathbb{R}^2 \otimes \mathbb{R}^3 = \mathbb{R}^{2 \times 3}$, und dieser Raum hat die Dimension 6.

Antwort: Wahr

Das ist Beispiel 9.4.3 mit $m = 2$ und $n = 3$: Die sechs Matrizen $\mathbf{e}_i \otimes \mathbf{e}_j = \mathbf{E}_{ij}$ liegen im Spann der äußeren Produkte und bilden eine Basis von $\mathbb{R}^{2 \times 3}$. Nach Satz 9.4.7 ist die Dimension $2 \cdot 3 = 6$.

3. Jedes Element von $V \otimes W$ lässt sich als ein einzelnes Produkt $\mathbf{v} \otimes \mathbf{w}$ schreiben.

Antwort: Falsch

$V \otimes W$ ist der Spann der elementaren Tensoren, und Summen von Produkten sind im Allgemeinen selbst keine Produkte. Beispiel 9.4.5 zeigt das an $I_2 \in \mathbb{R}^2 \otimes \mathbb{R}^2$: Als Produkt hätte die Matrix höchstens den Rang 1, sie hat aber den Rang 2. Als Summe von zwei elementaren Tensoren geht es.

4. Für Vektorräume V und W gilt $\dim(V \otimes W) = \dim(V) + \dim(W)$.

Antwort: Falsch

Die Dimensionen multiplizieren sich, sie addieren sich nicht (Satz 9.4.7). Für $\dim(V) = 2$ und $\dim(W) = 3$ ist $\dim(V \otimes W) = 6$ und nicht 5. Die Basis besteht aus allen Paaren $\mathbf{v}_i \otimes \mathbf{w}_j$, und davon gibt es $\dim(V) \dim(W)$ Stück.

5. Jede Matrix $A \in \mathbb{R}^{m \times n}$ ist eine Summe von höchstens $\min(m, n)$ elementaren Tensoren.

Antwort: Wahr

Das ist Satz 9.4.6. Die Summenform der Singulärwertzerlegung liefert $A = \sum_{i=1}^r \sigma_i \mathbf{u}_i \otimes \mathbf{v}_i$ mit $r = \text{rang}(A)$, und der Rang übersteigt weder die Zeilen- noch die Spaltenzahl. Weniger als r Summanden reichen nicht, r ist also sogar die kleinstmögliche Anzahl.

6. In der Darstellung $\sum_i \sum_j c_{ij} \mathbf{v}_i \otimes \mathbf{w}_j$ haben die Koeffizienten stets die Form $c_{ij} = a_i b_j$.

Antwort: Falsch

Diese Produktform haben genau die elementaren Tensoren, wie die Entwicklung (9.4.3) zu Satz 9.4.7 zeigt. Allgemeine Elemente von $V \otimes W$ haben beliebige Koeffizienten; bezüglich der Standardbasen ist die Koeffizientenmatrix zu I_2 die Einheitsmatrix selbst, und die hat nicht die Form $\mathbf{a}\mathbf{b}^\top$ (Bemerkung 9.4.8).

7. Die Basis $\{1 \otimes 1, x \otimes 1, 1 \otimes y, x \otimes y\}$ spannt alle Polynome in zwei Variablen vom Gesamtgrad höchstens 2 auf.

Antwort: Falsch

Sie spannt die Polynome auf, die in jeder Variablen einzeln höchstens den Grad 1 haben, und das sind vier Basisfunktionen. Der Raum aller Polynome vom Gesamtgrad höchstens 2 hat dagegen die Dimension $\binom{4}{2} = 6$: Es fehlen x^2 und y^2 . Tensorprodukträume begrenzen den Grad in jeder Variablen einzeln, nicht den Gesamtgrad (Bemerkungen 9.4.11 und 9.4.12).

Vertiefung: vgl. MML §2.6 zu Basis und Dimension eines Vektorraums und MML §4.5 bis §4.6, wo die Zerlegung einer Matrix in Rang-1-Anteile ausgeführt wird.

9.5 Zusammenfassung

Vier Abschnitte, ein Faden: Begonnen haben wir mit Abbildungen, die mehrere Vektoren gleichzeitig verarbeiten, geendet sind wir bei Vektorräumen, die aus zwei anderen zusammengesetzt werden. Dazwischen stand der Tensor als das Objekt, das beides verbindet. Sammeln wir ein, was bleiben soll.

Die Kernkonzepte

Bemerkung 9.5.1 (Sechs Begriffe, die bleiben)

1. **Tensoren** verallgemeinern Vektoren und Matrizen auf mehr als zwei Indexpositionen. Ein Tensor der Stufe k ist eine durch k Indizes indizierte Familie reeller Zahlen, also ein Element von $\mathbb{R}^{n_1 \times \dots \times n_k}$ (Definition 9.2.3, Abschnitt 9.2); Stufe 1 ist ein Vektor, Stufe 2 eine Matrix. Gebraucht werden höhere Stufen im Deep Learning, in der Bild- und Videobearbeitung und in der modernen Statistik; ausführliche Bild- und Feature-Map-Beispiele stehen als Vertiefung in Abschnitt 9.2.
2. **Multilinearität** ist etwas anderes als Linearität. Linear heißt $f(c\mathbf{v} + c'\mathbf{w}) = c f(\mathbf{v}) + c' f(\mathbf{w})$, multilinear dagegen

$$f(\mathbf{v}_1, \dots, c\mathbf{v}_i + c'\mathbf{v}'_i, \dots, \mathbf{v}_n) = c f(\mathbf{v}_1, \dots, \mathbf{v}_i, \dots, \mathbf{v}_n) + c' f(\mathbf{v}_1, \dots, \mathbf{v}'_i, \dots, \mathbf{v}_n),$$

also Linearität in jedem einzelnen Argument bei festgehaltenen übrigen (Definition 9.1.1, Abschnitt 9.1). Die Matrizenmultiplikation ist bilinear, aber nicht linear (Beispiel 9.1.6). Der Grund gilt allgemein: Skalieren wir alle n Argumente mit demselben c , so tritt c^n heraus statt c (Bemerkung 9.1.2).

3. **Das äußere Produkt** $\mathbf{v} \otimes \mathbf{w} = \mathbf{v} \mathbf{w}^\top$ macht aus zwei Vektoren $\neq \mathbf{0}$ eine Matrix vom Rang 1 (Satz 9.3.4, Abschnitt 9.3). Die Singulärwertzerlegung zerlegt umgekehrt jede Matrix in genau solche Bausteine, $\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \otimes \mathbf{v}_i$ (Abschnitt 6.4, Bemerkung 9.3.5).
4. **Das Tensorprodukt** überträgt diese Vorschrift auf beliebige Stufen: Die Stufen addieren sich, die Einträge multiplizieren sich (Definition 9.3.7, Bemerkung 9.3.8).
5. **Das Kroneckerprodukt** ist die flach angeordnete Fassung für zwei Matrizen, dieselben Zahlen als Blockmatrix statt als Tensor der Stufe 4 (Definition 9.3.11, Bemerkung 9.3.12). Es trägt die Vektorisierung von Matrixgleichungen, $\text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = (\mathbf{B}^\top \otimes \mathbf{A}) \text{vec}(\mathbf{X})$ (Satz 9.5.3), und in der Statistik die Kronecker-strukturierten Kovarianzmatrizen multivariater Verteilungen, von Zeitreihen und von raumzeitlichen Daten (Definition 9.3.17).
6. **Tensorprodukte bauen größere Räume aus kleineren.** Der Raum $V \otimes W$ ist die lineare Hülle aller Produkte $\mathbf{v} \otimes \mathbf{w}$ (Definition 9.4.1, Abschnitt 9.4), und aus Basen der Faktoren entsteht durch Ausmultiplizieren eine Basis des Produktraums mit $\dim(V \otimes W) = \dim(V) \dim(W)$ (Satz 9.4.7). Mit Funktionenräumen als Faktoren liefert das multivariate Basen aus univariaten, die Grundlage der Funktionsapproximation und höherdimensionaler Splines.

Vektorisierung von Matrixgleichungen

Ein Punkt dieser Liste verdient mehr als einen Halbsatz, weil er Kapitel 9 an Kapitel 5 zurückbindet. Eine Gleichung der Bauart $\mathbf{A}\mathbf{X}\mathbf{B} = \mathbf{C}$ ist linear in der Unbekannten $\mathbf{X}$, aber diese Unbekannte ist eine Matrix. Die Verfahren aus Abschnitt 5.2 erwarten dagegen ein System $\mathbf{A}\mathbf{x} = \mathbf{b}$ mit einem Vektor als Unbekannter. Also stapeln wir die Matrix zu einem Vektor um.

Definition 9.5.2 (Vektorisierung)

Für $\mathbf{X} \in \mathbb{R}^{p \times q}$ mit den Spalten $\mathbf{x}_1, \dots, \mathbf{x}_q \in \mathbb{R}^p$ ist die *Vektorisierung* (vectorization)

$$\text{vec}(\mathbf{X}) = \begin{pmatrix} \mathbf{x}_1 \\ \vdots \\ \mathbf{x}_q \end{pmatrix} \in \mathbb{R}^{pq},$$

also der Vektor, der die Spalten von $\mathbf{X}$ untereinanderhängt. Gestapelt wird spaltenweise, so wie $\mathbf{R}$ eine Matrix ohnehin abspeichert.

Satz 9.5.3 (Vektorisierung eines Matrixprodukts)

Seien $\mathbf{A} \in \mathbb{R}^{m \times p}$, $\mathbf{X} \in \mathbb{R}^{p \times q}$ und $\mathbf{B} \in \mathbb{R}^{q \times n}$. Dann gilt

$$\text{vec}(\mathbf{A} \mathbf{X} \mathbf{B}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X}). \quad (9.5.1)$$

Im Kroneckerprodukt steht $\mathbf{B}^\top$ als erster und $\mathbf{A}$ als zweiter Faktor; die Matrix $\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}$ hat das Format $nm \times qp$, passend zu $\text{vec}(\mathbf{X}) \in \mathbb{R}^{pq}$ und $\text{vec}(\mathbf{A} \mathbf{X} \mathbf{B}) \in \mathbb{R}^{mn}$.

Vertiefung: Der vec-Trick, blockweise nachgerechnet

Beweis.

- (1) Wir lesen beide Seiten blockweise, mit Blöcken der Länge m . Sind $\mathbf{x}_1, \dots, \mathbf{x}_q$ die Spalten von $\mathbf{X}$ und b_{kj} die Einträge von $\mathbf{B}$, so ist die j -te Spalte von $\mathbf{A} \mathbf{X} \mathbf{B}$

$$\mathbf{A} \mathbf{X} \mathbf{B} \mathbf{e}_j = \mathbf{A} \mathbf{X} \begin{pmatrix} b_{1j} \\ \vdots \\ b_{qj} \end{pmatrix} = \sum_{k=1}^q b_{kj} \mathbf{A} \mathbf{x}_k.$$

$\mathbf{B} \mathbf{e}_j$ ist die j -te Spalte von $\mathbf{B}$; ein Matrix-Vektor-Produkt ist die Linearkombination der Spalten mit den Einträgen des Vektors

- (2) Auf der rechten Seite von (9.5.1) zerlegen wir $\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}$ nach Definition 9.3.11 in Blöcke der Größe $m \times p$. Der Block an der Stelle (j, k) ist $(\mathbf{B}^\top)_{jk} \mathbf{A} = b_{kj} \mathbf{A}$, und der k -te Block von $\text{vec}(\mathbf{X})$ ist $\mathbf{x}_k$. Der j -te Block des Produkts lautet damit $\sum_{k=1}^q b_{kj} \mathbf{A} \mathbf{x}_k$. Das ist genau die Spalte aus Schritt 1, und da j beliebig war, stimmen beide Seiten überein.

Blockmultiplikation: der j -te Block eines Produkts ist die Summe der Blockprodukte über k ; das Transponieren vertauscht die Indizes, deshalb steht dort b_{kj} und nicht b_{jk}

□

Interaktiv: Die Matrixgleichung als lineares System

Welche der beiden Rechnungen liefert denselben Vektor, wenn wir die Einträge von $\mathbf{X}$ verändern?

Interaktives Element — nur in der Web-Fassung

Die Vektorisierung bewahrt die lineare Wirkung von links und rechts und macht damit Standardverfahren für lineare Gleichungssysteme anwendbar.

Beispiel 9.5.4 (Die Identität an 2×2-Matrizen)

Für

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 1 & 0 \\ 2 & 3 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}$$

rechnen wir zuerst direkt:

$$\mathbf{A}\mathbf{X} = \begin{pmatrix} 5 & 6 \\ 2 & 3 \end{pmatrix}, \quad \mathbf{A}\mathbf{X}\mathbf{B} = \begin{pmatrix} 5 & 17 \\ 2 & 8 \end{pmatrix}, \quad \text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = \begin{pmatrix} 5 \\ 2 \\ 17 \\ 8 \end{pmatrix}.$$

Und nun über (9.5.1). Mit $\mathbf{B}^\top = \begin{pmatrix} 1 & 0 \\ 1 & 2 \end{pmatrix}$ ist

$$\mathbf{B}^\top \otimes_K \mathbf{A} = \begin{pmatrix} 1 \cdot \mathbf{A} & 0 \cdot \mathbf{A} \\ 1 \cdot \mathbf{A} & 2 \cdot \mathbf{A} \end{pmatrix} = \begin{pmatrix} 1 & 2 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 2 & 2 & 4 \\ 0 & 1 & 0 & 2 \end{pmatrix}, \quad \text{vec}(\mathbf{X}) = \begin{pmatrix} 1 \\ 2 \\ 0 \\ 3 \end{pmatrix},$$

und das Produkt der beiden ist $(5, 2, 17, 8)^\top$, wie versprochen. Auf die Reihenfolge kommt es dabei an: $\mathbf{A} \otimes_K \mathbf{B}^\top$ liefert $(1, 17, 0, 6)^\top$ und damit etwas ganz anderes. Die Formatprobe schützt uns davor grundsätzlich nicht: Zeilen- und Spaltenzahl eines Kroneckerprodukts entstehen als Produkte der Zeilen- bzw. Spaltenzahlen beider Faktoren, und Produkte hängen nicht von der Reihenfolge ab. $\mathbf{B}^\top \otimes_K \mathbf{A}$ und $\mathbf{A} \otimes_K \mathbf{B}^\top$ haben deshalb stets dasselbe Format, hier 4×4 , und beide lassen sich mit $\text{vec}(\mathbf{X})$ multiplizieren.

Beispiel 9.5.5 (Sylvester-Gleichung mit dem vec-Trick)

Die *Sylvester-Gleichung* lautet

$$\mathbf{A}\mathbf{X} + \mathbf{X}\mathbf{B} = \mathbf{C}.$$

Mit den beiden Spezialfällen von (9.5.1) wird daraus

$$(\mathbf{I}_2 \otimes_K \mathbf{A} + \mathbf{B}^\top \otimes_K \mathbf{I}_2) \text{vec}(\mathbf{X}) = \text{vec}(\mathbf{C}).$$

Betrachten wir

$$\mathbf{A} = \begin{pmatrix} 3 & 0 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}.$$

Das zugehörige lineare Gleichungssystem hat die Systemmatrix $\begin{pmatrix} 3 & 0 & 1 & 0 \\ 1 & 2 & 0 & 1 \\ 1 & 0 & 4 & 0 \\ 0 & 1 & 1 & 3 \end{pmatrix}$. Seine Lösung ist

$$\text{vec}(\mathbf{X}) = \begin{pmatrix} 1/11 \\ 27/55 \\ 8/11 \\ 51/55 \end{pmatrix}, \quad \mathbf{X} = \begin{pmatrix} 1/11 & 8/11 \\ 27/55 & 51/55 \end{pmatrix}.$$

In R folgt die Rechnung direkt der Formel:

```
A <- matrix(c(3, 1, 0, 2), 2, 2)
B <- matrix(c(0, 1, 1, 1), 2, 2)
C <- matrix(c(1, 2, 3, 4), 2, 2)
M <- kronecker(diag(2), A) + kronecker(t(B), diag(2))
```

```
X <- matrix(solve(M, as.vector(C)), 2, 2)
A %*% X + X %*% B
all.equal(A %*% X + X %*% B, C)
```

Die letzte Zeile ergibt TRUE. Lyapunov-Gleichungen, etwa für stationäre Kovarianzmatrizen von Zeitreihenmodellen, sind wichtige Spezialfälle. Für große Matrizen verwenden wir spezialisierte Löser, die die Struktur ausnutzen.

Aus $\mathbf{AXB} = \mathbf{C}$ wird mit (9.5.1) ein gewöhnliches lineares Gleichungssystem $(\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X}) = \text{vec}(\mathbf{C})$ mit mn Gleichungen und pq Unbekannten. Der Gewinn ist begrifflich, der Preis ist Speicher und Rechenzeit: Ausgeschrieben hat die Systemmatrix $mnpq$ Einträge. Praktisch nutzen wir deshalb die Struktur, statt sie auszumultiplizieren. Sind $\mathbf{A}$ und $\mathbf{B}$ quadratisch und invertierbar, so liefert $\mathbf{X} = \mathbf{A}^{-1} \mathbf{C} \mathbf{B}^{-1}$ die Lösung allein aus den beiden kleinen Faktoren. Das ist eine symbolische Identität; numerisch bilden wir die Inversen nicht explizit, sondern lösen nacheinander lineare Systeme mit $\mathbf{A}$ und $\mathbf{B}^\top$. Nach demselben Muster arbeiten die separierbaren Kovarianzmatrizen aus Abschnitt 9.3: Dort lesen wir die Eigenwerte von Σ als Produkte der Eigenwerte beider Faktoren ab, ohne Σ je aufzustellen.

Wie es weitergeht

Offen geblieben ist der Bogen zur Approximation. Eine gute Basis für univariate Funktionen, etwa Legendre-Polynome, eine Fourierbasis oder Wavelets, liefert über das Tensorprodukt sofort eine Basis für multivariate Funktionen. Wie wir damit tatsächlich approximieren und wie wir dem exponentiellen Wachstum $(d+1)^k$ aus Bemerkung 9.4.12 entkommen, ist das Thema von Kapitel 13.

Das nächste Kapitel, Kapitel 10, wendet sich der fortgeschrittenen Differentialrechnung zu. Tensoren begegnen uns dort gleich wieder: Leiten wir einen matrixwertigen Ausdruck nach einer Matrix ab (Abschnitt 10.4), so ist das Ergebnis von Haus aus ein mehrdimensionales Zahlenfeld, und die Vektorisierung aus Satz 9.5.3 ist eines der Werkzeuge, mit denen sich das ordnen lässt.

Selbsttest

Vier Aussagen quer durch das Kapitel. Welche davon stimmen?

1. Die Schätzung aus dem Kronecker-Widget zeigt: Vertauschen der Faktoren kann die Blockstruktur ändern, obwohl das Ergebnisformat gleich bleibt.

Antwort: Wahr

Das ist die Nicht-Kommutativität aus Bemerkung 9.3.14.

2. Mit spaltenweiser Vektorisierung gilt $\text{vec}(\mathbf{AXB}) = (\mathbf{A} \otimes_{\mathbb{K}} \mathbf{B}^\top) \text{vec}(\mathbf{X})$.

Antwort: Falsch

Die Faktoren stehen vertauscht. Richtig ist $\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes_{\mathbb{K}} \mathbf{A}) \text{vec}(\mathbf{X})$ nach (9.5.1): Der transponierte *rechte* Faktor steht im Kroneckerprodukt *links*. Die Formatprobe merkt den Tausch nie, denn beide Anordnungen führen auf dasselbe Format. In Beispiel 9.5.4 ist das 4×4 , und die vertauschte Fassung liefert dort $(1, 17, 0, 6)^\top$ statt des korrekten $\text{vec}(\mathbf{AXB}) = (5, 2, 17, 8)^\top$.

3. Für $\mathbf{v} \in \mathbb{R}^m$ und $\mathbf{w} \in \mathbb{R}^n$ hat das äußere Produkt $\mathbf{v} \otimes \mathbf{w}$ stets den Rang 1.

Antwort: Falsch

Satz 9.3.4 setzt $\mathbf{v} \neq \mathbf{0}$ und $\mathbf{w} \neq \mathbf{0}$ voraus, und diese Voraussetzung ist nötig: Ist einer der beiden Vektoren der Nullvektor, so sind nach (9.3.1) alle Einträge $v_i w_j$ gleich null, das Produkt ist die Nullmatrix und hat den Rang 0. Andernfalls stimmt die Aussage, denn dann spannt $\mathbf{v}$ allein den Spaltenraum auf.

4. Für $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $\mathbf{B} \in \mathbb{R}^{p \times q}$ enthalten das Tensorprodukt und das Kroneckerprodukt dieselben $mnpq$ Zahlen, nur anders angeordnet.

Antwort: Wahr

Beide Male entsteht jede Zahl als ein Produkt $a_{i_1 i_2} b_{j_1 j_2}$ (Bemerkung 9.3.12). Das Tensorprodukt legt sie in einen Tensor der Stufe 4 mit Format $m \times n \times p \times q$, das Kroneckerprodukt flacht dieselben Werte zu einer Matrix aus $\mathbb{R}^{mp \times nq}$ ab. Welche Lesart gemeint ist, verrät in der Praxis das Format des Ergebnisses.

5. Kombinieren wir k univariate Basen mit je $d + 1$ Funktionen zu einer Tensorproduktbasis, so hat diese $(d + 1)^k$ Elemente.

Antwort: Wahr

Satz 9.4.7 multipliziert die Dimensionen, und bei k Faktoren entsteht so das k -fache Produkt (Bemerkung 9.4.12). Für $d = 3$ und $k = 10$ sind das $4^{10} = 1\,048\,576$ Basisfunktionen, während der Raum der Polynome mit Gesamtgrad höchstens 3 in 10 Variablen nur $\binom{13}{10} = 286$ Dimensionen hat. Bequem gebaut, aber teuer bezahlt.

Vertiefung: Den vec-Operator und die Rechenregeln rund um $\text{vec}(\mathbf{AXB})$ entwickelt Kapitel 2 von Magnus und Neudecker, Matrix Differential Calculus with Applications in Statistics and Econometrics; von dort führt dasselbe Buch weiter in die Matrixdifferentialrechnung, die als Nächstes ansteht.

Differentialrechnung

10.1 Ableitung als lineare Approximation

Interaktiv: Landkarte für Analysis und Optimierung

Mit diesem Kapitel beginnt der zweite Teil des Skripts, *Analysis & Optimierung*. Die Karte zeigt, wie seine Konzepte zusammenhängen, von der Fréchet-Ableitung, die wir gleich einführen, bis zu den Optimierungsverfahren in Kapitel 12. Beim ersten Lesen genügt der Gesamteindruck; am Ende von Kapitel 12 lohnt sich ein zweiter Blick.

Interaktives Element — nur in der Web-Fassung

Wählen wir einen Begriff aus, hebt die Karte seine Voraussetzungen und Folgerungen hervor; verlinkte Kästen führen direkt zum zugehörigen Kapitel.

Was wir mitbringen

Dieses Kapitel steht auf zwei Beinen. Aus der linearen Algebra brauchen wir die Spur einer Matrix aus Abschnitt 3.1 und die Matrixnormen aus Abschnitt 3.2, dazu Determinante und Matrixinversion. Außerdem hantieren wir mit normierten Vektorräumen, also Vektorräumen mit einer Norm, und mit beschränkten linearen Abbildungen darauf. Aus der Analysis setzen wir die Ableitung $f'(x)$ voraus, die partiellen Ableitungen $\partial f(x)/\partial x_i$, die lineare Approximation $f(x+h) \approx f(x) + f'(x)h$ und die Grundzüge der Taylorreihen.

Ein Werkzeug verdient eine eigene Zeile, weil das ganze Kapitel daran hängt: die Landau-Notation für kleine Argumente. Für eine Funktion g schreiben wir

$$g(h) = o(h) \quad :\Longleftrightarrow \quad \lim_{h \rightarrow 0} \frac{g(h)}{h} = 0.$$

Ein $o(h)$ -Term verschwindet also nicht einfach nur, er verschwindet *schneller als h selbst*: Sein Verhältnis zu h geht gegen null. Für $g(h) = h^2$ etwa viertelt sich der Term, wenn wir h halbieren. Genauso schreiben wir $g(h) = o(|h|)$ und später $o(\|h\|)$, wenn der Quotient mit $|h|$ beziehungsweise $\|h\|$ gegen null geht; für reelles h ist $o(|h|)$ dasselbe wie $o(h)$, denn $g(h)/h$ und $g(h)/|h|$ unterscheiden sich höchstens im Vorzeichen. Die Landau-Symbole haben wir in Abschnitt 2.4 für große Argumente kennengelernt, wo sie Rechenaufwand messen; hier läuft das Argument gegen null statt gegen unendlich, die Rechenregeln bleiben dieselben (O-Notation).

Wohin geht die Reise? Ableitungen und Differenzierbarkeit kennen wir aus der Analysis für Funktionen einer reellen Variablen. Neu ist in diesem Kapitel ein allgemeinerer Ableitungsbegriff, die *Fréchet-Ableitung*. Mit ihm können wir Funktionen ableiten, die Vektoren oder Matrizen als Ein-

gabe oder als Ausgabe haben, und wir bleiben dabei nicht auf den $\mathbb{R}^n$ beschränkt. Die Spezialfälle mit Vektor- und Matrixargumenten füllen die Abschnitte 10.2 bis 10.4. Ab Abschnitt 10.5 geht es dann ums Rechnen mit dem neuen Begriff: Eigenschaften, Produkt- und Kettenregel, höhere Ableitungen und die Taylorapproximation. Der Leitgedanke steht schon in der Überschrift dieses Abschnitts: *Ableitung = lineare Approximation*.

Die Ableitung im Eindimensionalen

Fangen wir dort an, wo wir uns auskennen. Für eine Funktion einer reellen Variablen misst der Differenzenquotient, wie stark sich der Funktionswert ändert, wenn wir das Argument um h verrücken.

Definition 10.1.1 (Differenzierbarkeit)

Seien $S \subseteq \mathbb{R}$ offen und $f : S \rightarrow \mathbb{R}$ eine Funktion. Wir nennen f an der Stelle $x \in S$ *differenzierbar* mit *Ableitung* $f'(x)$, falls der Grenzwert

$$\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = f'(x) \quad (10.1.1)$$

existiert.

Geometrisch ist der Quotient in (10.1.1) die Steigung der Sekante durch die beiden Punkte $(x, f(x))$ und $(x+h, f(x+h))$. Lassen wir h gegen null laufen, so rutscht der zweite Punkt auf den ersten zu, und die Sekante dreht sich in die Tangente. Die Ableitung $f'(x)$ ist deren Steigung.

Das Wort „der Grenzwert“ in Definition 10.1.1 trägt mehr Gewicht, als es aussieht. Verlangt ist ein einziger Wert, gegen den der Differenzenquotient strebt, egal von welcher Seite und in welchem Tempo wir uns nähern. Daran kann es scheitern.

Bemerkung 10.1.2 (Wenn es keine eindeutige Tangente gibt)

Betrachten wir $f(x) = |x|$ an der Stelle $x = 0$. Für $h > 0$ ist der Differenzenquotient

$$\frac{|0+h| - |0|}{h} = \frac{h}{h} = 1,$$

für $h < 0$ dagegen $-h/h = -1$. Von rechts kommend messen wir also die Steigung 1, von links die Steigung -1 , und zwar für *jedes* h , nicht erst im Grenzwert. Der Grenzwert in (10.1.1) existiert damit nicht, f ist in $x = 0$ nicht differenzierbar. Anschaulich lässt sich an einen Knick keine eindeutige Tangente anlegen. Die verschiedenen einseitigen Sekantensteigungen zeigen bereits, warum. Daneben gibt es für jedes $-1 \leq m \leq 1$ eine Gerade $y = mx$, die den Graphen von unten stützt, denn $mx \leq |x|$ für alle x . Das sind *Stützgeraden*, keine Tangenten im Sinn der Ableitung; ihre Steigungen werden in der konvexen Optimierung später als Subgradienten bezeichnet. Stetig ist die Betragsfunktion trotzdem, Knicke sind also kein Widerspruch zur Stetigkeit.

Ableitung als lineare Approximation

Definition 10.1.1 beschreibt die Ableitung als Grenzwert. Für alles, was danach kommt, ist eine zweite Lesart nützlicher. Stellen wir (10.1.1) um und lösen nach $f(x+h)$ auf, so steht dort: In der Nähe von x verhält sich f wie eine Gerade, und $f'(x)$ ist deren Steigung. Der Fehler dieser Näherung ist klein, und das lässt sich präzise sagen.

Satz 10.1.3 (Ableitung als lineare Approximation)

Seien $S \subseteq \mathbb{R}$ offen, $f : S \rightarrow \mathbb{R}$, $x \in S$ und $a \in \mathbb{R}$. Dann sind die folgenden drei Aussagen gleichwertig:

1. $\lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h} = a$,
2. $\lim_{h \rightarrow 0} \frac{|f(x+h) - f(x) - ah|}{|h|} = 0$,
3. $f(x+h) = f(x) + ah + o(|h|)$ für $h \rightarrow 0$.

Trifft eine davon zu, so ist f in x differenzierbar mit $f'(x) = a$.

Vertiefung: Warum die drei Ableitungsfassungen gleichwertig sind

Beweis.

- (1) Zu (1) $\iff$ (2): Für $h \neq 0$ ziehen wir a in den Bruch hinein,

$$\frac{f(x+h) - f(x)}{h} - a = \frac{f(x+h) - f(x) - ah}{h}.$$

Der Differenzenquotient strebt also genau dann gegen a , wenn die rechte Seite gegen 0 strebt.

$a = ah/h$ für $h \neq 0$, dann beide Brüche zusammenfassen

- (2) Eine Größe strebt genau dann gegen null, wenn ihr Betrag es tut, und Beträge vertragen sich mit dem Bruch:

$$\left| \frac{f(x+h) - f(x) - ah}{h} \right| = \frac{|f(x+h) - f(x) - ah|}{|h|}.$$

Damit ist (1) gleichwertig zu (2).

$|z/h| = |z|/|h|$; und $\lim_{h \rightarrow 0} g(h) = 0$ gilt genau dann, wenn $\lim_{h \rightarrow 0} |g(h)| = 0$

- (3) Zu (2) $\implies$ (3): Wir taufen den Fehler auf einen Namen und setzen

$$r(h) := f(x+h) - f(x) - ah.$$

Aussage (2) besagt $|r(h)|/|h| \rightarrow 0$, also $r(h) = o(|h|)$. Einsetzen in die Definition von $r(h)$ liefert

$$f(x+h) = f(x) + ah + r(h) = f(x) + ah + o(|h|),$$

und das ist (3).

Definition des Landau-Symbols: $g(h) = o(|h|)$ heißt genau, dass $g(h)/|h|$ gegen null strebt

- (4) Zu (3) $\implies$ (2): Gilt (3), so ist die Differenz $f(x+h) - f(x) - ah$ ein Term der Ordnung $o(|h|)$. Nach Definition strebt sein Quotient mit $|h|$ gegen null, und dann tut es auch der Betrag dieses Quotienten. Damit ist der Ring geschlossen, und Aussage (1) ist wörtlich Definition 10.1.1 mit $f'(x) = a$.

wieder $\lim g = 0 \iff \lim |g| = 0$, diesmal in der Gegenrichtung

□

Aussage (3) ist die Fassung, die uns durch das ganze Kapitel trägt: *die Funktion, ihre lineare Näherung, und ein Rest, der schneller verschwindet als h* . Farblich halten wir das durch: blau die Funktion und ihre Werte, grün die lineare Approximation, rot der Restterm.

Bemerkung 10.1.4 (Was hier eigentlich linear ist)

Eine Verwechslung lohnt sich auszuräumen, bevor wir verallgemeinern. Die Ableitung f' selbst ist im Allgemeinen keine lineare Funktion: für $f(x) = \sin(x)$ etwa ist $f'(x) = \cos(x)$, und der Kosinus ist alles andere als linear.

Linear ist eine andere Abbildung, nämlich

$$D_x f : \mathbb{R} \rightarrow \mathbb{R}, \quad h \mapsto f'(x)h$$

bei festem x . Sie erfüllt $f'(x)(h_1 + h_2) = f'(x)h_1 + f'(x)h_2$ und $f'(x)(ch) = c f'(x)h$, ist also linear in h . Wie $f'(x)$ von x abhängt, bleibt davon völlig unberührt. Diese Trennung ist der Hebel für den Rest des Kapitels: Was wir verallgemeinern wollen, ist nicht die Zahl $f'(x)$, sondern die lineare Abbildung $D_x f$.

Zwei Fragen bleiben offen. Wie schnell verschwindet der Restterm, wenn wir h verkleinern? Und was geht schief, wenn die Funktion an der betrachteten Stelle einen Knick hat? Beides lässt sich an drei Beispielkurven ausprobieren.

Interaktiv: Sekante, Tangente und der Restterm zum Anfassen

Zur Wahl stehen zwei glatte Funktionen und die Betragsfunktion aus Bemerkung 10.1.2. Der rote Balken über $x + h$ ist der Restterm $r(h)$, der letzte Readout darunter der *relative Fehler* $|r(h)| / |h|$.

Interaktives Element — nur in der Web-Fassung

Dieser relative Fehler ist genau der Abstand zwischen Sekanten- und Tangentensteigung, und nach Satz 10.1.3(2) geht er gegen null. Bei $f(x) = x^2$ steht dort sogar exakt $|h|$, denn der Restterm ist h^2 (Beispiel 10.1.7); halbieren wir die Schrittweite, so fällt der Restterm auf ein Viertel und der relative Fehler auf die Hälfte. Die Betragsfunktion zeigt die Kehrseite: Schieben wir x auf 0, so trennen sich die beiden Sekanten und bleiben getrennt, wie klein h auch wird. Lehrreich ist eine Stelle knapp neben dem Knick – solange $|x| < h$ ist, liegt er noch im Fenster, und genau eine der beiden Sekanten läuft über ihn hinweg und misst etwas anderes als die Ableitung. Lineare Approximation ist eine lokale Angelegenheit.

Der allgemeine Ableitungsbegriff

Sobald wir Ableitung als lineare Approximation lesen, brauchen wir vom Definitionsbereich fast nichts mehr. Wir müssen Punkte verschieben können, $x + h$ muss also Sinn ergeben, wir müssen Länge messen können, um „der Fehler wird klein“ zu präzisieren, und wir brauchen einen Begriff von linearer Abbildung. Ein normierter Vektorraum liefert alle drei Zutaten. Auf metrischen Räumen ohne Vektorraumstruktur funktioniert es dagegen nicht, dort fehlt der lineare Teil.

Die folgende Definition geht auf Maurice Fréchet zurück. Sie reicht für alle endlich-dimensionalen Räume und für einige unendlich-dimensionale.

Definition 10.1.5 (Fréchet-Ableitung)

Seien $\mathbb{D}$ und $\mathbb{E}$ zwei normierte Vektorräume und $f : \mathbb{D} \rightarrow \mathbb{E}$. Wir nennen f an der Stelle $x \in \mathbb{D}$ (Fréchet-)differenzierbar, falls eine beschränkte lineare Abbildung $D_x f : \mathbb{D} \rightarrow \mathbb{E}$ und ein Restterm $r(h) \in \mathbb{E}$ existieren, sodass

$$f(x+h) = f(x) + D_x f(h) + r(h) \quad \text{und} \quad \lim_{h \rightarrow 0, h \neq 0} \frac{\|r(h)\|_{\mathbb{E}}}{\|h\|_{\mathbb{D}}} = 0. \quad (10.1.2)$$

Die Abbildung $D_x f$ heit dann *Frchet-Ableitung* von f an der Stelle x .

Wir schreiben $\mathbb{D}$ und $\mathbb{E}$ fr die beiden Rume; mit dem Erwartungswert hat das $\mathbb{E}$ hier nichts zu tun. Auch die beiden Normstriche in (10.1.2) meinen zwei verschiedene Dinge: $\|h\|$ ist die Norm auf $\mathbb{D}$, der Restterm wird dagegen in der Norm auf $\mathbb{E}$ gemessen. Die hufige Kurzschreibweise $r(h) = o(\|h\|)$ meint genau die Grenzwertbedingung in (10.1.2): Das o ist kein Skalar, den wir zu $f(x)$ addieren, sondern beschreibt die Gre des $\mathbb{E}$ -wertigen Restes.

Bemerkung 10.1.6 (Wie die Definition zu lesen ist)

Vier Punkte, die in der knappen Formulierung leicht untergehen.

Der Restterm. Setzen wir wie im Beweis von Satz 10.1.3

$$r(h) := f(x+h) - f(x) - D_x f(h),$$

so verlangt (10.1.2) genau $\|r(h)\| / \|h\| \rightarrow 0$ fr $h \rightarrow 0$. Die Gleichung selbst gilt fr jedes zulssige h , sie definiert ja blo $r(h)$; die Aussage steckt in der Kleinheit des Restes. Der Zusatz „fr $h \rightarrow 0$ “ gehrt deshalb nicht zur Gleichung, sondern zum Restterm: Er ist $o(\|h\|)$ fr $h \rightarrow 0$.

Beschrnktheit. Dass $D_x f$ beschrnkt ist, heit: es gibt ein $M < \infty$ mit

$$\|D_x f(h)\| \leq M \|h\| \quad \text{fr alle } h.$$

Das M darf von x abhngen, aber nicht von h . In endlich-dimensionalen Rumen ist jede lineare Abbildung automatisch beschrnkt, dort kostet die Forderung also nichts; im Unendlichdimensionalen ist sie eine echte Zusatzbedingung und gleichbedeutend mit Stetigkeit.

Linear in h , beliebig in x . Die Abbildung $h \mapsto D_x f(h)$ ist linear, so wie in Bemerkung 10.1.4. Wie $D_x f$ von x abhngt, schreibt die Definition in keiner Weise vor.

Der eindimensionale Fall. Fr $f: \mathbb{R} \rightarrow \mathbb{R}$ ist $D_x f(h) = f'(x)h$, und (10.1.2) ist wrtlich Aussage (3) aus Satz 10.1.3. Beschrnkt ist diese Abbildung, sobald $|f'(x)| < \infty$ ist, mit $M = |f'(x)|$. Die alte Ableitung ist also ein Spezialfall der neuen.

Prfen wir die Definition an dem einfachsten Beispiel, das nicht trivial ist.

Beispiel 10.1.7 (Frchet-Ableitung von $f(x) = x^2$)

Sei $f: \mathbb{R} \rightarrow \mathbb{R}$ mit $f(x) = x^2$. Behauptung: f ist Frchet-differenzierbar mit $D_x f(h) = 2xh$. Wir gehen die Definition Punkt fr Punkt durch. Zuerst multiplizieren wir aus:

$$\begin{aligned} f(x+h) &= (x+h)^2 = x^2 + 2xh + h^2 \\ &= f(x) + 2xh + h^2 \\ &= f(x) + D_x f(h) + h^2. \end{aligned}$$

Die Zerlegung liegt damit vor: Der Kandidat fr die Ableitung ist $D_x f(h) = 2xh$, und er ist linear in h . brig bleibt der Restterm h^2 .

Ist der Rest klein genug? Wir rechnen den Quotienten aus (10.1.2) nach:

$$\lim_{h \rightarrow 0} \frac{|h^2|}{|h|} = \lim_{h \rightarrow 0} |h| = 0 \quad \implies \quad h^2 = o(|h|). \quad \boxtimes$$

Bemerkenswert daran: Der relative Fehler ist hier nicht bloß klein, er ist exakt $|h|$. Bei $h = 0,1$ beträgt er 0,1, bei $h = 0,01$ schon 0,01. Das Widget oben zeigt diese Zahlen an.

Ist $D_x f$ beschränkt? Bei festem x ist

$$|D_x f(h)| = |2xh| = 2|x| \cdot |h| \leq M|h| \quad \text{für alle } h, \quad M = 2|x| < \infty. \quad \square$$

Beide Bedingungen sind erfüllt, f ist also Fréchet-differenzierbar mit $D_x f(h) = 2xh$. Das passt zur Schableitung: Nach Bemerkung 10.1.6 ist $D_x f(h) = f'(x)h$, und aus $f'(x)h = 2x \cdot h$ lesen wir $f'(x) = 2x$ ab.

Ein Wort zum Quantor in der Beschränktheit. Das $\forall$ in der Schranke gehört zum h , nicht zum x : Die Schranke M hängt ja gerade von x ab, und ein einziges M für alle x kann es nicht geben, weil $2|x|$ über jede Grenze wächst. Beschränktheit ist punktweise in x zu verstehen, und bei festem x gilt sie dann für alle h .

Bemerkung 10.1.8 (Ausblick: andere Ableitungsbegriffe)

Die Fréchet-Ableitung ist nicht die einzige Möglichkeit, den Ableitungsbegriff auf normierte Räume zu heben. Schwächere Varianten verlangen die Approximation nur richtungsweise oder nur gleichmäßig auf kompakten Richtungsmengen; sie heißen *Gateaux-* und *Hadamard-Ableitung* und spielen vor allem in der asymptotischen Statistik eine Rolle. Für dieses Kapitel reicht Fréchet vollständig aus, denn wir arbeiten durchweg in endlich-dimensionalen Räumen.

Selbsttest

In den ersten vier Aussagen sei $f : S \rightarrow \mathbb{R}$ an der Stelle x differenzierbar mit Ableitung $f'(x)$.

1. Ist f in x differenzierbar, so ist f' eine lineare Funktion.

Antwort: Falsch

Die Ableitung ist zunächst irgendeine Funktion von x , und die muss nichts mit Linearität zu tun haben: Für $f(x) = \sin(x)$ ist $f'(x) = \cos(x)$. Linear ist etwas anderes, nämlich die Abbildung $h \mapsto f'(x)h$ bei festem x (Bemerkung 10.1.4).

2. Für festes x ist $h \mapsto f'(x)h$ eine lineare Funktion.

Antwort: Wahr

Hier ist $f'(x)$ eine feste Zahl, und $h \mapsto f'(x)h$ ist die Multiplikation damit. Additivität und Homogenität sind in einer Zeile nachgerechnet. Diese Abbildung heißt in Definition 10.1.5 $D_x f$.

3. Es gilt $\lim_{h \rightarrow 0} \frac{|f(x+h) - f(x) - f'(x)h|}{|h|} = 0$.

Antwort: Wahr

Das ist Aussage (2) aus Satz 10.1.3. Der Zähler ist der Restterm $r(h)$, und Schritt 1 und 2 des Beweises zeigen, dass die Grenzwertdefinition der Ableitung genau dieses Verschwinden des relativen Fehlers bedeutet.

4. Es gilt $f(x+h) = f(x) + f'(x)h + o(|h|)$ für $h \rightarrow 0$.

Antwort: Wahr

Das ist Aussage (3) aus Satz 10.1.3, gleichwertig zur vorigen: Setzen wir $r(h) := f(x+h) - f(x) - f'(x)h$, so heißt $|r(h)|/|h| \rightarrow 0$ gerade $r(h) = o(|h|)$. Diese Zeile ist die Vorlage für Definition

10.1.5.

5. Für $f(x) = x^2$ gilt bei festem x die Schranke $|D_x f(h)| \leq M |h|$ für alle h , mit $M = 2|x|$.

Antwort: Wahr

So herum stimmt es: $|2xh| = 2|x||h|$, und $M = 2|x|$ ist bei festem x endlich. Der Quantor gehört zum h : Ein einziges M für alle x gäbe es nicht, denn $2|x|$ wächst unbeschränkt (Beispiel 10.1.7).

6. Die Fréchet-Ableitung $D_x f$ ist sowohl in h als auch in x linear.

Antwort: Falsch

Linear ist nur die Abhängigkeit von h . Wie $D_x f$ von der Stelle x abhängt, lässt Definition 10.1.5 offen, und schon $f(x) = x^2$ zeigt den Unterschied: $D_x f(h) = 2xh$ ist linear in h , die Zuordnung $x \mapsto D_x f$ ist es zufällig ebenfalls, bei $f(x) = x^3$ mit $D_x f(h) = 3x^2 h$ dagegen nicht mehr.

7. Im Widget: Wir wählen $f(x) = x^2$ und stellen $x = 0,60$ und $h = 0,60$ ein. Welchen Wert zeigt der Readout für den relativen Fehler $|r(h)| / |h|$?

Lösung: 0,6

Der Restterm ist $r(h) = h^2 = 0,36$, also $|r(h)| / |h| = |h| = 0,6$. Dasselbe steht als Abstand zwischen der Sekantensteigung 1,8 und der Tangentensteigung 1,2 in der Readout-Liste.

Vertiefung: MML §5.1 behandelt den eindimensionalen Ableitungsbegriff, §5.1.1 die Taylorreihen, die wir hier nur als Vorkenntnis gestreift haben, und §5.1.2 die Differentiationsregeln.

10.2 Der Gradient: Vektor zu Skalar

In Abschnitt 10.1 haben wir das Ableiten so weit gefasst, dass es in jedem normierten Raum funktioniert. Der Preis dafür ist Abstraktheit: Die Ableitung ist dort eine lineare Abbildung, kein Objekt, mit dem wir rechnen könnten. Für jede Kombination von Ein- und Ausgabeformat brauchen wir deshalb eine konkrete Gestalt, in der diese lineare Abbildung steckt, also eine Zahl, einen Vektor oder eine Matrix.

Fahrplan durch die Spezialfälle

Alle Ableitungen, die wir aus der Analysis kennen, sind Spezialfälle der Fréchet-Ableitung. Neu und für die Statistik wichtig sind die Fälle, in denen Skalare, Vektoren oder Matrizen als Eingabe oder als Ausgabe auftreten. Wir ordnen sie in einer Tabelle an, und diese Tabelle ist zugleich der Fahrplan für die nächsten drei Abschnitte.

Eingabe ↓ / Ausgabe →	Skalar y	Vektor $\mathbf{y}$	Matrix $\mathbf{Y}$
Skalar x	$\frac{dy}{dx}$	$\frac{d\mathbf{y}}{dx}$	$\frac{d\mathbf{Y}}{dx}$
Vektor $\mathbf{x}$	$\frac{\partial x}{\partial \mathbf{y}}$	$\frac{\partial \mathbf{x}}{\partial \mathbf{y}}$	
Matrix $\mathbf{X}$	$\frac{\partial \mathbf{x}}{\partial \mathbf{X}}$		

Die Einträge lesen sich so:

- Skalar zu Skalar ist die gewöhnliche Ableitung aus Abschnitt 10.1.
- Vektor zu Skalar ist der Fall dieses Abschnitts. Das zugehörige Objekt heißt *Gradient*.

- Vektor zu Vektor führt auf die Jacobimatrix in Abschnitt 10.3; der Fall Skalar zu Vektor steckt dort als Sonderfall mit einer einzigen Eingabevariablen drin.
- Skalar zu Matrix und Matrix zu Skalar behandelt Abschnitt 10.4.
- Die drei leeren Zellen lassen wir aus. Ob wir eine Matrix nach einem Vektor ableiten, einen Vektor nach einer Matrix oder eine Matrix nach einer Matrix: In allen drei Fällen bräuchten wir ein Objekt mit mindestens drei Indexpositionen, also einen Tensor (Kapitel 9). Wir kommen in Abschnitt 10.9 kurz darauf zurück.

Der Gradient

Der wichtigste Fall in der Statistik ist der zweite: Eine Verlust- oder Zielfunktion bewertet ein ganzes Bündel von Parametern und gibt dafür eine einzige Zahl zurück. Wie leiten wir nach einem ganzen Bündel ab? Die Analysis geht es Koordinate für Koordinate durch: Sie friert alle bis auf eine ein und leitet die übrig bleibende Funktion einer Variablen ganz gewöhnlich ab. Jede solche partielle Ableitung misst die Steigung von f entlang einer Koordinatenachse, und der Gradient ist die Liste dieser n Steigungen.

Definition 10.2.1 (Gradient)

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ eine in $\mathbf{x} \in \mathbb{R}^n$ differenzierbare Funktion. Der *Gradient* (gradient) von f in $\mathbf{x}$ ist der Zeilenvektor der partiellen Ableitungen,

$$\frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \nabla f(\mathbf{x}) = \left(\frac{\partial f(\mathbf{x})}{\partial x_1}, \dots, \frac{\partial f(\mathbf{x})}{\partial x_n} \right) \in \mathbb{R}^{1 \times n}.$$

Dass dieser Zeilenvektor wirklich die Ableitung im Sinne von Abschnitt 10.1 ist, kostet ein kurzes Argument. Die Formate passen jedenfalls: Das Produkt einer $1 \times n$ -Zeile mit einer $n \times 1$ -Spalte ist eine Zahl, und $\mathbf{h} \mapsto \nabla f(\mathbf{x})\mathbf{h}$ ist in $\mathbf{h}$ linear. Dass es genau diese lineare Abbildung sein muss, sehen wir, indem wir $\mathbf{h} = t\mathbf{e}_j$ mit dem j -ten Einheitsvektor $\mathbf{e}_j$ einsetzen und durch t teilen: Links steht dann der Differenzenquotient von f in Richtung der j -ten Koordinate, rechts $D_{\mathbf{x}}f(\mathbf{e}_j)$ plus ein Term, der mit $t \rightarrow 0$ verschwindet. Der Grenzwert links ist $\partial f(\mathbf{x})/\partial x_j$, also ist $D_{\mathbf{x}}f(\mathbf{e}_j)$ genau die j -te partielle Ableitung. Eine lineare Abbildung ist durch ihre Werte auf den n Koordinatenrichtungen schon festgelegt, es bleibt also keine andere Wahl. (Lemma 10.3.3 in Abschnitt 10.3 führt dasselbe Argument für beliebige Ausgabeformate aus.) Die lineare Approximation lautet damit

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|) \quad \text{mit} \quad D_{\mathbf{x}}f(\mathbf{h}) = \nabla f(\mathbf{x})\mathbf{h}. \quad (10.2.1)$$

Der Restterm $o(\|\mathbf{h}\|)$ ist dabei wie immer in der Landau-Notation zu lesen: Er schrumpft schneller als $\|\mathbf{h}\|$ selbst, wenn $\mathbf{h}$ gegen null geht. Anschaulich ist $\mathbf{h} \mapsto f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h}$ für $n = 2$ eine Ebene, die den Graphen von f im Punkt $(\mathbf{x}, f(\mathbf{x}))$ berührt, also die Tangentialebene. Sie übernimmt die Rolle, die für $n = 1$ die Tangente an den Funktionsgraphen spielt.

Bemerkung 10.2.2 (Zeilenvektor, nicht Spaltenvektor)

Vielerorts wird ∇f als *Spaltenvektor* definiert, während für uns die Zeilenform die praktischere ist. Wofür genau, zeigt sich erst in den nächsten Abschnitten, deshalb hier vorab die zwei Stellen.

Die Jacobimatrix aus Abschnitt 10.3 hat für $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ das Format $m \times n$, im Fall $m = 1$ also $\mathbb{R}^{1 \times n}$. In der Zeilenkonvention sind Gradient und Jacobimatrix damit dasselbe Objekt, und wir müssen die beiden Fälle weder unterscheiden noch ineinander umrechnen. In der

Kettenregel dürfen wir die Faktoren außerdem in der Reihenfolge hinschreiben, in der die Funktionen hintereinander ausgeführt werden; in der Spaltenkonvention kehrt sich diese Reihenfolge um und die Faktoren tragen Transponierte.

Bezahlen müssen wir dafür an einer anderen Stelle. Wo der Gradient als Punkt oder als Schritt im $\mathbb{R}^n$ auftritt, etwa in einem Optimierungsverfahren, steht der transponierte Gradient. Genau das passiert weiter unten im Gradientenabstieg.

Was der Gradient über f verrät

Die Definition zählt nur die Steigungen entlang der n Koordinatenachsen auf. Sie sagt scheinbar nichts über schräge Richtungen. Tatsächlich steckt darin schon die vollständige Information, und (10.2.1) liefert sie uns.

Definition 10.2.3 (Richtungsableitung)

Sei $\mathbf{d} \in \mathbb{R}^n$ mit $\|\mathbf{d}\|_2 = 1$. Die *Richtungsableitung* von f in $\mathbf{x}$ in Richtung $\mathbf{d}$ ist die Ableitung der eindimensionalen Funktion $t \mapsto f(\mathbf{x} + t\mathbf{d})$ an der Stelle $t = 0$. Setzen wir $\mathbf{h} = t\mathbf{d}$ in (10.2.1) ein, so ist sie gerade $\nabla f(\mathbf{x})\mathbf{d}$, denn

$$f(\mathbf{x} + t\mathbf{d}) = f(\mathbf{x}) + t \nabla f(\mathbf{x})\mathbf{d} + o(|t|). \quad (10.2.2)$$

Aus $\|\mathbf{d}\|_2 = 1$ folgt $\|t\mathbf{d}\|_2 = |t|$, deshalb steht rechts $o(|t|)$. Wir kennen die Steigung von f also in *jeder* Richtung, sobald wir den Gradienten kennen. Die Norm legt dabei fest, was ein gleich großer Schritt ist; im Folgenden verwenden wir die euklidische Norm. Unter allen so normierten Richtungen gibt es eine beste.

Satz 10.2.4 (Richtung des stärksten Anstiegs)

Sei f in $\mathbf{x}$ differenzierbar mit $\nabla f(\mathbf{x}) \neq \mathbf{0}$. Dann gilt für alle $\mathbf{d} \in \mathbb{R}^n$ mit $\|\mathbf{d}\|_2 = 1$

$$-\|\nabla f(\mathbf{x})\|_2 \leq \nabla f(\mathbf{x})\mathbf{d} \leq \|\nabla f(\mathbf{x})\|_2,$$

und die obere Schranke wird genau für

$$\mathbf{d}^* = \frac{\nabla f(\mathbf{x})^\top}{\|\nabla f(\mathbf{x})\|_2}$$

angenommen, die untere für $-\mathbf{d}^*$. Der transponierte Gradient zeigt also in die Richtung des stärksten Anstiegs, der negative in die des stärksten Abstiegs, und $\|\nabla f(\mathbf{x})\|_2$ ist die Steigung in dieser Richtung.

Vertiefung: Warum der Gradient die Richtung des stärksten Anstiegs angibt

Beweis.

- (1) Das Produkt aus Zeile und Spalte ist ein Skalarprodukt: Mit $\mathbf{g} = \nabla f(\mathbf{x})^\top \in \mathbb{R}^n$ ist $\nabla f(\mathbf{x})\mathbf{d} = \mathbf{g}^\top \mathbf{d}$. Die Cauchy-Schwarz-Ungleichung liefert

$$|\mathbf{g}^\top \mathbf{d}| \leq \|\mathbf{g}\|_2 \|\mathbf{d}\|_2 = \|\mathbf{g}\|_2,$$

also beide Schranken auf einmal.

$\|\mathbf{d}\|_2 = 1$; die euklidische Norm von $\mathbf{g}$ ist dieselbe wie die des Zeilenvektors $\nabla f(\mathbf{x})$

- (2) Gleichheit gilt in Cauchy-Schwarz genau dann, wenn $\mathbf{d}$ ein Vielfaches von $\mathbf{g}$ ist, also $\mathbf{d} = c \mathbf{g}$ mit $c \in \mathbb{R}$. Aus $\|\mathbf{d}\|_2 = 1$ folgt $|c| = 1/\|\mathbf{g}\|_2$. Das positive Vorzeichen gibt

$$\mathbf{g}^\top \mathbf{d}^* = \frac{\mathbf{g}^\top \mathbf{g}}{\|\mathbf{g}\|_2} = \frac{\|\mathbf{g}\|_2^2}{\|\mathbf{g}\|_2} = \|\mathbf{g}\|_2,$$

das negative den Wert $-\|\mathbf{g}\|_2$.

hier wird $\nabla f(\mathbf{x}) \neq \mathbf{0}$ gebraucht: sonst dürften wir nicht durch $\|\mathbf{g}\|_2$ teilen, und alle Richtungsableitungen wären null

□

Bemerkung 10.2.5 (Der Gradient steht senkrecht auf der Höhenlinie)

Interessant ist auch das andere Extrem. Für Richtungen $\mathbf{d}$ mit $\nabla f(\mathbf{x})\mathbf{d} = 0$ verschwindet in (10.2.2) der lineare Term, und f ändert sich in erster Ordnung überhaupt nicht. Solche Richtungen stehen senkrecht auf $\nabla f(\mathbf{x})^\top$. Ist f in einer Umgebung von $\mathbf{x}$ stetig differenzierbar und $\nabla f(\mathbf{x}) \neq \mathbf{0}$, so sind sie genau die Tangentialrichtungen der Niveaumenge $\{\mathbf{y} \in \mathbb{R}^n : f(\mathbf{y}) = f(\mathbf{x})\}$ durch $\mathbf{x}$; diese ist dort nach dem Satz über implizite Funktionen eine glatte Kurve oder Fläche, für $n = 2$ also die Höhenlinie durch $\mathbf{x}$. Verschwindet der Gradient dagegen, kann die Niveaumenge eine Ecke haben, und ein Beispiel dafür folgt gleich.

Beide Beobachtungen zusammen ergeben das vertraute Bild einer Landkarte: Der Gradient steht senkrecht auf der Höhenlinie und zeigt bergauf, und je enger die Höhenlinien beieinander liegen, desto größer ist $\|\nabla f(\mathbf{x})\|_2$.

Eine quadratische Funktion als Beispiel

Beispiel 10.2.6 (Gradient einer quadratischen Funktion)

Wir betrachten $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ mit

$$f(\mathbf{x}) = x_1^2 + 3x_1x_2 + 2x_2^2.$$

Schritt 1: partielle Ableitungen. Beim Ableiten nach x_1 ist x_2 eine Konstante und umgekehrt:

$$\frac{\partial f(\mathbf{x})}{\partial x_1} = 2x_1 + 3x_2, \quad \frac{\partial f(\mathbf{x})}{\partial x_2} = 3x_1 + 4x_2.$$

Schritt 2: als Zeile zusammensetzen.

$$\nabla f(\mathbf{x}) = (2x_1 + 3x_2 \quad 3x_1 + 4x_2) \in \mathbb{R}^{1 \times 2}.$$

Beachten wir das Format: ∇f bildet $\mathbb{R}^2$ nicht nach $\mathbb{R}^2$ ab, sondern nach $\mathbb{R}^{1 \times 2}$.

Probe an einer Stelle. In $\mathbf{x} = (1, 1)^\top$ ist $f(\mathbf{x}) = 6$ und $\nabla f(\mathbf{x}) = (5, 7)$. Für $\mathbf{h} = (0, 1; 0, 1)^\top$ sagt (10.2.1)

$$f(\mathbf{x} + \mathbf{h}) \approx 6 + 5 \cdot 0,1 + 7 \cdot 0,1 = 7,2, \quad \text{tatsächlich} \quad f(1,1; 1,1) = 7,26.$$

Der Restterm ist **0,06**. Weil f quadratisch ist, können wir ihn sogar exakt angeben: Ausmultiplizieren von $f(\mathbf{x} + \mathbf{h})$ liefert $h_1^2 + 3h_1h_2 + 2h_2^2$, also einen Ausdruck, der quadratisch in $\mathbf{h}$ ist. Halbieren wir $\mathbf{h}$, so viertelt er sich, während $\|\mathbf{h}\|$ nur halb so groß wird:

$\mathbf{h}$	Länge von $\mathbf{h}$	Restterm	Restterm geteilt durch die Länge
(0,1; 0,1)	0,1414	0,06	0,424
(0,05; 0,05)	0,0707	0,015	0,212
(0,01; 0,01)	0,0141	0,0006	0,042

Die letzte Spalte geht gegen null, und das ist genau die Aussage $o(\|\mathbf{h}\|)$.

Interaktiv: Gradient und Höhenlinie zum Verschieben

Bemerkung 10.2.5 behauptet etwas Geometrisches über einen Vektor aus lauter partiellen Ableitungen: Er steht senkrecht auf der Höhenlinie. Das lässt sich nachsehen, statt es zu glauben.

Interaktives Element — nur in der Web-Fassung

Zwei Tafeln zeigen denselben Zustand: links die Höhenlinien mit dem orangen Gradientenpfeil am gewählten Punkt, rechts dieselbe Funktion als Fläche über der Ebene, mit der grünen Tangentialebene aus (10.2.2) und demselben Gradientenpfeil auf dem Boden.

Im Widget steht der orange Pfeil an jeder Stelle senkrecht auf der blau gestrichelten Tangente an die Höhenlinie, und er wird lang, wo die Höhenlinien dicht liegen. Der zweite Knopf schaltet auf eine nicht quadratische Funktion um, deren Höhenlinien sich krümmen; die Aussage gilt dort unverändert, weil sie punktweise formuliert ist. Im Nullpunkt der quadratischen Funktion verschwindet der Gradient ganz, und dann zeichnet sich gar keine Richtung mehr aus: Das ist die Stelle, an der die Höhenlinie zum Niveau 0 aus zwei sich kreuzenden Geraden besteht.

Damit bleibt die zweite Behauptung, die von Satz 10.2.4: keine Richtung steigt steiler an als der Gradient. Und gleich die Anschlussfrage: Wie viel vom maximalen Anstieg bleibt übrig, wenn wir schräg dazu laufen?

Interaktiv: Der Kompass der Richtungen

Der Kompass zeigt für jede Richtung $\mathbf{d}$, wie stark f in diese Richtung ansteigt: Der Abstand der blauen Kurve vom Mittelpunkt ist $\nabla f(\mathbf{x})\mathbf{d}$, gemessen am Maximum. Rechts steht der Schnitt von f entlang des Strahls $\mathbf{x} + t\mathbf{d}$ samt der linearen Näherung aus (10.2.2).

Interaktives Element — nur in der Web-Fassung

Die blaue Kurve ist ein Kreis, der den Mittelpunkt berührt, also genau der Graph von $\theta \mapsto \|\nabla f(\mathbf{x})\|_2 \cos \theta$ in Polarkoordinaten. Ihr weitester Punkt liegt in Gradientenrichtung, dort erreicht die Richtungsableitung ihr Maximum $\|\nabla f(\mathbf{x})\|_2$ (Satz 10.2.4); senkrecht dazu, also längs der Höhenlinie, ist sie null. Dazwischen zählt allein der Kosinus des eingeschlossenen Winkels: 60° neben dem Gradienten bleiben noch 50 % übrig.

Zwei Gradienten zum Auswendiglernen

Zwei Funktionen tauchen in der Statistik so oft auf, dass sich ihre Gradienten einzuprägen lohnt: die lineare Funktion und die quadratische Form.

Beispiel 10.2.7 (Gradient einer linearen Funktion)

Seien $\mathbf{a}, \mathbf{x} \in \mathbb{R}^n$ und $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$. Was ist $\nabla f(\mathbf{x})$? Bevor wir weiterlesen, lohnt sich der eigene Versuch.

Lösung

Ausgeschrieben ist $f(\mathbf{x}) = \sum_{i=1}^n a_i x_i$. In dieser Summe kommt x_k in genau einem Summanden vor, nämlich in $a_k x_k$, und dort linear. Also

$$\frac{\partial f(\mathbf{x})}{\partial x_k} = a_k \quad \text{und damit} \quad \nabla f(\mathbf{x}) = (a_1, \dots, a_n) = \mathbf{a}^\top.$$

Der Gradient hängt hier nicht von $\mathbf{x}$ ab. Das muss so sein: f ist selbst schon linear, die lineare Approximation ist also f selbst, und der Restterm in (10.2.1) ist exakt null.

Satz 10.2.8 (Gradient der quadratischen Form)

Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetrisch und $f: \mathbb{R}^n \rightarrow \mathbb{R}$ mit $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$. Dann gilt

$$\nabla f(\mathbf{x}) = 2\mathbf{x}^\top \mathbf{A}. \quad (10.2.3)$$

Beweis.

- (1) Wir schreiben die quadratische Form als Doppelsumme aus:

$$f(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n x_i a_{ij} x_j.$$

der i -te Eintrag von $\mathbf{A} \mathbf{x}$ ist $\sum_j a_{ij} x_j$, und $\mathbf{x}^\top (\mathbf{A} \mathbf{x})$ summiert diese Einträge mit den Gewichten x_i

- (2) Jetzt leiten wir nach x_k ab. Die Variable x_k kommt in zwei Sorten von Summanden vor: einmal als linker Faktor ($i = k$), einmal als rechter ($j = k$). Die Produktregel liefert

$$\frac{\partial f(\mathbf{x})}{\partial x_k} = \sum_{i=1}^n x_i a_{ik} + \sum_{j=1}^n a_{kj} x_j.$$

der Summand mit $i = j = k$ enthält x_k zweimal; die Produktregel gibt dort $2a_{kk}x_k$, also je einen Beitrag $a_{kk}x_k$ an beide Summen

- (3) Erst hier geht die Symmetrie ein. Aus $a_{ik} = a_{ki}$ folgt, dass beide Summen dasselbe sind:

$$\frac{\partial f(\mathbf{x})}{\partial x_k} = \sum_{i=1}^n a_{ki} x_i + \sum_{j=1}^n a_{kj} x_j = 2 \sum_{i=1}^n a_{ki} x_i = 2 (\mathbf{A} \mathbf{x})_k.$$

die zweite Summe ist bis auf den Namen des Laufindex die erste

- (4) Der Gradient sammelt diese n Zahlen als Zeile ein, also

$$\nabla f(\mathbf{x}) = 2 (\mathbf{A} \mathbf{x})^\top = 2 \mathbf{x}^\top \mathbf{A}^\top = 2 \mathbf{x}^\top \mathbf{A}.$$

$(\mathbf{A} \mathbf{x})^\top = \mathbf{x}^\top \mathbf{A}^\top$, und noch einmal $\mathbf{A}^\top = \mathbf{A}$

□

Bemerkung 10.2.9 (Drei Nachträge zu Satz 10.2.8)

Der Spezialfall $A = I$. Dann ist $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x} = \|\mathbf{x}\|_2^2$ und (10.2.3) wird zu $\nabla f(\mathbf{x}) = 2\mathbf{x}^\top$. Das ist die mehrdimensionale Fassung von $(x^2)' = 2x$.

Ohne Symmetrie stimmt die Formel nicht. Schritt 3 des Beweises braucht $a_{ik} = a_{ki}$. Allgemein liefern die ersten beiden Schritte

$$\nabla(\mathbf{x}^\top A \mathbf{x}) = \mathbf{x}^\top (A + A^\top),$$

was für symmetrisches A wieder (10.2.3) ergibt. Ein Gegenbeispiel ist $A = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ mit $f(\mathbf{x}) = x_1 x_2$: Hier ist $\nabla f(\mathbf{x}) = (x_2, x_1)$, während $2\mathbf{x}^\top A = (0, 2x_1)$ etwas ganz anderes ist. In $\mathbf{x} = (1, 1)^\top$ steht $(1, 1)$ gegen $(0, 2)$.

Beispiel 10.2.6 ist ein Sonderfall. Mit $A = \begin{pmatrix} 1 & 3/2 \\ 3/2 & 2 \end{pmatrix}$ ist $\mathbf{x}^\top A \mathbf{x} = x_1^2 + 3x_1 x_2 + 2x_2^2$, und (10.2.3) gibt

$$2\mathbf{x}^\top A = \begin{pmatrix} 2x_1 + 3x_2 & 3x_1 + 4x_2 \end{pmatrix},$$

also genau den Gradienten, den wir dort partiell ausgerechnet haben. Diese Matrix ist allerdings *nicht* positiv definit, denn ihre Determinante ist $1 \cdot 2 - (3/2)^2 = -1/4 < 0$. Die Matrix der zweiten partiellen Ableitungen, die Hesse-Matrix $2A = \begin{pmatrix} 2 & 3 \\ 3 & 4 \end{pmatrix}$, hat entsprechend die Determinante -1 und die Eigenwerte $3 \pm \sqrt{10}$, also 6,162 und $-0,162$. Im Nullpunkt hat f deshalb kein Minimum, sondern einen Sattel: In der einen Eigenrichtung steigt f , in der anderen fällt es. Im Widget oben kreuzen sich dort die beiden Geraden zum Niveau 0.

Anwendung: Gradientenabstieg

Warum interessieren wir uns überhaupt für die Richtung des stärksten Anstiegs? Weil das Standardwerkzeug der Optimierung in hohen Dimensionen genau darauf beruht. Gesucht sind Parameter $\theta \in \mathbb{R}^p$, die eine Verlustfunktion $L(\theta)$ klein machen, etwa den mittleren quadratischen Fehler eines Modells. Für p in der Größenordnung von Millionen ist an eine geschlossene Lösung nicht zu denken, wohl aber an kleine Schritte bergab.

Algorithmus 10.2.10 (Gradientenabstieg)

Gegeben eine differenzierbare Verlustfunktion $L: \mathbb{R}^p \rightarrow \mathbb{R}$, ein Startwert $\theta^{(0)}$ und eine Lernrate $\alpha > 0$. Für $t = 0, 1, 2, \dots$ iteriere

$$\theta^{(t+1)} = \theta^{(t)} - \alpha \nabla L(\theta^{(t)})^\top. \quad (10.2.4)$$

Abgebrochen wird, wenn sich θ kaum noch ändert oder der Gradient klein genug ist.

Nach Satz 10.2.4 zeigt $-\nabla L(\theta)^\top$ in die Richtung des stärksten Abstiegs. Der Algorithmus läuft also in jedem Schritt in die aktuell steilste Richtung bergab, und zwar um die Länge $\alpha \|\nabla L(\theta^{(t)})\|$. Dass er damit tatsächlich am Ziel ankommt, hängt an α und an der Gestalt von L ; das Gradientenverfahren und seine Varianten sind die Grundlage für das Training neuronaler Netze und für komplexe Regressionsmodelle.

Bemerkung 10.2.11 (Warum in Gleichung 10.2.4 ein Transponiertes steht)

Das $^\top$ in (10.2.4) ist keine Kosmetik, sondern eine Formatfrage. Die Parameter $\theta^{(t)}$ bilden einen Spaltenvektor in $\mathbb{R}^p$, der Gradient $\nabla L(\theta^{(t)})$ nach Definition 10.2.1 dagegen eine Zeile

in $\mathbb{R}^{1 \times p}$. Eine Zeile von einer Spalte abzuziehen ist nicht definiert. Erst $\nabla L(\boldsymbol{\theta}^{(t)})^\top \in \mathbb{R}^p$ passt zum Rest der Gleichung. Wer den Gradienten von vornherein als Spalte definiert, spart hier das $^\top$ und handelt es sich dafür in der Kettenregel wieder ein (Bemerkung 10.2.2).

Die Lernrate α ist der einzige freie Parameter des Verfahrens. Gibt es eine beste Wahl, und woran erkennt man sie?

Interaktiv: Gradientenabstieg auf einer konvexen quadratischen Funktion

Zum Ausprobieren nehmen wir nicht die Funktion aus Beispiel 10.2.6, denn deren Sattel wäre für ein Abstiegsverfahren eine Falle. Stattdessen minimieren wir $L(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^\top \mathbf{A} \boldsymbol{\theta}$ mit der symmetrischen und positiv definiten Matrix $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, deren Eigenwerte $(5 \pm \sqrt{5})/2$ beide positiv sind. Nach Satz 10.2.8 und der Linearität des Ableitens ist $\nabla L(\boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{A}$, der Schritt also $-\alpha \mathbf{A} \boldsymbol{\theta}$.

Interaktives Element — nur in der Web-Fassung

Über den Erfolg entscheidet allein die Rate $\rho(\alpha) = \max_i |1 - \alpha \lambda_i|$: Die Lernrate ist durch das Verhältnis der Eigenwerte begrenzt, und die beste Wahl ist die, bei der beide Eigenrichtungen gleich schnell schrumpfen, hier $\alpha = 0,4$. Zu kleine Lernraten kriechen, zu große lassen die Iterierten davonlaufen. Wie eng dieser Spielraum ist, hängt an der Hesse-Matrix; Abschnitt 10.7 führt das aus, und Kapitel 12 nimmt die Konvergenzanalyse als eigenes Thema auf. Für die Funktion aus Beispiel 10.2.6 gäbe es dagegen gar kein brauchbares α : Ihre Hesse-Matrix hat den negativen Eigenwert $-0,162$, und entlang dieser Eigenrichtung wächst der Abstand zum Sattel bei jeder Lernrate.

Selbsttest

1. Ist $f: \mathbb{R}^n \rightarrow \mathbb{R}$ in $\mathbf{x}$ differenzierbar, so ist $\nabla f(\mathbf{x})$ ein Zeilenvektor in $\mathbb{R}^{1 \times n}$, und $\mathbf{h} \mapsto \nabla f(\mathbf{x}) \mathbf{h}$ ist die Fréchet-Ableitung von f in $\mathbf{x}$.

Antwort: Wahr

Das ist Definition 10.2.1 zusammen mit (10.2.1). Das Produkt aus einer $1 \times n$ -Zeile und einer $n \times 1$ -Spalte ist eine Zahl, und die Abbildung ist linear in $\mathbf{h}$.

2. Für $f(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$ hängt der Gradient von der Stelle $\mathbf{x}$ ab.

Antwort: Falsch

Er ist konstant gleich $\mathbf{a}^\top$ (Beispiel 10.2.7). Das ist kein Zufall: f ist selbst linear, also ist die beste lineare Näherung f selbst, und der Restterm in (10.2.1) verschwindet exakt.

3. Für jede Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ gilt $\nabla(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{x}^\top \mathbf{A}$.

Antwort: Falsch

Satz 10.2.8 verlangt Symmetrie, und die wird im dritten Beweisschritt gebraucht. Allgemein gilt $\nabla(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = \mathbf{x}^\top (\mathbf{A} + \mathbf{A}^\top)$. Für $\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ ist $f(\mathbf{x}) = x_1 x_2$ mit $\nabla f(\mathbf{x}) = (x_2, x_1)$, während $2\mathbf{x}^\top \mathbf{A} = (0, 2x_1)$ ist. In $\mathbf{x} = (1, 1)^\top$ steht $(1, 1)$ gegen $(0, 2)$.

4. Sei $\nabla f(\mathbf{x}) \neq \mathbf{0}$. Unter allen Richtungen $\mathbf{d}$ mit $\|\mathbf{d}\|_2 = 1$ ist die Richtungsableitung $\nabla f(\mathbf{x}) \mathbf{d}$ genau dann maximal, wenn $\mathbf{d}$ in Richtung $\nabla f(\mathbf{x})^\top$ zeigt, und ihr Maximalwert ist $\|\nabla f(\mathbf{x})\|_2$.

Antwort: Wahr

Das ist Satz 10.2.4, bewiesen mit dem Gleichheitsfall der Cauchy-Schwarz-Ungleichung. Ohne die Voraussetzung $\nabla f(\mathbf{x}) \neq \mathbf{0}$ wäre nichts zu holen: Dann sind alle Richtungsableitungen null, und keine Richtung zeichnet sich aus.

5. Das $^\top$ im Update $\boldsymbol{\theta} - \alpha \nabla L(\boldsymbol{\theta})^\top$ ist reine Schreibweise und darf weggelassen werden.

Antwort: Falsch

Ohne das $^\top$ passen die Formate nicht: $\boldsymbol{\theta}$ ist eine Spalte in $\mathbb{R}^p$, der Gradient eine Zeile in $\mathbb{R}^{1 \times p}$, und die Differenz wäre nicht definiert (Bemerkung 10.2.11).

6. Verschwindet der Gradient in einem Punkt, so liegt dort ein Minimum von f .

Antwort: Falsch

Ein verschwindender Gradient sagt nur, dass die lineare Näherung flach ist. Für $f(\mathbf{x}) = x_1^2 + 3x_1x_2 + 2x_2^2$ aus Beispiel 10.2.6 ist $\nabla f(\mathbf{0}) = (0, 0)$, und der Nullpunkt ist ein Sattel: Die Hesse-Matrix hat die Eigenwerte 6,162 und $-0,162$, also einen positiven und einen negativen (Bemerkung 10.2.9). Entlang der einen Eigenrichtung steigt f , entlang der anderen fällt es.

7. Im Kompass-Widget: Wir setzen $\mathbf{d}$ mit dem Knopf „d 60° daneben“ und lesen im Verdikt den Anteil am maximalen Anstieg ab. Wie viel Prozent sind es?

Lösung: 50 Prozent

Nach Satz 10.2.4 ist $\nabla f(\mathbf{x})\mathbf{d} = \|\nabla f(\mathbf{x})\|_2 \cos \theta$, hier also $\cos 60^\circ = 0,5$, und zwar an jeder Stelle, an der der Gradient nicht verschwindet.

8. Bei welcher Lernrate α ist die Rate $\rho(\alpha) = \max_i |1 - \alpha\lambda_i|$ des Abstiegs-Widgets am kleinsten?

Lösung: 0,4

Bei $\alpha = 2/(\lambda_{\min} + \lambda_{\max}) = 2/5 = 0,4$. Dort schneiden sich die beiden Kurven $|1 - \alpha\lambda_{\min}|$ und $|1 - \alpha\lambda_{\max}|$, beide Eigenrichtungen schrumpfen mit demselben Faktor $\sqrt{5}/5 \approx 0,447$.

Vertiefung: MML §5.2 (partielle Ableitungen und Gradient); für den Gradientenabstieg MML §7.1.

10.3 Die Jacobimatrix: Vektor zu Vektor

Der Gradient aus Abschnitt 10.2 setzt voraus, dass am Ende genau eine Zahl herauskommt. Viele Abbildungen, mit denen wir rechnen, liefern aber wieder ein ganzes Bündel. Das Residuum $\mathbf{x} \mapsto \mathbf{Ax} - \mathbf{b}$ der kleinsten Quadrate (Abschnitt 7.1) hat so viele Komponenten wie es Beobachtungen gibt. Eine Schicht eines neuronalen Netzes macht aus einem Vektor von Eingaben einen Vektor von Aktivierungen. Und eine Koordinatentransformation der Ebene ordnet jedem Punkt wieder einen Punkt zu.

Solche Funktionen $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ sind nichts weiter als m gestapelte skalarwertige Funktionen,

$$\mathbf{f}(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top, \quad f_i: \mathbb{R}^n \rightarrow \mathbb{R}.$$

Jede einzelne Komponente f_i hat nach Abschnitt 10.2 einen Gradienten, also eine Zeile mit n Einträgen. Stapeln wir diese m Zeilen übereinander, steht eine Matrix da. Sie trägt einen eigenen Namen.

Die Jacobimatrix

Definition 10.3.1 (Jacobimatrix)

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ in $\mathbf{x} \in \mathbb{R}^n$ differenzierbar mit Komponenten $f(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))^\top$. Die *Jacobimatrix* (Jacobian) von f in $\mathbf{x}$ ist die Matrix der partiellen Ableitungen

$$\mathbf{J}_f(\mathbf{x}) := \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} = \begin{pmatrix} \partial f_1(\mathbf{x})/\partial x_1 & \cdots & \partial f_1(\mathbf{x})/\partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_m(\mathbf{x})/\partial x_1 & \cdots & \partial f_m(\mathbf{x})/\partial x_n \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

Der Eintrag an der Stelle (i, j) ist $\partial f_i(\mathbf{x})/\partial x_j$.

Damit hat die Fréchet-Ableitung aus Abschnitt 10.1 auch in diesem Fall eine Gestalt, mit der sich rechnen lässt: Sie ist die Multiplikation mit der Jacobimatrix,

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + o(\|\mathbf{h}\|) \quad \text{mit} \quad D_{\mathbf{x}}f(\mathbf{h}) = \mathbf{J}_f(\mathbf{x})\mathbf{h}. \quad (10.3.1)$$

Die Formate passen: Eine $m \times n$ -Matrix mal eine $n \times 1$ -Spalte ergibt eine $m \times 1$ -Spalte, und dort lebt der Zuwachs von f . Der Restterm wird in der euklidischen Norm des $\mathbb{R}^m$ gemessen, das $\mathbf{h}$ im Nenner der Landau-Bedingung dagegen in der des $\mathbb{R}^n$.

Bemerkung 10.3.2 (Die Zeilen sind Gradienten)

Lesen wir Definition 10.3.1 zeilenweise, so steht in Zeile i genau der Gradient der i -ten Komponentenfunktion:

$$\mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} \nabla f_1(\mathbf{x}) \\ \vdots \\ \nabla f_m(\mathbf{x}) \end{pmatrix}.$$

Die Jacobimatrix bringt also nichts grundsätzlich Neues, sie ordnet nur an, was wir schon haben. Zwei Sonderfälle stecken bereits darin.

- Für $m = 1$ bleibt eine einzige Zeile übrig, und das ist der Gradient aus Definition 10.2.1. Dass wir ihn dort als Zeilenvektor definiert haben, zahlt sich hier aus: Gradient und Jacobimatrix sind dasselbe Objekt, nur mit unterschiedlich vielen Zeilen.
- Für $n = 1$ bleibt eine einzige Spalte übrig. Das ist der Fall Skalar zu Vektor aus der Übersicht in Abschnitt 10.2, etwa die Ableitung einer Kurve $t \mapsto \mathbf{z}(t)$ nach ihrem Parameter.

Gleichung (10.3.1) lässt sich auch rückwärts lesen. Finden wir auf irgendeinem Weg eine Matrix, die f bis auf einen $o(\|\mathbf{h}\|)$ -Rest beschreibt, so ist das bereits die Jacobimatrix.

Lemma 10.3.3 (Die Matrix der linearen Näherung)

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, sei $\mathbf{x} \in \mathbb{R}^n$, und sei $\mathbf{M} \in \mathbb{R}^{m \times n}$ eine Matrix mit

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \mathbf{M}\mathbf{h} + o(\|\mathbf{h}\|) \quad \text{für } \mathbf{h} \rightarrow \mathbf{0}.$$

Dann ist f in $\mathbf{x}$ differenzierbar und $\mathbf{M} = \mathbf{J}_f(\mathbf{x})$.

Beweis.

- (1) Differenzierbar ist f schon nach Definition 10.1.5, denn $\mathbf{h} \mapsto \mathbf{M}\mathbf{h}$ ist linear und beschränkt. Zu zeigen bleibt, dass $\mathbf{M}$ die Matrix der partiellen Ableitungen ist. Dazu setzen wir $\mathbf{h} = t\mathbf{e}_j$ mit $t \in \mathbb{R}$ und dem j -ten Einheitsvektor $\mathbf{e}_j$. Dann ist $\|\mathbf{h}\| = |t|$ und $\mathbf{M}\mathbf{h} = t\mathbf{M}\mathbf{e}_j$, also

$$\frac{f(\mathbf{x} + t\mathbf{e}_j) - f(\mathbf{x})}{t} = \mathbf{M}\mathbf{e}_j + \frac{o(|t|)}{t}.$$

$\mathbf{M}\mathbf{e}_j$ ist die j -te Spalte von $\mathbf{M}$; der Restterm hängt von t ab, nicht aber der erste Summand

- (2) Für $t \rightarrow 0$ verschwindet der zweite Summand rechts, denn genau das besagt $o(|t|)$. Links steht der Differenzenquotient in Richtung der j -ten Koordinate, sein Grenzwert ist die Spalte der partiellen Ableitungen $(\partial f_1/\partial x_j, \dots, \partial f_m/\partial x_j)^\top$. Beide Seiten stimmen also spaltenweise überein, und das für jedes $j = 1, \dots, n$.

der Grenzwert existiert, weil die rechte Seite konvergiert; damit existieren nebenbei alle partiellen Ableitungen

□

Welche Gestalt hat die Ableitung eigentlich, wenn wir Ein- und Ausgabedimension frei wählen? Die Übersichtstabelle aus Abschnitt 10.2 listet vier Fälle auf; Definition 10.3.1 sagt, dass es immer derselbe Bauplan ist.

Interaktiv: Welche Gestalt hat die Ableitung?

Für jede Ausgabekomponente entsteht eine Zeile, für jede Eingabevariable eine Spalte.

Interaktives Element — nur in der Web-Fassung

Damit ist die Tabelle der Formate vollständig: Die einzelne Zahl, der Gradient als Zeile, die Spalte der Ableitungen einer Kurve und die volle Matrix sind vier Ansichten desselben Objekts, und welche wir sehen, entscheiden nur n und m .

Die Bausteine und ihre Jacobimatrizen

Vier Funktionen tauchen so oft als Bausteine auf, dass sich das Auswendiglernen lohnt. Alle vier sind konstant oder linear, und entsprechend einfach fallen die Jacobimatrizen aus.

Satz 10.3.4 (Jacobimatrizen der Grundbausteine)

Für $\mathbf{x} \in \mathbb{R}^n$ gilt:

1. $f(\mathbf{x}) = \mathbf{c} \in \mathbb{R}^m$ konstant $\implies \mathbf{J}_f(\mathbf{x}) = \mathbf{0}_{m \times n}$.
2. $f(\mathbf{x}) = \mathbf{x} \implies \mathbf{J}_f(\mathbf{x}) = \mathbf{I}_n$.
3. $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n} \implies \mathbf{J}_f(\mathbf{x}) = \mathbf{A}$.
4. $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}$ mit $\mathbf{A} \in \mathbb{R}^{n \times m} \implies \mathbf{J}_f(\mathbf{x}) = \mathbf{A}^\top$.

Keine dieser vier Jacobimatrizen hängt von $\mathbf{x}$ ab.

Beweis.

- (1) Zu (1): Jede Komponente $f_i(\mathbf{x}) = c_i$ ist konstant, also verschwinden alle partiellen Ableitungen $\partial f_i / \partial x_j = 0$. Die Jacobimatrix ist die Nullmatrix im Format $m \times n$.

eine konstante Funktion ändert sich nicht, wenn wir an einer Koordinate wackeln

- (2) Zu (3): Schreiben wir f komponentenweise auf. Die i -te Komponente von $A\mathbf{x}$ ist

$$f_i(\mathbf{x}) = \sum_{j=1}^n a_{ij} x_j, \quad \text{also} \quad \frac{\partial f_i(\mathbf{x})}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{j=1}^n a_{ij} x_j \right) = a_{ik}.$$

in der Summe hängt nur der Summand mit $j = k$ von x_k ab, und zwar linear mit Faktor a_{ik} ; alle anderen sind bezüglich x_k konstant

- (3) Die partielle Ableitung an der Stelle (i, k) ist damit der Matrixeintrag a_{ik} selbst, und das Sammeln aller Einträge liefert

$$\mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} = A.$$

Definition 10.3.1: Zeilenindex i = Ausgabe, Spaltenindex k = Eingabe, exakt die Indizierung von A

- (4) Zu (2): Das ist der Sonderfall $A = I_n$ von (3), denn $\mathbf{x} = I_n \mathbf{x}$. Direkt gesehen: $f_i(\mathbf{x}) = x_i$ hat die partiellen Ableitungen $\partial f_i / \partial x_j = 1$ für $i = j$ und 0 sonst.

das ist gerade die Einheitsmatrix

- (5) Zu (4): Der Ausdruck $\mathbf{x}^\top A$ ist eine Zeile mit m Einträgen. Lesen wir sie als Spalte, so ist $(\mathbf{x}^\top A)^\top = A^\top \mathbf{x}$, und Teil (3) mit der Matrix $A^\top \in \mathbb{R}^{m \times n}$ liefert $\mathbf{J}_f(\mathbf{x}) = A^\top$.

Transponieren dreht ein Produkt um: $(\mathbf{x}^\top A)^\top = A^\top (\mathbf{x}^\top)^\top = A^\top \mathbf{x}$

□

Bemerkung 10.3.5 (Zur Identität für $\mathbf{x}^\top A$)

Streng genommen bildet $f(\mathbf{x}) = \mathbf{x}^\top A$ nicht in den $\mathbb{R}^m$ ab, sondern in den Raum der Zeilenvektoren $\mathbb{R}^{1 \times m}$. Definition 10.3.1 verlangt aber eine Spalte. Wir lesen die Zeile deshalb stillschweigend als Spalte, rechnen also mit $A^\top \mathbf{x}$, und erhalten so die im Satz genannte $m \times n$ -Matrix $A^\top$. Wer stattdessen im Zeilenformat bleibt, bekommt dieselben Zahlen in transponierter Anordnung. Solche Formatverabredungen sind kein Selbstzweck: Sie entscheiden, ob die Kettenregel weiter unten ohne zusätzliche Transponierte auskommt.

Korollar 10.3.6 (Lineare Abbildungen sind ihre eigene Näherung)

Für $f(\mathbf{x}) = A\mathbf{x}$ gilt

$$f(\mathbf{x} + \mathbf{h}) = A\mathbf{x} + A\mathbf{h} = f(\mathbf{x}) + \mathbf{J}_f(\mathbf{x})\mathbf{h},$$

und zwar exakt für jedes $\mathbf{h}$, ohne Restterm. Die lineare Näherung (10.3.1) ist hier keine Näherung, sondern eine Identität.

Das ist der Grund, warum die vier Bausteine so nützlich sind. Sie bilden zusammen mit der Kettenregel den Werkzeugkasten, mit dem sich fast alle Ableitungen dieses Kapitels zusammensetzen lassen, und bei ihnen selbst fällt kein Fehler an.

Was die Jacobimatrix geometrisch anstellt

Für eine nichtlineare Abbildung $f: \mathbb{R}^2 \rightarrow \mathbb{R}^2$ ist die Lage interessanter. Sie verbiegt die Ebene, obwohl Urbild und Bild dieselbe Dimension haben. Legen wir ein feines Gitter um einen Punkt $\mathbf{x}_0$, so wird daraus ein krummes Netz um $f(\mathbf{x}_0)$. Die Jacobimatrix beschreibt dieses krumme Netz näherungsweise durch ein Parallelogrammgitter: Die beiden Spalten $\mathbf{J}_f(\mathbf{x}_0)\mathbf{e}_1$ und $\mathbf{J}_f(\mathbf{x}_0)\mathbf{e}_2$ sind die Bilder der beiden Koordinatenrichtungen unter der Linearisierung, sie spannen die Maschen dieses Gitters auf, und je kleiner das Gitter, desto besser passt es auf das krumme Netz.

Beispiel 10.3.7 (Quadrieren in der Ebene)

Wir nehmen

$$f(\mathbf{x}) = \begin{pmatrix} x_1^2 - x_2^2 \\ 2x_1x_2 \end{pmatrix}, \quad \mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} 2x_1 & -2x_2 \\ 2x_2 & 2x_1 \end{pmatrix}.$$

Wer die Ebene als komplexe Zahlenebene liest, erkennt darin die Abbildung $z \mapsto z^2$. An der Stelle $\mathbf{x}_0 = (1; 0,5)^\top$ ist

$$f(\mathbf{x}_0) = \begin{pmatrix} 0,75 \\ 1 \end{pmatrix}, \quad \mathbf{J}_f(\mathbf{x}_0) = \begin{pmatrix} 2 & -1 \\ 1 & 2 \end{pmatrix}.$$

Der Restterm lässt sich hier vollständig ausrechnen. Setzen wir $\mathbf{x}_0 + \mathbf{h}$ ein und sortieren nach Potenzen von $\mathbf{h}$, so bleibt

$$\mathbf{r}(\mathbf{h}) = f(\mathbf{x}_0 + \mathbf{h}) - f(\mathbf{x}_0) - \mathbf{J}_f(\mathbf{x}_0)\mathbf{h} = \begin{pmatrix} h_1^2 - h_2^2 \\ 2h_1h_2 \end{pmatrix} = f(\mathbf{h}).$$

Der Fehler der linearen Näherung ist also die Funktion selbst, ausgewertet am Zuwachs. Seine Länge kennen wir damit exakt:

$$\|\mathbf{r}(\mathbf{h})\|^2 = (h_1^2 - h_2^2)^2 + 4h_1^2h_2^2 = (h_1^2 + h_2^2)^2, \quad \text{also} \quad \|\mathbf{r}(\mathbf{h})\| = \|\mathbf{h}\|^2.$$

Der relative Fehler ist demnach $\|\mathbf{r}(\mathbf{h})\| / \|\mathbf{h}\| = \|\mathbf{h}\|$, und das geht für $\mathbf{h} \rightarrow \mathbf{0}$ gegen null. Damit ist die $o(\|\mathbf{h}\|)$ -Bedingung nicht nur erfüllt, wir sehen auch, wie schnell: Bei $\mathbf{h} = (0,1; 0,1)^\top$ mit $\|\mathbf{h}\| \approx 0,1414$ ist der Fehler 0,02 lang, bei $\mathbf{h} = (0,01; 0,01)^\top$ nur noch 0,0002. Ein Zehntel Schrittweite kostet ein Hundertstel Fehler.

Vertiefung: Die Jacobi-Determinante als Flächenfaktor

Bemerkung 10.3.8 (Wie stark die Fläche verzerrt wird)

Das Bild dazu ist schnell erzählt: ein kleines Quadrat links, sein verbogenes Bild rechts, dazu das Parallelogramm der linearen Näherung. Wie stark f die Fläche dehnt oder staucht, steht in der Determinante der Jacobimatrix. Ein achsenparalleles Quadrat mit Kantenlänge a um $\mathbf{x}_0$ geht unter der Linearisierung in das Parallelogramm über, das von $a \mathbf{J}_f(\mathbf{x}_0) \mathbf{e}_1$ und $a \mathbf{J}_f(\mathbf{x}_0) \mathbf{e}_2$ aufgespannt wird; dessen Flächeninhalt ist das $|\det \mathbf{J}_f(\mathbf{x}_0)|$ -fache von a^2 . Weil sich das krumme Bild für kleine Quadrate immer besser an dieses Parallelogramm anschmiegt, gilt derselbe Faktor im Grenzwert auch für f selbst. In Beispiel 10.3.7 ist $\det \mathbf{J}_f(\mathbf{x}) = 4x_1^2 + 4x_2^2$, an der Stelle $\mathbf{x}_0$ also 5: Ein kleines Quadrat um $\mathbf{x}_0$ wird auf annähernd die fünffache Fläche aufgeblasen, und je kleiner es ist, desto genauer stimmt der Faktor. In der Wahrscheinlichkeitsrechnung ist genau dieser Faktor das, was beim Transformationssatz für Dichten den Ausgleich schafft.

Interaktiv: Linearisierung zum Verkleinern

Wie schnell verschwindet der Unterschied zwischen dem krummen Bild und dem Parallelogramm, wenn wir die Umgebung schrumpfen lassen? Und trifft der Flächenfaktor $|\det \mathbf{J}_f(\mathbf{x}_0)|$ wirklich zu, oder gilt er nur im Grenzwert? Beides ist eine Frage an ein Bild, nicht an eine Formel.

Interaktives Element — nur in der Web-Fassung

Links liegt ein regelmäßiges Gitter über dem ganzen Definitionsbereich, dazu das violette Fenster der halben Kantenlänge h um $\mathbf{x}_0$. In der Mitte steht das Bild desselben Gitters unter f , in der Lupe rechts das krumme Bild des Fensters (blau) gegen das Parallelogramm $f(\mathbf{x}_0) + \mathbf{J}_f(\mathbf{x}_0)\mathbf{h}$ (grün). Die roten Strecken in den vier Ecken sind der Restterm, die orangen Pfeile sind h mal die Spalten von $\mathbf{J}_f(\mathbf{x}_0)$.

Wie das Widget zeigt, fällt der rote Abstand beim Halbieren von h auf ungefähr ein Viertel, während das Fenster nur halb so groß wird. Der Quotient $\|\mathbf{r}(\mathbf{h})\| / \|\mathbf{h}\|$ halbiert sich also mit, und mehr verlangt (10.3.1) auch nicht. Bei der Voreinstellung „linear“ bleibt der Restterm exakt null, so wie es Korollar 10.3.6 verlangt, und das Flächenverhältnis ist dort für jedes h exakt $|\det \mathbf{J}_f(\mathbf{x}_0)| = 1,65$. Bei den krummen Abbildungen nähert es sich diesem Wert erst für kleines h : Beim Quadrieren an der Stelle $\mathbf{x}_0 = (1; 0,5)^\top$ steht bei $h = 0,3$ noch 5,24 und bei $h = 0,075$ bereits 5,015, gegen den Flächenfaktor $|\det \mathbf{J}_f(\mathbf{x}_0)| = 5$. Die Voreinstellung „Wirbel“ ist der Sonderfall dazu: Dort ist $\det \mathbf{J}_f(\mathbf{x}) = 1$ an jeder Stelle, die Abbildung verschiebt Fläche also, ohne sie zu verändern, und trotzdem verbiegt sie das Gitter kräftig.

Verkettung

Bisher haben wir einzelne Abbildungen abgeleitet. Interessant wird es, wenn mehrere hintereinandergeschaltet sind, wenn also die Ausgabe der einen die Eingabe der nächsten ist. Für die Verkettung gilt dieselbe Regel wie in der Analysis, nur dass jetzt Matrizen multipliziert werden.

Satz 10.3.9 (Kettenregel für Jacobimatrizen)

Seien $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ in $\mathbf{x}$ differenzierbar und $f : \mathbb{R}^m \rightarrow \mathbb{R}^p$ in $g(\mathbf{x})$ differenzierbar. Dann ist $f \circ g$ in $\mathbf{x}$ differenzierbar mit

$$\mathbf{J}_{f \circ g}(\mathbf{x}) = \mathbf{J}_f(g(\mathbf{x})) \mathbf{J}_g(\mathbf{x}). \quad (10.3.2)$$

Die Formatprobe erledigt sich von selbst: rechts steht eine $p \times m$ -Matrix mal einer $m \times n$ -Matrix, das Produkt ist $p \times n$, und genau das verlangt Definition 10.3.1 für eine Abbildung von $\mathbb{R}^n$ nach $\mathbb{R}^p$.

Der Beweis führt dieselbe Rechnung wie später Satz 10.6.7, dort ohne Koordinaten und für beliebige normierte Räume: den Zuwachs abkürzen, ihn gegen $\|\mathbf{h}\|$ abschätzen, die Linearität zum Aufspalten nutzen und beide Restterme auf denselben Bezugspunkt bringen. Wer die allgemeine Fassung liest, kann die folgende Vertiefung überspringen.

Vertiefung: Der Beweis in Koordinaten

Beweis.

- (1) Wir kürzen $\mathbf{y} := g(\mathbf{x})$ ab und bezeichnen mit $\mathbf{k} := g(\mathbf{x}+\mathbf{h}) - g(\mathbf{x})$ den Zuwachs, den g weiterreicht. Die Differenzierbarkeit von g heißt

$$\mathbf{k} = \mathbf{J}_g(\mathbf{x}) \mathbf{h} + \mathbf{r}_g(\mathbf{h}), \quad \|\mathbf{r}_g(\mathbf{h})\| / \|\mathbf{h}\| \rightarrow 0.$$

(10.3.1), angewandt auf g an der Stelle $\mathbf{x}$

- (2) Die Differenzierbarkeit von f an der Stelle $\mathbf{y}$ liefert $f(\mathbf{y} + \mathbf{k}) = f(\mathbf{y}) + \mathbf{J}_f(\mathbf{y}) \mathbf{k} + \mathbf{r}_f(\mathbf{k})$. Setzen wir den ersten Schritt ein und sortieren, so entsteht

$$f(g(\mathbf{x} + \mathbf{h})) = f(g(\mathbf{x})) + \mathbf{J}_f(\mathbf{y}) \mathbf{J}_g(\mathbf{x}) \mathbf{h} + \underbrace{\mathbf{J}_f(\mathbf{y}) \mathbf{r}_g(\mathbf{h}) + \mathbf{r}_f(\mathbf{k})}_{=: \mathbf{r}(\mathbf{h})}.$$

$g(\mathbf{x} + \mathbf{h}) = \mathbf{y} + \mathbf{k}$; die Matrixmultiplikation ist assoziativ, deshalb dürfen wir $\mathbf{J}_f(\mathbf{y})(\mathbf{J}_g(\mathbf{x})\mathbf{h})$ als $(\mathbf{J}_f(\mathbf{y})\mathbf{J}_g(\mathbf{x}))\mathbf{h}$ lesen

- (3) Bleibt zu zeigen, dass $\mathbf{r}(\mathbf{h})$ wirklich $o(\|\mathbf{h}\|)$ ist. Für den ersten Summanden schätzen wir mit der Matrixnorm ab,

$$\|\mathbf{J}_f(\mathbf{y}) \mathbf{r}_g(\mathbf{h})\| \leq \|\mathbf{J}_f(\mathbf{y})\| \|\mathbf{r}_g(\mathbf{h})\|,$$

und die rechte Seite ist ein festes Vielfaches von $\|\mathbf{r}_g(\mathbf{h})\| = o(\|\mathbf{h}\|)$.

$\|\mathbf{A}\mathbf{v}\| \leq \|\mathbf{A}\| \|\mathbf{v}\|$ für die induzierte Operatornorm, Abschnitt 3.3; $\|\mathbf{J}_f(\mathbf{y})\|$ hängt nicht von $\mathbf{h}$ ab

- (4) Für den zweiten Summanden brauchen wir, dass $\mathbf{k}$ mit $\mathbf{h}$ klein wird. Aus dem ersten Schritt folgt für hinreichend kleine $\mathbf{h}$ die Abschätzung $\|\mathbf{k}\| \leq (\|\mathbf{J}_g(\mathbf{x})\| + 1) \|\mathbf{h}\|$. Also ist $\|\mathbf{r}_f(\mathbf{k})\| / \|\mathbf{h}\| \leq (\|\mathbf{J}_g(\mathbf{x})\| + 1) \cdot \|\mathbf{r}_f(\mathbf{k})\| / \|\mathbf{k}\|$, und das geht gegen null. Damit erfüllt die lineare Abbildung $\mathbf{h} \mapsto \mathbf{J}_f(\mathbf{y})\mathbf{J}_g(\mathbf{x})\mathbf{h}$ die Bedingung aus Lemma 10.3.3, und dieses Lemma liefert (10.3.2).

der Bruch $\|\mathbf{r}_f(\mathbf{k})\| / \|\mathbf{k}\|$ geht gegen null, weil mit $\mathbf{h}$ auch $\mathbf{k}$ gegen $\mathbf{0}$ geht; für $\mathbf{k} = \mathbf{0}$ ist $\mathbf{r}_f(\mathbf{k}) = \mathbf{0}$, der Quotient also gar nicht gebildet

□

Anwendung: Backpropagation

Ein neuronales Netz ist eine Verkettung, wie sie im Buche steht. Aus einer Eingabe $\mathbf{x}$ wird über K Schichten eine Vorhersage $\hat{\mathbf{y}}$,

$$\mathbf{x} \xrightarrow{f_1} \mathbf{z}_1 \xrightarrow{f_2} \mathbf{z}_2 \xrightarrow{\dots} \mathbf{z}_{K-1} \xrightarrow{f_K} \hat{\mathbf{y}} = (f_K \circ \dots \circ f_2 \circ f_1)(\mathbf{x}).$$

Jede Schicht f_k hat eigene Parameter θ_k , also Gewichte und Biases, und rechnet typischerweise $f_k(\mathbf{z}_{k-1}) = \sigma_k(\mathbf{W}_k(\theta_k) \mathbf{z}_{k-1})$ mit einer Gewichtsmatrix $\mathbf{W}_k$ und einer komponentenweise wirken-

den Aktivierungsfunktion σ_k . Eine Verlustfunktion $L(\hat{\mathbf{y}}, \mathbf{y})$ misst anhand des Zielwerts $\mathbf{y}$, wie gut die Vorhersagen sind; in der folgenden Formel hängt sie bei festem $\hat{\mathbf{y}}$ nicht noch einmal direkt von den Parametern ab. Zum Trainieren brauchen wir für jede Schicht die Ableitung $\partial L / \partial \theta_k$, denn genau die verlangt der Gradientenabstieg aus Algorithmus 10.2.10.

Bemerkung 10.3.10 (Die Gradientenkette eines Netzes)

Zwischen θ_k und L liegen die Schichten $k + 1$ bis K und danach die Verlustfunktion. Von θ_k hängt dabei allein die Schicht k ab: Ihre Eingabe $\mathbf{z}_{k-1}$ entsteht aus den Schichten davor und bleibt fest, wenn wir θ_k verändern. Satz 10.3.9, mehrfach angewandt, ergibt

$$\frac{\partial L}{\partial \theta_k} = \frac{\partial L}{\partial \hat{\mathbf{y}}} \cdot \mathbf{J}_{f_K}(\mathbf{z}_{K-1}) \cdot \mathbf{J}_{f_{K-1}}(\mathbf{z}_{K-2}) \cdots \mathbf{J}_{f_{k+1}}(\mathbf{z}_k) \cdot \frac{\partial f_k(\mathbf{z}_{k-1}; \theta_k)}{\partial \theta_k}. \quad (10.3.3)$$

Die Formate lesen sich von links nach rechts wie eine Kette von Dominosteinen. Ganz links steht mit $\partial L / \partial \hat{\mathbf{y}}$ ein Zeilenvektor, weil L skalar ist. Jede Jacobimatrix $\mathbf{J}_{f_j}(\mathbf{z}_{j-1})$ hat so viele Zeilen wie $\mathbf{z}_j$ Komponenten und so viele Spalten wie $\mathbf{z}_{j-1}$. Der letzte Faktor leitet die Schicht k nach ihren *eigenen* Parametern ab, nicht nach ihrer Eingabe. Am Ende steht eine Zeile mit so vielen Einträgen, wie θ_k Parameter hat.

Enthält der Gesamtverlust zusätzlich einen direkten Parameterterm, etwa eine Regularisierung $R(\theta)$, kommt dessen Gradient additiv hinzu: $\partial(L + R) / \partial \theta_k$ ist die Kette aus (10.3.3) plus $\partial R / \partial \theta_k$.

Bemerkung 10.3.11 (Wo die Kette aufhört)

Eine Stolperstelle in (10.3.3) verdient einen eigenen Absatz, weil sie leicht zu übersehen ist. Das Produkt der Jacobimatrizen läuft nur bis $\mathbf{J}_{f_{k+1}}(\mathbf{z}_k)$, also bis zur Schicht *über* der gesuchten. Verlockend wäre, die Kette bis $\mathbf{J}_{f_k}(\mathbf{z}_{k-1})$ weiterzuziehen und erst dann nach θ_k abzuleiten. Das zählt $\mathbf{W}_k$ jedoch doppelt: $\mathbf{J}_{f_k}(\mathbf{z}_{k-1})$ ist die Ableitung der Schicht k nach ihrer Eingabe $\mathbf{z}_{k-1}$ und enthält $\mathbf{W}_k$ bereits als Faktor. Für die Parameter brauchen wir stattdessen die Ableitung derselben Schicht nach θ_k , bei festgehaltener Eingabe. Merkregel: Jede Schicht liefert genau einen Faktor, und für die Schicht k ist das der nach θ_k abgeleitete.

Beispiel 10.3.12 (Jacobimatrix eines ReLU-Layers)

Nehmen wir als Aktivierung die *ReLU-Funktion* (rectified linear unit) $\sigma(u) = \max(0, u)$, komponentenweise angewandt. Die Schicht rechnet also $f_k(\mathbf{z}_{k-1}) = \max(\mathbf{0}, \mathbf{W}_k \mathbf{z}_{k-1})$. Für die Ableitung nach der Eingabe verketteten wir zwei Bausteine: erst die lineare Abbildung $\mathbf{z} \mapsto \mathbf{W}_k \mathbf{z}$ mit Jacobimatrix $\mathbf{W}_k$ nach Satz 10.3.4(3), dann die komponentenweise Schwelle. Die zweite Abbildung reicht eine positive Komponente unverändert durch und setzt eine negative auf null; sie mischt die Komponenten nicht, also ist ihre Jacobimatrix diagonal, und auf der Diagonale steht die Ableitung von $\max(0, \cdot)$, je nach Vorzeichen eine 1 oder eine 0. Nach Satz 10.3.9 ist

$$\mathbf{J}_{f_k}(\mathbf{z}_{k-1}) = \text{diag}(\mathbb{1}_{\{\mathbf{W}_k \mathbf{z}_{k-1} > \mathbf{0}\}}) \cdot \mathbf{W}_k. \quad (10.3.4)$$

Die Diagonalmatrix trägt an der Stelle i eine 1, falls $(\mathbf{W}_k \mathbf{z}_{k-1})_i > 0$ ist, und sonst eine 0. Sie streicht also gerade die Zeilen von $\mathbf{W}_k$, die zu inaktiven Einheiten gehören. Ein Zahlenbeispiel mit

$$\mathbf{W} = \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix}, \quad \mathbf{z} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \mathbf{W}\mathbf{z} = \begin{pmatrix} -1 \\ 4 \end{pmatrix} :$$

Die erste Einheit ist inaktiv, die zweite aktiv, also

$$\mathbf{J}_f(\mathbf{z}) = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 2 & 1 \end{pmatrix}.$$

Vorsicht an den Schaltstellen: Ist $(\mathbf{W}_k \mathbf{z}_{k-1})_i = 0$ und die i -te Zeile von $\mathbf{W}_k$ nicht der Nullvektor, etwa für $\mathbf{z} = (1; 1)^\top$ mit $\mathbf{W}\mathbf{z} = (0; 3)^\top$, dann hat diese Ausgabekomponente dort einen Knick und der Layer ist nicht differenzierbar (Bemerkung 10.1.2). Eine Nullzeile ist die Ausnahme: Dann ist die betreffende Komponente konstant null und damit differenzierbar. In der Praxis wird an einem echten Knick meist per Verabredung die Ableitung null verwendet. Bei festen von null verschiedenen Zeilen und kontinuierlich verteilten Eingaben bilden die Schalthyperebenen eine Nullmenge; bei diskreten Daten oder Nullgewichten können sie dagegen durchaus exakt getroffen werden.

Vertiefung: Wie teuer die Kette ist, und warum sie von links ausgewertet wird

Bemerkung 10.3.13 (Wie die Kette ausgewertet wird)

Gleichung (10.3.3) ist ein Produkt aus vielen Matrizen, und Matrixmultiplikation ist assoziativ: Wir dürfen die Klammern setzen, wie wir wollen. Für den Rechenaufwand macht das einen großen Unterschied, denn eine $m \times n$ -Matrix mal eine $n \times p$ -Matrix kostet mnp Multiplikationen (Abschnitt 2.3).

Rechnen wir ein kleines Netz mit drei Schichten durch, mit $\mathbf{z}_1 \in \mathbb{R}^{50}$, $\mathbf{z}_2 \in \mathbb{R}^{20}$ und $\hat{\mathbf{y}} \in \mathbb{R}^{10}$, und zwar die Kette bis hinunter zur ersten Schicht. Beginnen wir in (10.3.3) ganz links, so bleiben wir immer bei einer Zeile: erst $(1 \times 10) \cdot (10 \times 20)$ mit 200 Multiplikationen, dann $(1 \times 20) \cdot (20 \times 50)$ mit 1000, zusammen 1200. Klammern wir dagegen zuerst die beiden Jacobimatrizen zusammen, so entsteht das volle Matrixprodukt $(10 \times 20) \cdot (20 \times 50)$ mit 10 000 Multiplikationen und danach noch einmal 500, zusammen 10 500, also fast das Neunfache.

Der Grund ist immer derselbe: Weil L skalar ist, ist der äußerste linke Faktor eine Zeile, und eine Zeile mal einer Matrix bleibt eine Zeile. Wer bei $\partial L / \partial \hat{\mathbf{y}}$ anfängt, hält also stets nur einen Vektor im Speicher; wer am anderen Ende beginnt, baut große Zwischenmatrizen auf. In der Sprache des Netzes heißt „links anfangen“ gerade: von der Ausgabe rückwärts Schicht für Schicht zur Schicht k laufen. Nichts anderes ist *Backpropagation*, und dazu kommt eine zweite Ersparnis: Das Teilprodukt für die Schicht k entsteht aus dem für die Schicht $k+1$ durch einen einzigen weiteren Faktor, nämlich $\mathbf{J}_{f_{k+1}}(\mathbf{z}_k)$. Ein Rückwärtslauf liefert deshalb die Gradienten *aller* Schichten, nicht nur den einer einzelnen. Neuronale Netze lassen sich überhaupt nur deshalb mit Millionen von Parametern trainieren.

Wo genau endet die Kette, wenn wir sie bis zu $\mathbf{W}_1$ zurückverfolgen? Und was steht unterwegs eigentlich in den Zwischengrößen, Matrizen oder Zeilen? Am kleinsten möglichen Netz lässt sich das Schritt für Schritt nachzählen.

Interaktiv: Vorwärts und rückwärts durch ein winziges Netz

Das Netz hat zwei Schichten: $\mathbf{a}_1 = \mathbf{W}_1 \mathbf{x}$, dann $z_1 = \max(\mathbf{0}, \mathbf{a}_1)$, dann $\hat{y} = \mathbf{W}_2 z_1$ und schließlich $L = \frac{1}{2}(\hat{y} - y)^2$ mit dem Zielwert $y = 1$. Unter den Knoten stehen blau die Zahlen des Vorwärtslaufs, über den Knoten orange die Ableitung von L nach dem jeweiligen Knoten.

Interaktives Element — nur in der Web-Fassung

Jede Zwischengröße des Rückwärtslaufs bleibt eine Zeile, weil links immer die Zeile des vorigen Schritts steht und rechts eine Jacobimatrix; erst der letzte Schritt ordnet das Ergebnis als Gewichtsmatrix an, denn $\mathbf{W}_1$ ist keine Liste, sondern ein Rechteck. Und die Kette für $\mathbf{W}_1$ endet bei $\mathbf{J}_{f_2}(\mathbf{z}_1) = \mathbf{W}_2$: Der letzte Faktor ist die Ableitung der ersten Schicht nach ihren eigenen Gewichten, nicht noch einmal $\mathbf{W}_1$ (Bemerkung 10.3.11). Stellen wir den Regler auf $x_1 = 2$, so landen wir auf einer Knickstelle der Art aus Beispiel 10.3.12.

1. Ob wir das Produkt in (10.3.3) von links oder von rechts auswerten, ändert nur die Reihenfolge der Rechenschritte, nicht den Aufwand.

Antwort: Falsch

Das Ergebnis ist dasselbe, der Aufwand nicht. Im Beispiel aus Bemerkung 10.3.13 kostet die Auswertung von links 1200 und die von rechts 10 500 Multiplikationen. Weil L skalar ist, bleibt von links jedes Zwischenergebnis eine Zeile; von rechts entstehen volle Matrizen. Genau diese Auswertungsreihenfolge ist die Backpropagation.

Selbsttest

1. Für $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ ist $\mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{m \times n}$, und die i -te Zeile ist der Gradient der i -ten Komponentenfunktion f_i .

Antwort: Wahr

Das ist Definition 10.3.1 zusammen mit Bemerkung 10.3.2: Pro Ausgabekomponente gibt es eine Zeile, pro Eingabevariable eine Spalte. Für $m = 1$ bleibt genau eine Zeile übrig, nämlich der Gradient aus Definition 10.2.1.

2. Für $f(\mathbf{x}) = \mathbf{A}\mathbf{x}$ mit $\mathbf{A} \in \mathbb{R}^{m \times n}$ hängt die Jacobimatrix von der Stelle $\mathbf{x}$ ab.

Antwort: Falsch

Sie ist konstant gleich $\mathbf{A}$ (Satz 10.3.4). Der Grund steht in Korollar 10.3.6: f ist selbst linear, also ist die beste lineare Näherung f selbst, und der Restterm verschwindet für jedes $\mathbf{h}$ exakt.

3. Die Kettenregel lautet $\mathbf{J}_{f \circ g}(\mathbf{x}) = \mathbf{J}_g(\mathbf{x}) \mathbf{J}_f(g(\mathbf{x}))$.

Antwort: Falsch

Die Reihenfolge ist vertauscht; richtig ist (10.3.2), $\mathbf{J}_{f \circ g}(\mathbf{x}) = \mathbf{J}_f(g(\mathbf{x})) \mathbf{J}_g(\mathbf{x})$. Schon die Formate verraten den Fehler: Für $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ und $f : \mathbb{R}^m \rightarrow \mathbb{R}^p$ wäre das falsch geordnete Produkt $(m \times n) \cdot (p \times m)$ gar nicht definiert, außer wenn zufällig $n = p$ gilt.

4. Für die Ableitung nach θ_k multiplizieren wir die Jacobimatrizen bis einschließlich $\mathbf{J}_{f_k}(\mathbf{z}_{k-1})$ und hängen danach noch $\partial \mathbf{W}_k(\theta_k) / \partial \theta_k$ an.

Antwort: Falsch

Damit ginge $\mathbf{W}_k$ zweimal in das Produkt ein. Die Matrix $\mathbf{J}_{f_k}(\mathbf{z}_{k-1})$ ist die Ableitung der Schicht k nach ihrer Eingabe und enthält $\mathbf{W}_k$ bereits, siehe (10.3.4) für den ReLU-Fall. Die Kette endet nach (10.3.3) bei $\mathbf{J}_{f_{k+1}}(\mathbf{z}_k)$, und der letzte Faktor ist $\partial f_k(\mathbf{z}_{k-1}; \theta_k) / \partial \theta_k$ (Bemerkung 10.3.11).

5. Ein ReLU-Layer $f(\mathbf{z}) = \max(\mathbf{0}, \mathbf{W}\mathbf{z})$ ist an einer Stelle mit $(\mathbf{W}\mathbf{z})_i = 0$ nicht differenzierbar, sofern die i -te Zeile von $\mathbf{W}$ nicht der Nullvektor ist.

Antwort: Wahr

Dort hat $\max(0, \cdot)$ einen Knick, links und rechts ergeben sich verschiedene Steigungen (Bemerkung 10.1.2). Formel (10.3.4) gilt deshalb nur abseits echter Schalthyperebenen; in der Praxis wird dort meist die Ableitung null verwendet. Bei einer Nullzeile wäre die Komponente dagegen konstant null und differenzierbar (Beispiel 10.3.12).

6. Im Linearisierungs-Widget: Wir wählen die Voreinstellung „Quadrieren“ und lassen $\mathbf{x}_0$ auf $(1; 0,5)^\top$ stehen. Welchen Wert zeigt $\det \mathbf{J}_f(\mathbf{x}_0)$?

Lösung: 5

$\mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} 2x_1 & -2x_2 \\ 2x_2 & 2x_1 \end{pmatrix}$ hat die Determinante $4(x_1^2 + x_2^2)$, an der Stelle $(1; 0,5)^\top$ also 5 (Beispiel 10.3.7). Dasselbe steht als Grenzwert des Flächenverhältnisses im Verdikt.

7. Um welchen Faktor fällt der Restterm $\|r(\mathbf{h})\|$ im Linearisierungs-Widget, wenn wir mit dem Knopf „h halbieren“ die Kantenlänge des Fensters halbieren?

Lösung: 4

Der Restterm fällt wie $\|\mathbf{h}\|^2$, also auf ein Viertel. Bei der Voreinstellung „Quadrieren“ ist das exakt so, bei den anderen krummen Abbildungen bis auf Terme höherer Ordnung.

Vertiefung: MML §5.3 (Jacobimatrix, Jacobi-Determinante als Flächenfaktor); MML §5.6 für Backpropagation und automatisches Differenzieren.

10.4 Ableitungen mit Matrizen

Zwei Felder der Tabelle aus Abschnitt 10.2 haben wir noch nicht besetzt. Im einen steht eine Matrix als Ausgabe und ein Skalar als Eingabe, im anderen genau umgekehrt. Beide Fälle kommen in der Statistik ständig vor, und beide brauchen kein neues Werkzeug: Wir leiten Eintrag für Eintrag ab und sortieren die Ergebnisse so, dass die Bedingung aus Definition 10.1.5 wieder in der gewohnten Gestalt dasteht. Die eigentliche Arbeit steckt in der Buchführung, nicht in der Analysis.

Skalar zu Matrix

Beginnen wir mit dem Fall, in dem eine ganze Matrix an einer einzigen Zahl hängt. Eine Kovarianzmatrix, die von einem Korrelationsparameter abhängt, ist so ein Objekt, ebenso eine Modellmatrix, in der ein Bandbreitenparameter steckt. Formal betrachten wir

$$\mathbf{F} : \mathbb{R} \rightarrow \mathbb{R}^{m \times n}, \quad \mathbf{F}(\mathbf{x}) = \begin{pmatrix} f_{11}(\mathbf{x}) & \cdots & f_{1n}(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ f_{m1}(\mathbf{x}) & \cdots & f_{mn}(\mathbf{x}) \end{pmatrix}.$$

Jeder der $m \cdot n$ Einträge ist eine gewöhnliche Funktion einer reellen Variablen. Wir können also jeden einzeln ableiten und die Ergebnisse an ihrem Platz stehen lassen.

Definition 10.4.1 (Ableitung einer matrixwertigen Funktion nach einem Skalar)

Sei $F: \mathbb{R} \rightarrow \mathbb{R}^{m \times n}$ so, dass alle Einträge f_{ij} an der Stelle $x \in \mathbb{R}$ differenzierbar sind. Dann heißt

$$\frac{\partial F(x)}{\partial x} = \begin{pmatrix} \frac{\partial f_{11}(x)}{\partial x} & \dots & \frac{\partial f_{1n}(x)}{\partial x} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_{m1}(x)}{\partial x} & \dots & \frac{\partial f_{mn}(x)}{\partial x} \end{pmatrix} \in \mathbb{R}^{m \times n} \quad (10.4.1)$$

die Ableitung von F nach x . Es gilt dann

$$F(x+h) = F(x) + D_x F(h) + o(|h|) \quad \text{mit} \quad D_x F(h) = \frac{\partial F(x)}{\partial x} h. \quad (10.4.2)$$

Bemerkung 10.4.2 (Das Format bleibt erhalten)

Drei Beobachtungen zur Definition.

Die Ableitung hat dieselbe Gestalt wie die Funktion. Sowohl $F(x)$ als auch $\partial F(x)/\partial x$ liegen in $\mathbb{R}^{m \times n}$. Das ist bequem, verführt aber dazu, beide zu verwechseln; die orange Matrix enthält Steigungen, die blaue Funktionswerte. Für $n = 1$ steht in (10.4.1) der Fall Skalar zu Vektor, den Abschnitt 10.3 als Jacobimatrix mit einer einzigen Spalte führt.

Warum die Ableitung eine Matrix sein darf. Nach Definition 10.1.5 ist die Ableitung eine lineare Abbildung $D_x F: \mathbb{R} \rightarrow \mathbb{R}^{m \times n}$. Eine lineare Abbildung mit eindimensionalem Definitionsbereich ist durch ihren Wert an der Stelle 1 vollständig festgelegt, denn $D_x F(h) = h \cdot D_x F(1)$. Dieser eine Wert ist die Matrix (10.4.1). Dasselbe Argument hat in Abschnitt 10.1 aus der Ableitung $f'(x)$ eine Zahl gemacht.

Der Restterm. In (10.4.2) misst $o(|h|)$ eine Matrix, wir brauchen dort also eine Matrixnorm, etwa die Frobenius-Norm aus Abschnitt 3.1. Da die Bedingung eintragsweise nachgerechnet wird und in endlich-dimensionalen Räumen alle Normen äquivalent sind, spielt die Wahl keine Rolle.

Beispiel 10.4.3 (Übung: eine 2×3 -Matrixfunktion)

Sei

$$F(x) = \begin{pmatrix} x^2 & 2e^x & 0 \\ 0 & x & \ln(x) \end{pmatrix}, \quad x > 0.$$

Die Einschränkung auf $x > 0$ kommt vom Logarithmus im letzten Eintrag. Was ist $\partial F(x)/\partial x$? Das ist eine gute Stelle für einen eigenen Versuch, bevor wir weiterlesen.

Lösung

Ableiten heißt hier schlicht: sechsmal die Schulregeln anwenden und das Ergebnis an derselben Stelle notieren.

$$\frac{\partial F(x)}{\partial x} = \begin{pmatrix} 2x & 2e^x & 0 \\ 0 & 1 & 1/x \end{pmatrix}.$$

Der konstante Eintrag 0 bleibt 0, und $2e^x$ steht unverändert da, weil die Exponentialfunktion ihre eigene Ableitung ist. An der Stelle $x = 2$ etwa lauten die Einträge 4 , $2e^2 \approx 14,778$, 0 , 0 , 1 und $0,5$.

Kennzahlen von F ableiten

Interessant wird es, wenn wir nicht F selbst ableiten wollen, sondern eine Größe, die aus F gewonnen wird: die Spur, die Determinante oder die Inverse. Für alle drei gibt es geschlossene Formeln, die mit $\partial F(x)/\partial x$ auskommen. Ein Wort zu den Voraussetzungen vorweg: Alle drei Größen sind nur für quadratische Matrizen erklärt. Ab hier setzen wir deshalb $F : \mathbb{R} \rightarrow \mathbb{R}^{n \times n}$ voraus; nur Definition 10.4.1 selbst kommt mit beliebigen Formaten aus.

Satz 10.4.4 (Identitäten für Skalar zu Matrix)

Sei $F : \mathbb{R} \rightarrow \mathbb{R}^{n \times n}$ an der Stelle x differenzierbar. Dann gilt

$$\frac{\partial \operatorname{tr}(F(x))}{\partial x} = \operatorname{tr}\left(\frac{\partial F(x)}{\partial x}\right), \quad (10.4.3)$$

und falls $F(x)$ zusätzlich invertierbar ist, auch

$$\frac{\partial \det(F(x))}{\partial x} = \det(F(x)) \operatorname{tr}\left[F(x)^{-1} \frac{\partial F(x)}{\partial x}\right] \quad (10.4.4)$$

sowie

$$\frac{\partial F(x)^{-1}}{\partial x} = -F(x)^{-1} \frac{\partial F(x)}{\partial x} F(x)^{-1}. \quad (10.4.5)$$

Gleichung (10.4.4) heißt *Jacobi-Formel* (Jacobi's formula).

Beweis.

- (1) Wir beweisen die Spur-Identität (10.4.3); die beiden anderen Formeln übernehmen wir aus der Literatur (MML §5.5) und prüfen sie am Ende dieses Abschnitts numerisch nach. Der Beweis kommt mit einer einzigen Beobachtung aus, denn die Spur liest nur die Diagonale:

$$\operatorname{tr}(F(x)) = \sum_{i=1}^n f_{ii}(x).$$

Definition 3.1.2 der Spur (Abschnitt 3.1): Summe der Diagonaleinträge

- (2) Die rechte Seite ist eine endliche Summe von Funktionen einer Variablen. Wir dürfen also gliedweise ableiten:

$$\frac{\partial}{\partial x} \operatorname{tr}(F(x)) = \frac{\partial}{\partial x} \sum_{i=1}^n f_{ii}(x) = \sum_{i=1}^n \frac{\partial f_{ii}(x)}{\partial x}.$$

die Ableitung einer endlichen Summe ist die Summe der Ableitungen; jedes f_{ii} ist nach Voraussetzung differenzierbar

- (3) Rechts steht nun die Summe der Diagonaleinträge derjenigen Matrix, die in (10.4.1) definiert wurde, also

$$\sum_{i=1}^n \frac{\partial f_{ii}(x)}{\partial x} = \operatorname{tr}\left(\frac{\partial F(x)}{\partial x}\right).$$

der Eintrag (i, i) von $\partial F(x)/\partial x$ ist gerade $\partial f_{ii}(x)/\partial x$

□

Hinter dem Beweis steckt ein Prinzip, das über die Spur hinausreicht: Die Spur ist eine lineare Abbildung von $\mathbb{R}^{n \times n}$ nach $\mathbb{R}$, und lineare Abbildungen vertauschen mit dem Ableiten. Wo eine Größe linear in den Einträgen ist, dürfen wir Ableiten und Auswerten in beliebiger Reihenfolge ausführen. Determinante und Inverse sind dagegen nicht linear, und man sieht ihren Formeln (10.4.4) und (10.4.5) an, dass dort mehr passiert.

Vertiefung: Die Jacobi-Formel an einem Diagonalbeispiel nachgerechnet

Beispiel 10.4.5 (Die Jacobi-Formel an einem Diagonalbeispiel)

Sei

$$\mathbf{F}(x) = \begin{pmatrix} x & 0 \\ 0 & 2x \end{pmatrix}.$$

Direkt. Die Determinante einer Diagonalmatrix ist das Produkt der Diagonaleinträge, also

$$\det(\mathbf{F}(x)) = x \cdot 2x = 2x^2 \implies \frac{\partial}{\partial x} \det(\mathbf{F}(x)) = 4x.$$

Über die Jacobi-Formel. Wir brauchen die beiden Zutaten

$$\frac{\partial \mathbf{F}(x)}{\partial x} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad \mathbf{F}(x)^{-1} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/(2x) \end{pmatrix} \quad (x \neq 0),$$

und multiplizieren sie:

$$\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/(2x) \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/x \end{pmatrix}.$$

Die Spur dieses Produkts ist $1/x + 1/x = 2/x$, und (10.4.4) liefert

$$\det(\mathbf{F}(x)) \cdot \operatorname{tr} \left[\mathbf{F}(x)^{-1} \frac{\partial \mathbf{F}(x)}{\partial x} \right] = 2x^2 \cdot \frac{2}{x} = 4x.$$

Beide Wege stimmen überein. Wichtig ist dabei die Spur auf der rechten Seite: Ohne sie stünde dort ein Produkt aus einer Zahl und einer Matrix, also gar keine Zahl, während links die Ableitung einer skalaren Größe steht. Und noch eine Feinheit: Die Jacobi-Formel verlangt $x \neq 0$, während die direkte Rechnung auch bei $x = 0$ durchläuft und dort 0 liefert. Die Invertierbarkeit in Satz 10.4.4 ist also eine Voraussetzung der Formel, keine Bedingung dafür, dass die Ableitung existiert.

Bemerkung 10.4.6 (Warum das in der Statistik gebraucht wird)

Die Log-Dichte eines multivariaten Normalmodells enthält die Kovarianzmatrix $\Sigma(\theta)$ gleich zweimal: einmal als $\log \det(\Sigma(\theta))$ und einmal als $\Sigma(\theta)^{-1}$ in der quadratischen Form. Wer die Score-Funktion aufstellen will, also die Ableitung der Log-Likelihood nach θ , braucht deshalb genau (10.4.4) und (10.4.5). Aus der Jacobi-Formel wird mit der Kettenregel

$$\frac{\partial \log \det(\Sigma(\theta))}{\partial \theta} = \operatorname{tr} \left[\Sigma(\theta)^{-1} \frac{\partial \Sigma(\theta)}{\partial \theta} \right],$$

denn die äußere Ableitung des Logarithmus liefert den Faktor $1/\det(\Sigma)$, und der kürzt sich gegen das $\det(\Sigma)$ aus (10.4.4). Das ist eine der meistbenutzten Formeln der Statistik.

Stimmen diese drei Formeln wirklich, oder merken wir uns nur Muster? Eine Probe ist schnell gemacht: Wir werten $\mathbf{F}$ numerisch an $x \pm 10^{-5}$ aus und stellen den Differenzenquotienten neben den Formelwert.

Interaktiv: Die drei Identitäten gegen numerische Ableitungen halten

Die numerische Spalte kennt keine der Formeln, die Formelspalte rechnet mit Satz 10.4.4. Drei Beispiele stehen zur Wahl: die Diagonalmatrix $F(x) = \text{diag}(x, 2x)$, eine Matrix, in der jeder Eintrag anders von x abhängt (einer gar nicht), und eine Drehmatrix.

Interaktives Element — nur in der Web-Fassung

Alle drei Identitäten halten über den ganzen Reglerbereich stand; die größte Abweichung liegt unter 10^{-9} und ist der Abbruchfehler des Differenzenquotienten, nicht ein Fehler der Formel. Zwei Stellen lohnen einen zweiten Blick: Die Drehmatrix hat die konstante Determinante 1, also muss $\text{tr}[F^{-1} \partial F / \partial x]$ verschwinden. Und bei der Diagonalmatrix zeigt der Schieber die Lücke bei $x = 0$: Dort ist F singulär, die Determinanten- und die Inversenformel setzen aber beide F^{-1} voraus, während die Spur-Identität auch dort gilt.

Matrix zu Skalar

Jetzt drehen wir die Richtung um. Eine Verlustfunktion bewertet eine ganze Matrix und gibt dafür eine einzige Zahl zurück. Die Gewichtsmatrix einer Netzschicht, die Faktormatrix einer Zerlegung, die Kovarianzmatrix eines Modells: In allen Fällen wollen wir wissen, wie sich der Wert ändert, wenn wir an einem einzelnen Eintrag drehen. So lautet dann auch die Definition.

Definition 10.4.7 (Ableitung nach einer Matrix)

Sei $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ an der Stelle $\mathbf{X} = (x_{ij}) \in \mathbb{R}^{m \times n}$ Fréchet-differenzierbar. Dann existieren alle partiellen Ableitungen nach den Einträgen von $\mathbf{X}$, und die Matrix

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = \begin{pmatrix} \partial f(\mathbf{X}) / \partial x_{11} & \cdots & \partial f(\mathbf{X}) / \partial x_{1n} \\ \vdots & \ddots & \vdots \\ \partial f(\mathbf{X}) / \partial x_{m1} & \cdots & \partial f(\mathbf{X}) / \partial x_{mn} \end{pmatrix} \in \mathbb{R}^{m \times n} \quad (10.4.6)$$

heißt die *Ableitung von f nach $\mathbf{X}$* oder *Gradientenmatrix*. Sie stellt die Fréchet-Ableitung dar:

$$f(\mathbf{X} + \mathbf{H}) = f(\mathbf{X}) + D_{\mathbf{X}} f(\mathbf{H}) + o(\|\mathbf{H}\|) \quad \text{mit} \quad D_{\mathbf{X}} f(\mathbf{H}) = \text{tr} \left(\left(\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \right)^{\top} \mathbf{H} \right). \quad (10.4.7)$$

Bemerkung 10.4.8 (Der Ableitungsterm ist ein Skalarprodukt)

Die Spur in (10.4.7) sieht auf den ersten Blick nach einem Trick aus. Sie ist aber genau das, was wir schon kennen. Für zwei Matrizen $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$ gilt

$$\text{tr}(\mathbf{A}^{\top} \mathbf{B}) = \sum_{i=1}^m \sum_{j=1}^n a_{ij} b_{ij}, \quad (10.4.8)$$

und rechts steht das Skalarprodukt der beiden Matrizen, wenn wir sie als lange Vektoren der Länge $m \cdot n$ lesen. Man nennt es das *Frobenius-Skalarprodukt*; für $\mathbf{A} = \mathbf{B}$ ergibt es das Quadrat der Frobenius-Norm aus Abschnitt 3.1, was Satz 3.1.10 dort als $\|\mathbf{A}\|_F = \sqrt{\text{tr}(\mathbf{A}^{\top} \mathbf{A})}$ festhält. Damit lesen wir (10.4.7) so:

$$D_{\mathbf{X}}f(\mathbf{H}) = \sum_{i=1}^m \sum_{j=1}^n \frac{\partial f(\mathbf{X})}{\partial x_{ij}} h_{ij}.$$

Der Ableitungsterm summiert also über alle Einträge das Produkt aus Steigung und Auslenkung. Beim Gradienten in Abschnitt 10.2 stand an dieser Stelle das gewöhnliche Skalarprodukt $\nabla f(\mathbf{x}) \mathbf{h}$; hier ist es dieselbe Rechnung, nur mit zwei Indexrichtungen. Setzen wir speziell $\mathbf{H} = h \cdot \mathbf{E}_{ij}$, wobei $\mathbf{E}_{ij}$ die Matrix mit einer 1 an der Stelle (i, j) und sonst lauter Nullen ist, so bleibt von der Summe nur ein Term übrig, nämlich $h \cdot \partial f(\mathbf{X}) / \partial x_{ij}$. Das ist die Rechtfertigung dafür, die Ableitung eintragsweise zu definieren.

Beispiel 10.4.9 (Ableitung von $f(\mathbf{X}) = \mathbf{a}^\top \mathbf{X} \mathbf{b}$)

Seien $\mathbf{a} \in \mathbb{R}^m$ und $\mathbf{b} \in \mathbb{R}^n$ fest und $f(\mathbf{X}) = \mathbf{a}^\top \mathbf{X} \mathbf{b}$. Schreiben wir das Produkt aus, so steht dort eine Doppelsumme:

$$f(\mathbf{X}) = \sum_{i=1}^m \sum_{j=1}^n a_i x_{ij} b_j \implies \frac{\partial f(\mathbf{X})}{\partial x_{ij}} = a_i b_j.$$

In der Doppelsumme kommt x_{ij} genau einmal vor, und zwar mit dem Vorfaktor $a_i b_j$; alle anderen Summanden sind bezüglich x_{ij} konstant. Sortieren wir die Ergebnisse zurück in eine Matrix, erkennen wir ein äußeres Produkt:

$$\frac{\partial \mathbf{a}^\top \mathbf{X} \mathbf{b}}{\partial \mathbf{X}} = \begin{pmatrix} a_1 b_1 & \cdots & a_1 b_n \\ \vdots & \ddots & \vdots \\ a_m b_1 & \cdots & a_m b_n \end{pmatrix} = \mathbf{a} \mathbf{b}^\top. \quad (10.4.9)$$

Setzen wir das in (10.4.7) ein, so lässt sich die Spur wieder auflösen:

$$D_{\mathbf{X}}f(\mathbf{H}) = \text{tr}((\mathbf{a} \mathbf{b}^\top)^\top \mathbf{H}) = \text{tr}(\mathbf{b} \mathbf{a}^\top \mathbf{H}) = \mathbf{a}^\top \mathbf{H} \mathbf{b}.$$

Im letzten Schritt steckt die zyklische Vertauschung aus Satz 3.1.4: $\mathbf{a}^\top \mathbf{H} \mathbf{b}$ ist eine 1×1 -Matrix und damit gleich ihrer eigenen Spur. Die Probe ist nun leicht, denn f ist linear in $\mathbf{X}$:

$$f(\mathbf{X} + \mathbf{H}) = \mathbf{a}^\top \mathbf{X} \mathbf{b} + \mathbf{a}^\top \mathbf{H} \mathbf{b} = f(\mathbf{X}) + D_{\mathbf{X}}f(\mathbf{H}).$$

Der Restterm $o(\|\mathbf{H}\|)$ ist hier also nicht bloß klein, sondern exakt null. Bei einer linearen Funktion muss das so sein: Ihre beste lineare Näherung ist sie selbst.

Identitäten zum Nachschlagen

Satz 10.4.10 (Identitäten für Matrix zu Skalar)

Sei $\mathbf{X} \in \mathbb{R}^{m \times n}$.

1. *Quadratische Form.* Für $\mathbf{A} \in \mathbb{R}^{n \times n}$ und $f(\mathbf{X}) = \text{tr}(\mathbf{X} \mathbf{A} \mathbf{X}^\top)$ gilt

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = \mathbf{X}(\mathbf{A} + \mathbf{A}^\top), \quad (10.4.10)$$

für symmetrisches $\mathbf{A}$ also $2\mathbf{X}\mathbf{A}$.

2. *Spur-Produkt.* Für $\mathbf{A} \in \mathbb{R}^{m \times n}$ und $f(\mathbf{X}) = \text{tr}(\mathbf{A}^\top \mathbf{X})$ gilt

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = \mathbf{A}. \quad (10.4.11)$$

3. *Frobenius-Norm.* Für $f(\mathbf{X}) = \|\mathbf{X}\|_F^2 = \text{tr}(\mathbf{X}^\top \mathbf{X})$ gilt

$$\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = 2\mathbf{X}. \quad (10.4.12)$$

Alle drei Formeln lassen sich als Muster lesen. Hinter (10.4.11) steckt nichts weiter als (10.4.8): Dort ist f das Frobenius-Skalarprodukt von $\mathbf{A}$ mit $\mathbf{X}$, und die Ableitung ist der Partner $\mathbf{A}$ selbst. Das ist die Matrixfassung von $\partial(\mathbf{a}^\top \mathbf{x})/\partial \mathbf{x} = \mathbf{a}^\top$ aus Beispiel 10.2.7. In (10.4.10) steht $\mathbf{X}$ einmal links und einmal rechts von $\mathbf{A}$. Ausgeschrieben ist $\text{tr}(\mathbf{X}\mathbf{A}\mathbf{X}^\top) = \sum_{i,j,k} x_{ij} a_{jk} x_{ik}$, und beim Ableiten nach x_{pq} liefert die Produktregel zwei Beiträge: aus dem linken Vorkommen wird $[\mathbf{X}\mathbf{A}^\top]_{pq}$, aus dem rechten $[\mathbf{X}\mathbf{A}]_{pq}$. Zusammen ist das $\mathbf{X}(\mathbf{A} + \mathbf{A}^\top)$. Und (10.4.12) ist der Spezialfall $\mathbf{A} = \mathbf{I}$ von (10.4.10), denn $\text{tr}(\mathbf{X}\mathbf{I}\mathbf{X}^\top) = \text{tr}(\mathbf{X}\mathbf{X}^\top) = \|\mathbf{X}\|_F^2$ und $\mathbf{X}(\mathbf{I} + \mathbf{I}) = 2\mathbf{X}$. Die Verwandtschaft zu $\partial x^2/\partial x = 2x$ ist kein Zufall.

Beispiel 10.4.11 (Probe der Frobenius-Identität)

Für (10.4.12) brauchen wir keine Matrixrechnung. Nach Definition der Frobenius-Norm ist

$$f(\mathbf{X}) = \|\mathbf{X}\|_F^2 = \sum_{i=1}^m \sum_{j=1}^n x_{ij}^2,$$

und in dieser Summe kommt x_{ij} genau einmal vor, nämlich quadriert. Also

$$\frac{\partial f(\mathbf{X})}{\partial x_{ij}} = 2x_{ij} \implies \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} = 2\mathbf{X}.$$

Anders als in Beispiel 10.4.9 ist der Restterm hier nicht null. Ausmultiplizieren liefert

$$f(\mathbf{X} + \mathbf{H}) = \sum_{i,j} (x_{ij} + h_{ij})^2 = f(\mathbf{X}) + 2\text{tr}(\mathbf{X}^\top \mathbf{H}) + \|\mathbf{H}\|_F^2,$$

und der mittlere Term ist wegen $\text{tr}((2\mathbf{X})^\top \mathbf{H}) = 2\text{tr}(\mathbf{X}^\top \mathbf{H})$ genau $D_{\mathbf{X}} f(\mathbf{H})$. Übrig bleibt $r(\mathbf{H}) = \|\mathbf{H}\|_F^2$, und dieser Ausdruck verschwindet im Verhältnis zu $\|\mathbf{H}\|_F$ wie verlangt.

Was sagt ein einzelner Eintrag der Gradientenmatrix konkret aus, etwa der an der Stelle $(2, 3)$? Die Antwort steckt in Bemerkung 10.4.8: Lenken wir nur in Richtung $\mathbf{E}_{ij}$ aus, so schrumpft die Summe über alle Einträge auf einen einzigen Summanden zusammen. Das lässt sich anstupsen.

Interaktiv: Auslenkung in Richtung einer einzelnen Koordinate

Wir fassen eine Zelle von $\mathbf{X}$ an und ziehen sie um h nach oben oder unten. Das Widget wertet f an der so veränderten Matrix aus und stellt die tatsächliche Änderung neben $h \cdot \partial f(\mathbf{X})/\partial x_{ij}$; die Zeile darunter trennt sie in den grünen Ableitungsterm und den roten Rest.

Interaktives Element — nur in der Web-Fassung

Der Eintrag (i, j) der Gradientenmatrix ist genau die Rate, mit der f auf einen Stups an dieser Stelle reagiert. Für die beiden in $\mathbf{X}$ linearen Funktionen bleibt der Rest exakt null, die lineare Vorhersage ist dort also nicht nur eine Näherung. Bei $\|\mathbf{X}\|_F^2$ ist der Rest exakt h^2 : sichtbar

von null verschieden, aber schneller fallend als h selbst, und damit ein Musterbeispiel für ein $\mathcal{O}(h)$.

Vertiefung: Anwendung: Matrixzerlegung und Matrix Completion

Zum Schluss ein Problem, in dem der Fall Matrix zu Skalar seine ganze Kraft zeigt. Gegeben sei eine Matrix $\mathbf{Y} \in \mathbb{R}^{m \times n}$, von der wir nur die Einträge an den Positionen einer Indexmenge Ω kennen. Das klassische Beispiel sind Bewertungen von Filmen: Die Zeilen sind Nutzer, die Spalten Filme, und die allermeisten Felder der Tabelle sind leer, weil niemand alle Filme sieht.

Die Hoffnung ist, dass die Tabelle in Wahrheit einfach gebaut ist. Wenige Geschmacksrichtungen erklären die meisten Bewertungen, die Matrix ist also näherungsweise von kleinem Rang. Wir suchen deshalb eine Rang- k -Approximation

$$\mathbf{Y} \approx \mathbf{U}\mathbf{V}^T, \quad \mathbf{U} \in \mathbb{R}^{m \times k}, \quad \mathbf{V} \in \mathbb{R}^{n \times k},$$

mit $k \ll \min(m, n)$. Statt $m \cdot n$ Zahlen sind dann nur noch $k(m+n)$ zu bestimmen, und die Lücken lesen wir am Produkt $\mathbf{U}\mathbf{V}^T$ einfach ab.

Niedriger Rang allein garantiert allerdings noch keine eindeutige Ergänzung. Auch Zahl und Lage der beobachteten Einträge müssen genügend Information über Zeilen- und Spaltenstruktur liefern; in Anwendungen kommen deshalb Annahmen an das Beobachtungsmuster und häufig Regularisierung hinzu. Das folgende Verfahren liefert Vorhersagen unter dem gewählten Rangmodell, nicht automatisch die einzig möglichen fehlenden Werte.

Bei vollständig beobachtetem $\mathbf{Y}$ wäre das Problem gelöst: Nach dem Satz von Eckart und Young (Abschnitt 6.4) liefert die abgeschnittene Singulärwertzerlegung die beste Rang- k -Approximation. Fehlende Einträge machen diese Tür zu, denn die Zerlegung braucht die ganze Matrix. Also formulieren wir eine Verlustfunktion, die nur die beobachteten Felder bewertet, und minimieren sie numerisch:

$$L(\mathbf{U}, \mathbf{V}) = \frac{1}{2} \sum_{(i,j) \in \Omega} \left(y_{ij} - [\mathbf{U}\mathbf{V}^T]_{ij} \right)^2 = \frac{1}{2} \|\mathbf{P}_\Omega \odot (\mathbf{Y} - \mathbf{U}\mathbf{V}^T)\|_F^2, \quad \mathbf{P}_\Omega = [\mathbb{1}_{(i,j) \in \Omega}]. \quad (10.4.13)$$

Dabei ist $\mathbf{P}_\Omega \in \mathbb{R}^{m \times n}$ die Maskenmatrix mit einer 1 an jeder beobachteten und einer 0 an jeder fehlenden Position, und $\odot$ bezeichnet das *elementweise Produkt* (Hadamard-Produkt), also $(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij}b_{ij}$. Die Maske blendet damit alle unbeobachteten Abweichungen aus, bevor die Frobenius-Norm sie aufsummieren kann. Ein gewöhnliches Matrixprodukt kann an dieser Stelle nicht gemeint sein: $\mathbf{P}_\Omega$ und $\mathbf{Y} - \mathbf{U}\mathbf{V}^T$ sind beide $m \times n$, und nur elementweise blendet die Maske die unbeobachteten Positionen aus.

Für die Optimierung brauchen wir die Ableitungen von L nach den beiden Faktormatrizen, und das ist genau der Fall Matrix zu Skalar.

Satz 10.4.12 (Gradienten der Completion-Verlustfunktion)

Mit $\mathbf{R} := \mathbf{P}_\Omega \odot (\mathbf{Y} - \mathbf{U}\mathbf{V}^T)$ gilt für L aus (10.4.13)

$$\frac{\partial L}{\partial \mathbf{U}} = -\mathbf{R}\mathbf{V} \in \mathbb{R}^{m \times k}, \quad \frac{\partial L}{\partial \mathbf{V}} = -\mathbf{R}^T\mathbf{U} \in \mathbb{R}^{n \times k}. \quad (10.4.14)$$

Beweis.

- (1) Nach (10.4.6) ist die Ableitung eintragsweise erklärt, also rechnen wir eintragsweise. Mit $r_{ij} = p_{ij}(y_{ij} - \sum_{s=1}^k u_{is}v_{js})$ und $p_{ij} \in \{0, 1\}$ ist

$$L(\mathbf{U}, \mathbf{V}) = \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^n r_{ij}^2.$$

die Maske p_{ij} setzt alle unbeobachteten Summanden auf null, die Summe über Ω wird dadurch zur Summe über alle Positionen

- (2) Nun leiten wir nach einem einzelnen Eintrag u_{ab} ab. Der Eintrag u_{ab} taucht nur in den Residuen der Zeile a auf, und dort mit $\partial r_{aj}/\partial u_{ab} = -p_{aj}v_{jb}$. Die Kettenregel liefert

$$\frac{\partial L}{\partial u_{ab}} = \sum_{j=1}^n r_{aj} \frac{\partial r_{aj}}{\partial u_{ab}} = - \sum_{j=1}^n p_{aj} r_{aj} v_{jb} = - \sum_{j=1}^n r_{aj} v_{jb} = -[\mathbf{RV}]_{ab}.$$

im vorletzten Schritt fällt die Maske weg, weil r_{aj} den Faktor p_{aj} bereits enthält und $p_{aj}^2 = p_{aj}$ für $p_{aj} \in \{0, 1\}$ gilt

- (3) Für $\mathbf{V}$ läuft die Rechnung genauso, nur über die Spalten. Der Eintrag v_{ab} steckt in den Residuen der Spalte a , mit $\partial r_{ia}/\partial v_{ab} = -p_{ia}u_{ib}$, also

$$\frac{\partial L}{\partial v_{ab}} = \sum_{i=1}^m r_{ia} (-p_{ia}u_{ib}) = - \sum_{i=1}^m r_{ia} u_{ib} = -[\mathbf{R}^T \mathbf{U}]_{ab}.$$

$\sum_i r_{ia} u_{ib}$ ist das Skalarprodukt der a -ten Spalte von $\mathbf{R}$ mit der b -ten Spalte von $\mathbf{U}$, also der Eintrag (a, b) von $\mathbf{R}^T \mathbf{U}$

□

Damit steht dem Gradientenabstieg aus Abschnitt 10.2 nichts mehr im Weg. Wir behandeln $\mathbf{U}$ und $\mathbf{V}$ als ein gemeinsames Parameterpaket und gehen in beiden zugleich einen Schritt gegen den Gradienten.

Algorithmus 10.4.13 (Gradientenabstieg für Matrix Completion)

Eingabe: beobachtete Werte $\mathbf{Y}$ auf Ω , Maske $\mathbf{P}_\Omega$, Rang k , Lernrate $\alpha > 0$, Startwerte $\mathbf{U}^{(0)}, \mathbf{V}^{(0)}$.

Für $t = 0, 1, 2, \dots$:

1. Residuum auf den beobachteten Feldern: $\mathbf{R}^{(t)} = \mathbf{P}_\Omega \odot (\mathbf{Y} - \mathbf{U}^{(t)}\mathbf{V}^{(t)\top})$.
2. Faktoren aktualisieren:

$$\mathbf{U}^{(t+1)} = \mathbf{U}^{(t)} + \alpha \mathbf{R}^{(t)} \mathbf{V}^{(t)}, \quad \mathbf{V}^{(t+1)} = \mathbf{V}^{(t)} + \alpha \mathbf{R}^{(t)\top} \mathbf{U}^{(t)}.$$

3. Abbruch, sobald sich L kaum noch ändert oder eine Schrittzahl erreicht ist.

Ausgabe: $\hat{\mathbf{Y}} = \mathbf{UV}^\top$; an den Positionen außerhalb von Ω stehen die vom Rangmodell und vom Optimierungslauf gelieferten Vorhersagen.

Das Pluszeichen in Schritt 2 ist kein Tippfehler: Wir gehen gegen den Gradienten, und der trägt nach (10.4.14) bereits ein Minus. Zwei Warnungen gehören dazu. Erstens ist L in $(\mathbf{U}, \mathbf{V})$ nicht konvex, das Produkt $\mathbf{UV}^\top$ ist ja quadratisch in den Unbekannten. Gradientenabstieg hat deshalb keine allgemeine Garantie für ein globales Minimum: Je nach Startpunkt und Lernrate kann er zu einem lokalen oder globalen Minimum beziehungsweise einem anderen stationären Punkt konvergieren, oszillieren oder divergieren. Zweitens ist die Zerlegung nie eindeutig, denn für jede invertierbare $\mathbf{T} \in \mathbb{R}^{k \times k}$ ist $(\mathbf{UT})(\mathbf{VT}^{-\top})^\top = \mathbf{UV}^\top$. Für die Vorhersage stört das nicht, wohl aber für jede Interpretation der Spalten von $\mathbf{U}$ und $\mathbf{V}$.

Anwendungen dieses Musters gibt es reichlich: Imputation fehlender Werte, Empfehlungssysteme und kollaboratives Filtern arbeiten alle nach demselben Rezept.

Damit stellt sich die Frage, die über den praktischen Wert des Verfahrens entscheidet: Wenn zwei Modelle den Verlust auf den beobachteten Einträgen beide auf null drücken, sagen sie dann auch dasselbe für die Lücken voraus?

Interaktiv: Matrix Completion an einer 2×3-Matrix

Vier von sechs Einträgen sind beobachtet, zwei sind Lücken. Das Widget führt Algorithmus 10.4.13 für $k = 1$ und $k = 2$ nebeneinander aus, mit derselben Lernrate, und zeigt alle Zwischengrößen: das Residuum $\mathbf{P}_Q \odot (\mathbf{Y} - \mathbf{UV}^T)$ in Rot, die beiden Gradienten in Orange und die Vorhersagen für die Lücken in Grün.

Interaktives Element — nur in der Web-Fassung

Der Verlust fällt bei beiden Rängen auf null, die Vorhersagen laufen aber auseinander: $k = 1$ landet bei $y_{13} = 2,5$ und $y_{22} = 2,4$, $k = 2$ bei 1,583 und 0,909. Bei $k = 2$ treffen zehn Parameter auf vier Beobachtungen; wohin die Vorhersage läuft, entscheidet dann der Startpunkt. Größerer Rang heißt also nicht bessere Rekonstruktion. Und wer die Lernrate hochschiebt, sieht ab etwa $\alpha = 0,14$ ein Pendeln des Verlusts und jenseits von etwa $\alpha = 0,25$ ein Davonlaufen der Einträge.

Selbsttest

1. Für $\mathbf{F} : \mathbb{R} \rightarrow \mathbb{R}^{m \times n}$ hat die Ableitung $\partial \mathbf{F}(x) / \partial x$ dasselbe Format $m \times n$ wie $\mathbf{F}(x)$ selbst.

Antwort: Wahr

Das ist Definition 10.4.1: Jeder Eintrag wird für sich abgeleitet und bleibt an seinem Platz. Verwechseln darf man die beiden Matrizen trotzdem nicht, die eine enthält Werte, die andere Steigungen (Bemerkung 10.4.2).

2. Die Identität $\partial \text{tr}(\mathbf{F}(x)) / \partial x = \text{tr}(\partial \mathbf{F}(x) / \partial x)$ gilt für jede Matrixfunktion $\mathbf{F} : \mathbb{R} \rightarrow \mathbb{R}^{m \times n}$.

Antwort: Falsch

Die Spur ist nur für quadratische Matrizen erklärt, die Aussage braucht also $m = n$. Deshalb setzt Satz 10.4.4 $\mathbf{F} : \mathbb{R} \rightarrow \mathbb{R}^{n \times n}$ voraus, während Definition 10.4.1 noch mit beliebigen Formaten auskommt.

3. Ist $\det(\mathbf{F}(x)) = 1$ für alle x , so gilt $\text{tr}[\mathbf{F}(x)^{-1} \partial \mathbf{F}(x) / \partial x] = 0$.

Antwort: Wahr

Die linke Seite der Jacobi-Formel (10.4.4) ist die Ableitung einer konstanten Funktion, also null. Rechts steht das Produkt aus $\det(\mathbf{F}(x)) = 1$ und der Spur, und ein Produkt mit Faktor 1 verschwindet nur, wenn der andere Faktor verschwindet. Die Drehmatrix im ersten Widget führt das vor.

4. Für $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$ ist der Ableitungsterm das Matrixprodukt $D_{\mathbf{X}} f(\mathbf{H}) = (\partial f(\mathbf{X}) / \partial \mathbf{X}) \mathbf{H}$.

Antwort: Falsch

Richtig ist $D_{\mathbf{X}} f(\mathbf{H}) = \text{tr}((\partial f(\mathbf{X}) / \partial \mathbf{X})^T \mathbf{H})$ nach (10.4.7). Das angegebene Produkt wäre für $m \neq n$ nicht einmal definiert, denn beide Faktoren haben das Format $m \times n$; und das Ergebnis müsste eine Zahl sein, keine Matrix.

5. Für $f(\mathbf{X}) = \text{tr}(\mathbf{A}^T \mathbf{X})$ ist $\partial f(\mathbf{X}) / \partial \mathbf{X} = \mathbf{A}$, und der Restterm in (10.4.7) ist exakt null.

Antwort: Wahr

Die Formel steht als (10.4.11) in Satz 10.4.10. Nach (10.4.8) ist f die Summe $\sum_{ij} a_{ij} x_{ij}$ und damit linear in $\mathbf{X}$, also ist die lineare Näherung exakt. Das zweite Widget zeigt denselben Effekt für die Funktion $\mathbf{a}^T \mathbf{X} \mathbf{b}$ aus (10.4.9).

6. Bei der Matrix Completion liefert ein größeres k stets eine bessere Rekonstruktion der unbeobachteten Einträge.

Antwort: Falsch

Ein größeres k senkt den Verlust auf den beobachteten Feldern, über die Lücken sagt es dagegen nichts. Im Widget passt $k = 2$ die vier Beobachtungen mit zehn Parametern exakt an, und die beiden Vorhersagen hängen dann nur noch am Startpunkt. Genau deshalb steht am Anfang die Annahme, dass die gesuchte Matrix kleinen Rang hat.

7. Im Completion-Widget: Wir lassen beide Modelle mit $\alpha = 0,05$ so lange laufen, bis der Verlust bei beiden praktisch null ist. Welchen Wert sagt das Rang-2-Modell für die Lücke y_{13} voraus?

Lösung: 1,583

1,583, während das Rang-1-Modell 2,5 vorhersagt. Beide erreichen denselben Verlust null auf den vier beobachteten Einträgen; über die Lücken sagt der Verlust nichts.

8. Im Anstups-Widget: Wir wählen $f(\mathbf{X}) = \|\mathbf{X}\|_F^2$ und stupsen einen Eintrag um $h = 0,4$ an. Wie groß ist der rote Restterm?

Lösung: 0,16

Exakt $h^2 = 0,16$, unabhängig davon, welchen Eintrag wir anfassen: Der quadratische Term in $(x_{ij} + h)^2 = x_{ij}^2 + 2x_{ij}h + h^2$ ist der ganze Rest.

Vertiefung: MML §5.4 (Ableitungen von Matrizen) und §5.5 (nützliche Identitäten für Gradienten).

10.5 Stetigkeit und Linearität

Woran wir anknüpfen

Damit ist der Begriffsteil des Kapitels abgeschlossen. Die Abschnitte 10.1 bis 10.4 haben geklärt, was eine Ableitung überhaupt ist, sobald Ein- und Ausgabe keine Zahlen mehr sind, und in welchem Format sie dann steht. Ab hier geht es um das Rechnen damit: Welche Eigenschaften erbt eine differenzierbare Funktion, und wie leiten wir zusammengesetzte Ausdrücke ab, ohne jedes Mal zur Definition zurückzukehren.

Der Begriff, an dem alles hängt, ist die *Fréchet-Ableitung*. Für normierte Vektorräume $\mathbb{D}$ und $\mathbb{E}$, eine Funktion $f : \mathbb{D} \rightarrow \mathbb{E}$ und eine Stelle $\mathbf{x} \in \mathbb{D}$ heißt f dort differenzierbar, wenn es eine beschränkte lineare Abbildung $D_{\mathbf{x}}f : \mathbb{D} \rightarrow \mathbb{E}$ und einen Rest $r(\mathbf{h}) \in \mathbb{E}$ gibt mit

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + r(\mathbf{h}), \quad \frac{\|r(\mathbf{h})\|_{\mathbb{E}}}{\|\mathbf{h}\|_{\mathbb{D}}} \rightarrow 0 \quad \text{für } \mathbf{h} \rightarrow \mathbf{0}. \quad (10.5.1)$$

Das ist Definition 10.1.5 aus Abschnitt 10.1, und die Notation von dort übernehmen wir mit: $\mathbb{D}$ und $\mathbb{E}$ sind die beiden normierten Vektorräume, mit dem Erwartungswert hat das $\mathbb{E}$ nichts zu tun. Gleichung (10.5.1) ist der Ausgangspunkt für jeden Beweis in diesem Abschnitt.

Die drei konkreten Gestalten, die wir laufend brauchen, stehen schon da: der Gradient für $f : \mathbb{R}^n \rightarrow \mathbb{R}$ (Abschnitt 10.2), die Jacobimatrix für $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ (Abschnitt 10.3) und die beiden Matrixableitungen $\partial F(\mathbf{x})/\partial \mathbf{x}$ und $\partial f(\mathbf{X})/\partial \mathbf{X}$ (Abschnitt 10.4).

Aus der Analysis setzen wir Produkt- und Kettenregel für Funktionen einer Variablen voraus, dazu höhere Ableitungen und die Taylorreihe; aus der linearen Algebra bilineare und multilineare

Abbildungen samt Skalarprodukt $\langle \cdot, \cdot \rangle$ (Abschnitt 9.1), die quadratischen Formen $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ und die Komposition von Funktionen.

Der Fahrplan für die zweite Kapitelhälfte: Dieser Abschnitt klärt zwei allgemeine Eigenschaften. Jede differenzierbare Funktion ist stetig, und die Ableitungsoperation selbst ist linear. Abschnitt 10.6 bringt Produkt- und Kettenregel in der allgemeinen Fassung, Abschnitt 10.7 die Ableitungen höheren Grades und Abschnitt 10.8 die Taylorapproximation.

Bemerkung 10.5.1 (Groß-O und klein-o für kleine Argumente)

In Abschnitt 10.1 haben wir $g(\mathbf{h}) = o(\|\mathbf{h}\|)$ so gelesen: Der Quotient $\|g(\mathbf{h})\| / \|\mathbf{h}\|$ geht gegen null, der Term verschwindet also schneller als $\|\mathbf{h}\|$ selbst. Der nächste Beweis braucht zusätzlich das große O , ebenfalls für kleine statt für große Argumente. Wir schreiben $g(\mathbf{h}) = O(\|\mathbf{h}\|)$, wenn es Konstanten $C < \infty$ und $\delta > 0$ gibt mit

$$\|g(\mathbf{h})\| \leq C \|\mathbf{h}\| \quad \text{für alle } \mathbf{h} \text{ mit } \|\mathbf{h}\| < \delta.$$

Ein O -Term darf also von derselben Größenordnung sein wie $\|\mathbf{h}\|$, er muss nur mitschrumpfen. Jeder o -Term ist damit automatisch ein O -Term: Strebt der Quotient gegen null, so liegt er für hinreichend kleine $\mathbf{h}$ unter 1, und das ist die Schranke mit $C = 1$. Umgekehrt geht es nicht, denn $g(h) = h$ ist $O(|h|)$, aber nicht $o(|h|)$. Die Landau-Symbole kennen wir aus Abschnitt 2.4 in ihrer üblichen Rolle als Aufwandsmaß; dort läuft das Argument gegen unendlich, hier gegen null, die Rechenregeln sind dieselben.

Differenzierbarkeit erzwingt Stetigkeit

Die lineare Approximation (10.5.1) sagt mehr, als sie auf den ersten Blick zugibt. Sie zwingt die Funktion nämlich, sich in der Nähe von $\mathbf{x}$ zahm zu verhalten: Wer sich in erster Ordnung linear approximieren lässt, kann an dieser Stelle nicht springen.

Satz 10.5.2 (Stetigkeit aus Differenzierbarkeit)

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Vektorräume und $f : \mathbb{D} \rightarrow \mathbb{E}$ in $\mathbf{x} \in \mathbb{D}$ differenzierbar. Dann ist f in $\mathbf{x}$ stetig.

Vertiefung: Warum Differenzierbarkeit Stetigkeit erzwingt

Beweis.

- (1) Die Ableitung $D_{\mathbf{x}}f$ ist nach Definition eine beschränkte lineare Abbildung. Es gibt also ein $M < \infty$ mit

$$\|D_{\mathbf{x}}f(\mathbf{h})\| \leq M \|\mathbf{h}\| \quad \text{für alle } \mathbf{h},$$

und damit ist $D_{\mathbf{x}}f(\mathbf{h}) = O(\|\mathbf{h}\|)$.

Definition 10.1.5 verlangt von $D_{\mathbf{x}}f$ nicht nur Linearität, sondern auch Beschränktheit; das M darf von der Stelle $\mathbf{x}$ abhängen, nicht aber von $\mathbf{h}$

- (2) Der Restterm $r(\mathbf{h}) := f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x}) - D_{\mathbf{x}}f(\mathbf{h})$ ist nach (10.5.1) von der Ordnung $o(\|\mathbf{h}\|)$. Nach Bemerkung 10.5.1 gibt es deshalb ein $\delta > 0$ mit

$$\|r(\mathbf{h})\| \leq \|\mathbf{h}\| \quad \text{für alle } \mathbf{h} \text{ mit } \|\mathbf{h}\| < \delta.$$

der Quotient $\|r(\mathbf{h})\| / \|\mathbf{h}\|$ strebt gegen null, liegt also ab einem δ unter 1

- (3) Beide Schranken zusammen liefern für alle $\mathbf{h}$ mit $\|\mathbf{h}\| < \delta$

$$\|f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})\| = \|D_{\mathbf{x}}f(\mathbf{h}) + r(\mathbf{h})\| \leq M \|\mathbf{h}\| + \|\mathbf{h}\| = (M + 1) \|\mathbf{h}\|.$$

erst die Definition von $r(\mathbf{h})$ umgestellt, dann die Dreiecksungleichung, dann die Schranken aus Schritt 1 und 2

- (4) Die rechte Seite geht für $\|\mathbf{h}\| \rightarrow 0$ gegen null, also auch die linke. Das heißt $f(\mathbf{x} + \mathbf{h}) \rightarrow f(\mathbf{x})$, und das ist die Stetigkeit von f in $\mathbf{x}$.

Stetigkeit in $\mathbf{x}$ bedeutet gerade, dass die Funktionswerte in der Nähe von $\mathbf{x}$ gegen $f(\mathbf{x})$ streben

□

Üblicherweise fasst man die beiden Zusatzterme zu einem einzigen zusammen und schreibt kurz $f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + O(\|\mathbf{h}\|)$. Schritt 3 ist genau diese Aussage, nur mit der Konstanten $M + 1$ ausgeschrieben. Nebenbei sehen wir, welche Zutat wirklich gebraucht wird: die Beschränktheit von $D_{\mathbf{x}}f$. In endlich-dimensionalen Räumen ist jede lineare Abbildung automatisch beschränkt (Bemerkung 10.1.6), dort kostet Schritt 1 also nichts. Im Unendlichdimensionalen ist die Forderung eine echte Zusatzbedingung, und ohne sie bricht der Beweis zusammen.

Stetig, aber nicht differenzierbar

Satz 10.5.2 hat eine Richtung. Die andere ist falsch, und das Gegenbeispiel ist so einfach, dass es kaum ein besseres gibt.

Beispiel 10.5.3 (Die Betragsfunktion am Nullpunkt)

Sei $f : \mathbb{R} \rightarrow \mathbb{R}$ mit $f(x) = |x|$, betrachte an der Stelle $x = 0$.

Stetig ist f dort. Für $h \rightarrow 0$ gilt $f(0 + h) = |h| \rightarrow 0 = f(0)$. $\square$

Differenzierbar ist f dort nicht. Diesen Knick haben wir in Abschnitt 10.1 schon vermessen: Der Differenzenquotient ist für jedes $h > 0$ exakt $+1$ und für jedes $h < 0$ exakt -1 (Bemerkung 10.1.2). Beide einseitigen Grenzwerte existieren also, sie sind bloß verschieden, während Definition 10.1.1 einen einzigen Wert für beide Seiten verlangt. Damit existiert $f'(0)$ nicht, und an den Knick lässt sich keine eindeutige Tangente anlegen.

Bemerkung 10.5.4 (Die Merkregel und ihre Kontraposition)

Satz 10.5.2 und Beispiel 10.5.3 zusammen ergeben

$$\text{differenzierbar in } \mathbf{x} \implies \text{stetig in } \mathbf{x}, \quad \text{stetig in } \mathbf{x} \not\implies \text{differenzierbar in } \mathbf{x}.$$

Stetigkeit ist für Differenzierbarkeit also notwendig, aber nicht hinreichend. In der Praxis benutzen wir meist die Kontraposition der ersten Zeile: Wo eine Funktion springt, brauchen wir gar nicht erst nach einer Ableitung zu suchen. Umgekehrt darf uns ein glatt aussehender Graph nicht beruhigen, denn Knicke sind mit Stetigkeit verträglich. Beides begegnet uns wieder, sobald wir in Abschnitt 10.7 Ableitungen höheren Grades verlangen: Jede Ableitungsstufe kostet eine Stufe Glattheit.

Die beiden Aussagen als Bild, alles an der Stelle $x_0 = 0$. Der grüne Bereich liegt vollständig im blauen; das ist Satz 10.5.2. Im Ring dazwischen wohnt die Betragsfunktion aus Beispiel 10.5.3, und dass dieser Ring nicht leer ist, verbietet die Umkehrung. Ganz außen steht die Sprungfunktion H , die in 0 nicht einmal stetig ist und deshalb auch nicht differenzierbar sein kann. Die Farben folgen dem Kapitel-Code: Blau gehört zur Funktion und ihren Werten, und Stetigkeit ist eine Aussage über Funktionswerte; Grün gehört zum Ableitungsterm, und Differenzierbarkeit ist die Existenz genau dieses Terms.

Ein zweiter Blick auf denselben Sachverhalt lohnt sich, und er ist der anschaulichere. Differenzierbarkeit in x heißt nach Definition 10.1.1, dass sich f dort bis auf einen Rest der Ordnung $o(\|h\|)$ durch eine lineare Abbildung ersetzen lässt. Wenn das stimmt, muss der Graph unter einer starken Lupe aussehen wie eine Gerade. Was passiert also, wenn wir in eine Stelle hineinzoomen: Bei welcher der drei Funktionen wird die Kurve irgendwann gerade, und bei welcher nicht?

Interaktiv: Unter der Lupe: lokal linear oder nicht

Wir legen ein quadratisches Fenster um die Stelle x_0 und ziehen es zusammen. Beide Achsen werden dabei gleich gestaucht, das Bild zeigt also wirklich einen Zoom und keine gestreckte Fassung. Gemessen wird, wie weit die Kurve im Fenster von der Geraden abweicht, die durch die beiden Fensterränder läuft.

Interaktives Element — nur in der Web-Fassung

Die drei Funktionen verhalten sich an der Stelle 0 grundverschieden. Bei $f(x) = x^2$ halbiert sich die Abweichung von der Geraden mit jeder Halbierung des Fensters; genau das besagt der Rest $o(\|h\|)$ in Definition 10.1.1. Bei $f(x) = |x|$ steht sie auf jeder Zoomstufe bei genau einem w , und bei $f(x) = \sqrt{|x|}$ wächst sie beim Hineinzoomen sogar wie $1/\sqrt{w}$. Nichtdifferenzierbarkeit hat also mehr als eine Bauform, und keine davon verträgt sich mit einer Geraden. Verschieben wir x_0 von der 0 weg, so wird auch der Betrag lokal linear – die Ausnahme steckt in der Stelle, nicht in der Funktion.

Die Ableitung ist linear

Die zweite allgemeine Eigenschaft betrifft nicht eine einzelne Funktion, sondern die Zuordnung $f \mapsto D_x f$ selbst. Sie ist verträglich mit Summen und mit Vielfachen, und das ist der Grund, warum wir Summen von Bausteinen Summand für Summand ableiten dürfen.

Satz 10.5.5 (Linearität der Ableitungsoperation)

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Vektorräume und $f, g : \mathbb{D} \rightarrow \mathbb{E}$ in $x \in \mathbb{D}$ differenzierbar. Dann ist für alle $c_1, c_2 \in \mathbb{R}$ auch $c_1 f + c_2 g$ in x differenzierbar, und es gilt

$$D_x(c_1 f + c_2 g) = c_1 D_x f + c_2 D_x g. \quad (10.5.2)$$

Vertiefung: Warum die Ableitungsoperation linear ist

Beweis.

- (1) Wir entwickeln beide Funktionen einzeln nach (10.5.1),

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + r_f(\mathbf{h}),$$

$$g(\mathbf{x} + \mathbf{h}) = g(\mathbf{x}) + D_{\mathbf{x}}g(\mathbf{h}) + r_g(\mathbf{h}),$$

mit $r_f(\mathbf{h}) = o(\|\mathbf{h}\|)$ und $r_g(\mathbf{h}) = o(\|\mathbf{h}\|)$.

Definition 10.1.5, einmal für f und einmal für g ; die beiden Restterme bekommen Namen, damit wir sie in Schritt 3 einzeln abschätzen können

- (2) Jetzt bilden wir die Linearkombination und sortieren die Terme nach Bauart:

$$\begin{aligned} (c_1f + c_2g)(\mathbf{x} + \mathbf{h}) &= c_1 [f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + r_f(\mathbf{h})] + c_2 [g(\mathbf{x}) + D_{\mathbf{x}}g(\mathbf{h}) + r_g(\mathbf{h})] \\ &= (c_1f + c_2g)(\mathbf{x}) + (c_1D_{\mathbf{x}}f + c_2D_{\mathbf{x}}g)(\mathbf{h}) + (c_1r_f(\mathbf{h}) + c_2r_g(\mathbf{h})). \end{aligned}$$

nichts als Umsortieren: die Funktionswerte nach vorn, die in $\mathbf{h}$ linearen Terme in die Mitte, die Restterme nach hinten

- (3) Die Zerlegung hat die Gestalt von (10.5.1), sobald zwei Dinge stimmen. Erstens ist der mittlere Term eine beschränkte lineare Abbildung in $\mathbf{h}$: Linearkombinationen linearer Abbildungen sind linear, und mit den Schranken M_f und M_g von $D_{\mathbf{x}}f$ und $D_{\mathbf{x}}g$ gilt

$$\|(c_1D_{\mathbf{x}}f + c_2D_{\mathbf{x}}g)(\mathbf{h})\| \leq (|c_1|M_f + |c_2|M_g)\|\mathbf{h}\|.$$

Zweitens ist der hintere Term wieder $o(\|\mathbf{h}\|)$, denn

$$\frac{\|c_1r_f(\mathbf{h}) + c_2r_g(\mathbf{h})\|}{\|\mathbf{h}\|} \leq |c_1| \frac{\|r_f(\mathbf{h})\|}{\|\mathbf{h}\|} + |c_2| \frac{\|r_g(\mathbf{h})\|}{\|\mathbf{h}\|} \xrightarrow{\|\mathbf{h}\| \rightarrow 0} 0.$$

Dreiecksungleichung und Homogenität der Norm; eine endliche Linearkombination von $o(\|\mathbf{h}\|)$ -Termen ist wieder $o(\|\mathbf{h}\|)$

- (4) Damit erfüllt $c_1D_{\mathbf{x}}f + c_2D_{\mathbf{x}}g$ die Definition der Fréchet-Ableitung für $c_1f + c_2g$: Die Funktion ist in $\mathbf{x}$ differenzierbar, und die angegebene Abbildung leistet das Verlangte. Eine zweite kann es nicht geben. Erfüllen nämlich A und B beide (10.5.1), so ist $(A - B)(\mathbf{h}) = o(\|\mathbf{h}\|)$; setzen wir für festes $\mathbf{u} \neq \mathbf{0}$ nun $\mathbf{h} = t\mathbf{u}$ mit $t \rightarrow 0$ ein, so kürzt sich der Faktor $|t|$ heraus,

$$\frac{\|(A - B)(t\mathbf{u})\|}{\|t\mathbf{u}\|} = \frac{|t| \|(A - B)(\mathbf{u})\|}{|t| \|\mathbf{u}\|} = \frac{\|(A - B)(\mathbf{u})\|}{\|\mathbf{u}\|}.$$

Der Quotient hängt also gar nicht von t ab, muss aber gegen null streben und ist damit selbst null. Also stimmen A und B überein, und (10.5.2) gilt mit Gleichheitszeichen.

Eindeutigkeit der Fréchet-Ableitung; für Jacobimatrizen steht dasselbe Argument als Lemma 10.3.3 in Abschnitt 10.3

□

Der Kern dieses Beweises steckt in Schritt 3: Beide Zutaten der Definition, Linearität mit Beschränktheit und die Kleinheit des Restes, überstehen das Bilden von Linearkombinationen unbeschadet.

Was die Linearität in den einzelnen Formaten bedeutet

Bemerkung 10.5.6 (Fünf Spezialfälle derselben Aussage)

Gleichung (10.5.2) ist eine Aussage über lineare Abbildungen. In den Spezialfällen aus den Abschnitten 10.2 bis 10.4 steckt diese Abbildung jeweils in einem konkreten Objekt, und die Linearität überträgt sich wörtlich auf dieses Objekt. Es sind fünf Fälle, und wir gehen sie in derselben Reihenfolge durch wie der Fahrplan in Abschnitt 10.2. Überall sind $c_1, c_2 \in \mathbb{R}$ beliebig, und die beteiligten Funktionen sind an der jeweiligen Stelle differenzierbar.

1. $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}$: die gewöhnliche Ableitung (Abschnitt 10.1).

$$(c_1 f + c_2 g)' = c_1 f' + c_2 g'$$

2. $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}$: der Gradient (Abschnitt 10.2).

$$\nabla [c_1 f(\mathbf{x}) + c_2 g(\mathbf{x})] = c_1 \nabla f(\mathbf{x}) + c_2 \nabla g(\mathbf{x})$$

3. $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}^m$: die Jacobimatrix (Abschnitt 10.3).

$$\mathbf{J}_{c_1 f + c_2 g}(\mathbf{x}) = c_1 \mathbf{J}_f(\mathbf{x}) + c_2 \mathbf{J}_g(\mathbf{x})$$

4. $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}^{m \times n}$: eine matrixwertige Funktion eines Skalars (Definition 10.4.1 in Abschnitt 10.4).

$$\frac{\partial [c_1 F(x) + c_2 G(x)]}{\partial x} = c_1 \frac{\partial F(x)}{\partial x} + c_2 \frac{\partial G(x)}{\partial x}$$

5. $\mathbb{D} = \mathbb{R}^{m \times n}, \mathbb{E} = \mathbb{R}$: die Ableitung nach einer Matrix (Definition 10.4.7 in Abschnitt 10.4).

$$\frac{\partial [c_1 f(\mathbf{X}) + c_2 g(\mathbf{X})]}{\partial \mathbf{X}} = c_1 \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} + c_2 \frac{\partial g(\mathbf{X})}{\partial \mathbf{X}}$$

Alle fünf Zeilen sagen dasselbe: Die Ableitungsobjekte werden eintragsweise addiert und mit Skalaren multipliziert. Überraschend ist das nicht, denn jeder ihrer Einträge ist eine gewöhnliche oder eine partielle Ableitung, und für die kennen wir die Linearität schon aus der Analysis. Der Gewinn liegt woanders: Satz 10.5.5 gilt in jedem normierten Raum, also auch dort, wo wir gar keine Einträge zum Nachrechnen haben. Die drei leeren Zellen der Tabelle aus Abschnitt 10.2 bleiben auch hier unbesetzt; sie bräuchten Tensoren mit mindestens drei Indexpositionen (Abschnitt 9.2).

Beispiel 10.5.7 (Linearität an einem Zahlenbeispiel)

Gegeben sind $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x}$ und $g(\mathbf{x}) = \mathbf{a}^\top \mathbf{x}$ mit $\mathbf{a} = (1, 2)^\top$. Gesucht ist $\nabla h(\mathbf{x})$ für $h(\mathbf{x}) = 3f(\mathbf{x}) - 2g(\mathbf{x})$ an der Stelle $\mathbf{x} = (1, -1)^\top$. Der Buchstabe h ist hier ausnahmsweise ein Funktionsname und nicht der Zuwachs aus (10.5.1); ein Zuwachs kommt in dieser Rechnung nicht vor.

Schritt 1: die Gradienten der Bausteine. Ausgeschrieben ist $f(\mathbf{x}) = x_1^2 + x_2^2$, also

$$\nabla f(\mathbf{x}) = (2x_1, 2x_2) = 2\mathbf{x}^\top.$$

Das ist Satz 10.2.8 mit $\mathbf{A} = \mathbf{I}$. Für $g(\mathbf{x}) = x_1 + 2x_2$ liefert Beispiel 10.2.7

$$\nabla g(\mathbf{x}) = \mathbf{a}^\top = (1, 2).$$

Schritt 2: Linearität anwenden. Zeile 2 aus Bemerkung 10.5.6 mit $c_1 = 3$ und $c_2 = -2$ ergibt

$$\nabla h(\mathbf{x}) = 3\nabla f(\mathbf{x}) - 2\nabla g(\mathbf{x}) = 3(2x_1, 2x_2) - 2(1, 2) = (6x_1 - 2, 6x_2 - 4).$$

Schritt 3: die Stelle einsetzen.

$$\nabla h\left(\begin{pmatrix} 1 \\ -1 \end{pmatrix}\right) = (6 \cdot 1 - 2, 6 \cdot (-1) - 4) = (4, -10).$$

Probe. Multiplizieren wir h zuerst aus, so steht dort $h(\mathbf{x}) = 3x_1^2 + 3x_2^2 - 2x_1 - 4x_2$, und die beiden partiellen Ableitungen sind $6x_1 - 2$ und $6x_2 - 4$. ☐ Bei diesem kleinen Beispiel sind beide Wege etwa gleich lang. Sobald f und g komplizierter werden, spielt der Umweg über die Bausteine seinen Vorteil aus, denn wir dürfen dann auf fertige Identitäten aus Abschnitt 10.2 zurückgreifen, statt eine unübersichtliche Summe von Hand zu differenzieren.

Selbsttest

1. Ist f in $\mathbf{x}$ differenzierbar, so ist f in $\mathbf{x}$ stetig.

Antwort: Wahr

Das ist Satz 10.5.2. Der Beweis schätzt beide Zusatzterme der linearen Approximation nach oben ab: den Ableitungsterm durch $M \|\mathbf{h}\|$ und den Restterm für kleine $\mathbf{h}$ durch $\|\mathbf{h}\|$. Zusammen bleibt $\|f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x})\| \leq (M + 1) \|\mathbf{h}\|$, und diese Schranke geht gegen null.

2. Ist f in $\mathbf{x}$ stetig, so ist f in $\mathbf{x}$ differenzierbar.

Antwort: Falsch

Die Umkehrung von Satz 10.5.2 gilt nicht. Die Betragsfunktion ist in 0 stetig, ihr Differenzenquotient strebt dort aber von rechts gegen +1 und von links gegen -1 (Beispiel 10.5.3). Stetigkeit ist notwendig, nicht hinreichend.

3. Am Knick von $f(x) = |x|$ existieren die einseitigen Grenzwerte des Differenzenquotienten nicht.

Antwort: Falsch

Sie existieren beide, sie stimmen nur nicht überein: +1 von rechts, -1 von links, und zwar schon für jedes einzelne $h \neq 0$. Genau daran scheitert Definition 10.1.1, die einen einzigen Grenzwert für beide Seiten verlangt.

4. Weil die Ableitungsoperation nach Satz 10.5.5 linear ist, hängt $D_{\mathbf{x}}f$ linear von der Stelle $\mathbf{x}$ ab.

Antwort: Falsch

Hier werden zwei Linearitäten verwechselt. Satz 10.5.5 sagt etwas über die Zuordnung $f \mapsto D_{\mathbf{x}}f$ bei fester Stelle. Wie $D_{\mathbf{x}}f$ von $\mathbf{x}$ abhängt, bleibt völlig offen: Für $f(x) = x^3$ ist $D_x f(h) = 3x^2 h$, und $x \mapsto 3x^2$ ist nicht linear (Bemerkung 10.1.4).

5. Jeder Term der Ordnung $O(\|\mathbf{h}\|)$ ist auch $o(\|\mathbf{h}\|)$.

Antwort: Falsch

Es gilt nur die eine Richtung. Jeder o -Term ist ein O -Term (Bemerkung 10.5.1), aber $g(h) = h$ erfüllt $|g(h)| \leq |h|$ und ist damit $O(|h|)$, während $g(h)/|h|$ betragsmäßig konstant 1 bleibt und nicht gegen null geht. Deshalb steht in (10.5.1) das kleine o und nicht das große O : Mit O wäre die

Forderung fast leer, denn sobald überhaupt eine beschränkte lineare Abbildung passte, passte jede andere genauso; die Differenz zweier solcher Abbildungen ist selbst $O(\|\mathbf{h}\|)$.

6. Für in $\mathbf{x}$ differenzierbare $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ gilt $\nabla [3f(\mathbf{x}) - 2g(\mathbf{x})] = 3\nabla f(\mathbf{x}) - 2\nabla g(\mathbf{x})$.

Antwort: Wahr

Das ist Zeile 2 aus Bemerkung 10.5.6, also Satz 10.5.5 im Fall $\mathbb{D} = \mathbb{R}^n$, $\mathbb{E} = \mathbb{R}$ mit $c_1 = 3$ und $c_2 = -2$. Beispiel 10.5.7 rechnet genau diesen Fall durch und erhält an der Stelle $(1, -1)^\top$ den Gradienten $(4, -10)$.

7. Stellen wir im Lupen-Widget $f(x) = |x|$ und $x_0 = 0$ ein und zoomen so weit hinein, wie der Regler reicht. Welchen Wert hat die angezeigte Abweichung von der Geraden dann, gemessen in Vielfachen der halben Fensterbreite w ?

Lösung: 1

Genau 1, und zwar auf jeder einzelnen Zoomstufe. Die Sehne durch die beiden Fensterränder ist bei $x_0 = 0$ waagrecht und liegt auf der Höhe w , während die Kurve in der Mitte den Wert 0 hat; die Abweichung ist also w selbst, in Vielfachen von w gerechnet 1. Der Knick verschwindet unter keiner Lupe. Zum Vergleich: Bei $f(x) = x^2$ steht dort w selbst, also eine Zahl, die mit jedem Zoomschritt kleiner wird.

Vertiefung: Die Summenregel in MML §5.1.2 ist der Spezialfall $\mathbb{D} = \mathbb{E} = \mathbb{R}$ von Satz 10.5.5; die übrigen Regeln von dort nimmt Abschnitt 10.6 auf.

10.6 Produkt- und Kettenregel

Warum zwei Regeln fehlen

Die Linearität aus Abschnitt 10.5 erledigt Summen und Vielfache. Damit allein kommen wir nicht weit. Die Zielfunktionen der Statistik sind selten Summen fertiger Bausteine, sondern Produkte und Verkettungen: Eine Quadratsumme $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2$ ist ein Skalarprodukt einer Funktion mit sich selbst, eine Log-Likelihood schiebt einen linearen Score durch eine nichtlineare Antwortfunktion, und ein neuronales Netz stapelt Dutzende solcher Schichten übereinander.

Aus der Analysis kennen wir beide Regeln für Funktionen einer reellen Variablen. Sie gelten wörtlich weiter, sobald wir Ableitung als lineare Approximation lesen. Bei der Produktregel ist allerdings zuerst zu klären, was „Produkt“ überhaupt heißen soll: Zwei Vektoren lassen sich nicht ohne Weiteres miteinander multiplizieren.

Produkte sind bilineare Abbildungen

Für $f, g : \mathbb{D} \rightarrow \mathbb{R}^m$ gibt es kein kanonisches Produkt $f(\mathbf{x})g(\mathbf{x})$. Was es gibt, sind mehrere nützliche Multiplikationen: das Skalarprodukt zweier Vektoren, das Matrixprodukt, das Produkt aus Zahl und Vektor. Alle drei teilen eine Eigenschaft. Halten wir ein Argument fest, so ist die Abbildung im anderen linear. Diese Abbildungen sind uns in Abschnitt 9.1 schon als multilineare Abbildungen begegnet; hier brauchen wir den Fall zweier Argumente und zusätzlich eine Größenschranke.

Definition 10.6.1 (Beschränkte bilineare Abbildung)

Seien $\mathbb{E}_1, \mathbb{E}_2$ und $\mathbb{B}$ normierte Vektorräume. Eine Abbildung $\langle \cdot, \cdot \rangle : \mathbb{E}_1 \times \mathbb{E}_2 \rightarrow \mathbb{B}$ heißt *bilinear*, wenn sie in jedem Argument linear ist, wenn also für alle $u, u_1, u_2 \in \mathbb{E}_1, v, v_1, v_2 \in \mathbb{E}_2$ und $c_1, c_2 \in \mathbb{R}$

$$\langle c_1 u_1 + c_2 u_2, v \rangle = c_1 \langle u_1, v \rangle + c_2 \langle u_2, v \rangle, \quad \langle u, c_1 v_1 + c_2 v_2 \rangle = c_1 \langle u, v_1 \rangle + c_2 \langle u, v_2 \rangle$$

gilt. Sie heißt *beschränkt*, wenn es ein $K < \infty$ gibt mit

$$\|\langle u, v \rangle\| \leq K \|u\| \|v\| \quad \text{für alle } u \in \mathbb{E}_1, v \in \mathbb{E}_2. \quad (10.6.1)$$

Bemerkung 10.6.2 (Beispiele, und warum die Schranke selten etwas kostet)

Vier Multiplikationen, die uns in diesem Abschnitt begegnen, samt ihrer Schranke.

1. Das Skalarprodukt $\langle u, v \rangle = u^\top v$ auf $\mathbb{R}^m \times \mathbb{R}^m \rightarrow \mathbb{R}$. Die Cauchy-Schwarz-Ungleichung liefert $K = 1$.
2. Zahl mal Vektor, $\mathbb{R} \times \mathbb{R}^m \rightarrow \mathbb{R}^m$, mit $\|c v\| = |c| \|v\|$, also ebenfalls $K = 1$.
3. Das Matrixprodukt $\mathbb{R}^{m \times n} \times \mathbb{R}^{n \times p} \rightarrow \mathbb{R}^{m \times p}$. Für eine submultiplikative Matrixnorm ist $K = 1$, siehe Abschnitt 3.5.
4. Die Spur eines Matrixprodukts, $(F, G) \mapsto \text{tr}(FG)$, mit $F \in \mathbb{R}^{m \times n}$ und $G \in \mathbb{R}^{n \times m}$. Bilinear ist sie, weil Matrixprodukt und Spur es sind. Wegen $\text{tr}(FG) = \sum_{i,j} f_{ij} g_{ji}$ ist sie das Frobenius-Skalarprodukt von F und $G^\top$, und Cauchy-Schwarz gibt wieder $K = 1$, diesmal in der Frobenius-Norm aus Abschnitt 3.1.

In endlich-dimensionalen Räumen ist jede bilineare Abbildung automatisch beschränkt: Wir entwickeln u und v in Basen, ziehen die endlich vielen Werte $\langle e_i, e_j \rangle$ heraus und erhalten eine Schranke aus deren Maximum. Erst im Unendlichdimensionalen ist die Forderung eine echte Zusatzbedingung. Der Beweis des folgenden Satzes braucht (10.6.1) ausdrücklich: Ohne diese Schranke lässt sich der Kreuzterm nicht abschätzen.

Damit ist der Rahmen gesteckt, und die Regel selbst sieht aus wie die aus der Schule.

Satz 10.6.3 (Produktregel)

Seien $\mathbb{D}, \mathbb{E}_1, \mathbb{E}_2$ und $\mathbb{B}$ normierte Vektorräume, $\langle \cdot, \cdot \rangle : \mathbb{E}_1 \times \mathbb{E}_2 \rightarrow \mathbb{B}$ beschränkt bilinear und $f : \mathbb{D} \rightarrow \mathbb{E}_1, g : \mathbb{D} \rightarrow \mathbb{E}_2$ in $x \in \mathbb{D}$ differenzierbar. Dann ist auch $b(x) = \langle f(x), g(x) \rangle$ in x differenzierbar mit

$$D_x b(h) = \langle D_x f(h), g(x) \rangle + \langle f(x), D_x g(h) \rangle. \quad (10.6.2)$$

Beweis.

- (1) Die Differenzierbarkeit von f und g an der Stelle x heißt

$$f(x+h) = f(x) + D_x f(h) + r_f(h), \quad g(x+h) = g(x) + D_x g(h) + r_g(h)$$

mit $\|r_f(h)\| / \|h\| \rightarrow 0$ und ebenso für r_g . Beide Ableitungen sind beschränkt, es gibt also $M_f, M_g < \infty$ mit $\|D_x f(h)\| \leq M_f \|h\|$ und $\|D_x g(h)\| \leq M_g \|h\|$.

Definition 10.1.5 der Fréchet-Ableitung, Abschnitt 10.1; die Beschränktheit der linearen Abbildung gehört dort zur Definition

- (2) Wir setzen beides ein und multiplizieren mit der Bilinearität gliedweise aus:

$$\begin{aligned} b(\mathbf{x} + \mathbf{h}) &= \langle f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + r_f(\mathbf{h}), g(\mathbf{x}) + D_{\mathbf{x}}g(\mathbf{h}) + r_g(\mathbf{h}) \rangle \\ &= b(\mathbf{x}) + \underbrace{\langle D_{\mathbf{x}}f(\mathbf{h}), g(\mathbf{x}) \rangle + \langle f(\mathbf{x}), D_{\mathbf{x}}g(\mathbf{h}) \rangle}_{=: L(\mathbf{h})} + R(\mathbf{h}) \end{aligned}$$

mit dem Sammelrest

$$R(\mathbf{h}) = \underbrace{\langle D_{\mathbf{x}}f(\mathbf{h}), D_{\mathbf{x}}g(\mathbf{h}) \rangle}_{\text{Kreuzterm}} + \langle f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + r_f(\mathbf{h}), r_g(\mathbf{h}) \rangle + \langle r_f(\mathbf{h}), g(\mathbf{x}) + D_{\mathbf{x}}g(\mathbf{h}) \rangle.$$

neun Terme wie beim Ausmultiplizieren zweier Summen; sortiert nach dem Term ohne $\mathbf{h}$, den beiden in $\mathbf{h}$ linearen Termen und dem Rest

- (3) Der Kreuzterm ist von der Ordnung $\|\mathbf{h}\|^2$:

$$\|\langle D_{\mathbf{x}}f(\mathbf{h}), D_{\mathbf{x}}g(\mathbf{h}) \rangle\| \leq K \|D_{\mathbf{x}}f(\mathbf{h})\| \|D_{\mathbf{x}}g(\mathbf{h})\| \leq KM_f M_g \|\mathbf{h}\|^2.$$

(10.6.1) mit $u = D_{\mathbf{x}}f(\mathbf{h})$ und $v = D_{\mathbf{x}}g(\mathbf{h})$, danach die beiden Schranken aus Schritt 1

- (4) Die beiden übrigen Summanden von $R(\mathbf{h})$ sind $o(\|\mathbf{h}\|)$. Für den mittleren etwa gilt

$$\|\langle f(\mathbf{x}) + D_{\mathbf{x}}f(\mathbf{h}) + r_f(\mathbf{h}), r_g(\mathbf{h}) \rangle\| \leq K(\|f(\mathbf{x})\| + M_f \|\mathbf{h}\| + \|r_f(\mathbf{h})\|) \|r_g(\mathbf{h})\|,$$

und die Klammer bleibt für kleine $\mathbf{h}$ unter einer festen Schranke, während $\|r_g(\mathbf{h})\| / \|\mathbf{h}\|$ gegen null geht. Der letzte Summand wird genauso behandelt.

wieder (10.6.1); der erste Faktor hängt nur beschränkt von $\mathbf{h}$ ab, der zweite ist nach Schritt 1 von kleinerer Ordnung als $\|\mathbf{h}\|$

- (5) Ordnung $\|\mathbf{h}\|^2$ ist kleiner als Ordnung $\|\mathbf{h}\|$: Teilen wir die Schranke aus Schritt 3 durch $\|\mathbf{h}\|$, so bleibt $KM_f M_g \|\mathbf{h}\|$ stehen, und das geht gegen null. Der Kreuzterm ist also ebenfalls $o(\|\mathbf{h}\|)$, und eine Summe aus endlich vielen solchen Termen ist wieder $o(\|\mathbf{h}\|)$. Insgesamt ist $R(\mathbf{h}) = o(\|\mathbf{h}\|)$.

$a(\mathbf{h}) = O(\|\mathbf{h}\|^2)$ heißt $\|a(\mathbf{h})\| \leq C \|\mathbf{h}\|^2$ nahe 0, also $\|a(\mathbf{h})\| / \|\mathbf{h}\| \leq C \|\mathbf{h}\| \rightarrow 0$; die Schreibweisen stehen in Bemerkung 10.5.1, die Landau-Rechenregeln in Abschnitt 2.4, dort nur mit Argument gegen unendlich statt gegen null

- (6) Bleibt zu prüfen, dass L als Kandidat überhaupt zulässig ist. Linear in $\mathbf{h}$ ist L , weil $D_{\mathbf{x}}f$ und $D_{\mathbf{x}}g$ es sind und die Multiplikation im jeweils anderen, festgehaltenen Argument linear ist. Beschränkt ist sie wegen

$$\|L(\mathbf{h})\| \leq K(M_f \|g(\mathbf{x})\| + M_g \|f(\mathbf{x})\|) \|\mathbf{h}\|.$$

Damit hat b die Gestalt $b(\mathbf{x} + \mathbf{h}) = b(\mathbf{x}) + L(\mathbf{h}) + o(\|\mathbf{h}\|)$ mit beschränktem, linearem L , und das ist Definition 10.1.5 mit $D_{\mathbf{x}}b = L$.

Bilinearität gibt die Additivität und Homogenität in jedem Argument einzeln; für die Schranke wieder (10.6.1), diesmal mit den festen Vektoren $f(\mathbf{x})$ und $g(\mathbf{x})$

□

Die Reihenfolge in (10.6.2) ist kein Zierrat. Sobald das Produkt nicht kommutativ ist, etwa beim Matrixprodukt, steht der abgeleitete Faktor genau dort, wo vorher der ursprüngliche stand.

Beispiel 10.6.4 (Die Produktregel in vier Bauformen)

Je nach Wahl der Räume und der Multiplikation liest sich (10.6.2) anders.

Reelle Funktionen, $\mathbb{D} = \mathbb{E}_1 = \mathbb{E}_2 = \mathbb{R}$, gewöhnliche Multiplikation:

$$b(x) = f(x)g(x) \implies b'(x) = f'(x)g(x) + f(x)g'(x).$$

Zwei skalare Felder, $\mathbb{D} = \mathbb{R}^n$, $\mathbb{E}_1 = \mathbb{E}_2 = \mathbb{R}$:

$$b(\mathbf{x}) = f(\mathbf{x})g(\mathbf{x}) \implies \nabla b(\mathbf{x}) = g(\mathbf{x})\nabla f(\mathbf{x}) + f(\mathbf{x})\nabla g(\mathbf{x}) \in \mathbb{R}^{1 \times n}.$$

Skalarprodukt zweier vektorwertiger Funktionen, $\mathbb{D} = \mathbb{R}^n$, $\mathbb{E}_1 = \mathbb{E}_2 = \mathbb{R}^m$:

$$b(\mathbf{x}) = f(\mathbf{x})^\top g(\mathbf{x}) \implies \nabla b(\mathbf{x}) = g(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x}) + f(\mathbf{x})^\top \mathbf{J}_g(\mathbf{x}).$$

Die Formatprobe geht auf: $1 \times m$ mal $m \times n$ ergibt $1 \times n$, wie es sich für den Gradienten einer skalaren Funktion auf dem $\mathbb{R}^n$ gehört.

Spur eines Matrixprodukts, $\mathbb{D} = \mathbb{R}$:

$$b(x) = \text{tr}[\mathbf{F}(x)\mathbf{G}(x)] \implies b'(x) = \text{tr}\left[\frac{\partial \mathbf{F}(x)}{\partial x}\mathbf{G}(x)\right] + \text{tr}\left[\mathbf{F}(x)\frac{\partial \mathbf{G}(x)}{\partial x}\right].$$

Damit FG quadratisch ist und die Spur überhaupt existiert, muss $\mathbf{F}(x) \in \mathbb{R}^{m \times n}$ und $\mathbf{G}(x) \in \mathbb{R}^{n \times m}$ sein; für quadratische Faktoren ($m = n$) ist das von selbst erfüllt. Zur Kontrolle rechnen wir den Fall einmal mit Zahlen nach. Für

$$\mathbf{F}(x) = \begin{pmatrix} x^2 & 1 \\ 0 & e^x \end{pmatrix}, \quad \mathbf{G}(x) = \begin{pmatrix} 1 & x \\ x & 2 \end{pmatrix}$$

liefern die Produktregel und der zentrale Differenzenquotient an der Stelle $x = 0,3$ beide den Wert 4,2997.

Die dritte Bauform ist die nützlichste, weil sich damit quadratische Formen in drei Zeilen ableiten lassen.

Beispiel 10.6.5 (Gradient einer quadratischen Form)

Gesucht ist $\nabla b(\mathbf{x})$ für $b(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ mit $\mathbf{A} \in \mathbb{R}^{n \times n}$. Wir lesen die quadratische Form als Skalarprodukt,

$$b(\mathbf{x}) = \langle f(\mathbf{x}), g(\mathbf{x}) \rangle \quad \text{mit} \quad f(\mathbf{x}) = \mathbf{x}, \quad g(\mathbf{x}) = \mathbf{A} \mathbf{x}.$$

Die beiden Jacobimatrizen kennen wir aus Abschnitt 10.3: $\mathbf{J}_f(\mathbf{x}) = \mathbf{I}_n$ und $\mathbf{J}_g(\mathbf{x}) = \mathbf{A}$. Die Produktregel gibt

$$\begin{aligned} \nabla b(\mathbf{x}) &= g(\mathbf{x})^\top \mathbf{J}_f(\mathbf{x}) + f(\mathbf{x})^\top \mathbf{J}_g(\mathbf{x}) \\ &= (\mathbf{A} \mathbf{x})^\top \mathbf{I}_n + \mathbf{x}^\top \mathbf{A} = \mathbf{x}^\top \mathbf{A}^\top + \mathbf{x}^\top \mathbf{A} = \mathbf{x}^\top (\mathbf{A} + \mathbf{A}^\top), \end{aligned}$$

und für symmetrisches $\mathbf{A}$ vereinfacht sich das zu $\nabla b(\mathbf{x}) = 2\mathbf{x}^\top \mathbf{A}$.

Die Unterscheidung ist keine Förmelerei. Für $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ und $\mathbf{x} = (1, 2)^\top$ ist $\mathbf{A} + \mathbf{A}^\top = \begin{pmatrix} 4 & 1 \\ 1 & 6 \end{pmatrix}$ und damit $\nabla b(\mathbf{x}) = (6, 13)$, während die Symmetriefformel $2\mathbf{x}^\top \mathbf{A} = (4, 14)$ liefern würde. Bei der symmetrischen Matrix $\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ stimmen beide Wege überein und geben $(8, 14)$.

In Abschnitt 10.2 haben wir denselben Gradienten schon einmal bestimmt, dort durch Aufschreiben der Doppelsumme und Ableiten nach jeder Komponente einzeln. Die Produktregel ersetzt diese Indexrechnung.

Beispiel 10.6.6 (Ridge-Regression)

Die Ridge-Regression ergänzt die lineare Regression um einen Strafterm. Mit Response $\mathbf{y} \in \mathbb{R}^n$, Designmatrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ und Regularisierungsparameter $\lambda > 0$ minimieren wir

$$\ell(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2.$$

Beide Summanden sind Skalarprodukte einer Funktion mit sich selbst. Für den ersten setzen wir $f(\boldsymbol{\beta}) = \mathbf{y} - \mathbf{X}\boldsymbol{\beta}$ mit $\mathbf{J}_f(\boldsymbol{\beta}) = -\mathbf{X}$ und erhalten aus der Produktregel

$$\begin{aligned} \nabla \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 &= f(\boldsymbol{\beta})^\top \mathbf{J}_f(\boldsymbol{\beta}) + f(\boldsymbol{\beta})^\top \mathbf{J}_f(\boldsymbol{\beta}) = -2(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{X} \\ &= 2(\mathbf{X}^\top \mathbf{X}\boldsymbol{\beta} - \mathbf{X}^\top \mathbf{y})^\top. \end{aligned}$$

Der Strafterm ist derselbe Fall mit $f(\boldsymbol{\beta}) = \boldsymbol{\beta}$ und $\mathbf{J}_f(\boldsymbol{\beta}) = \mathbf{I}_p$, sein Gradient also $2\lambda\boldsymbol{\beta}^\top$. Zusammen ergibt die Linearität aus Abschnitt 10.5

$$\nabla \ell(\boldsymbol{\beta}) = 2(\mathbf{X}^\top \mathbf{X}\boldsymbol{\beta} - \mathbf{X}^\top \mathbf{y} + \lambda\boldsymbol{\beta})^\top. \quad (10.6.3)$$

Nullsetzen führt auf $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p)\boldsymbol{\beta} = \mathbf{X}^\top \mathbf{y}$, also auf die Normalgleichungen aus Abschnitt 7.3 mit einem Zuschlag auf der Diagonalen. Der Zuschlag ist der eigentliche Gewinn: Ohne vollen Spaltenrang von $\mathbf{X}$ ist $\mathbf{X}^\top \mathbf{X}$ nur positiv semidefinit und singulär, $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p$ dagegen für $\lambda > 0$ stets positiv definit und invertierbar. Das gestrafte System ist deshalb stets *eindeutig* lösbar. Die ungestraften Normalgleichungen sind auch bei Rangdefizienz konsistent und besitzen Lösungen, diese sind dann aber nicht eindeutig; vollen Spaltenrang brauchen wir nur für ihre Eindeutigkeit.

Ein Zahlenbeispiel zur Kontrolle. Mit

$$\mathbf{X} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{y} = (1, 2, 2)^\top, \quad \lambda = 1,5$$

ist $\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix}$ und $\mathbf{X}^\top \mathbf{y} = (5, 11)^\top$. An der Stelle $\boldsymbol{\beta} = (0,5, 0,4)^\top$ liefert (10.6.3) den Gradienten $(-0,7, -3,6)$, was ein numerischer Differenzenquotient bestätigt. Die Lösung des gestraften Systems ist $\boldsymbol{\beta} = (0,341, 0,578)^\top$ gegenüber $(0,667, 0,500)^\top$ ohne Strafterm. Die Länge des Koeffizientenvektors sinkt dabei von 0,833 auf 0,671, die zweite Komponente wächst aber sogar leicht: Der Strafterm verkürzt den Vektor, nicht jede einzelne Komponente.

Eine Notationsfalle lauert beim Aufschreiben: Der Zeilenvektor $-2(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top \mathbf{X}$ und der Spaltenvektor $-2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})$ tragen dieselbe Information, sind aber verschiedene Objekte und dürfen nicht in derselben Zeile gleichgesetzt werden. Wir bleiben bei der Zeilenkonvention aus Abschnitt 10.2 und transponieren sichtbar.

Die Kettenregel

Verkettungen sind die zweite Art, aus einfachen Funktionen komplizierte zu bauen. Für Jacobimatrizien haben wir die zugehörige Regel in Abschnitt 10.3 schon bewiesen. Ohne Koordinaten wird sie noch einfacher, weil dann nicht einmal mehr ein Matrixprodukt dasteht, sondern schlicht eine Hintereinanderausführung.

Satz 10.6.7 (Kettenregel)

Seien $\mathbb{D}_1, \mathbb{D}_2, \mathbb{D}_3$ normierte Vektorräume, $f: \mathbb{D}_1 \rightarrow \mathbb{D}_2$ in $\mathbf{x} \in \mathbb{D}_1$ differenzierbar und $g: \mathbb{D}_2 \rightarrow \mathbb{D}_3$ in $\mathbf{y} = f(\mathbf{x}) \in \mathbb{D}_2$ differenzierbar. Dann ist die Verkettung $g \circ f$ in $\mathbf{x}$ differenzierbar mit

$$D_{\mathbf{x}}(g \circ f) = D_{\mathbf{y}}g \circ D_{\mathbf{x}}f. \quad (10.6.4)$$

Rechts steht die Verkettung zweier linearer Abbildungen, und die ist wieder linear und beschränkt. In Koordinaten wird daraus ein Matrixprodukt, dessen Reihenfolge die Verkettung spiegelt: zuerst wirkt $D_{\mathbf{x}}f$, also steht es rechts.

Beweis.

- (1) Wir kürzen den Zuwachs ab, den f weiterreicht:

$$\mathbf{k} := f(\mathbf{x} + \mathbf{h}) - f(\mathbf{x}) = D_{\mathbf{x}}f(\mathbf{h}) + r_f(\mathbf{h}).$$

Für hinreichend kleine $\mathbf{h}$ ist $\|\mathbf{k}\| \leq (M_f + 1)\|\mathbf{h}\|$: Der erste Summand ist durch $M_f\|\mathbf{h}\|$ beschränkt, der zweite für kleine $\mathbf{h}$ durch $\|\mathbf{h}\|$.

Differenzierbarkeit von f in $\mathbf{x}$; $\|r_f(\mathbf{h})\|/\|\mathbf{h}\| \rightarrow 0$ heißt insbesondere, dass der Quotient irgendwann unter 1 liegt

- (2) Die Differenzierbarkeit von g an der Stelle $\mathbf{y}$ liefert

$$(g \circ f)(\mathbf{x} + \mathbf{h}) = g(\mathbf{y} + \mathbf{k}) = g(\mathbf{y}) + D_{\mathbf{y}}g(\mathbf{k}) + r_g(\mathbf{k}).$$

$f(\mathbf{x} + \mathbf{h}) = \mathbf{y} + \mathbf{k}$ nach Schritt 1, und für g gilt Definition 10.1.5 an der Stelle $\mathbf{y}$

- (3) Weil $D_{\mathbf{y}}g$ linear ist, dürfen wir den Zuwachs aufspalten:

$$D_{\mathbf{y}}g(\mathbf{k}) = D_{\mathbf{y}}g(D_{\mathbf{x}}f(\mathbf{h})) + D_{\mathbf{y}}g(r_f(\mathbf{h})).$$

Weil sie zusätzlich beschränkt ist, ist der zweite Summand klein: $\|D_{\mathbf{y}}g(r_f(\mathbf{h}))\| \leq M_g \|r_f(\mathbf{h})\| = o(\|\mathbf{h}\|)$.

Linearität erlaubt $D(u + v) = D(u) + D(v)$; Beschränktheit heißt $\|D_{\mathbf{y}}g(u)\| \leq M_g \|u\|$, und ein festes Vielfaches eines $o(\|\mathbf{h}\|)$ -Terms bleibt $o(\|\mathbf{h}\|)$

- (4) Auch $r_g(\mathbf{k})$ ist $o(\|\mathbf{h}\|)$. Nach Schritt 1 ist nämlich

$$\frac{\|r_g(\mathbf{k})\|}{\|\mathbf{h}\|} \leq (M_g + 1) \frac{\|r_g(\mathbf{k})\|}{\|\mathbf{k}\|},$$

und mit $\mathbf{h} \rightarrow \mathbf{0}$ geht auch $\mathbf{k} \rightarrow \mathbf{0}$, der rechte Quotient also gegen null. Dieser Schritt wird leicht übersprungen: Der Restterm von g ist zunächst nur gemessen an $\mathbf{k}$ klein und nicht an $\mathbf{h}$. Erst die Schranke aus Schritt 1 wechselt den Bezugspunkt.

für $\mathbf{k} = \mathbf{0}$ ist $r_g(\mathbf{k}) = \mathbf{0}$, der Quotient wird dann gar nicht gebildet; sonst gilt die Abschätzung aus Schritt 1

- (5) Alles zusammengesetzt:

$$(g \circ f)(\mathbf{x} + \mathbf{h}) = (g \circ f)(\mathbf{x}) + (D_{\mathbf{y}}g \circ D_{\mathbf{x}}f)(\mathbf{h}) + o(\|\mathbf{h}\|).$$

Die Verkettung zweier beschränkter linearer Abbildungen ist beschränkt und linear, mit Schranke $M_g M_f$. Damit ist Definition 10.1.5 erfüllt, und (10.6.4) ist bewiesen.

$$\|D_{\mathbf{y}}g(D_{\mathbf{x}}f(\mathbf{h}))\| \leq M_g \|D_{\mathbf{x}}f(\mathbf{h})\| \leq M_g M_f \|\mathbf{h}\|$$

Beispiel 10.6.8 (Die Kettenregel in fünf Bauformen)

Wie bei der Produktregel hängt das Aussehen an den beteiligten Räumen.

Reelle Funktionen, $f, g : \mathbb{R} \rightarrow \mathbb{R}$:

$$[g(f(x))]' = g'(f(x)) f'(x).$$

Skalares Feld, dann reelle Funktion, $f : \mathbb{R}^n \rightarrow \mathbb{R}, g : \mathbb{R} \rightarrow \mathbb{R}$:

$$\nabla [g(f(\mathbf{x}))] = g'(f(\mathbf{x})) \nabla f(\mathbf{x}) \in \mathbb{R}^{1 \times n}.$$

Ein Skalar mal ein Zeilenvektor. Solange $g'(f(\mathbf{x})) \neq 0$ ist, bleibt die Richtung des Gradienten bis auf das Vorzeichen erhalten und nur seine Länge ändert sich; an einer Stelle mit $g'(f(\mathbf{x})) = 0$ verschwindet der Gradient dagegen ganz, gleichgültig wie steil f dort ist.

Zwei vektorwertige Abbildungen, $f : \mathbb{R}^n \rightarrow \mathbb{R}^m, g : \mathbb{R}^m \rightarrow \mathbb{R}^p$:

$$\mathbf{J}_{g \circ f}(\mathbf{x}) = \mathbf{J}_g(f(\mathbf{x})) \mathbf{J}_f(\mathbf{x}) \in \mathbb{R}^{p \times n}.$$

Das ist Satz 10.3.9, der Spezialfall, auf dem Backpropagation beruht.

Matrixargument, dann reelle Funktion, $f : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}, g : \mathbb{R} \rightarrow \mathbb{R}$:

$$\frac{\partial (g \circ f)(\mathbf{X})}{\partial \mathbf{X}} = g'(f(\mathbf{X})) \frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \in \mathbb{R}^{m \times n}.$$

Matrixwertige Kurve, dann reelle Funktion, $\mathbf{F} : \mathbb{R} \rightarrow \mathbb{R}^{m \times n}, g : \mathbb{R}^{m \times n} \rightarrow \mathbb{R}$, mit $\mathbf{Y} = \mathbf{F}(x)$:

$$\frac{\partial (g \circ \mathbf{F})}{\partial x} = D_{\mathbf{Y}} g \left(\frac{\partial \mathbf{F}(x)}{\partial x} \right) = \text{tr} \left(\left(\frac{\partial g(\mathbf{Y})}{\partial \mathbf{Y}} \right)^{\top} \frac{\partial \mathbf{F}(x)}{\partial x} \right) \in \mathbb{R}.$$

Hier zahlt sich die Lesart aus Abschnitt 10.4 aus: Die Fréchet-Ableitung einer skalaren Funktion mit Matrixargument wirkt über das Frobenius-Skalarprodukt, $D_{\mathbf{Y}} g(\mathbf{H}) = \text{tr}((\partial g / \partial \mathbf{Y})^{\top} \mathbf{H})$. Eingesetzt wird die Richtung $\mathbf{H} = \partial \mathbf{F}(x) / \partial x$.

Zwei der drei Voreinstellungen sind harmlos. Bei der dritten, $h(x) = \sqrt{x^2}$, laufen beide Faktoren aus dem Ruder: Die äußere Rate wächst zum Nullpunkt hin über jede Grenze, die innere geht dorthin gegen null. Was kommt dann heraus, und was passiert bei $x = 0$ selbst?

Interaktiv: Warum im Eindimensionalen ein Produkt herauskommt

Satz 10.6.7 verkettet zwei lineare Abbildungen. Ist $\mathbb{D}_1 = \mathbb{D}_2 = \mathbb{D}_3 = \mathbb{R}$, so ist jede lineare Abbildung eine Multiplikation mit einer Zahl, und das Hintereinanderausführen zweier Multiplikationen ist die Multiplikation der beiden Zahlen. Deshalb steht in (10.6.4) dort, wo sonst eine Verkettung oder ein Matrixprodukt stünde, schlicht ein Produkt zweier Steigungen. Die Tafel führt beide Zahlen einzeln vor und rechnet die Ableitung zur Kontrolle noch einmal ohne die Kettenregel nach, über einen zentralen Differenzenquotienten mit Schrittweite 10^{-6} .

Interaktives Element — nur in der Web-Fassung

Bei $h(x) = \sqrt{x^2}$ bleibt das Produkt zahm, obwohl beide Faktoren es nicht sind: Für jedes $x \neq 0$ ist $g'(f(x))f'(x) = \frac{1}{2|x|} \cdot 2x$ exakt das Vorzeichen von x , wie es sich für die Ableitung von $|x|$ gehört. Bei $x = 0$ dagegen ist die Voraussetzung des Satzes verletzt, weil die Wurzel an der

Stelle $u = f(0) = 0$ nicht differenzierbar ist; übrig bleibt der Knick aus Abschnitt 10.1, und das Produkt aus ∞ und 0 ist keine Zahl. Bemerkenswert ist, dass die numerische Gegenprobe davon nichts merkt und brav 0 meldet: Der zentrale Differenzenquotient mittelt über beide Seiten des Knicks. Beispiel 10.6.11 wird genau diesen Vorbehalt für die euklidische Norm brauchen.

Vertiefung: Alternatives Beispiel zur Kettenregel: logistische Regression

Kaum ein Modell zeigt die Kettenregel so sauber wie die logistische Regression. Wir wollen eine binäre Zielgröße $Y \in \{0, 1\}$ aus Merkmalen $\mathbf{x} \in \mathbb{R}^p$ vorhersagen und modellieren dafür

$$P(Y = 1 \mid \mathbf{x}) = \sigma(\boldsymbol{\beta}^\top \mathbf{x}), \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

Die Sigmoidfunktion σ presst den linearen Score $\boldsymbol{\beta}^\top \mathbf{x}$ in das Intervall $(0, 1)$, sodass die Vorhersage als Wahrscheinlichkeit lesbar ist. Der Verlust einer einzelnen Beobachtung $(\mathbf{x}, y)$ ist die negative Log-Likelihood

$$\ell(\boldsymbol{\beta}) = -[y \log(\sigma(\boldsymbol{\beta}^\top \mathbf{x})) + (1 - y) \log(1 - \sigma(\boldsymbol{\beta}^\top \mathbf{x}))] = \begin{cases} -\log P(Y = 1 \mid \mathbf{x}) & \text{für } y = 1, \\ -\log P(Y = 0 \mid \mathbf{x}) & \text{für } y = 0. \end{cases}$$

Für $y = 1$ bleibt vom Verlust also der erste Summand, für $y = 0$ der zweite.

Beispiel 10.6.9 (Gradient des logistischen Verlusts)

Zuerst brauchen wir die Ableitung von σ , und schon dafür arbeitet die Kettenregel. Mit $u(t) = 1 + e^{-t}$ und der äußeren Funktion $w \mapsto w^{-1}$ ist

$$\sigma'(t) = -\frac{1}{(1 + e^{-t})^2} \cdot (-e^{-t}) = \frac{e^{-t}}{(1 + e^{-t})^2} = \sigma(t)(1 - \sigma(t)), \quad (10.6.5)$$

wobei wir im letzten Schritt $1 - \sigma(t) = e^{-t}/(1 + e^{-t})$ benutzt haben. Die Ableitung der Sigmoidfunktion lässt sich also aus ihrem eigenen Funktionswert ablesen, was jede Implementierung dankbar ausnutzt.

Fall $y = 1$. Hier ist $\ell = g \circ f$ mit

$$f(\boldsymbol{\beta}) = \boldsymbol{\beta}^\top \mathbf{x}, \quad \nabla f(\boldsymbol{\beta}) = \mathbf{x}^\top, \quad g(t) = -\log \sigma(t).$$

Für die äußere Funktion liefert (10.6.5)

$$g'(t) = -\frac{\sigma'(t)}{\sigma(t)} = -(1 - \sigma(t)) = \sigma(t) - 1,$$

und die zweite Bauform aus Beispiel 10.6.8 setzt beides zusammen. Mit der Abkürzung $\hat{y} = \sigma(\boldsymbol{\beta}^\top \mathbf{x})$ ist

$$\nabla \ell(\boldsymbol{\beta}) = g'(f(\boldsymbol{\beta})) \nabla f(\boldsymbol{\beta}) = (\hat{y} - 1) \mathbf{x}^\top.$$

Fall $y = 0$. Jetzt ist $g(t) = -\log(1 - \sigma(t))$, also

$$g'(t) = \frac{\sigma'(t)}{1 - \sigma(t)} = \sigma(t),$$

und damit $\nabla \ell(\boldsymbol{\beta}) = \hat{y} \mathbf{x}^\top$. Das Vorzeichen ist hier positiv, und erst so fügen sich beide Fälle zusammen zu

$$\nabla \ell(\boldsymbol{\beta}) = (\hat{y} - y) \mathbf{x}^\top. \quad (10.6.6)$$

Ein Zahlenbeispiel, beide Fälle numerisch geprüft: Für $\mathbf{x} = (1, -2, 0.5)^\top$ und $\boldsymbol{\beta} = (0.3, -0.7, 1.1)^\top$ ist $\boldsymbol{\beta}^\top \mathbf{x} = 2.25$ und $\hat{y} = 0.9047$. Bei $y = 1$ ergibt (10.6.6) den Gradienten $(-0.0953, 0.1907, -0.0477)$, bei $y = 0$ dagegen $(0.9047, -1.8093, 0.4523)$.

Bemerkung 10.6.10 (Fehler mal Merkmal)

Gleichung (10.6.6) ist eine der meistbenutzten Formeln des maschinellen Lernens, und sie liest sich wie ein Satz: *Der Gradient ist der Vorhersagefehler, gewichtet mit dem Merkmalsvektor*. Zwei Konsequenzen.

Erstens die Richtung. Ist $\hat{y} > y$, sagt das Modell also zu viel voraus, so zeigt der Gradient in Richtung $\mathbf{x}^\top$. Der Gradientenabstieg aus Abschnitt 10.2 verschiebt β entgegen dieser Richtung, senkt damit den Score $\beta^\top \mathbf{x}$ und mit ihm die Vorhersage.

Zweitens die Länge. Es gilt stets $|\hat{y} - y| < 1$, weil σ die Werte 0 und 1 nie annimmt, also ist $\|\nabla \ell(\beta)\| = |\hat{y} - y| \|\mathbf{x}\| < \|\mathbf{x}\|$ für jedes $\mathbf{x} \neq \mathbf{0}$. Eine einzelne Beobachtung kann das Modell nie beliebig weit reißen, egal wie falsch die Vorhersage liegt.

Wie weit kann eine einzelne Beobachtung den Parameter reißen, wenn das Modell maximal danebenliegt? Und was bleibt vom Gradienten übrig, wenn das Merkmal selbst null ist?

Interaktiv: Der Gradient am Regler: eine Beobachtung, ein Merkmal

Die Tafel zeigt den einfachsten denkbaren Fall, ein Merkmal und einen Parameter. Links wandert der Punkt auf der Sigmoidkurve, wenn wir β verschieben; der orange Balken ist der Fehler $\hat{y} - y$. Rechts steht der Verlust als Funktion von β , die grüne Tangente hat die Steigung aus (10.6.6), und der orange Pfeil zeigt, wohin ein Abstiegsschritt mit Schrittweite 0,5 führt.

Interaktives Element — nur in der Web-Fassung

In beiden Klassen steht derselbe Ausdruck. Schalten wir auf $y = 0$ und schieben β nach oben, so wird das Modell selbstbewusst falsch, der Gradient nähert sich seiner Obergrenze $|\mathbf{x}|$, erreicht sie aber nie – das ist die zweite Konsequenz aus Bemerkung 10.6.10, ablesbar am Fehlerbalken, der nie über die Höhe 1 hinauskommt. Stellen wir umgekehrt $x = 0$ ein, so ist die Verlustkurve eine waagerechte Gerade auf der Höhe $\log 2 = 0,693147$, und der Gradient verschwindet für jedes β . Vor allem aber bestätigt die numerische Gegenprobe in beiden Klassen die Formel (10.6.6), Vorzeichen eingeschlossen.

1. Für den logistischen Verlust einer Beobachtung gilt $\nabla \ell(\beta) = (\hat{y} - y)\mathbf{x}^\top$ für $y = 0$ und für $y = 1$.

Antwort: Wahr

Beide Fälle führen über die Kettenregel auf dieselbe Form, siehe (10.6.6). Für $y = 1$ ist die äußere Ableitung $\sigma(t) - 1$, für $y = 0$ ist sie $\sigma(t)$, und beides ist $\hat{y} - y$.

Übung: Gradient der euklidischen Norm

Zum Schluss eine Rechnung, die beide Regeln dieses Abschnitts in Anspruch nimmt. Gesucht ist $\nabla h(\mathbf{x})$ für

$$h(\mathbf{x}) = \|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^\top \mathbf{x}}.$$

Bevor wir weiterlesen, lohnt der eigene Versuch: Welche innere und welche äußere Funktion bieten sich an?

Beispiel 10.6.11 (Gradient der euklidischen Norm)

Wir zerlegen $h = g \circ f$ mit

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{x}, \quad g(u) = \sqrt{u} = u^{1/2}.$$

Die innere Ableitung kennen wir aus Beispiel 10.6.5 mit $\mathbf{A} = \mathbf{I}_n$, denn $\mathbf{x}^\top \mathbf{x}$ ist die quadratische Form zur Einheitsmatrix: $\nabla f(\mathbf{x}) = 2\mathbf{x}^\top$. Die äußere ist $g'(u) = \frac{1}{2}u^{-1/2} = 1/(2\sqrt{u})$. Die Kettenregel gibt

$$\nabla h(\mathbf{x}) = g'(f(\mathbf{x}))\nabla f(\mathbf{x}) = \frac{2\mathbf{x}^\top}{2\sqrt{\mathbf{x}^\top \mathbf{x}}} = \frac{\mathbf{x}^\top}{\|\mathbf{x}\|_2}.$$

Der Gradient der Norm ist also der auf Länge 1 normierte Vektor selbst, als Zeile geschrieben. Er zeigt radial vom Ursprung weg, was zur Anschauung passt: Am schnellsten wächst der Abstand zum Ursprung, wenn wir uns direkt von ihm entfernen. Für $\mathbf{x} = (3, 4)^\top$ etwa ist $\nabla h(\mathbf{x}) = (0,6, 0,8)$, für $\mathbf{x} = (1, -2, 2)^\top$ entsprechend $(1/3, -2/3, 2/3)$.

Der Vorbehalt. Die Rechnung gilt nur für $\mathbf{x} \neq \mathbf{0}$. Die Kettenregel verlangt Differenzierbarkeit von g an der Stelle $u = f(\mathbf{x}) = \|\mathbf{x}\|_2^2$, und für $\mathbf{x} = \mathbf{0}$ ist das gerade die Stelle $u = 0$, an der g' über jede Grenze wächst. Der Nullpunkt ist dabei nicht bloß eine Lücke unserer Herleitung, dort gibt es überhaupt keine Fréchet-Ableitung. Gäbe es nämlich eine lineare Abbildung L mit $\|\mathbf{h}\|_2 = L(\mathbf{h}) + o(\|\mathbf{h}\|)$, so setzten wir $\mathbf{h} = t\mathbf{d}$ ein und teilten durch t : Für $t > 0$ folgte $L(\mathbf{d}) = \|\mathbf{d}\|_2$, für $t < 0$ dagegen $L(\mathbf{d}) = -\|\mathbf{d}\|_2$. Für $\mathbf{d} \neq \mathbf{0}$ widerspricht sich das. Im Fall $n = 1$ ist es genau der Knick der Betragsfunktion aus Abschnitt 10.1.

Selbsttest

1. Für zwei differenzierbare Abbildungen $f, g: \mathbb{R}^n \rightarrow \mathbb{R}^m$ mit $m > 1$ liefert die Produktregel den Gradienten von $f(\mathbf{x})g(\mathbf{x})$.

Antwort: Falsch

Für $m > 1$ ist $f(\mathbf{x})g(\mathbf{x})$ gar nicht erklärt: Zwei Vektoren lassen sich nicht ohne Weiteres multiplizieren. Satz 10.6.3 braucht eine bilineare Abbildung, die aus den beiden Werten ein neues Objekt macht, etwa das Skalarprodukt $f(\mathbf{x})^\top g(\mathbf{x})$. Erst dann steht die Regel bereit.

2. Für jede Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ gilt $\nabla(\mathbf{x}^\top \mathbf{A} \mathbf{x}) = 2\mathbf{x}^\top \mathbf{A}$.

Antwort: Falsch

Allgemein gilt $\mathbf{x}^\top (\mathbf{A} + \mathbf{A}^\top)$; die Kurzform mit dem Faktor 2 setzt $\mathbf{A} = \mathbf{A}^\top$ voraus (Beispiel 10.6.5). Für $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ und $\mathbf{x} = (1, 2)^\top$ ist der Gradient $(6, 13)$, die Symmetrieformel gäbe $(4, 14)$.

3. Der Kreuzterm $\langle D_{\mathbf{x}} f(\mathbf{h}), D_{\mathbf{x}} g(\mathbf{h}) \rangle$ im Beweis der Produktregel verschwindet für kleine $\mathbf{h}$, deshalb taucht er in der Formel nicht auf.

Antwort: Falsch

Er verschwindet nicht, er ist nur von zu kleiner Ordnung. Nach Schritt 3 des Beweises ist er $O(\|\mathbf{h}\|^2)$, und ein solcher Term geht nach Teilen durch $\|\mathbf{h}\|$ gegen null. Für $b(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} \mathbf{x}$ mit $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ und $\mathbf{h} = (0,1, -0,05)^\top$ etwa hat er den Wert 0,0225, also durchaus nicht null.

4. Im Unendlichdimensionalen genügt Bilinearität für Satz 10.6.3 nicht, dort brauchen wir zusätzlich die Schranke (10.6.1).

Antwort: Wahr

Ohne die Schranke lässt sich der Kreuzterm nicht kontrollieren, und der ganze Beweis bricht an Schritt 3. In endlicher Dimension ist jede bilineare Abbildung automatisch beschränkt (Bemerkung 10.6.2), weshalb die Voraussetzung dort nichts kostet.

5. Für $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ und $g : \mathbb{R}^m \rightarrow \mathbb{R}^p$ gilt $J_{g \circ f}(\mathbf{x}) = J_f(\mathbf{x}) J_g(f(\mathbf{x}))$.

Antwort: Falsch

Die Reihenfolge ist vertauscht. Richtig ist $J_{g \circ f}(\mathbf{x}) = J_g(f(\mathbf{x})) J_f(\mathbf{x})$, also die äußere Ableitung links. Schon die Formate zeigen es: J_g ist $p \times m$, J_f ist $m \times n$, und $p \times m$ mal $m \times n$ ergibt $p \times n$. Die falsche Reihenfolge stellt dagegen $m \times n$ neben $p \times m$ und ist für $n \neq p$ nicht einmal definiert.

6. Der Gradient $\nabla \|\mathbf{x}\|_2 = \mathbf{x}^\top / \|\mathbf{x}\|_2$ hat für jedes $\mathbf{x} \neq \mathbf{0}$ die Länge 1.

Antwort: Wahr

Wir teilen einen Vektor durch seine eigene Länge, das Ergebnis hat Länge 1. Anschaulich heißt das: Der Abstand zum Ursprung wächst in radialer Richtung mit Rate 1, und keine Richtung ist besser. In $\mathbf{x} = \mathbf{0}$ existiert der Gradient nicht (Beispiel 10.6.11).

7. Im Kettenregel-Widget steht die erste Voreinstellung $h(x) = (2x + 1)^4$ mit $x = 0,3$. Welchen Wert hat dort $h'(x)$?

Lösung: 32.768

32,768. Die innere Rate ist $f'(x) = 2$, die äußere $g'(f(x)) = 4 \cdot 1,6^3 = 16,384$, und Satz 10.6.7 multipliziert beide. Die numerische Gegenprobe im Widget landet auf demselben Wert; sie weicht erst in der zehnten Stelle ab.

8. Stellen wir im Logistik-Widget das Merkmal auf $x = 0$. Auf welcher Höhe liegt die Verlustkurve dann?

Lösung: 0.693147

Bei $\log 2 = 0,693147$, und zwar für jedes β . Mit $x = 0$ ist der Score $t = \beta x$ konstant null, also $\hat{y} = \sigma(0) = 0,5$ und $\ell = -\log 0,5 = \log 2$. Der Gradient $(\hat{y} - y)x$ verschwindet entsprechend, unabhängig von der Klasse: Ein Merkmal ohne Variation trägt keine Information über β .

Vertiefung: MML §5.1.2 stellt die Differentiationsregeln im eindimensionalen Fall zusammen, §5.2.1 die Produkt- und Summenregel für partielle Ableitungen und §5.2.2 die Kettenregel für verkettete Abbildungen.

10.7 Ableitungen höheren Grades

Wo die erste Ableitung aufhört

Die Fréchet-Ableitung ersetzt f in der Nähe von $\mathbf{x}$ durch eine lineare Abbildung. Für sehr vieles reicht das. Sie versagt aber ausgerechnet dort, wo wir am meisten wissen wollen. Sei $f : S \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ und $\mathbf{x}^*$ ein *kritischer Punkt*, also eine Stelle mit $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Die lineare Näherung lautet dort

$$f(\mathbf{x}^* + \mathbf{h}) \approx f(\mathbf{x}^*) + \nabla f(\mathbf{x}^*) \mathbf{h} = f(\mathbf{x}^*),$$

sie ist konstant. Ob $\mathbf{x}^*$ ein Minimum, ein Maximum oder keines von beidem ist, steht darin nicht. Davon hängt aber ab, ob ein Optimierungsverfahren angekommen ist, und in der Statistik, wie genau ein Schätzer den Parameter trifft.

Der Ausweg ist derselbe wie in der Schule: Wir leiten noch einmal ab. Was das für Funktionen mehrerer Variabler heißt, klärt dieser Abschnitt. Die Antwort heißt Hesse-Matrix, sie ist wegen des Satzes von Schwarz symmetrisch, ihre Eigenwerte entscheiden über die Art des kritischen Punktes, und in der Statistik führt sie direkt zur Fisher-Information.

Ableitungen höheren Grades

Wir definieren höhere Ableitungen in einem Zug für normierte Vektorräume. Die Definition wirkt beim ersten Lesen sperrig, sagt aber etwas sehr Einfaches, das wir gleich danach auspacken.

Definition 10.7.1 (k -mal (Fréchet-)differenzierbar)

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Vektorräume, $U \subseteq \mathbb{D}$ offen und $f : U \rightarrow \mathbb{E}$. Mit $\mathcal{L}^j(\mathbb{D}, \mathbb{E})$ bezeichnen wir den normierten Vektorraum der beschränkten j -linearen Abbildungen $\mathbb{D}^j \rightarrow \mathbb{E}$; für $j = 0$ setzen wir $\mathcal{L}^0(\mathbb{D}, \mathbb{E}) := \mathbb{E}$. Für $j \geq 1$ ist seine Operatornorm

$$\|A\|_{\text{op}} := \sup_{\|h_1\|, \dots, \|h_j\| \leq 1} \|A(h_1, \dots, h_j)\|.$$

Wir setzen $D^0 f := f$. Rekursiv heißt f auf U k -mal (Fréchet-)differenzierbar, wenn für jedes $j = 1, \dots, k$ die Abbildung

$$U \longrightarrow \mathcal{L}^{j-1}(\mathbb{D}, \mathbb{E}), \quad y \longmapsto D_y^{j-1} f$$

Fréchet-differenzierbar ist. Ihre Ableitung an x , eine beschränkte lineare Abbildung $\mathbb{D} \rightarrow \mathcal{L}^{j-1}(\mathbb{D}, \mathbb{E})$, identifizieren wir mit der beschränkten j -linearen Abbildung $D_x^j f \in \mathcal{L}^j(\mathbb{D}, \mathbb{E})$. Äquivalent gilt

$$\lim_{\substack{h \rightarrow 0 \\ x+h \in U}} \frac{\|D_{x+h}^{j-1} f - D_x^{j-1} f - D_x^j f(\cdot, \dots, \cdot, h)\|_{\text{op}}}{\|h\|} = 0. \quad (10.7.1)$$

Von k -maliger Differenzierbarkeit an x sprechen wir, wenn diese Voraussetzungen in einer offenen Umgebung von x erfüllt sind.

Bemerkung 10.7.2 (Wie die Definition zu lesen ist)

Sechs Beobachtungen, und die sperrige Formel wird handzahn.

Für $j = 1$ steht dort die alte Definition. Dann ist $\mathcal{L}^0(\mathbb{D}, \mathbb{E}) = \mathbb{E}$ und $D^0 f = f$, also lautet die Restbedingung

$$f(x+h) = f(x) + D_x f(h) + r(h), \quad \frac{\|r(h)\|}{\|h\|} \longrightarrow 0,$$

und das ist die Fréchet-Ableitung aus Definition 10.1.5. Die höheren Stufen sind also keine neue Idee, sondern dieselbe Idee, mehrfach angewandt.

Warum eine Umgebung vorkommt. Um $D_x^j f$ zu bilden, müssen die Werte $D_{x+h}^{j-1} f$ für kleine h bereits existieren. Ableitungen nur an der einzelnen Stelle x reichen dafür nicht.

Was hier wonach abgeleitet wird. Halten wir $h_1, \dots, h_{j-1}$ fest. Dann ist

$$x \longmapsto D_x^{j-1} f(h_1, \dots, h_{j-1})$$

eine Abbildung von $\mathbb{D}$ nach $\mathbb{E}$. Aus (10.7.1) folgt insbesondere: Ihre Fréchet-Ableitung an der Stelle x , angewandt auf h , ist $D_x^j f(h_1, \dots, h_{j-1}, h)$. In einem Satz: Die j -te Ableitung ist die lineare Näherung an die Änderung der $(j-1)$ -ten. Deshalb rechnen wir höhere Ableitungen wie gewohnt iterativ aus, Stufe für Stufe.

Warum die Operatornorm entscheidend ist. Sie verlangt mehr als eine Restabschätzung für feste Richtungen: Der Rest muss gleichmäßig für alle Richtungen mit Norm höchstens eins klein werden. Genau diese Gleichmäßigkeit braucht der allgemeine Taylor-Satz.

Multilinear und beschränkt. Multilinear heißt: linear in jedem Argument, wenn die übrigen festgehalten werden. Solche Abbildungen kennen wir aus Abschnitt 9.1 und aus der Produktregel in Abschnitt 10.6. Beschränkt heißt hier

$$\|D_{\mathbf{x}}^j f(\mathbf{h}_1, \dots, \mathbf{h}_j)\| \leq M \prod_{i=1}^j \|\mathbf{h}_i\|$$

für ein $M < \infty$, das von $\mathbf{x}$ abhängen darf. In endlicher Dimension ist das automatisch erfüllt und kostet uns nichts.

Die Objekte werden größer. Für $f : \mathbb{R}^n \rightarrow \mathbb{R}$ braucht $D_{\mathbf{x}}^1 f$ einen Zeilenvektor mit n Zahlen, $D_{\mathbf{x}}^2 f$ eine Matrix mit n^2 Zahlen, $D_{\mathbf{x}}^3 f$ einen Tensor dritter Stufe mit n^3 Zahlen. Die Stufe wächst mit j , und ab $j = 3$ verlassen wir die Matrixsprache (Abschnitt 9.2). Die Definition sagt noch nichts darüber, wie wir $D_{\mathbf{x}}^j f$ ausrechnen. Das tun die nächsten Seiten für den wichtigsten Fall, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, und danach für $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

Vektor zu Skalar: die Hesse-Matrix

Für skalarwertige Funktionen sammelt die zweite Ableitung alle partiellen Ableitungen zweiter Ordnung in einer Matrix.

Definition 10.7.3 (Hesse-Matrix)

Die Hesse-Matrix $\mathbf{H}_f(\mathbf{x})$ einer zweimal partiell differenzierbaren Funktion $f : S \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$ ist die Matrix der zweiten partiellen Ableitungen,

$$\mathbf{H}_f(\mathbf{x})_{i,j} := \frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} \in \mathbb{R}^{n \times n}.$$

Bloße Differenzierbarkeit reicht hier nicht. Damit die Einträge überhaupt existieren, brauchen wir zweite partielle Ableitungen, und für alles Weitere sogar deren Stetigkeit. Wir schreiben dafür kurz $f \in \mathcal{C}^2$, gemeint ist: alle zweiten partiellen Ableitungen existieren und sind stetig.

Unter dieser Bedingung fällt eine Symmetrie ab, die den halben Rechenaufwand spart. Wir stellen sie den Formeln für $D_{\mathbf{x}}^2 f$ voran, weil diese sie schon brauchen.

Satz 10.7.4 (Satz von Schwarz)

Seien $S \subseteq \mathbb{R}^n$ offen und $f : S \rightarrow \mathbb{R}$ zweimal stetig differenzierbar ($f \in \mathcal{C}^2$). Dann gilt für alle $\mathbf{x} \in S$ und alle i, j

$$\frac{\partial^2 f(\mathbf{x})}{\partial x_i \partial x_j} = \frac{\partial^2 f(\mathbf{x})}{\partial x_j \partial x_i}.$$

Vertiefung: Der Beweis: zweimal Mittelwertsatz an derselben doppelten Differenz

Beweis.

- (1) Wir schreiben kurz $\partial_i f := \partial f / \partial x_i$ und $\partial_j \partial_i f := \partial(\partial_i f) / \partial x_j$; das ist der Eintrag $\mathbf{H}_f(\mathbf{x})_{j,i}$. Für $i = j$ ist nichts zu zeigen, sei also $i \neq j$. Zu kleinen $s, t \neq 0$ betrachten wir die *doppelte Differenz*

$$\Delta(s, t) := f(\mathbf{x} + s\mathbf{e}_i + t\mathbf{e}_j) - f(\mathbf{x} + s\mathbf{e}_i) - f(\mathbf{x} + t\mathbf{e}_j) + f(\mathbf{x}).$$

Diese Größe ändert sich nicht, wenn wir die Rollen von (i, s) und (j, t) vertauschen: Es werden nur die vier Summanden umsortiert.

der erste und der letzte Summand sind schon symmetrisch, die beiden mittleren tauschen die Plätze

- (2) Wir setzen $\varphi(u) := f(\mathbf{x} + u\mathbf{e}_i + t\mathbf{e}_j) - f(\mathbf{x} + u\mathbf{e}_i)$, also $\Delta(s, t) = \varphi(s) - \varphi(0)$. Die Funktion φ hängt nur von einer Variablen ab und ist differenzierbar mit

$$\varphi'(u) = \partial_i f(\mathbf{x} + u\mathbf{e}_i + t\mathbf{e}_j) - \partial_i f(\mathbf{x} + u\mathbf{e}_i).$$

Nach dem Mittelwertsatz gibt es ein σ zwischen 0 und s mit $\Delta(s, t) = s\varphi'(\sigma)$.

Mittelwertsatz für φ auf dem Intervall zwischen 0 und s ; φ ist differenzierbar, weil f nach x_i stetig partiell differenzierbar ist

- (3) Jetzt dasselbe noch einmal in der anderen Richtung. Mit $\psi(v) := \partial_i f(\mathbf{x} + \sigma\mathbf{e}_i + v\mathbf{e}_j)$ ist $\varphi'(\sigma) = \psi(t) - \psi(0)$, und ψ ist differenzierbar mit $\psi'(v) = \partial_j \partial_i f(\mathbf{x} + \sigma\mathbf{e}_i + v\mathbf{e}_j)$. Der Mittelwertsatz liefert ein τ zwischen 0 und t mit

$$\Delta(s, t) = st \partial_j \partial_i f(\mathbf{x} + \sigma\mathbf{e}_i + \tau\mathbf{e}_j).$$

zweite Anwendung des Mittelwertsatzes; ψ ist differenzierbar, weil die zweite partielle Ableitung $\partial_j \partial_i f$ nach Voraussetzung existiert

- (4) Nun wiederholen wir die Schritte 2 und 3 mit vertauschten Rollen von i und j . Weil $\Delta(s, t)$ dabei nach Schritt 1 dasselbe bleibt, gibt es $\tilde{\sigma}$ zwischen 0 und t sowie $\tilde{\tau}$ zwischen 0 und s mit

$$st \partial_j \partial_i f(\mathbf{x} + \sigma\mathbf{e}_i + \tau\mathbf{e}_j) = st \partial_i \partial_j f(\mathbf{x} + \tilde{\tau}\mathbf{e}_i + \tilde{\sigma}\mathbf{e}_j).$$

Wir teilen durch $st \neq 0$ und lassen $s, t \rightarrow 0$ laufen. Beide Argumente streben gegen $\mathbf{x}$, und weil die zweiten partiellen Ableitungen stetig sind, streben beide Seiten gegen $\partial_j \partial_i f(\mathbf{x})$ beziehungsweise $\partial_i \partial_j f(\mathbf{x})$.

hier geht die Voraussetzung $f \in \mathcal{C}^2$ ein; ohne Stetigkeit der zweiten Ableitungen könnten die beiden Grenzwerte auseinanderfallen, und es gibt Gegenbeispiele dieser Art

□

Bemerkung 10.7.5 (Was die Symmetrie spart)

Aus Satz 10.7.4 folgt sofort $\mathbf{H}_f(\mathbf{x}) = \mathbf{H}_f(\mathbf{x})^\top$: Die Hesse-Matrix einer $\mathcal{C}^2$ -Funktion ist symmetrisch. Die Reihenfolge, in der wir partiell ableiten, spielt also keine Rolle.

Das ist mehr als eine Schönheit. Statt aller n^2 Einträge müssen wir nur die $n(n+1)/2$ Einträge auf und über der Diagonalen berechnen und speichern. Für $n = 100$ sind das 5050 statt 10 000 Zahlen, für $n = 1000$ schon 500 500 statt einer Million. Vorsicht mit der naheliegenden Kurzfassung: Das ist etwas *mehr* als die Hälfte, nicht weniger, denn $n(n+1)/2 = n^2/2 + n/2$. Der Quotient fällt mit wachsendem n von 0,75 bei $n = 2$ auf 0,5005 bei $n = 1000$.

Die Symmetrie zahlt sich zweimal aus: bei der Berechnung, und beim Lösen von Systemen mit $\mathbf{H}_f$. Ist $\mathbf{H}_f$ zusätzlich positiv definit, also gerade in der Nähe eines Minimums, so bietet sich das Cholesky-Verfahren aus Abschnitt 5.4 an; Symmetrie allein reicht dafür nicht, denn Satz 5.4.2 verlangt positive Definitheit. Für symmetrische Matrizen gilt außerdem der Spektralsatz: Es gibt eine Orthonormalbasis aus Eigenvektoren, und alle Eigenwerte sind reell. Darauf beruht alles Weitere in diesem Abschnitt.

Jetzt können wir die Ableitungen erster und zweiter Stufe für $f : \mathbb{R}^n \rightarrow \mathbb{R}$ ausrechnen.

Satz 10.7.6 (Erste und zweite Ableitung in Koordinaten)

Seien $S \subseteq \mathbb{R}^n$ offen, $f \in \mathcal{C}^2(S)$ und $\mathbf{x} \in S$. Dann gilt für alle $\mathbf{h}, \mathbf{h}_1, \mathbf{h}_2 \in \mathbb{R}^n$

$$D_{\mathbf{x}}f(\mathbf{h}) = \nabla f(\mathbf{x}) \mathbf{h} \quad \text{und} \quad D_{\mathbf{x}}^2f(\mathbf{h}_1, \mathbf{h}_2) = \mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_2.$$

Vertiefung: Warum Gradient und Hesse-Matrix die ersten beiden Ableitungen darstellen

Beweis.

- (1) Die erste Formel ist Gleichung (10.2.1) aus Abschnitt 10.2: Der Gradient ist der Zeilenvektor der partiellen Ableitungen, und $\mathbf{h} \mapsto \nabla f(\mathbf{x}) \mathbf{h}$ ist die zugehörige lineare Abbildung.

dort mit $\mathbf{h} = \mathbf{e}_j$ hergeleitet: eine lineare Abbildung ist durch ihre Werte auf den Koordinatenrichtungen festgelegt

- (2) Für die zweite Formel setzen wir $j = 2$ in (10.7.1). Das Produkt über $i = 1, \dots, 1$ ist $\|\mathbf{h}_1\|$, und beide D^1 -Terme schreiben wir nach Schritt 1 in Gradientenform:

$$D_{\mathbf{x}}^2f(\mathbf{h}_1, \mathbf{h}) = (\nabla f(\mathbf{x} + \mathbf{h}) - \nabla f(\mathbf{x})) \mathbf{h}_1 + o(\|\mathbf{h}\|) \|\mathbf{h}_1\|.$$

(10.7.1) mit $j = 2$; $D_{\mathbf{x}+\mathbf{h}}^1f(\mathbf{h}_1) = \nabla f(\mathbf{x} + \mathbf{h}) \mathbf{h}_1$ und ebenso an der Stelle $\mathbf{x}$

- (3) Der Gradient ist selbst eine Abbildung, nämlich $\mathbf{g} : S \rightarrow \mathbb{R}^n$, $\mathbf{g}(\mathbf{x}) := \nabla f(\mathbf{x})^\top$. Ihre Jacobimatrix hat die Einträge

$$J_{\mathbf{g}}(\mathbf{x})_{ij} = \frac{\partial}{\partial x_j} \frac{\partial f(\mathbf{x})}{\partial x_i} = \mathbf{H}_f(\mathbf{x})_{ji} = \mathbf{H}_f(\mathbf{x})_{ij},$$

also $J_{\mathbf{g}}(\mathbf{x}) = \mathbf{H}_f(\mathbf{x})$. Weil $f \in \mathcal{C}^2$ ist, sind alle diese Einträge stetig, und damit ist $\mathbf{g}$ differenzierbar:

$$\nabla f(\mathbf{x} + \mathbf{h})^\top = \nabla f(\mathbf{x})^\top + \mathbf{H}_f(\mathbf{x}) \mathbf{h} + \mathbf{r}(\mathbf{h}), \quad \|\mathbf{r}(\mathbf{h})\| = o(\|\mathbf{h}\|).$$

Satz 10.7.4 macht $\mathbf{H}_f$ symmetrisch, deshalb dürfen wir die Indizes tauschen; dass stetige partielle Ableitungen schon die volle Differenzierbarkeit von $\mathbf{g}$ nach sich ziehen, ist ein Standardsatz der Analysis, den wir hier als bekannt voraussetzen

- (4) Transponieren wir die letzte Zeile und setzen sie in Schritt 2 ein, so wird aus der Differenz der Gradienten $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) + \mathbf{r}(\mathbf{h})^\top$, und

$$D_{\mathbf{x}}^2f(\mathbf{h}_1, \mathbf{h}) = \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_1 + \mathbf{r}(\mathbf{h})^\top \mathbf{h}_1 + o(\|\mathbf{h}\|) \|\mathbf{h}_1\|.$$

Der ganze rote Rest ist $o(\|\mathbf{h}\|) \|\mathbf{h}_1\|$, denn $|\mathbf{r}(\mathbf{h})^\top \mathbf{h}_1| \leq \|\mathbf{r}(\mathbf{h})\| \|\mathbf{h}_1\|$ nach Cauchy-Schwarz. Halten wir nun $\mathbf{h}_1$ fest, so sind $\mathbf{h} \mapsto D_{\mathbf{x}}^2f(\mathbf{h}_1, \mathbf{h})$ und $\mathbf{h} \mapsto \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_1$ zwei lineare Abbildungen, die sich nur um $o(\|\mathbf{h}\|)$ unterscheiden, und solche Abbildungen sind gleich. Also ist $D_{\mathbf{x}}^2f(\mathbf{h}_1, \mathbf{h}) = \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_1$, und wegen der Symmetrie von $\mathbf{H}_f(\mathbf{x})$ ist das dasselbe wie $\mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$. Mit $\mathbf{h}_2 := \mathbf{h}$ steht die Behauptung da.

das Eindeutigkeitsargument aus dem letzten Schritt des Beweises zu Satz 10.5.5: $\mathbf{h} = t\mathbf{u}$ einsetzen, der Quotient hängt nicht von t ab und muss null sein

□

Die Formel $D_{\mathbf{x}}^2f(\mathbf{h}_1, \mathbf{h}_2) = \mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_2$ ist in beiden Argumenten linear, also eine bilineare Abbildung im Sinne von Definition 10.6.1. Für $\mathbf{h}_1 = \mathbf{h}_2 = \mathbf{h}$ wird daraus die quadratische Form $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$, und die misst gleich, wie sich f in Richtung $\mathbf{h}$ krümmt.

Beispiel 10.7.7 (Gradient und Hesse-Matrix einer kubischen Funktion)

Sei $f(\mathbf{x}) = \frac{1}{3}x_1^3 - x_1 + x_2^2$. Die ersten partiellen Ableitungen sind $\partial f/\partial x_1 = x_1^2 - 1$ und $\partial f/\partial x_2 = 2x_2$, also

$$\nabla f(\mathbf{x}) = \begin{pmatrix} x_1^2 - 1 & 2x_2 \end{pmatrix} \in \mathbb{R}^{1 \times 2}, \quad \mathbf{H}_f(\mathbf{x}) = \begin{pmatrix} 2x_1 & 0 \\ 0 & 2 \end{pmatrix}.$$

Die Hesse-Matrix ist symmetrisch, wie Satz 10.7.4 es verlangt, und sie hängt hier von $\mathbf{x}$ ab. Kritische Punkte gibt es zwei, nämlich $(1, 0)$ und $(-1, 0)$.

Im Punkt $(1, 0)$ ist $\mathbf{H}_f = \text{diag}(2, 2)$ mit den Eigenwerten 2 und 2, beide positiv. Im Punkt $(-1, 0)$ ist $\mathbf{H}_f = \text{diag}(-2, 2)$, ein Eigenwert positiv, der andere negativ. Rechnen wir nach, was das bedeutet:

$$f(-0,8; 0) - f(-1; 0) = -0,0373 < 0,$$

$$f(-1; 0,2) - f(-1; 0) = +0,0400 > 0.$$

Von $(-1, 0)$ aus fällt f also entlang der ersten Koordinate und steigt entlang der zweiten. Ein Minimum ist das nicht, ein Maximum auch nicht. Im Punkt $(1, 0)$ dagegen steigt f in jede Richtung, dort liegt ein lokales Minimum mit Wert $f(1, 0) = -2/3$. Der nächste Satz macht daraus eine Regel.

Bemerkung 10.7.8 (Die dritte Ableitung ist ein Tensor)

Für $j = 3$ passt kein Matrixformat mehr. Wir schreiben $D_{\mathbf{x}}^3 f \in \mathbb{R}^{n \times n \times n}$ mit den Einträgen

$$(D_{\mathbf{x}}^3 f)_{i,j,k} = \frac{\partial^3 f(\mathbf{x})}{\partial x_k \partial x_j \partial x_i}, \quad D_{\mathbf{x}}^3 f(\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3) = \sum_{i,j,k} (D_{\mathbf{x}}^3 f)_{i,j,k} h_{1i} h_{2j} h_{3k}.$$

Das ist ein Tensor dritter Stufe, wie wir ihn in Abschnitt 9.2 kennengelernt haben, mit n^3 Einträgen. Für $f \in \mathcal{C}^3$ ist er in allen drei Indizes symmetrisch, sodass wieder nur ein Bruchteil davon wirklich zu berechnen ist.

Zur Notation halten wir ein für alle Mal fest: Der Index an Position r gehört zum r -ten Argument, und in welcher Reihenfolge die partiellen Ableitungen ausgeführt werden, spielt unter den hier durchgehend vorausgesetzten Stetigkeitsannahmen keine Rolle (Satz 10.7.4 für $\mathcal{C}^2$, die entsprechende Symmetrie für $\mathcal{C}^3$). Der Speicherbedarf ist dagegen der eigentliche Grund, warum in der Praxis fast nie über die zweite Stufe hinausgegangen wird: Ein solcher Tensor verlangt für $n = 1000$ schon eine Milliarde Zahlen.

Krümmung, Definitheit und kritische Punkte

Die Hesse-Matrix beschreibt die lokale Krümmung von f . Weil sie symmetrisch ist, hat sie nach dem Spektralsatz reelle Eigenwerte und eine Orthonormalbasis aus Eigenvektoren. Das Vorzeichenmuster dieser Eigenwerte entscheidet über die Art des kritischen Punktes.

Satz 10.7.9 (Hesse-Kriterium für kritische Punkte)

Seien $S \subseteq \mathbb{R}^n$ offen, $f \in \mathcal{C}^2(S)$ und $\mathbf{x}^* \in S$ ein kritischer Punkt, also $\nabla f(\mathbf{x}^*) = \mathbf{0}$. Dann gilt:

1. Ist $\mathbf{H}_f(\mathbf{x}^*)$ positiv definit, gilt also $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} > 0$ für alle $\mathbf{h} \neq \mathbf{0}$, so ist $\mathbf{x}^*$ ein striktes lokales Minimum.

2. Ist $\mathbf{H}_f(\mathbf{x}^*)$ negativ definit, gilt also $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} < 0$ für alle $\mathbf{h} \neq \mathbf{0}$, so ist $\mathbf{x}^*$ ein striktes lokales Maximum.
3. Ist $\mathbf{H}_f(\mathbf{x}^*)$ indefinit, hat also positive und negative Eigenwerte, so ist $\mathbf{x}^*$ ein *Sattelpunkt*: In jeder noch so kleinen Umgebung von $\mathbf{x}^*$ liegen Punkte mit größerem und Punkte mit kleinerem Funktionswert.

Den Beweis liefert die Taylorentwicklung zweiter Ordnung aus Abschnitt 10.8. Die Idee lässt sich aber schon hier in einem Satz sagen. In einem kritischen Punkt fällt der lineare Term weg, und es bleibt

$$f(\mathbf{x}^* + \mathbf{h}) = f(\mathbf{x}^*) + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} + o(\|\mathbf{h}\|^2).$$

Ist $\mathbf{H}_f(\mathbf{x}^*)$ positiv definit mit kleinstem Eigenwert $\lambda_{\min} > 0$, so ist der Krümmungsterm mindestens $\frac{1}{2} \lambda_{\min} \|\mathbf{h}\|^2$, während der rote Rest schneller als $\|\mathbf{h}\|^2$ verschwindet. Für hinreichend kleine $\mathbf{h} \neq \mathbf{0}$ gewinnt also die Krümmung, und $f(\mathbf{x}^* + \mathbf{h}) > f(\mathbf{x}^*)$. Bei einem indefiniten $\mathbf{H}_f(\mathbf{x}^*)$ führen wir dasselbe Argument entlang der beiden Eigenrichtungen zu einem positiven und einem negativen Eigenwert getrennt durch.

Bemerkung 10.7.10 (Wenn die Hesse-Matrix nichts entscheidet)

Die drei Fälle in Satz 10.7.9 decken nicht alles ab. Offen bleibt genau der Grenzfall, in dem $\mathbf{H}_f(\mathbf{x}^*)$ semidefinit, aber nicht definit ist: Mindestens ein Eigenwert ist null, und die übrigen tragen alle dasselbe Vorzeichen. Ein Eigenwert null allein reicht dafür übrigens nicht; kommt ein gemischtes Vorzeichenmuster hinzu, etwa bei $\mathbf{H}_f(\mathbf{x}^*) = \text{diag}(1, -1, 0)$, so ist die Matrix indefinit und Fall 3 greift nach wie vor. Im semidefiniten Grenzfall dagegen entscheidet die zweite Ableitung gar nichts. Schon in einer Variablen sehen wir das an drei Funktionen mit $f'(0) = f''(0) = 0$:

$$f(x) = x^4 \text{ (Minimum),} \quad f(x) = -x^4 \text{ (Maximum),} \quad f(x) = x^3 \text{ (weder noch).}$$

In allen drei Fällen ist $\mathbf{H}_f(0) = (0)$, also zugleich positiv und negativ semidefinit. Erst die dritte beziehungsweise vierte Ableitung trennt die Fälle.

Und in der Gegenrichtung wird die Aussage schwächer: Liegt in $\mathbf{x}^*$ ein lokales Minimum, so folgt daraus nur, dass $\mathbf{H}_f(\mathbf{x}^*)$ positiv *semidefinit* ist. Die strikte Ungleichung darf verloren gehen, wie $f(x) = x^4$ zeigt.

Ein Bild macht das greifbar: die Funktion $f(\mathbf{x}) = x_1^2 + 4x_2^2$ mit ihren Höhenlinien. Zeichnen wir sie und machen die Eigenwerte gleich verstellbar. Der Aufbau nutzt aus, dass sich jede symmetrische Matrix nach dem Spektralsatz als $\mathbf{H} = \mathbf{R}(\varphi) \text{diag}(\lambda_1, \lambda_2) \mathbf{R}(\varphi)^\top$ schreiben lässt, mit einer Drehung $\mathbf{R}(\varphi)$. Wir stellen also nicht die Einträge von $\mathbf{H}$ ein, sondern gleich die Eigenwerte und die Lage der Eigenbasis. Zur Funktion $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{H} \mathbf{x}$ gehört dann an jeder Stelle dieselbe Hesse-Matrix $\mathbf{H}$, und $\mathbf{0}$ ist ein kritischer Punkt von ihr, solange $\mathbf{H}$ invertierbar ist sogar der einzige.

Interaktiv: Hesse-Kriterium zum Schieben

Damit lässt sich Satz 10.7.9 abfahren wie eine Landkarte. Was passiert mit den Höhenlinien, wenn ein Eigenwert das Vorzeichen wechselt? Welche Achse gehört zur langen Halbachse einer Ellipse? Und was bleibt an der Grenze übrig, wenn ein Eigenwert genau auf null steht?

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, hängt die ganze Fallunterscheidung an den Vorzeichen der beiden Eigenwerte. Das Beispiel von oben liegt auf dem ersten Knopf: $\lambda = (2, 8)$ und $\varphi = 0$ ergibt $\mathbf{H} = \text{diag}(2, 8)$ und damit $f(\mathbf{x}) = x_1^2 + 4x_2^2$. Die Höhenlinien sind Ellipsen um das Minimum in $\mathbf{0}$; das hängt an der Definitheit und geht bei jedem Vorzeichenwechsel verloren. Ihre Hauptachsen liegen in Richtung der Eigenvektoren, hier also auf den Koordinatenachsen, und die Krümmung entlang einer Hauptachse ist der zugehörige Eigenwert, denn $f(t\mathbf{v}_i) = \frac{1}{2}\lambda_i t^2$. Die letzten beiden Beobachtungen gelten für jede symmetrische Hesse-Matrix, die erste nur im definiten Fall. Die lange Halbachse gehört dabei zum *kleinen* Eigenwert: Die Niveaumenge zum Wert 4 hat die Halbachsen 2 und 1, und $2/1 = \sqrt{8/2}$. Der Regler φ dreht Höhenlinien und Hauptachsen gemeinsam, lässt Spur und Determinante aber unberührt – die Eigenwerte hängen nicht davon ab, wie wir das Koordinatensystem legen.

Schieben wir einen Eigenwert unter null, so zerfallen die Ellipsen in Hyperbeln, und das dick gezeichnete Niveau 0 wird zu einem Geradenkreuz. Das ist der Sattel aus Satz 10.7.9(3), und in der Raumtafel daneben ist es die Sattelfläche. Dieselbe Situation kennen wir schon: Das Gradientenfeld-Widget in Abschnitt 10.2 zeigt $f(\mathbf{x}) = x_1^2 + 3x_1x_2 + 2x_2^2$ mit der Hesse-Matrix $\begin{pmatrix} 2 & 3 \\ 3 & 4 \end{pmatrix}$, deren Eigenwerte $3 \pm \sqrt{10}$, also 6,162 und $-0,162$, genau dieses Vorzeichenmuster haben.

Konvexität

Die Definitheit im kritischen Punkt ist eine lokale Aussage. Verlangen wir sie überall, so bekommen wir eine globale Eigenschaft.

Satz 10.7.11 (Konvexität und positive Semidefinitheit)

Seien $S \subseteq \mathbb{R}^n$ offen und *konvex* und $f \in \mathcal{C}^2(S)$. Dann gilt

$$f \text{ ist konvex auf } S \iff \mathbf{H}_f(\mathbf{x}) \succeq 0 \text{ (positiv semidefinit) für alle } \mathbf{x} \in S.$$

Für eine konvexe Funktion liegt die Verbindungsstrecke zwischen zwei Punkten des Graphen nie unter dem Graphen. Satz 10.7.11 übersetzt das in eine punktweise Bedingung an die zweite Ableitung, und das ist der Grund, warum konvexe Optimierungsprobleme handhabbar sind: Jeder kritische Punkt ist dort schon ein globales Minimum, und es gibt keine Sattelpunkte, an denen ein Verfahren hängenbleiben könnte.

Vertiefung: Zwei Gegenbeispiele zu den Voraussetzungen offen und konvex

Bemerkung 10.7.12 (Warum die Menge offen und konvex sein muss)

Beide Zusätze in Satz 10.7.11 werden leicht überlesen, nötig sind sie trotzdem.

S muss konvex sein. Sonst ist die linke Seite gar nicht definiert: Die Konvexitätsbedingung spricht über Punkte $tx + (1-t)y$ auf der Verbindungsstrecke, und die kann S verlassen. Nehmen wir $S = (-2, -1) \cup (1, 2) \subset \mathbb{R}$, offen, aber nicht konvex, und darauf $f \equiv -5$ auf dem linken und $f \equiv 0$ auf dem rechten Stück. Dann ist $f'' \equiv 0 \geq 0$ auf ganz S . Trotzdem gibt es keine konvexe Funktion auf $\mathbb{R}$, die f fortsetzt: Eine konvexe Funktion hat nichtfallende Steigung, und wer links die Steigung 0 hat und danach um 5 steigt, kann rechts nicht wieder bei Steigung 0 ankommen. *S muss offen sein.* Sonst scheitert die Richtung von links nach rechts. Sei S die x_1 -Achse im $\mathbb{R}^2$, also konvex, aber ohne innere Punkte, und $f(\mathbf{x}) = x_1^2 - x_2^2$. Auf S ist $f(x_1, 0) = x_1^2$, also konvex. Die Hesse-Matrix ist überall $\text{diag}(2, -2)$ und damit indefinit. Die Äquivalenz bricht, weil die Bedingung an $\mathbf{H}_f$ Information aus Richtungen zieht, die in S gar nicht vorkommen. Sobald S offen und konvex ist, stimmt die Äquivalenz.

1. Gilt $\mathbf{H}_f(\mathbf{x}) \succeq 0$ für alle $\mathbf{x}$ einer beliebigen Menge $S \subseteq \mathbb{R}^n$, so ist f auf S konvex.

Antwort: Falsch

Ohne Zusatzbedingungen an S ist die Behauptung nicht einmal sinnvoll. Auf $S = (-2, -1) \cup (1, 2)$ mit $f \equiv -5$ links und $f \equiv 0$ rechts ist $f'' \equiv 0$, aber die Verbindungsstrecke zwischen zwei Punkten verlässt S , und eine konvexe Fortsetzung auf $\mathbb{R}$ gibt es nicht. Satz 10.7.11 verlangt S offen und konvex (Bemerkung 10.7.12).

Die Konvexität und was sie für Optimierungsverfahren bedeutet, ist ein Thema für sich; Kapitel 11 nimmt es sich vor.

Warum das in der Praxis zählt

Bemerkung 10.7.13 (Praxisrelevanz der Hesse-Matrix)

Vier Punkte, die zusammen erklären, warum die Hesse-Matrix in der Praxis überall auftaucht. *Warum Sattelpunkte in hoher Dimension plausibel sind.* Ein kritischer Punkt mit invertierbarer Hesse-Matrix ist nur dann ein Extremum, wenn *alle* n Eigenwerte dasselbe Vorzeichen haben; im ausgearteten Fall entscheidet die Hesse-Matrix nichts (Bemerkung 10.7.10). Schon eine grobe Zählung macht klar, wie selten das ist: Wären die n Vorzeichen wie Münzwürfe, so hätte ein Minimum (alle Vorzeichen positiv) die Wahrscheinlichkeit 2^{-n} und irgendein Extremum, Maximum eingeschlossen, die Wahrscheinlichkeit 2^{1-n} . Für $n = 100$ sind das $7,9 \cdot 10^{-31}$ beziehungsweise $1,6 \cdot 10^{-30}$. Die Eigenwerte sind nicht unabhängig, die Zählung ist deshalb nur eine Faustregel. Sie motiviert aber, warum indefinite kritische Punkte in hochdimensionalen Verlustlandschaften eine wichtige Rolle spielen können. Auch das Rauschen des stochastischen Gradientenabstiegs *kann* helfen, instabile Sattelregionen zu verlassen; eine Garantie gegen schlechte lokale Minima oder jeden Sattelpunkt folgt aus der Münzwurfrechnung nicht. Augenzwinkernd ließe sich anhängen, dass die Singularität also bevorstehe, vielleicht.

Statistik. Der wichtigste Spezialfall einer Funktion $\mathbb{R}^p \rightarrow \mathbb{R}$ ist die Log-Likelihood $\ell(\theta)$. Die Hesse-Matrix der negativen Log-Likelihood heißt *beobachtete Fisher-Information* und bestimmt, wie genau ein Schätzer den Parameter trifft. Der nächste Unterabschnitt führt das aus.

Maschinelles Lernen. Im streng konvexen quadratischen Modell bestimmt die Hesse-Matrix die Kondition exakt. Für den Gradientenabstieg aus Abschnitt 10.2 hängt die Konvergenzrate dann am Verhältnis $\kappa = \lambda_{\max}/\lambda_{\min}$ der extremen Eigenwerte: Bei optimal gewählter Lernrate

schrumpft der Abstand zum Minimum je Schritt auf das $(\kappa-1)/(\kappa+1)$ -fache. Dort wird $L(\theta) = \frac{1}{2}\theta^\top A\theta$ mit $A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ minimiert, und weil $H_L(\theta) = A$ an jeder Stelle gilt, sind die Eigenwerte von A genau die gesuchten Krümmungen: $\lambda_{\max} = 3,618$, $\lambda_{\min} = 1,382$ und $\kappa = 2,618$, der Faktor also 0,447. Bei $\kappa = 1000$ stünde dort 0,998, und das Verfahren käme kaum voran. Für eine allgemeine Verlustfunktion ist die Hesse-Matrix ortsabhängig. Dieselbe Aussage gilt dort nur lokal beziehungsweise unter zusätzlichen Annahmen wie einer gleichmäßigen Schranke $\mu I \leq H_L(\theta) \leq LI$.

Optimierung. Verfahren, die H_f nicht nur analysieren, sondern benutzen, konvergieren erheblich schneller. Das Newton-Verfahren in Abschnitt 10.8 ist der Prototyp.

Anwendung: Fisher-Information

Definition 10.7.14 (Fisher-Informationsmatrix)

Für ein statistisches Modell mit Log-Likelihood $\ell(\theta)$ und Parameter $\theta \in \mathbb{R}^p$ ist unter den üblichen Regularitätsbedingungen, die insbesondere das Differenzieren unter dem Erwartungswert erlauben, die *Fisher-Informationsmatrix* definiert als

$$I(\theta) := \mathbb{E}[-H_\ell(\theta)] = -\mathbb{E}\left[\frac{\partial^2 \ell(\theta)}{\partial \theta_i \partial \theta_j}\right]_{i,j} \in \mathbb{R}^{p \times p}.$$

Das $\mathbb{E}$ ist hier wieder der Erwartungswert und nicht der Zielraum aus Definition 10.7.1; erwartet wird über die Daten, bei festem wahren θ . Lassen wir den Erwartungswert weg, so erhalten wir die *beobachtete* Fisher-Information $-H_\ell(\theta)$ am jeweiligen Parameterwert; häufig wird sie am ML-Schätzer $\hat{\theta}$ ausgewertet. Unter denselben Regularitätsbedingungen ist die erwartete Score-Funktion null und es gilt außerdem $I(\theta) = \mathbb{E}[\nabla \ell(\theta)^\top \nabla \ell(\theta)]$.

Die Intuition steckt schon in der Krümmung. Eine stark gekrümmte Log-Likelihood fällt neben ihrem Maximum steil ab: Benachbarte Parameterwerte erklären die Daten deutlich schlechter, die Daten sind also *informativ* über θ . Eine flache Log-Likelihood lässt einen weiten Bereich von Parametern fast gleich gut aussehen, und der Parameter ist schwer zu schätzen. Beides misst dieselbe zweite Ableitung.

Vertiefung: Die Fisher-Information im Bernoulli-Modell

Beispiel 10.7.15 (Fisher-Information im Bernoulli-Modell)

Seien $y_1, \dots, y_n$ unabhängige, identisch verteilte Bernoulli-Variablen mit Erfolgswahrscheinlichkeit $0 < p < 1$. Die Log-Likelihood ist

$$\ell(p) = \sum_{i=1}^n [y_i \log p + (1 - y_i) \log(1 - p)].$$

Zweimal ableiten ergibt

$$\ell''(p) = - \sum_{i=1}^n \left[\frac{y_i}{p^2} + \frac{1 - y_i}{(1 - p)^2} \right],$$

und mit $\mathbb{E}[y_i] = p$ folgt

$$\mathbf{I}_n(p) = \mathbb{E}[-\ell''(p)] = n \left[\frac{p}{p^2} + \frac{1 - p}{(1 - p)^2} \right] = n \left[\frac{1}{p} + \frac{1}{1 - p} \right] = \frac{n}{p(1 - p)}.$$

Der ML-Schätzer ist hier der Mittelwert $\hat{p} = \bar{y}$, und seine Varianz ist bekannt: $\text{Var}(\hat{p}) = p(1 - p)/n$. Das ist exakt $\mathbf{I}_n(p)^{-1}$, in diesem Modell also keine Näherung. Mit $p = 0,2$ und $n = 100$ steht dort $\mathbf{I}_n = 625$, damit $\text{Var}(\hat{p}) = 0,0016$ und eine Standardabweichung von 0,04.

Die Krümmungs-Intuition lässt sich am selben Beispiel ablesen, weiterhin mit $n = 100$. Am flachsten ist die Log-Likelihood bei $p = 0,5$: Dort steuert eine einzelne Beobachtung nur $\mathbf{I}_1 = 4$ bei, und $\text{Var}(\hat{p}) = 0,0025$ ist am größten. Bei $p = 0,05$ ist $\mathbf{I}_1 = 21,05$, also gut das Fünffache, und $\text{Var}(\hat{p}) = 0,000475$: Nahe am Rand des Parameterbereichs sind die Daten deutlich informativer.

Vertiefung: Was die Merkregel zur Varianz des ML-Schätzers wirklich sagt**Bemerkung 10.7.16 (Cramér-Rao, sauber formuliert)**

Als Merkregel kursiert $\text{Var}(\hat{\theta}) \rightarrow \mathbf{I}(\theta)^{-1}$ für $n \rightarrow \infty$. Gemeint ist das Richtige, aber so verkürzt vermischt sie drei Dinge, die zu trennen sind.

Erstens die Buchführung über n . Ein konsistenter Schätzer hat $\text{Var}(\hat{\theta}) \rightarrow \mathbf{0}$; gegen eine feste Matrix kann die Varianz also nicht streben. Die Aussage trägt nur, wenn $\mathbf{I}$ die Information der ganzen Stichprobe meint. Für unabhängige, identisch verteilte Beobachtungen ist $\mathbf{I}_n(\theta) = n \mathbf{I}_1(\theta)$, und die Faustformel lautet

$$\text{Var}(\hat{\theta}) \approx \mathbf{I}_n(\theta)^{-1} = \frac{1}{n} \mathbf{I}_1(\theta)^{-1}.$$

Zweitens die saubere asymptotische Fassung. Unter Regularitätsbedingungen gilt für den ML-Schätzer

$$\sqrt{n}(\hat{\theta}_n - \theta) \xrightarrow{d} N(\mathbf{0}, \mathbf{I}_1(\theta)^{-1}),$$

und daraus lesen wir die Faustformel als Näherung für endliches n ab.

Drittens der Name. Die Cramér-Rao-Schranke selbst ist keine asymptotische Aussage, sondern gilt bei festem n und für erwartungstreue Schätzer: Deren Kovarianzmatrix erfüllt $\text{Var}(\hat{\theta}) \succeq \mathbf{I}_n(\theta)^{-1}$ im Sinne der Definitheit. Der ML-Schätzer ist im Allgemeinen nicht erwartungstreu und erreicht die Schranke erst asymptotisch. Im Bernoulli-Beispiel 10.7.15 fällt beides zusammen, weil $\hat{p}$ erwartungstreu ist und die Schranke exakt annimmt.

In allen drei Fassungen bleibt die Merkregel dieselbe: mehr Information, kleinere Varianz, präzisere Schätzung.

1. Für den ML-Schätzer gilt bei jedem Stichprobenumfang $\text{Var}(\hat{\theta}) = \mathbf{I}(\theta)^{-1}$.

Antwort: Falsch

Gemeint ist eine asymptotische Aussage. Bei festem n gilt die Cramér-Rao-Schranke $\text{Var}(\hat{\theta}) \succeq \mathbf{I}_n(\theta)^{-1}$, und zwar für erwartungstreue Schätzer; der ML-Schätzer ist im Allgemeinen verzerrt und erreicht die Schranke erst für $n \rightarrow \infty$, in der Form $\sqrt{n}(\hat{\theta}_n - \theta) \rightarrow N(0, \mathbf{I}_1(\theta)^{-1})$ (Bemerkung 10.7.16). Das Bernoulli-Modell ist ein Sonderfall, in dem Gleichheit schon für endliches n gilt.

Vektor zu Vektor

Zum Schluss der allgemeine Fall $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$. Er ist in der Statistik nicht exotisch. Die *Score-Funktion* $\nabla \ell(\theta)^\top$ ist genau von dieser Bauart, sie bildet $\mathbb{R}^p$ nach $\mathbb{R}^p$ ab, und ihre Jacobimatrix ist die Hesse-Matrix $\mathbf{H}_\ell(\theta)$, wie Schritt 3 im Beweis zu Satz 10.7.6 zeigt; aus ihr entsteht die Fisher-Information von Definition 10.7.14. Momentengleichungen sind ein zweites Beispiel. Auf das Transponierte kommt es dabei an: In der Zeilenkonvention aus Abschnitt 10.2 ist $\nabla \ell(\theta)$ eine Zeile, und erst die transponierte Fassung ist eine Abbildung von $\mathbb{R}^p$ nach $\mathbb{R}^p$.

Nach Definition 10.7.1 ist die j -te Ableitung eine multilineare Abbildung $D_{\mathbf{x}}^j f: (\mathbb{R}^n)^j \rightarrow \mathbb{R}^m$. Als Zahlenschema ist das ein Tensor der Stufe $j+1$ mit den Dimensionen

$$m \times \underbrace{n \times n \times \cdots \times n}_{j\text{-mal}},$$

also $m n^j$ Einträgen. In Koordinaten lautet die Auswertung

$$D_{\mathbf{x}}^j f(\mathbf{h}_1, \dots, \mathbf{h}_j) = \sum_{i_1, \dots, i_j=1}^n \underbrace{\frac{\partial^j f(\mathbf{x})}{\partial x_{i_j} \cdots \partial x_{i_1}}}_{\in \mathbb{R}^m} h_{1,i_1} \cdots h_{j,i_j}.$$

Die Ausgabe ist also eine Summe von Vektoren aus $\mathbb{R}^m$, gewichtet mit Produkten von Komponenten der $\mathbf{h}_i$. Für die ersten drei Stufen schreiben wir das aus, und es lohnt sich, die rechten Seiten anzusehen: Dort steht jeweils dieselbe Rechnung, ausgedrückt über die Jacobimatrix.

Satz 10.7.17 (Die ersten drei Stufen für Vektor-zu-Vektor)

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ hinreichend oft stetig differenzierbar. Dann gilt

$$\begin{aligned} D_{\mathbf{x}} f(\mathbf{h}) &= \sum_{i=1}^n \frac{\partial f(\mathbf{x})}{\partial x_i} h_i = \mathbf{J}_f(\mathbf{x}) \mathbf{h}, \\ D_{\mathbf{x}}^2 f(\mathbf{h}_1, \mathbf{h}_2) &= \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 f(\mathbf{x})}{\partial x_j \partial x_i} h_{1,i} h_{2,j} = \sum_{i=1}^n \frac{\partial \mathbf{J}_f(\mathbf{x})}{\partial x_i} \mathbf{h}_1 \cdot h_{2,i}, \\ D_{\mathbf{x}}^3 f(\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3) &= \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n \frac{\partial^3 f(\mathbf{x})}{\partial x_k \partial x_j \partial x_i} h_{1,i} h_{2,j} h_{3,k} = \sum_{i=1}^n \sum_{j=1}^n \frac{\partial^2 \mathbf{J}_f(\mathbf{x})}{\partial x_j \partial x_i} \mathbf{h}_1 \cdot h_{2,i} \cdot h_{3,j}. \end{aligned} \tag{10.7.2}$$

Vertiefung: Warum die höheren Ableitungen dieselbe Indexrechnung ausdrücken

Beweis.

- (1) Die erste Zeile ist die Jacobimatrix aus Abschnitt 10.3: Die i -te Spalte von $\mathbf{J}_f(\mathbf{x})$ ist $\partial f(\mathbf{x})/\partial x_i \in \mathbb{R}^m$, und das Produkt $\mathbf{J}_f(\mathbf{x})\mathbf{h}$ summiert die Spalten mit den Gewichten h_i .

Definition 10.3.1: $\mathbf{J}_f(\mathbf{x})_{aj} = \partial f_a(\mathbf{x})/\partial x_j$; Matrix mal Vektor ist die gewichtete Spaltensumme

- (2) Die zweite und die dritte Zeile sind reine Buchführung über Indexnamen: Schreiben wir beide Seiten komponentenweise aus, so stehen nach Umbenennung der Summationsindizes zweimal dieselben Summanden.

Umbenennen gebundener Summationsindizes ändert nichts; der Satz von Schwarz wird hier nirgends gebraucht, die Identität ist rein kombinatorisch

- (3) Zur Kontrolle der Spezialfall $m = 1$. Dann ist $\mathbf{J}_f(\mathbf{x}) = \nabla f(\mathbf{x})$ ein Zeilenvektor, und $\partial \nabla f(\mathbf{x})/\partial x_i$ ist die i -te Zeile von $\mathbf{H}_f(\mathbf{x})$. Die zweite Zeile von (10.7.2) wird damit zu

$$\sum_{i=1}^n (\mathbf{H}_f(\mathbf{x}) \mathbf{h}_1)_i h_{2,i} = \mathbf{h}_2^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_1 = \mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_2,$$

also genau Satz 10.7.6.

der letzte Schritt nutzt die Symmetrie aus Satz 10.7.4; ohne sie stünde dort die Transponierte

□

Wie groß werden diese Objekte? Für $f : \mathbb{R}^{10} \rightarrow \mathbb{R}^5$ hat $D_{\mathbf{x}}^1 f$ genau 50 Einträge, $D_{\mathbf{x}}^2 f$ schon 500 und $D_{\mathbf{x}}^3 f$ ganze 5000. Der Satz von Schwarz drückt die zweite Stufe auf $5 \cdot 10 \cdot 11/2 = 275$. Das erklärt, warum in der Praxis zweite Ableitungen fast nur für skalarwertige Funktionen wirklich aufgestellt werden, und auch dort oft nur als Näherung.

Selbsttest

Fünf Aussagen zu diesem Abschnitt. Welche davon stimmen?

1. Ist $f : S \rightarrow \mathbb{R}$ auf der offenen Menge S zweimal stetig differenzierbar, so ist die Hesse-Matrix in jedem Punkt symmetrisch.

Antwort: Wahr

Das ist der Satz von Schwarz (Satz 10.7.4). Der Beweis vergleicht zwei Mittelwertsätze an derselben doppelten Differenz $\Delta(s, t)$, und die Stetigkeit der zweiten Ableitungen sorgt dafür, dass beide Grenzwerte für $s, t \rightarrow 0$ zusammenfallen.

2. Der Satz von Schwarz senkt die Zahl der zu berechnenden zweiten Ableitungen von n^2 auf $n(n+1)/2$, also auf weniger als die Hälfte.

Antwort: Falsch

Die Zahlen stimmen, der Zusatz nicht: $n(n+1)/2 = n^2/2 + n/2$ ist etwas *mehr* als die Hälfte von n^2 . Für $n = 100$ sind es 5050 statt 5000. Erst für große n nähert sich der Quotient der Hälfte an, bei $n = 1000$ liegt er bei 0,5005 (Bemerkung 10.7.5).

3. Verschwindet der Gradient in $\mathbf{x}^*$ und ist $\mathbf{H}_f(\mathbf{x}^*)$ positiv semidefinit, so liegt in $\mathbf{x}^*$ ein lokales Minimum.

Antwort: Falsch

Semidefinit reicht nicht. Für $f(x) = x^3$ ist $f'(0) = f''(0) = 0$, die Hesse-Matrix (0) ist positiv semidefinit, und trotzdem liegt in 0 weder ein Minimum noch ein Maximum. Satz 10.7.9 verlangt positive *Definitheit*; im semidefiniten Grenzfall entscheidet die zweite Ableitung nichts

(Bemerkung 10.7.10).

4. Die Abbildung $(\mathbf{h}_1, \mathbf{h}_2) \mapsto \mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_2$ ist in jedem der beiden Argumente linear.

Antwort: Wahr

Halten wir $\mathbf{h}_2$ fest, so ist die Zuordnung $\mathbf{h}_1 \mapsto \mathbf{h}_1^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}_2$ das Skalarprodukt mit einem festen Vektor, also linear; für $\mathbf{h}_1$ fest genauso. Damit ist $D_{\mathbf{x}}^2 f$ eine bilineare Abbildung im Sinne von Definition 10.6.1, so wie es Definition 10.7.1 für alle Stufen verlangt.

5. Nach der Münzwurf-Heuristik aus Bemerkung 10.7.13 sind in sehr hohen Dimensionen die meisten kritischen Punkte lokale Minima oder Maxima.

Antwort: Falsch

Im Münzwurfmodell aus Bemerkung 10.7.13 ist es umgekehrt. Sobald die Hesse-Matrix Eigenwerte beider Vorzeichen trägt, liegt ein Sattelpunkt vor; ein Extremum verlangt bei invertierbarer Hesse-Matrix, dass alle n Eigenwerte dasselbe Vorzeichen haben. Eine grobe Zählung mit unabhängigen Vorzeichen gibt 2^{-n} für ein Minimum und 2^{1-n} für ein Extremum irgendeiner Sorte, für $n = 100$ also $7,9 \cdot 10^{-31}$ beziehungsweise $1,6 \cdot 10^{-30}$. Das ist eine qualitative Motivation, keine Verteilungsaussage über reale Verlustlandschaften. Bei indefiniter Hesse-Matrix existiert eine Richtung negativer Krümmung, und Rauschen kann einem stochastischen Verfahren helfen, eine solche instabile Region zu verlassen; automatisch gelingt das nicht.

6. Stellen wir im Definitheits-Widget die Voreinstellung „Rinne (halbdefinit)“ ein. Welchen Wert hat dort $\det \mathbf{H}$?

Lösung: 0

Null. Die Voreinstellung setzt $\lambda = (3, 0)$, und wegen $\det \mathbf{H} = \lambda_1 \lambda_2$ ist die Determinante damit null, die Matrix also singulär. Im Bild wird das zu Parallelen statt Ellipsen: Entlang der Hauptachse zum Eigenwert 0 bleibt f konstant, statt eines isolierten kritischen Punktes liegt dort eine ganze Gerade davon. Satz 10.7.9 greift in diesem Fall nicht mehr (Bemerkung 10.7.10).

Vertiefung: MML §5.7 behandelt die höheren Ableitungen und die Hesse-Matrix; die Fisher-Information und die Cramér-Rao-Schranke stehen in jedem Lehrbuch zur mathematischen Statistik, etwa bei Casella und Berger, Statistical Inference, Kapitel 7.

10.8 Taylorapproximation

Vom linearen Blick zum Polynom

Abschnitt 10.1 hat die Ableitung als lineare Näherung eingeführt: In der Nähe von $\mathbf{x}$ verhält sich f wie eine lineare Abbildung, und der Fehler ist $o(\|\mathbf{h}\|)$. Diese Näherung ist billig und oft gut genug, aber sie ist eben nur die erste Stufe. Mit den höheren Ableitungen aus Abschnitt 10.7 haben wir jetzt das Material, um weiterzubauen: eine quadratische Näherung, eine kubische, und so fort. Das Ergebnis heißt Taylorapproximation, und sie ist eines der meistbenutzten Werkzeuge des ganzen Skripts.

Zwei Verwendungen laufen durch dieses Skript. Erstens rechnen wir mit dem Polynom statt mit f , weil sich seine Ableitungen und kritischen Punkte leichter bestimmen lassen, ohne dass wir über f mehr wissen müssen als ein paar Ableitungen an einer einzigen Stelle. Der Newton-Schritt am Ende dieses Abschnitts macht davon Gebrauch: Bestimmt wird der stationäre Punkt der quadratischen Näherung; bei positiver Hesse-Matrix ist er deren Minimum. Zweitens sind Taylorentwicklungen das Standardwerkzeug der Konvergenz- und Asymptotikanalyse. Die asymptotische

Normalität des Maximum-Likelihood-Schätzers etwa entsteht, indem wir die Score-Funktion um den wahren Parameter entwickeln; wir kommen in Bemerkung 10.8.13 darauf zurück.

Der Farbcode des Kapitels bleibt: blau die Funktion und ihre Werte, grün die Näherung T_k und die abstrakten Ableitungen $D_x^j f$, rot der Restterm. Gradient, Hesse-Matrix und die übrigen konkreten Ableitungsobjekte tragen das Orange aus Abschnitt 10.7, in den Formeln wie in den Widgets.

Der eindimensionale Fall

Fangen wir dort an, wo sich alles nachrechnen lässt. Gesucht ist ein Polynom, das sich an einer Stelle x an f anschmiegt, und zwar so eng wie möglich. Die Bedingung „so eng wie möglich“ lässt sich präzise fassen: Das Polynom soll in x denselben Wert, dieselbe Steigung, dieselbe Krümmung und so weiter haben wie f .

Definition 10.8.1 (Taylorpolynom)

Seien $S \subseteq \mathbb{R}$ offen und $f : S \rightarrow \mathbb{R}$ an der Stelle $x \in S$ k -mal differenzierbar. Das Polynom

$$T_k(h) := \sum_{j=0}^k \frac{1}{j!} f^{(j)}(x) h^j = f(x) + f'(x)h + \frac{1}{2} f''(x)h^2 + \dots + \frac{1}{k!} f^{(k)}(x)h^k \quad (10.8.1)$$

heißt *Taylorpolynom der Ordnung k von f im Entwicklungspunkt x* . Sein Grad ist höchstens k und kann kleiner sein, wenn die höchsten Koeffizienten verschwinden.

Wir lesen T_k als Funktion des Zuwachses h bei festgehaltenem Entwicklungspunkt, ganz wie die lineare Abbildung $D_x f$ aus Definition 10.1.5. Der Ausdruck rechts hängt vom Zuwachs h ab und nicht von x allein; Bemerkung 10.8.4 stellt die gebräuchliche zweite Schreibweise daneben.

Die Anschmiegebedingung steckt bereits in (10.8.1): Leiten wir das Polynom j -mal nach h ab und setzen $h = 0$, so überlebt genau der j -te Summand, und es bleibt $f^{(j)}(x)$ stehen. Für $k = 1$ steht dort die Tangente aus Abschnitt 10.1, für $k = 0$ die Konstante $f(x)$.

Satz 10.8.2 (Taylorentwicklung I)

Seien $S \subseteq \mathbb{R}$ offen und $f : S \rightarrow \mathbb{R}$ $(k+1)$ -mal stetig differenzierbar. Dann gilt für $x \in S$ und $h \rightarrow 0$

$$f(x+h) = f(x) + \sum_{j=1}^k \frac{1}{j!} f^{(j)}(x) h^j + o(|h|^k). \quad (10.8.2)$$

Genauer gibt es zu jedem hinreichend kleinen $h \neq 0$ eine Zwischenstelle ξ strikt zwischen x und $x+h$ mit

$$f(x+h) - T_k(h) = \frac{f^{(k+1)}(\xi)}{(k+1)!} h^{k+1}. \quad (10.8.3)$$

Vertiefung: Der Beweis: zwei Hilfsfunktionen, eine Teleskopsumme, der Cauchy-Mittelwertsatz

Beweis.

- (1) Sei $h \neq 0$ so klein, dass das abgeschlossene Intervall zwischen x und $x + h$ ganz in S liegt. Wir bauen zwei Hilfsfunktionen auf diesem Intervall, beide in der Variablen t :

$$F(t) := \sum_{j=0}^k \frac{f^{(j)}(t)}{j!} (x + h - t)^j, \quad G(t) := (x + h - t)^{k+1}.$$

An den beiden Endpunkten stehen genau die Größen, um die es geht: $F(x + h) = f(x + h)$, denn alle Summanden mit $j \geq 1$ verschwinden, und $F(x) = T_k(h)$ nach (10.8.1). Ebenso ist $G(x + h) = 0$ und $G(x) = h^{k+1}$.

In F ist der Entwicklungspunkt die Variable und $x+h$ fest; genau umgekehrt zu (10.8.1). Stünde hier durchgehend $f^{(j)}(x)$ statt $f^{(j)}(t)$, so wäre F ein Polynom in t mit konstanten Koeffizienten und der nächste Schritt scheiterte

- (2) Jetzt leiten wir F nach t ab. Jeder Summand ist ein Produkt, die Produktregel liefert

$$\frac{d}{dt} \left[\frac{f^{(j)}(t)}{j!} (x + h - t)^j \right] = \frac{f^{(j+1)}(t)}{j!} (x + h - t)^j - \frac{f^{(j)}(t)}{(j-1)!} (x + h - t)^{j-1}.$$

Der erste Term für den Index j ist bis aufs Vorzeichen der zweite Term für den Index $j + 1$. Beim Aufsummieren hebt sich deshalb fast alles weg, und es bleibt nur der erste Term des letzten Summanden übrig:

$$F'(t) = \frac{f^{(k+1)}(t)}{k!} (x + h - t)^k. \quad (10.8.4)$$

Teleskopsumme: der Abzugsterm zu $j + 1$ löscht den Zuwachsterm zu j ; für $j = 0$ ist der Abzugsterm leer, weil dort kein $(j - 1)$ -Summand existiert

- (3) Auch G leiten wir ab, $G'(t) = -(k + 1)(x + h - t)^k$. Für t strikt zwischen x und $x + h$ ist $x + h - t \neq 0$, also $G'(t) \neq 0$. Damit sind die Voraussetzungen des Mittelwertsatzes in der Cauchyschen Fassung erfüllt: Es gibt ein ξ strikt zwischen x und $x + h$ mit

$$\frac{F(x + h) - F(x)}{G(x + h) - G(x)} = \frac{F'(\xi)}{G'(\xi)}.$$

F und G sind stetig auf dem abgeschlossenen und differenzierbar auf dem offenen Intervall, weil f nach Voraussetzung $(k + 1)$ -mal stetig differenzierbar ist

- (4) Beide Seiten sind jetzt bekannt. Links steht

$$\frac{f(x + h) - T_k(h)}{0 - h^{k+1}},$$

rechts kürzt sich der Faktor $(x + h - \xi)^k$ aus (10.8.4) gegen denselben Faktor in $G'(\xi)$ weg:

$$\frac{F'(\xi)}{G'(\xi)} = \frac{f^{(k+1)}(\xi) / k!}{-(k + 1)} = -\frac{f^{(k+1)}(\xi)}{(k + 1)!}.$$

Multiplizieren wir mit $-h^{k+1}$, so steht (10.8.3) da.

Der Faktor $(x + h - \xi)^k$ ist nach Schritt 3 von null verschieden, das Kürzen ist also erlaubt. In der Zwischenstelle steht $f^{(k+1)}(\xi)$ und nicht $f^{(k+1)}(x)$ – sonst bräute man den Mittelwertsatz um seine Aussage

- (5) Bleibt die Kleinheitsaussage (10.8.2). Wir wählen ein $\delta > 0$ so, dass das abgeschlossene Intervall $[x - \delta, x + \delta]$ noch in S liegt. Dort ist $f^{(k+1)}$ stetig auf einer abgeschlossenen und beschränkten Menge, also beschränkt, sagen wir durch M . Für $0 < |h| \leq \delta$ folgt

$$\frac{|f(x + h) - T_k(h)|}{|h|^k} = \frac{|f^{(k+1)}(\xi)|}{(k + 1)!} |h| \leq \frac{M}{(k + 1)!} |h| \xrightarrow{h \rightarrow 0} 0,$$

und das heißt gerade $f(x+h) - T_k(h) = o(|h|^k)$.

Stetigkeit in dem einen Punkt x liefert keine Schranke für die Werte an den Zwischenstellen ξ . Gebraucht wird Beschränktheit auf einer ganzen kompakten Umgebung, und die gibt es, weil $f^{(k+1)}$ dort stetig ist

□

Bemerkung 10.8.3 (Was der Satz sagt und was nicht)

Zwei Restglieder. Die Fassung (10.8.3) mit der Zwischenstelle heißt *Lagrange-Restglied*; sie ist eine Gleichung und liefert konkrete Schranken, sobald wir $f^{(k+1)}$ abschätzen können. Die Fassung (10.8.2) mit $o(|h|^k)$ sagt nur etwas über das Verhalten für $h \rightarrow 0$, dafür kommt sie mit weniger Voraussetzungen aus. Für uns ist die o -Fassung die Arbeitsform, weil sie sich auf normierte Vektorräume mit vollständigem Zielraum überträgt.

Warum $|h|^k$ und nicht $|h|^{k+1}$. Der Rest ist nach (10.8.3) von der Größenordnung $|h|^{k+1}$, also sogar noch kleiner, als (10.8.2) behauptet. Die schwächere Aussage steht dort, weil sie ohne die Zusatzvoraussetzung „ $(k+1)$ -mal stetig differenzierbar“ auskommt und weil genau sie die Bauform von Definition 10.1.5 hat: Näherung plus Rest, der schneller verschwindet als die letzte mitgenommene Potenz.

Der Grenzfall $k = 1$. Dann ist (10.8.2) wörtlich $f(x+h) = f(x) + f'(x)h + o(|h|)$, also Aussage (3) aus Satz 10.1.3. Die Taylorentwicklung verallgemeinert die Ableitungsdefinition, sie ersetzt sie nicht.

Bemerkung 10.8.4 (Die zweite Schreibweise)

Statt Entwicklungspunkt und Zuwachs zu trennen, wird das Taylorpolynom oft als Funktion der Auswertungsstelle geschrieben. Mit x_0 als Entwicklungspunkt und $h = x - x_0$ wird aus (10.8.1)

$$T_k(x) = \sum_{j=0}^k \frac{1}{j!} f^{(j)}(x_0) (x - x_0)^j, \quad f(x) = T_k(x) + o(|x - x_0|^k) \quad \text{für } x \rightarrow x_0.$$

Beide Fassungen sind gebräuchlich. Wir benutzen die h -Fassung, wenn es um die Struktur „Funktion gleich Näherung plus Rest“ geht, und die x_0 -Fassung, wenn wir das Polynom zeichnen wollen.

Bemerkung 10.8.5 (Höher ist nicht automatisch besser)

Oft heißt es: je höher der Grad k , desto besser und globaler die Approximation. Als Faustregel für gutartige Funktionen stimmt das, und (10.8.3) erklärt auch, warum: Der Faktor $|h|^{k+1}/(k+1)!$ wird mit wachsendem k rasch klein. Eine Garantie ist es nicht, denn der andere Faktor $f^{(k+1)}(\xi)$ kann mindestens ebenso rasch wachsen.

Das Standardbeispiel ist $f(x) = 1/(1+x^2)$ mit Entwicklungspunkt 0. Bei $x = 0,5$ läuft alles wie erhofft, die Fehler der Ordnungen $k = 2, 4, 8, 16$ sind $5,0 \cdot 10^{-2}$, $1,3 \cdot 10^{-2}$, $7,8 \cdot 10^{-4}$ und $3,1 \cdot 10^{-6}$. Bei $x = 1,5$ dagegen wachsen sie: 1,6, 3,5, 17,7 und 455. Die Taylorreihe dieser Funktion konvergiert nur für $|x| < 1$, obwohl f auf ganz $\mathbb{R}$ beliebig oft differenzierbar ist. Höhere Ordnung hilft also lokal, nicht global.

Beispiel 10.8.6 (Taylorapproximation der Exponentialfunktion)

Wir entwickeln $f(x) = e^x$ um $x_0 = 0$ bis zur Ordnung $k = 3$ und messen den Fehler bei $x = 0,5$.

Ableitungen. Für die Exponentialfunktion ist $f^{(j)}(x) = e^x$ für jedes $j \geq 0$, im Entwicklungspunkt also $f^{(j)}(0) = e^0 = 1$. Alle Koeffizienten sind damit $1/j!$.

Die Polynome. In der Schreibweise von Bemerkung 10.8.4 mit $x_0 = 0$ lesen wir aus (10.8.1) ab:

$$T_1(x) = 1 + x,$$

$$T_2(x) = 1 + x + \frac{1}{2!}x^2 = 1 + x + \frac{x^2}{2},$$

$$T_3(x) = 1 + x + \frac{x^2}{2} + \frac{1}{3!}x^3 = 1 + x + \frac{x^2}{2} + \frac{x^3}{6}.$$

Auswertung bei $x = 0,5$. Der wahre Wert ist $e^{0,5} \approx 1,6487$, und die drei Polynome liefern

$$T_1(0,5) = 1 + 0,5 = 1,5, \quad |f(0,5) - T_1(0,5)| \approx 0,1487,$$

$$T_2(0,5) = 1 + 0,5 + \frac{0,25}{2} = 1,625, \quad |f(0,5) - T_2(0,5)| \approx 0,0237,$$

$$T_3(0,5) = 1 + 0,5 + 0,125 + \frac{0,125}{6} \approx 1,6458, \quad |f(0,5) - T_3(0,5)| \approx 0,0029.$$

Jede Ordnung drückt den Fehler also auf etwa ein Sechstel bis ein Achtel. Das Lagrange-Restglied erklärt, warum: Der Rest ist $e^\xi \cdot 0,5^{k+1}/(k+1)!$ mit einem $\xi \in (0; 0,5)$, beim Übergang von k auf $k+1$ kommt also im Wesentlichen der Faktor $0,5/(k+2)$ dazu; die Zwischenstelle ξ verschiebt sich dabei mit, weshalb die gemessenen Quotienten 6,3 und 8,2 die vorhergesagten 6 und 8 nur ungefähr treffen. Für $k = 1$ lässt sich das ξ sogar ausrechnen, indem wir (10.8.3) nach ihm auflösen: Aus $e^{0,5} - 1,5 = e^\xi \cdot 0,25/2$ folgt $\xi = 0,1738$, und das liegt wie versprochen zwischen 0 und 0,5.

Beispiel 10.8.6 hat den Fehler an einer einzigen Stelle für drei Ordnungen ausgerechnet. Lohnt sich der Sprung von T_2 auf T_3 überhaupt, und gilt der Gewinn auch weit weg vom Entwicklungspunkt?

Interaktiv: Drei Ordnungen an einem Regler

Die drei Näherungen T_1 , T_2 und T_3 derselben Funktion hängen hier an einem Regler, dazu kommt ein zweiter für die Stelle, an der wir den Fehler ablesen. Voreingestellt ist $x = 0,5$, also genau die Stelle aus Beispiel 10.8.6.

Interaktives Element — nur in der Web-Fassung

Nahe am Entwicklungspunkt ist der Gewinn beträchtlich: Bei $x = 0,5$ fällt der Fehler von T_2 auf T_3 um den Faktor 8,2, von T_3 auf T_4 sogar um 10,2. Weiter draußen kippt das Bild, und der linke Rand bleibt hartnäckig: Für $x \rightarrow -\infty$ geht e^x gegen null, jedes Taylorpolynom positiven Grades dagegen gegen $\pm\infty$. Bei $x = -3$ wächst der Fehler beim Schritt von T_0 auf T_1 deshalb von 0,950 auf 2,050, statt zu fallen. Der Faustwert $|x|/(k+1)$ aus dem Restglied beschreibt eben das Verhalten für kleine $|x|$; solange $|x|$ groß gegen $k+1$ ist, gewinnt zunächst die Potenz. Das ist Bemerkung 10.8.5 in Zahlen.

Der allgemeine Fall

Satz 10.8.2 lebt vom Rechnen mit Potenzen h^j , und die gibt es im $\mathbb{R}^n$ oder in einem Funktionenraum nicht. Die höheren Ableitungen aus Abschnitt 10.7 liefern den Ersatz: $D_x^j f$ ist eine multilineare Abbildung, die j Zuwächse frisst. Setzen wir überall denselben Zuwachs h ein, so entsteht der

passende Ersatz für den Term $f^{(j)}(x)h^j$.

Satz 10.8.7 (Taylorentwicklung II)

Seien V ein normierter Vektorraum, W ein Banachraum, $U \subseteq V$ offen, $k \geq 1$ und $f : U \rightarrow W$ in einer Umgebung von $\mathbf{x} \in U$ k -mal stetig Fréchet-differenzierbar; kurz $f \in \mathcal{C}^k$. Dann gibt es einen Rest $r_k(\mathbf{h}) \in W$ mit

$$f(\mathbf{x} + \mathbf{h}) = f(\mathbf{x}) + \sum_{j=1}^k \frac{1}{j!} D_{\mathbf{x}}^j f(\underbrace{\mathbf{h}, \dots, \mathbf{h}}_{j\text{-mal}}) + r_k(\mathbf{h}), \quad \frac{\|r_k(\mathbf{h})\|}{\|\mathbf{h}\|^k} \rightarrow 0 \quad \text{für } \mathbf{h} \rightarrow \mathbf{0}. \quad (10.8.5)$$

Vertiefung: Der Beweis: von der Verbindungsstrecke zum Integralrestglied

Beweis.

- (1) Für hinreichend kleines $\mathbf{h}$ liegt die Strecke von $\mathbf{x}$ nach $\mathbf{x} + \mathbf{h}$ in U . Wir laufen sie mit einer Funktion einer reellen Variablen ab:

$$\psi : [0, 1] \rightarrow W, \quad \psi(t) := f(\mathbf{x} + t\mathbf{h}).$$

Damit ist $\psi(0) = f(\mathbf{x})$ und $\psi(1) = f(\mathbf{x} + \mathbf{h})$: Die ganze Frage ist auf die Verbindungsstrecke zurückgeführt

- (2) Die Kettenregel aus Satz 10.6.7 liefert die Ableitungen von ψ . Die innere Abbildung $t \mapsto \mathbf{x} + t\mathbf{h}$ ist affin mit Ableitung $\mathbf{h}$, also setzt jedes weitere Ableiten ein $\mathbf{h}$ mehr ein:

$$\psi^{(j)}(t) = D_{\mathbf{x}+t\mathbf{h}}^j f(\underbrace{\mathbf{h}, \dots, \mathbf{h}}_{j\text{-mal}}).$$

Induktiv: $t \mapsto D_{\mathbf{x}+t\mathbf{h}}^j f(\mathbf{h}, \dots, \mathbf{h})$ ist wieder eine Verkettung, deren Ableitung D^{j+1} an einem zusätzlichen $\mathbf{h}$ ausgewertet

- (3) Für eine W -wertige $\mathcal{C}^k$ -Kurve gilt die Taylorformel mit Integralrest:

$$\psi(1) = \sum_{j=0}^{k-1} \frac{1}{j!} \psi^{(j)}(0) + \frac{1}{(k-1)!} \int_0^1 (1-t)^{k-1} \psi^{(k)}(t) dt.$$

Wir addieren und subtrahieren $\psi^{(k)}(0)$ im Integral. Weil $\int_0^1 (1-t)^{k-1} dt = 1/k$, entsteht daraus genau der fehlende Summand $\psi^{(k)}(0)/k!$ und der Rest

$$r_k(\mathbf{h}) = \frac{1}{(k-1)!} \int_0^1 (1-t)^{k-1} (D_{\mathbf{x}+t\mathbf{h}}^k f - D_{\mathbf{x}}^k f)(\mathbf{h}, \dots, \mathbf{h}) dt.$$

Das Integral ist für stetige Kurven in einem Banachraum definiert; in endlichdimensionalen Zielräumen wird einfach komponentenweise integriert

- (4) Für kleines $s > 0$ mit $\{\mathbf{z} : \|\mathbf{z} - \mathbf{x}\| \leq s\} \subseteq U$ setzen wir

$$\omega(s) := \sup_{\|\mathbf{z}-\mathbf{x}\| \leq s} \|D_{\mathbf{z}}^k f - D_{\mathbf{x}}^k f\|_{\text{op}}.$$

Die Stetigkeit von $D^k f$ in $\mathbf{x}$ liefert $\omega(s) \rightarrow 0$. Mit der Operatornorm und der Dreiecksungleichung für Integrale folgt

$$\|r_k(\mathbf{h})\| \leq \frac{\omega(\|\mathbf{h}\|)}{k!} \|\mathbf{h}\|^k.$$

Nach Division durch $\|\mathbf{h}\|^k$ geht die rechte Seite gegen null. Das beweist (10.8.5).

Die Operatornorm schätzt den Integranden gleichzeitig für jede Richtung ab; das Integral der Gewichtsfunktion liefert den Faktor $1/k$

□

Bemerkung 10.8.8 (Warum hier die Integralform steht)

Das Lagrange-Restglied aus Satz 10.8.2 besitzt eine Zwischenstelle ξ und ist ein skalares Resultat. Für eine vektorwertige Funktion können verschiedene Komponenten verschiedene Zwischenstellen benötigen; in einem allgemeinen Banachraum gibt es gar keine komponentenweise Formulierung. Die Integralform umgeht dieses Problem und liefert zusammen mit der Operatornorm unmittelbar eine Schranke, die für alle Richtungen zugleich gilt. Genau deshalb reicht $f \in \mathcal{C}^k$, während das Restglied trotzdem $o(\|\mathbf{h}\|^k)$ ist.

Gradient, Hesse-Matrix und die ersten drei Ordnungen

Für den wichtigsten Spezialfall $f: \mathbb{R}^n \rightarrow \mathbb{R}$ tragen die ersten Ableitungen aus Abschnitt 10.7 vertraute Namen, und (10.8.5) wird zu einer Formel, die sich merken lässt.

Korollar 10.8.9 (Taylorapproximation für Vektor zu Skalar)

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ in einer Umgebung von $\mathbf{x}$ dreimal stetig differenzierbar, sei $\nabla f(\mathbf{x}) \in \mathbb{R}^{1 \times n}$ der Gradient und $\mathbf{H}_f(\mathbf{x}) \in \mathbb{R}^{n \times n}$ die Hesse-Matrix aus Definition 10.7.3. Dann gilt für $\mathbf{h} \rightarrow \mathbf{0}$

$$\begin{aligned} f(\mathbf{x} + \mathbf{h}) &= f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + o(\|\mathbf{h}\|) \\ &= f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h} + o(\|\mathbf{h}\|^2) \\ &= f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h} + \frac{1}{6} \sum_{i,j,l=1}^n \frac{\partial^3 f(\mathbf{x})}{\partial x_i \partial x_j \partial x_l} h_i h_j h_l + o(\|\mathbf{h}\|^3). \end{aligned} \quad (10.8.6)$$

Mit dem Entwicklungspunkt $\mathbf{x}_0$ und $\mathbf{h} = \mathbf{x} - \mathbf{x}_0$ lautet dieselbe Aussage

$$\begin{aligned} f(\mathbf{x}) &= f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + o(\|\mathbf{x} - \mathbf{x}_0\|) \\ &= f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \mathbf{H}_f(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + o(\|\mathbf{x} - \mathbf{x}_0\|^2). \end{aligned}$$

Die ersten beiden Zeilen kommen mit $f \in \mathcal{C}^1$ bzw. $f \in \mathcal{C}^2$ aus.

Dass hier überhaupt Gradient und Hesse-Matrix auftauchen, ist Satz 10.7.6: Er übersetzt $D_{\mathbf{x}} f(\mathbf{h})$ in $\nabla f(\mathbf{x}) \mathbf{h}$ und $D_{\mathbf{x}}^2 f(\mathbf{h}, \mathbf{h})$ in $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$, und danach ist (10.8.6) nur noch (10.8.5) in Koordinaten.

Die Formate gehen auf: Der Gradient ist ein Zeilenvektor, $\mathbf{h}$ ein Spaltenvektor, also ist $\nabla f(\mathbf{x}) \mathbf{h}$ eine Zahl. Ebenso ist $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$ eine quadratische Form und damit skalar. Ab der dritten Ordnung gibt es keine Matrixschreibweise mehr, weil $D_{\mathbf{x}}^3 f$ drei Indexpositionen hat, also ein Tensor dritter Stufe ist (Abschnitt 9.2); deshalb steht dort die ausgeschriebene Summe. Praktisch verwendet werden fast nur die ersten beiden Zeilen.

Bemerkung 10.8.10 (Ebene und Quadrik)

Geometrisch sind die beiden Näherungen leicht zu benennen. Der Graph von T_1 ist die *Tangentialebene* an den Graphen von f im Punkt $(\mathbf{x}, f(\mathbf{x}))$; solange $\nabla f(\mathbf{x}) \neq \mathbf{0}^\top$ ist, sind ih-

re Höhenlinien parallele Geraden senkrecht zum Gradienten (Abschnitt 10.2), im kritischen Punkt dagegen ist T_1 konstant. Seine Niveaumenge zum konstanten Wert ist dann der ganze Definitionsraum, alle anderen sind leer; reguläre Höhenlinien mit einer ausgezeichneten Gradientenrichtung gibt es nicht. Der Graph von T_2 ist eine Quadrik, und ihre Höhenlinien verraten die Definitheit der Hesse-Matrix: Ellipsen bei definit, Hyperbeln bei indefinit Hesse-Matrix. Damit ist der Anschluss an Satz 10.7.9 hergestellt, wo dieselbe Fallunterscheidung über Minimum, Maximum und Sattelpunkt entscheidet, sofern die Hesse-Matrix definit oder indefinit ist (Bemerkung 10.7.10). Genauer ist die zweite Zeile von (10.8.6) genau das Werkzeug, das der Beweis dieses Satzes braucht: Im kritischen Punkt fällt der lineare Term weg, und weil der rote Rest schneller verschwindet als $\|\mathbf{h}\|^2$, entscheidet für kleine $\mathbf{h}$ das Vorzeichen von $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h}$.

Ein Sonderfall ist lehrreich: Ist f selbst ein Polynom zweiten Grades, so verschwinden alle Ableitungen ab der dritten, das Restglied ist null und $T_2 = f$ gilt exakt, an jeder Stelle und für jedes $\mathbf{h}$. Auf dieser Beobachtung beruht das Verfahren im nächsten Unterabschnitt.

Korollar 10.8.9 macht eine quantitative Aussage: Der Restterm von T_1 wächst wie $\|\mathbf{h}\|^2$, der von T_2 wie $\|\mathbf{h}\|^3$. Halbieren wir also den Abstand zum Entwicklungspunkt, um welchen Faktor fällt dann der Fehler?

Interaktiv: Tangentialebene und Quadrik im Höhenlinienbild

Statt nur der Flächen zeichnen wir die Höhenlinien, und zwar die von f und die des Taylorpolynoms im selben Fenster und auf denselben Niveaus; daneben steht dieselbe Lage im Raum. Gemessen wird der größte Fehler auf einem Kreis mit Radius r um den Entwicklungspunkt, im Vergleich zu dem auf dem halb so großen Kreis. In der Raumtafel ist dieser Fehler der senkrechte Abstand zwischen dem blauen und dem grünen Ring.

Dieselbe Idee zeigt außerdem eine Shiny-App mit Flächen im Raum.

Interaktives Element — nur in der Web-Fassung

Die Vorhersage trifft: In der Voreinstellung fällt der Fehler von T_1 beim Halbieren des Radius um den Faktor 4,34 und der von T_2 um 8,07, also ungefähr um 4 und 8. Die Umschaltung zwischen den beiden Ordnungen zeigt zugleich den strukturellen Unterschied: Die Tangentialebene liefert parallele Geraden als Höhenlinien, die Quadrik krümmt sich mit. Die Faustzahlen setzen allerdings voraus, dass der jeweils führende Restterm nicht selbst verschwindet; wo die Hesse-Matrix nahezu null ist, springt der T_1 -Quotient auf etwa 8. Auf der Quadrik im dritten Menüpunkt fällt der T_2 -Fehler auf Rundungsniveau: Dort ist die Näherung exakt, und genau darauf beruht der eine exakte Newton-Schritt im nächsten Unterabschnitt.

Anwendung: das Newton-Raphson-Verfahren

Gesucht ist ein lokales Minimum, sofern eines existiert, für eine zweimal stetig differenzierbare Funktion $f: \mathbb{R}^n \rightarrow \mathbb{R}$. Direkt lösen können wir das im Allgemeinen nicht. Für eine quadratische Funktion mit positiv definiten Hesse-Matrix dagegen schon: Dort ist die Bedingung „Gradient gleich null“ ein lineares Gleichungssystem, und lineare Gleichungssysteme lösen wir seit Kapitel 5. Die Idee des Verfahrens des Newton-Raphson ist deshalb, an der aktuellen Stelle $\mathbf{x}^{(k)}$ die quadratische Näherung aus Korollar 10.8.9 hinzulegen und ihren stationären Punkt zu bestimmen:

$$f(\mathbf{x}) \approx T_2(\mathbf{x}) = f(\mathbf{x}^{(k)}) + \nabla f(\mathbf{x}^{(k)}) (\mathbf{x} - \mathbf{x}^{(k)}) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}_f(\mathbf{x}^{(k)}) (\mathbf{x} - \mathbf{x}^{(k)}).$$

Den Gradienten dieser Funktion kennen wir aus Abschnitt 10.6: Der lineare Term steuert $\nabla f(\mathbf{x}^{(k)})$ bei, und die quadratische Form hat nach Beispiel 10.6.5 den Gradienten $(\mathbf{x} - \mathbf{x}^{(k)})^\top (\mathbf{H}_f + \mathbf{H}_f^\top)/2$, was wegen der Symmetrie der Hesse-Matrix (Satz von Schwarz, Satz 10.7.4) gerade $(\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}_f(\mathbf{x}^{(k)})$ ist. Nullsetzen liefert

$$\begin{aligned} \nabla T_2(\mathbf{x}) &= \nabla f(\mathbf{x}^{(k)}) + (\mathbf{x} - \mathbf{x}^{(k)})^\top \mathbf{H}_f(\mathbf{x}^{(k)}) \stackrel{!}{=} \mathbf{0}^\top \\ \implies (\mathbf{x} - \mathbf{x}^{(k)})^\top &= -\nabla f(\mathbf{x}^{(k)}) \mathbf{H}_f(\mathbf{x}^{(k)})^{-1}. \end{aligned}$$

Der letzte Schritt setzt voraus, dass $\mathbf{H}_f(\mathbf{x}^{(k)})$ invertierbar ist; sonst hat T_2 entweder gar keinen kritischen Punkt oder einen affinen Lösungsraum positiver Dimension. Ansonsten sind beide Seiten Zeilenvektoren, wie es die Konvention aus Abschnitt 10.2 verlangt. Transponieren bringt uns zurück zu Spalten, und damit steht das Verfahren da.

Algorithmus 10.8.11 (Newton-Raphson-Verfahren)

Gegeben seien $f : \mathbb{R}^n \rightarrow \mathbb{R}$ zweimal stetig differenzierbar und ein Startpunkt $\mathbf{x}^{(0)}$. Für $k = 0, 1, 2, \dots$ wiederhole, solange $\mathbf{H}_f(\mathbf{x}^{(k)})$ invertierbar ist:

1. Berechne $\nabla f(\mathbf{x}^{(k)})$ und $\mathbf{H}_f(\mathbf{x}^{(k)})$.
2. Setze

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left(\nabla f(\mathbf{x}^{(k)}) \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \right)^\top. \quad (10.8.7)$$

3. Brich ab, wenn $\|\nabla f(\mathbf{x}^{(k)})\|$ oder die Schrittlänge $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$ unter eine vorgegebene Schranke fällt.

Bemerkung 10.8.12 (Drei Vorbehalte)

Der Schritt sucht kritische Punkte, keine Minima. Konstruiert wurde (10.8.7) als Nullstelle von ∇T_2 . Ein solcher Punkt ist ein Minimum der Näherung nur dann, wenn $\mathbf{H}_f(\mathbf{x}^{(k)})$ positiv definit ist. Bei indefiniter beziehungsweise negativ definiter Hesse-Matrix ist der stationäre Punkt des lokalen quadratischen Modells ein Sattel beziehungsweise Maximum, und der Schritt muss keine Abstiegsrichtung sein. Das klassische Newton-Verfahren sucht Nullstellen des Gradienten und kann deshalb je nach Startpunkt zu Minima, Maxima oder Sattelpunkten konvergieren. Praktische Varianten erzwingen deshalb Definitheit, etwa indem sie ein Vielfaches der Einheitsmatrix addieren.

Die Inverse ist eine Schreibweise, keine Rechenanweisung. In (10.8.7) steht $\mathbf{H}_f^{-1}$, gerechnet wird aber das Gleichungssystem $\mathbf{H}_f(\mathbf{x}^{(k)}) \mathbf{d} = -\nabla f(\mathbf{x}^{(k)})^\top$ mit anschließendem $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}$. Das kostet weniger und ist stabiler (Abschnitt 5.3); bei positiv definiter Hesse-Matrix ist die Cholesky-Zerlegung aus Abschnitt 5.4 das Mittel der Wahl, und ihr Gelingen ist zugleich die Definitheitsprobe.

Nur lokal. Weit weg vom Ziel kann die quadratische Näherung so schlecht sein, dass der Schritt in die falsche Richtung zeigt. Die Gegenmittel heißen Schrittweitensteuerung und Trust-Region und gehören in ein eigenes Kapitel über Optimierung, das dieses Skript später aufmacht.

Bemerkung 10.8.13 (Warum Statistik und ML voll davon sind)

Die Maximum-Likelihood-Schätzung maximiert die Log-Likelihood $\ell(\theta)$, sucht also eine Nullstelle der Score-Funktion $\nabla \ell(\theta)$. Genau dafür ist (10.8.7) gebaut, und die dabei gebrauchte Hesse-Matrix ist bis aufs Vorzeichen die beobachtete Fisher-Information aus Definition 10.7.14. Ersetzen wir sie durch ihren Erwartungswert, so heißt das Verfahren *Fisher-Scoring*; für verallgemeinerte lineare Modelle fällt es mit den *iterativ gewichteten kleinsten Quadraten* (IRLS) zusammen, in denen jeder Schritt ein gewichtetes Kleinste-Quadrate-Problem im Sinne von Kapitel 7 ist.

Vertiefung: Die Asymptotik des ML-Schätzers als Taylorentwicklung

Auch die Asymptotik selbst ist eine Taylorentwicklung. Entwickeln wir die Score um den wahren Parameter θ und werten sie am Schätzer $\hat{\theta}$ aus, wo sie verschwindet, so bleibt in erster Ordnung $0^\top \approx \nabla \ell(\theta) + (\hat{\theta} - \theta)^\top \mathbf{H}_\ell(\theta)$ stehen. Umstellen und Normieren macht daraus unter den üblichen Regularitätsbedingungen die bekannte Aussage $\sqrt{n}(\hat{\theta} - \theta) \rightarrow N(0, \mathbf{I}_1(\theta)^{-1})$: Der Faktor $n^{-1/2} \nabla \ell(\theta)^\top$ ist eine normierte Summe unabhängiger Beiträge und wird nach dem zentralen Grenzwertsatz normal, und $-\mathbf{H}_\ell(\theta)/n$ konvergiert gegen die Fisher-Information $\mathbf{I}_1(\theta)$ einer einzelnen Beobachtung. Der Restterm aus (10.8.6) ist dabei das, was die Argumentation auf eine asymptotische festlegt.

Wie schnell ist „quadratisch konvergent“ in Zahlen? Rechnen wir das Verfahren Schritt für Schritt mit und schauen, wie viele Schritte es bis zur Maschinengenauigkeit braucht.

Interaktiv: Newton Schritt für Schritt

Die Tafel zeigt die Höhenlinien, den Weg der Iterierten und eine Tabelle mit Gradientennorm, Fehler und dem Quotienten e_k/e_{k-1}^2 . Bleibt dieser Quotient beschränkt, so wird der Fehler in jedem Schritt im Wesentlichen quadriert; das ist die *quadratische Konvergenz*, die Newton so beliebt macht (Konvergenzordnung). Der Schrittreger lässt sich vor- und zurückfahren, ohne dass die Iteration neu startet.

Voreingestellt ist $f(\mathbf{x}) = x_1^3/3 - x_1 + x_2^2/2$ mit Start $(2; 1,5)$. Drei Versuche lohnen sich: ein Startpunkt mit $x_1 < 0$, ein Startpunkt mit $x_1 = 0$, und der zweite Menüpunkt, die Quadrik aus Bemerkung 10.8.10.

Interaktives Element — nur in der Web-Fassung

In der x_2 -Richtung sitzt der erste Schritt sofort exakt, denn dort ist f quadratisch. In der x_1 -Richtung läuft die Iteration über 1,25, 1,025, 1,00030 und 1,00000005 gegen 1, mit den Fehlern 0,25, 0,025, $3,05 \cdot 10^{-4}$ und $4,65 \cdot 10^{-8}$; im fünften Schritt steht $1,1 \cdot 10^{-15}$. Fünf Schritte für fünfzehn Stellen: Der Fehler wird von Schritt zu Schritt im Wesentlichen quadriert, der Quotient e_k/e_{k-1}^2 läuft gegen $1/(2x_1^*) = 0,5$. Die drei Versuche zeigen ebenso schnell die Grenzen: Ein Start mit $x_1 < 0$ führt in den Sattelpunkt $(-1; 0)$ statt ins Minimum (Bemerkung 10.8.12), bei $x_1 = 0$ ist die Hesse-Matrix $\text{diag}(0, 1)$ singulär und der Schritt gar nicht definiert, und auf der Quadrik ist nach einem einzigen Schritt Schluss, weil T_2 dort mit f übereinstimmt.

Selbsttest

1. Im Restglied von Satz 10.8.2 steht die $(k + 1)$ -te Ableitung an der Entwicklungsstelle, also $f^{(k+1)}(x)$.

Antwort: Falsch

Sie steht an einer Zwischenstelle ξ zwischen x und $x + h$, die der Mittelwertsatz liefert und über die wir sonst nichts wissen. Stünde dort $f^{(k+1)}(x)$, so wäre der Rest ein exakt bekannter Ausdruck und der Mittelwertsatz im Beweis überflüssig. Bekannt ist nur eine Schranke für $f^{(k+1)}$ in einer Umgebung, und genau daraus wird die o -Aussage.

2. Je größer der Grad k , desto kleiner der Fehler, gleichgültig an welcher Stelle wir auswerten.

Antwort: Falsch

Lokal stimmt die Faustregel, global nicht. Für $f(x) = 1/(1+x^2)$ mit Entwicklungspunkt 0 fallen die Fehler bei $x = 0,5$ von $5,0 \cdot 10^{-2}$ auf $3,1 \cdot 10^{-6}$, wenn wir von $k = 2$ auf $k = 16$ gehen, bei $x = 1,5$ wachsen sie im selben Schritt von 1,6 auf 455 (Bemerkung 10.8.5). Im Lagrange-Restglied kann der Faktor $f^{(k+1)}(\xi)$ schneller wachsen, als $(k+1)!$ ihn drückt.

3. Ist f ein Polynom zweiten Grades, so gilt $T_2 = f$ überall, nicht nur näherungsweise.

Antwort: Wahr

Alle Ableitungen ab der dritten sind null, also verschwindet das Restglied (10.8.3) für $k = 2$ identisch. Bei invertierbarer Hesse-Matrix trifft der Newton-Schritt deshalb den eindeutigen kritischen Punkt in einem Zug. Ein Minimum ist dieser nur bei positiver Definitheit, wie im zweiten Menüpunkt des Widgets.

4. Der Newton-Schritt (10.8.7) liefert stets ein lokales Minimum von f .

Antwort: Falsch

Er liefert einen kritischen Punkt der quadratischen Näherung, mehr nicht. Ist die Hesse-Matrix indefinit, so ist dieser Punkt ein Sattel; im Widget läuft jeder Start mit $x_1 < 0$ zielstrebig nach $(-1; 0)$, wo der Gradient verschwindet und die Hesse-Matrix $\text{diag}(-2, 1)$ indefinit ist. Erst positive Definitheit macht aus dem kritischen Punkt ein Minimum (Bemerkung 10.8.12).

5. Halbieren wir $\|\mathbf{h}\|$, so fällt der Fehler der Tangentialebene auf etwa ein Viertel, der Fehler der quadratischen Näherung auf etwa ein Achtel.

Antwort: Wahr

Nach Korollar 10.8.9 ist der erste Rest von der Größenordnung $\|\mathbf{h}\|^2$ und der zweite von der Größenordnung $\|\mathbf{h}\|^3$. Das Widget misst diesen Quotienten auf einem Kreis: In der Voreinstellung stehen dort 4,34 und 8,07 für $\sin(x_1) + \cos(x_2)$, für die Glockenfunktion an derselben Stelle 3,99 und 8,78.

6. Der Beweis von Satz 10.8.7 überträgt die eindimensionale Aussage einfach auf jede Gerade durch $\mathbf{x}$, mehr ist nicht zu zeigen.

Antwort: Falsch

Der Kern ist das, aber die Aussage $o(\|\mathbf{h}\|^k)$ verlangt Kleinheit gleichmäßig über alle Richtungen, während die Entwicklung von ψ zunächst nur für eine feste Richtung $\mathbf{u}$ gilt. Die Stetigkeit der k -ten Ableitung liefert eine gemeinsame Normschranke für alle Richtungen und damit die benötigte gleichmäßige Kleinheit (Herleitung in der Vertiefung Bemerkung 10.8.8).

7. Fahren wir den Schrittreger im Newton-Widget von der Voreinstellung aus hoch. In welchem Schritt unterschreitet der Abstand zum Minimum zum ersten Mal 10^{-10} ?

Lösung: 5

In Schritt 5. Die Fehlerspalte fällt von $4,65 \cdot 10^{-8}$ auf $1,1 \cdot 10^{-15}$, überspringt also sieben Größenordnungen auf einmal. Das ist quadratische Konvergenz in Zahlen: Sobald der Fehler klein genug ist, verdoppelt sich die Zahl der gültigen Stellen mit jedem Schritt.

Vertiefung: MML §5.1.1 behandelt die Taylorreihe im eindimensionalen Fall, §5.8 die Linearisierung und die mehrdimensionale Taylorentwicklung samt der Tensorschreibweise für die höheren Terme.

10.9 Zusammenfassung

Ein einziger Gedanke trägt dieses Kapitel: Ableiten heißt linear approximieren. Alles Weitere war Buchführung darüber, in welchem Format die lineare Näherung steht und nach welchen Regeln sich mit ihr rechnen lässt. Sammeln wir ein, was bleiben soll, und sagen wir dazu, was wir bewusst liegengelassen haben.

Die Kernkonzepte

Die ersten vier Abschnitte haben geklärt, was eine Ableitung überhaupt ist, sobald Ein- und Ausgabe keine Zahlen mehr sind.

Bemerkung 10.9.1 (Fünf Begriffe, die bleiben)

1. **Die Fréchet-Ableitung** ist eine beschränkte lineare Abbildung $D_x f$ mit $f(x+h) = f(x) + D_x f(h) + o(\|h\|)$ (Definition 10.1.5). Linear ist die Abhängigkeit von h , nicht die von der Stelle x (Bemerkung 10.1.4).
2. **Der Gradient** sammelt die partiellen Ableitungen einer skalarwertigen Funktion als Zeile (Definition 10.2.1); transponiert zeigt er in die Richtung des stärksten Anstiegs (Satz 10.2.4), und daran hängt der Gradientenabstieg (Algorithmus 10.2.10).
3. **Die Jacobimatrix** stapelt diese Zeilen (Definition 10.3.1); vier Bausteine (Satz 10.3.4) und die Kettenregel als Matrixprodukt (Satz 10.3.9) reichen für fast jede Rechnung dieses Kapitels.
4. **Skalar zu Matrix** heißt: jeden Eintrag einzeln ableiten und stehen lassen, wo er steht (Definition 10.4.1); für Spur, Determinante und Inverse eines quadratischen $F(x)$ gibt es geschlossene Formeln (Satz 10.4.4).
5. **Matrix zu Skalar** läuft ebenfalls eintragsweise, und der Ableitungsterm ist das Frobenius-Skalarprodukt (Definition 10.4.7, Bemerkung 10.4.8); drei Identitäten decken die häufigsten Fälle ab (Satz 10.4.10).

Zwei Anordnungen laufen dabei nebeneinander her. Solange nur Skalare und Vektoren beteiligt sind, gilt: pro Ausgabekomponente eine Zeile, pro Eingabevariable eine Spalte. Sobald eine Matrix als Eingabe oder als Ausgabe auftritt, sortieren wir die Ableitungen dagegen so, dass ihr Format erhalten bleibt (Bemerkung 10.4.2); der Ableitungsterm entsteht dann nicht als Matrixprodukt, sondern im Fall Matrix zu Skalar über die Spur (Bemerkung 10.4.8). Einzeln ausrechnen müssen wir die partiellen Ableitungen nur dann, wenn wir die lineare Näherung nicht schon auf anderem Weg haben. Finden wir irgendeine Matrix M mit $f(x+h) = f(x) + Mh + o(\|h\|)$, so ist sie bereits die Jacobimatrix (Lemma 10.3.3). Ausmultiplizieren ist oft kürzer als Ableiten.

Die zweite Kapitelhälfte hat geklärt, wie wir mit diesem Begriff rechnen. Vier Bausteine sind dabei entstanden, und alle vier gehen auf dieselbe Gleichung zurück, auf die lineare Approximation aus Definition 10.1.5.

Bemerkung 10.9.2 (Vier Bausteine, die bleiben)

1. **Eigenschaften** (Abschnitt 10.5). Differenzierbar zieht stetig nach sich (Satz 10.5.2), umgekehrt nicht (Beispiel 10.5.3). Die Zuordnung $f \mapsto D_{\mathbf{x}}f$ ist linear (Satz 10.5.5), in jedem der fünf Formate (Bemerkung 10.5.6).
2. **Rechenregeln** (Abschnitt 10.6). Die Produktregel gilt für jede beschränkte bilineare Multiplikation (Satz 10.6.3), die Kettenregel ist ohne Koordinaten schlicht eine Verketten (Satz 10.6.7) und in Koordinaten ein Matrixprodukt (Satz 10.3.9).
3. **Ableitungen höheren Grades** (Abschnitt 10.7). Die j -te Ableitung ist die lineare Näherung an die Änderung der $(j-1)$ -ten (Definition 10.7.1). Für $f \in \mathcal{C}^2$ mit $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ist die zweite Stufe die Hesse-Matrix (Definition 10.7.3, Satz 10.7.6); sie ist symmetrisch (Satz 10.7.4), ihre Definitheit entscheidet im kritischen Punkt über Minimum, Maximum und Sattel (Satz 10.7.9) und überall über Konvexität (Satz 10.7.11). In der Statistik steht dort die Fisher-Information (Definition 10.7.14).
4. **Taylorapproximation** (Abschnitt 10.8). Das Taylorpolynom schmiegt sich bis zur k -ten Ableitung an (Definition 10.8.1); für den Fehler gibt es das Lagrange-Restglied und die Ordnungsaussage (Satz 10.8.2), und dieselbe Aussage trägt bis in Banachräume (Satz 10.8.7). Für $f: \mathbb{R}^n \rightarrow \mathbb{R}$ stehen in den ersten beiden Ordnungen Gradient und Hesse-Matrix (Korollar 10.8.9), geometrisch Tangentialebene und Quadrik (Bemerkung 10.8.10); der stationäre Punkt der Quadrik ist der Newton-Schritt (Algorithmus 10.8.11, Bemerkung 10.8.12).

Die meisten Beweise dieser vier Abschnitte folgen demselben Muster. Wir setzen die Definition für jeden Baustein einzeln ein, sortieren die entstehenden Terme nach Bauart und zeigen, dass alles außerhalb des Kandidaten $o(\|\mathbf{h}\|)$ ist. So entstehen Satz 10.5.5, die Produktregel, die Kettenregel und die Hesse-Formel in Satz 10.7.6. Satz 10.5.2 kommt mit weniger aus, denn dort gibt es gar keinen Kandidaten: Wir schätzen beide Zusatzterme nach oben ab und lassen $\mathbf{h}$ gegen null gehen. Das Rechnen mit Landau-Symbolen ist deshalb kein Beiwerk, sondern das eigentliche Handwerkszeug (Bemerkung 10.5.1). Weisen wir den Kandidaten zusätzlich als linear und beschränkt nach, so ist Definition 10.1.5 erfüllt, und mehr braucht es für Produkt- und Kettenregel nicht. Zweimal kommt ein Zusatzargument dazu, im letzten Schritt zu Satz 10.5.5 und in Schritt 4 zu Satz 10.7.6: Zwei lineare Abbildungen, die sich nur um $o(\|\mathbf{h}\|)$ unterscheiden, sind gleich. Aus dem Muster fallen der Satz von Schwarz und die eindimensionale Taylorentwicklung heraus, denn beide brauchen einen Mittelwertsatz; der allgemeine Satz 10.8.7 verwendet statt einer skalaren Zwischenstelle die Integralform des Restglieds.

Vertiefung: Weitere Ableitungsformate und praktische Nachschlagewege

Was wir ausgelassen haben

Bemerkung 10.9.3 (Die drei leeren Felder)

In der Übersicht aus Abschnitt 10.2 sind drei Zellen leer geblieben: Matrix zu Vektor, Vektor zu Matrix und Matrix zu Matrix. Sie fehlen nicht, weil sie schwierig wären, sondern weil ihre Ableitungen aus dem Matrixschema herausfallen. Zählen wir nach. Eine Funktion $f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{p \times q}$ hat $mnpq$ partielle Ableitungen, jede davon adressiert durch zwei Indizes für die Eingabe und zwei für die Ausgabe. Ordentlich sortiert ergibt das ein Zahlenfeld mit vier Indexpositionen, also einen Tensor der Stufe 4 (Abschnitt 9.2). Für 2×2 -Matrizen auf beiden Seiten sind das 16 Zahlen in $\mathbb{R}^{2 \times 2 \times 2 \times 2}$. Bei den gemischten Fällen Vektor zu Matrix und Matrix zu Vektor bleiben drei Indexpositionen übrig.

Gebraucht werden diese Fälle seltener als die sechs ausgefüllten Zellen, und wo sie doch auftreten, gibt es zwei Wege. Entweder wir rechnen mit Tensoren weiter, oder wir klopfen alles flach und stapeln jede Matrix zu einem Vektor: Dafür stehen die Vektorisierung und das Kroneckerprodukt aus Abschnitt 9.5 bereit, die eine Matrixgleichung in ein gewöhnliches lineares Gleichungssystem verwandeln.

Eine zweite Lücke ist keine Formatfrage. Definition 10.1.5 lässt für $\mathbb{D}$ und $\mathbb{E}$ beliebige normierte Vektorräume zu, also auch Räume von Funktionen. Ableitungen von Abbildungen zwischen Funktionenräumen bleiben hier eine Übungsaufgabe. Der Begriff trägt dorthin, doch die Beschränktheit von $D_x f$ ist dann keine Formalie mehr, sondern eine echte Zusatzbedingung (Bemerkung 10.1.6).

Ableiten in eine Richtung

Bemerkung 10.9.4 (Störungsanalyse als Richtungsableitung)

Ein Werkzeug lohnt den Nachtrag, weil es die Matrixfälle auf den eindimensionalen zurückführt. Wollen wir wissen, wie empfindlich eine Matrixrechnung auf eine Störung reagiert, so halten wir eine Störungsrichtung $\mathbf{E}$ fest und betrachten die Funktion einer einzigen reellen Variablen

$$\varphi(h) := f(\mathbf{X} + h\mathbf{E}).$$

Ableiten nach dem Skalar h an der Stelle $h = 0$ liefert genau den Ableitungsterm in Richtung $\mathbf{E}$, für skalarwertiges f also

$$\varphi'(0) = D_{\mathbf{X}} f(\mathbf{E}) = \text{tr} \left(\left(\frac{\partial f(\mathbf{X})}{\partial \mathbf{X}} \right)^{\top} \mathbf{E} \right).$$

Nachzurechnen ist das schnell: Setzen wir $\mathbf{H} = h\mathbf{E}$ in (10.4.7) ein, so zieht die Linearität den Faktor heraus, $D_{\mathbf{X}} f(h\mathbf{E}) = h D_{\mathbf{X}} f(\mathbf{E})$, und der Restterm ist $o(|h|)$, weil $\|h\mathbf{E}\| = |h| \|\mathbf{E}\|$ gilt. Das ist dieselbe Idee wie die Richtungsableitung aus Definition 10.2.3, nur mit einer Matrix als Richtung. Anders als dort verlangen wir von $\mathbf{E}$ keine Länge 1; deshalb steht „Richtung“ hier in Anführungszeichen. Rechnen wir ein kleines Beispiel nach. Für $f(\mathbf{X}) = \|\mathbf{X}\|_F^2$ mit $\mathbf{X} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ und $\mathbf{E} = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ ist $\varphi(h) = 30 + 4h + h^2$, also $\varphi'(0) = 4$, und das ist der Eintrag (1, 2) von $2\mathbf{X}$ nach (10.4.12). Diese Wahl von $\mathbf{E}$ ist gerade das $\mathbf{E}_{ij}$ aus Bemerkung 10.4.8, das Anstupsen eines einzelnen Eintrags; beliebige $\mathbf{E}$ stupsen alle Einträge zugleich an.

Für die Praxis lesen wir das als Störungsanalyse: $\mathbf{E}$ gibt an, wohin gestört wird, und $\varphi'(0)$ misst in erster Ordnung, wie stark diese Störung im Ergebnis ankommt. Über alle Störungsrichtungen zugleich beschränken die Konditionszahlen aus Abschnitt 4.2, wie stark ein relativer Datenfehler anwachsen kann.

Zum Nachschlagen

Die Identitäten dieses Kapitels sind eine kleine Auswahl. Brauchen wir eine Ableitung, die hier nicht steht, so schlagen wir sie besser nach, statt sie neu herzuleiten. Das *Matrix Cookbook* von Petersen und Pedersen (2012) sammelt die Formeln, die in Anwendungen tatsächlich vorkommen, von Determinan-

ten und Inversen bis zu Spur- und Normausdrücken. Eine Warnung gehört dazu: Die Konventionen unterscheiden sich zwischen den Quellen, und dieselbe Formel steht je nach Zeilen- oder Spaltenkonvention transponiert (Bemerkung 10.2.2). Bei rechteckigen Formaten fällt das schon beim Nachzählen von Zeilen und Spalten auf. Bei quadratischen passt die transponierte Fassung genauso gut, und dann hilft nur eine numerische Probe an einem kleinen Beispiel.

1. Die Ableitung einer matrixwertigen Funktion nach einer Matrix passt im Allgemeinen in kein rechteckiges Zahlenschema mehr.

Antwort: Wahr

Für $f: \mathbb{R}^{m \times n} \rightarrow \mathbb{R}^{p \times q}$ entstehen $mnpq$ partielle Ableitungen mit vier Indexpositionen, also ein Tensor der Stufe 4 (Bemerkung 10.9.3). Deshalb bleiben die drei Zellen der Übersicht aus Abschnitt 10.2 leer. Wer den Fall braucht, rechnet mit Tensoren oder stapelt die Matrizen per Vektorisierung zu Vektoren um (Abschnitt 9.5).

Das Wichtigste in Kürze

Abschnitt	Thema	Was bleibt
10.1	Ableitung als lineare Approximation	Fréchet-Ableitung auf normierten Räumen: $f(x+h) = f(x) + D_x f(h) + r(h)$ mit linearem, beschränktem $D_x f$ und einem Restterm $r(h)$, der schneller verschwindet als h
10.2	Der Gradient	$\nabla f(\mathbf{x}) \in \mathbb{R}^{1 \times n}$ als Zeile der partiellen Ableitungen; Richtung des stärksten Anstiegs; Gradientenabstieg
10.3	Die Jacobimatrix	$J_f(\mathbf{x}) \in \mathbb{R}^{m \times n}$ mit Gradienten als Zeilen; Kettenregel als Matrixprodukt; Backpropagation als Auswertung von links
10.4	Ableitungen mit Matrizen	eintragsweise Ableitungen für Skalar zu Matrix und Matrix zu Skalar; Spur, Jacobi-Formel, Inverse; Frobenius-Skalarprodukt als Ableitungsterm
10.5	Stetigkeit und Linearität	Differenzierbar zieht stetig nach sich, nicht umgekehrt; die Ableitungsoperation $f \mapsto D_x f$ ist linear, in jedem der fünf Formate aus den Abschnitten 10.1 bis 10.4
10.6	Produkt- und Kettenregel	Produktregel für jede beschränkte bilineare Multiplikation $\langle \cdot, \cdot \rangle$; Kettenregel als Verkettung $D_y g \circ D_x f$, in Koordinaten ein Matrixprodukt
10.7	Ableitungen höheren Grades	$D_x^j f$ als multilineare Abbildung; Hesse-Matrix $H_f(\mathbf{x})$, Satz von Schwarz, Definitheit im kritischen Punkt trennt lokales Minimum, Maximum und Sattel, entscheidet aber nichts, wenn sie semidefinit und nicht definit ist
10.8	Taylorapproximation	$f(\mathbf{x}+\mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x})\mathbf{h} + \frac{1}{2}\mathbf{h}^\top H_f(\mathbf{x})\mathbf{h} + r(\mathbf{h})$ mit einem Rest, der schneller verschwindet als die letzte mitgenommene Potenz; Newton bestimmt den stationären Punkt der Quadrik

Nächstes Kapitel

Offen geblieben ist eine Frage, die in diesem Kapitel zweimal aufgetaucht ist, ohne dass wir sie beantwortet hätten: Wann ist ein lokaler Befund auch ein globaler? Satz 10.7.11 verknüpft die Konvexität mit der positiven Semidefinitheit der Hesse-Matrix, zieht daraus aber keinen Schluss über globale Minima. Und Bemerkung 10.8.12 nennt als ersten Vorbehalt des Newton-Verfahrens, dass ein kritischer Punkt eben noch kein Minimum ist.

Für konvexe Funktionen lautet die Antwort „immer“, und das ändert die Lage grundlegend. Jeder kritische Punkt ist dann ein globales Minimum, Sattelpunkte gibt es keine, und ein Abstiegsverfahren kann nirgends an der falschen Stelle hängenbleiben. Damit bekommen Optimierungsverfahren Garantien statt Faustregeln. Das nächste Kapitel nimmt sich die Konvexität deshalb als eigenes Thema vor.

Selbsttest

Zwei Runden. In der ersten stehen sechs Aussagen zum Ableitungsbegriff und zu den Formaten aus den Abschnitten 10.1 bis 10.4, dazu eine Frage, die sich nur an einem Widget dieser Abschnitte beantworten lässt. Welche Aussagen stimmen?

1. Für $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$ und $f : \mathbb{R}^m \rightarrow \mathbb{R}$ gilt $\nabla(f \circ g)(\mathbf{x}) = \nabla f(g(\mathbf{x})) \mathbf{J}_g(\mathbf{x})$, ganz ohne Transponierte.

Antwort: Wahr

Genau dafür ist der Gradient eine Zeile (Bemerkung 10.2.2): Das Produkt einer $1 \times m$ -Zeile mit einer $m \times n$ -Matrix ist wieder eine Zeile, und für $m = 1$ ist der Gradient der Sonderfall der Jacobimatrix (Bemerkung 10.3.2). Zahlenprobe mit $g(\mathbf{x}) = \mathbf{A}\mathbf{x}$, $\mathbf{A} = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \end{pmatrix}$ und $f(y) = \|y\|^2$: In $\mathbf{x} = (1, 2, 3)^\top$ ist $\mathbf{A}\mathbf{x} = (5, 11)^\top$, also $\nabla f(\mathbf{A}\mathbf{x}) \mathbf{A} = 2(5, 11)\mathbf{A} = (10, 42, 66)$. Direkt ausgerechnet ist $\nabla f(g(\mathbf{x})) = \mathbf{x}^\top \mathbf{A}^\top \mathbf{A} \mathbf{x}$ mit Gradient $2\mathbf{x}^\top \mathbf{A}^\top \mathbf{A}$ nach Satz 10.2.8, dessen Symmetrieforderung $\mathbf{A}^\top \mathbf{A}$ erfüllt. Das ergibt dieselbe Zeile.

2. Die Forderung in Definition 10.1.5, dass $D_{\mathbf{x}}f$ beschränkt sein soll, ist überflüssig, denn lineare Abbildungen sind ohnehin beschränkt.

Antwort: Falsch

In endlich-dimensionalen Räumen stimmt das, dort kostet die Forderung nichts (Bemerkung 10.1.6). Im Unendlichdimensionalen ist sie eine echte Zusatzbedingung. Auf dem Raum der Polynome auf $[0, 1]$ mit der Supremumsnorm etwa ist das Ableiten $p \mapsto p'$ linear, aber unbeschränkt: Für $p_n(x) = x^n$ ist $\|p_n\|_\infty = 1$, während $p'_n(x) = nx^{n-1}$ die Norm n hat. Genau deshalb bleiben Abbildungen zwischen Funktionenräumen in diesem Kapitel außen vor.

3. Für $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A}$ mit $\mathbf{A} \in \mathbb{R}^{n \times m}$ ist $\mathbf{J}_f(\mathbf{x}) = \mathbf{A}$.

Antwort: Falsch

Richtig ist $\mathbf{J}_f(\mathbf{x}) = \mathbf{A}^\top$ nach Satz 10.3.4(4), gelesen über $(\mathbf{x}^\top \mathbf{A})^\top = \mathbf{A}^\top \mathbf{x}$ (Bemerkung 10.3.5). Für $n \neq m$ verraten schon die Formate den Fehler: f bildet von $\mathbb{R}^n$ nach $\mathbb{R}^m$ ab, die Jacobimatrix ist also $m \times n$, während $\mathbf{A}$ das Format $n \times m$ hat. Bei $n = m$ passt die falsche Antwort formal, dort fällt sie erst beim Nachrechnen auf. Für $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 3 & -1 \end{pmatrix} \in \mathbb{R}^{3 \times 2}$ ist $\mathbf{J}_f(\mathbf{x}) = \begin{pmatrix} 1 & 0 & 3 \\ 2 & 1 & -1 \end{pmatrix}$.

4. Für differenzierbares $F : \mathbb{R} \rightarrow \mathbb{R}^{n \times n}$ gilt $\partial \det(\mathbf{F}(\mathbf{x})) / \partial \mathbf{x} = \det(\partial \mathbf{F}(\mathbf{x}) / \partial \mathbf{x})$.

Antwort: Falsch

Die Determinante ist nicht linear, Ableiten und Auswerten lassen sich hier also nicht vertauschen. Für $\mathbf{F}(\mathbf{x}) = \text{diag}(x, 2x)$ ist $\det(\mathbf{F}(\mathbf{x})) = 2x^2$ mit Ableitung $4x$, während $\det(\partial \mathbf{F}(\mathbf{x}) / \partial \mathbf{x}) = \det(\text{diag}(1, 2)) = 2$ konstant ist. Beide stimmen nur zufällig bei $x = 0,5$ überein. Richtig ist die Jacobi-Formel (10.4.4), die den Faktor $\det(\mathbf{F}(\mathbf{x}))$ und die Spur mitführt; für die Spur dagegen dürfen wir vertauschen (10.4.3), denn sie ist linear.

5. Für feste $\mathbf{a} \in \mathbb{R}^m$ und $\mathbf{b} \in \mathbb{R}^n$ mit $\mathbf{a}, \mathbf{b} \neq \mathbf{0}$ ist $\partial(\mathbf{a}^\top \mathbf{X} \mathbf{b}) / \partial \mathbf{X} = \mathbf{a} \mathbf{b}^\top$, und diese Matrix hat den Rang 1.

Antwort: Wahr

Die Formel ist (10.4.9), und rechts steht ein äußeres Produkt, das nach Satz 9.3.4 (Abschnitt 9.3)

für Vektoren $\neq \mathbf{0}$ genau den Rang 1 hat. Für $\mathbf{a} = (1, 2)^\top$ und $\mathbf{b} = (3, -1)^\top$ ist $\mathbf{ab}^\top = \begin{pmatrix} 3 & -1 \\ 6 & -2 \end{pmatrix}$ mit Determinante 0; die zweite Zeile ist das Doppelte der ersten.

6. Im Gradienten-Widget aus Abschnitt 10.2: Wir drehen $\mathbf{d}$ so, dass die Richtung längs der Höhenlinie zeigt. Welchen Wert nimmt die Richtungsableitung $\nabla f(\mathbf{x})\mathbf{d}$ dann an?

Lösung: 0

Null. Entlang der Höhenlinie ändert sich f in erster Ordnung nicht, und genau deshalb steht der Gradient senkrecht auf ihr (Bemerkung 10.2.5). Im Schnitt rechts liegt die grüne Gerade dann waagrecht.

Die zweite Runde nimmt sich die Abschnitte 10.5 bis 10.8 vor: acht Aussagen zu Eigenschaften, Rechenregeln, Hesse-Matrix und Taylor.

1. Existieren in $\mathbf{x}$ alle partiellen Ableitungen von $f : \mathbb{R}^n \rightarrow \mathbb{R}$, so ist f dort differenzierbar.

Antwort: Falsch

Die Existenz der partiellen Ableitungen sagt nur etwas über die Koordinatenrichtungen aus. Für

$$f(\mathbf{x}) = \frac{x_1 x_2}{x_1^2 + x_2^2} \quad \text{für } \mathbf{x} \neq \mathbf{0}, \quad f(\mathbf{0}) = 0$$

verschwindet f auf beiden Achsen, also existieren im Nullpunkt beide partiellen Ableitungen und sind 0. Auf der Diagonalen $x_1 = x_2 = t$ ist f dagegen konstant 0,5 und auf $x_2 = -x_1$ konstant $-0,5$, für jedes noch so kleine $t \neq 0$. Damit ist f in $\mathbf{0}$ nicht einmal stetig und nach der Kontraposition von Satz 10.5.2 erst recht nicht differenzierbar (Bemerkung 10.5.4). Sind die partiellen Ableitungen dagegen *stetig*, so ist f differenzierbar; davon lebt Schritt 3 im Beweis zu Satz 10.7.6.

2. Für in $\mathbf{x}$ differenzierbare $f, g : \mathbb{R}^n \rightarrow \mathbb{R}$ gilt $\nabla(fg)(\mathbf{x}) = g(\mathbf{x})\nabla f(\mathbf{x}) + f(\mathbf{x})\nabla g(\mathbf{x})$.

Antwort: Wahr

Das ist Satz 10.6.3 mit der Multiplikation zweier Zahlen als bilinearer Abbildung, die wegen $|uv| = |u||v|$ die Schranke $K = 1$ hat (Bemerkung 10.6.2, Punkt 2 mit $m = 1$). Beide Summanden sind Zeilenvektoren, skaliert mit einer Zahl. Zahlenprobe mit $f(\mathbf{x}) = x_1^2 + x_2$ und $g(\mathbf{x}) = e^{x_1} x_2$ an der Stelle $\mathbf{x} = (0,4; -0,7)^\top$: Dort ist $f = -0,54$ und $g = -1,0443$, und beide Wege liefern $\nabla(fg) = (-0,2715; -1,8499)$.

3. Für $\mathbf{F} : \mathbb{R} \rightarrow \mathbb{R}^{n \times n}$ und $g : \mathbb{R}^{n \times n} \rightarrow \mathbb{R}$ ist $\partial(g \circ \mathbf{F})(x)/\partial x$ das Matrixprodukt $(\partial g(\mathbf{Y})/\partial \mathbf{Y})^\top \partial \mathbf{F}(x)/\partial x$ an der Stelle $\mathbf{Y} = \mathbf{F}(x)$.

Antwort: Falsch

Links steht eine Zahl, rechts eine Matrix. Es fehlt die Spur: Nach der fünften Bauform in Beispiel 10.6.8 ist die Ableitung $\text{tr}((\partial g/\partial \mathbf{Y})^\top \partial \mathbf{F}(x)/\partial x)$, denn die Fréchet-Ableitung einer skalaren Funktion mit Matrixargument wirkt über das Frobenius-Skalarprodukt (Bemerkung 10.4.8). Nachgerechnet mit $g(\mathbf{Y}) = \|\mathbf{Y}\|_F^2$ und $\mathbf{F}(x) = \begin{pmatrix} x & 1 \\ 0 & e^x \end{pmatrix}$: Direkt ist $g(\mathbf{F}(x)) = x^2 + 1 + e^{2x}$ mit Ableitung $2x + 2e^{2x}$, bei $x = 0,3$ also 4,2442. Das Matrixprodukt ist dort $\begin{pmatrix} 0,6 & 0 \\ 2 & 3,6442 \end{pmatrix}$, und erst seine Spur ergibt 4,2442.

4. Der Satz von Schwarz gilt für jede Funktion, deren zweite partielle Ableitungen existieren.

Antwort: Falsch

Er verlangt zusätzlich deren Stetigkeit, also $f \in \mathcal{C}^2$ (Satz 10.7.4). Schon für die Einträge der Hesse-Matrix reicht bloße Differenzierbarkeit nicht (Definition 10.7.3). Das Standardgegenbeispiel ist

$$f(\mathbf{x}) = \frac{x_1 x_2 (x_1^2 - x_2^2)}{x_1^2 + x_2^2} \quad \text{für } \mathbf{x} \neq \mathbf{0}, \quad f(\mathbf{0}) = 0.$$

Auf den Achsen lässt sich alles ausrechnen: $\partial f(0, x_2)/\partial x_1 = -x_2$ und $\partial f(x_1, 0)/\partial x_2 = x_1$. Einmal mehr abgeleitet ergibt das im Nullpunkt die beiden Werte -1 und $+1$, je nach Reihenfolge. Die Hesse-Matrix ist dort also nicht symmetrisch, und alles, was auf ihrer Symmetrie aufbaut, von Satz 10.7.6 bis zum Spektralsatz-Argument in Satz 10.7.9, steht ohne Stetigkeit in der Luft.

5. Ist $\mathbf{x}^*$ ein kritischer Punkt von $f \in \mathcal{C}^2(S)$ mit $S \subseteq \mathbb{R}^2$ offen und gilt $\det \mathbf{H}_f(\mathbf{x}^*) < 0$, so ist $\mathbf{x}^*$ ein Sattelpunkt.

Antwort: Wahr

Die Determinante ist das Produkt der beiden Eigenwerte, und die sind reell, weil $\mathbf{H}_f(\mathbf{x}^*)$ nach Satz 10.7.4 symmetrisch ist. Ein negatives Produkt lässt nur eine Möglichkeit: ein positiver und ein negativer Eigenwert, also eine indefinite Hesse-Matrix und damit Satz 10.7.9(3). Beispiel ist die Funktion $f(\mathbf{x}) = x_1^2 + 3x_1x_2 + 2x_2^2$ aus dem Gradientenfeld-Widget in Abschnitt 10.2 mit $\mathbf{H}_f = \begin{pmatrix} 2 & 3 \\ 3 & 4 \end{pmatrix}$, Determinante -1 und Eigenwerten $3 \pm \sqrt{10}$, also $6,162$ und $-0,162$. Ab $n = 3$ trägt das Vorzeichen der Determinante nicht mehr: $\text{diag}(-1, -1, -1)$ hat ebenfalls Determinante -1 , ist aber negativ definit und zeigt ein Maximum an.

6. Weil der Restterm der quadratischen Näherung $o(\|\mathbf{h}\|^2)$ ist, fällt er beim Halbieren von $\mathbf{h}$ höchstens auf ein Viertel.

Antwort: Falsch

Eine o -Aussage beschreibt einen Grenzwert und verspricht über einzelne Schritte gar nichts. Schon im Eindimensionalen: Die Funktion $r(h) = h^3(2 + \sin(1/h))$ erfüllt $|r(h)|/h^2 \leq 3|h| \rightarrow 0$, ist also $o(|h|^2)$. Bei $h = 0,095$ steht trotzdem $|r(h/2)|/|r(h)| = 0,317$. Die Faustregel „ein Achtel“, die das Widget in Abschnitt 10.8 misst, stützt sich nicht auf die o -Aussage, sondern auf das Lagrange-Restglied (10.8.3) mit beschränkter dritter Ableitung.

7. Für $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top \mathbf{A} \mathbf{x} - \mathbf{b}^\top \mathbf{x}$ mit symmetrischem, positiv definitem $\mathbf{A}$ landet der Newton-Schritt (10.8.7) aus jedem Startpunkt in der Lösung des Gleichungssystems $\mathbf{A} \mathbf{x} = \mathbf{b}$.

Antwort: Wahr

Nach Beispiel 10.6.5 mit $\mathbf{A} = \mathbf{A}^\top$, Satz 10.3.4(3) für den linearen Term und der Linearität aus Satz 10.5.5 ist $\nabla f(\mathbf{x}) = \mathbf{x}^\top \mathbf{A} - \mathbf{b}^\top$, und $\mathbf{H}_f(\mathbf{x}) = \mathbf{A}$ ist konstant. Eingesetzt in (10.8.7) gibt das $\mathbf{x}^{(1)} = \mathbf{x}^{(0)} - \mathbf{A}^{-1}(\mathbf{A} \mathbf{x}^{(0)} - \mathbf{b}) = \mathbf{A}^{-1} \mathbf{b}$, unabhängig vom Start. Zahlenprobe mit $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$ und $\mathbf{b} = (1, 2)^\top$: Von $(5, -4)^\top$, von $\mathbf{0}$ und von $(-13, 7; 8, 2)^\top$ aus steht nach einem Schritt jeweils exakt $(0, 2; 0, 6)^\top$ da. Der Grund ist Bemerkung 10.8.10: Bei einer quadratischen Funktion ist $T_2 = f$, das Verfahren minimiert also die Funktion selbst. Gerechnet wird der Schritt als Gleichungssystem und nicht über die Inverse, bei positiv definitem $\mathbf{A}$ per Cholesky-Zerlegung (Abschnitt 5.4).

8. Ist $f \in \mathcal{C}^2(S)$ auf einer offenen, konvexen Menge S konvex, so ist jeder kritische Punkt von f ein globales Minimum auf S .

Antwort: Wahr

Nach Satz 10.7.11 ist $\mathbf{H}_f(\mathbf{x}) \succeq 0$ für alle $\mathbf{x} \in S$. Sei $\mathbf{x}^*$ kritisch und $\mathbf{y} \in S$ beliebig. Weil S konvex ist, liegt die Strecke dazwischen ganz in S , und wir betrachten $\varphi(t) := f(\mathbf{x}^* + t(\mathbf{y} - \mathbf{x}^*))$ für $t \in [0, 1]$. Die Kettenregel gibt $\varphi'(0) = \nabla f(\mathbf{x}^*)(\mathbf{y} - \mathbf{x}^*) = 0$ und

$\varphi''(t) = (\mathbf{y} - \mathbf{x}^*)^\top \mathbf{H}_f(\mathbf{x}^* + t(\mathbf{y} - \mathbf{x}^*))(\mathbf{y} - \mathbf{x}^*) \geq 0$. Also ist φ' nichtfallend und ab $t = 0$ nicht-negativ, damit $\varphi(1) \geq \varphi(0)$, und das heißt $f(\mathbf{y}) \geq f(\mathbf{x}^*)$. Sattelpunkte kann es aus demselben Grund nicht geben. Darauf beruht der Ausblick auf das nächste Kapitel.

Vertiefung: MML §5.7 behandelt die höheren Ableitungen, §5.8 die Linearisierung und die multivariaten Taylorreihen, und §5.9 sammelt weiterführende Literatur zur Matrixdifferentialrechnung. Die Konvexität und die Verfahren, die auf ihr aufbauen, stehen in MML §7.3; der Gradientenabstieg aus Abschnitt 10.2 steht dort in §7.1 daneben.

Konvexität

11.1 Konvexkombinationen und konvexe Hülle

Wozu Konvexität

Kapitel 10 hat die Ableitungen bereitgestellt. Gradient, Jacobi- und Hesse-Matrix beschreiben, wie sich eine Funktion in der Nähe eines Punktes verhält. Über ihr Verhalten im Großen sagen sie wenig. Diese Lücke füllt die Konvexität.

Der Begriff hat zwei eng verwandte Ausprägungen: *konvexe Mengen* (Abschnitt 11.2) und *konvexe Funktionen* (Abschnitt 11.3). Beide verallgemeinern Linearität, und zwar durch eine Einschränkung auf der einen und eine Lockerung auf der anderen Seite. Bei einer Linearkombination $\sum_i w_i \mathbf{x}_i$ dürfen die Koeffizienten beliebig sein; lassen wir nur nichtnegative mit Summe 1 zu, entsteht der Begriff dieses Abschnitts. Und wo eine lineare Funktion die Gleichung $f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) = \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ exakt erfüllt, verlangt Konvexität davon nur noch die Ungleichung $\leq$.

Warum sich das lohnt, zeigt sich an zwei Stellen. In der Optimierung und damit im maschinellen Lernen und in der Statistik ist ein konvexes Problem eines, bei dem kein Verfahren in einem schlechten Tal hängenbleiben kann: Jedes lokale Minimum ist dort automatisch ein globales. Darauf bauen viele Standardwerkzeuge auf, etwa die Maximum-Likelihood-Schätzung in verallgemeinerten linearen Modellen, Support Vector Machines und die Ridge-Regression, deren Zielfunktion wir in Abschnitt 10.6 abgeleitet haben. Bei der Likelihood lohnt ein Blick auf die Richtung: Konvex ist die *negative* Log-Likelihood, die wir minimieren. Die Log-Likelihood selbst ist in diesen Modellen konkav, und eine konkave Funktion zu maximieren ist dasselbe, wie ihr Negatives zu minimieren.

Vorsicht bei Formulierungen, die konvexen Problemen gleich *eindeutige* globale Minima zusprechen. Verlässlich ist nur die Aussage über lokal und global. Eindeutigkeit bekommen wir erst mit strikter Konvexität, denn eine konstante Funktion ist konvex und wird an jeder Stelle minimal. Und ob überhaupt ein Minimum angenommen wird, ist noch einmal eine eigene Frage: $f(x) = e^x$ ist konvex, kommt ihrem Infimum 0 auf $\mathbb{R}$ aber nur beliebig nahe. Sauber getrennt stehen die drei Aussagen in Abschnitt 11.5, und in Kapitel 12 hängen daran die Konvergenzgarantien der Verfahren.

In der Wahrscheinlichkeitstheorie liefert Konvexität Ungleichungen für Erwartungswerte. Die Jensen-Ungleichung ist der Prototyp (Abschnitt 11.4); die Kullback-Leibler-Divergenz ist konvex, was in der Variationsinferenz gebraucht wird; und die Nichtnegativität der Varianz ist nichts anderes als die Jensen-Ungleichung für $x \mapsto x^2$:

$$\mathbb{E}[X^2] \geq (\mathbb{E}[X])^2.$$

Dieser Abschnitt legt das Fundament für all das. Wir brauchen dafür wenig: Vektorräume und Linearkombinationen, den Umgang mit Normen und Skalarprodukten, aus der Analysis Grenzwerte,

Supremum und Infimum sowie abgeschlossene Mengen, und aus der Stochastik den Erwartungswert als gewichteten Durchschnitt. Später im Kapitel kommen der Gradient und positiv definite Matrizen dazu.

Konvexkombinationen

Beginnen wir mit dem Baustein, aus dem alles Weitere entsteht.

Definition 11.1.1 (Konvexkombination)

Sei V ein Vektorraum und $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_k\} \subset V$. Wir nennen $\mathbf{x}$ eine *Konvexkombination* (convex combination) von Vektoren in $\mathcal{X}$, falls es Koeffizienten $w_1, \dots, w_k \geq 0$ gibt mit

$$\mathbf{x} = \sum_{i=1}^k w_i \mathbf{x}_i \quad \text{und} \quad \sum_{i=1}^k w_i = 1.$$

Bemerkung 11.1.2 (Ein gewichteter Durchschnitt)

Konvexkombinationen sind spezielle Linearkombinationen. Zwei Bedingungen kommen hinzu, und jede von beiden zieht etwas nach sich.

Die Gewichte sind nichtnegativ, sie dürfen also keine Richtung umkehren. Sie summieren sich zu 1, wir können sie also nicht gemeinsam hochskalieren. Zusammen heißt das: Die Gewichte $w_1, \dots, w_k$ sind nichts anderes als eine Wahrscheinlichkeitsverteilung auf den k Punkten, und $\mathbf{x}$ ist deren *gewichteter Durchschnitt*.

Zwei Sonderfälle sind schnell abgehakt. Für $k = 1$ bleibt nur $w_1 = 1$, also $\mathbf{x} = \mathbf{x}_1$. Und wählen wir alle Gewichte gleich, so ist $w_i = \frac{1}{k}$ und $\mathbf{x}$ das arithmetische Mittel der Punkte.

Nützlich ist außerdem die Lesart als Verschiebung. Setzen wir $w_1 = 1 - \sum_{i \geq 2} w_i$ in die Definition ein, so wird daraus

$$\mathbf{x} = \mathbf{x}_1 + \sum_{i=2}^k w_i (\mathbf{x}_i - \mathbf{x}_1).$$

Wir starten also in $\mathbf{x}_1$ und gehen ein Stück in Richtung der übrigen Punkte. Die Summenbedingung ist der Grund, warum hier lauter *Differenzen* stehen und das Ergebnis nicht davon abhängt, wo der Ursprung des Koordinatensystems liegt.

Beispiel 11.1.3 (Der Erwartungswert ist eine Konvexkombination)

Sei X eine diskrete Zufallsvariable mit endlichem Träger $\mathcal{X} = T_X = \{x_1, \dots, x_k\} \subset \mathbb{R}$; wir arbeiten also in Definition 11.1.1 mit $V = \mathbb{R}$. Ihr Erwartungswert ist

$$\mathbb{E}(X) = \sum_{x_j \in T_X} \mathbb{P}(X = x_j) x_j,$$

und das ist wörtlich Definition 11.1.1: Die Wahrscheinlichkeiten sind nichtnegativ und summieren sich zu 1. Der Erwartungswert ist also eine Konvexkombination der Werte des Trägers, mit den Wahrscheinlichkeiten als Gewichten. Bei abzählbar unendlichem Träger wird daraus eine Reihe, und wir bräuchten einen Grenzübergang.

Beim fairen Würfel etwa tragen alle sechs Augenzahlen das Gewicht $\frac{1}{6}$, und wir erhalten das arithmetische Mittel $\mathbb{E}(X) = 3,5$. Daran hängt eine Aussage, die uns durch das Kapitel

begleitet: Ein gewichteter Durchschnitt verlässt den Bereich seiner Zutaten nicht. $\mathbb{E}(X)$ liegt zwischen 1 und 6, obwohl kein einzelner Wurf jemals 3,5 zeigt.

Rechnen wir dagegen erst und mitteln dann, ändert sich das Bild. Für dieselbe Zufallsvariable ist $\mathbb{E}(X^2) = \frac{91}{6} \approx 15,17$, während $(\mathbb{E}(X))^2 = 12,25$ ist. Die Differenz $\frac{35}{12} \approx 2,92$ ist die Varianz. Dass hier immer dieses Vorzeichen steht, ist kein Zufall des Würfels, sondern die Jensen-Ungleichung; wir beweisen sie in Abschnitt 11.4.

Zwei Punkte: das Liniensegment

Der einfachste nichttriviale Fall hat $k = 2$, und er ist bereits das geometrische Herz des ganzen Kapitels.

Satz 11.1.4 (Konvexkombinationen zweier Vektoren)

Seien $\mathbf{x}_1, \mathbf{x}_2 \in V$. Die Menge aller Konvexkombinationen von $\mathbf{x}_1$ und $\mathbf{x}_2$ ist das *Liniensegment*

$$[\mathbf{x}_1, \mathbf{x}_2] := \{\mathbf{x}_1 + \lambda(\mathbf{x}_2 - \mathbf{x}_1) : \lambda \in [0, 1]\},$$

also die Verbindungsstrecke der beiden Punkte.

Vertiefung: Beweis für Konvexkombinationen zweier Vektoren

Beweis.

- (1) Sei $\mathbf{x}$ eine Konvexkombination, also $\mathbf{x} = w_1 \mathbf{x}_1 + w_2 \mathbf{x}_2$ mit $w_1, w_2 \geq 0$ und $w_1 + w_2 = 1$. Setzen wir $\lambda := w_2$, so ist $\lambda \in [0, 1]$ und $w_1 = 1 - \lambda$, also

$$\mathbf{x} = (1 - \lambda) \mathbf{x}_1 + \lambda \mathbf{x}_2 = \mathbf{x}_1 + \lambda(\mathbf{x}_2 - \mathbf{x}_1).$$

die Summenbedingung erlaubt es, w_1 zu eliminieren; danach sortieren wir nach $\mathbf{x}_1$

- (2) Sei umgekehrt $\lambda \in [0, 1]$ gegeben. Mit $w_1 := 1 - \lambda$ und $w_2 := \lambda$ sind beide Gewichte nichtnegativ, ihre Summe ist 1, und die zugehörige Konvexkombination ist gerade $\mathbf{x}_1 + \lambda(\mathbf{x}_2 - \mathbf{x}_1)$. Beide Mengen enthalten einander, sind also gleich.

$\lambda \in [0, 1]$ garantiert $1 - \lambda \geq 0$; ohne diese Schranke bekämen wir die ganze Gerade durch $\mathbf{x}_1$ und $\mathbf{x}_2$ statt nur die Strecke

□

Der Parameter λ misst den Fortschritt entlang der Strecke: $\lambda = 0$ gibt $\mathbf{x}_1$, $\lambda = \frac{1}{2}$ den Mittelpunkt, $\lambda = 1$ gibt $\mathbf{x}_2$. Wer die Schranken $0 \leq \lambda \leq 1$ fallen lässt, erhält die ganze Gerade durch die beiden Punkte; die Nichtnegativität der Gewichte ist also genau das, was Strecke von Gerade trennt.

Halten wir fest, worauf das hinausläuft. Eine Menge wird gleich *konvex* heißen, wenn sie mit je zwei Punkten auch deren Verbindungsstrecke enthält (Abschnitt 11.2). Satz 11.1.4 sagt: Diese Strecke ist nichts anderes als die Menge aller Konvexkombinationen der beiden Punkte. Konvexe Mengen sind also genau die Mengen, die unter Konvexkombinationen von je zwei Punkten abgeschlossen sind. Dass dann auch Konvexkombinationen von mehr als zwei Punkten nicht hinausführen, ist eine Induktion und steht dort als eigener Satz.

Die konvexe Hülle

Bisher haben wir von einer festen, endlichen Punktliste aus kombiniert. Die konvexe Hülle dreht den Blick um: Wir sammeln alles ein, was sich aus einer Menge $\mathcal{X}$ überhaupt kombinieren lässt.

Definition 11.1.5 (Konvexe Hülle)

Sei $\mathcal{X} \subset V$ eine beliebige Menge von Vektoren. Die Menge

$$\text{conv}(\mathcal{X}) = \left\{ \mathbf{x} = \sum_{i=1}^N w_i \mathbf{x}_i : N \in \mathbb{N}, \mathbf{x}_i \in \mathcal{X}, w_i \geq 0, \sum_{i=1}^N w_i = 1 \right\}$$

aller Konvexkombinationen von endlich vielen Vektoren aus $\mathcal{X}$ heißt *konvexe Hülle* (convex hull) von $\mathcal{X}$.

Bemerkung 11.1.6 (Warum endlich viele)

Drei Details der Definition verdienen einen zweiten Blick.

$\mathcal{X}$ selbst darf unendlich sein, eine Kugel etwa oder eine Kurve. Jede *einzelne* Kombination benutzt aber nur endlich viele Punkte daraus. Und N ist nicht ein für alle Mal festgelegt, sondern darf von Punkt zu Punkt anders ausfallen; deshalb steht das $N \in \mathbb{N}$ mit in der Mengenklammer.

Für endliches $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_k\}$ reicht immer $N = k$. Wir dürfen ja Gewichte 0 vergeben, brauchen also nie mehr Summanden als Punkte, und Wiederholungen bringen nichts Neues, weil sich die Gewichte zweier gleicher Punkte einfach addieren.

Schließlich gilt $\mathcal{X} \subseteq \text{conv}(\mathcal{X})$: Für $N = 1$ ist $w_1 = 1$ erzwungen, und wir bekommen die Punkte von $\mathcal{X}$ selbst zurück. Wer neugierig ist: Im $\mathbb{R}^n$ genügen sogar stets $N \leq n + 1$ Punkte pro Kombination, in der Ebene also drei. Das ist der klassische Satz von Carathéodory, den wir hier nicht brauchen.

Unter den Punkten von $\mathcal{X}$ gibt es solche, die sich nicht wegdiskutieren lassen, weil sie aus den anderen nicht zu bauen sind.

Definition 11.1.7 (Extrempunkt)

Ein Punkt $\mathbf{x} \in \mathcal{X}$ heißt *Extrempunkt* (extreme point) von $\text{conv}(\mathcal{X})$, wenn er sich nur auf die triviale Weise als Konvexkombination von Punkten aus $\mathcal{X}$ schreiben lässt, nämlich mit dem Gewicht 1 auf $\mathbf{x}$ selbst und 0 auf allen anderen Punkten von $\mathcal{X}$.

Bemerkung 11.1.8 (Extrempunkt zu sein ist keine Eigenschaft des Punktes allein)

Die Definition prüft eine Aussage über *alle* Darstellungen. Ob ein Punkt Extrempunkt ist, hängt deshalb an der ganzen Menge $\mathcal{X}$, nicht am Punkt für sich. Nehmen wir zu $\mathcal{X}$ weitere Vektoren hinzu, so kann ein bisheriger Extrempunkt seinen Status verlieren, sobald er zwischen den neuen liegt. Das zweite Widget unten führt das vor.

In der Literatur wird der Begriff meist etwas allgemeiner gefasst, nämlich für eine beliebige konvexe Menge statt für eine Punktliste: Extrem heißt dort, im Inneren keiner Strecke zu liegen, die ganz in der Menge verläuft. Diese Fassung brauchen wir ab Abschnitt 11.2 und führen sie dort als Definition 11.2.5 ein; für eine endliche Punktmenge $\mathcal{X}$ wählen beide Lesarten dieselben Punkte aus. Im Dreiecksbeispiel unten bleiben deshalb genau die drei Ecken übrig.

Bemerkung 11.1.9 (Die kleinste konvexe Obermenge)

Der Name *Hülle* ist mit Bedacht gewählt: $\text{conv}(\mathcal{X})$ ist die kleinste konvexe Menge, die $\mathcal{X}$ enthält. Das sind drei Behauptungen auf einmal: $\text{conv}(\mathcal{X})$ ist konvex, sie enthält $\mathcal{X}$, und sie liegt in *jeder* konvexen Menge, die $\mathcal{X}$ enthält.

Die zweite steht schon in Bemerkung 11.1.6. Für die anderen beiden fehlt uns noch der Begriff der konvexen Menge, deshalb holen wir den Beweis in Abschnitt 11.2 nach. Die Beweisidee lässt sich aber jetzt schon in einem Satz sagen: Eine konvexe Menge ist unter Konvexkombinationen von zwei Punkten abgeschlossen, und daraus folgt per Induktion die Abgeschlossenheit unter Konvexkombinationen von N Punkten.

Interaktiv: Wie eine Punktwolke ihre Hülle aufspannt

Hier steht eine Punktwolke mit ihrer konvexen Hülle, und wir lassen die Wolke wachsen. Die Hülle selbst kann dabei nur größer werden. Aber gilt das auch für die Liste ihrer Extrempunkte?

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, wächst die Fläche monoton, die Liste der Extrempunkte dagegen nicht: Bei $k = 5, 6, 8, 9, 10, 11$ und 12 fällt jeweils eine alte Ecke heraus, während der neue Punkt nachrückt. Die Anzahl bleibt dabei stehen und nur die Zusammensetzung ändert sich; fallen kann sie ebenfalls, nämlich sobald ein einziger neuer Punkt gleich zwei alte Ecken einschließt. Am Ende tragen sieben der vierzehn Punkte die ganze Hülle. Genau das meint Bemerkung 11.1.8: Extrempunkt zu sein ist keine Eigenschaft des Punktes, sondern der Menge.

Der grüne Punkt ist der Mittelwert der freigeschalteten Punkte, also die Konvexkombination mit lauter Gewichten $\frac{1}{k}$ aus Bemerkung 11.1.2. Er liegt immer in der blauen Fläche. Das ist die geometrische Fassung derselben Aussage, die in Beispiel 11.1.3 den Erwartungswert zwischen 1 und 6 hält.

Vertiefung: Die konvexe Hülle dreier Punkte ausführlich berechnet

Beispiel im $\mathbb{R}^2$: das Dreieck

Beispiel 11.1.10 (Konvexe Hülle dreier Punkte)

Sei $\mathcal{X} = \{\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3\} \subset \mathbb{R}^2$ mit

$$\mathbf{x}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad \mathbf{x}_2 = \begin{pmatrix} 2 \\ 0 \end{pmatrix}, \quad \mathbf{x}_3 = \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$

also den Ecken eines Dreiecks.

Extrempunkte. Alle drei Punkte sind Extrempunkte im Sinne von Definition 11.1.7. Keiner von ihnen liegt auf der Verbindungsstrecke der beiden anderen, also erreichen wir ihn nur mit $w_i = 1$ und $w_j = 0$ für $j \neq i$.

Konvexe Hülle. Nach Definition 11.1.5 ist $\text{conv}(\mathcal{X})$ die Menge aller Punkte

$$\mathbf{x} = w_1 \begin{pmatrix} 0 \\ 0 \end{pmatrix} + w_2 \begin{pmatrix} 2 \\ 0 \end{pmatrix} + w_3 \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 2w_2 + w_3 \\ 2w_3 \end{pmatrix}$$

mit $w_1, w_2, w_3 \geq 0$ und $w_1 + w_2 + w_3 = 1$, und das ist die Dreiecksfläche samt Rand.

Ein konkreter Punkt. Für $w_1 = w_2 = w_3 = \frac{1}{3}$ ergibt sich

$$\mathbf{x} = \frac{1}{3} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \frac{1}{3} \begin{pmatrix} 2 \\ 0 \end{pmatrix} + \frac{1}{3} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{2}{3} \end{pmatrix},$$

der Schwerpunkt des Dreiecks. Er liegt im Inneren, denn alle drei Gewichte sind positiv.

Bemerkung 11.1.11 (Rand und Inneres an den Gewichten ablesen)

Im Dreiecksbeispiel sind die Gewichte durch den Punkt eindeutig bestimmt, weil $\mathbf{x}_2 - \mathbf{x}_1$ und $\mathbf{x}_3 - \mathbf{x}_1$ linear unabhängig sind. Deshalb dürfen wir die Lage eines Punktes direkt an seinen Gewichten ablesen. Sind alle drei positiv, liegt er im Inneren. Ist genau eines gleich 0, liegt er auf der gegenüberliegenden Kante, die nach Satz 11.1.4 gerade das Liniensegment der beiden anderen Ecken ist. Sind zwei gleich 0, sind wir in einer Ecke.

Die Gewichte haben hier eine hübsche geometrische Bedeutung: w_i ist der Flächenanteil des Teildreiecks, das aus $\mathbf{x}$ und den beiden anderen Ecken gebildet wird. Beim Schwerpunkt sind alle drei Teilflächen gleich groß, jede also ein Drittel der Gesamtfläche 2. Das Widget rechnet diese Probe live mit.

Interaktiv: Konvexkombinationen zum Schieben

Ein einziges Widget deckt Liniensegment und Dreieck zugleich ab, wenn wir den Punkt selbst bewegen dürfen. Zwei Fragen lassen sich daran klären: Welche Punkte der Ebene sind überhaupt Konvexkombinationen der drei Ecken, und woran sehen wir den Gewichten an, ob der Punkt im Inneren, auf einer Kante oder in einer Ecke liegt?

Interaktives Element — nur in der Web-Fassung

Der grüne Punkt verlässt das Dreieck nie: Negative Gewichte sind nicht zugelassen, und genau diese Nichtnegativität hält ihn drinnen. Jede Kante entsteht, sobald ein Gewicht null wird: Übrig bleibt die Konvexkombination zweier Ecken, also nach Satz 11.1.4 ihr Liniensegment.

Die drei Regler geben unnormierte Werte; geteilt wird durch ihre Summe, damit $w_1 + w_2 + w_3 = 1$ gilt. Nur eine Reglerstellung ist verboten, nämlich alle drei auf 0: Dann ist die Summe 0, die Normierung nicht definiert, und die Nebenbedingung lässt sich gar nicht erfüllen.

Die gestrichelte Hilfsstrecke zeigt, warum die drei Ecken die gesamte Fläche erzeugen. Wir mischen zuerst $\mathbf{x}_2$ und $\mathbf{x}_3$ zum Punkt $\mathbf{y}$ auf der Kante zwischen diesen beiden und mischen dann $\mathbf{x}_1$ mit $\mathbf{y}$. Läuft $\mathbf{y}$ über die ganze Kante, so überstreichen die Strecken von $\mathbf{x}_1$ aus das ganze Dreieck. Formal steckt dahinter nichts anderes als das Zusammenfassen von Gewichten:

$$\mathbf{x} = w_1 \mathbf{x}_1 + (w_2 + w_3) \underbrace{\left(\frac{w_2}{w_2 + w_3} \mathbf{x}_2 + \frac{w_3}{w_2 + w_3} \mathbf{x}_3 \right)}_{=\mathbf{y}},$$

und die beiden inneren Gewichte summieren sich wieder zu 1. Vorausgesetzt ist dabei $w_2 + w_3 > 0$; sonst bleibt schlicht $\mathbf{x} = \mathbf{x}_1$, und es gibt nichts zu mischen. Dieselbe Rechnung, ein paar Punkte weiter getrieben, ist die Induktion, die wir in Abschnitt 11.2 für die Hülle brauchen.

Selbsttest

1. Jede Konvexkombination ist eine Linearkombination, aber nicht jede Linearkombination ist eine Konvexkombination.

Antwort: Wahr

Konvexkombinationen sind Linearkombinationen mit zwei Zusatzbedingungen (Definition 11.1.1), also ist die erste Hälfte klar. Für die zweite genügt ein Gegenbeispiel. Mit $\mathbf{x}_1 = (1, 0)^\top$ und $\mathbf{x}_2 = (0, 1)^\top$ ist $2\mathbf{x}_1 = (2, 0)^\top$ eine Linearkombination, aber keine Konvexkombination: Nach Satz 11.1.4 sind das genau die Punkte der Strecke zwischen $\mathbf{x}_1$ und $\mathbf{x}_2$, und $(2, 0)^\top$ liegt nicht darauf. Ebenso wenig tut es $\mathbf{x}_1 - \mathbf{x}_2 = (1, -1)^\top$. Auf die konkreten Vektoren kommt es dabei an: Wäre $\mathbf{x}_1 = \mathbf{0}$, so wäre $2\mathbf{x}_1 = \mathbf{x}_1$ sehr wohl eine Konvexkombination.

2. Ist ein Optimierungsproblem konvex, so besitzt es genau ein globales Minimum.

Antwort: Falsch

So formuliert verspricht die Aussage zu viel. Zuverlässig ist: Jedes lokale Minimum ist global. Eindeutigkeit folgt daraus nicht, denn eine konstante Funktion ist konvex und wird an jeder Stelle minimal; dafür braucht es strikte Konvexität. Und die Existenz ist eine dritte, davon unabhängige Frage: $f(x) = e^x$ ist konvex und nimmt ihr Infimum 0 auf $\mathbb{R}$ nie an. Die drei Aussagen werden in Abschnitt 11.5 sauber getrennt.

3. Für die Dreiecksecken $\mathbf{x}_1 = (0, 0)^\top$, $\mathbf{x}_2 = (2, 0)^\top$ und $\mathbf{x}_3 = (1, 2)^\top$ liegt der Punkt $(1, 1)^\top$ nicht in $\text{conv}(\mathcal{X})$.

Antwort: Falsch

Er liegt darin, und wir können die Gewichte angeben: $\frac{1}{4}\mathbf{x}_1 + \frac{1}{4}\mathbf{x}_2 + \frac{1}{2}\mathbf{x}_3 = (\frac{1}{2} + \frac{1}{2}, 1)^\top = (1, 1)^\top$. Alle drei Gewichte sind positiv und summieren sich zu 1, der Punkt liegt also sogar im Inneren des Dreiecks.

4. Ist $\mathbf{x}$ ein Extrempunkt von $\text{conv}(\mathcal{X})$, so bleibt $\mathbf{x}$ auch dann Extrempunkt, wenn wir $\mathcal{X}$ um weitere Punkte vergrößern.

Antwort: Falsch

Extrempunkt zu sein ist eine Aussage über alle Darstellungen und hängt damit an der ganzen Menge (Bemerkung 11.1.8). Im Hüllen-Widget oben ist $(1,5; 1,4)$ bei $k = 4$ noch Extrempunkt und verliert diesen Status bei $k = 5$, weil der neu hinzukommende Punkt ihn einschließt. Umgekehrt kann ein Punkt seinen Status nie gewinnen, wenn wir Punkte hinzunehmen.

5. Der Erwartungswert einer diskreten Zufallsvariablen mit endlichem Träger ist eine Konvexkombination der Trägerwerte.

Antwort: Wahr

Die Gewichte sind die Wahrscheinlichkeiten $\mathbb{P}(X = x_j)$: nichtnegativ und mit Summe 1 (Beispiel 11.1.3). Daraus folgt sofort, dass $\mathbb{E}(X)$ zwischen dem kleinsten und dem größten Trägerwert liegt, denn ein gewichteter Durchschnitt verlässt die konvexe Hülle seiner Zutaten nicht.

6. In der Definition der konvexen Hülle muss die Zahl N der beteiligten Punkte für alle Elemente dieselbe sein, und $\mathcal{X}$ muss endlich sein.

Antwort: Falsch

Beides ist zu eng gelesen. N steht mit in der Mengenklammer und darf von Punkt zu Punkt verschieden ausfallen, und $\mathcal{X}$ selbst darf ohne Weiteres unendlich sein; nur jede einzelne Kombination benutzt endlich viele Punkte (Bemerkung 11.1.6). Für endliches $\mathcal{X}$ mit k Elementen reicht dagegen immer $N = k$, weil Gewichte 0 erlaubt sind.

7. Schieben wir im Hüllen-Widget den Regler ganz nach rechts, also auf $k = 14$. Wie viele der vierzehn Punkte sind dann noch Extrempunkte?

Lösung: 7

Sieben. Die anderen sieben liegen im Inneren der Hülle und lassen sich als Konvexkombination der Ecken schreiben; sie steuern zur Hülle nichts bei. Genau darin liegt der Nutzen der Extrempunkte: Sie sind die sparsame Beschreibung einer konvexen Menge. Die Hülle hat dann die Fläche 7,46, und keiner der sieben inneren Punkte könnte sie vergrößern.

Vertiefung: Boyd und Vandenberghe, Convex Optimization, §2.1 behandelt Liniensegmente, konvexe Mengen, Konvexkombinationen und die konvexe Hülle mit weiteren Beispielen.

11.2 Konvexe Mengen

Mengen, die ihre Verbindungsstrecken enthalten

In Abschnitt 11.1 haben wir aus gegebenen Punkten neue gemacht: Konvexkombinationen mischen endlich viele Vektoren mit nichtnegativen Gewichten, die sich zu eins addieren. Jetzt drehen wir die Frage um. Statt zu fragen, welche Punkte aus einer Menge entstehen, fragen wir, welche Mengen unter dieser Konstruktion schon abgeschlossen sind.

Das klingt nach einer technischen Spielerei, ist aber die Eigenschaft, an der in der Optimierung

alles hängt. Die zulässige Menge eines Optimierungsproblems sammelt alle Punkte, die die Nebenbedingungen erfüllen. Ist sie konvex, so darf ein Verfahren, das zwei zulässige Punkte kennt, jeden Punkt dazwischen ausprobieren, ohne aus dem Erlaubten zu fallen. Ist sie es nicht, kann schon der Weg zwischen zwei guten Lösungen durch verbotenes Gebiet führen.

Definition 11.2.1 (Konvexe Menge)

Sei V ein Vektorraum. Eine Menge $\mathcal{X} \subseteq V$ heißt *konvex* (convex), falls für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und alle $\lambda \in [0, 1]$ gilt:

$$\lambda \mathbf{x} + (1 - \lambda) \mathbf{y} \in \mathcal{X}.$$

Bemerkung 11.2.2 (Was die Bedingung verlangt)

Der grüne Ausdruck ist genau die Konvexkombination zweier Punkte aus Abschnitt 11.1. Läuft λ von 0 nach 1, so wandert er von $\mathbf{y}$ nach $\mathbf{x}$ und durchläuft dabei die ganze Verbindungsstrecke. Konvex heißt also: Mit je zwei Punkten liegt auch die Strecke zwischen ihnen in der Menge. Anschaulich hat eine konvexe Menge weder Einbuchtungen noch Löcher.

Drei Kleinigkeiten sind der Definition anzusehen. Erstens sind die Randfälle $\lambda = 0$ und $\lambda = 1$ zugelassen; sie geben $\mathbf{y}$ beziehungsweise $\mathbf{x}$ zurück und sind damit automatisch erfüllt. Zweitens ist die leere Menge konvex, und einelementige Mengen sind es auch: Im ersten Fall gibt es kein Paar zu prüfen, im zweiten ist $\mathbf{x} = \mathbf{y}$ und die Konvexkombination liefert wieder denselben Punkt.

Drittens, und das ist beim Rechnen das Wichtigste, sind Nachweis und Widerlegung ungleich schwer. Ein einziges Paar, dessen Strecke heraushängt, widerlegt die Konvexität. Für den Nachweis dagegen müssen wir über *alle* Paare und alle λ argumentieren, und dazu brauchen wir eine Rechnung, keine Beispiele.

Interaktiv: Welche Mengen sind konvex?

Statt hier zwei Bilder abzdrukken, eine konvexe und eine nicht konvexe Menge, machen wir daraus einen Test. Vier Mengen stehen zur Wahl, zwei Punkte lassen sich frei hineinziehen: Bei welchen der vier finden wir ein Paar, dessen Verbindungsstrecke herausläuft, und wie schwer ist die Suche?

Interaktives Element — nur in der Web-Fassung

Die Kreisscheibe und das Dreieck geben nicht nach, so oft wir es auch versuchen. Beim Kreisring und bei der Parabelunterseite finden wir dagegen schnell ein Paar, dessen Strecke hinausläuft. Der vorbereitete Knopf setzt beim Ring die Punkte $(1,1; 0)^T$ und $(0; 1,1)^T$; ihr Mittelpunkt hat die Norm $1,1/\sqrt{2} \approx 0,778$ und unterschreitet damit den Innenradius 0,8. Bei der Parabel verlässt die Strecke die Menge sogar auf ganzer Länge. Kreisscheibe und Parabelmenge kommen gleich im Selbsttest wieder vor, dort mit Rechnung statt mit Mausklicks; der Ring vertritt dort die Einheitssphäre, die als Kurve ohne Dicke nicht anklickbar wäre.

Die Definition spricht nur über zwei Punkte. Gemeint ist aber mehr, und der folgende Satz holt es nach. Wir brauchen ihn, sobald wir über die konvexe Hülle argumentieren.

Satz 11.2.3 (Konvexe Mengen enthalten alle Konvexkombinationen)

Sei $\mathcal{X} \subseteq V$ konvex, seien $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}$ und $w_1, \dots, w_N \geq 0$ mit $\sum_{i=1}^N w_i = 1$. Dann gilt

$$\sum_{i=1}^N w_i \mathbf{x}_i \in \mathcal{X}.$$

Vertiefung: Beweis per Induktion über die Zahl der Punkte

Beweis.

- (1) Induktion über N . Für $N = 1$ ist $w_1 = 1$ und die Summe gleich $\mathbf{x}_1 \in \mathcal{X}$. Für $N = 2$ ist $w_2 = 1 - w_1$, und die Behauptung ist wörtlich Definition 11.2.1.

die Nebenbedingung $\sum_i w_i = 1$ macht aus zwei Gewichten gerade das Paar λ und $1 - \lambda$

- (2) Sei die Aussage für N Punkte bewiesen, und seien $\mathbf{x}_1, \dots, \mathbf{x}_{N+1}$ und Gewichte $w_1, \dots, w_{N+1}$ wie im Satz gegeben. Ist $w_{N+1} = 1$, so sind alle übrigen Gewichte null und die Summe ist $\mathbf{x}_{N+1} \in \mathcal{X}$. Sei also $w_{N+1} < 1$ und

$$s := \sum_{i=1}^N w_i = 1 - w_{N+1} > 0.$$

die Gewichte sind nichtnegativ und addieren sich zu eins, also ist $s = 1 - w_{N+1}$ und $s > 0$ genau dann, wenn $w_{N+1} < 1$

- (3) Wir teilen die ersten N Gewichte durch s . Die Zahlen w_i/s sind nichtnegativ und addieren sich zu 1, also ist

$$\mathbf{u} := \sum_{i=1}^N \frac{w_i}{s} \mathbf{x}_i \in \mathcal{X}$$

nach Induktionsvoraussetzung. Damit lässt sich die volle Summe als Konvexkombination zweier Punkte schreiben:

$$\sum_{i=1}^{N+1} w_i \mathbf{x}_i = s \mathbf{u} + (1 - s) \mathbf{x}_{N+1} \in \mathcal{X},$$

denn $s \in (0, 1)$ und $\mathbf{u}, \mathbf{x}_{N+1} \in \mathcal{X}$.

Definition 11.2.1 mit $\lambda = s$; der Trick ist das Normieren der ersten N Gewichte, das aus einer $(N+1)$ -Kombination eine Zweierkombination von etwas schon Bekanntem macht

- (4) Zusammen mit $\mathcal{X} \subseteq \text{conv}(\mathcal{X})$, was man an der Einerkombination $w_1 = 1$ abliest, folgt für konvexe Mengen

$$\text{conv}(\mathcal{X}) = \mathcal{X}.$$

Eine Menge ist also genau dann konvex, wenn sie mit ihrer konvexen Hülle übereinstimmt.

der Satz liefert $\text{conv}(\mathcal{X}) \subseteq \mathcal{X}$, die Einerkombination die umgekehrte Inklusion; die Definition der konvexen Hülle steht in Abschnitt 11.1

□

Fünf Mengen im Selbsttest

Fünf Mengen stehen zur Wahl. Rechnen wir sie einzeln durch.

1. Die leere Menge $\emptyset$ ist konvex.

Antwort: Wahr

Definition 11.2.1 fordert etwas über alle Paare $\mathbf{x}, \mathbf{y} \in \mathcal{X}$. In der leeren Menge gibt es kein solches Paar, also ist nichts zu prüfen und die Bedingung leer erfüllt. Mit demselben Argument ist jede einelementige Menge konvex, denn dort ist $\mathbf{x} = \mathbf{y}$ und $\lambda \mathbf{x} + (1 - \lambda)\mathbf{x} = \mathbf{x}$.

2. Ein Vektorraum V ist, als Teilmenge seiner selbst gelesen, konvex.

Antwort: Wahr

Ein Vektorraum ist gegen Linearkombinationen abgeschlossen, und $\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}$ ist eine solche. Dasselbe gilt für jeden Untervektorraum und für affine Teilräume. Konvexe Mengen sind damit eine echte Verallgemeinerung: Sie verlangen die Abgeschlossenheit nur für Gewichte, die nichtnegativ sind und sich zu eins addieren.

3. Die Einheitssphäre $\{\mathbf{x} \in V : \|\mathbf{x}\| = 1\}$ ist konvex.

Antwort: Falsch

Sie ist es nicht. Ein Gegenbeispiel lebt im $\mathbb{R}^2$: Für $\mathbf{x} = (1, 0)^\top$ und $\mathbf{y} = (0, 1)^\top$ ist $\|\mathbf{x}\| = \|\mathbf{y}\| = 1$, aber der Mittelpunkt $\lambda = \frac{1}{2}$ liefert

$$\frac{1}{2}\mathbf{x} + \frac{1}{2}\mathbf{y} = (0,5; 0,5)^\top, \quad \|(0,5; 0,5)^\top\| = \sqrt{0,25 + 0,25} = \frac{1}{\sqrt{2}} \approx 0,7071 \neq 1.$$

Kürzer und ohne Koordinaten geht es auch: Sobald $V \neq \{\mathbf{0}\}$ ist, gibt es ein $\mathbf{x}$ mit $\|\mathbf{x}\| = 1$, und dann liegt auch $-\mathbf{x}$ auf der Sphäre. Deren Mittelpunkt ist $\mathbf{0}$ mit Norm 0. Der Sphäre fehlt genau das Innere, das den Ball konvex macht.

4. Der geschlossene Ball $\{\mathbf{x} \in V : \|\mathbf{x}\| \leq r\}$ ist konvex.

Antwort: Wahr

Das ist die Dreiecksungleichung plus absolute Homogenität der Norm. Für $\lambda \in [0, 1]$ sind λ und $1 - \lambda$ nichtnegativ, also

$$\|\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}\| \leq \lambda \|\mathbf{x}\| + (1 - \lambda)\|\mathbf{y}\| \leq \lambda r + (1 - \lambda)r = r.$$

Bemerkenswert daran ist, dass nirgends eine bestimmte Norm gebraucht wird: Kugeln bezüglich jeder Norm sind konvex, auch die Würfel und Rauten der ∞ - und 1-Norm aus Abschnitt 3.2.

5. Die Menge $\{(x, y) \in \mathbb{R}^2 : y \leq x^2\}$ ist konvex.

Antwort: Falsch

Sie ist es nicht, denn sie liegt *unter* einer nach oben geöffneten Parabel. Die Punkte $(-1, 1)$ und $(1, 1)$ gehören dazu, denn $1 \leq (\pm 1)^2 = 1$. Ihr Mittelpunkt ist $(0, 1)$, und dort scheitert die Bedingung: $1 \leq 0^2 = 0$ ist falsch. Die Strecke verlässt die Menge sogar für jedes λ zwischen 0 und 1, wie das Widget vorführt.

Konvex wäre die Menge oberhalb der Parabel, $\{(x, y) : y \geq x^2\}$. Diese Umkehrung ist kein Zufall, sie hängt daran, dass $x \mapsto x^2$ eine konvexe Funktion ist, und wir kommen in Abschnitt 11.3 darauf zurück.

6. Setzen wir im Widget oben den Kreisring und drücken auf „Gegenbeispiel setzen“. Ab welchem λ verlässt die Strecke die Menge?

Lösung: 0.38

Ab $\lambda \approx 0,380$, und sie kehrt bei 0,620 zurück; dazwischen liegt sie im Loch. Der Mittelpunkt $\lambda = \frac{1}{2}$ hat die Norm $1,1/\sqrt{2} \approx 0,778$ und unterschreitet den Innenradius 0,8 am deutlichsten. Das Widget schärft diese Grenzen per Bisektion nach; für die Widerlegung genügt ohnehin ein einziges λ .

Weitere Beispiele

Beispiel 11.2.4 (Offener Ball)

Für $r > 0$ ist der offene Ball $\mathcal{B} = \{\mathbf{x} \in V : \|\mathbf{x}\| < r\}$ ebenfalls konvex. Sei $\lambda \in [0, 1]$. Mindestens eines der beiden Gewichte λ und $1 - \lambda$ ist echt positiv, und schon deshalb wird die Abschätzung aus dem Selbsttest strikt:

$$\|\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}\| \leq \lambda \|\mathbf{x}\| + (1 - \lambda) \|\mathbf{y}\| < \lambda r + (1 - \lambda) r = r.$$

Der strikte Schritt kommt dabei immer von einem der beiden Summanden: Für $\lambda = 0$ trägt ihn der zweite, sonst der erste. Eine Fallunterscheidung brauchen wir deshalb nicht.

Der nächste Begriff beschreibt die Ecken einer konvexen Menge. In Abschnitt 11.1 sind Extrempunkte über die Gewichte eingeführt worden, als Punkte also, die sich nur mit einem einzigen $w_i = 1$ erreichen lassen. Hier brauchen wir die allgemeine Fassung, denn die folgenden Sätze sprechen über konvexe Mengen, die niemand als Hülle einer Punktwolke gegeben hat.

Definition 11.2.5 (Extrempunkt)

Sei $\mathcal{K} \subseteq V$ konvex. Ein Punkt $\mathbf{z} \in \mathcal{K}$ heißt *Extrempunkt* (extreme point) von $\mathcal{K}$, falls aus

$$\mathbf{z} = \lambda \mathbf{u} + (1 - \lambda) \mathbf{w} \quad \text{mit} \quad \mathbf{u}, \mathbf{w} \in \mathcal{K}, \lambda \in (0, 1)$$

stets $\mathbf{u} = \mathbf{w} = \mathbf{z}$ folgt.

Vertiefung: Warum beide Fassungen dieselben Punkte auswählen

Für endliches $\mathcal{X}$ und $\mathcal{K} = \text{conv}(\mathcal{X})$ stimmt Definition 11.2.5 mit Definition 11.1.7 überein. Sei zunächst $\mathbf{z} \in \mathcal{X}$ nur trivial darstellbar, und sei $\mathbf{z} = \lambda \mathbf{u} + (1 - \lambda) \mathbf{w}$ mit $\mathbf{u}, \mathbf{w} \in \mathcal{K}$ und $\lambda \in (0, 1)$. Setzen wir für $\mathbf{u}$ und $\mathbf{w}$ ihre Darstellungen als Konvexkombinationen von Punkten aus $\mathcal{X}$ ein und sortieren um, so steht rechts wieder eine Konvexkombination von Punkten aus $\mathcal{X}$. Sie muss also die triviale sein, und daraus folgt $\mathbf{u} = \mathbf{w} = \mathbf{z}$.

Umgekehrt lösen wir eine nichttriviale Darstellung nach $\mathbf{z}$ auf und erhalten $\mathbf{z}$ als Konvexkombination der übrigen Punkte. Mindestens zwei von ihnen tragen positives Gewicht, denn ein einzelner Punkt mit Gewicht 1 wäre $\mathbf{z}$ selbst, und der kommt unter den übrigen nicht vor. Spalten wir einen davon ab, so liegt $\mathbf{z}$ im Inneren einer Strecke innerhalb von $\mathcal{K}$. Dasselbe Argument zeigt nebenbei, dass Punkte aus $\mathcal{K} \setminus \mathcal{X}$ von vornherein ausscheiden.

Ein Extrempunkt liegt also auf keiner Strecke, die ganz in der Menge verläuft und ihn im Inneren trifft. Der offene Ball aus Beispiel 11.2.4 hat deshalb gar keinen Extrempunkt: Jeder seiner Punkte ist Mittelpunkt zweier anderer Punkte des Balls. Beim geschlossenen *euklidischen* Ball dagegen ist jeder Randpunkt extrem. Auf die Norm kommt es dabei an: Der Einheitsball der ∞ -Norm im $\mathbb{R}^2$ ist das Quadrat $[-1, 1]^2$, und dessen Randpunkt $(1, 0)^\top$ ist der Mittelpunkt von $(1, -\frac{1}{2})^\top$ und $(1, \frac{1}{2})^\top$, die beide dazugehören. Extrem sind dort nur die vier Ecken.

Beispiel 11.2.6 (Der Simplex)

Der Simplex

$$\Delta^N = \left\{ \mathbf{x} \in \mathbb{R}^N : x_i \geq 0 \text{ für alle } i, \sum_{i=1}^N x_i = 1 \right\}$$

ist konvex. Seien $\mathbf{x}, \mathbf{y} \in \Delta^N$ und $\lambda \in [0, 1]$. Die Komponenten von $\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$ sind als Summe nichtnegativer Zahlen wieder nichtnegativ, und für die Gesamtsumme gilt

$$\sum_{i=1}^N (\lambda x_i + (1 - \lambda) y_i) = \lambda \sum_{i=1}^N x_i + (1 - \lambda) \sum_{i=1}^N y_i = \lambda \cdot 1 + (1 - \lambda) \cdot 1 = 1.$$

Extrempunkte. Die Extrempunkte von Δ^N sind genau die Standard-Basisvektoren $\mathbf{e}_1, \dots, \mathbf{e}_N$. Dass $\mathbf{e}_1$ extrem ist, sieht man so: Gilt $\mathbf{e}_1 = \lambda \mathbf{u} + (1 - \lambda) \mathbf{w}$ mit $\mathbf{u}, \mathbf{w} \in \Delta^N$ und $\lambda \in (0, 1)$, so ist für jedes $i \geq 2$ die Zahl $\lambda u_i + (1 - \lambda) w_i = 0$ eine Summe zweier nichtnegativer Terme mit positiven Vorfaktoren, also $u_i = w_i = 0$; weil sich die Komponenten zu eins addieren, bleibt $u_1 = w_1 = 1$. Für die übrigen $\mathbf{e}_i$ läuft das Argument genauso, und umgekehrt ist jeder Punkt mit zwei positiven Komponenten $x_j, x_k > 0, j \neq k$, der Mittelpunkt von $\mathbf{x} \pm \varepsilon(\mathbf{e}_j - \mathbf{e}_k)$ und deshalb nicht extrem. Dass die Ecken den Simplex auch aufspannen, sieht man ohne Umweg: Jedes $\mathbf{x} \in \Delta^N$ ist $\mathbf{x} = \sum_{i=1}^N x_i \mathbf{e}_i$, und die x_i sind gerade nichtnegative Gewichte mit Summe eins. Also ist $\Delta^N = \text{conv}\{\mathbf{e}_1, \dots, \mathbf{e}_N\}$.

Geometrie. Δ^2 ist die Strecke von $(1, 0)^\top$ nach $(0, 1)^\top$, Δ^3 das Dreieck mit den Ecken $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$ im $\mathbb{R}^3$, Δ^4 ein Tetraeder. Der Simplex ist außerdem abgeschlossen und beschränkt, denn er ist durch Gleichungen und nichtstrikte Ungleichungen beschrieben und erfüllt $\|\mathbf{x}\|_1 = 1$.

Warum das in der Statistik zählt. Ein Punkt von Δ^N ist nichts anderes als eine Wahrscheinlichkeitsverteilung über N Ereignisse. Der Vektor $\mathbf{p} = (0,2; 0,5; 0,3)^\top$ beschreibt eine kategoriale Verteilung mit drei Kategorien, und die Extrempunkte $\mathbf{e}_i$ sind die entarteten Verteilungen, die ihre ganze Masse auf eine Kategorie legen. Modelle, die Verteilungen vorhersagen, müssen deshalb in den Simplex hinein abbilden. Die übliche Wahl ist die Softmax-Abbildung

$$\text{SoftMax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_{j=1}^N \exp(z_j)},$$

deren Werte nichtnegativ sind und sich nach Konstruktion zu eins addieren. Für $\mathbf{z} = (1; 2; 0,5)^\top$ etwa liefert sie $(0,231; 0,629; 0,140)^\top$. Alle Komponenten sind echt positiv, die Ecken werden also nie erreicht, sondern nur im Grenzwert angesteuert. Dieselbe Menge trägt die multinomiale Regression und die Dirichlet-Verteilung, die Verteilungen über Δ^N beschreibt.

Der Kegel der positiv semidefiniten Matrizen

Das dritte Beispiel lebt nicht im $\mathbb{R}^n$, sondern in einem Raum von Matrizen. Das ist kein Sonderfall, denn die $n \times n$ -Matrizen bilden mit der üblichen Addition und Skalarmultiplikation selbst einen Vektorraum, und Definition 11.2.1 fragt nach nichts anderem.

Definition 11.2.7 (Positiv semidefinit)

Eine symmetrische Matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ heißt *positiv semidefinit* (PSD), falls

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0 \quad \text{für alle } \mathbf{x} \in \mathbb{R}^n.$$

Wir schreiben dafür $\mathbf{A} \succeq 0$ und sammeln alle diese Matrizen in

$$\mathcal{P}_n := \{\mathbf{A} \in \mathbb{R}^{n \times n} : \mathbf{A} = \mathbf{A}^\top, \mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0 \quad \forall \mathbf{x} \in \mathbb{R}^n\}.$$

Die Symmetrie ist dabei keine Nebensache. Die quadratische Form $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ sieht von $\mathbf{A}$ nur den symmetrischen Anteil $(\mathbf{A} + \mathbf{A}^\top)/2$, denn der schiefsymmetrische Teil liefert stets null; ohne Symmetrie wäre $\mathbf{A} \succeq 0$ also gar keine Eigenschaft von $\mathbf{A}$ allein. Vor allem hängen die Aussagen, für die wir Semidefinitheit brauchen, an ihr: Nur für symmetrische Matrizen gibt der Spektralsatz die Äquivalenz zu „alle Eigenwerte sind nichtnegativ“ her, und nur dort greift das Cholesky-Verfahren aus Abschnitt 5.4.

Satz 11.2.8 (Die positiv semidefiniten Matrizen bilden eine konvexe Menge)

$\mathcal{P}_n$ ist konvex.

Beweis.

- (1) Seien $\mathbf{A}_1, \mathbf{A}_2 \in \mathcal{P}_n$ und $\lambda \in [0, 1]$. Wir setzen $\mathbf{A}_\lambda := \lambda \mathbf{A}_1 + (1 - \lambda) \mathbf{A}_2$ und prüfen zuerst die Symmetrie:

$$\mathbf{A}_\lambda^\top = \lambda \mathbf{A}_1^\top + (1 - \lambda) \mathbf{A}_2^\top = \lambda \mathbf{A}_1 + (1 - \lambda) \mathbf{A}_2 = \mathbf{A}_\lambda.$$

Transponieren ist linear, und beide Summanden sind nach Voraussetzung symmetrisch

- (2) Sei nun $\mathbf{x} \in \mathbb{R}^n$ beliebig. Dann ist

$$\begin{aligned} \mathbf{x}^\top \mathbf{A}_\lambda \mathbf{x} &= \mathbf{x}^\top [\lambda \mathbf{A}_1 + (1 - \lambda) \mathbf{A}_2] \mathbf{x} \\ &= \lambda \mathbf{x}^\top \mathbf{A}_1 \mathbf{x} + (1 - \lambda) \mathbf{x}^\top \mathbf{A}_2 \mathbf{x} \\ &\geq \lambda \cdot 0 + (1 - \lambda) \cdot 0 = 0. \end{aligned}$$

Damit ist $\mathbf{A}_\lambda \in \mathcal{P}_n$.

die mittlere Zeile nutzt nur, dass das Matrix-Vektor-Produkt in der Matrix linear ist; die Abschätzung nutzt $\mathbf{A}_1, \mathbf{A}_2 \succeq 0$ und $\lambda, 1 - \lambda \geq 0$

□

Bemerkung 11.2.9 (Kovarianzmatrizen sind semidefinit, nicht immer definit)

Als Anwendung liest man häufig, Kovarianzmatrizen seien SPD, also symmetrisch und positiv *definit*, und bildeten deshalb eine konvexe Menge. Die Konvexität stimmt, die Begründung greift zu hoch.

Richtig ist: Kovarianzmatrizen sind stets symmetrisch und positiv *semidefinit*. Für einen Zufallsvektor $\mathbf{X}$ mit Kovarianzmatrix Σ und beliebiges $\mathbf{a} \in \mathbb{R}^n$ gilt

$$\mathbf{a}^\top \Sigma \mathbf{a} = \text{Var}(\mathbf{a}^\top \mathbf{X}) \geq 0,$$

und eine Varianz ist nie negativ. Positiv definit ist Σ dagegen nur, wenn es keine lineare Degeneration gibt, wenn also kein $\mathbf{a} \neq \mathbf{0}$ existiert, für das $\mathbf{a}^\top \mathbf{X}$ fast sicher konstant ist. Ein Gegenbeispiel ist schnell gebaut: Für $\mathbf{X} = (Z, Z)^\top$ mit $\text{Var}(Z) = 1$ ist

$$\Sigma = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{a} = (1, -1)^\top, \quad \mathbf{a}^\top \Sigma \mathbf{a} = 0,$$

und die Eigenwerte sind 2 und 0. In der Praxis passiert genau das, sobald zwei Merkmale exakt linear zusammenhängen oder eine empirische Kovarianzmatrix aus zu wenigen Beobachtungen geschätzt wird. Das ist kein Haarspalten: Das Cholesky-Verfahren aus Abschnitt 5.4 verlangt positive Definitheit und scheitert im degenerierten Fall.

Die konvexe Menge, die hier gemeint ist, ist also $\mathcal{P}_n$, der Kegel der semidefiniten Matrizen. Die positiv definiten Matrizen bilden übrigens ebenfalls eine konvexe Menge, mit demselben Beweis und strikten Ungleichungen; sie ist das Innere von $\mathcal{P}_n$ innerhalb des Raums der symmetrischen Matrizen. Im vollen $\mathbb{R}^{n \times n}$ hätte $\mathcal{P}_n$ dagegen gar kein Inneres, denn die Forderung $\mathbf{A} = \mathbf{A}^\top$ sperrt sie in einen echten Untervektorraum.

Für $n = 2$ lässt sich $\mathcal{P}_2$ vollständig hinzeichnen, denn eine symmetrische 2×2 -Matrix besteht aus nur drei Zahlen: a, b und c in $\mathbf{A} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$. Wie sieht die Menge der semidefiniten unter ihnen aus, und woran erkennen wir ihren Rand?

Interaktiv: Der Kegel zum Anfassen

Die Bedingung aus Definition 11.2.7 lässt sich für $n = 2$ in drei Ungleichungen übersetzen: $a \geq 0$, $c \geq 0$ und $ac \geq b^2$. Die linke Tafel zeigt davon den Schnitt bei festem b , die rechte die ganze Menge im Raum der drei Zahlen.

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, ist $\mathcal{P}_2$ ein Kegel mit gekrümmtem Rand. Kegel heißt: Mit $\mathbf{A}$ gehört der ganze Halbstrahl $t\mathbf{A}$, $t \geq 0$, dazu, denn alle drei Bedingungen sind homogen. Der Rand $ac = b^2$ besteht genau aus den semidefiniten Matrizen mit Determinante 0, also denen vom Rang 1 oder 0; dort ist der kleinere Eigenwert gerade null. Innen liegen die positiv definiten Matrizen, außen die indefiniten und die negativ semidefiniten. Die drei Voreinstellungen mit Spur 2 machen den Unterschied an Zahlen fest: Die Einheitsmatrix hat die Eigenwerte 1 und 1, die Rang-1-Matrix mit $b = 1$ die Eigenwerte 0 und 2, und mit $b = 2$ kippt es auf -1 und 3. Satz 11.2.8 ist im Bild die Aussage, dass die Strecke zwischen zwei Punkten des Kegels nie unter die gekrümmte Randfläche taucht.

Nebenbei fällt eine hübsche Beobachtung ab: Schneiden wir den Kegel mit der Ebene $a+c = 2$, so entsteht genau die Einheitskreisscheibe $\left(\frac{a-c}{2}\right)^2 + b^2 \leq 1$. Alle Matrizen mit Spur 2, die im Kegel liegen, sind also so angeordnet wie die Punkte einer Kreisscheibe.

1. Stellen wir im Kegel-Widget $a = c = 1$ und $b = 1$ ein. Wie groß ist der kleinere Eigenwert von $\mathbf{A}$?

Lösung: 0

Null. Für $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$ ist $\det \mathbf{A} = 1 \cdot 1 - 1^2 = 0$, die Eigenwerte sind 0 und 2. Die Matrix ist semidefinit, aber nicht definit, und liegt deshalb auf dem Rand des Kegels. Genau diese Matrix taucht in Bemerkung 11.2.9 als Kovarianzmatrix von $(Z, Z)^\top$ wieder auf.

2. Weil $\mathcal{P}_2$ ein Kegel ist, gehört mit $\mathbf{A}$ auch $-\mathbf{A}$ dazu.

Antwort: Falsch

Ein Kegel im hier gemeinten Sinn ist unter Multiplikation mit *nichtnegativen* Skalaren abgeschlossen, nicht unter Vorzeichenwechsel. Im Widget liegt $-\mathbf{A}$ im gespiegelten Kegel, und bei-

de berühren sich nur in der Nullmatrix. Wäre $\mathcal{P}_2$ unter $t < 0$ abgeschlossen, so wäre die Menge ein Untervektorraum, und $\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$ könnte nur für $\mathbf{A} = \mathbf{0}$ gelten.

Operationen, die Konvexität erhalten

Bis hierhin haben wir jede Menge einzeln geprüft. Das wird schnell mühsam. Bequemer ist es, wenige Grundbausteine als konvex nachzuweisen und dann Regeln zu haben, die aus konvexen Mengen neue konvexe Mengen machen. Vier solche Regeln stellen wir zusammen.

Satz 11.2.10 (Konvexitäts-erhaltung)

Sei $\mathcal{X}$ konvex und sei $(\mathcal{X}_i)_{i \in I}$ eine Familie konvexer Mengen über einer beliebigen Indexmenge I . Dann sind auch die folgenden Mengen konvex:

1. *Schnitt*: $\mathcal{S} = \bigcap_{i \in I} \mathcal{X}_i$, sofern alle $\mathcal{X}_i$ im selben Vektorraum liegen;
2. *Kartesisches Produkt*: $\mathcal{X}_1 \times \dots \times \mathcal{X}_k$, hier dürfen die Faktoren in verschiedenen Vektorräumen leben, und das Produkt trägt die komponentenweisen Verknüpfungen;
3. *Bild unter einer affinen Abbildung*: Für $\mathcal{X} \subseteq \mathbb{R}^n$ und $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, $f(\mathbf{x}) = \mathbf{A}\mathbf{x} + \mathbf{b}$, ist $f(\mathcal{X}) = \{\mathbf{y} = f(\mathbf{x}) : \mathbf{x} \in \mathcal{X}\}$ konvex;
4. *Summe*: $\mathcal{X}_1 \oplus \mathcal{X}_2 := \{\mathbf{x}_1 + \mathbf{x}_2 : \mathbf{x}_1 \in \mathcal{X}_1, \mathbf{x}_2 \in \mathcal{X}_2\}$ für $\mathcal{X}_1, \mathcal{X}_2 \subseteq \mathbb{R}^n$.

Vertiefung: Beweis der Konvexitäts-erhaltung

Beweis.

- (1) *Schnitt*. Seien $\mathbf{x}, \mathbf{y} \in \mathcal{S}$, $\lambda \in [0, 1]$ und $\mathbf{z} = \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$. Aus $\mathbf{x}, \mathbf{y} \in \mathcal{S}$ folgt $\mathbf{x}, \mathbf{y} \in \mathcal{X}_i$ für jedes $i \in I$. Weil jedes $\mathcal{X}_i$ konvex ist, liegt $\mathbf{z}$ in jedem $\mathcal{X}_i$, also in ihrem Schnitt.

hier wird über die Indexmenge nichts vorausgesetzt; das Argument läuft für endlich, abzählbar oder überabzählbar viele Mengen gleich

- (2) *Kartesisches Produkt*. Seien $(\mathbf{x}_1, \dots, \mathbf{x}_k)$ und $(\mathbf{y}_1, \dots, \mathbf{y}_k)$ zwei Elemente des Produkts und $\lambda \in [0, 1]$. Addition und Skalarmultiplikation laufen im Produktraum komponentenweise, also ist

$$\begin{aligned} & \lambda(\mathbf{x}_1, \dots, \mathbf{x}_k) + (1 - \lambda)(\mathbf{y}_1, \dots, \mathbf{y}_k) \\ &= (\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{y}_1, \dots, \lambda \mathbf{x}_k + (1 - \lambda) \mathbf{y}_k) \in \mathcal{X}_1 \times \dots \times \mathcal{X}_k, \end{aligned}$$

denn die j -te Komponente liegt nach Voraussetzung in $\mathcal{X}_j$.

jede Komponente wird für sich gemischt, also greift die Konvexität von $\mathcal{X}_j$ genau auf der j -ten Stelle

- (3) *Affines Bild*. Seien $\mathbf{y}_1 = f(\mathbf{x}_1)$ und $\mathbf{y}_2 = f(\mathbf{x}_2)$ zwei Punkte von $f(\mathcal{X})$ mit $\mathbf{x}_1, \mathbf{x}_2 \in \mathcal{X}$, und sei $\lambda \in [0, 1]$. Dann ist

$$\begin{aligned} \lambda \mathbf{y}_1 + (1 - \lambda) \mathbf{y}_2 &= \lambda(\mathbf{A}\mathbf{x}_1 + \mathbf{b}) + (1 - \lambda)(\mathbf{A}\mathbf{x}_2 + \mathbf{b}) \\ &= \mathbf{A}(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) + \mathbf{b} \\ &= f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \in f(\mathcal{X}). \end{aligned}$$

in der mittleren Zeile heben sich die beiden $\mathbf{b}$ -Anteile zu einem einzigen $\mathbf{b}$ auf, weil $\lambda + (1 - \lambda) = 1$ ist; genau dafür braucht es Konvex- statt Linearkombinationen

- (4) *Summe*. Das folgt aus den beiden vorigen Punkten. Nach Schritt 2 ist $\mathcal{X}_1 \times \mathcal{X}_2 \subseteq \mathbb{R}^{2n}$ konvex, und $f(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1 + \mathbf{x}_2$ ist linear, in Matrixform $f(\mathbf{z}) = (\mathbf{I}_n \ \mathbf{I}_n) \mathbf{z}$. Nach Schritt 3 ist das Bild konvex, und dieses Bild ist gerade $\mathcal{X}_1 \oplus \mathcal{X}_2$.

Schritt 3 mit $\mathbf{A} = (\mathbf{I}_n \ \mathbf{I}_n) \in \mathbb{R}^{n \times 2n}$ und $\mathbf{b} = \mathbf{0}$

□

Bemerkung 11.2.11 (Drei Feinheiten zum Satz)

Das Produkt heißt *kartesisch*. Punkt 2 meint das kartesische Produkt der Mengen, also die Menge aller Tupel. Mit dem *äußeren* Produkt $\mathbf{v} \otimes \mathbf{w} = \mathbf{vw}^\top$ aus Abschnitt 9.3 hat es nichts zu tun; die Namen ähneln sich, die Begriffe nicht. Wir schreiben deshalb durchgehend *kartesisches* Produkt.

Die Abbildung ist *affin*. In Punkt 3 steht $f(\mathbf{x}) = \mathbf{Ax} + \mathbf{b}$. Das ist eine affine Abbildung, und linear ist sie nur für $\mathbf{b} = \mathbf{0}$. Der Beweis benutzt tatsächlich nur die Affinität, die Aussage gilt also in dieser allgemeineren Form. Der Verschiebungsanteil ist übrigens der Grund, warum Konvexkombinationen und nicht beliebige Linearkombinationen betrachtet werden: Erst $\lambda + (1 - \lambda) = 1$ lässt aus zwei $\mathbf{b}$ -Anteilen wieder genau ein $\mathbf{b}$ werden.

Der Schnitt darf *beliebig groß sein*. Punkt 1 lässt eine beliebige Indexmenge I zu und nicht nur eine Abzählung $\bigcap_{i=1}^\infty$. Der Beweis kennt gar keine Abzählung, und wir brauchen die allgemeine Fassung gleich: Satz 11.2.13 schneidet über *alle* konvexen Obermengen einer Menge, und das sind im Allgemeinen überabzählbar viele.

Beispiel 11.2.12 (Ein Dreieck als Schnitt dreier Halbräume)

Im $\mathbb{R}^2$ betrachten wir die drei Mengen

$$\mathcal{X}_1 = \{(x, y) : x + y \leq 1\}, \quad \mathcal{X}_2 = \{(x, y) : x \geq 0\}, \quad \mathcal{X}_3 = \{(x, y) : y \geq 0\}.$$

Jede von ihnen ist ein *Halbraum*, also eine der beiden Seiten einer Hyperebene und damit von der Form $\{\mathbf{z} : \mathbf{a}^\top \mathbf{z} \leq \beta\}$; für $\mathcal{X}_2$ etwa mit $\mathbf{a} = (-1, 0)^\top$ und $\beta = 0$. Jeder Halbraum ist konvex:

$$\mathbf{a}^\top (\lambda \mathbf{z}_1 + (1 - \lambda) \mathbf{z}_2) = \lambda \mathbf{a}^\top \mathbf{z}_1 + (1 - \lambda) \mathbf{a}^\top \mathbf{z}_2 \leq \lambda \beta + (1 - \lambda) \beta = \beta.$$

Nach Satz 11.2.10(1) ist damit auch der Schnitt konvex, und der ist

$$\mathcal{S} = \mathcal{X}_1 \cap \mathcal{X}_2 \cap \mathcal{X}_3 = \{(x, y) : x \geq 0, y \geq 0, x + y \leq 1\},$$

also das Dreieck mit den Ecken $(0, 0)$, $(1, 0)$ und $(0, 1)$. Genau diese Menge steckt hinter dem Knopf „Dreieck“ im Widget oben, samt ihren drei orangen Extrempunkten.

Das Muster trägt weit. Endliche Schnitte von Halbräumen heißen *Polyeder*, und die zulässigen Mengen linearer Optimierungsprobleme sind von dieser Bauart: endlich viele lineare Ungleichungen, jede ein Halbraum, zusammen ein konvexes Polyeder. Die Optimierung unter solchen Nebenbedingungen ist Thema von Abschnitt 12.5.

Die konvexe Hülle als kleinste konvexe Obermenge

In Abschnitt 11.1 ist $\text{conv}(\mathcal{X})$ als Menge aller Konvexkombinationen endlich vieler Punkte aus $\mathcal{X}$ eingeführt worden, mit dem Zusatz, sie sei die kleinste konvexe Menge, die $\mathcal{X}$ enthält. Der folgende Satz sagt genau das, und zwar in einer Form, die sich beweisen lässt.

Satz 11.2.13 (Konvexe Hülle als Durchschnitt)

Für jede Menge $\mathcal{X} \subseteq V$ gilt

$$\text{conv}(\mathcal{X}) = \bigcap_{\mathcal{X} \subseteq \mathcal{Y}, \mathcal{Y} \text{ konvex}} \mathcal{Y}. \quad (11.2.1)$$

Beweis.

- (1) Zuerst halten wir fest, dass $\text{conv}(\mathcal{X})$ überhaupt konvex ist. Der Name legt es nahe, nachzurechnen ist es trotzdem. Seien $\mathbf{z} = \sum_{i=1}^N w_i \mathbf{x}_i$ und $\mathbf{z}' = \sum_{j=1}^M w'_j \mathbf{x}'_j$ zwei Konvexkombinationen von Punkten aus $\mathcal{X}$ und $\lambda \in [0, 1]$. Dann ist

$$\lambda \mathbf{z} + (1 - \lambda) \mathbf{z}' = \sum_{i=1}^N \lambda w_i \mathbf{x}_i + \sum_{j=1}^M (1 - \lambda) w'_j \mathbf{x}'_j$$

wieder eine Konvexkombination von endlich vielen Punkten aus $\mathcal{X}$: Alle Gewichte sind nicht-negativ, und ihre Summe ist $\lambda \cdot 1 + (1 - \lambda) \cdot 1 = 1$.

Konvexkombinationen von Konvexkombinationen sind wieder Konvexkombinationen; das ist dieselbe Rechnung wie im Induktionsschritt zu Satz 11.2.3, nur in die andere Richtung gelesen

- (2) Sei $\mathcal{S}$ die rechte Seite von (11.2.1). Über wie viele Mengen dieser Schnitt läuft, brauchen wir nicht zu wissen; leer ist die Familie jedenfalls nicht, denn V selbst ist konvex und enthält $\mathcal{X}$. Wir zeigen $\text{conv}(\mathcal{X}) \subseteq \mathcal{S}$. Sei dazu $\mathcal{Y}$ irgendeine konvexe Menge mit $\mathcal{X} \subseteq \mathcal{Y}$, und sei $\mathbf{z} \in \text{conv}(\mathcal{X})$, also $\mathbf{z} = \sum_i w_i \mathbf{x}_i$ mit $\mathbf{x}_i \in \mathcal{X}$. Dann sind alle $\mathbf{x}_i$ auch Punkte von $\mathcal{Y}$, und weil $\mathcal{Y}$ konvex ist, liegt nach Satz 11.2.3 die ganze Konvexkombination in $\mathcal{Y}$. Das gilt für jedes solche $\mathcal{Y}$, also auch für den Schnitt.

genau hier wird Satz 11.2.3 gebraucht: Die Definition der Konvexität spricht nur über zwei Punkte, die Konvexkombination hat aber N Summanden

- (3) Umgekehrt gilt $\mathcal{S} \subseteq \text{conv}(\mathcal{X})$. Nach Schritt 1 ist $\mathcal{Y} = \text{conv}(\mathcal{X})$ selbst eine konvexe Menge, die $\mathcal{X}$ enthält, denn jedes $\mathbf{x} \in \mathcal{X}$ ist die Einerkombination seiner selbst. Der Schnitt läuft also unter anderem über diese Menge und ist damit in ihr enthalten.

ein Durchschnitt ist in jedem seiner Teilnehmer enthalten; der Trick ist nur, $\text{conv}(\mathcal{X})$ als einen dieser Teilnehmer zu erkennen

- (4) Beide Inklusionen zusammen geben (11.2.1). Damit ist $\text{conv}(\mathcal{X})$ in einem präzisen Sinn die kleinste konvexe Obermenge von $\mathcal{X}$: Sie ist konvex, sie enthält $\mathcal{X}$, und sie steckt in jeder anderen Menge mit diesen beiden Eigenschaften.

Schritt 2 sagt „steckt in jeder anderen“, Schritt 1 und die Einerkombination sagen „ist selbst eine solche“

□

Anwendung: lineare Ziele werden in Ecken angenommen

Zum Abschluss eine Anwendung. Sie erklärt, warum sich der ganze Aufwand mit Hüllen und Extrempunkten in der Optimierung lohnt.

Sei $\mathcal{X} \subseteq \mathbb{N}^n$ eine Menge von Zusammenstellungen: Ein Vektor $\mathbf{x} \in \mathcal{X}$ sagt mit $x_i = k$, dass k Einheiten des i -ten Gegenstands vorhanden sind. Jeder Gegenstand hat einen Wert v_i , und gesucht ist die wertvollste zulässige Zusammenstellung,

$$\mathbf{x}^* \in \arg \max_{\mathbf{x} \in \mathcal{X}} \sum_{i=1}^n v_i x_i = \arg \max_{\mathbf{x} \in \mathcal{X}} \mathbf{v}^\top \mathbf{x}.$$

Das Elementzeichen ist mit Bedacht gesetzt: Das Maximum ist der erreichte *Wert* und nicht die Stelle, an der er angenommen wird; gesucht ist das Argmax, und mehrdeutig ist es obendrein.

Ist $\mathcal{X}$ groß, so ist das Absuchen aller Vektoren aussichtslos. Der folgende Satz verkleinert die Suche drastisch.

Satz 11.2.14 (Lineare Ziele und Extrempunkte)

Sei $\mathcal{X} \subset \mathbb{R}^n$ endlich und nichtleer, sei $\mathbf{v} \in \mathbb{R}^n$ und $M := \max_{\mathbf{x} \in \mathcal{X}} \mathbf{v}^\top \mathbf{x}$. Dann gilt:

1. $\max_{\mathbf{z} \in \text{conv}(\mathcal{X})} \mathbf{v}^\top \mathbf{z} = M$, das Maximum wächst durch den Übergang zur konvexen Hülle also nicht;
2. unter den Maximierern in $\mathcal{X}$ gibt es einen Extrempunkt $\mathbf{x}^*$ von $\text{conv}(\mathcal{X})$;
3. jeder Extrempunkt von $\text{conv}(\mathcal{X})$ liegt in $\mathcal{X}$.

Vertiefung: Der Beweis des Extrempunkt-Satzes

Für die Anwendung des Satzes genügt seine Aussage. Wer wissen will, wie der Nachweis läuft, findet ihn hier; der technische Kern ist ein Existenzbeweis für Extrempunkte.

Beweis.

- (1) Zu 1. Wegen $\mathcal{X} \subseteq \text{conv}(\mathcal{X})$ ist das Maximum über die Hülle mindestens M . Umgekehrt sei $\mathbf{z} = \sum_i w_i \mathbf{x}_i \in \text{conv}(\mathcal{X})$. Dann ist

$$\mathbf{v}^\top \mathbf{z} = \sum_i w_i \mathbf{v}^\top \mathbf{x}_i \leq \sum_i w_i M = M,$$

also ist das Maximum über die Hülle höchstens M .

ein gewichteter Durchschnitt von Zahlen, die alle höchstens M sind, ist höchstens M ; die Gewichte addieren sich zu eins

- (2) Sei $\mathcal{X}_M := \{\mathbf{x} \in \mathcal{X} : \mathbf{v}^\top \mathbf{x} = M\}$ die Menge der Maximierer und $\mathcal{F} := \{\mathbf{z} \in \text{conv}(\mathcal{X}) : \mathbf{v}^\top \mathbf{z} = M\}$. Wir zeigen $\mathcal{F} = \text{conv}(\mathcal{X}_M)$. Von rechts nach links rechnen wir direkt nach: Für $\mathbf{z} = \sum_i w_i \mathbf{x}_i$ mit $\mathbf{x}_i \in \mathcal{X}_M$ ist $\mathbf{v}^\top \mathbf{z} = \sum_i w_i M = M$. Für die andere Richtung sei $\mathbf{z} = \sum_i w_i \mathbf{x}_i \in \mathcal{F}$ mit $\mathbf{x}_i \in \mathcal{X}$ und $w_i > 0$. Aus $\sum_i w_i (M - \mathbf{v}^\top \mathbf{x}_i) = M - \mathbf{v}^\top \mathbf{z} = 0$ und $M - \mathbf{v}^\top \mathbf{x}_i \geq 0$ folgt $\mathbf{v}^\top \mathbf{x}_i = M$ für jedes dieser i .

eine Summe nichtnegativer Terme mit positiven Gewichten ist nur dann null, wenn jeder Term null ist

- (3) Zu 2, in der Skizze. Unter den endlich vielen Punkten von $\mathcal{X}_M$ wählen wir einen mit größter euklidischer Norm und nennen ihn $\mathbf{x}^*$. Zerlegen wir ihn als $\mathbf{x}^* = \lambda \mathbf{u} + (1 - \lambda) \mathbf{w}$ mit $\mathbf{u}, \mathbf{w} \in \text{conv}(\mathcal{X}_M)$ und $\lambda \in (0, 1)$, so liefert Ausmultiplizieren

$$\lambda \|\mathbf{u}\|^2 + (1 - \lambda) \|\mathbf{w}\|^2 - \|\lambda \mathbf{u} + (1 - \lambda) \mathbf{w}\|^2 = \lambda(1 - \lambda) \|\mathbf{u} - \mathbf{w}\|^2,$$

und links steht höchstens null, rechts mindestens null. Also sind beide Seiten null und $\mathbf{u} = \mathbf{w} = \mathbf{x}^*$, das heißt $\mathbf{x}^*$ ist Extrempunkt von $\text{conv}(\mathcal{X}_M)$. Für eine Zerlegung innerhalb von $\text{conv}(\mathcal{X})$ zwingt Schritt 1 die beiden Teilpunkte ohnehin nach $\mathcal{F} = \text{conv}(\mathcal{X}_M)$, und damit ist $\mathbf{x}^*$ auch dort extrem.

die Identität entsteht aus $\|\lambda \mathbf{u} + (1 - \lambda) \mathbf{w}\|^2 = \lambda^2 \|\mathbf{u}\|^2 + 2\lambda(1 - \lambda) \mathbf{u}^\top \mathbf{w} + (1 - \lambda)^2 \|\mathbf{w}\|^2$ und $\lambda - \lambda^2 = \lambda(1 - \lambda)$; links ist höchstens null, weil $\|\mathbf{u}\|$ und $\|\mathbf{w}\|$ die maximale Norm $\|\mathbf{x}^\|$ nicht überschreiten*

- (4) Zu 3. Sei $\mathbf{z}$ ein Extrempunkt von $\text{conv}(\mathcal{X})$, geschrieben als $\mathbf{z} = \sum_{i=1}^N w_i \mathbf{x}_i$ mit $\mathbf{x}_i \in \mathcal{X}$ und $w_i > 0$. Für $N = 1$ ist $\mathbf{z} = \mathbf{x}_1 \in \mathcal{X}$, und für $N \geq 2$ lesen wir $\mathbf{z} = w_1 \mathbf{x}_1 + (1 - w_1) \mathbf{u}$ mit $\mathbf{u} = \sum_{i=2}^N \frac{w_i}{1 - w_1} \mathbf{x}_i \in \text{conv}(\mathcal{X})$ als Zweierkombination; weil $\mathbf{z}$ extrem ist, folgt wieder $\mathbf{z} = \mathbf{x}_1 \in \mathcal{X}$.

dieselbe Normierung wie im Beweis zu Satz 11.2.3, hier benutzt, um eine N -Kombination als Zweierkombination zu lesen

□

Beispiel 11.2.15 (Auswahl unter einer Budgetschränke)

Zwei Gegenstände, der erste kostet 2 und ist 5 wert, der zweite kostet 3 und ist 8 wert. Das Budget beträgt 12, ganze Einheiten sind zu wählen:

$$\mathcal{X} = \{\mathbf{x} \in \mathbb{N}^2 : 2x_1 + 3x_2 \leq 12\}, \quad \mathbf{v} = (5, 8)^\top.$$

Diese Menge hat 19 Elemente. Ihre konvexe Hülle ist das Dreieck $\{\mathbf{x} \in \mathbb{R}^2 : x_1 \geq 0, x_2 \geq 0, 2x_1 + 3x_2 \leq 12\}$ mit den drei Extrempunkten

$$(0, 0), \quad (6, 0), \quad (0, 4),$$

denn alle Punkte von $\mathcal{X}$ erfüllen die drei Ungleichungen, und die drei Ecken liegen selbst in $\mathcal{X}$. Die Zielwerte dort sind 0, 30 und 32. Nach Satz 11.2.14 ist $\mathbf{x}^* = (0, 4)$ mit dem Wert 32 eine Lösung, und wir haben dafür drei Punkte geprüft statt neunzehn.

Lehrreich ist der Zweitplatzierte. Der Punkt $(3, 2)$ schöpft das Budget voll aus ($2 \cdot 3 + 3 \cdot 2 = 12$) und erreicht den Wert 31; er liegt damit auf der Kante von $\text{conv}(\mathcal{X})$ zwischen $(6, 0)$ und $(0, 4)$. Ein Extrempunkt ist er trotzdem nicht, denn er ist genau deren Mittelpunkt. Wer nur die Ecken absucht, übersieht ihn und verliert dadurch nichts.

Bemerkung 11.2.16 (Was der Satz leistet und was nicht)

Nicht jeder Maximierer ist ein Extrempunkt. Satz 11.2.14(2) verspricht die Existenz eines solchen, nicht mehr: Bei $\mathcal{X} = \{(0, 0), (1, 0), (2, 0)\}$ und $\mathbf{v} = (0, 1)^\top$ sind alle drei Punkte Maximierer, der mittlere ist aber der Mittelpunkt der beiden äußeren. Bei Gleichständen taugt der Satz zum Finden einer Lösung, nicht zum Beschreiben aller. Und die Endlichkeit ist nicht kosmetisch: Der offene Ball aus Beispiel 11.2.4 hat gar keine Extrempunkte, und für $\mathbf{v} \neq \mathbf{0}$ erreicht $\mathbf{v}^\top \mathbf{x}$ dort sein Supremum nie.

Die Arbeit verschiebt sich, sie verschwindet nicht. Der Satz ersetzt die Suche über $\mathcal{X}$ durch die Suche über die Extrempunkte von $\text{conv}(\mathcal{X})$, und das nützt nur, wenn wir diese Hülle beschreiben können. In Beispiel 11.2.15 war sie leicht zu bekommen; für allgemeine Teilmengen von $\mathbb{N}^n$ ist sie dagegen selbst schwer zu bestimmen, und genau darin liegt die Schwierigkeit der ganzzahligen Optimierung. Ist die zulässige Menge von vornherein ein Polyeder wie in Beispiel 11.2.12, so ist die Suche über die Ecken die Grundidee der linearen Programmierung. Dieses Verfahren behandelt das Skript nicht; Abschnitt 12.5 geht die Optimierung unter Nebenbedingungen über Lagrange-Multiplikatoren und die Karush-Kuhn-Tucker-Bedingungen an.

Vertiefung: Boyd und Vandenberghe, *Convex Optimization*, §2.2 und §2.3 behandeln dieselben Themen ausführlicher, mit §2.1 zu affinen und konvexen Mengen, §2.2 zu den wichtigen Beispielen einschließlich Simplex und semidefinitem Kegel und §2.3 zu den Operationen, die Konvexität erhalten.

11.3 Projektion und konvexe Funktionen

Der nächste Punkt einer konvexen Menge

Abschnitt 11.2 hat konvexe Mengen von innen betrachtet: Zwei Punkte drin, die ganze Strecke drin. Jetzt schauen wir von außen darauf. Gegeben ein Punkt $\mathbf{x}$, der nicht in der Menge liegt: Welcher Punkt der Menge kommt ihm am nächsten?

Die Frage ist keine Spielerei. Verfahren für Optimierungsprobleme mit Nebenbedingungen rechnen einen Schritt oft erst frei aus und holen das Ergebnis danach in die zulässige Menge zurück, und zwar an die nächstgelegene Stelle. Auch die Kleinste-Quadrate-Rechnung aus Abschnitt 7.1 ist eine solche Aufgabe: Dort suchen wir im Spaltenraum von $\mathbf{A}$ den Punkt mit dem kleinsten Abstand zu $\mathbf{y}$. Damit ein Verfahren darauf bauen kann, brauchen wir zwei Zusagen. Es muss einen nächstgelegenen Punkt überhaupt geben, und es darf nur einen geben. Beides liefert Konvexität.

Satz 11.3.1 (Projektionstheorem)

Sei V ein endlichdimensionaler Skalarproduktraum und $\mathcal{X} \subseteq V$ eine nichtleere, abgeschlossene und konvexe Teilmenge. Dann gibt es zu jedem $\mathbf{x} \in V$ genau einen Punkt $\hat{\mathbf{x}} \in \mathcal{X}$ mit

$$\|\mathbf{x} - \hat{\mathbf{x}}\| = \min_{\mathbf{y} \in \mathcal{X}} \|\mathbf{x} - \mathbf{y}\|. \quad (11.3.1)$$

Wir nennen $\hat{\mathbf{x}}$ die *Projektion* von $\mathbf{x}$ auf $\mathcal{X}$.

Vertiefung: Beweisskizze des Projektionstheorems

Beweis.

- (1) *Existenz.* Wir setzen $d := \inf_{\mathbf{y} \in \mathcal{X}} \|\mathbf{x} - \mathbf{y}\|$. Das Infimum existiert, weil die Menge der Abstände nichtleer und nach unten durch 0 beschränkt ist. Nun greifen wir uns irgendein $\mathbf{y}_0 \in \mathcal{X}$ und schneiden die Menge mit einer Kugel:

$$\mathcal{K} := \mathcal{X} \cap \{\mathbf{y} \in V : \|\mathbf{x} - \mathbf{y}\| \leq \|\mathbf{x} - \mathbf{y}_0\|\}.$$

$\mathcal{K}$ ist nichtleer, abgeschlossen und beschränkt, und außerhalb von $\mathcal{K}$ sind alle Abstände größer als in $\mathbf{y}_0$. Es genügt also, über $\mathcal{K}$ zu minimieren.

$\mathcal{K}$ ist als Schnitt zweier abgeschlossener Mengen abgeschlossen und liegt in einer Kugel um $\mathbf{x}$, ist also beschränkt

- (2) Die Abbildung $\mathbf{y} \mapsto \|\mathbf{x} - \mathbf{y}\|$ ist stetig, und in endlicher Dimension ist $\mathcal{K}$ als abgeschlossene und beschränkte Menge kompakt. Nach dem Satz von Weierstraß nimmt eine stetige Funktion auf einer kompakten Menge ihr Minimum an. Es gibt also ein $\hat{\mathbf{x}} \in \mathcal{K} \subseteq \mathcal{X}$ mit $\|\mathbf{x} - \hat{\mathbf{x}}\| = d$.

hier steckt die Vollständigkeit des Raums: In endlicher Dimension liefert der Satz von Bolzano-Weierstraß zu jeder beschränkten Folge eine konvergente Teilfolge, und die Abgeschlossenheit hält den Grenzwert in der Menge

- (3) *Eindeutigkeit.* Angenommen, es gäbe zwei Punkte $\hat{\mathbf{x}}_1 \neq \hat{\mathbf{x}}_2$ in $\mathcal{X}$, beide mit Abstand d zu $\mathbf{x}$. Wir setzen $\mathbf{v} := \hat{\mathbf{x}}_2 - \hat{\mathbf{x}}_1 \neq \mathbf{0}$ und betrachten den Abstand entlang der Verbindungsstrecke, quadriert:

$$g(\lambda) := \|\mathbf{x} - (\hat{\mathbf{x}}_1 + \lambda \mathbf{v})\|^2, \quad \lambda \in [0, 1]. \quad (11.3.2)$$

Wegen der Konvexität von $\mathcal{X}$ liegt $\hat{\mathbf{x}}_1 + \lambda \mathbf{v} = (1 - \lambda)\hat{\mathbf{x}}_1 + \lambda\hat{\mathbf{x}}_2$ für jedes solche λ in $\mathcal{X}$. Nach Schritt 2 ist d der kleinstmögliche Abstand, also gilt $g(\lambda) \geq d^2$ auf dem ganzen Intervall.

genau hier geht die Konvexität ein: ohne sie wüssten wir über die Punkte der Strecke gar nichts

- (4) Jetzt multiplizieren wir (11.3.2) aus. Mit der Bilinearität des Skalarprodukts ist

$$g(\lambda) = \|\mathbf{x} - \hat{\mathbf{x}}_1\|^2 - 2\lambda \langle \mathbf{x} - \hat{\mathbf{x}}_1, \mathbf{v} \rangle + \lambda^2 \|\mathbf{v}\|^2,$$

eine quadratische Funktion in λ mit positivem Leitkoeffizienten. Aus $g(0) = d^2$ und $g(1) = \|\mathbf{x} - \hat{\mathbf{x}}_2\|^2 = d^2$ folgt $\langle \mathbf{x} - \hat{\mathbf{x}}_1, \mathbf{v} \rangle = \frac{1}{2} \|\mathbf{v}\|^2$.

$$g(1) - g(0) = -2\langle \mathbf{x} - \hat{\mathbf{x}}_1, \mathbf{v} \rangle + \|\mathbf{v}\|^2 = 0$$

- (5) Eine Parabel mit gleichen Werten an den Rändern hat ihren Scheitel in der Mitte. Setzen wir $\lambda = \frac{1}{2}$ ein:

$$g\left(\frac{1}{2}\right) = d^2 - \langle \mathbf{x} - \hat{\mathbf{x}}_1, \mathbf{v} \rangle + \frac{1}{4} \|\mathbf{v}\|^2 = d^2 - \frac{1}{4} \|\mathbf{v}\|^2 < d^2,$$

denn $\mathbf{v} \neq \mathbf{0}$. Der Mittelpunkt der beiden Kandidaten liegt also echt näher an $\mathbf{x}$ als d , und das widerspricht Schritt 3. Folglich war die Annahme falsch, und es gilt $\hat{\mathbf{x}}_1 = \hat{\mathbf{x}}_2$.

der rote Term ist der ganze Gewinn: Er misst, wie weit die beiden Kandidaten auseinanderliegen, und genau um ihn unterbietet der Mittelpunkt den angeblich kleinsten Abstand

□

Bemerkung 11.3.2 (Jede Voraussetzung wird gebraucht)

Der Satz stellt drei Bedingungen, und entbehrlich ist keine davon.

Abgeschlossen. Für den offenen Einheitsball $\{\mathbf{z} : \|\mathbf{z}\| < 1\}$ und $\mathbf{x} = (2, 0)^\top$ ist das Infimum der Abstände 1, angenommen wird es nie: Zu jedem $\mathbf{y}$ des Balls gibt es einen näheren. Ein Minimum wie in (11.3.1) gibt es also nicht.

Konvex. Für die Einheitssphäre $\{\mathbf{z} : \|\mathbf{z}\| = 1\}$, die abgeschlossen, aber nicht konvex ist, und $\mathbf{x} = \mathbf{0}$ hat jeder Punkt der Menge den Abstand 1. Statt eines nächsten Punktes gibt es unendlich viele. Noch einfacher ist die zweielementige Menge $\{-1, +1\} \subseteq \mathbb{R}$ mit $\mathbf{x} = 0$: zwei nächste Punkte.

Endliche Dimension. Die Abgeschlossenheit allein trägt die Existenz nicht. Sie sorgt dafür, dass der Grenzwert einer Minimierungsfolge wieder in der Menge liegt; dass die Folge überhaupt konvergiert, ist die eigentliche Arbeit, und in endlicher Dimension besorgt es Bolzano-Weierstraß. In unendlichdimensionalen Räumen gilt der Satz ebenfalls, dort allerdings nur für *vollständige* Räume, und die Rechnung aus Schritt 5 wird dann zur Parallelogrammgleichung, mit der sich jede Minimierungsfolge als Cauchy-Folge erweist.

Der nächstgelegene Punkt lässt sich auch ohne Minimierung erkennen. Das folgende Kriterium sagt, woran wir ihn sehen. Für einen Untervektorraum wird daraus die vertraute Orthogonalität, für allgemeine konvexe Mengen bleibt nur die Ungleichung.

Satz 11.3.3 (Kriterium des stumpfen Winkels)

Seien $\mathcal{X} \subseteq V$ nichtleer und konvex, $\mathbf{x} \in V$ und $\hat{\mathbf{x}} \in \mathcal{X}$. Dann hat $\hat{\mathbf{x}}$ genau dann unter allen Punkten von $\mathcal{X}$ den kleinsten Abstand zu $\mathbf{x}$, wenn

$$\langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{y} - \hat{\mathbf{x}} \rangle \leq 0 \quad \text{für alle } \mathbf{y} \in \mathcal{X}. \quad (11.3.3)$$

Abgeschlossenheit und endliche Dimension braucht dieses Kriterium nicht. Es sagt allerdings auch nichts darüber, ob ein nächstgelegener Punkt existiert; es erkennt ihn nur, wenn es ihn gibt.

Geometrisch sagt (11.3.3) für $\mathbf{x} \notin \mathcal{X}$: Der Vektor von der Projektion zu $\mathbf{x}$ und der Vektor von der Projektion zu einem beliebigen weiteren Punkt $\mathbf{y} \neq \hat{\mathbf{x}}$ der Menge schließen einen Winkel von mindestens 90° ein. Die ganze Menge liegt also auf einer Seite der Geraden (im $\mathbb{R}^n$: der Hyperebene) durch $\hat{\mathbf{x}}$, die senkrecht auf $\mathbf{x} - \hat{\mathbf{x}}$ steht. Liegt $\mathbf{x}$ dagegen selbst in $\mathcal{X}$, so ist $\mathbf{x} - \hat{\mathbf{x}} = \mathbf{0}$ und von einem Winkel keine Rede mehr.

Ist $\mathcal{X} = \mathcal{U}$ sogar ein Untervektorraum, so wird aus (11.3.3) eine Gleichung. Mit $\mathbf{y} = \hat{\mathbf{x}} \pm \mathbf{u}$ für beliebiges $\mathbf{u} \in \mathcal{U}$ liefert (11.3.3) nämlich $\pm \langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{u} \rangle \leq 0$, also $\langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{u} \rangle = 0$. Das Residuum steht dann wirklich senkrecht auf dem ganzen Unterraum, und aus dieser Bedingung entstehen die Normalgleichungen der Kleinsten Quadrate aus Abschnitt 7.1. Für eine konvexe Menge bleibt davon die Ungleichung übrig, und die reicht aus.

Vertiefung: Der Beweis des Winkelkriteriums und zwei Projektionsformeln

Der Beweis ist eine Rechnung mit demselben quadratischen Ausdruck wie im Eindeutigkeits teil oben; die beiden Formeln danach zeigen, wie eine Projektion aussieht, wenn man sie ausnahmsweise hinschreiben kann.

Beweis.

- (1) *Aus der Ungleichung folgt die Minimalität.* Sei (11.3.3) erfüllt und $\mathbf{y} \in \mathcal{X}$ beliebig. Wir zerlegen $\mathbf{x} - \mathbf{y} = (\mathbf{x} - \hat{\mathbf{x}}) + (\hat{\mathbf{x}} - \mathbf{y})$ und quadrieren:

$$\|\mathbf{x} - \mathbf{y}\|^2 = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 - 2\langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{y} - \hat{\mathbf{x}} \rangle + \|\mathbf{y} - \hat{\mathbf{x}}\|^2 \geq \|\mathbf{x} - \hat{\mathbf{x}}\|^2.$$

Beide weggelassenen Terme sind nichtnegativ, der mittlere nach Voraussetzung.

$$\|\mathbf{a} + \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + 2\langle \mathbf{a}, \mathbf{b} \rangle + \|\mathbf{b}\|^2 \text{ mit } \mathbf{a} = \mathbf{x} - \hat{\mathbf{x}} \text{ und } \mathbf{b} = \hat{\mathbf{x}} - \mathbf{y}, \text{ also } \langle \mathbf{a}, \mathbf{b} \rangle = -\langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{y} - \hat{\mathbf{x}} \rangle$$

- (2) *Aus der Minimalität folgt die Ungleichung.* Sei nun $\hat{\mathbf{x}}$ die Projektion und $\mathbf{y} \in \mathcal{X}$. Für $t \in (0, 1]$ liegt $\mathbf{z}_t := \hat{\mathbf{x}} + t(\mathbf{y} - \hat{\mathbf{x}})$ wegen der Konvexität in $\mathcal{X}$, also ist $\|\mathbf{x} - \mathbf{z}_t\|^2 \geq \|\mathbf{x} - \hat{\mathbf{x}}\|^2$. Ausmultipliziert heißt das

$$-2t\langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{y} - \hat{\mathbf{x}} \rangle + t^2\|\mathbf{y} - \hat{\mathbf{x}}\|^2 \geq 0.$$

Wir teilen durch $2t > 0$ und erhalten $\langle \mathbf{x} - \hat{\mathbf{x}}, \mathbf{y} - \hat{\mathbf{x}} \rangle \leq \frac{t}{2}\|\mathbf{y} - \hat{\mathbf{x}}\|^2$. Weil das für jedes $t \in (0, 1]$ gilt, dürfen wir $t \rightarrow 0$ laufen lassen, und die rechte Seite verschwindet.

dieselbe quadratische Rechnung wie im Eindeutigkeitsbeweis, nur mit dem Laufparameter t statt λ

□

Die Eindeutigkeit fällt nebenbei ab: Erfüllen $\hat{\mathbf{x}}_1$ und $\hat{\mathbf{x}}_2$ beide (11.3.3), so setzen wir einmal $\mathbf{y} = \hat{\mathbf{x}}_2$ und einmal $\mathbf{y} = \hat{\mathbf{x}}_1$ ein und addieren die beiden Ungleichungen. Übrig bleibt $\|\hat{\mathbf{x}}_2 - \hat{\mathbf{x}}_1\|^2 \leq 0$.

Beispiel 11.3.4 (Zwei Projektionen, die wir hinschreiben können)

Selten lässt sich die Projektion in einer Formel angeben; in diesen beiden Fällen geht es.

Kugel. Für $\mathcal{X} = \{\mathbf{z} : \|\mathbf{z}\| \leq r\}$ ist

$$\hat{\mathbf{x}} = \begin{cases} \mathbf{x}, & \|\mathbf{x}\| \leq r, \\ r \frac{\mathbf{x}}{\|\mathbf{x}\|}, & \|\mathbf{x}\| > r. \end{cases}$$

Für $r = 1$ und $\mathbf{x} = (1, 6; 1, 2)^\top$ ist $\|\mathbf{x}\| = 2$, also $\hat{\mathbf{x}} = (0, 8; 0, 6)^\top$ und $d = 1$. Die Probe mit (11.3.3): Es ist $\mathbf{x} - \hat{\mathbf{x}} = (0, 8; 0, 6)^\top = \hat{\mathbf{x}}$, und für jedes $\mathbf{y}$ mit $\|\mathbf{y}\| \leq 1$ gilt $\langle \hat{\mathbf{x}}, \mathbf{y} - \hat{\mathbf{x}} \rangle = \langle \hat{\mathbf{x}}, \mathbf{y} \rangle - 1 \leq \|\mathbf{y}\| - 1 \leq 0$ nach Cauchy-Schwarz.

Halbraum. Für $\mathcal{X} = \{\mathbf{z} : \mathbf{a}^\top \mathbf{z} \leq \beta\}$ mit $\mathbf{a} \neq \mathbf{0}$ ist

$$\hat{\mathbf{x}} = \mathbf{x} - \frac{\max\{0, \mathbf{a}^\top \mathbf{x} - \beta\}}{\|\mathbf{a}\|^2} \mathbf{a},$$

wir laufen also entlang der Normalenrichtung gerade so weit, bis die Nebenbedingung mit Gleichheit erfüllt ist. Liegt $\mathbf{x}$ schon im Halbraum, passiert nichts.

Für ein Dreieck oder ein Polyeder gibt es keine solche Formel mehr. Dort rechnen wir die Projektion auf jede Seite einzeln aus, jeweils auf diese Seite beschränkt, und nehmen die beste; Ecken sind damit von selbst abgedeckt. Genau so rechnet das Widget unten.

Interaktiv: Die Projektion zum Ziehen

In dieser Skizze ist der Punkt beweglich. Zwei Fragen lassen sich damit klären: Wandert $\hat{x}$ immer mit, wenn wir x bewegen, und wo liegt der nächste Punkt, wenn x hinter einer Ecke steht?

Interaktives Element — nur in der Web-Fassung

Der gestrichelte Kreis um x mit Radius d berührt die Menge nur in $\hat{x}$. Ein weiterer Punkt der Menge im Inneren des Kreises wäre selbst näher an x ; läge er auf dem Kreis, so wäre nach Schritt 5 des Beweises der Mittelpunkt der beiden näher. Die gestrichelte Gerade durch $\hat{x}$ ist die Trennlinie aus Satz 11.3.3, senkrecht zur Abstandsstrecke. Wir dürfen x um die Menge herumziehen: Die Gerade dreht sich mit, die Menge bleibt aber immer auf derselben Seite. Beim Dreieck wandert $\hat{x}$ dabei über Kanten und Ecken, und in einer Ecke bleibt es eine Weile stehen, während x weiterläuft.

Der Epigraph

Bisher ging es um Mengen. Für die Optimierung brauchen wir aber vor allem Aussagen über *Funktionen*, und daraus wird keine neue Theorie: Wir führen die Konvexität einer Funktion auf die Konvexität einer Menge zurück. Diese Menge ist alles, was über dem Graphen liegt.

Definition 11.3.5 (Epigraph)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ und $f : \mathcal{X} \rightarrow \mathbb{R}$. Der *Epigraph* (epigraph) von f ist die Menge

$$\text{epi}(f) := \{(\mathbf{x}, t) \in \mathcal{X} \times \mathbb{R} : t \geq f(\mathbf{x})\}.$$

Der Name kommt aus dem Griechischen und heißt wörtlich „darüber geschrieben“. Für $f : \mathbb{R} \rightarrow \mathbb{R}$ ist $\text{epi}(f) \subseteq \mathbb{R}^2$ die Fläche über der Kurve, den Graphen eingeschlossen. Die Punkte darunter gehören nicht dazu.

Interaktives Element — nur in der Web-Fassung

Definition 11.3.6 (Konvexe Funktion)

Eine Funktion $f : \mathcal{X} \rightarrow \mathbb{R}$ heißt *konvex* (convex), falls ihr Epigraph $\text{epi}(f)$ eine konvexe Teilmenge des Umgebungsraums $\mathbb{R}^n \times \mathbb{R}$ ist.

Bemerkung 11.3.7 (Der Definitionsbereich muss mitspielen)

Dass $\mathcal{X}$ dabei konvex ist, muss man nicht eigens fordern. Es ist keine zusätzliche Forderung, sondern eine Folgerung. Sind nämlich $\mathbf{x}, \mathbf{y} \in \mathcal{X}$, so liegen $(\mathbf{x}, f(\mathbf{x}))$ und $(\mathbf{y}, f(\mathbf{y}))$ in $\text{epi}(f)$, und ist der Epigraph konvex, so liegt auch $\lambda(\mathbf{x}, f(\mathbf{x})) + (1 - \lambda)(\mathbf{y}, f(\mathbf{y}))$ darin. Die erste Komponente dieses Punktes ist $\lambda\mathbf{x} + (1 - \lambda)\mathbf{y}$, und weil alle Punkte des Epigraphen ihre erste Komponente in $\mathcal{X}$ haben, liegt sie in $\mathcal{X}$. Ein konvexer Epigraph erzwingt also einen konvexen Definitionsbereich.

Umgekehrt brauchen wir die Konvexität von $\mathcal{X}$ sehr wohl als Voraussetzung, sobald wir gleich mit Ungleichungen arbeiten: Dort steht $f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y})$, und dieser Ausdruck muss überhaupt erst definiert sein. Wir setzen deshalb ab jetzt durchgehend voraus, dass $\mathcal{X}$ konvex ist.

Die Epigraph-Fassung erklärt, warum konvexe Funktionen in dieses Kapitel gehören. Zum Rechnen ist sie unhandlich. Gängiger ist die Formulierung über eine Ungleichung. Beide sagen dasselbe.

Satz 11.3.8 (Konvexität als Ungleichung)

Sei $\mathcal{X}$ konvex und $f : \mathcal{X} \rightarrow \mathbb{R}$. Dann ist f genau dann konvex im Sinne von Definition 11.3.6, wenn für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und alle $\lambda \in [0, 1]$ gilt

$$f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y}). \quad (11.3.4)$$

Vertiefung: Beweis der Ungleichungscharakterisierung

Beweis.

- (1) *Vom Epigraphen zur Ungleichung.* Sei $\text{epi}(f)$ konvex und seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$, $\lambda \in [0, 1]$. Die Punkte $(\mathbf{x}, f(\mathbf{x}))$ und $(\mathbf{y}, f(\mathbf{y}))$ liegen in $\text{epi}(f)$, also auch ihre Konvexkombination

$$(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}, \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})) \in \text{epi}(f).$$

Nach Definition des Epigraphen heißt das genau (11.3.4).

ein Paar (z, t) liegt im Epigraphen, wenn $t \geq f(z)$ ist; hier ist $z = \lambda \mathbf{x} + (1 - \lambda)\mathbf{y}$ und $t = \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$

- (2) *Von der Ungleichung zum Epigraphen.* Gelte (11.3.4), und seien $(\mathbf{x}, s), (\mathbf{y}, t)$ zwei Punkte aus $\text{epi}(f)$, also $s \geq f(\mathbf{x})$ und $t \geq f(\mathbf{y})$. Für $\lambda \in [0, 1]$ rechnen wir

$$\lambda s + (1 - \lambda)t \geq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y}) \geq f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}).$$

Die erste Ungleichung gilt, weil λ und $1 - \lambda$ nichtnegativ sind, die zweite ist die Voraussetzung. Zusammen mit $\lambda \mathbf{x} + (1 - \lambda)\mathbf{y} \in \mathcal{X}$ liegt die Konvexkombination der beiden Punkte damit wieder in $\text{epi}(f)$.

hier wird gebraucht, dass $\mathcal{X}$ konvex ist; sonst wäre die rechte Seite gar nicht definiert

□

Bemerkung 11.3.9 (Wie wir die Ungleichung lesen)

Für $f : \mathbb{R}^n \rightarrow \mathbb{R}$ sagt (11.3.4) etwas sehr Anschauliches. Die rechte Seite ist der Wert, den die Verbindungsstrecke zwischen den Graphenpunkten $(\mathbf{x}, f(\mathbf{x}))$ und $(\mathbf{y}, f(\mathbf{y}))$ an der Stelle $\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}$ hat, die linke der Funktionswert dort. Die *Sehne* liegt also nie unter dem Graphen, oder umgekehrt: Zwischen zwei Punkten hängt der Graph durch. Konvexe Funktionen sind nach oben geöffnet.

Aus derselben Ungleichung entstehen zwei weitere Begriffe. Wir nennen f *konkav* (concave), wenn $-f$ konvex ist; für konkave Funktionen dreht sich in (11.3.4) das Ungleichheitszeichen um, die Sehne liegt nie über dem Graphen. Und wir nennen f *strikt konvex* (strictly convex), wenn in (11.3.4) für $\mathbf{x} \neq \mathbf{y}$ und $\lambda \in (0, 1)$ sogar die strikte Ungleichung gilt. Gebräuchlich sind dafür beide Namen, „strikt konvex“ und „streng konvex“; wir bleiben einheitlich bei strikt. Strikt konvexe Funktionen dürfen also keine geraden Stücke haben. Warum das wichtig ist, sehen wir in Abschnitt 11.5: Konvexität macht jedes lokale Minimum global, strikte

Konvexität macht das Minimum eindeutig.

Eine Bildtafel stellt konvex und konkav gegenüber, und daran hängt ein beweglicher Test.

Interaktives Element — nur in der Web-Fassung

Blau ist jeweils der Graph, rot markiert die Stellen, an denen er über der Sehne liegt; die Sehne selbst ist grün, solange sie nirgends unterschritten wird, und sonst rot. Bei der konkaven Funktion ist das ganze Zwischenstück rot, bei der Doppelmulde rechts nur ein Teil davon. Beides widerlegt Konvexität, konkav ist von den vier Funktionen aber nur die zweite. In der dritten Tafel liegen beide Endpunkte rechts vom Knick des Betrags. Dort verdeckt die grüne Sehne den blauen Graphen vollständig, denn auf diesem Ast steht in (11.3.4) Gleichheit; genau darin unterscheiden sich konvex und strikt konvex. Die Parabel links kennt diesen Fall nicht, sie bleibt zwischen ihren Endpunkten überall echt unter der Sehne.

Interaktiv: Konvexität mit einer beweglichen Sehne

Wählen wir die Endpunkte selbst, wird aus der Tafel ein Werkzeug. Wie viele Paare müssen wir prüfen, um Konvexität zu widerlegen, und wie viele, um sie zu beweisen?

Interaktives Element — nur in der Web-Fassung

Nachweis und Widerlegung sind ebenso ungleich schwer wie bei den Mengen in Bemerkung 11.2.2: Ein einziges schlechtes Paar erledigt die Konvexität, während noch so viele gelungene Proben sie nicht beweisen. Die Doppelmulde führt beides vor. In der Voreinstellung $x = -1,55$ und $y = 1,25$ ragt der Graph um 1,891 über die Sehne, damit ist die Sache entschieden. Rücken wir dagegen beide Endpunkte in denselben Talgrund, etwa auf $x = -1,8$ und $y = -0,9$, so besteht das Paar die Probe, und bewiesen ist damit nichts. Beim Betrag deckt die grüne Sehne den Graphen auf jedem Paar mit gleichem Vorzeichen vollständig ab, dort steht in (11.3.4) Gleichheit, und genau daran unterscheiden sich konvex und strikt konvex.

Nachweis von Hand: der Betrag

Beweisen lässt sich Konvexität nur mit einer Rechnung über alle Paare. Wie so eine Rechnung aussieht, führen wir am Betrag vor.

Beispiel 11.3.10 (Der Betrag ist konvex)

Die Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = |x|$, ist konvex.

Beweis.

- (1) Seien $x, y \in \mathbb{R}$ und $\lambda \in [0, 1]$ beliebig. Zu zeigen ist $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$. Wir setzen die Definition von f ein und schätzen mit der Dreiecksungleichung ab:

$$\begin{aligned} f(\lambda x + (1 - \lambda)y) &= |\lambda x + (1 - \lambda)y| \\ &\leq |\lambda x| + |(1 - \lambda)y| \\ &= \lambda |x| + (1 - \lambda) |y| \\ &= \lambda f(x) + (1 - \lambda)f(y). \end{aligned}$$

in der zweiten Zeile die Dreiecksungleichung $|a + b| \leq |a| + |b|$, in der dritten $|\lambda x| = |\lambda| |x| = \lambda |x|$, weil $\lambda \geq 0$ und $1 - \lambda \geq 0$ sind

- (2) Die Rechnung benutzt vom Betrag nur zwei Eigenschaften, die absolute Homogenität und die Dreiecksungleichung. Beides haben alle Normen, und deshalb tragen wir denselben Beweis unten in Satz 11.3.15 gleich für jede Norm ein.

$|\cdot|$ ist die Norm $\|\cdot\|_1$ auf $\mathbb{R}^1$

□

Der Betrag ist konvex, aber nicht strikt konvex: Für $x, y > 0$ steht in der Rechnung durchgehend Gleichheit. Und er zeigt noch etwas: Konvexe Funktionen dürfen Knicke haben, sie müssen nicht differenzierbar sein. Die Ableitungskriterien, die Abschnitt 11.4 bringt, sind deshalb bequem, aber nicht die Definition.

Selbsttest

1. Die affine Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = ax + b$ mit $a, b \in \mathbb{R}$, ist konvex.

Antwort: Wahr

In (11.3.4) steht hier sogar Gleichheit: $a(\lambda x + (1 - \lambda)y) + b = \lambda(ax + b) + (1 - \lambda)(ay + b)$, weil sich die beiden b -Anteile wegen $\lambda + (1 - \lambda) = 1$ zu einem einzigen b zusammensetzen. Damit ist f konvex und zugleich konkav, aber für kein a strikt konvex.

2. Die Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = x^4$, ist konvex.

Antwort: Wahr

Sie ist es, und zwar streng. Ein Beweis in einer Zeile folgt in Abschnitt 11.4 aus $f''(x) = 12x^2 \geq 0$. Dass die zweite Ableitung im Nullpunkt verschwindet, schadet dabei nicht: Die Sehnenungleichung verlangt nichts über Ableitungen, und die Funktion ist auch über den Nullpunkt hinweg strikt konvex.

3. Die Wurzelfunktion $f : \mathbb{R}_+ \rightarrow \mathbb{R}$, $f(x) = \sqrt{x}$, ist konvex.

Antwort: Falsch

Sie ist konkav. Die Sehne zwischen $(0, 0)$ und $(4, 2)$ hat in der Mitte den Wert 1, die Funktion dort aber $\sqrt{2} \approx 1,414$ und liegt damit über der Sehne. Für $-f$ ist die Ungleichung erfüllt, also ist $\sqrt{\cdot}$ konkav im Sinne von Bemerkung 11.3.9.

4. Die Exponentialfunktion $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = e^x$, ist konvex.

Antwort: Wahr

Sogar strikt konvex. Nebenbei ist die Exponentialfunktion das Standardbeispiel dafür, dass Konvexität allein noch kein Minimum liefert: $e^x > 0$ für alle x , das Infimum 0 wird nie angenommen.

5. Der Logarithmus $f : \mathbb{R}_+ \rightarrow \mathbb{R}$, $f(x) = \ln x$, ist konvex.

Antwort: Falsch

Er ist konkav, wie die Wurzel. Zwischen $x = 1$ und $x = 4$ etwa hat die Sehne in der Mitte den Wert $\frac{1}{2} \ln 4 \approx 0,693$, der Logarithmus dort aber $\ln 2,5 \approx 0,916$. Beide steigen mit wachsendem Argument immer langsamer, und das ist die Anschauung hinter konkav.

6. Ist f konkav, so ist ihr Epigraph konvex.

Antwort: Falsch

Genau andersherum. Konkav heißt, dass $-f$ konvex ist, und das ist gleichbedeutend damit, dass die Menge *unter* dem Graphen konvex ist. Für $f(x) = -x^2$ etwa ist der Epigraph das Gebiet über einer nach unten geöffneten Parabel, und die Verbindungsstrecke zwischen $(-1, -1)$ und $(1, -1)$ läuft durch $(0, -1)$, was wegen $-1 < f(0) = 0$ nicht im Epigraphen liegt. Konvex ist der Epigraph einer konkaven Funktion nur, wenn sie affin ist.

7. Wählen wir im Sehnentest die Doppelmulde in ihrer Voreinstellung $x = -1,55$, $y = 1,25$. Wie dick ist der rote Streifen an seiner stärksten Stelle, also wie weit ragt der Graph über die Sehne?

Lösung: 1.891

Um 1,891. Das ist die größte Verletzung von (11.3.4) auf diesem Paar, und mehr braucht es nicht: Satz 11.3.8 fordert die Ungleichung für alle Paare, ein einziges Gegenbeispiel entscheidet die Frage. Zum Vergleich: Auf dem Paar $x = -1,8$, $y = -0,9$ bleibt der Streifen leer, bewiesen ist damit aber nichts.

Beispiele konvexer Funktionen

Zum Schluss stehen drei Klassen zusammen, die in der Statistik ständig vorkommen. Wir gehen sie der Reihe nach durch, mit Beweis, wo er kurz ist.

Beispiel 11.3.11 (Affine Funktionen)

Für $\mathbf{b} \in \mathbb{R}^n$ und $a \in \mathbb{R}$ ist $f(\mathbf{x}) = \mathbf{b}^\top \mathbf{x} + a$ konvex und konkav zugleich, denn in (11.3.4) steht Gleichheit:

$$\mathbf{b}^\top (\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) + a = \lambda (\mathbf{b}^\top \mathbf{x} + a) + (1 - \lambda) (\mathbf{b}^\top \mathbf{y} + a).$$

Satz 11.3.12 (Quadratische Funktionen)

Seien $\mathbf{Q} \in \mathbb{R}^{n \times n}$, $\mathbf{b} \in \mathbb{R}^n$ und $a \in \mathbb{R}$, und sei

$$f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + a.$$

Ist der symmetrische Anteil $\mathbf{Q}_{\text{sym}} := \frac{1}{2}(\mathbf{Q} + \mathbf{Q}^\top)$ positiv semidefinit, so ist f konvex. Ist $\mathbf{Q}_{\text{sym}}$ sogar positiv definit, so ist f strikt konvex.

Vertiefung: Beweis des Kriteriums für quadratische Funktionen

Beweis.

- (1) Wir rechnen die Differenz zwischen rechter und linker Seite von (11.3.4) aus. Mit $\mathbf{z} := \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$ und $\mathbf{d} := \mathbf{x} - \mathbf{y}$ gilt

$$\lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}) - f(\mathbf{z}) = \lambda(1 - \lambda) \mathbf{d}^T \mathbf{Q} \mathbf{d}.$$

Die affinen Anteile $\mathbf{b}^T \mathbf{x} + a$ fallen dabei heraus, weil sie nach Beispiel 11.3.11 die Gleichung mit Gleichheit erfüllen; zu rechnen bleibt der quadratische Teil.

ausmultiplizieren: $\mathbf{z}^T \mathbf{Q} \mathbf{z} = \lambda^2 \mathbf{x}^T \mathbf{Q} \mathbf{x} + \lambda(1 - \lambda)(\mathbf{x}^T \mathbf{Q} \mathbf{y} + \mathbf{y}^T \mathbf{Q} \mathbf{x}) + (1 - \lambda)^2 \mathbf{y}^T \mathbf{Q} \mathbf{y}$, und mit $\lambda - \lambda^2 = \lambda(1 - \lambda)$ bleibt genau der angegebene Term; numerisch über 200000 Zufallsfälle bestätigt

- (2) Nun ist $\mathbf{d}^T \mathbf{Q} \mathbf{d} = \mathbf{d}^T \mathbf{Q}_{\text{sym}} \mathbf{d}$, denn der schiefsymmetrische Anteil $\frac{1}{2}(\mathbf{Q} - \mathbf{Q}^T)$ liefert in einer quadratischen Form stets null. Ist $\mathbf{Q}_{\text{sym}}$ positiv semidefinit, so ist dieser Ausdruck nichtnegativ, und wegen $\lambda(1 - \lambda) \geq 0$ ist die ganze Differenz nichtnegativ. Das ist (11.3.4). Ist $\mathbf{Q}_{\text{sym}}$ positiv definit, so ist der Ausdruck für $\mathbf{d} \neq \mathbf{0}$ und $\lambda \in (0, 1)$ echt positiv, und wir bekommen die strikte Ungleichung.

$\mathbf{d}^T \mathbf{A} \mathbf{d} = (\mathbf{d}^T \mathbf{A} \mathbf{d})^T = \mathbf{d}^T \mathbf{A}^T \mathbf{d}$, also verschwindet $\mathbf{d}^T (\mathbf{A} - \mathbf{A}^T) \mathbf{d}$

□

Bemerkung 11.3.13 (Warum der symmetrische Anteil entscheidet)

Die Voraussetzung des Satzes ist bewusst an $\mathbf{Q}_{\text{sym}}$ geknüpft und nicht an $\mathbf{Q}$ selbst. Verlangten wir „ $\mathbf{Q}$ positiv semidefinit“, so wäre die Aussage unnötig eng, denn die quadratische Form sieht von $\mathbf{Q}$ nur den symmetrischen Anteil, wie Schritt 2 zeigt. Für

$$\mathbf{Q} = \begin{pmatrix} 1 & 5 \\ -5 & 1 \end{pmatrix}$$

ist $\mathbf{Q}_{\text{sym}} = \mathbf{I}$, die Funktion $f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x} = x_1^2 + x_2^2$ also strikt konvex, obwohl $\mathbf{Q}$ selbst nicht symmetrisch und damit nach Definition 11.2.7 nicht einmal ein Kandidat für „positiv semidefinit“ ist. In der Praxis fällt das selten ins Gewicht, weil wir $\mathbf{Q}$ ohnehin symmetrisch wählen; die allgemeinere Fassung kostet aber nichts.

Beispiel 11.3.14 (Eine Quadrik ausgerechnet)

Rechnen wir ein Beispiel vor. Sei $f(\mathbf{x}) = \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{b}^T \mathbf{x} + a$ mit

$$\mathbf{Q} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ -2 \end{pmatrix}, \quad a = 3.$$

$\mathbf{Q}$ ist symmetrisch mit den Eigenwerten 2 und 1, also positiv definit. Nach Satz 11.3.12 ist f strikt konvex. Auswerten an der Stelle $\mathbf{x} = (1, 2)^T$:

$$\begin{aligned} f(\mathbf{x}) &= \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 1 & -2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + 3 \\ &= \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 2 \end{pmatrix} + (1 - 4) + 3 = 6 - 3 + 3 = 6. \end{aligned}$$

Weil $\mathbf{Q}$ hier diagonal ist, lässt sich f auch als $2x_1^2 + x_2^2 + x_1 - 2x_2 + 3$ schreiben, und daran sehen wir die strikte Konvexität ohne Matrixrechnung: eine Summe zweier nach oben geöffneter Parabeln plus einer affinen Funktion.

Satz 11.3.15 (Jede Norm ist konvex)

Sei $\|\cdot\|$ eine beliebige Norm auf $\mathbb{R}^n$. Dann ist $f(\mathbf{x}) = \|\mathbf{x}\|$ konvex.

Beweis.

- (1) Seien $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ und $\lambda \in [0, 1]$. Dann ist

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) = \|\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}\| \leq \|\lambda \mathbf{x}\| + \|(1 - \lambda) \mathbf{y}\| = \lambda \|\mathbf{x}\| + (1 - \lambda) \|\mathbf{y}\|.$$

Dreiecksungleichung, dann absolute Homogenität $\|c\mathbf{x}\| = |c| \|\mathbf{x}\|$ mit $c = \lambda \geq 0$ beziehungsweise $c = 1 - \lambda \geq 0$

- (2) Von den drei definierenden Eigenschaften einer Norm haben wir nur zwei gebraucht: die absolute Homogenität und die Dreiecksungleichung. Die Definitheit ($f(\mathbf{x}) = 0 \implies \mathbf{x} = \mathbf{0}$) geht nirgends ein. Der Beweis trägt deshalb auch für Halbnormen, und für $n = 1$ ist er wörtlich der aus Beispiel 11.3.10.

die drei Normeigenschaften stehen als Definition 3.2.1 in Kapitel 3, dort für Matrizen formuliert; Bemerkung 3.2.2 hält fest, dass es dieselben Axiome sind

□

Strikt konvex ist keine dieser Normen. Liegen $\mathbf{x}$ und $\mathbf{y}$ auf demselben Strahl durch den Ursprung, so steht in Schritt 1 überall Gleichheit. Für das *Quadrat* der euklidischen Norm sieht es anders aus: $f(\mathbf{x}) = \|\mathbf{x}\|_2^2 = \mathbf{x}^\top \mathbf{I} \mathbf{x}$ ist nach Satz 11.3.12 strikt konvex, weil $\mathbf{I}$ positiv definit ist. Und diese Funktion ist es, die uns in der Statistik ständig begegnet.

Beispiel 11.3.16 (Kleinste Quadrate und Ridge)

Sei $\mathbf{A} \in \mathbb{R}^{m \times n}$ die Designmatrix und $\mathbf{y} \in \mathbb{R}^m$ der Beobachtungsvektor. Der Kleinste-Quadrate-Verlust lässt sich ausmultiplizieren zu

$$f(\mathbf{w}) = \|\mathbf{A}\mathbf{w} - \mathbf{y}\|^2 = \mathbf{w}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{w} - 2\mathbf{y}^\top \mathbf{A}\mathbf{w} + \|\mathbf{y}\|^2,$$

also zu einer quadratischen Funktion mit $\mathbf{Q} = \mathbf{A}^\top \mathbf{A}$. Diese Matrix ist symmetrisch und stets positiv semidefinit, denn

$$\mathbf{w}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{w} = (\mathbf{A}\mathbf{w})^\top (\mathbf{A}\mathbf{w}) = \|\mathbf{A}\mathbf{w}\|^2 \geq 0.$$

Nach Satz 11.3.12 ist der Kleinste-Quadrate-Verlust deshalb immer konvex, ganz gleich, wie die Daten aussehen. Das ist der Grund, warum wir uns in Kapitel 7 nie um lokale Minima kümmern mussten.

Strikt konvex ist er dagegen nur bei vollem Spaltenrang. Hat $\mathbf{A}$ zwei gleiche Spalten, etwa

$$\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 2 & 2 \\ 3 & 3 \end{pmatrix}, \quad \mathbf{A}^\top \mathbf{A} = \begin{pmatrix} 14 & 14 \\ 14 & 14 \end{pmatrix},$$

so ist $\mathbf{d}^\top (\mathbf{A}^\top \mathbf{A}) \mathbf{d} = 0$ für $\mathbf{d} = (1, -1)^\top$, und entlang dieser Richtung ist f konstant. Die Lösung ist dann nicht eindeutig.

Hier setzt die *Ridge-Regression* an. Sie addiert einen Strafterm,

$$f_\lambda(\mathbf{w}) = \|\mathbf{A}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2 = \mathbf{w}^\top (\mathbf{A}^\top \mathbf{A} + \lambda \mathbf{I}) \mathbf{w} - 2\mathbf{y}^\top \mathbf{A}\mathbf{w} + \|\mathbf{y}\|^2,$$

und für $\lambda > 0$ ist $\mathbf{A}^\top \mathbf{A} + \lambda \mathbf{I}$ positiv definit: $\mathbf{w}^\top (\mathbf{A}^\top \mathbf{A} + \lambda \mathbf{I}) \mathbf{w} = \|\mathbf{A}\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|^2 > 0$ für $\mathbf{w} \neq \mathbf{0}$. Nach Satz 11.3.12 ist f_λ also strikt konvex, und die Lösung ist eindeutig, auch bei rangdefizientem $\mathbf{A}$. Die Rechnung dazu, samt Gradient, steht in Beispiel 10.6.6.

Der Strafterm hebt dabei alle Eigenwerte um λ an und drückt so die Konditionszahl: Das rangdefiziente $\mathbf{A}^\top \mathbf{A}$ von oben hat die Eigenwerte 28 und 0, κ_2 ist also unendlich; mit $\lambda = 1$ stehen dort 29 und 1, und κ_2 fällt auf 29.

An der quadratischen Form hängt mehr als die Frage konvex oder nicht. Für $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + a$ ist die Hesse-Matrix überall dieselbe, nämlich $\mathbf{H}_f = \mathbf{Q} + \mathbf{Q}^\top = 2\mathbf{Q}_{\text{sym}}$, und ihre Eigenwerte messen die Krümmung in den Hauptrichtungen (Abschnitt 10.7); Eigenwerte nahe null bedeuten flache Richtungen, in denen die Lösung schlecht bestimmt ist. In der Statistik ist das keine Randnotiz, denn die negative Hesse-Matrix der Log-Likelihood ist die beobachtete Fisher-Information, aus der die Standardfehler kommen. Wie sich Konvexität und Krümmung genau zueinander verhalten, klärt Abschnitt 11.4.

Vertiefung: Boyd und Vandenberghe, Convex Optimization, Kapitel 3 führt konvexe Funktionen über dieselbe Ungleichung ein, sammelt die Beispiele (Normen, quadratische Funktionen) und behandelt die Epigraph-Charakterisierung ausführlich. Das Projektionstheorem ist der endlichdimensionale Fall des Projektionssatzes für Hilberträume, den jedes Lehrbuch zur Funktionalanalysis bringt.

11.4 Eigenschaften konvexer Funktionen

Regeln statt Nachrechnen

In Abschnitt 11.3 haben wir Konvexität direkt aus der Definition nachgewiesen, für den Betrag mit der Dreiecksungleichung. Für jede neue Funktion dasselbe zu tun, wäre mühsam. Bei den Zielfunktionen der Statistik ist es aussichtslos, denn die entstehen aus Bausteinen: Ein Verlust wird über alle Beobachtungen summiert, ein Strafterm kommt dazu, über Klassen wird maximiert, und am Ende steht vielleicht noch ein Grenzübergang.

Dieser Abschnitt liefert das Handwerkszeug dafür. Zuerst einen Baukasten aus vier Operationen, unter denen Konvexität erhalten bleibt. Dann die Jensen-Ungleichung, die die Definition von zwei auf beliebig viele Punkte hebt. Danach ein Kriterium, das die Frage an die zweite Ableitung weiterreicht und damit an das Rechnen aus Abschnitt 10.7 anschließt. Zum Schluss den Subgradienten, der die Tangente ersetzt, wo es keine gibt.

Vier Operationen, die Konvexität erhalten

Satz 11.4.1 (Operationen, die Konvexität erhalten)

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und seien $f_1, f_2, \dots : \mathcal{X} \rightarrow \mathbb{R}$ konvexe Funktionen. Dann sind auch die folgenden Funktionen auf $\mathcal{X}$ konvex:

1. die Summe $f(\mathbf{x}) = f_1(\mathbf{x}) + f_2(\mathbf{x})$;
2. das nichtnegative Vielfache $f(\mathbf{x}) = c f_1(\mathbf{x})$ mit $c \geq 0$;
3. das punktweise Supremum $f(\mathbf{x}) = \sup_i f_i(\mathbf{x})$, sofern dieser Wert an jeder Stelle endlich ist; bei endlich vielen Funktionen ist das das Maximum $\max_i f_i(\mathbf{x})$;

4. der punktweise Grenzwert $f(\mathbf{x}) = \lim_{k \rightarrow \infty} f_k(\mathbf{x})$, sofern dieser Grenzwert für jedes $\mathbf{x} \in \mathcal{X}$ existiert.

Wir führen die Beweise für (2) und (4) in der Vertiefung aus; (1) und (3) bleiben Übungen, die wir gleich im Anschluss lösen.

Vertiefung: Beweise zu Skalarmultiplikation und Grenzwerten

Beweis.

- (1) Zu (2). Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und $\lambda \in [0, 1]$. Dann gilt

$$\begin{aligned} f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) &= c f_1(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \\ &\leq c(\lambda f_1(\mathbf{x}) + (1 - \lambda) f_1(\mathbf{y})) \\ &= \lambda c f_1(\mathbf{x}) + (1 - \lambda) c f_1(\mathbf{y}) = \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \end{aligned}$$

im zweiten Schritt geht beides ein: die Konvexität von f_1 liefert die Ungleichung, und $c \geq 0$ erlaubt es, mit c zu multiplizieren, ohne das Ungleichheitszeichen zu drehen. Für $c < 0$ kippt es, und $c f_1$ ist dann konkav

- (2) Zu (4). Wieder seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und $\lambda \in [0, 1]$. Wir setzen $\mathbf{z} := \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$, das nach Voraussetzung wieder in $\mathcal{X}$ liegt, und rechnen nach:

$$\begin{aligned} f(\mathbf{z}) &= \lim_{k \rightarrow \infty} f_k(\mathbf{z}) = \limsup_{k \rightarrow \infty} f_k(\mathbf{z}) \\ &\leq \limsup_{k \rightarrow \infty} [\lambda f_k(\mathbf{x}) + (1 - \lambda) f_k(\mathbf{y})] \\ &\leq \lambda \limsup_{k \rightarrow \infty} f_k(\mathbf{x}) + (1 - \lambda) \limsup_{k \rightarrow \infty} f_k(\mathbf{y}) \\ &= \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \end{aligned}$$

Zeile 1: f ist als Grenzwert definiert, und eine konvergente Folge hat Limes und Limes superior gemeinsam. Zeile 2: jedes einzelne f_k ist konvex, und der Limes superior ist monoton, aus $a_k \leq b_k$ folgt also $\limsup a_k \leq \limsup b_k$. Zeile 3: Subadditivität des Limes superior, zusammen mit $\lambda, 1 - \lambda \geq 0$. Zeile 4: beide Folgen konvergieren nach Voraussetzung, ihr Limes superior ist also ihr Grenzwert

- (3) Weil hier alle drei Folgen konvergieren, kämen wir auch mit den gewöhnlichen Grenzwertregeln aus. Der Umweg über den Limes superior kostet nichts und ist robuster: Er existiert immer, notfalls mit dem Wert $+\infty$, und braucht die Konvergenz erst in der letzten Zeile.

dieselbe Kette zeigt damit ohne Zusatzarbeit: Ist $\limsup_k f_k$ überall endlich, so ist auch diese Funktion konvex, ganz ohne Konvergenzannahme

□

Jetzt die beiden Übungen. Wer will, deckt die Lösungen ab und rechnet selbst.

Beispiel 11.4.2 (Übung: die Summe zweier konvexer Funktionen)

Zu zeigen ist Teil (1) von Satz 11.4.1: Sind f_1 und f_2 konvex, so ist $f = f_1 + f_2$ konvex. Zwei Ungleichungen stehen bereit, gesucht ist eine.

Lösung

Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und $\lambda \in [0, 1]$. Die Konvexität von f_1 und von f_2 liefert je eine Ungleichung, und beide zeigen in dieselbe Richtung:

$$\begin{aligned}
f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) &= f_1(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) + f_2(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) \\
&\leq [\lambda f_1(\mathbf{x}) + (1 - \lambda) f_1(\mathbf{y})] + [\lambda f_2(\mathbf{x}) + (1 - \lambda) f_2(\mathbf{y})] \\
&= \lambda [f_1(\mathbf{x}) + f_2(\mathbf{x})] + (1 - \lambda) [f_1(\mathbf{y}) + f_2(\mathbf{y})] \\
&= \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).
\end{aligned}$$

Der einzige Schritt, der etwas verlangt, ist der zweite: Zwei Ungleichungen mit demselben Zeichen dürfen addiert werden. Sortiert wird danach nur noch nach $\mathbf{x}$ und $\mathbf{y}$.

Beispiel 11.4.3 (Übung: das punktweise Maximum)

Zu zeigen ist Teil (3) von Satz 11.4.1 für endlich viele Funktionen: Sind $f_1, \dots, f_m$ konvex, so ist $f(\mathbf{x}) = \max_{i \leq m} f_i(\mathbf{x})$ konvex. Der Trick steckt in der Frage, *welches* i das Maximum liefert.

Lösung

Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und $\lambda \in [0, 1]$, und sei wieder $\mathbf{z} = \lambda \mathbf{x} + (1 - \lambda) \mathbf{y}$. Weil es nur endlich viele Funktionen sind, wird das Maximum an der Stelle $\mathbf{z}$ angenommen: Wir wählen ein

$$j \in \arg \max_{i \leq m} f_i(\mathbf{z}), \quad \text{also} \quad f(\mathbf{z}) = f_j(\mathbf{z}).$$

Für dieses eine j rechnen wir:

$$f(\mathbf{z}) = f_j(\mathbf{z}) \leq \lambda f_j(\mathbf{x}) + (1 - \lambda) f_j(\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}).$$

Die erste Ungleichung ist die Konvexität von f_j . Die zweite benutzt $f_j \leq f$ punktweise, denn f ist ja das Maximum über alle i , und dass die Gewichte λ und $1 - \lambda$ nichtnegativ sind.

Der Index j hängt von $\mathbf{z}$ ab, und an den Stellen $\mathbf{x}$ und $\mathbf{y}$ maximiert womöglich ein ganz anderer Index. Das stört nicht: Gebraucht wird j nur auf der linken Seite, und rechts schätzen wir sofort wieder gegen das Maximum ab. Genau deshalb funktioniert die Fallunterscheidung, ohne dass wir die Stücke des Maximums je einzeln verfolgen müssten.

Für unendlich viele Funktionen gibt es kein Argmax mehr. Dann argumentieren wir ohne Fallunterscheidung: Für jedes einzelne i ist

$$f_i(\mathbf{z}) \leq \lambda f_i(\mathbf{x}) + (1 - \lambda) f_i(\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}),$$

und die rechte Seite hängt nicht mehr von i ab. Sie ist damit eine obere Schranke für alle $f_i(\mathbf{z})$, also auch für deren Supremum $f(\mathbf{z})$.

Vertiefung: Was die vier Regeln zusammen hergeben

Bemerkung 11.4.4 (Was die vier Regeln zusammen ergeben)

Aus (1) und (2) folgt per Induktion sofort mehr: Jede *nichtnegative Linearkombination* $\sum_{i=1}^m c_i f_i$ mit $c_i \geq 0$ konvexer Funktionen ist konvex. Die konvexen Funktionen auf $\mathcal{X}$ bilden also einen konvexen Kegel, ganz analog zum Kegel der positiv semidefiniten Matrizen aus Satz 11.2.8.

Die Vorzeichenbedingung ist unverzichtbar. Für $c < 0$ dreht sich die Ungleichung, $c f_1$ ist dann konkav. Deshalb ist auch die Differenz zweier konvexer Funktionen im Allgemeinen weder konvex noch konkav: Mit $f_1(x) = x^2$ und $f_2(x) = 2x^2$ ist $f_1 - f_2 = -x^2$ konkav, mit $f_2(x) = \frac{1}{2}x^2$ dagegen bliebe die Differenz konvex.

Zu (3) trägt nur die Supremum-Fassung, denn ein Maximum über unendlich viele Werte muss weder angenommen werden noch endlich sein: Für $f_i(x) = i x^2$ ist das Supremum in jedem $x \neq 0$ gleich $+\infty$, und über solche Funktionen redet Definition 11.3.6 nicht.

Regel (4) erhält die Konvexität, nicht aber die Glattheit: Die Funktionen $f_k(x) = |x|^{1+1/k}$ sind konvex und überall differenzierbar, auch in $x = 0$ mit $f'_k(0) = 0$, streben punktweise aber gegen $|x|$ und damit gegen eine Funktion mit Knick. Das ist einer der Gründe, warum wir am Ende dieses Abschnitts einen Ersatz für die Tangente brauchen.

Beispiel 11.4.5 (Zielfunktionen aus Bausteinen)

Die Regeln erledigen Fälle, die aus der Definition heraus mühsam wären. Wir sammeln vier Beispiele ein, deren Bausteine wir schon kennen: Normen sind konvex, quadratische Funktionen mit positiv semidefinitem symmetrischen Anteil ebenfalls, und affine Funktionen sind sogar zugleich konvex und konkav (alles Abschnitt 11.3).

Ridge-Regression. Die Zielfunktion $f(\mathbf{w}) = \|\mathbf{A}\mathbf{w} - \mathbf{y}\|^2 + \lambda \|\mathbf{w}\|^2$ ist eine Summe zweier konvexer Funktionen, der zweite Summand zusätzlich mit dem Faktor $\lambda \geq 0$ versehen. Regeln (1) und (2) geben die Konvexität, ohne dass wir eine einzige Ungleichung nachrechnen. Beispiel 11.3.16 ist denselben Weg noch über die quadratische Form gegangen und hat dabei mehr herausgeholt, nämlich strikte Konvexität für $\lambda > 0$; die Regeln allein liefern das nicht. Ihr Vorteil zeigt sich beim nächsten Punkt.

LASSO. Ersetzen wir den Strafterm durch $\lambda \|\mathbf{w}\|_1$, ändert sich am Argument nichts, denn auch die ℓ_1 -Norm ist eine Norm – Satz 11.3.12 greift hier dagegen nicht mehr. Die Zielfunktion bleibt konvex, ist aber an den Stellen mit $w_j = 0$ nicht differenzierbar.

Hinge-Verlust und Support Vector Machines. Der Verlust $\ell(u) = \max\{0, 1 - u\}$ ist das Maximum zweier affiner Funktionen, nach Regel (3) also konvex. Setzen wir $u = \mathbf{y}_i \mathbf{x}_i^\top \boldsymbol{\beta}$ ein, so steht dort $\max\{0, 1 - \mathbf{y}_i \mathbf{x}_i^\top \boldsymbol{\beta}\}$, wieder ein Maximum zweier affiner Funktionen, diesmal in $\boldsymbol{\beta}$. Regel (3) greift also unmittelbar, und die Summe über die Beobachtungen ist nach Regel (1) wieder konvex. Den Umweg über eine Regel für Verkettungen brauchen wir nicht.

Vom empirischen zum theoretischen Risiko. Ist $\boldsymbol{\theta} \mapsto \ell(\boldsymbol{\theta}; \mathbf{z})$ für jede Beobachtung konvex, so ist das empirische Risiko $R_n(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell(\boldsymbol{\theta}; \mathbf{z}_i)$ als nichtnegative Linearkombination konvex, und weil es nach dem Gesetz der großen Zahlen punktweise gegen das theoretische Risiko $R(\boldsymbol{\theta}) = \mathbb{E}[\ell(\boldsymbol{\theta}; \mathbf{Z})]$ strebt, überträgt Regel (4) die Konvexität auf den Grenzwert.

Die Jensen-Ungleichung

Die Definition der Konvexität spricht über zwei Punkte. Konvexkombinationen aus Abschnitt 11.1 dürfen beliebig viele Punkte mischen. Der folgende Satz schließt die Lücke.

Satz 11.4.6 (Jensen-Ungleichung)

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und $f: \mathcal{X} \rightarrow \mathbb{R}$ konvex. Dann gilt für alle $N \in \mathbb{N}$, alle $\mathbf{x}_1, \dots, \mathbf{x}_N \in \mathcal{X}$ und alle Gewichte $w_1, \dots, w_N \geq 0$ mit $\sum_{i=1}^N w_i = 1$

$$f\left(\sum_{i=1}^N w_i \mathbf{x}_i\right) \leq \sum_{i=1}^N w_i f(\mathbf{x}_i). \quad (11.4.1)$$

Vertiefung: Induktionsbeweis der Jensen-Ungleichung

Beweis.

- (1) Induktion über N . Für $N = 1$ ist $w_1 = 1$, und auf beiden Seiten steht $f(\mathbf{x}_1)$. Für $N = 2$ ist (11.4.1) mit $\lambda := w_1$ und $w_2 = 1 - \lambda$ wörtlich die Definition einer konvexen Funktion.

die Jensen-Ungleichung ist also keine neue Eigenschaft, sondern die Definition, mehrfach angewandt

- (2) Sei nun $N \geq 3$ und die Behauptung für $N - 1$ Punkte bereits bewiesen. Ist $w_N = 1$, so sind alle übrigen Gewichte null, und beide Seiten sind $f(\mathbf{x}_N)$. Sei also $w_N < 1$ und

$$s := \sum_{i=1}^{N-1} w_i = 1 - w_N > 0, \quad \mathbf{z} := \sum_{i=1}^{N-1} \frac{w_i}{s} \mathbf{x}_i.$$

Die Koeffizienten w_i/s sind nichtnegativ und summieren sich zu 1, also ist $\mathbf{z}$ eine Konvexkombination von $\mathbf{x}_1, \dots, \mathbf{x}_{N-1}$ und liegt in $\mathcal{X}$.

hier zählt sich Satz 11.2.3 aus: Eine konvexe Menge enthält nicht nur die Verbindungsstrecken je zweier Punkte, sondern alle endlichen Konvexkombinationen. Ohne diese Aussage wüssten wir nicht, dass $f(\mathbf{z})$ überhaupt definiert ist

- (3) Nach Konstruktion ist $\sum_{i=1}^N w_i \mathbf{x}_i = s \mathbf{z} + w_N \mathbf{x}_N$ eine Konvexkombination der beiden Punkte $\mathbf{z}$ und $\mathbf{x}_N$, denn $s + w_N = 1$ und beide Gewichte sind nichtnegativ. Damit rechnen wir

$$\begin{aligned} f\left(\sum_{i=1}^N w_i \mathbf{x}_i\right) &= f(s \mathbf{z} + w_N \mathbf{x}_N) \\ &\leq s f(\mathbf{z}) + w_N f(\mathbf{x}_N) \\ &\leq s \sum_{i=1}^{N-1} \frac{w_i}{s} f(\mathbf{x}_i) + w_N f(\mathbf{x}_N) = \sum_{i=1}^N w_i f(\mathbf{x}_i). \end{aligned}$$

die erste Ungleichung ist die Definition der Konvexität für die zwei Punkte $\mathbf{z}$ und $\mathbf{x}_N$, die zweite ist die Induktionsvoraussetzung, multipliziert mit $s \geq 0$. Im letzten Schritt kürzt sich s heraus

□

Bemerkung 11.4.7 (Gewichte als Wahrscheinlichkeiten)

Beispiel 11.1.3 hat den Erwartungswert einer Zufallsvariablen mit endlichem Träger als Konvexkombination gelesen: Die Gewichte sind die Wahrscheinlichkeiten, der Erwartungswert ist also selbst eine Mischung. In dieser Sprache lautet (11.4.1)

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)],$$

und das ist die Form, in der die Jensen-Ungleichung meist zitiert wird. Erst mischen und dann auswerten gibt höchstens so viel wie erst auswerten und dann mischen.

Zwei Einschränkungen sind dabei zu beachten. Satz 11.4.6 deckt nur endlich viele Punkte ab, also Zufallsvariablen mit endlichem Träger. Für allgemeine integrierbare X gilt die Unglei-

chung ebenfalls, der Beweis läuft dann aber nicht mehr über die Induktion; am Ende dieses Abschnitts fällt er als Nebenprodukt der Stützgeraden ab (Bemerkung 11.4.17). Und für konkave f dreht sich alles um: Dort gilt $f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$, was etwa beim Logarithmus in Likelihood-Rechnungen ständig gebraucht wird.

Beispiel 11.4.8 (Die Varianz ist nicht negativ)

Aus der Jensen-Ungleichung folgt eine Aussage, die wir längst benutzen, ohne sie je bewiesen zu haben. Die Funktion $f(x) = x^2$ ist konvex auf $\mathbb{R}$. Für eine Zufallsvariable X mit endlichem Träger liefert Bemerkung 11.4.7 damit

$$(\mathbb{E}[X])^2 = f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)] = \mathbb{E}[X^2],$$

und Umstellen ergibt

$$\text{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \geq 0.$$

Der faire Würfel aus Beispiel 11.1.3 zeigt die Größenordnung: $\mathbb{E}[X] = 3,5$ und $(\mathbb{E}[X])^2 = 12,25$ stehen gegen $\mathbb{E}[X^2] = \frac{91}{6} \approx 15,17$, und die Differenz $\frac{35}{12} \approx 2,92$ ist die Varianz. Gleichheit tritt genau dann ein, wenn die Varianz verschwindet, wenn also X mit Wahrscheinlichkeit 1 konstant ist. Die Jensen-Lücke misst hier direkt die Streuung.

Interaktiv: Die Jensen-Ungleichung zum Schieben

Der Satz sagt, dass zwei Zahlen in einer bestimmten Reihenfolge stehen. Sehen wir sie uns an. Drei feste Stützstellen bekommen einstellbare Gewichte, und das Widget zeichnet beide Seiten von (11.4.1) über derselben Stelle $\sum_i w_i x_i$: orange den Funktionswert der Mischung, grün die Mischung der Funktionswerte. Wie groß ist der Abstand zwischen beiden, und bei welcher Reglerstellung verschwindet er?

Interaktives Element — nur in der Web-Fassung

Das grüne Dreieck ist die konvexe Hülle der drei Punkte auf dem Graphen. Bei konvexem f liegt es nie unter der Kurve, seine Ecken berühren sie ja. Der grüne Punkt kann den orangenen deshalb nie unterbieten. Für $f(x) = x^2$ und gleiche Gewichte stehen 3,3611 gegen 4,9167, die Lücke beträgt 1,5556. Das ist genau die gewichtete Varianz der drei Stützstellen, denn für $x \mapsto x^2$ ist die Jensen-Lücke nach Beispiel 11.4.8 die Differenz $\mathbb{E}[X^2] - (\mathbb{E}[X])^2$. Bei $f(x) = e^x$ wächst die Lücke auf 6,8273, weil die Krümmung dort viel stärker zunimmt. Die dritte Wahl ist eine Gegenprobe: $\sqrt{x}$ ist konkav, und prompt tauschen die beiden Markierungen die Plätze, 1,2676 gegen 1,3540. Und wer alles Gewicht auf eine einzige Stützstelle schiebt, erzeugt den Gleichheitsfall: Dann steht auf beiden Seiten derselbe Wert, so wie bei einer Zufallsvariablen mit Varianz 0.

Konvexität an der Ableitung ablesen

Bisher haben wir Konvexität aus Konvexität gemacht. Die Bausteine selbst mussten wir immer noch aus der Definition heraus prüfen. Für zweimal stetig differenzierbare Funktionen geht es bequemer, und das Ergebnis verbindet dieses Kapitel mit Abschnitt 10.7.

Satz 11.4.9 (Konvexe Funktionen von Vektoren zu Skalaren)

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ eine offene, konvexe Menge und $f : \mathcal{X} \rightarrow \mathbb{R}$ zweimal stetig differenzierbar, also $f \in \mathcal{C}^2$. Dann sind die folgenden Aussagen äquivalent:

1. f ist konvex.
2. Mit dem Gradienten gilt für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})(\mathbf{y} - \mathbf{x}). \quad (11.4.2)$$

3. Die Hesse-Matrix $\mathbf{H}_f(\mathbf{x})$ ist positiv semidefinit für alle $\mathbf{x} \in \mathcal{X}$.

Alle drei Aussagen beschreiben dieselbe Krümmungseigenschaft, jede aus einer anderen Entfernung. Aussage (1) ist die Sehnbedingung: Die Verbindungsstrecke zwischen zwei Punkten des Graphen verläuft nie unter dem Graphen. Aussage (2) ist die Tangentenbedingung: Die Tangentialebene im Punkt $\mathbf{x}$ verläuft nie über dem Graphen, die lineare Näherung unterschätzt f also überall. Aussage (3) ist die punktweise Krümmungsbedingung: In keiner Richtung und an keiner Stelle krümmt sich f nach unten, denn die quadratische Form $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$ ist überall nichtnegativ. In einer Variablen schrumpft (3) auf die Schulregel $f'' \geq 0$ zusammen.

Die Äquivalenz von (1) und (3) haben wir in Abschnitt 10.7 als Satz 10.7.11 schon einmal gesehen, damals ohne Beweis. Den holen wir jetzt nach, und zwar über (2) als Drehscheibe.

Vertiefung: Beweis der drei Charakterisierungen

Beweis.

- (1) $(1) \Rightarrow (2)$. Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und $\lambda \in (0, 1)$. Wir setzen $t := 1 - \lambda \in (0, 1)$ und beobachten $\lambda \mathbf{x} + (1 - \lambda) \mathbf{y} = \mathbf{x} + t(\mathbf{y} - \mathbf{x})$. Die Konvexität liefert

$$\begin{aligned} f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) &\leq \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y}) \\ \Leftrightarrow f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - f(\mathbf{x}) &\leq t[f(\mathbf{y}) - f(\mathbf{x})] \\ \Leftrightarrow \frac{f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - f(\mathbf{x})}{t} &\leq f(\mathbf{y}) - f(\mathbf{x}). \end{aligned}$$

in der zweiten Zeile ist $\lambda = 1 - t$ eingesetzt und $f(\mathbf{x})$ auf beiden Seiten abgezogen; in der dritten wird durch $t > 0$ geteilt, das Ungleichheitszeichen bleibt also stehen. Dass der Punkt $\mathbf{x} + t(\mathbf{y} - \mathbf{x})$ überhaupt in $\mathcal{X}$ liegt, ist die Konvexität der Menge

- (2) Auf der linken Seite steht ein Differenzenquotient in Richtung $\mathbf{h} := \mathbf{y} - \mathbf{x}$. Weil f differenzierbar ist, strebt er für $t \rightarrow 0^+$ gegen die Richtungsableitung $\nabla f(\mathbf{x}) \mathbf{h}$. Die rechte Seite hängt nicht von t ab, die Ungleichung überlebt den Grenzübergang, und wir erhalten

$$\nabla f(\mathbf{x})(\mathbf{y} - \mathbf{x}) \leq f(\mathbf{y}) - f(\mathbf{x}),$$

also (11.4.2). Der Fall $\mathbf{y} = \mathbf{x}$ war nicht mitgerechnet, dort steht in (11.4.2) aber ohnehin nur $f(\mathbf{x}) \geq f(\mathbf{x})$.

der Grenzwert eines Differenzenquotienten ist die Ableitung in Richtung $\mathbf{h}$, siehe Abschnitt 10.2; schwache Ungleichungen bleiben beim Grenzübergang erhalten, strikte nicht

- (3) $(2) \Rightarrow (1)$. Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ und $\lambda \in [0, 1]$. Wir setzen $\mathbf{z} := \lambda \mathbf{x} + (1 - \lambda) \mathbf{y} \in \mathcal{X}$ und wenden (11.4.2) zweimal an, einmal mit $\mathbf{z}$ als Entwicklungspunkt und $\mathbf{x}$ als Argument, einmal mit $\mathbf{y}$:

$$\begin{aligned} \lambda f(\mathbf{x}) &\geq \lambda f(\mathbf{z}) + \lambda \nabla f(\mathbf{z})(\mathbf{x} - \mathbf{z}), \\ (1 - \lambda)f(\mathbf{y}) &\geq (1 - \lambda)f(\mathbf{z}) + (1 - \lambda)\nabla f(\mathbf{z})(\mathbf{y} - \mathbf{z}). \end{aligned}$$

Addieren wir beide Zeilen, so fällt der Gradiententerm weg:

$$\lambda(\mathbf{x} - \mathbf{z}) + (1 - \lambda)(\mathbf{y} - \mathbf{z}) = \lambda\mathbf{x} + (1 - \lambda)\mathbf{y} - \mathbf{z} = \mathbf{0},$$

und es bleibt $\lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y}) \geq f(\mathbf{z})$, also genau die Definition der Konvexität.

beide Ungleichungen werden mit den nichtnegativen Zahlen λ und $1 - \lambda$ multipliziert, das Zeichen bleibt. Der Gradient $\nabla f(\mathbf{z})$ ist in beiden Zeilen derselbe Zeilenvektor und lässt sich ausklammern, deshalb verschwindet er mit der Klammer

- (4) (2) $\Rightarrow$ (3). Sei $\mathbf{x} \in \mathcal{X}$ und $\mathbf{h} \in \mathbb{R}^n$ beliebig. Weil $\mathcal{X}$ offen ist, liegt $\mathbf{x} + t\mathbf{h}$ für alle hinreichend kleinen $t > 0$ noch in $\mathcal{X}$. Die Taylorentwicklung zweiter Ordnung aus Korollar 10.8.9 lautet

$$f(\mathbf{x} + t\mathbf{h}) = f(\mathbf{x}) + t \nabla f(\mathbf{x}) \mathbf{h} + \frac{t^2}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h} + o(t^2).$$

Nach (11.4.2) mit $\mathbf{y} = \mathbf{x} + t\mathbf{h}$ ist die linke Seite mindestens $f(\mathbf{x}) + t \nabla f(\mathbf{x}) \mathbf{h}$. Umgestellt heißt das

$$\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h} = \frac{2[f(\mathbf{x} + t\mathbf{h}) - f(\mathbf{x}) - t \nabla f(\mathbf{x}) \mathbf{h}]}{t^2} + o(1) \geq 0 + o(1) \rightarrow 0 \quad (t \rightarrow 0^+).$$

Die linke Seite hängt nicht von t ab, also ist sie nichtnegativ. Da $\mathbf{h}$ beliebig war, ist $\mathbf{H}_f(\mathbf{x})$ positiv semidefinit.

der Faktor 2 vor der eckigen Klammer kommt aus dem $\frac{1}{2}$ der Taylorformel; wer ihn vergisst, verschiebt die Kette um den Faktor 2, am Vorzeichen und damit an der Schlussfolgerung ändert das nichts

- (5) (3) $\Rightarrow$ (2). Seien $\mathbf{x}, \mathbf{y} \in \mathcal{X}$. Weil $\mathcal{X}$ konvex ist, liegt die ganze Verbindungsstrecke in $\mathcal{X}$, und wir dürfen f darauf einschränken:

$$g(s) := f(\mathbf{x} + s(\mathbf{y} - \mathbf{x})), \quad s \in [0, 1].$$

Nach der Kettenregel aus Abschnitt 10.6 ist g zweimal stetig differenzierbar mit

$$g'(s) = \nabla f(\mathbf{x} + s(\mathbf{y} - \mathbf{x}))(\mathbf{y} - \mathbf{x}), \quad g''(s) = (\mathbf{y} - \mathbf{x})^\top \mathbf{H}_f(\mathbf{x} + s(\mathbf{y} - \mathbf{x}))(\mathbf{y} - \mathbf{x}).$$

g ist die Verkettung von f mit der affinen Kurve $s \mapsto \mathbf{x} + s(\mathbf{y} - \mathbf{x})$, deren Ableitung konstant $\mathbf{y} - \mathbf{x}$ ist; die zweite Ableitung ist dann die quadratische Form der Hesse-Matrix an der laufenden Stelle, vgl. Satz 10.7.6

- (6) Auf g wenden wir die eindimensionale Taylorentwicklung mit Lagrange-Restglied an (Satz 10.8.2 mit $k = 1$): Es gibt ein $t \in (0, 1)$ mit $g(1) = g(0) + g'(0) + \frac{1}{2}g''(t)$, ausgeschrieben

$$f(\mathbf{y}) = f(\mathbf{x}) + \nabla f(\mathbf{x})(\mathbf{y} - \mathbf{x}) + \frac{1}{2}(\mathbf{y} - \mathbf{x})^\top \mathbf{H}_f(\mathbf{x} + t(\mathbf{y} - \mathbf{x}))(\mathbf{y} - \mathbf{x}).$$

Der letzte Summand ist nach (3) nichtnegativ, denn die Zwischenstelle $\mathbf{x} + t(\mathbf{y} - \mathbf{x})$ liegt in $\mathcal{X}$. Weglassen macht die rechte Seite also nur kleiner, und es folgt (11.4.2).

Damit ist der Ring geschlossen: Die Schritte 1 bis 3 zeigen (1) $\Leftrightarrow$ (2), die Schritte 4 bis 6 zeigen (2) $\Leftrightarrow$ (3).

hier geht die Konvexität von $\mathcal{X}$ ein zweites Mal ein: Ohne sie wüssten wir nicht, dass die Hesse-Matrix an der Zwischenstelle überhaupt zur Verfügung steht

□

Bemerkung 11.4.10 (Wo die Voraussetzungen stecken)

Der Satz nennt drei Bedingungen an $\mathcal{X}$ und f , und jede wird an einer bestimmten Stelle gebraucht.

Konvex muss $\mathcal{X}$ sein, damit in den Schritten 1 und 5 überhaupt Punkte auf der Verbindungsstrecke zur Verfügung stehen. Offen muss $\mathcal{X}$ sein, damit in Schritt 4 zu jeder Richtung $\mathbf{h}$ ein

Stück Weg in $\mathcal{X}$ bleibt; an einem Randpunkt gäbe es Richtungen, in denen wir sofort hinauslaufen, und die Aussage über $\mathbf{H}_f$ zöge Information aus Richtungen, die gar nicht vorkommen. Bemerkung 10.7.12 führt beide Fälle mit Gegenbeispielen vor.

Zweimal stetig differenzierbar muss f sein, damit die Hesse-Matrix existiert und die Taylorformeln gelten. Für (1) $\Leftrightarrow$ (2) allein genügt einfache Differenzierbarkeit.

Zwei Verschärfungen liegen nahe und gelten nur halb. Ist $\mathbf{H}_f(\mathbf{x})$ an jeder Stelle positiv definit, so ist f strikt konvex. Die Umkehrung ist falsch: $f(x) = x^4$ ist strikt konvex, hat aber $f''(0) = 0$. Wir kennen den Grenzfall aus Bemerkung 10.7.10, wo dieselbe Funktion zeigt, dass die zweite Ableitung im semidefiniten Fall nichts entscheidet.

Beispiel 11.4.11 (Quadratische Funktionen, mit dem Kriterium nachgerechnet)

Satz 11.3.12 hat für die quadratische Funktion $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{b}^\top \mathbf{x} + a$ schon gezeigt: Ist der symmetrische Anteil $\mathbf{Q}_{\text{sym}} = \frac{1}{2}(\mathbf{Q} + \mathbf{Q}^\top)$ positiv semidefinit, so ist f konvex. Bewiesen haben wir das damals über den Sehnendefekt. Satz 11.4.9 liefert denselben Schluss in zwei Zeilen und obendrein die Umkehrung, macht die Bedingung also zu einem *genau dann, wenn*. Den Gradienten der quadratischen Form kennen wir aus Abschnitt 10.2, und noch einmal ableiten gibt die Hesse-Matrix:

$$\nabla f(\mathbf{x}) = \mathbf{x}^\top (\mathbf{Q} + \mathbf{Q}^\top) + \mathbf{b}^\top, \quad \mathbf{H}_f(\mathbf{x}) = \mathbf{Q} + \mathbf{Q}^\top.$$

Die Hesse-Matrix ist konstant, und nach (3) ist f genau dann konvex, wenn $\mathbf{Q} + \mathbf{Q}^\top \succeq 0$ gilt. Für symmetrisches $\mathbf{Q}$ steht dort $2\mathbf{Q}$, und die Bedingung ist wörtlich $\mathbf{Q} \succeq 0$.

Bemerkung 11.3.13 hat eine Matrix gezeigt, die trotz fehlender Symmetrie eine strikt konvexe Funktion erzeugt. Die andere Richtung ist ebenso lehrreich: $\mathbf{Q}$ kann lauter positive Eigenwerte haben und die Funktion trotzdem nicht konvex sein. Für $\mathbf{Q} = \begin{pmatrix} 2 & 3 \\ 0 & 1 \end{pmatrix}$ stehen auf der Diagonalen der Dreiecksmatrix die Eigenwerte 2 und 1, beide positiv. Es ist aber $\mathbf{Q} + \mathbf{Q}^\top = \begin{pmatrix} 4 & 3 \\ 3 & 2 \end{pmatrix}$ mit den Eigenwerten 6,162 und $-0,162$, und die zugehörige Funktion ist nicht konvex. Zum negativen Eigenwert gehört ungefähr die Richtung $\mathbf{d} = (2; -3)^\top$, denn $\mathbf{d}^\top (\mathbf{Q} + \mathbf{Q}^\top) \mathbf{d} = 16 - 36 + 18 = -2$. Mit $\mathbf{x} = (1; -1,5)^\top$, $\mathbf{y} = -\mathbf{x}$, $\mathbf{b} = \mathbf{0}$ und $a = 0$ steht im Mittelpunkt $\mathbf{0}$ der Wert $f(\mathbf{0}) = 0$ gegen den Mittelwert $\frac{1}{2}f(\mathbf{x}) + \frac{1}{2}f(\mathbf{y}) = -0,25$: Die Sehne verläuft unter dem Graphen. Die Richtung muss dabei sorgfältig gewählt sein – die ähnlich aussehende $\mathbf{d} = (1; -1)^\top$ liefert $4 - 6 + 2 = 0$ und damit exakt Gleichheit, zeigt also gar nichts.

Vertiefung: Die logistische Regression als konvexes Problem

Beispiel 11.4.12 (Logistische Regression ist ein konvexes Problem)

Die negative Log-Likelihood der logistischen Regression aus Abschnitt 10.6 lautet mit $\pi_i(\boldsymbol{\beta}) = 1/(1 + e^{-\mathbf{x}_i^\top \boldsymbol{\beta}})$

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^n \left[\log(1 + e^{\mathbf{x}_i^\top \boldsymbol{\beta}}) - y_i \mathbf{x}_i^\top \boldsymbol{\beta} \right], \quad \nabla \ell(\boldsymbol{\beta}) = \sum_{i=1}^n (\pi_i(\boldsymbol{\beta}) - y_i) \mathbf{x}_i.$$

Noch einmal ableiten gibt

$$\mathbf{H}_\ell(\boldsymbol{\beta}) = \sum_{i=1}^n \pi_i(\boldsymbol{\beta}) (1 - \pi_i(\boldsymbol{\beta})) \mathbf{x}_i \mathbf{x}_i^\top.$$

Jeder Summand ist positiv semidefinit, denn $\mathbf{h}^\top \mathbf{x}_i \mathbf{x}_i^\top \mathbf{h} = (\mathbf{x}_i^\top \mathbf{h})^2 \geq 0$, und der Vorfaktor $\pi_i(1 - \pi_i)$ ist wegen $\pi_i \in (0, 1)$ positiv; sein größter Wert $\frac{1}{4}$ wird bei $\pi_i = \frac{1}{2}$ angenommen, also genau dann, wenn $\mathbf{x}_i^\top \boldsymbol{\beta} = 0$ ist. Eine Summe positiv semidefiniter Matrizen ist wieder positiv semidefinit, also ist $\mathbf{H}_\ell(\boldsymbol{\beta}) \succeq 0$ an jeder Stelle, und nach Satz 11.4.9 ist ℓ konvex. Die Log-Likelihood selbst ist damit konkav, und ihre Maximierung ist ein konvexes Problem.

Ein Zahlenbeispiel mit vier Beobachtungen, $\mathbf{x}_i^\top = (1, x_i)$ und $\mathbf{x} = (0,5; -1,2; 2,0; 0,1)$: An der Stelle $\boldsymbol{\beta} = (0,3; -0,7)^\top$ ist

$$\mathbf{H}_\ell(\boldsymbol{\beta}) = \begin{pmatrix} 0,8675 & 0,3040 \\ 0,3040 & 1,0788 \end{pmatrix},$$

mit den Eigenwerten 1,2950 und 0,6513, also sogar positiv definit. Definitheit statt Semidefinitheit bekommen wir hier, weil die Regressormatrix vollen Spaltenrang hat. Bei linear abhängigen Regressoren bliebe nur Semidefinitheit, und mit ihr die Mehrdeutigkeit des Maximums.

Das Spektrum der Hesse-Matrix**Bemerkung 11.4.13 (Eigenwerte als Krümmungen)**

Für $f \in \mathcal{C}^2$ ist $\mathbf{H}_f(\mathbf{x})$ nach dem Satz von Schwarz symmetrisch (Satz 10.7.4). Der Spektralsatz liefert deshalb reelle Eigenwerte $\lambda_1, \dots, \lambda_n$ und eine Orthonormalbasis aus Eigenvektoren $\mathbf{v}_1, \dots, \mathbf{v}_n$. Zerlegen wir einen Zuwachs $\mathbf{h}$ in dieser Basis, so wird aus der Taylorentwicklung zweiter Ordnung

$$f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h} = f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \sum_{i=1}^n \lambda_i (\mathbf{v}_i^\top \mathbf{h})^2.$$

Die letzte Gleichung ist die Spektralzerlegung $\mathbf{H}_f = \sum_i \lambda_i \mathbf{v}_i \mathbf{v}_i^\top$, eingesetzt in die quadratische Form. Sie zerlegt die Krümmung in Richtungen, und jede Richtung bekommt ihre eigene Zahl:

- $\lambda_i > 0$: In Richtung $\mathbf{v}_i$ krümmt sich f nach oben, wir stehen in einem Tal.
- $\lambda_i < 0$: In Richtung $\mathbf{v}_i$ krümmt sich f nach unten, wir stehen auf einer Kuppe.
- $|\lambda_i|$ misst, wie stark die Krümmung in Richtung $\mathbf{v}_i$ ausfällt.

Konvexität ist in dieser Sprache leicht zu sagen: Nach Satz 11.4.9(3) ist f genau dann konvex, wenn an jeder Stelle *alle* Eigenwerte nichtnegativ sind. Es gibt dann nirgends eine Richtung, in der die Funktion nach unten wegkippt, und damit auch keine Sattelpunkte, in denen ein Verfahren steckenbleiben könnte.

Der Gradiententerm darf in dieser Näherung nicht fehlen: Die verkürzte Fassung $f(\mathbf{x} + \mathbf{h}) \approx f(\mathbf{x}) + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$ stimmt nur an kritischen Punkten, sonst ist der weggelassene Term für kleine $\mathbf{h}$ sogar der größere von beiden. Zum Anschauen eignet sich das Definitheits-Widget in Abschnitt 10.7, dessen Höhenlinien von Ellipsen zu Hyperbeln wechseln, sobald ein Eigenwert unter null rutscht.

Stützgeraden und Subgradienten

Aussage (2) von Satz 11.4.9 ist eine bemerkenswerte Eigenschaft: Eine konvexe Funktion liegt nirgends unter ihrer Tangentialebene. Was aber, wenn es keine Tangente gibt? Der Betrag hat im Nullpunkt keine, der LASSO-Strafterm ebenfalls nicht, und nach Bemerkung 11.4.4 kann selbst der Grenzwert glatter konvexer Funktionen einen Knick bekommen. Es zeigt sich, dass die Existenz einer Stützgeraden erhalten bleibt, auch wenn die Ableitung fehlt.

Definition 11.4.14 (Subgradient und Subdifferential)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ konvex, $f: \mathcal{X} \rightarrow \mathbb{R}$ und $\mathbf{x} \in \mathcal{X}$. Ein Vektor $\mathbf{v} \in \mathbb{R}^n$ heißt *Subgradient* (subgradient) von f in $\mathbf{x}$, falls

$$f(\mathbf{y}) \geq f(\mathbf{x}) + \mathbf{v}^\top (\mathbf{y} - \mathbf{x}) \quad \text{für alle } \mathbf{y} \in \mathcal{X}. \quad (11.4.3)$$

Die Menge aller Subgradienten in $\mathbf{x}$ heißt *Subdifferential* und wird $\partial f(\mathbf{x})$ geschrieben.

Ungleichung (11.4.3) ist (11.4.2) mit $\mathbf{v}^\top$ anstelle des Gradienten. Sie sagt, dass die affine Funktion $\mathbf{y} \mapsto f(\mathbf{x}) + \mathbf{v}^\top (\mathbf{y} - \mathbf{x})$ nirgends über dem Graphen von f verläuft und ihn in $\mathbf{x}$ berührt. Ist f in einem inneren Punkt $\mathbf{x}$ differenzierbar, so ist $\nabla f(\mathbf{x})^\top$ ein solcher Vektor, und dann ist er auch der einzige. Die Schritte 1 und 2 im Beweis von Satz 11.4.9 brauchen dafür nur Differenzierbarkeit in genau diesem einen Punkt, nicht die $\mathcal{C}^2$ -Voraussetzung des Satzes. Am Knick können dagegen viele auftreten: Für $f(x) = |x|$ ist $\partial f(0) = [-1, 1]$ (Herleitung in der Vertiefung Beispiel 11.4.16).

Satz 11.4.15 (Existenz von Subgradienten im Inneren)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und $f: \mathcal{X} \rightarrow \mathbb{R}$ konvex. Dann gibt es zu jedem $\mathbf{x} \in \text{int}(\mathcal{X})$ einen Subgradienten, es ist also $\partial f(\mathbf{x}) \neq \emptyset$.

Vertiefung: Beweis, Subdifferential des Betrags und wozu Subgradienten dienen

Beweis.

- (1) *Der eindimensionale Fall, in der Skizze.* Schreiben wir für eine Richtung $\mathbf{h}$ und $0 < s \leq t$ mit $\mathbf{x} + t\mathbf{h} \in \mathcal{X}$ den Punkt

$$\mathbf{x} + s\mathbf{h} = \left(1 - \frac{s}{t}\right) \mathbf{x} + \frac{s}{t} (\mathbf{x} + t\mathbf{h})$$

als Konvexkombination, so liefert die Konvexität von f nach Umstellen

$$\frac{f(\mathbf{x} + s\mathbf{h}) - f(\mathbf{x})}{s} \leq \frac{f(\mathbf{x} + t\mathbf{h}) - f(\mathbf{x})}{t},$$

der Differenzenquotient wächst also monoton in der Schrittweite. In einem inneren Punkt $\mathbf{x}$ existieren deshalb beide einseitigen Ableitungen als monotone, beschränkte Grenzwerte, und es gilt $f'_-(\mathbf{x}) \leq f'_+(\mathbf{x})$. Jedes $\mathbf{v} \in [f'_-(\mathbf{x}), f'_+(\mathbf{x})]$ erfüllt dann (11.4.3): Für $\mathbf{y} > \mathbf{x}$ ist der Differenzenquotient

mindestens $f'_+(\mathbf{x})$ und damit mindestens v , für $y < \mathbf{x}$ höchstens $f'_-(\mathbf{x})$ und damit höchstens v , und die Multiplikation mit $y - \mathbf{x}$ dreht im zweiten Fall das Ungleichheitszeichen um.

die Monotonie ist Schritt 1 im Beweis von Satz 11.4.9 ohne den Grenzübergang; eine monotone, beschränkte Folge konvergiert, und die Schranke liefert jeweils der andere einseitige Quotient. Ist f differenzierbar, fallen f'_- und f'_+ zusammen

- (2) Für $n > 1$ liefert dasselbe Argument in jeder einzelnen Richtung eine Steigung, aber noch keinen Vektor $\mathbf{v}$, der alle Richtungen gleichzeitig bedient. Den Rest erledigt eine Trennungsaussage: Der Epigraph $\text{epi}(f) = \{(\mathbf{y}, r) : \mathbf{y} \in \mathcal{X}, r \geq f(\mathbf{y})\} \subseteq \mathbb{R}^{n+1}$ aus Definition 11.3.5 ist konvex, und $(\mathbf{x}, f(\mathbf{x}))$ ist ein Randpunkt. Eine stützende Hyperebene in diesem Randpunkt, also eine, die $\text{epi}(f)$ ganz auf einer Seite lässt, hat wegen $\mathbf{x} \in \text{int}(\mathcal{X})$ eine nichtverschwindende letzte Koordinate; nach Division durch diese Koordinate steht (11.4.3) da. Die Trennungsaussage selbst beweisen wir hier nicht, sie lässt sich aus dem Projektionssatz in Abschnitt 11.3 gewinnen.

dass der Epigraph konvex ist, ist Definition 11.3.6, die konvexe Funktionen genau darüber erklärt; läge $\mathbf{x}$ auf dem Rand von $\mathcal{X}$, könnte die stützende Hyperebene senkrecht stehen, und die Division wäre nicht möglich. Genau daran scheitert Bemerkung 11.4.17

□

Beispiel 11.4.16 (Das Subdifferential des Betrags)

Sei $f(x) = |x|$ auf $\mathcal{X} = \mathbb{R}$. Für $x > 0$ ist f differenzierbar mit Ableitung 1, und $\partial f(x) = \{1\}$; für $x < 0$ entsprechend $\{-1\}$. Interessant ist der Knick in $x = 0$. Dort verlangt (11.4.3)

$$|y| \geq v y \quad \text{für alle } y \in \mathbb{R},$$

und das ist für $v \in [-1, 1]$ erfüllt, sonst nicht: Für $v > 1$ scheitert $y = 1$, denn $1 < v$; für $v < -1$ scheitert $y = -1$. Also ist $\partial f(0) = [-1, 1]$, ein ganzes Intervall von Stützgeraden. Der Satz sagt nur, dass es mindestens eine gibt; eindeutig ist sie an einem Knick gerade nicht.

Diese Vielfalt hat eine praktische Kehrseite, die den LASSO erklärt. Für die eindimensionale Zielfunktion $g(w) = \frac{1}{2}(w - z)^2 + \lambda |w|$ mit $\lambda > 0$ ist $0 \in \partial g(w^*)$ die Optimalitätsbedingung, und sie führt auf die Schwellenwertregel

$$w^* = \text{sign}(z) \max\{|z| - \lambda, 0\}.$$

Für $z = 2$ und $\lambda = 0,5$ steht dort 1,5, für $z = 0,3$ dagegen exakt 0. Weil das Subdifferential im Nullpunkt ein ganzes Intervall ist, kann die Bedingung dort für einen ganzen Bereich von z erfüllt sein, und der LASSO setzt Koeffizienten wirklich auf null statt nur nahe an null.

Bemerkung 11.4.17 (Randpunkte, und wozu Subgradienten gut sind)

Der innere Punkt in Satz 11.4.15 ist keine Formalie: Am Rand kann die Aussage scheitern. Auf $\mathcal{X} = [0, \infty)$ ist $f(x) = -\sqrt{x}$ konvex, aber im Randpunkt 0 gibt es keinen Subgradienten. Für $v \geq 0$ scheitert schon $y = 1$, denn dort verlangt (11.4.3) die falsche Aussage $-1 \geq v$. Und für $v = -c$ mit $c > 0$ nimmt $f(y) - v y = -\sqrt{y} + c y$ bei $y = 1/(4c^2)$ den Wert $-1/(4c) < 0$ an, während (11.4.3) dort ≥ 0 verlangt. Für $c = 10$ etwa ist das $-0,025$ an der Stelle $y = 0,0025$. Anschaulich wird die Funktion am Rand unendlich steil, und keine Gerade kommt hinterher. Satz 11.4.15 verlangt deshalb einen inneren Punkt; im Inneren von $[0, \infty)$ existiert der Subgradient wie versprochen. Die Existenz von Stützgeraden ist mehr wert, als sie zunächst aussieht. Sie liefert etwa die Jensen-Ungleichung für beliebige integrierbare Zufallsvariablen, die Satz 11.4.6 mit seinen endlich vielen Gewichten nicht erreicht: Ist $\mu = \mathbb{E}[X]$ ein innerer Punkt von $\mathcal{X}$ und $v \in \partial f(\mu)$, so gilt punktweise $f(X) \geq f(\mu) + v^T(X - \mu)$, und Erwartungswertbildung auf beiden Seiten lässt den zweiten Summanden verschwinden. Übrig bleibt $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$.

Algorithmisch sind Subgradienten die Eintrittskarte in die nichtdifferenzierbare Optimierung. Wo der Gradient fehlt, tritt ein beliebiger Subgradient an seine Stelle, und Verfahren wie der Gradientenabstieg bleiben in ihrer Subgradienten-Fassung anwendbar. Typische Anwendungen sind der LASSO und die Quantilregression, deren Verlustfunktion $\rho_\tau(u) = u(\tau - 1\{u < 0\})$ genau wie der Betrag einen Knick in $u = 0$ hat und für $\tau = \frac{1}{2}$ bis auf den Faktor $\frac{1}{2}$ mit ihm übereinstimmt. In beiden Fällen ist die Zielfunktion konvex, aber nicht differenzierbar. Kapitel 12 entwickelt die Verfahren für den differenzierbaren Fall, vom Gradientenabstieg bis zu Newton und seinen Verwandten; die Subgradienten-Varianten selbst gehören nicht mehr zum Stoff dieses Skripts, brauchen aber genau das Werkzeug von hier.

Selbsttest

Sieben Aussagen zu diesem Abschnitt. Welche davon stimmen?

1. Sind f_1 und f_2 konvex, so ist auch die Differenz $f_1 - f_2$ konvex.

Antwort: Falsch

Satz 11.4.1(2) verlangt einen Faktor $c \geq 0$, und $-f_2$ verletzt das. Mit $f_1(x) = x^2$ und $f_2(x) = 2x^2$ ist die Differenz $-x^2$, also konkav. Je nach f_2 kann die Differenz konvex, konkav oder keines von beidem sein; für $f_2(x) = \frac{1}{2}x^2$ etwa bliebe sie konvex.

2. Das punktweise Maximum endlich vieler konvexer Funktionen ist konvex, auch wenn keine der beteiligten Funktionen differenzierbar ist.

Antwort: Wahr

Das ist Satz 11.4.1(3), und der Beweis in Beispiel 11.4.3 kommt ohne Ableitungen aus: Wir wählen an der gemischten Stelle einen maximierenden Index j , benutzen die Konvexität von f_j und schätzen rechts sofort wieder gegen das Maximum ab. Der Hinge-Verlust $\max\{0, 1 - u\}$ zeigt zugleich, dass dabei ein Knick entstehen darf: In $u = 1$ wechselt das Maximum von einem Stück zum anderen, und die Funktion bleibt trotzdem konvex.

3. Das punktweise Minimum zweier konvexer Funktionen ist konvex.

Antwort: Falsch

Für das Minimum gibt es keine Regel. Zu $f_1(x) = x^2$ und $f_2(x) = (x - 3)^2$ ist $f = \min\{f_1, f_2\}$ in $x = 0$ und $x = 3$ jeweils null, in der Mitte aber $f(1,5) = 2,25$. Der Mittelpunkt liegt also über der Sehne, und die Konvexitätsbedingung ist verletzt. Der Beweis aus Beispiel 11.4.3 bricht genau an der Stelle, an der wir nach oben abschätzen: Beim Minimum zeigt die entsprechende Ungleichung in die falsche Richtung.

4. Für $N = 2$ ist die Jensen-Ungleichung wörtlich die Definition einer konvexen Funktion.

Antwort: Wahr

Mit $\lambda = w_1$ und $1 - \lambda = w_2$ steht in (11.4.1) genau $f(\lambda \mathbf{x}_1 + (1 - \lambda) \mathbf{x}_2) \leq \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2)$. Der Beweis von Satz 11.4.6 nimmt das als Induktionsanfang und schiebt in jedem Schritt einen weiteren Punkt nach, wobei Satz 11.2.3 sicherstellt, dass die Zwischenpunkte in $\mathcal{X}$ bleiben.

5. Eine Funktion $f \in \mathcal{C}^2$ auf einer offenen, konvexen Menge ist genau dann konvex, wenn ihre Hesse-Matrix überall positiv definit ist.

Antwort: Falsch

Satz 11.4.9(3) verlangt positive *Semidefinitheit*. Jede affine Funktion hat $\mathbf{H}_f \equiv \mathbf{0}$ und ist konvex, und $f(x) = x^4$ ist sogar strikt konvex, obwohl $f''(0) = 0$ ist. Definitheit ist hinreichend für strikte Konvexität, aber nicht notwendig (Bemerkung 11.4.10).

6. In der Taylorentwicklung zweiter Ordnung von $f(\mathbf{x} + t\mathbf{h})$ steht der Krümmungsterm $t^2 \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$.

Antwort: Falsch

Der Faktor $\frac{1}{2}$ fehlt: Korollar 10.8.9 liefert $\frac{t^2}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}$. Nachprüfen lässt sich das an $f(x) = x^2$ mit $x = h = 1$ und $t = 0,1$: Der Krümmungsterm ist 0,01, die Fassung ohne den Faktor gäbe 0,02. Am Ergebnis $\mathbf{h}^\top \mathbf{H}_f \mathbf{h} \geq 0$ ändert der positive Faktor nichts.

7. Jede konvexe Funktion besitzt in jedem Punkt ihres Definitionsbereichs einen Subgradienten.

Antwort: Falsch

Im Inneren ja (Satz 11.4.15), am Rand nicht unbedingt. Auf $[0, \infty)$ ist $-\sqrt{x}$ konvex, und in $x = 0$ gibt es keinen Subgradienten: Für $v = -c$ mit $c > 0$ unterschreitet $f(y) - v y$ bei $y = 1/(4c^2)$ den Wert null, und $v \geq 0$ scheitert schon an $y = 1$ (Bemerkung 11.4.17).

8. Stellen wir im Jensen-Widget $f(x) = x^2$ und gleiche Gewichte ein. Wie groß ist die Differenz zwischen der grünen und der orangen Markierung?

Lösung: 1.5556

1,5556, nämlich $4,9167 - 3,3611$. Diese Lücke ist die gewichtete Varianz der drei Stützstellen 0,5, 1,5 und 3,5 um ihren Mittelwert $\frac{11}{6} = 1,8333$ (Beispiel 11.4.8). Sie verschwindet genau dann, wenn alles Gewicht auf einer Stützstelle liegt; für die konkave Wurzel wechselt sie das Vorzeichen.

Vertiefung: Boyd und Vandenberghe, *Convex Optimization*, §3.1 behandelt die Bedingungen erster und zweiter Ordnung sowie die Jensen-Ungleichung, §3.2 die Operationen, die Konvexität erhalten.

11.5 Konvexe Optimierung und Zusammenfassung

Warum konvexe Probleme leichter sind

Bis hierher haben wir Konvexität beschrieben. Jetzt ernten wir. Die Frage, um die es in der Optimierung wirklich geht, lautet nicht „ist die Funktion konvex“, sondern: Wenn ein Verfahren stehenbleibt, weil es keine Abstiegsrichtung mehr findet, haben wir dann das Beste gefunden oder nur ein Zwischenergebnis? Für allgemeine Funktionen ist die Antwort ernüchternd. Ein Gradient, der verschwindet, kann zu einem Minimum gehören, zu einem Maximum oder zu einem Sattelpunkt (Satz 10.7.9 in Abschnitt 10.7), und selbst ein echtes lokales Minimum kann weit über dem globalen liegen.

Für konvexe Funktionen bricht diese Unterscheidung zusammen. Das ist der eigentliche Grund, warum sich der Aufwand der letzten vier Abschnitte lohnt.

Satz 11.5.1 (Kritischer Punkt und globales Minimum)

Sei $f: \mathbb{R}^n \rightarrow \mathbb{R}$ differenzierbar und konvex, und sei $\mathbf{x}^* \in \mathbb{R}^n$. Dann sind die folgenden beiden Aussagen äquivalent:

1. $\mathbf{x}^*$ ist ein globales Minimum von f .
2. $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$.

Vertiefung: Beweis: Kritische Punkte konvexer Funktionen sind global

Beweis.

- (1) $(1) \Rightarrow (2)$. Sei $\mathbf{h} \in \mathbb{R}^n$ beliebig und $\varphi(t) := f(\mathbf{x}^* + t\mathbf{h})$ für $t \in \mathbb{R}$. Nach der Kettenregel ist φ differenzierbar mit $\varphi'(0) = \nabla f(\mathbf{x}^*)\mathbf{h}$. Weil $\mathbf{x}^*$ global minimiert, gilt $\varphi(t) \geq \varphi(0)$ für alle t , und damit

$$\frac{\varphi(t) - \varphi(0)}{t} \geq 0 \quad \text{für } t > 0, \quad \frac{\varphi(t) - \varphi(0)}{t} \leq 0 \quad \text{für } t < 0.$$

Beide Differenzenquotienten streben für $t \rightarrow 0$ gegen $\varphi'(0)$. Also ist $\varphi'(0)$ zugleich nicht negativ und nicht positiv, folglich $\nabla f(\mathbf{x}^*)\mathbf{h} = 0$. Mit $\mathbf{h} = \mathbf{e}_j$ verschwindet so jeder einzelne Eintrag des Gradienten.

in diesem Schritt geht die Konvexität nirgends ein; er gilt für jede differenzierbare Funktion an einer Minimalstelle im Inneren des Definitionsbereichs und heißt notwendige Bedingung erster Ordnung

- (2) $(2) \Rightarrow (1)$. Hier trägt die Konvexität. Nach Satz 11.4.9(2), der Tangentenbedingung aus Abschnitt 11.4, gilt für jedes $\mathbf{y} \in \mathbb{R}^n$

$$f(\mathbf{y}) \geq f(\mathbf{x}^*) + \underbrace{\nabla f(\mathbf{x}^*)(\mathbf{y} - \mathbf{x}^*)}_{= \mathbf{0}^\top} = f(\mathbf{x}^*).$$

Damit ist $f(\mathbf{x}^*)$ der kleinste Funktionswert überhaupt.

Satz 11.4.9 ist für $f \in \mathcal{C}^2$ formuliert; für die Äquivalenz seiner Aussagen (1) und (2) genügt nach Bemerkung 11.4.10 einfache Differenzierbarkeit, denn der zugehörige Beweisschritt rechnet nur mit Differenzenquotienten

□

Zwei Konsequenzen stecken in diesem Satz, und sie sind die wichtigsten des ganzen Kapitels. Wir gehen sie einzeln durch.

Bemerkung 11.5.2 (Was daraus folgt, und was nicht)

Jeder kritische Punkt ist ein globales Minimum. Das ist Schritt 2 des Beweises, wörtlich. Die vorsichtigeren Fassung „jeder kritische Punkt ist ein lokales Minimum“ bleibt unter dem, was der Satz hergibt.

Jedes lokale Minimum ist ein globales Minimum. Für differenzierbares f folgt das aus dem Satz, indem wir beide Schritte hintereinanderschalten: In einem lokalen Minimum verschwindet der Gradient nach Schritt 1, und Schritt 2 macht daraus ein globales. Bemerkenswerterweise brauchen wir dafür aber gar keine Ableitung. Sei $\mathcal{X} \subseteq \mathbb{R}^n$ konvex, $f: \mathcal{X} \rightarrow \mathbb{R}$ konvex und $\mathbf{x}^*$ ein lokales Minimum. Gäbe es ein $\mathbf{y} \in \mathcal{X}$ mit $f(\mathbf{y}) < f(\mathbf{x}^*)$, so läge für jedes $\lambda \in (0, 1]$ der Punkt $\mathbf{z}_\lambda = (1 - \lambda)\mathbf{x}^* + \lambda\mathbf{y}$ in $\mathcal{X}$, und die Konvexitätsungleichung (11.3.4) gäbe

$$f(\mathbf{z}_\lambda) \leq (1 - \lambda)f(\mathbf{x}^*) + \lambda f(\mathbf{y}) < f(\mathbf{x}^*).$$

Für kleines λ liegt $\mathbf{z}_\lambda$ aber in jeder noch so kleinen Umgebung von $\mathbf{x}^*$, im Widerspruch zur lokalen Minimalität. Nichtdifferenzierbare konvexe Funktionen wie der Betrag oder der LASSO-Strafterm sind also genauso gutmütig.

Es gibt keine Sattelpunkte, in denen ein Verfahren hängenbleiben könnte. An einem Sattelpunkt verschwindet der Gradient, und in jeder Umgebung lägen Punkte mit kleinerem Funktionswert. Nach Satz 11.5.1 ist ein solcher Punkt aber ein globales Minimum, und kleinere Werte gibt es dort nicht. Konvexe Landschaften haben nur Talböden.

Nicht folgt: Eindeutigkeit, und ebenso wenig Existenz. Eine konstante Funktion ist konvex und wird an jeder Stelle minimal – die erste der drei Landschaften weiter unten zeigt denselben Effekt mit einem flachen Stück; $f(x) = e^x$ ist konvex und nimmt ihr Infimum nie an. Bemerkung 11.5.6 sortiert die drei Aussagen. Der Merksatz „wenn wir ein konvexes Optimierungsproblem lösen, finden wir garantiert ein globales Optimum“ ist deshalb genau so zu lesen, wie er dasteht: *falls* wir eine Lösung finden, ist sie global.

Und ohne Ableitung? Ist f konvex, so tritt an die Stelle des Gradienten das Subdifferential aus Definition 11.4.14:

$$\mathbf{x}^* \text{ ist globales Minimum} \iff \mathbf{0} \in \partial f(\mathbf{x}^*).$$

Beide Richtungen stehen schon in der Definition des Subgradienten, und zwar an jeder Stelle des Definitionsbereichs, Randpunkte eingeschlossen. Ist $\mathbf{0} \in \partial f(\mathbf{x}^*)$, so lautet (11.4.3) mit $\mathbf{v} = \mathbf{0}$ gerade $f(\mathbf{y}) \geq f(\mathbf{x}^*)$ für alle $\mathbf{y}$. Und ist umgekehrt $\mathbf{x}^*$ ein globales Minimum, so erfüllt $\mathbf{v} = \mathbf{0}$ die Ungleichung, ist also ein Subgradient. Die Voraussetzung eines inneren Punktes brauchen wir an anderer Stelle: Sie sichert nach Satz 11.4.15, dass $\partial f(\mathbf{x}^*)$ überhaupt nicht leer ist. Für $f(x) = |x|$ gilt am Knick $\partial f(0) = [-1, 1]$; mit dem Kriterium $\mathbf{0} \in \partial f$ folgt daraus die Schwellenwertregel des LASSO (Herleitung in der Vertiefung Beispiel 11.4.16).

Wenn die Minimalstelle nicht eindeutig ist, ist wenigstens die Menge aller Minimalstellen handlich. Sie ist selbst wieder konvex, und das ist der Grund, warum in der Statistik von der *Lösungsmenge* eines konvexen Problems die Rede ist.

Satz 11.5.3 (Die Minimalstellen bilden eine konvexe Menge)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und $f : \mathcal{X} \rightarrow \mathbb{R}$ konvex. Dann ist

$$\arg \min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) := \{\mathbf{x} \in \mathcal{X} : f(\mathbf{x}) \leq f(\mathbf{y}) \text{ für alle } \mathbf{y} \in \mathcal{X}\}$$

eine konvexe Teilmenge von $\mathcal{X}$.

Vertiefung: Beweis: Die Minimalstellen bilden eine konvexe Menge

Beweis.

- (1) Ist die Menge leer oder einelementig, so ist sie konvex und wir sind fertig. Seien also $\mathbf{u}$ und $\mathbf{w}$ zwei Minimalstellen. Beide tragen denselben Wert: Aus der Minimalität von $\mathbf{u}$ folgt $f(\mathbf{u}) \leq f(\mathbf{w})$, aus der von $\mathbf{w}$ folgt $f(\mathbf{w}) \leq f(\mathbf{u})$. Wir setzen $m := f(\mathbf{u}) = f(\mathbf{w})$.

die leere Menge und einelementige Mengen sind konvex, weil die Bedingung aus Definition 11.2.1 dort nichts fordert oder trivial erfüllt ist

- (2) Sei $\lambda \in [0, 1]$ und $\mathbf{z} = \lambda \mathbf{u} + (1 - \lambda) \mathbf{w}$. Weil $\mathcal{X}$ konvex ist, liegt $\mathbf{z}$ in $\mathcal{X}$, und die Konvexität von f gibt

$$f(\mathbf{z}) \leq \lambda f(\mathbf{u}) + (1 - \lambda) f(\mathbf{w}) = m.$$

Andererseits ist m der kleinste Wert von f auf $\mathcal{X}$, also $f(\mathbf{z}) \geq m$. Damit ist $f(\mathbf{z}) = m$, und $\mathbf{z}$ ist selbst eine Minimalstelle.

hier werden beide Konvexitäten gebraucht: die der Menge, damit $\mathbf{z}$ überhaupt zulässig ist, und die der Funktion für die Abschätzung nach oben

□

Dieselbe Rechnung trägt mehr, als der Satz behauptet. Für jedes $c \in \mathbb{R}$ ist die *Subniveaumenge* $\{\mathbf{x} \in \mathcal{X} : f(\mathbf{x}) \leq c\}$ konvex, denn aus $f(\mathbf{u}) \leq c$ und $f(\mathbf{w}) \leq c$ folgt $f(\mathbf{z}) \leq \lambda c + (1 - \lambda)c = c$. Die Minimalstellen sind davon der Sonderfall $c = \min f$, sofern dieses Minimum angenommen wird. Konvexe Funktionen haben also konvexe *Subniveaumengen*, und daran hängt in Abschnitt 12.5 die Beschreibung zulässiger Bereiche. Für exakte Niveaumengen $\{\mathbf{x} : f(\mathbf{x}) = c\}$ gilt das nicht: Das Niveau 1 von $f(x) = x^2$ besteht aus $\{-1, 1\}$ und ist nicht konvex.

Strikte Konvexität und Eindeutigkeit

Eindeutigkeit fehlt in Satz 11.5.1 aus einem einfachen Grund: Die Konvexitätsungleichung erlaubt Gleichheit, und Gleichheit heißt geometrisch, dass der Graph auf einem Stück gerade verläuft. Verboten wir das, so bekommen wir die Eindeutigkeit geschenkt.

Definition 11.5.4 (Strikte Konvexität)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und $f : \mathcal{X} \rightarrow \mathbb{R}$. Wir nennen f *strikt konvex* (strictly convex), falls für alle $\mathbf{x}, \mathbf{y} \in \mathcal{X}$ mit $\mathbf{x} \neq \mathbf{y}$ und alle $\lambda \in (0, 1)$ gilt

$$f(\lambda \mathbf{x} + (1 - \lambda) \mathbf{y}) < \lambda f(\mathbf{x}) + (1 - \lambda) f(\mathbf{y}). \quad (11.5.1)$$

Gegenüber der Sehnenungleichung (11.3.4) aus Satz 11.3.8 haben sich drei Dinge geändert, und alle drei sind nötig. Aus $\leq$ wird $<$, das ist der Punkt. Dafür müssen $\mathbf{x}$ und $\mathbf{y}$ verschieden sein und λ die Randwerte 0 und 1 auslassen, denn sonst stünde links und rechts ohnehin derselbe Wert und keine einzige Funktion wäre strikt konvex. Eingeführt ist der Begriff samt seiner Zweitbezeichnung „streng konvex“ schon in Bemerkung 11.3.9; wir bleiben bei strikt.

Satz 11.5.5 (Höchstens eine Minimalstelle)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und $f : \mathcal{X} \rightarrow \mathbb{R}$ strikt konvex. Dann besitzt f höchstens eine Minimalstelle.

Vertiefung: Beweis der Eindeutigkeit bei strikter Konvexität

Beweis.

- (1) Angenommen, es gäbe zwei verschiedene Minimalstellen $\mathbf{x}_1 \neq \mathbf{x}_2$. Wie in Schritt 1 des Beweises zu Satz 11.5.3 tragen beide denselben Wert $m := f(\mathbf{x}_1) = f(\mathbf{x}_2)$.

jede der beiden Stellen ist minimal, also höchstens so groß wie die andere

- (2) Wir betrachten den Mittelpunkt $\mathbf{z} = \frac{1}{2}\mathbf{x}_1 + \frac{1}{2}\mathbf{x}_2$. Er liegt in $\mathcal{X}$, weil $\mathcal{X}$ konvex ist, und Definition 11.5.4 mit $\lambda = \frac{1}{2}$ liefert

$$f(\mathbf{z}) < \frac{1}{2}f(\mathbf{x}_1) + \frac{1}{2}f(\mathbf{x}_2) = m.$$

Damit gäbe es einen zulässigen Punkt mit einem kleineren Wert als dem Minimum, ein Widerspruch. Also kann es keine zwei verschiedenen Minimalstellen geben.

hier wird jede Zutat von Definition 11.5.4 gebraucht: $\mathbf{x}_1 \neq \mathbf{x}_2$ und $\frac{1}{2} \in (0, 1)$ machen die strikte Ungleichung anwendbar, und die Konvexität von $\mathcal{X}$ sichert die Zulässigkeit des Mittelpunkts

□

Der Beweis ist kurz, und er zeigt, wo die Grenze verläuft: Genommen wird nur ein einziges λ , nämlich $\frac{1}{2}$. Dasselbe Argument lässt sich auch aus Satz 11.5.3 ablesen. Die Minimalstellen bilden eine konvexe Menge; enthielte sie zwei verschiedene Punkte, so enthielte sie die ganze Strecke dazwischen, und auf deren Innerem ist f nach (11.5.1) echt kleiner als an den Endpunkten. Eine konvexe Funktion, die auf einer Strecke konstant ist, kann also nicht strikt konvex sein.

Bemerkung 11.5.6 (Drei Aussagen, die auseinanderzuhalten sind)

Damit sind die drei Versprechen sortiert, die in der Einleitung von Abschnitt 11.1 noch zusammen aufgetreten sind.

1. *Lokal ist global.* Das leistet schon gewöhnliche Konvexität, ohne jede Differenzierbarkeit (Satz 11.5.1, Bemerkung 11.5.2).
2. *Höchstens eine Lösung.* Dafür braucht es strikte Konvexität (Satz 11.5.5).
3. *Mindestens eine Lösung.* Das folgt aus keinem von beiden.

Punkt 3 wird am leichtesten übersehen. Die Exponentialfunktion ist auf $\mathbb{R}$ sogar strikt konvex, denn $f''(x) = e^x > 0$ überall, und trotzdem gibt es kein Minimum. Für die Existenz sorgen andere Argumente: eine kompakte zulässige Menge und Stetigkeit nach dem Satz von Weierstraß, wie in Satz 11.3.1, oder Koerzivität, also $f(\mathbf{x}) \rightarrow \infty$ für $\|\mathbf{x}\| \rightarrow \infty$. Im zweiten Fall genügt es, auf einer hinreichend großen abgeschlossenen Kugel zu minimieren, und wir sind wieder im ersten.

Erst beides zusammen, strikte Konvexität und Existenz, gibt die *genau eine* Lösung. Das Kapitel hat davon schon zwei Beispiele geliefert. Der Kleinste-Quadrate-Verlust ist bei vollem Spaltenrang der Designmatrix strikt konvex und koerziv, das Minimum existiert und ist eindeutig (Beispiel 11.3.16, Kapitel 7 rechnet es aus). Und die Ridge-Regression ist es für jedes $\lambda > 0$ auch dann, wenn die Designmatrix rangdefizient ist.

Ein statistisches Gegenbeispiel zeigt, dass Punkt 3 keine Spitzfindigkeit ist. Bei perfekt trennbaren Daten bleibt die negative Log-Likelihood der logistischen Regression strikt konvex, angenommen wird ihr Infimum aber nicht: Die Koeffizienten laufen ins Unendliche, während der Wert gegen 0 fällt. Schon mit den beiden Beobachtungen $(x, y) = (1, 1)$ und $(x, y) = (-1, 0)$ und dem Modell $\pi(\beta) = 1/(1 + e^{-\beta x})$ ist $\ell(\beta) = \log(1 + e^\beta) - \beta + \log(1 + e^{-\beta})$, und dieser Wert fällt von 1,386 bei $\beta = 0$ auf $9,1 \cdot 10^{-5}$ bei $\beta = 10$, bleibt aber für jedes endliche β positiv. Ein Ridge-Strafterm repariert hier nicht die Eindeutigkeit, die schon da ist, sondern die Existenz: Der Zusatz $\lambda\beta^2$ ist koerziv, und für $\lambda = 0,2$ liegt das Minimum bei $\beta = 1,18$.

Optimierungslandschaften

Der Unterschied lässt sich als Bildtafel mit drei Flächen nebeneinander zeigen. Wir bauen ihn in einer Variablen nach, weil sich dort alles einzeichnen lässt, worauf es ankommt: erst die drei Fälle als Tafel, danach ein Verfahren, das in einem von ihnen scheitert.

Interaktiv: Drei Landschaften nebeneinander

Woran sehen wir einer Landschaft an, welcher der drei Fälle vorliegt? Die Antwort steckt nicht im Verlauf des Graphen, sondern in der Menge der Minimalstellen.

Interaktives Element — nur in der Web-Fassung

Die mittlere Tafel ist der Fall, den Satz 11.5.5 abdeckt: strikt konvex, also höchstens eine Minimalstelle, und weil die Parabel koerziv ist, wird sie auch angenommen. Links steht eine konvexe Funktion mit einem flachen Stück. Sie erfüllt (11.3.4) mit $\leq$, aber auf dem Plateau steht in (11.5.1) Gleichheit, sie ist also nicht strikt konvex. Ihre Minimalstellen bilden das ganze Intervall von $-0,8$ bis $0,8$, und das ist eine konvexe Menge, wie Satz 11.5.3 es verlangt. Rechts ist die Konvexität verletzt, und damit fällt die ganze Theorie dieses Abschnitts weg: Zwei kritische Punkte tragen dort positive Krümmung, nur einer von beiden ist global.

Vertiefung: Ein Abstiegsverfahren auf der Doppelmulde

Wie sich der Unterschied auf ein Verfahren auswirkt, gehört eigentlich schon zu Kapitel 12. Ein Blick voraus lohnt sich trotzdem: Wir lassen den einfachsten Vertreter laufen, den Gradientenabstieg aus Abschnitt 10.2, der wiederholt ein Stück in Richtung des stärksten Abstiegs geht. Auf der Doppelmulde entscheidet allein der Startwert, in welchem Tal er endet.

Interaktiv: Ein Abstieg auf der Doppelmulde

Der Startwert lässt sich Schritt für Schritt verschieben; die Tafel zeigt jedes Mal die ausgewählten Zwischenstationen und den Punkt, an dem die Folge zur Ruhe kommt. Wo genau verläuft die Grenze zwischen den beiden Einzugsgebieten?

Interaktives Element — nur in der Web-Fassung

Wie das Widget zeigt, trennt der Höcker bei $-0,1699$ die beiden Einzugsbereiche. Am Endpunkt selbst ist der Unterschied nicht abzulesen: In beiden Tälern verschwindet die Ableitung, in beiden ist die Krümmung positiv, in beiden meldet jedes Abbruchkriterium Erfolg. Wer ein nicht-konvexes Problem rechnet, muss deshalb mehrere Startwerte durchprobieren; bei einer konvexen Zielfunktion erübrigt sich das, denn nach Satz 11.5.1 ist jeder Punkt mit verschwindendem Gradienten schon das Ziel.

Konvexe und nicht-konvexe Probleme in Statistik und maschinellem Lernen

Zum Abschluss eine Landkarte. Sie sortiert die gängigen Verfahren nach der Frage, die dieser Abschnitt beantwortet hat, und ist beim Lesen von Software-Dokumentation erstaunlich nützlich: Sie sagt voraus, ob ein Startwert das Ergebnis beeinflussen kann.

Bemerkung 11.5.7 (Eine Landkarte der Optimierungsprobleme)

Strikt konvexe Probleme haben höchstens eine Lösung (Satz 11.5.5), und wo sie existiert, ist sie eindeutig und global. Jeder korrekt konvergierte Lauf muss daher dieselbe Lösung liefern; Rechenweg, Laufzeit und Abbruchstatus können trotzdem vom Startwert abhängen.

- Kleinste Quadrate, $\min_{\beta} \|y - X\beta\|_2^2$, sofern X vollen *Spaltenrang* hat (Beispiel 11.3.16). „Voller Rang“ allein genügt nicht; entscheidend ist der Spaltenrang, denn nur er macht $X^T X$ positiv definit.
- Allgemeiner die Maximum-Likelihood-Schätzung in verallgemeinerten linearen Modellen mit kanonischem Link. Dort hat die negative Log-Likelihood die Hesse-Matrix $X^T W X$ mit einer nichtnegativen Diagonalmatrix W , ist also stets konvex, und bei vollem Spaltenrang und positiven Gewichten strikt konvex (Beispiel 11.4.12 rechnet den logistischen Fall vor).
- Support Vector Machines. Die Zielfunktion $\frac{1}{2} \|w\|_2^2 + C \sum_i \max\{0, 1 - y_i(w^T x_i + b)\}$ ist als Summe konvexer Funktionen konvex (Satz 11.4.1), und im Argument w ist der erste Summand strikt konvex; damit ist der Trennvektor w nach Satz 11.5.5 eindeutig. Im Paar (w, b) ist die Zielfunktion dagegen nicht strikt konvex, denn der Achsenabschnitt b geht nur über den stückweise linearen Hinge-Verlust ein und kann bei trennbaren Daten mit Sicherheitsabstand mehrdeutig bleiben.
- Ridge-Regression, $\min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2$, für jedes $\lambda > 0$ und jedes X (Beispiel 11.3.16), ebenso das *Elastic Net*, solange sein Ridge-Anteil positiv ist.
- Graphisches LASSO, also die regularisierte Schätzung einer Präzisionsmatrix. Optimierte wird über die positiv definiten Matrizen. Sie bilden nach Bemerkung 11.2.9 das Innere des Kegels aus Satz 11.2.8 *innerhalb des Raums der symmetrischen Matrizen*, und der Term $-\log \det$ ist darauf strikt konvex.

Konvexe, aber nicht strikt konvexe Probleme haben nach Satz 11.5.3 eine konvexe Lösungsmenge, die mehr als einen Punkt enthalten darf. Jede gefundene Lösung ist global, aber verschiedene Implementierungen dürfen verschiedene Koeffizienten melden.

- Das LASSO, $\min_{\beta} \|y - X\beta\|_2^2 + \lambda \|\beta\|_1$. Der Strafterm ist konvex, aber stückweise linear und damit nicht strikt; bei rangdefizienter Designmatrix, dem Regelfall für $p > n$, überträgt sich das auf die Zielfunktion.
- Quantilregression. Die Verlustfunktion $\rho_{\tau}(u) = u(\tau - \mathbb{1}\{u < 0\})$ ist konvex, besteht aber aus zwei Geradenstücken und ist auf keinem Intervall strikt konvex. Optimierte wird deshalb mit Subgradienten oder als lineares Programm.
- Nicht-kanonische Links in verallgemeinerten linearen Modellen. Hier entfällt die Garantie: Der Term, der beim kanonischen Link die Form $X^T W X$ erzwingt, fällt weg. Konvex bleibt die negative Log-Likelihood trotzdem oft, etwa im Probit-Modell, weil die Verteilungsfunktion der Normalverteilung log-konkav ist. Automatisch ist es eben nicht mehr.

Nicht-konvexe Probleme können mehrere lokale Minima und Sattelpunkte haben. Verschiedene Startwerte können dann in verschiedene Einzugsgebiete führen, und ein bloß stationäres Ergebnis trägt kein globales Gütesiegel.

- Neuronale Netze. Schon das Vertauschen zweier Neuronen einer Schicht liefert ein anderes Parameterset mit demselben Verlust, das Problem ist also nicht einmal identifizierbar.

- Clusteranalyse: k -means und Verwandte, Mischverteilungsmodelle. Beide Verfahren werden in der Praxis mehrfach neu gestartet, und das beste Ergebnis gewinnt.
- Bäume und darauf aufbauende Verfahren, CART, Random Forests, XGBoost. Hier ist der Suchraum überhaupt nicht kontinuierlich, optimiert wird gierig Schnitt für Schnitt.
- Modelle mit latenten Variablen: gemischte Modelle, Strukturgleichungsmodelle, Faktorenanalyse.
- Variablenselektion mit L_0 -Strafe, *Best Subset*, *Spike and Slab*. Das Zählen der Nicht-Null-Koeffizienten ist keine Norm und nicht konvex, das Problem ist kombinatorisch.
- Dimensionsreduktion und Einbettungen: tSNE, UMAP, word2vec.
- Matrixvervollständigung und Tensorzerlegungen: nichtnegative Matrixfaktorisierung, dünn besetzte Hauptkomponenten. Die Zielfunktion aus Abschnitt 10.4 ist in U und in V jeweils konvex, in beiden zusammen aber nicht.
- Graphische Modelle, wenn nicht nur die Gewichte, sondern die Struktur selbst gesucht wird.

Die Wahl von Hyperparametern gehört nur mit Vorbehalt in dieselbe Liste: Dort ist die Zielfunktion, meist ein Kreuzvalidierungsfehler, oft nicht einmal stetig. Von Sattelpunkten lässt sich da gar nicht sprechen, und deshalb kommen Gittersuche, Zufallssuche und Bayessche Optimierung zum Einsatz statt Abstiegsverfahren.

Die Kernkonzepte des Kapitels

Bemerkung 11.5.8 (Fünf Bausteine, die bleiben)

1. **Konvexkombinationen** (Abschnitt 11.1): Gewichte $w_i \geq 0$ mit Summe 1 (Definition 11.1.1), für zwei Punkte die Verbindungsstrecke (Satz 11.1.4), für endlich viele die konvexe Hülle $\text{conv}(\mathcal{X})$ (Definition 11.1.5); der Erwartungswert ist der Prototyp (Beispiel 11.1.3).
2. **Konvexe Mengen** (Abschnitt 11.2): mit je zwei Punkten die ganze Strecke (Definition 11.2.1), per Induktion alle endlichen Konvexkombinationen (Satz 11.2.3), stabil unter Durchschnitt, kartesischem Produkt, affinem Bild und Summe (Satz 11.2.10), und $\text{conv}(\mathcal{X})$ als kleinste konvexe Obermenge (Satz 11.2.13); wichtigstes Beispiel für die Statistik ist der Kegel der positiv semidefiniten Matrizen (Satz 11.2.8).
3. **Konvexe Funktionen** (Abschnitt 11.3): konvexer Epigraph (Definition 11.3.6), gleichwertig die Sehnenungleichung (11.3.4) (Satz 11.3.8); konvex sind affine und quadratische Funktionen sowie alle Normen (Beispiel 11.3.11, Satz 11.3.12, Satz 11.3.15); dazu das Projektionstheorem (Satz 11.3.1) als Brücke zur Geometrie der kleinsten Quadrate.
4. **Eigenschaften konvexer Funktionen** (Abschnitt 11.4): der Baukasten aus Summe, nichtnegativem Vielfachem, Supremum und Grenzwert (Satz 11.4.1), die Jensen-Ungleichung (Satz 11.4.6), die Äquivalenz von Sehnen-, Tangenten- und Krümmungsbedingung (Satz 11.4.9) und der Subgradient dort, wo die Tangente fehlt (Definition 11.4.14, Satz 11.4.15).
5. **Konvexe Optimierungsprobleme** (Abschnitt 11.5): kritischer Punkt und globales Minimum fallen zusammen, lokale Minima sind global, Sattelpunkte gibt es nicht (Satz 11.5.1, Bemerkung 11.5.2); die Lösungsmenge ist konvex (Satz 11.5.3), strikte Konvexität macht sie höchstens einelementig (Definition 11.5.4, Satz 11.5.5), und die Existenz

ist davon unabhängig (Bemerkung 11.5.6).

Das Wichtigste in Kürze

Abschnitt	Thema	Was bleibt
11.1	Konvexkombinationen	Gewichte $w_i \geq 0$ mit $\sum_i w_i = 1$; zwei Punkte spannen eine Strecke auf, endlich viele die konvexe Hülle $\text{conv}(\mathcal{X})$
11.2	Konvexe Mengen	Mit je zwei Punkten die ganze Strecke; stabil unter Durchschnitt, kartesischem Produkt, affinen Abbildungen und Summe; der Kegel $\{A = A^T : A \succeq 0\}$ als Beispiel
11.3	Konvexe Funktionen	Konvexer Epigraph, gleichbedeutend mit $f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$; die Projektion auf eine nichtleere abgeschlossene konvexe Menge existiert und ist eindeutig
11.4	Eigenschaften	Baukasten aus Summe, nichtnegativem Vielfachem, Supremum und Grenzwert; Jensen; für $f \in \mathcal{C}^2$ auf offenem, konvexem Bereich ist Konvexität gleichbedeutend mit $H_f(x) \geq 0$ überall; Subgradienten im Knick
11.5	Konvexe Optimierung	Für differenzierbares konvexes f bedeutet $\nabla f(x^*) = 0^T$ globales Minimum; lokal ist global, keine Sattelpunkte; konvexe Lösungsmenge; strikt konvex bedeutet höchstens eine Lösung, Existenz ist eine eigene Frage

Ein Satz fasst das Kapitel zusammen. Konvexität kostet uns die Gleichheit der Linearität und gibt uns dafür eine globale Aussage, die keine lokale Rechnung je liefern könnte.

Nächstes Kapitel

Wir wissen jetzt, wann ein lokaler Befund ein globaler ist. Offen ist die andere Hälfte: wie wir überhaupt dorthin kommen. Davon handelt Kapitel 12 – von Abstiegsverfahren und ihren Schrittweiten, von Newton-Verfahren, die die Krümmung mitnehmen, und von Nebenbedingungen. Die Ergebnisse dieses Kapitels sind dort die Eintrittskarte für alle Garantien, die über „das Verfahren bleibt irgendwo stehen“ hinausgehen.

Selbsttest

Acht Aussagen aus dem ganzen Kapitel. Welche davon stimmen?

1. Die konvexe Hülle der Einheitssphäre $\{x \in \mathbb{R}^n : \|x\|_2 = 1\}$ ist wieder die Einheitssphäre.

Antwort: Falsch

Sie ist die abgeschlossene Einheitskugel. Schon der Mittelpunkt von e_1 und e_2 liegt in der Hülle und hat die Norm $1/\sqrt{2} = 0,7071$, gehört also nicht zur Sphäre. Umgekehrt lässt sich jeder Punkt der Kugel als Konvexkombination zweier Sphärenpunkte schreiben: Wir legen eine Gerade durch ihn und nehmen die beiden Schnittpunkte mit der Sphäre. Für $x = (0,3; -0,2)^T$ und die waagerechte Richtung sind das $(0,9798; -0,2)^T$ und $(-0,9798; -0,2)^T$ mit den Gewichten 0,6531 und 0,3469. Die Sphäre selbst ist nicht konvex (Selbsttest in Abschnitt 11.2), ihre Hülle ist es nach Satz 11.2.13.

2. Ist $f : \mathbb{R}^n \rightarrow \mathbb{R}$ differenzierbar und konvex und hat f mehrere kritische Punkte, so haben sie alle denselben Funktionswert.

Antwort: Wahr

Nach Satz 11.5.1 ist jeder kritische Punkt ein globales Minimum, und globale Minima tragen alle den kleinsten Wert. Die erste Landschaft im Widget zeigt genau das: Auf dem Plateau von $-0,8$ bis $0,8$ verschwindet die Ableitung überall, und überall steht der Wert 0. Nach Satz 11.5.3 bilden diese Stellen sogar eine konvexe Menge, hier ein Intervall.

3. Jede strikt konvexe Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ besitzt genau ein globales Minimum.

Antwort: Falsch

Sie besitzt *höchstens* eines (Satz 11.5.5). Die Existenz ist eine unabhängige Frage: $f(x) = e^x$ ist wegen $f''(x) = e^x > 0$ strikt konvex, nähert sich ihrem Infimum 0 aber nur an. Statistisch relevant ist derselbe Effekt bei perfekt trennbaren Daten in der logistischen Regression, wo die Koeffizienten ins Unendliche laufen (Bemerkung 11.5.6). Gesichert wird die Existenz erst durch Kompaktheit oder Koerzitivität.

4. Ist f_1 konvex und f_2 strikt konvex, so ist $f_1 + f_2$ strikt konvex.

Antwort: Wahr

Wir addieren die beiden Ungleichungen (11.3.4) und (11.5.1) für $\mathbf{x} \neq \mathbf{y}$ und $\lambda \in (0, 1)$: Links steht $\leq$, rechts $<$, in der Summe also $<$. Davon lebt die Ridge-Regression. Der Kleinste-Quadrate-Term ist immer konvex und bei rangdefizienter Designmatrix nur nicht strikt konvex; der Strafterm $\lambda \|\boldsymbol{\beta}\|_2^2$ ist für $\lambda > 0$ strikt konvex, und die Summe erbt die strikte Konvexität (Beispiel 11.3.16, Satz 11.4.1). Ohne den strikten Summanden geht es nicht: $|x|$ und $\max\{0, x\}$ sind beide konvex, ihre Summe stimmt auf $(-\infty, 0]$ aber mit $|x| = -x$ überein und verläuft dort geradlinig.

5. Aus der Jensen-Ungleichung folgt für eine Zufallsvariable X mit endlichem Träger sowohl $\mathbb{E}[X]^2 \leq \mathbb{E}[X^2]$ als auch $\mathbb{E}[\sqrt{X}] \leq \sqrt{\mathbb{E}[X]}$, sofern $X \geq 0$ ist.

Antwort: Wahr

Beides ist derselbe Satz 11.4.6, einmal mit der konvexen Funktion $x \mapsto x^2$ und einmal mit der konkaven Wurzel, bei der sich das Ungleichheitszeichen umdreht (Bemerkung 11.3.9). Am fairen Würfel gerechnet: $\mathbb{E}[X] = 3,5$ und $\mathbb{E}[X^2] = 15,1667$ gegen $\mathbb{E}[X]^2 = 12,25$, die Differenz 2,9167 ist die Varianz (Beispiel 11.4.8). Und $\mathbb{E}[\sqrt{X}] = 1,8053$ liegt unter $\sqrt{3,5} = 1,8708$.

6. Die quadratische Funktion $f(\mathbf{x}) = \mathbf{x}^\top \mathbf{Q} \mathbf{x}$ ist genau dann konvex, wenn $\mathbf{Q}$ positiv semidefinit ist.

Antwort: Falsch

Die quadratische Form sieht von $\mathbf{Q}$ nur den symmetrischen Anteil $(\mathbf{Q} + \mathbf{Q}^\top)/2$, und nach Satz 11.3.12 entscheidet allein dieser. Für $\mathbf{Q} = \begin{pmatrix} 1 & 5 \\ -5 & 1 \end{pmatrix}$ ist $\mathbf{x}^\top \mathbf{Q} \mathbf{x} = x_1^2 + x_2^2$, also sogar strikt konvex, während $\mathbf{Q}$ nicht einmal symmetrisch ist und damit nach Definition 11.2.7 gar nicht positiv semidefinit heißen kann. An der Stelle $\mathbf{x} = (2, 3)^\top$ steht auf beiden Seiten 13. Die Bedingung an $\mathbf{Q}$ selbst ist also zu eng (Bemerkung 11.3.13).

7. Im Knick $x = 0$ hat $f(x) = |x|$ keinen Subgradienten, weil dort die Ableitung fehlt.

Antwort: Falsch

Der Subgradient braucht keine Ableitung, das ist gerade sein Zweck. Bedingung (11.4.3) verlangt in $x = 0$ nur $|y| \geq v y$ für alle y , und das erfüllt jedes $v \in [-1, 1]$. Das Subdifferential ist also ein ganzes Intervall, $\partial f(0) = [-1, 1]$, und weil 0 darin liegt, ist 0 nach Bemerkung 11.5.2 die globale Minimalstelle (Beispiel 11.4.16). Umgekehrt wird ein Rand gefährlich: Auf $[0, \infty)$ hat das konvexe $-\sqrt{x}$ in 0 wirklich keinen Subgradienten (Bemerkung 11.4.17).

8. Erfüllen zwei verschiedene Koeffizientenvektoren eines LASSO-Problems beide die Subgradienten-Optimalitätsbedingung, so sind beide global optimal.

Antwort: Wahr

Die LASSO-Zielfunktion ist als Summe zweier konvexer Funktionen konvex (Satz 11.4.1), also ist jeder Punkt mit $\mathbf{0}$ im Subdifferential global optimal (Bemerkung 11.5.2). Strikt konvex ist sie bei rangdefizienter Designmatrix nicht, die Lösungsmenge kann deshalb mehr als einen Punkt enthalten. Nach Satz 11.5.3 ist sie konvex, also liegt sogar die ganze Verbindungsstrecke zwischen den beiden Lösungen in der Lösungsmenge, und alle Punkte darauf haben denselben Zielwert. Zwei bloße Softwareausgaben wären dagegen noch kein Beweis: Zuerst sind Konvergenzstatus und Optimalitätsresiduum zu prüfen. Bei nicht-konvexen Verfahren wie k -means wäre selbst erfüllte lokale Optimalität kein globales Gütesiegel (Bemerkung 11.5.7).

9. Im Abstiegs-Widget landet ein Lauf, der bei $x_0 = -0,16$ startet, im tiefen Tal rechts.

Antwort: Wahr

Er landet dort, obwohl er weit links vom tiefen Tal beginnt. Die Wasserscheide ist der Höcker bei $-0,1699$, und $-0,16$ liegt knapp rechts davon; der negative Gradient zeigt dort nach rechts, und die Folge rollt über den ganzen Höcker hinweg bis $1,30084$. Ein Start bei $-0,17$ endet dagegen bei $-1,13090$. Ein Reglerschritt von $0,01$ entscheidet also über einen Zielwert, der um $2,44$ auseinanderliegt.

Vertiefung: Boyd und Vandenberghe, Convex Optimization, behandeln in Kapitel 4 die konvexen Optimierungsprobleme selbst, darunter die Aussage, dass jedes lokale Minimum global ist, und das Optimalitätskriterium für differenzierbare Zielfunktionen. Das Buch ist frei verfügbar.

Nichtlineare Gleichungen & Optimierung

12.1 Nichtlineare Gleichungen

Was wir mitbringen

Der zweite Block des Skripts, *Analysis und Optimierung*, läuft auf dieses Kapitel zu. Alles, was wir dafür brauchen, steht schon da, und es lohnt sich, es kurz nebeneinanderzulegen.

Aus Abschnitt 10.2 kommt der Gradient $\nabla f(\mathbf{x})$ einer Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Wo er nicht verschwindet, zeigt er in die Richtung des steilsten Anstiegs, genauer gesagt tut das sein Transponiertes, denn in der Konvention dieses Skripts ist der Gradient ein Zeilenvektor, eine Richtung im $\mathbb{R}^n$ dagegen eine Spalte. Die Voraussetzung $\nabla f(\mathbf{x}) \neq \mathbf{0}^\top$ von Satz 10.2.4 ist hier wichtig: In einem kritischen Punkt gibt es keine ausgezeichnete Anstiegsrichtung mehr, und kritische Punkte sind im Rest des Kapitels unser Ziel. Aus Abschnitt 10.7 kommt die Hesse-Matrix $\mathbf{H}_f(\mathbf{x})$ der zweiten Ableitungen, die die Krümmung misst. Und aus Kapitel 11 kommt die Konvexität von Mengen und Funktionen, die später darüber entscheidet, ob ein gefundenes Minimum auch das globale ist.

Dazu zwei Bausteine aus der Grundvorlesung. *Notwendig* für ein Extremum einer differenzierbaren Funktion in einem inneren Punkt des Definitionsbereichs ist $f'(\mathbf{x}^*) = 0$ beziehungsweise $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$. *Hinreichend* für ein Minimum ist das zusammen mit $f''(\mathbf{x}^*) > 0$, im Mehrdimensionalen mit einer positiv definiten Hesse-Matrix. Beide Teile werden gebraucht: Ohne die notwendige Bedingung daneben sagt die Krümmungsbedingung nichts, denn eine positive Krümmung allein macht aus einem beliebigen Punkt kein Minimum.

Der Farbcode dieses Kapitels: blau die Iterierten $\mathbf{x}^{(k)}$ und die Wege, die sie zurücklegen, grün das Ziel $\mathbf{x}^*$, orange die Ableitungsobjekte und die aus ihnen gebauten Schritte, rot die Nebenbedingungen und die Warnzeichen für Divergenz.

Drei Sorten von Problemen

Der Rest des Kapitels behandelt drei Fragestellungen, die enger zusammengehören, als sie zunächst aussehen.

Definition 12.1.1 (Nichtlineares Gleichungssystem)

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$. Ein *nichtlineares Gleichungssystem* ist die Aufgabe, ein $\mathbf{x}^* \in \mathbb{R}^n$ zu finden mit

$$f(\mathbf{x}^*) = \mathbf{0}.$$

Für $n = m = 1$ sprechen wir von einer *Nullstelle* (root) von f .

Der Zusatz „nichtlinear“ grenzt gegen Kapitel 5 ab: Ist f affin, also $f(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$, so ist das ein lineares Gleichungssystem, und dafür haben wir fertige Zerlegungen. Sobald f krumm ist, gibt es keine Formel mehr, die uns die Lösung in endlich vielen Schritten hinstellt.

Definition 12.1.2 (Unbeschränktes und beschränktes Optimierungsproblem)

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Ein *unbeschränktes Optimierungsproblem* (unconstrained optimization) ist die Aufgabe

$$\mathbf{x}^* \in \arg \min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}) \quad \text{beziehungsweise} \quad \mathbf{x}^* \in \arg \max_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}).$$

Ein *beschränktes Optimierungsproblem* (constrained optimization) schränkt die Suche auf eine Menge $S \subseteq \mathbb{R}^n$ ein,

$$\mathbf{x}^* \in \arg \min_{\mathbf{x} \in S} f(\mathbf{x}),$$

wobei die *Nebenbedingungen* typischerweise selbst durch Funktionen beschrieben werden:

$$S = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{g}_i(\mathbf{x}) = 0, i = 1, \dots, m\} \quad \text{oder} \quad S = \{\mathbf{x} \in \mathbb{R}^n : \mathbf{h}_j(\mathbf{x}) \leq 0, j = 1, \dots, p\}$$

oder eine Kombination aus beidem. Die Funktion f heißt Zielfunktion.

„Beschränkt“ bedeutet hier *mit Nebenbedingungen* und nicht, dass S eine beschränkte Menge sein muss. Eindeutiger ist deshalb auch die Bezeichnung *Optimierung mit Nebenbedingungen*.

Bemerkung 12.1.3 (Zwei Probleme, ein Werkzeugkasten)

Maximieren und Minimieren sind dieselbe Aufgabe, denn $\arg \max_{\mathbf{x}} f(\mathbf{x}) = \arg \min_{\mathbf{x}} (-f(\mathbf{x}))$. Wir reden deshalb ab jetzt nur noch über Minima.

Wichtiger ist der zweite Zusammenhang. Ist f differenzierbar, so muss im Minimum eines *unbeschränkten* Problems $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ gelten; welche Bedingung an die Stelle tritt, sobald Nebenbedingungen im Spiel sind, klärt Abschnitt 12.5. Das ist ein nichtlineares Gleichungssystem für die Funktion $\mathbf{x} \mapsto \nabla f(\mathbf{x})^\top$ von $\mathbb{R}^n$ nach $\mathbb{R}^n$. Wer Nullstellen suchen kann, kann also auch kritische Punkte suchen, und umgekehrt taucht jedes Verfahren dieses Abschnitts im Rest des Kapitels wieder auf. Deshalb steht die Nullstellensuche am Anfang.

Alle drei Probleme lösen wir *iterativ*: Wir starten irgendwo und bauen eine Folge $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$, die hoffentlich gegen $\mathbf{x}^*$ läuft. Dieselbe Bauform kennen wir schon von der Potenzmethode und den Splitting-Verfahren aus Kapitel 8.

Warum Optimierung?

Beispiel 12.1.4 (Optimierungsprobleme in Statistik und maschinellem Lernen)

Ein erstaunlich großer Teil beider Fächer besteht daraus, ein Minimum zu suchen.

Maximum-Likelihood-Schätzung. Für unabhängige, identisch verteilte Beobachtungen $y_1, \dots, y_n$ mit Dichte $f_Y(y | \theta)$ ist der ML-Schätzer

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^n \log f_Y(y_i | \theta).$$

Nur in Lehrbuchfällen wie der Normalverteilung lässt sich das nach θ auflösen; sonst bleibt die Iteration.

Überwachtes Lernen. Mit einer Verlustfunktion $L(y, \hat{y})$ und Vorhersagen $\hat{y}_i := p_{\theta}(\mathbf{x}_i)$ aus Merkmalen $\mathbf{x}_i$ lautet die Aufgabe

$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^n L(y_i, p_{\theta}(\mathbf{x}_i)).$$

Für die lineare Regression mit quadratischem Verlust ist das $\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2$, und dieses eine Problem konnten wir in Kapitel 7 noch geschlossen lösen. Steht bei p_{θ} dagegen ein neuronales Netz, so ist θ hochdimensional, die Zielfunktion nicht konvex, und es bleibt nur die Iteration.

Regularisierte Regression. Ridge und Lasso hängen einen Strafterm an:

$$\hat{\beta} = \arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 + \lambda \|\beta\|_p^p, \quad p = 2 \text{ (Ridge)}, \quad p = 1 \text{ (Lasso)}.$$

Dieselbe Aufgabe lässt sich als beschränktes Problem schreiben,

$$\arg \min_{\beta} \|\mathbf{y} - \mathbf{X}\beta\|_2^2 \quad \text{so dass} \quad \|\beta\|_p \leq c,$$

und darin sitzt die geometrische Anschauung. Verletzt der KQ-Schätzer die Nebenbedingung, so blasen wir die Niveaumengen des Kleinste-Quadrate-Verlusts um ihn herum so lange auf, bis eine von ihnen die p -Norm-Kugel vom Radius c gerade berührt; der Berührungspunkt ist die Lösung. Bei $p = 1$ hat diese Kugel Ecken auf den Achsen, und an einer Ecke kommt die wachsende Niveaumenge besonders leicht zuerst an; deshalb setzt Lasso Koeffizienten exakt auf null.

Zwei Feinheiten sind dabei leicht zu übersehen. Erstens gehört zu jeder penalisierten Lösung ein Budget c , das dieselbe Lösung liefert – in der Gegenrichtung aber nur, solange die Nebenbedingung wirklich bindet. Warum das so ist und warum es keine allgemeine Umrechnungsformel gibt, klärt Beispiel 12.5.10 in Abschnitt 12.5. Zweitens steht im Strafterm die p -te Potenz der Norm. Für $p = 1$ ist das die Norm selbst, für $p = 2$ dagegen $\lambda \|\beta\|_2^2$ – und nur mit dieser quadrierten Fassung ergibt sich die geschlossene Ridge-Lösung $\hat{\beta} = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I}_p)^{-1} \mathbf{X}^T \mathbf{y}$ aus Beispiel 10.6.6 in Abschnitt 10.6. Die unquadrierte Fassung wäre ein anderes, ebenfalls konvexes Problem. In der Nebenbedingungsform spielt der Unterschied keine Rolle, weil $\|\beta\|_2 \leq c$ und $\|\beta\|_2^2 \leq c^2$ dieselbe Menge beschreiben.

Wann hat $f(x) = 0$ überhaupt eine Lösung?

Bevor wir ein Verfahren bauen, sollten wir wissen, wonach wir suchen. Sei $[a, b] \subset \mathbb{R}$ und $f : [a, b] \rightarrow \mathbb{R}$; gesucht sind Lösungen von $f(x) = 0$. Welche der folgenden Aussagen stimmen?

1. Ist f stetig, so hat $f(x) = 0$ genau eine Lösung.

Antwort: Falsch

Stetigkeit allein liefert weder Existenz noch Eindeutigkeit. Die konstante Funktion $f \equiv 1$ ist stetig und hat gar keine Nullstelle, die konstante Funktion $f \equiv 0$ ist stetig und hat lauter Nullstellen. Was Stetigkeit beiträgt, ist erst in Verbindung mit einem Vorzeichenwechsel etwas wert.

2. Gilt $f(a) < 0$ und $f(b) > 0$, so hat $f(x) = 0$ mindestens eine Lösung.

Antwort: Falsch

Das wäre der Zwischenwertsatz, aber der verlangt Stetigkeit, und die steht in der Voraussetzung nicht. Ein Gegenbeispiel ist die Sprungfunktion auf $[-1, 1]$ mit $f(x) = -1$ für $x < 0$ und $f(x) = 1$ für $x \geq 0$: Die Vorzeichen an den Rändern wechseln, eine Nullstelle gibt es nicht.

3. Ist f streng monoton, so hat $f(x) = 0$ höchstens eine Lösung.

Antwort: Wahr

Streng monotone Funktionen sind injektiv: Aus $x_1 < x_2$ folgt $f(x_1) \neq f(x_2)$. Der Wert 0 wird also höchstens einmal angenommen. Für die Existenz sagt das nichts, dafür brauchen wir Stetigkeit und einen Vorzeichenwechsel. Diese Aufteilung steckt im folgenden Satz.

Satz 12.1.5 (Existenz und Eindeutigkeit einer Nullstelle)

Sei $f : [a, b] \rightarrow \mathbb{R}$ streng monoton und stetig mit $f(a) \cdot f(b) \leq 0$. Dann hat die Gleichung $f(x) = 0$ genau eine Lösung $x^* \in [a, b]$.

Beweis.

- (1) *Höchstens eine Lösung.* Sind $x_1 < x_2$ zwei Punkte aus $[a, b]$, so ist $f(x_1) \neq f(x_2)$, weil f streng monoton ist. Der Wert 0 kann also höchstens an einer Stelle angenommen werden.

strenge Monotonie heißt: $x_1 < x_2$ erzwingt $f(x_1) < f(x_2)$ (steigend) oder $f(x_1) > f(x_2)$ (fallend), in beiden Fällen ungleiche Werte

- (2) *Der Randfall.* Ist $f(a) \cdot f(b) = 0$, so ist $f(a) = 0$ oder $f(b) = 0$, und wir sind bereits fertig: Die Nullstelle liegt am Rand.

ein Produkt reeller Zahlen ist genau dann null, wenn einer der Faktoren null ist

- (3) *Der interessante Fall.* Sei nun $f(a) \cdot f(b) < 0$, die beiden Randwerte haben also verschiedene Vorzeichen. Ohne Beschränkung der Allgemeinheit nehmen wir $f(a) < 0 < f(b)$ an.

im anderen Fall betrachten wir $-f$; diese Funktion ist ebenfalls stetig und streng monoton und hat genau dieselben Nullstellen

- (4) *Existenz.* Weil f stetig ist, liefert der Zwischenwertsatz zu jedem Wert zwischen $f(a)$ und $f(b)$ eine Urbildstelle, insbesondere zum Wert 0. Es gibt also ein $x^* \in (a, b)$ mit $f(x^*) = 0$. Zusammen mit Schritt 1 ist diese Lösung eindeutig.

der Zwischenwertsatz gilt für stetige Funktionen auf einem Intervall; ohne Stetigkeit ist er falsch, wie die Sprungfunktion aus dem Quiz zeigt

□

Bemerkung 12.1.6 (Hinreichend, aber nicht notwendig)

Der Satz gibt eine Garantie, keine Charakterisierung. Die Funktion $f(x) = x^2 - 1$ auf $[-2, 2]$ ist weder monoton noch wechselt sie das Vorzeichen zwischen den Rändern, und trotzdem hat sie dort zwei Nullstellen. Die Bedingungen des Satzes sind also *hinreichend*, aber nicht *notwendig*.

Praktisch heißt das: Findet ein Verfahren keine Nullstelle, so ist damit nicht gesagt, dass es keine gibt. Umgekehrt liefert ein Vorzeichenwechsel plus Stetigkeit eine belastbare Zusage, und darauf baut das erste Verfahren auf.

Das Bisektionsverfahren

Die Idee des Bisektionsverfahrens ist so einfach, dass sie fast keine Erklärung braucht. Wir kennen ein Intervall, in dem eine Nullstelle stecken muss, weil f an den Rändern das Vorzeichen wechselt. Wir halbieren es, schauen nach, in welcher Hälfte der Vorzeichenwechsel geblieben ist, und werfen die andere weg.

Algorithmus 12.1.7 (Bisektionsverfahren)

Gegeben seien eine stetige Funktion $f : \mathbb{R} \rightarrow \mathbb{R}$, zwei Punkte $a < b$ mit $f(a) \cdot f(b) < 0$ und eine Zielgenauigkeit $\epsilon > 0$. Solange $b - a > \epsilon$:

1. Setze $m = a + (b - a)/2$.
2. Haben $f(a)$ und $f(m)$ dasselbe Vorzeichen, so setze $a \leftarrow m$, andernfalls $b \leftarrow m$.

Als Näherung geben wir die Mitte $a + (b - a)/2$ des Endintervalls zurück.

Das ϵ ist hier die gewünschte Genauigkeit, nicht die Maschinengenauigkeit aus Kapitel 4. Wie viele Schritte kostet uns das?

Satz 12.1.8 (Schrittzahl der Bisektion)

Es gelten die Voraussetzungen von Algorithmus 12.1.7, und das Startintervall sei noch zu grob, $b - a > \epsilon$. Dann hat das Intervall nach k Durchläufen die Länge $(b - a)/2^k$, und es enthält weiterhin eine Nullstelle von f . Die Schleife bricht nach genau

$$k = \left\lceil \log_2 \left(\frac{b - a}{\epsilon} \right) \right\rceil$$

Durchläufen ab, und die zurückgegebene Intervallmitte $\hat{x}$ erfüllt $|x^* - \hat{x}| \leq \epsilon/2$.

Beweis.

- (1) *Der Vorzeichenwechsel überlebt.* Schreiben wir $[a_k, b_k]$ für das Intervall nach k Durchläufen. Haben $f(a_{k-1})$ und $f(m)$ dasselbe Vorzeichen, so hat $f(m)$ das andere Vorzeichen als $f(b_{k-1})$, und der Wechsel sitzt in $[m, b_{k-1}]$; diese Hälfte behalten wir. Andernfalls sitzt er in $[a_{k-1}, m]$, und wir behalten diese. In beiden Fällen gilt weiter $f(a_k) \cdot f(b_k) \leq 0$.

das leistet die Fallunterscheidung des Algorithmus. Gleichheit tritt nur im Ausnahmefall $f(m) = 0$ ein: Dann haben die beiden Werte nicht dasselbe Vorzeichen, es wird $b \leftarrow m$ gesetzt, und die Nullstelle sitzt anschließend auf dem rechten Rand

- (2) *Es gibt immer noch eine Nullstelle.* Auf $[a_k, b_k]$ ist f stetig, und die Randwerte haben verschiedene Vorzeichen oder einer von ihnen ist null. Im ersten Fall liefert der Zwischenwertsatz eine Nullstelle x^* im Inneren, im zweiten liegt sie auf dem Rand. Monotonie brauchen wir dafür nicht; nur für die Eindeutigkeit wäre sie nötig.

das sind die Schritte 2 und 4 des Beweises von Satz 12.1.5, angewandt auf das kleinere Intervall

- (3) *Die Länge halbiert sich.* Der Mittelpunkt teilt das Intervall in zwei gleich lange Hälften, also ist $b_k - a_k = (b_{k-1} - a_{k-1})/2$ und per Induktion $b_k - a_k = (b - a)/2^k$.

Induktionsanfang $k = 0$ ist das Startintervall selbst

- (4) *Die Abbruchbedingung.* Die Schleife läuft, solange $b_k - a_k > \epsilon$ ist. Sie endet also beim kleinsten k mit

$$\frac{b-a}{2^k} \leq \epsilon \iff 2^k \geq \frac{b-a}{\epsilon} \iff k \geq \log_2 \left(\frac{b-a}{\epsilon} \right),$$

und die kleinste solche ganze Zahl ist die aufgerundete rechte Seite.

der Logarithmus zur Basis 2 ist streng monoton wachsend, die Äquivalenzen bleiben also erhalten

- (5) **Die Fehlerschranke.** Die Nullstelle x^* liegt nach Schritt 2 im Endintervall, und von dessen Mitte $\hat{x}$ ist jeder Punkt des Intervalls höchstens eine halbe Intervalllänge entfernt. Also ist

$$|x^* - \hat{x}| \leq \frac{b_k - a_k}{2} \leq \frac{\epsilon}{2}.$$

hier zählt sich die Rückgabe der Intervallmitte aus; ein Randpunkt des Endintervalls hätte nur die Schranke ϵ

□

Ein Beispiel, das uns durch den Rest des Abschnitts begleitet: Für $f(x) = x^2 - 2$ auf $[1, 2]$, also die Suche nach $\sqrt{2}$, und $\epsilon = 10^{-6}$ ist $\log_2(10^6) = 19,93$, also brauchen wir 20 Schritte, und das Endintervall hat die Länge $2^{-20} = 9,54 \cdot 10^{-7}$. Die Bisektion gewinnt pro Schritt genau ein Bit, also $\log_{10} 2 \approx 0,301$ Dezimalstellen; für eine weitere gültige Stelle brauchen wir rund 3,3 Schritte. Das ist zuverlässig und langsam.

Bemerkung 12.1.9 (Die Bisektion in sieben Zeilen)

Kompakt geschrieben ist das Verfahren sieben Zeilen lang:

```
bisect <- function(f, a, b, eps) {
  while (b - a > eps) {
    mid <- (a + b) / 2
    if (f(a) * f(mid) <= 0) b <- mid else a <- mid
  }
  return((a + b) / 2)
}
```

Auf die letzte Zeile kommt es dabei an. Gäbe sie den *letzten Mittelpunkt* zurück, so wäre das nach der Zuweisung stets ein Randpunkt des Endintervalls: Statt der garantierten Genauigkeit $\epsilon/2$ aus Satz 12.1.8 bliebe nur ϵ , ein voller Halbierungsschritt verschenkt.

Vertiefung: Fließkomma-Fallen der Bisektion

Zwei Kleinigkeiten dieser Fassung sind Fließkomma-Handwerk aus Kapitel 4. Die erste betrifft den Mittelpunkt. Haben a und b dasselbe Vorzeichen, so ist $a + (b - a)/2$ die verlässlichere Formel: Die Differenz $b - a$ ist dann betragsmäßig höchstens so groß wie der größere der beiden Endpunkte, ihre Hälfte erst recht, und der Ausdruck bleibt auch gerundet zwischen a und b . Die Summe $a + b$ dagegen ist ein Zwischenergebnis, das größer sein kann als beide Endpunkte, und sie kann überlaufen, obwohl die Mitte selbst darstellbar wäre. Die zweite betrifft den Vorzeichentest. Nahe der Nullstelle sind $f(a)$ und $f(m)$ beide winzig; ihr Produkt unterläuft dann auf null, und der Produkttest kann einen echten Vorzeichenwechsel nicht mehr von einem Unterlauf unterscheiden. Vergleichen wir stattdessen die Vorzeichen, so stehen im Produkt nur noch ± 1 , und dort kann nichts mehr unterlaufen. So sieht die robuste Fassung aus:

```
bisect <- function(f, a, b, eps) {
  fa <- f(a); fb <- f(b)
  stopifnot(is.finite(fa), is.finite(fb), sign(fa) * sign(fb) < 0)
  while (b - a > eps) {
    mid <- a + (b - a) / 2          # bleibt auch gerundet in [a, b]
```



```

fmid <- f(mid)
if (fmid == 0) return(mid)      # exakt getroffen
if (sign(fa) * sign(fmid) < 0) {
  b <- mid
} else {
  a <- mid; fa <- fmid          # f(a) nur neu auswerten, wenn a sich bewegt
}
}
a + (b - a) / 2                # Mitte des Endintervalls
}

```

Der Aufruf `stopifnot` macht aus einer stillschweigenden Voraussetzung eine geprüfte, und das Zwischenspeichern von `fa` halbiert die Zahl der Funktionsauswertungen. Bei einer Zielfunktion, deren Auswertung eine Minute dauert, ist das kein Schönheitsfehler.

Bemerkung 12.1.10 (Umkehrfunktionen, Binärsuche und die Grenze des Verfahrens)

Umkehrfunktionen auswerten. Suchen wir $f^{-1}(y)$ für ein gegebenes y , so lösen wir einfach $f(x) - y = 0$. Für die Statistik ist der wichtigste Fall die Quantilfunktion: Ist F eine stetige, streng wachsende Verteilungsfunktion, so ist das p -Quantil die Lösung von $F(x) - p = 0$. Satz 12.1.5 ist hier wörtlich anwendbar, und genau so berechnen viele Programmbibliotheken Quantile, für die es keine geschlossene Formel gibt.

Verwandtschaft zur Binärsuche. Die Bisektion halbiert in jedem Schritt den Suchbereich, wie die binäre Suche in einer sortierten Liste. Auch der Aufwand ist derselbe, nämlich logarithmisch in der geforderten Auflösung (Abschnitt 2.3). Der Vorzeichenwechsel übernimmt dabei die Rolle des Größenvergleichs: Er sagt uns, in welcher Hälfte weiterzusuchen ist.

Und die Grenze. Beide Verfahren leben davon, dass $\mathbb{R}$ angeordnet ist. Im $\mathbb{R}^n$ ist es das nicht: Ein „Vorzeichenwechsel“ eines vektorwertigen f ist gar nicht definiert, und ein Gebiet lässt sich nicht so halbieren, dass eine Nullstelle nachweislich in einer Hälfte bleibt. Deshalb eignet sich die Bisektion nur für univariate Funktionen; für $n > 1$ brauchen wir andere Ideen, und die einfachste davon steht am Ende dieses Abschnitts.

Zwei Fragen bleiben offen. Wie viele Halbierungen kostet eine vorgegebene Genauigkeit wirklich, und was passiert, wenn im Startintervall mehr als eine Nullstelle liegt? Beides lässt sich am Verfahren selbst ablesen.

Interaktiv: Halbieren, Schritt für Schritt

Voreingestellt ist $f(x) = x^2 - 2$ auf $[1, 2]$, also die Suche nach $\sqrt{2}$; die Tabelle daneben protokolliert den Intervall-Verlauf. Der zweite Menüpunkt wechselt zu $f(x) = x^3 - 3x + 1$ auf $[-2, 2]$, wo die Voraussetzung von Satz 12.1.5 verletzt ist.

Schätzfrage. Wie viele Halbierungen braucht die Bisektion auf $[1, 2]$, bis das Intervall kürzer ist als $\epsilon = 10^{-6}$?

Lösung: 20 Schritte

Satz 12.1.8 sagt $\lceil \log_2((b-a)/\epsilon) \rceil = 20$ voraus, und das ist keine Schätzung, sondern die exakte Zahl.

Die Schranke aus Satz 12.1.8 trifft exakt: 10 Schritte für $\epsilon = 10^{-3}$, 20 für 10^{-6} , 34 für 10^{-10} , also $1/\log_{10} 2 = 3,32$ Schritte je gültiger Dezimalstelle. Der zweite Menüpunkt zeigt, was der Satz *nicht* verspricht: $f(x) = x^3 - 3x + 1$ hat auf $[-2, 2]$ drei Nullstellen bei $-1,879385$, $0,347296$ und $1,532089$, und schon der erste Mittelpunkt entscheidet, welche davon wir finden – weil $f(0) = 1$ dasselbe Vorzeichen trägt wie $f(2) = 3$, fliegt die rechte Hälfte samt zwei Nullstellen

sofort hinaus. Für die Eindeutigkeit fehlt hier die strenge Monotonie aus Satz 12.1.5.

Das Newton-Raphson-Verfahren

Die Bisektion benutzt von f nur das Vorzeichen. Ist f differenzierbar, verschenken wir damit Information: Die Ableitung sagt uns nicht nur, auf welcher Seite die Nullstelle liegt, sondern auch ungefähr wie weit.

Der Ansatz ist die Taylorentwicklung erster Ordnung aus Abschnitt 10.8. Um die aktuelle Stelle $x^{(k)}$ herum ersetzen wir f durch seine Tangente und verlangen von dieser, dass sie null wird:

$$f(x) \approx f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)}) \stackrel{!}{=} 0.$$

Das ist eine lineare Gleichung in x , und sie lässt sich hinschreiben, solange $f'(x^{(k)}) \neq 0$ ist. Auflösen liefert die nächste Iterierte.

Algorithmus 12.1.11 (Newton-Raphson-Verfahren für Nullstellen)

Gegeben seien eine differenzierbare Funktion $f: \mathbb{R} \rightarrow \mathbb{R}$ und ein Startwert $x^{(0)}$. Für $k = 0, 1, 2, \dots$ setze

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}, \quad (12.1.1)$$

solange $f'(x^{(k)}) \neq 0$ ist, und brich ab, sobald $|f(x^{(k)})|$ oder die Schrittlänge $|x^{(k+1)} - x^{(k)}|$ unter eine vorgegebene Schranke fällt.

Geometrisch ist $x^{(k+1)}$ der Schnittpunkt der Tangente an f im Punkt $(x^{(k)}, f(x^{(k)}))$ mit der x -Achse. Der Bruch in (12.1.1) sagt das noch einmal in Zahlen: Wir teilen die Höhe, die abzubauen ist, durch die Rate, mit der die Tangente sie abbaut.

Vertiefung: Newton-Raphson einmal vollständig durchgerechnet

Beispiel 12.1.12 (Die Wurzel aus zwei)

Wir bleiben bei $f(x) = x^2 - 2$ mit $f'(x) = 2x$ und $x^* = \sqrt{2} = 1,41421356$. Die Vorschrift (12.1.1) wird zu

$$x^{(k+1)} = x^{(k)} - \frac{(x^{(k)})^2 - 2}{2x^{(k)}} = \frac{1}{2} \left(x^{(k)} + \frac{2}{x^{(k)}} \right),$$

also gerade dem Heron-Verfahren, das seit der Antike Wurzeln zieht. Von $x^{(0)} = 2$ aus laufen die Iterierten so:

k	$x^{(k)}$	$e_k := x^{(k)} - x^* $	e_k/e_{k-1}^2
0	2,000000000000	$5,86 \cdot 10^{-1}$	-
1	1,500000000000	$8,58 \cdot 10^{-2}$	0,250
2	1,416666666667	$2,45 \cdot 10^{-3}$	0,333
3	1,414215686275	$2,12 \cdot 10^{-6}$	0,353
4	1,414213562375	$1,59 \cdot 10^{-12}$	0,354

Die letzte Spalte pendelt sich bei $1/(2x^*) = 0,3536$ ein. Die Zahl der gültigen Dezimalstellen läuft dabei über 0,2, 1,1, 2,6, 5,7 und 11,8: Sie verdoppelt sich in jedem Schritt grob. Zum Vergleich brauchte die Bisektion auf demselben Intervall 20 Schritte für sechs Stellen.

Bemerkung 12.1.13 (Quadratische Konvergenz)

Was wir in der letzten Spalte gesehen haben, ist *quadratische Konvergenz* (Konvergenzordnung): Der Fehler des nächsten Schrittes verhält sich wie das Quadrat des aktuellen,

$$e_{k+1} \approx C \cdot e_k^2,$$

mit einer Konstanten C , die von f abhängt. Ist f in einer Umgebung einer einfachen Nullstelle zweimal stetig differenzierbar, so ist asymptotisch $C = |f''(x^*)| / (2|f'(x^*)|)$, für unser Beispiel also $2/(2 \cdot 2\sqrt{2}) = 0,354$, genau der beobachtete Wert. Auf C kommt es an: Zur Merkregel „der Fehler quadriert sich pro Schritt“ abgekürzt, stimmt die Aussage schon in unserem eigenen Beispiel nicht, denn $e_0^2 = 0,343$ steht gegen $e_1 = 0,086$. Praktisch heißt quadratische Konvergenz: Sobald wir nah genug dran sind, verdoppelt jeder Schritt die Zahl der richtigen Stellen, und wenige Schritte genügen bis zur Maschinengenauigkeit.

Der Preis dafür steht im Kleingedruckten. Erstens braucht das Verfahren die Ableitung, die Bisektion kam mit Funktionswerten aus. Zweitens gilt die Aussage nur *lokal*: Weit weg vom Ziel kann die Tangente irgendwohin zeigen, und die Iteration läuft dann davon. Drittens muss die Nullstelle einfach und f' hinreichend glatt sein; eine lokal Lipschitz-stetige Ableitung ist eine übliche hinreichende Annahme. Ist $f'(x^*) = 0$, so bricht die Konvergenz auf lineare Geschwindigkeit ein, und unterwegs kann ein $f'(x^{(k)}) = 0$ den Schritt ganz undefiniert machen. Die Bisektion hat keinen dieser Vorbehalte, dafür braucht sie für dieselbe Genauigkeit ein Vielfaches an Schritten. In der Praxis kombinieren wir beides: erst klammern, dann Newton. Wir kommen in Abschnitt 12.4 darauf zurück.

Das Kleingedruckte klingt harmlos. Wie schlimm wird es tatsächlich, wenn die Tangente flach liegt?

Interaktiv: Die Tangente als Wegweiser

Die Tafel zeigt Algorithmus 12.1.11 Schritt für Schritt: violett der Graph von f , orange die Tangente im aktuellen Punkt, und wo sie die x -Achse trifft, sitzt die nächste blaue Iterierte.

Der Startpunkt lässt sich auf der Achse ziehen oder mit dem Regler setzen; die gestrichelten roten Senkrechten im zweiten Beispiel markieren die Stellen mit $f' = 0$.

Schätzfrage. Die dritte Funktion, $f(x) = \arctan x$, hat genau EINE Nullstelle. Findet Newton sie von jedem Startpunkt aus?

Lösung: *nein*

Die Schwelle liegt bei $|x^{(0)}| = 1,3917$.

Auf $f(x) = x^2 - 2$ läuft alles glatt: Die Fehlerspalte fällt über $8,6 \cdot 10^{-2}$ und $2,5 \cdot 10^{-3}$ auf $1,6 \cdot 10^{-12}$, und der Quotient e_{k+1}/e_k^2 pendelt sich bei 0,3536 ein. Auf $f(x) = x^3 - 3x + 1$ entscheidet dagegen der Startpunkt, welche der drei Nullstellen wir finden, und in der Nähe von $x = 1$ wird es unangenehm: Bei $x^{(0)} = 1,05$ ist $f' = 0,31$, der erste Schritt springt nach 4,28, und der Lauf braucht zehn Schritte statt fünf. Am deutlichsten ist der Arkustangens: Er hat genau eine Nullstelle, aber ab $|x^{(0)}| > 1,3917$ überholt jeder Schritt den vorigen, und die Iterierten $1,5 \rightarrow -1,694 \rightarrow 2,321 \rightarrow -5,114 \rightarrow 32,30 \rightarrow -1575,3$ laufen auseinander. Die Bisektion hätte hier ohne Weiteres funktioniert; sie ist eben langsam, aber nicht zu überlisten.

Bemerkung 12.1.14 (Von der Nullstelle zum Minimum)

Jetzt schließt sich der Kreis zu Bemerkung 12.1.3. Suchen wir das Minimum einer differenzierbaren Funktion f , so suchen wir eine Nullstelle von ∇f . Wenden wir (12.1.1) auf diese Funktion an, so steht im Nenner die Ableitung des Gradienten, also die Hesse-Matrix:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \left(\nabla f(\mathbf{x}^{(k)}) \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \right)^\top.$$

Das ist wörtlich Algorithmus 10.8.11 aus Abschnitt 10.8. Newton für $\min_{\mathbf{x}} f(\mathbf{x})$ ist Newton-Raphson für $\nabla f(\mathbf{x}) = \mathbf{0}^\top$, und das Transponierte steht da, weil der Gradient in unserer Konvention eine Zeile ist. In Abschnitt 12.4 bauen wir dieses Verfahren zu einem praxistauglichen aus.

Für $f: \mathbb{R}^n \rightarrow \mathbb{R}$ tritt in (12.1.1) an die Stelle der Division die Lösung des linearen Systems $\mathbf{J}_f(\mathbf{x}^{(k)}) \mathbf{d} = -f(\mathbf{x}^{(k)})$ mit der Jacobimatrix aus Abschnitt 10.3; die nächste Iterierte ist dann $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}$. Das ist teuer: pro Schritt eine ganze Matrix von Ableitungen und eine Zerlegung.

Vertiefung: Fixpunktiteration erster Ordnung

Die einfachste Alternative ersetzt die Ableitung durch eine feste Zahl. Das Ergebnis ist ein Nebenweg, aber ein lehrreicher: Es nimmt den Gradientenabstieg aus Abschnitt 12.3 in Reinform vorweg.

Algorithmus 12.1.15 (Fixpunktiteration erster Ordnung)

Gegeben seien $f: \mathbb{R}^n \rightarrow \mathbb{R}$, ein Startpunkt $\mathbf{x}^{(0)}$ und eine Schrittweite $\gamma > 0$. Für $k = 1, 2, \dots$ setze

$$\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \gamma f(\mathbf{x}^{(k-1)}). \quad (12.1.2)$$

Der Name kommt daher, dass (12.1.2) eine Fixpunktiteration $\mathbf{x} \mapsto g(\mathbf{x})$ für $g(\mathbf{x}) := \mathbf{x} - \gamma f(\mathbf{x})$ ist: Wegen $\gamma \neq 0$ gilt $g(\mathbf{x}^*) = \mathbf{x}^*$ genau dann, wenn $f(\mathbf{x}^*) = \mathbf{0}$ ist. Fixpunkte von g und Nullstellen von f sind dasselbe.

Für affines $f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$ ist (12.1.2) übrigens wörtlich das Richardson-Verfahren aus Abschnitt 8.3, und die dortige Konvergenzbedingung ist der Spezialfall des folgenden Satzes.

Satz 12.1.16 (Konvergenzrate der Fixpunktiteration)

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}^n$ in einer Umgebung von $\mathbf{x}^*$ differenzierbar mit $f(\mathbf{x}^*) = \mathbf{0}$, und sei

$$\rho := \|\mathbf{I}_n - \gamma \mathbf{J}_f(\mathbf{x}^*)\| < 1 \quad (12.1.3)$$

für eine induzierte Matrixnorm. Dann gibt es zu jedem $\rho' \in (\rho, 1)$ ein $\delta > 0$, sodass für jeden Startpunkt mit $\|\mathbf{x}^{(0)} - \mathbf{x}^*\| \leq \delta$ die Iteration (12.1.2) gegen $\mathbf{x}^*$ konvergiert, und zwar mit

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\| \leq (\rho')^k \|\mathbf{x}^{(0)} - \mathbf{x}^*\| = O((\rho')^k).$$

Ist f affin, so gilt die Schranke mit ρ selbst und für jeden Startpunkt.

Beweis.

- (1) *Fehlerrekursion und Kontraktion.* Schreiben wir $\mathbf{e}_k := \mathbf{x}^{(k)} - \mathbf{x}^*$. Weil f in $\mathbf{x}^*$ differenzierbar ist und dort verschwindet, ist $f(\mathbf{x}^{(k-1)}) = \mathbf{J}_f(\mathbf{x}^*) \mathbf{e}_{k-1} + r(\mathbf{e}_{k-1})$ mit $\|r(\mathbf{h})\| = o(\|\mathbf{h}\|)$. Einsetzen in (12.1.2) und Abziehen von $\mathbf{x}^*$ auf beiden Seiten liefert

$$\mathbf{e}_k = (\mathbf{I}_n - \gamma \mathbf{J}_f(\mathbf{x}^*)) \mathbf{e}_{k-1} - \gamma r(\mathbf{e}_{k-1}), \quad \text{also} \quad \|\mathbf{e}_k\| \leq \rho \|\mathbf{e}_{k-1}\| + \gamma \|r(\mathbf{e}_{k-1})\|.$$

Sei nun $\rho' \in (\rho, 1)$ beliebig. Weil $\|r(\mathbf{h})\| / \|\mathbf{h}\| \rightarrow 0$ für $\mathbf{h} \rightarrow \mathbf{0}$ gilt, gibt es ein $\delta > 0$ mit $\gamma \|r(\mathbf{h})\| \leq (\rho' - \rho) \|\mathbf{h}\|$ für alle $\|\mathbf{h}\| \leq \delta$, und für $\|\mathbf{e}_{k-1}\| \leq \delta$ folgt daraus $\|\mathbf{e}_k\| \leq \rho' \|\mathbf{e}_{k-1}\|$.

die Zerlegung von f ist Definition 10.1.5, die Fréchet-Differenzierbarkeit: Funktionswert gleich lineare Näherung plus Restterm, der schneller als linear verschwindet. Die Abschätzung benutzt danach die Dreiecksungleichung und die Verträglichkeit der induzierten Norm, $\|\mathbf{M}\mathbf{v}\| \leq \|\mathbf{M}\| \|\mathbf{v}\|$ (Abschnitt 3.3). Genau hier wird die Aussage lokal: Sie gilt nur in einer Kugel um $\mathbf{x}^*$, deren Radius von ρ' abhängt

- (2) *Induktion.* Ist $\|\mathbf{e}_0\| \leq \delta$, so ist $\|\mathbf{e}_1\| \leq \rho' \|\mathbf{e}_0\| \leq \delta$, und dasselbe Argument greift erneut. Per Induktion bleibt die ganze Folge in der Kugel und es gilt $\|\mathbf{e}_k\| \leq (\rho')^k \|\mathbf{e}_0\|$. Wegen $\rho' < 1$ konvergiert die rechte Seite gegen null.

$\rho' < 1$ sorgt dafür, dass die Kugel nie verlassen wird; ohne diese Beobachtung wäre die Induktion nicht abgeschlossen

- (3) *Der affine Fall.* Ist $f(\mathbf{x}) = \mathbf{A}\mathbf{x} - \mathbf{b}$, so ist $\mathbf{J}_f = \mathbf{A}$ konstant und $r \equiv \mathbf{0}$. Die Abschätzung aus Schritt 1 gibt dann direkt $\|\mathbf{e}_k\| \leq \rho \|\mathbf{e}_{k-1}\|$, ohne Einschränkung an den Startpunkt.

eine affine Abbildung ist ihre eigene lineare Näherung, der Restterm entfällt

□

Kurzgefasst lautet das Ergebnis $\|\mathbf{x}^{(k)} - \mathbf{x}^*\| = O(\rho^k)$. Das trifft den Kern, unterschlägt aber zwei Vorbehalte: Die Aussage gilt nur in der Nähe von $\mathbf{x}^*$, und der Restterm verwässert die Rate im nichtaffinen Fall um ein beliebig kleines Stück.

Eine Voraussetzung ist dabei leicht zu übersehen. Damit überhaupt ein $\gamma > 0$ mit $\rho < 1$ existiert, muss die Ableitung im Zielpunkt in die richtige Richtung zeigen: Im Eindimensionalen ist $\rho = |1 - \gamma f'(x^*)| < 1$ genau für $0 < \gamma < 2/f'(x^*)$, und das setzt $f'(x^*) > 0$ voraus. Ist f dort fallend, so treibt uns (12.1.2) mit positivem γ von der Nullstelle weg – das Gegenmittel ist wohlfeil, wir arbeiten mit $-f$, das dieselben Nullstellen hat. Deshalb wird die Fixpunktiteration oft gleich mit der Annahme formuliert, f sei monoton steigend.

Bemerkung 12.1.17 (Das Dilemma der Schrittweite)

Die Vor- und Nachteile des Verfahrens hängen an einer einzigen Zahl. Positiv: Es ist denkbar simpel, funktioniert für multivariates f und braucht keine Jacobimatrix, sondern nur Funktionsauswertungen; bei gutartigem f , also kleinem ρ , ist es zudem schnell. Negativ: Die Schrittweite γ muss passen.

Zu klein heißt langsam. Mit ρ nahe bei 1 schrumpft der Fehler pro Schritt kaum, und wir brauchen $\log(10)/(-\log \rho)$ Schritte je Dezimalstelle. *Zu groß* heißt, dass der Schritt über die Nullstelle hinausschießt und dabei größer wird als der Fehler, den er beheben sollte. Dann ist $\rho \geq 1$, und Satz 12.1.16 sagt nichts mehr zu.

Wie eng das Fenster ist, lässt sich im symmetrisch positiv definiten Fall exakt angeben. Hat $J_f(\mathbf{x}^*)$ die Eigenwerte $0 < \lambda_{\min} \leq \dots \leq \lambda_{\max}$, so ist

$$\gamma^* = \frac{2}{\lambda_{\min} + \lambda_{\max}} \quad \text{mit} \quad \rho = \frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = \frac{\kappa - 1}{\kappa + 1}$$

die beste Wahl, und für $\gamma > 2/\lambda_{\max}$ läuft die Iteration von jedem Startpunkt aus auseinander, dessen Fehler einen Anteil in der steilsten Eigenrichtung hat. Beides steht in Satz 12.3.15, wo dieselbe Eigenzerlegung von $I - \gamma A$ den Gradientenabstieg trägt: Dort ist $\max_i |1 - \gamma \lambda_i|$ der Fehlerfaktor, und γ^* ist genau die Schrittweite, die ihn kleinstmöglich macht – nur steht dort die Hesse-Matrix, wo hier die Jacobimatrix steht. Der Buchstabe $\kappa = \lambda_{\max}/\lambda_{\min}$ ist die Konditionszahl aus Abschnitt 3.5. Schlechte Kondition heißt also wörtlich langsame Konvergenz: Bei $\kappa = 10$ ist selbst die beste Rate noch $9/11 = 0,818$. Und wer die optimale Schrittweite verdoppelt, landet stets auf oder jenseits der Divergenzgrenze, denn $4/(\lambda_{\min} + \lambda_{\max}) \geq 2/\lambda_{\max}$ mit Gleichheit nur für $\lambda_{\min} = \lambda_{\max}$.

Im Eindimensionalen ist die beste Wahl besonders sprechend. Dort drückt $\gamma^* = 1/f'(x^*)$ die Rate auf $\rho = 0$, und die Iteration ist nichts anderes als das Newton-Verfahren mit eingefrorener Ableitung. Der Haken ist offensichtlich: Wir müssten $f'(x^*)$ schon kennen. Newton umgeht das, indem es die Ableitung in jedem Schritt neu auswertet, und bezahlt sie mit einer Auswertung pro Schritt.

In der Praxis wählen wir γ_k adaptiv, etwa durch eine Liniensuche: Ein zu großer Vorschlag wird verkleinert, bis er genügend Fortschritt liefert. Die akzeptierten Schrittweiten müssen dabei nicht gegen null gehen. Folgen mit $\gamma_k \rightarrow 0$ spielen später beim stochastischen Gradientenabstieg eine eigene Rolle und brauchen dort zusätzliche Bedingungen. Die deterministische Schrittweitensteuerung behandelt Abschnitt 12.3.

Bleibt die Frage, wie weit uns diese Schranke trägt. Sie hängt an einer einzigen Zahl, und die lässt sich für drei kleine Systeme durchspielen: Ab welcher Schrittweite kippt die Iteration?

Interaktiv: Die Schrittweite als Regler

Alle drei Beispiele sind affin, $f(\mathbf{x}) = A(\mathbf{x} - \mathbf{x}^*)$ mit $\mathbf{x}^* = \mathbf{0}$. Damit ist die Jacobimatrix überall A , und $\rho = \|I - \gamma A\|_2$ ist nach dem affinen Fall im Beweis eine echte Schranke ohne Restterm.

Schätzfrage. Das dritte System hat $A = \text{diag}(1, 10)$, also Kondition 10. Ab welchem γ divergiert es?

Lösung: $0,2 \gamma$

Die Grenze liegt bei $2/\lambda_{\max} = 0,2$, das beste erreichbare ρ bei $9/11 = 0,818$.

Entschieden wird alles von ρ . Das erste System ist symmetrisch mit den Eigenwerten 4,618 und 2,382; die Voreinstellung $\gamma = 0,25$ ist die aus dem Richardson-Beispiel in Abschnitt 8.3 und liefert dieselbe Rate 0,4045, das Optimum liegt bei $\gamma^* = 2/7$ mit $\rho = 0,319$. Beim zweiten sind die Eigenwerte komplex, der Weg wird zur Spirale, und der Drehanteil lässt sich durch keine Schrittweite wegregeln: Selbst im Optimum bleibt $2/\sqrt{5} = 0,894$. Das dritte führt Bemerkung 12.1.17 vor – bei $\gamma = 0,25$ ist $\rho = 1,5$, erst unterhalb von 0,2 läuft die Iteration zusammen, und die beste Rate bleibt $9/11$. Sehenswert ist dort das Zickzack: Die steile Richtung überschießt, während die flache kaum vorankommt – dasselbe Bild wie beim Gradientenabstieg in Abschnitt 12.3.

1. Für $f(\mathbf{x}) = \mathbf{Ax} - \mathbf{b}$ ist die Fixpunktiteration (12.1.2) genau das Richardson-Verfahren aus Abschnitt 8.3.

Antwort: Wahr

Einsetzen gibt $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} - \gamma(\mathbf{Ax}^{(k-1)} - \mathbf{b})$, also die Korrektur des Residuums mit fester Schrittweite. Die dortige Konvergenzbedingung $\|\mathbf{I} - \gamma\mathbf{A}\| < 1$ ist wörtlich (12.1.3), und weil f affin ist, greift der affine Fall im Beweis von Satz 12.1.16: Die Schranke gilt für jeden Startpunkt.

2. Ist $\rho = \|\mathbf{I}_n - \gamma\mathbf{J}_f(\mathbf{x}^*)\| < 1$, so konvergiert die Fixpunktiteration von jedem Startpunkt aus gegen $\mathbf{x}^*$.

Antwort: Falsch

Satz 12.1.16 ist eine *lokale* Aussage: Die Schranke im Beweis gilt nur in einer Kugel um $\mathbf{x}^*$, deren Radius wir nicht kennen. Weiter draußen taugt die lineare Näherung nichts mehr, und die Iteration kann zu einer anderen Nullstelle laufen oder ganz davonlaufen. Nur im affinen Fall verschwindet der Restterm und die Aussage wird global.

3. Verdoppeln wir im symmetrisch positiv definiten Fall die optimale Schrittweite, so ist die Konvergenzgarantie verloren.

Antwort: Wahr

Die optimale Wahl ist $\gamma^* = 2/(\lambda_{\min} + \lambda_{\max})$, divergiert wird ab $2/\lambda_{\max}$. Wegen $4/(\lambda_{\min} + \lambda_{\max}) \geq 2/\lambda_{\max}$ liegt das Doppelte immer auf oder jenseits der Grenze, mit Gleichheit nur für $\lambda_{\min} = \lambda_{\max}$. Bei $\mathbf{A} = \text{diag}(1, 10)$ etwa ist $\gamma^* = 2/11 = 0,18$ und die Grenze 0,2; das Doppelte 0,36 liegt weit darüber.

Damit ist der Werkzeugkasten für Nullstellen beisammen. Für die Optimierung ist allerdings gerade die Monotonie heikel, von der die Fixpunktiteration lebt: Der Gradient einer beliebigen Zielfunktion ist im Allgemeinen nicht monoton. Abschnitt 12.2 setzt deshalb neu an und fragt zuerst, was ein Minimum überhaupt auszeichnet.

Selbsttest

1. Das Bisektionsverfahren braucht die Ableitung von f .

Antwort: Falsch

Es braucht nur Funktionswerte, genauer sogar nur deren Vorzeichen, und einen Startbereich mit Vorzeichenwechsel. Das ist seine große Stärke: Es funktioniert für nicht differenzierbare, nur numerisch auswertbare oder schlicht hässliche Funktionen, und es kann nach Satz 12.1.8 nicht divergieren. Dafür kostet jede weitere gültige Dezimalstelle rund 3,3 Schritte, während Newton in der Nähe der Nullstelle ihre Zahl verdoppelt.

2. Eine zusätzliche gültige Dezimalstelle kostet die Bisektion rund 3,3 Schritte.

Antwort: Wahr

Jeder Schritt halbiert die Intervalllänge, gewinnt also genau ein Bit oder $\log_{10} 2 \approx 0,301$ Dezimalstellen. Für eine ganze Stelle brauchen wir $1/\log_{10} 2 = 3,32$ Schritte. Nach Satz 12.1.8 sind das für $[1, 2]$ und $\epsilon = 10^{-6}$ zusammen 20 Durchläufe.

3. Ab welchem $|x^{(0)}|$ läuft Newton auf $f(x) = \arctan x$ auseinander?

Lösung: 1.39

Die Schwelle liegt bei 1,3917; sie löst $\arctan(\xi)(1 + \xi^2) = 2\xi$. Darunter zieht sich das Pendeln zusammen, darüber wächst es. Dass es überhaupt eine solche Schwelle gibt, obwohl die Funktion nur eine einzige Nullstelle hat, ist die Pointe von Bemerkung 12.1.13: Quadratische Konvergenz ist eine *lokale* Aussage, und wie groß die Umgebung ist, sagt sie nicht.

4. Das Newton-Raphson-Verfahren konvergiert quadratisch, gleichgültig wo wir starten und welche Nullstelle wir suchen.

Antwort: Falsch

Beides ist zu viel versprochen. Quadratisch wird es erst in der Nähe der Nullstelle, und nur wenn diese einfach ist, also $f'(x^*) \neq 0$ gilt. Bei einer mehrfachen Nullstelle sinkt die Ordnung auf 1, und weit weg vom Ziel kann die Tangente die Iteration beliebig weit forttragen (Bemerkung 12.1.13).

Vertiefung: Heath §5.1 ordnet Existenz, Eindeutigkeit und Konvergenzraten von Nullstellenproblemen ein, §5.5 behandelt Bisektion, Fixpunktiteration und Newton im Eindimensionalen, §5.6 die Systeme. Heath Kapitel 6 ist der Optimierung gewidmet und trägt die folgenden Abschnitte dieses Kapitels.

12.2 Optimalität und Sattelpunkte

Vom Nullstellen- zum Optimierungsproblem

Abschnitt 12.1 hat Gleichungen gelöst: gesucht war ein x mit $f(x) = 0$. Ab hier suchen wir etwas anderes, nämlich die Stelle, an der eine Funktion ihren kleinsten Wert annimmt. Für die Statistik ist das der Normalfall. Kleinste Quadrate minimieren eine Residuenquadratsumme, Maximum-Likelihood maximiert eine Log-Likelihood, ein neuronales Netz minimiert einen empirischen Verlust. In allen drei Fällen steht am Ende dieselbe Frage: Wo liegt das Optimum, und woran erkennen wir es, wenn wir dort sind?

Dieser Abschnitt beantwortet die zweite Hälfte der Frage. Er sagt, was ein Optimum ist, welche Bedingungen an einer optimalen Stelle gelten und warum Konvexität den Unterschied zwischen „irgendein Minimum“ und „das Minimum“ ausmacht. Die Verfahren, die das Optimum dann tatsächlich suchen, kommen in den Abschnitten 12.3 und 12.4.

Der Farbcode des Kapitels: blau die Iterierten eines Verfahrens, grün das Optimum und Grenzwerte, rot die Richtungen, in denen etwas davonläuft, und Nebenbedingungen. Gradient und Hesse-Matrix behalten das Orange aus Kapitel 10; aus ihnen bauen die Verfahren ihre Suchrichtungen. Der stationäre Punkt des Sattelbeispiels bekommt Violett, weil er keine dieser Rollen spielt.

Definition 12.2.1 (Lokales und globales Minimum)

Seien $\mathcal{X} \subseteq \mathbb{R}^n$ die *zulässige Menge* (feasible set) und $f : \mathcal{X} \rightarrow \mathbb{R}$ die Zielfunktion (objective function), das Problem also

$$\mathbf{x}^* \in \arg \min_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}). \quad (12.2.1)$$

Ein Punkt $\mathbf{x}^* \in \mathcal{X}$ heißt

- *globales Minimum*, wenn $f(\mathbf{x}^*) \leq f(\mathbf{x})$ für alle $\mathbf{x} \in \mathcal{X}$ gilt;
- *lokales Minimum*, wenn es eine Umgebung U von $\mathbf{x}^*$ gibt, sodass $f(\mathbf{x}^*) \leq f(\mathbf{x})$ für alle $\mathbf{x} \in U \cap \mathcal{X}$ gilt.

Gilt jeweils die strikte Ungleichung für $\mathbf{x} \neq \mathbf{x}^*$, so heißt das Minimum *strikt*.

Das Problem (12.2.1) hat Abschnitt 12.1 schon aufgeschrieben, dort mit S für die zulässige Menge. Wir schreiben hier $\mathcal{X}$, weil der Buchstabe S in den Sätzen unten für den offenen Definitionsbereich gebraucht wird. Neu ist die Unterscheidung zwischen lokal und global, und um die dreht sich der

ganze Abschnitt.

Zwei Sprachfallen stecken in der Schreibweise. Erstens ist $\arg \min$ streng genommen die Menge aller Minimierer; wir schreiben $\mathbf{x}^*$ für ein Element daraus, sofern überhaupt eines existiert. Zweitens meint „unbeschränkte Optimierung“ die Optimierung ohne Nebenbedingungen, also $\mathcal{X} = \mathbb{R}^n$, und nicht etwa eine unbeschränkte Funktion. Die Sätze zur Konvexität unten formulieren wir trotzdem für eine allgemeine zulässige Menge, weil gerade dort sichtbar wird, was $\mathcal{X}$ beiträgt; die Verfahren der Abschnitte 12.3 und 12.4 arbeiten dann durchweg mit $\mathcal{X} = \mathbb{R}^n$, und wie Nebenbedingungen die Bedingungen ändern, klärt Abschnitt 12.5.

Bemerkung 12.2.2 (Maximieren ist Minimieren)

Ein Maximierungsproblem müssen wir nicht getrennt behandeln. Eine einzige Zeile hält das fest, und sie meint zweierlei:

$$\max_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) = -\min_{\mathbf{x} \in \mathcal{X}} (-f(\mathbf{x})), \quad \arg \max_{\mathbf{x} \in \mathcal{X}} f(\mathbf{x}) = \arg \min_{\mathbf{x} \in \mathcal{X}} (-f(\mathbf{x})).$$

Die Werte kippen im Vorzeichen, die Argumente bleiben dieselben; die zweite Gleichung stand schon in Abschnitt 12.1. Deshalb sind die gängigen Bibliotheken Minimierer, und deshalb heißt die Zielfunktion der Maximum-Likelihood-Schätzung dort in aller Regel die *negative* Log-Likelihood. Wir formulieren im Folgenden alles für Minima.

Die Bedingung erster Ordnung

Abschnitt 12.1 hat die Bedingung schon benutzt, und Kapitel 10 hat um kritische Punkte herum das Hesse-Kriterium gebaut. Bewiesen haben wir die Bedingung selbst an keiner der beiden Stellen. Für die Optimierung ist sie der Ausgangspunkt jedes Verfahrens, deshalb holen wir den Beweis hier nach.

Satz 12.2.3 (Notwendige Bedingung erster Ordnung)

Seien $S \subseteq \mathbb{R}^n$ offen, $f : S \rightarrow \mathbb{R}$ und $\mathbf{x}^* \in S$ ein lokales Minimum von f . Ist f in $\mathbf{x}^*$ differenzierbar, so gilt

$$\nabla f(\mathbf{x}^*) = \mathbf{0}^\top. \quad (12.2.2)$$

Dieselbe Aussage gilt für lokale Maxima.

Beweis.

- (1) Sei $\mathbf{h} \in \mathbb{R}^n$ beliebig. Weil S offen ist, gibt es ein $\delta > 0$, sodass $\mathbf{x}^* + t\mathbf{h} \in S$ für alle $|t| < \delta$ gilt. Wir betrachten die Funktion einer einzigen Variablen

$$g : (-\delta, \delta) \rightarrow \mathbb{R}, \quad g(t) := f(\mathbf{x}^* + t\mathbf{h}).$$

Weil $\mathbf{x}^*$ ein lokales Minimum von f ist, hat g in $t = 0$ ein lokales Minimum.

Für kleine $|t|$ liegt $\mathbf{x}^* + t\mathbf{h}$ in der Umgebung U aus Definition 12.2.1, dort ist $f(\mathbf{x}^*) \leq f(\mathbf{x}^* + t\mathbf{h})$, also $g(0) \leq g(t)$

- (2) g ist in $t = 0$ differenzierbar mit $g'(0) = \nabla f(\mathbf{x}^*) \mathbf{h}$. Nun schauen wir den Differenzenquotienten von zwei Seiten an. Für hinreichend kleine $t > 0$ ist $(g(t) - g(0))/t \geq 0$, weil der Zähler nicht negativ ist; im Grenzwert folgt $g'(0) \geq 0$. Für hinreichend kleine $t < 0$ dreht der Nenner das

Vorzeichen um, wir erhalten $(g(t) - g(0))/t \leq 0$ und im Grenzwert $g'(0) \leq 0$. Beides zusammen ergibt $g'(0) = 0$.

Der Wert der Ableitung ist die Richtungsableitung aus Abschnitt 10.2; die Kettenregel auf die affine Abbildung $t \mapsto \mathbf{x}^* + t\mathbf{h}$ angewandt gibt genau $\nabla f(\mathbf{x}^*)\mathbf{h}$

- (3) Der zweite Schritt gilt für *jedes* $\mathbf{h}$. Setzen wir nacheinander die Einheitsvektoren $\mathbf{h} = \mathbf{e}_j$ ein, so steht dort $\nabla f(\mathbf{x}^*)\mathbf{e}_j = \partial f(\mathbf{x}^*)/\partial x_j = 0$ für $j = 1, \dots, n$, und das ist (12.2.2). Für ein lokales Maximum wenden wir das Gezeigte auf $-f$ an und benutzen Bemerkung 12.2.2.

Ein Zeilenvektor, der auf jedem Einheitsvektor verschwindet, ist der Nullvektor

□

Definition 12.2.4 (Stationärer Punkt)

Ein Punkt $\mathbf{x}$ mit $\nabla f(\mathbf{x}) = \mathbf{0}^\top$ heißt *stationärer Punkt* von f . In Kapitel 10 heißt derselbe Punkt *kritischer Punkt*; beide Wörter sind gebräuchlich und meinen dasselbe.

Zwei Warnungen gehören sofort dazu. Die Bedingung ist *notwendig*, nicht *hinreichend*: Jedes lokale Minimum ist stationär, aber ein stationärer Punkt kann ebenso ein Maximum oder ein Sattelpunkt sein. Und sie gilt nur im *Inneren*. Liegt das Minimum auf dem Rand der zulässigen Menge, so darf der Gradient dort sehr wohl von null verschieden sein. Verboten ist dann nur noch eine zulässige Abstiegsrichtung: Für jede Richtung $\mathbf{d}$, die von $\mathbf{x}^*$ aus in $\mathcal{X}$ hineinführt, muss $\nabla f(\mathbf{x}^*)\mathbf{d} \geq 0$ gelten, der *negative* Gradient darf also nicht ins Innere zeigen. Bei $f(x) = x$ auf $\mathcal{X} = [0, 1]$ etwa liegt das Minimum in 0 mit $f'(0) = 1$: Der Gradient zeigt dort sogar ins Innere, der negative Gradient dagegen hinaus. Genau davon handelt Abschnitt 12.5.

Bemerkung 12.2.5 (Optimieren heißt Gleichungen lösen)

Gleichung (12.2.2) ist ein System aus n Gleichungen in n Unbekannten, meist nichtlinear. Damit sind wir wieder bei Abschnitt 12.1, und das ganze dortige Werkzeug steht bereit. Insbesondere ist

$$\Phi(\mathbf{x}) := \mathbf{x} - \gamma \nabla f(\mathbf{x})^\top \quad (12.2.3)$$

eine Fixpunktabbildung, deren Fixpunkte genau die stationären Punkte von f sind, denn $\Phi(\mathbf{x}) = \mathbf{x}$ gilt für $\gamma \neq 0$ genau dann, wenn $\nabla f(\mathbf{x}) = \mathbf{0}^\top$ ist. Die zu (12.2.3) gehörige Fixpunktiteration $\mathbf{x}^{(k+1)} = \Phi(\mathbf{x}^{(k)})$ ist der Gradientenabstieg aus Abschnitt 12.3.

Analytisch lösen lässt sich (12.2.2) nur in Sonderfällen. Einer davon ist wichtig genug, um ihn zu nennen: Bei den kleinsten Quadraten ist der Gradient linear in β , aus (12.2.2) werden die Normalgleichungen, und die lösen wir mit den Mitteln aus Kapitel 7 in einem Zug. Sobald das Modell nichtlinear wird, ist damit Schluss, und wir iterieren.

Lokal oder global

Ein Verfahren, das nur die Umgebung des aktuellen Punktes ansieht, kann grundsätzlich nur lokale Minima finden. Ob das genügt, entscheidet die Zielfunktion. Kapitel 11 hat die beiden Antworten schon bewiesen, und wir zitieren sie hier, statt sie ein zweites Mal zu führen.

Lokal ist global. Sind $\mathcal{X} \subseteq \mathbb{R}^n$ konvex und $f: \mathcal{X} \rightarrow \mathbb{R}$ konvex, so ist jedes lokale Minimum von f auf $\mathcal{X}$ auch ein globales, und zwar ohne jede Differenzierbarkeit (Bemerkung 11.5.2). Ist f differenzierbar, so steht in Satz 11.5.1 sogar die schärfere Fassung: Dann ist schon jeder stationäre

Punkt ein globales Minimum. Ein lokal suchendes Verfahren kann bei konvexem f also nichts verpassen, und deshalb ist Konvexität die wichtigste Struktureigenschaft der ganzen Optimierung.

Höchstens eine Lösung. Ist f darüber hinaus strikt konvex, gilt also $f(\lambda \mathbf{x} + (1 - \lambda)\mathbf{y}) < \lambda f(\mathbf{x}) + (1 - \lambda)f(\mathbf{y})$ für alle $\mathbf{x} \neq \mathbf{y}$ in $\mathcal{X}$ und alle $\lambda \in (0, 1)$, so hat f auf $\mathcal{X}$ nach Satz 11.5.5 höchstens einen Minimierer; der Beweis dort nimmt den Mittelpunkt zweier Minimierer und findet an ihm einen strikt kleineren Wert. Über die Existenz eines Minimierers sagt keine der beiden Aussagen etwas: $f(x) = e^x$ ist auf $\mathbb{R}$ strikt konvex und hat kein Minimum.

Beide Aussagen kosten dasselbe Kleingedruckte.

Bemerkung 12.2.6 (Beide Sätze brauchen zwei Konvexitätsbedingungen)

Leicht zu übersehen ist, dass in beiden Sätzen die Konvexität der zulässigen Menge genauso tragend ist wie die der Zielfunktion. Die Beweise in Abschnitt 11.5 zeigen, warum: Der Vergleichspunkt muss *zulässig* sein, dafür sorgt die Menge, und er muss *besser* sein, dafür sorgt die Funktion.

Ohne konvexe Menge nützt die beste Zielfunktion nichts. Wir nehmen $\mathcal{X} = \{-1, +1\}$ und $f(y) = y^2$. Die Zielfunktion ist strikt konvex, und trotzdem sind $y = -1$ und $y = +1$ beide optimal mit dem Wert 1. Der Mittelpunkt $z = 0$ hätte den kleineren Wert 0, er ist aber nicht zulässig, und der Widerspruch aus Satz 11.5.5 entsteht gar nicht erst. Die Eindeutigkeit fällt mit der Konvexität der Menge.

Das Projektionstheorem ist der Spezialfall, bei dem die zweite Bedingung geschenkt ist. Es minimiert die Distanz zu einem festen Punkt $\mathbf{x}$ über eine abgeschlossene konvexe Menge, also $\min_{\mathbf{y} \in \mathcal{X}} \|\mathbf{x} - \mathbf{y}\|^2$ (Kapitel 11). Sichtbar ist dort nur die Bedingung an die Menge, weil die strikte Konvexität der quadrierten Distanz in $\mathbf{y}$ schon in der Struktur des Raumes steckt.

Vertiefung: Warum die quadrierte Norm strikt konvex ist

Die Dreiecksungleichung liefert $\|\lambda \mathbf{a} + (1 - \lambda)\mathbf{b}\| \leq \lambda \|\mathbf{a}\| + (1 - \lambda)\|\mathbf{b}\|$ und damit nur die Konvexität der Norm, nicht die strikte. Gleichheit tritt nämlich ein, sobald $\mathbf{a}$ und $\mathbf{b}$ in dieselbe Richtung zeigen: Für $\mathbf{a} = (1; 0)$, $\mathbf{b} = (2; 0)$ und $\lambda = \frac{1}{2}$ stehen auf beiden Seiten 1,5. Die *quadrierte* Norm ist dagegen strikt konvex, und zwar wegen der Identität

$$\|\lambda \mathbf{a} + (1 - \lambda)\mathbf{b}\|^2 = \lambda \|\mathbf{a}\|^2 + (1 - \lambda)\|\mathbf{b}\|^2 - \lambda(1 - \lambda)\|\mathbf{a} - \mathbf{b}\|^2, \quad (12.2.4)$$

die durch Ausmultiplizieren mit der Bilinearität des Skalarprodukts entsteht und der Parallelogrammgleichung entspricht. Für $\mathbf{a} \neq \mathbf{b}$ und $\lambda \in (0, 1)$ ist der rote Term strikt negativ, und genau das ist die strikte Konvexität. Weil $t \mapsto t^2$ auf $[0, \infty)$ streng wächst, haben Distanz und quadrierte Distanz dieselben Minimierer; das Quadrieren kostet also nichts und bringt die strikte Konvexität. Gebraucht wird dafür das Skalarprodukt, nicht bloß eine Norm.

Konvexe Verlustfunktionen in Statistik und ML

Beispiel 12.2.7 (Vier konvexe Verlustfunktionen)

Vier Standardfälle stehen hier nebeneinander. Alle vier begegnen uns im Skript wieder.

Kleinste Quadrate. Für $L(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2$ ist der Gradient $\nabla L(\boldsymbol{\beta}) = 2(\mathbf{X}\boldsymbol{\beta} - \mathbf{y})^\top \mathbf{X}$ und die Hesse-Matrix

$$\mathbf{H}_L = 2\mathbf{X}^\top \mathbf{X}.$$

Sie ist stets positiv semidefinit, denn $\mathbf{h}^\top \mathbf{X}^\top \mathbf{X} \mathbf{h} = \|\mathbf{X}\mathbf{h}\|_2^2 \geq 0$, und positiv definit genau dann,

wenn $\mathbf{X}$ vollen Spaltenrang hat: Nur dann folgt aus $\mathbf{h} \neq \mathbf{0}$ auch $\mathbf{Xh} \neq \mathbf{0}$. Mit vollem Spaltenrang ist L also strikt konvex und der Kleinste-Quadrate-Schätzer nach Satz 11.5.5 eindeutig. Ein Zahlenbeispiel: Für die Designmatrix mit den Zeilen $(1, 1)$, $(1, 2)$, $(1, 3)$ und $(1, 5)$ ist

$\mathbf{H}_L = \begin{pmatrix} 8 & 22 \\ 22 & 78 \end{pmatrix}$ mit den Eigenwerten 84,34 und 1,66, also positiv definit.

Ridge-Regression. Der Strafterm addiert eine Krümmung dazu,

$$f(\boldsymbol{\beta}) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_2^2, \quad \mathbf{H}_f = 2(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}_p).$$

Die Eigenwerte von $\mathbf{H}_f$ sind $2(\mu_i + \lambda)$, wenn $\mu_i \geq 0$ die Eigenwerte von $\mathbf{X}^\top \mathbf{X}$ sind. Für $\lambda > 0$ sind sie alle positiv, und zwar unabhängig vom Rang von $\mathbf{X}$. Ein rangdefektes Beispiel: Hat $\mathbf{X}$ zwei identische Spalten, etwa mit den Zeilen $(1, 1)$, $(2, 2)$, $(3, 3)$, so hat $\mathbf{X}^\top \mathbf{X}$ die Eigenwerte 28 und 0; mit $\lambda = 1,5$ stehen in $\mathbf{H}_f$ die Eigenwerte 59 und 3. Ridge macht aus einem Problem mit unendlich vielen Lösungen eines mit genau einer; den Gradienten dazu haben wir in Abschnitt 10.6 schon gerechnet.

Negative Log-Likelihood. Ist die Log-Likelihood konkav, so ist ihr Negatives konvex, und die Maximum-Likelihood-Schätzung wird zu einem konvexen Minimierungsproblem. Das ist bei vielen Standardmodellen so, etwa bei der Normalverteilung mit bekannter Varianz, wo dabei wieder genau das Kleinste-Quadrate-Problem herauskommt, und bei der logistischen Regression aus Abschnitt 10.6. Automatisch ist die Konkavität nicht: Die Log-Likelihood einer Mischverteilung ist nicht konkav, und solche Modelle sind der Standardfall für mehrere lokale Optima.

Hinge-Loss. Die Support-Vector-Machine summiert über die Daten den *Hinge-Verlust* $L(y, \hat{y}) = \max(0, 1 - y\hat{y})$ und addiert einen Strafterm. Als punktweises Maximum zweier affiner Funktionen ist der Hinge-Loss konvex (Kapitel 11), aber weder strikt konvex noch überall differenzierbar: Für $y\hat{y} \geq 1$ ist er konstant null, und bei $y\hat{y} = 1$ hat er einen Knick. Verfahren, die nur Gradienten kennen, brauchen dort das Subgradienten-Werkzeug aus Kapitel 11.

Sattelpunkte

Bei nicht-konvexen Funktionen sagt $\nabla f(\mathbf{x}) = \mathbf{0}^\top$ wenig. Der Punkt kann ein Minimum sein, ein Maximum, oder keines von beidem.

Definition 12.2.8 (Sattelpunkt)

Ein stationärer Punkt $\mathbf{x}^*$ von f heißt *Sattelpunkt*, wenn jede Umgebung von $\mathbf{x}^*$ Punkte $\mathbf{x}_-$ und $\mathbf{x}_+$ enthält mit

$$f(\mathbf{x}_-) < f(\mathbf{x}^*) < f(\mathbf{x}_+).$$

Ist die Hesse-Matrix $\mathbf{H}_f(\mathbf{x}^*)$ *indefinit*, hat sie also sowohl positive als auch negative Eigenwerte, so sprechen wir genauer von einem *Sattelpunkt mit indefiniter Hesse-Matrix*.

Ist f zweimal stetig differenzierbar, wie es Satz 10.7.9 verlangt, so ist die indefinite Hesse-Matrix ein hinreichendes Kriterium für die Definition: In jeder Umgebung liegen Punkte mit größerem und Punkte mit kleinerem Funktionswert. Die Umkehrung gilt nicht. Bei $f(x) = x^3$ ist $x = 0$ ebenfalls ein Sattelpunkt, obwohl $f''(0) = 0$ ist und das Kriterium zweiter Ordnung nichts entscheidet. Praktisch wichtig ist vor allem der Fall der indefiniten Hesse-Matrix.

Beispiel 12.2.9 (Der Standardsattel)

Wir rechnen den einfachsten Fall vollständig durch. Sei

$$f(x, y) = x^2 - y^2.$$

Stationäre Punkte. Der Gradient ist $\nabla f(x, y) = (2x, -2y)$, und er verschwindet genau im Ursprung. Der einzige stationäre Punkt ist also $\mathbf{x}^* = (0; 0)$.

Hesse-Matrix. Zweimaliges Ableiten gibt

$$\mathbf{H}_f(x, y) = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix},$$

und zwar an jeder Stelle dieselbe Matrix, weil f quadratisch ist. Sie ist diagonal, ihre Eigenwerte stehen also ablesbar auf der Diagonalen: $\lambda_1 = 2 > 0$ zum Eigenvektor $\mathbf{e}_1$ und $\lambda_2 = -2 < 0$ zum Eigenvektor $\mathbf{e}_2$. Die Vorzeichen sind verschieden, die Matrix ist indefinit, und $\mathbf{x}^*$ ist ein Sattelpunkt.

Was das geometrisch heißt. Schneiden wir f entlang der beiden Achsen, so sehen wir zwei Parabeln mit entgegengesetzter Öffnung:

$$f(t, 0) = t^2 \quad (\text{Minimum in } t = 0), \quad f(0, t) = -t^2 \quad (\text{Maximum in } t = 0).$$

Derselbe Punkt ist in der einen Richtung der tiefste und in der anderen der höchste seiner Umgebung. Die Höhenlinien machen das sichtbar: Das Niveau $f = 0$ besteht aus den beiden Geraden $y = \pm x$, die positiven Niveaus sind Hyperbeln, die in x -Richtung öffnen, die negativen Hyperbeln öffnen in y -Richtung.

Damit ist die Lage beschrieben. Offen ist die praktisch entscheidende Frage: Kann ein Verfahren an so einem Punkt hängenbleiben?

Interaktiv: Zwei Verfahren am Sattelpunkt

Die Tafel zeichnet die Höhenlinien von Beispiel 12.2.9 und hängt zwei Verfahren daran. Drei Voreinstellungen sind die drei Experimente, die sich lohnen: der gewöhnliche Start, der Sonderfall auf der grünen Achse und ein schräger Start für den Newton-Knopf.

Schätzfrage. Von welchen Startpunkten aus läuft der *Gradientenabstieg* in den Sattelpunkt?

Lösung: achse

Die Voreinstellung „Startstrahl $y = 0$ “ trifft genau diesen Fall.

Die beiden Verfahren gehen gegensätzlich mit dem Sattel um. Der Newton-Knopf landet von jedem Startpunkt aus in einem einzigen Schritt exakt in $\mathbf{x}^*$, denn f ist quadratisch und seine Näherung zweiter Ordnung damit exakt. Der Gradientenabstieg dagegen entkommt: Mit $y = 0,25$ steht er aus $(1,5; 0,4)$ nach acht Schritten bei $(0,0059; 10,25)$ – die erste Komponente fällt gegen null, die zweite läuft davon. Nur auf dem Startstrahl $y = 0$ kriecht auch er hinein, weil dort die zweite Komponente in jedem Schritt null bleibt; aus $x = 1,5$ stehen die Iterierten dann nach zwanzig Schritten bei $1,4 \cdot 10^{-6}$. Warum diese Ausnahme in der Praxis keine Rolle spielt, sagt Bemerkung 12.2.10.

Bemerkung 12.2.10 (Was Sattelpunkte für die Verfahren bedeuten)

Newton läuft hinein. Der Newton-Schritt aus Abschnitt 10.8 sucht eine Nullstelle des Gradienten, und ein Sattelpunkt ist eine. Für unser f rechnen wir das in einer Zeile nach. Der Schritt lautet $\mathbf{x}^{(1)} = \mathbf{x} - \mathbf{H}_f^{-1} \nabla f(\mathbf{x})^\top$ mit der Spalte $\nabla f(\mathbf{x})^\top$, und mit $\mathbf{H}_f^{-1} = \text{diag}(\frac{1}{2}, -\frac{1}{2})$ ist

$$\mathbf{x}^{(1)} = \begin{pmatrix} x \\ y \end{pmatrix} - \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & -\frac{1}{2} \end{pmatrix} \begin{pmatrix} 2x \\ -2y \end{pmatrix} = \begin{pmatrix} x - x \\ y - y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix},$$

und zwar von jedem Startpunkt aus. Wer Krümmungsinformation benutzt, ohne auf ihr Vorzeichen zu achten, wird von Sattelpunkten genauso angezogen wie von Minima. Abschnitt 12.4 zeigt die Gegenmittel.

Der Gradientenabstieg entkommt. Mit Schrittweite γ lautet die Iteration komponentenweise

$$\mathbf{x}^{(k+1)} = (1 - 2\gamma) \mathbf{x}^{(k)}, \quad \mathbf{y}^{(k+1)} = (1 + 2\gamma) \mathbf{y}^{(k)}. \quad (12.2.5)$$

Für $0 < \gamma < \frac{1}{2}$ schrumpft nach (12.2.5) die erste Komponente, die zweite wächst in jedem Schritt um den Faktor $1 + 2\gamma > 1$. Nur wer exakt auf der x -Achse startet, also mit $\mathbf{y}^{(0)} = 0$, landet im Sattel. Diese Startmenge ist eine Gerade im $\mathbb{R}^2$ und hat das Volumen null. In der Praxis genügt schon Rundungsrauschen, um sie zu verfehlen, und der stochastische Gradientenabstieg aus Abschnitt 12.4 bringt sein eigenes Rauschen mit.

Eine hochdimensionale Vorzeichenheuristik. Solange kein Eigenwert von $\mathbf{H}_f$ null ist, entscheidet allein das Vorzeichenmuster: Ein Minimum verlangt, dass *alle* n Eigenwerte positiv sind, für einen Sattelpunkt mit indefiniter Hesse-Matrix genügen gemischte Vorzeichen. (Ein Eigenwert null lässt beide Fälle offen, dazu Bemerkung 12.2.12.) Die Münzwurf-Faustregel aus Abschnitt 10.7 beziffert den Unterschied: Wären die n Vorzeichen unabhängige faire Münzwürfe, so hätte ein Minimum die Wahrscheinlichkeit 2^{-n} und irgendein Extremum 2^{1-n} , für $n = 100$ also $7,9 \cdot 10^{-31}$ und $1,6 \cdot 10^{-30}$. Ein Toy-Modell ist das und keine Verteilungsaussage über die stationären Punkte eines konkreten Problems; belastbar bleibt die Geometrie dahinter, dass positive Definitheit ein einheitliches Vorzeichen verlangt und Indefinitheit nur ein gemischtes Muster. Wie Verfahren auf negative Krümmung reagieren, zeigen die Abschnitte 12.3 und 12.4.

Optimalitätsbedingungen im Überblick

Damit haben wir alle Bausteine zusammen. Der folgende Satz stellt sie nebeneinander, für eine zweimal stetig differenzierbare Zielfunktion.

Satz 12.2.11 (Bedingungen erster und zweiter Ordnung)

Seien $S \subseteq \mathbb{R}^n$ offen, $f \in \mathcal{C}^2(S)$ und $\mathbf{x}^* \in S$.

1. *Notwendig, erste Ordnung.* Ist $\mathbf{x}^*$ ein lokales Minimum, so gilt $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$.
2. *Notwendig, zweite Ordnung.* Ist $\mathbf{x}^*$ ein lokales Minimum, so ist $\mathbf{H}_f(\mathbf{x}^*) \succeq 0$, also positiv semidefinit.
3. *Hinreichend, zweite Ordnung.* Gilt $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ und ist $\mathbf{H}_f(\mathbf{x}^*) \succ 0$, also positiv definit, so ist $\mathbf{x}^*$ ein striktes lokales Minimum.

Vertiefung: Beweis der notwendigen Bedingung zweiter Ordnung

Beweis.

- (1) Teil 1 ist Satz 12.2.3, und Teil 3 ist Satz 10.7.9(1) aus Abschnitt 10.7. Zu zeigen bleibt Teil 2. Sei $\mathbf{h} \in \mathbb{R}^n$ beliebig und wieder $g(t) = f(\mathbf{x}^* + t\mathbf{h})$ wie im Beweis von Satz 12.2.3. Die Taylorentwicklung zweiter Ordnung aus Abschnitt 10.8 gibt mit $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ aus Teil 1

$$g(t) = g(0) + \frac{1}{2} t^2 \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} + o(t^2) \quad (t \rightarrow 0).$$

Der lineare Term fällt weg, weil $\mathbf{x}^$ nach Teil 1 stationär ist*

- (2) Weil $\mathbf{x}^*$ ein lokales Minimum ist, gilt $g(t) \geq g(0)$ für alle hinreichend kleinen $|t|$. Wir ziehen $g(0)$ ab und teilen durch $t^2 > 0$:

$$0 \leq \frac{g(t) - g(0)}{t^2} = \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} + \frac{o(t^2)}{t^2}.$$

Der rote Quotient verschwindet für $t \rightarrow 0$, also bleibt $\mathbf{h}^\top \mathbf{H}_f(\mathbf{x}^*) \mathbf{h} \geq 0$ stehen. Weil $\mathbf{h}$ beliebig war, ist $\mathbf{H}_f(\mathbf{x}^*)$ positiv semidefinit.

Genau hier bleibt nur die schwache Ungleichung übrig: Der Grenzwert einer Folge nichtnegativer Zahlen ist nichtnegativ, aber nicht notwendig positiv

□

Bemerkung 12.2.12 (Warum die Rückrichtung nicht gilt)

Verlockend wäre, aus Teil 3 eine Äquivalenz zu machen und zu schreiben: $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ und $\mathbf{H}_f(\mathbf{x}^*) \succ 0$ genau dann, wenn $\mathbf{x}^*$ ein lokales Minimum ist. Die Richtung von links nach rechts stimmt, das ist Teil 3 des Satzes. Die Rückrichtung ist falsch.

Das Gegenbeispiel ist schon in einer Variablen zu haben: $f(x) = x^4$ hat in $x = 0$ ein striktes globales Minimum, denn $f(x) > 0 = f(0)$ für alle $x \neq 0$. Es ist aber $f'(0) = 0$ und $f''(0) = 0$, die Hesse-Matrix $\mathbf{H}_f(0) = (0)$ ist also nicht positiv definit. Was aus einem lokalen Minimum wirklich folgt, steht in Teil 2 des Satzes: positive Semidefinitheit, mehr nicht. Bemerkung 10.7.10 in Abschnitt 10.7 führt neben x^4 noch $-x^4$ und x^3 vor, drei Funktionen mit derselben Hesse-Matrix im Nullpunkt und drei verschiedenen Antworten.

Die Fälle lassen sich in einer Tabelle zusammenfassen. Sie gilt für stationäre Punkte, also unter der Voraussetzung $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$:

$\mathbf{H}_f(\mathbf{x}^*)$	$\mathbf{x}^*$ ist
$\succ 0$ (positiv definit)	striktes lokales Minimum
$\prec 0$ (negativ definit)	striktes lokales Maximum
indefinit	Sattelpunkt
semidefinit, aber nicht definit	unklar, weitere Analyse nötig

Die letzte Zeile ist keine Bequemlichkeit, sondern eine Tatsache: In diesem Fall entscheidet die zweite Ableitung nichts, und wir müssen höhere Ableitungen ansehen oder anders argumentieren. Ein einzelner Eigenwert null genügt dafür übrigens nicht. Bei $\mathbf{H}_f = \text{diag}(1, -1, 0)$ ist die Matrix indefinit, und die dritte Zeile greift nach wie vor.

Bemerkung 12.2.13 (Was Konvexität aus der Tabelle macht)

Sei f auf einer offenen konvexen Menge S konvex und zweimal stetig differenzierbar. Dann ist $\mathbf{H}_f(\mathbf{x}) \succeq 0$ an jeder Stelle (Satz 10.7.11), eine indefinite Hesse-Matrix kann also gar nicht

auftreten: Die dritte Zeile der Tabelle bleibt leer, eine konvexe Funktion hat keine Sattelpunkte. Mehr noch, die Bedingung erster Ordnung wird für konvexe Funktionen hinreichend: Jeder stationäre Punkt ist dann bereits ein globales Minimum (Satz 11.5.1, dessen Beweis nur $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ in die Gradientenungleichung einsetzt).

Für die Praxis heißt das: Bei einem konvexen Problem sagt ein *verschwindender* Gradient schon alles, wir stehen dann im globalen Minimum. Ein bloß *kleiner* Gradient garantiert das nicht von selbst; wie viel er wert ist, hängt an der Krümmung, und darauf kommt Abschnitt 12.3 mit den Abbruchkriterien zurück. Bei einem nicht-konvexen Problem heißt ein kleiner Gradient ohnehin nur, dass wir irgendwo stationär stehen, und die Tabelle sagt, welche drei Möglichkeiten das offenlässt.

Selbsttest

1. Ein Punkt mit $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ ist ein lokales Minimum von f .

Antwort: Falsch

Die Bedingung ist notwendig, nicht hinreichend (Satz 12.2.3). Der Ursprung ist ein stationärer Punkt von $f(x, y) = x^2 - y^2$, aber weder Minimum noch Maximum, sondern ein Sattelpunkt: entlang der x -Achse geht es nach oben, entlang der y -Achse nach unten.

2. Ein Maximierungsproblem lässt sich ohne Verlust in ein Minimierungsproblem umschreiben.

Antwort: Wahr

Es ist $\max f = -\min(-f)$, und die Maximierer von f sind genau die Minimierer von $-f$ (Bemerkung 12.2.2). Die Werte kippen im Vorzeichen, die Argumente bleiben dieselben. Deshalb minimieren Softwarebibliotheken die *negative* Log-Likelihood.

3. Ist die Zielfunktion strikt konvex, so hat das Optimierungsproblem höchstens eine Lösung, gleichgültig wie die zulässige Menge aussieht.

Antwort: Falsch

Satz 11.5.5 braucht beide Konvexitätsbedingungen. Auf $\mathcal{X} = \{-1, +1\}$ ist $f(y) = y^2$ strikt konvex, und trotzdem sind beide Punkte optimal mit dem Wert 1. Der Mittelpunkt 0 wäre besser, ist aber nicht zulässig, und ohne ihn entsteht der Widerspruch des Beweises gar nicht erst (Bemerkung 12.2.6).

4. Die Hesse-Matrix $2\mathbf{X}^\top\mathbf{X}$ des Kleinste-Quadrate-Problems ist immer positiv semidefinit, aber nur bei vollem Spaltenrang von $\mathbf{X}$ positiv definit.

Antwort: Wahr

Es ist $\mathbf{h}^\top \mathbf{X}^\top \mathbf{X} \mathbf{h} = \|\mathbf{X}\mathbf{h}\|_2^2 \geq 0$, also stets semidefinit. Gleichheit tritt für ein $\mathbf{h} \neq \mathbf{0}$ genau dann ein, wenn $\mathbf{X}\mathbf{h} = \mathbf{0}$ gilt, wenn also die Spalten linear abhängig sind. Ohne vollen Spaltenrang ist die Zielfunktion nur konvex, nicht strikt konvex, und das Minimum ist nicht mehr eindeutig. Ridge repariert das, indem es 2λ auf jeden Eigenwert addiert (Beispiel 12.2.7).

5. Aus $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$ und positiv semidefiniter Hesse-Matrix folgt, dass $\mathbf{x}^*$ ein lokales Minimum ist.

Antwort: Falsch

Semidefinit reicht nicht, das ist die letzte Zeile der Klassifikationstabelle. Für $f(x) = x^3$ ist $f'(0) = 0$ und $f''(0) = 0$, die Hesse-Matrix also positiv semidefinit, und trotzdem liegt in 0 kein Minimum: links davon ist f negativ. Positive *Definitheit* dagegen genügt (Satz 12.2.11).

6. Das Newton-Verfahren umgeht Sattelpunkte, weil es die Krümmung mitbenutzt.

Antwort: Falsch

Es läuft eher hinein. Auf $f(x, y) = x^2 - y^2$ landet ein einziger Newton-Schritt von jedem Startpunkt aus exakt im Sattel $(0; 0)$, wie die Rechnung in Bemerkung 12.2.10 zeigt. Der Schritt sucht eine Nullstelle des Gradienten, und ein Sattelpunkt ist eine. Der Gradientenabstieg entkommt dagegen, weil er die Komponente in der Richtung mit negativem Eigenwert in jedem Schritt um den Faktor $1 + 2\gamma$ vergrößert.

7. Mit welchem Faktor wächst im Widget die y -Komponente des Gradientenabstiegs pro Schritt, wenn $\gamma = 0,25$ eingestellt ist?

Lösung: 1.5

Die Iteration lautet $y^{(k+1)} = y^{(k)} - \gamma \cdot (-2y^{(k)}) = (1 + 2\gamma) y^{(k)}$, mit $\gamma = 0,25$ also der Faktor 1,5. Die Ablesetafel des Widgets nennt ihn direkt; in der x -Richtung steht daneben $|1 - 2\gamma| = 0,5$. Diese Gegenläufigkeit ist der ganze Grund, warum der Abstieg dem Sattel entkommt.

8. Im Münzwurfmodell für Hesse-Eigenwertvorzeichen wird positive Definitheit mit wachsender Dimension exponentiell selten.

Antwort: Wahr

Sind die n Vorzeichen unabhängig und fair, müssen für positive Definitheit alle n positiv ausfallen; das hat Wahrscheinlichkeit 2^{-n} . Für irgendein Extremum dürfen alle positiv oder alle negativ sein, also 2^{1-n} . Bei $n = 100$ sind das $7,9 \cdot 10^{-31}$ beziehungsweise $1,6 \cdot 10^{-30}$. Für reale Probleme ist die Unabhängigkeitsannahme im Allgemeinen falsch; der Selbsttest prüft deshalb ausdrücklich nur die Konsequenz des Toy-Modells (Bemerkung 12.2.10).

Vertiefung: Heath §6.1 stellt Optimierungsprobleme und ihre Sprechweisen zusammen, §6.2.1 sammelt die Folgen der Konvexität und §6.2.2 die Optimalitätsbedingungen ohne Nebenbedingungen; die Bedingungen mit Nebenbedingungen (Heath §6.2.3) folgen in Abschnitt 12.5.

12.3 Nelder-Mead und Gradientenabstieg

Wie viel Ableitung darf es sein?

Abschnitt 12.2 hat die Bedingung geliefert, an der ein Minimum zu erkennen ist: In einem inneren Minimum verschwindet der Gradient. Nur ist $\nabla f(\mathbf{x}) = \mathbf{0}^\top$ ein nichtlineares Gleichungssystem, und solche Systeme lösen wir nach Abschnitt 12.1 iterativ. Ab hier geht es also um Verfahren, die eine Folge $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$ erzeugen und hoffen, dass sie gegen ein Minimum $\mathbf{x}^*$ läuft.

Die Verfahren unterscheiden sich vor allem darin, wie viel Ableitungsinformation sie überhaupt benutzen. Das ist keine akademische Einteilung: Jede Ableitung, die ein Verfahren verlangt, muss jemand bereitstellen, sei es von Hand, per automatischem Differenzieren oder durch Differenzenquotienten, und jede Ableitung kostet Rechenzeit in jedem einzelnen Schritt.

Bemerkung 12.3.1 (Taxonomie nach Ableitungsordnung)

Ordnung	benutzt	Beispiele
nullte	nur Funktionswerte $f(\mathbf{x})$	Nelder-Mead, Gittersuche
erste	f und ∇f	Gradientenabstieg, Momentum-Verfahren, SGD
zweite	f , ∇f und $\mathbf{H}_f$	Newton-Verfahren

Der Tausch hat in beide Richtungen seinen Preis. Verfahren nullter Ordnung sind robust und laufen auf allem, was sich auswerten lässt, auch auf Funktionen mit Knicken oder auf Simulationsergebnissen; dafür brauchen sie viele Iterationen. Höhere Ordnung bringt schnellere Konvergenz, macht aber jeden einzelnen Schritt teurer, und die Hesse-Matrix hat n^2 Einträge. In der Praxis greifen wir oft zu einem Kompromiss, den superlinearen Quasi-Newton-Verfahren wie BFGS, die eine Näherung der inversen Hesse-Matrix aus den bereits berechneten Gradienten aufbauen (Abschnitt 12.4).

Dieser Abschnitt behandelt die ersten beiden Zeilen: ein Verfahren nullter Ordnung als robusten Notnagel und den Gradientenabstieg als das Arbeitspferd, auf dem das ganze maschinelle Lernen läuft.

Nelder-Mead: Optimieren ohne Ableitungen

Manchmal gibt es keine Ableitung. Die Zielfunktion ist das Ergebnis einer Simulation, einer Datenbankabfrage oder eines Programms, dessen Innenleben wir nicht kennen; sie kann Knicke haben oder gar nicht überall differenzierbar sein. Dann bleibt nur, Funktionswerte zu vergleichen. Das bekannteste Verfahren dieser Art arbeitet mit einem *Simplex*, also mit $n + 1$ Punkten im $\mathbb{R}^n$: einem Dreieck in der Ebene, einem Tetraeder im Raum. Der Simplex kriecht durch den Suchraum, dehnt sich und schrumpft wieder, indem er in jedem Schritt seine schlechteste Ecke durch eine bessere ersetzt.

Algorithmus 12.3.2 (Nelder-Mead-Simplexverfahren)

Gegeben seien $f : \mathbb{R}^n \rightarrow \mathbb{R}$ und ein Startsimplex aus $n + 1$ Punkten. Wiederhole, bis der Simplex klein genug ist:

1. *Sortieren*. Ordne die Ecken nach Funktionswert, $\mathbf{x}_{(1)}$ sei die beste und $\mathbf{x}_{(n+1)}$ die schlechteste. Bilde den Schwerpunkt $\mathbf{c}$ der n besten Ecken.
2. *Reflexion*. Spiegle die schlechteste Ecke am Schwerpunkt, $\mathbf{x}_r = \mathbf{c} + (\mathbf{c} - \mathbf{x}_{(n+1)})$. Ist $f(\mathbf{x}_r)$ besser als die zweitschlechteste Ecke, aber nicht besser als die beste, so übernahm $\mathbf{x}_r$ und beginne von vorn.
3. *Expansion*. Ist $f(\mathbf{x}_r)$ besser als alles Bisherige, so gehe in derselben Richtung noch weiter, $\mathbf{x}_e = \mathbf{c} + 2(\mathbf{c} - \mathbf{x}_{(n+1)})$, und übernahm den besseren der beiden Punkte.
4. *Kontraktion*. Ist $\mathbf{x}_r$ nicht besser als die zweitschlechteste Ecke, so probiere einen Punkt auf halbem Weg zwischen Schwerpunkt und dem besseren der beiden Punkte $\mathbf{x}_r$ und $\mathbf{x}_{(n+1)}$; übernahm ihn, falls er besser ist als beide.
5. *Schrumpfen*. Hilft auch das nicht, so ziehe alle Ecken zur besten Ecke $\mathbf{x}_{(1)}$ hin.

Zwei Bewegungsmuster stecken in diesen vier Zügen. Solange es bergab geht, schaukelt sich der Simplex über Reflexionen und Expansionen voran und wird dabei lang; sobald er ein Tal umschließt, ziehen ihn Kontraktionen und Schrumpfschritte zusammen. Die Schrittweite steuert er dabei selbst, ohne dass wir einen Parameter einstellen müssten. Das ist der eigentliche Reiz des Verfahrens.

Bemerkung 12.3.3 (Wann sich Nelder-Mead lohnt)

Situation	geeignet?
Ableitungen nicht verfügbar	ja, das ist der Hauptgrund
niedrige Dimension, $n \leq 10$	ja
nicht glatte Zielfunktion	ja, es werden nur Werte verglichen
hohe Dimension, $n > 50$	nein, zu langsam
präzise Konvergenz nötig	nein, im Allgemeinen ohne Garantie

Kosten. Ein Simplex im $\mathbb{R}^n$ hat $n + 1$ Ecken. Ein gewöhnlicher Schritt kostet ein bis zwei neue Funktionsauswertungen, ein Schrumpfschritt dagegen n neue Ecken; pro Iteration fallen also $O(n)$ Auswertungen an (Landau-Notation), und es sind meist sehr viele Iterationen nötig. In hoher Dimension wird beides zum Problem.

Garantien. Es gibt keinen allgemeinen Konvergenzsatz. Schlimmer noch, es sind strikt konvexe Funktionen bekannt, auf denen das Verfahren gegen einen Punkt läuft, der gar kein stationärer Punkt ist (McKinnon 1998). In der Praxis wird Nelder-Mead deshalb gern als robuste Startpunktsuche vor einem gradientenbasierten Verfahren eingesetzt.

In R. Nelder-Mead ist die Voreinstellung von `optim()`; wer nichts angibt, optimiert also ableitungsfrei:

```
# Nelder-Mead ist der Default in optim()!
optim(c(0, 0), f)                # verwendet Nelder-Mead
optim(c(0, 0), f, method = "Nelder-Mead") # explizit
```

Das ist bequem und für viele Modelle ausreichend, erklärt aber auch, warum eine `optim()`-Anpassung mit vielen Parametern manchmal überraschend lange läuft. Wer Ableitungen hat, gibt sie über `gr` an und wählt dazu ein gradientenbasiertes `method`; Abschnitt 12.6 geht die Optionen durch.

Vier Züge, aber wie oft fällt welcher? Das entscheidet über die Kosten des Verfahrens, denn die vier sind verschieden teuer. Sehen wir zu.

Interaktiv: Der Simplex bei der Arbeit

Hier läuft das Verfahren Schritt für Schritt, auf der Testfunktion $f(\mathbf{x}) = (1 - x_1)^2 + 5(x_2 - x_1^2)^2$ mit Minimum in $(1; 1)$. Der orange Strahl markiert jeweils den nächsten Versuchspunkt; der Schrittreger lässt sich vorwärts und rückwärts schieben, der Abspielknopf lässt den ganzen Lauf durchlaufen.

Schätzfrage. Welcher der vier Züge fällt auf dieser Testfunktion am häufigsten?

Lösung: *kontraktion*

Der Zugsähler im Widget führt mit.

Die vier Züge verteilen sich sehr ungleich. Aus dem voreingestellten Simplex braucht das Verfahren vierzig Schritte, bis der beste Eckpunkt unter 10^{-6} liegt, und der Zähler steht dann auf 23 Kontraktionen, 13 Reflexionen und 4 Expansionen; ein Schrumpfschritt, der teuerste Zug, kommt auf diesem Weg gar nicht vor, erst der zweite Startsimplex führt ihn im vierten Schritt vor. Auch die Form verrät, wo der Simplex steht: Auf den geraden Stücken des Tals wird er lang und dünn und legt sich in Laufrichtung, in der Biegung um $x_1 \approx 0$ schrumpft er auf ein fast gleichseitiges Dreieck und dreht sich in die neue Richtung. Diese Anpassung an die lokale Geometrie ersetzt hier die fehlende Ableitung.

Eine sehr sehenswerte Animation derselben Züge, mit einstellbaren Parametern und mehreren Testfunktionen, liegt unter alexadowad.github.io/visualizing-nelder-mead.

Der Gradientenabstieg

Der Gradient zeigt bergauf. Diese eine Beobachtung aus Abschnitt 10.2 reicht schon für ein Optimierungsverfahren: Wenn wir minimieren wollen, laufen wir in die Gegenrichtung.

Algorithmus 12.3.4 (Gradientenabstieg)

Gegeben seien eine differenzierbare Funktion $f: \mathbb{R}^n \rightarrow \mathbb{R}$, ein Startpunkt $\mathbf{x}^{(0)}$ und eine Schrittweite $\gamma > 0$. Für $k = 0, 1, 2, \dots$ setze

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top \quad (12.3.1)$$

und brich ab, sobald eines der Kriterien aus Bemerkung 12.3.17 greift.

Das Transponierte gehört dazu. Der Gradient ist in diesem Skript ein Zeilenvektor (Gradient, Abschnitt 10.2), eine Richtung im $\mathbb{R}^n$ dagegen eine Spalte. In der Sprache des maschinellen Lernens heißt γ *Lernrate* (learning rate), und das Verfahren selbst heißt Gradientenabstieg (gradient descent).

Bemerkung 12.3.5 (Drei Lesarten desselben Schritts)

Steilster Abstieg. Sei $\nabla f(\mathbf{x}) \neq \mathbf{0}^\top$. Unter allen Richtungen $\mathbf{d}$ mit $\|\mathbf{d}\| = 1$ macht $\mathbf{d} = -\nabla f(\mathbf{x})^\top / \|\nabla f(\mathbf{x})\|$ die Richtungsableitung $\nabla f(\mathbf{x})\mathbf{d}$ so klein wie möglich; das ist der Gleichheitsfall der Cauchy-Schwarz-Ungleichung und steht als Satz in Abschnitt 10.2. Lokal gibt es also keine bessere Richtung als die des negativen Gradienten.

Abstiegsrichtung. Für $\nabla f(\mathbf{x}) \neq \mathbf{0}^\top$ ist $\nabla f(\mathbf{x})(-\nabla f(\mathbf{x})^\top) = -\|\nabla f(\mathbf{x})\|^2 < 0$, und nach der Taylorentwicklung erster Ordnung (Abschnitt 10.8) sinkt f deshalb für alle hinreichend kleinen γ tatsächlich. Zwei Zusätze wären dabei zu großzügig. Abstiegsrichtung ist $-\nabla f(\mathbf{x})^\top$ auch ohne Konvexität, solange der Gradient nicht verschwindet – Konvexität wird dafür gar nicht gebraucht. Und *hin zum Minimum* zeigt sie im Allgemeinen gerade nicht, wie das Zickzack-Bild weiter unten vorführt. Was die Konvexität aus Kapitel 11 beisteuert, ist etwas anderes und Wichtigeres: Sie sorgt dafür, dass jeder stationäre Punkt schon ein globales Minimum ist (Abschnitt 12.2), das Verfahren also nirgends sonst hängen bleiben kann.

Fixpunktiteration. Mit $\mathbf{g}(\mathbf{x}) := \mathbf{x} - \gamma \nabla f(\mathbf{x})^\top$ ist (12.3.1) wörtlich die Fixpunktiteration $\mathbf{x}^{(k+1)} = \mathbf{g}(\mathbf{x}^{(k)})$ aus Abschnitt 12.1, und die Fixpunkte von $\mathbf{g}$ sind genau die stationären Punkte von f . Alles, was dort über Schrittweiten und Konvergenzraten steht, gilt deshalb hier weiter; wir bekommen es gleich in schärferer Form, weil ∇f mehr Struktur hat als ein beliebiges $\mathbf{g}$.

Beispiel 12.3.6 (Gradientenabstieg von Hand)

Minimieren wir $f(x) = (x - 2)^2 + 1$ mit Startwert $x^{(0)} = 4,5$ und Schrittweite $\gamma = 0,6$. Die Ableitung ist $f'(x) = 2x - 4$, das Minimum liegt bei $x^* = 2$, denn dort und nur dort verschwindet sie.

$$x^{(0)} = 4,5,$$

$$x^{(1)} = x^{(0)} - 0,6 f'(x^{(0)}) = 4,5 - 0,6 \cdot (2 \cdot 4,5 - 4) = 4,5 - 3 = 1,5,$$

$$x^{(2)} = x^{(1)} - 0,6 f'(x^{(1)}) = 1,5 - 0,6 \cdot (2 \cdot 1,5 - 4) = 1,5 + 0,6 = 2,1,$$

$$x^{(3)} = x^{(2)} - 0,6 f'(x^{(2)}) = 2,1 - 0,6 \cdot (2 \cdot 2,1 - 4) = 2,1 - 0,12 = 1,98.$$

Der erste Schritt schießt weit über das Ziel hinaus, von 4,5 auf 1,5. Danach wird die Iteration schnell brav: Die Fehler $x^{(k)} - x^*$ sind 2,5, -0,5, 0,1 und -0,02. Sie wechseln in jedem Schritt

das Vorzeichen und schrumpfen dabei exakt auf ein Fünftel.

Dass dort der Faktor $-0,2$ steht, ist kein Zufall. Bevor wir den Grund ausrechnen, probieren wir die Schrittweite selbst aus: Welche Werte von γ führen ans Ziel, welche nicht?

Interaktiv: Die Schrittweite am eindimensionalen Beispiel

Die Tafel zeigt Beispiel 12.3.6 mit frei einstellbarem γ . Weil $f'' \equiv 2$ ist, liegen die beiden Schwellen direkt auf dem Reglerraster und lassen sich einzeln ansteuern; der Schrittregler läuft vorwärts wie rückwärts.

Schätzfrage. Ab welchem γ läuft die Iteration davon, statt sich dem Minimum zu nähern?

Lösung: $1/\gamma$

Die Grenze ist $2/L$, hier also $\gamma = 1$. Dort pendelt die Folge; darüber wächst der Fehler.

Die Beobachtungen ordnen sich zu einer vollständigen Fallunterscheidung, und die lohnt sich auszurechnen: Sie erklärt das ganze Verhalten des Verfahrens auf einen Schlag.

Bemerkung 12.3.7 (Zu klein, zu groß, gerade richtig)

Für eine Parabel $f(x) = \frac{L}{2}(x - x^*)^2$ mit Krümmung $L > 0$ ist $f'(x) = L(x - x^*)$, und (12.3.1) wird zu einer Rekursion für den Fehler $e^{(k)} := x^{(k)} - x^*$:

$$e^{(k+1)} = e^{(k)} - \gamma L e^{(k)} = (1 - \gamma L) e^{(k)}, \quad \text{also} \quad e^{(k)} = (1 - \gamma L)^k e^{(0)}. \quad (12.3.2)$$

Den Buchstaben L für die Krümmung wählen wir mit Absicht; Definition 12.3.8 wird zeigen, dass diese Größe die Schrittweite auch im allgemeinen Fall begrenzt. Entschieden ist damit alles, und zwar allein durch den Betrag von $1 - \gamma L$:

- $0 < \gamma < 1/L$: Der Faktor liegt in $(0, 1)$, der Fehler behält sein Vorzeichen und schrumpft. Für sehr kleines γ liegt er dicht an 1, und das Verfahren kriecht.
- $\gamma = 1/L$: Der Faktor ist null, ein einziger Schritt trifft x^* exakt.
- $1/L < \gamma < 2/L$: Der Faktor ist negativ mit Betrag unter 1; die Iterierten springen über das Minimum hinweg und nähern sich trotzdem.
- $\gamma = 2/L$: Der Betrag ist genau 1. Die Folge pendelt für immer zwischen zwei Punkten.
- $\gamma > 2/L$: Der Betrag übersteigt 1, jeder Schritt vergrößert den Fehler, das Verfahren divergiert. Und das, obwohl jeder einzelne Schritt in die richtige Richtung startet.

Im Beispiel 12.3.6 ist $L = 2$, also $1/L = 0,5$ und $2/L = 1$; mit $\gamma = 0,6$ landen wir im dritten Fall und lesen $1 - 0,6 \cdot 2 = -0,2$ ab. Das sind genau die fünf Bilder, die das Widget oben liefert: bei $\gamma = 0,2$ die zähe einseitige Annäherung, bei $\gamma = 0,5$ der Treffer im ersten Schritt (es ist der Newton-Schritt aus Abschnitt 10.8), bei $\gamma = 0,6$ die Zahlen der Tabelle, bei $\gamma = 1$ das ewige Pendeln und darüber die Flucht aus dem Bild. Das ist das Dilemma der Schrittweite in Reinform, dem wir schon bei der Fixpunktiteration in Abschnitt 12.1 begegnet sind: klein genug für Konvergenz, groß genug für Tempo. Nur wird es im $\mathbb{R}^n$ noch unangenehmer, denn dort gibt es einen solchen Faktor je Krümmungsrichtung, und ein einziges γ muss sie alle zugleich bedienen.

Zwischenfrage: ein Schritt im $\mathbb{R}^2$

Sei $f(\mathbf{x}) = x_1^2 + x_2^2$ und $\mathbf{x}^{(0)} = (4, 3)^\top$ mit Schrittweite $\gamma = 0,25$. Wo liegt $\mathbf{x}^{(1)}$ nach einem Schritt des Gradientenabstiegs? Vier Vorschläge, jeder einzeln zu beurteilen.

1. $\mathbf{x}^{(1)} = (2; 1,5)^\top$.

Antwort: Wahr

Der Gradient ist $\nabla f(\mathbf{x}) = (2x_1, 2x_2)$, an der Stelle $(4, 3)^\top$ also $\nabla f(\mathbf{x}^{(0)}) = (8, 6)$. Eingesetzt in (12.3.1): $\mathbf{x}^{(1)} = (4, 3)^\top - 0,25 \cdot (8, 6)^\top = (4, 3)^\top - (2; 1,5)^\top = (2; 1,5)^\top$.

2. $\mathbf{x}^{(1)} = (0, 0)^\top$, das Verfahren ist sofort fertig.

Antwort: Falsch

Das wäre der Treffer in einem Schritt, und den gibt es hier bei $\gamma = 1/L = 0,5$, nicht bei 0,25. Die Hesse-Matrix ist $\mathbf{H}_f = \text{diag}(2, 2)$, also $L = 2$; mit $\gamma = 0,25$ ist der Fehlerfaktor $1 - \gamma L = 0,5$, und die Iterierten halbieren nur ihren Abstand zum Ursprung: $(2; 1,5)$, $(1; 0,75)$, $(0,5; 0,375)$.

3. $\mathbf{x}^{(1)} = (3, 2)^\top$.

Antwort: Falsch

Hier ist von jeder Komponente 1 abgezogen worden, der Schritt hätte also in beiden Richtungen dieselbe Länge. Der Gradientenabstieg skaliert die Richtung aber nicht auf Einheitslänge; er nimmt den Gradienten so, wie er ist, und die Komponente mit dem größeren Betrag bekommt den längeren Schritt.

4. $\mathbf{x}^{(1)} = (8, 6)^\top$.

Antwort: Falsch

Das ist der Gradient selbst. Er ist der Bauteil des Schritts, nicht das Ergebnis; abgesehen davon zeigt $+\nabla f$ bergauf, und $(8, 6)^\top$ liegt weiter vom Minimum entfernt als der Startpunkt.

Wie schnell ist der Gradientenabstieg?

Bemerkung 12.3.7 hat die Schrittweite an die Krümmung gekoppelt. Für eine allgemeine Funktion brauchen wir dafür einen Begriff, der die Krümmung nach oben begrenzt, ohne zweite Ableitungen zu verlangen.

Definition 12.3.8 (Lipschitz-Stetigkeit)

Eine Funktion $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ heißt *Lipschitz-stetig* (Lipschitz continuous) mit *Lipschitz-Konstante* $L > 0$, falls

$$\|f(\mathbf{x}) - f(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\| \quad \text{für alle } \mathbf{x}, \mathbf{y} \in \mathbb{R}^n.$$

Anschaulich: Die Funktionswerte ändern sich höchstens L -mal so schnell wie die Argumente. Je größer L , desto zackiger darf f verlaufen. Lipschitz-Stetigkeit ist deutlich stärker als bloße Stetigkeit, mit Differenzierbarkeit aber ohne Zusatzannahmen nicht vergleichbar: $f(x) = |x|$ ist mit $L = 1$ Lipschitz-stetig und im Nullpunkt trotzdem nicht differenzierbar; $f(x) = x^2$ ist auf $\mathbb{R}$ differenzierbar, aber nicht global Lipschitz-stetig. Auf einem kompakten Intervall ist eine stetig differenzierbare Funktion wegen ihrer dort beschränkten Ableitung Lipschitz-stetig.

Für den Gradientenabstieg brauchen wir die Bedingung nicht für f , sondern für den Gradienten.

Bemerkung 12.3.9 (Lipschitz-stetiger Gradient und die Spektralnorm)

Wir nennen ∇f *Lipschitz-stetig mit Konstante L* , wenn Definition 12.3.8 auf die Abbildung $\mathbf{x} \mapsto \nabla f(\mathbf{x})^\top \in \mathbb{R}^n$ zutrifft:

$$\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|.$$

Die Steigung von f ändert sich also nicht zu abrupt, die Krümmung ist beschränkt. Ist f auf einer konvexen offenen Menge zweimal stetig differenzierbar, so ist das gleichwertig zu

$$\|\mathbf{H}_f(\mathbf{x})\|_2 \leq L \quad \text{für alle } \mathbf{x}, \quad \text{also} \quad L_{\min} = \sup_{\mathbf{x}} \|\mathbf{H}_f(\mathbf{x})\|_2 = \sup_{\mathbf{x}} \max_i |\lambda_i(\mathbf{H}_f(\mathbf{x}))|, \quad (12.3.3)$$

wobei $L_{\min}$ die kleinste mögliche Lipschitz-Konstante bezeichnet; jede größere Zahl ist ebenfalls eine gültige Konstante.

Denn für symmetrische Matrizen ist die Spektralnorm der betragsgrößte Eigenwert. Satz 3.3.7 in Abschnitt 3.3 berechnet $\|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}$; für symmetrisches $\mathbf{A}$ ist $\mathbf{A}^\top \mathbf{A} = \mathbf{A}^2$, und der Spektralsatz gibt dieser Matrix die Eigenwerte λ_i^2 , also $\|\mathbf{A}\|_2 = \max_i |\lambda_i(\mathbf{A})|$. Dieselbe Umrechnung steckt hinter der Formel für κ_2 symmetrischer Matrizen in Abschnitt 3.5.

Auf den Betrag kommt es an. Die kürzere Schreibweise $L = \sup_{\mathbf{x}} \lambda_{\max}(\mathbf{H}_f(\mathbf{x}))$ geht nur gut, solange f konvex ist, denn dann ist $\mathbf{H}_f$ positiv semidefinit und der größte Eigenwert zugleich der betragsgrößte. Für nicht konvexes f ist sie falsch: Für $f(x) = -x^2$ ist $f'(x) = -2x$ mit der exakten Lipschitz-Konstanten 2, während $\lambda_{\max}(\mathbf{H}_f) = -2$ nicht einmal positiv ist. Im Mehrdimensionalen genügt schon eine Sattelrichtung: Bei $\mathbf{H}_f = \text{diag}(0,5; -8)$ ist $L = 8$, das Supremum der größten Eigenwerte aber 0,5. Gerade in der nicht konvexen Optimierung, wo die Schrittweite an L hängt, wäre das ein Faktor 16 zu viel Vertrauen.

Satz 12.3.10 (Konvergenzrate bei konvexem f)

Sei $f : \mathbb{R}^n \rightarrow \mathbb{R}$ konvex und differenzierbar mit Lipschitz-stetigem Gradienten und Lipschitz-Konstante L , und f nehme sein Minimum in $\mathbf{x}^*$ an. Dann gilt für den Gradientenabstieg (12.3.1) mit fester Schrittweite $\gamma \leq 1/L$

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*) \leq \frac{\|\mathbf{x}^{(0)} - \mathbf{x}^*\|^2}{2\gamma k}. \quad (12.3.4)$$

Bemerkung 12.3.11 (Was eine Rate der Ordnung $1/k$ praktisch heißt)

Die Schranke fällt wie $1/k$. Um den Fehler unter ε zu drücken, brauchen wir also $k \sim 1/\varepsilon$ Schritte, und das ist wenig erfreulich: Eine Dezimalstelle mehr Genauigkeit kostet die zehnfache Iterationszahl, der Weg von 10^{-2} auf 10^{-6} das Zehntausendfache. Zum Vergleich hat das Newton-Verfahren aus Abschnitt 12.1 in seinem Einzugsbereich die Zahl der korrekten Stellen in jedem Schritt verdoppelt.

Zwei Feinheiten sind der Formulierung wert. Erstens misst (12.3.4) den Fehler im Funktionswert, nicht den Abstand zu $\mathbf{x}^*$; die beiden können weit auseinanderliegen, wie Bemerkung 12.3.17 gleich beziffert. Zweitens setzt der Satz voraus, dass ein Minimierer überhaupt existiert. Für $f(x) = \sqrt{1+x^2} - x$ ist alles andere erfüllt, denn $f''(x) = (1+x^2)^{-3/2} \leq 1$ macht f konvex mit $L = 1$; das Infimum 0 wird aber nirgends angenommen, und der Gradientenabstieg läuft brav, aber ergebnislos nach rechts.

Schneller wird es, wenn die Krümmung nicht nur nach oben, sondern auch nach unten beschränkt ist. Die Funktion darf dann nirgends flach werden.

Definition 12.3.12 (Starke Konvexität)

Eine zweimal stetig differenzierbare Funktion $f : \mathbb{R}^n \rightarrow \mathbb{R}$ heißt *stark konvex* (μ -strongly convex) mit Parameter $\mu > 0$, falls

$$\mathbf{H}_f(\mathbf{x}) - \mu \mathbf{I} \succeq 0 \quad \text{für alle } \mathbf{x},$$

also $\lambda_{\min}(\mathbf{H}_f(\mathbf{x})) \geq \mu$ für alle $\mathbf{x}$. Der größte solche Wert ist $\mu = \inf_{\mathbf{x}} \lambda_{\min}(\mathbf{H}_f(\mathbf{x}))$, die kleinste vorkommende Krümmung.

Ohne Ableitungen formuliert heißt dasselbe: $\mathbf{x} \mapsto f(\mathbf{x}) - \frac{\mu}{2} \|\mathbf{x}\|^2$ ist konvex. In dieser Fassung braucht die Definition keine zweiten Ableitungen, und sie zeigt zugleich, was starke Konvexität aussagt: Unter f steckt an jeder Stelle eine Parabel mit Krümmung mindestens μ .

Satz 12.3.13 (Konvergenzrate bei starker Konvexität)

Sei f stark konvex mit Parameter $\mu > 0$ und habe einen Lipschitz-stetigen Gradienten mit Konstante L . Dann gilt für den Gradientenabstieg (12.3.1) mit Schrittweite $\gamma \leq 1/L$

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*) \leq \rho^k \cdot (f(\mathbf{x}^{(0)}) - f(\mathbf{x}^*)), \quad \rho = 1 - \gamma\mu < 1, \quad (12.3.5)$$

insbesondere $\rho = 1 - \mu/L$ für $\gamma = 1/L$.

Der Gewinn gegenüber Satz 12.3.10 ist beträchtlich. Dort brauchten wir $k \sim 1/\varepsilon$ Schritte, hier genügen wegen $\rho^k < \varepsilon$ schon $k \sim \log(1/\varepsilon)$: Jede weitere Dezimalstelle kostet nicht mehr das Zehnfache, sondern denselben festen Zuschlag. Das ist derselbe Unterschied wie zwischen linearer und exponentieller Fehlerreduktion in Abschnitt 12.1.

Die Existenz von $\mathbf{x}^*$ muss dieser Satz nicht mehr eigens voraussetzen. Starke Konvexität legt unter f an jeder Stelle eine Parabel mit Krümmung $\mu > 0$, und eine solche Parabel wächst in jeder Richtung über alle Grenzen; ein Minimum wird also angenommen, und weil f dann auch strikt konvex ist, gibt es nur dieses eine (Abschnitt 11.5).

Die Schrittweite gehört dabei zwingend zur Rate. Mit einem sehr kleinen γ bleibt zwar die Konvergenz, aber eben mit $1 - \gamma\mu$ statt mit $1 - \mu/L$; die Rate $\rho = 1 - \mu/L$ ist die beste, die dieser Satz hergibt, und sie gehört zu $\gamma = 1/L$.

Bemerkung 12.3.14 (Die Konditionszahl einer Funktion)

Die beiden Konstanten spannen die Krümmung von beiden Seiten ein,

$$\mu \mathbf{I} \preceq \mathbf{H}_f(\mathbf{x}) \preceq L \mathbf{I} \quad \text{für alle } \mathbf{x},$$

wobei die obere Hälfte für konvexes f gerade (12.3.3) ist. Das Verhältnis der beiden Konstanten heißt *Konditionszahl* der Funktion,

$$\kappa_f = \frac{L}{\mu} = \frac{\sup_{\mathbf{x}} \lambda_{\max}(\mathbf{H}_f(\mathbf{x}))}{\inf_{\mathbf{x}} \lambda_{\min}(\mathbf{H}_f(\mathbf{x}))}, \quad \text{damit} \quad \rho = 1 - \frac{\mu}{L} = \frac{\kappa_f - 1}{\kappa_f}.$$

Der Name ist mit Bedacht gewählt. Für eine Quadrik $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^\top \mathbf{A} \mathbf{x}$ mit positiv definitem $\mathbf{A}$ ist $\mathbf{H}_f = \mathbf{A}$ überall, also $L = \lambda_{\max}(\mathbf{A})$, $\mu = \lambda_{\min}(\mathbf{A})$ und

$$\kappa_f = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})} = \kappa_2(\mathbf{A}),$$

genau die Konditionszahl der Matrix aus Abschnitt 3.5. Für allgemeines f wandert die Hesse-Matrix mit $\mathbf{x}$, und κ_f misst das Verhältnis der extremen Krümmungen über den ganzen Definitionsbereich hinweg.

Wie teuer ein großes κ_f ist, lässt sich beziffern. Aus $\rho^k \leq 10^{-1}$ folgt $k \geq \ln 10 / (-\ln \rho)$, und weil $-\ln(1 - 1/\kappa_f) \approx 1/\kappa_f$ ist, wächst diese Zahl ungefähr proportional zu κ_f : Nach dieser Schranke kostet eine Dezimalstelle bei $\kappa_f = 10$ rund 21,9 Schritte, bei $\kappa_f = 20$ schon 44,9 und bei $\kappa_f = 100$ ganze 229,1. Geometrisch heißt großes κ_f , dass die Höhenlinien lange, schmale Ellipsen sind; bei einer Quadrik gilt das überall, sonst in der Nähe eines Minimums, wo die Hesse-Matrix positiv definit ist. In solchen Canyons läuft der Gradientenabstieg im Zickzack: Er steht quer zum Tal, statt es entlangzulaufen.

Warum das so ist, lässt sich auf einer Quadrik vollständig ausrechnen. Der folgende Satz behandelt diesen Sonderfall schärfer als Satz 12.3.13: Er sagt, was tatsächlich passiert, statt nur eine Schranke zu geben, und er deckt jede Schrittweite unterhalb der Divergenzgrenze ab.

Satz 12.3.15 (Gradientenabstieg auf einer Quadrik)

Sei $\mathbf{A} \in \mathbb{R}^{n \times n}$ symmetrisch und positiv definit mit Eigenwerten $0 < \mu = \lambda_1 \leq \dots \leq \lambda_n = L$, und sei $f(\mathbf{x}) = \frac{1}{2}(\mathbf{x} - \mathbf{x}^*)^\top \mathbf{A}(\mathbf{x} - \mathbf{x}^*)$. Dann gilt für den Gradientenabstieg mit fester Schrittweite γ und den Fehler $\mathbf{e}^{(k)} := \mathbf{x}^{(k)} - \mathbf{x}^*$

$$\mathbf{e}^{(k)} = (\mathbf{I} - \gamma \mathbf{A})^k \mathbf{e}^{(0)} \quad \text{und} \quad f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*) \leq q^{2k} (f(\mathbf{x}^{(0)}) - f(\mathbf{x}^*)) \quad (12.3.6)$$

mit $q := \max_i |1 - \gamma \lambda_i|$. Die Iteration konvergiert genau dann für jeden Startpunkt, wenn $q < 1$ ist, und das trifft für $0 < \gamma < 2/L$ zu.

Vertiefung: Beweis: Fehlerzerlegung nach Eigenrichtungen

Beweis.

- (1) *Fehlerrekursion und Spektralzerlegung.* Der Gradient der quadratischen Form ist $\nabla f(\mathbf{x}) = (\mathbf{x} - \mathbf{x}^*)^\top \mathbf{A}$. Einsetzen in (12.3.1) und Abziehen von $\mathbf{x}^*$ auf beiden Seiten gibt $\mathbf{e}^{(k+1)} = (\mathbf{I} - \gamma \mathbf{A}) \mathbf{e}^{(k)}$, also nach k Schritten die erste Aussage von (12.3.6). Weil $\mathbf{A}$ symmetrisch ist, liefert der Spektralsatz eine Orthonormalbasis $\mathbf{v}_1, \dots, \mathbf{v}_n$ aus Eigenvektoren, und in ihr zerfällt der Fehler in n unabhängige Anteile: Mit $\mathbf{e}^{(0)} = \sum_i c_i \mathbf{v}_i$ ist

$$(\mathbf{I} - \gamma \mathbf{A}) \mathbf{v}_i = (1 - \gamma \lambda_i) \mathbf{v}_i \quad \implies \quad \mathbf{e}^{(k)} = \sum_{i=1}^n (1 - \gamma \lambda_i)^k c_i \mathbf{v}_i.$$

Jede Eigenrichtung hat also ihren eigenen Fehlerfaktor $1 - \gamma \lambda_i$; (12.3.2) ist der Fall $n = 1$.

der Gradient der quadratischen Form $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ ist für symmetrisches $\mathbf{A}$ gerade $2\mathbf{x}^\top \mathbf{A}$ (Abschnitt 10.6), und der Faktor $\frac{1}{2}$ kürzt die 2 weg; der Fehler erfüllt dieselbe Rekursion wie die Iterierten, weil $\mathbf{x}^$ konstant ist. $\mathbf{I} - \gamma \mathbf{A}$ hat dieselben Eigenvektoren wie $\mathbf{A}$, mit den Eigenwerten $1 - \gamma \lambda_i$*

- (2) *Der Funktionswert.* In derselben Basis ist wegen $\mathbf{A} \mathbf{v}_i = \lambda_i \mathbf{v}_i$ und $\mathbf{v}_i^\top \mathbf{v}_j = \delta_{ij}$

$$f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*) = \frac{1}{2} \mathbf{e}^{(k)\top} \mathbf{A} \mathbf{e}^{(k)} = \frac{1}{2} \sum_{i=1}^n \lambda_i (1 - \gamma \lambda_i)^{2k} c_i^2 \leq q^{2k} \cdot \frac{1}{2} \sum_{i=1}^n \lambda_i c_i^2,$$

und $\frac{1}{2} \sum_i \lambda_i c_i^2$ ist gerade $f(\mathbf{x}^{(0)}) - f(\mathbf{x}^*)$. Das ist die zweite Aussage von (12.3.6).

Jeder einzelne Faktor $(1 - \gamma\lambda_i)^{2k}$ ist höchstens q^{2k} , und alle Summanden $\lambda_i c_i^2$ sind wegen $\lambda_i > 0$ nichtnegativ, dürfen also mit demselben Faktor abgeschätzt werden

- (3) Die Bedingung an γ . Es ist $q < 1$ genau dann, wenn $0 < \gamma\lambda_i < 2$ für alle i gilt, und weil alle λ_i positiv sind und L der größte ist, ist das gleichwertig zu $0 < \gamma < 2/L$. Ist umgekehrt $q \geq 1$, so fällt die Komponente in der zugehörigen Eigenrichtung nicht mehr gegen null.

Die kritische Bedingung stammt immer vom GRÖSSTEN Eigenwert: $\gamma\lambda_i < 2$ für alle i ist dasselbe wie $\gamma L < 2$

□

Vertiefung: Schranke und tatsächliche Rate an einem Zahlenbeispiel

Beispiel 12.3.16 (Warum der Satz eine Schranke ist und keine Formel)

Nehmen wir $A = \text{diag}(1, 10)$, also $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10x_2^2)$, mit $\mathbf{x}^* = \mathbf{0}$. Dann ist $\mu = 1$, $L = 10$ und $\kappa_f = 10$.

Mit $\gamma = 1/L = 0,1$ sind die beiden Fehlerfaktoren $1 - \gamma\mu = 0,9$ und $1 - \gamma L = 0$. Die steile Richtung wird also in einem einzigen Schritt exakt getroffen, danach lebt der Fehler nur noch in der flachen Richtung und wird je Schritt mit 0,9 multipliziert. Nach Satz 12.3.15 fällt der Funktionswert deshalb ab dem zweiten Schritt auf exakt das $0,9^2 = 0,81$ -fache. Satz 12.3.13 verspricht an derselben Stelle nur $\rho = 1 - \mu/L = 0,9$. Beides ist richtig, denn (12.3.5) ist eine obere Schranke: Der beobachtete Verlauf darf schneller sein, und hier ist er es um genau den Faktor ρ je Schritt. Wer aus dem Satz „der Fehler fällt genau wie ρ^k “ liest, liest zu viel hinein.

Mit $\gamma = 0,18$ dagegen sind die Faktoren 0,82 und $-0,8$: Die steile Richtung wechselt in jedem Schritt das Vorzeichen, die flache kriecht. Das ist das Zickzack aus Bemerkung 12.3.14, und der Grund steht daneben: Die Schrittweite ist an die steilste Richtung gebunden, denn dort droht Divergenz; vorankommen müssen wir aber in der flachsten.

Mit $\gamma = 0,2 = 2/L$ pendelt die zweite Komponente zwischen zwei Werten, ohne kleiner zu werden, und darüber läuft sie davon.

Wie teuer wird eine schlechte Kondition also wirklich? Das lässt sich zählen.

Interaktiv: Zickzack im Canyon

Die obere Tafel zeigt die Höhenlinien von $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + \kappa x_2^2)$ und den Weg der Iterierten, die mittlere den Verlauf von $f(\mathbf{x}^{(k)}) - f(\mathbf{x}^*)$ auf logarithmischer Skala, daneben steht dieselbe Quadrik als Fläche. Der Regler für γ ist in Vielfachen von $2/L$ geeicht, damit die Schwellen unabhängig von κ an derselben Stelle liegen.

Schätzfrage. Bei $\kappa = 10$ braucht der Abstieg in der Voreinstellung 35 Schritte, bis f auf ein Millionstel gefallen ist. Wie viele bei $\kappa = 100$?

Lösung: 336 Schritte

Der Aufwand wächst proportional zu κ ; die Ablesetafel des Widgets nennt die Zahl.

Der Aufwand wächst ungefähr proportional zur Kondition: bei $\kappa = 10$ sind es 35 Schritte bis zum Millionstel, bei $\kappa = 25$ schon 88 und bei $\kappa = 100$ dann 336. Bei $\kappa = 1$ sind die Höhenlinien Kreise, der negative Gradient zeigt direkt auf das Minimum, und mit einem halben Anteil ist das Verfahren nach einem Schritt fertig. Bei $\kappa = 10$ und $\gamma = 1/L$ stehen dieselben Zahlen auf der Tafel: gemessener Quotient 0,81 gegen die gestrichelte Schranke 0,9. Und bei einem Anteil um 0,9 entsteht das Zickzack; die Schranke wird dann gar nicht mehr gezeichnet, denn Satz 12.3.13 setzt $\gamma \leq 1/L$ voraus.

Wann hören wir auf?

Die Iteration (12.3.1) läuft von sich aus ewig. In der Praxis brauchen wir eine Regel, wann wir sie beenden, und alle brauchbaren Regeln messen etwas, das wir tatsächlich ausrechnen können. Der Abstand $\|\mathbf{x}^{(k)} - \mathbf{x}^*\|$ gehört nicht dazu, denn $\mathbf{x}^*$ ist ja gerade das Gesuchte.

Bemerkung 12.3.17 (Drei Abbruchkriterien und ihre Grenzen)

1. Der Gradient ist klein: $\|\nabla f(\mathbf{x}^{(k)})\| < \varepsilon_{\text{grad}}$.
2. Der Funktionswert stagniert: $|f(\mathbf{x}^{(k+1)}) - f(\mathbf{x}^{(k)})|/|f(\mathbf{x}^{(k)})| < \varepsilon_f$.
3. Das Budget ist aufgebraucht: $k > k_{\text{max}}$.

Üblich sind $\varepsilon_{\text{grad}} \approx 10^{-6}$ bis 10^{-8} und $k_{\text{max}} \approx 1000$ bis $10\,000$, und üblich ist es auch, mehrere Kriterien zu kombinieren: Die ersten beiden sagen etwas über die Lösung, das dritte verhindert eine Endlosschleife, wenn keines der anderen greift.

Vorsicht. Bei schlecht konditionierten oder nicht stark konvexen Problemen können die Kriterien versagen. Für stark konvexes f gilt zwar $\|\nabla f(\mathbf{x})\| \geq \mu \|\mathbf{x} - \mathbf{x}^*\|$ und damit $\|\mathbf{x} - \mathbf{x}^*\| \leq \|\nabla f(\mathbf{x})\|/\mu$, doch diese Schranke ist nur so gut wie μ . Ein Beispiel: $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10^{-8}x_2^2)$ hat an der Stelle $\mathbf{x} = (0; 10)^\top$ den Gradienten $\nabla f = (0; 10^{-7})$, unterschreitet also jede Gradientenschranke ab 10^{-6} , und liegt trotzdem 10 Einheiten vom Minimum entfernt. Der Funktionswert ist dort allerdings schon fast optimal, nämlich $f - f^* = 5 \cdot 10^{-7}$. Das ist keine Ausrede, sondern die ehrliche Auskunft: In einer flachen Richtung ist die Stelle des Minimums schlecht bestimmt, sein Wert dagegen gut.

Das zweite Kriterium ist noch schwächer. Ein kleiner Fortschritt kann daran liegen, dass wir angekommen sind, aber genauso daran, dass γ zu klein gewählt ist; und ist $f(\mathbf{x}^{(k)})$ nahe null, so ist der relative Quotient gar nicht mehr aussagekräftig. Wer es ernst meint, prüft am Ende zusätzlich, ob der Gradient wirklich klein *relativ zur Skala des Problems* ist.

Schrittweitsuche: die Armijo-Bedingung

Eine feste Schrittweite ist immer ein Kompromiss. Bemerkung 12.3.7 hat gezeigt, wie schmal der brauchbare Bereich sein kann, und L kennen wir in der Regel nicht. Naheliegender ist deshalb, γ in jedem Schritt neu zu bestimmen: klein genug, dass es sicher bergab geht, groß genug, dass sich der Schritt lohnt. Diese Suche heißt *Liniensuche* (line search), weil sie sich mit der Suchrichtung $\mathbf{d}$ auf der Geraden $\gamma \mapsto \mathbf{x}^{(k)} + \gamma \mathbf{d}$ abspielt.

Den exakten Minimierer auf dieser Geraden zu bestimmen, wäre wieder ein Optimierungsproblem. Billiger und in der Praxis besser ist eine Bedingung, die nur „genügend Abstieg“ verlangt. Ein Warnhinweis zur Notation vorweg: Der Buchstabe ρ steht im folgenden Algorithmus für den Faktor, um den die Schrittweite verkleinert wird, und hat mit der Konvergenzrate ρ aus Satz 12.3.13 nichts zu tun. Beide Bezeichnungen sind eingebürgert, und wir behalten sie bei.

Algorithmus 12.3.18 (Backtracking-Liniensuche nach Armijo)

Gegeben seien die aktuelle Stelle $\mathbf{x}^{(k)}$, die Suchrichtung $\mathbf{d} = -\nabla f(\mathbf{x}^{(k)})^\top$ und Parameter $c \in (0, 1)$ sowie $\rho \in (0, 1)$.

1. Starte mit $\gamma = 1$.
2. Solange

$$f(\mathbf{x}^{(k)} + \gamma \mathbf{d}) > f(\mathbf{x}^{(k)}) + c\gamma \nabla f(\mathbf{x}^{(k)})^\top \mathbf{d}, \quad (12.3.7)$$

setze $\gamma \leftarrow \rho \gamma$.

3. Gib γ zurück und setze $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \gamma \mathbf{d}$.

Bemerkung 12.3.19 (Was die Armijo-Bedingung fordert)

Das Vorzeichen ist der Kern. Mit $\mathbf{d} = -\nabla f(\mathbf{x}^{(k)})^\top$ ist

$$\nabla f(\mathbf{x}^{(k)})^\top \mathbf{d} = -\|\nabla f(\mathbf{x}^{(k)})\|^2 < 0 \quad \text{für } \nabla f(\mathbf{x}^{(k)}) \neq \mathbf{0}^\top,$$

der Zusatzterm $c\gamma \nabla f(\mathbf{x}^{(k)})^\top \mathbf{d}$ auf der rechten Seite von (12.3.7) ist also *negativ*. Verschwindet der Gradient, gibt es überhaupt keine Suchrichtung mehr, und die Abbruchkriterien haben längst gegriffen. Die rechte Seite liegt damit unter dem alten Funktionswert, und darauf zielt die Bedingung: Der Funktionswert muss nicht bloß sinken, sondern um einen Betrag sinken, der proportional zu γ und zum Quadrat der Gradientennorm ist. Wo es steil bergab geht, wird viel verlangt, wo es flach ist, wenig.

Die geometrische Lesart. Die Gerade $\gamma \mapsto f(\mathbf{x}^{(k)}) + \gamma \nabla f(\mathbf{x}^{(k)})^\top \mathbf{d}$ ist die Tangente an den Schnitt $\varphi(\gamma) = f(\mathbf{x}^{(k)} + \gamma \mathbf{d})$ im Nullpunkt, also das Beste, was die erste Ordnung überhaupt verspricht. Die Armijo-Gerade ist dieselbe Gerade, um den Faktor c flacher gelegt. Gefordert wird, unter dieser flacheren Geraden zu bleiben, und je kleiner c , desto bescheidener die Forderung.

Warum die Schleife endet. Nach der Taylorentwicklung erster Ordnung ist $\varphi(\gamma) = \varphi(0) + \gamma\varphi'(0) + o(\gamma)$ mit $\varphi'(0) = \nabla f(\mathbf{x}^{(k)})^\top \mathbf{d} < 0$. Für kleine γ ist die linke Seite von (12.3.7) deshalb ungefähr $\varphi(0) + \gamma\varphi'(0)$, und das liegt unter der rechten Seite, weil $\gamma\varphi'(0) < c\gamma\varphi'(0)$ für $c < 1$ und $\varphi'(0) < 0$ gilt. Solange der Gradient nicht verschwindet, bricht die Schleife also nach endlich vielen Verkleinerungen ab.

Typische Parameter. $c = 10^{-4}$ und $\rho = 0,5$, die Schrittweite wird also halbiert. Mit so kleinem c ist die Armijo-Gerade praktisch waagerecht, und die Bedingung heißt kaum mehr als „der Funktionswert muss wirklich sinken“; gerade das macht sie robust, denn verworfen wird nur ein Schritt, der über das Ziel hinausschießt.

Ausblick. Die Liniensuche ist kein Zubehör für den Gradientenabstieg allein. Quasi-Newton-Verfahren wie BFGS bestimmen ihre Suchrichtung anders, benutzen darüber aber dieselbe Backtracking-Schleife; Abschnitt 12.4 baut darauf auf.

Wie oft muss die Schleife also halbieren, bis sie fündig wird? Sehen wir am Schnitt nach.

Interaktiv: Die Armijo-Bedingung am Schnitt

Hier die drei Kurven zum Selberschieben: der Schnitt $\varphi(\gamma)$ in Violett, die Tangente in Orange und die Armijo-Gerade gestrichelt in Grau. Die roten Punkte sind verworfene Probeschritte, der blaue ist der akzeptierte. Als Zielfunktion dient die Quadrik $f(\mathbf{x}) = \frac{1}{2}x_1^2 + \frac{5}{2}x_2^2$ mit $\kappa_f = 5$, als Suchrichtung wie immer $\mathbf{d} = -\nabla f(\mathbf{x})^\top$.

Interaktives Element — nur in der Web-Fassung

Ein bis zwei Halbierungen genügen. In der Voreinstellung wird $\gamma = 1$ verworfen und $\gamma = 0,5$ angenommen, obwohl der exakte Minimierer bei $\gamma^* = 1/3$ läge; mit dem Lehrbuchwert $c = 0,3$ ist eine zweite Halbierung nötig, und $\gamma = 0,25$ wird angenommen. Auch das gehört zur

Idee: Der billige Kompromiss ist gut genug, weil ohnehin ein nächster Schritt folgt. Wer c nach rechts schiebt, sieht die Forderung steiler werden und die Zahl der Verkleinerungen steigen. Über alle Reglerstellungen hinweg sind höchstens 16 Halbierungen nötig – die Schleife endet also nicht nur theoretisch, sondern auch schnell.

Selbsttest

1. Auf der Quadrik mit $\mathbf{H}_f = \text{diag}(1, 10)$ fällt der Funktionswert mit $\gamma = 1/L$ in jedem Schritt genau auf das $\rho = 0,9$ -fache.

Antwort: Falsch

(12.3.5) ist eine obere Schranke, keine Gleichung. Tatsächlich fällt der Wert ab dem zweiten Schritt auf das 0,81-fache, also auf das ρ^2 -fache: Die steile Richtung ist wegen $1 - \gamma L = 0$ schon nach dem ersten Schritt erledigt, und in der flachen schrumpft der Fehler mit 0,9, der Funktionswert also mit $0,9^2$.

2. Auch für nicht konvexes f ist $-\nabla f(\mathbf{x})^\top$ eine Abstiegsrichtung, solange $\nabla f(\mathbf{x}) \neq \mathbf{0}^\top$ ist.

Antwort: Wahr

Es ist $\nabla f(\mathbf{x})(-\nabla f(\mathbf{x})^\top) = -\|\nabla f(\mathbf{x})\|^2 < 0$, und daraus folgt mit der Taylorentwicklung erster Ordnung, dass f für hinreichend kleine Schritte sinkt. Konvexität wird dafür nicht gebraucht. Was sie liefert, ist etwas anderes: dass jeder stationäre Punkt, an dem das Verfahren zur Ruhe kommt, schon global optimal ist (Bemerkung 12.3.5).

3. Ist $\|\nabla f(\mathbf{x}^{(k)})\| < 10^{-6}$, so liegt $\mathbf{x}^{(k)}$ nahe am Minimum.

Antwort: Falsch

Beide Schranken, die es dazu gibt, hängen an der kleinsten Krümmung μ : $\|\mathbf{x} - \mathbf{x}^*\| \leq \|\nabla f(\mathbf{x})\| / \mu$ für die Stelle und $f(\mathbf{x}) - f(\mathbf{x}^*) \leq \|\nabla f(\mathbf{x})\|^2 / (2\mu)$ für den Wert. Bei kleinem μ taugt vor allem die erste nichts. Für $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + 10^{-8}x_2^2)$ ist der Gradient in $(0; 10)^\top$ nur 10^{-7} lang, der Abstand zum Minimum aber 10, während der Funktionswert mit $5 \cdot 10^{-7}$ tatsächlich schon fast optimal ist (Bemerkung 12.3.17).

4. Für einen Lipschitz-stetigen Gradienten genügt $L = \sup_{\mathbf{x}} \lambda_{\max}(\mathbf{H}_f(\mathbf{x}))$ als Konstante.

Antwort: Falsch

Gebraucht wird die Spektralnorm, also der betragsgrößte Eigenwert $\sup_{\mathbf{x}} \max_i |\lambda_i|$. Bei konvexem f fällt beides zusammen, weil die Hesse-Matrix dann positiv semidefinit ist; sonst nicht. Für $f(x) = -x^2$ ist die exakte Konstante 2, während der größte Eigenwert -2 beträgt (Bemerkung 12.3.9).

5. Aus dem voreingestellten Simplex fällt im Nelder-Mead-Widget in vierzig Schritten kein einziger Schrumpfschritt.

Antwort: Wahr

Der Zugzähler steht nach vierzig Schritten auf 13 Reflexionen, 4 Expansionen, 23 Kontraktionen und null Schrumpfschritten. Der Schrumpfschritt ist der teuerste der vier Züge aus Algorithmus 12.3.2, und dass er hier gar nicht gebraucht wird, ist der Grund für die verhältnismäßig guten Kosten. Erst der zweite Startsimplex („flach von links“) erzwingt ihn, und zwar im vierten Schritt.

6. Nelder-Mead braucht pro Iteration $O(n^2)$ Funktionsauswertungen.

Antwort: Falsch

Es sind $O(n)$. Der Simplex hat $n + 1$ Ecken, aber in einem gewöhnlichen Schritt werden nur ein oder zwei neue Punkte ausgewertet; erst der Schrumpfschritt kostet n neue Ecken. Langsam wird das Verfahren in hoher Dimension nicht wegen der Kosten pro Iteration, sondern wegen der Zahl der Iterationen (Bemerkung 12.3.3).

7. Verdoppeln wir κ_f , so verdoppelt sich ungefähr die Schrittzahl, die Satz 12.3.13 für dieselbe Genauigkeit garantiert.

Antwort: Wahr

Die garantierte Zahl der Schritte je Dezimalstelle ist $\ln 10 / (-\ln \rho)$ mit $\rho = 1 - 1/\kappa_f$, und wegen $-\ln(1 - 1/\kappa_f) \approx 1/\kappa_f$ wächst sie näherungsweise linear in κ_f : 21,9 bei $\kappa_f = 10$, 44,9 bei 20, 229,1 bei 100 und 459,4 bei 200 (Bemerkung 12.3.14). Der tatsächliche Verlauf darf schneller sein, wie das Canyon-Widget zeigt, langsamer aber nicht.

Vertiefung: Heath §6.5.1 behandelt die direkten Suchverfahren einschließlich Nelder-Mead, §6.5.2 den steilsten Abstieg samt Zickzack-Bild, und die eindimensionalen Verfahren hinter der Liniensuche stehen in §6.4.

12.4 Newton, Quasi-Newton und SGD

Krümmung statt nur Steigung

Der Gradientenabstieg aus Abschnitt 12.3 kennt von f nur die Steigung. Er ersetzt die Funktion an der aktuellen Stelle durch ihre Tangentialebene und läuft dann bergab. Wie weit er laufen darf, sagt ihm niemand, deshalb die Schrittweite γ und deshalb der Zickzack in engen Tälern.

Die Taylorentwicklung liefert die nächstbessere Näherung gratis. Satz 10.8.7 aus Abschnitt 10.8 gibt sie für $f \in \mathcal{C}^2$, also für zweimal stetig differenzierbares f , mit einem Restterm $o(\|\mathbf{h}\|^2)$. Korollar 10.8.9 schreibt dieselbe Zeile für $f: \mathbb{R}^n \rightarrow \mathbb{R}$ in Gradient und Hesse-Matrix aus, verlangt dafür aber $f \in \mathcal{C}^3$, weil es gleich noch eine Ordnung weiter geht. Uns genügt die Ordnung zwei:

$$f(\mathbf{x} + \mathbf{h}) \approx T_2(\mathbf{h}) = f(\mathbf{x}) + \nabla f(\mathbf{x}) \mathbf{h} + \frac{1}{2} \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \mathbf{h}. \quad (12.4.1)$$

Statt einer Geraden legen wir also eine Parabel an, im $\mathbb{R}^n$ eine quadratische Form über dem aktuellen Punkt. Und anders als bei einer Geraden hat eine nach oben geöffnete Parabel einen tiefsten Punkt. Den können wir ausrechnen.

Dazu leiten wir (12.4.1) nach dem Zuwachs $\mathbf{h}$ ab. Der Gradient ist in unserer Konvention eine Zeile (Abschnitt 10.2); der lineare Term steuert $\nabla f(\mathbf{x})$ bei, und die quadratische Form hat nach Beispiel 10.6.5 den Gradienten $\mathbf{h}^\top (\mathbf{H}_f + \mathbf{H}_f^\top)/2 = \mathbf{h}^\top \mathbf{H}_f$, denn die Hesse-Matrix ist nach dem Satz von Schwarz (Satz 10.7.4) symmetrisch:

$$\nabla_{\mathbf{h}} T_2(\mathbf{h}) = \nabla f(\mathbf{x}) + \mathbf{h}^\top \mathbf{H}_f(\mathbf{x}) \stackrel{!}{=} \mathbf{0}^\top.$$

Beide Seiten sind Zeilenvektoren. Auflösen nach $\mathbf{h}^\top$ und Transponieren liefert den Schritt, den wir suchen. Dieselbe Rechnung lässt sich auch in Spalten führen, als $\nabla f(\mathbf{x})^\top + \mathbf{H}_f(\mathbf{x}) \mathbf{h} = \mathbf{0}$; sie liefert denselben Schritt. Nur mischen darf man die beiden Konventionen nicht – links zwei Spaltenvektoren und rechts eine Zeile ergibt keine Gleichung.

Algorithmus 12.4.1 (Newton-Verfahren für die Optimierung)

Gegeben seien $f : \mathbb{R}^n \rightarrow \mathbb{R}$ zweimal stetig differenzierbar und ein Startpunkt $\mathbf{x}^{(0)}$. Für $k = 0, 1, 2, \dots$ wiederhole:

1. Berechne $\nabla f(\mathbf{x}^{(k)})$ und $\mathbf{H}_f(\mathbf{x}^{(k)})$.
2. Setze

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f(\mathbf{x}^{(k)})^{-1} \nabla f(\mathbf{x}^{(k)})^\top, \quad (12.4.2)$$

sofern $\mathbf{H}_f(\mathbf{x}^{(k)})$ invertierbar ist.

3. Brich ab, sobald $\|\nabla f(\mathbf{x}^{(k)})\|$ oder die Schrittlänge $\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|$ unter eine vorgegebene Schranke fällt, wie beim Gradientenabstieg in Abschnitt 12.3.

Bemerkung 12.4.2 (Was der Schritt voraussetzt und wie wir ihn rechnen)

Die Hesse-Matrix muss invertierbar sein. Sonst hat die quadratische Näherung entweder gar keinen kritischen Punkt oder gleich einen ganzen affinen Unterraum davon, und (12.4.2) steht nicht einmal auf dem Papier. „Bei konvexem f ist $\mathbf{H}_f \succ 0$ “ wäre dafür ein zu bequemes Argument: Konvexität liefert nur positive Semidefinitheit (Abschnitt 11.4), und selbst strikte Konvexität reicht nicht, wie $f(x) = x^4$ im Nullpunkt zeigt. Gebraucht wird eine positiv definite Hesse-Matrix, und die ist eine eigene Annahme.

Die Inverse ist eine Schreibweise, keine Rechenanweisung. Gerechnet wird nicht $\mathbf{H}_f^{-1}$, sondern das lineare Gleichungssystem

$$\mathbf{H}_f(\mathbf{x}^{(k)}) \mathbf{d}^{(k)} = -\nabla f(\mathbf{x}^{(k)})^\top, \quad \mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{d}^{(k)}.$$

Das ist billiger und stabiler (Abschnitt 5.3). Ist $\mathbf{H}_f$ positiv definit, so ist die Cholesky-Zerlegung aus Abschnitt 5.4 das Mittel der Wahl, und ihr Gelingen ist zugleich die Definitheitsprobe. Die Merkregel lautet knapper: *Invertiere niemals eine Matrix.*

Newton-Raphson und Newton-Optimierung sind dasselbe Verfahren

In Abschnitt 12.1 haben wir Newton-Raphson als Verfahren für Nullstellen kennengelernt, hier als Verfahren für Minima. Das ist kein Zufall, sondern ein und dieselbe Vorschrift, angewandt auf zwei verschiedene Funktionen.

Bemerkung 12.4.3 (Eine Vorschrift, zwei Spalten)

Nebeneinandergestellt sehen die beiden Fassungen so aus:

	Nullstellensuche	Optimierung
Aufgabe	löse $f(x) = 0$	löse $\nabla f(\mathbf{x}) = \mathbf{0}^\top$
Schritt	$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$	$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{H}_f^{-1} \nabla f(\mathbf{x}^{(k)})^\top$

Der Übergang ist wörtlich zu nehmen. Setzen wir $\mathbf{g}(\mathbf{x}) := \nabla f(\mathbf{x})^\top$, so ist die Optimierung genau die Nullstellensuche für $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^n$, und deren Jacobimatrix ist die Hesse-Matrix, $\mathbf{J}_g = \mathbf{H}_f$ – das ist die Definition der Hesse-Matrix (Definition 10.7.3). Der mehrdimensionale Newton-Raphson-Schritt $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{J}_g^{-1} \mathbf{g}$ aus Abschnitt 12.1 ist damit wörtlich (12.4.2), wie Bemerkung 12.1.14 ihn schon einmal aufgeschrieben hat. Alles, was wir über Newton-

Raphson wissen, gilt deshalb hier weiter: Unter ausreichender lokaler Glattheit, etwa einer lokal Lipschitz-stetigen Hesse-Matrix, erhalten wir quadratische Konvergenz in der Nähe einer Lösung mit invertierbarer Hesse-Matrix. Es bleiben die Empfindlichkeit gegen schlechte Startpunkte und der Ärger, wenn die Ableitung im Nenner verschwindet.

Wie schnell das geht

Dass sich die Näherungsparabel Schritt für Schritt an das Minimum heranarbeitet, ist als Bild schnell gezeichnet. Interessanter ist, wie schnell das geht. Rechnen wir es an einer Funktion nach, deren Vorschrift wir kennen.

Beispiel 12.4.4 (Newton auf einer nicht-quadratischen Funktion)

Wir minimieren

$$f(x) = x - 2 \ln x, \quad x > 0,$$

mit $f'(x) = 1 - 2/x$ und $f''(x) = 2/x^2 > 0$. Die Funktion ist also strikt konvex, und $f'(x) = 0$ hat die einzige Lösung $x^* = 2$ (Abschnitt 12.2). Statistisch ist das kein Kunstprodukt: Bis auf eine additive Konstante ist f die negative Log-Dichte einer Gammaverteilung mit Formparameter 3 und Rate 1, und 2 ist deren Modus.

Der Newton-Schritt (12.4.2) wird in einer Dimension zu

$$x^{(k+1)} = x^{(k)} - \frac{f'(x^{(k)})}{f''(x^{(k)})} = x^{(k)} - \frac{1 - 2/x^{(k)}}{2/(x^{(k)})^2} = \frac{x^{(k)}(4 - x^{(k)})}{2}.$$

Von $x^{(0)} = 1$ aus laufen die Iterierten so:

k	$x^{(k)}$	$e_k := x^{(k)} - x^* $	e_k/e_{k-1}^2
0	1,0000000000	$1,00 \cdot 10^0$	-
1	1,5000000000	$5,00 \cdot 10^{-1}$	0,500
2	1,8750000000	$1,25 \cdot 10^{-1}$	0,500
3	1,9921875000	$7,81 \cdot 10^{-3}$	0,500
4	1,9999694824	$3,05 \cdot 10^{-5}$	0,500
5	1,9999999995	$4,66 \cdot 10^{-10}$	0,500

Die letzte Spalte ist hier nicht bloß ungefähr konstant, sondern exakt $1/2$. Denn aus der Schrittformel folgt

$$x^{(k+1)} - 2 = \frac{4x^{(k)} - (x^{(k)})^2 - 4}{2} = -\frac{(x^{(k)} - 2)^2}{2},$$

also $e_{k+1} = e_k^2/2$ in jedem einzelnen Schritt. Der Faktor $1/2$ ist kein Zufallswert, sondern das Gegenstück zu der Konstanten aus Bemerkung 12.1.13: Weil wir Newton-Raphson auf f' anwenden, rutschen die Ableitungen um eine Ordnung hoch, und aus $|f''|/(2|f'|)$ wird $|f'''(x^*)|/(2f''(x^*)) = 0,5/(2 \cdot 0,5) = 0,5$.

Lokal ist die Aussage aber auch hier. Für $x^{(0)} = 4$ liefert die Schrittformel 0, für $x^{(0)} = 4,5$ sogar $-1,125$. Beide Male landet der erste Schritt außerhalb von $(0, \infty)$, wo f gar nicht erklärt ist.

Interaktiv: Die Parabel am Startpunkt

Die Tafel zeigt, was (12.4.1) meint. Orange liegt das quadratische Modell am aktuellen Punkt, sein Scheitel ist die nächste Iterierte, und die blauen Punkte zeigen, wohin die Reise geht. Voreingestellt ist Beispiel 12.4.4 mit $x^{(0)} = 1$, sodass sich die Zahlen der Tabelle direkt wiederfinden lassen; der zweite Menüpunkt ist die nicht-konvexe Funktion aus Bemerkung 12.4.5.

Schätzfrage. Wo landet Newton auf der nicht-konvexen Funktion, wenn wir bei $x^{(0)} = 0,5$ starten?

Lösung: *max*

Der zweite Menüpunkt und der Startpunktregler zeigen es in einem einzigen Schritt. Alles hängt am Startpunkt. Auf der konvexen Funktion verlässt der erste Schritt ab $x^{(0)} = 4$ den Definitionsbereich, wie am Ende von Beispiel 12.4.4 gerechnet. Auf der nicht-konvexen führt -2 in das *lokale* Minimum bei -1 , $2,5$ in das globale bei 2 , und $0,5$ landet in einem einzigen Schritt exakt auf dem lokalen Maximum bei 0 – dort ist $f'' = -2,25$, die Modellparabel öffnet nach unten, und ihr Scheitel ist ein Hochpunkt. Am lehrreichsten ist $x^{(0)} = 1,2$: Dort ist $f'' = -0,08$ fast null, der erste Schritt springt nach $-25,2$, und 14 Schritte später steht die Iteration im *lokalen* Minimum bei -1 – obwohl der Startpunkt näher am globalen lag.

Bemerkung 12.4.5 (Newton bei nicht-konvexen Funktionen)

Was (12.4.2) sucht, sind Nullstellen des Gradienten, nicht Minima. Bei einer konvexen Funktion fällt das zusammen (Abschnitt 12.2), sonst nicht. Zwei Dinge können schiefgehen.

Ein lokales Optimum statt des globalen. Sichtbar wird das an einer Landschaft mit mehreren Mulden; im Widget übernimmt das $f(x) = x^4/4 - x^3/3 - x^2 + 2$ mit den kritischen Punkten -1 , 0 und 2 . Bei -1 liegt ein lokales Minimum mit $f(-1) \approx 1,583$, bei 2 das globale mit $f(2) \approx -0,667$. Welches der beiden Täler die Iteration findet, entscheidet allein der Startpunkt. Das ist keine Schwäche gerade dieses Verfahrens, sondern der Preis dafür, dass alle Verfahren dieses Kapitels nur lokale Information benutzen.

Ein Sattelpunkt oder ein Maximum. Ist $H_f(x^{(k)})$ indefinit, so ist der Scheitel der Näherung ein Sattel, und ist sie negativ definit, ein Maximum. Der Schritt läuft dann genauso zielstrebig dorthin, wie er sonst ins Minimum läuft (Abschnitt 12.2). Im Widget genügt der Startpunkt $0,5$, um das vorzuführen.

Praktische Fassungen des Verfahrens sorgen deshalb zunächst für einen Abstiegsschritt: Sie addieren ein Vielfaches der Einheitsmatrix, bis $H_f + \mu I$ positiv definit ist, begrenzen die Schrittlänge (Trust-Region) oder dämpfen den Schritt mit einer Liniensuche wie in Abschnitt 12.3. Das hilft gegen Schritte zu Sattelpunkten oder Maxima und verbessert die Konvergenz aus entfernten Startpunkten. Welches von mehreren lokalen Minima gefunden wird, repariert es nicht; ein globales Optimum ist bei nicht-konvexem f weiterhin nicht garantiert. Für sehr kleines Vertrauen in das quadratische Modell geht die modifizierte Richtung in Richtung des negativen Gradienten über.

Newton gegen Gradientenabstieg

Beide Verfahren lassen sich an einem Fall gegenüberstellen, der sich vollständig im Kopf rechnen lässt.

Beispiel 12.4.6 (Ein Zug statt vieler)

Wir minimieren $f(x) = x^2$ mit $f'(x) = 2x$, $f''(x) = 2$ und Startwert $x^{(0)} = 4$.

Gradientenabstieg mit $\gamma = 0,4$:

$$x^{(1)} = 4 - 0,4 \cdot 8 = 0,8,$$

$$x^{(2)} = 0,8 - 0,4 \cdot 1,6 = 0,16,$$

$$x^{(3)} = 0,16 - 0,4 \cdot 0,32 = 0,032.$$

Jeder Schritt multipliziert die Iterierte mit $1 - 2\gamma = 0,2$. Das ist lineare Konvergenz: Die Zahl der richtigen Stellen wächst um einen festen Betrag pro Schritt, erreicht wird die Null nie.

Newton:

$$x^{(1)} = 4 - \frac{8}{2} = 0.$$

Ein Schritt, und wir stehen exakt im Minimum. Auch das ist kein Zufall: Für eine quadratische Funktion ist die Näherung (12.4.1) exakt (Bemerkung 10.8.10), der Scheitel der Parabel ist der kritische Punkt der Funktion selbst. Ist die Hesse-Matrix dabei positiv definit, so ist dieser Punkt das Minimum, und Newton löst das Problem in einem Zug, gleichgültig, wo wir starten. Ist sie es nicht, so trifft derselbe eine Schritt eben den Sattel oder das Maximum (Bemerkung 12.4.5).

Bemerkung 12.4.7 (Warum Newton die Kondition nicht spürt)

In Abschnitt 12.3 hing die Konvergenzrate des Gradientenabstiegs an der Konditionszahl $\kappa(\mathbf{H}_f)$: Je länglicher das Tal, desto langsamer. Beispiel 12.4.6 zeigt daneben eine Rechnung, in der die Kondition gar nicht vorkommt. Der Grund ist, dass Newton die Krümmung nicht schätzt, sondern benutzt: Sein Schritt ist gegen lineare Koordinatenwechsel unempfindlich, und schlechte Kondition ist nichts anderes als ein ungünstig gewähltes Koordinatensystem. Der Gradientenschritt hat diese Eigenschaft nicht.

Gemeint ist damit die Zahl der Schritte, nicht die Rechnung darin. Numerisch bleibt die Kondition sehr wohl spürbar: Das lineare System aus Bemerkung 12.4.2 wird mit $\kappa(\mathbf{H}_f)$ schlechter konditioniert, und der gelöste Schritt trägt diesen Fehler weiter (Abschnitt 4.2).

Bezahlt wird das an anderer Stelle. Ein Newton-Schritt braucht die $n(n+1)/2$ zweiten Ableitungen, ihren Speicher der Größenordnung n^2 und eine Zerlegung der Größenordnung n^3 (Abschnitt 5.3). Für $n = 10\,000$ Parameter, in der Statistik keine ungewöhnliche Zahl, sind das 10^8 Einträge allein für die Matrix. Der nächste Unterabschnitt handelt davon, wie sich dieser Preis drücken lässt.

Vertiefung: Warum sich der Koordinatenwechsel im Newton-Schritt herauskürzt

Rechnen wir in neuen Koordinaten $\mathbf{x} = \mathbf{A}\mathbf{y}$ mit invertierbarem $\mathbf{A}$, so wird aus dem Gradienten $\nabla f \mathbf{A}$ und aus der Hesse-Matrix $\mathbf{A}^\top \mathbf{H}_f \mathbf{A}$. Im Newton-Schritt heben sich die Faktoren weg,

$$-(\mathbf{A}^\top \mathbf{H}_f \mathbf{A})^{-1} (\nabla f \mathbf{A})^\top = -\mathbf{A}^{-1} \mathbf{H}_f^{-1} \nabla f^\top,$$

und übrig bleibt genau das Bild des alten Schrittes. Beim Gradientenschritt $-\gamma (\nabla f \mathbf{A})^\top = -\gamma \mathbf{A}^\top \nabla f^\top$ bleibt der Faktor $\mathbf{A}^\top$ dagegen stehen, und genau er trägt die Kondition.

Quasi-Newton: die Krümmung schätzen statt rechnen

Newton ist schnell und teuer, der Gradientenabstieg billig und langsam. Die Quasi-Newton-Verfahren sitzen dazwischen, und die Idee ist so einfach wie naheliegend: Wir sammeln die Krümmungsinformation aus den Gradienten ein, die wir ohnehin berechnen.

Algorithmus 12.4.8 (Quasi-Newton-Schritt)

Gegeben seien $f : \mathbb{R}^n \rightarrow \mathbb{R}$ stetig differenzierbar, ein Startpunkt $\mathbf{x}^{(0)}$ und eine Startmatrix $\mathbf{B}_0$, meist $\mathbf{B}_0 = \mathbf{I}$. Für $k = 0, 1, 2, \dots$ setze

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \gamma_k \mathbf{B}_k \nabla f(\mathbf{x}^{(k)})^\top, \quad (12.4.3)$$

wobei $\mathbf{B}_k \approx \mathbf{H}_f(\mathbf{x}^{(k)})^{-1}$ aus den bisherigen Schritten geschätzt wird und die Schrittweite γ_k aus einer Liniensuche stammt (Abschnitt 12.3).

Die Schrittweite γ_k gehört dabei zwingend dazu. Ohne sie ist der erste Schritt aus $\mathbf{B}_0 = \mathbf{I}$ ein ungebremster Gradientenschritt, und der kann den Funktionswert erhöhen. Das Widget weiter unten führt genau das vor.

Der Unterschied zu (12.4.2) ist die Matrix. Newton setzt dort die exakte inverse Hesse-Matrix ein und zahlt jedes Mal $O(n^3)$; (12.4.3) benutzt eine Näherung, die schon da ist, und kostet nur noch eine Matrix-Vektor-Multiplikation. Wie diese Näherung gebaut wird, unterscheidet die Verfahren der Familie: Häufig ist $\mathbf{B}_k$ eine Diagonalmatrix oder eine Einheitsmatrix plus eine Korrektur kleinen Ranges, weil beides sich billig speichern und anwenden lässt.

Bleibt die Frage, woher $\mathbf{B}_k$ kommt. Fangen wir in einer Dimension an. Dort ist die zweite Ableitung die Ableitung der ersten, und die schätzen wir wie jede Ableitung durch einen Differenzenquotienten aus zwei benachbarten Werten:

$$f''(x) \approx \frac{f'(x^{(k+1)}) - f'(x^{(k)})}{x^{(k+1)} - x^{(k)}}.$$

Diese Forderung lässt sich in den $\mathbb{R}^n$ übertragen. Mit den Abkürzungen

$$\mathbf{s}_k := \mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}, \quad \mathbf{y}_k := (\nabla f(\mathbf{x}^{(k+1)}) - \nabla f(\mathbf{x}^{(k)}))^\top$$

verlangen wir von der Näherung der *inversen* Hesse-Matrix die

$$\text{Sekantenbedingung:} \quad \mathbf{B}_{k+1} \mathbf{y}_k = \mathbf{s}_k. \quad (12.4.4)$$

In einer Dimension ist das genau der Differenzenquotient von oben, nur auf den Kopf gestellt: $b = s/y$ statt $f'' \approx y/s$. Beide Gleichungen heißen „Sekantenbedingung“, und wer den Kehrwert überliest, hält $\mathbf{B}_k$ für eine Näherung der Hesse-Matrix statt ihrer Inversen. Im $\mathbb{R}^n$ ist (12.4.4) ein System aus n Gleichungen für die $n(n+1)/2$ Einträge einer symmetrischen Matrix, also weit unterbestimmt. Die verbleibende Freiheit nutzen wir, um $\mathbf{B}_{k+1}$ möglichst wenig von $\mathbf{B}_k$ abweichen zu lassen. Die bekannteste Wahl ist diese:

Definition 12.4.9 (BFGS-Update)

Sei $\mathbf{B}_k \in \mathbb{R}^{n \times n}$ symmetrisch und gelte $\mathbf{y}_k^\top \mathbf{s}_k \neq 0$. Mit $\rho_k := 1/(\mathbf{y}_k^\top \mathbf{s}_k)$ heißt

$$\mathbf{B}_{k+1} = (\mathbf{I} - \rho_k \mathbf{s}_k \mathbf{y}_k^\top) \mathbf{B}_k (\mathbf{I} - \rho_k \mathbf{y}_k \mathbf{s}_k^\top) + \rho_k \mathbf{s}_k \mathbf{s}_k^\top \quad (12.4.5)$$

das *BFGS-Update*, benannt nach Broyden, Fletcher, Goldfarb und Shanno.

Der Nenner von ρ_k hat eine anschauliche Bedeutung. Auf einer Quadrik mit Hesse-Matrix $\mathbf{H}$ ist $\mathbf{y}_k = \mathbf{H} \mathbf{s}_k$, also

$$\frac{\mathbf{y}_k^\top \mathbf{s}_k}{\mathbf{s}_k^\top \mathbf{s}_k} = \frac{\mathbf{s}_k^\top \mathbf{H} \mathbf{s}_k}{\mathbf{s}_k^\top \mathbf{s}_k},$$

und das ist die Krümmung in Richtung $\mathbf{s}_k$. Die Krümmungsbedingung $\mathbf{y}_k^\top \mathbf{s}_k > 0$ aus Bemerkung 12.4.11 sagt damit schlicht: Entlang des gerade gegangenen Schrittes ist f nach oben gekrümmt.

Der Ausdruck (12.4.5) sieht schlimmer aus, als er ist. Beide Klammern sind Einheitsmatrizen plus eine Rang-1-Störung, der letzte Summand ist ebenfalls vom Rang 1. Ausmultipliziert steht dort $\mathbf{B}_k$ plus eine Korrektur, die von $\mathbf{s}_k$ und $\mathbf{B}_k \mathbf{y}_k$ aufgespannt wird und deshalb höchstens Rang 2 hat. Dass die Formel tut, wozu sie gebaut wurde, rechnen wir in drei Zeilen nach.

Satz 12.4.10 (Das BFGS-Update erfüllt die Sekantenbedingung)

Unter den Voraussetzungen von Definition 12.4.9 gilt $\mathbf{B}_{k+1} \mathbf{y}_k = \mathbf{s}_k$.

Vertiefung: Beweis: Die BFGS-Aktualisierung erfüllt die Sekantenbedingung

Beweis.

- (1) Wir lassen die Indizes weg und multiplizieren (12.4.5) von rechts mit $\mathbf{y}$. Die rechte Klammer trifft dabei zuerst auf $\mathbf{y}$:

$$(\mathbf{I} - \rho \mathbf{y} \mathbf{s}^\top) \mathbf{y} = \mathbf{y} - \rho \mathbf{y} (\mathbf{s}^\top \mathbf{y}) = \mathbf{y} - \mathbf{y} = \mathbf{0}.$$

$\mathbf{s}^\top \mathbf{y}$ ist eine Zahl und darf vor den Vektor gezogen werden; nach Definition von ρ ist $\rho (\mathbf{s}^\top \mathbf{y}) = 1$

- (2) Damit verschwindet der ganze erste Summand von (12.4.5), denn er hat diese Klammer als rechten Faktor:

$$(\mathbf{I} - \rho \mathbf{s} \mathbf{y}^\top) \mathbf{B} \underbrace{(\mathbf{I} - \rho \mathbf{y} \mathbf{s}^\top) \mathbf{y}}_{=\mathbf{0}} = \mathbf{0}.$$

Matrixmultiplikation ist assoziativ, wir dürfen also von rechts nach links auswerten

- (3) Übrig bleibt der zweite Summand, und der liefert genau den gesuchten Vektor:

$$\rho \mathbf{s} \mathbf{s}^\top \mathbf{y} = \mathbf{s} (\rho \mathbf{s}^\top \mathbf{y}) = \mathbf{s}.$$

wieder dieselbe Zahl $\rho (\mathbf{s}^\top \mathbf{y}) = 1$; die Sekantenbedingung (12.4.4) ist also nicht näherungsweise, sondern exakt erfüllt

□

Bemerkung 12.4.11 (Eigenschaften, Kosten und L-BFGS)

Symmetrie und Definitheit erben sich. Ist $\mathbf{B}_k$ symmetrisch, so ist es $\mathbf{B}_{k+1}$ nach (12.4.5) auch. Ist $\mathbf{B}_k$ zusätzlich positiv definit und gilt die Krümmungsbedingung $\mathbf{y}_k^\top \mathbf{s}_k > 0$, so bleibt auch die Definitheit erhalten. Das ist wichtig, denn positiv definites $\mathbf{B}_k$ garantiert $\nabla f \mathbf{B}_k \nabla f^\top > 0$ und damit, dass $-\mathbf{B}_k \nabla f(\mathbf{x}^{(k)})^\top$ bergab zeigt, und zwar für jeden Gradienten, der auftreten kann. Erzwingen lässt sich die Krümmungsbedingung durch eine Liniensuche, die neben dem Abstiegs Kriterium aus Abschnitt 12.3 auch eine Bedingung an die Steigung im neuen Punkt stellt.

Kosten. Ein Schritt besteht aus einer Matrix-Vektor-Multiplikation und zwei Rang-1-Korrekturen, zusammen $O(n^2)$ statt $O(n^3)$ bei Newton. Eine Zerlegung fällt nie an, und zweite Ableitungen braucht das Verfahren gar nicht.

Konvergenz. Zwischen linear und quadratisch liegt *superlinear*: Der Quotient e_{k+1}/e_k geht gegen null, ohne dass e_{k+1}/e_k^2 beschränkt bleiben müsste (Konvergenzordnung). In der Praxis ist BFGS deutlich schneller als der Gradientenabstieg und kaum langsamer als Newton.

L-BFGS. Für großes n ist schon das Speichern von $\mathbf{B}_k$ zu teuer. Die Variante mit beschränktem Speicher (*limited memory*) merkt sich deshalb gar keine Matrix, sondern nur die letzten m Paare $(\mathbf{s}_j, \mathbf{y}_j)$, meist m zwischen 5 und 20, und baut daraus bei Bedarf das Produkt $\mathbf{B}_k \nabla f^\top$ zusammen. Der Speicher fällt damit von $O(n^2)$ auf $O(mn)$. Bei $n = 10\,000$ und $m = 10$ stehen 10^8 gespeicherte Zahlen einigen Hunderttausend gegenüber, also drei Größenordnungen weniger.

Interaktiv: BFGS Schritt für Schritt

Wir sehen dem Verfahren beim Lernen zu. Minimiert wird die Quadrik $f(\mathbf{x}) = 0,5 x_1^2 + 2,5 x_2^2$ mit der Hesse-Matrix $\text{diag}(1; 5)$, gestartet wird bei $(5; 1)$ mit $\mathbf{B}_0 = \mathbf{I}$. Über die Krümmung weiß das Verfahren zu diesem Zeitpunkt nichts, der erste Schritt ist also ein reiner Gradientenschritt.

Die letzte Zeile des Ablesefelds prüft in jedem Schritt die Sekantenbedingung nach. Offen bleibt, was die Schrittweite damit zu tun hat.

Interaktives Element — nur in der Web-Fassung

Das Residuum $\|\mathbf{B}_k \mathbf{y} - \mathbf{s}\|$ bleibt in jedem Schritt auf Rundungsfehlerniveau, wie Satz 12.4.10 es verspricht. Frei ist dagegen die Schrittweite, und daran hängt das Verhalten: Lassen wir $\gamma_k = 1$ stehen, so ist der erste Schritt zu lang, f steigt von 15 auf 40, und $\mathbf{B}_k$ bleibt von $\text{diag}(1; 0,2)$ sichtbar entfernt (nach sechs Schritten noch 0,011 in der Frobeniusnorm). Mit dem Häkchen für die exakte Schrittweite liefert dagegen jeder Schritt eine Sekantenbedingung für eine neue Richtung, und im $\mathbb{R}^2$ sind nach zwei Schritten alle vergeben: Die Iterierte sitzt im Minimum und $\mathbf{B}_2$ stimmt mit $\text{diag}(1; 0,2)$ überein. Auf einer Quadrik im $\mathbb{R}^n$ ist BFGS mit exakter Liniensuche deshalb nach höchstens n Schritten fertig.

Vier Verfahren nebeneinander

Damit sind die vier Verfahren beisammen, die für unbeschränkte Probleme zur Wahl stehen. Nebeneinandergestellt:

Eigenschaft	Nelder-Mead	Gradientenabstieg	Quasi-Newton	Newton
Ordnung	nullte	erste	dazwischen	zweite
braucht	nur f	∇f	∇f	$\nabla f, \mathbf{H}_f$
lineare Algebra je Schritt	$O(n)$	$O(n)$	$O(n^2)$	$O(n^3)$
Konvergenz	keine Garantie	linear	superlinear	lokal quadratisch
Schrittweite	automatisch	kritisch (γ)	Liniensuche	automatisch
Skalierbarkeit	schlecht ($n \lesssim 10$)	sehr gut	gut	schlecht

Bemerkung 12.4.12 (Wie die Tabelle zu lesen ist)

Die Aufwandsspalte zählt die lineare Algebra je Schritt, nicht die Funktions- und Gradientenauswertungen. Die können ihrerseits teuer sein: Bei einem empirischen Risiko über N Be-

obachtungen kostet schon ein einziger Gradient $O(N)$, und genau davon handelt der letzte Unterabschnitt.

Die Konvergenzspalte ist asymptotisch und lokal. Die quadratische Rate von Newton gilt in der Nähe eines Minimums mit invertierbarer und lokal hinreichend glatter Hesse-Matrix, die lineare Rate des Gradientenabstiegs unter den Voraussetzungen aus Abschnitt 12.3. Über das Verhalten weit vom Ziel sagt die Zeile nichts.

Drei der vier sind Fixpunktiterationen. Gradientenabstieg, Quasi-Newton und Newton haben dieselbe Bauform

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \mathbf{M}_k \nabla f(\mathbf{x}^{(k)})^\top, \quad \mathbf{M}_k \in \{\gamma \mathbf{I}, \gamma_k \mathbf{B}_k, \mathbf{H}_f(\mathbf{x}^{(k)})^{-1}\},$$

sind also allesamt Fixpunktiterationen für $\nabla f(\mathbf{x}) = \mathbf{0}^\top$ im Sinne von Abschnitt 12.1. Die Wahl von $\mathbf{M}_k$ ist genau die Wahl zwischen Rechenzeit und Schrittqualität, dieselbe Abwägung wie bei den Splitting-Verfahren in Abschnitt 8.3.

Wo sie eingesetzt werden. Nelder-Mead ist der Notnagel ohne Ableitungen (Abschnitt 12.3), der Gradientenabstieg die Grundlage der Optimierer für neuronale Netze, BFGS der Standardkompromiss für mittelgroße glatte Probleme, und Newton steckt in der Likelihood-Inferenz: Das Fisher-Scoring für verallgemeinerte lineare Modelle ist ein Newton-Verfahren mit erwarteter statt beobachteter Hesse-Matrix und läuft dort unter dem Namen *iterativ gewichtete kleinste Quadrate* (IWLS oder IRLS, Bemerkung 10.8.13); auch das Boosting-Verfahren XGBoost baut seine Bäume aus Gradient und zweiter Ableitung. In R bekommen wir BFGS mit `method = "BFGS"`; die Voreinstellung von `optim()` ist es nicht (Abschnitt 12.6).

Momentum: Schwung gegen den Zickzack

Zurück zum billigsten der Verfahren. Der Gradientenabstieg leidet an schlecht konditionierten Problemen: In einem langen, schmalen Tal zeigt der negative Gradient fast quer zur Talrichtung, die Iterierten pendeln zwischen den Hängen hin und her und kommen der Länge nach kaum voran (Abschnitt 12.3). Große Schritte verstärken das Pendeln, kleine machen den Fortschritt entlang des Tals noch langsamer.

Ein Bild aus der Mechanik hilft weiter. Eine Kugel, die den Hang hinunterrollt, folgt nicht in jedem Augenblick der lokalen Falllinie, sondern sammelt Schwung. Quer zum Tal wird sie von den Hängen abwechselnd nach links und rechts geschubst, das mittelt sich weg; entlang des Tals ziehen alle Schubser in dieselbe Richtung und addieren sich auf.

Algorithmus 12.4.13 (Gradientenabstieg mit Heavy-Ball-Momentum)

Gegeben seien $f : \mathbb{R}^n \rightarrow \mathbb{R}$ stetig differenzierbar, ein Startpunkt $\mathbf{x}^{(0)}$, eine Schrittweite $\gamma > 0$ und ein Momentumparameter $\alpha \in [0, 1)$. Setze die *Geschwindigkeit* $\mathbf{v}^{(0)} = \mathbf{0}$, die im Folgenden die bisherigen Schritte aufsammelt, und für $k = 0, 1, 2, \dots$

$$\begin{aligned} \mathbf{v}^{(k+1)} &= \alpha \mathbf{v}^{(k)} - \gamma \nabla f(\mathbf{x}^{(k)})^\top, \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)}. \end{aligned} \tag{12.4.6}$$

Für $\alpha = 0$ ist das der gewöhnliche Gradientenabstieg.

Bemerkung 12.4.14 (Was der Schwung bewirkt)

Gleichgerichtete Gradienten summieren sich auf. Bleibt der Gradient über mehrere Schritte

ungefähr gleich, so wird aus (12.4.6) eine geometrische Reihe, und die Geschwindigkeit läuft gegen $\mathbf{v} \rightarrow -\gamma \nabla f^T / (1 - \alpha)$. Der effektive Schritt in einer flachen, aber konsistenten Richtung ist also um den Faktor $1/(1 - \alpha)$ größer als beim reinen Gradientenabstieg; für den üblichen Wert $\alpha = 0,9$ ist das ein Faktor 10.

Wechselnde Gradienten mitteln sich weg. Kippt die Gradientenrichtung dagegen in jedem Schritt, so heben sich die Beiträge in (12.4.6) weitgehend auf. Das ist die Dämpfung des Zickzacks. Der gelegentlich zu lesende Name „Dämpfungsfaktor“ für α führt dabei leicht in die Irre: Gedämpft wird die Schwingung, nicht die Bewegung, und in der mechanischen Analogie ist gerade $1 - \alpha$ die Reibung. Je größer α , desto weniger Reibung und desto mehr Gedächtnis. *Größere Schritte sind erlaubt.* Auf einer Quadrik mit Krümmungen zwischen μ und L konvergiert der Gradientenabstieg genau für $\gamma L < 2$; mit Momentum verschiebt sich die Grenze auf $\gamma L < 2(1 + \alpha)$. Bei $\alpha = 0,9$ dürfen die Schritte also fast doppelt so lang sein.

Aber nicht jedes α hilft. Für eine Quadrik lässt sich die beste Wahl ausrechnen: Mit $\alpha^* = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2$ und $\gamma^* = 4/(\sqrt{L} + \sqrt{\mu})^2$ fällt der Fehler pro Schritt auf das $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$ -fache statt auf das $(\kappa - 1)/(\kappa + 1)$ -fache. Statt κ steht dort also $\sqrt{\kappa}$, und das ist bei $\kappa = 100$ der Unterschied zwischen 0,98 und 0,82 pro Schritt. Der Standardwert 0,9 passt zu κ in der Größenordnung 10^3 ; bei gut konditionierten Problemen ist er zu groß und macht das Verfahren *langsamer* als den reinen Gradientenabstieg. Das Widget führt beides vor.

Und die Garantie gilt nicht überall. Für Quadriken ist die Beschleunigung ein Satz. Für allgemeine glatte, strikt konvexe Funktionen ist sie es nicht: Es gibt Beispiele, auf denen Heavy-Ball mit den für Quadriken optimalen Parametern in einen Zyklus läuft und gar nicht konvergiert. Die eng verwandte Variante von Nesterov trägt dort eine bewiesene Schranke. Der Merksatz „bei strikt konvexen Funktionen beschleunigt Momentum die Konvergenz deutlich“ ist also eine Faustregel, kein Theorem.

Die Verwandtschaft. Adagrad, RMSprop und ADAM bauen auf derselben Beobachtung auf, dass in der Vergangenheit der Gradienten Information steckt. Sie merken sich zusätzlich, wie groß die bisherigen Gradienten typischerweise waren, und geben jeder Koordinate daraus eine eigene Schrittweite; ADAM verbindet das mit dem Schwung aus (12.4.6). Wir nennen sie hier nur; der Kurs *Optimization for ML* im Master geht ihnen nach.

Interaktiv: Mit und ohne Schwung

Beide Verfahren laufen auf derselben Quadrik $f(\mathbf{x}) = \frac{1}{2}(x_1^2 + c x_2^2)$ mit Konditionszahl $\kappa = c$, blau ohne und violett mit Momentum. Die Schrittweite geben wir als Vielfaches von $1/L$ an, damit die beiden Stabilitätsgrenzen 2 und $2(1 + \alpha)$ am Regler ablesbar bleiben.

Drei Voreinstellungen decken die Fallunterscheidung ab: gut, mittel und schlecht konditioniert, jeweils mit dem Standardwert $\alpha = 0,9$.

Schätzfrage. Hilft der Standardwert $\alpha = 0,9$ bei einer *gut* konditionierten Quadrik, etwa $\kappa = 5$?

Lösung: *nein*

Die Ablesezeile zählt beide Verfahren mit.

Der Nutzen des Schwungs hängt allein an der Kondition. Bei $\kappa = 5$ dreht sich das Ergebnis sogar um: 106 Schritte mit gegen 31 ohne Momentum. Bei $\kappa = 25$ steht es 103 zu 161, bei $\kappa = 100$ dann 121 zu 608 – je schlechter die Kondition, desto größer der Gewinn. Und bei $\gamma L = 2,5$ explodiert der blaue Weg, während der violette stabil bleibt, denn seine Grenze liegt bei $2 \cdot 1,9 = 3,8$. Der Standardwert 0,9 stammt aus dem Deep Learning, wo die Konditionszahlen um Größenordnungen höher liegen als hier.

Eine Shiny-App zeigt Gradientenabstieg und Momentum auf weiteren Landschaften.

Stochastischer Gradientenabstieg

Bisher haben wir gefragt, wie viele Schritte ein Verfahren braucht. Jetzt fragen wir, was ein einzelner Schritt kostet. In Statistik und maschinellem Lernen hat die Zielfunktion fast immer dieselbe Bauform, nämlich einen Mittelwert über Daten. Wir minimieren das *empirische Risiko*

$$R(\theta) = \frac{1}{N} \sum_{i=1}^N L(y_i, p_{\theta}(\mathbf{x}_i)), \quad (12.4.7)$$

wobei L den Fehler einer einzelnen Vorhersage misst und N die Zahl der Beobachtungen ist. Beispiel 12.1.4 hat diese Gestalt schon aufgestellt, und Beispiel 12.2.7 rechnet mit dem Kleinste-Quadrate-Verlust und der negativen Log-Likelihood zwei ihrer Vertreter durch. Auch der Kreuzentropie-Verlust gehört dazu: Er ist die negative Log-Likelihood eines Bernoulli-Modells, also der Verlust der logistischen Regression.

Der Gradient erbt die Summe:

$$\nabla R(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla L(y_i, p_{\theta}(\mathbf{x}_i)).$$

Ein einziger Gradientenschritt läuft also einmal durch den gesamten Datensatz. Bei N in Millionenhöhe ist das der Flaschenhals, und zwar unabhängig davon, wie gut die Schrittrichtung ist. Die Idee des *stochastischen Gradientenabstiegs* ist, diesen Mittelwert durch einen unverzerrten Schätzer zu ersetzen, statt ihn auszurechnen.

Satz 12.4.15 (Der Gradient einer zufällig gezogenen Beobachtung ist unverzerrt)

Seien θ fest, der Datensatz fest, und sei i gleichverteilt auf $\{1, \dots, N\}$ gezogen. Dann gilt

$$\mathbb{E}[\nabla L(y_i, p_{\theta}(\mathbf{x}_i))] = \nabla R(\theta).$$

Vertiefung: Beweis der Unverzerrtheit

Beweis.

- (1) Der Erwartungswert läuft allein über den Index i , und der nimmt jeden der N Werte mit Wahrscheinlichkeit $1/N$ an:

$$\mathbb{E}[\nabla L(y_i, p_{\theta}(\mathbf{x}_i))] = \sum_{j=1}^N \mathbb{P}(i = j) \nabla L(y_j, p_{\theta}(\mathbf{x}_j)) = \frac{1}{N} \sum_{j=1}^N \nabla L(y_j, p_{\theta}(\mathbf{x}_j)),$$

und das ist $\nabla R(\theta)$.

Erwartungswert einer Funktion einer diskreten Zufallsvariablen; die y_j und $\mathbf{x}_j$ sind hier Konstanten, zufällig ist nur, welche von ihnen wir erwischen

□

Der Beweis ist kurz, aber die Voraussetzung verdient Aufmerksamkeit. Kurz hingeschrieben lautet die Aussage $\mathbb{E}[\nabla L(y_i, p_{\theta}(\mathbf{x}_i))] = \nabla R(\theta)$, und dann ist leicht zu übersehen, worüber gemittelt wird: über den Zug des Index, nicht über die Verteilung der Daten. Deshalb ist die Aussage auch keine Annahme über das Modell, sondern eine Konsequenz daraus, wie wir ziehen: Wir bauen den unverzerrten Schätzer selbst.

Algorithmus 12.4.16 (Stochastischer Gradientenabstieg, SGD)

Gegeben seien das empirische Risiko (12.4.7), ein Startwert $\theta^{(0)}$ und Schrittweiten $\gamma^{(k)} > 0$. Für $k = 0, 1, 2, \dots$

1. ziehe einen Index i_k gleichverteilt aus $\{1, \dots, N\}$,
2. setze

$$\theta^{(k+1)} = \theta^{(k)} - \gamma^{(k)} \nabla L(y_{i_k}, p_{\theta^{(k)}}(\mathbf{x}_{i_k}))^\top. \quad (12.4.8)$$

Vertiefung: Mini-Batches, Lernraten und praktische Varianten**Bemerkung 12.4.17 (Rauschen, Mini-Batches und Lernraten)**

Der Preis und der Gewinn. Ein Schritt kostet jetzt $O(1)$ statt $O(N)$, und zwar unabhängig von der Datenmenge. Dafür ist die Richtung nur im Mittel richtig. Die Iterierten laufen nicht mehr glatt bergab, sondern zappeln; in der Nähe des Minimums bleibt bei fester Schrittweite ein Rauschband, das nicht kleiner wird.

Mini-Batches. In der Praxis ziehen wir nicht eine Beobachtung, sondern einen ganzen Stapel von b Stück und mitteln deren Gradienten; an die Stelle des einzelnen Summanden in (12.4.8) tritt dann dieses Mittel. Die Stapelgröße nennen wir b , weil n in diesem Abschnitt schon die Zahl der Parameter ist. Der Schätzer bleibt unverzerrt, und bei unabhängigen Ziehungen fällt seine Varianz auf den b -ten Teil, die Standardabweichung also auf den $\sqrt{b}$ -ten. Das ist die entscheidende Buchführung: Der Aufwand wächst linear in b , die Genauigkeit nur mit $\sqrt{b}$. Ein Stapel von 32 statt einer einzelnen Beobachtung kostet das 32-fache und drückt das Rauschen nur um den Faktor 5,7. Übliche Größen liegen zwischen 32 und 256, und diese Wahl folgt weniger der Statistik als der Hardware, die viele gleichartige Rechnungen gleichzeitig erledigt.

Lernraten. Damit die Iterierten nicht im Rauschband hängenbleiben, lassen wir $\gamma^{(k)}$ langsam fallen. Klassisch verlangen wir $\sum_k \gamma^{(k)} = \infty$, damit die Iteration überhaupt beliebig weit kommt, und $\sum_k (\gamma^{(k)})^2 < \infty$, damit sich das Rauschen herausmittelt; $\gamma^{(k)} \sim 1/k$ erfüllt beides. In der Praxis kommen dazu Aufwärmphasen und stufenweise Absenkungen (*learning rate schedules*).

Und was wirklich passiert. Statt unabhängig zu ziehen, mischen die gängigen Implementierungen den Datensatz einmal pro Durchlauf und gehen ihn der Reihe nach durch; Satz 12.4.15 gilt dann nur noch gemittelt über alle Mischungen, praktisch funktioniert es aber besser.

Warum das wichtig ist. SGD mit Mini-Batches und Momentum ist die Grundlage, auf der sämtliche modernen neuronalen Netze trainiert werden. Der MSc-Kurs Optimization for ML behandelt ADAM, Muon und die Lernratenpläne im Detail.

Damit ist der Werkzeugkasten für unbeschränkte Probleme beisammen. Im nächsten Abschnitt kommen Nebenbedingungen dazu, und mit ihnen ein Optimalitätsbegriff, der mehr verlangt als einen verschwindenden Gradienten (Abschnitt 12.5).

Selbsttest

1. Ein Newton-Schritt liefert stets ein lokales Minimum der Funktion.

Antwort: Falsch

Er liefert einen kritischen Punkt der quadratischen Näherung, mehr nicht. Ist die Hesse-Matrix indefinit, so ist das ein Sattel, ist sie negativ definit, ein Maximum. Im Widget genügt der Startpunkt 0,5 auf der nicht-konvexen Funktion: Ein einziger Schritt landet exakt auf dem lokalen Maximum bei 0, weil dort ebenfalls $f' = 0$ gilt (Bemerkung 12.4.5).

2. Auf einer strikt konvexen quadratischen Funktion trifft Newton das Minimum in einem Schritt, ganz gleich, wie schlecht konditioniert sie ist und wo wir starten.

Antwort: Wahr

Für ein Polynom zweiten Grades ist die Näherung (12.4.1) exakt. Ist seine Hesse-Matrix positiv definit, ist der Scheitel das eindeutige Minimum selbst (Bemerkung 10.8.10). Beispiel 12.4.6 rechnet das vor: $4 - 8/2 = 0$. Bei negativ definiten Hesse-Matrix träge Newton stattdessen ein Maximum, bei indefiniten einen Sattel, und bei singulärer wäre der angegebene Schritt nicht definiert. Der Gradientenabstieg schafft den Treffer im Allgemeinen nicht, seine Schrittzahl hängt an der Kondition. In einer Dimension gibt es allerdings eine Ausnahme: Mit $\gamma = 1/f''$ trifft auch er sofort, und genau diese Schrittweite ist der Newton-Schritt.

3. Damit BFGS funktioniert, muss B_k gegen die inverse Hesse-Matrix konvergieren.

Antwort: Falsch

Muss es nicht. Im Widget steht nach sechs Schritten mit $\gamma_k = 1$ eine Matrix, die $\text{diag}(1; 0,2)$ nur nahekommt, und das Verfahren konvergiert trotzdem. Gebraucht wird eine gute Richtung, nicht die richtige Matrix. Nur im Sonderfall der Quadrik mit exakter Liniensuche wird die Näherung nach n Schritten exakt.

4. Das BFGS-Update erfüllt die Sekantenbedingung nicht näherungsweise, sondern exakt.

Antwort: Wahr

Das ist Satz 12.4.10, und der Beweis braucht nur die Definition $\rho_k = 1/(\mathbf{y}_k^\top \mathbf{s}_k)$: Die rechte Klammer in (12.4.5) schickt $\mathbf{y}_k$ auf null, der erste Summand fällt also weg, und der zweite liefert genau $\mathbf{s}_k$. Näherungsweise ist die *Sekantenbedingung selbst*, denn ein Differenzenquotient ist nicht die Ableitung.

5. Momentum mit $\alpha = 0,9$ ist immer mindestens so schnell wie der reine Gradientenabstieg.

Antwort: Falsch

Bei gut konditionierten Problemen schadet es. Im Widget braucht der Abstieg mit $\alpha = 0,9$ auf $\kappa = 5$ ganze 106 Schritte, ohne Momentum sind es 31. Rechnerisch optimal wäre dort $\alpha^* \approx 0,15$. Der Wert 0,9 stammt aus dem Deep Learning, wo die Konditionszahl um Größenordnungen höher liegt (Bemerkung 12.4.14).

6. Ein Mini-Batch aus 32 Beobachtungen macht die Schätzung des Gradienten 32-mal genauer als eine einzelne Beobachtung.

Antwort: Falsch

Die *Varianz* fällt auf ein Zweiunddreißigstel, die Standardabweichung also nur um den Faktor $\sqrt{32} \approx 5,7$. Genau deshalb sind Mini-Batches ein Kompromiss und keine Lösung: Der Aufwand wächst linear, die Genauigkeit mit der Wurzel.

7. Wie viele Schritte braucht der reine Gradientenabstieg im Momentum-Widget bei $\kappa = 5$ und $\gamma L = 1$ für einen Faktor 10^{-6} im Funktionswert?

Lösung: 31 Schritte

Es sind 31. Der Abstieg *mit* Momentum ($\alpha = 0,9$) braucht dafür 106, also mehr als dreimal so viele. Bei guter Kondition schadet der Schwung, weil die Iterierten über das Tal hinausschießen; erst bei $\kappa = 100$ dreht sich das Verhältnis auf 121 gegen 608 (Bemerkung 12.4.14).

8. Die Unverzerrtheit des SGD-Gradienten folgt aus der Art, wie wir den Index ziehen, und nicht aus einer Annahme über die Daten.

Antwort: Wahr

In Satz 12.4.15 sind Datensatz und Parameter fest; zufällig ist einzig, welchen Summanden von (12.4.7) wir erwischen. Weil jeder Index Wahrscheinlichkeit $1/N$ hat, ist der Erwartungswert der Mittelwert aller Summanden, also der volle Gradient.

Vertiefung: Heath §6.5.3 behandelt das Newton-Verfahren für die unbeschränkte Optimierung, §6.5.4 die Quasi-Newton-Verfahren im Überblick und §6.5.5 die Sekanten-Updates samt BFGS; §5.6 zeigt die Newton-Verfahren für nichtlineare Gleichungssysteme, aus denen sie hervorgehen. Zum stochastischen Gradientenabstieg und seinen Varianten führt der MSc-Kurs Optimization for ML weiter.

12.5 Beschränkte Optimierung

Die Verfahren der Abschnitte 12.3 und 12.4 haben f auf ganz $\mathbb{R}^n$ minimiert. Oft ist aber nicht jeder Punkt erlaubt: Wahrscheinlichkeiten müssen nichtnegativ sein und sich zu 1 summieren, ein Budget hat eine Obergrenze, Erhaltungsgrößen der Physik stehen nicht zur Disposition. Solche Einschränkungen formulieren wir als Gleichungen und Ungleichungen an $\mathbf{x}$.

Definition 12.5.1 (Beschränktes Optimierungsproblem)

Definition 12.1.2 in Abschnitt 12.1 hat das *beschränkte Optimierungsproblem* (constrained optimization) schon eingeführt. Wir schreiben es jetzt in der Form aus, mit der wir rechnen werden:

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) && \text{(Zielfunktion)} \\ \text{unter} \quad & g_i(\mathbf{x}) = 0, \quad i = 1, \dots, m && \text{(Gleichungen)} \\ & h_j(\mathbf{x}) \leq 0, \quad j = 1, \dots, p && \text{(Ungleichungen).} \end{aligned}$$

Die Menge S aller Punkte, die sämtliche Nebenbedingungen erfüllen, heißt *zulässiger Bereich* (feasible set).

Bemerkung 12.5.2 (Das Optimum liegt oft auf der Nebenbedingung)

Minimieren wir etwa $f(x, y) = x^2 + y^2$ unter der Gleichungsnebenbedingung $g(x, y) = x + y - 1 = 0$, so ist das unbeschränkte Minimum $(0, 0)$ nicht zulässig. Das beschränkte Optimum muss auf der Geraden $x + y = 1$ liegen, und zwar dort, wo die Gerade die niedrigste Höhenlinie von f berührt. Dieses Berührbild trägt das ganze folgende Verfahren.

Die Idee der Lagrange-Multiplikatoren

Dürfen wir uns nur entlang der Kurve $g(\mathbf{x}) = 0$ bewegen, dann zählt am Optimum nur noch die Richtung entlang der Kurve: Gäbe es dort eine Richtung mit Steigung nach unten, könnten wir den Zielwert weiter senken. Am Optimum $\mathbf{x}^*$ hat f also entlang der Nebenbedingung keine Steigung mehr. Das heißt geometrisch:

- $\nabla f(\mathbf{x}^*)$ steht senkrecht auf der Tangente der Nebenbedingungskurve,
- und weil auch $\nabla g(\mathbf{x}^*)$ senkrecht auf dieser Tangente steht (Abschnitt 10.2: der Gradient steht senkrecht auf den Höhenlinien), sind die beiden Gradienten *parallel*:

$$\exists \lambda \in \mathbb{R} : \quad \nabla f(\mathbf{x}^*) + \lambda \nabla g(\mathbf{x}^*) = \mathbf{0}^\top.$$

Bemerkung 12.5.3 (Auch null ist ein zulässiger Multiplikator)

Verlockend wäre, an dieser Stelle $\lambda \in \mathbb{R} \setminus \{0\}$ zu verlangen. Das stimmt aber nicht. Liegt das unbeschränkte Minimum zufällig auf der Nebenbedingung, dann ist dort schon $\nabla f(\mathbf{x}^*) = \mathbf{0}^\top$, und die Bedingung gilt mit $\lambda = 0$. Der Multiplikator darf also jeden reellen Wert annehmen; sein Vorzeichen ist bei Gleichungsnebenbedingungen frei, wie gleich Beispiel 12.5.6 zeigt. Damit die Parallelitätsbedingung das Optimum wirklich erfasst, brauchen wir außerdem $\nabla g(\mathbf{x}^*) \neq \mathbf{0}^\top$, sonst legt die Nebenbedingung ihre Tangente nicht fest.

Definition 12.5.4 (Lagrange-Funktion)

Für das Problem $\min f(\mathbf{x})$ unter $\mathbf{g}(\mathbf{x}) = \mathbf{0}$ mit $\mathbf{g}(\mathbf{x}) = (g_1(\mathbf{x}), \dots, g_m(\mathbf{x}))^\top$ heißt

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \boldsymbol{\lambda}^\top \mathbf{g}(\mathbf{x}), \quad \boldsymbol{\lambda} = (\lambda_1, \dots, \lambda_m)^\top \in \mathbb{R}^m,$$

Lagrange-Funktion; die λ_i heißen Lagrange-Multiplikatoren.

Satz 12.5.5 (Notwendige Bedingung von Lagrange)

Seien f und $\mathbf{g}$ stetig differenzierbar und sei $\mathbf{x}^*$ ein lokales Minimum von f unter $\mathbf{g}(\mathbf{x}) = \mathbf{0}$. Sind die Gradienten $\nabla g_1(\mathbf{x}^*), \dots, \nabla g_m(\mathbf{x}^*)$ linear unabhängig, so existiert ein $\boldsymbol{\lambda}^* \in \mathbb{R}^m$ mit

$$\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*) = \mathbf{0}^\top \quad \text{und} \quad \mathbf{g}(\mathbf{x}^*) = \mathbf{0}.$$

Die erste Gleichung ist die Parallelitätsbedingung von oben, für alle m Nebenbedingungen gleichzeitig; die zweite ist die Zulässigkeit. Statt eines Minimierungsproblems mit Zwang lösen wir also ein Gleichungssystem in den $n + m$ Unbekannten $(\mathbf{x}, \boldsymbol{\lambda})$, und dafür haben wir die Verfahren aus Abschnitt 12.1.

Die Unabhängigkeitsbedingung an die Gradienten ist keine Formsache. Minimieren wir $f(x, y) = x$ unter $\mathbf{g}(x, y) = y^2 - x^3 = 0$, so erzwingt die Nebenbedingung $x \geq 0$, das Minimum liegt also im Ursprung. Dort ist wegen $\nabla \mathbf{g}(x, y) = (-3x^2, 2y)$ aber $\nabla \mathbf{g}(0, 0) = \mathbf{0}^\top$, während $\nabla f = (1, 0)$ nirgends verschwindet: Kein λ erfüllt die Stationaritätsgleichung. An dieser Spitze der zulässigen Kurve greift der Satz von Lagrange schlicht nicht.

Hinreichende Bedingungen sind feiner (zweite Ableitungen der Lagrange-Funktion auf dem Tangentialraum); dazu Heath, Ch. 6.2.3.

Beispiel 12.5.6 (Minimieren auf einer Geraden)

Wir minimieren $f(x, y) = x^2 + y^2$ unter $\mathbf{x} + \mathbf{y} - \mathbf{1} = \mathbf{0}$.

Die Lagrange-Funktion ist $\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$, und ihre partiellen Ableitungen liefern drei Gleichungen:

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial x} &= 2x + \lambda = 0 & \implies & x = -\lambda/2, \\ \frac{\partial \mathcal{L}}{\partial y} &= 2y + \lambda = 0 & \implies & y = -\lambda/2, \\ \frac{\partial \mathcal{L}}{\partial \lambda} &= x + y - 1 = 0 & \implies & -\lambda = 1. \end{aligned}$$

Also $\lambda^* = -1$ und $x^* = y^* = \frac{1}{2}$ mit $f(x^*, y^*) = \frac{1}{2}$. Das passt zur Geometrie: Im Punkt $(\frac{1}{2}, \frac{1}{2})$ ist $\nabla f = (1, 1)$ parallel zu $\nabla g = (1, 1)$, und das Vorzeichen von λ^* ist negativ. Bei Gleichungsnebenbedingungen ist das erlaubt.

Woran erkennen wir am Bild, dass ein Punkt der Geraden optimal ist? Probieren wir es aus.

Interaktiv: Höhenlinien, Gerade und zwei Pfeile

Das Widget zeigt das Beispiel live: die Höhenlinien von f in Grau, die gerade erreichte violett, die Nebenbedingung in Rot, an einem verschiebbaren Punkt auf der Geraden die beiden Gradienten in Orange. Daneben steht dieselbe Funktion als Fläche, auf die die Nebenbedingung als rote Kurve gehoben ist. Die drei Knöpfe wechseln zwischen der Gleichung und den beiden Ungleichungen.

Interaktives Element – nur in der Web-Fassung

Abzulesen ist die *Parallelität* der beiden Pfeile: Nur an einer Stelle der Geraden decken sie sich, und nur dort liefern beide Stationaritätsgleichungen denselben Multiplikator, nämlich $\lambda^* = -1$ im Punkt $(0,5; 0,5)$ mit $f^* = 0,5$. Überall sonst schneidet die Höhenlinie die Gerade in zwei Punkten, statt sie zu berühren. Die beiden Ungleichungs-Modi nehmen die folgenden KKT-Bedingungen vorweg: Bei $x + y \geq 1$ zeigen die Pfeile im Optimum gegeneinander, $\mu = 1 > 0$, die Nebenbedingung bindet; bei $x + y \leq 1$ verlangte jeder Randpunkt $\mu < 0$, und stattdessen gewinnt das unbeschränkte Minimum $(0; 0)$ mit $\mu = 0$.

Ungleichungen: die KKT-Bedingungen

Bei Ungleichungs-Nebenbedingungen $h_j(\mathbf{x}) \leq 0$ kommt eine neue Unterscheidung dazu: Eine Ungleichung kann im Optimum *aktiv* sein (es gilt Gleichheit, sie blockiert die Bewegung) oder *inaktiv* (es gilt strikte Ungleichheit, lokal spürt das Problem sie gar nicht).

Satz 12.5.7 (Karush-Kuhn-Tucker-Bedingungen)

Sei $\mathbf{x}^*$ ein lokales Minimum von f unter $g_i(\mathbf{x}) = 0$ und $h_j(\mathbf{x}) \leq 0$, und seien die Gradienten aller Gleichungen zusammen mit denen der in $\mathbf{x}^*$ aktiven Ungleichungen linear unabhängig. Dann existieren Multiplikatoren λ_i, μ_j mit:

1. *Stationarität*: $\nabla f(\mathbf{x}^*) + \sum_i \lambda_i \nabla g_i(\mathbf{x}^*) + \sum_j \mu_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}^\top$
2. *Primale Zulässigkeit*: $g_i(\mathbf{x}^*) = 0$ und $h_j(\mathbf{x}^*) \leq 0$
3. *Duale Zulässigkeit*: $\mu_j \geq 0$
4. *Komplementarität*: $\mu_j h_j(\mathbf{x}^*) = 0$ für alle j

Bemerkung 12.5.8 (Komplementarität: bindet oder abgeschaltet)

Bedingung 4 ist ein Entweder-oder je Ungleichung:

- $h_j(\mathbf{x}^*) < 0$ (inaktiv) $\implies \mu_j = 0$: Die Nebenbedingung trägt nichts zur Stationarität bei.

- $\mu_j > 0 \implies h_j(\mathbf{x}^*) = 0$ (aktiv): Die Nebenbedingung bindet, und ihr Gradient stemmt sich in der Stationaritätsgleichung gegen ∇f .

Anders als bei Gleichungen ist das Vorzeichen hier festgelegt: $\mu_j \geq 0$, denn eine Ungleichung kann nur in eine Richtung drücken. Die λ_i der Gleichungen bleiben vorzeichenfrei.

Beispiel 12.5.9 (Eine Box-Beschränkung)

Wir minimieren $f(x) = (x - 3)^2$ unter $0 \leq x \leq 2$, also mit $h_1(x) = -x \leq 0$ und $h_2(x) = x - 2 \leq 0$.

Die KKT-Bedingungen lauten

$$2(x^* - 3) - \mu_1 + \mu_2 = 0, \quad \mu_1, \mu_2 \geq 0, \quad \mu_1 x^* = 0, \quad \mu_2(x^* - 2) = 0. \quad (12.5.1)$$

Das unbeschränkte Minimum $x = 3$ ist unzulässig, der Kandidat ist der rechte Rand: $x^* = 2$. Komplementarität erzwingt dann $\mu_1 = 0$ (denn $x^* = 2 \neq 0$), und (12.5.1) liefert $2(2 - 3) + \mu_2 = 0$, also $\mu_2 = 2 > 0$. Alle vier Bedingungen sind erfüllt: Die obere Schranke ist *aktiv*, die untere *inaktiv*.

Die beiden anderen Kandidaten scheitern an denselben Bedingungen. Am linken Rand $x = 0$ ist $\mu_2 = 0$, und (12.5.1) verlangte dann $\mu_1 = 2(0 - 3) = -6 < 0$, was die duale Zulässigkeit verletzt. Im Inneren wären beide Multiplikatoren null und (12.5.1) verlangte $x^* = 3$, was die Box verlässt.

Anschaulich zeigt die negative Ableitung $-f'(2) = 2$ nach rechts, Richtung des unbeschränkten Minimums, aber die aktive Schranke h_2 blockiert jede weitere Bewegung. Genau diese blockierte Abstiegsrichtung bezahlt das Problem mit $\mu_2 > 0$.

Anwendung: Ridge und Lasso als beschränkte Optimierung

Die regularisierte Regression aus Abschnitt 12.1 lässt sich in einer penalisierten und einer beschränkten Form schreiben; die zweite ist ein beschränktes Problem im Sinn von Definition 12.5.1:

$$\begin{aligned} \text{penalisiert:} \quad & \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_p^p \\ \text{beschränkt:} \quad & \min_{\boldsymbol{\beta}} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 \quad \text{unter} \quad \|\boldsymbol{\beta}\|_p^p \leq c, \end{aligned}$$

mit $p = 2$ (Ridge) beziehungsweise $p = 1$ (Lasso).

Beispiel 12.5.10 (KKT-Stationarität für Ridge)

Für Ridge ($p = 2$) ist die Lagrange-Funktion der beschränkten Form

$$\mathcal{L}(\boldsymbol{\beta}, \mu) = \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \mu (\|\boldsymbol{\beta}\|_2^2 - c),$$

und die Stationarität liefert mit den Gradienten aus Abschnitt 10.6

$$\nabla_{\boldsymbol{\beta}} \mathcal{L}^\top = -2\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) + 2\mu\boldsymbol{\beta} = \mathbf{0} \implies \hat{\boldsymbol{\beta}}_{\text{Ridge}} = (\mathbf{X}^\top\mathbf{X} + \mu\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}.$$

Das ist die Ridge-Lösung mit dem Multiplikator μ in der Rolle des Strafparameters. Ist die penalisiert geschätzte Lösung zu $\lambda > 0$ von null verschieden, so gehört mit $c = \|\hat{\boldsymbol{\beta}}(\lambda)\|_2^2 > 0$ eine bindende beschränkte Form mit derselben Lösung dazu. Im Sonderfall $\hat{\boldsymbol{\beta}}(\lambda) = \mathbf{0}$ ist das aktive

Budget $c = 0$; ein größeres Budget kann dieselbe Lösung enthalten, ist dann aber inaktiv und hat $\mu = 0$. Die Umkehrung gilt nur, solange die Nebenbedingung wirklich bindet: Ist c so groß, dass der Kleinste-Quadrate-Schätzer selbst zulässig ist, dann erzwingt die Komplementarität $\mu = 0$, und das entspricht $\lambda = 0$. Welches c zu welchem λ gehört, hängt außerdem von den Daten ab; eine Umrechnungsformel gibt es nicht (Abschnitt 12.1). Für $\mu > 0$ ist $\mathbf{X}^\top \mathbf{X} + \mu \mathbf{I}$ symmetrisch positiv definit, das System also eindeutig lösbar (Abschnitt 5.4), auch wenn $\mathbf{X}$ keinen vollen Rang hat.

Bemerkung 12.5.11 (Kreis gegen Raute: warum Lasso Nullen erzeugt)

Geometrisch minimieren beide Verfahren dieselben elliptischen Höhenlinien des Kleinste-Quadrate-Verlusts, nur über verschiedenen zulässigen Bereichen: Ridge über der Kreisscheibe $\|\boldsymbol{\beta}\|_2^2 \leq c$, Lasso über der Raute $|\beta_1| + |\beta_2| \leq c$. Verletzt der Kleinste-Quadrate-Schätzer die Nebenbedingung, so liegt die Lösung dort, wo die um ihn herum wachsende Ellipse den Bereich zuerst berührt. Einen Kreis trifft sie fast immer in einem glatten Randpunkt, an dem beide Koordinaten von null verschieden sind. Die Raute dagegen streckt ihre Ecken auf die Achsen hinaus, und an einer Ecke kommt die wachsende Ellipse besonders leicht zuerst an; dort ist eine Koordinate exakt null. Zwingend ist das nicht: Wird das Budget c groß genug, rutscht auch die Lasso-Lösung auf eine Kante der Raute, und beide Koordinaten sind wieder besetzt. Am Regler des Widgets lassen sich beide Regime durchfahren. Der Ecken-Fall ist der geometrische Grund für die Sparsity des Lasso; formal steckt dahinter das Subdifferential der Betragsfunktion (Abschnitt 11.4).

Interaktiv: Kreis gegen Raute

Das Widget parametrisiert beide Mengen über ihren direkt sichtbaren Radius r : Es zeigt $\|\boldsymbol{\beta}\|_2 \leq r$ und $\|\boldsymbol{\beta}\|_1 \leq r$. Für Ridge entspricht das dem Budget $c = r^2$ in der Formel oben, für Lasso $c = r$. Die zulässigen Mengen sind dieselben; nur die Parametrisierung unterscheidet sich.

Schätzfrage. Bis zu welchem Radius r bleibt die Lasso-Lösung in der Ecke, setzt also β_2 exakt auf null?

Lösung: 1,342857 r

Der Ecken-Vermerk in der rechten Tafel verschwindet an dieser Schwelle.

Die Eckenschwelle liegt bei $r = 1,3429$: Darunter steht die Lasso-Lösung exakt auf $(r; 0)$, darüber rutscht sie auf eine Kante der Raute, und beide Koordinaten sind wieder besetzt. Die Ridge-Lösung wandert dagegen glatt am Kreisrand entlang und wird nie exakt null. Nach oben verschwindet der Zwang ganz: Ab $\|\hat{\boldsymbol{\beta}}\|_2 = 1,8358$ ist die Ridge-Nebenbedingung inaktiv, ab $\|\hat{\boldsymbol{\beta}}\|_1 = 2,5$ die des Lasso, und beide Male erzwingt die Komplementarität aus Satz 12.5.7 dann $\mu = 0$.

Satz 12.5.12 (KKT und Konvexität)

Sind f und alle h_j konvex und alle g_i affin, so ist jeder zulässige Punkt $\mathbf{x}^*$, der die KKT-Bedingungen erfüllt, ein globales Minimum.

Der Beweis ist eine Zeile: Mit festen λ_i^* und $\mu_j^* \geq 0$ ist $\mathcal{L}(\cdot, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*)$ konvex, die Stationarität macht $\mathbf{x}^*$ zu ihrem Minimierer, und für jedes zulässige $\mathbf{x}$ gilt

$$f(\mathbf{x}) \geq \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) \geq \mathcal{L}(\mathbf{x}^*, \boldsymbol{\lambda}^*, \boldsymbol{\mu}^*) = f(\mathbf{x}^*).$$

Die erste Ungleichung nutzt $g_i(\mathbf{x}) = 0$, $h_j(\mathbf{x}) \leq 0$ und $\mu_j^* \geq 0$, die letzte Gleichheit die Komplementarität.

Diese Richtung braucht also keine Regularitätsbedingung. Die umgekehrte aus Satz 12.5.7 braucht sie weiterhin, auch bei konvexen Problemen – die beiden Richtungen sind nicht symmetrisch, und die verbreitete Kurzformel „im konvexen Fall sind die KKT-Bedingungen notwendig *und* hinreichend“ verwischt genau das. Bei $\min x$ unter $h(x) = x^2 \leq 0$ sind Ziel und Nebenbedingung konvex, der einzige zulässige Punkt $x^* = 0$ ist das Optimum, und trotzdem erfüllt kein $\mu \geq 0$ die Stationarität $1 + \mu \cdot 2x^* = 0$: Wegen $x^* = 0$ steht links immer 1.

Für konvexe Probleme ist ein KKT-Punkt demnach bereits ein globales Optimum, und viele statistische Probleme sind konvex: Kleinste Quadrate, Ridge, Lasso, die negative Log-Likelihood einer Exponentialfamilie in ihrer kanonischen Parametrisierung (Abschnitt 11.5). Bei nicht-konvexen Problemen wie neuronalen Netzen liefern die KKT-Bedingungen dagegen nur Kandidaten, unter denen auch lokale Optima und Sattelpunkte sind (Abschnitt 12.2).

Sind Ziel und alle Nebenbedingungen affin, heißt die Aufgabe *lineares Programm*; ihr zulässiger Bereich ist dann ein Polyeder (Abschnitt 11.2), und dafür gibt es eigene Verfahren, die dieses Skript nicht behandelt.

Selbsttest

1. Ist $\mathbf{x}^*$ ein lokales Minimum von f unter $g(\mathbf{x}) = 0$, so gibt es immer ein λ^* mit $\nabla f(\mathbf{x}^*) + \lambda^* \nabla g(\mathbf{x}^*) = \mathbf{0}^\top$.

Antwort: Falsch

Nur unter der Bedingung aus Satz 12.5.5. Bei $f(x, y) = x$ unter $g(x, y) = y^2 - x^3 = 0$ liegt das Minimum im Ursprung, dort ist $\nabla g(0, 0) = \mathbf{0}^\top$, und $\nabla f = (1, 0)$ lässt sich durch kein Vielfaches des Nullvektors ausgleichen.

2. Bei einer Gleichungsnebenbedingung darf der Multiplikator den Wert null annehmen.

Antwort: Wahr

Das ist Bemerkung 12.5.3. Minimieren wir etwa $f(x, y) = x^2 + y^2$ unter $x + y = 0$, so liegt das unbeschränkte Minimum $(0, 0)$ selbst auf der Nebenbedingung; dort ist $\nabla f = \mathbf{0}^\top$, und die Stationarität gilt mit $\lambda^* = 0$.

3. Ist eine Ungleichung im Optimum aktiv, so ist ihr Multiplikator positiv.

Antwort: Falsch

Die Komplementarität $\mu_j h_j(\mathbf{x}^*) = 0$ ist auch dann erfüllt, wenn beide Faktoren null sind. Bei $\min x^2$ unter $h(x) = x \leq 0$ ist $x^* = 0$, die Ungleichung also aktiv, und die Stationarität $2x^* + \mu = 0$ liefert trotzdem $\mu = 0$. Nur die andere Richtung trägt: inaktiv erzwingt $\mu_j = 0$.

4. Ist die Lösung der penalisierten Ridge-Regression zu $\lambda > 0$ von null verschieden, so liefert $c = \|\hat{\boldsymbol{\beta}}(\lambda)\|_2^2$ eine bindende beschränkte Form mit derselben Lösung.

Antwort: Wahr

Wir lösen die penalisierte Form und setzen $c := \|\hat{\boldsymbol{\beta}}(\lambda)\|_2^2$. Dann ist $\hat{\boldsymbol{\beta}}(\lambda)$ zulässig, die Nebenbedingung aktiv, und mit $\mu = \lambda$ sind alle KKT-Bedingungen erfüllt; Satz 12.5.12 macht daraus das globale Optimum. Für X und y aus Beispiel 10.6.6 (Abschnitt 10.6) und $\lambda = 1,5$ ist das $\hat{\boldsymbol{\beta}} = (0,341; 0,578)$ mit $c = 0,450$. Die Umkehrung gilt dagegen nur, solange die Nebenbedingung bindet. Ist die penalisierte Lösung null, ergibt die aktive Zuordnung $c = 0$; genau deshalb steht die Nichtnull-Voraussetzung im Selbsttest.

5. Bei einem konvexen Problem erfüllt das globale Minimum stets die KKT-Bedingungen.

Antwort: Falsch

Satz 12.5.12 trägt nur die andere Richtung. Bei $\min x$ unter $h(x) = x^2 \leq 0$ sind Ziel und Nebenbedingung konvex, der einzige zulässige Punkt $x^* = 0$ ist damit das Optimum, und trotzdem scheitert die Stationarität $1 + \mu \cdot 2x^* = 0$ an jedem $\mu \geq 0$. Für die notwendige Richtung braucht es eine Regularitätsbedingung.

6. Die Lasso-Lösung muss nicht in einer Ecke der Raute liegen.

Antwort: Wahr

Die Ecke gewinnt nur, solange das Budget klein genug ist. Im Widget sitzt die Lösung für $c \leq 1,30$ exakt in $(c; 0)$, bei $c = 1,40$ dagegen liest das Readout $(1,36; 0,04)$: beide Koordinaten sind besetzt. Die Sparsity des Lasso ist eine Tendenz der Geometrie, keine Garantie.

Vertiefung: Heath behandelt die beschränkte Optimierung in §6.2.3 (Lagrange-Multiplikatoren, hinreichende Bedingungen) sowie §6.7 (Verfahren für Probleme mit Nebenbedingungen); die konvexe Theorie samt KKT-Dualität entwickeln Boyd und Vandenberghe, Convex Optimization, Kapitel 5.

12.6 Optimierung in R und Zusammenfassung

Zum Abschluss übersetzen wir die Verfahren dieses Kapitels in R-Aufrufe. Die beiden Arbeitspferde sind `optimize()` für univariate und `optim()` für multivariate Zielfunktionen. Die Ausgaben der Aufrufe drucken wir hier nicht ab; stattdessen rechnen wir die Beispielfunktion im Widget selbst nach. Das Widget läuft mit Gradientenabstieg und nicht mit den Verfahren aus `optim()`, es zeigt also die Landschaft und ihre Mulden, nicht den Iterationsweg von R.

Univariat: optimize

```
f <- function(x) (x - 3)^2
optimize(f, interval = c(0, 10))
```

Der Aufruf sucht das Minimum von f auf dem Intervall $[0, 10]$. Das analytische Minimum ist $x^* = 3$ mit $f(x^*) = 0$, und weiter als bis auf das voreingestellte `tol` kommt der Aufruf nicht heran: Es steht auf `.Machine$double.eps^0.25`, also auf rund $1,2 \cdot 10^{-4}$.

Bemerkung 12.6.1 (Golden Section Search)

Im Kern verwendet `optimize()` die *Golden Section Search*, eine Verwandte der Bisektion aus Abschnitt 12.1: Statt eines Vorzeichenwechsels von f schachtelt sie ein Minimum ein. Dazu hält sie zwei innere Testpunkte, die das Intervall im Verhältnis des goldenen Schnitts teilen; in jedem Schritt fällt das Teilintervall weg, das das Minimum nicht enthalten kann, und ein einziger neuer Funktionswert genügt, weil der andere Testpunkt wiederverwendet wird. Das Intervall schrumpft pro Schritt auf das 0,618-fache, ganz ohne Ableitungen; verlässlich ist das für *unimodale* Funktionen: Bis zu einer Minimalstelle steigen sie nicht an, danach fallen sie nicht mehr. Ein bloß eindeutiges globales Minimum ohne diese Formbedingung genügt nicht, weil zusätzliche Wellen die Intervallsuche täuschen können.

Der goldene Schnitt ist aber nur der eine Baustein. Tatsächlich kombiniert `optimize()` ihn mit einer parabolischen Interpolation: Solange sich die drei zuletzt berechneten Punkte gutartig verhalten, springt der nächste Testpunkt in den Scheitel der Parabel durch sie, und erst

wenn dieser Sprung aus dem eingeschachtelten Intervall führt oder zu wenig einbringt, greift wieder der goldene Schnitt. Die Einschachtelung bleibt dabei erhalten; ein ungeeigneter Parabelschritt zerstört also nicht die robuste Intervallverkleinerung des goldenen Schnitts.

Multivariat: optim

Als Testfall dient uns die Funktion

$$f(\mathbf{x}) = \log(1 + (x_1^2 + \sin 3x_2)^2) + 0,1 x_1^2 + 0,1 x_2^2, \quad (12.6.1)$$

deren erster Term entlang der Kurven $x_1^2 + \sin 3x_2 = 0$ verschwindet und deren Regularisierungsterme die Werte zum Ursprung hin senken. Sie hat das globale Minimum $\mathbf{x}^* = (0; 0)$ mit $f(\mathbf{x}^*) = 0$, und im Fenster des Widgets liegen daneben genau zwei lokale Mulden, bei $(0; \pm 1,04)$ mit $f \approx 0,11$. Die Regularisierung verschiebt sie leicht: Der erste Term verschwindet bei $x_2 = \pm\pi/3 = \pm 1,047$, die Talsohle liegt bei $\pm 1,036$.

```
f <- function(x) log1p((x[1]^2 + sin(3*x[2]))^2) +
  .1 * x[1]^2 + .1 * x[2]^2
optim(c(-1, -0.5), f) # Default: Nelder-Mead
optim(c(-1, -0.5), f, method = "BFGS")
```

Ohne method-Argument verwendet `optim()` das ableitungsfreie Nelder-Mead-Verfahren aus Abschnitt 12.3; mit `method = "BFGS"` das Quasi-Newton-Verfahren aus Abschnitt 12.4, dessen Gradienten es mangels `gr`-Argument per finiter Differenzen schätzt. Ein über `gr` mitgeliefertes Gradientenfeld wechselt die Methode übrigens nicht von selbst; Nelder-Mead bleibt die Voreinstellung, bis `method` etwas anderes sagt. Von diesem Startpunkt aus landen beide Verfahren im globalen Minimum.

Bemerkung 12.6.2 (Falsche Konvergenz, und keiner warnt)

Zwei Fallen zeigt schon dieses kleine Beispiel; drei Aufrufe genügen:

```
optim(c(-1, 1), f) # Nelder-Mead
optim(c(-0.5, -1), f, method = "BFGS") # BFGS
optim(c(-1, -0.5), f, control = list(maxit = 50)) # abgeschnitten
```

Lokale Minima. Die Beispielfunktion hat neben dem globalen Minimum zwei spiegelbildliche Mulden bei $(0; \pm 1,04)$. Vom Startpunkt $(-1; 1)$ aus liegt die obere davon im Weg, von $(-0,5; -1)$ aus die untere; der Gradientenabstieg im Widget landet von beiden Startpunkten genau dort. Das Verfahren meldet dann Konvergenz, obwohl der gefundene Wert $f \approx 0,11$ über dem globalen Minimum 0 liegt. Der convergence-Code von `optim()` sagt nur, dass das Verfahren zum Stillstand kam, nicht, dass das Ergebnis global optimal ist (Abschnitt 12.2). *Abgebrochene Läufe.* Begrenzen wir die Iterationen, etwa mit `control = list(maxit = 50)`, kann `optim()` mitten am Hang stehen bleiben. Das Ergebnis sieht aus wie immer, nur der convergence-Code (dann $\neq 0$) verrät den Abbruch; eine Warnung gibt es nicht. Abbruchkriterien prüfen heißt hier: den Rückgabewert lesen, nicht auf `warnings()` warten (Abschnitt 12.3, Abbruchkriterien).

Wie schlimm ist das in der Praxis? Probieren wir es an der Beispielfunktion selbst aus.

Interaktiv: Die Landkarte zur Beispielfunktion

Das Widget zeichnet die Funktion aus (12.6.1) als Karte und lässt uns den Startpunkt setzen, per Klick, per Zug oder über die beiden Regler. Weil das Widget mit Gradientenabstieg rechnet, sind das die Einzugsgebiete *dieses* Verfahrens; Nelder-Mead und BFGS laufen andere Wege und können deshalb an anderen Startpunkten umkippen als die blaue Spur.

Schätzfrage. Wie viele verschiedene Grenzwerte findet der Gradientenabstieg auf diesem Ausschnitt?

Lösung: drei

Es sind das globale Minimum (0; 0) und zwei lokale Mulden bei (0; ±1,04).

Es sind genau drei. Von (−1; −0,5) aus geht es ins globale Minimum (0; 0) mit $f = 0$, von (−1; 1) in die obere und von (−0,5; −1) in die untere Mulde, beide bei (0; ±1,0357) mit $f = 0,1085$. Alle drei Läufe enden ordnungsgemäß – Gradient null, Hesse-Matrix positiv definit, kein Warnhinweis –, und trotzdem landen zwei von dreien am falschen Ort. Gegen falsche Konvergenz hilft deshalb keine bessere Schrittweite, sondern nur ein zweiter Startpunkt.

Der analytische Gradient

Wer den Gradienten kennt, sollte ihn `optim()` übergeben: Finite Differenzen kosten pro Gradient zusätzliche Funktionsauswertungen, bei einseitigen Differenzen n und bei zentralen $2n$, und sie bringen Rundungsfehler mit (Kapitel 4); der analytische Gradient ist schneller und genauer.

Beispiel 12.6.3 (Kettenregel für die Beispielfunktion)

Mit $u(\mathbf{x}) = x_1^2 + \sin 3x_2$ ist $f = \log(1 + u^2) + 0,1x_1^2 + 0,1x_2^2$, und die Kettenregel (Abschnitt 10.6) liefert wegen $\frac{d}{du} \log(1 + u^2) = \frac{2u}{1+u^2}$:

$$\nabla f(\mathbf{x}) = \left(\frac{4x_1 u}{1 + u^2} + 0,2 x_1, \quad \frac{6 \cos(3x_2) u}{1 + u^2} + 0,2 x_2 \right).$$

```
grad_f <- function(x) {
  u <- x[1]^2 + sin(3*x[2])
  d <- 2 * u / (1 + u^2)
  c(d * 2 * x[1] + .2 * x[1],
    d * 3 * cos(3*x[2]) + .2 * x[2])
}
optim(c(-1, -0.5), f, gr = grad_f, method = "BFGS")
optim(c(-1, 0.5), f, gr = grad_f, method = "L-BFGS-B",
      lower = c(-1, -1), upper = c(1, 1))
```

L-BFGS-B ist die speicherschonende BFGS-Variante aus Abschnitt 12.4, erweitert um Box-Nebenbedingungen `lower/upper`; damit sind wir zurück bei der beschränkten Optimierung aus Abschnitt 12.5.

Wann welches Verfahren

Die Methodenwahl aus diesem Kapitel, zusammengefasst:

Methode	Wann verwenden?
Nelder-Mead	Keine Ableitungen, niedrige Dimension ($n \leq 10$)
Gradientenabstieg	Große Probleme, einfache Implementierung
Quasi-Newton (BFGS)	Mittlere Probleme, guter Kompromiss
Newton	Kleine Probleme, schnelle Konvergenz nötig
Momentum	Gradientenabstieg bei schlecht konditionierten Problemen
SGD	Sehr große Datensätze und hohe Dimension
Lagrange/KKT	Nebenbedingungen

Ein verbreitetes Missverständnis dazu: BFGS ist zwar in vielen Optimierungsbibliotheken die Voreinstellung, in `optim()` aber nicht. Dort ist Nelder-Mead die Voreinstellung, und daran ändert auch ein über `gr` übergebener Gradient nichts. Der gute Kompromiss ist BFGS trotzdem, wir müssen ihn nur anfordern.

Was bleibt

Bemerkung 12.6.4 (Kernkonzepte des Kapitels)

1. **Nichtlineare Gleichungen** lösen wir iterativ: Die Bisektion halbiert verlässlich, aber langsam; Newton-Raphson konvergiert lokal quadratisch, $e_{k+1} \approx C e_k^2$ mit einer Konstanten C , die von f abhängt, und überträgt sich wörtlich ins Multivariate. Wo die Jacobimatrix pro Schritt zu teuer wird, ersetzt die Fixpunktiteration sie durch eine feste Schrittweite (Abschnitt 12.1).
2. **Optimalität** heißt stationär plus Krümmungsbedingung; ohne Konvexität drohen lokale Minima und vor allem Sattelpunkte (Abschnitt 12.2).
3. **Gradientenabstieg** lebt von der Schrittweite: Konvergenzraten $O(1/k)$ beziehungsweise linear mit $\rho = 1 - \gamma\mu$, für $\gamma = 1/L$ also $\rho = 1 - \mu/L$, und die Kondition $\kappa_f = L/\mu$ bestimmt das Zickzack (Abschnitt 12.3).
4. **Newton und Quasi-Newton** kaufen mit Krümmungsinformation schnellere Konvergenz; BFGS approximiert die Hesse-Inverse aus Gradienten, SGD ersetzt den Gradienten durch den einer zufällig gezogenen Beobachtung, der im Mittel über diese Ziehung der gesuchte ist (Abschnitt 12.4).
5. **Nebenbedingungen** behandeln Lagrange-Multiplikatoren und die KKT-Bedingungen; Ridge und Lasso sind die statistischen Musterbeispiele (Abschnitt 12.5).

Selbsttest

Sechs Aussagen quer durch das Kapitel.

1. Die Bisektion auf $[0, 8]$ braucht für die Zielgenauigkeit $\epsilon = 10^{-3}$ genau 13 Halbierungen.

Antwort: Wahr

Nach Satz 12.1.8 sind es $\lceil \log_2((b-a)/\epsilon) \rceil = \lceil \log_2 8000 \rceil$ Schritte, und wegen $2^{12} = 4096 < 8000 \leq 8192 = 2^{13}$ sind das 13.

2. Der Gradientenabstieg auf $f(x) = x^2$ konvergiert für jede feste Schrittweite $\gamma > 0$.

Antwort: Falsch

Der Schritt lautet $x^{(k+1)} = (1 - 2\gamma)x^{(k)}$. Für $\gamma \geq 1$ ist $|1 - 2\gamma| \geq 1$: Bei $\gamma = 1$ springt die Iteration ewig zwischen $x^{(0)}$ und $-x^{(0)}$, für $\gamma > 1$ wächst der Betrag sogar. Konvergenz braucht hier $\gamma < 1 = 2/L$ mit $L = 2$ (Abschnitt 12.3).

3. Auf einer strikt konvexen quadratischen Funktion findet das Newton-Verfahren das Minimum in einem einzigen Schritt, unabhängig vom Startpunkt.

Antwort: Wahr

Die quadratische Taylor-Approximation einer Quadrik ist die Quadrik selbst, der Newton-Schritt springt also direkt in deren Minimum (Beispiel in Abschnitt 12.4; für $f(x) = x^2$ mit $x^{(0)} = 4$: $x^{(1)} = 4 - 8/2 = 0$).

4. In den KKT-Bedingungen müssen auch die Multiplikatoren λ_i der Gleichungsnebenbedingungen nichtnegativ sein.

Antwort: Falsch

Nur die μ_j der Ungleichungen tragen die Vorzeichenbedingung $\mu_j \geq 0$; die λ_i sind frei. Beispiel 12.5.6 hat $\lambda^* = -1$, und alles ist in Ordnung.

5. Ist eine Ungleichungs-Nebenbedingung im Optimum inaktiv, so ist ihr KKT-Multiplikator null.

Antwort: Wahr

Das ist die Komplementarität $\mu_j h_j(\mathbf{x}^*) = 0$ aus Satz 12.5.7: Aus $h_j(\mathbf{x}^*) < 0$ folgt $\mu_j = 0$. In Beispiel 12.5.9 gilt genau das für die untere Schranke.

6. Welchen Funktionswert erreicht der Gradientenabstieg im Landkarten-Widget, wenn wir bei $(-1; 1)$ starten?

Lösung: 0.1085

Er landet in der oberen Mulde bei $(0; 1,0357)$ mit $f = 0,1085$. Das globale Minimum liegt bei $(0; 0)$ mit $f = 0$, also gut ein Zehntel tiefer. Beide Läufe enden mit verschwindendem Gradienten und ohne Warnung – genau das meint Bemerkung 12.6.2 mit „falscher Konvergenz“.

7. Weil $-\nabla f$ die Richtung des steilsten Abstiegs ist, verkleinert jeder Gradientenabstiegs-Schritt den Funktionswert.

Antwort: Falsch

Die Richtung stimmt, solange $\nabla f \neq \mathbf{0}^\top$ ist, aber die Schrittweite kann den Abstieg überschießen: Auf $f(x) = x^2$ mit $\gamma = 1$ gilt $x^{(k+1)} = -x^{(k)}$ und der Funktionswert bleibt exakt gleich, mit $\gamma > 1$ wächst er. Genau dagegen sichert die Armijo-Bedingung aus Abschnitt 12.3 ab.

Vertiefung: Heath, Scientific Computing, behandelt in Kapitel 5 die nichtlinearen Gleichungen und in Kapitel 6 die Optimierung, jeweils mit den hier gezeigten Verfahren; die R-Seite dokumentieren die Hilfeseiten zu `optimize()` und `optim()`. Zum stochastischen Gradientenabstieg führt der MSc-Kurs Optimization for ML weiter.

Funktionsapproximation

13.1 Approximation, Interpolation, Glättung

Mit diesem Kapitel beginnt der dritte Block des Skripts (Abschnitt 1.1). Die Aufgabe klingt zunächst harmlos: Gegeben sind n Datenpunkte $(x_1, y_1), \dots, (x_n, y_n)$ mit paarweise verschiedenen x_i , gesucht ist eine Funktion $\hat{f}$, die zu ihnen passt. Manchmal stammen die Punkte gar nicht aus einer Messung, sondern sind Auswertungen einer bekannten Funktion f , die zu teuer ist, um sie an beliebig vielen Stellen auszurechnen.

Interaktiv: Landkarte der Funktionsapproximation

Wie die Konzepte dieses Kapitels zusammenhängen und auf welche Bausteine aus den Kapiteln 1–9 sie zurückgreifen, zeigt die Karte; Teil 2 wird hier nirgends gebraucht. Beim ersten Lesen genügt der Gesamteindruck.

Interaktives Element — nur in der Web-Fassung

Wählen wir einen Begriff aus, hebt die Karte seine Voraussetzungen und Folgerungen hervor; verlinkte Kästen führen direkt zum zugehörigen Kapitel.

„Passt“ lässt sich allerdings auf sehr verschiedene Weisen verlangen, und deshalb stehen gleich drei Varianten nebeneinander. Der Rechenweg bleibt dabei der aus den bisherigen Kapiteln: Am Ende steht wieder ein lineares Gleichungssystem, neu ist nur, dass die Unbekannten darin die Koeffizienten einer Funktion sind.

Vorkenntnisse

Der Abschnitt führt kaum eigene Technik ein, greift dafür aber auf mehrere frühere Kapitel zurück.

- Lineare Gleichungssysteme $\mathbf{B}\mathbf{a} = \mathbf{y}$ und ihre Lösbarkeit (Abschnitt 5.2). Jede Interpolationsaufgabe dieses Kapitels endet in einem solchen System.
- Die Konditionszahl einer Matrix als Maß dafür, wie stark sich Daten- und Rundungsfehler verstärken (Abschnitt 4.2 und Abschnitt 3.5). Sie entscheidet ab Abschnitt 13.3 darüber, welches Basissystem brauchbar ist.
- Basen und lineare Unabhängigkeit, diesmal in einem Vektorraum, dessen Elemente Funktionen sind.
- Polynome samt Polynomgrad und Monombasis, Stetigkeit und Differenzierbarkeit, außerdem stückweise definierte Funktionen.
- Normen (Abschnitt 3.2), hier vor allem die *Supremumsnorm* einer Funktion: Für g auf $[a, b]$ ist $\|g\|_\infty = \sup_{x \in [a, b]} |g(x)|$, bei stetigem g auf einem abgeschlossenen Intervall also der

größte Betrag, den g dort annimmt.

Drei Varianten desselben Wunsches

Alle drei Aufgaben liefern am Ende ein $\hat{f}$, verlangen aber sehr Verschiedenes von ihm.

Definition 13.1.1 (Approximationsproblem)

Gegeben seien eine Funktion f auf $[a, b]$, eine Norm $\|\cdot\|$ auf einem Funktionenraum, der f enthält, und eine vorgegebene, einfacher handhabbare Klasse $\mathcal{A}$ von Funktionen. Gesucht ist ein $\hat{f} \in \mathcal{A}$, das den Abstand zu f minimiert oder dem bestmöglichen Abstand wenigstens nahek kommt:

$$\|f - \hat{f}\| \approx \inf_{g \in \mathcal{A}} \|f - g\|.$$

Die Bedingung ist global: $\hat{f}$ soll f überall auf $[a, b]$ nahek kommen, muss es aber an keiner einzigen Stelle exakt treffen. Existiert ein Minimierer, steht in der Anzeige exakt ein Gleichheitszeichen.

Definition 13.1.2 (Interpolationsproblem)

Gegeben seien Datenpunkte (x_i, y_i) , $i = 1, \dots, n$, mit paarweise verschiedenen x_i . Gesucht ist ein $\hat{f}$ mit

$$\hat{f}(x_i) = y_i, \quad i = 1, \dots, n. \quad (13.1.1)$$

Jede Funktion mit dieser Eigenschaft heißt *Interpolant* (interpolant) der Daten, die x_i heißen *Stützstellen* (nodes). Die Bedingung ist punktweise und exakt: An den n Stützstellen ist nichts erlaubt außer Treffen, zwischen ihnen alles.

Definition 13.1.3 (Glättungsproblem)

Gegeben seien Datenpunkte (x_i, y_i) , $i = 1, \dots, n$, die aus einer unbekannten Funktion f und Fehlern entstanden sind:

$$y_i = f(x_i) + \varepsilon_i, \quad i = 1, \dots, n.$$

Gesucht ist eine Schätzung $\hat{f}$ von f , die das Signal trifft, statt jeden beobachteten Fehler mitzuzeichnen. Die ε_i sind in aller Regel Zufallsvariablen; die Residuen $y_i - \hat{f}(x_i)$ sind dagegen aus den Daten berechenbare Abweichungen und nicht mit den unbeobachteten Fehlern gleichzusetzen.

Bemerkung 13.1.4 (Daten oder Funktion?)

Die Interpolationsbedingung wird mal als $f(x_i) = \hat{f}(x_i)$ geschrieben, mal als $y_i = \hat{f}(x_i)$. Beides meint dasselbe, solange die Daten rauschfrei sind, also $y_i = f(x_i)$ gilt. Der Unterschied wird erst bei der Glättung wichtig: Dort ist f unbekannt und nur die verrauschten y_i liegen vor, weshalb die Bedingung $\hat{f}(x_i) = f(x_i)$ gar nicht überprüfbar wäre. Wir schreiben deshalb durchgehend y_i und sagen dazu, ob $y_i = f(x_i)$ gelten soll.

Interaktiv: Die drei Aufgaben nebeneinander

Wann wird aus Glättung wieder Interpolation? Wir variieren dafür nur das Rauschen.

Interaktives Element — nur in der Web-Fassung

Wie die drei Aufgaben zusammenhängen**Bemerkung 13.1.5 (Approximation über Interpolation)**

Ein wichtiger Weg zu einer Approximation führt über Interpolation: Statt die Norm $\|f - \hat{f}\|$ direkt über einem Funktionenraum zu minimieren, werten wir f an endlich vielen Stellen x_i aus und interpolieren die so entstandenen Paare $(x_i, f(x_i))$. Damit ist das konkrete Approximationsverfahren auf ein Interpolationsproblem zurückgeführt, und aus dessen Bedingung (13.1.1) wird in Abschnitt 13.2 ein lineares Gleichungssystem. Andere Verfahren minimieren eine Norm tatsächlich direkt, etwa Kleinste Quadrate oder eine Minimax-Approximation. Sind die Auswertungen fehlerbehaftet, tritt an die Stelle der Interpolation ein Glättungsproblem.

Von den drei Varianten ist die Glättung die schwierigste: Sie muss zusätzlich entscheiden, welcher Teil der Daten Signal und welcher Rauschen ist, und dafür braucht es ein stochastisches Modell für die ε_i . Dieses Kapitel behandelt deshalb die Interpolation; die Glättung folgt in Abschnitt 13.7.

Wozu wir das brauchen

Klassisch sind drei Anlässe. Erstens die glatte Kurve durch eine Punktwolke, etwa für eine Grafik. Zweitens der Wunsch, eine teure Funktion schneller auszuwerten: Wenn $f(x) = \int_0^x g(y) dy$ oder $f(x) = g^{-1}(x)$ jedes Mal ein numerisches Verfahren erfordert, ist es billiger, f an wenigen Stellen zu bestimmen und dazwischen zu interpolieren. Drittens Ableitungen und Integrale von f : Ein Polynom oder ein Spline lässt sich exakt differenzieren und integrieren, und wir nehmen das Ergebnis als Näherung für f' beziehungsweise für $\int f$. Auf diesem Umweg beruhen die klassischen Verfahren der numerischen Differentiation und Integration, die dieses Skript nicht weiter verfolgt.

Bemerkung 13.1.6 (Interpolation im maschinellen Lernen, und was nur so aussieht)

Zwei der gängigen Anwendungsbeispiele sind Interpolation im Sinn von Definition 13.1.2:

- *Computergrafik.* Wird ein Bild gedreht oder skaliert, liegen die neuen Pixelmittelpunkte zwischen den alten. Bilineare und bikubische Verfahren legen eine Funktion durch die umliegenden Pixelwerte und werten sie an der neuen Stelle aus.
- *Zeitreihen.* Fehlende Werte in Sensor- oder Finanzdaten werden aufgefüllt, indem eine Kurve durch die vorhandenen Messzeitpunkte gelegt wird.

Zwei weitere werden gern danebengestellt, sind aber bloße Analogien:

- *Latent-Space-Interpolation* in VAEs und GANs interpoliert linear zwischen zwei Codes, $z(t) = (1 - t)z_0 + tz_1$. Das ist Interpolation im Argumentraum mit $n = 2$ Stützstellen. Über das erzeugte Bild sagt sie nichts: Ob der Übergang glatt aussieht, hängt am Decoder, nicht an einer Interpolationsbedingung.

- *Positional Encodings* wie RoPE werten feste Sinus- und Kosinusfunktionen an den Token-Positionen aus. Eine Bedingung der Form $\hat{f}(x_i) = y_i$ kommt darin nicht vor; gemeinsam ist mit unserem Thema nur der Baukasten aus Sinusfunktionen. Interpolation im engen Sinn taucht dort erst auf, wenn ein trainiertes Modell auf längere Kontexte umskaliert wird.

Ein echter Grenzfall ist die Idee, die feste Aktivierungsfunktion eines neuronalen Netzes durch eine lernbare, spline-basierte zu ersetzen (*Kolmogorow-Arnold-Netze*). Die Splines aus Abschnitt 13.4 sind dort das Modell selbst; ihre Koeffizienten werden allerdings trainiert und nicht aus Interpolationsbedingungen bestimmt.

Warum die Interpolationsbedingung zu wenig ist

Interpolation stellt n Gleichungen an eine Funktion. Eine Funktion hat unendlich viele Freiheitsgrade, und deshalb bleibt nach diesen n Gleichungen sehr viel offen. Wie viel, zeigt schon das kleinste sinnvolle Beispiel.

Beispiel 13.1.7 (Vier Interpolanten durch drei Punkte)

Betrachten wir die $n = 3$ Punkte $(0, 1)$, $(1, 2)$, $(2, 5)$. Die folgenden vier Funktionen interpolieren sie alle:

$$\begin{aligned}\hat{f}_1(x) &= 1 + x^2, \\ \hat{f}_2(x) &= \begin{cases} x + 1 & x \in [0, 1] \\ 3x - 1 & x \in [1, 2] \end{cases}, \\ \hat{f}_3(x) &= x^3 - 2x^2 + 2x + 1, \\ \hat{f}_4(x) &= 1 + x^2 + 0,5 \sin(2\pi x).\end{aligned}$$

Die Probe ist in allen vier Fällen eine Zeile. Für $\hat{f}_1$ etwa ist $1 + 0 = 1$, $1 + 1 = 2$ und $1 + 4 = 5$. Für $\hat{f}_3$ rechnen wir $0 - 0 + 0 + 1 = 1$, $1 - 2 + 2 + 1 = 2$ und $8 - 8 + 4 + 1 = 5$. Bei $\hat{f}_4$ genügt der Hinweis, dass $\sin(2\pi x)$ an jeder ganzen Zahl verschwindet, und $\hat{f}_2$ ist auf beiden Teilintervallen die Gerade durch die jeweiligen Endpunkte.

Zwischen den Stützstellen sehen die vier Funktionen dagegen völlig verschieden aus. Bei $x = 1,3$ liefern sie 2,69, 2,90, 2,42 und 3,17; die höchste und die niedrigste trennt dort 0,75. Viel weiter gehen sie auf $[0, 2]$ nirgends auseinander, die größte Spanne beträgt 0,7499 und wird bei $x \approx 1,288$ erreicht.

Interaktiv: Vier Interpolanten, ein Datensatz

Wie weit können vier Interpolanten, die dieselben drei Punkte treffen, zwischen zwei Stützstellen auseinanderliegen?

Schätzfrage. Schätzen wir die größte Spanne auf $[0, 2]$.

Lösung: 0,75

Wie das Widget zeigt, liegt die größte Spanne bei etwa 0,75; Satz 13.1.8 erklärt, warum die Interpolationsbedingungen diese Zwischenwerte nicht festlegen.

Vier Beispiele sind noch kein Argument. Der folgende Satz beschreibt die gesamte Lösungsmenge und macht damit klar, dass es nicht bei vieren bleibt.

Satz 13.1.8 (Gestalt aller Interpolanten)

Seien Daten (x_i, y_i) , $i = 1, \dots, n$, gegeben und sei p irgendein Interpolant dieser Daten. Dann ist eine Funktion $\hat{f}$ genau dann ebenfalls ein Interpolant, wenn sie sich schreiben lässt als

$$\hat{f} = p + g \quad \text{mit} \quad g(x_i) = 0 \quad \text{für } i = 1, \dots, n. \quad (13.1.2)$$

Vertiefung: Beweis der allgemeinen Gestalt aller Interpolanten

Beweis.

- (1) Sei zunächst g eine Funktion mit $g(x_i) = 0$ für alle i . Dann gilt für jedes i

$$(p + g)(x_i) = p(x_i) + g(x_i) = y_i + 0 = y_i,$$

also interpoliert $p + g$ die Daten.

Funktionen werden punktweise addiert, und p ist nach Voraussetzung ein Interpolant

- (2) Sei umgekehrt $\hat{f}$ ein Interpolant. Wir setzen $g := \hat{f} - p$ und erhalten für jedes i

$$g(x_i) = \hat{f}(x_i) - p(x_i) = y_i - y_i = 0.$$

Mit diesem g ist $\hat{f} = p + g$ von der Form (13.1.2).

beide Funktionen erfüllen die Interpolationsbedingung (13.1.1) an derselben Stelle x_i

□

Bemerkung 13.1.9 (Unendlich viele Lösungen)

Die Menge der Funktionen g mit $g(x_i) = 0$ für alle i ist ein Untervektorraum des Funktionenraums, und er ist unendlichdimensional: Für jedes Polynom q liegt $q(x) \prod_{i=1}^n (x - x_i)$ darin, und diese Produkte sind für verschiedene Grade von q linear unabhängig. Schon der Spezialfall $g_c(x) = c \prod_{i=1}^n (x - x_i)$ liefert für jedes $c \in \mathbb{R}$ einen weiteren Interpolanten. Damit hat das Interpolationsproblem aus Definition 13.1.2 stets unendlich viele Lösungen, sobald wir $\hat{f}$ nicht weiter einschränken.

Die drei Zusatzfunktionen aus Beispiel 13.1.7 sind genau von dieser Bauart. Mit $p = \hat{f}_1$ ist

$$\hat{f}_3 - p = x(x-1)(x-2), \quad \hat{f}_4 - p = 0,5 \sin(2\pi x),$$

und beide verschwinden an 0, 1 und 2; das erste ist gerade g_1 von oben, das zweite eine Sinusschwingung, die zufällig an denselben Stellen Nullstellen hat. Auch die stückweise lineare Differenz $\hat{f}_2 - p$ verschwindet dort, sie lautet $x(1-x)$ auf $[0, 1]$ und $-(x-1)(x-2)$ auf $[1, 2]$.

Interpolation allein legt also nichts fest. Wir müssen die Auswahl einschränken, und zwar so, dass genau so viele Freiheitsgrade übrig bleiben, wie wir Datenpunkte haben. Das leisten *Basissysteme*: Wir wählen K Basisfunktionen und lassen nur noch Linearkombinationen aus ihnen zu. Aus der Interpolationsbedingung wird dann ein lineares Gleichungssystem in den K Koeffizienten (Abschnitt 13.2), und die Wahl der Basis entscheidet über alles Weitere: über Eindeutigkeit (Abschnitt 13.3), über die Kondition des Systems und über die Frage, ob eine lokale Datenänderung die Kurve global verbiegt (Abschnitt 13.4).

Selbsttest

1. Zu Daten (x_i, y_i) , $i = 1, \dots, n$, mit paarweise verschiedenen x_i gibt es unendlich viele Funktionen $\hat{f}$ mit $\hat{f}(x_i) = y_i$ für alle i .

Antwort: Wahr

Das ist Bemerkung 13.1.9. Ist p ein Interpolant, so ist nach Satz 13.1.8 auch $p + c \prod_i (x - x_i)$ für jedes $c \in \mathbb{R}$ einer, und verschiedene c liefern verschiedene Funktionen.

2. Aus $\hat{f}(x_i) = y_i$ für $i = 1, \dots, n$ folgt $\|f - \hat{f}\| \approx 0$.

Antwort: Falsch

Die Interpolationsbedingung sagt über die Stellen zwischen den Stützstellen nichts. In Beispiel 13.1.7 erfüllen alle vier Funktionen dieselben drei Bedingungen und liegen bei $x = 1,3$ trotzdem 0,75 auseinander; höchstens eine von ihnen kann einem gegebenen f nahekommen. Wie groß der Abstand tatsächlich wird, klärt erst eine Fehlerabschätzung für eine konkrete Funktionenklasse.

3. Beschränken wir $\hat{f}$ auf Polynome vom Grad höchstens $n - 1$, so hat das Interpolationsproblem bei paarweise verschiedenen x_i genau eine Lösung.

Antwort: Wahr

Das ist der Eindeigkeitsatz der Polynominterpolation aus Abschnitt 13.3. Ohne den Zusatz „vom Grad höchstens $n - 1$ “ wäre die Aussage falsch; so gelesen widerspräche sie der ersten Aussage dieses Selbsttests.

4. Beim Glättungsproblem verlangen wir wie bei der Interpolation, dass $\hat{f}$ durch alle Datenpunkte läuft.

Antwort: Falsch

Definition 13.1.3 lässt die Residuen $\epsilon_i = y_i - \hat{f}(x_i)$ ausdrücklich zu. Bei verrauschten Daten wäre eine Kurve durch alle Punkte die falsche Antwort, weil sie das Rauschen mit abbildet. Im Widget der ersten Vertiefung fallen die beiden Aufgaben nur für $\sigma = 0$ zusammen.

5. Ein Interpolant muss ein Polynom sein.

Antwort: Falsch

Definition 13.1.2 verlangt nur die n Gleichungen an den Stützstellen. $\hat{f}_2$ aus Beispiel 13.1.7 ist stückweise linear und an $x = 1$ nicht differenzierbar, $\hat{f}_4$ enthält einen Sinusterm; beide interpolieren. Polynome sind eine besonders bequeme Wahl, keine Vorschrift.

Vertiefung: Heath, Scientific Computing, Kapitel 7 (Interpolation), führt in §7.1 dieselbe Aufgabenstellung ein und diskutiert, nach welchen Kriterien eine Funktionenklasse für die Interpolation ausgewählt wird.

13.2 Interpolation durch Basisdarstellung

Abschnitt 13.1 hat uns mit einem Ärgernis zurückgelassen: Durch dieselben drei Punkte laufen eine Parabel, ein Polygonzug, ein kubisches Polynom und eine wellige Sinuskurve, und alle vier erfüllen die Interpolationsbedingungen exakt. Solange wir nur „irgendeine Funktion durch die Punkte“ verlangen, ist die Aufgabe hoffnungslos unterbestimmt. Dieser Abschnitt räumt das auf. Die Idee ist in einem Satz gesagt: Wir suchen den Interpolanten nicht mehr unter allen Funktionen, sondern nur noch in einem kleinen, endlichdimensionalen Ansatzraum. Aus der Suche nach einer

Funktion wird damit die Suche nach endlich vielen Zahlen, und dafür haben wir seit Kapitel 5 das Werkzeug.

Funktionen als Vektoren

Der erste Schritt ist ein Perspektivwechsel: Funktionen sind Vektoren.

Definition 13.2.1 (Der Vektorraum der Funktionen)

Sei $\mathcal{F} = \{f : [a, b] \rightarrow \mathbb{R}\}$ die Menge aller reellwertigen Funktionen auf einem Intervall $[a, b]$. Mit den punktweise erklärten Verknüpfungen

$$(f + g)(x) = f(x) + g(x), \quad (\alpha \cdot f)(x) = \alpha \cdot f(x), \quad \alpha \in \mathbb{R},$$

ist $\mathcal{F}$ ein Vektorraum über $\mathbb{R}$.

Nachzurechnen ist daran nichts Aufregendes: Assoziativität, Kommutativität und die Distributivgesetze erbt $\mathcal{F}$ punktweise von $\mathbb{R}$, die Nullfunktion $x \mapsto 0$ ist das neutrale Element, und $-f$ ist das Inverse zu f . Wir dürfen Funktionen also addieren und strecken wie Vektoren im $\mathbb{R}^n$. Der Unterschied liegt in der Größe des Raums, und genau die ist unser Problem.

Was heißt der Befund aus Abschnitt 13.1 in dieser Sprache? Dort war jeder Interpolant von der Gestalt $p + g$ mit einer Funktion g , die an allen Stellen x_i verschwindet. Genau so sieht die Lösungsmenge eines linearen Gleichungssystems aus, und das ist kein Zufall.

Bemerkung 13.2.2 (Unendlich viele Freiheitsgrade, endlich viele Bedingungen)

Die Auswertungsabbildung

$$E : \mathcal{F} \rightarrow \mathbb{R}^n, \quad E(f) = (f(x_1), \dots, f(x_n))^T,$$

ist linear, und die Interpolanten der Daten $\mathbf{y}$ sind genau die Urbilder von $\mathbf{y}$ unter E : eine spezielle Lösung plus der Kern (Abschnitt 5.1), also genau die Gestalt aus Abschnitt 13.1. Der Kern ist unendlichdimensional, denn E misst nur n Zahlen aus einem unendlichdimensionalen Raum (Rangsatz).

Die Diagnose lautet damit: unendlich viele Freiheitsgrade gegen n Bedingungen. Die Therapie liegt auf der Hand. Wir suchen von vornherein in einem Raum, der nur endlich viele Freiheitsgrade hat.

Ansatzräume und Basisdarstellung

Wenn die Bedingungen zu wenig festlegen, engen wir den Suchraum ein. Statt in ganz $\mathcal{F}$ suchen wir nur noch in einem endlichdimensionalen Unterraum.

Definition 13.2.3 (Ansatzraum, Basisdarstellung, Basiskoeffizienten)

Seien $\phi_1, \dots, \phi_K : [a, b] \rightarrow \mathbb{R}$ fest gewählte Funktionen, genannt *Basisfunktionen* (basis functions). Der von ihnen aufgespannte Unterraum

$$\mathcal{F}_K = \text{span}\{\phi_1, \dots, \phi_K\} = \left\{ \sum_{k=1}^K a_k \phi_k \mid a_1, \dots, a_K \in \mathbb{R} \right\} \subset \mathcal{F}$$

heißt *Ansatzraum*. Eine Darstellung $f = \sum_{k=1}^K a_k \phi_k$ heißt *Basisdarstellung* von f , und $a_1, \dots, a_K$ heißen *Basiskoeffizienten*.

Ein Element von $\mathcal{F}_K$ ist damit durch seinen Koeffizientenvektor $\mathbf{a} = (a_1, \dots, a_K)^\top \in \mathbb{R}^K$ beschrieben. Ob umgekehrt auch jede Funktion in $\mathcal{F}_K$ nur *einen* solchen Vektor besitzt, ist eine echte Zusatzforderung an die ϕ_k .

Satz 13.2.4 (Wann K Zahlen eine Funktion festlegen)

Die Koordinatenabbildung

$$\Phi: \mathbb{R}^K \rightarrow \mathcal{F}_K, \quad \Phi(\mathbf{a}) = \sum_{k=1}^K a_k \phi_k,$$

ist linear und surjektiv. Sie ist genau dann bijektiv, wenn $\phi_1, \dots, \phi_K$ linear unabhängig sind. In diesem Fall bilden sie eine Basis von $\mathcal{F}_K$, es gilt $\dim \mathcal{F}_K = K$, und jedes $f \in \mathcal{F}_K$ hat genau einen Koeffizientenvektor.

Vertiefung: Beweis: Wann Koeffizienten eine Funktion eindeutig festlegen

Beweis.

- (1) Linearität und Surjektivität stehen schon in Definition 13.2.3.

$\mathcal{F}_K$ ist als Menge aller Linearkombinationen definiert, also ist jedes Element ein Bild unter Φ ; die Linearität folgt aus den punktweisen Rechenregeln

- (2) Φ ist genau dann bijektiv, wenn $\text{Kern}(\Phi) = \{\mathbf{0}\}$ ist, und das ist genau die lineare Unabhängigkeit der ϕ_k .

eine surjektive lineare Abbildung ist genau dann bijektiv, wenn sie injektiv ist, und injektiv ist sie genau bei trivialem Kern; $\Phi(\mathbf{a}) = 0$ heißt ausgeschrieben $\sum_k a_k \phi_k(x) = 0$ für alle x , und dass daraus $\mathbf{a} = \mathbf{0}$ folgt, ist die Definition der linearen Unabhängigkeit

- (3) Ist Φ bijektiv, so ist es ein Isomorphismus zwischen $\mathbb{R}^K$ und $\mathcal{F}_K$; damit ist $\dim \mathcal{F}_K = K$, und die Basisdarstellung jedes $f \in \mathcal{F}_K$ ist eindeutig.

ein Isomorphismus bildet eine Basis auf eine Basis ab; das Bild der Standardbasis von $\mathbb{R}^K$ ist gerade $\phi_1, \dots, \phi_K$

□

Bemerkung 13.2.5 (Das Wort Basisfunktion trägt eine Voraussetzung mit)

Dass jedes $f \in \mathcal{F}_K$ durch K Koeffizienten eindeutig bestimmt sei, klingt harmlos. In der Leserichtung von den Koeffizienten zur Funktion ist das immer richtig, denn Φ ist eine Abbildung. Interessant ist die Gegenrichtung, und die braucht Satz 13.2.4. Wählen wir etwa $\phi_1(x) = 1$, $\phi_2(x) = x$ und $\phi_3(x) = 1 + x$, so ist $K = 3$, aber $\mathcal{F}_3$ besteht nur aus den Polynomen vom Grad höchstens 1 und hat die Dimension 2. Die Funktion $f(x) = 1 + x$ hat dort die Koeffizientenvektoren $(1, 1, 0)^\top$, $(0, 0, 1)^\top$ und unendlich viele weitere. Das Wort *Basisfunktion* setzt die lineare Unabhängigkeit stillschweigend voraus; wir prüfen sie im Zweifelsfall nach.

Beispiel 13.2.6 (Zwei Ansatzräume)

1. *Polynome vom Grad höchstens $n - 1$* : Mit der *Monombasis* $\phi_k(x) = x^{k-1}$ und $K = n$ ist $\mathcal{F}_K$ der Raum aller Polynome vom Grad höchstens $n - 1$. Die Monome sind linear

unabhängig, also ist $\dim \mathcal{F}_K = n$. Diesen Ansatzraum nimmt sich Abschnitt 13.3 vor.

2. *Splines vom Grad q über $m + 1$ Knoten:* Stückweise Polynome vom Grad höchstens q , die an den $m - 1$ inneren Knoten C^{q-1} -glatt aneinanderstoßen, bilden ebenfalls einen Ansatzraum. Bei einfachen inneren Knoten und dieser größtmöglichen Glattheit hat er $K = m + q$ Freiheitsgrade. Woher diese Zahl kommt und welche Basis man dafür wählt, klärt Abschnitt 13.4; wir nehmen sie hier als Vorschau mit.

Der Ansatz und sein Gleichungssystem

Jetzt können wir das Interpolationsproblem umformulieren. Gesucht ist nicht mehr eine Funktion, sondern ein Vektor aus K Zahlen.

Algorithmus 13.2.7 (Interpolation durch Basisdarstellung)

Gegeben seien Daten (x_i, y_i) , $i = 1, \dots, n$, mit paarweise verschiedenen x_i .

1. *Basissystem wählen:* Lege $\phi_1, \dots, \phi_K$ fest. Diese Funktionen sind bekannt.
2. *Ansatz machen:* $\hat{f}(x) = \sum_{k=1}^K a_k \phi_k(x)$ mit den unbekannten Koeffizienten $a_1, \dots, a_K$.
3. *Bedingungen hinschreiben:* $y_i = \hat{f}(x_i)$ für alle i ergibt das lineare Gleichungssystem $\mathbf{B}\mathbf{a} = \mathbf{y}$ aus Satz 13.2.8.
4. *System lösen* (Kapitel 5) und die Koeffizienten in den Ansatz einsetzen.

Satz 13.2.8 (Das Interpolationsproblem ist ein lineares Gleichungssystem)

Sei $\mathbf{B} \in \mathbb{R}^{n \times K}$ die Matrix mit den Einträgen $B_{ik} = \phi_k(x_i)$, also

$$\mathbf{B} = \begin{pmatrix} \phi_1(x_1) & \cdots & \phi_K(x_1) \\ \vdots & \ddots & \vdots \\ \phi_1(x_n) & \cdots & \phi_K(x_n) \end{pmatrix} \in \mathbb{R}^{n \times K}, \quad \mathbf{a} = \begin{pmatrix} a_1 \\ \vdots \\ a_K \end{pmatrix} \in \mathbb{R}^K, \quad \mathbf{y} = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} \in \mathbb{R}^n.$$

Dann interpoliert $\hat{f} = \sum_{k=1}^K a_k \phi_k$ die Daten genau dann, wenn $\mathbf{B}\mathbf{a} = \mathbf{y}$ gilt.

Der Beweis ist eine Zeile: Die i -te Komponente von $\mathbf{B}\mathbf{a}$ ist $\sum_{k=1}^K \phi_k(x_i) a_k = \hat{f}(x_i)$, die i -te Zeile der Gleichung also wörtlich die i -te Interpolationsbedingung.

Bemerkung 13.2.9 (Was in $\mathbf{B}$ steckt und was nicht)

Die Matrix $\mathbf{B}$ liest sich zeilenweise wie eine Datentabelle: *eine Zeile je Datenpunkt, eine Spalte je Basisfunktion*. Drei Beobachtungen dazu.

- $\mathbf{B}$ hängt nur von den Stellen x_i und vom Basissystem ab, nicht von den gemessenen Werten y_i . Kommen neue Messwerte an denselben Stellen herein, bleibt die Matrix stehen, und eine einmal berechnete Zerlegung lässt sich wiederverwenden (Abschnitt 5.3).
- Für $n = K$ ist $\mathbf{B}$ quadratisch. Das System ist genau dann für jede rechte Seite eindeutig lösbar, wenn $\mathbf{B}$ invertierbar ist (Abschnitt 5.1). Das ist der Normalfall der Interpolation, und die folgenden Abschnitte drehen sich um die Frage, wie gut diese Matrix konditioniert ist.
- Für $n > K$ haben wir mehr Bedingungen als Unbekannte. Ein exakter Interpolant

existiert dann in der Regel nicht, und wir minimieren stattdessen die Residuen, also Kleinste Quadrate (Kapitel 7). Das ist der Übergang zur Glättung, den Abschnitt 13.1 angekündigt hat. Für $n < K$ bleibt das Problem unterbestimmt und braucht weitere Forderungen.

Ein Beispiel von Hand

Beispiel 13.2.10 (Basisdarstellung konkret)

Gegeben seien die $n = 3$ Punkte $(0, 1)$, $(1, 2)$, $(2, 5)$ aus Abschnitt 13.1, dazu die Monombasis $\phi_k(x) = x^{k-1}$ mit $K = 3$. Der Ansatzraum ist der Raum der Polynome vom Grad höchstens 2, der Ansatz lautet $\hat{f}(x) = a_1 + a_2x + a_3x^2$.

Die Matrix entsteht durch Auswerten der drei Basisfunktionen an den drei Stellen:

$$B = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad y = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix}.$$

Wir lösen $Ba = y$ durch Einsetzen. Die erste Zeile liefert a_1 sofort, danach eliminieren wir:

$$\text{Zeile 1: } a_1 = 1,$$

$$\text{Zeile 2: } a_1 + a_2 + a_3 = 2 \implies a_2 + a_3 = 1 \implies a_2 = 1 - a_3,$$

$$\text{Zeile 3: } a_1 + 2a_2 + 4a_3 = 5 \implies 2a_2 + 4a_3 = 4 \implies 2 + 2a_3 = 4.$$

In Zeile 3 steckt bereits $a_2 = 1 - a_3$ aus Zeile 2. Aus der letzten Gleichung folgt $a_3 = 1$ und damit $a_2 = 0$. Der Koeffizientenvektor ist $a = (1, 0, 1)^\top$, der Interpolant also

$$\hat{f}(x) = 1 + x^2.$$

Die Probe an allen drei Stellen geht auf: $\hat{f}(0) = 1$, $\hat{f}(1) = 2$, $\hat{f}(2) = 5$.

Bemerkung 13.2.11 (Einsetzen ist hier nicht Vorwärtssubstitution)

Der Rechenweg sieht aus wie das Lösen eines Dreieckssystems, ist aber keines: Vorwärtssubstitution setzt eine untere Dreiecksmatrix voraus, und in Zeile 2 von B steht rechts der Diagonalen $\phi_3(x_2) = 1 \neq 0$. Dass die erste Zeile so billig ist, liegt allein an $x_1 = 0$; ab Zeile 2 rechnen wir echte Gauß-Elimination (Abschnitt 5.2).

Dass es auch anders geht, zeigt ein zweiter Anlauf mit demselben Ansatzraum.

Beispiel 13.2.12 (Dieselbe Funktion, andere Koeffizienten)

Wir behalten die Daten und wechseln nur das Basissystem, und zwar zur *Newton-Basis* zu den Knoten 0 und 1:

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \phi_3(x) = x(x - 1).$$

Auch diese drei Funktionen spannen die Polynome vom Grad höchstens 2 auf, der Ansatzraum ist also unverändert. Die Matrix dagegen ändert ihre Gestalt:

$$\mathbf{B} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 2 & 2 \end{pmatrix}.$$

Sie ist eine untere Dreiecksmatrix, denn ϕ_2 verschwindet in 0 und ϕ_3 verschwindet in 0 und in 1. Jetzt greift die Vorwärtssubstitution wirklich, Zeile für Zeile von oben nach unten:

$$\begin{aligned} a_1 = 1 &\implies a_1 = 1, \\ a_1 + a_2 = 2 &\implies a_2 = 1, \\ a_1 + 2a_2 + 2a_3 = 5 &\implies 2a_3 = 2 \implies a_3 = 1. \end{aligned}$$

Der Koeffizientenvektor ist diesmal $\mathbf{a} = (1, 1, 1)^\top$, und ausmultipliziert steht dort

$$\hat{f}(x) = 1 + x + x(x - 1) = 1 + x^2,$$

dieselbe Funktion wie in Beispiel 13.2.10. Anders sein kann sie auch nicht: Die Differenz zweier Polynome vom Grad höchstens 2, die beide durch alle drei Punkte laufen, hat mindestens drei Nullstellen. Ein Polynom vom Grad höchstens 2 mit drei Nullstellen ist aber das Nullpolynom, wie Abschnitt 13.3 allgemein festhält.

Der Ansatzraum bestimmt also den Interpolanten, das Basissystem nur seine Koeffizienten und den Rechenweg dorthin. Für die Numerik ist der Rechenweg allerdings alles andere als gleichgültig: Ein Dreieckssystem kostet $O(K^2)$ Operationen, die volle Elimination $O(K^3)$ (Abschnitt 5.3).

Interaktiv: Zwei Basen, ein Interpolant

Der Rechner führt beide Beispiele vor. Der Schalter wechselt das Basissystem, die Regler verschieben die drei Messwerte. Beobachten lässt sich dabei alles, worauf es in diesem Abschnitt ankommt: Die Matrix bleibt beim Schieben stehen, weil sie nur von den Stellen und der Basis abhängt; die Koeffizienten springen beim Basiswechsel, die grüne Kurve nicht; und die orangen Bausteine $a_k \phi_k$ summieren sich punktweise zu genau dieser Kurve.

Interaktives Element — nur in der Web-Fassung

Woran sich Basissysteme unterscheiden

Interpolationsverfahren unterscheiden sich im Wesentlichen durch die Wahl des Basissystems, und manche Systeme sind erheblich brauchbarer als andere. Vier Kriterien geben den Fahrplan für den Rest des Kapitels vor.

- *Kondition.* Wie empfindlich reagieren die berechneten Koeffizienten auf Störungen der Daten? Für eine quadratische invertierbare Matrix kontrolliert $\|\mathbf{B}^{-1}\|$ die absolute Verstärkung einer Störung der rechten Seite; die Konditionszahl $\kappa(\mathbf{B})$ liefert die übliche relative Störungsabschätzung (Abschnitt 4.2). Bei der Monombasis wächst sie mit K dramatisch, und das ist ihr Hauptproblem (Abschnitt 13.3).
- *Stabilität.* Ändert sich ein einziger Messwert ein wenig, sollte sich der Interpolant auch nur ein wenig ändern, und zwar am besten nur in der Nähe dieses Punktes. Ein globales Polynom schlägt stattdessen im ganzen Intervall aus, oft am heftigsten weit weg von der Störung (Abschnitt 13.3).

- *Lokalität.* Ist jede Basisfunktion nur auf einem kleinen Teilintervall von null verschieden, so hat $\mathbf{B}$ viele Nullen und wird zur Bandmatrix. Genau das leisten die B-Splines aus Abschnitt 13.4.
- *Effiziente Lösbarkeit.* Aus der Struktur von $\mathbf{B}$ wird Rechenzeit: Dreiecks- und Bandmatrizen lassen sich mit einem Aufwand lösen, der linear in der Zahl der Unbekannten wächst, statt kubisch (Abschnitt 5.3, Abschnitt 2.4).

Selbsttest

1. Im System $\mathbf{B}\mathbf{a} = \mathbf{y}$ hat $\mathbf{B}$ genau n Zeilen und K Spalten, und in Zeile i , Spalte k steht $\phi_k(x_i)$.

Antwort: Wahr

So steht es in Satz 13.2.8: Jede Zeile gehört zu einem Datenpunkt, jede Spalte zu einer Basisfunktion. Die Unbekannte ist der Koeffizientenvektor $\mathbf{a} \in \mathbb{R}^K$, die rechte Seite der Datenvektor $\mathbf{y} \in \mathbb{R}^n$.

2. Die Matrix $\mathbf{B}$ hängt von den gemessenen Werten y_i ab.

Antwort: Falsch

In $\mathbf{B}$ stehen ausschließlich Werte der Basisfunktionen an den Stellen x_i . Die Messwerte stehen komplett auf der rechten Seite. Deshalb dürfen wir für neue Daten an denselben Stellen dieselbe Matrix und dieselbe Zerlegung weiterverwenden (Bemerkung 13.2.9).

3. Sobald wir uns auf einen endlichdimensionalen Ansatzraum $\mathcal{F}_K$ beschränken, ist der Interpolant eindeutig.

Antwort: Falsch

Die Einschränkung allein genügt nicht. Suchen wir zu unseren drei Punkten in den Polynomen vom Grad höchstens 3, also mit $K = 4 > 3 = n$, so hat das System mehr Unbekannte als Gleichungen und weiterhin unendlich viele Lösungen; der kubische Interpolant aus Abschnitt 13.1 ist eine davon. Genau eine Lösung für jede rechte Seite gibt es erst, wenn $\mathbf{B}$ quadratisch und invertierbar ist.

4. Wechseln wir von der Monombasis zur Newton-Basis, so ändert sich der Interpolant.

Antwort: Falsch

Es ändern sich nur die Koeffizienten, von $(1, 0, 1)^T$ auf $(1, 1, 1)^T$. Beide Basen spannen denselben Ansatzraum auf, und in ihm gibt es zu drei verschiedenen Stellen nur einen Interpolanten, hier $1 + x^2$ (Beispiel 13.2.12). Am Widget lässt sich das nachfahren: Die grüne Kurve springt beim Umschalten nicht.

5. Weil sich das System aus Beispiel 13.2.10 durch Einsetzen lösen lässt, ist $\mathbf{B}$ dort eine untere Dreiecksmatrix.

Antwort: Falsch

Über der Diagonalen steht in Zeile 2 der Eintrag $\phi_3(x_2) = 1$, die Matrix ist also keine Dreiecksmatrix (Bemerkung 13.2.11). Dass die erste Zeile sofort a_1 liefert, liegt allein an $x_1 = 0$. Erst mit der Newton-Basis entsteht wirklich Dreiecksgestalt.

6. Wählen wir K Basisfunktionen, so hat der Ansatzraum $\mathcal{F}_K$ die Dimension K .

Antwort: Falsch

Nur wenn die ϕ_k linear unabhängig sind (Satz 13.2.4). Mit $\phi_1(x) = 1$, $\phi_2(x) = x$ und $\phi_3(x) = 1 + x$ ist $K = 3$, aber $\mathcal{F}_3$ enthält nur die Polynome vom Grad höchstens 1 und hat die Dimension 2; jede Funktion darin hat dann unendlich viele Koeffizientenvektoren (Bemerkung 13.2.5).

Vertiefung: Heath §7.1 (Interpolation, Wahl des Basissystems) und §7.2 (Existenz, Eindeutigkeit, Kondition).

13.3 Polynominterpolation

Abschnitt 13.2 hat das Interpolationsproblem auf das lineare Gleichungssystem $\mathbf{B}\mathbf{a} = \mathbf{y}$ zurückgeführt. Damit ist die Wahl des Ansatzraums die eigentliche Entscheidung, und die naheliegendste ist zugleich die älteste: Polynome, aufgeschrieben in der Monombasis. Dieser Abschnitt zeigt beide Seiten dieser Wahl. Theoretisch ist der Fall erledigt, und zwar so schön, wie man es sich nur wünschen kann: Durch n Punkte läuft genau ein Polynom vom Grad höchstens $n - 1$. Numerisch dagegen sammelt derselbe Ansatz drei Probleme ein, eine katastrophal konditionierte Matrix, eine global instabile Lösung und ein Konvergenzverhalten, das an einem harmlos aussehenden Beispiel spektakulär versagt.

Wie viele Nullstellen ein Polynom haben kann

Die Eindeutigkeit hängt an einer einzigen Aussage über Nullstellen, und die kommt aus der Algebra.

Satz 13.3.1 (Fundamentalsatz der Algebra)

Jedes Polynom $p(x) = a_n x^n + \dots + a_1 x + a_0$ mit $a_n \neq 0$ und $n \geq 1$ hat mindestens eine Nullstelle in $\mathbb{C}$.

Gleichwertig dazu: Jedes Polynom vom Grad n hat in $\mathbb{C}$ genau n Nullstellen, wenn wir sie mit Vielfachheit zählen.

Beide Fassungen gehen durch Polynomdivision auseinander hervor. Ist x_0 eine Nullstelle von p , so lässt sich der Linearfaktor abspalten, $p(x) = (x - x_0)q(x)$ mit $\deg q = n - 1$, und auf q wenden wir dieselbe Überlegung an. Nach n Schritten ist p vollständig in Linearfaktoren zerlegt. Wir brauchen den Satz allerdings gar nicht in dieser Stärke, sondern nur seine Kehrseite.

Korollar 13.3.2 (Zu viele Nullstellen erzwingen das Nullpolynom)

Sei p ein Polynom vom Grad höchstens $n - 1$ mit n paarweise verschiedenen Nullstellen. Dann ist p das Nullpolynom, es gilt also $p(x) = 0$ für alle x .

Beweis.

- (1) Nehmen wir an, p sei nicht das Nullpolynom. Dann besitzt p einen Grad d mit $0 \leq d \leq n-1$ und einen führenden Koeffizienten $a_d \neq 0$.

der Grad eines Polynoms ist der größte Index mit $a_d \neq 0$; nur das Nullpolynom hat gar keinen solchen Index

- (2) Im Fall $d = 0$ ist p die konstante Funktion $a_0 \neq 0$ und hat überhaupt keine Nullstelle. Das widerspricht der Voraussetzung, denn $n \geq 1$.
- (3) Im Fall $d \geq 1$ hat p nach Satz 13.3.1 mit Vielfachheit gezählt genau d Nullstellen in $\mathbb{C}$, also höchstens $d \leq n-1$ verschiedene. Auch das widerspricht der Voraussetzung von n verschiedenen Nullstellen.

verschiedene Nullstellen zählen mindestens einfach, ihre Anzahl ist also höchstens die Zahl der mit Vielfachheit gezählten

- (4) Beide Fälle sind unmöglich, die Annahme fällt. Also ist p das Nullpolynom.

□

Beispiel 13.3.3 (Drei Nullstellen bei Grad höchstens zwei)

Ein Polynom vom Grad höchstens 2 hat höchstens zwei verschiedene Nullstellen; das kennen wir von der Parabel. Finden wir also ein solches Polynom mit drei Nullstellen, etwa $p(-1) = p(0) = p(1) = 0$, so bleibt nur ein Schluss: p ist das Nullpolynom, und dann ist auch $p(7) = 0$ und $p(10^6) = 0$. Aus drei Nullstellen wird eine Aussage über die gesamte reelle Achse, und genau diese Hebelwirkung brauchen wir gleich.

Genau ein Polynom durch n Punkte

Wir halten zuerst den Ansatzraum und seine Basismatrix fest. Beides kennen wir schon aus Beispiel 13.2.6, hier bekommen sie ihre Namen.

Definition 13.3.4 (Monombasis und Vandermonde-Matrix)

Die n Funktionen

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \dots, \quad \phi_n(x) = x^{n-1}$$

heißen *Monombasis* (monomial basis). Sie sind linear unabhängig und spannen den Ansatzraum $\mathcal{F}_n$ aller Polynome vom Grad höchstens $n-1$ auf, der damit die Dimension n hat.

Die zugehörige Basismatrix aus Satz 13.2.8 zu Stellen $x_1, \dots, x_n$ heißt *Vandermonde-Matrix*:

$$\mathbf{B} = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \dots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^{n-1} \end{pmatrix} \in \mathbb{R}^{n \times n}.$$

Dass die Monome linear unabhängig sind, ist Korollar 13.3.2 in anderer Verpackung: Verschwindet $\sum_{k=1}^n a_k x^{k-1}$ für alle x , so hat dieses Polynom vom Grad höchstens $n-1$ unendlich viele Nullstellen, ist also das Nullpolynom, und damit sind alle a_k null.

Satz 13.3.5 (Existenz und Eindeutigkeit der Polynominterpolation)

Zu n Paaren (x_i, y_i) , $i = 1, \dots, n$, mit paarweise verschiedenen x_i gibt es *genau ein* Polynom p vom Grad *höchstens* $n - 1$ mit

$$p(x_i) = y_i \quad \text{für } i = 1, \dots, n.$$

Gleichbedeutend damit ist: Die Vandermonde-Matrix zu paarweise verschiedenen Stellen ist invertierbar.

Beweis.

- (1) *Eindeutigkeit.* Seien p und q zwei Polynome vom Grad höchstens $n - 1$, die beide interpolieren. Für ihre Differenz gilt an jeder Stelle

$$(p - q)(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0.$$

beide erfüllen dieselbe Interpolationsbedingung an derselben Stelle

- (2) Damit hat $p - q$ mindestens n verschiedene Nullstellen, obwohl der Grad höchstens $n - 1$ ist. Nach Korollar 13.3.2 ist die Differenz das Nullpolynom, also $p = q$.

die Differenz zweier Polynome vom Grad höchstens $n - 1$ hat wieder Grad höchstens $n - 1$

- (3) *Invertierbarkeit von B.* Sei $\mathbf{a} \in \mathbb{R}^n$ mit $\mathbf{B}\mathbf{a} = \mathbf{0}$. Nach Satz 13.2.8 heißt das: Das Polynom $\sum_{k=1}^n a_k x^{k-1}$ verschwindet an allen n Stellen. Korollar 13.3.2 macht daraus das Nullpolynom, und weil die Monome linear unabhängig sind, folgt $\mathbf{a} = \mathbf{0}$.

$\mathbf{B}\mathbf{a}$ ist der Vektor der Werte dieses Polynoms an den Stellen $x_1, \dots, x_n$

- (4) $\mathbf{B}$ ist quadratisch und hat nur den trivialen Kern, ist also invertierbar (Abschnitt 5.1). Damit besitzt $\mathbf{B}\mathbf{a} = \mathbf{y}$ für jede rechte Seite genau eine Lösung, und das ist die *Existenz*: Zu jedem Datenvektor gehört ein interpolierendes Polynom, nämlich $p = \sum_{k=1}^n a_k \phi_k$.

□

Höchstens, nicht genau**Bemerkung 13.3.6 (Ein häufiges Missverständnis)**

Satz 13.3.5 verspricht genau ein Polynom vom Grad *höchstens* $n - 1$. Er verspricht nicht den Grad *genau* $n - 1$. Der führende Koeffizient darf null sein, und wenn die Daten schon auf einem Polynom kleineren Grades liegen, ist er es auch. Der Ansatzraum $\mathcal{F}_n$ enthält diese Polynome ja ebenfalls.

Beispiel 13.3.7 (Drei Punkte auf einer Geraden)

Für die $n = 3$ Punkte $(0, 0)$, $(1, 1)$, $(2, 2)$ lautet das System aus Satz 13.2.8

$$\begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} a_1 \\ a_2 \\ a_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 2 \end{pmatrix}, \quad \text{Lösung } \mathbf{a} = (0, 1, 0)^\top.$$

Das eindeutige Polynom vom Grad höchstens 2 ist also

$$p(x) = 0 \cdot x^2 + 1 \cdot x + 0 = x,$$

eine Gerade und damit vom Grad 1. Die Eindeutigkeit gilt trotzdem: Ein zweites Polynom vom Grad höchstens 2 durch dieselben drei Punkte gibt es nicht.

Der Rechenweg und sein Preis

Algorithmus 13.3.8 (Polynominterpolation in der Monombasis)

Gegeben seien Daten (x_i, y_i) , $i = 1, \dots, n$, mit paarweise verschiedenen x_i .

1. *Vandermonde-Matrix aufstellen:* $B_{ik} = x_i^{k-1}$.
2. *System lösen:* $\mathbf{B}\mathbf{a} = \mathbf{y}$ mit Gauß-Elimination (Abschnitt 5.2), Aufwand $O(n^3)$.
3. *Auswerten:* $p(x) = \sum_{k=1}^n a_k x^{k-1}$ an den gewünschten Stellen.

In R sind das wenige Zeilen, hier für $n = 10$ Stellen im Intervall $[0, 1]$; die Funktion f liefert dabei die Werte, die interpoliert werden sollen.

```
x <- seq(0, 1, length = 10)
y <- f(x)

B <- outer(x, 0:9, "^") # Vandermonde-Matrix: cbind(1, x, x^2, ...)
a <- solve(B, y)

xnew <- seq(0, 1, length = 100)
fhat <- outer(xnew, 0:9, "^") %*% a

ggplot() + geom_point(aes(x, y)) + geom_line(aes(xnew, fhat))
```

`outer(x, 0:9, "^")` baut genau die Matrix aus Definition 13.3.4, `solve` ruft die Elimination auf, und die letzte Auswertung ist wieder ein Matrix-Vektor-Produkt, diesmal mit einem feinen Auswertungsgitter. So weit, so unauffällig. Nur ist dieses 10×10 -System bereits so schlecht konditioniert, dass von den rund 16 verlässlichen Dezimalstellen der doppelten Genauigkeit etwa sieben verloren gehen. Warum, klärt der nächste Abschnitt.

Problem 1: die Kondition der Monombasis

Bemerkung 13.3.9 (Warum die Spalten fast parallel werden)

Die k -te Spalte der Vandermonde-Matrix ist der Vektor

$$\mathbf{b}_k = (x_1^{k-1}, x_2^{k-1}, \dots, x_n^{k-1})^\top.$$

Für wachsendes k unterscheiden sich diese Vektoren immer weniger in ihrer *Richtung*. Auf $[0, 1]$ drückt jedes weitere Potenzieren die Einträge gegen null, den größten aber gar nicht, denn er gehört zu $x_n = 1$. Damit konzentriert sich jeder dieser Vektoren mehr und mehr auf seine letzten Komponenten, und alle zeigen am Ende in dieselbe Richtung. Bei $n = 10$ gleichmäßig verteilten Stellen in $[0, 1]$ schließen $\mathbf{b}_1$ und $\mathbf{b}_2$ noch einen Winkel von $32,6^\circ$ ein, $\mathbf{b}_5$ und $\mathbf{b}_6$ nur noch $5,5^\circ$, und $\mathbf{b}_9$ und $\mathbf{b}_{10}$ gerade einmal $2,7^\circ$.

Fast parallele Spalten heißen: fast linear abhängig, also eine Matrix, die einer singulären sehr nahe kommt. Genau das misst die Konditionszahl $\kappa(\mathbf{B})$ (Abschnitt 4.2), und sie wächst hier dramatisch.

Beispiel 13.3.10 (Konditionszahlen, der Größenordnung nach)

Wir interpolieren an n gleichmäßig verteilten Stellen in $[0, 1]$ durch ein Polynom vom Grad $n - 1$ und sehen uns die Konditionszahl der Vandermonde-Matrix an.

n	Polynomgrad	$\kappa_2(\mathbf{B})$	$\kappa_\infty(\mathbf{B})$
5	4	$6,9 \cdot 10^2$	$1,7 \cdot 10^3$
10	9	$1,5 \cdot 10^7$	$4,8 \cdot 10^7$
15	14	$4,0 \cdot 10^{11}$	$1,6 \cdot 10^{12}$
20	19	$1,1 \cdot 10^{16}$	$4,9 \cdot 10^{16}$

Die Werte sind *illustrativ* und als Größenordnungen zu lesen. Sie hängen von der verwendeten Norm ab, wie der Vergleich der beiden Spalten zeigt, und ebenso von der Lage der Stellen: Dieselbe Rechnung auf $[-1, 1]$ statt auf $[0, 1]$ liefert bei $n = 20$ nur noch $\kappa_2 \approx 2,7 \cdot 10^8$.

Entscheidend ist der Trend, und der ist in jeder Norm derselbe: exponentielles Wachstum. Die Faustregel aus Abschnitt 4.2 beschränkt den relativen Fehler der berechneten Koeffizienten der Größenordnung nach durch $\kappa(\mathbf{B}) \cdot \varepsilon$ mit $\varepsilon \approx 2,2 \cdot 10^{-16}$ (Maschinengenauigkeit, Abschnitt 4.1). Bei $n = 10$ steht dort $3 \cdot 10^{-9}$, bei $n = 15$ schon 10^{-4} , und bei $n = 20$ ist die Schranke mit 2 größer als die gesuchten Zahlen selbst: Von der Lösung bleibt rechnerisch nichts Verlässliches übrig.

Bemerkung 13.3.11 (Der Ausweg: orthogonalisierte Basen)

Die schlechte Kondition ist eine Eigenschaft der *Basis*, nicht des Ansatzraums. Nach Abschnitt 13.2 ändert ein Basiswechsel den Interpolanten nicht, nur die Koeffizienten und den Rechenweg. Wählen wir Basisfunktionen, die sich weniger ähneln, wird die Basismatrix besser konditioniert. Klassisch sind dafür die *orthogonalen Polynome* (Legendre, Chebyshev, Hermite), die bezüglich eines Skalarprodukts von Funktionen orthogonal sind. An denselben gleichmäßig verteilten Stellen wie oben liefern die Chebyshev-Polynome $\kappa_2 \approx 2,2$ statt $6,9 \cdot 10^2$ bei $n = 5$ und $\kappa_2 \approx 4,8 \cdot 10^3$ statt $1,1 \cdot 10^{16}$ bei $n = 20$, ein Gewinn von zwölf Zehnerpotenzen.

Noch weiter geht R mit `poly(x, 3)` anstelle der drei nichtkonstanten Spalten `cbind(x, x^2, x^3)`. Der Aufruf erzeugt bezüglich der *gegebenen* Stellen orthonormale Spalten, im Kern über eine stabile Orthogonalisierung der Vandermonde-Spalten. `poly(x, 3)` allein hat deshalb in der 2-Norm Konditionszahl 1. Ergänzen wir für ein Regressionsmodell die unnormierte konstante Spalte, also `cbind(1, poly(x, 3))`, stehen weiterhin alle Spalten senkrecht aufeinander, aber ihre Längen sind $\sqrt{n}, 1, 1, 1$; die Konditionszahl ist dann $\sqrt{n}$. Normieren wir auch die erste Spalte, fällt sie wieder auf 1.

Interaktiv: Die Monombasis und ihre Kondition

Warum sind die Spalten einer Vandermonde-Matrix auf $[0, 1]$ fast parallel? Die erste Tafel macht die Ursache sichtbar.

Interaktives Element — nur in der Web-Fassung

Diese Ähnlichkeit ist es, die das Gleichungssystem verdirbt. Der Regler zieht die Zahl der Stellen hoch, die Schalter wechseln zwischen drei Basissystemen desselben Ansatzraums.

Interaktives Element — nur in der Web-Fassung

Problem 2: Stabilität und Lokalität

Die zweite Schwäche zeigt sich, wenn wir an den Daten wackeln. Ändern wir einen einzigen Messwert, so ändert sich der Interpolant im gesamten Intervall, und keineswegs am stärksten dort, wo wir gewackelt haben.

Satz 13.3.12 (Wie sich eine Datenänderung fortpflanzt)

Seien p das interpolierende Polynom vom Grad höchstens $n - 1$ zu den Daten (x_i, y_i) und $\tilde{p}$ das interpolierende Polynom zu denselben Daten, bei denen nur y_j um δ verändert wurde. Dann gilt

$$\tilde{p} - p = \delta \cdot \ell_j,$$

wobei ℓ_j das eindeutige Polynom vom Grad höchstens $n - 1$ mit $\ell_j(x_j) = 1$ und $\ell_j(x_i) = 0$ für $i \neq j$ ist.

Beweis.

- (1) Die Differenz $\tilde{p} - p$ ist ein Polynom vom Grad höchstens $n - 1$, und an den Stellen nimmt sie die Werte $0, \dots, 0, \delta, 0, \dots, 0$ an, mit δ an der j -ten Position.

beide Polynome interpolieren dieselben Daten bis auf den j -ten Wert, ihre Differenz interpoliert also die Differenz der Daten

- (2) Dieselben Werte hat $\delta \cdot \ell_j$, und auch dieses Polynom hat Grad höchstens $n - 1$. Nach der Eindeutigkeit in Satz 13.3.5 sind beide Polynome gleich.

□

Bemerkung 13.3.13 (Was das in Zahlen heißt)

Die Polynome ℓ_j heißen *Lagrange-Grundpolynome*, und Satz 13.3.12 sagt: Der Ausschlag einer Datenänderung ist überall dort groß, wo $|\ell_j|$ groß ist. Rechnen wir das für $n = 10$ gleichmäßig verteilte Stellen in $[0, 1]$ aus und heben den fünften Wert bei $x_5 = 4/9 \approx 0,444$ um $\delta = 0,1$ an, so verschiebt sich der Interpolant um bis zu **0,40**, das Vierfache der Störung. Erreicht wird dieses Maximum bei $x \approx 0,035$, also am *anderen Ende* des Intervalls. Der größte solche Verstärkungsfaktor liegt bei diesen zehn Stellen bei 4,03 und gehört zu den beiden mittleren, x_5 und x_6 .

Damit sind beide Schwächen beziffert. Die Methode ist *instabil*, weil kleine Datenänderungen groß herauskommen, und sie ist *nicht lokal*, weil jedes ℓ_j auf dem ganzen Intervall von null verschieden ist. Beides ist kein Malheur der Monombasis, sondern eine Eigenschaft globaler Polynome: ℓ_j hängt allein vom Ansatzraum ab. Abhilfe schaffen erst Basisfunktionen, die außerhalb eines kleinen Bereichs verschwinden (Abschnitt 13.4).

Vertiefung: Wenn alle Werte gleichzeitig gestört werden

Wie schlimm es werden kann, hängt an der Zahl der Stellen. Stören wir alle Werte gleichzeitig um höchstens δ , so ändert sich der Interpolant um höchstens $\Lambda_n \cdot \delta$ mit $\Lambda_n = \max_x \sum_{j=1}^n |\ell_j(x)|$, und diese Schranke wird bei passend gewählten Vorzeichen auch angenommen. Für gleichmäßig verteilte Stellen wächst diese Zahl rasant, nämlich auf 2,2 bei $n = 5$, 17,9 bei $n = 10$, 283 bei $n = 15$ und 5890 bei $n = 20$. Für die Chebyshev-Knoten, die uns gleich wieder begegnen, bleibt sie im selben Bereich zwischen 2,0 und 2,9.

Das Runge-Phänomen

Bleibt die Frage, ob wenigstens der Interpolant selbst besser wird, wenn wir mehr Stützstellen nehmen. Bei einer beliebig oft differenzierbaren Funktion sollte man das erwarten. Carl Runge hat 1901 gezeigt, dass die Erwartung trügt.

Beispiel 13.3.14 (Runge 1901)

Wir interpolieren

$$f(x) = \frac{1}{1 + 25x^2} \quad \text{auf } [-1, 1]$$

an n gleichmäßig verteilten Knoten durch das Polynom p_{n-1} vom Grad $n - 1$. Die Funktion f ist so gutartig, wie man es sich wünschen kann, sie ist beliebig oft differenzierbar, also $f \in \mathcal{C}^\infty$, und beschränkt durch 1.

Trotzdem konvergieren die Interpolanten nicht gleichmäßig gegen f . Der Fehler $\|f - p_{n-1}\|_\infty = \max_{x \in [-1, 1]} |f(x) - p_{n-1}(x)|$ wächst über alle Grenzen, und zwar durch immer heftigere Oszillationen nahe den Intervallenden. In der Mitte passt sich das Polynom dabei immer besser an.

Bemerkung 13.3.15 (Divergenz schon, aber nicht monoton)

Die knappe Schreibweise $\|f - p_{n-1}\|_\infty \rightarrow \infty$ für $n \rightarrow \infty$ legt nahe, dass jeder zusätzliche Knoten die Lage verschlimmert. Das ist nicht so. Nachgerechnet ergeben sich für gleichmäßig verteilte Knoten diese größten Fehler:

n Knoten	Polynomgrad	größter Fehler	Stelle des Maximums
5	4	0,44	$x \approx \pm 0,80$
10	9	0,30	$x \approx \pm 0,93$
15	14	7,2	$x \approx \pm 0,96$
20	19	8,6	$x \approx \pm 0,97$

Von $n = 5$ auf $n = 10$ fällt der Fehler. Erst danach dreht die Folge nach oben, und dann sehr entschlossen: bei $n = 25$ steht dort bereits 257. Die Aussage lautet also präzise, dass der Fehler asymptotisch unbeschränkt wächst, nicht dass er in jedem Schritt zunimmt. Zugleich zeigt die letzte Spalte, wo das Unheil sitzt: Das Fehlermaximum wandert mit wachsendem n immer näher an die Ränder ± 1 .

Vertiefung: Chebyshev-Knoten als Gegenmittel

Bemerkung 13.3.16 (Ein Ausweg: die Knoten anders legen)

Am Ansatzraum liegt es nicht, und an der Basis auch nicht: Satz 13.3.5 gibt zu jeder Knotenwahl dasselbe eindeutige Polynom her. Es liegt an den *Knoten*. Legen wir sie an den Rändern dichter, statt sie gleichmäßig zu verteilen, verschwindet das Problem. Die übliche Wahl sind die *Chebyshev-Knoten*

$$x_i = \cos\left(\frac{(2i-1)\pi}{2n}\right), \quad i = 1, \dots, n,$$

die Nullstellen des n -ten Chebyshev-Polynoms. Für Runge's Funktion sinkt der größte Fehler damit auf 0,047 bei $n = 15$ und 0,038 bei $n = 20$, gegen 7,2 und 8,6 oben, und er fällt weiter: $1,3 \cdot 10^{-5}$ bei $n = 60$. Ehrlicherweise gehört dazu, dass dieser Gewinn asymptotisch ist. Bei $n = 4$, 6 und 8 Knoten schneiden die gleichmäßig verteilten Knoten sogar etwas besser ab.

Chebyshev-Knoten zähmen das Runge-Phänomen, setzen aber voraus, dass wir die Stellen frei wählen dürfen (in der Vertiefung Bemerkung 13.3.16 erklärt). Bei gemessenen Daten liegen sie fest, und dann hilft nur ein anderer Ansatzraum.

Interaktiv: Runge's Funktion, zwei Knotenfamilien

Fällt der Fehler bei äquidistanten Knoten von $n = 5$ auf $n = 10$, oder steigt er schon dort?

Schätzfrage. Welche Richtung nimmt der Fehler von $n = 5$ auf $n = 10$ bei äquidistanten Knoten?

Lösung: *faellt*

Der Vergleich zeigt den nichtmonotonen Start aus Bemerkung 13.3.15: Von $n = 5$ auf $n = 10$ fällt der Fehler zunächst, bevor die Randszillationen dominieren.

Damit ist die Bilanz der Polynominterpolation gemischt. Der Satz von der Existenz und Eindeutigkeit ist so sauber, wie ein Satz nur sein kann. Der praktische Nutzen des globalen Polynoms hoher Ordnung ist trotzdem gering: Es rechnet sich schlecht, es reagiert auf jeden Messwert im ganzen Intervall, und es konvergiert nicht einmal für harmlose Funktionen. Der Schluss daraus trägt den Rest des Kapitels. Statt eines globalen Polynoms hohen Grades nehmen wir viele Polynome kleinen Grades, jedes nur auf einem kurzen Stück gültig, und kleben sie glatt aneinander. Das sind die *Splines* aus Abschnitt 13.4.

Selbsttest

1. Durch n Punkte mit paarweise verschiedenen x_i läuft genau ein Polynom vom Grad höchstens $n - 1$.

Antwort: Wahr

Das ist Satz 13.3.5. Die Eindeutigkeit folgt aus Korollar 13.3.2 über die Differenz zweier Interpolanten, die Existenz aus der Invertierbarkeit der Vandermonde-Matrix.

2. Das interpolierende Polynom durch n Punkte hat den Grad genau $n - 1$.

Antwort: Falsch

Es hat Grad *höchstens* $n - 1$. Für die drei Punkte $(0, 0)$, $(1, 1)$, $(2, 2)$ liefert das System die Koeffizienten $(0, 1, 0)^T$, das Polynom ist also $p(x) = x$ und damit vom Grad 1 (Beispiel 13.3.7).

3. Ein Polynom vom Grad höchstens 5 mit sechs verschiedenen Nullstellen kann noch von null verschiedene Werte annehmen.

Antwort: Falsch

Nach Korollar 13.3.2 ist es das Nullpolynom, nimmt also überall den Wert null an. Ein Polynom vom Grad $d \geq 1$ hat mit Vielfachheit gezählt genau d Nullstellen, ein konstantes Polynom ungleich null gar keine; sechs verschiedene Nullstellen bei Grad höchstens 5 sind in beiden Fällen zu viele.

4. Die Vandermonde-Matrix ist zu paarweise verschiedenen Stellen stets invertierbar und kann trotzdem numerisch unbrauchbar sein.

Antwort: Wahr

Invertierbarkeit ist eine Ja-Nein-Frage, Kondition eine Frage des Ausmaßes. Bei $n = 20$ gleichmäßig verteilten Stellen in $[0, 1]$ ist $\det \mathbf{B} \neq 0$, aber $\kappa_2(\mathbf{B}) \approx 10^{16}$, und damit ist die Schranke $\kappa \cdot \varepsilon$ für den relativen Fehler von der Größe 1 (Beispiel 13.3.10).

5. Bei äquidistanten Knoten wird der Fehler $\max |f - p_{n-1}|$ für Runges Funktion mit jedem zusätzlichen Knoten größer.

Antwort: Falsch

Er wächst asymptotisch über alle Grenzen, aber nicht monoton: von $n = 5$ auf $n = 10$ fällt er von 0,44 auf 0,30 (Bemerkung 13.3.15). Erst danach setzt das Wachstum durch, mit 7,2 bei $n = 15$ und 8,6 bei $n = 20$.

6. Wechseln wir von der Monombasis zu den Chebyshev-Polynomen, so verschwindet bei gleichmäßig verteilten Knoten auch das Runge-Phänomen.

Antwort: Falsch

Der Basiswechsel bessert die Kondition des Gleichungssystems, mehr nicht. Nach Satz 13.3.5 gehört zu den Daten dasselbe eindeutige Polynom, gleich in welcher Basis wir es aufschreiben, also auch derselbe Fehler. Gegen die Oszillationen hilft nur, die Knoten anders zu legen oder den Ansatzraum zu wechseln (Bemerkung 13.3.16).

7. Heben wir bei fester Knotenmenge einen einzigen Wert y_j um δ an, so ändert sich der Interpolant an jeder Stelle x um $\delta \ell_j(x)$.

Antwort: Wahr

So steht es in Satz 13.3.12, und der Beweis ist die Eindeutigkeit aus Satz 13.3.5. Weil ℓ_j nur an den anderen Knoten verschwindet, wirkt die Änderung im gesamten Intervall, bei $n = 10$ mit einem Faktor bis 4,03 und am stärksten fern von x_j (Bemerkung 13.3.13).

Vertiefung: Heath, Scientific Computing, §7.3.1 (Monombasis und Vandermonde-Matrix), §7.3.4 (orthogonale Polynome) und §7.3.5 (Interpolation stetiger Funktionen mit Runge-Phänomen und Chebyshev-Knoten); die Lagrange-Grundpolynome aus Satz 13.3.12 stehen dort in §7.3.2.

13.4 Splines und B-Splines

Abschnitt 13.3 endet mit einer unbequemen Beobachtung: Ein einzelnes Polynom, das durch viele Datenpunkte gezwungen wird, schlägt zwischen ihnen aus, und der Ausschlag wächst mit der Zahl der Punkte. Der Grund liegt im Ansatzraum selbst. Ein Polynom vom Grad $n - 1$ ist ein **globales** Objekt: Jeder Koeffizient wirkt überall, jede Messung zieht an der ganzen Kurve.

Der Ausweg dieses Abschnitts kehrt das um. Wir zerlegen das Intervall in kleine Stücke und legen auf jedes ein Polynom von *festem, niedrigem* Grad. Der Grad wächst dann nicht mehr mit der Datenmenge, es wächst nur die Anzahl der Stücke. Damit daraus keine zerbrochene Linie wird,

verlangen wir an den Nahtstellen so viel Glattheit, wie der Grad hergibt.

Stückweise Polynome mit vorgeschriebener Glattheit

Definition 13.4.1 (Polynom-Spline vom Grad q)

Sei $[a, b] \subset \mathbb{R}$ ein Intervall und sei

$$a = \xi_0 < \xi_1 < \dots < \xi_m = b$$

ein Gitter von *Knoten* (knots). Eine Funktion $s : [a, b] \rightarrow \mathbb{R}$ heißt *Polynom-Spline vom Grad q* (polynomial spline of degree q) zu diesem Gitter, wenn

1. auf jedem Teilintervall ein Polynom steht: $s(x) = p_k(x)$ für $x \in [\xi_{k-1}, \xi_k)$ mit Polynomen p_k vom Grad höchstens q , $k = 1, \dots, m$ (auf dem letzten Teilintervall abgeschlossen), und
2. die Nahtstellen glatt sind: $s \in \mathcal{C}^{q-1}([a, b])$, also sind s und seine ersten $q-1$ Ableitungen auf ganz $[a, b]$ stetig.

Die Menge aller solchen Funktionen bezeichnen wir mit $\mathcal{S}_q$.

Bemerkung 13.4.2 (Was in dieser Definition steckt)

Drei Punkte lohnen einen zweiten Blick.

Grad höchstens q . Die kurze Sprechweise „Polynome q -ten Grades“ meint stets „vom Grad höchstens q “. Sonst wäre nicht einmal die Nullfunktion ein Spline, und $\mathcal{S}_q$ wäre kein Vektorraum.

Die Glattheit ist eine Forderung, kein Geschenk. Eine stückweise durch Polynome vom Grad höchstens q definierte Funktion ist von sich aus gar nichts: Sie darf an jedem Knoten springen. Erst Bedingung 2 macht sie zum Spline. $\mathcal{C}^{q-1}$ ist dabei die *größtmögliche* Glattheit, die man verlangen kann, ohne den Spline zu einem einzigen globalen Polynom zu zwingen. Fordern wir nämlich zusätzlich, dass auch die q -te Ableitung stetig ist, so stimmen benachbarte Polynome in allen Ableitungen überein und sind damit gleich.

Der Fall $q = 0$. Dann bedeutet $\mathcal{C}^{-1}$ schlicht: keine Stetigkeitsforderung. Ein Spline vom Grad 0 ist eine Treppenfunktion, die an jedem Knoten springen darf. Ab $q = 1$ ist s stetig, ab $q = 2$ auch knickfrei.

Beispiel 13.4.3 (Die Grade 0, 1 und 3)

Auf dem Gitter $0 < 1 < 2 < 3$ sehen die drei gebräuchlichsten Grade so aus.

- $q = 0$: konstante Stücke, Sprünge an den Knoten erlaubt. Das ist der Histogramm-Fall.
- $q = 1$: der Streckenzug durch die Daten, stetig, mit Knicken an den Knoten. Für Interpolation braucht er keine Rechnung, wir verbinden die Punkte.
- $q = 3$: kubische Stücke, an den Knoten stimmen Funktionswert, Steigung und Krümmung überein. Ein Knick ist nicht mehr zu sehen, und darum ist $q = 3$ der Standardfall.

Höhere Grade sind möglich, bringen aber wenig: Die Kurve wird kaum glatter fürs Auge, und die Stücke fangen wieder an zu schwingen.

Wie viele Parameter hat ein Spline?

Bevor wir interpolieren, sollten wir wissen, wie groß der Ansatzraum überhaupt ist. Nach Abschnitt 13.2 entscheidet diese Zahl darüber, wie viele Bedingungen wir stellen dürfen.

Satz 13.4.4 (Dimension des Spline-Raums)

Sei $q \geq 1$ und sei $\mathcal{S}_q$ der Raum der Polynom-Splines vom Grad q zum Gitter $a = \xi_0 < \dots < \xi_m = b$. Dann ist $\mathcal{S}_q$ ein Vektorraum der Dimension

$$\dim \mathcal{S}_q = m + q, \quad (13.4.1)$$

und die $m + q$ Funktionen

$$1, x, \dots, x^q, (x - \xi_1)_+^q, \dots, (x - \xi_{m-1})_+^q \quad \text{mit} \quad u_+ := \max(u, 0)$$

bilden eine Basis, die *Basis der abgeschnittenen Potenzen* (truncated power basis).

Vertiefung: Warum die abgeschnittenen Potenzen eine Basis bilden

Der Beweis baut einen Spline von links nach rechts aus Monomen und abgeschnittenen Potenzen auf und zeigt, dass diese Familie linear unabhängig ist. Die Abzählung darunter kommt ohne ihn aus.

Beweis.

- (1) Jede der genannten Funktionen liegt in $\mathcal{S}_q$.

Die Monome $1, x, \dots, x^q$ sind auf ganz $[a, b]$ Polynome vom Grad höchstens q und beliebig oft differenzierbar. Für die abgeschnittene Potenz $g_i(x) = (x - \xi_i)_+^q$ ist $g_i \equiv 0$ links von ξ_i und $g_i(x) = (x - \xi_i)^q$ rechts davon, auf beiden Seiten also ein Polynom vom Grad höchstens q .

Für $j \leq q - 1$ ist die j -te Ableitung rechts von ξ_i gleich $q(q-1) \cdots (q-j+1)(x - \xi_i)^{q-j}$ mit Exponent $q-j \geq 1$; sie verschwindet also in ξ_i , ebenso wie die Ableitung von links. Damit ist $g_i \in \mathcal{C}^{q-1}$, und erst die q -te Ableitung springt (von 0 auf $q!$).

- (2) Jedes $s \in \mathcal{S}_q$ ist eine Linearkombination dieser Funktionen.

Sei p_1 das Polynom auf dem ersten Teilintervall. Wir zeigen per Induktion über k , dass es Zahlen $d_1, \dots, d_{k-1}$ gibt mit

$$s(x) = p_1(x) + \sum_{i=1}^{k-1} d_i (x - \xi_i)_+^q \quad \text{für } x \in [a, \xi_k).$$

Für $k = 1$ ist das die Definition von p_1 . Gilt die Darstellung bis ξ_k , so ist die Differenz r zwischen s und der rechten Seite auf $[\xi_k, \xi_{k+1})$ ein Polynom vom Grad höchstens q , und ihre Ableitungen der Ordnungen $0, \dots, q-1$ verschwinden in ξ_k .

Ein Polynom vom Grad höchstens q , dessen Ableitungen bis zur Ordnung $q-1$ in ξ_k verschwinden, ist aber ein Vielfaches von $(x - \xi_k)^q$ (Taylor-Entwicklung um ξ_k). Das liefert d_k und den Induktionsschritt.

Beide Funktionen sind dort $\mathcal{C}^{q-1}$ und stimmen links von ξ_k überein, also stimmen auch ihre einseitigen Ableitungen bis zur Ordnung $q-1$ überein und die der Differenz sind null.

- (3) Die Funktionen sind linear unabhängig.

Verschwindet eine Linearkombination auf ganz $[a, b]$, so verschwindet sie insbesondere auf $[a, \xi_1)$. Dort tragen nur die Monome bei, also ist das Polynom $\sum_j c_j x^j$ auf einem ganzen Intervall null und damit das Nullpolynom, $c_0 = \dots = c_q = 0$.

Auf $[\xi_1, \xi_2)$ bleibt dann nur $d_1(x - \xi_1)^q$ übrig, also $d_1 = 0$, und so weiter bis $d_{m-1} = 0$.

Ein Polynom vom Grad höchstens q hat höchstens q Nullstellen, sofern es nicht das Nullpolynom ist; ein Intervall enthält unendlich viele Punkte.

- (4) Damit ist die angegebene Familie eine Basis, und ihre Länge ist $(q + 1) + (m - 1) = m + q$.

□

Bemerkung 13.4.5 (Die Parameterzählung als Gegenprobe)

Zum selben Ergebnis kommt man ohne Basis, durch bloßes Abzählen.

- *Ohne Glattheitsforderung:* m Teilintervalle mit je $q + 1$ Koeffizienten, also $m(q + 1)$ freie Parameter.
- *An jedem der $m - 1$ inneren Knoten:* q Bedingungen, nämlich Stetigkeit von s und der ersten $q - 1$ Ableitungen.

Das ergibt

$$m(q + 1) - (m - 1)q = mq + m - mq + q = m + q.$$

Für $q = 3$ und $m = 5$ sind das $5 \cdot 4 - 4 \cdot 3 = 20 - 12 = 8 = m + q$. Diese Rechnung setzt allerdings voraus, dass die $(m - 1)q$ Bedingungen linear unabhängig sind, sonst dürften wir sie nicht einfach abziehen. Satz 13.4.4 liefert genau das nach.

Beide Wege stützen sich auf zwei Annahmen, die wir nicht weglassen dürfen: *einfache innere Knoten* (kein ξ_i tritt doppelt auf) und *maximale Glattheit* $\mathcal{C}^{q-1}$. Erlauben wir an einem Knoten weniger Glattheit, so fällt dort eine Bedingung weg und die Dimension steigt um eins.

Interpolation mit kubischen Splines

Jetzt können wir zählen, ob ein Interpolationsproblem aufgeht. Wir haben $n = m + 1$ Datenpunkte an den Knoten selbst und wählen $q = 3$.

Bemerkung 13.4.6 (Zwei Bedingungen fehlen: Randbedingungen)

Nach (13.4.1) hat der Ansatzraum die Dimension $m + 3$, die Daten liefern aber nur $n = m + 1$ Gleichungen. Es fehlen also genau *zwei* Bedingungen, und ohne sie hat das Interpolationsproblem unendlich viele Lösungen. Üblich sind drei Sorten von *Randbedingungen* (boundary conditions):

Typ	Bedingung
natürlich	$s''(a) = s''(b) = 0$
eingespannt	$s'(a)$ und $s'(b)$ vorgegeben
periodisch	$s'(a) = s'(b)$ und $s''(a) = s''(b)$

Liegt keine Information über den Rand vor, ist der *natürliche Spline* der Standard. Er hat einen guten Grund für sich: Unter allen Interpolanten mit zwei stetigen Ableitungen hat er die kleinste Gesamtkrümmung $\int_a^b |g''(x)|^2 dx$ (Abschnitt 13.5).

Im periodischen Fall ist die Wahl der beiden Bedingungen heikler, als sie aussieht: $s(a) = s(b)$ taugt nicht als eine davon. Sind die Daten periodisch, gilt also $y_0 = y_m$, so folgt $s(a) = s(b)$ schon aus den Interpolationsbedingungen und ist keine neue Gleichung; sind sie es nicht, so widerspricht die Forderung ihnen. Für $q = 3$ und $m = 4$ nachgerechnet: Mit $\{s(a) = s(b), s'(a) = s'(b)\}$ hat die 16×16 -Matrix nur den Rang 15, ein Freiheitsgrad bleibt offen. Erst $\{s'(a) = s'(b), s''(a) = s''(b)\}$ macht sie regulär.

Beispiel 13.4.7 (Kubischer Spline durch vier Punkte)

Gegeben seien die vier Punkte $(0, 0)$, $(1, 1)$, $(2, 0)$, $(3, -1)$, also $m = 3$ Teilintervalle. Auf jedes legen wir ein kubisches Polynom $p_k(x) = c_{k1} + c_{k2}x + c_{k3}x^2 + c_{k4}x^3$; das sind $3 \cdot 4 = 12$ Unbekannte. Die Bedingungen:

Typ	Anzahl	Beispiel
Interpolation	4	$p_1(0) = 0, p_1(1) = 1, p_2(2) = 0, p_3(3) = -1$
s stetig	2	$p_1(1) = p_2(1)$
s' stetig	2	$p_1'(1) = p_2'(1)$
s'' stetig	2	$p_1''(1) = p_2''(1)$
natürlicher Rand	2	$s''(0) = s''(3) = 0$

Zusammen $4 + 2 + 2 + 2 + 2 = 12$ Bedingungen für 12 Unbekannte, und das lineare Gleichungssystem ist eindeutig lösbar. Beachten wir, dass jeder Datenpunkt nur *einmal* gezählt wird: Dass etwa auch $p_2(1) = 1$ gilt, erzwingt schon die Stetigkeitszeile.

Die Lösung lautet

$$p_1(x) = \frac{1}{15}(23x - 8x^3), \quad p_2(x) = \frac{1}{15}(-18 + 77x - 54x^2 + 10x^3), \quad p_3(x) = \frac{1}{15}(78 - 67x + 18x^2 - 2x^3).$$

Die Probe geht glatt auf: $p_1(1) = \frac{15}{15} = 1 = p_2(1)$, und an der Nahtstelle $x = 1$ ist $p_1'(1) = p_2'(1) = -\frac{1}{15}$ sowie $p_1''(1) = p_2''(1) = -\frac{48}{15}$. Am linken Rand ist $p_1''(0) = 0$, am rechten $p_3''(3) = \frac{36-36}{15} = 0$, wie die natürliche Randbedingung verlangt.

Interaktiv: Das Zwölf-mal-zwölf-System live

Das Widget baut die zwölf Zeilen genau in der Reihenfolge der Tabelle auf und löst das System mit Spaltenpivotierung (Abschnitt 5.2). Die vier Messwerte lassen sich verschieben, und der Schalter tauscht die letzten beiden Zeilen gegen die eingespannte Variante.

Interaktives Element — nur in der Web-Fassung

Zwei Dinge sind daran wichtig. Erstens ist die Naht unsichtbar, obwohl links und rechts davon verschiedene Polynome stehen; das Readout zeigt, dass Wert, Steigung und Krümmung bis auf Rundung übereinstimmen. Zweitens ändern sich beim Verschieben eines einzigen Messwerts *alle* zwölf Koeffizienten.

B-Splines

Die stückweise Monomdarstellung aus Beispiel 13.4.7 ist zum Rechnen schlecht geeignet. Sie erbt die schlechte Kondition der Monombasis (Abschnitt 13.3), ihre Matrix hat keine erkennbare Struktur, und jeder Koeffizient hängt an allen Daten: Verschieben wir einen einzigen Messwert, so ändern sich alle zwölf Koeffizienten, in dieser Darstellung ist Spline-Interpolation also global. Wir suchen deshalb eine *Basis* des Raums $\mathcal{S}_q$ im Sinn von Abschnitt 13.2, aus der sich die Systemmatrix B mit den Einträgen $B_{ik} = \phi_k(x_i)$ besser verhält. Nach Satz 13.4.4 muss eine solche Basis genau $m + q$ Funktionen haben.

Die abgeschnittenen Potenzen aus dem Beweis wären eine, taugen aber nicht: Sie sind fast alle

auf dem halben Intervall von null verschieden, und für großes x werden sie riesig. Die B-Splines sind die Basis, die stattdessen gebraucht wird. Ihr Name kommt von *Basis*, ihre Gestalt von der Rekursion, die sie erzeugt.

Definition 13.4.8 (Erweiterte Knotenfolge und B-Splines)

Zum Gitter $a = \xi_0 < \dots < \xi_m = b$ und zum Grad $q \geq 0$ setzen wir die *erweiterte Knotenfolge* $\tau_1 \leq \dots \leq \tau_{m+2q+1}$ fest durch

$$\tau_1 = \dots = \tau_{q+1} = \xi_0, \quad \tau_{q+1+i} = \xi_i \quad (i = 1, \dots, m-1), \quad \tau_{m+q+1} = \dots = \tau_{m+2q+1} = \xi_m. \quad (13.4.2)$$

Die Randknoten treten also $(q+1)$ -fach auf, die inneren einfach; zusammen sind das $(q+1) + (m-1) + (q+1) = m+2q+1$ Knoten.

Die B-Splines $B_k^{(q)}$, $k = 1, \dots, m+q$, sind rekursiv über den Grad erklärt. Für $q = 0$ ist

$$B_k^{(0)}(x) = \mathbb{1}(\tau_k \leq x < \tau_{k+1}),$$

und für $q > 0$ gilt die *Cox-de-Boor-Rekursion*

$$B_k^{(q)}(x) = \frac{x - \tau_k}{\tau_{k+q} - \tau_k} B_k^{(q-1)}(x) + \frac{\tau_{k+q+1} - x}{\tau_{k+q+1} - \tau_{k+1}} B_{k+1}^{(q-1)}(x). \quad (13.4.3)$$

Ein Summand mit verschwindendem Nenner wird dabei als 0 gelesen; das betrifft nur die mehrfachen Randknoten. Am rechten Rand lesen wir das letzte nichtleere Knotenintervall als abgeschlossen, damit die Basisfunktionen auch in b definiert sind.

Bemerkung 13.4.9 (Warum die Knotenfolge so lang sein muss)

Zu $m+q$ Basisfunktionen nur $m+q$ Knoten anzusetzen, liegt nahe, reicht aber nicht: Die Rekursion (13.4.3) greift für $k = m+q$ auf τ_{m+2q+1} zu, es braucht also $m+2q+1$ Knoten. Ebenso wichtig ist, dass sich die drei Vorschriften in (13.4.2) nicht ins Gehege kommen. Der linke Randblock belegt die Indizes $1, \dots, q+1$, die inneren Knoten die Indizes $q+2, \dots, q+m$, der rechte Randblock die Indizes $m+q+1, \dots, m+2q+1$: Jeder Index bekommt genau einen Wert. Begänne der rechte Block schon bei $m+1$, so bekäme für $q = 3$ und $m = 5$ der Knoten τ_6 sowohl ξ_2 als auch ξ_5 zugewiesen.

Beispiel 13.4.10 (Grad 1: die Hutfunktionen)

Für $\xi = (0, 1, 2)$, also $m = 2$, und $q = 1$ lautet die erweiterte Knotenfolge

$$\tau = (0, 0, 1, 2, 2),$$

das sind $m+2q+1 = 5$ Knoten und damit $m+q = 3$ Basisfunktionen. Die Rekursion liefert

$$B_1^{(1)}(x) = \begin{cases} 1-x, & 0 \leq x < 1 \\ 0 & \text{sonst,} \end{cases} \quad B_2^{(1)}(x) = \begin{cases} x, & 0 \leq x < 1 \\ 2-x, & 1 \leq x \leq 2 \end{cases} \quad B_3^{(1)}(x) = \begin{cases} x-1, & 1 \leq x \leq 2 \\ 0 & \text{sonst.} \end{cases}$$

An der Stelle $x = 0,25$ etwa stehen dort die Werte 0,75, 0,25 und 0, bei $x = 1,5$ die Werte 0, 0,5 und 0,5. In beiden Fällen ist die Summe 1, und das gilt an jeder Stelle. Jedes dieser drei Dächer ist selbst ein Spline vom Grad 1, jedes ist nichtnegativ, und jedes ist nur auf höchstens $q+1 = 2$ Gitterintervallen von null verschieden. An den Rändern sorgen die doppelten Knoten dafür, dass der Träger kürzer ausfällt.

Satz 13.4.11 (Die B-Splines sind eine Basis)

Mit den Bezeichnungen aus Definition 13.4.8 gilt für $q \geq 1$:

1. *Lokaler Träger:* $B_k^{(q)}(x) = 0$ für $x \notin [\tau_k, \tau_{k+q+1}]$, und $B_k^{(q)}(x) > 0$ für $\tau_k < x < \tau_{k+q+1}$.
2. *Zerlegung der Eins:* $\sum_{k=1}^{m+q} B_k^{(q)}(x) = 1$ für alle $x \in [a, b]$.
3. *Glattheit:* $B_k^{(q)} \in \mathcal{C}^{q-1}$, jedes $B_k^{(q)}$ liegt also selbst in $\mathcal{S}_q$.
4. *Basis:* $B_1^{(q)}, \dots, B_{m+q}^{(q)}$ sind linear unabhängig und spannen $\mathcal{S}_q$ auf. Jeder Spline s vom Grad q besitzt daher genau eine Darstellung

$$s(x) = \sum_{k=1}^{m+q} a_k B_k^{(q)}(x). \quad (13.4.4)$$

Einen Beweis führen wir nicht; die Rekursion selbst brauchen wir im Weiteren nur zur Referenz. Plausibel ist die Aussage aber sofort: Aussage 1 und 3 lesen sich direkt an einem Rekursionsschritt ab (siehe die Vertiefungen), und die Anzahl $m + q$ ist genau die Dimension aus Satz 13.4.4, sodass ein aufspannendes System automatisch eine Basis ist.

Damit ist der Anschluss an Abschnitt 13.2 hergestellt: Wir wählen $\phi_k = B_k^{(q)}$ als Basisfunktionen, stellen das System $\mathbf{B}\mathbf{a} = \mathbf{y}$ auf und lösen es. Neu ist nur, wie gut sich dieses System benimmt.

Interaktiv: Die B-Spline-Basis erkunden

Wie viele Basisfunktionen sind an einer Stelle aktiv, und wie entsteht ihre Form aus der Rekursion?

Interaktives Element — nur in der Web-Fassung

Die gestrichelte Summe liegt über dem ganzen Gitter konstant bei eins, obwohl die einzelnen Kurven ganz verschieden aussehen. Und an jeder Stelle sind höchstens $q + 1$ Basisfunktionen von null verschieden: Das ist die Zahl, die gleich die Rechenzeit bestimmt. Der aufgeklappte Rekursionsschritt zeigt die beiden Gewichtsrampen an derselben Idee und erklärt, warum der Grad um eins steigt.

Bemerkung 13.4.12 (B-Splines in R)

In R liefert `splines::bs()` die Auswertungsmatrix $\mathbf{B}$ direkt. Ein typischer Aufruf sieht so aus:

```
y <- f(x)
B <- splines::bs(x, df = 10, degree = 3, intercept = TRUE)
a <- solve(B, y)

xnew <- seq(0, 1, length = 100)
Bnew <- splines::bs(xnew, knots = attr(B, "knots"),
                    Boundary.knots = attr(B, "Boundary.knots"),
                    degree = attr(B, "degree"),
                    intercept = TRUE)

fhat <- Bnew %*% a
```

Die erste Hälfte ist der Algorithmus aus Abschnitt 13.2 in drei Zeilen: Basis auswerten, Sys-

tem lösen, fertig. Die zweite Hälfte wertet den Interpolanten an neuen Stellen aus, und dabei steckt die eigentliche Fußangel: Die neue Basismatrix muss zu *derselben* Basis gehören wie die alte. Deshalb werden innere Knoten, Randknoten und Grad weitergereicht, statt `bs()` Teile davon aus `xnew` neu bestimmen zu lassen. Vergisst man das, geht nichts schief, was man merken würde: Die Spaltenzahl stimmt weiterhin, es gibt keine Fehlermeldung, und heraus kommt eine glatte Kurve – nur eben nicht die geschätzte. Am sichersten erledigt das bei einem angepassten Modell die zugehörige `predict()`-Methode.

Warum B-Splines? Kondition und Aufwand

Bemerkung 13.4.13 (Konditionszahlen: eine Größenordnung, kein Messwert)

Für $n = 20$ Punkte in $[0, 1]$ stellen wir zwei Basissysteme gegenüber. Die Konditionszahlen sind *illustrativ*: Sie hängen von der verwendeten Norm, von der Lage der Auswertungsstellen und von der Knotenwahl ab.

Basis	Grad	Konditionszahl
Monombasis	19	$\kappa(\mathbf{B}_{\text{mono}}) \approx 10^{16}$
kubische B-Splines	3	$\kappa(\mathbf{B}_{\text{spline}}) \approx 10^1 \text{ bis } 10^2$

Die obere Zeile setzt die Tabelle aus Abschnitt 13.3; wir haben sie in exakter rationaler Arithmetik nachgerechnet und erhalten $\kappa_1 = 4,4 \cdot 10^{16}$. Die kubische B-Spline-Kollokationsmatrix mit ebenfalls 20 Basisfunktionen kommt in derselben Norm auf $\kappa_1 = 37$. Zwischen beiden liegen gut fünfzehn Größenordnungen; rundet man den B-Spline-Wert großzügig auf 10^2 auf, werden es vierzehn. Auf die genaue Zahl kommt es nicht an, wohl aber auf ihre Deutung: Bei $\kappa \approx 10^{16}$ ist von den etwa sechzehn signifikanten Stellen eines `double` (Abschnitt 4.2) im Ergebnis nichts mehr sicher, während $\kappa \approx 40$ nicht einmal zwei Stellen kostet.

Bemerkung 13.4.14 (Bandstruktur und Aufwand)

Der zweite Vorteil ist die Struktur der Matrix. Nach Satz 13.4.11 ist an jeder Stelle x_i höchstens $q + 1$ der Basisfunktionen von null verschieden, in der i -ten Zeile von $\mathbf{B}$ stehen also höchstens $q + 1$ Einträge ungleich null, und sie stehen nebeneinander. $\mathbf{B}$ ist eine dünn besetzte *Bandmatrix* mit Bandbreite ungefähr q .

Für die Elimination heißt das: Wo bei einer vollbesetzten Matrix in jedem Schritt alle verbleibenden Zeilen angefasst werden, sind es hier nur die höchstens q Zeilen, die unterhalb des Pivots noch im Band liegen, und in jeder davon nur die höchstens q Einträge des Bandes (Abschnitt 5.3). Bei einer Systemgröße N kostet das

$$O(Nq^2) \quad \text{statt} \quad O(N^3)$$

(Abschnitt 2.4). Hier bezeichnet N die Größe des Systems, also die Zahl der Basisfunktionen $m + q$; bei Interpolation an n Punkten fällt sie mit n zusammen, sonst nicht. Es lohnt sich, die beiden Größen auseinanderzuhalten.

Für $N = 1000$ und $q = 3$ ist das Verhältnis $N^3/(Nq^2) = N^2/q^2 = 10^6/9 \approx 111\,000$, also ein Beschleunigungsfaktor der Größenordnung 10^5 . Verglichen werden dabei Operationszahlen, keine gemessenen Laufzeiten.

Eigenschaften

Splines sind heute das gebräuchlichste Basissystem zur Funktionsapproximation, und in aller Regel wird mit $q = 3$ gearbeitet. Zum Abschluss sammeln wir die Eigenschaften, die diese Vorliebe erklären.

Satz 13.4.15 (Eigenschaften von Splines und B-Splines)

Sei $q \geq 1$.

1. *Approximierbarkeit:* Zu jeder stetigen Funktion f auf $[a, b]$ und jedem $\varepsilon > 0$ gibt es ein Gitter, dessen maximale Knotenweite $\max_k |\xi_k - \xi_{k-1}|$ klein genug ist, sodass der zugehörige Raum $\mathcal{S}_q$ ein s mit $\|f - s\|_\infty < \varepsilon$ enthält.
2. *Ableitung und Integral:* Die Ableitung eines Splines vom Grad q ist ein Spline vom Grad $q - 1$ zum selben Gitter, eine Stammfunktion ist ein Spline vom Grad $q + 1$.
3. *Lokaler Träger:* $B_k^{(q)}$ verschwindet außerhalb von $[\tau_k, \tau_{k+q+1}]$, also außerhalb von höchstens $q + 1$ Gitterintervallen.

Aussage 2 ist schnell eingesehen: Leiten wir stückweise ab, so sinkt der Grad jedes Stücks um eins, und aus $s \in \mathcal{C}^{q-1}$ wird $s' \in \mathcal{C}^{q-2}$. Das ist wieder genau die maximale Glattheit zum Grad $q - 1$. Für $q = 3$ heißt das: Die Ableitung eines kubischen Splines ist ein quadratischer Spline, die Krümmung eine stückweise lineare Funktion. Aussage 1 ist im Fall $q = 1$ nichts anderes als die gleichmäßige Stetigkeit von f , für höhere Grade ein klassisches Resultat der Approximationstheorie; wie schnell der Fehler mit der Knotenweite fällt, klärt Abschnitt 13.6.

Bemerkung 13.4.16 (Vorsicht bei den Ableitungen)

Aus Aussage 1 wird gern gefolgert, dass auch die Ableitungen und das Integral des Splines die entsprechenden Größen von f approximieren. Für das Integral stimmt das ohne Weiteres, denn aus $\|f - s\|_\infty < \varepsilon$ folgt sofort eine Schranke $(b - a)\varepsilon$ für die Differenz der Integrale. Bei den Ableitungen ist Vorsicht geboten: Sie brauchen erstens, dass f selbst genügend oft differenzierbar ist, und sie konvergieren zweitens langsamer. Jede abgeleitete Ordnung kostet eine Potenz der Knotenweite. Eine kleine Abweichung in den Funktionswerten sagt für sich genommen noch nichts über die Steigungen aus.

Bemerkung 13.4.17 (Was die Lokalität praktisch bedeutet)

Der lokale Träger ist ein Hauptgrund, warum B-Splines in der Praxis benutzt werden. Verschieben wir einen einzigen Messwert, so ändert sich zunächst nur die entsprechende rechte Seite; die zugehörige Zeile der Basismatrix enthält nur die wenigen Basisfunktionen in seiner Umgebung. Die Lösung des Gleichungssystems koppelt diese Koeffizienten allerdings weiter: Die Inverse einer Bandmatrix ist im Allgemeinen dicht. Für gut konditionierte, reguläre Bandsysteme fällt ihr Einfluss typischerweise schnell und häufig geometrisch mit dem Abstand ab, aber der kompakte Träger allein garantiert weder einen universellen Faktor noch exakte Abschottung. Ein globales Polynom verhält sich im Vergleich meist deutlich weniger lokal. Abschnitt 13.3 hat den Verstärkungsfaktor dort schon beziffert: Die Störung eines einzigen Werts wirkt über das ganze Intervall und schlägt am weitesten dort aus, wo gar nichts passiert ist.

Wie stark die Kopplung mit dem Abstand abfällt, rechnen wir im gleichmäßigen Beispiel unten nach; die Zahlen dort sind Messwerte dieses Systems, keine allgemeine B-Spline-Konstante.

Interaktiv: Ein Datenpunkt wandert

Das Widget stört einen einzelnen Datenpunkt. Links steht das Interpolationspolynom vom Grad 8, rechts der natürliche kubische Spline zu denselben neun Punkten.

Interaktives Element — nur in der Web-Fassung

In der Voreinstellung wandert der fünfte Punkt um 1 nach oben. Der Spline ändert sich um höchstens 1,000, das Polynom um 2,320, also um mehr als die Störung selbst. Noch deutlicher wird der Unterschied im Fernbereich: Mehr als zwei Knoten vom verschobenen Punkt entfernt bleibt die Spline-Änderung unter 0,037, während das Polynom dort weiterhin 2,320 ausschlägt, und zwar am äußeren Rand. Die Balken zeigen, wo die Änderung sitzt: Von 1,732 fällt sie über 0,464, 0,124, 0,031 auf 0,010, also je Knotenabstand auf gut ein Viertel. Neun der elf Koeffizienten reagieren messbar, obwohl an der verschobenen Stelle nur drei Basisfunktionen überhaupt von null verschieden sind.

Selbsttest

1. Ein Spline vom Grad q auf m Teilintervallen ist ein Polynom vom Grad mq .

Antwort: Falsch

Er ist überhaupt kein einzelnes Polynom, sondern eine Zusammensetzung aus m Polynomen vom Grad *höchstens* q . Der Grad wächst gerade nicht mit der Zahl der Stücke; das ist der ganze Punkt der Konstruktion. Nur im Sonderfall $m = 1$ liegt ein einzelnes Polynom vor, und auch dann vom Grad höchstens q .

2. Zur Beschreibung eines Splines vom Grad q auf $m + 1$ Knoten brauchen wir $m + q$ Parameter.

Antwort: Wahr

Das ist Satz 13.4.4, unter seinen beiden Annahmen: einfache innere Knoten und maximale Glattheit $\mathcal{C}^{q-1}$. Die Abzählung dazu lautet $m(q + 1) - (m - 1)q = m + q$; für $q = 3$ und $m = 5$ also $20 - 12 = 8$. Verlangen wir an einem Knoten weniger Glattheit, steigt die Zahl.

3. Jede Funktion, die auf jedem Teilintervall mit einem Polynom vom Grad höchstens q übereinstimmt, ist ein Spline vom Grad q .

Antwort: Falsch

Die Glattheit gehört zur Definition (Bemerkung 13.4.2). Die Funktion, die auf $[0, 1)$ konstant 0 und auf $[1, 2]$ konstant 1 ist, besteht aus zwei Polynomen vom Grad höchstens 3, ist aber nicht einmal stetig und damit kein kubischer Spline. Als Spline vom Grad 0 geht sie durch, denn dort fordert $\mathcal{C}^{-1}$ nichts.

4. Fordern wir von einem kubischen Spline zusätzlich, dass auch s''' stetig ist, so bekommen wir eine glattere, aber immer noch stückweise verschiedene Kurve.

Antwort: Falsch

Dann stimmen benachbarte kubische Polynome an ihrer Nahtstelle in allen vier Ableitungen der Ordnungen 0 bis 3 überein und sind deshalb identisch. Aus dem Spline wird ein einziges globales Polynom vom Grad höchstens 3, und wir sind zurück bei dem Ansatz, den wir gerade verlassen haben. $\mathcal{C}^{q-1}$ ist die höchste sinnvolle Glattheitsforderung.

5. Interpolieren wir $n = m + 1$ Datenpunkte mit einem kubischen Spline zum Gitter der Datenpunkte, so ist der Interpolant eindeutig bestimmt.

Antwort: Falsch

Der Ansatzraum hat die Dimension $m + 3$, die Daten liefern aber nur $m + 1$ Gleichungen. Zwei Bedingungen fehlen, und ohne sie gibt es unendlich viele Lösungen (Bemerkung 13.4.6). Erst eine Randbedingung wie $s''(a) = s''(b) = 0$ macht das System quadratisch und regulär.

6. Die Ableitung eines kubischen Splines ist ein quadratischer Spline zum selben Gitter.

Antwort: Wahr

Stückweises Ableiten senkt den Grad jedes Stücks von 3 auf 2, und aus $\mathcal{C}^2$ wird $\mathcal{C}^1$ (Satz 13.4.15). Das ist genau die maximale Glattheit zum Grad 2. Entsprechend ist die zweite Ableitung eines kubischen Splines stückweise linear und stetig, während die dritte stückweise konstant ist und an den Knoten springen darf.

7. Für $m + q$ B-Splines vom Grad q genügt eine Knotenfolge mit $m + q$ Gliedern.

Antwort: Falsch

Die Rekursion (13.4.3) greift für $k = m + q$ auf τ_{k+q+1} zu, es braucht also $m + 2q + 1$ Knoten. Für $q = 3$ und $m = 17$ sind das 24 statt 20 (Bemerkung 13.4.9).

8. Verschieben wir einen Datenpunkt, so ändern sich nur die Koeffizienten derjenigen B-Splines, die an dieser Stelle von null verschieden sind.

Antwort: Falsch

Die Änderung ist lokalisiert, aber nicht exakt lokal. An einem inneren Datenpunkt sind bei $q = 3$ nur drei Basisfunktionen von null verschieden, im Beispiel des Widgets reagieren jedoch neun von elf Koeffizienten messbar: 1,732, dann 0,464, 0,124, 0,031, 0,010. Das Gleichungssystem koppelt die Koeffizienten über das Band, und die Kopplung fällt je Knotenabstand auf gut ein Viertel, statt abubrechen.

Vertiefung: Heath behandelt die stückweise polynomiale Interpolation in §7.4, die kubischen Splines in §7.4.2 und die B-Splines samt Rekursion und Bandstruktur in §7.4.3; die klassische Referenz zur Sache ist Carl de Boor, A Practical Guide to Splines.

13.5 Minimale Krümmung

Bis hierher war Interpolation ein lineares Problem: Basis wählen, System lösen (Abschnitt 13.2). Gelandet sind wir bei stückweisen Polynomen, weil globale Polynome hohen Grades am Rand ausschlagen. Eine Frage ist dabei offen geblieben: Warum laufen ausgerechnet *kubische* Splines überall als Standard? Grad 2 wäre billiger, Grad 5 glatter.

Die Antwort dieses Abschnitts ist ein Optimalitätssatz. Der kubische Spline ist nicht irgendein Interpolant, sondern die Lösung eines Optimierungsproblems: Unter allen zweimal stetig differenzierbaren Funktionen durch dieselben Punkte ist er derjenige mit der geringsten Gesamtkrümmung. Die Eigenschaft trägt den englischen Namen *minimal wiggleness*, und der trifft das Bild gut. Der kubische Spline zappelt am wenigsten.

Bemerkung 13.5.1 (Was die zweite Kapitelhälfte voraussetzt)

Von hier an lebt das Kapitel von Ergebnissen aus mehreren Ecken des Skripts. Wir sammeln sie mit den zugehörigen Abschnitten:

- **Lineare Algebra:** Kleinste Quadrate (Abschnitt 7.1) und die Pseudoinverse (Abschnitt 7.6). Beide brauchen wir ab Abschnitt 13.7, wenn aus dem Interpolationsproblem eine Regression wird. Die Pseudoinverse liefert dabei eine einheitliche Schreibweise auch bei Rangdefekt; unter vollem Spaltenrang lässt sie sich durch $(\mathbf{B}^\top \mathbf{B})^{-1} \mathbf{B}^\top$ ersetzen, wie die Varianzrechnung in Abschnitt 13.8 zeigt.
- **Analysis:** Glattheitsklassen $\mathcal{C}^k$, also k -mal stetig differenzierbare Funktionen. In diesem Abschnitt zählt $\mathcal{C}^2$, in Abschnitt 13.6 dann $\mathcal{C}^4$. Dazu Integrale der Bauart $\int |f''(x)|^2 dx$ und die partielle Integration.
- **Statistik:** Designmatrix eines linearen Modells und die Zerlegung des mittleren quadratischen Fehlers in Bias und Varianz (Abschnitt 13.8).

Splines, kurz erinnert

Bemerkung 13.5.2 (Spline)

Ein *Spline* setzt Polynome vom Grad höchstens q an vorgegebenen Stellen $\xi_1 < \dots < \xi_{m-1}$ zusammen, den *Knoten* (knots), und zwar so glatt wie möglich: An jedem Knoten passen die Stücke bis zur $(q-1)$ -ten Ableitung zusammen (Abschnitt 13.4). Ein kubischer Spline ($q=3$) ist demnach eine $\mathcal{C}^2$ -Funktion: Wert, erste und zweite Ableitung gehen ohne Sprung über die Knoten. Genau dieser Fall ist der Standard, und dieser Abschnitt sagt, warum.

Ein Wort zur Benennung, weil im Satz weiter unten andere Buchstaben stehen: $\xi_1, \dots, \xi_{m-1}$ sind hier nur die *inneren* Trennstellen. Im Krümmungssatz fallen die Knoten mit den Stützstellen zusammen und heißen deshalb $x_1, \dots, x_n$, die beiden Intervallenden mitgezählt. Bei n Datenpunkten ist also $m = n - 1$ und $\xi_k = x_{k+1}$.

Krümmung als Zielgröße

Wollen wir „möglichst glatt“ zu einer rechenbaren Forderung machen, brauchen wir eine Zahl, die einer Funktion ihre Zappeligkeit zuordnet. Die zweite Ableitung f'' ist der natürliche Kandidat: Sie ist genau dort groß, wo sich die Steigung schnell ändert, und sie verschwindet genau für Geraden. Weil uns nur die Stärke der Biegung interessiert und nicht ihre Richtung, quadrieren wir und integrieren über das ganze Intervall.

Definition 13.5.3 (Krümmungsfunktional)

Für $g \in \mathcal{C}^2[a, b]$ heißt

$$J(g) := \int_a^b |g''(x)|^2 dx \quad (13.5.1)$$

das *Krümmungsfunktional* von g auf $[a, b]$.

J ist eine Zielfunktion auf einem Funktionenraum, und $J(g) = 0$ gilt genau für die affinen Funktionen $g(x) = ax + \beta$. Je größer $J(g)$, desto mehr Biegearbeit steckt in der Kurve.

Streng genommen misst J nicht die geometrische Krümmung. Die ist $\kappa(x) = |g''(x)| / (1 + g'(x)^2)^{3/2}$, und das Integral darüber wäre weder quadratisch noch bequem. Für flache Kurven mit $|g'| \ll 1$ ist $\kappa \approx |g''|$, und J ist die dazu passende quadratische Größe. Der Name „minimale Krümmung“ ist in der Spline-Literatur eingebürgert, gemeint ist immer (13.5.1). Dass J quadratisch ist,

ist kein Schönheitsfehler, sondern der Grund für alles Folgende: Quadratische Ziele führen auf lineare Gleichungen, genau wie bei den Kleinsten Quadraten in Kapitel 7.

Damit lässt sich die Ausgangsfrage präzise stellen. Gegeben sind Punkte, gesucht ist unter allen $\mathcal{C}^2$ -Funktionen, die durch diese Punkte laufen, die mit dem kleinsten J . Interpolanten gibt es unendlich viele (Abschnitt 13.1), das Minimum dagegen ist eindeutig, und es hat einen Namen.

Satz 13.5.4 (Kubische Splines haben minimale Krümmung)

Seien $n \geq 2$ und $(x_1, y_1), \dots, (x_n, y_n) \in [a, b] \times \mathbb{R}$ mit $a = x_1 < x_2 < \dots < x_n = b$. Sei s ein natürlicher kubischer Spline mit Knoten $x_1, \dots, x_n$, der diese Punkte interpoliert, also $s(x_i) = y_i$ für alle i und

$$s''(a) = s''(b) = 0.$$

Sei $g \in \mathcal{C}^2[a, b]$ eine weitere Funktion mit $g(x_i) = y_i$ für alle i . Dann gilt

$$\int_a^b |s''(x)|^2 dx \leq \int_a^b |g''(x)|^2 dx,$$

also $J(s) \leq J(g)$.

Im Kern besteht der Beweis aus zwei partiellen Integrationen, die wir über die Knoten hinweg zusammensetzen. Die beiden Randbedingungen des natürlichen Splines sind darin nicht Zierrat, sondern genau der Grund, warum ein störender Term verschwindet.

Beweis.

- (1) Wir setzen $h := g - s$. Dann ist $h \in \mathcal{C}^2[a, b]$, und weil s und g dieselben Werte interpolieren, gilt

$$h(x_i) = 0 \quad \text{für } i = 1, \dots, n.$$

s ist auf jedem Teilintervall ein Polynom und an den Knoten zweimal stetig differenzierbar, liegt also selbst in $\mathcal{C}^2$; die Differenz zweier $\mathcal{C}^2$ -Funktionen ist wieder $\mathcal{C}^2$

- (2) Ausmultiplizieren von $g'' = s'' + h''$ unter dem Integral liefert die Zerlegung

$$J(g) = J(s) + 2 \int_a^b s''(x) h''(x) dx + \int_a^b (h''(x))^2 dx. \quad (13.5.2)$$

Zu zeigen bleibt, dass der mittlere Term verschwindet.

binomische Formel $(u + v)^2 = u^2 + 2uv + v^2$ mit $u = s''$ und $v = h''$; alle drei Integrale existieren, weil s'' und h'' stetig sind

- (3) Auf jedem einzelnen Knotenintervall $[x_{i-1}, x_i]$ ist s ein Polynom vom Grad höchstens 3, dort also beliebig oft differenzierbar. Deshalb zerlegen wir das mittlere Integral und integrieren auf jedem Stück zweimal partiell:

$$\int_{x_{i-1}}^{x_i} s'' h'' dx = \left[s'' h' - s^{(3)} h \right]_{x_{i-1}}^{x_i} + \int_{x_{i-1}}^{x_i} s^{(4)} h dx.$$

zweimal partiell integriert: der erste Durchgang schiebt einen Strich von h'' auf s'' und hinterlässt den Randterm $s'' h'$ samt $-\int s^{(3)} h'$, der zweite behandelt dieses Integral genauso und liefert den Randterm $-s^{(3)} h$ und $+\int s^{(4)} h$

- (4) Als kubisches Polynom hat s auf jedem Teilintervall $s^{(4)} \equiv 0$. Die Integrale fallen weg, übrig bleiben die Randterme:

$$\int_a^b s'' h'' dx = \sum_{i=2}^n \left[s'' h' - s^{(3)} h \right]_{x_{i-1}}^{x_i}.$$

die dritte Ableitung eines kubischen Polynoms ist konstant, die vierte null; das gilt stückweise, an den Knoten selbst ist $s^{(3)}$ im Allgemeinen unstetig

- (5) Die Summe teleskopiert, allerdings mit zwei verschiedenen Begründungen. Für den ersten Anteil sind s'' und h' an jedem inneren Knoten stetig, der Beitrag vom rechten Rand des einen Intervalls hebt also den vom linken Rand des nächsten auf:

$$\sum_{i=2}^n \left[s'' h' \right]_{x_{i-1}}^{x_i} = \left[s'' h' \right]_a^b = s''(b) h'(b) - s''(a) h'(a) = 0.$$

Der zweite Anteil verschwindet sogar summandenweise, weil h an jedem Knoten null ist.

$s''(a) = s''(b) = 0$ ist die natürliche Randbedingung; für den zweiten Anteil ist $s^{(3)}(x_i) h(x_i) = 0$ von beiden Seiten, ganz gleich wie $s^{(3)}$ dort springt

- (6) Damit ist der mittlere Term in (13.5.2) null, und es bleibt

$$J(g) = J(s) + \int_a^b (h''(x))^2 dx \geq J(s). \quad (13.5.3)$$

der verbleibende Integrand ist ein Quadrat, also nirgends negativ, und damit ist auch sein Integral nicht negativ

- (7) Für den Gleichheitsfall lesen wir (13.5.3) rückwärts. Aus $J(g) = J(s)$ folgt $\int_a^b (h'')^2 dx = 0$, und weil $(h'')^2$ stetig und nirgends negativ ist, folgt daraus $h'' \equiv 0$ auf $[a, b]$. Also ist h affin, $h(x) = \alpha x + \beta$. Wegen $n \geq 2$ hat h mindestens zwei verschiedene Nullstellen $x_1 \neq x_2$, und eine affine Funktion mit zwei Nullstellen ist die Nullfunktion. Also $h \equiv 0$ und $g = s$.

hätte $(h'')^2$ an einer Stelle einen positiven Wert, wäre es aus Stetigkeitsgründen auf einer ganzen Umgebung positiv und das Integral echt größer als null; aus $\alpha x_1 + \beta = \alpha x_2 + \beta = 0$ mit $x_1 \neq x_2$ folgt $\alpha = 0$ und dann $\beta = 0$

□

Bemerkung 13.5.5 (Was die Randterme wirklich brauchen)

Die Rechnung im Beweis ist empfindlicher, als sie aussieht, und es lohnt sich, die drei Zutaten getrennt zu benennen.

1. Das Teleskopieren von $s'' h'$ braucht die Stetigkeit von s'' und von h' über die Knoten hinweg. Beides ist vorhanden: s ist als kubischer Spline in $\mathcal{C}^2$, und h ist es nach Voraussetzung an g . Wäre g nur stückweise glatt mit Knick, brähe der Schritt zusammen.
2. Die dritte Ableitung $s^{(3)}$ darf an den Knoten dagegen springen, und sie tut es im Allgemeinen auch. Gerettet wird der Term allein durch $h(x_i) = 0$: Der Sprung wird mit null multipliziert. Genau deshalb müssen die Knoten des Splines die Datenpunkte sein.
3. Für den letzten Rest $[s'' h']_a^b$ hilft kein Verschwinden von h mehr, denn dort steht h' . Die natürliche Bedingung $s''(a) = s''(b) = 0$ macht ihn null, und das ist der bequemste Weg. Es ist nicht der einzige: Verlangen wir stattdessen von allen Kandidaten vorgeschriebene Randsteigungen, so ist $h'(a) = h'(b) = 0$, und der Term verschwindet ebenso. Das führt auf den eingespannten Spline (clamped spline), für den derselbe Satz in seiner

Klasse gilt.

Eine Voraussetzung im Satzkopf ist dabei leicht zu übersehen: $a = x_1$ und $x_n = b$, die äußersten Knoten sind also die Intervallenden. Liegen die Daten echt im Inneren, setzt man den Spline außerhalb linear fort; dort ist $s'' = 0$, $J(s)$ bleibt unverändert, und die Ungleichung überträgt sich auf das größere Intervall.

Korollar 13.5.6 (Der natürliche kubische Spline ist der einzige Minimierer)

Unter den Voraussetzungen von Satz 13.5.4 ist s der *eindeutige* Minimierer von J in der Menge

$$\{g \in \mathcal{C}^2[a, b] : g(x_i) = y_i \text{ für } i = 1, \dots, n\}.$$

Das ist genau der letzte Beweisschritt: Jeder andere Interpolant mit demselben Krümmungswert ist s selbst.

Satz 13.5.4 setzt voraus, dass ein natürlicher kubischer Spline durch die Punkte überhaupt existiert. Er tut es, und zwar genau einer: Die Abzählung von Unbekannten und Bedingungen geht auf, und das zugehörige tridiagonale Gleichungssystem für die zweiten Ableitungen ist strikt diagonaldominant und damit regulär.

Vertiefung: Die Abzählung und das tridiagonale System dahinter

Bemerkung 13.5.7 (Existenz: die Abzählung geht auf)

Bei n Knoten gibt es $n - 1$ Teilintervalle mit je 4 Koeffizienten, zusammen $4(n - 1)$ Unbekannte. Dem stehen $2(n - 1)$ Interpolationsbedingungen gegenüber (jedes Stück trifft beide Endwerte), $2(n - 2)$ Glattheitsbedingungen an den inneren Knoten (erste und zweite Ableitung) und die 2 natürlichen Randbedingungen. Summe: $2(n - 1) + 2(n - 2) + 2 = 4(n - 1)$, also genauso viele Gleichungen wie Unbekannte. Für $n = 5$ etwa sind es 16 auf beiden Seiten.

Abzählen allein ist noch kein Existenzbeweis, denn gleich viele Gleichungen wie Unbekannte heißt nicht, dass die Matrix regulär ist. Das sieht man aber, wenn man die $4(n - 1)$ Koeffizienten zugunsten der Momente $M_i := s''(x_i)$ eliminiert. Mit den Abständen $h_i := x_{i+1} - x_i$ bleibt für jeden inneren Knoten $i = 2, \dots, n - 1$ genau eine Gleichung übrig,

$$h_{i-1} M_{i-1} + 2(h_{i-1} + h_i) M_i + h_i M_{i+1} = 6 \left(\frac{y_{i+1} - y_i}{h_i} - \frac{y_i - y_{i-1}}{h_{i-1}} \right),$$

dazu $M_1 = M_n = 0$ aus der natürlichen Randbedingung. Das ist ein tridiagonales System in $n - 2$ Unbekannten, und sein Diagonaleintrag ist mit $2(h_{i-1} + h_i)$ doppelt so groß wie die Summe der beiden Nebeneinträge. Damit ist es strikt diagonaldominant, also regulär: Der natürliche kubische Spline existiert und ist eindeutig. Zugleich ist es ein Bandsystem, wie es Abschnitt 5.3 als ausnutzbare Sonderstruktur aufführt: Die Elimination verlässt das Band nie, der Aufwand wächst also nur linear in n statt kubisch. Genau deshalb bleiben Splines auch bei vielen Punkten billig. Für die drei Punkte aus Beispiel 13.5.8 bleibt davon die einzelne Gleichung $4M_2 = -12$ mit $M_2 = -3$, und das ist genau das $s''(1) = -3$, das wir dort ausrechnen. Abschnitt 13.4 zählt denselben Ansatz für vier Punkte durch.

Beispiel: Spline gegen Parabel

Der Satz verspricht, dass der natürliche kubische Spline gegen jeden anderen $\mathcal{C}^2$ -Interpolanten gewinnt. An drei Punkten lässt sich das komplett von Hand nachrechnen.

Beispiel 13.5.8 (Drei Punkte, zwei Interpolanten)

Gegeben: die Punkte $(0, 0)$, $(1, 1)$, $(2, 0)$, also $a = 0$, $b = 2$ und Knoten $x_1 = 0$, $x_2 = 1$, $x_3 = 2$. Der natürliche kubische Spline ist

$$s(x) = \begin{cases} 1,5x - 0,5x^3 & x \in [0, 1], \\ 1,5(2-x) - 0,5(2-x)^3 & x \in [1, 2]. \end{cases}$$

Die drei Eigenschaften prüfen wir der Reihe nach.

Interpolation: $s(0) = 0$, $s(1) = 1,5 - 0,5 = 1$ und $s(2) = 0$, jeweils aus dem passenden Ast.

Glattheit im inneren Knoten: Aus $s'(x) = 1,5 - 1,5x^2$ links und $s'(x) = -1,5 + 1,5(2-x)^2$ rechts wird an der Stelle $x = 1$ beidseitig 0; aus $s''(x) = -3x$ links und $s''(x) = -3(2-x)$ rechts beidseitig -3 . Der Spline ist also $\mathcal{C}^2$.

Natürlichkeit: $s''(0) = 0$ und $s''(2) = -3 \cdot 0 = 0$.

Damit rechnen wir das Krümmungsintegral aus, getrennt nach Teilintervall:

$$\begin{aligned} J(s) &= \int_0^1 (-3x)^2 dx + \int_1^2 (-3(2-x))^2 dx \\ &= \int_0^1 9x^2 dx + \int_0^1 9u^2 du \\ &= 9 \cdot \left[\frac{x^3}{3} \right]_0^1 + 9 \cdot \left[\frac{u^3}{3} \right]_0^1 = 3 + 3 = 6. \end{aligned}$$

Im zweiten Integral haben wir $u = 2 - x$ substituiert, also $du = -dx$ und die Grenzen $x = 1, 2$ zu $u = 1, 0$ gedreht; das Minuszeichen der Substitution und die gedrehten Grenzen heben sich auf.

Zum Vergleich die Parabel $p(x) = -x^2 + 2x$. Auch sie interpoliert, denn $p(0) = 0$, $p(1) = 1$ und $p(2) = 0$. Ihre Ableitungen sind $p'(x) = -2x + 2$ und $p''(x) = -2$, also

$$J(p) = \int_0^2 (-2)^2 dx = 4 \cdot 2 = 8.$$

Interpolant	$\int_0^2 f''(x) ^2 dx$
Spline s	6
Parabel p	8

Der Spline gewinnt, wie Satz 13.5.4 es verlangt. Der Überschuss lässt sich mit (13.5.3) sogar benennen: Mit $h = p - s$ ist $h''(x) = -2 + 3x$ auf $[0, 1]$ und

$$\int_0^1 (3x - 2)^2 dx = \left[\frac{(3x-2)^3}{9} \right]_0^1 = \frac{1}{9} - \left(-\frac{8}{9} \right) = 1,$$

auf $[1, 2]$ aus Symmetriegründen ebenso. Zusammen ist $\int_0^2 (h'')^2 dx = 2$, und tatsächlich ist $6 + 2 = 8$.

Interaktiv: Eine ganze Schar von Interpolanten

Spline und Parabel sind nur zwei Punkte in einer unendlichen Menge. Weil $h = p - s$ an allen drei Stützstellen verschwindet, interpoliert auch jede Mischung $g_t = s + t h$ dieselben Daten, für jedes $t \in \mathbb{R}$. Mit (13.5.3) ist ihr Krümmungsintegral

$$J(g_t) = J(s) + t^2 \int_0^2 (h'')^2 dx = 6 + 2t^2,$$

eine nach oben geöffnete Parabel in t mit Minimum bei $t = 0$. Der Regler fährt diese Schar ab: $t = 0$ ist der Spline, $t = 1$ die Parabel des Beispiels. Die zweite Ansicht zeigt, woher der Unterschied kommt, nämlich aus zwei Geradenstücken von s'' gegen die Konstante -2 von p'' . Die Farben tragen dieselben Rollen wie im Text: blau die Daten, orange die Knoten, grün der Spline als Lösung des Minimierungsproblems, rot die Abweichung h und mit ihr jeder Konkurrent g_t .

Welche Mischung hat nach unserer Vermutung die kleinste Krümmungsenergie?

Schätzfrage. Bei welchem t wird die Krümmungsenergie minimal?

Lösung: $0 \leq t$

Bemerkenswert an Satz 13.5.4 ist, was er *nicht* sagt. Über die Approximationsgüte fällt kein Wort: Alle Konkurrenten treffen die Daten exakt, sie unterscheiden sich nur in der Zappeligkeit dazwischen. Wie weit ein Spline von einer zugrunde liegenden Funktion abweicht, ist eine ganz andere Frage. Sie ist der Gegenstand von Abschnitt 13.6.

Selbsttest

1. Jeder kubische Spline, der die Punkte interpoliert, minimiert das Krümmungsfunktional J .

Antwort: Falsch

Nur der *natürliche*. Ohne $s''(a) = s''(b) = 0$ bleibt im Beweis der Randterm $[s'' h']_a^b$ stehen, und der kann jedes Vorzeichen haben. Kubische Spline-Interpolanten zu denselben Daten gibt es unendlich viele, weil zwei Randbedingungen frei sind; nur eine Wahl davon löst das Minimierungsproblem. Die Parabel aus Beispiel 13.5.8 ist selbst so ein Fall: Sie ist stückweise ein Polynom vom Grad höchstens 3 und beliebig oft differenzierbar, also ein kubischer Spline-Interpolant, aber wegen $p''(0) = -2 \neq 0$ nicht der natürliche, und ihr J ist um 2 größer.

2. Beim Teleskopieren der Randterme darf $s^{(3)}$ an den Knoten springen.

Antwort: Wahr

Der Term $s^{(3)} h$ wird an jedem Knoten mit $h(x_i) = 0$ multipliziert und verschwindet dadurch von beiden Seiten, egal wie der Sprung aussieht. Für den anderen Term $s'' h'$ gilt das nicht, dort braucht es die Stetigkeit von s'' und h' und am Rand die natürlichen Bedingungen (Bemerkung 13.5.5).

3. Der Beweis setzt voraus, dass auch die Vergleichsfunktion $g''(a) = g''(b) = 0$ erfüllt.

Antwort: Falsch

Diese Forderung wird nirgends gebraucht. Sie wäre sogar schädlich: Sie schlosse die meisten $\mathcal{C}^2$ -Interpolanten aus, und gerade gegen die soll der Spline gewinnen. Verwendet werden nur $g \in \mathcal{C}^2$ und $g(x_i) = y_i$.

4. Der Überschuss $J(p) - J(s) = 2$ im Beispiel ist genau $\int_0^2 (h'')^2 dx$ mit $h = p - s$.

Antwort: Wahr

Das ist Gleichung (13.5.3), und im Beispiel rechnet es sich aus: Auf $[0, 1]$ ist $h''(x) = -2 + 3x$ mit $\int_0^1 (3x - 2)^2 dx = 1$, auf $[1, 2]$ aus Symmetriegründen ebenso, zusammen $2 = 8 - 6$. Der Kreuzterm aus (13.5.2) fällt weg, sonst stimmte die Rechnung nicht.

5. Aus $J(g) = J(s)$ folgt stets $g = s$, auch bei nur einer Stützstelle.

Antwort: Falsch

Bei $n = 1$ ist jede affine Funktion durch den einen Punkt ein Interpolant mit $J = 0$, und Minimierer gibt es unendlich viele. Der Schluss von $h'' \equiv 0$ auf $h \equiv 0$ braucht zwei verschiedene Nullstellen von h , also $n \geq 2$. Die Kurzfassung „Gleichheit nur für $h \equiv 0$ “ verschweigt das.

Vertiefung: Heath führt natürliche kubische Splines in Kapitel 7 (Interpolation), Abschnitt 7.4.2 ein und rechnet dort denselben Drei-Punkte-Fall mit acht Bedingungen vor. Die Minimaleigenschaft selbst steht als Satz 7.57 bei Deuflhard und Hohmann, Numerische Mathematik 1, Abschnitt 7.4; dort findet sich auch die Rechtfertigung, warum man statt der geometrischen Krümmung deren Näherung f'' integriert.

13.6 Approximationsfehler

Über Splines wissen wir bisher zweierlei. Sie setzen sich aus Polynomstücken zusammen und sind darum billig zu rechnen (Abschnitt 13.4), und unter allen Interpolanten mit stetiger zweiter Ableitung hat der natürliche kubische Spline die kleinste Krümmung (Abschnitt 13.5). Dazu kam die Zusage, dass ein hinreichend feines Knotengitter jede stetige Funktion beliebig genau einfängt. Diese Zusage nennt kein Tempo. Sie sagt weder, wie viele Knoten wir brauchen, noch was wir gewinnen, wenn wir die Zahl der Teilintervalle verdoppeln. Genau das holen wir jetzt nach.

Gitterweite und Fehlerschranke

Zuerst brauchen wir ein Maß für die Feinheit des Gitters.

Definition 13.6.1 (Partition und Gitterweite)

Eine *Partition* des Intervalls $[a, b]$ ist eine geordnete Folge von Stellen

$$a = x_0 < x_1 < \dots < x_n = b.$$

Ihre *Gitterweite* (mesh width) ist die Länge des größten Teilintervalls,

$$h := \max_{i=1, \dots, n} |x_i - x_{i-1}|.$$

Zwei Feinheiten stecken schon in dieser Definition. Das Gitter ist geordnet und schöpft $[a, b]$ aus, die beiden Randpunkte sind also selbst Knoten. Und h ist ein Maximum, kein Mittelwert: Ein einziges breites Teilintervall bestimmt die Gitterweite, wie fein die übrigen auch liegen mögen.

Satz 13.6.2 (Approximationsfehler kubischer Splines)

Sei $a = x_0 < \dots < x_n = b$ eine Partition mit Gitterweite h , und sei $f \in \mathcal{C}^4[a, b]$. Dann gibt es einen kubischen Spline s mit Knoten $x_0, \dots, x_n$, der f an allen Knoten interpoliert,

$$s(x_i) = f(x_i) \quad \text{für } i = 0, \dots, n,$$

und dessen Fehler die Schranke

$$\max_{x \in [a, b]} |f(x) - s(x)| \leq C \cdot h^4 \cdot \max_{x \in [a, b]} |f^{(4)}(x)| \quad (13.6.1)$$

erfüllt, mit einer kleinen Konstanten C . Für kubische Splines mit passenden Randbedingungen gilt sie mit $C = \frac{5}{384}$.

Die Voraussetzung $f \in \mathcal{C}^4[a, b]$ verlangt vier stetige Ableitungen, also eine glatte Funktion im üblichen Sinn. Die rechte Seite der Schranke hat drei Faktoren, und jeder erzählt etwas anderes.

Bemerkung 13.6.3 (Wie wir die Schranke lesen)

1. Die Gitterweite steht in der *vierten* Potenz. Halbieren wir h , so fällt die Schranke auf $2^{-4} = \frac{1}{16}$ ihres Werts. Je enger das Gitter, desto besser die Approximation, und zwar sehr viel besser.
2. Der zweite Faktor gehört der Funktion, nicht dem Verfahren. Je kleiner $|f^{(4)}|$, desto kleiner die Schranke; je glatter also f ist, desto weniger Knoten kostet dieselbe Genauigkeit. Für ein Polynom vom Grad höchstens drei steht rechts sogar null.
3. Beide Faktoren sind Maxima über das *ganze* Intervall. Rechnet man lokal, so steht in der Abschätzung für ein Teilintervall dessen eigene Länge neben dem dortigen Maximum von $|f^{(4)}|$. Das ist der Grund, das Gitter dort zu verdichten, wo f stark gekrümmt ist und schwingt, und es dort auszudünnen, wo f fast gerade verläuft. Für den einfachsten Fall führen wir diese lokale Rechnung gleich in Satz 13.6.5 vor.

Wie ernst Punkt 3 gemeint ist, zeigt eine kleine Rechnung mit $f(x) = \sin(2\pi x)$ auf $[0, 1]$. Legen wir zehn Knoten so, dass die linke Hälfte des Intervalls ein einziges Teilstück bleibt und die rechte Hälfte in acht gleiche Stücke zerfällt, dann ist $h = 0,5$, und der maximale Fehler des natürlichen kubischen Splines beträgt 0,479. Ein gleichmäßiges Gitter mit *neun* Knoten kommt auf 0,00107 und ist damit rund 450-mal genauer. Ein Knoten mehr nützt nichts, solange eine Lücke bleibt.

Bemerkung 13.6.4 (Welcher Spline die Konstante trägt)

Satz 13.6.2 behauptet die Existenz eines kubischen Splines, nicht die Güte eines beliebigen Randabschlusses. Das ist keine Spitzfindigkeit. Die scharfe Konstante $\frac{5}{384}$ gilt etwa für den *vollständigen* kubischen Spline, der $s'(a) = f'(a)$ und $s'(b) = f'(b)$ erfüllt, sowie für die passende Randbedingung zweiter Ordnung $s''(a) = f''(a)$ und $s''(b) = f''(b)$. Der *natürliche* Spline verlangt stattdessen $s''(a) = s''(b) = 0$; pauschal steht ihm die Konstante deshalb nicht zu. Passen diese Randwerte nicht zu $f''(a)$ und $f''(b)$, so ist der natürliche Spline am Rand schlechter, und sein maximaler Fehler fällt global nur wie h^2 .

Zwei eigene Rechnungen zeigen das. Für $f(x) = e^x$ auf $[0, 1]$ ist $f''(0) = 1 \neq 0$, und der maximale Fehler des natürlichen Splines sinkt beim Halbieren von h nur auf ein Viertel: $2,08 \cdot 10^{-3}$, $5,21 \cdot 10^{-4}$, $1,30 \cdot 10^{-4}$ und $3,26 \cdot 10^{-5}$ für $n = 8, 16, 32, 64$. Die Fehlerspitze rückt dabei immer näher an den rechten Rand ($x \approx 0,95$, dann 0,976, 0,988 und 0,994), der Rest des Intervalls ist längst genauer. Noch schärfer ist $f(x) = x^3$: Dort steht rechts in (13.6.1) eine Null, denn $f^{(4)} \equiv 0$, und tatsächlich ist f selbst ein kubischer Spline und interpoliert sich fehlerfrei. Der natürliche Spline auf vier gleich langen Teilintervallen weicht dagegen um 0,018 ab, weil er $s''(1) = 0$ erzwingt, während $f''(1) = 6$ ist.

Für das Beispiel unten schreiben wir deshalb gleich die Randableitungen vor, $s'(0) = f'(0)$ und $s'(1) = f'(1)$; die Konstante $\frac{5}{384}$ ist dort also ehrlich verdient. Wie viel daran hängt, zeigt der Vergleich mit dem natürlichen Spline derselben Funktion. Auf groben Gittern ist der Unterschied winzig, weil die Funktion dort an beiden Rändern fast flach ausläuft: bei fünf Knoten messen wir 0,341 statt 0,324. Sobald das Gitter fein wird, schlägt der Randfehler durch. Bei 33 Knoten sitzt der größte Fehler des natürlichen Splines nicht mehr an der Spitze

im Inneren, sondern bei $x \approx 0,012$ am linken Rand, und bei 65 Knoten ist er mit $1,84 \cdot 10^{-5}$ sogar *größer* als die Schranke $1,49 \cdot 10^{-5}$ aus (13.6.1). Ein Widerspruch wäre das nur, wenn der Satz die Schranke für jeden Randabschluss behauptete.

Woher der Exponent kommt

Warum vier? Der Exponent hängt am Grad der Polynomstücke, und der Zusammenhang lässt sich am einfachsten Fall vollständig durchrechnen. Statt kubischer Stücke nehmen wir dafür gerade Stücke, also den Streckenzug durch die Punkte $(x_i, f(x_i))$.

Satz 13.6.5 (Fehler der stückweise linearen Interpolation)

Sei $f \in \mathcal{C}^2[a, b]$ und sei p der Streckenzug durch die Punkte $(x_i, f(x_i))$ zu einer Partition mit Gitterweite h . Dann gilt

$$\max_{x \in [a, b]} |f(x) - p(x)| \leq \frac{h^2}{8} \max_{x \in [a, b]} |f''(x)|. \quad (13.6.2)$$

Vertiefung: Wie die Schranke für Geradenstücke entsteht

Der Beweis arbeitet auf einem einzelnen Teilintervall, schiebt eine Hilfsfunktion mit drei Nullstellen dazwischen und wendet zweimal den Satz von Rolle an. Die Bemerkung darunter zieht daraus das Muster, auf das es hier ankommt.

Beweis.

- (1) Wir arbeiten auf einem einzelnen Teilintervall $[x_{i-1}, x_i]$ der Länge h_i und halten darin eine Stelle $\tilde{x}$ mit $x_{i-1} < \tilde{x} < x_i$ fest. Dort ist p die Gerade durch die beiden Endpunkte.

An den Knoten selbst ist die Differenz null, dort ist nichts zu zeigen; jede Stelle aus $[a, b]$ liegt in einem solchen Teilintervall

- (2) Wir setzen $w(t) := (t - x_{i-1})(t - x_i)$ und wählen $\lambda \in \mathbb{R}$ so, dass die Hilfsfunktion

$$g(t) := f(t) - p(t) - \lambda w(t)$$

auch in $\tilde{x}$ verschwindet.

Wegen $x_{i-1} < \tilde{x} < x_i$ ist $w(\tilde{x}) \neq 0$, also ist $\lambda = (f(\tilde{x}) - p(\tilde{x}))/w(\tilde{x})$ die einzige mögliche Wahl

- (3) Damit hat g drei Nullstellen in $[x_{i-1}, x_i]$: die beiden Knoten und $\tilde{x}$. Zwischen je zwei benachbarten Nullstellen liegt eine Nullstelle von g' , also hat g' zwei Nullstellen, und dasselbe Argument liefert ein ξ im Inneren mit $g''(\xi) = 0$.

Satz von Rolle, der Spezialfall des Mittelwertsatzes mit gleichen Funktionswerten an beiden Enden; g ist zweimal stetig differenzierbar, weil f es ist und p, w Polynome sind

- (4) Auf dem Teilintervall ist p linear, also $p'' \equiv 0$, und $w'' \equiv 2$. Aus $g''(\xi) = 0$ folgt daher

$$f''(\xi) - 2\lambda = 0, \quad \text{also} \quad \lambda = \frac{1}{2} f''(\xi).$$

- (5) Setzen wir das in $g(\tilde{x}) = 0$ ein, so steht die punktweise Darstellung des Fehlers da:

$$f(\tilde{x}) - p(\tilde{x}) = \frac{1}{2} f''(\xi) (\tilde{x} - x_{i-1})(\tilde{x} - x_i).$$

Der Betrag des Produkts ist $(\tilde{x} - x_{i-1})(x_i - \tilde{x})$; zwei nichtnegative Zahlen mit fester Summe h_i haben ihr größtes Produkt, wenn beide gleich $h_i/2$ sind, also ist er höchstens $h_i^2/4$

- (6) Auf dem Teilintervall gilt somit $|f - p| \leq \frac{h_i^2}{8} \max |f''|$, wobei das Maximum nur über dieses Teilintervall läuft. Wegen $h_i \leq h$ und weil das Maximum über $[a, b]$ mindestens so groß ist, gilt (13.6.2) auf jedem Teilintervall und damit auf ganz $[a, b]$.

□

Bemerkung 13.6.6 (Das Muster hinter dem Exponenten)

Der Beweis zeigt, woher der Exponent kommt. Das Hilfsprodukt w hat einen Faktor je Interpolationsbedingung im Teilintervall, jeder Faktor ist von der Größenordnung h , und genauso oft wird differenziert. Bei Geradenstücken sind es zwei Bedingungen, zwei Ableitungen und h^2 . Das Muster setzt sich fort: Bei Polynomstücken vom Grad q steht am Ende h^{q+1} neben der $(q+1)$ -ten Ableitung, für kubische Stücke also h^4 neben $f^{(4)}$. Wer den Restterm im Satz von Taylor kennt, erkennt die Bauart wieder: Dort steht eine höhere Ableitung an einer Zwischenstelle mal eine Potenz des Abstands, hier $\frac{1}{2}f''(\xi)$ mal ein Produkt von Abständen. Für Splines ist der Beweis deutlich länger als der obige, weil die Anschlussbedingungen die Teilintervalle koppeln: Der Spline auf einem Stück hängt über das Gleichungssystem aus Abschnitt 13.4 an *allen* Daten, und genau davon lebt ja seine Glattheit. Die Buchführung über die Ordnung bleibt dieselbe.

Bemerkenswert ist dabei, was *nicht* gebraucht wird. Kein Schritt verlangt einen wachsenden Polynomgrad. Wir verfeinern das Gitter und lassen den Grad bei drei, und deshalb passiert hier nichts von dem, was die Polynominterpolation hohen Grades ruiniert (Abschnitt 13.3).

Ein numerisches Beispiel

Beispiel 13.6.7 (Ein Buckel auf dem Einheitsintervall)

Wir nehmen einen schmalen Buckel,

$$f(x) = \exp\left(-\alpha\left(x - \frac{2}{5}\right)^2\right) \quad \text{mit } \alpha = 40, \quad x \in [0, 1].$$

Mit $t := x - \frac{2}{5}$ ist

$$f^{(4)}(x) = (12\alpha^2 - 48\alpha^3 t^2 + 16\alpha^4 t^4) f(x), \quad M_4 := \max_{x \in [0, 1]} |f^{(4)}(x)| = 12\alpha^2 = 19200,$$

und das Maximum sitzt in der Spitze $t = 0$. Dort, auf einem schmalen Streifen, steckt fast der ganze Fehler; an beiden Rändern ist f praktisch null.

Auf gleichmäßigen Gittern mit $n+1$ Knoten ist $h = 1/n$, und (13.6.1) liefert die Schranke $\frac{5}{384}h^4 M_4$. Die vierte Spalte ist unsere eigene Rechnung: Wir bestimmen den vollständigen kubischen Spline über das tridiagonale System seiner zweiten Ableitungen an den Knoten und messen $\max |f - s|$ auf einem feinen Raster.

Knoten $n+1$	Gitterweite h	Schranke $Ch^4 M_4$	gemessener Fehler	Faktor
5	0,25	0,977	0,324	–
9	0,125	0,0610	0,0239	13,6
17	0,0625	$3,81 \cdot 10^{-3}$	$1,11 \cdot 10^{-3}$	21,6
33	0,03125	$2,38 \cdot 10^{-4}$	$5,16 \cdot 10^{-5}$	21,4
65	0,015625	$1,49 \cdot 10^{-5}$	$3,05 \cdot 10^{-6}$	16,9
129	0,0078125	$9,31 \cdot 10^{-7}$	$1,87 \cdot 10^{-7}$	16,3

Zweierlei ist hier zu sehen. Erstens ist der Fehler auf groben Gittern mit bloßem Auge zu erkennen: Bei fünf Knoten beträgt er 0,324, also rund ein Drittel der Buckelhöhe 1. Der Spline drückt die Spitze auf 0,71 herunter und schwingt rechts davon bis auf $-0,056$ unter die Nulllinie.

Zweitens stellt sich der Faktor 16 nicht sofort ein. Die Schranke fällt bei jeder Halbierung exakt auf ein Sechzehntel, denn h^4 tut genau das; der gemessene Fehler springt dagegen erst auf 13,6, dann auf 21,6 und 21,4 und beruhigt sich ab 65 Knoten bei 16,9 und 16,3. Der Grund steht in Bemerkung 13.6.3: M_4 ist ein Maximum über das *ganze* Intervall, tatsächlich aber lebt $|f^{(4)}|$ fast nur auf der Breite $1/\sqrt{\alpha} \approx 0,16$ um die Spitze. Solange h von dieser Größenordnung ist, hängt der gemessene Fehler stark davon ab, wie die Knoten gerade zur Spitze liegen, und die Ordnung 4 ist noch nicht zu sehen. $O(h^4)$ ist eine Aussage für kleine h , und „klein“ heißt hier: klein gegen die Breite, auf der sich $f^{(4)}$ ändert.

Bemerkung 13.6.8 (Schranke und Messung sind zweierlei)

Die beiden Fehlerspalten in der Tabelle oben sehen ähnlich aus, sagen aber Verschiedenes. Die Schranke gilt für *jedes* $f \in \mathcal{C}^4[a, b]$ und für jede Partition; der gemessene Fehler gehört zu genau einer Funktion auf genau einem Gitter. Aus dem Faktor 16 wird deshalb keine Gleichung, sondern eine Aussage über die Größenordnung, die wir in Landau-Notation als $\|f - s\|_\infty = O(h^4)$ schreiben. Wir sagen dazu, der Fehler sei *von vierter Ordnung* in h ; das ist dieselbe Sprechweise wie bei der Konvergenzordnung iterativer Verfahren, nur läuft hier kein Iterationszähler, sondern die Gitterweite gegen null. Unsere Messung liegt in Beispiel 13.6.7 durchweg unter der Schranke – das Verhältnis von gemessenem Fehler zu Schranke beträgt 0,33, 0,39, 0,29, 0,22 und dann zweimal 0,20 –, aber die Faktoren erreichen die 16 erst, wenn das Gitter fein genug ist. Wer eine Rechnung „mit Faktor genau 16“ erwartet, wird enttäuscht, ohne dass etwas falsch wäre.

Interaktiv: Vom groben zum feinen Gitter

Das Widget rechnet den Buckel aus Beispiel 13.6.7 noch einmal nach und beginnt schon bei drei Knoten. Im oberen Bild ist zu sehen, wie der Spline die Spitze zunächst gar nicht findet: Mit drei Knoten bleibt er bei 0,67 hängen, mit fünf kommt er auf 0,71 und schwingt rechts der Spitze unter null. Erst ab neun Knoten liegen Funktion und Interpolant nahezu übereinander. Im unteren Bild verschwindet der Fehler an jedem Knoten und bildet dazwischen Bäuche; sie sammeln sich um die Spitze, wo $|f^{(4)}|$ groß ist. Darunter legt eine halblogarithmische Tafel Messung und Schranke übereinander, und die Tabelle listet dieselben Zahlen. Die Schranke fällt bei jeder Halbierung von h exakt auf ein Sechzehntel; der gemessene Faktor schwankt auf den groben Gittern kräftig und nähert sich der asymptotischen 16 erst am rechten Ende. Um welchen Faktor erwarten wir beim Halbieren von h einen kleineren Fehler?

Schätzfrage. Schätzen wir den asymptotischen Faktor beim Halbieren von h .

Lösung: 16

Was Splines stark macht

Damit ist der theoretische Werkzeugkasten für Splines beisammen, und er ist bemerkenswert gut bestückt.

Bemerkung 13.6.9 (Vier Stärken auf einen Blick)

1. *Effiziente Berechnung.* In der B-Spline-Basis führt die Interpolation auf ein Gleichungssystem mit Bandstruktur (Abschnitt 13.4), und Bandstruktur ist genau die Sorte Struktur, die den Aufwand drückt (Bemerkung 5.3.10 in Abschnitt 5.3). Der kubische Spline-Interpolant dieses Abschnitts ist das kleinste Beispiel dafür: Seine zweiten Ableitungen an den Knoten lösen ein *tridiagonales* System — bei natürlichem wie bei eingespanntem Randabschluss —, und weil die Elimination dessen Band erhält, kostet es $O(n)$ statt $O(n^3)$ Operationen. Genau so rechnet das Widget oben.
2. *Stabilität und Lokalität.* B-Splines haben kompakten Träger, jede Basisfunktion lebt also nur über wenigen Teilintervallen. Eine Änderung an einem Datenpunkt wirkt lokal, und die Basis ist weit besser konditioniert als die Monombasis (Abschnitt 13.3, Kondition in Abschnitt 4.2).
3. *Minimale Krümmung.* Unter allen Interpolanten aus $\mathcal{C}^2$ macht der natürliche kubische Spline $\int_a^b |f''(x)|^2 dx$ am kleinsten (Abschnitt 13.5).
4. *Approximationsfehler $O(h^4)$ bei passenden Randbedingungen.* Das ist Satz 13.6.2: Verdoppeln wir auf einem gleichmäßigen Gitter die Zahl der Teilintervalle, halbiert sich h , und die Schranke fällt um den Faktor 16, ohne dass wir den Polynomgrad anrühren. Für einen natürlichen Spline ohne passende Randkrümmung gilt diese globale Ordnung im Allgemeinen nicht (Bemerkung 13.6.4).

Alle vier Punkte gelten für *Interpolation*, also für Daten ohne Rauschen. Genau diese Voraussetzung fällt im nächsten Abschnitt weg. Sobald die y_i eine Zufallskomponente tragen, ist es nicht mehr wünschenswert, sie exakt zu treffen, und aus dem Interpolationsproblem wird ein Glättungsproblem (Abschnitt 13.7).

Selbsttest

1. Halbieren wir die Gitterweite, so fällt die Schranke aus Satz 13.6.2 auf ein Sechzehntel.

Antwort: Wahr

Die Schranke ist Ch^4M_4 , und nur h ändert sich. Aus $(h/2)^4 = h^4/16$ folgt der Faktor $\frac{1}{16}$ exakt. In Beispiel 13.6.7 sind das die Werte 0,977, 0,0610, $3,81 \cdot 10^{-3}$, $2,38 \cdot 10^{-4}$ und so fort.

2. Auch der tatsächliche Fehler wird bei jeder Halbierung von h genau 16-mal kleiner.

Antwort: Falsch

(13.6.1) ist eine obere Schranke, keine Gleichung. Unsere eigenen Messungen am Buckel aus Beispiel 13.6.7 ergeben der Reihe nach die Faktoren 13,6, 21,6, 21,4, 16,9 und 16,3: Erst wenn das Gitter die Spitze auflöst, pendeln sie sich bei 16 ein, und getroffen wird die Zahl auch dann nicht. Als Aussage über die Größenordnung bleibt die Faustregel „Fehler wird $2^4 = 16$ -mal kleiner“ trotzdem richtig.

3. Weil $f(x) = x^3$ die Bedingung $f^{(4)} \equiv 0$ erfüllt, interpoliert der natürliche kubische Spline diese Funktion fehlerfrei.

Antwort: Falsch

Fehlerfrei ist der Spline aus Satz 13.6.2, und das ist hier f selbst: Ein kubisches Polynom ist ein kubischer Spline. Der natürliche Spline erzwingt zusätzlich $s''(a) = s''(b) = 0$, während $f''(1) = 6$ ist. Auf vier gleich langen Teilintervallen von $[0, 1]$ weicht er um 0,018 ab, obwohl die rechte Seite von (13.6.1) null ist. Bemerkung 13.6.4 sagt, warum das kein Widerspruch ist.

4. Ein einziges breites Teilintervall verdirbt die Schranke, auch wenn alle übrigen Knoten dicht liegen.

Antwort: Wahr

h ist das Maximum der Teilintervall-Längen (Definition 13.6.1), und die Schranke wächst mit seiner vierten Potenz. Unsere Rechnung zu $\sin(2\pi x)$ zeigt es auch am gemessenen Fehler: Zehn Knoten mit einer Lücke der Breite 0,5 liefern 0,479, neun gleichmäßig verteilte Knoten dagegen 0,00107.

5. Satz 13.6.2 gibt eine Fehlerschranke für jede stetige Funktion f auf $[a, b]$.

Antwort: Falsch

Der Satz verlangt $f \in \mathcal{C}^4[a, b]$, denn auf der rechten Seite steht $\max |f^{(4)}|$. Dass sich stetige Funktionen durch Splines beliebig genau approximieren lassen, ist eine andere Aussage (Abschnitt 13.4); sie nennt keine Geschwindigkeit. Wer weniger Glattheit mitbringt, bekommt eine schwächere Ordnung: Satz 13.6.5 verlangt nur $f \in \mathcal{C}^2$ und liefert dafür auch nur $O(h^2)$.

6. Bei stückweise linearer Interpolation bringt eine Halbierung der Gitterweite nur den Faktor 4.

Antwort: Wahr

Satz 13.6.5 gibt die Schranke $h^2/8 \cdot \max |f''|$, und h^2 viertelt sich beim Halbieren. Für $\sin(2\pi x)$ messen wir die Fehler 0,211, 0,0704, 0,0188 und 0,00479 bei $n = 4, 8, 16, 32$, also die Faktoren 2,99, 3,73 und 3,93. Der Gewinn pro Verfeinerung ist damit deutlich kleiner als beim kubischen Spline, und das ist einer der Gründe, warum kubisch die übliche Wahl ist.

Vertiefung: Heath entwickelt die stückweise Polynominterpolation und die kubischen Splines in Kapitel 7. Die scharfe Konstante $5/384$ für den vollständigen kubischen Spline geht auf C. A. Hall und W. W. Meyer, Optimal error bounds for cubic spline interpolation, Journal of Approximation Theory 16 (1976), 105–122, zurück.

13.7 Glättung und Regression

Bisher waren die Daten exakt. Ein Interpolant läuft durch jeden vorgegebenen Punkt, und Satz 13.6.2 sagt, wie weit er dazwischen danebenliegen kann. Messwerte sind aber selten exakt. Ein Sensor rauscht, eine Stichprobe streut, eine Befragung trifft nur zufällig ausgewählte Personen. Wer solche Werte interpoliert, zwingt die Kurve durch jeden Messfehler hindurch.

Abschnitt 13.1 hat die beiden Aufgaben schon nebeneinandergestellt: Interpolation trifft die Daten exakt, Glättung nur ungefähr. In diesem Abschnitt rechnen wir die zweite Aufgabe aus, und sie stellt sich als altbekanntes Problem heraus.

Das Modell

Definition 13.7.1 (Regressionsmodell mit additivem Fehler)

An Stellen $x_1, \dots, x_n \in [a, b]$ beobachten wir

$$y_i = f(x_i) + \varepsilon_i, \quad i = 1, \dots, n, \quad (13.7.1)$$

mit einer unbekannten Funktion f und Zufallsfehlern $\varepsilon_1, \dots, \varepsilon_n$, die

$$\mathbb{E}[\varepsilon_i | x_i] = 0$$

erfüllen. Gesucht ist eine Schätzung $\hat{f}$ von f .

Der Unterschied zur Interpolation steckt allein im roten Term. Ohne ihn wären die y_i Funktionswerte, und wir wären wieder in Abschnitt 13.6.

Bemerkung 13.7.2 (Was in der Fehlerannahme steckt)

Die Bedingung $\mathbb{E}[\varepsilon_i | x_i] = 0$ sieht harmlos aus, trägt aber die ganze Aufgabe.

1. Sie sagt, dass die Beobachtungen an jeder Stelle *um* $f(x_i)$ herum streuen und nicht systematisch daneben. Anders gesagt ist $f(x)$ der bedingte Erwartungswert $\mathbb{E}[y | x]$, und ε_i ist reine Streuung.
2. Ohne sie wäre f überhaupt nicht bestimmt. Zu jeder Funktion g und jeder Konstanten c liefert das Paar $(g + c, \varepsilon - c)$ dieselben Beobachtungen wie (g, ε) . Erst die Forderung, dass der Fehler im Mittel verschwindet, macht aus (13.7.1) eine Aussage über f .
3. Über Abhängigkeit und Streuung der ε_i sagt die Annahme noch nichts. Für die algebraische KQ-Herleitung dieses Abschnitts brauchen wir beides nicht; Unkorreliertheit und die gemeinsame Varianz σ^2 kommen erst ins Spiel, wenn wir den Fehler der Schätzung zerlegen (Abschnitt 13.8).

Wie messen wir, ob eine Kandidatin $\hat{f}$ zu den Daten passt? Mit derselben Größe wie in Kapitel 7, der Summe der quadrierten Abweichungen:

$$\text{RSS}(\hat{f}) := \sum_{i=1}^n (y_i - \hat{f}(x_i))^2. \quad (13.7.2)$$

Die Abkürzung steht für *residual sum of squares*, die Residuenquadratsumme. Die einzelnen Differenzen $r_i = y_i - \hat{f}(x_i)$ heißen *Residuen*.

Bemerkung 13.7.3 (Die Aufgabe ist so noch entartet)

Suchten wir das Minimum von (13.7.2) unter *allen* Funktionen, wäre die Antwort langweilig und nutzlos: Das Minimum ist null, und angenommen wird es von jedem Interpolanten der Daten. Davon gibt es unendlich viele (Abschnitt 13.1). Alle übernehmen an den Datenstellen jedes ε_i ; die Trainings-RSS kann dabei zwischen einem brauchbaren und einem beliebig schlechten Verlauf zwischen den Punkten nicht unterscheiden.

Die Zielfunktion allein legt also nichts fest. Zwei Auswege gibt es, und beide kommen in diesem Abschnitt vor:

- Wir minimieren nur über einen *kleinen* Funktionenraum. Das ist der Weg der nächsten Seiten.
- Wir minimieren über einen großen Raum, bestrafen aber unruhige Kandidaten. Darauf kommen wir weiter unten zurück.

Der Basisansatz macht daraus ein KQ-Problem

Den ersten Weg kennen wir schon aus der Interpolation (Abschnitt 13.2): Wir legen endlich viele Funktionen fest und lassen nur deren Linearkombinationen zu.

Definition 13.7.4 (Ansatzraum und Designmatrix)

Seien $\phi_1, \dots, \phi_K$ fest gewählte Funktionen auf $[a, b]$, die *Basisfunktionen*, und sei

$$\mathcal{F}_K := \left\{ \sum_{k=1}^K a_k \phi_k : a_1, \dots, a_K \in \mathbb{R} \right\}$$

der von ihnen aufgespannte Ansatzraum. Die *Designmatrix* zu den Stellen $x_1, \dots, x_n$ ist

$$\mathbf{B} := \begin{pmatrix} \phi_1(x_1) & \cdots & \phi_K(x_1) \\ \vdots & \ddots & \vdots \\ \phi_1(x_n) & \cdots & \phi_K(x_n) \end{pmatrix} \in \mathbb{R}^{n \times K}, \quad b_{ik} = \phi_k(x_i).$$

Ihre i -te Zeile sammelt die Werte aller Basisfunktionen an der i -ten Beobachtungsstelle, ihre k -te Spalte die Werte der k -ten Basisfunktion an allen Stellen.

Für Splines sind die ϕ_k die B-Splines zu einem gewählten Knotenvektor $\xi_1 < \dots < \xi_{m-1}$ (Abschnitt 13.4). Wir halten die beiden Rollen auseinander: x_i sind die Stellen, an denen *gemessen* wurde, ξ_j die Stellen, an denen die Polynomstücke *zusammengesetzt* werden. Beide dürfen völlig verschieden liegen, und bei der Glättung tun sie das auch.

Satz 13.7.5 (Glättung ist ein lineares Kleinste-Quadrate-Problem)

Sei $\hat{f} = \sum_{k=1}^K a_k \phi_k \in \mathcal{F}_K$ mit Koeffizientenvektor $\mathbf{a} = (a_1, \dots, a_K)^\top$. Dann gilt

$$\sum_{i=1}^n (y_i - \hat{f}(x_i))^2 = \|\mathbf{y} - \mathbf{B}\mathbf{a}\|_2^2, \quad \mathbf{y} = (y_1, \dots, y_n)^\top, \quad (13.7.3)$$

und $\mathbf{a}$ minimiert die linke Seite genau dann, wenn es die Normalengleichungen

$$\mathbf{B}^\top \mathbf{B} \mathbf{a} = \mathbf{B}^\top \mathbf{y} \quad (13.7.4)$$

erfüllt. Hat $\mathbf{B}$ vollen Spaltenrang K , so ist die Lösung eindeutig, $\hat{\mathbf{a}} = (\mathbf{B}^\top \mathbf{B})^{-1} \mathbf{B}^\top \mathbf{y}$. Andernfalls ist die Lösungsmenge ein affiner Raum, und $\hat{\mathbf{a}} = \mathbf{B}^+ \mathbf{y}$ ist ihr Element kleinster Norm. Der Vektor der angepassten Werte $\mathbf{B}\hat{\mathbf{a}}$ ist in beiden Fällen eindeutig.

Beweis.

- (1) Nach Definition der Designmatrix ist der Wert von $\hat{f}$ an der Stelle x_i gerade die i -te Komponente von $\mathbf{B}\mathbf{a}$:

$$\hat{f}(x_i) = \sum_{k=1}^K a_k \phi_k(x_i) = \sum_{k=1}^K b_{ik} a_k = (\mathbf{B}\mathbf{a})_i.$$

Die i -te Zeile von $\mathbf{B}$ enthält genau die Werte $\phi_1(x_i), \dots, \phi_K(x_i)$, das Skalarprodukt mit $\mathbf{a}$ ist die Auswertung

- (2) Damit ist die Summe der quadrierten Residuen die quadrierte euklidische Norm des Vektors $\mathbf{y} - \mathbf{B}\mathbf{a}$, also (13.7.3).

$$\|\mathbf{v}\|_2^2 = \sum_i v_i^2 \text{ mit } v_i = y_i - (\mathbf{B}\mathbf{a})_i$$

- (3) Jedes Element von $\mathcal{F}_K$ hat einen Koeffizientenvektor, und jeder Koeffizientenvektor liefert ein Element von $\mathcal{F}_K$. Das Minimieren über $\mathcal{F}_K$ ist deshalb dasselbe wie das Minimieren über $\mathbf{a} \in \mathbb{R}^K$, und das ist wortwörtlich das lineare Kleinste-Quadrate-Problem $\min_{\mathbf{a}} \|\mathbf{y} - \mathbf{B}\mathbf{a}\|_2$ aus Abschnitt 7.1.

Die Zuordnung $\mathbf{a} \mapsto \sum_k a_k \phi_k$ bildet $\mathbb{R}^K$ auf $\mathcal{F}_K$ ab; ein Minimum der einen Aufgabe ist eines der anderen

- (4) Die Minimierer von $\|\mathbf{y} - \mathbf{B}\mathbf{a}\|_2$ sind genau die Lösungen der Normalengleichungen (13.7.4). Das ist die Projektions-Eigenschaft aus Abschnitt 7.1: Der Abstand wird minimal, wenn $\mathbf{B}\mathbf{a}$ die orthogonale Projektion von $\mathbf{y}$ auf den Spaltenraum von $\mathbf{B}$ ist, und das heißt, dass das Residuum auf allen Spalten senkrecht steht, $\mathbf{B}^\top(\mathbf{y} - \mathbf{B}\mathbf{a}) = \mathbf{0}$.

Ausmultipliziert ist $\mathbf{B}^\top(\mathbf{y} - \mathbf{B}\mathbf{a}) = \mathbf{0}$ dasselbe wie $\mathbf{B}^\top\mathbf{B}\mathbf{a} = \mathbf{B}^\top\mathbf{y}$

- (5) Ist $\text{rang}(\mathbf{B}) = K$, so ist $\mathbf{B}^\top\mathbf{B}$ symmetrisch positiv definit und damit invertierbar (Abschnitt 7.3), die Lösung von (13.7.4) also eindeutig. Ist $\text{rang}(\mathbf{B}) < K$, so unterscheiden sich je zwei Lösungen um ein Element des Kerns von $\mathbf{B}$; unter ihnen wählt die Pseudoinverse die mit der kleinsten Norm (Abschnitt 7.6).

$\mathbf{a}^\top \mathbf{B}^\top \mathbf{B} \mathbf{a} = \|\mathbf{B}\mathbf{a}\|_2^2 > 0$ für $\mathbf{a} \neq \mathbf{0}$ gilt genau bei vollem Spaltenrang

- (6) Die angepassten Werte sind die orthogonale Projektion von $\mathbf{y}$ auf den Spaltenraum, und eine Projektion hängt nur vom Unterraum ab, nicht von der gewählten Darstellung. Sie ist deshalb auch bei Rangdefekt eindeutig.

□

Bemerkung 13.7.6 (Wie wir das System wirklich lösen)

Die Normalengleichungen sind die kürzeste Herleitung, aber nicht der beste Rechenweg. Sie quadrieren die Konditionszahl, $\kappa_2(\mathbf{B}^\top\mathbf{B}) = \kappa_2(\mathbf{B})^2$, weshalb Kapitel 7 die QR-Zerlegung vorzieht (Abschnitt 7.4, Vergleich der Verfahren in Abschnitt 7.6). Der R-Code am Ende dieses Abschnitts nutzt darum `qr.solve`.

Für B-Splines ist der Schaden allerdings begrenzt, denn sie sind gut konditioniert. Für den Datensatz unseres Widgets messen wir $\kappa_2(\mathbf{B}) \approx 5,6$ bei $K = 10$ und $\kappa_2(\mathbf{B}) \approx 65$ bei $K = 40$. Die Monombasis derselben Größenordnung liegt viele Zehnerpotenzen darüber (Abschnitt 13.3). Dazu kommt die Bandstruktur: Jede Basisfunktion lebt nur über wenigen Teilintervallen, also ist $\mathbf{B}^\top\mathbf{B}$ dünn besetzt und die Zerlegung billig (Abschnitt 5.3).

Bemerkung 13.7.7 (Wann die Designmatrix vollen Spaltenrang hat)

Vollen Spaltenrang bekommen wir nicht geschenkt. Notwendig ist $K \leq n$, aber das reicht nicht: Die Basisfunktionen müssen auch *auf den Daten* linear unabhängig sein, und dafür müssen die Knoten zu den Messstellen passen. Anschaulich muss jede Basisfunktion einen eigenen, passend geordneten Datenpunkt belegen können; der zuständige Satz von Schoenberg und Whitney macht daraus eine Indexbedingung. Mit Knoten auf den Quantilen der x_i hält sie bis $K = n$, und deshalb legt das Widget seine Knoten auf Quantile, wie es auch `splines::bs()` tut.

Vertiefung: Die Schoenberg-Whitney-Bedingung im Detail

Für B-Splines vom Grad q mit erweitertem Knotenvektor $\boldsymbol{\tau}$ lautet die Bedingung: Es müssen sich Indizes $i_1 < \dots < i_K$ finden, sodass die jeweils zugeordnete Basisfunktion am eigenen Datenpunkt positiv ist,

$$B_k^{(q)}(x_{i_k}) > 0, \quad k = 1, \dots, K.$$

Im Inneren des Knotengebiets ist das gleichbedeutend mit $\tau_k < x_{i_k} < \tau_{k+q+1}$. Bei einem offenen, an den Rändern geklemmten Knotenvektor gehören die positiven Randwerte der ersten und letzten Ba-

sisfunktion ausdrücklich dazu. Anschaulich muss jede Basisfunktion also einen eigenen, passend geordneten Datenpunkt belegen können.

Die naheliegende Abschwächung „jede Basisfunktion sieht mindestens einen Datenpunkt“ genügt dafür nicht. Unsere eigene Rechnung mit den 50 Punkten des Widgets, aber *gleichmäßigen* Knoten auf $[0, 2\pi]$, zeigt es: Bei $K = 35$ hat jede der 35 Basisfunktionen mindestens einen Datenpunkt im Träger, und trotzdem bricht die Cholesky-Zerlegung von $\mathbf{B}^T \mathbf{B}$ ab, weil sieben der 32 Knotenintervalle leer bleiben und die Bedingung damit verletzt ist.

Interpolation und Glättung sind derselbe Rahmen

Der Rahmen aus Satz 13.7.5 deckt beide Aufgaben ab. Unterschieden werden sie allein durch die Größe des Ansatzraums.

Definition 13.7.8 (Interpolation und Glättung im Basisansatz)

Eine Anpassung *interpoliert* die Daten, wenn ihre Residuen verschwinden, also wenn

$$\mathbf{B}\hat{\mathbf{a}} = \mathbf{y} \quad \text{beziehungsweise} \quad \hat{f}(x_i) = y_i \quad \text{für alle } i$$

gilt. Bei einer *Glättung* dürfen Residuen verbleiben, damit die Schätzung nicht jede beobachtete Schwankung übernehmen muss.

Für eine unbestrafte Anpassung sind $K = n$ und invertierbares $\mathbf{B}$ der klassische Interpolationsfall. Ein vollrangiger Ansatz mit $K < n$ ist das klassische Regressions- oder Glättungsregime: Er kann nicht jeden beliebigen Datenvektor interpolieren. Die Zahlen K und n allein definieren die beiden Begriffe jedoch nicht; entscheidend sind Rang, konkrete rechte Seite und ein möglicher Strafterm.

Bei $K = n$ ist $\mathbf{B}$ quadratisch. Ist sie zusätzlich invertierbar, so hat (13.7.4) die Lösung $\hat{\mathbf{a}} = \mathbf{B}^{-1}\mathbf{y}$, das Residuum ist der Nullvektor, und $\hat{f}$ trifft jeden Datenpunkt exakt. Bei $K < n$ ist das System überbestimmt; genau dann bleibt ein Rest, wenn $\mathbf{y} \notin \text{Bild}(\mathbf{B})$ gilt.

Bemerkung 13.7.9 (Warum Interpolation das Rauschen erbt)

Warum ist das exakte Treffen schlecht? Setzen wir (13.7.1) in die Interpolationsbedingung ein:

$$\hat{f}(x_i) = y_i = f(x_i) + \varepsilon_i \quad \implies \quad \hat{f}(x_i) - f(x_i) = \varepsilon_i.$$

An jeder Beobachtungsstelle ist der Schätzfehler exakt der Messfehler. Er wird nicht kleiner, wenn wir mehr Daten erheben, denn jeder neue Punkt bringt seinen eigenen Fehler mit, den der Interpolant pflichtschuldig übernimmt. Zwischen den Punkten wird es eher schlimmer: Liegen zwei Stellen dicht beieinander und ziehen ihre Fehler in verschiedene Richtungen, muss die Kurve dazwischen steil werden. Modelliert eine Schätzung auf diese Weise den Zufallsanteil der Daten mit, spricht die Statistik von Überanpassung.

Bei vollem Spaltenrang und $K < n$ ist ein exakter Fit nicht mehr für jeden Datenvektor möglich; bei generischen verrauschten Daten bleibt ein Rest. Jeder Koeffizient wird dann aus mehreren Beobachtungen bestimmt, und in dieser Mittelung hebt sich ein Teil der Fehler auf. Ausnahmsweise kann auch ein kleiner Raum exakt interpolieren, nämlich wenn $\mathbf{y}$ zufällig in $\text{Bild}(\mathbf{B})$ liegt. Der Preis des kleinen Ansatzraums steht auf der anderen Seite: Er enthält f vielleicht gar nicht. Wie sich beides gegeneinander verrechnet, ist das Thema von Abschnitt 13.8.

Beispiel 13.7.10 (Fünfundzwanzig verrauschte Punkte)

Wir ziehen einmal $n = 50$ Stellen gleichverteilt auf $[0, 2\pi]$, werten dort

$$f(x) = \sin(x) + \frac{1}{2} \sin(2x)$$

aus und addieren normalverteilte Fehler mit $\sigma = 0,3$. Die Ziehung hat einen festen Startwert; die empirische Streuung der gezogenen Fehler beträgt 0,266. Auf diese Daten passen wir kubische B-Splines mit K Basisfunktionen an, Knoten auf den Quantilen der x_i . Vier Werte von K im Vergleich, alle Zahlen von unserem Widget selbst gerechnet:

K	RSS	$\hat{\sigma}$	Abstand zu f	größte Abweichung
4	6,421	0,374	0,247	0,467
10	3,084	0,278	0,086	0,532
20	2,460	0,286	0,269	2,223
40	0,557	0,236	1,908	8,574

Dabei ist $\hat{\sigma} = \sqrt{\text{RSS} / (n - K)}$ der übliche Residualstandardfehler. Als Schätzung von σ ist er bei vollem Rang, unkorrelierten homoskedastischen Fehlern und korrekt spezifiziertem Mittelwertmodell gerechtfertigt; fehlt f im Ansatzraum, enthält er zusätzlich Lack-of-fit. „Abstand zu f “ ist die Wurzel aus dem mittleren quadrierten Abstand

$$\frac{1}{2\pi} \int_0^{2\pi} (\hat{f}(x) - f(x))^2 dx.$$

Die beiden mittleren Spalten sind aus Daten ausrechenbar, die beiden rechten nicht, denn sie brauchen f . Und sie laufen auseinander: Von $K = 10$ auf $K = 40$ fällt die Residuenquadratsumme auf weniger als ein Fünftel, während der Abstand zur wahren Funktion auf das Zweiundzwanzigfache steigt. Am besten trifft in unserem Lauf $K = 11$ mit einem Abstand von 0,072.

Die Tabelle zeigt drei Regime. Bei $K = 4$ ist der Ansatzraum gerade der Raum der kubischen Polynome, und die Kurve ist zu starr, um beide Wellen von f mitzunehmen; zwischen $K = 6$ und $K = 15$ liegt sie sauber im Rauschband, und $\hat{\sigma}$ bewegt sich dort zwischen 0,278 und 0,306, also dicht an der wahren Streuung 0,3; ab $K = 16$ beginnen die Ausschläge, und der Abstand zu f wächst wieder, während die Residuen weiter fallen.

Bemerkung 13.7.11 (Die Residuen fallen nicht immer)

Beim Blick auf die Tabelle liegt die Vermutung nahe, die Residuenquadratsumme falle mit wachsendem K immer. Für *geschachtelte* Räume stimmt das: Kommt eine Basisfunktion hinzu und bleiben die alten erhalten, so ist der alte Minimierer weiterhin zulässig, das Minimum kann also nur sinken. Legen wir die Knoten aber auf Quantile, so wandern sie mit K , und die Räume sind nicht geschachtelt. In unserem Lauf steigt die Residuenquadratsumme bei elf der 36 Schritte von K auf $K + 1$ wieder an, etwa von $K = 6$ auf $K = 7$.

Interaktiv: Ein Regler für die Flexibilität

Das Widget rechnet den Fit für jedes K zwischen 4 und 40 neu: Cox-de-Boor für die Basis, Normalgleichungen mit Cholesky-Zerlegung für die Koeffizienten. Im unteren Bild stehen die K Basisfunktionen selbst, sie werden mit wachsendem K schmaler und zahlreicher. Die drei Regime lassen sich am Regler abfahren. Ab $K = 16$ beginnen die Ausschläge, und

von $K = 30$ an verlässt die Kurve bei den meisten Reglerstellungen das Bildfenster, ohne dass die Residuen davon noch viel besser würden.

Die rot gezeichneten Residuen zeigen dasselbe von der anderen Seite: Sie werden im Mittel kürzer, während die grüne Kurve immer weniger mit der violetten zu tun hat.

Welche der drei voreingestellten Knotenzahlen dürfte die unbekannte wahre Funktion am besten treffen?

Schätzfrage. Welcher der drei Fälle wirkt als Schätzer am plausibelsten?

Lösung: *passend*

Der zweite Weg: strafen statt weglassen

Bei der Glättung über einen kleinen Ansatzraum steuert K die Glattheit in groben Stufen, und wir müssen uns vorab auf Zahl und Lage der Knoten festlegen. Der zweite Weg aus Bemerkung 13.7.3 dreht das um: Wir lassen viele Knoten zu und bestrafen dafür unruhige Lösungen.

Vertiefung: Glättungssplines und Penalized Splines im Detail

Definition 13.7.12 (Penalisiertes Kleinste-Quadrate-Kriterium)

Für $\lambda \geq 0$ heißt

$$\sum_{i=1}^n (y_i - g(x_i))^2 + \lambda \int_a^b |g''(x)|^2 dx \quad (13.7.5)$$

das *penalisierte* Kleinste-Quadrate-Kriterium, und ein Minimierer über $\mathcal{C}^2[a, b]$ heißt *Glättungsspline* (smoothing spline). Der Parameter λ heißt *Glättungsparameter*.

Der Strafterm ist keine neue Größe. Er ist das Krümmungsfunktional $J(g) = \int_a^b |g''|^2 dx$ aus Abschnitt 13.5, also dasselbe Maß für Unruhe, das dort den natürlichen kubischen Spline ausgezeichnet hat.

Bemerkung 13.7.13 (Was der Strafterm bewirkt)

Drei Beobachtungen ordnen das Kriterium ein.

1. *Der Minimierer ist wieder ein Spline.* Für $\lambda > 0$ und mindestens zwei verschiedene Beobachtungsstellen ist der Minimierer von (13.7.5) über $\mathcal{C}^2[a, b]$ ein natürlicher kubischer Spline mit Knoten in *allen* verschiedenen Datenpunkten. Das ist der Satz von Schoenberg und Reinsch, den wir hier nicht beweisen; Green und Silverman führen ihn in Kapitel 2 vor. Bei paarweise verschiedenen x_i hat der daraus entstehende natürliche Spline-Raum Dimension $K = n$, und trotzdem entsteht für $\lambda > 0$ keine Interpolation.
2. *Die beiden Grenzfälle sind vertraut.* Für $\lambda \rightarrow 0$ verschwindet die Strafe. Bei paarweise verschiedenen Datenstellen bleiben im Grenzwert die Interpolanten, und unter ihnen wählt der Strafterm den mit der kleinsten Krümmung, also den natürlichen kubischen Spline aus Satz 13.5.4. Bei wiederholten Stellen mit verschiedenen Beobachtungen kann es dagegen keinen Interpolanten aller Einzelwerte geben. Für $\lambda \rightarrow \infty$ wird jede Kandidatin mit $J(g) > 0$ unbezahlbar; übrig bleiben die Funktionen mit $g'' \equiv 0$, also die Geraden, und unter ihnen die Kleinste-Quadrate-Gerade. Der Glättungsparameter fährt die Schätzung stufenlos zwischen Interpolation und einfacher linearer Regression hin und her.
3. *Gesteuert wird jetzt mit λ , nicht mit K .* Statt einer ganzen Zahl drehen wir an einer stetigen Größe, und die Knotenwahl fällt als Frage weg. Beahlt wird das mit einem Gleichungssystem der Größe n statt K .

Bemerkung 13.7.14 (Penalized Splines und die Nähe zu Ridge)

Zwischen beiden Wegen liegt der in der Praxis übliche Kompromiss: eine großzügige, aber nicht datengroße Basis ($K < n$) und ein Strafterm. Solche *Penalized Splines* führen auf ein Kriterium, das sich vollständig in Matrixform schreiben lässt. Mit $\mathbf{P}_{jk} := \int_a^b \phi_j''(x) \phi_k''(x) dx$ ist $\int_a^b |\hat{f}''|^2 dx = \mathbf{a}^\top \mathbf{P} \mathbf{a}$, und (13.7.5) wird zu

$$\|\mathbf{y} - \mathbf{B}\mathbf{a}\|_2^2 + \lambda \mathbf{a}^\top \mathbf{P} \mathbf{a}.$$

Ableiten nach $\mathbf{a}$ und Nullsetzen liefert die gestraften Normalengleichungen

$$(\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{P}) \mathbf{a} = \mathbf{B}^\top \mathbf{y}.$$

Mit $\mathbf{P} = \mathbf{I}$ steht dort wortwörtlich die Ridge-Regression aus Abschnitt 12.5, und der Rechenrick ist derselbe: Der Zuschlag macht das System auch dann eindeutig lösbar, wenn $\mathbf{B}$ keinen vollen Spaltenrang hat. Beim Krümmungs-Strafterm ist $\mathbf{P}$ allerdings nur positiv semidefinit, denn affine Funktionen haben Strafe null. Positiv definit wird $\mathbf{B}^\top \mathbf{B} + \lambda \mathbf{P}$ dann, sobald keine affine Funktion an allen Datenstellen verschwindet, wofür zwei verschiedene x_i genügen. Der Unterschied zu Ridge liegt in der Bedeutung: Dort werden *große* Koeffizienten bestraft, hier *unruhige* Funktionen. Wie λ gewählt wird, ist dieselbe Frage wie die nach dem richtigen K ; Abschnitt 13.8 kommt darauf zurück.

Wann welche Methode?

Situation	Methode
präzise, rauschfreie Daten	Interpolation
verrauschte Messungen	Glättung
Vorhersage auf neuen Daten	Glättung

Die dritte Zeile ist die wichtigste und die am wenigsten offensichtliche. Wer nur die vorliegenden n Punkte beschreiben will, fährt mit der Interpolation nicht schlecht: Sie hat auf den Daten die kleinste Residuenquadratsumme, nämlich null. Sobald wir aber an einer *neuen* Stelle vorhersagen wollen, zählt der Abstand zu f , und der ist bei einer glatten Schätzung kleiner. Beispiel 13.7.10 führt beide Größen nebeneinander vor.

Beispiel: Glättung in R

Der ganze Abschnitt passt in acht Zeilen R, hier für den Datensatz aus Beispiel 13.7.10:

```
B <- splines::bs(x, df = 10, intercept = TRUE) # n x 10 Designmatrix
a_hat <- qr.solve(B, y) # KQ-Loesung ueber QR

x_new <- seq(0, 2 * pi, length = 200)
B_new <- splines::bs(x_new,
                     knots = attr(B, "knots"),
                     Boundary.knots = attr(B, "Boundary.knots"),
                     intercept = TRUE)
f_hat <- B_new %*% a_hat
```

Drei Details lohnen den zweiten Blick.

- `bs()` baut die B-Spline-Designmatrix $\mathbf{B}$. Das Argument `df = 10` zusammen mit `inter-`

`cept = TRUE` fordert $K = 10$ Spalten; bei Grad 3 bleiben dafür $10 - 3 - 1 = 6$ innere Knoten, die die Funktion auf die Quantile der x_i legt. Wir wählen also nicht die Knoten, sondern ihre Anzahl.

- `qr.solve(B, y)` löst das überbestimmte System über die QR-Zerlegung (Abschnitt 7.4) und nicht über die Normalgleichungen. Das ist der stabilere der beiden Wege (Bemerkung 13.7.6).
- Für die Vorhersage an neuen Stellen braucht `B_new` *dieselbe* Basis. Deshalb werden Knoten und Randknoten von `B` übernommen. Lässt man `Boundary.knots` weg, so setzt `bs()` sie auf den Wertebereich des *neuen* Gitters: `B_new` gehört dann zu einer anderen Basis als `a_hat`, und zwar lautlos (Bemerkung 13.4.12). Beide Argumente, `knots` und `Boundary.knots`, gehören zusammen übergeben.

Ausgaben drucken wir hier keine ab. Wer sehen will, wie die Kurve auf einen konkreten Datensatz reagiert, findet oben im Widget eine eigene Realisierung, deren Zahlen wir selbst nachgerechnet haben.

Selbsttest

1. Im Glättungs-Widget kann $K = 40$ eine kleinere RSS und zugleich eine schlechtere Annäherung an die wahre Funktion liefern.

Antwort: Wahr

Die RSS misst nur die Anpassung an die verrauschten Daten. Bei großem K folgt der Fit einzelnen Störungen und überanpasst.

2. Minimieren wir $\sum_i (y_i - \hat{f}(x_i))^2$ über alle Funktionen $\hat{f}$, so erhalten wir die bestmögliche Schätzung von f .

Antwort: Falsch

Das Minimum ist null und wird von jedem Interpolanten angenommen, und davon gibt es unendlich viele (Abschnitt 13.1). Alle diese Funktionen übernehmen die Fehler ε_i punktweise (Bemerkung 13.7.9). Erst eine Einschränkung des Suchraums oder ein Strafterm macht die Aufgabe sinnvoll.

3. Ist $K = n$ und hat $\mathbf{B}$ vollen Rang, so ist die Residuenquadratsumme null.

Antwort: Wahr

Dann ist $\mathbf{B}$ quadratisch und invertierbar, (13.7.4) hat die Lösung $\hat{\mathbf{a}} = \mathbf{B}^{-1}\mathbf{y}$, und der Fit trifft jeden Punkt exakt. Für den Datensatz aus Beispiel 13.7.10 rechnen wir mit $K = n = 50$ eine Residuenquadratsumme von 10^{-16} nach. Ein guter Schätzer ist das nicht: Die Koeffizienten erreichen dabei Beträge über $3 \cdot 10^4$, und $\kappa_2(\mathbf{B})$ steigt auf rund $2 \cdot 10^5$.

4. Damit $\mathbf{B}^\top \mathbf{B}$ invertierbar ist, genügt es, dass jede Basisfunktion mindestens einen Datenpunkt in ihrem Träger hat.

Antwort: Falsch

Gebraucht wird die Bedingung von Schoenberg und Whitney aus Bemerkung 13.7.7, und die verlangt für jede Basisfunktion einen *eigenen* Datenpunkt in der richtigen Reihenfolge. Unsere Gegenrechnung mit gleichmäßigen Knoten und $K = 35$ auf 50 Punkten erfüllt die schwächere Forderung, und trotzdem bricht die Cholesky-Zerlegung ab.

5. Auch bei Rangdefekt von $\mathbf{B}$ sind die angepassten Werte $\mathbf{B}\hat{\mathbf{a}}$ eindeutig bestimmt.

Antwort: Wahr

Sie sind die orthogonale Projektion von $\mathbf{y}$ auf den Spaltenraum von $\mathbf{B}$, und die hängt nur vom Unterraum ab (letzter Beweisschritt zu Satz 13.7.5). Nicht eindeutig sind dagegen die Koeffizienten und mit ihnen die Funktion $\hat{f}$ selbst: Zwei Lösungen unterscheiden sich um ein Element des Kerns von $\mathbf{B}$, und die zugehörige Funktion verschwindet nur an den Datenstellen, nicht überall.

6. Mit wachsendem K fällt die Residuenquadratsumme in jedem Schritt.

Antwort: Falsch

Für geschachtelte Ansatzräume stimmt es, denn der alte Minimierer bleibt zulässig. Wandern die Knoten mit K , etwa weil sie auf Quantilen liegen, so sind die Räume nicht geschachtelt: In unserem Lauf steigt die Residuenquadratsumme bei elf von 36 Schritten wieder an, von $K = 6$ auf $K = 7$ etwa von 3,435 auf 4,016 (Bemerkung 13.7.11).

7. Für $\lambda \rightarrow \infty$ nähert sich der Glättungsspline der Kleinste-Quadrate-Geraden.

Antwort: Wahr

Der Strafterm $\lambda \int |g''|^2 dx$ zwingt im Grenzwert $g'' \equiv 0$, also affines g . Unter den affinen Funktionen minimiert das Kriterium nur noch die Residuenquadratsumme, und das ist die einfache lineare Regression.

Vertiefung: Die Theorie der Glättungssplines samt Existenz- und Eindeutigkeitsatz entwickeln P. J. Green und B. W. Silverman, Nonparametric Regression and Generalized Linear Models, Chapman und Hall 1994, Kapitel 2; die Penalized Splines und die additiven Modelle dahinter stehen bei S. N. Wood, Generalized Additive Models: An Introduction with R, 2. Auflage 2017. Die Rangbedingung aus Bemerkung 13.7.7 geht auf I. J. Schoenberg und A. Whitney (1953) zurück und wird bei C. de Boor, A Practical Guide to Splines, Springer 2001, bewiesen.

13.8 Bias-Varianz und Modellwahl

In Abschnitt 13.7 haben wir die Interpolation aufgegeben und stattdessen mit $K < n$ Basisfunktionen geglättet. Eine Frage blieb dabei offen, und es ist die einzige, die in der Praxis wirklich zu beantworten ist: Wie groß soll K sein? Zwei Kräfte ziehen in entgegengesetzte Richtungen. Zu wenige Basisfunktionen können f gar nicht darstellen, zu viele lassen den Schätzer dem Rauschen hinterherlaufen. Dieser Abschnitt macht aus dieser Intuition zwei Sätze und zeigt anschließend an einem asymptotischen Balance-Modell, wie eine theoretische Wahl von K aussehen kann.

Bemerkung 13.8.1 (Modell, Designmatrix, Schätzer)

Wir sammeln die Bezeichnungen aus Abschnitt 13.7, mit denen wir hier rechnen. An festen Stellen $x_1 < \dots < x_n$ in $[a, b]$ beobachten wir

$$y_i = f(x_i) + \varepsilon_i, \quad \mathbb{E}[\varepsilon_i] = 0, \quad \text{Var}[\varepsilon_i] = \sigma^2,$$

mit unkorrelierten Fehlern. Zu einer Basis $\phi_1, \dots, \phi_K$ von Splinefunktionen (Abschnitt 13.2) bilden wir die Designmatrix $\mathbf{B} \in \mathbb{R}^{n \times K}$ mit Einträgen $B_{ik} = \phi_k(x_i)$, schreiben $\phi(x) = (\phi_1(x), \dots, \phi_K(x))^T$ und erhalten den Kleinste-Quadrate-Schätzer

$$\hat{\mathbf{a}} = \mathbf{B}^+ \mathbf{y}, \quad \hat{f}(x) = \phi(x)^T \hat{\mathbf{a}}. \quad (13.8.1)$$

Dabei ist $\mathbf{B}^+$ die Pseudoinverse (Abschnitt 7.6); hat $\mathbf{B}$ vollen Spaltenrang K , so ist $\mathbf{B}^+ = (\mathbf{B}^\top \mathbf{B})^{-1} \mathbf{B}^\top$, und (13.8.1) ist die gewöhnliche Lösung der Normalengleichungen (Abschnitt 7.1).

Zwei Dinge sind an (13.8.1) wichtig. Der Schätzer ist *linear* in den Daten, er entsteht also durch Multiplikation mit einer festen Matrix. Und mit $\mathbf{y} = \mathbf{f} + \boldsymbol{\epsilon}$, wobei $\mathbf{f} = (f(x_1), \dots, f(x_n))^\top$ die rauschfreien Werte sammelt, zerfällt er in

$$\hat{\mathbf{a}} = \mathbf{B}^+ \mathbf{f} + \mathbf{B}^+ \boldsymbol{\epsilon}.$$

Der erste Summand ist deterministisch, der zweite trägt den ganzen Zufall. Diese Aufteilung ist die Bias-Varianz-Zerlegung in Rohform.

Bias: der Preis für zu wenige Basisfunktionen

Der erste Summand ist bereits vollständig bestimmt, bevor eine einzige Zufallszahl gezogen wird. Er misst, wie gut sich f überhaupt im Spann unserer K Basisfunktionen darstellen lässt.

Satz 13.8.2 (Der Bias ist der Approximationsfehler ohne Rauschen)

Hat $\mathbf{B}$ vollen Spaltenrang, so gilt für jedes $x \in [a, b]$

$$\mathbb{E}[\hat{f}(x)] = \boldsymbol{\phi}(x)^\top \mathbf{B}^+ \mathbf{f}. \quad (13.8.2)$$

Der Erwartungswert des Schätzers ist damit dieselbe Anpassung, die wir aus rauschfreien Daten bekämen. Der *Bias* $\mathbb{E}[\hat{f}(x)] - f(x)$ hängt damit weder von σ noch von der einzelnen Ziehung ab.

Liegen die Knoten gleichmäßig auf $[a, b]$, sind die ϕ_k eine Basis der kubischen Splines zu diesen Knoten und ist $f \in \mathcal{C}^4[a, b]$, so gilt zusätzlich

$$\frac{1}{n} \sum_{i=1}^n (\mathbb{E}[\hat{f}(x_i)] - f(x_i))^2 \leq \left(Ch^4 \max_{x \in [a, b]} |f^{(4)}(x)| \right)^2 = O(K^{-8}), \quad (13.8.3)$$

wobei h die Gitterweite der Knoten ist.

Vertiefung: Beweis der Bias-Darstellung

Beweis.

- (1) Nach (13.8.1) ist $\hat{f}(x)$ eine feste Linearkombination der $\mathbf{y}_i$. Aus der Linearität des Erwartungswerts folgt

$$\mathbb{E}[\hat{f}(x)] = \boldsymbol{\phi}(x)^\top \mathbf{B}^+ \mathbb{E}[\mathbf{y}] = \boldsymbol{\phi}(x)^\top \mathbf{B}^+ \mathbf{f}.$$

$\mathbf{B}^+$ und $\boldsymbol{\phi}(x)$ hängen nur von den Stellen x_i und der Basis ab, nicht von den Beobachtungen; und $\mathbb{E}[\mathbf{y}] = \mathbf{f}$, weil $\mathbb{E}[\boldsymbol{\epsilon}] = 0$

- (2) Die rechte Seite von (13.8.2) ist der Kleinste-Quadrate-Fit an die rauschfreien Werte $\mathbf{f}$. Nennen wir ihn $\hat{f}_0$. Er minimiert unter allen Funktionen s des Ansatzraums $\mathcal{F}_K$, also des Spanns von $\phi_1, \dots, \phi_K$ (Abschnitt 13.7), die Summe $\sum_{i=1}^n (f(x_i) - s(x_i))^2$.

Das ist die definierende Eigenschaft der Kleinste-Quadrate-Lösung, angewendet auf den Datenvektor $\mathbf{f}$ statt auf $\mathbf{y}$

- (3) Zu den Knoten unserer Basis gehört nach Satz 13.6.2 ein kubischer Spline s^* , der f dort interpoliert und $\max_x |f(x) - s^*(x)| \leq Ch^4 \max_x |f^{(4)}(x)|$ erfüllt. Dieser Spline liegt in $\mathcal{F}_K$.

Kubische Splines zu m Teilintervallen bilden einen Raum der Dimension $m + 3 = K$, und $\phi_1, \dots, \phi_K$ ist eine Basis davon (Abschnitt 13.4)

- (4) Mit Schritt 2 und Schritt 3 folgt

$$\sum_{i=1}^n (f(x_i) - \hat{f}_0(x_i))^2 \leq \sum_{i=1}^n (f(x_i) - s^*(x_i))^2 \leq n \left(Ch^4 \max_x |f^{(4)}(x)| \right)^2,$$

und Division durch n gibt (13.8.3).

- (5) Gleichmäßige Knoten auf einem festen Intervall $[a, b]$ bedeuten $h = (b-a)/(K-3)$, also $h = O(1/K)$ und damit $h^8 = O(K^{-8})$.

K Basisfunktionen vom Grad 3 gehören zu $K - 3$ Teilintervallen (Schritt 3), die Gitterweite fällt also wie $1/K$

□

Bemerkung 13.8.3 (Woran die Ordnung hängt)

Die Annahmen verdienen eine eigene Zeile, denn ohne sie ist die Aussage falsch. Gebraucht werden gleichmäßige Knoten auf einem festen Intervall, $f \in \mathcal{C}^4$ und kubische Stücke. Fällt eine dieser drei Bedingungen weg, ändert sich der Exponent. Bei stückweise linearen Stücken steht h^2 statt h^4 (Satz 13.6.5), bei einer Funktion mit nur zwei stetigen Ableitungen ebenfalls, und ein einziges breites Teilintervall verdirbt die Gitterweite unabhängig davon, wie fein der Rest liegt (Definition 13.6.1).

Die Kurzform $\max_x |f(x) - \hat{f}(x)| = O(h^4)$ ist die punktweise Fassung ohne Rauschen. Satz 13.8.2 mittelt stattdessen über die Entwurfsstellen und quadriert; das ist die Größe, die gleich in den MSE eingeht, und sie erbt die Ordnung $O(K^{-8})$.

Varianz: der Preis für zu viele Basisfunktionen

Der zweite Summand $\mathbf{B}^+ \epsilon$ trägt den Zufall. Er ist es, der bei großem K aus dem Ruder läuft, und über ihn lässt sich mehr sagen als eine Ordnung. Anschaulich heißt das: Ziehen wir die Daten mehrfach neu, so liegen die Schätzkurven bei kleinem K fast übereinander und verfehlen f alle in dieselbe Richtung, während sie bei großem K im Mittel richtig liegen, einzeln aber weit auffächern, weil jede ihrem eigenen Rauschen folgt.

Satz 13.8.4 (Gemittelte Varianz eines linearen Schätzers)

Seien die ϵ_i unkorreliert mit $\text{Var}[\epsilon_i] = \sigma^2$ (homoskedastisch), und habe $\mathbf{B}$ vollen Spaltenrang K . Dann gilt für jedes x

$$\text{Var}[\hat{f}(x)] = \sigma^2 \phi(x)^\top (\mathbf{B}^\top \mathbf{B})^{-1} \phi(x), \quad (13.8.4)$$

und über die Entwurfsstellen gemittelt sogar exakt

$$\frac{1}{n} \sum_{i=1}^n \text{Var}[\hat{f}(x_i)] = \frac{\sigma^2 K}{n}. \quad (13.8.5)$$

Vertiefung: Beweis der gemittelten Varianzformel*Beweis.*

- (1) Für die Kovarianzmatrix des Koeffizientenvektors gilt, wobei $\text{Var}[\cdot]$ für einen Vektor die Matrix der Kovarianzen bezeichnet,

$$\text{Var}[\hat{\mathbf{a}}] = \mathbf{B}^+ \text{Var}[\mathbf{y}] (\mathbf{B}^+)^T = \sigma^2 \mathbf{B}^+ (\mathbf{B}^+)^T = \sigma^2 (\mathbf{B}^T \mathbf{B})^{-1}.$$

$\text{Var}[\mathbf{M}\mathbf{y}] = \mathbf{M} \text{Var}[\mathbf{y}] \mathbf{M}^T$ für feste $\mathbf{M}$; $\text{Var}[\mathbf{y}] = \sigma^2 \mathbf{I}_n$ wegen Unkorreliertheit und Homoskedastizität; und mit $\mathbf{B}^+ = (\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T$ kürzt sich $\mathbf{B}^T \mathbf{B}$ heraus

- (2) Daraus folgt (13.8.4), denn $\hat{f}(x) = \phi(x)^T \hat{\mathbf{a}}$ ist eine Linearkombination der Koeffizienten.

$\text{Var}[\mathbf{c}^T \mathbf{v}] = \mathbf{c}^T \text{Var}[\mathbf{v}] \mathbf{c}$, hier mit $\mathbf{c} = \phi(x)$

- (3) Nun summieren wir (13.8.4) über die Entwurfsstellen. Der Vektor $\phi(x_i)^T$ ist die i -te Zeile von $\mathbf{B}$, also ist

$$\sum_{i=1}^n \phi(x_i)^T (\mathbf{B}^T \mathbf{B})^{-1} \phi(x_i) = \text{tr} \left((\mathbf{B}^T \mathbf{B})^{-1} \sum_{i=1}^n \phi(x_i) \phi(x_i)^T \right) = \text{tr}(\mathbf{I}_K) = K.$$

Jeder Summand ist ein Skalar und damit gleich seiner Spur; die Zyklizität der Spur (Abschnitt 3.1) zieht den Vektor nach vorn, und $\sum_{i=1}^n \phi(x_i) \phi(x_i)^T = \mathbf{B}^T \mathbf{B}$

- (4) Multiplikation mit σ^2/n liefert (13.8.5). □

Bemerkung 13.8.5 (Was die Varianzformel erzählt)

Gleichung (13.8.5) ist bemerkenswert schlicht. Rechts steht keine Eigenschaft der Basis, keine Knotenlage, keine Konditionszahl, sondern nur σ^2 , K und n . Im Mittel kostet uns jede Basisfunktion genau σ^2/n an Varianz, egal welche. Das ist die exakte Fassung der Ordnungsaussage $\text{Var}[\hat{f}(x)] = O(K/n)$.

Der Beweis liefert nebenbei eine Größe, die uns im letzten Teil dieses Abschnitts wieder begegnet. Die Matrix $\mathbf{H} = \mathbf{B}(\mathbf{B}^T \mathbf{B})^{-1} \mathbf{B}^T$ bildet die Beobachtungen auf die angepassten Werte ab, und Schritt 3 zeigt $\text{tr}(\mathbf{H}) = K$. Diese Spur heißt in der Statistik die *Anzahl effektiver Freiheitsgrade* des Schätzers, und bei einer unbestraften Regression mit K Basisfunktionen ist sie schlicht K .

Vorsicht bei der punktweisen Lesart. Formel (13.8.5) mittelt, (13.8.4) nicht. An einer Stelle, in deren Nähe kaum Daten liegen, kann $\text{Var}[\hat{f}(x)]$ um Größenordnungen darüber liegen. Werten wir (13.8.4) im Beispiel unten nicht an den 100 Entwurfsstellen aus, sondern auf einem gleichmäßigen Gitter über $[0, 2\pi]$, so ist die mittlere Varianz bei $K = 40$ nicht 0,036, sondern 5,04: Zwischen den Datenpunkten und an den Rändern schwingt der Schätzer weit stärker als dort, wo er angepasst wurde. Die Ordnung $O(K/n)$ wird meist punktweise notiert; als Aussage über den gemittelten Fehler ist sie richtig, als Aussage über jedes einzelne x nicht.

Der Kompromiss: mittlerer quadratischer Fehler**Satz 13.8.6 (Zerlegung des mittleren quadratischen Fehlers)**

Für jedes x gilt

$$\underbrace{\mathbb{E}[(\hat{f}(x) - f(x))^2]}_{\text{MSE}} = \underbrace{(\mathbb{E}[\hat{f}(x)] - f(x))^2}_{\text{Bias}^2} + \underbrace{\text{Var}[\hat{f}(x)]}_{\text{Varianz}}. \quad (13.8.6)$$

Vertiefung: Beweis der Bias-Varianz-Zerlegung

Beweis.

- (1) Wir schreiben $\mu := \mathbb{E}[\hat{f}(x)]$ und schieben diesen Wert ein:

$$(\hat{f}(x) - f(x))^2 = (\hat{f}(x) - \mu)^2 + 2(\hat{f}(x) - \mu)(\mu - f(x)) + (\mu - f(x))^2.$$

- (2) Jetzt nehmen wir den Erwartungswert. Der mittlere Term fällt weg, der erste ist die Varianz, der letzte ist deterministisch.

$\mathbb{E}[\hat{f}(x) - \mu] = 0$ nach Definition von μ , und $\mu - f(x)$ ist keine Zufallsgröße, darf also aus dem Erwartungswert gezogen werden

□

Setzen wir Satz 13.8.2 und Satz 13.8.4 in (13.8.6) ein und mitteln über die Entwurfsstellen, so erhalten wir unter den Voraussetzungen von Satz 13.8.2 für eine Konstante $C_1 > 0$ die zentrale Obergrenze dieses Abschnitts:

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E}[(\hat{f}(x_i) - f(x_i))^2] \leq \underbrace{C_1 K^{-8}}_{\text{obere Schranke für Bias}^2} + \underbrace{\frac{\sigma^2 K}{n}}_{\text{Varianz}}. \quad (13.8.7)$$

Die Schranke für den ersten Term fällt mit wachsendem K rasant, der zweite Term wächst exakt linear. Damit besitzt die *rechte Seite* ein Minimum dazwischen. Der tatsächliche Bias darf unter seiner Schranke liegen oder bei nicht geschachtelten Räumen schwanken; aus (13.8.7) allein folgt daher noch kein tatsächliches MSE-Minimum an derselben Stelle.

Bemerkung 13.8.7 (Heuristisches Balance-Modell für die Knotenzahl)

Nehmen wir stärker an, dass der quadrierte Bias in dem interessierenden Bereich tatsächlich wie $c_1 K^{-8}$ mit $c_1 > 0$ verläuft, statt nur nach oben so beschränkt zu sein, erhalten wir das Proxy-Kriterium $g(K) = c_1 K^{-8} + c_2 K/n$. Ableiten und Nullsetzen gibt

$$g'(K) = -8c_1 K^{-9} + \frac{c_2}{n} \stackrel{!}{=} 0 \implies K^* = \left(\frac{8c_1}{c_2} \right)^{1/9} n^{1/9},$$

und dort sind beide Summanden von derselben Ordnung:

$$c_1 (K^*)^{-8} \sim n^{-8/9}, \quad \frac{c_2 K^*}{n} \sim n^{1/9-1} = n^{-8/9}.$$

Unter dieser zusätzlichen Äquivalenzannahme ergeben sich also $K^* \asymp n^{1/9}$ und eine Proxy-Rate $n^{-8/9}$. Der Exponent $1/9$ entsteht aus $8 + 1$, also aus der angenommenen Bias-Ordnung plus dem linearen Varianzterm. Als rigorose Optimalitäts- oder Minimax-Aussage bräuchte die Rate zusätzlich eine passende untere Schranke und genaue Annahmen an Funktionsklasse, Design und Folge der Ansatzräume.

Aus derselben Rechnung folgt $c_1 (K^*)^{-8} = \frac{1}{8} c_2 K^*/n$: Der quadrierte Bias trägt im Optimum dieses Modells ein Neuntel der Summe, die Varianz die übrigen acht Neuntel – eine Eigenschaft der angenommenen Potenzform, keine des tatsächlichen MSE.

Dass $K^* \asymp n^{1/9}$ eine *rein asymptotische* Ordnung ist, verdient mehr als eine Fußnote: Mit Konstante 1 liefert die Formel für $n = 100$ den Wert $K^* = 1,67$, und niemand approximiert eine schwingende Funktion mit zwei Basisfunktionen. Was fehlt, ist der Faktor $(8c_1/c_2)^{1/9}$, und der hängt an der vierten Ableitung von f und an σ . Merken lässt sich aus der heuristischen Formel deshalb der Balance-Exponent, nicht eine konkrete Knotenzahl.

Ein numerisches Beispiel

Beispiel 13.8.8 (Bias und Varianz beim Sinusbeispiel)

Wir wählen das Setup $f(x) = \sin(3x)$ auf $[0, 2\pi]$, $n = 100$ feste Stellen, $\sigma = 0,3$. Die Stellen ziehen wir einmal aus einer Gleichverteilung und behalten sie bei; anschließend simulieren wir 200 Datensätze, passen jeweils einen kubischen Regressionsspline mit K Basisfunktionen an und schätzen daraus Bias, Varianz und MSE an den Entwurfsstellen. Alle Zahlen der folgenden Tabelle sind unsere eigene Rechnung.

K	Bias ²	Varianz	$\sigma^2 K/n$	MSE
5	0,4103	0,0044	0,0045	0,4147
8	0,0344	0,0071	0,0072	0,0415
9	0,1174	0,0080	0,0081	0,1254
12	0,0013	0,0107	0,0108	0,0120
15	0,0001	0,0135	0,0135	0,0136
40	0,0001	0,0358	0,0360	0,0360

Drei Beobachtungen. Erstens trifft die geschätzte Varianz die Vorhersage $\sigma^2 K/n$ aus Satz 13.8.4 in jeder Zeile auf höchstens zwei Prozent genau; die Abweichung ist der Simulationsfehler von 200 Wiederholungen. Zweitens liegt das Minimum des MSE bei $K = 12$ mit 0,0120. Bei $K = 5$ ist der MSE das 34,5-fache davon, bei $K = 40$ das Dreifache. Drittens trägt beim simulierten Minimum der Bias nur noch 11 Prozent zum MSE bei. Das stimmt hier numerisch mit dem Ein-Neuntel-Verhältnis des Proxy-Modells aus Bemerkung 13.8.7 überein, ist aber keine theoretisch erzwungene Übereinstimmung. Das diskrete Optimum liegt dort, wo der nächste zulässige Schritt mehr Varianz kostet, als er in dieser Simulation an Bias einspart.

Bemerkung 13.8.9 (Der Bias fällt nicht monoton)

Zwischen $K = 8$ und $K = 9$ steigt der quadrierte Bias in der Tabelle wieder an. Beide Anpassungen sind schlecht, denn fünf beziehungsweise sechs Teilintervalle sind für drei Sinusperioden zu wenig. Welche von beiden weniger schlecht ausfällt, entscheidet die Lage der Knoten relativ zu f : Bei $K = 9$ sitzen die fünf inneren Knoten exakt auf den Nullstellen $k\pi/3$ von $\sin(3x)$, dort also, wo auch f'' verschwindet, während die Funktion zwischen den Knoten am stärksten gekrümmt ist.

Ein Widerspruch zu Satz 13.8.2 ist das nicht. Die Aussage $O(K^{-8})$ ist eine obere Schranke für große K , keine Garantie, dass jeder einzelne Schritt eine Verbesserung bringt. Wir kennen dieselbe Unterscheidung aus Abschnitt 13.6, wo die gemessenen Faktoren erst allmählich gegen die vorhergesagten 16 liefen.

Interaktiv: Zwölf Kurven, drei Balken, ein Regler

Das Widget rechnet die Tabelle live nach und zeigt zugleich, woher die beiden Anteile kommen. Links stehen die Daten einer einzelnen Ziehung in Blau, die wahre Funktion violett gestrichelt, die inneren Knoten als orange Marken auf der Achse und in Grün die Schätzer der ersten zwölf Wiederholungen samt ihrem Mittelwert.

Bei kleinem K liegen die zwölf Kurven fast übereinander, bei $K = 40$ fächern sie weit auf, und nur noch ihr Mittelwert liegt auf f . Die Balken rechts zeigen die Anteile am MSE, die Tafel darunter den Verlauf aller drei Größen über K auf logarithmischer Skala. Dort ist der Knick gut zu sehen: Der rote Bias stürzt ab, die grüne Varianz steigt langsam und stetig, und die graue Summe hat ihr Minimum dazwischen.

Alle Zufallszahlen stammen aus einem festen Startwert. Es ist bei jedem Aufruf dieselbe Simulation, und die Zahlen im Widget stimmen mit denen der Tabelle überein.

Wo vermuten wir vor dem Blick auf die MSE-Kurve ihr Minimum?

Schätzfrage. Welche Knotenzahl minimiert in dieser festen Simulation den MSE?

Lösung: 12 K

Wie wählt man K in der Praxis?

Die ganze bisherige Rechnung hat einen Haken, und er ist deutlich zu benennen: Wir kennen f nicht. Bias und MSE sind damit nicht ausrechenbar, sondern nur die Residuen. In geschachtelten Räumen werden sie mit wachsendem K immer kleiner, bis ein vollrangiger Fall mit $K = n$ jeden Datenpunkt exakt trifft; bei wandernden Knoten dürfen einzelne Schritte davon abweichen (Bemerkung 13.7.11). Der allgemeine Trend bleibt: Trainingsanpassung belohnt Flexibilität, nicht Vorhersagegüte. Als Auswahlkriterium taugt sie daher nicht.

Bemerkung 13.8.10 (Drei Auswege)

1. *Kreuzvalidierung.* Wir halten einen Teil der Daten zurück, passen auf dem Rest an und messen den Fehler auf dem zurückgehaltenen Teil. Punkte, die nicht an der Anpassung beteiligt waren, bestrafen Überanpassung von selbst. Gewählt wird das K mit dem kleinsten Validierungsfehler.
2. *Informationskriterien.* AIC und BIC addieren zur Anpassungsgüte einen Strafterm, der mit der Parameterzahl wächst. Für normalverteilte Fehler sind sie bis auf Konstanten $n \log(\text{RSS}_K/n) + 2K$ beziehungsweise $n \log(\text{RSS}_K/n) + \log(n) K$; BIC bestraft schärfer, sobald $n \geq 8$ ist. Beide gehen auf die Log-Likelihood zurück.
3. *Generalisierte Kreuzvalidierung.* Sie schätzt den Kreuzvalidierungsfehler aus einer einzigen Anpassung.

Definition 13.8.11 (Generalisierte Kreuzvalidierung)

Mit der Residuenquadratsumme $\text{RSS}_K = \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$ heißt

$$\text{GCV}(K) = \frac{\text{RSS}_K/n}{(1 - K/n)^2} \quad (13.8.8)$$

der *GCV-Kriteriumswert*. Gewählt wird das K , das ihn minimiert.

Hier ist der Schätzer ein unbestrafter KQ-Fit mit vollem Spaltenrang, sodass $\text{tr}(\mathbf{H}) = K$ gilt. Für einen allgemeinen linearen Glätter steht im Nenner die effektive Freiheitsgradzahl $\text{tr}(\mathbf{H})$

anstelle von K .

Bemerkung 13.8.12 (Warum der Nenner nötig ist)

Bei geschachtelten Modellräumen fällt der Zähler monoton in K und wäre als Auswahlkriterium nutzlos. Bei den hier verwendeten, mit K wandernden Quantilsknoten kann er einzelne Male steigen (Bemerkung 13.7.11), bevorzugt im Trend aber weiterhin unnötig flexible Trainingsfits. Der Nenner sorgt für den nötigen Gegendruck, und er kommt nicht aus der Luft: Er entsteht aus der Kreuzvalidierung, die jeden Punkt einmal weglässt.

Die Faustregel dahinter ist einfach: Je näher K an n rückt, desto stärker bläst der Nenner den beobachteten Fehler auf, weil ein Schätzer mit vielen Parametern seine eigenen Residuen künstlich klein rechnet.

Vertiefung: Woher der Nenner kommt

Für einen linearen Schätzer $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ lässt sich das Residuum, das beim Weglassen des i -ten Punktes entstünde, ohne erneute Anpassung ausrechnen: Es ist $(y_i - \hat{f}(x_i))/(1 - H_{ii})$, was man mit der Sherman-Morrison-Formel nachrechnet. Die Kreuzvalidierung wird damit zu einer einzigen Formel. Ersetzt man darin alle H_{ii} durch ihren Mittelwert $\text{tr}(\mathbf{H})/n$, so steht (13.8.8) da, denn nach Bemerkung 13.8.5 ist $\text{tr}(\mathbf{H}) = K$.

Die Leave-one-out-Identität hält dabei die Designmatrix und damit die Basis fest. Würden wir nach dem Weglassen jedes Punkts auch quantilbasierte Knoten neu bestimmen, entstünde ein anderes Verfahren, für das die Formel nicht exakt dieselbe Kreuzvalidierung berechnet.

Beispiel 13.8.13 (Was die Kriterien in unserem Datensatz wählen)

Wir nehmen die erste der 200 Ziehungen aus Beispiel 13.8.8, also einen einzigen Datensatz, und werten (13.8.8) für $K = 4, \dots, 40$ aus. Das Minimum liegt bei $K = 10$ mit $\text{GCV} = 0,0895$; AIC und BIC in den Formen aus Bemerkung 13.8.10 wählen dasselbe K .

Der wahre MSE ist bei $K = 12$ am kleinsten. Mit $K = 10$ zahlen wir 0,0157 statt 0,0120, also rund 30 Prozent Aufschlag. Für ein Kriterium, das f überhaupt nicht kennt und mit einer einzigen Ziehung auskommt, ist das ein gutes Ergebnis, und es zeigt zugleich die Grenze: Die Kriterien schätzen den Fehler, sie kennen ihn nicht. Auf einer anderen Ziehung fällt die Wahl anders aus.

Bemerkung 13.8.14 (Der praktische Ausweg: groß wählen und bestrafen)

In der Anwendung wird K meist gar nicht feinjustiert. Man wählt es großzügig, sodass der Bias sicher klein ist, und steuert die Glattheit stattdessen über einen Strafterm. Das führt auf *P-Splines* und weiter auf additive Modelle; in R ist `mgcv : : gam()` die Standardadresse dafür. Der Vorteil ist mehr als Bequemlichkeit. Die Knotenzahl ist eine ganze Zahl, der Strafparameter dagegen stufenlos, und ein stufenloser Regler lässt sich mit denselben Kriterien viel feiner einstellen. Formal führt das auf gestrafte Normalengleichungen; die Details stehen in der Vertiefung zu Glättungs- und Penalized Splines in Abschnitt 13.7. Auch die effektiven Freiheitsgrade $\text{tr}(\mathbf{H})$ aus Bemerkung 13.8.5 bleiben brauchbar; sie sind dann keine ganze Zahl mehr, sondern rutschen mit wachsendem Strafparameter stetig nach unten. Die Glättungs-splines aus Abschnitt 13.7 sind der Grenzfall $K = n$ mit Strafterm.

Selbsttest

1. Je größer K , desto kleiner der Bias: Die Verzerrung fällt monoton.

Antwort: Falsch

Satz 13.8.2 gibt eine obere Schranke der Ordnung $O(K^{-8})$, keine monotone Folge. In unserer eigenen Simulation steigt der quadrierte Bias von 0,0344 bei $K = 8$ auf 0,1174 bei $K = 9$: Beide Gitter sind viel zu grob, und bei $K = 9$ fallen die inneren Knoten zusätzlich genau auf die Nullstellen von $\sin(3x)$ (Bemerkung 13.8.9). Erst über größere Sprünge in K setzt sich der Trend durch.

2. Der Bias hängt nicht von der Rauschstärke σ ab.

Antwort: Wahr

Nach (13.8.2) ist $\mathbb{E}[\hat{f}(x)] = \phi(x)^\top \mathbf{B}^+ \mathbf{f}$, und darin kommt σ nicht vor. Der Bias ist der Approximationsfehler, den wir auch bei rauschfreien Daten hätten. Stärkeres Rauschen vergrößert die Varianz, nicht die Verzerrung.

3. Die über die Entwurfsstellen gemittelte Varianz ist exakt $\sigma^2 K/n$, unabhängig davon, wo die Knoten liegen.

Antwort: Wahr

Das ist (13.8.5). Der Beweis benutzt nur, dass der Schätzer die Kleinste-Quadrate-Anpassung an eine Designmatrix mit vollem Spaltenrang K ist, dazu Unkorreliertheit und Homoskedastizität. Weder die Basis noch die Knotenlage gehen ein, denn am Ende steht $\text{tr}(\mathbf{I}_K) = K$. Unsere Simulation bestätigt es in jeder Zeile der Tabelle aus Beispiel 13.8.8.

4. Dann ist auch die Varianz an jeder einzelnen Stelle x gleich $\sigma^2 K/n$.

Antwort: Falsch

Gemittelt wird über die n Entwurfsstellen, und dort ist die Aussage exakt. Zwischen den Datenpunkten und an den Rändern kann die Varianz weit darüber liegen: Bei $K = 40$ misst unsere Simulation über ein feines Gitter auf $[0, 2\pi]$ eine mittlere Varianz von 5,04 statt 0,036. Punktweise gilt nur (13.8.4), und dort steht $\phi(x)$ im Ausdruck.

5. Im Minimum des MSE sind quadrierter Bias und Varianz allgemein gleich groß.

Antwort: Falsch

Schon im glatten Proxy-Modell sind am Optimum die *Änderungsraten*, nicht die Beiträge gleich: $g'(K^*) = 0$ heißt, dass eine infinitesimale Verfeinerung genauso viel Varianz kostet, wie sie an Bias spart. Für die spezielle Form $c_1 K^{-8} + c_2 K/n$ trägt der Bias dort ein Neuntel der Summe. Das ist eine Eigenschaft dieses Modells, keine allgemeine MSE-Identität. Im simulierten Beispiel liegt der Anteil bei $K = 12$ zufällig ebenfalls nahe daran (Beispiel 13.8.8).

6. GCV lässt sich ausrechnen, ohne f zu kennen.

Antwort: Wahr

In (13.8.8) stehen nur die Residuenquadratsumme, n und K , alles beobachtbare Größen. Darum geht es bei den Modellwahlkriterien: Sie schätzen den Vorhersagefehler, statt ihn zu messen, und deshalb treffen sie das MSE-Optimum auch nur ungefähr: In Beispiel 13.8.13 wählt GCV $K = 10$, während $K = 12$ am besten wäre.

Vertiefung: Die Bias-Varianz-Zerlegung und die Modellwahlkriterien behandeln T. Hastie, R. Tibshirani und J. Friedman, *The Elements of Statistical Learning*, Kapitel 7; die generalisierte Kreuzvalidierung geht auf P. Craven und G. Wahba, *Smoothing noisy data with spline functions*, *Numerische Mathe-*

matik 31 (1979), 377–403, zurück, *P-Splines und additive Modelle* entwickelt S. N. Wood, *Generalized Additive Models: An Introduction with R*, 2. Auflage 2017.

13.9 Multivariat und Zusammenfassung

Alles, was wir bisher gerechnet haben, spielte auf einem Intervall: eine Variable, Knoten darauf, eine Kurve. Reale Datensätze bringen mehr als eine Kovariable mit. Dieser Abschnitt überträgt den Basisansatz auf $f : \mathbb{R}^p \rightarrow \mathbb{R}$. Die Übertragung selbst gelingt mühelos, sie ist eine Anwendung des Tensorprodukts aus Abschnitt 9.4. Bezahlen müssen wir sie mit einem Aufwand, der in p exponentiell wächst. Danach ziehen wir Bilanz, für das Kapitel und für das ganze Skript.

Tensor-Produkt-Basen

Eine Basis für Funktionen mehrerer Variablen bauen wir nicht neu, sondern aus den univariaten Bausteinen zusammen, die wir schon haben. Aus K Basisfunktionen pro Variable entstehen alle möglichen Produkte.

Definition 13.9.1 (Tensor-Produkt-Basis)

Für jede Variable x_r sei ein K -dimensionaler Funktionenraum auf einem Intervall I_r mit Basis $\phi_{r1}, \dots, \phi_{rK}$ gegeben, etwa ein B-Spline-Raum aus Abschnitt 13.4. Die zugehörige *Tensor-Produkt-Basis* auf $I_1 \times \dots \times I_p$ besteht aus den K^p Produkten

$$\phi_{1j_1}(x_1) \times \dots \times \phi_{pj_p}(x_p), \quad j_1, \dots, j_p \in \{1, \dots, K\},$$

und der Ansatz für $f : I_1 \times \dots \times I_p \rightarrow \mathbb{R}$ lautet

$$\hat{f}(\mathbf{x}) = \sum_{j_1=1}^K \dots \sum_{j_p=1}^K a_{j_1, \dots, j_p} (\phi_{1j_1}(x_1) \times \dots \times \phi_{pj_p}(x_p)) \quad (13.9.1)$$

mit dem Koeffizienten-Tensor

$$\mathbf{A} = (a_{j_1, \dots, j_p})_{j_i=1, \dots, K, i=1, \dots, p} \in \mathbb{R}^{K \times \dots \times K}.$$

Bemerkung 13.9.2 (Was Kapitel 9 dazu schon gesagt hat)

Die Konstruktion ist genau das Tensorprodukt von Funktionenräumen aus Abschnitt 9.4. Dort haben wir gesehen, dass die Produkte von Basiselementen wieder eine Basis bilden und dass sich die Dimensionen dabei multiplizieren, für p Faktoren also $K \cdot K \cdots K = K^p$. Für Polynome vom Grad höchstens d in jeder Variablen stand dort die Zahl $(d+1)^K$, und der Name für das, was daraus folgt, fiel ebenfalls schon: *Fluch der Dimensionalität* (curse of dimensionality).

Anschaulich legen wir ein Gitter über den p -dimensionalen Raum. In jeder Variablen sitzen K Basisfunktionen mit ihren Knoten, und jede Basisfunktion des Produkts gehört zu einer Zelle dieses Gitters. Der Ansatz ist bequem, weil wir für jede Variable dieselben eindimensionalen Bausteine verwenden können, und teuer, weil das Gitter mit jeder Variablen um einen Faktor K wächst.

Bemerkung 13.9.3 (Es bleibt ein lineares Kleinste-Quadrate-Problem)

Der Ansatz (13.9.1) ist linear in den Koeffizienten, und daran ändert die Tensorschreibweise nichts. Nummerieren wir die K^p Indexkombinationen $(j_1, \dots, j_p)$ durch, so wird aus $\mathbf{A}$ ein Vektor $\mathbf{a} \in \mathbb{R}^{K^p}$, und mit der Designmatrix $\mathbf{B} \in \mathbb{R}^{n \times K^p}$, deren Zeilen die K^p Produkte an der Beobachtungsstelle $\mathbf{x}_i$ enthalten, steht wieder

$$\min_{\mathbf{a}} \|\mathbf{y} - \mathbf{B} \mathbf{a}\|_2^2$$

da: dasselbe Problem wie in Satz 13.7.5, nur mit mehr Spalten. Gelöst wird es mit denselben Mitteln, über die Normalgleichungen oder besser über eine QR-Zerlegung (Abschnitt 7.4). Die Spaltenzahl ist die schlechte Nachricht. Vollen Spaltenrang kann $\mathbf{B}$ nur haben, wenn $n \geq K^p$ ist, und das ist eine harte Schranke: Mit $K = 10$ Basisfunktionen je Variable und $p = 5$ Kovariablen brauchen wir mindestens 100 000 Beobachtungen, bevor die Anpassung überhaupt eindeutig ist.

Der Fluch der Dimensionalität

Drei Größen wachsen exponentiell in p , und jede für sich reicht aus, um den vollen Tensoransatz jenseits weniger Variablen unbrauchbar zu machen: der Speicher für $\mathbf{A}$, der Aufwand einer Auswertung und die Datenmenge, die für eine feste Genauigkeit nötig ist.

Beispiel 13.9.4 (Zehn Basisfunktionen je Variable)

Wir rechnen mit $K = 10$ und speichern jeden Koeffizienten als Gleitkommazahl doppelter Genauigkeit, also mit 8 Byte (Abschnitt 4.1).

Dimension p	Koeffizienten K^p	Speicher
1	10	80 Bytes
2	100	800 Bytes
3	1000	8 kB
5	100 000	800 kB
10	10^{10}	80 GB

Bis $p = 3$ ist der Koeffizienten-Tensor ein Nichts, bei $p = 5$ noch handlich, bei $p = 10$ passt er auf keinen üblichen Arbeitsspeicher. Die Speicherangaben verwenden dezimale Präfixe, 1 kB sind also 1000 Bytes; binär gerechnet sind die $8 \cdot 10^{10}$ Bytes der letzten Zeile 74,5 GiB. Der Sprung von $p = 5$ auf $p = 10$ ist ein Faktor 10^5 , und der kommt allein daher, dass fünf weitere Variablen dazukommen.

Der statistische Preis ist derselbe. Er lässt sich mit dem, was wir in Abschnitt 13.8 gerechnet haben, direkt hinschreiben.

Satz 13.9.5 (Eine MSE-Obergrenze im Multivariaten)

Wir beobachten $y_i = f(\mathbf{x}_i) + \epsilon_i$ an n festen Stellen mit unkorrelierten, homoskedastischen Fehlern vom Mittelwert null und passen (13.9.1) mit kubischen Spline-Bausteinen auf einem gleichmäßigen Gitter an. Hat $\mathbf{B}$ vollen Spaltenrang K^p und besitzt die verwendete Tensorprodukt-Splinefolge für f eine gleichmäßige Approximationsschranke $\inf_{s \in \mathcal{F}_K} \|f - s\|_\infty \leq CK^{-4}$ (etwa unter den üblichen Glattheitsannahmen an die partiellen Ableitungen vierter Ordnung), so gilt für den an den Entwurfsstellen gemittelten Fehler

$$\text{MSE}(\hat{f}) \leq \underbrace{C_1 K^{-8}}_{\text{Bias}^2\text{-Schranke}} + \underbrace{\frac{\sigma^2 K^p}{n}}_{\text{Varianz}} \quad (13.9.2)$$

mit einer Konstante $C_1 > 0$. Die Wahl $K \asymp n^{1/(8+p)}$ balanciert die rechte Seite und liefert die Garantie

$$\text{MSE}(\hat{f}) = O(n^{-8/(8+p)}). \quad (13.9.3)$$

Vertiefung: Wie sich die Rate im Multivariaten ausbalanciert

Der Beweis ist die Rechnung aus Abschnitt 13.8 mit K^p statt K Spalten: Varianzsatz übernehmen, Bias-Schranke voraussetzen, Summe minimieren, einsetzen.

Beweis.

- (1) Der Varianzterm ist Satz 13.8.4, wörtlich. Dessen Beweis benutzt vom Schätzer nur, dass er die Kleinste-Quadrate-Anpassung an eine Designmatrix mit vollem Spaltenrang ist; wie viele Spalten sie hat und wie ihre Einträge zustande kommen, geht nicht ein. Hier sind es K^p Spalten, also ist die gemittelte Varianz $\sigma^2 K^p/n$.

am Ende des Beweises steht $\text{tr}(\mathbf{I}_{K^p}) = K^p$ statt $\text{tr}(\mathbf{I}_K) = K$

- (2) Für den Bias verwenden wir die im Satz ausdrücklich vorausgesetzte Tensorprodukt-Approximation. Der rauschfreie KQ-Fit ist an den Entwurfsstellen mindestens so gut wie dieser Vergleichsspline, also ist der gemittelte quadrierte Bias höchstens $C_1 K^{-8}$.

dieselbe Vergleichsrechnung wie in Satz 13.8.2; die genaue Herleitung der Tensorprodukt-Schranke gehört zur mehrdimensionalen Approximationstheorie und wird hier als Voraussetzung sichtbar gemacht

- (3) Wir minimieren nun die bewiesene Obergrenze $g(K) = c_1 K^{-8} + c_2 K^p/n$ mit Konstanten $c_1, c_2 > 0$. Ableiten und Nullsetzen gibt

$$g'(K) = -8c_1 K^{-9} + \frac{c_2 p K^{p-1}}{n} \stackrel{!}{=} 0 \implies K^{8+p} = \frac{8c_1 n}{c_2 p},$$

also $K^* \sim n^{1/(8+p)}$.

Multiplikation mit $K^9 n/(c_2 p)$ sammelt alle K -Potenzen auf einer Seite; g fällt für kleine und wächst für große K , die einzige positive Nullstelle ist also das Minimum

- (4) Einsetzen von K^* in beide Summanden liefert

$$c_1 (K^*)^{-8} \sim n^{-8/(8+p)}, \quad \frac{c_2 (K^*)^p}{n} \sim n^{p/(8+p)-1} = n^{-8/(8+p)},$$

und damit (13.9.3).

$$p/(8+p) - 1 = (p-8-p)/(8+p) = -8/(8+p)$$

□

Bemerkung 13.9.6 (Was die Rate über Datenmengen sagt)

Gleichung (13.9.3) ist eine Obergrenze bei einer Wahl $K \asymp n^{1/(8+p)}$, nicht der Fehler für jedes K . Sie beschreibt eine garantierte Balance der beiden Schranken; dass diese Wahl den tatsächlichen MSE minimiert, würde zusätzliche untere Schranken verlangen.

Als grobe Planungsheuristik setzen wir die in der $\mathcal{O}$ -Notation verborgene Konstante gleich eins. Welches n legt diese Proxy-Rechnung nahe, damit der MSE unter eine Schranke ε fällt? Aus $n^{-8/(8+p)} \leq \varepsilon$ folgt $n \geq \varepsilon^{-(8+p)/8}$, für $\varepsilon = 0,01$ also $n \geq 10^{(8+p)/4}$.

Dimension p	Rate	Proxy- n bei Konstante 1
1	$n^{-8/9}$	$10^{2,25} \approx 178$
2	$n^{-8/10} = n^{-0,8}$	$10^{2,5} \approx 316$
5	$n^{-8/13}$	$10^{3,25} \approx 1778$
10	$n^{-8/18}$	$10^{4,5} \approx 31\,623$

Die dritte Spalte ist eine reine Orientierung und keine garantierte Stichprobenplanung, denn die entscheidenden Konstanten hängen am Problem und nicht an der Rate; in Bemerkung 13.8.7 haben wir für $p = 1$ gesehen, wie weit die versteckte Konstante eine solche Zahl verschieben kann.

Die Botschaft der Spalte bleibt davon unberührt, und sie steckt im Exponenten $(8 + p)/4$. Jede weitere Variable multipliziert die nötige Datenmenge mit $10^{1/4} \approx 1,78$, bei schärferer Genauigkeitsforderung mit noch mehr: für $\varepsilon = 10^{-4}$ mit $\varepsilon^{-1/8} \approx 3,16$ je Dimension. Der Datenhunger wächst also exponentiell in p , ganz so wie der Speicherbedarf.

Bemerkung 13.9.7 (Auch die Auswertung kostet)

Für eine einzelne Auswertung von $\hat{f}$ steht zunächst der Aufwand $O(K^p)$, und das ist die ehrliche Zahl für eine Basis ohne besondere Struktur: Die Summe in (13.9.1) hat K^p Summanden. Mit B-Splines wird es besser, denn die haben kompakten Träger (Abschnitt 13.4). An einer festen Stelle x sind von den K Basisfunktionen einer Variablen höchstens $q + 1$ von null verschieden, bei kubischen also vier. Im Produkt bleiben damit höchstens $(q+1)^p$ Summanden übrig, für $p = 10$ und $q = 3$ noch $4^{10} = 1\,048\,576$ statt 10^{10} . Das ist eine Ersparnis um fünf Größenordnungen und ändert nichts am Grundproblem: 4^p wächst exponentiell wie 10^p , nur mit kleinerer Basis. Gespeichert werden müssen die K^p Koeffizienten ohnehin alle.

Additive Modelle statt voller Produkte

Der Ausweg ist keine bessere Numerik, sondern eine Annahme über f . Wenn wir darauf verzichten, beliebige Wechselwirkungen zwischen den Variablen darzustellen, schrumpft der Ansatzraum dramatisch.

Definition 13.9.8 (Additives Modell)

Ein *additives Regressionsmodell* setzt statt des vollen Tensorprodukts eine Summe univariater Funktionen an:

$$\hat{f}(\mathbf{x}) = \beta_0 + \sum_{i=1}^p f_i(x_i), \quad (13.9.4)$$

wobei jede Komponente f_i in einem eigenen K -dimensionalen Ansatzraum liegt, etwa dem der kubischen Splines aus Abschnitt 13.7.

Ein *generalisiertes additives Modell* (GAM) erweitert diese Form wie ein generalisiertes lineares Modell um eine Verteilung für $Y \mid X$ und eine Linkfunktion g :

$$g(\mathbb{E}[Y \mid X = \mathbf{x}]) = \beta_0 + \sum_{i=1}^p f_i(x_i).$$

Für Gaußfehler und die Identitätslink ist das genau das additive Modell oben.

Bemerkung 13.9.9 (Der Gewinn, und was er kostet)

Skalierbar. In der rohen Darstellung stehen $pK + 1$ Koeffizienten statt K^p . Nach den nötigen Zentrierungsbedingungen sind davon $p(K - 1) + 1 = O(pK)$ frei wählbar. Für $p = 10$ und $K = 10$ sind das 91 freie Parameter gegen 10^{10} , ein Faktor von gut 10^8 . Aus dem Wachstum in p wird ein lineares.

Die geläufige Kurzformel lautet pK ; um diese Zahl sauber zu bekommen, räumen wir kurz auf: Zu den pK Komponenten-Koeffizienten kommt der Achsenabschnitt β_0 , dafür ist jede Komponente nur bis auf eine additive Konstante bestimmt, denn eine Konstante ließe sich von f_1 nach f_2 verschieben, ohne (13.9.4) zu ändern. Üblich ist deshalb eine Zentrierungsbedingung an jede Komponente, etwa dass sich ihre Werte an den Beobachtungsstellen zu null summieren; dann bleiben $p(K - 1) + 1$ frei wählbare Koeffizienten. Die Größenordnung pK stimmt, die Buchführung ist ein wenig feiner.

Interpretierbar. Jede Komponente ist eine Funktion einer einzigen Variablen und lässt sich als Kurve zeichnen. Den Effekt einer Kovariablen können wir also ansehen; beim vollen Tensorprodukt ist er nur gemeinsam mit allen anderen Variablen definiert.

Der Preis. Wechselwirkungen fallen weg. Ein GAM kann nicht darstellen, dass die Wirkung von x_1 vom Wert von x_2 abhängt. Wo einzelne solche Wechselwirkungen gebraucht werden, nehmen wir sie gezielt als Tensorprodukt-Term für zwei oder drei Variablen dazu und behalten für den Rest die additive Struktur. In R ist `mgcv::gam()` die Standardadresse, mit automatischer Wahl der Glättungsparameter über die Kriterien aus Abschnitt 13.8.

Interaktiv: Zwei Kurven auf logarithmischer Achse

Das Widget stellt die beiden Zählungen nebeneinander. Orange die K^p Koeffizienten des vollen Tensorprodukts, grün die $p(K - 1) + 1$ freien Parameter des zentrierten additiven Modells, beides über der Dimension p und mit logarithmischer senkrechter Achse. Der Tensoransatz ist dort eine Gerade, das additive Modell eine flach liegende Kurve.

Der zweite Regler ändert K . Er verschiebt die grüne Kurve kaum und kippt die orange Gerade steiler oder flacher: Bei einer Geraden im logarithmischen Bild sitzt K in der Steigung. Die Tafel daneben rechnet Speicherbedarf, Mindestzahl an Beobachtungen und die Konvergenzrate zum eingestellten Zustand mit.

Wie groß vermuten wir den Speicherbedarf des vollen Tensorprodukts bei zehn Variablen und zehn Basisfunktionen?

Schätzfrage. Schätzen wir den Speicherbedarf in GB für $K = 10$ und $p = 10$.

Lösung: 80 GB

Vertiefung: Additivität ist mehr als ein Verzicht**Bemerkung 13.9.10 (Warum das mehr ist als eine Notlösung)**

Die Additivität sieht nach einem bloßen Verzicht aus. Ist f tatsächlich additiv und bleibt p fest, so ist sie mehr als das: Unter den üblichen Regularitäts- und Designannahmen lassen sich die einzelnen Komponenten mit derselben eindimensionalen Rate schätzen wie eine univariate Funktion; p geht dann in die Konstante ein. Für vierfach glatte Komponenten wird aus der vollen Rate $n^{-8/(8+p)}$ wieder die univariate Rate $n^{-8/9}$. Der Fluch der Dimensionalität ist damit nicht bekämpft, sondern durch eine Strukturannahme umgangen. Entsprechende Optimalitätsresultate gehen auf C. J. Stone (1985) zurück und sind ein Grund, warum additive Modelle in der Statistik so verbreitet sind.

Verstreute Punkte ohne Gitter

Tensor-Produkt-Basis und additives Modell haben eines gemeinsam: Beide behandeln die Variablen getrennt und setzen die Bausteine anschließend zusammen. Für Stellen, die als verstreute Punkte im $\mathbb{R}^p$ anfallen, passt das schlecht: Messstationen auf einer Landkarte liegen nicht auf einem Gitter, und ein Gitter, das sie alle abdeckt, wäre in weiten Teilen leer. Die übliche Antwort sind *radiale Basisfunktionen* (radial basis functions), die statt eines Gitters nur Zentren brauchen und den Ansatz linear in den Koeffizienten lassen.

Vertiefung: Basisfunktionen, die nur vom Abstand abhängen

Bemerkung 13.9.11 (Radiale Basisfunktionen)

Eine radiale Basisfunktion hängt nur vom Abstand zu einem Zentrum $\mathbf{c}_k$ ab,

$$\phi_k(\mathbf{x}) = \rho(\|\mathbf{x} - \mathbf{c}_k\|), \quad \text{etwa mit } \rho(r) = \exp(-r^2/(2h^2)),$$

oft nimmt man als Zentren die Datenpunkte selbst. Es ist wieder das Kleinste-Quadrate-Problem aus Bemerkung 13.9.3, nur mit anderen Spalten.

Bezahlt wird die Freiheit mit der Struktur. Die Basismatrix ist im Allgemeinen voll besetzt, denn eine Gaußglocke wird nirgends exakt null, und damit ist die Lokalität der B-Splines aus Abschnitt 13.4 dahin, solange wir nicht eigens Kerne mit beschränktem Träger wählen. Der Bandbreitenparameter h spielt dabei die Rolle des Knotenabstands: klein heißt flexibel und unruhig, groß heißt glatt und träge.

Vertiefung: Splines im maschinellen Lernen

Zwei aktuelle Verwendungen benutzen Splines als Modellbaustein, nicht als Interpolationsverfahren. *Kolmogorow-Arnold-Netze* ersetzen die feste Aktivierungsfunktion eines neuronalen Netzes durch eine lernbare, stückweise polynomiale; die Netzgewichte sind dann B-Spline-Koeffizienten, bestimmt durch Training statt durch Interpolationsbedingungen (Bemerkung 13.1.6).

Normalizing Flows bauen eine komplizierte Wahrscheinlichkeitsverteilung aus einer einfachen auf, indem sie eine invertierbare Abbildung dazwischenschalten. Die Dichte der transformierten Größe enthält den Betrag der Determinante ihrer Jacobimatrix (Abschnitt 10.3), und deshalb muss die Abbildung zweierlei können: sich invertieren und ihre Determinante billig ausrechnen lassen. Geeignet parametrisierte *streng monotone* Splines leisten beides: Strikte Monotonie macht die eindimensionale Abbildung bijektiv, und ihre Ableitung steht stückweise sofort da. Häufig verwendet werden dafür monotone rationale quadratische Splines.

Was vom Kapitel bleibt

Am Anfang stand ein Wunsch, der sich in einem Satz sagen lässt: eine Funktion finden, die zu gegebenen Daten passt. Am Ende steht ein lineares Gleichungssystem, dessen Matrix wir uns selbst aussuchen dürfen. Alles Interessante entscheidet sich bei dieser Wahl, und deshalb war dieses Kapitel weniger eine Sammlung von Formeln als eine Reihe von Entscheidungen samt ihren Folgen.

Bemerkung 13.9.12 (Kernkonzepte des Kapitels)

1. **Drei Aufgaben, ein Rechenweg.** Approximation misst global in einer Norm, Interpolation verlangt punktweise exakte Treffer, Glättung lässt Residuen zu, weil die Daten Rauschen tragen. Einen wichtigen Zugang zur ersten Aufgabe erhalten wir über die

zweite, indem wir f an endlich vielen Stellen auswerten und interpolieren; andere Verfahren minimieren den Approximationsfehler direkt (Abschnitt 13.1). Die Interpolationsbedingung allein legt dabei nichts fest: Zu jedem Interpolanten p und jeder Funktion g mit $g(x_i) = 0$ ist auch $p + g$ einer, und solche g gibt es unendlich viele (Satz 13.1.8).

2. **Die Basisdarstellung macht daraus ein Gleichungssystem.** Wir wählen K Basisfunktionen $\phi_1, \dots, \phi_K$, setzen $\hat{f} = \sum_k a_k \phi_k$ an und erhalten $\mathbf{B}\mathbf{a} = \mathbf{y}$ mit $B_{ik} = \phi_k(x_i)$: eine Zeile je Datenpunkt, eine Spalte je Basisfunktion (Abschnitt 13.2). Die Matrix hängt nur von den Stellen und der Basis ab, nicht von den Messwerten.
3. **Polynome sind theoretisch schön und numerisch heikel.** Durch n Punkte läuft genau ein Polynom vom Grad höchstens $n - 1$ (Abschnitt 13.3). Die Monombasis erkaufte das mit einer Kondition, die exponentiell wächst, mit einer Störung, die im ganzen Intervall wirkt, und mit dem Runge-Phänomen: Selbst bei beliebig oft differenzierbarem f muss die Folge der Interpolanten an gleichmäßig verteilten Knoten nicht konvergieren.
4. **Splines halten den Grad klein und die Wirkung lokal.** Statt eines Polynoms hohen Grades nehmen wir viele Polynome vom festen Grad q und kleben sie $\mathcal{C}^{q-1}$ -glatt aneinander; der Raum $\mathcal{S}_q$ hat die Dimension $m + q$ (Abschnitt 13.4). Die B-Spline-Basis dazu hat lokale Träger, macht $\mathbf{B}$ zu einer dünn besetzten Bandmatrix und senkt den Aufwand von $O(N^3)$ auf $O(Nq^2)$.
5. **Minimale Krümmung.** Unter allen $\mathcal{C}^2$ -Funktionen, die die Daten interpolieren, minimiert der natürliche kubische Spline $J(f) = \int (f'')^2 dx$. Getragen wird der Beweis von den natürlichen Randbedingungen $s''(a) = s''(b) = 0$; die dritte Ableitung darf an den Knoten springen (Abschnitt 13.5).
6. **Approximationsordnung.** Für $f \in \mathcal{C}^4$ ist der Fehler des interpolierenden kubischen Splines durch $Ch^4 \max_x |f^{(4)}(x)|$ beschränkt, mit passenden Randbedingungen sogar mit $C = 5/384$: Halbierte Gitterweite heißt Schranke durch 16. Die Ordnung hängt am Grad und an der Glattheit, stückweise linear liefert nur h^2 (Abschnitt 13.6).
7. **Glättung statt Interpolation.** Mit einem vollrangigen kleinen Ansatzraum, typischerweise $K < n$, wird die Anpassung zu einem Kleinste-Quadrate-Problem $\min_{\mathbf{a}} \|\mathbf{y} - \mathbf{B}\mathbf{a}\|_2^2$. Der zweite Weg benutzt einen großen, bis zu datengroßen Raum mit einem Strafterm $\lambda \int (g'')^2 dx$, also einen Glättungsspline (Abschnitt 13.7). Nicht K allein, sondern Residuen und Strafterm entscheiden zwischen Interpolation und Glättung.
8. **Bias-Varianz.** Der gemittelte quadrierte Bias besitzt unter den genannten Glattheitsannahmen eine obere Schranke $C_1 K^{-8}$, die gemittelte Varianz ist exakt $\sigma^2 K/n$. Unter der stärkeren Proxy-Annahme $\text{Bias}^2 \asymp c_1 K^{-8}$ balanciert $K \asymp n^{1/9}$ beide Terme und liefert die Rate $n^{-8/9}$. In der Praxis wird K über Kreuzvalidierung, AIC, BIC oder GCV gewählt (Abschnitt 13.8).
9. **Fluch der Dimensionalität.** Die Tensor-Produkt-Basis braucht K^p Koeffizienten. Das Ausbalancieren der gezeigten MSE-Obergrenze liefert $O(n^{-8/(8+p)})$. Additive Modelle ersetzen das Produkt durch eine Summe, kommen mit $O(pK)$ Koeffizienten aus und sind zusätzlich interpretierbar.

Bemerkung 13.9.13 (Drei Stellschrauben, drei Wirkungen)

Drei Entscheidungen der ersten Kapitelhälfte haben klar getrennte Folgen, und das ist ihre kürzeste Fassung.

- Der *Ansatzraum* bestimmt den Interpolanten. Polynome vom Grad höchstens $n-1$ und kubische Splines liefern verschiedene Kurven durch dieselben Punkte.
- Das *Basissystem* bestimmt Koeffizienten, Kondition und Rechenweg, aber nicht die Kurve. Monombasis und Newton-Basis spannen denselben Raum auf und ergeben dieselbe Funktion (Beispiel 13.2.12); orthogonalisierte Basen ändern an ihr nichts und an der Kondition alles (Bemerkung 13.3.11).
- Die *Knotenwahl* bestimmt, wie gut der Interpolant zwischen den Daten liegt. Bei frei wählbaren Stellen zähmen Chebyshev-Knoten das Runge-Phänomen (in der Vertiefung Bemerkung 13.3.16 erklärt); bei gemessenen Daten liegen die Stellen fest, und dann bleibt nur der Wechsel des Ansatzraums.

Selbsttest zum Kapitel

Der Selbsttest geht quer durch das Kapitel, von den drei Aufgaben aus Abschnitt 13.1 bis zum Fluch der Dimensionalität.

1. Damit der Interpolant zu n Datenpunkten eindeutig ist, genügt es, genau $K = n$ Basisfunktionen zu wählen.

Antwort: Falsch

Die Zahl allein reicht nicht, $\mathbf{B}$ muss auch invertierbar sein. Mit $\phi_1(x) = 1$, $\phi_2(x) = x$ und $\phi_3(x) = 1 + x$ und den Stellen 0, 1, 2 ist die dritte Spalte die Summe der ersten beiden, also $\det \mathbf{B} = 0$ (Satz 13.2.4, Bemerkung 13.2.5). Mit der Monombasis dagegen, also mit Polynomen vom Grad höchstens $n-1$, steht dort die Vandermonde-Matrix mit $\det \mathbf{B} \neq 0$, und Satz 13.3.5 sichert die Eindeutigkeit.

2. Es gibt beliebig oft differenzierbare Funktionen, für die der größte Fehler der Interpolation an gleichmäßig verteilten Knoten über alle Grenzen wächst, obwohl der Fehler an den Knoten selbst stets null ist.

Antwort: Wahr

Das ist Runge's Funktion $f(x) = 1/(1 + 25x^2)$ auf $[-1, 1]$ (Beispiel 13.3.14). Nachgerechnet steigt der größte Fehler von 0,44 bei $n = 5$ über 7,2 bei $n = 15$ auf 257 bei $n = 25$, während die Interpolationsbedingung an jedem einzelnen Knoten exakt erfüllt bleibt. Interpolation sagt eben nur etwas über die Stützstellen aus.

3. In der Basismatrix zu kubischen B-Splines stehen in jeder Zeile höchstens vier Einträge ungleich null.

Antwort: Wahr

An jeder Stelle sind höchstens $q + 1 = 4$ der Basisfunktionen von null verschieden, denn $B_k^{(q)}$ verschwindet außerhalb von $[\tau_k, \tau_{k+q+1}]$ (Satz 13.4.11). Für 20 Stützstellen und 20 kubische B-Splines haben wir das nachgezählt: nirgends mehr als vier, an den Rändern weniger. Daraus wird die Bandstruktur und mit ihr der Aufwand $O(Nq^2)$ statt $O(N^3)$, bei $N = 1000$ und $q = 3$ ein Verhältnis von $N^2/q^2 \approx 111\,000$ (Bemerkung 13.4.14).

4. Zwei verschiedene Basen desselben Ansatzraums liefern denselben Interpolanten, können aber Basismatrizen sehr unterschiedlicher Kondition haben.

Antwort: Wahr

Der Ansatzraum bestimmt die Kurve, die Basis nur die Koeffizienten: Monom- und Newton-Basis geben beide $\hat{f}(x) = 1 + x^2$, einmal mit $\mathbf{a} = (1, 0, 1)^T$ und einmal mit $\mathbf{a} = (1, 1, 1)^T$ (Beispiel

13.2.12). An der Kondition ändert ein solcher Wechsel dagegen sehr viel, bei $n = 20$ gleichmäßig verteilten Stellen um zwölf Zehnerpotenzen (Bemerkung 13.3.11).

5. Der Übergang von der Monombasis zu den kubischen B-Splines ist ein Basiswechsel im Sinn der vorigen Aussage, der Interpolant bleibt also derselbe.

Antwort: Falsch

Hier wechselt nicht die Basis, sondern der Ansatzraum. Die n Monome spannen die Polynome vom Grad höchstens $n - 1$ auf, die $m + q$ kubischen B-Splines den Spline-Raum $\mathcal{S}_3$, und für $n > 4$ ist das eine ganz andere Menge von Funktionen. Entsprechend verschieden sehen die Kurven aus: Im Störungs-Widget von Abschnitt 13.4 liegen links ein Polynom vom Grad 8 und rechts ein stückweise kubischer Spline durch dieselben neun Punkte.

6. Zu jeder stetigen Funktion f auf $[a, b]$ und jeder Genauigkeit $\varepsilon > 0$ gibt es ein Gitter, auf dem bereits ein stückweise linearer Spline s mit $\|f - s\|_\infty < \varepsilon$ existiert.

Antwort: Wahr

Das ist Satz 13.4.15, Punkt 1, im Fall $q = 1$; dort ist er nichts anderes als die gleichmäßige Stetigkeit von f auf einem abgeschlossenen Intervall. Bemerkenswert ist der Gegensatz zum Runge-Phänomen: Der Weg über wachsenden Polynomgrad kann scheitern, der Weg über feinere Gitter bei festem Grad nicht.

7. Bei der Spline-Interpolation wächst mit jedem zusätzlichen Datenpunkt der Grad des Interpolanten.

Antwort: Falsch

Der Grad q ist fest gewählt, meist $q = 3$. Was mit den Daten wächst, ist die Zahl der Teilintervalle und damit die Dimension $m + q$ des Ansatzraums (Satz 13.4.4). Genau darin liegt der Unterschied zur Polynominterpolation, bei der der Ansatzraum für n Punkte Polynome bis zum Grad $n - 1$ zulässt (Satz 13.3.5). Das tatsächliche Interpolationspolynom darf kleineren Grad haben. Am wachsenden maximalen Grad hängt, dass die Oszillationen aus Abschnitt 13.3 ausbleiben.

8. Unter allen Funktionen, die die Punkte (x_i, y_i) interpolieren, minimiert der natürliche kubische Spline $J(f) = \int_a^b (f'')^2 dx$.

Antwort: Falsch

Ohne Glattheitsforderung ist die Aussage falsch. Der Polygonzug durch die Punkte ist zwischen den Knoten linear, hat dort also $f'' = 0$ und käme damit auf $J = 0$; in Beispiel 13.5.8 stünde das gegen $J(s) = 6$. An den Knoten selbst existiert seine zweite Ableitung nicht, J ist für ihn streng genommen undefiniert. Satz 13.5.4 vergleicht deshalb nur $\mathcal{C}^2$ -Interpolanten, und unter denen gewinnt der natürliche Spline.

9. Verdoppeln wir die Zahl der Teilintervalle eines gleichmäßigen Gitters, so fällt die Fehlerschranke für kubische Spline-Interpolation auf ein Sechzehntel, die für stückweise lineare Interpolation dagegen nur auf ein Viertel.

Antwort: Wahr

In (13.6.1) steht h^4 , in Satz 13.6.5 steht h^2 , und die verdoppelte Teilintervallzahl halbiert h . Bei N Knoten gehen wir dafür auf $2N - 1$ Knoten, wie in der Folge 5, 9, 17, 33. Halbiertes h gibt die Faktoren $2^4 = 16$ beziehungsweise $2^2 = 4$. Gemessen bestätigen unsere Rechnungen beides: Auf hinreichend feinen Gittern laufen die gemessenen Faktoren im kubischen Fall gegen 16, im linearen gegen 4 (Beispiel 13.6.7).

10. Interpolation und Glättung sind zwei verschiedene Verfahren; die Interpolation fällt nicht unter den Kleinste-Quadrate-Ansatz dieses Kapitels.

Antwort: Falsch

Interpolation ist der Spezialfall des KQ-Ansatzes mit verschwindenden Residuen. Mit $K = n$ Basisfunktionen und invertierbarer Designmatrix ist die Kleinste-Quadrate-Lösung $\hat{\mathbf{a}} = \mathbf{B}^{-1}\mathbf{y}$, die Residuenquadratsumme null und jeder Punkt exakt getroffen. Ein kleiner vollrangiger Raum mit $K < n$ lässt für generische Daten Residuen stehen; ein großer Raum kann durch einen Strafterm ebenfalls glätten (Definition 13.7.8). Ein interpolierender Schätzer übernimmt die Fehler ϵ_i an den Datenstellen punktweise.

11. Mit wachsendem K fällt der quadrierte Bias monoton und die Varianz wächst monoton.

Antwort: Falsch

Die zweite Hälfte stimmt exakt: Die gemittelte Varianz ist $\sigma^2 K/n$ (Satz 13.8.4), sie wächst also linear. Für den Bias gibt Satz 13.8.2 nur die obere Schranke $O(K^{-8})$, und die verlangt keine Monotonie. In unserer Simulation steigt der quadrierte Bias von $K = 8$ auf $K = 9$ sogar an, weil die inneren Knoten dort gerade auf die Nullstellen von $\sin(3x)$ fallen (Bemerkung 13.8.9).

12. Allein aus der Bias-Obergrenze folgt, dass die tatsächlich MSE-optimale Knotenzahl von der Ordnung $n^{1/9}$ ist.

Antwort: Falsch

Bewiesen ist nur $\text{Bias}^2 \leq C_1 K^{-8}$. Minimieren wir die daraus entstehende Obergrenze oder nehmen stärker $\text{Bias}^2 \asymp c_1 K^{-8}$ an, erhalten wir $K \asymp n^{1/9}$ und die Garantie beziehungsweise Proxy-Rate $n^{-8/9}$ (Bemerkung 13.8.7). Für eine Aussage über das tatsächliche oder minimax-optimale K braucht es zusätzlich eine passende untere Schranke und genaue Annahmen an Funktionsklasse und Design.

13. Bei $p = 10$ Kovariablen und $K = 10$ Basisfunktionen je Variable ist der Koeffizienten-Tensor mit rund 80 MB noch gut zu handhaben.

Antwort: Falsch

Es sind 10^{10} Koeffizienten, und mit 8 Byte je Zahl macht das $8 \cdot 10^{10}$ Bytes, also 80 GB und nicht 80 MB (Beispiel 13.9.4). Dazu kämen mindestens ebenso viele Beobachtungen, damit die Designmatrix vollen Spaltenrang haben kann. Genau daran scheitert der volle Tensoransatz in hohen Dimensionen.

14. Ein additives Modell mit p Variablen und K Basisfunktionen je Komponente hat eine Koeffizientenzahl der Ordnung pK , wächst also nur linear in p .

Antwort: Wahr

Das ist (13.9.4): pK Komponenten-Koeffizienten plus Achsenabschnitt, nach Zentrierung $p(K - 1) + 1$ frei wählbare. Für $p = 10$ und $K = 10$ sind das 91 freie Parameter statt 10^{10} . Bezahlt wird der Gewinn mit den Wechselwirkungen, die der Ansatz nicht darstellen kann (Bemerkung 13.9.9).

15. Die Rate $O(n^{-8/(8+p)})$ ist eine obere Schranke für den MSE, gleich welches K wir wählen.

Antwort: Falsch

Sie ist die Garantie aus der ausbalancierten Obergrenze und gilt für eine Wahl $K \asymp n^{1/(8+p)}$ unter den Voraussetzungen von Satz 13.9.5. Für ein beliebiges K steht nur die zweigliedrige Schranke (13.9.2) zur Verfügung: K zu klein lässt eine große Bias-Schranke, K zu groß macht die Varianz groß.

Wie es weitergeht

Drei Fragen hat dieses Kapitel nur angerissen, und jede füllt eine eigene Vorlesung: die theoretische Analyse statistischer Verfahren, die Eigenschaften von Parameterschätzern und die Funktionsapproximation über das hier Gezeigte hinaus. Dafür bieten sich die Vorlesungen von Thomas Nagler an, *Statistical Learning Theory* im Master und *Mathematical Statistics* für Bachelor und Master. Additive Modelle und ihre Verwandten behandelt die Vorlesung *Statistik V: Konzepte statistischer Modellierung* im Bachelor.

Finito

Damit endet das Skript. Begonnen haben wir mit der Frage, warum Rechnen mit endlicher Genauigkeit überhaupt schiefgehen kann, und mit dem Handwerkszeug dagegen: Kondition, Stabilität, Aufwandsanalyse. Darauf baute die numerische lineare Algebra auf, von der Elimination über die Zerlegungen bis zur Singulärwertzerlegung, und darauf wiederum die Analysis mehrerer Veränderlicher mit Ableitungen, Konvexität und Optimierung. Dieses letzte Kapitel hat alles zusammengeführt: Eine Spline-Anpassung ist ein Kleinste-Quadrate-Problem, ihre Lösung ein Zerlegungsverfahren, ihre Güte eine Frage von Approximationsordnung und Kondition, und ihre Modellwahl ein Optimierungsproblem.

Vertiefung: Heath, *Scientific Computing*, Kapitel 7 fasst *Interpolation, stückweise polynomiale Interpolation und Splines samt Übungsaufgaben* zusammen; die klassische Referenz zur Sache ist C. de Boor, *A Practical Guide to Splines*. Die gestrafte Glättung entwickeln P. Eilers und B. Marx, *Flexible Smoothing with B-splines and Penalties*, *Statistical Science* 11 (1996), 89–121; *Tensorprodukt-Splines und additive Modelle samt mgcv* behandelt S. N. Wood, *Generalized Additive Models: An Introduction with R*, 2. Auflage 2017. Die Rate additiver Schätzer geht auf C. J. Stone, *Additive regression and other nonparametric models*, *Annals of Statistics* 13 (1985), 689–705, zurück, der Begriff „Fluch der Dimensionalität“ auf R. Bellman, *Dynamic Programming*, 1957.